Databricks Certified Data Engineer Professional — Study Guide

Generated November 12, 2026

Contents

Databricks Data Engineer Professional **5**
- Exam Overview . . . 5
- Exam Domain Weights (official — Oct 9, 2026) . . . 5
- Study Topics . . . 6
- Interview Preparation . . . 6
- Prerequisites . . . 7
- Study Progress Tracker . . . 7
- Official Resources . . . 7
- Recommended Courses . . . 7

Section 1: Developing Code for Data Processing using Python and SQL (23 % of Exam) **9**
- Topics Overview . . . 9
- Section Contents . . . 9
- Exam Skill Coverage . . . 9
- Key Concepts to Master . . . 10
- Related Resources . . . 10
- Python Project Structure and Dependencies . . . 10
- UDFs and Unity Catalog Functions . . . 21
- Incremental Processing . . . 31
- Structured Streaming — Part 1 . . . 42
- Structured Streaming — Part 2 . . . 51
- Streaming Joins & Stateful Operations — Part 1 . . . 58
- Streaming Joins & Stateful Operations — Part 2 . . . 67
- Declarative Pipelines . . . 74
- AUTO CDC API . . . 86
- Lakeflow Jobs — Part 1 . . . 95
- Lakeflow Jobs — Part 2 . . . 105
- Databricks Compute — Part 1 . . . 114
- Databricks Compute — Part 2 . . . 123
- Workspace and Notebooks — Part 1 . . . 134
- Workspace and Notebooks — Part 2 . . . 140
- Unit Testing — Part 1 . . . 149
- Unit Testing — Part 2 . . . 160
- Advanced Testing & Operations — Part 1 . . . 170
- Advanced Testing & Operations — Part 2 . . . 183

Section 2: Data Ingestion & Acquisition (12 % of Exam) **196**
- Topics Overview . . . 196
- Section Contents . . . 196
- Exam Skill Coverage . . . 196

Key Concepts to Master . . . 196
Related Resources . . . 197
Auto Loader . . . 197
COPY INTO . . . 210
Streaming Ingestion from Message Buses . . . 212
Change Data Capture (CDC) — Part 1 . . . 220
Change Data Capture (CDC) — Part 2 . . . 226
Lakeflow Connect Database CDC . . . 233
OpenSharing . . . 242
Clean Rooms . . . 251
Lakehouse Federation . . . 259

Section 3: Data Manipulation (12 % of Exam) 266
Topics Overview . . . 266
Section Contents . . . 266
Exam Skill Coverage . . . 266
Key Concepts to Master . . . 266
Related Resources . . . 267
Batch ETL Pipelines — Part 1 . . . 267
Batch ETL Pipelines — Part 2 . . . 277
Delta Lake Operations — Part 1 . . . 283
Delta Lake Operations — Part 2 . . . 291
Data Deduplication . . . 297
VARIANT and Semi-Structured Data . . . 308
AI Functions in Data Pipelines . . . 317
Expectations and Data Quality . . . 326

Section 4: Monitoring and Alerting (10 % of Exam) 336
Topics Overview . . . 336
Section Contents . . . 336
Exam Skill Coverage . . . 336
Key Concepts to Master . . . 336
Related Resources . . . 337
System Tables . . . 337
Lakeflow/Lakeflow Pipelines Event Logs . . . 350
Lakehouse Alerts . . . 360
Databricks CLI — Part 1 . . . 370
Databricks CLI — Part 2 . . . 377
Databricks REST API — Part 1: Jobs, Clusters, DBFS & Workspace APIs . . . 385
Databricks REST API — Part 2: Monitoring with SDK, REST & CLI; Permissions, SQL & Error Handling . . . 395
Streaming Monitoring & Optimization . . . 405

Section 5: Cost & Performance Optimization (15 % of Exam) 416
Topics Overview . . . 416
Section Contents . . . 416
Exam Skill Coverage . . . 416
Key Concepts to Master . . . 416
Related Resources . . . 417
Managed Tables, Predictive Optimization, and Automatic Clustering . . . 417
File Sizing and Compaction . . . 428
Data Skipping: Liquid Clustering vs Partitioning and Z-ORDER . . . 438
Disk Caching . . . 445
Query Profile . . . 452
Spark Tuning . . . 459
EXPLAIN Plans & Adaptive Query Execution . . . 470

Photon, Diagnostics & Query Optimization — Part 1 . . . 479
Photon, Diagnostics & Query Optimization — Part 2 . . . 487
Streaming Performance Optimization . . . 495
Cost Optimization . . . 503

Section 6: Ensuring Data Security and Compliance (8 % of Exam) 514
Topics Overview . . . 514
Section Contents . . . 514
Exam Skill Coverage . . . 514
Key Concepts to Master . . . 514
Related Resources . . . 515
Access Control . . . 515
ABAC Policies and Governed Tags . . . 525
Anonymization, PII Controls, and Data Purging . . . 534
Data Classification, Compliance & Permissions . . . 544
Secret Management . . . 557
Audit Logging, Data Lineage & Network Security . . . 566

Section 7: Data Governance (5 % of Exam) 580
Topics Overview . . . 580
Section Contents . . . 580
Exam Skill Coverage . . . 580
Key Concepts to Master . . . 580
Related Resources . . . 580
Unity Catalog . . . 580
Tags, Comments, and Discoverability . . . 594
DBFS and Cloud Storage . . . 600

Section 8: Debugging and Deploying (10 % of Exam) 612
Topics Overview . . . 612
Section Contents . . . 612
Exam Skill Coverage . . . 612
Key Concepts to Master . . . 612
Related Resources . . . 613
Spark UI Debugging . . . 613
Job Repair and Parameter Overrides . . . 625
Declarative Automation Bundles — Part 1 . . . 634
Declarative Automation Bundles — Part 2 . . . 644
Bundle Deployment Strategies — Part 1: Bundle Patterns & CI/CD Pipelines . . . 652
Bundle Deployment Strategies — Part 2: Rollback, Feature Flags & OIDC . . . 663
CI/CD Integration — Part 1: Platforms & Configuration . . . 671
CI/CD Integration — Part 2: Testing, Secrets & Monitoring . . . 682
Git Folders . . . 688

Section 9: Data Modeling (5 % of Exam) 701
Topics Overview . . . 701
Section Contents . . . 701
Exam Skill Coverage . . . 701
Key Concepts to Master . . . 701
Related Resources . . . 701
Medallion Architecture . . . 702
Delta Lake Fundamentals . . . 714
Partitioning Strategies . . . 727
Schema Management . . . 740
Slowly Changing Dimension (SCD) Patterns . . . 752
Dimensional Modeling and Metric Views . . . 766

Data Engineer Professional - Exam Resources **776**
Available Resources 776
Exam Tips and Strategies 776
Final Review — DE Professional (20-minute exam-morning scan) 780
Official Documentation Links 783
Cheat Sheets - Data Engineer Professional 785
Practice Questions 801
Mock Exam 1 — Databricks Data Engineer Professional 862
Mock Exam 2 — Databricks Data Engineer Professional 892

Databricks Data Engineer Professional

Important

What changed in the Oct 9, 2026 exam guide (new exam version, published and effective Oct 9, 2026)

- **9 domains instead of 10.** *Data Sharing and Federation* is no longer its own domain — OpenSharing, Clean Rooms and Lakehouse Federation now sit under **Data Ingestion & Acquisition**, which rises from 7 % to **12 %**
- **Data Manipulation** (12 %) replaces *Data Transformation, Cleansing, and Quality* (10 %) and adds the **VARIANT** type and **AI functions (`ai_query`)**
- Weights changed: Developing Code 22 → **23 %**, Cost & Performance 13 → **15 %**, Security & Compliance 10 → **8 %**, Data Governance 7 → **5 %**, Data Modeling 6 → **5 %** (Monitoring and Debugging stay at 10 %)
- **New skills:** AUTO CDC APIs (formerly APPLY CHANGES), Lakeflow Connect CDC from SQL Server / MySQL / PostgreSQL, OpenSharing (D2D and Databricks-to-Open), Clean Rooms, ABAC with governed tags, `CLUSTER BY AUTO`, Unity Catalog metric views, Lakehouse alerts (including AI agent quality), Python project structure and dependency troubleshooting for **Declarative Automation Bundles** (formerly Databricks Asset Bundles / DABs)
- **60 scored questions** (was 59), 120 minutes, USD 200; the guide lists **English** only. No numeric passing score is published

Source of truth: the Oct 9, 2026 exam guide PDF. The folder structure below matches its 9 sections 1 : 1, in the same order.

Databricks-side discrepancy (checked Oct 26, 2026)

The certification page links the Oct 9, 2026 PDF but still lists the previous 10 domains, 59 questions and four languages (English, Japanese, Portuguese BR, Korean). This guide follows the PDF. Check the page again two weeks before your exam, as the PDF itself advises.

Exam Overview

Detail	Information
Certification	Databricks Certified Data Engineer Professional
Exam guide	Oct 9, 2026 (exam published Oct 9, 2026)
Scored questions	60 multiple-choice (unscored items may be added; extra time is built in)
Duration	120 minutes
Result	Pass / fail (no numeric threshold in the Oct 9, 2026 exam guide)
Languages	English (per the Oct 9, 2026 PDF)
Code in stems	Python and SQL
Experience	One year of hands-on data engineering on Databricks (recommended)
Recertification	Every 2 years — see Renewal Guide
Cost	USD 200, plus applicable taxes
Delivery	Online proctored or test center

Exam Domain Weights (official — Oct 9, 2026)

```
pie title Exam Topic Distribution (9 domains)
    "Developing Code for Data Processing" : 23
    "Cost & Performance Optimization" : 15
    "Data Ingestion & Acquisition" : 12
    "Data Manipulation" : 12
```

```
"Monitoring and Alerting" : 10
"Debugging and Deploying" : 10
"Ensuring Data Security and Compliance" : 8
"Data Governance" : 5
"Data Modeling" : 5
```

Study Topics

The folders below match the 9 sections of the Oct 9, 2026 exam guide, in the guide's order. Each folder's `README.md` lists its files and maps every exam-guide skill bullet to the file that covers it.

Section	Weight	Focus
01 — Developing Code for Data Processing using Python and SQL	23 %	Bundle project structure and dep
02 — Data Ingestion & Acquisition	12 %	Auto Loader, message buses, CD
03 — Data Manipulation	12 %	Window functions, joins, aggrega
04 — Monitoring and Alerting	10 %	System tables, pipeline event log
05 — Cost & Performance Optimization	15 %	Managed tables, Predictive Optin
06 — Ensuring Data Security and Compliance	8 %	Least-privilege ACLs, ABAC, anon
07 — Data Governance	5 %	Permission inheritance, tags and
08 — Debugging and Deploying	10 %	Spark UI, logs, job repair, Declara
09 — Data Modeling	5 %	Table layout, SCD, dimensional m

Product names used in this guide

This guide uses the exam guide's names. Current docs names, where different: **Lakeflow Pipelines** → Lakeflow Spark Declarative Pipelines (renamed Nov 12, 2025); **Declarative Automation Bundles** were Databricks Asset Bundles until Mar 16, 2026; **OpenSharing** was Delta Sharing until Jun 10, 2026; the "Delta cache" is the **disk cache** in the docs. For hands-on practice use Free Edition — Community Edition was retired on Jan 1, 2026.

Practice Exams

Resource	Description
Mock Exam	63-question full-length practice exam, labelled by 2026 section
Mock Exam 2	60-question advanced practice exam, labelled by 2026 section
Practice Questions	155 questions across the 9 sections, including new 2026-blueprint questions

Quick Reference

Resource	Purpose
Cheat Sheets	Quick reference cards for key topics
Exam Tips	Exam strategies and common traps
Final Review	Exam-morning cram scan
Official Links	Databricks documentation references

Interview Preparation

After completing this certification, explore advanced architecture and design questions:

- Interview Prep Resource - System design, Delta Lake internals, pipeline architecture, performance optimization, and more

Prerequisites

Before starting this certification, review:

- Delta Lake Basics
- Spark Fundamentals
- Medallion Architecture
- Unity Catalog Basics

Study Progress Tracker

Phase 1: Foundations (weeks 1–2)

☐ Delta Lake fundamentals
☐ Medallion architecture
☐ Unity Catalog basics
☐ Spark fundamentals

Phase 2: Core Processing (weeks 3–4)

☐ Section 01 — Developing Code for Data Processing using Python and SQL
☐ Section 02 — Data Ingestion & Acquisition
☐ Section 03 — Data Manipulation

Phase 3: Operations & Optimization (weeks 5–6)

☐ Section 05 — Cost & Performance Optimization
☐ Section 04 — Monitoring and Alerting
☐ Section 08 — Debugging and Deploying

Phase 4: Security, Governance & Modeling (week 7)

☐ Section 06 — Ensuring Data Security and Compliance
☐ Section 07 — Data Governance
☐ Section 09 — Data Modeling

Phase 5: Exam Prep (week 8)

☐ Review cheat sheets
☐ Complete practice questions (target 70 %+ per section)
☐ Work the 10 sample questions in the official exam guide PDF
☐ Mock Exam 1 (timed)
☐ Mock Exam 2 (timed)

Official Resources

- Databricks Certification Page
- Oct 9, 2026 exam guide (PDF)
- Databricks Documentation
- Databricks Academy

Recommended Courses

From the Oct 9, 2026 exam guide:

1. **Advanced Data Engineering with Databricks** — instructor-led

2. **Advanced Techniques with Apache Spark™ Declarative Pipeline** — self-paced
3. **Databricks Data Privacy** — self-paced
4. **Databricks Performance Optimization** — self-paced
5. **Automated Deployment with Declarative Automation Bundles** — self-paced

Section 1: Developing Code for Data Processing using Python and SQL (23 % of Exam)

The highest-weighted domain (23 %). It covers how production code is structured, packaged, tested and run on Databricks: Python project layout for Declarative Automation Bundles, library dependencies, UDFs, Lakeflow Pipelines (streaming tables, materialized views, AUTO CDC), Structured Streaming state semantics, Lakeflow Jobs control flow, and compute choice.

Topics Overview

```
flowchart LR
    D[Developing Code]
    D --> T1["Python Project Structure and Dependencies"]
    D --> T2["UDFs and Unity Catalog Functions"]
    D --> T3["Incremental Processing"]
    D --> T4["Structured Streaming"]
    D --> T5["Streaming Joins & Stateful Operations"]
    D --> T6["Declarative Pipelines"]
    D --> T7["AUTO CDC API"]
    D --> T8["Lakeflow Jobs"]
    D --> T9["Databricks Compute"]
    D --> T10["Workspace and Notebooks"]
    D --> T11["Unit Testing"]
    D --> T12["Advanced Testing & Operations"]
```

Section Contents

File	Topic
01-python-project-structure-and-dependencies.md	Bundle project layout, wheels, `pyproject.toml`, depende
02-udfs-and-unity-catalog-functions.md	Python, pandas and SQL UDFs; Unity Catalog functions; c
03-incremental-processing.md	High-water marks, checkpoints, `availableNow`, idempote
04-structured-streaming-part1.md	Sources, sinks, triggers, output modes, watermarks, winc
04-structured-streaming-part2.md	Stateful operations, state store, `foreachBatch`, checkpoi
05-streaming-joins-stateful-part1.md	Stream-stream and stream-static joins, `mapGroupsWithSt`
05-streaming-joins-stateful-part2.md	Advanced watermarking, streaming deduplication
06-declarative-pipelines.md	Lakeflow Pipelines: streaming tables vs materialized view
07-auto-cdc-api.md	AUTO CDC APIs (formerly APPLY CHANGES), SCD Type 1 /
08-lakeflow-jobs-part1.md	Tasks, dependencies, task values, If/else conditions, For e
08-lakeflow-jobs-part2.md	Triggers, compute, notifications, parameters, UI / API / CL
09-databricks-compute-part1.md	Classic compute: all-purpose vs job compute, access mo
09-databricks-compute-part2.md	Serverless environments, dependencies, performance mo
10-workspace-and-notebooks-part1.md	Workspace layout, magic commands, widgets
10-workspace-and-notebooks-part2.md	`dbutils`, notebook permissions, notebook formats
11-unit-testing-part1.md	pytest, `assertDataFrameEqual`, `assertSchemaEqual`, `Da`
11-unit-testing-part2.md	Testing patterns, integration tests, unit testing Lakeflow p
12-advanced-testing-operations-part1.md	Integration testing patterns
12-advanced-testing-operations-part2.md	Deployment validation, GitOps

Exam Skill Coverage

Every skill bullet in Section 1 of the Oct 9, 2026 exam guide, with the files that cover it.

Exam guide skill
Implement scalable Python project structures for Declarative Automation Bundles (formerly Databricks Asset
Apply troubleshooting techniques to dependency conflicts and installation failures for external libraries (PyPI,
Develop User-Defined Functions (UDFs) using Pandas, Python, and SQL, including Unity Catalog functions for t
Create production-ready data pipelines for streaming data using Lakeflow Pipelines and Auto Loader.
Create and automate ETL workloads using Lakeflow Jobs via the UI, API, and CLI.
Choose between a streaming table and a materialized view for a given latency, cost, and refresh requirement.
Implement CDC pipelines using AUTO CDC APIs (formerly APPLY CHANGES) in Lakeflow Pipelines, including SC
Choose between Spark Structured Streaming and Apache Spark™ Declarative Pipelines for scalable ETL given
Create Lakeflow Jobs that use control-flow operators such as If/Else conditions and For Each loops.
Choose appropriate compute and configuration for environments and dependencies — including serverless co
Develop unit and integration tests using assertDataFrameEqual, assertSchemaEqual, DataFrame.transform, a
Apply Structured Streaming stateful-processing semantics, including watermarks, output modes, foreachBatch

Key Concepts to Master

Concept	Why it matters
Streaming table vs materialized view	Streaming tables process each appended source row once; mater
AUTO CDC	Declarative CDC into SCD Type 1 or 2 tables with keys, sequence_
Watermark + checkpoint	The watermark bounds state for late data; the checkpoint restores
Bundle project layout	`databricks.yml` + `resources/` + `src/` package + `tests/`, built a
Lakeflow Jobs control flow	If/else condition tasks branch on values; For each tasks loop a nes
`assertDataFrameEqual`	PySpark's built-in DataFrame equality assertion for unit tests (pys

Related Resources

- Lakeflow Pipelines cheat sheet (shared)
- Streaming quick reference (shared)
- PySpark API quick reference (shared)

Python Project Structure and Dependencies

Overview

Production Databricks code is a Python project, not a pile of notebooks: a bundle root with `databricks.yml`, resource definitions, an importable package under `src/`, tests, and a `pyproject.toml` that declares dependencies. This page covers how to lay that project out for Declarative Automation Bundles, and how to troubleshoot libraries that fail to install or conflict on serverless compute, Lakeflow Pipelines, and bundle-deployed jobs.

Overview

- A bundle has exactly one `databricks.yml` at its root; every other config file (for example `resources/*.yml`) must be listed in `include`.
- Shared logic lives in a package under `src/`, built into a wheel by the `artifacts` mapping (`type: whl`) and installed through job `environments`, pipeline `environment`, or task `libraries`.
- Each runtime has its own dependency mechanism: serverless uses environments (base environment + `dependencies`), pipelines use the pipeline environment, classic compute uses compute-scoped, notebook-scoped, or task libraries.

- Most installation failures come from unpinned versions, cached environments, unsupported mechanisms (init scripts, PySpark installs, restarting Python in a pipeline), CPU-architecture-specific wheels, or private repositories that aren't configured.

What the Exam Tests

- "Implement scalable Python project structures for Declarative Automation Bundles (formerly Databricks Asset Bundles / DABs) to support modular development and CI/CD integration."
- "Apply troubleshooting techniques to dependency conflicts and installation failures for external libraries (PyPI, local wheels, source archives) across serverless, pipeline, and bundle-deployed environments."
- Choosing where a dependency must be declared (job environment, pipeline environment, notebook environment, cluster library) for a given compute type.
- Matching an installation symptom to its root cause and the documented fix.

Platform update (Oct 2026)

Databricks Asset Bundles were renamed **Declarative Automation Bundles** on March 16, 2026. The CLI command group is still `databricks bundle ...`. Source: Declarative Automation Bundles release notes

---

Bundle Project Structure

Start from a template `databricks bundle init` creates a bundle from a template. If you don't name one, the CLI prompts with the default templates.

```
databricks bundle init default-python   # job + ETL pipeline + Python package,
↪ requires uv
databricks bundle init pydabs           # same idea, but resources defined in
↪ Python
databricks bundle init /path/to/custom-template
```

Template	What it generates
`default-minimal`	Empty bundle with only required files and catalog variables
`default-python`	Python bundle with a job and an ETL pipeline; requires uv
`pydabs`	`default-python` variant that uses Python instead of YAML for resource configuration
`default-sql`	Job that runs SQL queries on a SQL warehouse
`default-scala`	Builds a Scala JAR configured for serverless compute
`dbt-sql`	dbt-core project deployed as a job with a dbt task
`mlops-stacks`	Full MLOps Stacks project

During `default-python` initialization the CLI asks whether to include a sample notebook job, an ETL pipeline, a stub (sample) Python package, whether to **use serverless**, the default catalog, and whether to use a personal schema per user. Since CLI 0.258.0 the template builds the wheel with uv and `pyproject.toml`; since CLI 0.276.0 it follows Lakeflow Pipelines conventions (pipeline source as files, not notebooks).

A layout that scales

```
my_project/
├── databricks.yml                # bundle name, include, artifacts, variables,
 ↪  targets
├── pyproject.toml                # package metadata, dependencies, build backend
├── uv.lock                       # optional lock file (uv lock)
├── README.md
├── resources/
│   ├── my_project.job.yml        # Lakeflow Jobs definitions
│   └── my_project_etl.pipeline.yml
├── src/
│   ├── my_project/               # importable package -> built into a wheel
│   │   ├── __init__.py
│   │   ├── main.py               # entry point for python_wheel_task
│   │   └── transforms.py         # pure DataFrame -> DataFrame functions
│   └── my_project_etl/
│       └── transformations/      # Lakeflow Pipelines source files
└── tests/
    └── test_transforms.py        # run locally with: uv run pytest
```

Element	Why it supports modular development and CI/CD
One `databricks.yml` + `include`	Keeps the root small; teams add `resources/*.yml` files without editing
`resources/` split per job or pipeline	Each workload is reviewed and deployed as code; target overrides chan
`src/<package>/`	Logic is importable by jobs, pipelines, notebooks, and tests, and version
`pyproject.toml`	Single source of truth for runtime and dev dependencies (`[dependency-`
`tests/`	`uv run pytest` runs in CI before `databricks bundle deploy`
`targets` (`dev`, `prod`)	Same code promoted across workspaces with `mode: development` / `mod`

`databricks.yml`: include, artifacts, variables, targets

```yaml
bundle:
  name: my_project

include:
  - resources/*.yml            # every config file other than databricks.yml must
 ↪  be included

artifacts:
  python_artifact:
    type: whl                  # whl, jar, or tgz
    build: uv build --wheel    # runs locally before upload
    path: .                    # folder that holds pyproject.toml / setup.py
    # dynamic_version: true    # patch the wheel version with the build timestamp

variables:
  catalog:
    description: Target catalog
    default: dev_catalog

targets:
  dev:
    mode: development          # [dev <user>] prefix, schedules paused
    default: true
  prod:
```

```
    mode: production
    variables:
      catalog: prod_catalog
```

Key facts about these mappings:

- `include` paths are globs relative to the file that declares them, and `include` can appear only as a top-level mapping.
- `artifacts.<name>.build` is optional. For wheels, if you omit it the CLI runs `python setup.py bdist_wheel`; if you omit `path`, it looks for `setup.py` in the bundle root.
- `dynamic_version: true` (CLI 0.245.0+, `type: whl` only) patches the wheel version from the file timestamp, so you can redeploy new code without editing the version in `setup.py` or `pyproject.toml`. The `artifacts_dynamic_version` target preset does the same per target; `default-python` turns it on for dev when you choose classic compute.
- If a wheel is already built and you only want to deploy it, omit `artifacts`; the CLI deploys the `whl` files listed in `libraries`.
- To install a wheel on a cluster running Databricks Runtime 12.2 LTS or below, add the top-level `experimental: python_wheel_wrapper: true` mapping.

`pyproject.toml` for a bundle

```toml
[project]
name = "my_project"
version = "0.0.1"
requires-python = ">=3.10,<3.13"
dependencies = [
    # Runtime dependencies for jobs and pipelines in this project.
    "numpy==1.25.2",
]

[dependency-groups]
dev = [
    "pytest",
    "databricks-connect>=15.4,<15.5",
]

[project.scripts]
main = "my_project.main:main"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

Runtime dependencies go under `[project] dependencies`; local tooling (pytest, Databricks Connect, linters) goes in a dev dependency group so it never ships to the cluster. The documented template notes that, for pipelines, dependencies are cached during development, so add them to the pipeline's `environment` section as well.

Wiring the package into jobs and pipelines Paths inside a resource file are relative to that file, so a resource in `resources/` refers to `../src/...` and `../dist/*.whl`.

```
# resources/my_project.job.yml
resources:
  jobs:
    my_project_job:
      name: my_project_job
      tasks:
```

```yaml
        # Serverless: Python wheel / script tasks must reference an environment
        - task_key: main
          python_wheel_task:
            package_name: my_project
            entry_point: main
          environment_key: default
      environments:
        - environment_key: default
          spec:
            environment_version: "4"
            dependencies:
              - ../dist/*.whl          # the wheel built by the artifacts mapping
              - simplejson==3.19.*

# Classic job compute: attach the wheel as a task library instead
tasks:
  - task_key: main
    new_cluster:
      spark_version: 15.4.x-scala2.12
      node_type_id: i3.xlarge
      num_workers: 1
    python_wheel_task:
      package_name: my_project
      entry_point: main
    libraries:
      - whl: ../dist/*.whl
      - pypi:
          package: numpy==1.25.2
```

```yaml
# resources/my_project_etl.pipeline.yml
resources:
  pipelines:
    my_project_etl:
      name: my_project_etl
      catalog: ${var.catalog}
      schema: my_project
      serverless: true
      root_path: ../src/my_project_etl
      libraries:
        - glob:
            include: ../src/my_project_etl/transformations/**
      environment:
        dependencies:
          - --editable ${workspace.file_path}   # install the bundle's own
↪  pyproject.toml
```

Workload	Where the wheel / package goes
Serverless Python wheel or script task	`environments[].spec.dependencies` + task `environment_key`
Serverless notebook task	Notebook's own environment (Environment side pane) or a job-level e
Classic job task	Task `libraries` (`whl`, `pypi`, `maven`, `jar`, `requirements`)
Serverless pipeline	Pipeline `environment.dependencies` (CLI 0.258+)

Defining resources in Python Python support for bundles (GA October 27, 2025, CLI 0.275.0+) lets you define, generate, and mutate resources in Python alongside YAML. Initialize with `databricks bundle init pydabs`, then point `databricks.yml` at the loader:

```
python:
  venv_path: .venv                # environment that has databricks-bundles
 ↪  installed
  resources:
    - 'resources:load_resources'  # function that returns the Python-defined
 ↪  resources
  mutators:
    - 'mutators:add_email_notifications'
```

```
from databricks.bundles.core import Bundle, Resources,
 ↪  load_resources_from_current_package_module
from databricks.bundles.jobs import Job


def create_job(country: str) -> Job:
    return Job.from_dict({
        "name": f"my_job_{country}",
        "tasks": [{"task_key": "nb", "notebook_task": {"notebook_path":
         ↪  "files/my_notebook.py"}}],
    })


def load_resources(bundle: Bundle) -> Resources:
    resources = load_resources_from_current_package_module()
    for country in ["US", "NL"]:
        resources.add_resource(f"my_job_{country}", create_job(country))
    return resources
```

Use this when many near-identical jobs come from metadata (one per country or source system). A mutator that raises an exception aborts the deployment.

Sharing code across notebooks, files, and pipelines

Approach	When to use
Import from workspace files	Code co-versioned with the notebooks in th
`sys.path.append(os.path.abspath(".."))`	Module lives in another directory
Build a wheel	Code reused across bundles, pipelines, or t
Pipeline `utilities/` folder	Helpers specific to one pipeline
Path in the pipeline environment (`-e /Workspace/.../shared/`)	Shared module used by several pipelines
`%run`	Legacy notebook includes

On serverless compute the working directory is not guaranteed, so relative file paths and relative imports can fail; use absolute paths and import shared code as workspace files.

CI/CD flow

```
flowchart LR
    PR[Pull request] --> CI[CI: uv sync + uv run pytest]
    CI --> V[databricks bundle validate -t staging]
    V --> D1[databricks bundle deploy -t staging]
    D1 --> IT[databricks bundle run integration_job]
    IT --> D2[databricks bundle deploy -t prod<br/>as a service principal]
```

Deployment commands, targets, and CI pipelines are covered in Declarative Automation Bundles — Part 1 and CI/CD Integration — Part 1.

Dependency Mechanisms by Environment

Environment	Declare dependencies with
Serverless notebook	Environment side pane: base environment + Dependencies; `%pip` / `%uv pip` (envi
Serverless job task	Job `environments` spec (`environment_version`, `dependencies`) referenced by en
Lakeflow Pipelines	Pipeline Environment settings (`environment.dependencies`); workspace-file impo
Classic compute	Compute-scoped libraries, notebook-scoped `%pip`, job task `libraries`, init scripts
Unity Catalog Python UDF	`ENVIRONMENT (dependencies = '[...]', environment_version = '...')` in

Environment versions, base environments, and memory settings for serverless are covered in Databricks Compute — Part 2.

Dependency entry formats Serverless notebook, job, and pipeline dependencies accept any line that is valid in a `requirements.txt` file:

```
simplejson==3.19.*                                    # PyPI, pinned
/Workspace/Shared/libs/my_pkg-0.1.0-py3-none-any.whl  # wheel in workspace files
 ↪  (absolute path)
/Volumes/main/default/libs/my_pkg-0.1.0-py3-none-any.whl  # wheel in a UC volume
/Workspace/helper_utils                               # unpackaged project folder
 ↪  with pyproject.toml
-r /Workspace/Shared/requirements.txt                 # requirements file
--index-url https://<private-repo>/simple             # private index
git+https://github.com/databricks/databricks-cli      # VCS source
```

Local wheels and source archives

- **Where to store files:** Unity Catalog volumes or workspace files. Library files in the DBFS root are deprecated and disabled by default on DBR 15.1+ because any user can modify them.
- **Volume uploads:** uploading to a volume renames files (spaces, periods, hyphens become underscores), but wheel filenames are left unchanged because pip requires periods in the version and hyphens as separators.
- **Source projects:** a directory containing `pyproject.toml` or `setup.py` can be added as a serverless dependency or a pipeline environment entry. For classic compute, the Workspace library source accepts wheels, zipped wheelhouses (`*.wheelhouse.whl`), ZIP, tar, and `requirements.txt` files.
- **Eggs:** supported only on DBR 13.3 LTS and below with dedicated or no-isolation shared access, and never from volumes or workspace files. Use wheels or PyPI.
- **CPU architecture:** serverless compute can run on `aarch64` or `x86_64`, and the architecture can change between runs. A wheel with native (C) extensions built for one architecture fails on the other. Prefer pure-Python wheels (`py3-none-any`), or ship both variants constrained with a `platform_machine` environment marker.

Private package repositories

```
token = dbutils.secrets.get(scope="scope", key="key")
```

```
%pip install --index-url
 ↪  https://<user>:$token@<your-package-repository>.com/<path/to/repo>
 ↪  <package>==<version> --extra-index-url https://pypi.org/simple/
```

- On serverless, dependencies are resolved before your code runs, and compute-scoped environment variables aren't supported, so a proxy set at runtime has no effect. Put `--index-url` / `--extra-index-url` in the dependency specification instead.
- Workspace admins can configure **default package repositories** (Settings → Compute → Default Package Repositories, or the `databricks-package-management` secret scope with keys `pip-index-url`, `pip-extra-index-urls`, `pip-cert`). They apply to notebooks, jobs, Lakeflow Pipelines, and Databricks Apps; an index URL specified in code or a notebook takes precedence over the workspace default.
- Changes to the repository secrets take effect after reattaching (serverless notebooks), rerunning (serverless jobs), restarting the compute (classic), or on the next pipeline run.

Notebook-scoped installs and restarting Python

- `%pip` doesn't restart the Python process. After installing or upgrading a package that's already loaded, run `dbutils.library.restartPython()`. Install everything at the top of the notebook, then restart, because the restart clears Python state.
- On DBR 11.3 LTS+, `%pip`, `%sh pip`, and `!pip` all install notebook-scoped libraries.
- `%uv pip` (serverless environment version 5+) is faster than `%pip` and shares the same virtual environment, but it doesn't check for conflicts against preinstalled libraries.
- `%pip uninstall` can't remove a library that ships with Databricks Runtime or was installed as a cluster library; it can only revert a different version you installed.
- Lakeflow Pipelines don't support `dbutils.library.restartPython()`, so declare all Python dependencies in the pipeline environment instead of installing them at runtime.

Python library precedence When two libraries share a name, `import` loads the one with the highest precedence:

1. Current working directory (Git folders only)
2. Git folder root directory (Git folders only)
3. Notebook-scoped libraries (`%pip install`)
4. Compute-scoped libraries (UI, CLI, or API)
5. Libraries included in Databricks Runtime (init-script installs can land before or after these)
6. Current working directory (outside Git folders)
7. Workspace directories manually appended to `sys.path`

Troubleshooting Dependency Conflicts and Installation Failures

```
flowchart TD
    S[Library fails to install or import] --> W{Where does it run?}
    W -->|Serverless notebook / job| SV[Check environment spec,
↪   pins,<br/>architecture, private index]
    W -->|Lakeflow pipeline| PL[Check pipeline environment;<br/>no restartPython,
↪   no JVM libs]
    W -->|Classic compute| CL[Check library source, access mode,<br/>allowlist,
↪   precedence]
    W -->|Bundle deploy| BD[Check artifacts build, paths,<br/>environment_key,
↪   version bump]
```

Symptom	Likely cause
Serverless notebook session stops with an error after adding a dependency	The package is PySpark or pul
Serverless job keeps running the old code of your custom package	Environment cache: same pac
Wheel installs on one run and fails on the next	Native wheel built for one CPU
Behaviour changes between runs with no code change	Unpinned dependency resolve

Symptom	Likely cause
New version installed, but `import` still shows the old one	Python process not restarted,
Private package works on classic but not on serverless	Runtime proxy variables are ig
Pipeline update fails on `restartPython()` or a mid-pipeline install	Not supported in Lakeflow Pipe
JAR / Maven library needed in a pipeline	Pipelines support only SQL and
Init script or cluster library has no effect on serverless	Not supported on serverless
Python wheel task on serverless fails validation	No environment configured for
Notebook task on serverless ignores task `libraries`	Task libraries aren't supported
Library stored in DBFS root won't install on DBR 15.1+	DBFS-root library storage disal
Maven or JAR install blocked on standard access mode	Coordinates and paths must b
Two Maven packages need different versions of a transitive dependency	Load order decides which vers
Local `pytest` against Databricks Connect breaks after `pip install pyspark`	Databricks Connect and PySpa
Relative import or file path fails only on serverless	Working directory isn't guaran

To inspect what was installed on a serverless notebook, open the Environment side pane: the **Installed** tab lists packages and **pip logs** at the bottom shows the install log.

Beta: `pyproject.toml`-Driven Serverless Environments

Platform update (Oct 2026)

Two Beta features released October 22, 2026 let `pyproject.toml` define the serverless environment: **serverless job environments from `pyproject.toml`** (notebook and Python script tasks; configurable in the Jobs UI, Jobs API, or a bundle) and **Shared Serverless for Asset Bundles** (notebooks and files in one bundle share serverless compute and the environment defined in the bundle-root `pyproject.toml`). A workspace admin must enable **Project-Scoped Shared Compute** on the Previews page. Source: October 2026 release notes

```toml
[project]
name = "my-project"
version = "0.1.0"
dependencies = ["cowsay==6.1"]

[build-system]                    # only needed to install the project itself
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.databricks.environment]
environment_version = "6"         # serverless environment version
```

```yaml
resources:
  jobs:
    my_project_job:
      name: my-project-job
      environments:
        - environment_key: project
          spec:
            project_environment: ../pyproject.toml   # must be the only
↪  environment-source field
      tasks:
        - task_key: main
          environment_key: project
```

```
      spark_python_task:
        python_file: ../src/main.py
```

- Requires serverless environment version 5+, `pyproject.toml` in workspace files, and Databricks CLI 1.21.0+ for `project_environment` in a bundle.
- With `project_environment`, omit `environment_version`, `base_environment`, and `dependencies` from the spec; set them in `pyproject.toml`.
- Installing the project itself needs a build backend and an up-to-date `uv.lock` (`uv lock`). With a missing or stale lock file, the runtime falls back to `uv pip install -r pyproject.toml`, which installs only the listed dependencies without enforcing locked versions. Check with `%uv lock --check`.
- In Shared Serverless, `%uv add <package>` updates `pyproject.toml`, `uv.lock`, and the shared environment; run `%uv sync` after editing `pyproject.toml` directly. Compute is shared per user and per serverless usage policy.

Use Cases

- **Multi-team ETL monorepo**: Each team owns `resources/<team>_*.yml` and a sub-package under `src/`; `include: resources/*.yml` picks up new files without edits to the root config, and one wheel ships the shared transformations.
- **Serverless job with a private library**: The job `environments` spec lists `--extra-index-url` and the pinned internal package, so the same bundle deploys unchanged to dev and prod workspaces.
- **Fast iteration on a custom wheel**: `dynamic_version: true` on the artifact means every `bundle deploy` produces a new version, so serverless jobs never reuse a stale cached install.

Common Issues & Errors

Resource file is not deployed **Scenario:** A new `resources/ingest.job.yml` exists, but `bundle deploy` doesn't create the job.

Fix: Add the file (or a matching glob) to the top-level `include` array. Only `databricks.yml` is read automatically.

Wheel path not found at deploy time **Scenario:** `libraries: - whl: ./dist/*.whl` fails from a file in `resources/`.

Fix: Paths in a resource file resolve relative to that file; use `../dist/*.whl`.

Build step fails on the CI runner **Scenario:** `bundle deploy` stops while running the artifact `build` command.

Fix: Install the build tool on the runner (uv for `uv build --wheel`, or Poetry / `setuptools` + `wheel`), and run the same build locally first.

Mixed environment fields with a project environment **Scenario:** A job spec sets both `project_environment` and `dependencies`.

Fix: `project_environment` must be the only environment-source field; move dependencies into `pyproject.toml`.

Exam Tips

Tip

- A bundle has **one** `databricks.yml`; every other config file must be listed in `include`.
- Shared code belongs in `src/<package>/`, built as a `whl` artifact; tests run locally with `uv run pytest`.
- On **serverless**, Python wheel and script tasks need `environment_key` + an `environments` spec. Init scripts, cluster libraries, and compute policies don't exist there.
- Custom package changed but serverless still runs old code → **bump the version** (or `dynamic_version: true`).
- Pipelines: declare packages in the **pipeline environment**; `restartPython()` and JVM libraries are not supported.
- Never install PySpark (directly or transitively) on serverless; never mix `databricks-connect` and `pyspark` locally.
- Store wheels in **UC volumes or workspace files**, not the DBFS root.

Key Takeaways

- **Bundle root**: `databricks.yml` (`name`, `include`, `artifacts`, `variables`, `targets`) plus `resources/`, `src/<package>/`, `tests/`, and `pyproject.toml` is the documented, template-generated layout.
- **Artifacts**: `type: whl` with `build: uv build --wheel` builds the wheel locally before upload; `dynamic_version: true` patches the version on each deploy.
- **Dependency placement**: serverless jobs use `environments` + `environment_key`; serverless notebooks use the Environment side pane; pipelines use `environment.dependencies`; classic compute uses task or compute-scoped `libraries`.
- **Formats**: any `requirements.txt` line works for serverless and pipeline dependencies, including volume or workspace wheel paths, project folders, `-r` files, and `--index-url`.
- **Private repos**: put index URLs in the dependency spec or configure default package repositories; runtime proxies don't affect serverless installs.
- **Architecture**: serverless may run on `aarch64` or `x86_64`; prefer `py3-none-any` wheels.
- **Python for bundles**: `pydabs` template, `python.resources` loader and `python.mutators`, for generating many resources from metadata.
- **Beta (Oct 22, 2026)**: `project_environment: ../pyproject.toml` and Shared Serverless for Asset Bundles need the Project-Scoped Shared Compute preview and environment version 5+.

Related Topics

- UDFs and Unity Catalog Functions - `ENVIRONMENT` clause for UDF dependencies
- Databricks Compute — Part 2 - Serverless environment versions, performance mode, high memory
- Unit Testing — Part 1 - Testing the package in `src/`
- Declarative Automation Bundles — Part 1 - Bundle commands and targets
- CI/CD Integration — Part 1 - Pipelines that deploy bundles
- DBFS and UC Volumes - Where to store wheels

Official Documentation

- Declarative Automation Bundles project templates
- Build a Python wheel file using Declarative Automation Bundles
- Declarative Automation Bundles configuration
- Bundle configuration reference (artifacts, include, presets)
- Declarative Automation Bundles library dependencies

- Bundle configuration in Python
- Bundle configuration examples
- Develop pipelines with Declarative Automation Bundles
- Configure the serverless environment
- Use Shared Serverless for Git folders and bundles
- Serverless compute limitations
- Manage Python dependencies for pipelines
- Import Python modules from Git folders or workspace files
- Install libraries
- Notebook-scoped Python libraries
- Compute-scoped libraries
- Install libraries from a package repository
- Configure default package repositories
- Work with Python and R modules
- Testing for Databricks Connect for Python

UDFs and Unity Catalog Functions

Overview

User-defined functions (UDFs) extend Spark when built-in functions can't express the logic. On Databricks you choose along two axes: **language and execution model** (SQL, row-at-a-time Python, Arrow-optimized Python, vectorized pandas, batch, table functions) and **scope** (session-scoped in a notebook or job, or governed and shared as a Unity Catalog function).

Overview

- Built-in functions and SQL UDFs are the most efficient options; Python UDFs pay serialization cost, and pandas UDFs reduce it with Apache Arrow (up to 100x faster than row-at-a-time Python UDFs).
- Session-scoped UDFs (`@udf`, `@pandas_udf`, `applyInPandas`, `@udtf`, `CREATE TEMPORARY FUNCTION`) live only in the current SparkSession.
- Unity Catalog functions (`CREATE FUNCTION catalog.schema.fn ... LANGUAGE SQL | PYTHON`) are persisted, governed with `EXECUTE`, discoverable, and callable from notebooks, jobs, pipelines, and pro/serverless SQL warehouses.
- Batch Unity Catalog Python UDFs (`PARAMETER STYLE PANDAS` + `HANDLER`) and Unity Catalog Python UDTFs bring vectorized and table-returning logic under governance.

What the Exam Tests

- ”Develop User-Defined Functions (UDFs) using Pandas, Python, and SQL, including Unity Catalog functions for the given constraints.”
- Picking the UDF type that satisfies a stated constraint: performance, governance and reuse, SQL warehouse or serverless support, external libraries, one-to-many output.
- Writing correct `CREATE FUNCTION` syntax for SQL and Python UDFs, and granting `EXECUTE`.
- Recognizing compute and access-mode limits (DBR versions, serverless memory limit, classic SQL warehouse restriction).

Built-in Functions First

Use UDFs for logic that's hard to express with built-in Spark functions; common examples are encryption, decryption, hashing, JSON parsing, and validation. Databricks recommends UDFs for

ad hoc queries, manual data cleaning, exploratory analysis, and small-to-medium datasets, and recommends built-in Spark functions and methods for large datasets and for workloads that run regularly or continuously, such as ETL jobs and streaming.

Option	Relative performance	Why
Built-in functions, SQL UDFs	Fastest	Run inside the engine with no Python seri
Scala UDFs	Faster than Python	Avoid (or reduce) moving data out of the
pandas UDFs	Up to 100x faster than Python UDFs	Apache Arrow transfers data in columnar
Row-at-a-time Python UDFs	Slowest	Each row is serialized to the Python worke

---

Session-Scoped vs Unity Catalog Governed

Consideration	Unity Catalog governed UDFs
Best for	Sharing functions securely across teams, notebooks, jobs, and pro/serverless SQL warehouses
Languages	SQL, Python, Scala, Java
Governance	Unity Catalog permissions (EXECUTE), discoverable in Catalog Explorer
Persistence	Persisted in Unity Catalog, reusable across sessions
Reach	Every workspace attached to the same metastore (unless the catalog is bound to specific wor

---

Session-Scoped Python UDFs

Scalar Python UDF A scalar UDF receives one row's values and returns one value. The default return type is `StringType`, so declare it explicitly.

```
from pyspark.sql.functions import col, udf
from pyspark.sql.types import StringType


@udf(returnType=StringType())
def format_phone(phone):
    if phone and len(phone) == 10:
        return f"({phone[:3]}) {phone[3:6]}-{phone[6:]}"
    return phone


df_out = df.withColumn("formatted_phone", format_phone(col("phone")))
```

Register the same function for SQL with `spark.udf.register`:

```
from pyspark.sql.types import LongType


def squared(s):
    return s * s


spark.udf.register("squaredWithPython", squared, LongType())
spark.sql("SELECT id, squaredWithPython(id) AS id_squared FROM test")
```

Arrow-optimized Python UDFs An Arrow Python UDF still runs row by row, but uses Apache Arrow for batch transfer and serialization, which also gives more consistent type coercion than pickled UDFs.

```
@udf(returnType="int", useArrow=True)    # Arrow-optimized
def arrow_slen(s):
    return len(s)
```

Enable it for the whole session with `spark.sql.execution.pythonUDF.arrow.enabled=true`; the config applies only when `useArrow` is unset or `None`. In open-source Apache Spark 4.2, Arrow optimization is the default for regular Python UDFs.

Evaluation order and null handling Spark SQL doesn't guarantee the order in which subexpressions are evaluated, and `AND` / `OR` don't short-circuit. A `WHERE s IS NOT NULL AND my_udf(s) > 1` filter can still pass `NULL` into the UDF.

```
spark.udf.register("strlen_nullsafe", lambda s: len(s) if s is not None else -1,
 ↪  "int")
spark.sql("SELECT s FROM test1 WHERE s IS NOT NULL AND strlen_nullsafe(s) > 1")
 ↪  # ok
spark.sql("SELECT s FROM test1 WHERE if(s IS NOT NULL, strlen(s), null) > 1")
 ↪  # ok
```

Make the UDF null-aware, or guard the call with `IF` / `CASE WHEN`. UDFs are deterministic by default; mark random or time-dependent ones with `.asNondeterministic()`.

PySpark UDF limitations

- **Serverless memory:** each PySpark UDF is limited to 1 GB; exceeding it raises `UDF_PYSPARK_USER_CODE_E`. On standard access mode the limit follows the instance's available memory (`UDF_PYSPARK_USER_CODE_ERR`).
- **Broadcast variables and instance profiles** aren't supported in PySpark UDFs on standard access mode or serverless.
- **File access:** on DBR 14.2 and below, PySpark UDFs on shared (standard) clusters can't access Git folders, workspace files, or volumes.
- **Hive UDFs** aren't supported on standard access mode, and serverless doesn't support `CREATE FUNCTION (External)`. Use Unity Catalog functions instead.

---

pandas UDFs (Vectorized)

A pandas UDF uses Apache Arrow to transfer data and pandas to process it in batches. You define it with `@pandas_udf` and Python type hints; the hints determine the UDF type.

Series to Series

```
import pandas as pd
from pyspark.sql.functions import col, pandas_udf


@pandas_udf("double")
def calculate_discount(amount: pd.Series, rate: pd.Series) -> pd.Series:
    return amount * (1 - rate)


df_out = df.withColumn("discounted", calculate_discount(col("amount"),
 ↪  col("rate")))
```

The function must return a Series of the same length as its input. Spark splits each partition into Arrow batches, calls the function per batch, and concatenates the results.

Iterator of Series to Iterator of Series Use this when each executor must do expensive one-time setup, such as loading a model, before processing batches.

```
from typing import Iterator

import pandas as pd
from pyspark.sql.functions import pandas_udf


@pandas_udf("long")
def plus_offset(batch_iter: Iterator[pd.Series]) -> Iterator[pd.Series]:
    offset = 1                      # one-time setup, e.g. load a model, before any
 ↪  batch
    try:
        for batch in batch_iter:
            yield batch + offset
    finally:
        pass                         # release resources here
```

For several input columns, the hint is `Iterator[Tuple[pd.Series, ...]] -> Iterator[pd.Series]`.

Series to scalar (grouped aggregate)

```
@pandas_udf("double")
def mean_udf(v: pd.Series) -> float:
    return v.mean()


df.groupBy("id").agg(mean_udf(df["v"]))
```

A Series-to-scalar UDF works with `select`, `groupBy().agg()`, and windows. It doesn't support partial aggregation; all data for each group is loaded into memory.

pandas function APIs pandas function APIs apply a native Python function to a whole DataFrame or group. Type hints are optional.

```
def subtract_mean(pdf: pd.DataFrame) -> pd.DataFrame:
    return pdf.assign(v=pdf.v - pdf.v.mean())


result = df.groupBy("id").applyInPandas(subtract_mean, schema="id long, v
 ↪  double")
```

API	Input → output
`groupBy().applyInPandas(func, schema)`	Each group as a `pandas.DataFrame` → panda
`mapInPandas(func, schema)`	Iterator of `pandas.DataFrame` → iterator of p
`groupBy().cogroup(other.groupBy()).applyInPandas(...)`	Two co-grouped DataFrames → `pandas.Data`

All rows of a group are loaded into memory before the function runs, and `spark.sql.execution.arrow.maxReco` doesn't apply to groups, so skewed groups can cause out-of-memory errors. On classic compute

`maxRecordsPerBatch` (default 10,000) limits Arrow batch size for pandas UDFs; on serverless the platform manages batch sizing and the setting has no effect.

pandas UDF type	Type hint	Rows out
Series to Series	`pd.Series, ... -> pd.Series`	Same as input
Iterator of Series	`Iterator[pd.Series] -> Iterator[pd.Series]`	Same as input
Iterator of multiple Series	`Iterator[Tuple[pd.Series, ...]] -> Iterator[pd.Series]`	Same as input
Series to scalar	`pd.Series, ... -> scalar`	One per group

SQL UDFs

Scalar SQL UDF

```
CREATE OR REPLACE FUNCTION main.analytics.calculate_bmi(weight DOUBLE, height
 ↪  DOUBLE)
RETURNS DOUBLE
LANGUAGE SQL
DETERMINISTIC
COMMENT 'BMI from kilograms and metres'
RETURN weight / (height * height);

SELECT person_id, main.analytics.calculate_bmi(weight_kg, height_m) AS bmi
FROM main.analytics.person_data;
```

- RETURNS is optional for SQL UDFs; the type is derived from the body.
- The RETURN body is an expression or a query. It can't contain aggregate, window, or ranking functions, or row-producing functions such as explode.
- DEFAULT values on parameters are supported for LANGUAGE SQL only; once one parameter has a default, all following parameters need one.
- SQL and Python UDFs support named parameter invocation (`fn(num_sides => 10, num_dice => 3)`).
- Mark functions such as `rand()`-based ones `NOT DETERMINISTIC`.

SQL table function

```
CREATE OR REPLACE FUNCTION main.hr.getemps(deptno INT)
RETURNS TABLE (name STRING)
RETURN SELECT name FROM main.hr.employee e WHERE e.deptno = getemps.deptno;

SELECT * FROM main.hr.getemps(10);
```

Temporary (session-scoped) functions `CREATE TEMPORARY FUNCTION` creates a function that is visible only in the current session, with no catalog entry. It works with `LANGUAGE SQL` and `LANGUAGE PYTHON`.

Unity Catalog Python UDFs

Create and call

```
CREATE OR REPLACE FUNCTION main.analytics.mask_email(email STRING)
RETURNS STRING
LANGUAGE PYTHON
```

```
DETERMINISTIC
AS $$
parts = email.split('@', 1)
if len(parts) != 2:
    return None
username, domain = parts
return f"{username[0]}{'*' * (len(username) - 2)}{username[-1]}@{domain}"
$$;
```

```
from pyspark.sql.functions import expr

masked = df.withColumn("masked_email", expr("main.analytics.mask_email(email)"))
```

- The Python body sits between `$$` delimiters and must return a value matching `RETURNS`.
- Python UDFs in Unity Catalog need a serverless or pro SQL warehouse, serverless compute, or a cluster on DBR 13.3 LTS or above.
- A view that uses a Unity Catalog Python UDF fails on **classic** SQL warehouses.
- Python UDFs run in an isolated environment without access to file systems or internal services, must handle `NULL` themselves, and follow Databricks SQL type mappings.
- If you omit the catalog and schema, the function is registered in the current schema.

To upgrade a session `@udf` to a governed function, move the body into `CREATE FUNCTION ... LANGUAGE PYTHON AS $$ ... $$`; the syntax and semantics differ from session UDFs.

External libraries: the `ENVIRONMENT` clause

```
CREATE OR REPLACE FUNCTION main.analytics.mixed_process(data STRING)
RETURNS STRING
LANGUAGE PYTHON
ENVIRONMENT (
  dependencies = '["simplejson==3.19.3",
 ↪  "/Volumes/main/libs/pkgs/custom_package-1.0.0-py3-none-any.whl"]',
  environment_version = '6'
)
AS $$
import simplejson as json
import custom_package
return json.dumps(custom_package.process(data))
$$;
```

- `dependencies` is a JSON array string of pip requirement lines: PyPI packages, wheels in Unity Catalog volumes, or HTTPS URLs to wheels.
- `environment_version` is required whenever `ENVIRONMENT` is present. Values are a version of 3 or above, or `'None'` for the default Python environment.
- Supported on serverless notebooks and jobs, pro and serverless SQL warehouses, and classic compute on DBR 16.2+. On DBR 16.2–18.1 only `'None'` is allowed; other values need DBR 18.2+.
- The creator needs `READ VOLUME` on any source volume. With `environment_version` 6+, callers need only `EXECUTE`; with an older or `'None'` version, callers also need `READ VOLUME`.
- Serverless can run on `aarch64` or `x86_64`; a native wheel built for one architecture fails with `ISOLATION_ENVIRONMENT_USER_ERROR.GENERIC`. Ship both variants with `platform_machine` markers.

Features that need an explicit environment version On serverless compute and pro/serverless SQL warehouses, these require `environment_version = '6'` or above in the function definition (classic compute instead needs the listed runtime):

Feature	Classic compute requirement
Named `HANDLER` for a scalar Python UDF	DBR 18.1+
`CREDENTIALS` (service credentials) in a scalar Python UDF	DBR 18.1+
`SECRETS` (Unity Catalog secrets) in scalar or batch UDFs	DBR 19+, standard access mode
More than five UDF calls in one query	DBR 18.1+
PySpark-compatible `TIMESTAMP` input (timezone-naive UTC)	DBR 18.1+

Isolation and determinism

- On DBR 18.1+, Unity Catalog Python UDFs with the same owner and session can share an isolation environment. Add `STRICT ISOLATION` for UDFs that run `eval()` / `exec()`, write local files, modify global or system state, or read environment variables.
- Add `DETERMINISTIC` when the same inputs always produce the same output, so the optimizer can use it. Batch Unity Catalog Python UDFs are treated as non-deterministic unless you declare otherwise.

Batch Unity Catalog Python UDFs

Batch UDFs process an iterator of `pandas.Series` batches instead of one row at a time, while keeping 1:1 input/output row parity. They are generally available and need DBR 16.3+ on classic compute, or serverless compute, or a pro/serverless SQL warehouse.

```
CREATE OR REPLACE FUNCTION main.analytics.calculate_bmi_pandas(weight_kg DOUBLE,
 ↪  height_m DOUBLE)
RETURNS DOUBLE
LANGUAGE PYTHON
DETERMINISTIC
PARAMETER STYLE PANDAS
HANDLER 'handler_function'
AS $$
import pandas as pd
from typing import Iterator, Tuple

model_factor = 1.0   # expensive setup goes here: runs once, outside the per-batch
 ↪  loop

def handler_function(batch_iter: Iterator[Tuple[pd.Series, pd.Series]]) ->
 ↪  Iterator[pd.Series]:
    for weight, height in batch_iter:
        yield weight / (height ** 2) * model_factor
$$;
```

- `PARAMETER STYLE PANDAS` declares batch processing; `HANDLER` names the generator function.
- The handler must yield a Series with the same length as each input batch.
- Put expensive setup outside the handler so it runs once, not per batch.
- Batch UDFs receive timestamps inside `pandas.Series` objects.

User-Defined Table Functions (UDTFs)

A UDTF returns zero or more rows (and possibly several columns) per input and is called in the `FROM` clause.

```sql
CREATE OR REPLACE FUNCTION main.analytics.square_numbers(start INT, end INT)
RETURNS TABLE (num INT, squared INT)
LANGUAGE PYTHON
HANDLER 'SquareNumbers'
DETERMINISTIC
AS $$
class SquareNumbers:
    def eval(self, start: int, end: int):
        for num in range(start, end + 1):
            yield (num, num * num)
$$;

SELECT * FROM main.analytics.square_numbers(1, 5);
```

```python
from pyspark.sql.functions import lit, udtf


@udtf(returnType="sum: int, diff: int")
class GetSumDiff:
    def eval(self, x: int, y: int):
        yield x + y, x - y


GetSumDiff(lit(1), lit(2)).show()
spark.udtf.register("get_sum_diff", GetSumDiff)   # make it callable from SQL
```

- The class has a mandatory `eval` method that yields rows. With TABLE arguments (DBR 17.2+), `eval` runs once per input row and the optional `terminate` method runs once at the end of each partition, which is where aggregations or cleanup go.
- Unity Catalog Python UDTFs run on serverless, standard access mode on DBR 17.1+, and pro/serverless SQL warehouses. They became generally available in DBR 18 on July 28, 2026.
- Session-scoped Python UDTFs are in Public Preview on DBR 14.3 LTS+. If a UDTF receives little input but emits a large table, set `useArrow=True` on `@udtf`.

Governance: Privileges and Sharing

Action	Required privileges
Create a function	`USE CATALOG` + `USE SCHEMA`, `CREATE FUNCTION` on the schema; `SELECT` or
Replace with `CREATE OR REPLACE`	Ownership or `MANAGE` on the function, or on its schema or catalog
Call a function	`EXECUTE` on the function, plus `USE CATALOG` and `USE SCHEMA`

```sql
GRANT EXECUTE ON FUNCTION main.analytics.calculate_bmi TO `data-analysts`;
REVOKE EXECUTE ON FUNCTION main.analytics.calculate_bmi FROM
 ↪  `contractor@example.com`;
```

- Grants in a metastore apply in every workspace attached to that metastore, and a catalog is reachable from all of them unless it's bound to specific workspaces. That's what makes Unity Catalog functions reusable across workspaces.
- SQL and Python UDFs are the building blocks for row filters and column masks; see Access Control.
- Databricks recommends a dedicated catalog and schema for shared UDFs, and a docstring with version, changelog, parameters, return value, and an example.

Compute Support

UDF type	Serverless notebooks/jobs	Classic (standard a
Unity Catalog Python UDF	Yes	DBR 13.3 LTS+
Batch Unity Catalog Python UDF	Yes	DBR 16.3+
Unity Catalog Python UDTF	Yes	DBR 17.1+
Session Python scalar UDF	Yes	DBR 13.3 LTS+
Session pandas UDF / `applyInPandas` / `mapInPandas` / UDTF	Yes	DBR 14.3 LTS+

SQL UDFs (`LANGUAGE SQL`) apply to both Databricks SQL and Databricks Runtime.

Which UDF for Which Constraint

Constraint in the scenario	Choose
Logic expressible in SQL; best performance	Built-in function or SQL UDF
Governed, discoverable, reused across teams and workspaces	Unity Catalog function (`CREATE FUNCTION`
Python logic called from dashboards or a SQL warehouse	Unity Catalog Python UDF on a pro/serverl
Numeric/vectorizable Python over large data in a notebook or job	pandas UDF (Series to Series)
Same, but governed	Batch Unity Catalog Python UDF
Expensive one-time setup (load a model)	Iterator pandas UDF, or batch UDF with se
Custom logic per group, needs the whole group	`groupBy().applyInPandas()`
One input row produces many rows	UDTF (session `@udtf` or Unity Catalog `RETU`
Needs an external PyPI library in a governed function	Unity Catalog Python UDF with `ENVIRONME`
Row filter or column mask	SQL UDF in Unity Catalog
Quick exploration in one notebook	Session-scoped `@udf` / `@pandas_udf`

Use Cases

- **PII masking for BI users**: A Unity Catalog SQL or Python function such as `mask_email` is granted with EXECUTE to an analyst group and used in a view queried from a serverless SQL warehouse.
- **Batch inference in ETL**: An iterator pandas UDF loads a model once per task and scores each Arrow batch, avoiding per-row model loads.
- **Exploding nested API payloads**: A Unity Catalog Python UDTF turns one JSON document per row into one row per line item, callable from SQL.

Common Issues & Errors

UDF receives NULL despite a `WHERE ... IS NOT NULL` filter **Scenario:** `TypeError` inside a UDF on rows that the filter should have removed.

Fix: Spark doesn't guarantee evaluation order; handle `None` inside the UDF or wrap the call in `IF` / `CASE WHEN`.

`UDF_PYSPARK_USER_CODE_ERROR.MEMORY_LIMIT_SERVERLESS` **Scenario:** A PySpark UDF holds a large lookup in memory on serverless.

Fix: Stay under the 1 GB per-UDF limit: replace the lookup with a join or use built-in functions.

View works in a notebook but fails on a SQL warehouse **Scenario:** The view calls a Unity Catalog Python UDF and is queried from a classic SQL warehouse.

Fix: Use a pro or serverless SQL warehouse.

Out-of-memory in `applyInPandas` **Scenario:** One key has millions of rows.

Fix: The whole group is loaded into memory and `maxRecordsPerBatch` doesn't split it; salt or pre-aggregate the skewed key.

Permission denied calling a function **Scenario:** A user can see the table but gets an error calling `main.analytics.mask_email`.

Fix: Grant `EXECUTE` on the function and `USE CATALOG` / `USE SCHEMA` on its parents.

Exam Tips

Tip

- Prefer built-in functions; if a UDF is needed, SQL UDF > pandas UDF > row-at-a-time Python UDF for performance.
- Governed and shared across teams or SQL warehouses → **Unity Catalog function**; quick notebook logic → session-scoped UDF.
- Running a UDF needs **`EXECUTE`** (plus `USE CATALOG` / `USE SCHEMA`); creating needs **`CREATE FUNCTION`** on the schema.
- `PARAMETER STYLE PANDAS` + `HANDLER` = **Batch Unity Catalog Python UDF**; `RETURNS TABLE` + class with `eval` = **UDTF**.
- Python UDF libraries in a governed function go in the **`ENVIRONMENT`** clause (`dependencies`, `environment_version`).
- `applyInPandas` loads each whole group into memory; Series-to-scalar pandas UDFs don't do partial aggregation.
- Serverless: 1 GB memory per PySpark UDF, no broadcast variables, no `CREATE FUNCTION (External)` / Hive UDFs.

Key Takeaways

- **Scope**: session-scoped UDFs disappear with the SparkSession; Unity Catalog functions are persisted securables governed by `EXECUTE` and visible across workspaces on the same metastore.
- **pandas UDF types**: Series to Series, Iterator of Series, Iterator of multiple Series, and Series to scalar; pandas function APIs are `applyInPandas`, `mapInPandas`, and cogrouped `applyInPandas`.
- **Arrow-optimized Python UDFs**: `useArrow=True` or `spark.sql.execution.pythonUDF.arrow.enabled` still row-at-a-time logic but Arrow serialization.
- **SQL UDF syntax**: `CREATE [OR REPLACE] [TEMPORARY] FUNCTION name(params) RETURNS type RETURN expr`; `RETURNS TABLE (...) RETURN SELECT ...` for table functions; `DEFAULT` parameters for SQL only.
- **Unity Catalog Python UDF**: `LANGUAGE PYTHON AS $$ ... $$`; DBR 13.3 LTS+ or pro/serverless SQL warehouse; views using it fail on classic SQL warehouses.
- **Batch UDF**: `PARAMETER STYLE PANDAS` + `HANDLER`, same row count out as in, DBR 16.3+ on classic.
- **UDTF**: Python class with `eval` (and optional `terminate`), `RETURNS TABLE`, called in `FROM`; Unity Catalog Python UDTFs GA in DBR 18.
- **Environment version 6** unlocks named handlers, secrets, service credentials, more than five UDF calls per query, and caller access to volume dependencies without `READ VOLUME`.

Related Topics

- Python Project Structure and Dependencies - Packaging shared code and dependency troubleshooting
- Batch ETL Pipelines — Part 1 - DataFrame transformations that usually replace UDFs
- Access Control - Row filters and column masks built on UDFs
- Unity Catalog - Securables and privilege inheritance

Official Documentation

- What are user-defined functions (UDFs)?
- Python scalar user-defined functions (UDFs)
- pandas user-defined functions
- pandas function APIs
- SQL and Python user-defined functions (UDFs) in Unity Catalog
- Batch Python user-defined functions (UDFs) in Unity Catalog
- Python user-defined table functions (UDTFs) in Unity Catalog
- Python user-defined table functions (UDTFs)
- CREATE FUNCTION (SQL, Python, Scala, and Java)
- Serverless compute limitations
- Standard compute requirements and limitations
- Securable objects in Unity Catalog
- Apache Spark: Arrow in PySpark (Arrow Python UDFs)

Incremental Processing

Incremental processing is essential for efficient data pipelines. Understanding when and how to process only new or changed data can significantly impact performance and cost.

Overview

```
flowchart TD
    subgraph Full Load
        F1[Read All Data] --> F2[Process Everything]
        F2 --> F3[Overwrite Target]
    end

    subgraph Incremental Load
        I1[Track Progress] --> I2[Read New Data Only]
        I2 --> I3[Process Changes]
        I3 --> I4[Merge/Append]
        I4 --> I5[Update Progress]
    end
```

Overview

- **High-water mark pattern**: track progress on a reliable timestamp or sequence column in a control table; it misses updates and deletes without CDC handling
- **`availableNow=True`** replaces the deprecated `once=True` trigger: it processes all available data in multiple batches, respecting rate limits, then stops
- **Checkpoints** give streaming queries exactly-once progress; `startingVersion` / `startingTimestamp` and `ignoreDeletes` / `ignoreChanges` control a Delta streaming source
- **MERGE is idempotent**, which makes it the safe write for reprocessed or `foreachBatch`

batches

What the Exam Tests

- Apply Structured Streaming stateful-processing semantics, including watermarks, output modes, foreachBatch, and checkpoints for fault-tolerant exactly-once state recovery.
- Choose between a full load and an incremental load, and pick an incremental strategy (high-water mark, checkpointed stream, Delta source options) for a source
- Recognise which write patterns stay correct when a batch is rerun

Full Load vs Incremental Load

Aspect	Full Load	Incremental Load
Data processed	All data every time	Only new/changed data
Processing time	Grows with data size	Relatively constant
Cost	Higher	Lower
Complexity	Simple	More complex
Use case	Small datasets, initial loads	Large datasets, frequent updates

When to Use Each

Scenario	Recommendation
Initial data load	Full load
Small reference tables	Full load
Large fact tables	Incremental load
Frequent updates (hourly/daily)	Incremental load
Source supports change tracking	Incremental load
No reliable change indicator	Full load or CDC

Incremental Strategies

```
flowchart TD
    Start[Choose Strategy] --> Source{Source Type?}

    Source -->|Files| Files[File-Based]
    Source -->|Tables| Tables[Table-Based]

    Files --> AL[Auto Loader]
    Files --> Manual[Manual File Tracking]

    Tables --> WM[Watermark Column]
    Tables --> CDF[Change Data Feed]
    Tables --> CDC[External CDC]

    WM --> HWM[High-Water Mark Pattern]
```

Strategy Comparison

Strategy	Best For	Pros	Cons
Auto Loader	File ingestion	Automatic tracking, scalable	Files only
High-Water Mark	Append-only tables	Simple, widely applicable	Misses updates/deletes
Change Data Feed	Delta tables	Captures all changes	Delta only
External CDC	Source systems	Complete change capture	Requires source setup

High-Water Mark Pattern

The most common pattern for incremental processing using a timestamp or sequence column.

Concept

```
sequenceDiagram
    participant Source as Source Table
    participant Control as Control Table
    participant Pipeline as Pipeline
    participant Target as Target Table

    Pipeline->>Control: Get last watermark
    Control-->>Pipeline: 2024-01-15 10:00:00

    Pipeline->>Source: Read WHERE updated_at > watermark
    Source-->>Pipeline: New/changed records

    Pipeline->>Target: Write records
    Pipeline->>Control: Update watermark to max(updated_at)
```

Implementation

```
# Get last processed watermark from control table

last_watermark = spark.sql("""
    SELECT MAX(watermark_value) as watermark
    FROM control.incremental_tracking
    WHERE table_name = 'orders'
""").collect()[0]["watermark"]

# Handle first run (no watermark)

if last_watermark is None:
    last_watermark = "1900-01-01"

# Read only new records

new_records = spark.sql(f"""
    SELECT *
    FROM source.orders
    WHERE updated_at > '{last_watermark}'
""")

# Process and write

(new_records.write
    .format("delta")
```

```
    .mode("append")
    .save("/path/to/target"))

# Update watermark

new_max = new_records.agg(max("updated_at")).collect()[0][0]
if new_max:
    spark.sql(f"""
        INSERT INTO control.incremental_tracking
        VALUES ('orders', '{new_max}', current_timestamp())
    """)
```

Control Table Schema

```
CREATE TABLE control.incremental_tracking (
    table_name STRING,
    watermark_value TIMESTAMP,
    last_updated TIMESTAMP
) USING DELTA;
```

SQL-Based Implementation

```
-- Create or replace view with incremental logic
CREATE OR REPLACE VIEW incremental_orders AS
WITH watermark AS (
    SELECT COALESCE(MAX(watermark_value), '1900-01-01') as wm
    FROM control.incremental_tracking
    WHERE table_name = 'orders'
)
SELECT o.*
FROM source.orders o
CROSS JOIN watermark w
WHERE o.updated_at > w.wm;
```

Checkpoint Management

Checkpoints track streaming progress for exactly-once processing.

Checkpoint Location

```
query = (df.writeStream
    .option("checkpointLocation", "/path/to/checkpoint")
    .start())
```

Checkpoint Structure

Directory	Purpose
`commits/`	Completed micro-batches
`offsets/`	Source positions (Kafka offsets, file paths)
`sources/`	Source-specific metadata
`state/`	Stateful operation data

Checkpoint Recovery When a streaming query restarts:

1. Reads checkpoint to find last committed batch
2. Resumes from that position
3. Reprocesses any incomplete batch (exactly-once)

```
# Streaming query automatically recovers from checkpoint

query = (df.writeStream
    .option("checkpointLocation", "/path/to/checkpoint")
    .start())

# After failure, restart the same query
# It will resume from last checkpoint
```

When to Reset Checkpoints

Scenario	Action
Schema change in source	May need reset
Logic change in transformations	Usually safe to keep
New streaming query	New checkpoint location
Change output location	New checkpoint location
Reprocess all data	Delete checkpoint directory

```
# To reprocess from beginning, delete checkpoint

dbutils.fs.rm("/path/to/checkpoint", recurse=True)
```

Delta Lake as Incremental Source

Streaming from Delta

```
# Basic streaming read

df = (spark.readStream
    .format("delta")
    .load("/path/to/delta/table"))

# Start from specific version

df = (spark.readStream
    .format("delta")
    .option("startingVersion", 10)
    .load("/path/to/delta/table"))

# Start from timestamp

df = (spark.readStream
    .format("delta")
    .option("startingTimestamp", "2024-01-01")
    .load("/path/to/delta/table"))
```

Handling Updates and Deletes By default, streaming from Delta fails if source has updates or deletes.

```python
# Skip deletes

df = (spark.readStream
    .format("delta")
    .option("ignoreDeletes", "true")
    .load("/path/to/table"))

# Skip updates and deletes

df = (spark.readStream
    .format("delta")
    .option("ignoreChanges", "true")
    .load("/path/to/table"))
```

Option	Behavior
Default	Fails on updates/deletes
`ignoreDeletes`	Skips delete operations
`ignoreChanges`	Skips update and delete operations

Deletion Vectors Impact Tables with Deletion Vectors enabled mark rows as deleted without rewriting files. This affects incremental reads:

- Streaming reads handle DVs transparently
- `ignoreDeletes` works normally with DV-enabled tables
- DVs improve UPDATE/DELETE/MERGE performance on source tables
- Periodic VACUUM cleans up DV files, which may trigger file rewrites

Rate Limiting

```python
# Limit files per trigger

df = (spark.readStream
    .format("delta")
    .option("maxFilesPerTrigger", 100)
    .load("/path/to/table"))

# Limit bytes per trigger

df = (spark.readStream
    .format("delta")
    .option("maxBytesPerTrigger", "10g")
    .load("/path/to/table"))
```

MERGE for Incremental Updates

MERGE combines insert, update, and delete in one operation.

Incremental MERGE Pattern

```python
from delta.tables import DeltaTable

# Read incremental changes

changes = spark.sql(f"""
```

```
    SELECT *
    FROM source_table
    WHERE updated_at > '{last_watermark}'
""")

# MERGE into target

target = DeltaTable.forPath(spark, "/path/to/target")

target.alias("t").merge(
    changes.alias("s"),
    "t.id = s.id"
).whenMatchedUpdateAll(
).whenNotMatchedInsertAll(
).execute()
```

MERGE with Delete Handling

```
target.alias("t").merge(
    changes.alias("s"),
    "t.id = s.id"
).whenMatchedDelete(
    condition="s.is_deleted = true"
).whenMatchedUpdateAll(
    condition="s.is_deleted = false"
).whenNotMatchedInsertAll(
    condition="s.is_deleted = false"
).execute()
```

Batch Incremental Patterns

Pattern 1: Timestamp-Based

```
def process_incremental(table_name, source_query, target_path):
    # Get watermark
    watermark = get_watermark(table_name)

    # Read incremental data
    df = spark.sql(source_query.format(watermark=watermark))

    if df.count() > 0:
        # Write to target
        df.write.format("delta").mode("append").save(target_path)

        # Update watermark
        new_watermark = df.agg(max("updated_at")).collect()[0][0]
        update_watermark(table_name, new_watermark)
```

Pattern 2: Version-Based

```
# Track processed Delta version

last_version = get_last_processed_version("orders")

# Get current version
```

```
current_version = (DeltaTable.forPath(spark, source_path).history(1)
    .select("version").collect()[0][0])

# Read changes between versions

changes = (spark.read.format("delta")
    .option("versionAsOf", current_version)
    .load(source_path))

# Filter to only new versions (time travel comparison)
# ... process changes ...

# Update tracked version

update_processed_version("orders", current_version)
```

availableNow Trigger

The `availableNow` trigger processes all available data in multiple batches, then stops.

Use Case Ideal for scheduled incremental jobs that need streaming semantics (checkpoints, exactly-once) but run as batch jobs.

```
# Process all available data, then stop

query = (df.writeStream
    .format("delta")
    .trigger(availableNow=True)
    .option("checkpointLocation", "/checkpoint")
    .start("/output"))

# Wait for completion

query.awaitTermination()
```

Comparison with once=True

Aspect	once=True (Deprecated)	availableNow=True
Batches	Single batch	Multiple batches
Rate limiting	Ignored	Respected
Large backlogs	May OOM	Handles gracefully
Status	Deprecated	Recommended

Scheduled Pipeline Example

```
# Run daily via Lakeflow Jobs

def daily_incremental_job():
    # Read stream from source
    source = (spark.readStream
        .format("delta")
        .option("maxFilesPerTrigger", 1000)
        .load("/source/path"))
```

```
    # Transform
    transformed = source.transform(apply_transformations)

    # Write with availableNow
    query = (transformed.writeStream
        .format("delta")
        .trigger(availableNow=True)
        .option("checkpointLocation", "/checkpoint")
        .start("/target/path"))

    query.awaitTermination()
```

Idempotent Incremental Processing

Ensure same input produces same output, even on rerun.

MERGE for Idempotency

```
def idempotent_incremental(batch_df, batch_id):
    # MERGE handles duplicates naturally
    target = DeltaTable.forPath(spark, "/target")

    target.alias("t").merge(
        batch_df.alias("s"),
        "t.id = s.id"
    ).whenMatchedUpdateAll(
    ).whenNotMatchedInsertAll(
    ).execute()

# Use with foreachBatch

query = (source_stream.writeStream
    .foreachBatch(idempotent_incremental)
    .option("checkpointLocation", "/checkpoint")
    .start())
```

Deduplication Before Write

```
def process_with_dedup(batch_df, batch_id):
    # Deduplicate within batch
    deduped = batch_df.dropDuplicates(["id"])

    # Then MERGE
    # ...
```

Monitoring Incremental Jobs

Track Processing Metrics

```
# Create metrics table

spark.sql("""
    CREATE TABLE IF NOT EXISTS monitoring.incremental_metrics (
        job_name STRING,
        run_time TIMESTAMP,
        records_processed LONG,
```

```
        processing_duration_seconds DOUBLE,
        watermark_before STRING,
        watermark_after STRING
    ) USING DELTA
""")

# Log metrics after each run

def log_metrics(job_name, count, duration, wm_before, wm_after):
    spark.sql(f"""
        INSERT INTO monitoring.incremental_metrics
        VALUES ('{job_name}', current_timestamp(), {count},
                {duration}, '{wm_before}', '{wm_after}')
    """)
```

Alerting on Lag

```
# Check for processing lag

lag_check = spark.sql("""
    SELECT
        table_name,
        watermark_value,
        current_timestamp() - watermark_value as lag
    FROM control.incremental_tracking
    WHERE current_timestamp() - watermark_value > INTERVAL 2 HOURS
""")

if lag_check.count() > 0:
    # Send alert
    pass
```

Use Cases

Understanding when to apply specific incremental patterns is crucial for the exam.

Scenario	Recommended Strategy	Why?
Standard Database Source	High-Water Mark	Simple, effective for append-only or updated-at ti
Delta Lake Source	Delta Source / CDF	Built-in efficiency, handles deletes/updates if conf
Cloud Files Ingestion	Auto Loader	Handles schema evolution and new file discovery
Scheduled Batch Job	`availableNow=True`	Incremental processing with batch scheduling effi
Complex Transformations	Checkpoints + Streaming	Ensures exactly-once processing even during com

Common Issues & Errors

AnalysisException: Checkpoint path does not exist or invalid **Scenario:** Restarting a query with a new location but pointing to old checkpoint.

Fix: Ensure checkpoint path is unique per query. Delete checkpoint if logic changes fundamentally (not recommended for production recovery).

Exam Context: Questions about why a stream failed to start after code changes.

Stream-Static Join Result Empty **Scenario:** Joining a stream with a static table, but updates to the static table aren't reflected.

Fix: With a Delta static table, each micro-batch already reads the latest valid version, so check that the static side is actually a Delta table and that the update was committed before the micro-batch ran. The documented latest-version behavior assumes the static data is stored in Delta Lake (stream-static joins).

Delta Source: "Detected deleted data" **Scenario:** Source Delta table had a DELETE/UPDATE operation.

Fix: Use `.option("ignoreChanges", "true")` or `.option("ignoreDeletes", "true")` if strict consistency isn't required, or use Delta CDF for exact propagation.

Lag Growing Indefinitely **Scenario:** Processing rate is slower than input rate.

Fix: Scale up cluster (add cores), optimize code, or check backpressure configurations.

Best Practices

- Use `availableNow` for scheduled batch-style streaming jobs
- Store watermarks in a control table with audit columns
- Always use checkpoints for streaming queries
- Deduplicate before MERGE to handle source duplicates
- Monitor lag between watermark and current time
- Use `ignoreChanges` when updates don't need downstream propagation
- Test incremental logic with edge cases (empty batches, duplicates)

Exam Tips

1. **High-water mark** requires a reliable timestamp/sequence column in source
2. **Checkpoints** enable exactly-once semantics in streaming
3. **availableNow** is the recommended replacement for `once=True`
4. **ignoreChanges** skips updates and deletes when streaming from Delta
5. **MERGE** is inherently idempotent - same input = same output
6. **startingVersion/startingTimestamp** control where streaming starts
7. **Rate limiting** (maxFilesPerTrigger) prevents OOM on large backlogs
8. **Predictive Optimization** can auto-schedule OPTIMIZE/VACUUM, reducing manual maintenance
9. **Deletion Vectors** improve incremental read performance by marking deletes without rewriting files

Key Takeaways

- **High-water mark pattern** tracks progress using a reliable timestamp or sequence column stored in a control table; it misses updates and deletes without additional CDC handling
- **`availableNow=True`** is the recommended replacement for the deprecated `once=True` trigger; it processes all available data across multiple batches then stops, respecting rate limits
- **Checkpoints** (`checkpointLocation`) provide exactly-once semantics in streaming; delete and recreate them only when changing schema or reprocessing from the beginning
- **`ignoreDeletes`** skips delete operations when streaming from a Delta source; **`ignoreChanges`** skips both updates and deletes
- **`startingVersion` / `startingTimestamp`** options control where a Delta streaming source begins reading
- **MERGE is inherently idempotent**: applying the same CDC batch twice produces the same result, making it the safe choice for `foreachBatch` writes

- **Deletion Vectors** mark rows as deleted without rewriting files, improving incremental read performance on Delta sources
- **Rate limiting** (`maxFilesPerTrigger`, `maxBytesPerTrigger`) prevents OOM when processing large backlogs from Delta or Auto Loader sources

Related Topics

- Structured Streaming - Streaming fundamentals
- Auto Loader - File-based incremental
- Change Data Capture - CDF for incremental
- Delta Lake Operations - MERGE patterns

Official Documentation

- Delta Lake Streaming
- Structured Streaming
- availableNow Trigger

Structured Streaming — Part 1

Structured Streaming is heavily tested on the exam. This part covers streaming fundamentals, sources, sinks, trigger types, output modes, watermarking, windowed aggregations, and stream-stream joins.

Overview

```
flowchart LR
    subgraph Sources
        S1[Delta Tables]
        S2[Kafka]
        S3[Auto Loader]
        S4[Rate Source]
    end

    subgraph Processing
        S1 & S2 & S3 & S4 --> Stream[Streaming DataFrame]
        Stream --> Transform[Transformations]
        Transform --> Stateful[Stateful Operations]
    end

    subgraph Sinks
        Stateful --> D1[Delta Tables]
        Stateful --> D2[Kafka]
        Stateful --> D3[Console]
        Stateful --> D4[foreachBatch]
    end
```

Overview

- **Sources and sinks**: Delta, Kafka, Auto Loader, and rate sources; Delta, Kafka, console, and `foreachBatch` sinks
- **Triggers** (including once vs `availableNow`) and **output modes** (append, `complete`, `update`), with the operations each mode supports

- **Watermarks** bound state for late data in tumbling, sliding, and session windows
- **Stream-stream joins** need watermarks and a time range condition

What the Exam Tests

- Apply Structured Streaming stateful-processing semantics, including watermarks, output modes, foreachBatch, and checkpoints for fault-tolerant exactly-once state recovery.
- Pick the trigger and output mode that a scenario's latency and operation require
- Predict which late events a watermark drops and when window state is cleaned up

Streaming Fundamentals

Micro-Batch vs Continuous

Aspect	Micro-Batch	Continuous
Latency	~100ms minimum	~1ms
Processing	Batch-at-a-time	Record-at-a-time
Guarantees	Exactly-once	At-least-once
Status	Production ready	Experimental

Databricks uses **micro-batch processing** by default, providing exactly-once semantics.

Basic Streaming Query

```
# Read stream from Delta

stream_df = (spark.readStream
    .format("delta")
    .load("/path/to/source"))

# Apply transformations

transformed = (stream_df
    .filter(col("status") == "active")
    .select("id", "name", "amount"))

# Write stream to Delta

query = (transformed.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "/path/to/checkpoint")
    .start("/path/to/target"))
```

Streaming Sources

Delta Table Source

```
# Basic Delta stream

df = spark.readStream.format("delta").load("/path/to/table")
df = spark.readStream.table("catalog.schema.table_name")
```

```python
# Start from specific version

df = (spark.readStream
    .format("delta")
    .option("startingVersion", 10)
    .load("/path/to/table"))

# Start from timestamp

df = (spark.readStream
    .format("delta")
    .option("startingTimestamp", "2024-01-01")
    .load("/path/to/table"))

# Handle schema changes
# Skip updates/deletes

df = (spark.readStream
    .format("delta")
    .option("ignoreChanges", "true")
    .load("/path/to/table"))
```

Option	Behavior
`startingVersion`	Start reading from specific version
`startingTimestamp`	Start reading from specific timestamp
`ignoreDeletes`	Skip delete operations
`ignoreChanges`	Skip update and delete operations
`maxFilesPerTrigger`	Limit files processed per batch
`maxBytesPerTrigger`	Limit bytes processed per batch

Kafka Source

```python
df = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "broker1:9092,broker2:9092")
    .option("subscribe", "topic1,topic2")
    .option("startingOffsets", "latest")
    .load())

# Kafka schema: key, value, topic, partition, offset, timestamp
# Value is binary - parse as needed

parsed = df.select(
    col("key").cast("string"),
    from_json(col("value").cast("string"), schema).alias("data")
).select("key", "data.*")
```

Option	Values
`startingOffsets`	`earliest`, `latest`, or JSON offsets
`subscribe`	Topic names (comma-separated)
`subscribePattern`	Regex pattern for topics

Auto Loader (cloudFiles)

```
df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/path/to/schema")
    .load("/path/to/files"))
```

See Auto Loader for detailed coverage.

Rate Source (Testing)

```
# Generate test data

df = (spark.readStream
    .format("rate")
    .option("rowsPerSecond", 100)
    .load())
# Schema: timestamp, value (Long)
```

Streaming Sinks

Delta Sink

```
query = (df.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "/path/to/checkpoint")
    .start("/path/to/table"))

# Write to table

query = (df.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "/path/to/checkpoint")
    .toTable("catalog.schema.table_name"))
```

Kafka Sink

```
query = (df.selectExpr("CAST(id AS STRING) AS key", "to_json(struct(*)) AS value")
    .writeStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "broker:9092")
    .option("topic", "output_topic")
    .option("checkpointLocation", "/path/to/checkpoint")
    .start())
```

Console Sink (Debugging)

```
query = (df.writeStream
    .format("console")
    .outputMode("append")
    .option("truncate", False)
    .start())
```

foreachBatch Sink

```python
def process_batch(batch_df, batch_id):
    # Custom processing for each micro-batch
    (batch_df.write
        .format("delta")
        .mode("append")
        .save("/path/to/table"))

query = (df.writeStream
    .foreachBatch(process_batch)
    .option("checkpointLocation", "/path/to/checkpoint")
    .start())
```

Trigger Types (Exam Critical)

```mermaid
flowchart TD
    Trigger[Trigger Type] --> Default[Default/Unspecified]
    Trigger --> PT[processingTime]
    Trigger --> Once[once - DEPRECATED]
    Trigger --> AN[availableNow]
    Trigger --> Cont[continuous]

    Default -.- D1[Process immediately when data available]
    PT -.- PT1[Fixed interval batches]
    Once -.- O1[Single batch, then stop]
    AN -.- AN1[All available data, then stop]
    Cont -.- C1[Low-latency experimental]
```

Trigger Comparison

Trigger	Behavior	Use Case
Default	Process ASAP when data arrives	Low-latency streaming
`processingTime("10 seconds")`	Fixed interval micro-batches	Controlled throughput
`once=True`	**Deprecated** - Single batch	Legacy batch-streaming
`availableNow=True`	Process all available, multiple batches	Scheduled batch jobs
`continuous`	Low-latency (~1ms), experimental	Not for production

once vs availableNow (Exam Important)

```python
# DEPRECATED: once=True - processes ONE batch only

query = (df.writeStream
    .trigger(once=True)
    .start())

# RECOMMENDED: availableNow=True - processes ALL available data

query = (df.writeStream
    .trigger(availableNow=True)
    .start())
```

Aspect	once=True	availableNow=True
Batches	Single batch only	Multiple batches (rate limited)
Data processed	May miss data	All available data
Backlog handling	Poor	Excellent
Status	Deprecated	Recommended

Trigger Syntax

```
from pyspark.sql.streaming import Trigger

# Processing time trigger

query = (df.writeStream
    .trigger(processingTime="10 seconds")
    .start())

# Available now trigger (batch-style streaming)

query = (df.writeStream
    .trigger(availableNow=True)
    .start())

# Continuous trigger (experimental)

query = (df.writeStream
    .trigger(continuous="1 second")
    .start())
```

Output Modes (Exam Critical)

```
flowchart TD
    Mode[Output Mode] --> Append
    Mode --> Complete
    Mode --> Update

    Append -.- A1[Only new rows]
    Append -.- A2[Works with most operations]

    Complete -.- C1[Entire result table]
    Complete -.- C2[Required for aggregations]

    Update -.- U1[Only changed rows]
    Update -.- U2[For stateful operations]
```

Mode	Output	Use Case	Restrictions
append	New rows only	Inserts, filters, maps	No aggregations
complete	All rows in result	Aggregations	Must have aggregation
update	Changed rows only	Stateful updates	Not all sinks support

Mode Compatibility

Operation	append	complete	update
select, filter	Yes	No	Yes
Aggregation without watermark	No	Yes	Yes
Aggregation with watermark	Yes (after watermark)	Yes	Yes
mapGroupsWithState	Depends	No	Yes
flatMapGroupsWithState	Depends	No	Yes

Watermarking

Watermarks handle late-arriving data and enable state cleanup in streaming aggregations.

Watermark Concept

```
sequenceDiagram
    participant Events as Event Stream
    participant Watermark as Watermark (max event time - threshold)
    participant State as State Store

    Events->>Watermark: Event time: 10:05
    Note over Watermark: Watermark: 10:05 - 10min = 9:55

    Events->>State: Late event time: 9:50
    Note over State: 9:50 < 9:55 = DROPPED

    Events->>State: Event time: 9:56
    Note over State: 9:56 > 9:55 = PROCESSED
```

Watermark Syntax

```python
from pyspark.sql.functions import window

# Define watermark

df_with_watermark = (df
    .withWatermark("event_time", "10 minutes"))

# Windowed aggregation with watermark

result = (df_with_watermark
    .groupBy(
        window("event_time", "5 minutes"),
        "device_id"
    )
    .agg(avg("temperature").alias("avg_temp")))
```

Watermark Behavior

Scenario	Behavior
Event time > watermark	Processed normally
Event time <= watermark	Dropped (late data)
Watermark advances	State for old windows cleaned up

Key Point: Watermark = max(event_time) - threshold

Choosing Watermark Delay

```
# Short delay: Less memory, drops more late data

df.withWatermark("event_time", "1 minute")

# Long delay: More memory, accepts more late data

df.withWatermark("event_time", "1 hour")
```

Delay	Memory Usage	Late Data Tolerance
Short (minutes)	Low	Low
Long (hours)	High	High

Windowed Aggregations

Tumbling Windows Non-overlapping, fixed-size windows.

```
from pyspark.sql.functions import window

# 5-minute tumbling windows

(df.withWatermark("event_time", "10 minutes")
    .groupBy(
        window("event_time", "5 minutes"),
        "device_id"
    )
    .agg(count("*").alias("event_count")))
```

```
gantt
    title Tumbling Windows (5 min)
    dateFormat HH:mm
    axisFormat %H:%M
    section Windows
    Window 1 :w1, 00:00, 5m
    Window 2 :w2, 00:05, 5m
    Window 3 :w3, 00:10, 5m
```

Sliding Windows Overlapping windows with a slide interval.

```
# 10-minute windows, sliding every 5 minutes

(df.withWatermark("event_time", "10 minutes")
    .groupBy(
        window("event_time", "10 minutes", "5 minutes"),
        "device_id"
    )
    .agg(avg("value").alias("avg_value")))
```

```
gantt
    title Sliding Windows (10 min window, 5 min slide)
    dateFormat HH:mm
    axisFormat %H:%M
    section Windows
    Window 1 :w1, 00:00, 10m
    Window 2 :w2, 00:05, 10m
    Window 3 :w3, 00:10, 10m
```

Session Windows Dynamic windows based on activity gaps.

```
from pyspark.sql.functions import session_window

# Session windows with 5-minute gap

(df.withWatermark("event_time", "10 minutes")
    .groupBy(
        session_window("event_time", "5 minutes"),
        "user_id"
    )
    .agg(count("*").alias("events_in_session")))
```

Window Type	Size	Overlap	Use Case
Tumbling	Fixed	No	Periodic aggregations
Sliding	Fixed	Yes	Moving averages
Session	Dynamic	No	User sessions

Stream-Stream Joins

Inner Join with Watermarks

```
# Both streams need watermarks

impressions = (impressions_df
    .withWatermark("impression_time", "2 hours"))

clicks = (clicks_df
    .withWatermark("click_time", "3 hours"))

# Join with time range condition

joined = impressions.join(
    clicks,
    (impressions.ad_id == clicks.ad_id) &
    (clicks.click_time >= impressions.impression_time) &
    (clicks.click_time <= impressions.impression_time + expr("INTERVAL 1 HOUR"))
)
```

```
sequenceDiagram
    participant L as Left Stream (Impressions)
    participant R as Right Stream (Clicks)
    participant Out as Output

    L->>L: Watermark: 2 hours
    R->>R: Watermark: 3 hours

    L->>Out: Impression at 10:00
    Note over Out: Buffer, wait for click

    R->>Out: Click at 10:30 (same ad_id)
    Note over Out: Match found, emit join result

    Note over L,R: After watermarks pass, state cleaned
```

Stream-Stream Join Requirements

Join Type	Watermark Required	State
Inner	Both sides	Buffered until watermark
Left Outer	Right side required	Left buffered
Right Outer	Left side required	Right buffered
Full Outer	Both sides	Both buffered

Continue reading: Part 2 — Stream-Static Joins, Stateful Operations, State Store Management, Query Management & Exam Tips

---

Structured Streaming — Part 2

Overview

This part covers stream-static joins, stateful operations, state store management, query management, checkpoints, use cases, and exam tips for Structured Streaming.

> For streaming fundamentals, sources, sinks, triggers, output modes, watermarking, and windowed aggregations, see Part 1.

Overview

- **Stream-static joins**, stateful operations, and state timeouts (`ProcessingTimeTimeout`, `EventTimeTimeout`, `NoTimeout`)
- **State stores**: the HDFS-backed default suits small state; RocksDB suits state larger than executor memory
- **`foreachBatch`** runs batch operations such as MERGE inside a stream and, with checkpoints, gives exactly-once semantics
- **Checkpoint compatibility**: changing groupBy keys, watermark delay, output mode, or stateful operators needs a new checkpoint location

What the Exam Tests

- Apply Structured Streaming stateful-processing semantics, including watermarks, output modes, foreachBatch, and checkpoints for fault-tolerant exactly-once state recovery.
- Choose a state store and timeout for a given state size and expiry rule
- Know which query changes break an existing checkpoint, and detect unbounded state with `query.lastProgress`

Stream-Static Joins

Join a streaming DataFrame with a static (batch) DataFrame.

```
# Static dimension table

dim_products = spark.table("catalog.schema.dim_products")

# Stream of events

events_stream = spark.readStream.format("delta").load("/events")

# Join stream with static
```

```
enriched = events_stream.join(
    dim_products,
    events_stream.product_id == dim_products.id,
    "left"
)
```

Important: When the static side is a Delta table, each micro-batch joins against the **latest valid version** of that table, so updates to the static table are picked up by later micro-batches without a restart. Because the static side can change between batches, the output is not deterministic if you reprocess the same stream (stream-static joins).

Stateful Operations

mapGroupsWithState Custom stateful processing with exactly-once semantics.

```
from pyspark.sql.streaming import GroupState, GroupStateTimeout

def update_session(key, events, state: GroupState):
    # Custom state management logic
    if state.exists:
        current_count = state.get
    else:
        current_count = 0

    new_count = current_count + len(list(events))
    state.update(new_count)

    return (key, new_count)

result = (df
    .groupByKey(lambda x: x.user_id)
    .mapGroupsWithState(
        update_session,
        outputMode="update",
        timeoutConf=GroupStateTimeout.ProcessingTimeTimeout
    ))
```

flatMapGroupsWithState Similar to mapGroupsWithState but can emit multiple output records.

```
def emit_alerts(key, events, state: GroupState):
    alerts = []
    # Process events and potentially emit multiple alerts
    for event in events:
        if event.value > threshold:
            alerts.append(Alert(key, event.value, event.timestamp))
    return iter(alerts)
```

State Store Management

Understanding state management is critical for production streaming applications.

State Store Backends

```
# Default: HDFS-based state store

spark.conf.get("spark.sql.streaming.stateStore.providerClass")

# org.apache.spark.sql.execution.streaming.state.HDFSBackedStateStoreProvider

# RocksDB state store (better for large state)

spark.conf.set(
    "spark.sql.streaming.stateStore.providerClass",
    "org.apache.spark.sql.execution.streaming.state.RocksDBStateStoreProvider"
)
```

Backend	Best For	Memory Usage
HDFS (default)	Small to medium state	State in memory
RocksDB	Large state (GB+)	Spills to disk

When to Use RocksDB

- State size exceeds executor memory
- High-cardinality groupBy keys
- Long watermark delays
- Complex aggregations with many groups

```
# RocksDB configuration

spark.conf.set("spark.sql.streaming.stateStore.rocksdb.compactOnCommit", "true")
spark.conf.set(
 ↪  "spark.sql.streaming.stateStore.rocksdb.changelogCheckpointing.enabled",
 ↪  "true")
```

State Monitoring

```
# Monitor state via query progress

progress = query.lastProgress

# State metrics are in stateOperators

if progress and "stateOperators" in progress:
    for op in progress["stateOperators"]:
        print(f"Operator: {op.get('operatorName')}")
        print(f"  numRowsTotal: {op.get('numRowsTotal')}")  # Total rows in state
        print(f"  numRowsUpdated: {op.get('numRowsUpdated')}")  # Rows updated
         ↪  this batch
        print(f"  memoryUsedBytes: {op.get('memoryUsedBytes')}")  # Memory usage
        print(f"  numRowsDroppedByWatermark:
         ↪  {op.get('numRowsDroppedByWatermark')}")
```

Key State Metrics

Metric	Meaning	Alert If
numRowsTotal	Total state size	Growing unboundedly

Metric	Meaning	Alert If
`memoryUsedBytes`	Memory for state	Approaching heap limit
`numRowsDroppedByWatermark`	Late events dropped	Too high (adjust watermark)
`customMetrics.rocksdbMemUsage`	RocksDB memory	Exceeds configured limits

State Timeout Configuration

```python
from pyspark.sql.streaming import GroupStateTimeout

# Processing time timeout - based on clock time

result = df.groupByKey(...).mapGroupsWithState(
    func,
    outputMode="update",
    timeoutConf=GroupStateTimeout.ProcessingTimeTimeout
)

# Event time timeout - based on watermark

result = (df.withWatermark("timestamp", "1 hour")
    .groupByKey(...).mapGroupsWithState(
        func,
        outputMode="update",
        timeoutConf=GroupStateTimeout.EventTimeTimeout
    ))

# No timeout - state never expires (dangerous!)

result = df.groupByKey(...).mapGroupsWithState(
    func,
    outputMode="update",
    timeoutConf=GroupStateTimeout.NoTimeout  # State grows forever!
)
```

Timeout Type	Behavior	Use Case
`ProcessingTimeTimeout`	Expires after wall clock time	Sessions with idle timeout
`EventTimeTimeout`	Expires when watermark passes	Event-time based expiry
`NoTimeout`	Never expires	Small, bounded state only

State Cleanup Strategies

```python
# Use watermarks for automatic cleanup

(df.withWatermark("event_time", "1 hour")
    .groupBy(window("event_time", "10 minutes"))
    .count())  # State cleaned after watermark passes window

# Set timeout in mapGroupsWithState

def update_with_timeout(key, events, state):
    if state.hasTimedOut:
        state.remove()  # Clean up expired state
        return None
```

```
    # ... process events
    state.setTimeoutDuration("30 minutes")  # Reset timeout
```

Debugging State Issues

```
# Check checkpoint directory for state size

dbutils.fs.ls("/checkpoint/state/0/")

# View state schema

spark.read.format("delta").load("/checkpoint/state/0/").printSchema()

# Monitor state growth over time

progress_history = query.recentProgress
for p in progress_history:
    if p and "stateOperators" in p:
        print(f"Batch {p['batchId']}:
         ↪  {p['stateOperators'][0].get('numRowsTotal')} rows")
```

Query Management

Starting Queries

```
# Start with path

query = (df.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoint")
    .start("/output/path"))

# Start with table

query = (df.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoint")
    .toTable("catalog.schema.table"))

# Named query

query = (df.writeStream
    .queryName("my_streaming_query")
    .format("delta")
    .start("/output/path"))
```

Monitoring Queries

```
# Get active queries

spark.streams.active

# Query status

query.status
```

```
# Last progress

query.lastProgress

# Recent progress

query.recentProgress

# Check if running

query.isActive

# Exception (if failed)

query.exception()
```

Stopping Queries

```
# Wait for termination

query.awaitTermination()

# Wait with timeout

query.awaitTermination(timeout=3600)  # 1 hour

# Stop query

query.stop()

# Stop all queries

for q in spark.streams.active:
    q.stop()
```

Checkpoints

Checkpoints store query progress for fault tolerance.

```
query = (df.writeStream
    .option("checkpointLocation", "/path/to/checkpoint")
    .start())
```

Checkpoint Contents

Directory	Contents
`commits/`	Completed batch info
`offsets/`	Source offsets for each batch
`sources/`	Source-specific state
`state/`	Aggregation state
`metadata`	Query metadata

Checkpoint Best Practices

- Use cloud storage (S3, ADLS, GCS) for durability
- One checkpoint location per query
- Don't share checkpoints between different queries
- Keep checkpoints in same region as data

Use Cases

Scenario	Recommended Trigger/Mode	Why?
Real-time Dashboard	Continuous / Low `processingTime`	Minimizes latency for live viewing.
Daily ETL	`availableNow=True`	Processes all data efficiently, then shuts
Aggregates (Counts/Sums)	`complete` Output Mode	Validates total counts, usually requires w
De-duplication	`dropDuplicates` + Watermark	Ensures unique records without unbound

Common Issues & Errors

AnalysisException: Append output mode not supported **Scenario:** Trying to use append mode with aggregations without watermarking.

Fix: Add watermark or switch to `complete` or `update` mode.

Exam Context: Identifying incompatible source/sink/transformation combinations.

Massive State Store Growth (OOM) **Scenario:** Running a stream-stream join or deduplication without watermarks.

Fix: Define watermarks on both sides of join or on the dedup stream to allow state cleanup.

Checkpoint Incompatibility **Scenario:** Changing stateful operations (like grouping keys) and trying to resume from old checkpoint.

Fix: New query structure requires a new checkpoint location.

Output Mode "Update" Not Supported by Sink **Scenario:** Using `update` mode with a file sink (Parquet/ORC).

Fix: File sinks only support append mode. Use Delta sink for `delete`/`update` capabilities (via MERGE in `foreachBatch`).

Best Practices

- Always specify checkpoint location for production queries
- Use watermarks with streaming aggregations
- Choose appropriate trigger based on latency needs
- Monitor query progress with `lastProgress`
- Use `availableNow` for scheduled batch-style streaming
- Test with rate source before connecting production sources

Exam Tips

1. **Triggers**: `availableNow=True` replaces deprecated `once=True` - processes all available data in multiple batches
2. **Output modes**: append for inserts, `complete` for aggregations, `update` for stateful
3. **Watermarks**: Required for streaming aggregations to enable state cleanup
4. **Stream-stream joins**: Both sides need watermarks for inner joins

5. **Checkpoints**: Required for exactly-once semantics and failure recovery
6. **ignoreChanges**: Use when source has updates/deletes you want to skip
7. **foreachBatch**: Enables batch operations (like MERGE) in streaming
8. **RocksDB state store**: Use for large state that exceeds memory
9. **State timeouts**: `ProcessingTimeTimeout` vs `EventTimeTimeout` vs `NoTimeout`
10. **Monitor `numRowsTotal`** in stateOperators to detect unbounded state growth

Key Takeaways

- **Output modes**: `append` for inserts only, `complete` for full aggregation results, `update` for rows changed in the current batch — not every mode is compatible with every operation or sink
- **Watermarks** are required for streaming aggregations to enable state cleanup; without them, state grows unboundedly and causes OOM
- **Stream-static joins** (Delta static side) join each micro-batch against the latest version of the static table; no restart is needed to pick up changes
- **RocksDB state store** is recommended when state exceeds available executor heap memory (GB+ scale or high-cardinality keys); HDFS-backed store is the default and suits smaller state
- **State timeouts**: `ProcessingTimeTimeout` expires based on wall clock; `EventTimeTimeout` expires when the watermark advances past the set timestamp; `NoTimeout` causes unbounded state growth
- **`foreachBatch`** enables batch operations (such as MERGE) within a streaming context, providing exactly-once semantics when combined with checkpoints
- **Checkpoint compatibility**: changing groupBy keys, watermark delay, output mode, or stateful operators requires a new checkpoint location
- **`query.lastProgress`** and `stateOperators[].numRowsTotal` are the primary metrics for detecting unbounded state growth in production

Related Topics

- Incremental Processing - Checkpoint management
- Auto Loader - File ingestion streaming
- Data Deduplication - Streaming dedup

Official Documentation

- Structured Streaming Guide
- Streaming Triggers
- Watermarks

Streaming Joins & Stateful Operations — Part 1

Overview

This part covers advanced Structured Streaming join patterns and arbitrary stateful operations: stream-stream joins, stream-static joins, and `mapGroupsWithState` / `flatMapGroupsWithState`. Advanced watermarking and streaming deduplication are in Part 2.

> For streaming basics (triggers, output modes, sources, sinks, foreachBatch), see Structured Streaming.

Overview

- **Stream-stream joins**: inner joins need watermarks on both sides plus a time range condition; outer joins need a watermark on the opposite side
- **Stream-static joins** need no watermark on the static side; an uncached static DataFrame is re-read each micro-batch
- **Arbitrary state**: `mapGroupsWithState` emits one record per group per batch, `flatMapGroupsWithState` zero or more, with processing-time or event-time timeouts

What the Exam Tests

- Apply Structured Streaming stateful-processing semantics, including watermarks, output modes, foreachBatch, and checkpoints for fault-tolerant exactly-once state recovery.
- Identify the watermark and join-predicate requirements for each stream-stream join type
- Choose between built-in aggregations and custom stateful operations for a scenario

Stream-Stream Joins

Stream-stream joins combine two unbounded streaming DataFrames. Both sides must buffer state until matching records arrive, making watermarks essential for bounding state.

Architecture

```
flowchart LR
    subgraph LeftStream["Left Stream (Impressions)"]
        L1[Event A at 10:00]
        L2[Event B at 10:05]
        L3[Event C at 10:15]
    end

    subgraph StateStore["State Store"]
        LBuf[Left Buffer]
        RBuf[Right Buffer]
    end

    subgraph RightStream["Right Stream (Clicks)"]
        R1[Event X at 10:02]
        R2[Event Y at 10:12]
    end

    LeftStream --> LBuf
    RightStream --> RBuf
    LBuf -- "Match on key + time range" --> Output[Joined Output]
    RBuf -- "Match on key + time range" --> Output

    Output --> Cleanup[Watermark advances -> State cleanup]
```

Inner Stream-Stream Join Both streams must define watermarks. A time range condition in the join predicate bounds how long state is retained.

```
from pyspark.sql.functions import expr

# Define watermarks on BOTH streams
```

```
impressions = (
    spark.readStream
    .format("delta")
    .load("/data/impressions")
    .withWatermark("impression_time", "2 hours")
)

clicks = (
    spark.readStream
    .format("delta")
    .load("/data/clicks")
    .withWatermark("click_time", "3 hours")
)

# Inner join with time range condition

joined = impressions.join(
    clicks,
    expr("""
        impressions.ad_id = clicks.ad_id AND
        click_time >= impression_time AND
        click_time <= impression_time + INTERVAL 1 HOUR
    """)
)

query = (
    joined.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoints/impression_clicks")
    .outputMode("append")
    .start("/output/impression_clicks")
)
```

```
-- SQL equivalent for stream-stream inner join
CREATE OR REFRESH STREAMING TABLE impression_clicks AS
SELECT
    i.ad_id,
    i.impression_time,
    c.click_time,
    c.user_id,
    TIMESTAMPDIFF(SECOND, i.impression_time, c.click_time) AS time_to_click
FROM STREAM(impressions) i
JOIN STREAM(clicks) c
    ON i.ad_id = c.ad_id
    AND c.click_time >= i.impression_time
    AND c.click_time <= i.impression_time + INTERVAL 1 HOUR;
```

Outer Stream-Stream Joins Outer joins produce NULL-padded results when no match is found within the watermark window. The requirements differ by join type.

```
# Left outer join: RIGHT side must have watermark
# Left rows emit with NULLs if no right match found after watermark

left_outer_joined = impressions.join(
    clicks,
```

```
    expr("""
        impressions.ad_id = clicks.ad_id AND
        click_time >= impression_time AND
        click_time <= impression_time + INTERVAL 1 HOUR
    """),
    "leftOuter"
)

# Right outer join: LEFT side must have watermark

right_outer_joined = impressions.join(
    clicks,
    expr("""
        impressions.ad_id = clicks.ad_id AND
        click_time >= impression_time AND
        click_time <= impression_time + INTERVAL 1 HOUR
    """),
    "rightOuter"
)
```

Stream-Stream Join Requirements

Join Type	Left Watermark	Right Watermark	Time Range Condition	Output Mode
Inner	Required	Required	Required for state cleanup	Append
Left Outer	Optional	**Required**	Required	Append
Right Outer	**Required**	Optional	Required	Append
Full Outer	Required	Required	Required	Append

Time Range Condition Explained The time range condition in the join predicate controls how long buffered state is retained on each side.

```
# The time range condition determines state retention
# Without it, state grows unboundedly even with watermarks

# Example: clicks must occur within 1 hour after impression

expr("""
    click_time >= impression_time AND
    click_time <= impression_time + INTERVAL 1 HOUR
""")

# This means:
#   - Impressions are buffered up to: watermark(click) + 1 hour
#   - Clicks are buffered up to: watermark(impression) + 0
#   - Wider range = more state retained = more memory
```

Time Range Width	State Size	Late Match Tolerance	Use Case
1 minute	Small	Very low	Real-time correlation
1 hour	Medium	Moderate	Ad click attribution
24 hours	Large	High	Daily event matching

Practice Question: Stream-Stream Joins You have two streaming DataFrames, `orders` and `shipments`. You want to join them so that every order appears in the output, even if no matching shipment is found within 48 hours. Both streams have watermarks defined. Which join type should you use?

A) Inner join B) Left outer join C) Right outer join D) Cross join

Answer

Correct Answer: B
A left outer join ensures every row from the left stream (orders) appears in the output. If no matching shipment arrives within the time range defined by the watermark, the shipment columns are filled with NULLs. Inner join would drop unmatched orders. Right outer would guarantee all shipments appear, not all orders.

Stream-Static Joins

Stream-static joins combine a streaming DataFrame with a batch (static) DataFrame, commonly used for enrichment lookups such as adding dimension data to streaming events.

Key Behavior

```
flowchart LR
    subgraph Streaming["Streaming Side"]
        S1[Micro-batch 1]
        S2[Micro-batch 2]
        S3[Micro-batch 3]
    end

    subgraph Static["Static Side (Re-read Each Batch)"]
        D[Dimension Table]
    end

    S1 --> |Join| D
    S2 --> |Join| D
    S3 --> |Join| D

    D --> Note["Static table re-read<br>each micro-batch"]
```

Stream-Static Join Code

```
# Static dimension table (read as batch)

dim_products = spark.table("catalog.schema.dim_products")

# Streaming events

event_stream = (
    spark.readStream
    .format("delta")
    .load("/data/sales_events")
)

# Stream-static join (no watermark needed on static side)

enriched = event_stream.join(
```

```
    dim_products,
    event_stream.product_id == dim_products.product_id,
    "left"
)

query = (
    enriched.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoints/enriched_sales")
    .outputMode("append")
    .start("/output/enriched_sales")
)
```

```
-- SQL stream-static join pattern
CREATE OR REFRESH STREAMING TABLE enriched_sales AS
SELECT
    e.*,
    p.product_name,
    p.category,
    p.price
FROM STREAM(sales_events) e
LEFT JOIN dim_products p
    ON e.product_id = p.product_id;
```

Stream-Static vs Stream-Stream Comparison

Aspect	Stream-Static	Stream-Stream
Static side re-read	Each micro-batch	N/A (both streaming)
Watermark required	Not on static side	Both sides (for inner)
State management	No state for static	State for both sides
Dimension changes	Picked up each batch	N/A
Use case	Enrichment / lookup	Correlating two event streams
Memory impact	Low (broadcast possible)	High (state buffering)

When Static Data Changes The static DataFrame is re-read at the start of each micro-batch. If the underlying table is updated between batches, new micro-batches will see the updated data. However, there is no guarantee of consistency within a single micro-batch if the static table is being written to concurrently.

```
# Pattern: Force refresh of static table
# Option 1: The default behavior re-reads automatically

dim_table = spark.table("catalog.schema.dim_products")

# Option 2: Cache control for large static tables

dim_table = spark.table("catalog.schema.dim_products").cache()

# WARNING: Cached tables are NOT re-read each batch
# Only use cache if the static table rarely changes
```

Practice Question: Stream-Static Joins A data engineer has a streaming pipeline that enriches click events with user profile data from a Delta table. The user profile table is updated

hourly. The streaming query uses a processing time trigger of 30 seconds. How often does the streaming query see updates to the user profile table?

A) Only at query start B) Every 30 seconds (each micro-batch) C) Every hour when the table updates D) Never, unless the query is restarted

Answer

Correct Answer: B
In a stream-static join, the static DataFrame is re-read at the start of each micro-batch. With a 30-second processing time trigger, the static table is re-read every 30 seconds, so any updates to the user profile table are picked up at the next micro-batch boundary. This is a key behavioral difference from caching the static table.

Stateful Streaming Operations

mapGroupsWithState vs flatMapGroupsWithState Both enable arbitrary stateful processing beyond built-in aggregations. The key difference is output cardinality.

```
flowchart TD
    Input[Grouped Input Events] --> Decision{Choose Operation}

    Decision -->|Exactly one output per group| MGS[mapGroupsWithState]
    Decision -->|Zero or more outputs per group| FMGS[flatMapGroupsWithState]

    MGS --> Out1[Single Row Output]
    FMGS --> Out2[Iterator of Rows Output]

    MGS -.- Note1["Use for: session summary,<br>running totals, status tracking"]
    FMGS -.- Note2["Use for: alerts, anomaly detection,<br>session windowing with
 ↪  events"]
```

mapGroupsWithState Deep Dive Returns exactly one output record per group per micro-batch.

```
from pyspark.sql.streaming import GroupState, GroupStateTimeout
from pyspark.sql.types import StructType, StructField, StringType, LongType,
 ↪  DoubleType

# Define output schema

output_schema = StructType([
    StructField("device_id", StringType()),
    StructField("event_count", LongType()),
    StructField("avg_temperature", DoubleType()),
    StructField("last_event_time", StringType())
])

def update_device_stats(key, events, state: GroupState):
    """Track running statistics per device."""
    # Handle timeout (state expiry)
    if state.hasTimedOut:
        old_state = state.get
        state.remove()
        return (key[0], old_state["count"], old_state["avg_temp"], "EXPIRED")
```

```
    # Get current state or initialize
    if state.exists:
        current = state.get
    else:
        current = {"count": 0, "sum_temp": 0.0, "avg_temp": 0.0}

    # Process new events
    event_list = list(events)
    new_count = current["count"] + len(event_list)
    new_sum = current["sum_temp"] + sum(e.temperature for e in event_list)
    new_avg = new_sum / new_count
    last_time = str(max(e.event_time for e in event_list))

    # Update state
    new_state = {
        "count": new_count,
        "sum_temp": new_sum,
        "avg_temp": new_avg
    }
    state.update(new_state)

    # Set timeout (state expires if no new events within 30 minutes)
    state.setTimeoutDuration("30 minutes")

    return (key[0], new_count, new_avg, last_time)

# Apply mapGroupsWithState

result = (
    sensor_stream
    .withWatermark("event_time", "10 minutes")
    .groupByKey(lambda row: (row.device_id,))
    .mapGroupsWithState(
        update_device_stats,
        outputSchema=output_schema,
        outputMode="update",
        timeoutConf=GroupStateTimeout.ProcessingTimeTimeout
    )
)
```

flatMapGroupsWithState Deep Dive Returns zero or more output records per group per micro-batch.

```
from typing import Iterator, Tuple

alert_schema = StructType([
    StructField("device_id", StringType()),
    StructField("alert_type", StringType()),
    StructField("value", DoubleType()),
    StructField("timestamp", StringType())
])

def detect_anomalies(
    key: Tuple,
    events: Iterator,
    state: GroupState
```

```
) -> Iterator[Tuple]:
    """Emit alerts when temperature exceeds thresholds."""
    # Handle timeout
    if state.hasTimedOut:
        state.remove()
        return iter([])

    # Get or initialize state
    if state.exists:
        history = state.get
    else:
        history = {"readings": [], "alert_count": 0}

    alerts = []
    event_list = list(events)

    for event in event_list:
        history["readings"].append(event.temperature)

        # Keep only last 10 readings
        if len(history["readings"]) > 10:
            history["readings"] = history["readings"][-10:]

        # Check for spike: current reading > 2x average
        avg = sum(history["readings"]) / len(history["readings"])
        if event.temperature > 2 * avg and len(history["readings"]) >= 5:
            history["alert_count"] += 1
            alerts.append((
                key[0],
                "TEMPERATURE_SPIKE",
                event.temperature,
                str(event.event_time)
            ))

        # Check for sustained high temperature
        if all(r > 100 for r in history["readings"][-5:]):
            alerts.append((
                key[0],
                "SUSTAINED_HIGH_TEMP",
                event.temperature,
                str(event.event_time)
            ))

    # Update state
    state.update(history)
    state.setTimeoutDuration("1 hour")

    return iter(alerts)

# Apply flatMapGroupsWithState

alerts = (
    sensor_stream
    .withWatermark("event_time", "10 minutes")
    .groupByKey(lambda row: (row.device_id,))
```

```
    .flatMapGroupsWithState(
        detect_anomalies,
        outputSchema=alert_schema,
        outputMode="append",
        timeoutConf=GroupStateTimeout.ProcessingTimeTimeout
    )
)
```

GroupState Timeout Types

Timeout Type	Trigger Condition	Requires Watermark	Use Case
`ProcessingTimeTimeout`	Wall clock time since last event	No	Idle session exp
`EventTimeTimeout`	Watermark passes the timeout timestamp	Yes	Event-time bas
`NoTimeout`	Never expires	No	Permanent stat

```
# Processing time timeout
# State expires based on wall clock (system time)

state.setTimeoutDuration("30 minutes")

# Event time timeout
# State expires when watermark passes the set timestamp

state.setTimeoutTimestamp(event_time_ms + 3600000)  # 1 hour in ms

# No timeout (dangerous - state grows forever)
# Only appropriate when number of keys is bounded and small
```

When to Use Custom Stateful Operations vs Built-in

Scenario	Recommended Approach
Windowed counts, sums, averages	Built-in aggregations + watermark
Distinct counts per window	Built-in `approx_count_distinct`
Custom session definitions	`flatMapGroupsWithState`
Running totals across windows	`mapGroupsWithState`
Anomaly detection with history	`flatMapGroupsWithState`
Complex state machines	`flatMapGroupsWithState`
Simple deduplication	`dropDuplicatesWithinWatermark`

Streaming Joins & Stateful Operations — Part 2

Overview

This part covers advanced watermarking (multiple watermarks, the global watermark policy, boundary behavior, and state cleanup) and streaming deduplication, plus use cases, common issues, and exam tips for the whole topic.

> For stream-stream joins, stream-static joins, and `mapGroupsWithState` / `flatMapGroupsWithState`, see Part 1.

Overview

- **Global watermark policy**: with several watermarks, `min` (default) follows the slowest stream and `max` the fastest
- **Boundary behavior**: events with `event_time >= watermark` are processed, earlier events are dropped, and watermark advancement cleans up window state
- **Streaming deduplication**: `dropDuplicatesWithinWatermark` bounds state to the watermark window; pair it with `foreachBatch` + MERGE for end-to-end exactly-once

What the Exam Tests

- Apply Structured Streaming stateful-processing semantics, including watermarks, output modes, foreachBatch, and checkpoints for fault-tolerant exactly-once state recovery.
- Predict which events a watermark keeps and when window state is cleaned up
- Pick a streaming deduplication method that keeps state bounded

Advanced Watermarking

Multiple Watermarks in a Query When a streaming query involves multiple operators that each define watermarks (for example, a stream-stream join followed by a windowed aggregation), Spark must reconcile them using a global watermark policy.

```
# Stream A with 10-minute watermark

stream_a = (
    spark.readStream.format("delta").load("/data/stream_a")
    .withWatermark("event_time_a", "10 minutes")
)

# Stream B with 30-minute watermark

stream_b = (
    spark.readStream.format("delta").load("/data/stream_b")
    .withWatermark("event_time_b", "30 minutes")
)

# Join produces a query with two watermarks

joined = stream_a.join(
    stream_b,
    expr("""
        stream_a.key = stream_b.key AND
        event_time_b >= event_time_a AND
        event_time_b <= event_time_a + INTERVAL 5 MINUTES
    """)
)
```

Global Watermark Policy When multiple watermarks exist, Spark uses a global policy to determine the effective watermark for the entire query.

```
# Set global watermark policy

spark.conf.set(
    "spark.sql.streaming.multipleWatermarkPolicy",
    "min"  # default: uses the MINIMUM watermark
```

```
)

# Alternative: use the maximum watermark

spark.conf.set(
    "spark.sql.streaming.multipleWatermarkPolicy",
    "max"
)
```

Policy	Behavior	Data Safety	State Size
`min` (default)	Uses the slowest watermark	Safer (less data dropped)	Larger (more state retained)
`max`	Uses the fastest watermark	Aggressive (more data dropped)	Smaller (state cleaned sooner

```
flowchart TD
    subgraph Watermarks["Two Watermarks in Query"]
        WA["Watermark A: 10:05<br>(10 min delay)"]
        WB["Watermark B: 09:45<br>(30 min delay)"]
    end

    WA --> Policy{Global Policy}
    WB --> Policy

    Policy -->|min policy| MinResult["Effective Watermark:
↪   09:45<br>(conservative, more state)"]
    Policy -->|max policy| MaxResult["Effective Watermark: 10:05<br>(aggressive,
↪   less state)"]
```

Watermark Propagation Through Operations Watermarks propagate through transformations but have specific rules.

Operation	Watermark Behavior
`filter`, `select`, `map`	Watermark passes through unchanged
`union` of two streams	Min of both watermarks
Stream-stream join	Both watermarks retained, global policy applies
Aggregation	Watermark used for state cleanup
`flatMapGroupsWithState`	Watermark available via `state.getCurrentWatermarkMs()`

Watermark Boundary Edge Cases Understanding exact boundary behavior is critical for exam questions.

```
# Watermark = max(event_time) - threshold
# Example: threshold = 10 minutes, max event_time seen = 10:25
# Watermark = 10:15

# Events arriving at exactly the watermark boundary:
#   event_time = 10:15 -> PROCESSED (not dropped)
#   event_time = 10:14 -> DROPPED (strictly less than watermark)
#   event_time = 10:16 -> PROCESSED
```

Event Time	Watermark (10:15)	Result
10:16	10:15	Processed
10:15	10:15	Processed (at boundary = included)
10:14	10:15	Dropped (before boundary)
10:00	10:15	Dropped

> **Key rule**: Events at exactly the watermark boundary are **included**, not dropped. The condition is `event_time >= watermark`, not `event_time > watermark`.

Impact on State Store Cleanup Watermark advancement triggers state cleanup for completed windows.

```
# Window: 10:00-10:10, Watermark delay: 10 minutes
# When max event_time = 10:25 -> watermark = 10:15
# Window 10:00-10:10 end (10:10) < watermark (10:15) -> state cleaned
# Window 10:05-10:15 end (10:15) <= watermark (10:15) -> state cleaned
# Window 10:10-10:20 end (10:20) > watermark (10:15) -> state retained
```

Practice Question: Watermark Policy A streaming query joins two streams. Stream A has a watermark delay of 5 minutes and Stream B has a watermark delay of 30 minutes. The global watermark policy is set to "min". Stream A's max event time is 11:00 and Stream B's max event time is 10:50. What is the effective watermark for state cleanup?

A) 10:20 (Stream B: 10:50 - 30 min) B) 10:55 (Stream A: 11:00 - 5 min) C) 10:37 (average of both watermarks) D) 10:50 (max of the two event times minus min delay)

Answer

Correct Answer: A
With the "min" policy, the effective watermark is the minimum of all individual watermarks. Stream A's watermark = 11:00 - 5 min = 10:55. Stream B's watermark = 10:50 - 30 min = 10:20. The minimum is 10:20, which becomes the effective global watermark. This is conservative: state is retained longer, reducing the chance of dropping late data from either stream.

Streaming Deduplication

dropDuplicates vs dropDuplicatesWithinWatermark

```
flowchart TD
    subgraph Standard["dropDuplicates (Standard)"]
        S1[Tracks ALL keys seen] --> S2[State grows with every<br>unique key ever
↪   processed]
        S2 --> S3[Without watermark:<br>unbounded state growth]
    end

    subgraph WithinWM["dropDuplicatesWithinWatermark"]
        W1[Tracks keys only within<br>watermark window] --> W2[State bounded
↪   by<br>watermark duration]
        W2 --> W3[Automatically cleaned<br>when watermark advances]
    end
```

dropDuplicatesWithinWatermark This method only deduplicates events that arrive within the watermark window of each other. It is more memory-efficient but may not catch duplicates that arrive far apart.

```python
from pyspark.sql.functions import col

# Efficient: dedup only within watermark window

deduped = (
    event_stream
    .withWatermark("event_time", "15 minutes")
    .dropDuplicatesWithinWatermark(["transaction_id"])
)

query = (
    deduped.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoints/deduped_events")
    .outputMode("append")
    .start("/output/deduped_events")
)
```

```sql
-- SQL streaming deduplication with watermark
CREATE OR REFRESH STREAMING TABLE deduped_transactions AS
SELECT *
FROM STREAM(raw_transactions)
-- Note: In Lakeflow Pipelines, deduplication is handled via AUTO CDC (formerly
 ↪  APPLY CHANGES INTO)
-- For standard SQL streaming, use Python API
```

Deduplication State Comparison

Method	State Contents	State Growth
`dropDuplicates(["id"])`	All unique IDs ever seen	Unbounded (without watermark
`dropDuplicates(["id", "event_time"])`	ID + event_time pairs	Bounded by watermark
`dropDuplicatesWithinWatermark(["id"])`	IDs within watermark window	Bounded by watermark

Exactly-Once Guarantees with Deduplication Combining streaming deduplication with checkpoint-based exactly-once delivery.

```python
def idempotent_dedup_write(batch_df, batch_id):
    """Combine streaming dedup with MERGE for end-to-end exactly-once."""
    from delta.tables import DeltaTable

    # Deduplicate within the micro-batch
    deduped = batch_df.dropDuplicates(["transaction_id"])

    # MERGE into target for cross-batch deduplication
    target = DeltaTable.forName(spark, "catalog.schema.transactions")

    target.alias("t").merge(
        deduped.alias("s"),
        "t.transaction_id = s.transaction_id"
    ).whenNotMatchedInsertAll(
    ).execute()
```

```
# Streaming with foreachBatch for exactly-once dedup

query = (
    event_stream
    .withWatermark("event_time", "10 minutes")
    .dropDuplicatesWithinWatermark(["transaction_id"])
    .writeStream
    .foreachBatch(idempotent_dedup_write)
    .option("checkpointLocation", "/checkpoints/exact_once_dedup")
    .start()
)
```

Practice Question: Streaming Deduplication A streaming pipeline processes IoT sensor readings. Duplicate readings can arrive up to 5 minutes apart. The pipeline must minimize memory usage while preventing duplicates. Which approach is most appropriate?

A) `dropDuplicates(["sensor_id", "reading_time"])` without watermark B) `dropDuplicates(["sensor_id"])` with a 5-minute watermark C) `dropDuplicatesWithinWatermark(["sensor_id", "reading_time"])` with a 5-minute watermark D) `foreachBatch` with MERGE on sensor_id

Answer

Correct Answer: C
`dropDuplicatesWithinWatermark` with a 5-minute watermark is ideal because it only tracks keys within the watermark window, minimizing memory usage. Option A has no watermark, causing unbounded state growth. Option B deduplicates on sensor_id alone, which would drop all but the first reading per sensor. Option D works but uses more resources and is more complex than necessary.

Use Cases

- **Ad click attribution**: join an impressions stream to a clicks stream with watermarks on both sides and a time range condition (for example, clicks within 1 hour of the impression)
- **Dimension enrichment**: join an event stream to a dimension table with a stream-static join; no watermark is needed on the static side
- **Custom sessions and anomaly detection**: use `flatMapGroupsWithState` for custom session definitions, anomaly detection with history, and other state machines
- **Duplicate sensor readings**: use `dropDuplicatesWithinWatermark` with a watermark as long as the gap between duplicates (for example, 5 minutes)

Common Issues & Errors

Join State Grows Despite Watermarks **Scenario:** A stream-stream join has watermarks on both sides, but state keeps growing. **Fix:** Add a time range condition to the join predicate. Watermarks alone do not bound join state.

Stream-Static Join Misses Dimension Changes **Scenario:** Updates to the dimension table stop appearing in the joined output. **Fix:** A cached static DataFrame is not re-read each micro-batch. Remove `.cache()` unless the static table rarely changes.

Deduplication State Runs Out of Memory **Scenario:** `dropDuplicates(["id"])` without a watermark keeps every key ever seen. **Fix:** Add a watermark and use `dropDuplicatesWithinWatermark`, which only tracks keys inside the watermark window.

Group State Never Expires **Scenario:** State kept by `mapGroupsWithState` or `flatMapGroupsWithState` grows forever. **Fix:** `NoTimeout` never expires state. Set `ProcessingTimeTimeout`, or `EventTimeTimeout` together with a watermark.

Exam Tips

1. **Inner stream-stream joins** - watermarks on both sides plus a time range condition in the join predicate
2. **Outer stream-stream joins** - left outer needs the right-side watermark; right outer needs the left-side watermark; full outer needs both
3. **Stream-static joins** - no watermark on the static side; an uncached static side is re-read each micro-batch
4. **`mapGroupsWithState` vs `flatMapGroupsWithState`** - exactly one output record per group vs zero or more
5. **Timeouts** - `ProcessingTimeTimeout` uses wall clock time; `EventTimeTimeout` needs a watermark
6. **Global watermark policy** - `min` (default) keeps more state and drops less late data; `max` cleans state sooner and drops more
7. **Watermark boundary** - events with `event_time >= watermark` are processed; earlier events are dropped
8. **`dropDuplicatesWithinWatermark`** - bounds deduplication state to the watermark window

Key Takeaways

- **Stream-stream inner joins require watermarks on both sides** and a time range condition in the join predicate; watermarks alone are not sufficient — without a time range condition, state grows unboundedly
- **Left outer join**: the right-side stream must have a watermark; right outer join: the left-side stream must have a watermark; full outer: both sides need watermarks
- **Stream-static joins** re-read the static side at each micro-batch (unless cached); no watermark is needed on the static side, and changes to the static table are automatically picked up each batch
- **`mapGroupsWithState`** emits exactly one output record per group per micro-batch; **`flatMapGroupsWithState`** emits zero or more, enabling alerts and complex state machine patterns
- **`ProcessingTimeTimeout`** expires based on wall clock time (no watermark needed); **`EventTimeTimeout`** expires when the watermark advances past a set timestamp (watermark required)
- **Global watermark policy `min`** (default) uses the slowest watermark, retaining more state but reducing late-data loss; `max` uses the fastest watermark, cleaning state sooner but dropping more late events
- **Events at exactly the watermark boundary are included** (condition is `event_time >= watermark`, not `>`); events strictly before the watermark are dropped
- **`dropDuplicatesWithinWatermark`** bounds deduplication state to the watermark window, unlike `dropDuplicates` which tracks all keys ever seen

Related Topics

- Structured Streaming — Part 1 - Triggers, output modes, sources, sinks, and basic watermarking
- Structured Streaming — Part 2 - State store management, `foreachBatch`, and checkpoints

- Streaming Monitoring & Optimization - Back-pressure, monitoring and troubleshooting, state store deep dive
- Data Deduplication - Batch and streaming deduplication patterns

Official Documentation

- Stream-Stream Joins
- Stateful Streaming
- Watermarks
- Streaming Deduplication

Declarative Pipelines

Overview

Lakeflow Pipelines let you declare streaming tables, materialized views, and views in SQL or Python; the engine builds the dependency graph, manages checkpoints and retries, and refreshes results incrementally where it can. This file covers the dataset types, the current `pyspark.pipelines` (dp) API, Auto Loader ingestion, run modes, and the two exam decisions: **streaming table vs materialized view** and **Structured Streaming vs Spark Declarative Pipelines**.

Platform update (Oct 2026)

The exam says **Lakeflow Pipelines**. The docs renamed Lakeflow Declarative Pipelines to **Lakeflow Spark Declarative Pipelines** on Nov 12, 2025, and also use the short form "Lakeflow pipelines". The open-source framework underneath is **Apache Spark™ Declarative Pipelines (SDP)**, part of Apache Spark from 4.1. The Python `dlt` module has been replaced by `pyspark.pipelines` (`from pyspark import pipelines as dp`). `import dlt` still works, but Databricks recommends `pipelines`. Sources: November 2025 release notes, Lakeflow pipelines Python language reference

Overview

- **Streaming table**: each source row is processed once (append-only source); built for ingestion (Auto Loader, Kafka) and AUTO CDC targets
- **Materialized view**: its result always matches the batch query; it refreshes incrementally on serverless when the query allows, and reflects late updates and deletes
- **Python API**: `@dp.table` (streaming table), `@dp.materialized_view`, `@dp.temporary_view`; **SQL**: `CREATE OR REFRESH STREAMING TABLE ... FROM STREAM ...`
- **Lakeflow Pipelines add on top of SDP**: AUTO CDC, expectations, the queryable event log, update flows and foreachBatch sinks, and continuous mode

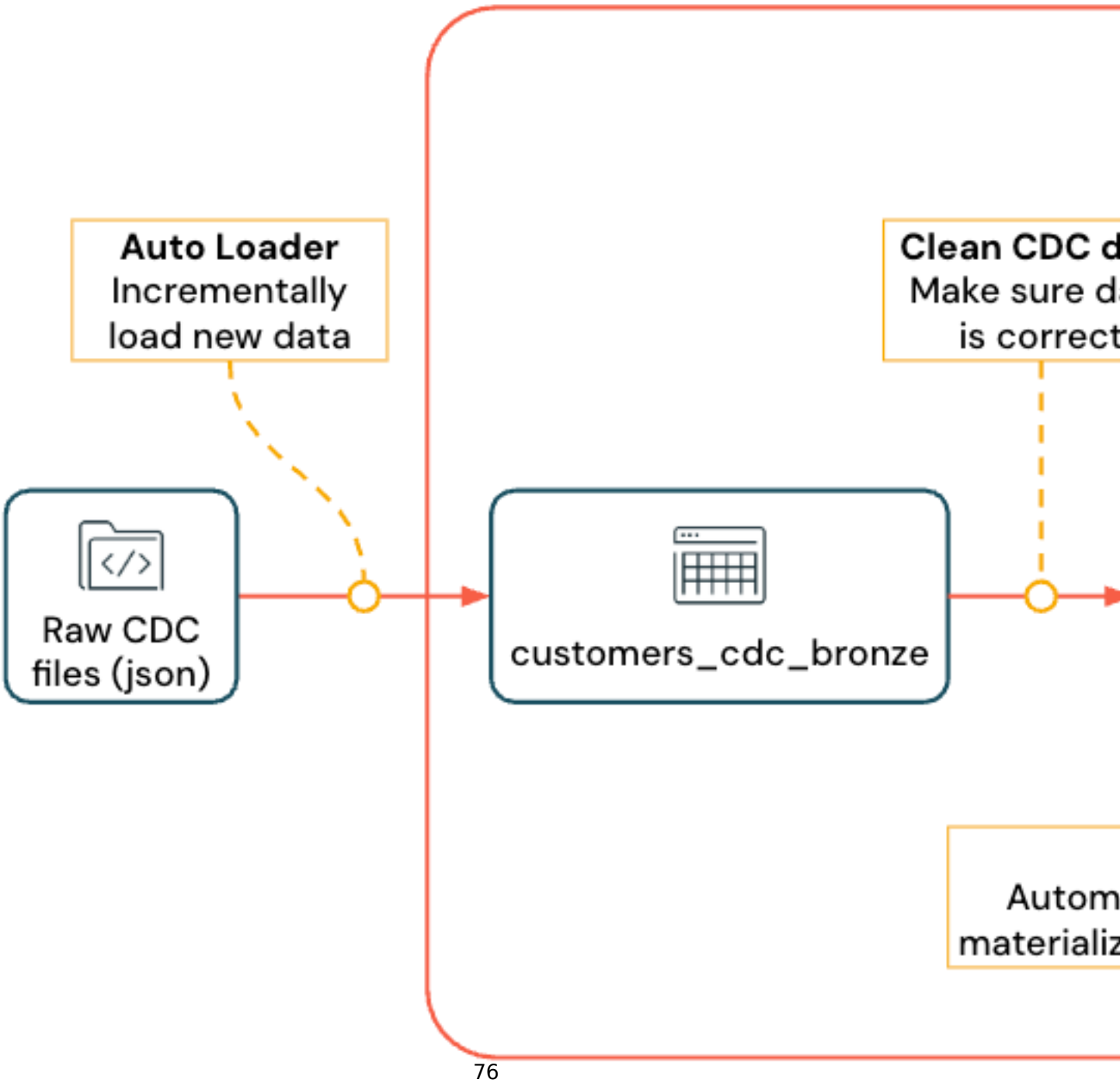
What the Exam Tests

- Create production-ready data pipelines for streaming data using Lakeflow Pipelines and Auto Loader
- Choose between a streaming table and a materialized view for a given latency, cost, and refresh requirement
- Choose between Spark Structured Streaming and Apache Spark™ Declarative Pipelines for scalable ETL given operational constraints
- Predict what a full refresh, a triggered run, or a continuous run does to each dataset type

---

Pipeline Architecture

```
flowchart LR
    subgraph Source["Source code (SQL / Python)"]
        Def[Dataset + flow definitions]
    end
    subgraph Engine["Pipeline engine"]
        Graph[Build dataflow graph]
        Run[Run flows in order,<br>max parallelism, retries]
    end
    subgraph Datasets["Unity Catalog datasets"]
        ST[Streaming tables]
        MV[Materialized views]
        V[Temporary views]
    end
    Def --> Graph --> Run --> Datasets
```


Auto Loader
Incrementally load new data
Clean CDC d
Make sure d
is correct
Raw CDC files (json)
customers_cdc_bronze
Autom
materializ

Lakeflow pipeline DAG showing Bronze → Silver → Gold data flow between tables.

You never call `writeStream`, `save()`, or `toTable()` in pipeline code. Python dataset functions return a DataFrame, and the engine decides how to write it. The docs warn against `collect()`, `count()`, `toPandas()`, `save()`, `saveAsTable()`, `start()`, and `toTable()` inside dataset definitions, because the source code is evaluated several times during planning.

Dataset Types

Type	SQL	Python
Streaming table	`CREATE OR REFRESH STREAMING TABLE`	`@dp.table` returning a streaming DataFrame
Materialized view	`CREATE OR REFRESH MATERIALIZED VIEW`	`@dp.materialized_view` returning a batch Da
View	`CREATE TEMPORARY VIEW`	`@dp.temporary_view`

Both streaming tables and materialized views are Unity Catalog managed **Delta** tables. A streaming table can receive several flows (append flows, AUTO CDC flows); a materialized view always has exactly the one flow implied by its query.

Streaming Table vs Materialized View — Decision Guide

The exam frames this as ”latency, cost, and refresh requirement”. The deciding question is usually: **must the result reflect changes to rows that were already processed?**

Requirement in the scenario	Choose
Ingest ever-growing files or Kafka events exactly once	Streaming table
Aggregate must reflect **late-arriving updates to prior days** or deletes upstream	Materialized view
Join where dimension changes must update old results	Materialized view
Lowest latency (seconds), continuous processing	Streaming table in a con
A full recompute of the source would be cost-prohibitive (very large, append-only)	Streaming table
Source rows get deleted or archived after processing but must stay downstream	Streaming table
Pre-computed gold aggregate read by dashboards on a schedule	Materialized view
Apply inserts/updates/deletes from a CDC feed	Streaming table + AUTO

Model scenario (exam guide sample)

A gold object aggregates daily order totals from bronze; BI reads it on a schedule; the aggregate must reflect the full history, **including late-arriving updates to prior days**. Answer: a **materialized view**. It incrementally maintains the full aggregate, can be refreshed on a schedule, and reflects late changes. A streaming table would only process newly appended rows and would not recompute earlier aggregates.

Refresh, scheduling, and cost levers

- **Incremental refresh needs serverless.** Materialized views refreshed by serverless pipelines (or SQL warehouses) refresh incrementally when the query supports it; on classic compute they are **always fully recomputed**.
- **Cost model**: by default (`REFRESH POLICY AUTO`) Databricks picks the cheaper of incremental refresh and full recompute. `INCREMENTAL` prefers incremental, `INCREMENTAL STRICT` fails rather than fall back to a full recompute, and `FULL` always recomputes. `REFRESH POLICY` is **Beta**.
- **Source tables**: enabling deletion vectors, row tracking, and change data feed on MV sources helps incremental refresh; some operators require row tracking.

- **Check the technique used**: the event log `planning_information` event reports `FULL_RECOMPUTE`, `NO_OP`, or an incremental technique such as `ROW_BASED` or `GROUP_AGGREGATE`.
- **Scheduling**: datasets in a pipeline refresh when the pipeline runs (triggered on a job schedule, or continuously). Standalone MVs and STs created in Databricks SQL can carry `SCHEDULE EVERY <n> HOURS`, `SCHEDULE CRON '...'`, or `TRIGGER ON UPDATE [AT MOST EVERY <interval>]`, and always run in triggered mode.

```
-- Standalone materialized view (Databricks SQL) with its own refresh schedule
CREATE OR REPLACE MATERIALIZED VIEW main.gold.daily_order_totals
SCHEDULE EVERY 1 HOUR
AS SELECT order_date, SUM(amount) AS total_amount
FROM main.silver.orders
GROUP BY order_date;
-- Alternative: TRIGGER ON UPDATE refreshes when upstream Delta sources change
 ↪  (at most once per minute by default)
```

```
flowchart TD
    Q1{Source append-only and<br>each row processed once?} -->|Yes| Q2{Need
 ↪  sub-minute latency or<br>full recompute too costly?}
    Q2 -->|Yes| ST[Streaming table]
    Q2 -->|No| Q3{Result must reflect updates/deletes<br>or late data in old
 ↪  rows?}
    Q1 -->|No: updates/deletes upstream| Q4{CDC feed with keys<br>and a sequence
 ↪  column?}
    Q4 -->|Yes| CDC[Streaming table + AUTO CDC]
    Q4 -->|No| MV[Materialized view]
    Q3 -->|Yes| MV
    Q3 -->|No| ST
```

SQL Syntax

Streaming tables

```
-- Auto Loader ingestion: read_files with the STREAM keyword
CREATE OR REFRESH STREAMING TABLE bronze_orders
COMMENT "Raw orders from the landing volume"
AS SELECT *, _metadata.file_path AS source_file
FROM STREAM read_files(
  '/Volumes/main/landing/orders/',
  format => 'json'
);

-- Kafka ingestion
CREATE OR REFRESH STREAMING TABLE bronze_kafka_events
AS SELECT
  CAST(key AS STRING) AS event_key,
  CAST(value AS STRING) AS event_value,
  topic, partition, offset, timestamp
FROM STREAM read_kafka(
  bootstrapServers => 'broker:9092',
  subscribe => 'events-topic'
);

-- Downstream streaming table with an expectation
CREATE OR REFRESH STREAMING TABLE silver_orders (
  CONSTRAINT valid_order_id EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW
)
```

```sql
CLUSTER BY (order_date)
AS SELECT order_id, customer_id, CAST(order_date AS DATE) AS order_date,
 ↪  total_amount
FROM STREAM bronze_orders
WHERE order_status <> 'cancelled';

-- Streaming read of a source that has update/delete commits
CREATE OR REFRESH STREAMING TABLE silver_orders_from_changing_source
AS SELECT * FROM STREAM main.ops.orders WITH (SKIPCHANGECOMMITS);
```

A streaming read fails if it encounters a change or delete commit in its source. `skipChangeCommits` ignores those commits; it does not propagate them. To propagate changes, read the change data feed and use AUTO CDC.

Materialized views

```sql
CREATE OR REFRESH MATERIALIZED VIEW gold_daily_sales
AS SELECT
  order_date,
  COUNT(*) AS order_count,
  SUM(CAST(total_amount AS DECIMAL(18,2))) AS total_sales
FROM silver_orders
GROUP BY order_date;

-- Explicit refresh policy (Beta)
CREATE OR REFRESH MATERIALIZED VIEW gold_customer_ltv
REFRESH POLICY INCREMENTAL STRICT
AS SELECT c.customer_id, c.segment, SUM(o.total_amount) AS lifetime_value
FROM silver_customers c
LEFT JOIN silver_orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.segment;
```

Do not use STREAM in a materialized view query. Sums over FLOAT/DOUBLE can force a full refresh; casting to DECIMAL keeps the query incrementalizable.

Temporary views

```sql
CREATE TEMPORARY VIEW v_enriched_orders
AS SELECT o.*, p.product_name, p.category
FROM silver_orders o
JOIN silver_products p ON o.product_id = p.product_id;
```

LIVE. is legacy

Older material references pipeline datasets as `LIVE.table_name` and `STREAM(LIVE.table_name)`. The LIVE virtual schema is **deprecated** and belongs to the legacy publishing mode. In the default publishing mode, `LIVE.` is silently ignored and unqualified names resolve to the pipeline's configured catalog and schema. Source: LIVE schema (legacy)

Python Syntax (pyspark.pipelines)

```python
from pyspark import pipelines as dp
from pyspark.sql.functions import col, current_timestamp, count, sum

# Streaming table: @dp.table + a streaming read
```

```python
@dp.table(name="bronze_orders", comment="Raw orders from the landing volume")
def bronze_orders():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.inferColumnTypes", "true")
        .load("/Volumes/main/landing/orders/")
    )

# Streaming table with expectations, reading another pipeline dataset
@dp.table(name="silver_orders", cluster_by=["order_date"])
@dp.expect_or_drop("valid_order_id", "order_id IS NOT NULL")
@dp.expect_or_drop("valid_amount", "total_amount > 0")
def silver_orders():
    return (
        spark.readStream.table("bronze_orders")
        .filter(col("order_status") != "cancelled")
        .select(
            "order_id",
            "customer_id",
            col("order_date").cast("date").alias("order_date"),
            col("total_amount").cast("decimal(18,2)").alias("total_amount"),
            current_timestamp().alias("processed_at"),
        )
    )

# Materialized view: @dp.materialized_view + a batch read
@dp.materialized_view(name="gold_daily_sales", refresh_policy="auto")
def gold_daily_sales():
    return (
        spark.read.table("silver_orders")
        .groupBy("order_date")
        .agg(count("*").alias("order_count"),
 ↪  sum("total_amount").alias("total_sales"))
    )

# Temporary view: computed when queried, not published
@dp.temporary_view(name="v_recent_orders")
def v_recent_orders():
    return spark.read.table("silver_orders").filter("order_date >=
     ↪  date_sub(current_date(), 30)")
```

Older `dlt` code	Current `dp` code
`import dlt`	`from pyspark import pipelines as dp`
`@dlt.table` returning a batch DataFrame	`@dp.materialized_view` (`@dp.table` still works for both, but MVs
`@dlt.view`	`@dp.temporary_view`
`@dlt.table(temporary=True)`	`@dp.table(private=True)` (private: not published to the catalog,
`dlt.read("t")` / `dlt.read_stream("t")`	Current docs examples use `spark.read.table("t")` / `spark.rea`
`dlt.apply_changes(...)`	`dp.create_auto_cdc_flow(...)`
`@dlt.expect...`	`@dp.expect...`

Common decorator arguments (`@dp.table`, `@dp.materialized_view`): `name`, `comment`, `spark_conf`, `table_properties`, `path`, `partition_cols`, `cluster_by`, `cluster_by_auto`, `schema`, `row_filter` (Public Preview), `private`. `refresh_policy` exists only on `@dp.materialized_view`.

Python loops that define tables

Pipelines evaluate dataset functions lazily. A function inside a `for` loop that reads the loop variable directly makes every table load the **last** value. Pass the value as a default argument (`def t(name=t_name)`) or wrap the decorator in a factory function. Source: Develop pipeline code with Python

Auto Loader in Pipelines

Databricks recommends Auto Loader plus streaming tables for incremental ingestion from cloud object storage or Unity Catalog volumes. In SQL, `read_files` invokes Auto Loader when used with STREAM (older material uses `cloud_files()`); in Python, use `spark.readStream.format("cloudFiles")`.

```sql
CREATE OR REFRESH STREAMING TABLE bronze_events
AS SELECT * FROM STREAM read_files(
  '/Volumes/main/landing/events/',
  format => 'json',
  schemaHints => 'event_id STRING, event_time TIMESTAMP',
  schemaEvolutionMode => 'addNewColumns'
);
```

```python
@dp.table(name="bronze_events")
def bronze_events():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.schemaHints", "event_id STRING, event_time TIMESTAMP")
        .option("cloudFiles.schemaEvolutionMode", "rescue")
        .load("/Volumes/main/landing/events/")
    )
```

In a pipeline you do **not** set `cloudFiles.schemaLocation` or a checkpoint location; Databricks manages schema location and checkpoint information for you.

`cloudFiles.schemaEvolutionMode`	Behavior when a new column appears
`addNewColumns` (default when no schema is provided)	Stream fails with `UnknownFieldException` after addi
`addNewColumnsWithTypeWidening`	Like `addNewColumns`, and widens supported types (int
`rescue`	Schema never evolves and the stream does not fail; r
`failOnNewColumns`	Stream fails and does not restart until you update the
`none` (default when a schema is provided)	Schema does not evolve, new columns are ignored, th

Because `addNewColumns` deliberately fails the stream once, the docs recommend running Auto Loader so it restarts automatically (for example, with Lakeflow Jobs); after the restart it continues with the evolved schema. For more Auto Loader detail see Auto Loader.

Reading Message Buses and Existing Tables

```python
@dp.table(name="bronze_kafka")
def bronze_kafka():
    return (
        spark.readStream.format("kafka")
        .option("kafka.bootstrap.servers", "broker:9092")
        .option("subscribe", "orders")
        .option("startingOffsets", "earliest")
```

```
        .load()
        .selectExpr("CAST(key AS STRING) AS key", "CAST(value AS STRING) AS
↪   value", "timestamp")
    )

# Existing Unity Catalog table as a streaming source
@dp.table(name="orders_from_delta")
def orders_from_delta():
    return spark.readStream.table("main.ops.orders")

# Small or static reference data: batch read into a materialized view
@dp.materialized_view(name="dim_country")
def dim_country():
    return spark.read.table("main.ref.countries")
```

Do not define a materialized view directly over Kafka: Kafka does not keep history, so a recompute cannot rebuild the result.

Spark Structured Streaming vs Spark Declarative Pipelines — Decision Guide

Both run the same Spark streaming engine. The difference is **who owns the operational work**: you (Structured Streaming in a notebook or script scheduled by Lakeflow Jobs), or the pipeline engine (SDP / Lakeflow Pipelines).

Concern	Spark Structured Streaming (+ Lakeflow Jobs)
Programming model	Procedural: you write `readStream ... writeStream ... start()`
Checkpoints	You set and manage `checkpointLocation` per query
Orchestration and dependencies	You order queries and tasks (multiple tasks or queries in a job)
Retries and recovery	Configure the job (continuous trigger restarts failed runs with exponential b
Sinks and custom writes	Any sink: `foreachBatch`, `foreach`, JDBC, multiple targets in one batch
Output modes and triggers	Full control (append, update, complete; `processingTime`, `availableNow`, re
Data quality	Hand-written filters and assertions
CDC / SCD	Hand-written `MERGE` in `foreachBatch` (with ordering and dedup logic)
Lineage and monitoring	Streaming query listener / metrics you collect
Continuous on serverless	Serverless notebooks and jobs support only `Trigger.AvailableNow()` and
Portability	Any Spark runtime

Choose **Structured Streaming** when you need fine-grained control the pipeline does not expose: custom `foreachBatch` logic across several systems, complete output mode, arbitrary stateful processing you tune yourself, explicit checkpoint management, or a specific trigger. Choose **Declarative Pipelines** when the priority is managed orchestration, automatic retries and checkpoints, expectations, AUTO CDC, and lineage with less code. Databricks recommends Lakeflow pipelines for most new ETL workloads.

```
# Structured Streaming: you own trigger, checkpoint, sink, and idempotency
from delta.tables import DeltaTable

def upsert_batch(batch_df, batch_id):
    (DeltaTable.forName(spark, "main.silver.orders").alias("t")
        .merge(batch_df.alias("s"), "t.order_id = s.order_id")
        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute())

(spark.readStream.table("main.bronze.orders")
```

```
    .writeStream
    .foreachBatch(upsert_batch)          # at-least-once: keep it idempotent
    .option("checkpointLocation", "/Volumes/main/chk/orders_upsert")
    .trigger(availableNow=True)
    .start())

# Same intent declaratively: the pipeline owns checkpoint, retries, ordering
from pyspark import pipelines as dp

dp.create_streaming_table("silver_orders")
dp.create_auto_cdc_flow(
    target="silver_orders",
    source="bronze_orders",
    keys=["order_id"],
    sequence_by="updated_at",
    stored_as_scd_type=1,
)
```

Middle ground: `@dp.foreach_batch_sink` gives pipeline-managed checkpoints with custom per-batch Python. You still own downstream cleanup: on a full refresh `batch_id` restarts at 0 and the pipeline does not clean data it wrote to external targets.

Pipeline Configuration



Lakeflow pipeline settings panel showing target schema, development mode, and trigger type.

```
# databricks.yml (Declarative Automation Bundle) — pipeline resource
resources:
  pipelines:
    orders_pipeline:
      name: "orders-pipeline-${bundle.target}"
      catalog: ${var.catalog}
      schema: sales           # use schema; `target` is the deprecated legacy field
      serverless: true          # recommended for new pipelines; omit clusters
      continuous: false         # triggered; run on a job schedule
      channel: CURRENT          # PREVIEW tests upcoming runtime changes
      configuration:
        source_path: /Volumes/main/landing/orders/
      libraries:
        - glob:
            include: ../src/transformations/**
      notifications:
        - email_recipients:
            - data-team@example.com
          alerts:
            - on-update-failure
            - on-flow-failure
```

Read a configuration value in Python with `spark.conf.get("source_path")`. Pipeline **parameters** (`parameters:` map, referenced as `:name` in SQL source) are a separate feature for SQL code. Editions: `CORE` (streaming ingest), `PRO` (adds CDC), `ADVANCED` (adds expectations, the default). Serverless pipelines can use AUTO CDC.

Run Modes, Retries, and Refresh Semantics

	Triggered (`continuous: false`, default)	Continuous (`continuous: true`)
Stops	When all tables are refreshed from data available at start	Runs until manually stopped
Freshness fit	Every 10 minutes, hourly, daily	Every 10 seconds to a few minutes
Cost	Compute only while updating	Always-running compute

Pipeline mode is independent of dataset type: both STs and MVs update in either mode. Databricks recommends running a continuous pipeline from a **continuous job** rather than the pipeline's built-in continuous setting; the job's mode overrides the pipeline's.

Run behavior depends on how the update starts. "Run now" or editor updates of a triggered pipeline use fast-start, debugging-focused behavior: the cluster is reused (two hours by default) and pipeline retries are disabled so errors surface immediately. Updates started from Jobs or the API, and all continuous updates, use automatic retry and restart behavior. `pipelines.maxFlowRetryAttempts` defaults to two retries per flow; `pipelines.numUpdateRetryAttempts` defaults to five for triggered pipelines and unlimited for continuous ones. The `development` flag (default `false`) selects development or production mode.

Update type	Materialized view	Streaming table
Refresh (default)	Brought up to date; incremental when cheaper	Processes only new
Full refresh	Recomputed from sources	**Clears data and c**
Reset streaming flow checkpoints (API)	Not applicable	Clears checkpoints

```
# Start an update; --full-refresh resets all tables first
databricks pipelines start-update <pipeline-id>
databricks pipelines start-update <pipeline-id> --full-refresh

# Refresh only selected tables (request body fields of the updates API)
databricks pipelines start-update <pipeline-id> \
  --json '{"refresh_selection": ["silver_orders"], "full_refresh_selection":
  ↪  ["gold_daily_sales"]}'
```

A full refresh of a streaming table can **lose data** when the source no longer holds it (short Kafka retention, lifecycle-deleted files, compliance deletes). Set `pipelines.reset.allowed = false` on tables that must never be fully refreshed.

Use Cases

- **Clickstream ingestion with a gold rollup**: a streaming table ingests JSON events from a volume with Auto Loader; a materialized view computes daily active users and is refreshed incrementally on a serverless pipeline that a job triggers hourly.
- **Schema drift in front-end events**: Auto Loader `rescue` mode keeps the bronze stream running when developers add fields; new fields land in `_rescued_data` for later promotion.
- **Choosing the engine**: a team that must write each micro-batch to an operational database and a Kafka topic, with custom retry logic, keeps that job on Structured Streaming `foreachBatch`; the rest of the medallion flow moves to a Lakeflow pipeline for expectations and lineage.

Common Issues & Errors

Symptom	Likely cause
Streaming table fails reading an upstream table	Upstream had UPDATE/DELETE/MERGE commits
Gold aggregate ignores corrections to old rows	Built as a streaming table
MV is fully recomputed every run	Classic compute, unsupported operator, FLOA
Changing a streaming table's query does not change old rows	Each row is processed once
Data missing after a full refresh	Source retention shorter than table history
All loop-generated tables contain the same data	Late binding of the loop variable
Unqualified table not found after migrating code	Default publishing mode resolves names to th
Unbounded stateful stream fails with memory pressure	No watermark on an aggregation or join

Exam Tips

Tip

- **Late updates to old aggregates → materialized view.** Append-only, exactly-once, lowest latency → streaming table.
- **Kafka or Auto Loader ingest → streaming table**, never an MV (the source cannot be recomputed).
- **Incremental MV refresh requires serverless**; on classic compute MVs fully recompute.
- **Full refresh of a streaming table clears data and checkpoints** and can drop records the source no longer holds.
- **Structured Streaming** = control (`foreachBatch`, custom sinks, explicit checkpoints, any output mode). **Declarative Pipelines** = managed orchestration, retries, checkpoints, expectations, AUTO CDC, lineage.
- **AUTO CDC, expectations, the event log, and continuous mode are Lakeflow-only**; plain SDP on Apache Spark does not have them.
- Current Python API: `from pyspark import pipelines as dp`; `@dp.table` (ST), `@dp.materialized_view`, `@dp.temporary_view`. `LIVE.` is legacy.

Key Takeaways

- **Streaming tables** process each appended source row once and are the default for ingestion and AUTO CDC targets; **materialized views** always equal the batch result of their query and reflect late changes; **views** are not persisted.
- **The exam decision**: if the result must reflect updates or late data in previously processed rows, choose a materialized view; if you need exactly-once, low-latency processing of a growing source, choose a streaming table.
- **Cost**: MVs refresh incrementally only on serverless, using a cost model (`REFRESH POLICY` can force or forbid incremental). STs avoid recompute entirely but cannot revise old rows without a full refresh.
- **Auto Loader in pipelines**: `STREAM read_files(...)` in SQL or `format("cloudFiles")` in Python; schema location and checkpoints are managed by the pipeline.
- **Structured Streaming vs SDP**: choose Structured Streaming for control over `foreachBatch`, sinks, state, triggers, and checkpoints; choose Declarative Pipelines for managed orchestration, retries, expectations, AUTO CDC, and lineage.
- **Run behavior**: UI/editor runs of triggered pipelines disable retries for fast debugging; job/API and continuous runs retry automatically.
- **Naming**: exam "Lakeflow Pipelines" = docs "Lakeflow Spark Declarative Pipelines"; Python moved from `dlt` to `pyspark.pipelines` (`dp`), with `import dlt` still working.

Related Topics

- AUTO CDC API — CDC flows into streaming tables, SCD Type 1 and 2
- Expectations and Data Quality — warn, drop, fail, quarantine
- Structured Streaming — Part 1 — triggers, output modes, checkpoints
- Streaming Joins and Stateful Processing — watermarks and state
- Auto Loader — file ingestion options
- Lakeflow Jobs — Part 1 — scheduling pipelines
- Lakeflow Event Logs — monitoring updates and refresh techniques

Official Documentation

- Pipeline concepts
- Streaming tables
- Materialized views
- Incremental refresh for materialized views
- REFRESH POLICY clause (pipelines)
- Apache Spark Declarative Pipelines
- Lakeflow pipelines Python language reference
- Pipeline SQL language reference
- CREATE STREAMING TABLE (pipelines)
- CREATE MATERIALIZED VIEW (Databricks SQL)
- Load data in pipelines
- Auto Loader schema inference and evolution
- Triggered vs. continuous pipeline mode
- Run a pipeline update
- Pipeline properties reference
- Use ForEachBatch to write to arbitrary data sinks in pipelines
- Procedural vs. declarative data processing
- Production considerations for Structured Streaming
- `databricks pipelines` CLI commands

AUTO CDC API

Overview

The AUTO CDC APIs apply a change feed (inserts, updates, deletes) to a streaming table in a Lakeflow pipeline. They handle out-of-order events and build SCD Type 1 or Type 2 tables without hand-written `MERGE` logic. `AUTO CDC` reads a CDC feed; `AUTO CDC FROM SNAPSHOT` derives the changes by comparing full snapshots.

Platform update (Oct 2026)

The AUTO CDC APIs **replace the APPLY CHANGES APIs and have the same syntax**. APPLY CHANGES still works, but Databricks recommends AUTO CDC. In Python, `dp.create_auto_cdc_flow()` replaces `apply_changes()` and `dp.create_auto_cdc_from_snapshot_flow()` replaces `apply_changes_from_snapshot()`, each with the same signature. Sources: The AUTO CDC APIs, create_auto_cdc_flow

Overview

- **Target is always a streaming table**, declared first (`CREATE OR REFRESH STREAMING TABLE` / `dp.create_streaming_table()`), then populated by an AUTO CDC flow

- **Required inputs**: KEYS / keys (row identity) and SEQUENCE BY / sequence_by (event order); STORED AS / stored_as_scd_type is optional and defaults to **SCD Type 1**
- **SCD Type 2** adds __START_AT and __END_AT, filled with **sequence values**; the current row has __END_AT IS NULL
- **Lakeflow-only**: AUTO CDC is not part of open-source Apache Spark Declarative Pipelines; it needs serverless or the Pro/Advanced edition

What the Exam Tests

- Implement CDC pipelines using AUTO CDC APIs (formerly APPLY CHANGES) in Lakeflow Pipelines, including SCD Type 1 and Type 2 via stored_as_scd_type
- Configure keys, sequence_by, apply_as_deletes, column selection, and track_history_column_list for a scenario
- Explain how late or out-of-order events are resolved and why __END_AT IS NULL identifies current rows
- Translate older APPLY CHANGES / dlt.apply_changes code to AUTO CDC

How AUTO CDC Works

```
flowchart LR
    Feed["CDC feed<br>(op, key, sequence, values)"] --> View["Source
↪   dataset<br>(view / streaming table)"]
    View --> Flow["AUTO CDC flow<br>KEYS + SEQUENCE BY"]
    Flow -->|STORED AS SCD TYPE 1| T1["Streaming table<br>latest row per key"]
    Flow -->|STORED AS SCD TYPE 2| T2["Streaming table<br>history with __START_AT
↪   / __END_AT"]
```

Concept	Meaning in AUTO CDC
Keys	Column(s) that uniquely identify a row; match events to target rows
Sequence	Sortable column (or STRUCT) giving the logical order of events; must be non-null, with one dis
Default action	INSERT and UPDATE events are **upserts**: update the matching row, or insert if none exists
Delete	Only when APPLY AS DELETE WHEN / apply_as_deletes matches
Truncate	Only when APPLY AS TRUNCATE WHEN / apply_as_truncates matches; **SCD Type 1 only**

Requirements: the pipeline must run on serverless or use the Pro or Advanced edition. The source must be a **streaming** source (read it with STREAM in SQL or spark.readStream in Python).

Name Mapping: APPLY CHANGES → AUTO CDC

Older material	Current
APPLY CHANGES INTO LIVE.target FROM STREAM(LIVE.src) ...	CREATE FLOW f AS AUTO CDC INTO tar
dlt.apply_changes(...)	dp.create_auto_cdc_flow(...) (same
dlt.apply_changes_from_snapshot(...)	dp.create_auto_cdc_from_snapshot_f
dlt.create_streaming_table(...)	dp.create_streaming_table(...) (cr
import dlt	from pyspark import pipelines as d

Clauses and arguments (KEYS, SEQUENCE BY, APPLY AS DELETE WHEN, COLUMNS * EXCEPT, STORED AS SCD TYPE, TRACK HISTORY ON, stored_as_scd_type, track_history_column_list, and so on) are unchanged.

SQL Syntax

```
CREATE OR REFRESH STREAMING TABLE table_name;

CREATE FLOW flow_name AS AUTO CDC [ONCE] INTO table_name
FROM source
KEYS (keys)
[IGNORE NULL UPDATES [ON {columnList | * EXCEPT (exceptColumnList)}]]
[APPLY AS DELETE WHEN condition]
[APPLY AS TRUNCATE WHEN condition]
SEQUENCE BY orderByColumn
[SYSTEM SEQUENCE BY systemOrderByColumn]
[COLUMNS {columnList | * EXCEPT (exceptColumnList)}]
[STORED AS {SCD TYPE 1 | SCD TYPE 2 | BITEMPORAL}]
[TRACK HISTORY ON {columnList | * EXCEPT (exceptColumnList)}]
[COLUMNS TO UPDATE columnName]
```

`KEYS` and `SEQUENCE BY` are required; everything else is optional. Expectations on the target use the normal `CONSTRAINT ... EXPECT` clause.

SCD Type 1 (latest value only)

```
CREATE OR REFRESH STREAMING TABLE customers_current;

CREATE FLOW customers_current_cdc AS AUTO CDC INTO customers_current
FROM STREAM(main.cdc.customers_feed)
KEYS (customer_id)
APPLY AS DELETE WHEN operation = 'DELETE'
APPLY AS TRUNCATE WHEN operation = 'TRUNCATE'
SEQUENCE BY sequence_num
COLUMNS * EXCEPT (operation, sequence_num)
STORED AS SCD TYPE 1;
```

SCD Type 2 (full history)

```
CREATE OR REFRESH STREAMING TABLE customers_history;

CREATE FLOW customers_history_cdc AS AUTO CDC INTO customers_history
FROM STREAM(main.cdc.customers_feed)
KEYS (customer_id)
APPLY AS DELETE WHEN operation = 'DELETE'
SEQUENCE BY sequence_num
COLUMNS * EXCEPT (operation, sequence_num)
STORED AS SCD TYPE 2
TRACK HISTORY ON * EXCEPT (last_login_ts);
```

`TRACK HISTORY ON` applies only to SCD Type 2. Without it, a change to **any** output column creates a new version (equivalent to `TRACK HISTORY ON *`). Columns excluded from tracking are updated **in place** on the current version. `APPLY AS TRUNCATE WHEN` is not allowed with SCD Type 2.

Column selection and partial updates

- `COLUMNS (customer_id, name, city)` keeps only the listed columns; `COLUMNS * EXCEPT (operation, sequence_num)` drops metadata columns. The default includes every source column.
- `COLUMNS` takes column names only. Do casts and renames in the source view, not in `COLUMNS`.

- `IGNORE NULL UPDATES`: a null in an incoming update keeps the existing target value (also for nested columns). Use it when the source sends only changed columns. By default, nulls overwrite.
- `IGNORE NULL UPDATES ON columnList` or `ON * EXCEPT (...)` limits that behavior to some columns, so the others can still apply an explicit null.
- `COLUMNS TO UPDATE colName`: a source column of type `array<string>` lists which columns each record updates, including explicit nulls. It cannot be combined with `IGNORE NULL UPDATES` and is not supported for bitemporal tables.

Note

`CREATE STREAMING TABLE ... FLOW AUTO CDC ...` (an inline flow on the table definition) is **Beta** (Databricks Runtime 17.3+ and the PREVIEW channel). `CREATE FLOW ... AS AUTO CDC INTO` is the standard form. Bitemporal storage (`STORED AS BITEMPORAL` with `SYSTEM SEQUENCE BY`) is also **Beta**.

Python Syntax

```
from pyspark import pipelines as dp

dp.create_auto_cdc_flow(
    target="<target-table>",
    source="<data-source>",
    keys=["key1", "key2"],
    sequence_by="<sequence-column>",
    system_sequence_by=None,                          # bitemporal only (Beta)
    ignore_null_updates=False,
    ignore_null_updates_column_list=None,
    ignore_null_updates_except_column_list=None,
    columns_to_update=None,
    apply_as_deletes=None,
    apply_as_truncates=None,
    column_list=None,
    except_column_list=None,
    stored_as_scd_type="1",                          # 1, 2, or "bitemporal"; default 1
    track_history_column_list=None,
    track_history_except_column_list=None,
    name=None,                                       # flow name; defaults to target
    once=False,                                      # one-time flow (backfill) on a
 ↪  batch source
)
```

Argument	Type	Notes
`target`	str	Required. C
`source`	str	Required. D
`keys`	list	Required. S
`sequence_by`	str, `col()`, or `struct()`	Required. S
`apply_as_deletes`	str or `expr()`	Condition th
`apply_as_truncates`	str or `expr()`	Condition th
`column_list` / `except_column_list`	list	Include or e
`stored_as_scd_type`	str or int	1 (default),
`track_history_column_list` / `track_history_except_column_list`	list	SCD Type 2
`ignore_null_updates`	bool	Keep existin

SCD Type 1 example

```python
from pyspark import pipelines as dp
from pyspark.sql.functions import col, expr

@dp.temporary_view
def customers_feed():
    return spark.readStream.table("main.cdc.customers_feed")

dp.create_streaming_table("customers_current")

dp.create_auto_cdc_flow(
    target="customers_current",
    source="customers_feed",
    keys=["customer_id"],
    sequence_by=col("sequence_num"),
    apply_as_deletes=expr("operation = 'DELETE'"),
    apply_as_truncates=expr("operation = 'TRUNCATE'"),
    except_column_list=["operation", "sequence_num"],
    stored_as_scd_type=1,
)
```

SCD Type 2 example with selective history

```python
dp.create_streaming_table(
    name="employees_history",
    comment="Employee history (SCD Type 2)",
)

dp.create_auto_cdc_flow(
    target="employees_history",
    source="employees_feed",
    keys=["employee_id"],
    sequence_by=col("event_ts"),
    apply_as_deletes=expr("operation_type = 'D'"),
    except_column_list=["operation_type", "event_ts"],
    stored_as_scd_type=2,
    track_history_column_list=["department_id", "salary", "title"],
)
```

Changes to `department_id`, `salary`, or `title` create a new version; changes to any other column update the current version in place.

SCD Type 2 Output

`__START_AT` and `__END_AT` hold the **sequence value** of the change that opened or closed each version, and they have the same data type as `sequence_by`. They are timestamps only when you sequence by a timestamp. If you declare the target schema yourself, you must include both columns with that data type.

Docs example: input sequences 1, 2, 5, 6 with a delete of user 123 at sequence 6.

```
| userId | name     | city        | __START_AT | __END_AT |
|--------|----------|-------------|------------|----------|
| 123    | Isabel   | Monterrey   | 1          | 5        |
| 123    | Isabel   | Chihuahua   | 5          | 6        |  <- closed by the
 ↪  DELETE at 6
```

```
| 124    | Raul     | Oaxaca      | 1          | null     |  <- current
| 125    | Mercedes | Tijuana     | 2          | 5        |
| 125    | Mercedes | Mexicali    | 5          | 6        |
| 125    | Mercedes | Guadalajara | 6          | null     |  <- current
| 126    | Lily     | Cancun      | 2          | null     |  <- current
```

With SCD Type 1 on the same feed, the target holds only 124, 125 (Guadalajara), and 126. The update at sequence 5 arrived **after** sequence 6 and is discarded because it is older.

```sql
-- Current rows
SELECT * FROM customers_history WHERE __END_AT IS NULL;

-- Version valid at sequence/time X
SELECT * FROM customers_history
WHERE __START_AT <= :x AND (__END_AT IS NULL OR __END_AT > :x);

-- Friendlier names for downstream users
CREATE VIEW customers_history_v AS
SELECT *, __START_AT AS valid_from, __END_AT AS valid_to FROM customers_history;
```

Sequencing and Out-of-Order Events

- The sequence column must be sortable and represent the true event order, with **one distinct update per key at each sequence value**. NULL sequence values are not supported.
- Break ties with a struct: `SEQUENCE BY STRUCT(event_ts, event_id)` in SQL, `sequence_by=struct("event_ts", "event_id")` in Python. Ordering uses the first field, then the next on ties.
- Late events with an older sequence never overwrite newer state (SCD Type 1). In SCD Type 2 they are placed in the correct position in history.
- **Deletes and late data**: a deleted row is kept temporarily as a **tombstone** in the underlying Delta table, and a view filters tombstones out. The retention defaults to two days and is set with the table property `pipelines.cdc.tombstoneGCThresholdInSeconds`. Auto Loader does not guarantee file order, so set this above the maximum expected delay of out-of-order deletes.

```python
from pyspark.sql.functions import struct

dp.create_auto_cdc_flow(
    target="orders_current",
    source="orders_feed",
    keys=["order_id"],
    sequence_by=struct("event_ts", "event_id"),
    stored_as_scd_type=1,
)
```

AUTO CDC FROM SNAPSHOT

Use this when the source has **no change feed**, only periodic full snapshots. The API compares consecutive snapshots: new keys become inserts, changed rows become updates, and keys missing from the new snapshot become **deletes**. It only sees state at snapshot times, so intermediate changes (A → B → C within one day) can appear as A → C.

```python
# Snapshot of a table or view, read on each pipeline update
dp.create_streaming_table("accounts_scd2")

dp.create_auto_cdc_from_snapshot_flow(
    target="accounts_scd2",
```

```
    source="accounts_snapshot_view",   # or a function returning (DataFrame,
 ↪  version) or None
    keys=["account_id"],
    stored_as_scd_type=2,
)
```

```
CREATE OR REFRESH STREAMING TABLE accounts_scd2;

CREATE FLOW accounts_snapshot_flow AS
AUTO CDC INTO accounts_scd2
FROM SNAPSHOT (SELECT account_id, status, tier FROM main.ops.accounts_snapshot)
WITH VERSION (
  SELECT version FROM main.ops.snapshot_version
  WHERE NOT EXISTS (SELECT 1 FROM last_snapshot_version())
     OR version > (SELECT version FROM last_snapshot_version())
)
KEYS (account_id)
STORED AS SCD TYPE 2;
```

Rules: snapshot versions must increase (a non-increasing version fails the update); there is no `SEQUENCE BY` (ordering comes from `WITH VERSION`); expectations are not supported on a snapshot flow. For SCD Type 1 targets in triggered pipelines, one snapshot flow can share a target with one or more AUTO CDC flows, for example a one-time snapshot backfill (`ONCE`) plus an ongoing CDC feed.

Source provides	Use
A change feed with operation and sequence (Debezium, GoldenGate, DMS, Delta CDF)	`AUTO CDC` / `create_a`
Only full table snapshots	`AUTO CDC FROM SNAP`
Initial history then a feed	`AUTO CDC` with a `once`

CDC Source Patterns

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, coalesce, expr, from_json

# Debezium envelope from Kafka. For deletes "after" is null, so take the key from
 ↪  "before".
@dp.temporary_view
def customers_debezium():
    return (
        spark.readStream.format("kafka")
        .option("kafka.bootstrap.servers", "broker:9092")
        .option("subscribe", "dbserver1.inventory.customers")
        .load()
        .select(from_json(col("value").cast("string"),
 ↪  debezium_schema).alias("d"))
        .select(
            coalesce(col("d.after.id"), col("d.before.id")).alias("customer_id"),
            col("d.after.name").alias("name"),
            col("d.after.email").alias("email"),
            col("d.op").alias("op"),                 # c / u / d / r
            col("d.ts_ms").alias("event_ts_ms"),
        )
    )

dp.create_streaming_table("customers")
```

```
dp.create_auto_cdc_flow(
    target="customers",
    source="customers_debezium",
    keys=["customer_id"],
    sequence_by=col("event_ts_ms"),
    apply_as_deletes=expr("op = 'd'"),
    except_column_list=["op", "event_ts_ms"],
    stored_as_scd_type=1,
)
```

For many tables, generate flows with a **factory function** so each flow captures its own table name (a plain `for` loop that reads the loop variable inside a decorated function binds the last value):

```
def cdc_table(name, keys):
    @dp.temporary_view(name=f"{name}_feed")
    def feed():
        return spark.readStream.table(f"main.cdc.{name}_feed")

    dp.create_streaming_table(name)
    dp.create_auto_cdc_flow(
        target=name,
        source=f"{name}_feed",
        keys=keys,
        sequence_by="event_ts",
        apply_as_deletes="operation = 'DELETE'",
        except_column_list=["operation", "event_ts"],
        stored_as_scd_type=1,
    )

for table_name, table_keys in [("customers", ["customer_id"]), ("orders",
 ↪  ["order_id"])]:
    cdc_table(table_name, table_keys)
```

Downstream of an AUTO CDC Target

- **Read the change data feed** of an AUTO CDC target streaming table like any Delta table (`readChangeFeed`, `table_changes`). The table must be published to Unity Catalog and the reader must use Databricks Runtime 15.2+. If a primary key changes (or `__START_AT` changes in SCD Type 2), the feed shows `delete` + `insert` instead of `update_preimage` / `update_postimage`.
- **Do not plain-stream the target**: it receives updates and deletes, so a streaming read of it fails unless you use the change feed (or `skipChangeCommits`, which ignores changes). A materialized view over the target is the simplest option for aggregates.
- **DML on targets**: you can run `INSERT`/`UPDATE`/`DELETE`/`MERGE` against a target published to Unity Catalog, but keep `__START_AT` / `__END_AT` valid and do not evolve the schema with DML.
- **Metrics**: AUTO CDC flows report `num_upserted_rows` and `num_deleted_rows` in the event log (not `num_output_rows`); snapshot flows do not report these.

Use Cases

- **OLTP mirror for reporting**: SCD Type 1 from a Debezium feed keeps a silver `customers` table equal to the source, with deletes applied, and late events ignored by sequence.
- **Audit history of compensation**: SCD Type 2 with `track_history_column_list=["salary", "department_id"]` keeps a version per change of those columns. Updates to `last_login_ts`

overwrite the current row instead of creating noise versions.
- **Legacy system without CDC**: nightly full extracts land as files; `AUTO CDC FROM SNAPSHOT` turns them into an SCD Type 2 dimension, including deletes for keys that disappear.

Common Issues & Errors

Symptom	Cause
Duplicate or ambiguous results for a key	Two events for one key with the same sequence valu
Errors or unreliable ordering for some keys	NULL values in the sequence column (not supported)
Deletes not applied	Delete condition does not match the actual operation
A deleted key reappears after late, out-of-order events	The delete tombstone was already garbage-collected
Error targeting a managed table with AUTO CDC	`CREATE TABLE ... FLOW` managed tables do not sup
Schema error after declaring the SCD Type 2 schema	`__START_AT` / `__END_AT` missing or wrong type
`apply_as_truncates` rejected	Used with SCD Type 2
SCD Type 2 table grows quickly	High-churn columns tracked
Cast inside `COLUMNS (...)` fails	`COLUMNS` accepts names only

Exam Tips

Tip

- **Required**: `KEYS` + `SEQUENCE BY`. **Optional**: `STORED AS` (default **SCD Type 1**), `APPLY AS DELETE WHEN`, `COLUMNS`, `TRACK HISTORY ON`.
- **`stored_as_scd_type=2`** → history with `__START_AT` / `__END_AT`; **current row = `__END_AT IS NULL`**.
- **Target must be a streaming table** created before the flow (`dp.create_streaming_table`).
- **Out-of-order events** are handled by the sequence column, not by watermarks you write.
- **Truncate is SCD Type 1 only.** Snapshot flows: no `SEQUENCE BY`, no expectations.
- **Formerly APPLY CHANGES**: same clauses; `dlt.apply_changes` → `dp.create_auto_cdc_flow`.
- AUTO CDC is a **Lakeflow** feature, not available in open-source Spark Declarative Pipelines.

Key Takeaways

- **AUTO CDC = APPLY CHANGES renamed**, same syntax; SQL uses `CREATE FLOW ... AS AUTO CDC INTO <streaming table> FROM STREAM(<source>)`, Python uses `dp.create_auto_cdc_flow()`.
- **Keys and sequence** are mandatory; the sequence must be sortable, non-null, and unique per key; use a `STRUCT` to break ties.
- **SCD Type 1** upserts and keeps only the latest row (default). **SCD Type 2** keeps versions with `__START_AT` / `__END_AT` set to sequence values.
- **Deletes** need `APPLY AS DELETE WHEN` / `apply_as_deletes`. They are kept as tombstones (default two days, `pipelines.cdc.tombstoneGCThresholdInSeconds`) to handle late arrivals.
- **Selective history**: `TRACK HISTORY ON` / `track_history_column_list` choose which column changes create versions; other columns update in place.
- **Snapshots only?** Use `AUTO CDC FROM SNAPSHOT` / `create_auto_cdc_from_snapshot_flow`, which infers inserts, updates, and deletes by comparing snapshots.
- **Downstream**: read the target's change data feed (DBR 15.2+, Unity Catalog), or build materialized views on it; do not plain-stream it.

Related Topics

- Declarative Pipelines — streaming tables, materialized views, run modes
- Expectations and Data Quality — validating CDC sources
- Change Data Capture (CDC) — Part 1 — Change Data Feed and manual MERGE patterns
- Change Data Capture (CDC) — Part 2 — Delta or Iceberg targets, multi-hop CDC
- SCD Patterns — SCD types in dimensional modeling
- Lakeflow Event Logs — CDC flow metrics

Official Documentation

- The AUTO CDC APIs: Simplify change data capture with pipelines
- AUTO CDC INTO (pipelines)
- CREATE FLOW (pipelines)
- create_auto_cdc_flow
- create_auto_cdc_from_snapshot_flow
- create_streaming_table
- Advanced AUTO CDC topics
- Change data capture and snapshots
- CREATE TABLE ... FLOW (pipelines)

Lakeflow Jobs — Part 1

Lakeflow Jobs (Lakeflow Jobs) provide orchestration capabilities for running notebooks, Lakeflow pipelines, and other tasks as coordinated workflows. This part covers job components, configuration, task dependencies, task values, conditional execution, and for-each loops.

Overview

```
flowchart TB
    subgraph Trigger["Trigger Types"]
        Schedule[Scheduled]
        Manual[Manual]
        FileArrival[File Arrival]
        API[API/External]
    end

    subgraph Job["Job Definition"]
        Tasks[Task Graph]
        Dependencies[Dependencies]
        Compute[Compute Config]
    end

    subgraph Execution["Execution"]
        Run[Job Run]
        Monitoring[Monitoring]
        Alerts[Notifications]
    end

    Trigger --> Job
    Job --> Execution
```

Overview

- **Jobs are task graphs**: notebook, pipeline, and other task types, defined in `databricks.yml` or with the Python SDK
- **depends_on** orders tasks; tasks with no dependencies run in parallel
- **Task values** pass small results between tasks with `dbutils.jobs.taskValues.set` / `get`
- **Control flow**: If/else condition tasks and Run if dependencies branch a run; For each tasks loop a nested task over a list of inputs

What the Exam Tests

- Create Lakeflow Jobs that use control-flow operators such as If/Else conditions and For Each loops.
- Create and automate ETL workloads using Lakeflow Jobs via the UI, API, and CLI.
- Read a task graph and predict which tasks run, and in what order, for a given outcome

Job Components

Job Structure

```
flowchart LR
    subgraph Job["Job Configuration"]
        subgraph Tasks["Task Graph"]
            T1[Task 1: Ingest]
            T2[Task 2: Transform]
            T3[Task 3: Validate]
            T4[Task 4: Publish]
        end

        T1 --> T2
        T2 --> T3
        T3 --> T4
    end

    subgraph Config["Configuration"]
        Schedule[Schedule/Trigger]
        Compute[Compute]
        Notify[Notifications]
        Params[Parameters]
    end
```



Task Types

Task type selector in the Lakeflow Jobs UI showing available task types (Notebook, Python, dbt, SQL, Lakeflow Pipelines).

Task Type	Use Case	Configuration
Notebook	Run notebook code	Notebook path, parameters
Lakeflow Pipelines Pipeline	Run Lakeflow pipeline	Pipeline ID

Task Type	Use Case	Configuration
Python Script	Run Python file	Script path, arguments
SQL	Run SQL statements	SQL warehouse, queries
JAR	Run Java/Scala JAR	JAR path, main class
Spark Submit	Submit Spark job	Application arguments
dbt	Run dbt models	dbt project, profiles
If/Else	Conditional logic	Condition expression
For Each	Loop over items	Input array, task

Job Configuration

databricks.yml Job Definition

```yaml
# resources/jobs.yml

resources:
  jobs:
    daily_etl_pipeline:
      name: "Daily ETL Pipeline - ${var.environment}"

      # Job-level settings
      tags:
        team: data-engineering
        environment: ${var.environment}

      # Schedule configuration
      schedule:
        quartz_cron_expression: "0 0 6 * * ?"
        timezone_id: "America/New_York"
        pause_status: UNPAUSED

      # Email notifications
      email_notifications:
        on_start:
          - team@company.com
        on_success:
          - team@company.com
        on_failure:
          - team@company.com
          - oncall@company.com

      # Webhook notifications
      webhook_notifications:
        on_failure:
          - id: slack_webhook_id

      # Task definitions
      tasks:
        - task_key: ingest_data
          notebook_task:
            notebook_path: ../src/notebooks/bronze/ingest.py
            base_parameters:
              source: ${var.source_path}
              catalog: ${var.catalog}
          job_cluster_key: etl_cluster
```

```yaml
        - task_key: transform_data
          depends_on:
            - task_key: ingest_data
          notebook_task:
            notebook_path: ../src/notebooks/silver/transform.py
            base_parameters:
              catalog: ${var.catalog}
          job_cluster_key: etl_cluster

        - task_key: run_dlt_pipeline
          depends_on:
            - task_key: transform_data
          pipeline_task:
            pipeline_id: ${resources.pipelines.my_dlt_pipeline.id}

        - task_key: validate_data
          depends_on:
            - task_key: run_dlt_pipeline
          notebook_task:
            notebook_path: ../src/notebooks/validation/validate.py
          job_cluster_key: etl_cluster

        - task_key: publish_metrics
          depends_on:
            - task_key: validate_data
          sql_task:
            warehouse_id: ${var.warehouse_id}
            file:
              path: ../src/sql/publish_metrics.sql

      # Job clusters
      job_clusters:
        - job_cluster_key: etl_cluster
          new_cluster:
            spark_version: "14.3.x-scala2.12"
            node_type_id: "Standard_DS3_v2"
            num_workers: 2
            spark_conf:
              spark.databricks.delta.preview.enabled: "true"

      # Retry and timeout
      max_concurrent_runs: 1
      timeout_seconds: 3600
```

Python Job Definition (SDK)

```python
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.jobs import (
    Task, NotebookTask, PipelineTask, CronSchedule,
    JobCluster, ClusterSpec, EmailNotifications
)

w = WorkspaceClient()

# Create job
```

```python
job = w.jobs.create(
    name="ETL Pipeline",
    tasks=[
        Task(
            task_key="ingest",
            notebook_task=NotebookTask(
                notebook_path="/Workspace/notebooks/ingest",
                base_parameters={"source": "/mnt/raw"}
            ),
            job_cluster_key="etl_cluster"
        ),
        Task(
            task_key="transform",
            depends_on=[{"task_key": "ingest"}],
            notebook_task=NotebookTask(
                notebook_path="/Workspace/notebooks/transform"
            ),
            job_cluster_key="etl_cluster"
        ),
        Task(
            task_key="run_pipeline",
            depends_on=[{"task_key": "transform"}],
            pipeline_task=PipelineTask(pipeline_id="abc-123")
        )
    ],
    job_clusters=[
        JobCluster(
            job_cluster_key="etl_cluster",
            new_cluster=ClusterSpec(
                spark_version="14.3.x-scala2.12",
                node_type_id="Standard_DS3_v2",
                num_workers=2
            )
        )
    ],
    schedule=CronSchedule(
        quartz_cron_expression="0 0 6 * * ?",
        timezone_id="UTC"
    ),
    email_notifications=EmailNotifications(
        on_failure=["team@company.com"]
    )
)

print(f"Created job: {job.job_id}")
```

Task Dependencies

Linear Dependencies

```yaml
tasks:
  - task_key: step_1
    # No dependencies - runs first

  - task_key: step_2
```

```yaml
    depends_on:
      - task_key: step_1
    # Runs after step_1

  - task_key: step_3
    depends_on:
      - task_key: step_2
    # Runs after step_2
```

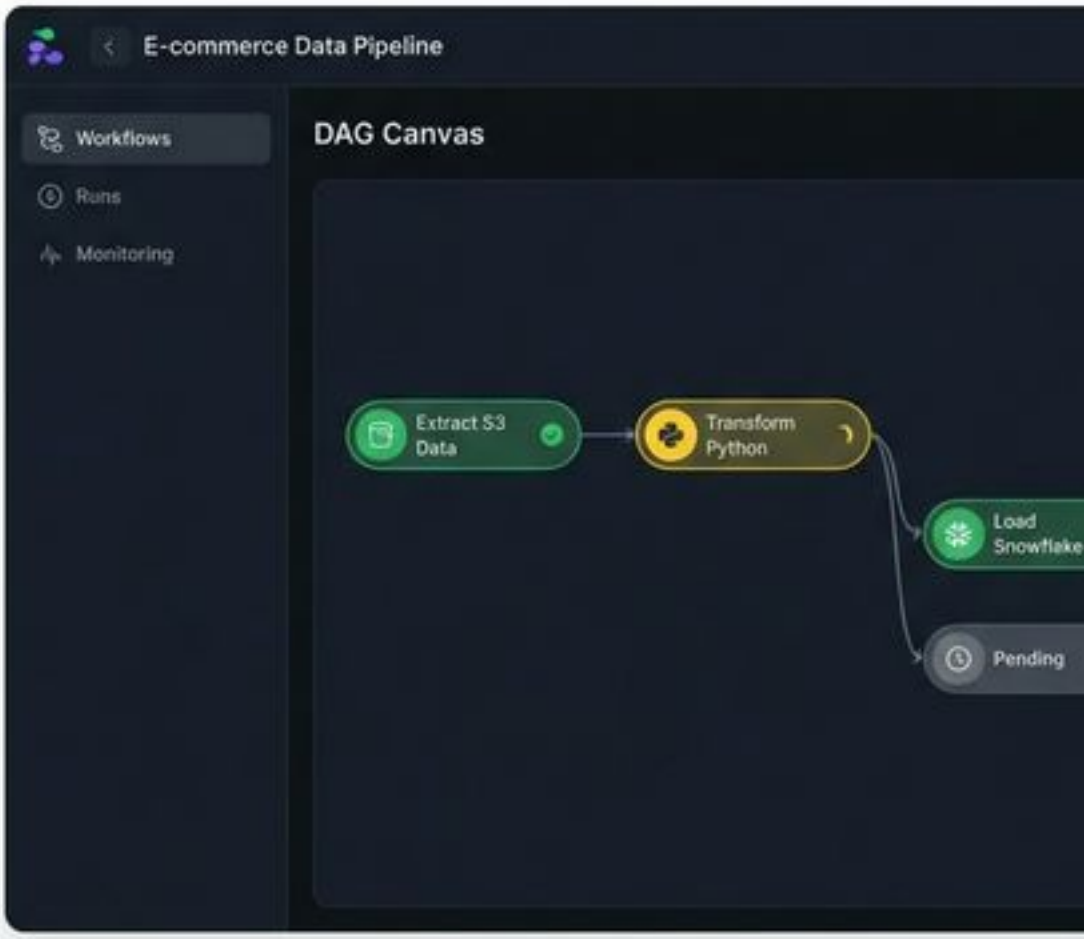
Parallel Execution

```yaml
tasks:
  - task_key: ingest_orders
    # No dependencies

  - task_key: ingest_customers
    # No dependencies - runs parallel with ingest_orders

  - task_key: ingest_products
    # No dependencies - runs parallel with others

  - task_key: join_data
    depends_on:
      - task_key: ingest_orders
      - task_key: ingest_customers
      - task_key: ingest_products
    # Waits for all three to complete
```



DAG Visualization

Workflow DAG visualizer showing task dependencies and execution ordering in a multi-task job.

```
flowchart TD
    A[Ingest Orders] --> D[Join Data]
    B[Ingest Customers] --> D
    C[Ingest Products] --> D
    D --> E[Transform]
    E --> F[Validate]
    E --> G[Archive]
    F --> H[Publish]
    G --> H
```

Task Values (Inter-Task Communication)

Setting Task Values

```python
# In a notebook task
# Set a value to be used by downstream tasks

# Using dbutils

dbutils.jobs.taskValues.set(key="record_count", value=10000)
dbutils.jobs.taskValues.set(key="output_path", value="/mnt/output/2024-01-15/")
dbutils.jobs.taskValues.set(key="status", value="success")

# Set complex values (JSON serializable)

dbutils.jobs.taskValues.set(
    key="metrics",
    value={"rows": 10000, "errors": 5, "duration": 120}
)
```

Getting Task Values

```python
# In a downstream notebook task
# Get value from upstream task

# Specify the task key that set the value

record_count = dbutils.jobs.taskValues.get(
    taskKey="ingest_data",
    key="record_count",
    default=0
)

output_path = dbutils.jobs.taskValues.get(
    taskKey="ingest_data",
    key="output_path",
    default="/mnt/output/default/"
)

# Get complex value

metrics = dbutils.jobs.taskValues.get(
    taskKey="ingest_data",
    key="metrics",
```

```
    default={}
)
print(f"Processed {metrics.get('rows', 0)} rows")
```

Task Values in SQL Tasks

```
-- Reference task values in SQL using parameters
-- Set in task configuration: ${tasks.upstream_task.values.output_table}

SELECT COUNT(*) FROM ${output_table}
WHERE processed_date = '${date}'
```

Conditional Execution

If/Else Task

```
tasks:
  - task_key: check_data_quality
    notebook_task:
      notebook_path: ../notebooks/quality_check.py
    # Sets task value: quality_passed = true/false

  - task_key: condition_check
    depends_on:
      - task_key: check_data_quality
    condition_task:
      op: EQUAL_TO
      left: "{{tasks.check_data_quality.values.quality_passed}}"
      right: "true"

  - task_key: process_good_data
    depends_on:
      - task_key: condition_check
        outcome: "true"
    notebook_task:
      notebook_path: ../notebooks/process.py

  - task_key: handle_bad_data
    depends_on:
      - task_key: condition_check
        outcome: "false"
    notebook_task:
      notebook_path: ../notebooks/quarantine.py
```

Run If Dependencies

```
tasks:
  - task_key: main_task
    notebook_task:
      notebook_path: ../notebooks/main.py

  - task_key: on_success
    depends_on:
      - task_key: main_task
    run_if: ALL_SUCCESS
    notebook_task:
```

```yaml
      notebook_path: ../notebooks/success_handler.py

  - task_key: on_failure
    depends_on:
      - task_key: main_task
    run_if: AT_LEAST_ONE_FAILED
    notebook_task:
      notebook_path: ../notebooks/failure_handler.py

  - task_key: always_run
    depends_on:
      - task_key: main_task
    run_if: ALL_DONE
    notebook_task:
      notebook_path: ../notebooks/cleanup.py
```

For Each Task (Loops)

Basic For Each

```yaml
tasks:
  - task_key: get_tables
    notebook_task:
      notebook_path: ../notebooks/get_table_list.py
    # Returns: ["table1", "table2", "table3"]

  - task_key: process_tables
    depends_on:
      - task_key: get_tables
    for_each_task:
      inputs: "{{tasks.get_tables.values.table_list}}"
      task:
        task_key: process_single_table
        notebook_task:
          notebook_path: ../notebooks/process_table.py
          base_parameters:
            table_name: "{{input}}"
```

Notebook Returning Loop Input

```python
# get_table_list.py

tables = ["orders", "customers", "products", "inventory"]

# Set as task value for for_each

dbutils.jobs.taskValues.set(
    key="table_list",
    value=tables
)
```

Processing Each Item

```python

# process_table.py
# Get the current iteration value
```

```python
table_name = dbutils.widgets.get("table_name")

# Process this table

df = spark.table(f"bronze.{table_name}")
df.write.format("delta").mode("overwrite").saveAsTable(f"silver.{table_name}")

print(f"Processed table: {table_name}")
```

> **Continue reading:** Part 2 — Triggers, Compute, Notifications, Parameters, Error Handling & Exam Tips

---

Lakeflow Jobs — Part 2

Overview

This part covers triggers and scheduling, compute configuration, notifications, job parameters, error handling, monitoring, common issues, and exam tips for Lakeflow Jobs (Lakeflow Jobs).

> For job components, configuration, task dependencies, task values, and for-each loops, see Part 1.

Overview

- **Triggers**: schedules use 7-field Quartz cron; the `file_arrival` trigger starts a run when new files land
- **Job clusters** are created per run and terminated afterwards, so they cost less than all-purpose clusters for scheduled work
- **Failure handling**: per-task retries (`max_retries`, `min_retry_interval_millis`, `retry_on_timeout`) and run conditions (`ALL_SUCCESS`, `AT_LEAST_ONE_FAILED`, `ALL_DONE`)
- **Monitoring**: `system.lakeflow.job_run_timeline` and `job_task_run_timeline` hold run history and durations

What the Exam Tests

- Create and automate ETL workloads using Lakeflow Jobs via the UI, API, and CLI.
- Use Lakeflow Jobs UI and Jobs API to monitor job status and performance metrics.
- Choose trigger, compute, retry, and notification settings for a production job

Triggers and Scheduling



Databricks Job trigger schedule configuration showing cron, continuous, and file-arrival options.

Cron Schedule

```yaml
schedule:
  quartz_cron_expression: "0 0 6 * * ?"  # Daily at 6 AM
  timezone_id: "America/New_York"
  pause_status: UNPAUSED
```

Common Cron Expressions

Expression	Description
`0 0 6 * * ?`	Daily at 6:00 AM
`0 0 * * * ?`	Every hour
`0 */15 * * * ?`	Every 15 minutes
`0 0 6 ? * MON-FRI`	Weekdays at 6:00 AM
`0 0 6 1 * ?`	First of month at 6:00 AM
`0 0 6 ? * SUN`	Every Sunday at 6:00 AM

File Arrival Trigger

```yaml
trigger:
  file_arrival:
    url: "s3://bucket/landing/data/"
    min_time_between_triggers_seconds: 60
    wait_after_last_change_seconds: 30
```

Continuous Trigger

```yaml
trigger:
  periodic:
    interval: 1
    unit: HOURS
```

Compute Configuration

Job Clusters vs All-Purpose

Aspect	Job Cluster	All-Purpose Cluster
Lifecycle	Created/destroyed per job	Always running
Cost	Pay per job	Pay for uptime
Startup	Cold start delay	Immediate
Best for	Production jobs	Development

Job Cluster Configuration

```yaml
job_clusters:
  - job_cluster_key: standard_etl
    new_cluster:
      spark_version: "14.3.x-scala2.12"
      node_type_id: "Standard_DS3_v2"
      num_workers: 2
      spark_conf:
        spark.databricks.delta.preview.enabled: "true"
        spark.sql.shuffle.partitions: "200"
      custom_tags:
        purpose: etl
        environment: ${var.environment}

  - job_cluster_key: large_processing
    new_cluster:
      spark_version: "14.3.x-scala2.12"
```

```
      node_type_id: "Standard_DS4_v2"
      autoscale:
        min_workers: 2
        max_workers: 10
      spark_conf:
        spark.databricks.adaptive.autoOptimizeShuffle.enabled: "true"
```

Serverless Compute

```
tasks:
  - task_key: serverless_task
    notebook_task:
      notebook_path: ../notebooks/process.py
    # Uses serverless compute (no cluster config)
    environment_key: default  # Or custom environment
```

Notifications

Email Notifications

```
email_notifications:
  on_start:
    - team@company.com
  on_success:
    - team@company.com
  on_failure:
    - team@company.com
    - oncall@company.com
  on_duration_warning_threshold_exceeded:
    - team@company.com
  no_alert_for_skipped_runs: true
```

Webhook Notifications

```
webhook_notifications:
  on_start:
    - id: ${var.slack_webhook_id}
  on_success:
    - id: ${var.slack_webhook_id}
  on_failure:
    - id: ${var.pagerduty_webhook_id}
    - id: ${var.slack_webhook_id}
```

Duration Warning

```
# Warn if job exceeds expected duration

health:
  rules:
    - metric: RUN_DURATION_SECONDS
      op: GREATER_THAN
      value: 1800  # 30 minutes
```

Job Parameters

Widget Parameters

```python
# In notebook

dbutils.widgets.text("date", "2024-01-15")
dbutils.widgets.dropdown("environment", "dev", ["dev", "staging", "prod"])
dbutils.widgets.text("source_path", "/mnt/landing/")

# Get values

date = dbutils.widgets.get("date")
env = dbutils.widgets.get("environment")
source = dbutils.widgets.get("source_path")
```

Job-Level Parameters

```yaml
parameters:
  - name: date
    default: "2024-01-15"
  - name: environment
    default: "dev"

tasks:
  - task_key: process
    notebook_task:
      notebook_path: ../notebooks/process.py
      base_parameters:
        date: "{{job.parameters.date}}"
        environment: "{{job.parameters.environment}}"
```

Dynamic Parameters

```bash
# Run job with parameters via CLI

databricks jobs run-now --job-id 12345 \
    --notebook-params '{"date": "2024-01-20", "mode": "full"}'

# Via API

curl -X POST "$WORKSPACE_URL/api/2.1/jobs/run-now" \
    -H "Authorization: Bearer $TOKEN" \
    -d '{"job_id": 12345, "notebook_params": {"date": "2024-01-20"}}'
```

Error Handling

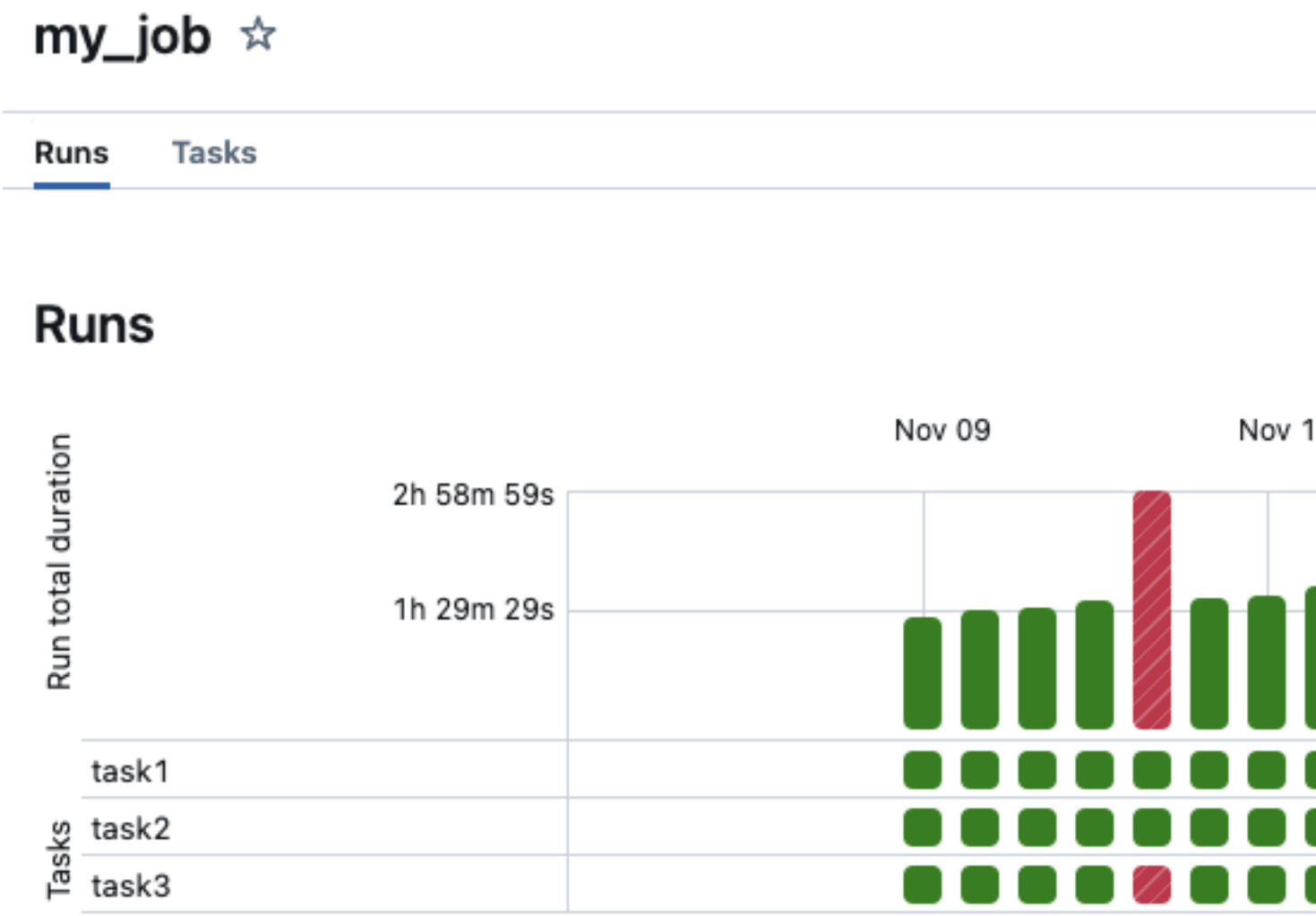
Retry Configuration

```yaml
tasks:
  - task_key: potentially_flaky_task
    notebook_task:
      notebook_path: ../notebooks/external_api.py
    # Retry on failure
    max_retries: 3
    min_retry_interval_millis: 60000  # 1 minute
    retry_on_timeout: true
    timeout_seconds: 600
```

Failure Handling Task

```yaml
tasks:
  - task_key: main_processing
    notebook_task:
      notebook_path: ../notebooks/main.py

  - task_key: handle_failure
    depends_on:
      - task_key: main_processing
    run_if: AT_LEAST_ONE_FAILED
    notebook_task:
      notebook_path: ../notebooks/failure_handler.py
```



Job run matrix showing historical success and failure status across multiple executions.

Jobs UI views

View	Where	What it shows
Jobs & pipelines list	Sidebar **Jobs & Pipelines**	All jobs and pipelines you ca
Runs tab (all jobs)	**Jobs & Pipelines** → **Runs**	Finished runs count graph (l
Matrix view (one job)	Job → **Runs**	One column per run. The **Ru**
Job run details	Click a run's **Start time**	**Graph**, **Timeline** (Gantt-sty
Run breakdown by phase	Hover the run's **Duration**	Queued → Waiting for resou
Streaming metrics (Public Preview)	Task run → **Metrics**	Backlog bytes, records, sec
Query metrics (serverless jobs)	Job and task run sidebars, Timeline view	Rows read and written, que

Job run outcome comes from the **leaf tasks**. **Succeeded with failures** means some tasks failed but every leaf task succeeded. Run history is kept for **60 days**. For longer, account-wide history, query `system.lakeflow.*`.

Health rules and notifications Set **Duration and streaming backlog thresholds** on the job (or **Metric thresholds** on a task). The **Warning** value under Run duration is the expected completion time, and exceeding it triggers a **Duration warning** event (the matrix and runs list also show a warning). A task's **Timeout** is the hard maximum. Streaming backlog notifications fire when the 10-minute average exceeds the threshold, repeat at most every 30 minutes, and aren't supported for pipeline tasks.

```yaml
health:
  rules:
    - metric: RUN_DURATION_SECONDS        # or STREAMING_BACKLOG_BYTES / _RECORDS
 ↪  / _SECONDS / _FILES
      op: GREATER_THAN
      value: 1800
email_notifications:
  on_failure: [oncall@company.com]
  on_duration_warning_threshold_exceeded: [team@company.com]
  on_streaming_backlog_exceeded: [team@company.com]
notification_settings:
  no_alert_for_skipped_runs: true
  no_alert_for_canceled_runs: true
```

Job-level notifications aren't sent when a failed task is retried. Use task notifications, with **Mute notifications until the last retry** (`alert_on_last_attempt`), when you need them. Each event type allows at most three system destinations.

Jobs API run fields

```python
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()

for run in w.jobs.list_runs(job_id=12345, limit=10):      # GET
 ↪  /api/2.2/jobs/runs/list
    print(run.run_id, run.status.state, run.status.termination_details)

run = w.jobs.get_run(run_id=67890)                        # GET
 ↪  /api/2.2/jobs/runs/get
print(f"Total: {run.run_duration / 1000} s, queued: {run.queue_duration / 1000}
 ↪  s")
```

```python
for task in run.tasks:
    print(task.task_key, task.status.state, task.execution_duration)
```

Field	Meaning
`status.state`	QUEUED, PENDING, BLOCKED, WA
`status.termination_details.code` / `.type`	Why it ended (SUCCESS, RUN_
`state.life_cycle_state` / `state.result_state`	**Deprecated.** Use `status` ins
`queue_duration`, `setup_duration`, `execution_duration`, `cleanup_duration`	Milliseconds. These are `0` at j
`run_duration`	Milliseconds for the run and a
`start_time`, `end_time`	Epoch milliseconds. `end_time`

See Databricks REST API — Part 2 for the CLI and pipelines equivalents.

System tables for job monitoring

```sql
-- Run outcomes and durations (rows are hourly slices; result_state only on the
 ↪  last row)
SELECT
    job_id,
    run_id,
    MIN(period_start_time)                                    AS run_start,
    MAX(period_end_time)                                      AS run_end,
    CAST(SUM(period_end_time - period_start_time) AS LONG)    AS duration_seconds,
    MAX(result_state)                                         AS result_state
FROM system.lakeflow.job_run_timeline
WHERE job_id = '12345'
  AND period_start_time >= current_timestamp() - INTERVAL 7 DAYS
GROUP BY job_id, run_id
ORDER BY run_start DESC;

-- Task-level durations
SELECT
    job_id,
    job_run_id,
    task_key,
    result_state,
    execution_duration_seconds
FROM system.lakeflow.job_task_run_timeline
WHERE job_id = '12345'
  AND result_state IS NOT NULL
ORDER BY period_start_time DESC;
```

Use Cases

- **Cross-Workspace Data Promotion**: Configuring a multi-task job that sequentially triggers an ingestion pipeline, executes a suite of PyTest validation code, and conditionally alerts stakeholders via Slack if any task fails using the `AT_LEAST_ONE_FAILED` condition.
- **Scheduled Feature Engineering**: Running a parameterized machine learning feature extraction notebook on a cron schedule using an isolated, cost-effective job cluster to process recent interactions before model retraining occurs.

Common Issues & Errors

Task Timeout **Scenario:** Task exceeds timeout and is killed.

Fix: Adjust timeout or optimize task:

```yaml
tasks:
  - task_key: long_running
    notebook_task:
      notebook_path: ../notebooks/heavy_processing.py
    timeout_seconds: 7200  # 2 hours
```

Cluster Start Failure **Scenario:** Job cluster fails to start.

Fix: Check cluster configuration and quotas:

```yaml
job_clusters:
  - job_cluster_key: fallback_cluster
    new_cluster:
      spark_version: "14.3.x-scala2.12"
      # Use smaller instance if quota issues
      node_type_id: "Standard_DS2_v2"
      num_workers: 1
```

Task Value Not Found **Scenario:** Downstream task can't read task value.

Fix: Verify task key and key name:

```python
# Upstream task - verify this sets the value

dbutils.jobs.taskValues.set(key="result", value="success")

# Downstream task - use exact task key

value = dbutils.jobs.taskValues.get(
    taskKey="upstream_task_key",  # Must match exactly
    key="result",
    default="unknown"
)
```

Circular Dependencies **Scenario:** Job fails validation due to circular deps.

Fix: Review and fix dependency graph:

```yaml
# Wrong - circular

tasks:
  - task_key: a
    depends_on: [{task_key: b}]
  - task_key: b
    depends_on: [{task_key: a}]

# Correct - acyclic

tasks:
  - task_key: a
  - task_key: b
    depends_on: [{task_key: a}]
```

Exam Tips

1. **Task types** - Notebook, Pipeline, SQL, Python, JAR, If/Else, For Each

2. **Dependencies** - depends_on defines execution order; tasks with no dependencies run in parallel
3. **Task values** - Inter-task communication with `dbutils.jobs.taskValues.set/get`
4. **Run conditions** - `ALL_SUCCESS`, `AT_LEAST_ONE_FAILED`, `ALL_DONE`
5. **Job clusters** - Created and destroyed per run; cost-effective for production
6. **Cron syntax** - Quartz format (7 fields including seconds); ? means "no specific value"
7. **File arrival trigger** - Starts job when new files are detected at a path
8. **Retry config** - `max_retries` and `min_retry_interval_millis` per task
9. **For each** - Parallel processing of array items from upstream task values
10. **Notifications** - Email and webhook notifications on start, success, and failure

Key Takeaways

- **Task execution order**: Tasks with no depends_on entries run in parallel; depends_on enforces sequential execution and the dependency graph must be a DAG (no cycles allowed).
- **Run conditions**: `ALL_SUCCESS` (default) runs only if all upstream tasks succeed; `AT_LEAST_ONE_FAILED` creates a failure-handler task; `ALL_DONE` runs regardless of upstream outcome.
- **Task values for inter-task data**: Use `dbutils.jobs.taskValues.set(key, value)` in upstream tasks and `dbutils.jobs.taskValues.get(taskKey, key)` in downstream tasks to share computed values within a run.
- **Job clusters are cost-effective**: Job clusters are created fresh per run and terminated on completion, costing ~60% less than keeping an All-Purpose cluster running 24/7.
- **Quartz cron format**: Databricks uses 7-field Quartz cron (includes seconds field); ? means "no specific value" and is used in day-of-month or day-of-week when the other is specified.
- **File arrival trigger**: The `file_arrival` trigger starts a job when new files are detected at a cloud storage path, eliminating the need for polling-based scheduling.
- **Retry configuration**: Set `max_retries` and `min_retry_interval_millis` per task to handle transient failures; `retry_on_timeout: true` retries tasks that exceed `timeout_seconds`.
- **system.lakeflow tables**: Query `system.lakeflow.job_run_timeline` and `job_task_run_timeline` for programmatic job run history, duration metrics, and result states.

Related Topics

- Declarative Pipelines - Lakeflow pipeline task type
- Declarative Automation Bundles - Job deployment via DABs
- CI/CD Integration - Automated deployment

Official Documentation

- Lakeflow Jobs
- Jobs API
- Task Dependencies
- Task Values
- Monitor Lakeflow Jobs
- Diagnose Lakeflow Jobs performance
- Add notifications on a job
- Jobs system table reference

Databricks Compute — Part 1

Understanding Databricks compute options is essential for building cost-effective, performant data pipelines. This part covers classic compute: all-purpose and job clusters, instance pools, cluster

policies, autoscaling, spot instances, and cluster permissions. Part 2 covers serverless compute (environments, dependencies, performance mode, high memory, auto-optimization), SQL warehouses, and Photon.

Overview

```
flowchart TB
    Compute[Databricks Compute] --> Interactive[Interactive]
    Compute --> Automated[Automated]
    Compute --> SQL[SQL Analytics]

    Interactive --> AllPurpose[All-Purpose Clusters]
    Interactive --> Serverless[Serverless Compute]

    Automated --> JobCluster[Job Clusters]
    Automated --> Pools[Instance Pools]
    Automated --> ServerlessJobs[Serverless Jobs and Pipelines]

    SQL --> SQLWarehouse[SQL Warehouses]
    SQL --> ServerlessSQL[Serverless SQL]
```

Overview

- Classic compute runs in your cloud account: all-purpose clusters for interactive work, job clusters created per run, and instance pools to cut startup time.
- Cluster policies (`fixed`, `allowlist`, `blocklist`, `range`, `unlimited`) enforce configuration and cost standards.
- Autoscaling, spot instances with `first_on_demand`, and cluster permissions round out classic configuration.

What the Exam Tests

- "Choose appropriate compute and configuration for environments and dependencies" — know when classic compute is still required and how it is configured.
- Picking job clusters vs all-purpose clusters, pools, policies, and spot settings for a given cost and reliability constraint.
- Serverless-specific settings (environments, performance mode, high memory, auto-optimization) are in Part 2.

Compute Types Comparison

Type	Use Case	Cost	Startup Time
All-Purpose Clusters	Interactive development	Higher	5-10 minutes
Job Clusters	Production jobs on classic compute	Lower	5-10 minutes
Serverless Compute	Notebooks, jobs, Lakeflow Pipelines	Pay per use	Seconds (performance-optimized); 4
Instance Pools	Reduce startup time	Moderate	Seconds
SQL Warehouse	SQL analytics	Per-query	Minutes
Serverless SQL	SQL analytics	Per-query	Seconds

All-Purpose Clusters

All-purpose clusters are designed for interactive workloads and collaborative development.

Characteristics

```
flowchart LR
    AP[All-Purpose Cluster] --> Multi["Multi-user"]
    AP --> Persistent["Persistent until terminated"]
    AP --> Languages["Python, SQL, Scala, R"]
    AP --> Notebooks["Notebook execution"]
```

Feature	Behavior
Persistence	Stays running until manually terminated or auto-terminated
Users	Multiple users can attach notebooks
Languages	Supports Python, SQL, Scala, R
Cost model	Pay while running (even if idle)
Auto-termination	Configurable: 10–10,000 minutes of inactivity, or 0 to disable

Creating All-Purpose Clusters

```python
# Via SDK

from databricks.sdk import WorkspaceClient

w = WorkspaceClient()

cluster = w.clusters.create(
    cluster_name="dev-cluster",
    spark_version="14.3.x-scala2.12",
    node_type_id="Standard_DS3_v2",
    num_workers=2,
    autotermination_minutes=60,
    spark_conf={
        "spark.speculation": "true",
        "spark.databricks.delta.preview.enabled": "true"
    },
    custom_tags={
        "team": "data-engineering",
        "environment": "development"
    }
).result()

print(f"Cluster ID: {cluster.cluster_id}")
```

```sql
-- Via SQL (for serverless SQL or existing cluster)
-- Clusters are typically created via UI or API, not SQL
```

Auto-Termination

```python
# Configure auto-termination

w.clusters.edit(
    cluster_id="xxx",
    autotermination_minutes=30  # Terminate after 30 mins idle
)

# Disable auto-termination (not recommended for cost)
```

```
w.clusters.edit(
    cluster_id="xxx",
    autotermination_minutes=0  # Never auto-terminate
)
```

Setting	Behavior
`autotermination_minutes=0`	Never auto-terminate
`autotermination_minutes=60`	Terminate after 60 mins idle
Not set (Clusters API)	Cluster is not automatically terminated

Job Clusters

Job clusters are created for specific job runs and terminated after completion.

Characteristics

```
flowchart LR
    JC[Job Cluster] --> Created["Created per run"]
    JC --> Isolated["Isolated execution"]
    JC --> Auto["Auto-terminated"]
    JC --> Cost["Lower cost"]
```

Feature	Behavior
Lifecycle	Created when job starts, terminated when job ends
Isolation	Dedicated to single job run
Cost model	Pay only during job execution
Configuration	Defined in job specification
Reuse	Cannot be reused between runs

Job Cluster Configuration

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.jobs import Task, NotebookTask

w = WorkspaceClient()

job = w.jobs.create(
    name="Daily ETL",
    tasks=[
        Task(
            task_key="etl_task",
            notebook_task=NotebookTask(
                notebook_path="/Jobs/etl_pipeline"
            ),
            new_cluster={  # Job cluster definition
                "spark_version": "14.3.x-scala2.12",
                "node_type_id": "Standard_DS3_v2",
                "num_workers": 4,
                "spark_conf": {
                    "spark.sql.shuffle.partitions": "200"
                }
            }
        )
```

```
    ]
)
```

Job Cluster vs Existing Cluster

```
# Using job cluster (recommended for production)

task_with_new_cluster = Task(
    task_key="task1",
    new_cluster={
        "spark_version": "14.3.x-scala2.12",
        "num_workers": 2,
        "node_type_id": "Standard_DS3_v2"
    },
    notebook_task=NotebookTask(notebook_path="/Jobs/task1")
)

# Using existing all-purpose cluster (for development)

task_with_existing_cluster = Task(
    task_key="task1",
    existing_cluster_id="1234-567890-abc",
    notebook_task=NotebookTask(notebook_path="/Jobs/task1")
)
```

Approach	Pros	Cons
Job Cluster	Isolated, cost-effective, consistent	Startup overhead
Existing Cluster	Faster start, shared resources	Resource contention, higher cost

Instance Pools

Instance pools maintain a set of idle instances ready for quick allocation.

How Pools Work

```
sequenceDiagram
    participant Job as Job/Cluster
    participant Pool as Instance Pool
    participant Cloud as Cloud Provider

    Job->>Pool: Request instances
    Pool->>Pool: Check idle instances
    alt Instances available
        Pool->>Job: Allocate from pool (seconds)
    else Need more instances
        Pool->>Cloud: Request new instances
        Cloud->>Pool: Provision (minutes)
        Pool->>Job: Allocate instances
    end
    Job->>Job: Run workload
    Job->>Pool: Return instances to pool
```

Pool Configuration

```python
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()

pool = w.instance_pools.create(
    instance_pool_name="etl-pool",
    node_type_id="Standard_DS3_v2",
    min_idle_instances=2,  # Always keep 2 instances ready
    max_capacity=10,       # Maximum instances in pool
    idle_instance_autotermination_minutes=30,  # Terminate idle after 30 mins
    preloaded_spark_versions=["14.3.x-scala2.12"]  # Pre-install Spark
)

print(f"Pool ID: {pool.instance_pool_id}")
```

Using Pools with Clusters

```python
# All-purpose cluster using pool

cluster = w.clusters.create(
    cluster_name="dev-cluster",
    spark_version="14.3.x-scala2.12",
    instance_pool_id="pool-123",  # Use pool for workers
    driver_instance_pool_id="pool-123",  # Use pool for driver
    num_workers=2
)

# Job cluster using pool

job = w.jobs.create(
    name="Pooled Job",
    tasks=[Task(
        task_key="task1",
        new_cluster={
            "spark_version": "14.3.x-scala2.12",
            "instance_pool_id": "pool-123",
            "driver_instance_pool_id": "pool-123",
            "num_workers": 4
        },
        notebook_task=NotebookTask(notebook_path="/Jobs/task1")
    )]
)
```

Pool Best Practices

Setting	Recommendation	Reason
`min_idle_instances`	1-2	Balance cost vs startup time
`max_capacity`	Based on peak usage	Prevent over-provisioning
`idle_instance_autotermination_minutes`	15-30	Reduce idle costs
`preloaded_spark_versions`	Your common versions	Faster startup

Cluster Policies

Cluster policies enforce configuration standards and cost controls.

Policy Structure

```
{
  "spark_version": {
    "type": "fixed",
    "value": "14.3.x-scala2.12"
  },
  "node_type_id": {
    "type": "allowlist",
    "values": ["Standard_DS3_v2", "Standard_DS4_v2"]
  },
  "num_workers": {
    "type": "range",
    "minValue": 1,
    "maxValue": 10
  },
  "autotermination_minutes": {
    "type": "fixed",
    "value": 60,
    "hidden": true
  },
  "custom_tags.team": {
    "type": "fixed",
    "value": "data-engineering"
  }
}
```

Policy Types

Type	Behavior	Example
`fixed`	Cannot be changed	Specific Spark version
`allowlist`	Must be from list	Approved instance types
`blocklist`	Cannot be from list	Expensive instances
`range`	Within min/max	1-10 workers
`unlimited`	No restriction	Any value allowed

Creating Policies

```
import json

policy = w.cluster_policies.create(
    name="Standard ETL Policy",
    definition=json.dumps({
        "spark_version": {"type": "fixed", "value": "14.3.x-scala2.12"},
        "node_type_id": {
            "type": "allowlist",
            "values": ["Standard_DS3_v2", "Standard_DS4_v2"]
        },
        "num_workers": {"type": "range", "minValue": 1, "maxValue": 20},
        "autotermination_minutes": {"type": "fixed", "value": 60}
    })
)
```

Using Policies

```python
# Create cluster with policy

cluster = w.clusters.create(
    cluster_name="policy-cluster",
    policy_id="policy-123",  # Apply policy
    num_workers=4  # Must comply with policy
)
```

Autoscaling

Cluster Autoscaling

```python
# Enable autoscaling

cluster = w.clusters.create(
    cluster_name="autoscale-cluster",
    spark_version="14.3.x-scala2.12",
    node_type_id="Standard_DS3_v2",
    autoscale={
        "min_workers": 2,
        "max_workers": 10
    }
)
```

Autoscaling Behavior

```
flowchart TD
    Start[Workload Starts] --> Check{Resource Usage}
    Check --> |High CPU/Memory| ScaleUp[Add Workers]
    Check --> |Low Usage| ScaleDown[Remove Workers]
    Check --> |Normal| Maintain[Keep Current]

    ScaleUp --> |Max reached| Cap[At max_workers]
    ScaleDown --> |Min reached| Floor[At min_workers]
```

Metric	Scale Up Trigger	Scale Down Trigger
Pending tasks	High queue	Empty queue
Executor memory	>90% used	<30% used
Shuffle	Spilling to disk	Under-utilized

Optimized Autoscaling

```python
# Enhanced autoscaling (Databricks-specific)

cluster = w.clusters.create(
    cluster_name="optimized-autoscale",
    spark_version="14.3.x-scala2.12",
    node_type_id="Standard_DS3_v2",
    autoscale={
        "min_workers": 1,
        "max_workers": 8
    },
    spark_conf={
        "spark.databricks.cluster.scaling.optimized.enabled": "true"
```

```
    }
)
```

Spot Instances

Use spot/preemptible instances for cost savings.

Spot Configuration

```
# AWS spot instances

cluster = w.clusters.create(
    cluster_name="spot-cluster",
    spark_version="14.3.x-scala2.12",
    node_type_id="m5.xlarge",
    num_workers=4,
    aws_attributes={
        "first_on_demand": 1,  # Driver on-demand
        "availability": "SPOT_WITH_FALLBACK",
        "spot_bid_price_percent": 100
    }
)

# Azure spot instances

cluster = w.clusters.create(
    cluster_name="spot-cluster",
    spark_version="14.3.x-scala2.12",
    node_type_id="Standard_DS3_v2",
    num_workers=4,
    azure_attributes={
        "first_on_demand": 1,
        "availability": "SPOT_WITH_FALLBACK_AZURE",
        "spot_bid_max_price": -1  # Use current price
    }
)
```

Spot Availability Options

Option	Behavior	Risk
`SPOT`	All spot instances	Higher interruption risk
`SPOT_WITH_FALLBACK`	Spot preferred, on-demand fallback	Lower risk, higher potential cost
`ON_DEMAND`	All on-demand	No interruption, highest cost

Cluster Permissions

Permission Levels

Permission	Capabilities
No Permission	Cannot see cluster
Can Attach To	Attach notebooks
Can Restart	Attach + restart cluster
Can Manage	Full control

Setting Permissions

```
from databricks.sdk.service.iam import AccessControlList, AccessControlRequest

w.permissions.set(
    object_type="clusters",
    object_id="cluster-id",
    access_control_list=[
        AccessControlRequest(
            user_name="user@company.com",
            permission_level="CAN_ATTACH_TO"
        ),
        AccessControlRequest(
            group_name="data-engineers",
            permission_level="CAN_RESTART"
        )
    ]
)
```

Continue reading: Part 2 — Serverless Compute, Environments, SQL Warehouses, Photon & Exam Tips

---

Databricks Compute — Part 2

This part covers serverless compute for notebooks, jobs, and Lakeflow Pipelines, including how to configure environments and dependencies, performance mode, high-memory notebook tasks, and auto-optimization, plus SQL warehouses and Photon.

> For all-purpose and job clusters, instance pools, cluster policies, autoscaling, spot instances, and cluster permissions, see Part 1.

Overview

Overview

- Serverless compute is Databricks-managed and available by default in Unity Catalog workspaces; Databricks recommends it for all job tasks that support it.
- Instead of a Databricks Runtime version, serverless workloads pick an **environment version** (Python version + preinstalled packages, supported for three years) and a **base environment**, then add dependencies.
- **Performance optimized** (default) vs **standard** performance mode trades startup latency (standard: 4–6 minutes) for fewer DBUs on jobs and pipelines.
- **High memory** (32 GB instead of 16 GB) applies to notebooks and notebook tasks; **serverless auto-optimization** retries failed tasks and can be disabled for at-most-once workloads.

What the Exam Tests

- ”Choose appropriate compute and configuration for environments and dependencies — including serverless compute (serverless environments, dependency management, performance mode), high-memory notebook tasks, and auto-optimization settings (e.g., disallowing retries).”
- Mapping a requirement (lowest cost, fastest start, driver OOM, non-idempotent task, classic-only feature) to the right serverless setting or to classic compute.

- Knowing which serverless settings exist for notebooks vs jobs vs pipelines.

Serverless Compute

Serverless compute is a Databricks-managed service for notebooks, Lakeflow Jobs, and Lakeflow Pipelines. Databricks allocates and scales the compute; autoscaling and Photon are enabled automatically for serverless jobs, and Databricks upgrades the underlying runtime without code changes.

```
flowchart LR
    User[User] --> Notebook[Notebook]
    Notebook --> Serverless[Serverless Compute]
    Serverless --> |Seconds| Execution[Code Execution]
    Job[Lakeflow Job] --> Serverless
    Pipeline[Lakeflow Pipeline] --> Serverless
```

Feature	Behavior
Startup time	Seconds in performance-optimized mode; 4–6 minutes in standard mode (jobs and pipe
Notebook languages	Python and SQL (Scala and R are not supported in notebooks)
Scaling	Automatic, including vertical autoscaling for serverless pipelines
Access mode	Standard access mode; requires Unity Catalog
Cost	Pay per compute usage; attribute with serverless usage policies
Management	No cluster configuration; cluster creation permission not required

Enabling serverless Serverless compute is available by default in most workspaces and needs no enablement; workspaces enabled for Unity Catalog have access automatically. Legacy workspaces without Unity Catalog must upgrade to use it. To use it, select **Serverless** in the notebook compute drop-down, keep serverless as the compute for supported job task types (it's the default), or check **Serverless** in pipeline settings (`serverless: true` in a bundle).

Serverless limitations

Supported	Not supported
Python and SQL notebooks	Scala and R in notebooks (R is not su
Notebook, Python script, Python wheel, dbt, and JAR job tasks	JAR libraries in notebooks; Maven coo
Spark Connect DataFrame and SQL APIs	Spark RDD APIs
`Trigger.AvailableNow()` (recommended) and `Trigger.Once()`	`Trigger.ProcessingTime` and `Trigg`
Base environments and environment dependencies	Init scripts, compute policies, comput
Allowlisted Spark configuration properties	Most other Spark configurations
Query profile for diagnostics	Spark UI and Spark logs
`df.cache()` / `df.persist()` (Public Preview, environment version 5+)	`CACHE TABLE`, `spark.catalog.cache`
Unity Catalog volumes and workspace files	DBFS mounts with instance profiles; g
Runs up to 7 days	Runs longer than 7 days (terminated

Lakeflow Pipelines aren't bound by the trigger restrictions: serverless pipelines support triggered, continuous, and real-time modes.

Serverless Environments and Configuration

Environment versions An environment version provides a stable client API, based on Spark Connect, so Databricks can upgrade the server independently. Each version fixes a Python version

and a set of preinstalled packages, gets security updates for three years, and is used by serverless notebooks and jobs (and by classic compute in the Environments dependency mode, Beta).

Version	Python	Databricks Connect	Released	End of support
6	3.12.3	19	September 3, 2026	September 3, 2029
5	3.12.3	18	February 25, 2026	February 25, 2029
4	3.12.3	17.3.2	August 13, 2025	August 13, 2028
3	3.12.3	16.4.2	June 13, 2025	June 13, 2028
2	3.11.10	15.4.5	November 21, 2024	November 21, 2027
1	3.10.12	14.3.7	March 12, 2024	March 12, 2027

After end of support, a version is no longer selectable for new jobs and notebooks in the UI (still configurable through the API), existing workloads keep running but might start more slowly, and new Unity Catalog UDFs that reference it trigger a warning. Several features need a minimum version: `%uv` commands and Shared Serverless need 5+, and DataFrame caching needs 5+.

Base environments

Base environment	What it provides
Standard	Default serverless environment with Databricks-provided libraries
ML	Python and system packages from Databricks Runtime for Machine Learning
AI	AI-optimized environment for serverless GPU (shown only when an accelerator is
Previous versions / Custom	Earlier Standard, ML, or AI versions, or a custom YAML specification
Workspace environments	Admin-built, cached YAML environments; load quickly and speed up job startup

A base environment is a shareable YAML specification. Build one in the Environment side pane and use **Export environment**, or write it directly:

```yaml
environment_version: '5'
dependencies:
  - --index-url https://pypi.org/simple
  - -r "/Workspace/Shared/requirements.txt"
  - my-library==6.1
  - /Workspace/Shared/Path/To/simplejson-3.19.3-py3-none-any.whl
  - git+https://github.com/databricks/databricks-cli
```

Workspace admins register workspace base environments under **Settings → Compute → Base environments for serverless compute**, optionally as the default for new notebooks.

Configure the environment per workload

Workload	Where you configure it
Serverless notebook	Environment side pane: base environment, dependencies, memory, serve
Notebook task	Uses the notebook's own environment by default; can be overridden with a
Python script / Python wheel task	Job environment referenced by `environment_key`
dbt task	Job-level environment
JAR task	Environment version + JAR dependencies
Serverless pipeline	Pipeline Environment settings (`environment.dependencies`, optional envi

The base environment must match the compute type: a GPU environment isn't compatible with CPU compute. Tasks that share the same dependency set within a run reuse the cached environ-

ment. If you change a custom package's code, also change its version number so serverless jobs pick up the new implementation.

Dependency formats, private repositories, and troubleshooting are covered in Python Project Structure and Dependencies.

Performance mode

Mode	Behavior
Performance optimized (default)	Faster startup and execution; Databricks keeps warm capacity
Standard	Uses less compute; typical startup 4–6 minutes; fewer DBUs on the same S

- **Jobs:** toggle **Performance optimized** in the job details page. The job needs at least one serverless task, and the setting affects only its serverless tasks. In the Jobs API and bundles the job field is `performance_target` (`PERFORMANCE_OPTIMIZED` default, or `STANDARD`). It doesn't apply to tasks on serverless GPU compute.
- **Pipelines:** for triggered pipelines, set **Performance optimized** in the pipeline scheduler. For a continuous pipeline in standard mode, run it with a continuous job and clear **Performance optimized** in that schedule.
- **Notebooks:** standard mode isn't available for notebooks.
- Since July 6, 2026, materialized view and streaming table refreshes orchestrated through a SQL task in a job respect the job's Performance optimized setting.

High memory for notebooks and notebook tasks

> **Note**
>
> High memory serverless compute is in Public Preview.

Memory option	Total memory
Standard	16 GB
High	32 GB

- Set it in the notebook's Environment side pane (**Memory → High memory → Apply**) when a notebook hits out-of-memory errors.
- It increases the REPL memory used to run the notebook's code, not the memory of the Spark session, and has a higher DBU emission rate than standard memory.
- It's available only for notebook task types. Notebook job tasks that use the notebook's memory preference pick it up, and a change applies from the next job run.
- It's separate from the 1 GB per-UDF memory limit on serverless.

Auto-optimization and disallowing retries Serverless auto-optimization automatically optimizes the compute that runs a job and **retries failed tasks**. It's on by default, and Databricks recommends leaving it on so critical workloads run successfully at least once. For workloads that must run **at most once**, such as non-idempotent jobs, turn it off per task:

1. In the task, next to **Retries**, click **Add** (or edit the existing retry policy).
2. In the **Retry Policy** dialog, uncheck **Enable serverless auto-optimization (may include additional retries)**.
3. Click **Confirm**, then **Save task**.

In the Jobs API the task field is `disable_auto_optimization` (default `false`); pair it with `max_retries: 0` (the default) so no explicit retry policy reruns the task.

```
{
  "name": "nightly_payment_export",
  "performance_target": "STANDARD",
  "environments": [
    {
      "environment_key": "default",
     "spec": {"environment_version": "4", "dependencies": ["simplejson==3.19.*"]}
    }
  ],
  "tasks": [
    {
      "task_key": "export",
      "environment_key": "default",
      "spec_python_task": {"python_file": "/Workspace/Shared/etl/export.py"},
      "max_retries": 0,
      "disable_auto_optimization": true
    }
  ]
}
```

Choosing compute and configuration

Requirement	Choose
Supported task type, no classic-only features	Serverless
Lowest cost for a nightly batch that tolerates a few minutes of startup	Serverless
Tight SLA or interactive response	Serverless
Notebook code (the REPL, not Spark) runs out of memory	**High mem**
Non-idempotent task must run at most once	Uncheck se
Reproducible Python packages across runs	Pin the env
Init scripts, Docker containers, R, RDD APIs, Spark UI, instance pools, or a run longer than 7 days	Classic com
Continuous streaming	Serverless

SQL Warehouses

SQL warehouses are optimized for SQL analytics and BI workloads.

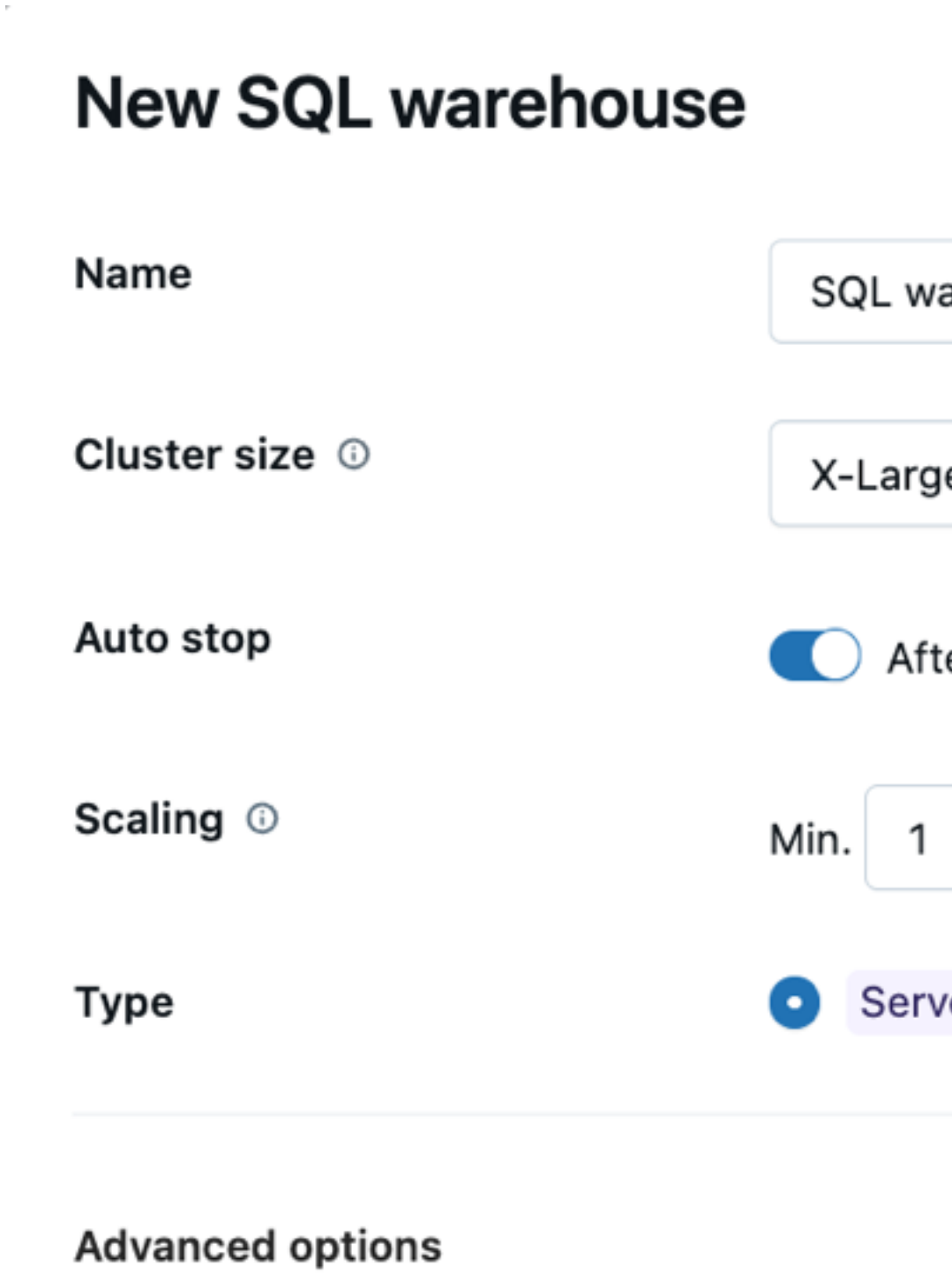
Warehouse Types

```
flowchart TB
    SQLWarehouse[SQL Warehouse] --> Classic[Classic]
    SQLWarehouse --> Pro[Pro]
    SQLWarehouse --> Serverless[Serverless]

    Classic --> C1["Standard SQL"]
    Pro --> P1["Enhanced security"]
    Pro --> P2["Query federation"]
    Serverless --> S1["Instant startup"]
    Serverless --> S2["Auto-scaling"]
```

Type	Features	Best For
Classic	Standard SQL, Photon	Cost-sensitive workloads
Pro	+ Query federation, enhanced security	Enterprise features
Serverless	Instant start, auto-scale, no management	Ad-hoc queries, variable load

Type	Features	Best For



Creating SQL Warehouse

SQL Warehouse creation form showing cluster size, auto-stop, and scaling settings.

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.sql import EndpointInfo

w = WorkspaceClient()

warehouse = w.warehouses.create(
    name="analytics-warehouse",
    cluster_size="Small",  # 2X-Small, X-Small, Small, Medium, Large, X-Large,
 ↪  2X-Large, 3X-Large, 4X-Large
    min_num_clusters=1,
    max_num_clusters=4,
    auto_stop_mins=30,
    enable_photon=True,
    warehouse_type="PRO",  # CLASSIC, PRO
    spot_instance_policy="COST_OPTIMIZED"  # COST_OPTIMIZED, RELIABILITY_OPTIMIZED
).result()

print(f"Warehouse ID: {warehouse.id}")
```

Warehouse Sizing

Size	Cluster DBU/hour	Use Case
2X-Small	2	Light queries
X-Small	4	Development
Small	8	Small team
Medium	16	Medium workloads
Large	32	Heavy analytics
X-Large+	64+	Enterprise scale

Auto-Scaling

```
# Configure scaling

warehouse = w.warehouses.edit(
    id="warehouse-id",
    min_num_clusters=1,   # Minimum clusters running
    max_num_clusters=10,  # Maximum concurrent clusters
    auto_stop_mins=15     # Stop after 15 mins idle
)
```

```
flowchart LR
    Load[Query Load] --> Scale{Scaling Decision}
    Scale --> |High load| ScaleUp[Add clusters]
    Scale --> |Low load| ScaleDown[Remove clusters]
    Scale --> |No queries| Stop[Auto-stop]
```

Photon Engine

Photon is Databricks' native vectorized query engine for accelerated SQL and DataFrame operations.

Photon Benefits

Workload	Speedup	Notes
Aggregations	2-8x	SIMD vectorization
Joins	2-5x	Optimized hash joins
Scans	2-4x	Columnar processing
Filters	2-5x	Predicate pushdown

Enabling Photon

```
# On clusters

cluster = w.clusters.create(
    cluster_name="photon-cluster",
    spark_version="14.3.x-photon-scala2.12",  # Use Photon runtime
    runtime_engine="PHOTON",  # Explicitly enable
    # ...
)

# On SQL warehouses (enabled by default)

warehouse = w.warehouses.create(
    name="photon-warehouse",
    enable_photon=True,  # Enable Photon
    # ...
)
```

Photon Limitations

Accelerated	Not Accelerated
SQL queries	Python UDFs
DataFrame operations	Scala UDFs
Delta Lake operations	Complex nested types
Parquet/ORC reads	Some window functions

Use Cases

Development Workflow

Stage	Compute Type
Exploration	Serverless notebook or small all-purpose cluster
Development	Serverless notebook (or all-purpose cluster for classic-only features)
Testing	Serverless job or job cluster
Production	Serverless job (standard mode for cost, performance optimized for SLAs) or job cluster with pc

Compute Decision Tree

```
flowchart TD
    Start[Choose Compute] --> Interactive{Interactive?}
    Interactive --> |Yes| ServerlessNB[Serverless notebook<br/>High memory if
 ↪  REPL OOM]
```

```
    Interactive --> |No| Supported{Task type supported on serverless<br/>and no
↪   classic-only features?}

    Supported --> |Yes| Latency{Startup latency acceptable?}
    Supported --> |No| Frequent{Frequent runs?}

    Latency --> |Yes| Standard[Serverless job<br/>standard performance mode]
    Latency --> |No| Optimized[Serverless job<br/>performance optimized]

    Frequent --> |Yes| Pool[Job Cluster + Pool]
    Frequent --> |No| JobCluster[Job Cluster]
```

Common Issues & Errors

Cluster Startup Timeout **Scenario:** Classic cluster takes too long to start.

Fix: Use instance pools or check cloud quotas, or move the task to serverless.

Out of Memory **Scenario:** Executor or driver OOM errors on classic compute.

Fix: Increase instance size or add workers:

```
# Increase driver memory

spark_conf = {
    "spark.driver.memory": "8g"
}
```

Serverless Notebook Runs Out of Memory **Scenario:** A serverless notebook or notebook task fails with out-of-memory errors in notebook code.

Fix: Select **High memory** (32 GB) in the Environment side pane. For a PySpark UDF hitting `UDF_PYSPARK_USER_CODE_ERROR.MEMORY_LIMIT_SERVERLESS`, reduce the UDF's footprint instead; the 1 GB per-UDF limit is separate.

Serverless Job Starts Slowly **Scenario:** A serverless job takes 4–6 minutes to start.

Fix: The job runs in standard performance mode. Enable **Performance optimized** (`performance_target: PERFORMANCE_OPTIMIZED`) if the SLA can't absorb the latency.

Non-Idempotent Task Ran Twice **Scenario:** A serverless task that sends payments or appends without deduplication ran more than once after a failure.

Fix: Serverless auto-optimization retries failed tasks. Uncheck **Enable serverless auto-optimization** in the task's retry policy (`disable_auto_optimization: true`) and keep `max_retries` at 0.

Unsupported Streaming Trigger on Serverless **Scenario:** A streaming query in a serverless job fails with `INFINITE_STREAMING_TRIGGER_NOT_SUPPORTED`.

Fix: Use `Trigger.AvailableNow()` (or a serverless pipeline in continuous mode). Without an explicit trigger, Spark defaults to `ProcessingTime("0 seconds")`, which serverless notebooks and jobs don't support.

Spot Instance Interruption **Scenario:** Spot instances preempted during job.

Fix: Use SPOT_WITH_FALLBACK or set first_on_demand:

```
aws_attributes = {
    "first_on_demand": 2,  # Keep driver + 1 worker on-demand
    "availability": "SPOT_WITH_FALLBACK"
}
```

Policy Violation **Scenario:** Cluster creation blocked by policy.

Fix: Adjust cluster config to comply with policy or request policy change.

Quota Exceeded **Scenario:** Cannot create cluster due to cloud quota.

Fix: Request quota increase from cloud provider or terminate unused resources.

Exam Tips

1. **Job vs All-Purpose** - Job clusters are isolated and cost-effective for production on classic compute
2. **Pools** - Reduce classic startup time from minutes to seconds
3. **Serverless first** - Databricks recommends serverless for all job tasks that support it; serverless notebooks are Python and SQL only (no Scala or R)
4. **No classic knobs on serverless** - No init scripts, cluster libraries, compute policies, instance pools, or Spark UI; use environments and the query profile
5. **Environment version** - Pins Python and preinstalled packages for serverless; each version is supported for three years
6. **Performance mode** - Performance optimized (default, fast) vs standard (4–6 min startup, fewer DBUs); jobs and pipelines only, not notebooks
7. **High memory** - 32 GB vs 16 GB; notebooks and notebook tasks only; raises REPL memory, not Spark memory
8. **Auto-optimization** - On by default and retries failed tasks; uncheck it for at-most-once (non-idempotent) tasks
9. **Photon** - Accelerates SQL/DataFrame ops, not UDFs
10. **Auto-termination** - 10–10,000 minutes of inactivity; `0` disables it (not set via the API = never terminates)
11. **Autoscaling** - Based on pending tasks and resource usage
12. **Spot instances** - Use first_on_demand for driver reliability
13. **Cluster permissions** - Can Attach To < Can Restart < Can Manage
14. **Policies** - fixed, allowlist, blocklist, range, unlimited types
15. **SQL Warehouse sizes** - Know Small = 8 DBU/hour baseline

Key Takeaways

- **Job clusters vs all-purpose clusters**: job clusters are created per run, isolated, and auto-terminated — lower cost for production on classic compute; all-purpose clusters persist and are shared across users — better for interactive development
- **Instance pools** keep a set of pre-provisioned idle instances that reduce cluster startup from minutes to seconds; clusters reference a pool via `instance_pool_id`
- **Serverless compute** is Databricks-managed, available by default with Unity Catalog, and recommended for supported job tasks; serverless notebooks support Python and SQL only, RDD APIs aren't supported, and JAR libraries can't be used in notebooks (JAR tasks in jobs are supported)
- **Environment versions** (1–6) fix the Python version and package set; choose a base environment (Standard, ML, AI, custom YAML, or an admin-built workspace environment) and add pinned dependencies

- **Where to configure serverless dependencies**: notebook Environment side pane; job `environments` + `environment_key` (required for Python script and wheel tasks); pipeline Environment settings
- **Performance mode**: `performance_target` `PERFORMANCE_OPTIMIZED` (default) or `STANDARD`; standard mode saves DBUs at the cost of 4–6 minute startup and isn't available for notebooks
- **High memory**: 32 GB REPL memory (Public Preview) for notebooks and notebook tasks, at a higher DBU rate
- **Auto-optimization** retries failed serverless tasks; disable it (`disable_auto_optimization`) for workloads that must run at most once
- **Photon** accelerates SQL queries and DataFrame operations (joins, aggregations, scans, filters) via vectorized C++ execution but does not accelerate Python or Scala UDFs
- **Auto-termination** accepts 10–10,000 minutes of inactivity; `autotermination_minutes=0` disables it (not recommended for cost management)
- **Cluster policy types**: `fixed` (enforced value), `allowlist` (must be in list), `blocklist` (must not be in list), `range` (min/max), `unlimited` (unrestricted)
- **SQL Warehouse types**: Classic (standard), Pro (query federation, enhanced security), Serverless (instant startup, auto-scale, no management overhead)
- **Spot instance availability**: `SPOT` (all spot, higher interruption risk), `SPOT_WITH_FALLBACK` (spot preferred with on-demand fallback), `ON_DEMAND` (no interruption, highest cost)

Related Topics

- Databricks Compute — Part 1 - Classic clusters, pools, policies, spot instances
- Python Project Structure and Dependencies - Dependency formats and troubleshooting
- UDFs and Unity Catalog Functions - Serverless UDF limits
- Lakeflow Jobs — Part 1 - Task types and retries
- Workspace and Notebooks — Part 2 - Compute attachment
- Cost Optimization - Reducing spend

Official Documentation

- Connect to serverless compute
- Configure the serverless environment
- Serverless environment versions
- Manage workspace base environments
- Run your Lakeflow Jobs with serverless compute for workflows
- Configure compute for jobs
- Configure a serverless pipeline
- Configure environment versions for pipelines
- Best practices for serverless compute
- Serverless compute limitations
- Jobs API: create a job (performance_target, disable_auto_optimization)
- July 2026 release notes (standard performance mode for MV/ST refreshes)
- Compute Overview
- Cluster Configuration
- Manage compute (automatic termination)
- Instance Pools
- SQL Warehouses
- Photon

Workspace and Notebooks — Part 1

Databricks notebooks are the primary interface for interactive data engineering. Understanding notebook features, magic commands, widgets, and dbutils is essential for building production

pipelines. This part covers workspace organization, notebook languages and magic commands, and widgets; Part 2 covers `dbutils`, notebook permissions, formats, and compute attachment.

Overview

```
flowchart TB
    subgraph Workspace["Databricks Workspace"]
        Folders[Folders] --> Notebooks[Notebooks]
        Folders --> Libraries[Libraries]
        Folders --> Repos[Git Repos]
    end

    subgraph Notebook["Notebook Features"]
        Cells[Code Cells] --> Magic[Magic Commands]
        Cells --> Widgets[Widgets]
        Cells --> Dbutils[dbutils]
    end

    Workspace --> Notebook
```

Overview

- **Workspace structure**: folders, notebooks, and Git folders; `/Workspace/...` paths for code vs `dbfs:/...` or `/dbfs/...` for DBFS data
- **Magic commands**: `%sql`, `%python`, and `%scala` switch language; `%fs`, `%pip`, and `%sh`; `%run` runs another notebook in the same context
- **Widgets** (`text`, `dropdown`, `combobox`, `multiselect`) parameterize notebooks; in job runs, job parameters override widget defaults

What the Exam Tests

- Create and automate ETL workloads using Lakeflow Jobs via the UI, API, and CLI. Notebooks are the most common job task, parameterized through widgets.
- Choose the widget type for a parameter and know how job parameters override widget defaults
- Know what `%run` shares with the calling notebook

Workspace Organization

Workspace Structure

```
/Workspace/
├── Users/
│   └── user@company.com/          # Personal user folders
│       ├── notebooks/
│       └── projects/
├── Shared/                        # Shared across organization
│   ├── common_functions/
│   └── templates/
├── Repos/                         # Git-connected repositories
│   └── project-name/
└── Applications/                  # Databricks Apps (if enabled)
```

Workspace vs DBFS Paths

Path Type	Format	Access	Use Case
Workspace	`/Workspace/Users/...`	Notebooks, files	Code, configs
DBFS	`dbfs:/...` or `/dbfs/...`	Data files	Data storage
Unity Catalog Volumes	`/Volumes/catalog/schema/volume/`	Governed data	Managed files

```
# Read file from workspace

with open("/Workspace/Users/user@company.com/config.json", "r") as f:
    config = json.load(f)

# Read file from DBFS

df = spark.read.format("csv").load("dbfs:/data/input.csv")

# Read from Unity Catalog Volume

df = spark.read.format("parquet").load("/Volumes/main/default/raw_data/")
```

Notebook Languages

Default Languages Notebooks have a default language set at creation:

Language	Extension	Use Case
Python	`.py`	General purpose, ML, ETL
SQL	`.sql`	Data analysis, queries
Scala	`.scala`	Performance-critical code
R	`.r`	Statistical analysis

Magic Commands Magic commands allow switching languages within a notebook.

```
# Default cell runs in notebook's default language

df = spark.read.table("my_table")
```

```
%sql
-- Switch to SQL for this cell
SELECT * FROM my_table LIMIT 10
```

```
%scala
// Switch to Scala for this cell
val df = spark.read.table("my_table")
```

Complete Magic Commands Reference

Command	Purpose	Example
`%python`	Run Python code	`%python print("Hello")`
`%sql`	Run SQL queries	`%sql SELECT * FROM table`
`%scala`	Run Scala code	`%scala val x = 1`
`%r`	Run R code	`%r print("Hello")`
`%md`	Render Markdown	`%md # Header`
`%run`	Execute another notebook	`%run ./utils/helpers`
`%fs`	DBFS file operations	`%fs ls /data/`

Command	Purpose	Example
`%sh`	Shell commands	`%sh ls -la`
`%pip`	Install Python packages	`%pip install pandas`

The %run Command `%run` executes another notebook in the same context, making its variables and functions available.

```
# In /Workspace/Users/user/utils/helpers

def clean_data(df):
    return df.dropna().dropDuplicates()

ENVIRONMENT = "production"
```

```
# In main notebook

%run ./utils/helpers

# Now clean_data() and ENVIRONMENT are available

cleaned_df = clean_data(raw_df)
print(ENVIRONMENT)  # "production"
```

%run Characteristics:

Feature	Behavior
Variable scope	Runs in same context - variables are shared
Relative paths	Relative to current notebook location
Parameters	Use widgets for parameterization (no direct args)
Return values	Cannot return values directly
Error handling	Errors propagate to calling notebook

```
# Pass parameters via widgets

%run ./etl_pipeline

# In etl_pipeline notebook:
# dbutils.widgets.text("date", "2024-01-01")
# date = dbutils.widgets.get("date")
```

The %fs Command `%fs` provides quick DBFS operations without Python/Scala code.

```
%fs ls /data/bronze/

%fs head /data/sample.csv

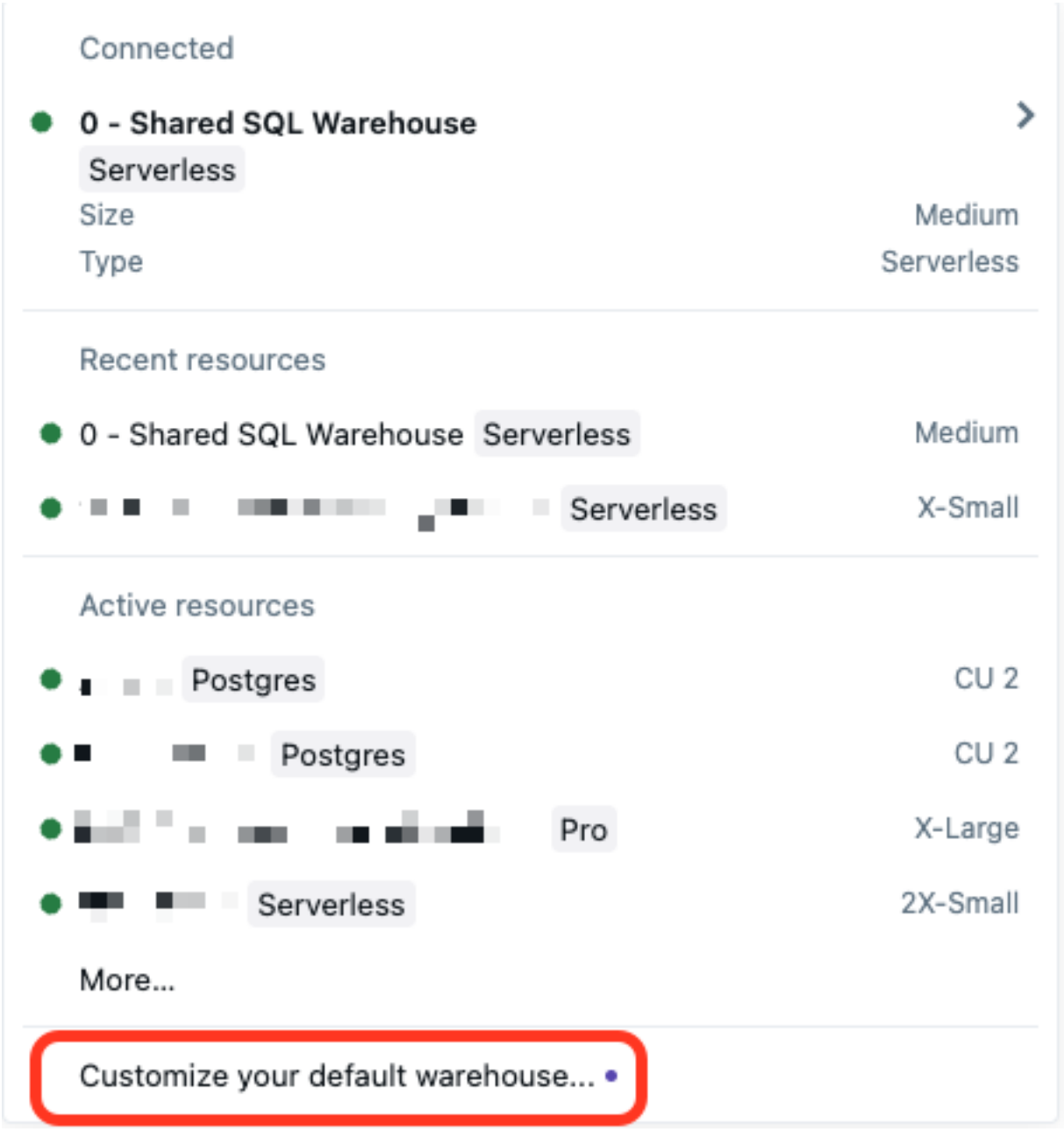
%fs cp /source/file.csv /dest/file.csv

%fs rm /data/temp/ --recurse
```

Operation	Command	Description
List files	`%fs ls /path/`	List directory contents
View file	`%fs head /path/file`	Show first 64KB of file

Operation	Command	Description
Copy	`%fs cp /src /dest`	Copy file or directory
Move	`%fs mv /src /dest`	Move file or directory
Remove	`%fs rm /path`	Delete file
Remove dir	`%fs rm /path --recurse`	Delete directory recursively
Make dir	`%fs mkdirs /path`	Create directory

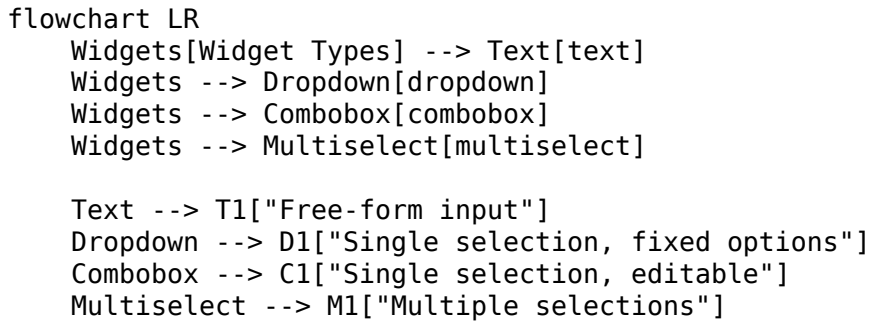
Widgets



Databricks SQL dashboard with dynamic parameters for interactive filtering.

Widgets provide parameterization for notebooks, enabling reusable pipelines.

Widget Types



Creating Widgets

```
# Text widget - free-form input

dbutils.widgets.text("start_date", "2024-01-01", "Start Date")

# Dropdown widget - single selection from list

dbutils.widgets.dropdown("environment", "dev", ["dev", "staging", "prod"],
 ↪  "Environment")

# Combobox widget - editable dropdown

dbutils.widgets.combobox("table_name", "customers", ["customers", "orders",
 ↪  "products"], "Table")

# Multiselect widget - multiple selections

dbutils.widgets.multiselect("regions", "US", ["US", "EU", "APAC"], "Regions")
```

Getting Widget Values

```
# Get single value

start_date = dbutils.widgets.get("start_date")
environment = dbutils.widgets.get("environment")

# Get multiselect values (returns comma-separated string)

regions = dbutils.widgets.get("regions")  # "US,EU"
region_list = regions.split(",")  # ["US", "EU"]
```

Managing Widgets

```
# Remove a specific widget

dbutils.widgets.remove("start_date")

# Remove all widgets from notebook

dbutils.widgets.removeAll()
```

Widget Scope and Jobs

Context	Widget Behavior
Interactive	Widget UI appears at top of notebook
Job run	Parameters passed via job configuration
%run	Widgets inherited from parent notebook

```
# When running as job, parameters override widget defaults
# Job configuration:
# {
#   "notebook_task": {
#     "notebook_path": "/path/to/notebook",
#     "base_parameters": {
#       "start_date": "2024-06-01",
#       "environment": "prod"
#     }
#   }
# }
```

SQL Widgets Widgets can be used directly in SQL cells:

```
-- Create widget in SQL
CREATE WIDGET TEXT start_date DEFAULT '2024-01-01';
CREATE WIDGET DROPDOWN env DEFAULT 'dev' CHOICES ['dev', 'staging', 'prod'];

-- Use widget in query
SELECT * FROM events
WHERE event_date >= '${start_date}'
AND environment = '${env}';

-- Get all widget values
SELECT getArgument('start_date') AS start_date;

-- Remove widgets
REMOVE WIDGET start_date;
```

Workspace and Notebooks — Part 2

Overview

This part covers `dbutils` (file system, secrets, notebook orchestration, job task values, libraries), notebook permissions, notebook formats, and compute attachment, plus use cases, common issues, and exam tips for the whole topic.

> For workspace organization, notebook languages and magic commands (including `%run`), and widgets, see Part 1.

Overview

- **`dbutils` modules**: `fs`, `secrets` (values redacted; `list()` returns key names only), `notebook`, `jobs`, and `library`
- **`dbutils.notebook.run`** runs an isolated child notebook with a timeout and a string return value; `ThreadPoolExecutor` runs children in parallel

- **`dbutils.jobs.taskValues`** passes values between tasks in a job run (job context only)
- **Notebook management**: permission levels (No Permission < Can Read < Can Run < Can Edit < Can Manage), export formats, and compute attachment

What the Exam Tests

- Create and automate ETL workloads using Lakeflow Jobs via the UI, API, and CLI. Notebooks are the most common job task, parameterized through widgets.
- Choose between `%run` and `dbutils.notebook.run` for a modularity or isolation need
- Know the notebook permission levels and what each allows

dbutils (Databricks Utilities)

`dbutils` is a utility library providing helper functions for file operations, secrets, widgets, and more.

dbutils Modules

```
flowchart TB
    dbutils --> fs["fs (File System)"]
    dbutils --> secrets["secrets"]
    dbutils --> widgets["widgets"]
    dbutils --> notebook["notebook"]
    dbutils --> jobs["jobs"]
    dbutils --> library["library"]
    dbutils --> credentials["credentials"]
```

dbutils.fs - File System Operations

```python
# List files

files = dbutils.fs.ls("/data/bronze/")
for file in files:
    print(f"{file.name} - {file.size} bytes")

# Check if path exists

def path_exists(path):
    try:
        dbutils.fs.ls(path)
        return True
    except Exception:
        return False

# Copy files

dbutils.fs.cp("/source/data.csv", "/dest/data.csv")
dbutils.fs.cp("/source/folder/", "/dest/folder/", recurse=True)

# Move files

dbutils.fs.mv("/old/path.csv", "/new/path.csv")

# Remove files
```

```
dbutils.fs.rm("/data/temp.csv")
dbutils.fs.rm("/data/temp_folder/", recurse=True)

# Create directory

dbutils.fs.mkdirs("/data/new_folder/")

# Read file head (first 64KB)

content = dbutils.fs.head("/data/sample.txt", maxBytes=1000)

# Write text file (not recommended for large data)

dbutils.fs.put("/data/output.txt", "Hello World", overwrite=True)
```

File System Operations Comparison

Operation	dbutils.fs	%fs magic	Spark
List	`dbutils.fs.ls()`	`%fs ls`	N/A
Copy	`dbutils.fs.cp()`	`%fs cp`	N/A
Move	`dbutils.fs.mv()`	`%fs mv`	N/A
Delete	`dbutils.fs.rm()`	`%fs rm`	N/A
Read data	N/A	N/A	`spark.read.format()`
Write data	N/A	N/A	`df.write.format()`

dbutils.secrets - Secret Management

```
# List available secret scopes

scopes = dbutils.secrets.listScopes()
for scope in scopes:
    print(scope.name)

# List secrets in a scope (names only, not values)

secrets = dbutils.secrets.list("my-scope")
for secret in secrets:
    print(secret.key)

# Get secret value

password = dbutils.secrets.get(scope="my-scope", key="db-password")
api_key = dbutils.secrets.get(scope="azure-kv", key="api-key")

# Secrets are redacted in logs

print(password)  # Prints [REDACTED] in notebook output
```

Secret Scope Types:

Type	Backend	Use Case
Databricks-backed	Databricks	Simple, managed secrets
Azure Key Vault	Azure Key Vault	Enterprise Azure integration

Type	Backend	Use Case
AWS Secrets Manager	AWS	Enterprise AWS integration

dbutils.notebook - Notebook Orchestration

```
# Run another notebook and get return value

result = dbutils.notebook.run(
    path="/Workspace/Users/user/etl/process_data",
    timeout_seconds=3600,
    arguments={"date": "2024-01-01", "env": "prod"}
)
print(f"Notebook returned: {result}")

# Exit notebook with return value

dbutils.notebook.exit("Success: Processed 1000 rows")

# In calling notebook, result contains the exit value
```

dbutils.notebook.run vs %run:

Feature	dbutils.notebook.run	%run
Return value	Yes (string)	No
Separate context	Yes (isolated)	No (shared)
Timeout	Configurable	None
Arguments	Pass directly	Use widgets
Error handling	Returns error	Raises exception
Parallel execution	Yes (via threading)	No

Parallel Notebook Execution

```
from concurrent.futures import ThreadPoolExecutor

def run_notebook(params):
    return dbutils.notebook.run(
        path="/etl/process_region",
        timeout_seconds=3600,
        arguments=params
    )

# Run notebooks in parallel

regions = [
    {"region": "US", "date": "2024-01-01"},
    {"region": "EU", "date": "2024-01-01"},
    {"region": "APAC", "date": "2024-01-01"}
]

with ThreadPoolExecutor(max_workers=3) as executor:
    results = list(executor.map(run_notebook, regions))

for result in results:
    print(result)
```

dbutils.jobs - Job Task Values Task values enable passing data between tasks in a job.

```
# Set task value (in upstream task)

dbutils.jobs.taskValues.set(key="row_count", value=1000)
dbutils.jobs.taskValues.set(key="status", value="success")
dbutils.jobs.taskValues.set(key="metrics", value={"processed": 1000, "failed":
 ↪  5})

# Get task value (in downstream task)
# Specify the task name that set the value

row_count = dbutils.jobs.taskValues.get(
    taskKey="extract_task",
    key="row_count",
    default=0
)

# Get complex value

metrics = dbutils.jobs.taskValues.get(
    taskKey="extract_task",
    key="metrics",
    default={}
)
```

```
sequenceDiagram
    participant Extract as Extract Task
    participant Transform as Transform Task
    participant Load as Load Task

    Extract->>Extract: Process data
    Extract->>Extract: taskValues.set("count", 1000)
    Extract->>Transform: Task completes

    Transform->>Transform: taskValues.get("extract", "count")
    Transform->>Transform: Process 1000 rows
    Transform->>Transform: taskValues.set("status", "success")
    Transform->>Load: Task completes

    Load->>Load: taskValues.get("transform", "status")
    Load->>Load: Final processing
```

dbutils.library - Library Management

```
# Restart Python interpreter (clears all variables)

dbutils.library.restartPython()

# Install library (deprecated - use %pip instead)
# dbutils.library.install("pypi-package")  # Not recommended
```

Best Practice: Use `%pip` for package installation:

```
%pip install requests==2.28.0 pandas==2.0.0

# Restart Python after installing
```

```
dbutils.library.restartPython()
```

Notebook Permissions

Permission Levels

Permission	Capabilities
No Permission	Cannot see notebook
Can Read	View notebook content
Can Run	Execute notebook (requires compute)
Can Edit	Modify notebook content
Can Manage	Full control including permissions

Setting Permissions

```
# Permissions are managed via UI or REST API
# Example REST API call to set permissions:
# POST /api/2.0/permissions/notebooks/{notebook_id}
# {
#   "access_control_list": [
#     {"user_name": "user@company.com", "permission_level": "CAN_EDIT"},
#     {"group_name": "data-engineers", "permission_level": "CAN_RUN"}
#   ]
# }
```

Notebook Formats

Export Formats

Format	Extension	Use Case
Source	`.py`, `.sql`, `.scala`, `.r`	Version control
DBC	`.dbc`	Archive with all cells
HTML	`.html`	Sharing results
Jupyter	`.ipynb`	Jupyter compatibility

Export/Import via CLI

```
# Export notebook as source file

databricks workspace export /Users/user/notebook /local/path/notebook.py --format
 ↪  SOURCE

# Export as DBC archive

databricks workspace export /Users/user/notebook /local/path/notebook.dbc
 ↪  --format DBC

# Import notebook

databricks workspace import /local/path/notebook.py /Users/user/notebook
 ↪  --language PYTHON
```

Compute Attachment

Attaching to Compute

```
flowchart LR
    Notebook --> Attach{Attach to}
    Attach --> AllPurpose[All-Purpose Cluster]
    Attach --> Serverless[Serverless Compute]
    Attach --> SQLWarehouse[SQL Warehouse]
```

Compute Type	Languages	Best For
All-Purpose Cluster	Python, SQL, Scala, R	Development, ML
Serverless Compute	Python, SQL	Quick development
SQL Warehouse	SQL only	BI, SQL analytics

Use Cases

Interactive Development

Scenario	Recommended Approach
Exploring data	SQL cells with `%sql` magic
Building pipelines	Python with widgets for parameters
Sharing results	Export to HTML or dashboard
Code reuse	`%run` for shared utilities

Production Pipelines

Scenario	Recommended Approach
Parameterized jobs	Widgets with job parameters
Orchestration	`dbutils.notebook.run` or Workflows
Secret handling	`dbutils.secrets.get`
Task communication	`dbutils.jobs.taskValues`

Multi-Notebook Architecture

```
flowchart TD
    Main[Main Orchestrator] --> Extract[Extract Notebook]
    Main --> Transform[Transform Notebook]
    Main --> Load[Load Notebook]

    subgraph Shared["Shared Utilities"]
        Utils[utils/helpers]
        Config[config/settings]
    end

    Extract -.-> Utils
    Transform -.-> Utils
    Load -.-> Utils
```

```
# Main orchestrator notebook

%run ./config/settings
%run ./utils/helpers
```

```python
# Sequential execution with error handling

try:
    result = dbutils.notebook.run("./etl/extract", 3600, {"date": process_date})
    print(f"Extract: {result}")

    result = dbutils.notebook.run("./etl/transform", 3600, {"date": process_date})
    print(f"Transform: {result}")

    result = dbutils.notebook.run("./etl/load", 3600, {"date": process_date})
    print(f"Load: {result}")

except Exception as e:
    print(f"Pipeline failed: {e}")
    dbutils.notebook.exit(f"FAILED: {e}")

dbutils.notebook.exit("SUCCESS")
```

Common Issues & Errors

Widget Not Found Error **Scenario:** Accessing widget that doesn't exist.

```
# Error: InputWidgetNotDefined: No widget named 'missing_widget' defined

value = dbutils.widgets.get("missing_widget")
```

Fix: Create widget first or use try-except:

```python
try:
    value = dbutils.widgets.get("missing_widget")
except Exception:
    value = "default_value"
```

%run Path Not Found **Scenario:** Incorrect relative path in %run.

```
# Error: Notebook not found: ./utils/helpers

%run ./utils/helpers
```

Fix: Verify path is relative to current notebook location or use absolute path:

```
%run /Workspace/Users/user/project/utils/helpers
```

dbutils.notebook.run Timeout **Scenario:** Child notebook exceeds timeout.

```
# Error: Run timed out after 3600 seconds

result = dbutils.notebook.run("/path/to/slow_notebook", timeout_seconds=3600)
```

Fix: Increase timeout or optimize notebook:

```
result = dbutils.notebook.run("/path/to/notebook", timeout_seconds=7200)
```

Secret Scope Access Denied **Scenario:** User doesn't have permission to secret scope.

```
# Error: User does not have READ permission on secret scope

secret = dbutils.secrets.get("restricted-scope", "key")
```

Fix: Request access to secret scope from admin.

Task Value Not Found **Scenario:** Getting task value that wasn't set or from wrong task.

```
# Returns default if task/key not found

value = dbutils.jobs.taskValues.get(
    taskKey="wrong_task_name",
    key="count",
    default=-1  # Always provide a default
)
```

Exam Tips

1. **%run vs dbutils.notebook.run** - Know the differences: shared context vs isolated, return values, timeouts
2. **Widget types** - text (free-form), dropdown (fixed), combobox (editable), multiselect (multiple)
3. **Widget in jobs** - Parameters in job config override widget defaults
4. **Secret scopes** - Databricks-backed vs Key Vault backed, secrets always redacted in output
5. **Task values** - Must specify task name when getting values, only works in job context
6. **Magic commands** - `%run` executes in same context, `%fs` for quick file ops
7. **Workspace paths** - `/Workspace/` for code, `dbfs:/` for data, `/Volumes/` for Unity Catalog
8. **Notebook permissions** - Can Read, Can Run, Can Edit, Can Manage hierarchy
9. **%pip vs dbutils.library** - Prefer `%pip` for package installation
10. **Parallel notebooks** - Use ThreadPoolExecutor with dbutils.notebook.run

Key Takeaways

- **`%run` vs `dbutils.notebook.run`**: `%run` executes in the same context (variables shared, no return value, no timeout); `dbutils.notebook.run` executes in an isolated context with a configurable timeout and a string return value
- **Widget types**: `text` (free-form), `dropdown` (fixed list, single select), `combobox` (editable dropdown), `multiselect` (comma-separated result); in job runs, job parameters override widget defaults
- **`dbutils.jobs.taskValues`**: set values in upstream tasks with `taskValues.set(key, value)` and retrieve them in downstream tasks with `taskValues.get(taskKey="task_name", key="key", default=...)`; only works in job context, not interactive
- **Secret scopes**: Databricks-backed (managed internally) vs Azure Key Vault-backed (enterprise); secret values are always redacted in notebook output; `dbutils.secrets.list()` returns key names only, never values
- **Workspace paths**: `/Workspace/...` for notebooks and code files; `dbfs:/...` (Spark) or `/dbfs/...` (Python) for data files; `/Volumes/catalog/schema/volume/` for Unity Catalog governed files
- **Magic commands**: `%sql` / `%python` / `%scala` for language switching; `%fs` for DBFS operations; `%pip` for package installation (preferred over `dbutils.library`); `%sh` for shell commands
- **Parallel notebook execution**: use `ThreadPoolExecutor` with `dbutils.notebook.run` to run independent child notebooks concurrently, then aggregate results
- **Notebook permissions hierarchy**: No Permission < Can Read < Can Run < Can Edit < Can Manage

Related Topics

- Databricks CLI — Part 1 - Command line management
- REST API - Programmatic workspace access

- Secret Management - Secure credential storage

Official Documentation

- Databricks Notebooks
- dbutils Reference
- Notebook Widgets
- Notebook Workflows

Unit Testing — Part 1

Robust testing practices are essential for reliable data pipelines. This part covers unit testing strategies, pytest fundamentals, writing unit tests, PySpark's built-in `assertDataFrameEqual` and `assertSchemaEqual` utilities, structuring code with `DataFrame.transform`, chispa comparisons, mocking, and the Nutter framework. Part 2 covers testing patterns, integration tests, unit testing Lakeflow Pipelines (Beta), CI/CD integration, and exam tips.

Overview

```
flowchart TB
    subgraph TestTypes["Testing Pyramid"]
        Unit[Unit Tests<br>Fast, Isolated]
        Integration[Integration Tests<br>Component Interaction]
        E2E[End-to-End Tests<br>Full Pipeline]
    end

    subgraph Tools["Testing Tools"]
        Pytest[pytest]
        Nutter[Nutter Framework]
        Great[Great Expectations]
    end

    Unit --> Pytest
    Integration --> Nutter
    E2E --> Great
```

Overview

- Unit tests check self-contained functions early and often; integration tests check how components work together against real compute and tables.
- PySpark ships `assertDataFrameEqual` and `assertSchemaEqual` in `pyspark.testing` (Spark 3.5+) for DataFrame and schema comparisons in any test framework.
- Writing each transformation as a `DataFrame -> DataFrame` function and chaining them with `DataFrame.transform` keeps logic testable without reading or writing tables.
- `pytest` fixtures (for example a session-scoped `spark` fixture in `conftest.py`) provide shared setup and teardown.

What the Exam Tests

- ”Develop unit and integration tests using assertDataFrameEqual, assertSchemaEqual, DataFrame.transform, and testing frameworks to ensure code correctness.”
- Choosing the right assertion and options (row order, float tolerance, schema-only checks) for a test.

- Structuring pipeline code so it can be unit-tested, and knowing where integration tests run.

Testing Pyramid for Data Engineering

Level	Scope	Speed	Tools	Example
Unit	Functions, transformations	Fast	pytest, unittest	Test a cleaning
Integration	Notebook workflows	Medium	Nutter, Databricks jobs	Test bronze-to-
End-to-End	Full pipelines	Slow	Databricks workflows	Test complete
Data Quality	Data validation	Varies	Great Expectations, Lakeflow Pipelines	Test data const

pytest Fundamentals

Project Setup

```
my-project/
├── src/
│   └── transformations/
│       ├── __init__.py
│       ├── cleaning.py
│       └── aggregations.py
├── tests/
│   ├── __init__.py
│   ├── conftest.py
│   └── unit/
│       ├── __init__.py
│       ├── test_cleaning.py
│       └── test_aggregations.py
├── pytest.ini
└── requirements-dev.txt
```

pytest Configuration

```
# pytest.ini

[pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
addopts = -v --tb=short
markers =
    unit: Unit tests
    integration: Integration tests
    slow: Slow running tests
```

requirements-dev.txt

```
pytest>=7.0.0
pytest-cov>=4.0.0
pytest-mock>=3.10.0
pyspark>=3.5.0   # pyspark.testing.assertDataFrameEqual / assertSchemaEqual
chispa>=0.9.0
```

Writing Unit Tests

Basic Test Structure

```
# tests/unit/test_cleaning.py

import pytest
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
from src.transformations.cleaning import remove_nulls, standardize_names

class TestCleaning:
    """Tests for cleaning transformations."""

    @pytest.fixture(scope="class")
    def spark(self):
        """Create SparkSession for tests."""
        return (SparkSession.builder
            .master("local[*]")
            .appName("unit-tests")
            .getOrCreate())

    def test_remove_nulls_removes_null_rows(self, spark):
        """Test that null rows are removed."""
        # Arrange
        schema = StructType([
            StructField("id", IntegerType(), True),
            StructField("name", StringType(), True)
        ])
        data = [(1, "Alice"), (2, None), (3, "Charlie")]
        df = spark.createDataFrame(data, schema)

        # Act
        result = remove_nulls(df, "name")

        # Assert
        assert result.count() == 2
        assert result.filter("name IS NULL").count() == 0

    def test_standardize_names_converts_to_lowercase(self, spark):
        """Test name standardization to lowercase."""
        # Arrange
        data = [(1, "ALICE"), (2, "BOB")]
        df = spark.createDataFrame(data, ["id", "name"])

        # Act
        result = standardize_names(df, "name")

        # Assert
        names = [row.name for row in result.collect()]
        assert names == ["alice", "bob"]
```

Source Code Being Tested

```
# src/transformations/cleaning.py
```

```python
from pyspark.sql import DataFrame
from pyspark.sql.functions import lower, trim

def remove_nulls(df: DataFrame, column: str) -> DataFrame:
    """Remove rows where specified column is null."""
    return df.filter(f"{column} IS NOT NULL")

def standardize_names(df: DataFrame, column: str) -> DataFrame:
    """Standardize names to lowercase and trimmed."""
    return df.withColumn(column, lower(trim(df[column])))

def deduplicate(df: DataFrame, key_columns: list) -> DataFrame:
    """Remove duplicate rows based on key columns."""
    return df.dropDuplicates(key_columns)
```

PySpark Built-in Test Utilities

What the exam names

The exam guide names `assertDataFrameEqual`, `assertSchemaEqual`, `DataFrame.transform`, and "testing frameworks". The two assertions ship with PySpark in `pyspark.testing` (new in Spark 3.5.0), work with Spark Connect and Databricks Connect, and plug into any framework such as `pytest` or `unittest`. chispa and Nutter, covered later on this page, are third-party alternatives you may meet in older codebases.

Write transformations as composable functions Code is testable when each step is a function that takes a DataFrame (plus parameters) and returns a DataFrame. `DataFrame.transform(func, *args, **kwargs)` chains those functions without nesting calls; passing extra positional and keyword arguments needs Spark 3.3+.

```python
# src/my_project/transforms.py
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def clean_names(df: DataFrame, column: str = "name") -> DataFrame:
    """Collapse repeated whitespace and trim."""
    return df.withColumn(column, F.trim(F.regexp_replace(F.col(column), r"\s+", "
    ↪  ")))


def keep_valid(df: DataFrame) -> DataFrame:
    return df.filter(F.col("amount") > 0)


def add_net_amount(df: DataFrame, tax_rate: float) -> DataFrame:
    return df.withColumn("net_amount", F.round(F.col("amount") * (1 - tax_rate),
    ↪  2))


def build_silver(df: DataFrame, tax_rate: float) -> DataFrame:
    """Compose the steps; each one is also unit-tested on its own."""
    return (df
        .transform(clean_names)
```

```
        .transform(keep_valid)
        .transform(add_net_amount, tax_rate=tax_rate))
```

Because none of these functions reads a table or writes output, a test can build a tiny input DataFrame, call one function, and compare the result. Reading and writing stay in a thin entry point (`main.py`, a notebook, or a pipeline file) that is covered by integration tests instead.

Share one SparkSession with a pytest fixture

```
# tests/conftest.py
import pytest
from pyspark.sql import SparkSession


@pytest.fixture(scope="session")
def spark():
    spark = (SparkSession.builder
        .master("local[*]")
        .appName("unit-tests")
        .config("spark.sql.shuffle.partitions", "1")
        .getOrCreate())
    yield spark
    spark.stop()
```

`scope="session"` creates the session once for the whole test run; `yield` hands it to every test that declares a `spark` argument, and the code after `yield` tears it down.

assertDataFrameEqual

```
# tests/test_transforms.py
from pyspark.sql import Row
from pyspark.testing import assertDataFrameEqual

from my_project.transforms import add_net_amount, build_silver, clean_names


def test_clean_names_collapses_spaces(spark):
    df = spark.createDataFrame([("John    D.",), ("  Alice G. ",)], ["name"])
    expected = spark.createDataFrame([("John D.",), ("Alice G.",)], ["name"])
    assertDataFrameEqual(clean_names(df), expected)


def test_add_net_amount_within_tolerance(spark):
    df = spark.createDataFrame([(1, 100.0)], ["id", "amount"])
    expected = [Row(id=1, amount=100.0, net_amount=80.0)]   # list of Rows also
 ↪  works
    assertDataFrameEqual(add_net_amount(df, tax_rate=0.2), expected, rtol=1e-5)


def test_build_silver_drops_invalid_rows(spark):
    df = spark.createDataFrame([(" Bob ", 10.0), ("Eve", -5.0)], ["name",
 ↪  "amount"])
    expected = spark.createDataFrame([("Bob", 10.0, 9.0)], ["name", "amount",
 ↪  "net_amount"])
    assertDataFrameEqual(build_silver(df, tax_rate=0.1), expected)
```

Parameter	Default	Meaning
`actual`, `expected`	—	Spark, Spark Connect, pandas, or panda
`checkRowOrder`	`False`	Row order is ignored unless True; DataF
`rtol`, `atol`	`1e-5`, `1e-8`	Float/decimal values pass when `abs(a`
`ignoreNullable`	`True`	Nullable flags are ignored in the schema
`ignoreColumnOrder`	`False`	Compare columns by name instead of p
`ignoreColumnName` / `ignoreColumnType`	`False`	Skip name or type checks in the schema
`maxErrors`, `showOnlyDiff`, `includeDiffRows`	`None`, `False`, `False`	Limit and shape the failure report (Spark

- The schema is checked only when `expected` is a DataFrame, not a list of Rows.
- On failure it raises `PySparkAssertionError` with `[DIFFERENT_ROWS]` (and the percentage of mismatched rows plus a diff) or `[DIFFERENT_SCHEMA]`.
- `ignoreColumnOrder` and `ignoreColumnName` can't both be `True`.

`assertSchemaEqual` `assertSchemaEqual` compares two `StructType` schemas, so pass `df.schema`, not the DataFrame. Use it to pin an output contract without comparing data.

```
from pyspark.sql.types import DoubleType, StringType, StructField, StructType
from pyspark.testing import assertSchemaEqual


def test_build_silver_schema(spark):
    df = spark.createDataFrame([("Bob", 10.0)], ["name", "amount"])
    expected = StructType([
        StructField("name", StringType(), True),
        StructField("amount", DoubleType(), True),
        StructField("net_amount", DoubleType(), True),
    ])
    assertSchemaEqual(build_silver(df, tax_rate=0.1).schema, expected)
```

In Spark 4.0+, `assertSchemaEqual` also accepts `ignoreNullable` (default `True`), `ignoreColumnOrder`, and `ignoreColumnName`. A failure raises `PySparkAssertionError: [DIFFERENT_SCHEMA]` with a diff of both schemas.

Choosing an assertion

Need	Use
Compare data and schema of two DataFrames	`assertDataFrameEqual(actual, expected)`
Compare only the output schema (contract test)	`assertSchemaEqual(actual.schema, expected_sche`
Floating-point results	`assertDataFrameEqual(..., rtol=..., atol=...)`
Result was explicitly sorted and order matters	`assertDataFrameEqual(..., checkRowOrder=True)`
Quick expected values without building a DataFrame	`expected=[Row(...), Row(...)]` (no schema check)
Count or a single value	Plain `assert df.count() == n`

Testing with chispa (Third-Party)

Note

chispa is an open-source library, not part of PySpark; `assertDataFrameEqual` and `assertSchemaEqual` cover the same checks with built-in utilities.

DataFrame Comparison

```python
# tests/unit/test_aggregations.py

import pytest
from chispa import assert_df_equality
from pyspark.sql import SparkSession
from src.transformations.aggregations import calculate_totals

class TestAggregations:

    @pytest.fixture(scope="class")
    def spark(self):
        return (SparkSession.builder
            .master("local[*]")
            .appName("unit-tests")
            .getOrCreate())

    def test_calculate_totals_sums_by_category(self, spark):
        """Test aggregation calculates correct totals."""
        # Arrange
        data = [
            ("electronics", 100),
            ("electronics", 200),
            ("clothing", 50)
        ]
        df = spark.createDataFrame(data, ["category", "amount"])

        expected_data = [
            ("electronics", 300),
            ("clothing", 50)
        ]
        expected = spark.createDataFrame(expected_data, ["category", "total"])

        # Act
        result = calculate_totals(df, "category", "amount")

        # Assert
        assert_df_equality(result, expected, ignore_row_order=True)

    def test_calculate_totals_handles_empty_df(self, spark):
        """Test aggregation handles empty DataFrame."""
        # Arrange
        schema = "category string, amount int"
        df = spark.createDataFrame([], schema)

        # Act
        result = calculate_totals(df, "category", "amount")

        # Assert
        assert result.count() == 0
```

Schema Comparison

```python
from chispa import assert_schema_equality
```

```
def test_output_schema_matches_expected(self, spark):
    """Verify output schema matches expected."""
    # Arrange
    expected_schema = StructType([
        StructField("category", StringType(), True),
        StructField("total", LongType(), True)
    ])

    data = [("electronics", 100)]
    df = spark.createDataFrame(data, ["category", "amount"])

    # Act
    result = calculate_totals(df, "category", "amount")

    # Assert
    assert_schema_equality(result.schema, expected_schema)
```

Mocking Spark and dbutils

Mocking SparkSession

```
# tests/conftest.py

import pytest
from unittest.mock import MagicMock, patch
from pyspark.sql import SparkSession

@pytest.fixture(scope="session")
def spark():
    """Shared SparkSession for all tests."""
    spark = (SparkSession.builder
        .master("local[*]")
        .appName("test")
        .config("spark.sql.shuffle.partitions", "1")
        .config("spark.default.parallelism", "1")
        .getOrCreate())
    yield spark
    spark.stop()

@pytest.fixture
def mock_spark():
    """Mock SparkSession for unit tests not needing real Spark."""
    mock = MagicMock(spec=SparkSession)
    return mock
```

Mocking dbutils

```
# tests/conftest.py

import pytest
from unittest.mock import MagicMock

@pytest.fixture
def mock_dbutils():
    """Mock dbutils for testing outside Databricks."""
    dbutils = MagicMock()
```

```
    # Mock widgets
    dbutils.widgets.get.return_value = "default_value"

    # Mock secrets
    dbutils.secrets.get.return_value = "mock_secret"

    # Mock fs
    dbutils.fs.ls.return_value = [
        MagicMock(path="/mnt/data/file1.parquet", name="file1.parquet",
↪   size=1000),
       MagicMock(path="/mnt/data/file2.parquet", name="file2.parquet", size=2000)
    ]

    return dbutils

# Using the mock

def test_function_using_dbutils(mock_dbutils):
    """Test function that uses dbutils."""
    with patch('src.my_module.dbutils', mock_dbutils):
        result = my_function_using_dbutils()
        assert result is not None
```

Mocking External Services

```
# tests/unit/test_api_calls.py

import pytest
from unittest.mock import patch, MagicMock
from src.connectors.api_client import fetch_data

class TestAPIClient:

    @patch('src.connectors.api_client.requests.get')
    def test_fetch_data_returns_records(self, mock_get):
        """Test API fetch returns expected data."""
        # Arrange
        mock_response = MagicMock()
        mock_response.status_code = 200
        mock_response.json.return_value = {
            "records": [{"id": 1, "value": "test"}]
        }
        mock_get.return_value = mock_response

        # Act
        result = fetch_data("https://api.example.com/data")

        # Assert
        assert len(result) == 1
        assert result[0]["id"] == 1

    @patch('src.connectors.api_client.requests.get')
    def test_fetch_data_handles_error(self, mock_get):
        """Test API fetch handles errors gracefully."""
        # Arrange
```

```
        mock_get.side_effect = Exception("Connection failed")

        # Act & Assert
        with pytest.raises(Exception) as exc_info:
            fetch_data("https://api.example.com/data")
        assert "Connection failed" in str(exc_info.value)
```

Nutter Framework

Nutter Overview

> **Note**
>
> Nutter is an open-source project (microsoft/nutter), not a Databricks product, and isn't named in the exam guide. For the exam, focus on pytest tests that use PySpark's built-in assertions.

```
Nutter is a Databricks-native testing framework:
- Runs tests inside Databricks notebooks
- Supports test fixtures (before/after)
- Integrates with CI/CD pipelines
- Generates JUnit-compatible reports
```

Installing Nutter

```
# Install via pip

pip install nutter

# Or add to cluster library
# Libraries → Install New → PyPI: nutter
```

Nutter Test Notebook

```
# Databricks notebook source
# MAGIC %pip install nutter

# COMMAND ----------

from runtime.nutterfixture import NutterFixture, tag

class TestBronzeIngestion(NutterFixture):
    """Tests for bronze layer ingestion."""

    def __init__(self):
        self.test_table = "test_catalog.test_schema.bronze_events_test"
        NutterFixture.__init__(self)

    def before_all(self):
        """Setup before all tests."""
        # Create test schema
        spark.sql("CREATE SCHEMA IF NOT EXISTS test_catalog.test_schema")

    def after_all(self):
        """Cleanup after all tests."""
        spark.sql(f"DROP TABLE IF EXISTS {self.test_table}")
```

```
        spark.sql("DROP SCHEMA IF EXISTS test_catalog.test_schema")

    def before_test_ingest_creates_table(self):
        """Setup for specific test."""
        # Prepare test data
        self.test_data = [
            (1, "event_a", "2024-01-01"),
            (2, "event_b", "2024-01-02")
        ]

    def run_test_ingest_creates_table(self):
        """Run the ingestion process."""
        df = spark.createDataFrame(
            self.test_data,
            ["event_id", "event_type", "event_date"]
        )
        df.write.format("delta").mode("overwrite").saveAsTable(self.test_table)

    def assertion_test_ingest_creates_table(self):
        """Verify the ingestion results."""
        count = spark.table(self.test_table).count()
        assert count == 2, f"Expected 2 rows, got {count}"

    @tag("schema")
    def run_test_schema_is_correct(self):
        """Test that schema matches expected."""
        df = spark.table(self.test_table)
        self.actual_columns = df.columns

    def assertion_test_schema_is_correct(self):
        """Verify schema columns."""
        expected = ["event_id", "event_type", "event_date"]
        assert self.actual_columns == expected

# COMMAND ----------

# Run tests

result = TestBronzeIngestion().execute_tests()
print(result.to_string())

# COMMAND ----------

# For CI/CD, check if tests passed

is_job = dbu-
 ↪  tils.notebook.entry_point.getDbutils().notebook().getContext().currentRunId().isDefined(
if is_job:
    result.exit(dbutils)
```

Running Nutter from CLI

```
# Run all tests in a folder

nutter run /Workspace/Users/user@company.com/project/tests/ \
    --cluster_id abc-123-def \
```

```
    --recursive

# Run specific test notebook

nutter run /Workspace/Users/user@company.com/project/tests/test_bronze \
    --cluster_id abc-123-def

# Run with tags

nutter run /Workspace/Users/user@company.com/project/tests/ \
    --cluster_id abc-123-def \
    --tag_filter schema

# Generate JUnit report

nutter run /Workspace/Users/user@company.com/project/tests/ \
    --cluster_id abc-123-def \
    --junit_report results.xml
```

Continue reading: Part 2 — Testing Patterns, CI/CD Integration, Best Practices & Exam Tips

---

Unit Testing — Part 2

Overview

This part covers testing patterns for Delta Lake and streaming, integration tests, unit testing Lakeflow Pipelines (Beta), CI/CD integration, best practices, common issues, and exam tips for unit testing in Databricks.

For pytest fundamentals, `assertDataFrameEqual` / `assertSchemaEqual`, `DataFrame.transform`, chispa comparisons, mocking, and the Nutter framework, see Part 1.

Overview

- **Testing patterns** for Delta Lake and streaming; run a test stream to completion with `processAllAvailable()` then `stop()`
- **Unit vs integration tests**: pure functions composed with `DataFrame.transform` are unit-tested; entry points are integration-tested on real compute
- **Pipeline unit testing (Beta)** runs in the Lakeflow Pipelines Editor with a `test_spark` session that redirects table operations to a temporary schema
- **CI integration**: pytest with `--junitxml` reports; a session-scoped SparkSession fixture keeps suites fast

What the Exam Tests

- Develop unit and integration tests using assertDataFrameEqual, assertSchemaEqual, DataFrame.transform, and testing frameworks to ensure code correctness.
- Separate unit tests from integration tests and know what each one needs
- Keep tests isolated and fast in CI (fixture scopes, low shuffle partitions)

Testing Patterns

Testing Delta Lake Operations

```python
# tests/unit/test_delta_operations.py

import pytest
from delta import DeltaTable
from pyspark.sql import SparkSession
import tempfile
import shutil

class TestDeltaOperations:

    @pytest.fixture
    def delta_path(self):
        """Create temp directory for Delta table."""
        path = tempfile.mkdtemp()
        yield path
        shutil.rmtree(path)

    @pytest.fixture
    def spark(self):
        return (SparkSession.builder
            .master("local[*]")
            .config("spark.sql.extensions",
↪   "io.delta.sql.DeltaSparkSessionExtension")
            .config("spark.sql.catalog.spark_catalog",
↪   "org.apache.spark.sql.delta.catalog.DeltaCatalog")
            .getOrCreate())

    def test_merge_upserts_correctly(self, spark, delta_path):
        """Test MERGE operation updates and inserts."""
        # Arrange - Create target table
        target_data = [(1, "old_value"), (2, "keep")]
        target_df = spark.createDataFrame(target_data, ["id", "value"])
        target_df.write.format("delta").save(delta_path)

        # Source data with update and insert
        source_data = [(1, "new_value"), (3, "insert")]
        source_df = spark.createDataFrame(source_data, ["id", "value"])

        # Act - Perform merge
        delta_table = DeltaTable.forPath(spark, delta_path)
        (delta_table.alias("target")
            .merge(source_df.alias("source"), "target.id = source.id")
            .whenMatchedUpdateAll()
            .whenNotMatchedInsertAll()
            .execute())

        # Assert
        result = spark.read.format("delta").load(delta_path)
        assert result.count() == 3

        updated_row = result.filter("id = 1").collect()[0]
        assert updated_row.value == "new_value"
```

Testing Streaming

```python
# tests/unit/test_streaming.py

import pytest
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, TimestampType
import tempfile
import time

class TestStreaming:

    @pytest.fixture
    def spark(self):
        return (SparkSession.builder
            .master("local[*]")
            .getOrCreate())

    def test_streaming_aggregation(self, spark, tmp_path):
        """Test streaming aggregation logic."""
        # Arrange
        input_path = str(tmp_path / "input")
        output_path = str(tmp_path / "output")
        checkpoint_path = str(tmp_path / "checkpoint")

        # Create test data
        data = [
            ("user1", "2024-01-01 00:00:00"),
            ("user1", "2024-01-01 00:01:00"),
            ("user2", "2024-01-01 00:00:00")
        ]
        df = spark.createDataFrame(data, ["user_id", "timestamp"])
        df.write.format("json").save(input_path)

        # Act - Run streaming query in batch mode for testing
        stream_df = (spark.readStream.format("json")
            .schema(df.schema)
            .load(input_path))

        agg_df = stream_df.groupBy("user_id").count()

        query = (agg_df.writeStream
            .format("delta")
            .outputMode("complete")
            .option("checkpointLocation", checkpoint_path)
            .start(output_path))

        query.processAllAvailable()
        query.stop()

        # Assert
        result = spark.read.format("delta").load(output_path)
        user1_count = result.filter("user_id = 'user1'").collect()[0]["count"]
        assert user1_count == 2
```

Testing with Fixtures

```python
# tests/conftest.py

import pytest
from pyspark.sql import SparkSession

@pytest.fixture(scope="session")
def spark():
    """Session-scoped SparkSession."""
    spark = (SparkSession.builder
        .master("local[*]")
        .appName("test")
        .getOrCreate())
    yield spark
    spark.stop()

@pytest.fixture
def sample_orders(spark):
    """Sample orders DataFrame for testing."""
    data = [
        (1, "2024-01-01", 100.00, "A"),
        (2, "2024-01-02", 200.00, "B"),
        (3, "2024-01-02", 150.00, "A"),
    ]
    return spark.createDataFrame(
        data,
        ["order_id", "order_date", "amount", "category"]
    )

@pytest.fixture
def sample_customers(spark):
    """Sample customers DataFrame for testing."""
    data = [
        (1, "Alice", "alice@email.com"),
        (2, "Bob", "bob@email.com"),
    ]
    return spark.createDataFrame(
        data,
        ["customer_id", "name", "email"]
    )
```

Integration Tests

Unit tests run pure transformations against a local SparkSession. Integration tests check that the pieces work together against Databricks compute, Unity Catalog tables, and workspace files. Run them against non-production data: create fake data as close as possible to production, especially for functions that add, remove, or change data.

Approach	How it runs
`pytest` + Databricks Connect	Local `pytest` process; Spark code executes on a Databricks cluster or serverle
`pytest` inside the workspace	A notebook or job task calls `pytest.main()` on test files stored next to the co
Workspace **Tests** pane	Discovers `test_*.py` / `*_test.py` files and `test`-prefixed functions; run with i
Bundle-deployed test job	`databricks bundle deploy -t staging` then `databricks bundle run <te`

```python
# tests/conftest.py for integration tests with Databricks Connect
import pytest
```

```
from databricks.connect import DatabricksSession


@pytest.fixture(scope="session")
def spark():
    # Uses the DEFAULT configuration profile when run from a terminal
    return DatabricksSession.builder.getOrCreate()
```

- Databricks Connect and PySpark are mutually exclusive in one Python environment; install only `databricks-connect` for these tests.
- From a terminal, `pytest` with Databricks Connect uses only the `DEFAULT` configuration profile, which must name the cluster or serverless compute to use.

Running `pytest` from a notebook or job task in the workspace:

```
import sys

import pytest

# Skip writing .pyc files on a read-only file system.
sys.dont_write_bytecode = True

retcode = pytest.main([".", "-v", "-p", "no:cacheprovider"])

# Fail the cell (and the job task) if any test fails.
assert retcode == 0, "The pytest invocation failed. See the log for details."
```

Unit Testing Lakeflow Pipelines (Beta)

Platform update (Oct 2026)

Unit testing for pipelines is in **Beta** (released July 26, 2026). You write Python tests in the Lakeflow Pipelines Editor and run Python or SQL pipeline logic against mock data, including AUTO CDC flows, streaming tables, expectations, and append flows. Source: July 2026 release notes

The framework gives tests a SparkSession (`test_spark`) that redirects table reads and writes **by name** to a temporary schema (`redirecting_<id>`) in the pipeline's default catalog, then drops it when the run ends.

```
import pytest
from pyspark.pipelines.testing import TestPipeline, test_spark
from pyspark.testing import assertDataFrameEqual

test_pipeline = TestPipeline.active()


def mock_users(session):
    session.sql("""
        CREATE TABLE catalog.schema.wanderbricks_users AS
        SELECT * FROM VALUES
            (1, 'alice@example.com', 'Alice', 'admin'),
            (2, NULL, 'Bob', 'user')
        AS t(user_id, email, name, user_type)
    """)
```

```python
def test_counts(test_spark):
    mock_users(test_spark)
    status = test_pipeline.run(test_spark, set(["catalog.schema.users",
↪   "catalog.schema.counts"]))
    assert status.is_success
    expected = test_spark.createDataFrame(
        [("admin", 1, 1), ("user", 1, 0)],
        schema=["user_type", "total_count", "count_valid_emails"],
    )
    assertDataFrameEqual(test_spark.table("catalog.schema.counts"), expected)
```

API	Purpose
`TestPipeline.active()`	The pipeline currently open in the edito
`test_pipeline.run(test_spark, set([...]))`	Synchronous selective refresh of the na
`status.is_success`, `status.error_message`, `status.error_class`	Outcome of the update
`status.event_log` / `test_pipeline.event_log`	Event log for this run / all runs in the te
`status.flow_info`	Per-flow `records_written`, `duration`,

Requirements and limits to remember:

- You must be the pipeline **owner** (`CAN MANAGE` isn't enough) and have `USE CATALOG` and `CREATE SCHEMA` on the default catalog.
- The pipeline must be **triggered** (not continuous) and run on DBR 18.1+. Spark Connect isn't supported. Tests run from the Lakeflow Pipelines Editor only, must be written in Python, and are billed as pipeline updates.
- Isolation covers name-based table operations only. Reading or writing by path, Auto Loader, Kafka, governance statements (`GRANT`, tags, policies), and `CREATE CATALOG` / `CREATE SCHEMA` reach real production systems.
- Only selective refresh is available (no full refresh), and you shouldn't run a test while the pipeline runs an update.

CI/CD Integration

GitHub Actions Test Workflow

```yaml
# .github/workflows/test.yml

name: Run Tests

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.10'
```

```yaml
      - name: Install dependencies
        run: |
          pip install -r requirements-dev.txt
          pip install pytest pytest-cov

      - name: Run unit tests
        run: |
          pytest tests/unit/ \
            --cov=src \
            --cov-report=xml \
            --cov-report=html \
            --junitxml=test-results.xml \
            -v

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          files: coverage.xml

      - name: Upload test results
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: test-results
          path: |
            test-results.xml
            htmlcov/

  nutter-tests:
    runs-on: ubuntu-latest
    needs: unit-tests
    steps:
      - uses: actions/checkout@v4

      - uses: databricks/setup-cli@main

      - name: Run Nutter tests
        run: |
          nutter run /Workspace/Shared/project/tests/ \
            --cluster_id ${{ secrets.TEST_CLUSTER_ID }} \
            --recursive \
            --junit_report nutter-results.xml
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}

      - name: Upload Nutter results
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: nutter-results
          path: nutter-results.xml
```

Test Environments

```yaml
# databricks.yml - Test environment target

targets:
  test:
    mode: development
    variables:
      catalog: test_catalog
      schema: test_${workspace.current_user.userName}
    resources:
      jobs:
        test_runner:
          name: "Test Runner"
          tasks:
            - task_key: run_tests
              notebook_task:
                notebook_path: ../tests/run_all_tests
              new_cluster:
                spark_version: "14.3.x-scala2.12"
                node_type_id: "Standard_DS3_v2"
                num_workers: 0
```

Use Cases

- **Ensuring Transformation Correctness**: Writing PyTest tests with Chispa to verify that a complex custom PySpark aggregation function outputs the exact expected DataFrame before the code allows for merging.
- **Testing composed transformations**: Each silver-layer step is a `DataFrame -> DataFrame` function chained with `DataFrame.transform`; `pytest` tests compare each step's output with `assertDataFrameEqual` and pin the final contract with `assertSchemaEqual`.
- **Validating Delta Lake Operations**: Using an isolated local temporary directory to test that a PySpark `MERGE` statement correctly applies updates and inserts without accidentally mutating real development tables.

Common Issues & Errors

SparkSession Not Available **Scenario:** Tests fail because SparkSession is None.

Fix: Use proper fixture scope:

```python
@pytest.fixture(scope="session")  # Not "function"
def spark():
    return SparkSession.builder.master("local[*]").getOrCreate()
```

Tests Interfere with Each Other **Scenario:** Tests pass individually but fail together.

Fix: Ensure test isolation:

```python
@pytest.fixture
def clean_table(spark):
    """Drop and recreate test table."""
    spark.sql("DROP TABLE IF EXISTS test_table")
    yield
    spark.sql("DROP TABLE IF EXISTS test_table")
```

Slow Test Execution **Scenario:** Tests take too long.

Fix: Optimize Spark configuration:

```
spark = (SparkSession.builder
    .master("local[*]")
    .config("spark.sql.shuffle.partitions", "1")
    .config("spark.default.parallelism", "1")
    .config("spark.executor.memory", "1g")
    .getOrCreate())
```

Delta Lake Not Available **Scenario:** Delta operations fail in tests.

Fix: Include Delta dependencies:

```
spark = (SparkSession.builder
    .config("spark.jars.packages", "io.delta:delta-core_2.12:2.4.0")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .getOrCreate())
```

Best Practices

Test Organization

```
tests/
├── conftest.py              # Shared fixtures
├── unit/                    # Unit tests (no external deps)
│   ├── test_transformations.py
│   ├── test_validations.py
│   └── test_utils.py
├── integration/             # Integration tests (real Spark)
│   ├── test_bronze_layer.py
│   ├── test_silver_layer.py
│   └── test_gold_layer.py
└── e2e/                     # End-to-end tests
    └── test_full_pipeline.py
```

Naming Conventions

```
# Good test names - descriptive and specific

def test_remove_nulls_removes_rows_with_null_values():
    pass

def test_calculate_totals_returns_zero_for_empty_input():
    pass

def test_merge_updates_existing_records_when_key_matches():
    pass

# Bad test names - vague

def test_function():
    pass

def test_it_works():
    pass
```

Test Isolation

```
# Each test should be independent

class TestTransformations:

    def test_transformation_a(self, spark):
        """Test A - creates its own data."""
        df = spark.createDataFrame([(1,)], ["id"])
        # ... test logic

    def test_transformation_b(self, spark):
        """Test B - doesn't depend on Test A."""
        df = spark.createDataFrame([(2,)], ["id"])
        # ... test logic
```

Exam Tips

Tip

- `pyspark.testing.assertDataFrameEqual(actual, expected)` compares schema and data; row order is ignored unless `checkRowOrder=True`; floats use `rtol` (1e-5) / `atol` (1e-8).
- `assertSchemaEqual` takes **schemas** (`df.schema`), not DataFrames.
- Write steps as `DataFrame -> DataFrame` functions and chain them with `DataFrame.transform(func, *args, **kwargs)` so each is unit-testable.
- Use a `scope="session"` spark fixture in `conftest.py`; integration tests can use `DatabricksSession` (never install PySpark next to Databricks Connect).
- Pipeline unit testing (Beta): `TestPipeline.active()`, `test_pipeline.run(test_spark, ...)`, owner permission, triggered mode, DBR 18.1+, isolation by table name only.
- Nutter and chispa are third-party; the exam names PySpark's built-in utilities.
- Test isolation, JUnit reports (`--junitxml`), and separate test catalogs/schemas still apply.

Key Takeaways

- **Built-in assertions**: `assertDataFrameEqual` (data + schema, tolerances, optional row order) and `assertSchemaEqual` (schemas only) are in `pyspark.testing` since Spark 3.5 and work in any framework.
- **Testable structure**: pure transformation functions composed with `DataFrame.transform` keep I/O out of unit tests; integration tests cover the entry points against real compute.
- **Pipeline unit testing (Beta)**: tests run in the Lakeflow Pipelines Editor with a `test_spark` session that redirects name-based table operations to a temporary schema.
- **Fixture scopes matter**: Use `scope="session"` for the SparkSession fixture to create it once for all tests; function-scoped fixtures recreate it per test and dramatically slow down the test suite.
- **Test isolation**: Each test must create its own data and clean up after itself — tests that depend on shared state or a specific execution order are fragile and unreliable in CI.
- **Delta in local tests**: To test Delta Lake operations locally, add `io.delta:delta-core_2.12:<version>` to `spark.jars.packages` and enable the Delta extension in the SparkSession configuration.
- **Chispa for DataFrame equality**: Use `chispa.assert_df_equality()` to compare two DataFrames with proper null and order handling, rather than converting to Python lists for comparison.
- **Nutter pattern**: Nutter notebook tests follow a `before` / `run_<name>` / `assertion_<name>` naming convention and are discovered automatically by the Nutter runner.

- **Streaming test trick**: Use `query.processAllAvailable()` followed by `query.stop()` to run a streaming query to completion in a test environment without setting up a continuous trigger.
- **JUnit reports**: Output test results with `--junitxml=test-results.xml` so CI platforms like GitHub Actions can parse and display per-test pass/fail results.
- **Low parallelism for speed**: Set `spark.sql.shuffle.partitions=1` and `spark.default.parallelism=` in local test SparkSessions to dramatically reduce test execution time.

Related Topics

- Unit Testing — Part 1 - Built-in assertions and `DataFrame.transform`
- Python Project Structure and Dependencies - Where tests live in a bundle
- Advanced Testing & Operations — Part 1 - Property-based and data quality testing
- Declarative Automation Bundles - Test deployment
- CI/CD Integration - Pipeline testing
- Git Folders - Test versioning

Official Documentation

- pyspark.testing.assertDataFrameEqual
- pyspark.testing.assertSchemaEqual
- DataFrame.transform
- Testing PySpark
- Unit testing for Databricks notebooks
- Python unit testing in the workspace
- Testing for Databricks Connect for Python
- Unit testing for pipelines (Beta)
- Testing Notebooks
- Nutter Framework
- pytest Documentation
- chispa Library

Advanced Testing & Operations — Part 1

Overview

This guide covers advanced testing strategies for Databricks — property-based testing, Lakeflow Pipelines testing, streaming pipeline testing, integration testing, and test isolation patterns.

> For deployment validation, GitOps patterns, and rollback strategies, see Part 2: Advanced Operations & Deployment.

Overview

- **Test pyramid** for data engineering, property-based testing with Hypothesis, and data quality tests with Great Expectations
- **Pipeline tests**: Lakeflow Pipelines and streaming pipelines, mocking Databricks-specific APIs, and code coverage for PySpark
- **Integration testing**: Databricks Connect for remote tests, test isolation strategies, and cost control for test clusters

What the Exam Tests

- Develop unit and integration tests using assertDataFrameEqual, assertSchemaEqual, DataFrame.transform, and testing frameworks to ensure code correctness.
- Pick a test isolation strategy that lets parallel CI runs share one workspace
- Decide which tests run locally and which run remotely on Databricks compute

Advanced Testing Strategies

Test Pyramid for Data Engineering

```
flowchart TB
    subgraph Pyramid["Test Pyramid"]
        E2E["End-to-End Tests<br>Full pipeline runs<br>~5% of tests"]
        Integration["Integration Tests<br>Multi-component interaction<br>~15% of
↪   tests"]
        Unit["Unit Tests<br>Individual functions<br>~80% of tests"]
    end

    E2E --- Speed1["Slowest - Minutes to hours"]
    Integration --- Speed2["Medium - Seconds to minutes"]
    Unit --- Speed3["Fastest - Milliseconds"]

    style E2E fill:#ff6b6b
    style Integration fill:#feca57
    style Unit fill:#48dbfb
```

Test Level	Runs Where	Spark Required	Typical Duration	CI Stage
Unit	Local / CI runner	Local SparkSession	< 1 min	Every PR
Integration	Databricks workspace	Cluster-based	5-30 min	Merge to main
End-to-End	Databricks workspace	Full cluster	30-120 min	Pre-production
Data Quality	Databricks workspace	SQL Warehouse	2-10 min	Post-deploy

Property-Based Testing with Hypothesis Property-based testing generates random inputs to discover edge cases that example-based tests miss.

```python
# tests/unit/test_transforms_property.py

"""Property-based tests for data transformations."""
import pytest
from hypothesis import given, strategies as st, settings
from hypothesis.extra.pandas import columns, data_frames, column
from pyspark.sql import SparkSession
from src.transformations.cleaning import remove_nulls, deduplicate

@pytest.fixture(scope="module")
def spark():
    return SparkSession.builder.master("local[*]").getOrCreate()

class TestCleaningProperties:

    @given(
        num_rows=st.integers(min_value=0, max_value=100),
        null_fraction=st.floats(min_value=0.0, max_value=1.0)
```

```
    )
    @settings(max_examples=50, deadline=None)
    def test_remove_nulls_never_returns_nulls(
        self, spark, num_rows, null_fraction
    ):
        """Property: After remove_nulls, the column must have zero NULLs."""
        import random

        data = []
        for i in range(num_rows):
            name = None if random.random() < null_fraction else f"name_{i}"
            data.append((i, name))

        df = spark.createDataFrame(data, ["id", "name"])
        result = remove_nulls(df, "name")

        # Property assertion: no nulls should remain
        assert result.filter("name IS NULL").count() == 0

    @given(
        data=st.lists(
            st.tuples(
                st.integers(min_value=1, max_value=10),
                st.text(min_size=1, max_size=10)
            ),
            min_size=0, max_size=50
        )
    )
    @settings(max_examples=30, deadline=None)
    def test_deduplicate_reduces_or_preserves_count(self, spark, data):
        """Property: Deduplication must not increase row count."""
        if not data:
            return

        df = spark.createDataFrame(data, ["id", "value"])
        result = deduplicate(df, ["id"])

        assert result.count() <= df.count()

    @given(
        data=st.lists(
            st.tuples(
                st.integers(min_value=1, max_value=5),
                st.text(min_size=1, max_size=10)
            ),
            min_size=1, max_size=50
        )
    )
    @settings(max_examples=30, deadline=None)
    def test_deduplicate_produces_unique_keys(self, spark, data):
        """Property: After dedup, all key values must be unique."""
        df = spark.createDataFrame(data, ["id", "value"])
        result = deduplicate(df, ["id"])
        unique_ids = result.select("id").distinct().count()
```

```
        assert unique_ids == result.count()
```

Data Quality Testing with Great Expectations

```
# tests/integration/test_data_quality_ge.py

"""Data quality tests using Great Expectations."""
import great_expectations as gx

def create_gx_context():
    """Create a GX context connected to Databricks."""
    context = gx.get_context()

    # Add Databricks data source
    datasource = context.data_sources.add_spark("databricks_source")
    return context, datasource

def test_bronze_events_quality():
    """Validate bronze events table data quality."""
    context, datasource = create_gx_context()

    # Define the data asset
    asset = datasource.add_dataframe_asset("bronze_events")
    batch = asset.add_batch_definition_whole_dataframe("full_table")

    # Create expectation suite
    suite = context.suites.add(
        gx.ExpectationSuite(name="bronze_events_suite")
    )

    # Define expectations
    suite.add_expectation(
        gx.expectations.ExpectColumnToExist(column="event_id")
    )
    suite.add_expectation(
        gx.expectations.ExpectColumnValuesToNotBeNull(column="event_id")
    )
    suite.add_expectation(
        gx.expectations.ExpectColumnValuesToBeUnique(column="event_id")
    )
    suite.add_expectation(
        gx.expectations.ExpectColumnValuesToBeInSet(
            column="event_type",
            value_set=["click", "view", "purchase", "signup"]
        )
    )
    suite.add_expectation(
        gx.expectations.ExpectColumnValuesToBeBetween(
            column="event_timestamp",
            min_value="2024-01-01",
            max_value="2026-12-31"
        )
    )

    # Run validation
    df = spark.table("prod_catalog.bronze.events")
```

```
    results = batch.validate(suite, dataframe=df)

    assert results.success, f"Data quality check failed: {results}"
```

Testing Lakeflow Pipelines Lakeflow pipelines cannot be unit-tested directly because they rely on the Lakeflow Pipelines runtime. Instead, extract the transformation logic and test it separately.

```
# src/pipelines/dlt_transforms.py

"""DLT transformation logic extracted for testability."""
from pyspark.sql import DataFrame
from pyspark.sql.functions import col, when, current_timestamp

def clean_events(df: DataFrame) -> DataFrame:
    """Clean raw events - logic used in DLT pipeline."""
    return (
        df
        .filter(col("event_id").isNotNull())
        .withColumn(
            "event_type",
            when(col("event_type").isNull(), "unknown")
            .otherwise(col("event_type"))
        )
        .withColumn("processed_at", current_timestamp())
        .dropDuplicates(["event_id"])
    )

def enrich_events(events_df: DataFrame, users_df: DataFrame) -> DataFrame:
    """Enrich events with user data - logic used in DLT pipeline."""
    return (
        events_df
        .join(users_df, events_df.user_id == users_df.user_id, "left")
        .select(
            events_df["*"],
            users_df["user_name"],
            users_df["user_segment"]
        )
    )


# src/pipelines/dlt_notebook.py (Databricks DLT notebook)
# This notebook uses DLT decorators but calls the testable functions

import dlt
from pyspark.sql.functions import col
from pipelines.dlt_transforms import clean_events, enrich_events

@dlt.table(
    comment="Cleaned events from raw source",
    table_properties={"quality": "silver"}
)
@dlt.expect_or_drop("valid_event_id", "event_id IS NOT NULL")
def silver_events():
    raw_df = dlt.read("bronze_events")
    return clean_events(raw_df)
```

```python
@dlt.table(
    comment="Events enriched with user data",
    table_properties={"quality": "gold"}
)
def gold_enriched_events():
    events = dlt.read("silver_events")
    users = dlt.read("dim_users")
    return enrich_events(events, users)
```

```python
# tests/unit/test_dlt_transforms.py

"""Unit tests for DLT transformation logic."""
import pytest
from pyspark.sql import SparkSession
from chispa import assert_df_equality
from src.pipelines.dlt_transforms import clean_events, enrich_events

@pytest.fixture(scope="module")
def spark():
    return SparkSession.builder.master("local[*]").getOrCreate()

class TestDLTTransforms:

    def test_clean_events_drops_null_event_id(self, spark):
        data = [(1, "click"), (None, "view"), (3, "purchase")]
        df = spark.createDataFrame(data, ["event_id", "event_type"])

        result = clean_events(df)

        assert result.count() == 2
        assert result.filter("event_id IS NULL").count() == 0

    def test_clean_events_replaces_null_event_type(self, spark):
        data = [(1, None), (2, "click")]
        df = spark.createDataFrame(data, ["event_id", "event_type"])

        result = clean_events(df)

        types = [row.event_type for row in result.collect()]
        assert "unknown" in types
        assert None not in types

    def test_clean_events_deduplicates(self, spark):
        data = [(1, "click"), (1, "click"), (2, "view")]
        df = spark.createDataFrame(data, ["event_id", "event_type"])

        result = clean_events(df)

        assert result.count() == 2

    def test_enrich_events_joins_user_data(self, spark):
        events = spark.createDataFrame(
            [(1, 100, "click"), (2, 200, "view")],
            ["event_id", "user_id", "event_type"]
        )
        users = spark.createDataFrame(
```

```
            [(100, "Alice", "premium"), (200, "Bob", "free")],
            ["user_id", "user_name", "user_segment"]
        )

        result = enrich_events(events, users)

        assert "user_name" in result.columns
        assert "user_segment" in result.columns
        alice = result.filter("event_id = 1").collect()[0]
        assert alice.user_name == "Alice"
```

Testing Streaming Pipelines

```
# tests/unit/test_streaming_logic.py

"""Test streaming transformations using batch mode."""
import pytest
import os
import shutil
from pyspark.sql import SparkSession
from pyspark.sql.functions import window, count

@pytest.fixture(scope="module")
def spark():
    return (SparkSession.builder
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "1")
        .getOrCreate())

@pytest.fixture
def stream_paths(tmp_path):
    """Provide temp paths for stream input, output, and checkpoint."""
    paths = {
        "input": str(tmp_path / "input"),
        "output": str(tmp_path / "output"),
        "checkpoint": str(tmp_path / "checkpoint"),
    }
    os.makedirs(paths["input"], exist_ok=True)
    return paths

class TestStreamingAggregation:

    def test_windowed_count_produces_expected_windows(
        self, spark, stream_paths
    ):
        """Test that streaming windowed count works correctly."""
        # Arrange: write test data as JSON
        data = [
            {"user_id": "u1", "ts": "2025-01-01T00:00:00"},
            {"user_id": "u1", "ts": "2025-01-01T00:05:00"},
            {"user_id": "u2", "ts": "2025-01-01T00:02:00"},
            {"user_id": "u1", "ts": "2025-01-01T01:00:00"},
        ]
        import json
        with open(os.path.join(stream_paths["input"], "batch1.json"), "w") as f:
            for record in data:
```

```
            f.write(json.dumps(record) + "\n")

    # Act: read as stream, apply windowed count, write to output
    from pyspark.sql.types import (
        StructType, StructField, StringType, TimestampType
    )
    schema = StructType([
        StructField("user_id", StringType()),
        StructField("ts", TimestampType()),
    ])

    stream_df = (
        spark.readStream
        .schema(schema)
        .json(stream_paths["input"])
    )

    windowed = (
        stream_df
        .groupBy(
            window("ts", "1 hour"),
            "user_id"
        )
        .agg(count("*").alias("event_count"))
    )

    query = (
        windowed.writeStream
        .format("json")
        .outputMode("complete")
        .option("checkpointLocation", stream_paths["checkpoint"])
        .start(stream_paths["output"])
    )

    query.processAllAvailable()
    query.stop()

    # Assert
    result = spark.read.json(stream_paths["output"])
    assert result.count() > 0

    # user1 should have events in 2 windows (00:00 and 01:00)
    u1_windows = result.filter("user_id = 'u1'").count()
    assert u1_windows == 2
```

Mocking Databricks-Specific APIs

```
# tests/conftest.py

"""Comprehensive mocks for Databricks-specific APIs."""
import pytest
from unittest.mock import MagicMock, patch, PropertyMock

@pytest.fixture
def mock_dbutils():
    """Full dbutils mock with all common methods."""
```

```
    dbutils = MagicMock()

    # Widgets
    widget_values = {
        "catalog": "test_catalog",
        "schema": "test_schema",
        "environment": "test",
    }
    dbutils.widgets.get.side_effect = lambda key: widget_values.get(key, "")
    dbutils.widgets.text = MagicMock()

    # Secrets
    secret_values = {
        ("scope1", "db-password"): "test_password",
        ("scope1", "api-key"): "test_api_key",
    }
    dbutils.secrets.get.side_effect = (
        lambda scope, key: secret_values.get((scope, key), "")
    )

    # Filesystem
    dbutils.fs.ls.return_value = [
        MagicMock(path="dbfs:/data/file1.parquet", name="file1.parquet",
                  size=1024, modificationTime=1700000000000),
        MagicMock(path="dbfs:/data/file2.parquet", name="file2.parquet",
                  size=2048, modificationTime=1700000001000),
    ]
    dbutils.fs.head.return_value = "sample file content"
    dbutils.fs.rm = MagicMock(return_value=True)
    dbutils.fs.mkdirs = MagicMock(return_value=True)
    dbutils.fs.cp = MagicMock(return_value=True)

    # Notebook context
    (dbutils.notebook.entry_point.getDbutils.return_value
        .notebook.return_value
        .getContext.return_value
        .tags.return_value
        .get).return_value = "interactive"

    # Notebook run
    dbutils.notebook.run.return_value = '{"status": "success"}'

    return dbutils

@pytest.fixture
def mock_spark_catalog():
    """Mock for spark.catalog operations."""
    catalog = MagicMock()
    catalog.tableExists.return_value = True
    catalog.listTables.return_value = [
        MagicMock(name="events", database="bronze", tableType="MANAGED"),
        MagicMock(name="sessions", database="silver", tableType="MANAGED"),
    ]
    return catalog
```

Code Coverage for PySpark

```
# pytest.ini - Coverage configuration

[pytest]
testpaths = tests
addopts =
    -v
    --tb=short
    --cov=src
    --cov-report=term-missing
    --cov-report=xml:coverage.xml
    --cov-report=html:htmlcov
    --cov-fail-under=80
```

```
# .github/workflows/coverage.yml

name: Coverage Check

on: [pull_request]

jobs:
  coverage:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.10"
      - run: pip install -r requirements-dev.txt

      - name: Run tests with coverage
        run: |
          pytest tests/unit/ \
            --cov=src \
            --cov-report=xml \
            --cov-fail-under=80

      - name: Upload to Codecov
        uses: codecov/codecov-action@v4
        with:
          files: coverage.xml
          fail_ci_if_error: true
```

```
# .coveragerc - Fine-tune what to measure

[run]
source = src
omit =
    src/notebooks/*
    src/*/__init__.py
    tests/*

[report]
exclude_lines =
    pragma: no cover
    def __repr__
```

```
    if __name__ == .__main__
    raise NotImplementedError
show_missing = True
fail_under = 80
```

Integration Testing Patterns

Databricks Connect for Remote Testing

```
# tests/integration/conftest.py

"""Integration test fixtures using Databricks Connect."""
import pytest
from databricks.connect import DatabricksSession

@pytest.fixture(scope="session")
def db_spark():
    """Create a Databricks Connect SparkSession."""
    spark = (DatabricksSession.builder
        .remote(
            host="https://adb-1234567890.1.azuredatabricks.net",
            token="dapi...",
            cluster_id="0123-456789-abcdef"
        )
        .getOrCreate())
    yield spark
    spark.stop()

@pytest.fixture
def test_schema(db_spark):
    """Create and drop an isolated test schema."""
    import uuid
    schema_name = f"test_{uuid.uuid4().hex[:8]}"
    full_name = f"test_catalog.{schema_name}"

    db_spark.sql(f"CREATE SCHEMA IF NOT EXISTS {full_name}")
    yield full_name
    db_spark.sql(f"DROP SCHEMA IF EXISTS {full_name} CASCADE")
```

```
# tests/integration/test_pipeline_integration.py

"""Integration tests running against a real Databricks workspace."""

class TestBronzeToSilverPipeline:

    def test_bronze_ingest_writes_to_delta(self, db_spark, test_schema):
        """Test that bronze ingestion creates a Delta table."""
        from src.pipelines.bronze import ingest_events

        ingest_events(
            db_spark,
            source_path="/Volumes/test_catalog/test/raw/events.json",
            target_table=f"{test_schema}.bronze_events"
        )

        result = db_spark.table(f"{test_schema}.bronze_events")
```

```
        assert result.count() > 0

    def test_silver_transform_cleans_data(self, db_spark, test_schema):
        """Test that silver transformation applies cleaning rules."""
        from src.pipelines.silver import transform_events

        # Setup: create bronze table with test data
        data = [(1, "click", None), (2, "view", "Chrome"), (None, "x", "FF")]
        df = db_spark.createDataFrame(data, ["id", "type", "browser"])
        df.write.format("delta").saveAsTable(f"{test_schema}.bronze_events")

        transform_events(
            db_spark,
            source_table=f"{test_schema}.bronze_events",
            target_table=f"{test_schema}.silver_events"
        )

        result = db_spark.table(f"{test_schema}.silver_events")
        # Null IDs should be filtered out
        assert result.filter("id IS NULL").count() == 0
```

Test Isolation Strategies

```
flowchart TB
    subgraph Isolation["Test Isolation Methods"]
        Schema[Unique Schema per Test Run]
        Prefix[Prefixed Table Names]
        Cleanup[Teardown Cleanup]
    end

    Schema --> Create["CREATE SCHEMA test_abc123"]
    Schema --> Run["Run tests against schema"]
    Schema --> Drop["DROP SCHEMA test_abc123 CASCADE"]

    Prefix --> PrefixCreate["test_run_42_bronze_events"]
    Cleanup --> After["after_each: DROP TABLE IF EXISTS"]
```

```
# tests/integration/test_isolation.py

"""Patterns for test isolation in Databricks."""
import pytest
import uuid
from datetime import datetime

@pytest.fixture(scope="session")
def test_run_id():
    """Generate unique ID for this test run."""
    return
     ↪  f"test_{datetime.now().strftime('%Y%m%d_%H%M%S')}_{uuid.uuid4().hex[:6]}"

@pytest.fixture(scope="session")
def isolated_schema(db_spark, test_run_id):
    """Create an isolated schema for the entire test session."""
    schema = f"test_catalog.{test_run_id}"
    db_spark.sql(f"CREATE SCHEMA IF NOT EXISTS {schema}")
    print(f"Created test schema: {schema}")
```

```
    yield schema
    # Cleanup after all tests
    db_spark.sql(f"DROP SCHEMA IF EXISTS {schema} CASCADE")
    print(f"Dropped test schema: {schema}")

@pytest.fixture
def isolated_table(db_spark, isolated_schema):
    """Provide a unique table name and clean up after the test."""
    table_id = uuid.uuid4().hex[:8]
    table_name = f"{isolated_schema}.table_{table_id}"
    yield table_name
    db_spark.sql(f"DROP TABLE IF EXISTS {table_name}")
```

Cost Management for Test Clusters

Strategy	Description	Savings
Single-node clusters	`num_workers: 0` for unit/integration tests	60-80%
Auto-termination	Set 10-min idle timeout on test clusters	Variable
Instance pools	Pre-warm pools for faster test starts	30-50% start time
Spot instances	Use spot/preemptible for test workloads	60-90%
Shared test cluster	Reuse cluster across CI runs with concurrency	40-60%
Serverless compute	Pay only for compute used during tests	Variable

```
# databricks.yml - Cost-optimized test target

targets:
  test:
    mode: development
    variables:
      catalog: test_catalog
    resources:
      jobs:
        integration_tests:
          name: "Integration Tests - ${workspace.current_user.userName}"
          tasks:
            - task_key: run_tests
              notebook_task:
                notebook_path: ../tests/integration/run_all
              new_cluster:
                spark_version: "15.4.x-scala2.12"
                node_type_id: "Standard_DS3_v2"
                num_workers: 0           # Single-node for tests
                spark_conf:
                  spark.master: "local[*]"
                  spark.databricks.cluster.profile: singleNode
                custom_tags:
                  ResourceClass: SingleNode
                  purpose: testing
                autotermination_minutes: 10
```

Continue reading: Part 2 — Deployment Validation, GitOps, Practice Questions & Exam Tips

---

Advanced Testing & Operations — Part 2

Overview

This file covers deployment validation, automated rollback, GitOps patterns, and common troubleshooting scenarios.

> For property-based testing, Lakeflow Pipelines testing, streaming tests, and integration patterns, see Part 1.

Overview

- **Deployment validation**: run a smoke test job right after deployment and roll back automatically (redeploy the previous commit) if it fails
- **GitOps**: mono-repo vs multi-repo, `paths:` filters that deploy only the changed bundle, and trunk-based development with feature flags
- **OIDC federation** (`permissions: id-token: write`) replaces stored CI secrets
- **CI guardrails**: coverage gating with `--cov-fail-under` and per-run test schemas dropped with `CASCADE`

What the Exam Tests

- Implement scalable Python project structures for Declarative Automation Bundles (formerly Databricks Asset Bundles / DABs) to support modular development and CI/CD integration.
- Design a validate-then-rollback step after a bundle deployment
- Choose a repository and branching strategy for a data engineering team

Deployment Validation

Post-Deployment Health Checks

```python
# scripts/post_deploy_checks.py

"""Post-deployment validation suite."""
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.sql import StatementState

class PostDeploymentValidator:
    """Run health checks after deployment."""

    def __init__(self, catalog: str, warehouse_id: str):
        self.client = WorkspaceClient()
        self.catalog = catalog
        self.warehouse_id = warehouse_id
        self.results = []

    def check_tables_exist(self, expected_tables: list[str]):
        """Verify all expected tables exist after deployment."""
        for table in expected_tables:
            result = self._run_sql(
                f"SELECT 1 FROM {self.catalog}.information_schema.tables "
                f"WHERE table_schema || '.' || table_name = '{table}'"
            )
            exists = len(result) > 0
            self.results.append({
```

```
            "check": f"table_exists_{table}",
            "passed": exists,
            "message": f"Table {table} {'exists' if exists else 'MISSING'}"
        })

    def check_row_counts(self, table_minimums: dict[str, int]):
        """Verify tables have minimum expected row counts."""
        for table, min_rows in table_minimums.items():
            result = self._run_sql(
                f"SELECT COUNT(*) as cnt FROM {self.catalog}.{table}"
            )
            actual = result[0]["cnt"] if result else 0
            passed = actual >= min_rows
            self.results.append({
                "check": f"row_count_{table}",
                "passed": passed,
                "message": (
                    f"{table}: {actual} rows "
                    f"(minimum: {min_rows})"
                )
            })

    def check_freshness(self, table: str, max_age_hours: int):
        """Verify table data is fresh enough."""
        result = self._run_sql(f"""
            SELECT MAX(updated_at) as latest
            FROM {self.catalog}.{table}
        """)
        if result and result[0]["latest"]:
            from datetime import datetime, timedelta
            latest = result[0]["latest"]
            cutoff = datetime.now() - timedelta(hours=max_age_hours)
            passed = latest >= cutoff
        else:
            passed = False

        self.results.append({
            "check": f"freshness_{table}",
            "passed": passed,
            "message": f"{table} freshness check"
        })

    def check_jobs_healthy(self, job_names: list[str]):
        """Verify critical jobs are not in a failed state."""
        for job_name in job_names:
            jobs = list(self.client.jobs.list(name=job_name))
            if not jobs:
                self.results.append({
                    "check": f"job_exists_{job_name}",
                    "passed": False,
                    "message": f"Job '{job_name}' not found"
                })
                continue

            job = jobs[0]
```

```
            runs = list(self.client.jobs.list_runs(
                job_id=job.job_id, limit=1
            ))
            if runs:
                last_run = runs[0]
                passed = str(last_run.state.result_state) != "FAILED"
            else:
                passed = True  # No runs yet is okay

            self.results.append({
                "check": f"job_healthy_{job_name}",
                "passed": passed,
                "message": f"Job '{job_name}' health check"
            })

    def report(self) -> bool:
        """Print results and return overall pass/fail."""
        all_passed = True
        for r in self.results:
            status = "PASS" if r["passed"] else "FAIL"
            print(f"  [{status}] {r['message']}")
            if not r["passed"]:
                all_passed = False

        print(f"\nOverall: {'PASSED' if all_passed else 'FAILED'}")
        return all_passed

    def _run_sql(self, statement: str):
        """Execute SQL and return results."""
        response = self.client.statement_execution.execute_statement(
            warehouse_id=self.warehouse_id,
            statement=statement,
            wait_timeout="30s"
        )
        if response.status.state == StatementState.SUCCEEDED:
            if response.result and response.result.data_array:
                columns = [c.name for c in response.manifest.schema.columns]
                return [
                    dict(zip(columns, row))
                    for row in response.result.data_array
                ]
        return []
```

```
#!/bin/bash
# scripts/post_deploy_validate.sh
# Run post-deployment validation as part of CI/CD

set -euo pipefail

ENVIRONMENT=$1

echo "Running post-deployment validation for ${ENVIRONMENT}..."

# Wait for any triggered jobs to start

sleep 30
```

```
# Run validation notebook

databricks bundle run post_deploy_checks -t "${ENVIRONMENT}"

# Check exit status

if [ $? -ne 0 ]; then
    echo "Post-deployment validation FAILED"
    echo "Initiating rollback..."
    ./scripts/rollback.sh "${ENVIRONMENT}"
    exit 1
fi

echo "Post-deployment validation PASSED"
```

Automated Rollback Triggers

```
# .github/workflows/deploy-with-rollback.yml

name: Deploy with Auto-Rollback

on:
  push:
    branches: [main]

jobs:
  deploy-and-validate:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 2  # Need previous commit for rollback

      - uses: databricks/setup-cli@main

      - name: Record current state
        id: pre-deploy
        run: |
          echo "commit_before=$(git rev-parse HEAD~1)" >> $GITHUB_OUTPUT

      - name: Deploy to production
        run: databricks bundle deploy -t prod
        env:
          DATABRICKS_HOST: ${{ secrets.PROD_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.PROD_TOKEN }}

      - name: Run smoke tests
        id: smoke
        continue-on-error: true
        run: databricks bundle run smoke_test_job -t prod
        env:
          DATABRICKS_HOST: ${{ secrets.PROD_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.PROD_TOKEN }}
```

```yaml
      - name: Rollback on failure
        if: steps.smoke.outcome == 'failure'
        run: |
          echo "Smoke tests failed. Rolling back to ${{
↪ steps.pre-deploy.outputs.commit_before }}"
          git checkout ${{ steps.pre-deploy.outputs.commit_before }}
          databricks bundle deploy -t prod
        env:
          DATABRICKS_HOST: ${{ secrets.PROD_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.PROD_TOKEN }}

      - name: Alert on rollback
        if: steps.smoke.outcome == 'failure'
        uses: slackapi/slack-github-action@v1
        with:
          channel-id: incidents
          slack-message: |
            ROLLBACK TRIGGERED for production deployment.
            Failed commit: ${{ github.sha }}
            Rolled back to: ${{ steps.pre-deploy.outputs.commit_before }}
        env:
          SLACK_BOT_TOKEN: ${{ secrets.SLACK_BOT_TOKEN }}
```

GitOps for Databricks

Mono-Repo vs Multi-Repo Strategies

Aspect	Mono-Repo	Multi-Repo
Structure	Single repo for all pipelines	Separate repo per project/team
Shared code	Easy internal imports	Requires package publishing
CI/CD	Longer pipelines, path-based triggers	Simpler per-repo pipelines
Ownership	CODEOWNERS file per directory	Repo-level permissions
Discoverability	All code in one place	Need a catalog/registry
Best for	Small-medium teams, shared infra	Large orgs, independent teams

```
# Mono-repo structure

databricks-platform/
├── CODEOWNERS
├── .github/workflows/
│   ├── ci.yml              # Shared CI
│   └── cd.yml              # Shared CD with path filters
├── projects/
│   ├── ingestion/           # Team A
│   │   ├── databricks.yml
│   │   ├── src/
│   │   └── tests/
│   ├── analytics/           # Team B
│   │   ├── databricks.yml
│   │   ├── src/
│   │   └── tests/
│   └── ml-features/         # Team C
│       ├── databricks.yml
```

```
│       ├── src/
│       └── tests/
├── shared/                    # Shared libraries
│   ├── etl-core/
│   └── data-quality/
└── infrastructure/            # Terraform/IaC
    ├── workspaces/
    └── unity-catalog/
```

```yaml
# .github/workflows/cd.yml - Path-filtered deployment for mono-repo

name: Deploy

on:
  push:
    branches: [main]
    paths:
      - 'projects/ingestion/**'
      - 'projects/analytics/**'
      - 'shared/**'

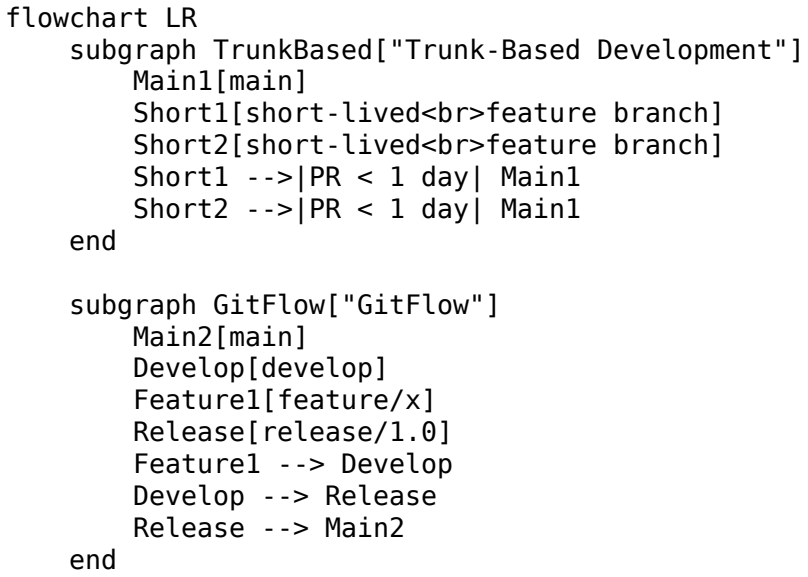
jobs:
  detect-changes:
    runs-on: ubuntu-latest
    outputs:
      ingestion: ${{ steps.filter.outputs.ingestion }}
      analytics: ${{ steps.filter.outputs.analytics }}
    steps:
      - uses: actions/checkout@v4
      - uses: dorny/paths-filter@v3
        id: filter
        with:
          filters: |
            ingestion:
              - 'projects/ingestion/**'
              - 'shared/**'
            analytics:
              - 'projects/analytics/**'
              - 'shared/**'

  deploy-ingestion:
    needs: detect-changes
    if: needs.detect-changes.outputs.ingestion == 'true'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - run: databricks bundle deploy -t prod
        working-directory: projects/ingestion
        env:
          DATABRICKS_HOST: ${{ secrets.PROD_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.PROD_TOKEN }}

  deploy-analytics:
    needs: detect-changes
```

```yaml
    if: needs.detect-changes.outputs.analytics == 'true'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - run: databricks bundle deploy -t prod
        working-directory: projects/analytics
        env:
          DATABRICKS_HOST: ${{ secrets.PROD_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.PROD_TOKEN }}
```

Branch Strategy Comparison



```
flowchart LR
    subgraph TrunkBased["Trunk-Based Development"]
        Main1[main]
        Short1[short-lived<br>feature branch]
        Short2[short-lived<br>feature branch]
        Short1 -->|PR < 1 day| Main1
        Short2 -->|PR < 1 day| Main1
    end

    subgraph GitFlow["GitFlow"]
        Main2[main]
        Develop[develop]
        Feature1[feature/x]
        Release[release/1.0]
        Feature1 --> Develop
        Develop --> Release
        Release --> Main2
    end
```

Strategy	Best For	Pros	Cons
Trunk-based	Small teams, frequent deploys	Simple, fast iteration	Requires feature flags
GitFlow	Large teams, scheduled releases	Structured, clear releases	Complex, slow merges
GitHub Flow	Medium teams	Simple, PR-based	No staging branch
Environment branches	Multi-env promotion	Clear env mapping	Branch divergence ris

Recommendation for data engineering: Trunk-based development with feature flags. Data pipelines benefit from frequent small changes rather than large batch releases.

Pull Request Templates for Data Engineering

```
<!-- .github/PULL_REQUEST_TEMPLATE.md -->
## Description

<!-- What does this change do? Why is it needed? -->

## Type of Change

- [ ] New pipeline / data source
- [ ] Pipeline modification
- [ ] Bug fix
- [ ] Schema change
```

```
- [ ] Configuration change
- [ ] Performance optimization

## Impact Assessment

- [ ] **Tables affected:** <!-- list tables -->
- [ ] **Downstream consumers:** <!-- who reads these tables? -->
- [ ] **Schema changes:** <!-- any column additions/removals? -->
- [ ] **Backfill required:** <!-- does historical data need reprocessing? -->

## Testing

- [ ] Unit tests pass locally
- [ ] Integration tests pass in staging
- [ ] Data quality checks verified
- [ ] Tested with production-scale data sample

## Deployment Notes

- [ ] No manual steps required
- [ ] Requires schema migration (attached migration file)
- [ ] Requires configuration change in target environment
- [ ] Requires backfill job after deployment

## Rollback Plan

<!-- How would you roll back this change if something goes wrong? -->


# CODEOWNERS - Enforce reviews by area
# .github/CODEOWNERS

# Data platform team reviews shared infrastructure

/shared/                    @data-platform-team
/infrastructure/            @data-platform-team

# Team-specific code ownership

/projects/ingestion/        @ingestion-team
/projects/analytics/        @analytics-team
/projects/ml-features/      @ml-team

# CI/CD changes require platform team review

/.github/                   @data-platform-team
**/databricks.yml           @data-platform-team
```

Practice Questions

Question 1: A data engineer is setting up CI/CD for a Databricks project that spans three teams, each owning separate pipelines. All teams share common cluster configurations and notification settings. What is the recommended approach for organizing their Declarative Automation Bundles?

A) Create a single databricks.yml with all resources and use variables to toggle per team B) Use the `include` directive to reference shared YAML files from each team's bundle C) Duplicate the

shared configuration in each team's databricks.yml for independence D) Store all configuration in environment variables and reference them at deploy time

Answer

Correct Answer: B
The `include` directive in databricks.yml allows multiple bundles to reference shared configuration files (e.g., common cluster definitions, notification settings) without duplication. Each team maintains their own databricks.yml while inheriting shared resources from a common directory. This follows DRY principles and ensures consistent configuration across teams.

Question 2: Which authentication method eliminates the need for long-lived secrets in GitHub Actions when deploying to Databricks?

A) Personal Access Tokens stored in GitHub Secrets B) Service principal client secret in environment variables C) OIDC federation with GitHub Actions identity provider D) SSH keys stored in the repository

Answer

Correct Answer: C
OIDC (OpenID Connect) federation allows GitHub Actions to authenticate with Databricks using short-lived JWT tokens. The CI runner requests a token from GitHub's OIDC provider, and Databricks validates it against a federation policy configured on the service principal. This eliminates storing long-lived secrets entirely.

Question 3: A data engineer needs to test Lakeflow pipeline transformation logic in a local pytest environment. What is the correct approach?

A) Install the Lakeflow Pipelines runtime locally and import `dlt` decorators in tests B) Extract transformation logic into plain PySpark functions and test those C) Run `databricks bundle run` from within pytest to execute Lakeflow Pipelines tests D) Use `unittest.mock` to fully mock the Lakeflow Pipelines runtime

Answer

Correct Answer: B
DLT pipelines depend on the Databricks DLT runtime, which cannot run locally. The recommended pattern is to extract transformation logic (joins, filters, aggregations) into standalone PySpark functions, then test those functions with a local SparkSession and standard pytest. The DLT notebook simply calls these functions within `@dlt.table` decorated functions.

Question 4: A team has deployed a new version of their production ETL pipeline, but post-deployment smoke tests are failing. They are using Declarative Automation Bundles. What is the fastest way to rollback?

A) Delete the production job and recreate it manually from the Databricks UI B) Run `databricks bundle destroy` and then redeploy from the previous commit C) Check out the previous Git commit and run `databricks bundle deploy -t prod` D) Use Delta Time Travel to restore all affected tables

Answer

Correct Answer: C
The fastest rollback approach with DAB is to checkout the previous known-good commit from Git and re-run `databricks bundle deploy -t prod`. This redeploys the exact previous configuration (job definitions, notebook code, cluster settings) without destroying existing resources first. Delta Time Travel (D) handles data rollback but does not restore job/pipeline configura-

tions.

Question 5: Which testing strategy best complements property-based testing when validating PySpark transformations?

A) Only run end-to-end tests in production B) Use `chispa.assert_df_equality` for exact output comparison alongside Hypothesis for edge case discovery C) Rely solely on Hypothesis to generate all test scenarios D) Skip unit tests and use Great Expectations for all validation

Answer

Correct Answer: B
Property-based testing (using Hypothesis) excels at discovering unexpected edge cases by generating random inputs and checking invariants (e.g., "deduplication never increases row count"). It complements example-based testing with chispa, which verifies exact expected outputs for specific known inputs. Together they provide thorough coverage: chispa for correctness of known cases, Hypothesis for robustness against unexpected inputs.

Use Cases

- **Post-deployment validation**: run a smoke test job right after each deployment and roll back automatically (check out the previous commit and redeploy) if it fails
- **Mono-repo with several bundles**: use `paths:` filters in GitHub Actions so only the bundle whose files changed is deployed
- **Reviewing data pipeline changes**: use a pull request template that records the type of change, impact assessment, testing, deployment notes, and rollback plan
- **Parallel CI runs**: give each integration test run its own `test_catalog.ci_<uuid>` schema and drop it with `CASCADE` in teardown

Common Issues & Errors

OIDC Token Exchange Fails **Scenario:** GitHub Actions OIDC authentication fails with "invalid token" error.

Fix: Verify the federation policy configuration:

```
# Check the subject claim matches exactly
# Format: repo:<org>/<repo>:ref:refs/heads/<branch>
# Common mistake: missing the 'ref:' prefix or wrong branch name

# Correct subject for main branch:
# repo:my-org/my-repo:ref:refs/heads/main

# Correct subject for environment:
# repo:my-org/my-repo:environment:production

# Verify in workflow with:

- name: Debug OIDC
  run: |
    echo "Subject: repo:${{ github.repository }}:ref:${{ github.ref }}"
```

Multi-Project Bundle Include Path Resolution **Scenario:** `include` paths fail with "file not found" when deploying.

Fix: Include paths are relative to the bundle root (location of databricks.yml):

```yaml
# If databricks.yml is in projects/ingestion/
# And shared files are in ../../shared/

include:
  - ../../shared/common-clusters.yml    # Relative to databricks.yml
  - resources/*.yml                      # Relative to databricks.yml
```

Coverage Drops Below Threshold **Scenario:** CI fails because PySpark code coverage is below the required threshold.

Fix: Configure `.coveragerc` to exclude untestable code:

```
# .coveragerc

[run]
omit =
    src/notebooks/*          # Notebook wrappers not unit-testable
    src/pipelines/dlt_*.py   # DLT decorators need runtime

[report]
exclude_lines =
    pragma: no cover
    if __name__ == .__main__
    # Databricks-specific lines that cannot run locally
    dbutils.widgets
    display\(
```

Integration Test Schema Collisions **Scenario:** Parallel CI runs cause test failures because they write to the same schema.

Fix: Use unique schema names per test run:

```python
import uuid

@pytest.fixture(scope="session")
def test_schema(db_spark):
    schema = f"test_catalog.ci_{uuid.uuid4().hex[:8]}"
    db_spark.sql(f"CREATE SCHEMA {schema}")
    yield schema
    db_spark.sql(f"DROP SCHEMA {schema} CASCADE")
```

Blue/Green View Switch Fails **Scenario:** Switching views between blue and green schemas fails due to permissions.

Fix: Ensure the service principal has `CREATE VIEW` permissions on the target schema:

```sql
-- Grant the deployment SP permission to create/replace views
GRANT CREATE TABLE ON SCHEMA prod_catalog.prod TO `prod-deploy-sp`;
GRANT USAGE ON SCHEMA prod_catalog.prod_blue TO `prod-deploy-sp`;
GRANT SELECT ON SCHEMA prod_catalog.prod_blue TO `prod-deploy-sp`;
GRANT USAGE ON SCHEMA prod_catalog.prod_green TO `prod-deploy-sp`;
GRANT SELECT ON SCHEMA prod_catalog.prod_green TO `prod-deploy-sp`;
```

Canary Deployment Timeout **Scenario:** Canary monitoring exceeds the CI job timeout.

Fix: Use async monitoring with a separate check job:

```
# Split canary into deploy + async monitor

- name: Deploy canary
  run: |
    databricks bundle deploy -t prod-canary
    databricks bundle run canary_job -t prod-canary --no-wait

- name: Wait for canary window
  run: sleep 600  # 10-minute observation window

- name: Check canary health
  run: |
    databricks bundle run canary_health_check -t prod-canary
```

Artifact Build Fails in CI **Scenario:** Python wheel build succeeds locally but fails in CI.

Fix: Ensure CI has all build dependencies:

```
- name: Install build tools
  run: |
    pip install poetry build setuptools wheel
    # For JAR artifacts:
    # sudo apt-get install -y maven

- name: Build artifacts
  run: |
    cd libs/etl-core
    poetry install
    poetry build
```

Exam Tips

1. **OIDC federation** - Understand that OIDC eliminates stored secrets by using short-lived JWT tokens exchanged between the CI provider and Databricks
2. **Bundle includes** - Know that `include` merges YAML files and paths are relative to the bundle root (databricks.yml location)
3. **Lookup references** - `${resources.jobs.my_job.id}` resolves after deployment; cannot be used before the resource is created
4. **Blue/green vs canary** - Blue/green swaps entire environments; canary routes a subset of traffic to the new version
5. **Testing Lakeflow Pipelines** - Extract transformation logic into plain functions; test those with local SparkSession; Lakeflow Pipelines decorators cannot run outside the Lakeflow Pipelines runtime
6. **Test isolation** - Use unique schemas per test run (`test_<uuid>`) and `DROP SCHEMA CASCADE` in teardown
7. **Coverage gating** - Use `--cov-fail-under=80` in pytest to enforce minimum coverage in CI
8. **Rollback with DAB** - Checkout previous commit and `databricks bundle deploy`; Delta Time Travel for data-level rollback
9. **Feature flags** - Store in Unity Catalog table; check in pipeline code to toggle behavior per environment
10. **Mono-repo CI** - Use path-based triggers (`paths:` filter in GitHub Actions) to deploy only changed projects

Key Takeaways

- **Post-deployment validation**: Always run a smoke test job immediately after deployment and trigger an automated rollback (git checkout previous commit + redeploy) if it fails.
- **OIDC replaces stored secrets**: OIDC federation with GitHub Actions requires `permissions: id-token: write` in the workflow and a matching federation policy on the Databricks service principal — no stored client secrets needed.
- **Bundle include paths**: All paths in the `include:` directive are relative to the location of `databricks.yml`, not the working directory from which the CLI is run.
- **DLT testing pattern**: DLT decorators cannot run locally — extract transformation logic into plain PySpark functions, test those with a local SparkSession, and wrap them in `@dlt.table` for the pipeline.
- **Test schema isolation**: Use `test_catalog.ci_<uuid>` schema names for integration tests and drop the schema with `CASCADE` in test teardown to prevent parallel CI run collisions.
- **Coverage gating**: Use `--cov-fail-under=80` in pytest CI to enforce a minimum code coverage threshold and block merges on coverage regressions.
- **Mono-repo path triggers**: Use `paths:` filters in GitHub Actions to deploy only the changed project bundle, preventing unnecessary redeployments of unrelated pipelines.
- **Trunk-based with feature flags**: For data engineering, trunk-based development (short-lived branches merged to main frequently) combined with feature flags is recommended over GitFlow to enable faster iteration.

Related Topics

- Declarative Automation Bundles - DAB fundamentals, structure, and targets
- CI/CD Integration - GitHub Actions, Azure DevOps basics
- Git Folders - Git integration and branching strategies
- Unit Testing - pytest, Nutter, and basic testing patterns
- Part 1 — Advanced Testing Strategies

Official Documentation

- Declarative Automation Bundles
- OIDC Federation with Databricks
- Databricks Connect
- CI/CD Best Practices
- Delta Lake Time Travel
- Great Expectations documentation (third-party)
- Lakeflow Pipelines Pipeline Testing

Section 2: Data Ingestion & Acquisition (12 % of Exam)

Getting data into (or in front of) the lakehouse: Auto Loader and `COPY INTO` for files, message buses (Kafka, Kinesis, Pub/Sub), incremental CDC with Lakeflow Pipelines into Delta or Iceberg, Lakeflow Connect database CDC (SQL Server, MySQL, PostgreSQL), OpenSharing, Clean Rooms, and Lakehouse Federation. The Oct 9, 2026 guide folds the former *Data Sharing and Federation* domain into this one, which is why its weight rose from 7 % to 12 %.

Topics Overview

```
flowchart LR
    D[Data Ingestion]
    D --> T1["Auto Loader"]
    D --> T2["COPY INTO"]
    D --> T3["Streaming Ingestion from Message Buses"]
    D --> T4["Change Data Capture (CDC)"]
    D --> T5["Lakeflow Connect Database CDC"]
    D --> T6["OpenSharing"]
    D --> T7["Clean Rooms"]
    D --> T8["Lakehouse Federation"]
```

Section Contents

File	Topic
01-auto-loader.md	Auto Loader: file discovery modes, schema inference and e
02-copy-into.md	`COPY INTO` idempotent batch loads and when to prefer Aut
03-streaming-ingestion-from-message-buses.md	Kafka, Kinesis, Pub/Sub, and a source × format matrix (Del
04-change-data-capture-part1.md	CDC fundamentals, Change Data Feed, MERGE-based CDC
04-change-data-capture-part2.md	Incremental CDC pipelines into Delta or Iceberg targets, mu
05-lakeflow-connect-database-cdc.md	Lakeflow Connect managed connectors for SQL Server, MyS
06-opensharing.md	OpenSharing (formerly Delta Sharing): D2D vs Databricks-t
07-clean-rooms.md	Clean Rooms for privacy-preserving collaboration
08-lakehouse-federation.md	Connections, foreign catalogs, UC permissions, connection-

Exam Skill Coverage

Every skill bullet in Section 2 of the Oct 9, 2026 exam guide, with the files that cover it.

Exam guide skill
Develop data ingestion pipelines to ingest a variety of data formats, including Delta Lake, Parquet, JSON, Iceb
Configure incremental CDC pipelines using Lakeflow Pipelines with Delta or Iceberg as the target table format.
Configure CDC ingestion pipelines from relational database sources, including SQL Server, MySQL, and Postgre
Implement OpenSharing (D2D and Databricks-to-Open) and Clean Rooms for privacy-preserving collaboration.
Configure Lakehouse Federation with governance across supported source systems, applying UC permissions

Key Concepts to Master

Concept	Why it matters
Auto Loader vs `COPY INTO`	Auto Loader for continuous / high-volume file discovery; `COPY I`
Lakeflow Connect database connectors	Managed CDC ingestion (inserts, updates, deletes) from operati

Concept	Why it matters
D2D vs Databricks-to-Open	D2D recipients use a UC-enabled workspace and a sharing iden
Clean Rooms	Collaborators run approved code on each other's data without s
Lakehouse Federation	Query external databases through a UC connection + foreign ca

Related Resources

- Auto Loader cheat sheet
- Streaming cheat sheet
- Unity Catalog Basics (shared)

Auto Loader

Auto Loader is Databricks' recommended solution for incrementally ingesting files from cloud storage. Understanding its modes and schema handling is critical for the exam.

Overview

```
flowchart LR
    subgraph Cloud Storage
        Files[New Files]
    end

    subgraph Auto Loader
        Files --> Detection[File Detection]
        Detection --> Ingest[Incremental Ingestion]
        Ingest --> Schema[Schema Handling]
        Schema --> Output[Streaming DataFrame]
    end

    subgraph Target
        Output --> Delta[Delta Table]
    end
```

Overview

- **`format("cloudFiles")`** with `cloudFiles.format` incrementally ingests new files from cloud storage
- **File detection**: directory listing (default, no cloud setup) vs file notification mode (cloud events, scales to millions of files)
- **Schema handling**: `cloudFiles.schemaLocation` persists the inferred schema; evolution modes are `addNewColumns`, `rescue`, `failOnNewColumns`, and `none`
- **`availableNow=True`** gives scheduled, batch-style incremental ingestion with exactly-once semantics

What the Exam Tests

- Create production-ready data pipelines for streaming data using Lakeflow Pipelines and Auto Loader.
- Develop data ingestion pipelines to ingest a variety of data formats, including Delta Lake, Parquet, JSON, Iceberg, CSV, and Binary from diverse sources such as message buses

(Kafka, Kinesis, Pub/Sub) and cloud storage.
- Pick the schema evolution mode and file detection mode for a scenario

Why Auto Loader?

Challenge	Auto Loader Solution
Tracking processed files	Automatic checkpointing
Handling new file arrivals	Event-driven or polling
Schema changes	Schema inference and evolution
Corrupt files	Rescue data column
Scale	Handles millions of files

Basic Usage

```
# Basic Auto Loader read

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/path/to/schema")
    .load("/path/to/files"))

# Write to Delta

query = (df.writeStream
    .format("delta")
    .option("checkpointLocation", "/path/to/checkpoint")
    .start("/path/to/target"))
```

Supported File Formats

Format	cloudFiles.format
JSON	`json`
CSV	`csv`
Parquet	`parquet`
Avro	`avro`
ORC	`orc`
Text	`text`
Binary	`binaryFile`

Directory Listing vs File Notification

```
flowchart TD
    Mode[Detection Mode] --> DL[Directory Listing]
    Mode --> FN[File Notification]

    DL --> DL1[Periodically scans directory]
    DL --> DL2[No cloud setup required]
    DL --> DL3[Slower for large directories]

    FN --> FN1[Cloud events trigger processing]
```

```
    FN --> FN2[Requires cloud setup]
    FN --> FN3[Faster, more efficient]
```

Directory Listing Mode (Default)

```
# Uses directory listing by default

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.useNotifications", "false")  # Default
    .option("cloudFiles.schemaLocation", "/schema")
    .load("/path/to/files"))
```

Aspect	Directory Listing
How it works	Periodically lists directory contents
Setup	No additional cloud setup
Scalability	Slower with millions of files
Cost	Directory listing API calls
Best for	Small to medium file volumes

File Notification Mode

```
# Use file notification mode

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.useNotifications", "true")
    .option("cloudFiles.schemaLocation", "/schema")
    .load("/path/to/files"))
```

Aspect	File Notification
How it works	Cloud events (SNS/SQS, Event Grid, Pub/Sub)
Setup	Requires cloud infrastructure setup
Scalability	Highly scalable, constant time
Cost	Event notification costs
Best for	High-volume file ingestion

Cloud Provider Setup

Cloud	Service	Setup
AWS	S3 + SNS + SQS	Auto Loader creates queue
Azure	ADLS + Event Grid + Queue	Auto Loader creates resources
GCP	GCS + Pub/Sub	Auto Loader creates subscription

Mode Comparison

Factor	Directory Listing	File Notification
Setup complexity	None	Medium
Latency	Higher	Lower
Cost at scale	Higher (listing)	Lower
File volume	< 1M files	Unlimited
Incremental setup	Recommended start	Migrate when needed

Schema Inference

Auto Loader can automatically infer and track schema.

Schema Location

```
# Schema is stored and tracked at this location

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/path/to/schema")
    .load("/path/to/files"))
```

The schema location stores:

- Inferred schema
- Schema evolution history
- Partition information

Schema Inference Options

```
df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("cloudFiles.inferColumnTypes", "true")  # Infer types (not just
↪   strings)
    .option("cloudFiles.schemaHints", "id INT, amount DOUBLE")  # Override
↪   specific columns
    .load("/path/to/files"))
```

Option	Default	Description
`inferColumnTypes`	true	Infer actual types vs all strings
`schemaHints`	none	Override types for specific columns

Explicit Schema

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

# Define schema explicitly

schema = StructType([
    StructField("id", IntegerType(), True),
    StructField("name", StringType(), True),
    StructField("amount", DoubleType(), True)
])
```

```
df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .schema(schema)
    .load("/path/to/files"))
```

Schema Evolution Modes (Exam Critical)

```
flowchart TD
    SE[Schema Evolution Mode] --> Add[addNewColumns]
    SE --> Rescue[rescue]
    SE --> Fail[failOnNewColumns]
    SE --> None[none]

    Add --> A1[Auto-add new columns to schema]
    Rescue --> R1[Put unexpected data in _rescued_data]
    Fail --> F1[Fail on new columns]
    None --> N1[Ignore schema changes]
```

Schema Evolution Options

```
df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load("/path/to/files"))
```

Mode	Behavior	Use Case
addNewColumns	Add new columns automatically	Flexible schema sources
rescue	Store unexpected in _rescued_data	Data quality monitoring
failOnNewColumns	Fail stream on new columns	Strict schema enforcement
none	Ignore schema changes	Fixed schema, drop new columns

addNewColumns Mode

```
# New columns are automatically added to the schema

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load("/path/to/files"))
```

When a new field appears:

1. Schema is updated at schemaLocation
2. Streaming query may need restart to pick up new schema
3. New column added to output

Rescue Data Column

```python
# Enable rescue column for unexpected data

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("cloudFiles.schemaEvolutionMode", "rescue")
    .load("/path/to/files"))

# _rescued_data column contains JSON of unexpected fields
```

Data Scenario	Rescued Column Contains
New field in record	`{"new_field": "value"}`
Type mismatch	`{"field": "wrong_type_value"}`
Nested schema change	Full unexpected nested structure

Combining Evolution and Rescue

```python
# Add new columns AND rescue corrupt data

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .option("rescuedDataColumn", "_rescued_data")
    .load("/path/to/files"))
```

Key Configuration Options

File Processing Options

```python
df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    # File selection
    .option("cloudFiles.includeExistingFiles", "true")  # Process existing files
    .option("pathGlobFilter", "*.json")  # Filter by pattern
    # Rate limiting
    .option("cloudFiles.maxFilesPerTrigger", 1000)
    .option("cloudFiles.maxBytesPerTrigger", "10g")
    .load("/path/to/files"))
```

Option	Default	Description
`includeExistingFiles`	true	Process files present at start
`pathGlobFilter`	none	Glob pattern to filter files
`maxFilesPerTrigger`	1000	Max files per micro-batch
`maxBytesPerTrigger`	none	Max bytes per micro-batch

Format-Specific Options

```
# JSON options

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("multiLine", "true")
    .option("primitivesAsString", "false")
    .load("/path/to/files"))

# CSV options

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("delimiter", ",")
    .option("inferSchema", "true")
    .load("/path/to/files"))
```

Handling Corrupt Records

Bad Records Path

```
df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("badRecordsPath", "/path/to/bad_records")
    .load("/path/to/files"))
```

Bad records are written to the specified path with:

- Original file content
- Error message
- Timestamp

Corrupt Record Column

```
# Add column for unparseable records

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("columnNameOfCorruptRecord", "_corrupt_record")
    .load("/path/to/files"))
```

Auto Loader with Unity Catalog

Unity Catalog integration affects how Auto Loader handles credentials, locations, and schema storage.

Ingesting to Managed Tables

```
# Read with Auto Loader

df = (spark.readStream
```

```
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .load("/path/to/source/files"))

# Write to Unity Catalog managed table

query = (df.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoint")
    .toTable("catalog.schema.table_name"))
```

Reading from Volumes

```
# Read from Unity Catalog Volume

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "parquet")
    .option("cloudFiles.schemaLocation", "/Volumes/catalog/schema/volume/schema")
    .load("/Volumes/catalog/schema/volume/data/"))
```

External Locations Writing to external locations requires appropriate Unity Catalog permissions.

```
# Reading from external location (requires READ FILES permission)

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "abfss://container@storage/schema/")
    .load("abfss://container@storage/landing/orders/"))

# Writing to external location (requires WRITE FILES permission)

query = (df.writeStream
    .format("delta")
    .option("checkpointLocation", "abfss://container@storage/checkpoint/")
    .start("abfss://container@storage/bronze/orders/"))
```

UC Permissions for Auto Loader

Operation	Required Permission
Read from external location	`READ FILES` on external location
Write to external location	`WRITE FILES` on external location
Write to managed table	`USE SCHEMA`, `CREATE TABLE`
Schema location (external)	`WRITE FILES` on external location
Schema location (volume)	`WRITE VOLUME`

Cross-Catalog Patterns

```
# Read from one catalog, write to another

source_df = (spark.readStream
    .format("cloudFiles")
```

```
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation",
↪   "/Volumes/source_catalog/schema/vol/schema")
    .load("/Volumes/source_catalog/schema/vol/data/"))

# Write to different catalog (requires permissions on both)

query = (source_df.writeStream
    .format("delta")
    .option("checkpointLocation",
↪   "/Volumes/target_catalog/schema/vol/checkpoint/")
    .toTable("target_catalog.target_schema.target_table"))
```

Schema Evolution with UC When using Unity Catalog, schema evolution respects table-level settings:

```
# UC tables may have schema enforcement

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load("/source/"))

# mergeSchema allows Auto Loader to add columns to UC table

query = (df.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoint")
    .option("mergeSchema", "true")  # Required for schema evolution
    .toTable("catalog.schema.table"))
```

Credential Management Auto Loader uses Unity Catalog credentials automatically:

- **Managed tables**: No explicit credentials needed
- **External locations**: UC external location credentials used
- **Volumes**: Volume storage credentials used

```
# No credential configuration needed - UC handles it

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/Volumes/catalog/schema/vol/schema")
    .load("/Volumes/catalog/schema/vol/data/"))  # UC credentials automatic
```

Common Patterns

Bronze Layer Ingestion

```
# Ingest raw data to Bronze layer

bronze_df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
```

```
    .option("cloudFiles.schemaLocation", "/bronze/schema/orders")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .option("rescuedDataColumn", "_rescued_data")
    .load("/landing/orders/"))

# Add metadata columns

bronze_with_metadata = (bronze_df
    .withColumn("_ingestion_timestamp", current_timestamp())
    .withColumn("_source_file", input_file_name()))

# Write to Bronze Delta table

query = (bronze_with_metadata.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoints/bronze_orders")
    .option("mergeSchema", "true")
    .toTable("bronze.orders"))
```

Multi-Format Ingestion

```
# Function to create Auto Loader stream for different formats

def create_ingestion_stream(format, source_path, schema_path):
    return (spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", format)
        .option("cloudFiles.schemaLocation", schema_path)
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .load(source_path))

# Create streams for different sources

json_stream = create_ingestion_stream("json", "/data/json/", "/schema/json/")
csv_stream = create_ingestion_stream("csv", "/data/csv/", "/schema/csv/")
parquet_stream = create_ingestion_stream("parquet", "/data/parquet/",
 ↪  "/schema/parquet/")
```

Scheduled Batch with availableNow

```
# Run as scheduled job

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("cloudFiles.maxFilesPerTrigger", 10000)
    .load("/source/path"))

query = (df.writeStream
    .format("delta")
    .trigger(availableNow=True)
    .option("checkpointLocation", "/checkpoint")
    .start("/target/path"))

query.awaitTermination()
```

Performance Tuning

Parallelism

```
# Increase parallelism for large files

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("maxFilesPerTrigger", 10000)
    .option("spark.sql.files.maxPartitionBytes", "128mb")
    .load("/path/to/files"))
```

Memory Optimization

```
# For very large schemas or many files

spark.conf.set("spark.databricks.cloudFiles.schemaInference.sampleSize.numFiles",
 ↪  100)
spark.conf.set("spark.databricks.cloudFiles.schemaInference.sampleSize.numBytes",
 ↪  "10mb")
```

Backlog Processing

```
# For processing large backlogs efficiently

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "parquet")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("maxFilesPerTrigger", 10000)
    .option("maxBytesPerTrigger", "100g")
    .load("/path/to/files"))

# Use availableNow to process all then stop

query = (df.writeStream
    .trigger(availableNow=True)
    .option("checkpointLocation", "/checkpoint")
    .start("/target"))
```

SQL Interface

```
-- Create streaming table with Auto Loader
CREATE OR REFRESH STREAMING TABLE bronze_orders
AS SELECT
    *,
    _metadata.file_path AS source_file,
    _metadata.file_modification_time AS file_mod_time
FROM STREAM read_files(
    '/landing/orders/',
    format => 'json',
    schemaLocation => '/schema/orders'
);
```

Troubleshooting

Common Issues

Issue	Cause	Solution
Schema mismatch	Source schema changed	Check schemaLocation, restart query
Missing files	Files processed already	Check checkpoint, use includeExistingFiles
Slow processing	Too many small files	Increase maxFilesPerTrigger
OOM errors	Large files or schema	Reduce maxBytesPerTrigger

Debugging

```
# Check Auto Loader metrics

query = (df.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoint")
    .start("/target"))

# View progress

print(query.lastProgress)
print(query.status)
```

Use Cases

Scenario	Recommended Configuration	Why?
Ingesting Millions of Files	`useNotifications=true`	Avoids slow directory listing and sc
Evolving Schema Sources	`schemaEvolutionMode="addNewColumns"`	Automatically adapts to upstream
Data Quality Monitoring	`schemaEvolutionMode="rescue"`	Captures bad data in `_rescued_da`
Unknown Schema	`cloudFiles.inferColumnTypes=true`	Detects proper types (Int, Double)

Common Issues & Errors

OutOfMemoryError (Driver) **Scenario:** Default `maxFilesPerTrigger` lists too many files for driver memory.

Fix: Reduce `cloudFiles.maxFilesPerTrigger` (e.g., to 1000) or `maxBytesPerTrigger`.

Schema Inference Sampling Miss **Scenario:** Auto Loader inferred a column as Integer, but later a float value appeared, causing failure.

Fix: Use `cloudFiles.schemaHints` to enforce `"col double"` or increase sampling size/files.

File Notification Setup Failure **Scenario:** Missing cloud permissions to create Queues/Topics.

Fix: Ensure Service Principal/Role has permission to create resources, or have admin pre-provision them.

"Path does not exist" on Schema Location **Scenario:** Logic changed but schema location still points to old schema state.

Fix: If schema logic changed fundamentally, delete the schema location directory to reset inference (caution: resets evolution history).

Best Practices

- Start with directory listing, switch to file notification at scale
- Always set schemaLocation for schema tracking
- Use `addNewColumns` for flexible sources, `rescue` for data quality monitoring
- Add `_source_file` and `_ingestion_timestamp` metadata columns
- Use `availableNow` for scheduled batch-style processing
- Set appropriate rate limits to prevent OOM
- Store schema and checkpoint in cloud storage for durability

Exam Tips

1. **Directory listing vs file notification** - Know when to use each
2. **Schema evolution modes** - `addNewColumns`, `rescue`, `failOnNewColumns`, `none`
3. **schemaLocation** is required for schema inference
4. **_rescued_data** column stores unexpected fields and type mismatches
5. **includeExistingFiles** controls whether existing files are processed
6. **maxFilesPerTrigger** controls processing rate
7. Auto Loader automatically handles **exactly-once** processing with checkpoints
8. **Unity Catalog** - Use Volumes for schema location and data paths
9. **UC permissions** - `READ FILES` / `WRITE FILES` required for external locations
10. **mergeSchema** option required when Auto Loader adds columns to UC tables

Key Takeaways

- **Auto Loader format**: use `format("cloudFiles")` with the `cloudFiles.format` option to specify the file type (json, csv, parquet, avro, etc.)
- **Directory listing** (default) periodically scans the source path and requires no cloud setup; **File Notification** mode uses cloud events (SNS/SQS, Event Grid, Pub/Sub) and scales to millions of files with lower latency
- **`cloudFiles.schemaLocation`** is required for schema inference and tracks the inferred schema and its evolution history; without it, Auto Loader cannot persist schema between runs
- **Schema evolution modes**: `addNewColumns` auto-expands the schema; `rescue` stores unexpected fields in `_rescued_data`; `failOnNewColumns` halts the stream; `none` silently drops new columns
- **`_rescued_data`** column captures fields that do not match the current schema (new fields or type mismatches) when using `rescue` mode
- **`includeExistingFiles`** (default `true`) determines whether files already present at the source path when the stream starts are processed
- **Unity Catalog integration**: read from Volumes or external locations requires `READ FILES` permission; writing to external locations requires `WRITE FILES`; `mergeSchema=true` is required on the write side when Auto Loader adds columns to a UC table
- **`availableNow=True`** trigger combined with Auto Loader is the standard pattern for scheduled batch-style incremental file ingestion with exactly-once semantics

Related Topics

- Structured Streaming - Streaming fundamentals
- Incremental Processing - Incremental patterns
- Change Data Capture - CDC from files

Official Documentation

- Auto Loader

- Auto Loader Options
- Schema Evolution

COPY INTO

Overview

`COPY INTO` is the SQL command for **idempotent batch ingestion** from cloud object storage into a Delta table. It tracks every file it has ever loaded into the target table's history, so re-running with the same source path is a no-op for already-processed files. It complements **Auto Loader** (continuous / streaming) and the **UI uploader** (one-off small files) by filling the "scheduled batch ingest of a few-to-many files" slot.

Overview

- **Idempotent**: each file is loaded at most once, tracked in the target table's Delta transaction log (commit log)
- **Schema-flexible**: `FORMAT_OPTIONS('inferSchema'='true')` infers; `mergeSchema='true'` evolves; declared target schema validates
- **Supports CSV / JSON / Parquet / Avro / ORC / TEXT / BINARYFILE**
- **Pattern-based**: `FILES` (explicit list) or `PATTERN` (glob/regex) restricts what's loaded
- **Force reload** with `COPY_OPTIONS('force' = 'true')` (or the docs' canonical `force = true` boolean inside `COPY_OPTIONS`) re-loads everything (use with care — duplicates if MERGE is not used)

What the Exam Tests

- `COPY INTO` vs Auto Loader: when to pick which (scheduled batch vs continuous incremental)
- The idempotency guarantee — re-running is safe by default
- How `FORMAT_OPTIONS` (parsing) differs from `COPY_OPTIONS` (load behaviour)
- That `PATTERN` and `FILES` let you control what gets ingested without changing the source path
- That `COPY INTO` reads from object storage paths (s3://, abfss://, gs://) or UC volumes — not from Delta tables

---

Anatomy of a `COPY INTO` statement

```
COPY INTO main.bronze.orders
FROM '/Volumes/main/landing/orders/'
FILEFORMAT = JSON
FORMAT_OPTIONS (
  'inferSchema' = 'true',
  'mergeSchema' = 'true'
)
COPY_OPTIONS (
  'mergeSchema' = 'true'
);
```

Clause	Purpose
`FROM <path>`	Cloud-storage path or UC volume path

Clause	Purpose
FILEFORMAT	One of CSV, JSON, PARQUET, AVRO, ORC, TEXT, BINARYFILE
FORMAT_OPTIONS	Parser options — inferSchema, header, delimiter, dateFormat, etc.
COPY_OPTIONS	Load behaviour — mergeSchema, force, validate
FILES (optional)	Explicit comma-separated file names to load (subset of the path)
PATTERN (optional)	Glob / regex on file names (e.g., *.json, orders_2026*.parquet)

Idempotency in practice

```
-- First run: loads 100 files
COPY INTO main.bronze.orders
FROM '/Volumes/main/landing/orders/'
FILEFORMAT = JSON;
-- Result: 100 files loaded

-- 5 minutes later, 3 new files arrive. Re-run:
COPY INTO main.bronze.orders
FROM '/Volumes/main/landing/orders/'
FILEFORMAT = JSON;
-- Result: 3 new files loaded; 100 originals skipped automatically
```

Databricks tracks the files-already-loaded set in the target table's commit log. You don't need an external watermark or processed-files table.

When to use COPY INTO vs Auto Loader

Scenario	Pick
One-time bulk load of historical data	COPY INTO (simpler, no streaming infra)
Scheduled daily / hourly batch from a known prefix	COPY INTO from a Lakeflow Job
Continuous low-latency ingest as files land	**Auto Loader** with cloudFiles streaming source
Millions of small files arriving every minute	**Auto Loader** in file-notification mode
Ad-hoc one-off CSV from a colleague	UI uploader (smaller blast radius than SQL)

Use Cases

- **Bronze ingestion in a medallion pipeline** — COPY INTO from raw-zone object storage to a Bronze Delta table
- **Reprocessing a specific date partition** — combine PATTERN with force = true to reload a known set
- **Validation before commit** — COPY_OPTIONS('validate'='true') parses files without writing, surfacing schema or format errors
- **Multi-cloud landing zones** — point at any S3 / ADLS / GCS path; UC connections supply credentials

Common Issues & Errors

- **Schema mismatch without mergeSchema** — COPY INTO defaults to schema validation; new columns in source cause errors unless mergeSchema = true
- **force = true creates duplicates** — when re-loading processed files, all rows are appended again unless the target uses a MERGE step downstream
- **DBFS mount paths are deprecated** — use UC volume paths (/Volumes/...) for new pipelines

- **`COPY INTO` reading Delta** — not supported; `COPY INTO` reads *files*, not other Delta tables. For Delta-to-Delta copies use `INSERT INTO ... SELECT FROM`, `CREATE TABLE ... AS SELECT`, or a Lakeflow Declarative Pipeline with a streaming-table source
- **`PATTERN` matches against full path, not just basename** — be explicit, e.g., `PATTERN = '*.parquet'` matches files anywhere under the source path

Exam Tips

Tip

- **Idempotent by default.** Re-running the same statement is safe.
- `FORMAT_OPTIONS` is **parsing**; `COPY_OPTIONS` is **load behaviour**. Don't confuse them.
- Reads **files**, not Delta tables. For Delta sources use SQL `INSERT INTO ... SELECT`.
- Prefer **UC volume paths** over DBFS mounts in new pipelines.
- `force = true` bypasses the de-duplication tracking — use only when you genuinely want to re-load everything.

Key Takeaways

- `COPY INTO` = idempotent batch ingest from object storage to a Delta table
- Tracks loaded files in the target table's commit log automatically
- Pair with Lakeflow Jobs for scheduled batch ingest; pair with Auto Loader for streaming
- `FORMAT_OPTIONS` (parser) vs `COPY_OPTIONS` (behaviour) — know the difference

Related Topics

- Auto Loader — the streaming-style sibling
- Streaming Ingestion from Message Buses — for non-file sources
- Lakeflow Jobs — scheduling the batch load

Official Documentation

- `COPY INTO` SQL reference
- `COPY INTO` patterns and examples
- Unity Catalog volumes

Streaming Ingestion from Message Buses

Overview

Object-storage ingestion covers files; message buses — **Apache Kafka** (including Confluent, Amazon MSK, Google Cloud Managed Kafka), **Azure Event Hubs**, **Amazon Kinesis**, and **Google Pub/Sub** — cover events. Databricks reads them as Structured Streaming sources (`kafka`, `kinesis`, `pubsub`) or SQL table-valued functions (`read_kafka`, `read_kinesis`, `read_pubsub`) and writes to Delta with checkpointed `writeStream` or Lakeflow Pipelines streaming tables. This page also maps every **data format** in the exam bullet (Delta Lake, Parquet, JSON, Iceberg, CSV, Binary) to the right ingestion API.

Overview

- **Kafka** (`format("kafka")`) — also the path for **Event Hubs** through its Kafka-compatible endpoint (port 9093); a managed **Lakeflow Connect Kafka connector** is in Beta

- **Kinesis** (`format("kinesis")`) — `streamName` or `streamARN`; polling or enhanced fan-out (`consumerMode = "efo"`)
- **Pub/Sub** (`format("pubsub")`) — `subscriptionId` + `topicId` + `projectId` required; exactly-once processing of fetched rows, but the bus itself can deliver duplicates
- Message payloads arrive as **binary** (`value`, `data`, `payload`) and must be decoded (`cast("string")`, `from_json`, `from_avro`)
- **Exactly-once into Delta** comes from a replayable source + a unique checkpoint per query; credentials come from **UC service credentials / connections** or secrets, never hard-coded

What the Exam Tests

- "Develop data ingestion pipelines to ingest a variety of data formats, including Delta Lake, Parquet, JSON, Iceberg, CSV, and Binary from diverse sources such as message buses (Kafka, Kinesis, Pub/Sub) and cloud storage"
- Required options per source and their defaults (`startingOffsets`, `initialPosition`, `maxOffsetsPerTrigger`)
- Which reader handles which format: Auto Loader `cloudFiles.format` values, `binaryFile`, Delta streaming reads, Iceberg read paths
- Checkpoints for exactly-once; Lakeflow Pipelines (declarative) vs raw `writeStream` (full control) vs a managed connector

---

Source × Format Matrix

Source	Reader
Kafka / Event Hubs	`spark.readStream.format("kafka")`, `read_kafka()`
Kinesis	`format("kinesis")`, `read_kinesis()`
Pub/Sub	`format("pubsub")`, `read_pubsub()`
Cloud storage files	**Auto Loader** (`cloudFiles`), `read_files()`, `COPY INTO`
Delta Lake tables	`spark.readStream.table(...)` / `format("delta")`
Iceberg tables	`spark.read.table(...)` on **managed Iceberg** (UC) or **foreign Iceberg** (catalog feder
OpenSharing	`format("deltaSharing")`

Note

Delta and Iceberg are *table* formats — they are read through table APIs, not through Auto Loader. Auto Loader and `read_files` ingest raw **files** (JSON, CSV, Parquet, Avro, ORC, text, XML, binary).

Kafka

```
from pyspark.sql.functions import col, from_json
from pyspark.sql.types import LongType, StringType, StructType

schema = StructType().add("order_id", StringType()).add("amount", LongType())

jaas = (
    'kafkashaded.org.apache.kafka.common.security.plain.PlainLoginModule required
     ↪  '
```

```
    f'username="{dbutils.secrets.get("kafka", "user")}" '
    f'password="{dbutils.secrets.get("kafka", "pwd")}";'
)

events = (
    spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "broker.example.com:9093")
    .option("subscribe", "orders")
    .option("startingOffsets", "earliest")
    .option("maxOffsetsPerTrigger", 100000)
    .option("kafka.security.protocol", "SASL_SSL")
    .option("kafka.sasl.mechanism", "PLAIN")
    .option("kafka.sasl.jaas.config", jaas)
    .load()
    .select(
        from_json(col("value").cast("string"), schema).alias("data"),
        col("timestamp").alias("ingest_ts"),
    )
    .select("data.*", "ingest_ts")
)

(events.writeStream
    .format("delta")
    .option("checkpointLocation", "/Volumes/main/landing/_checkpoints/orders")
    .outputMode("append")
    .toTable("main.bronze.orders"))
```

Option	Default	Purpose
`kafka.bootstrap.servers`	—	Broker `host:p`
`subscribe` / `subscribePattern` / `assign`	—	Exactly one to
`startingOffsets`	`latest` (streaming), `earliest` (batch)	Only applies w
`startingTimestamp`, `startingOffsetsByTimestamp`	—	Start from a p
`maxOffsetsPerTrigger`	—	Cap offsets pe
`minOffsetsPerTrigger` + `maxTriggerDelay`	— / `15m`	Wait to accum
`minPartitions`, `maxRecordsPerPartition`	—	Split skewed K
`failOnDataLoss`	`true`	Fail if offsets w
`includeHeaders`	`false`	Add Kafka hea

Schema: key and value (`binary`), `topic`, `partition`, `offset`, `timestamp`, `timestampType`.

Authentication:

- **UC service credential** (DBR 16.1+, recommended on standard-access clusters and serverless) for cloud-managed Kafka — Amazon MSK, Azure Event Hubs, Google Cloud Managed Kafka: set `databricks.serviceCredential`; no SASL/JAAS options needed.
- Otherwise SASL/SSL options passed as `kafka.*` properties (`SASL_SSL` + `PLAIN`, `SCRAM-SHA-256/512`, `OAUTHBEARER`, `AWS_MSK_IAM`, or SSL keystores).
- Any Kafka class referenced in options must use the **shaded** prefix `kafkashaded.`; unshaded names raise `RESTRICTED_STREAMING_OPTION_PERMISSION_ENFORCED`.

```
msk_stream = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "<bootstrap-hostname>:9092")
    .option("subscribe", "orders")
```

```
    .option("databricks.serviceCredential", "<service-credential-name>")
    .load())
```

Azure Event Hubs Event Hubs exposes an Apache Kafka-compatible interface, so you use the **Kafka source** with `kafka.bootstrap.servers` set to `<event-hubs-server>:9093` (the namespace server shown on the Event Hubs overview page). On Azure Databricks, authenticate with a **UC service credential** (DBR 16.1+; optionally `databricks.serviceCredential.scope`) or Microsoft Entra ID OAuth with a client ID and secret. Lakeflow Pipelines use the same Kafka connector.

Managed alternative: Lakeflow Connect Kafka connector (Beta) The managed Kafka connector stores the bootstrap servers and credentials in a **UC connection** and runs a serverless ingestion pipeline that writes streaming tables — no Structured Streaming code. Use raw Structured Streaming when you need custom offset handling or per-batch transformations. Lakeflow Connect also has a managed **RabbitMQ** connector (Beta); Kinesis, Pub/Sub, and Pulsar have no managed connector and are read with Structured Streaming or Lakeflow Pipelines.

Kinesis

```
from pyspark.sql.functions import col

kinesis = (
    spark.readStream
    .format("kinesis")
    .option("databricks.connection", "kinesis_conn")   # UC connection, DBR 19+
    .option("streamName", "orders-stream")
    .option("initialPosition", "trim_horizon")
    .load()
    .select(col("data").cast("string").alias("json"),
 ↪  "approximateArrivalTimestamp")
)
```

Option	Default	Notes
`streamName` **or** `streamARN`	—	Exactly one; `streamARN` ne
`initialPosition`	`latest`	`latest`, `trim_horizon` (=
`consumerMode`	`polling`	`efo` = enhanced fan-out, c
`region`	Locally resolved	Don't set with `databricks`
`maxFetchRate`, `minFetchPeriod`, `maxRecordsPerFetch`	1.0 MB/s, 400ms, 10000	Throttling controls
`shardsPerTask`	5	Shards prefetched per Spa

Authentication (preferred first): a **UC connection** of type `KINESIS` that references a service credential (`databricks.connection`, DBR 19+); a **service credential** (`serviceCredential`, DBR 16.1+); or instance profile, `roleArn`, or access keys (`awsAccessKey`/`awsSecretKey` from secrets). Combining `databricks.connection` with `region`, `roleArn`, or access-key options fails with `KINESIS_UC_CONNECTION_OPTION_CONFLICT`.

```
CREATE CONNECTION IF NOT EXISTS kinesis_conn TYPE KINESIS
OPTIONS (aws_region 'us-east-1', credential 'kinesis_service_cred');

GRANT USE CONNECTION ON CONNECTION kinesis_conn TO `stream-engineers`;
GRANT ACCESS ON SERVICE CREDENTIAL kinesis_service_cred TO `stream-engineers`;
```

Schema: `partitionKey`, `data` (binary), `stream`, `shardId`, `sequenceNumber`, `approximateArrivalTimestamp`. `Trigger.AvailableNow` is supported (DBR 13.3 LTS+) for incremental-batch runs.

Google Pub/Sub

```
auth_options = {
    "clientId": dbutils.secrets.get("pubsub", "clientId"),
    "clientEmail": dbutils.secrets.get("pubsub", "clientEmail"),
    "privateKey": dbutils.secrets.get("pubsub", "privateKey"),
    "privateKeyId": dbutils.secrets.get("pubsub", "privateKeyId"),
}

pubsub = (
    spark.readStream
    .format("pubsub")
    .option("subscriptionId", "orders-sub")   # created if it doesn't exist
    .option("topicId", "orders")
    .option("projectId", "my-project")
    .options(**auth_options)
    .load()
)
```

Option	Default	Notes
`subscriptionId`, `topicId`, `projectId`	—	**Required**
`serviceCredential`	—	Databricks service
`clientEmail`, `clientId`, `privateKey`, `privateKeyId`	—	Required when not
`numFetchPartitions`	Half the executors at stream start	Parallel fetch tasks
`maxRecordsPerFetch`	`1000`	Rows fetched per t
`maxBytesPerTrigger`	—	Soft limit per micro
`deleteSubscriptionOnStreamStop`	`false`	Needs `roles/pubs`

- **IAM roles**: `roles/pubsub.viewer` (or `roles/viewer`) and `roles/pubsub.subscriber` are required; `roles/pubsub.editor` is optional (create/delete subscriptions).
- **Schema**: `messageId`, `payload` (raw bytes), `attributes` (JSON string), `publishTimestampInMillis`.
- **Semantics**: the connector processes subscriber rows exactly once, but Pub/Sub can publish duplicates or deliver out of order — deduplicate downstream (for example on `messageId`).
- `Trigger.AvailableNow` processes everything published before the recorded start time; `spark.speculation` is not supported.

Cloud Storage Formats

Auto Loader for files

```
bronze_json = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/Volumes/main/landing/_schemas/orders")
    .option("cloudFiles.inferColumnTypes", "true")
    .load("/Volumes/main/landing/orders/"))
```

```python
bronze_csv = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("cloudFiles.schemaLocation",
↪   "/Volumes/main/landing/_schemas/customers")
    .load("/Volumes/main/landing/customers/"))
```

- `cloudFiles.format` is **required**: `avro`, `binaryFile`, `csv`, `json`, `orc`, `parquet`, `text`, `xml`.
- `cloudFiles.schemaLocation` is required for schema inference; JSON and CSV columns infer as strings unless `cloudFiles.inferColumnTypes` is `true`.
- `cloudFiles.schemaEvolutionMode`: `addNewColumns` (default without a schema), `rescue`, `failOnNewColumns`, `none`.
- Full Auto Loader coverage: Auto Loader; batch idempotent loads: COPY INTO.

Binary files

```python
images = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "binaryFile")
    .option("pathGlobFilter", "*.png")   # the path is only a prefix filter
    .load("/Volumes/main/raw/images/"))
```

The `binaryFile` source returns one row per file with a fixed schema: `path`, `modificationTime`, `length`, `content` (`BINARY`). Use `pathGlobFilter` for suffixes and `recursiveFileLookup` to ignore partition-style directories; batch reads use `spark.read.format("binaryFile")` or `read_files(path, format => 'binaryFile')`. Databricks recommends saving binary data to a Delta table for faster reads.

Delta Lake as a streaming source

```python
silver_feed = (spark.readStream
    .option("skipChangeCommits", "true")   # ignore updates/deletes, process
↪   appends only
    .table("main.bronze.orders"))

cdf_feed = (spark.readStream
    .option("readChangeFeed", "true")
    .option("startingVersion", 5)
    .table("main.silver.customers"))
```

`skipChangeCommits` replaces `ignoreChanges` on DBR 12.2 LTS+; `startingVersion` and `startingTimestamp` are mutually exclusive and only apply to new queries.

Iceberg

- **Managed Iceberg tables** in Unity Catalog (DBR 16.4 LTS+) are read like any UC table; external engines read and write them through the Iceberg REST Catalog.
- **Foreign Iceberg tables** registered in Glue, Hive metastore, or Snowflake Horizon are read through **catalog federation** — read-only, metadata from the external catalog. See Lakehouse Federation.
- **Converting** an Iceberg table directory to Delta in place: `CONVERT TO DELTA iceberg.\s3://bucket/path“` (paths only; managed Iceberg tables can't be converted).

```sql
-- Batch read of a managed or foreign Iceberg table, landed into a Delta bronze
↪   table
CREATE OR REPLACE TABLE main.bronze.vendor_orders AS
SELECT * FROM glue_catalog.sales.orders;
```

Lakeflow Pipelines Wrapper

For declarative ingestion with expectations, dependency tracking, managed checkpoints, retries, and the pipeline event log:

```python
from pyspark import pipelines as dp
from pyspark.sql.functions import col, from_json

@dp.table(name="bronze_orders", comment="Raw Kafka events")
@dp.expect_or_drop("valid_amount", "amount > 0")
def bronze_orders():
    return (
        spark.readStream.format("kafka")
        .option("kafka.bootstrap.servers", spark.conf.get("kafka.servers"))
        .option("subscribe", "orders")
        .load()
        .select(from_json(col("value").cast("string"), "order_id STRING, amount
 ↪  BIGINT").alias("data"))
        .select("data.*")
    )
```

```sql
CREATE OR REFRESH STREAMING TABLE kafka_raw AS
SELECT * FROM STREAM read_kafka(
  bootstrapServers => 'kafka_server:9092',
  subscribe => 'topic1'
);

CREATE OR REFRESH STREAMING TABLE pubsub_raw AS
SELECT * FROM STREAM read_pubsub(
  subscriptionId => 'my-subscription',
  projectId => 'my-project',
  topicId => 'my-topic',
  clientEmail => secret('pubsub-scope', 'clientEmail'),
  clientId => secret('pubsub-scope', 'clientId'),
  privateKeyId => secret('pubsub-scope', 'privateKeyId'),
  privateKey => secret('pubsub-scope', 'privateKey')
);
```

For low latency, run these streaming tables in a **continuous** pipeline; `read_kinesis` and `read_pulsar` follow the same pattern.

Exactly-once Semantics

Delta + checkpointing gives you exactly-once **into Delta**:

1. The streaming source records its progress (offsets, sequence numbers) in `checkpointLocation`.
2. The Delta sink commits each micro-batch atomically and records the batch in its log.
3. On restart the query resumes from the checkpoint, and Delta skips batches it already committed.

Caveats: the source must be replayable from a known position (Kafka offsets, Kinesis sequence numbers); a bus that delivers duplicates (Pub/Sub) still needs deduplication; side effects in UDFs or `foreachBatch` writes to other systems are not exactly-once unless made idempotent.

Use Cases

- **Kafka → bronze streaming table** as the first hop of a medallion pipeline
- **Debezium CDC → Kafka → Lakeflow Pipelines → silver** with the AUTO CDC APIs (formerly APPLY CHANGES)
- **Clickstream from Kinesis (EFO) → bronze → near-real-time dashboards**
- **IoT telemetry through Event Hubs' Kafka endpoint → bronze**
- **GCP event fan-in from Pub/Sub** with `messageId` deduplication
- **Images/PDFs via Auto Loader `binaryFile` → Delta** for ML or AI-function processing

Common Issues & Errors

Symptom	Cause
First run skips existing Kafka events	Streaming default `startingOffsets = latest`
Changing `startingOffsets` has no effect	Query resumes from its checkpoint
`RESTRICTED_STREAMING_OPTION_PERMISSION_ENFORCED`	Unshaded Kafka class in `kafka.sasl.jaas.config`
Two queries corrupt each other	Shared checkpoint location
Kinesis throttling	Polling consumers share per-shard read throughput
`KINESIS_UC_CONNECTION_OPTION_CONFLICT`	`region`/keys/role set alongside `databricks.connec`
Duplicate Pub/Sub rows	Pub/Sub at-least-once delivery
Hard-coded secrets in `spark.conf`	Fails security review, rotates badly
Hot Spark partition	Skewed Kafka partition key

Exam Tips

Tip

- **Sources to know**: `kafka` (also Event Hubs on port 9093), `kinesis`, `pubsub`; SQL: `read_kafka`, `read_kinesis`, `read_pubsub`.
- Payloads are **binary** — decode with `cast("string")` + `from_json` (or `from_avro`).
- `startingOffsets = latest` is the **streaming** default for Kafka (`earliest` for batch); Kinesis `initialPosition` defaults to `latest`; both apply only to a fresh checkpoint.
- Pub/Sub needs `subscriptionId`, `topicId`, `projectId`; it can deliver duplicates.
- Auto Loader formats: `avro`, `binaryFile`, `csv`, `json`, `orc`, `parquet`, `text`, `xml`. **Delta and Iceberg are read as tables**, not with Auto Loader.
- Checkpoint + Delta = exactly-once; credentials from UC service credentials/connections or secrets.

Key Takeaways

- Message-bus ingestion uses Structured Streaming sources (`kafka`, `kinesis`, `pubsub`) or their SQL TVFs inside Lakeflow Pipelines streaming tables.
- Event Hubs is consumed through the Kafka connector; a managed Lakeflow Connect Kafka connector (Beta) removes the code entirely.
- Each source has its own required options and defaults; message bodies are binary and must be decoded.
- File formats go through Auto Loader / `read_files` / `COPY INTO`; Delta streams via `read-Stream.table`; Iceberg via managed or foreign (federated) tables.
- Delta plus a unique checkpoint per query provides exactly-once into the lakehouse.

Related Topics

- Auto Loader — file ingestion in depth

- COPY INTO — idempotent batch file ingestion
- Change Data Capture (CDC) — Part 1 — CDF and change streams
- Lakeflow Connect Database CDC — managed database ingestion
- Lakehouse Federation — foreign Iceberg tables via catalog federation
- Structured Streaming — Part 1
- Lakeflow Pipelines

Official Documentation

- Connect to Apache Kafka
- Kafka authentication and Azure Event Hubs authentication (Azure Databricks)
- Connect to Amazon Kinesis and Kinesis authentication
- Subscribe to Google Pub/Sub
- Spark API options reference (Kafka, Kinesis, Pub/Sub, Auto Loader options)
- Streaming connectors in Lakeflow Connect and Kafka connector (Beta)
- Load data in pipelines
- `read_kafka`, `read_kinesis`, `read_files`
- What is Auto Loader? and Read binary files
- Delta Lake table streaming reads and writes
- What is Apache Iceberg in Databricks? and `CONVERT TO DELTA`

Change Data Capture (CDC) — Part 1

Overview

Change data capture (CDC) propagates row-level inserts, updates, and deletes instead of full reloads. This part covers CDC fundamentals, Delta **Change Data Feed (CDF)** (how to turn it on, read it, and how retention limits it), applying changes with `MERGE`, and the declarative alternative, **AUTO CDC**, in Lakeflow Pipelines. Part 2 covers Delta vs Iceberg targets, multi-hop CDC, and external CDC formats.

Platform update (Oct 2026)

Databricks now documents two CDF approaches. **Legacy change data feed** is the per-table `delta.enableChangeDataFeed = true` property tested on the exam (Delta only). **Automatic change data feed** became generally available on Sep 1, 2026: it computes changes at read time from row tracking (Delta) or row lineage (Iceberg v3), requires Databricks Runtime 19+, needs no per-table property, and uses the same `table_changes()` / `readChangeFeed` APIs. Legacy and automatic CDF can't be used together on one table. Sources: Use change data feed on Databricks, September 2026 release notes

Overview

- **CDF** records row-level changes between table versions with `_change_type` (`insert`, `update_preimage`, `update_postimage`, `delete`), `_commit_version`, and `_commit_timestamp`
- **Read CDF** in SQL with `table_changes('t', start[, end])` or in PySpark with `.option("readChangeFeed", "true")`, batch or streaming
- **CDF is not retroactive and not permanent**: only changes after it was enabled are recorded, and only within the table's retention window (`VACUUM` removes old change files)
- **Applying changes**: hand-written `MERGE` (Structured Streaming + `foreachBatch`) or declarative **AUTO CDC** in Lakeflow Pipelines

What the Exam Tests

- Apply Change Data Feed (CDF) to expose row-level changes (updates/deletes) for efficient incremental downstream processing
- Configure incremental CDC pipelines using Lakeflow Pipelines with Delta or Iceberg as the target table format
- Pick the right `_change_type` rows for a downstream task (for example, `update_postimage` for new values, `delete` to propagate removals)
- Recognize retention limits: a stream or batch read fails when the requested version is no longer in history

---

CDC Fundamentals

```
flowchart LR
    subgraph Sources
        DB[(OLTP database)]
        DT[(Delta table with CDF)]
        Snap[Periodic snapshots]
    end
    DB -->|Debezium / DMS / Lakeflow Connect| Feed[Change records]
    DT -->|readChangeFeed / table_changes| Feed
    Snap -->|compare snapshots| Feed
    Feed -->|MERGE in foreachBatch| T1[(Target table)]
    Feed -->|AUTO CDC flow| T2[(Streaming table)]
```

Approach	Data moved	Volume	Use case
Full load	All rows every run	High	Initial load, small tables
CDC feed	Changed rows only	Low	Incremental replication, SCD history
Snapshot diff	Full snapshots, diffed	High read, low write	Sources without a change feed

Each CDC record carries the operation (INSERT, UPDATE, DELETE), the row values, and a **sequence number or timestamp** that orders events so that late or out-of-order arrivals apply correctly.

Delta Change Data Feed (CDF)

CDF tracks row-level changes between versions of a table. Typical uses: incremental ETL that processes only changed rows, audit trails, and replicating changes to downstream tables or external systems.

Enabling legacy CDF

```sql
-- New table
CREATE TABLE main.sales.orders (
  order_id INT,
  customer_id INT,
  amount DECIMAL(18,2),
  status STRING
)
TBLPROPERTIES (delta.enableChangeDataFeed = true);

-- Existing table: only changes made after this command are recorded
```

```sql
ALTER TABLE main.sales.orders
SET TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

If you turn legacy CDF off for a while and then on again, the interval is not queryable through legacy CDF. Automatic CDF can cover such gaps on supported tables. To migrate a table to automatic CDF, `ALTER TABLE ... UNSET TBLPROPERTIES ('delta.enableChangeDataFeed')`.

CDF metadata columns

Column	Type	Values
`_change_type`	STRING	`insert`, `update_preimage` (value before the update), `update_postimage`
`_commit_version`	LONG	Table version that contains the change
`_commit_timestamp`	TIMESTAMP	When that commit was created

If the table already has columns with these names, CDF can't be used until you rename them.

How operations map to change types

```
sequenceDiagram
    participant U as Writer
    participant T as Delta table
    participant C as Change data feed
    U->>T: INSERT (id=1, value=100)
    T->>C: insert (id=1, value=100)
    U->>T: UPDATE id=1 SET value=150
    T->>C: update_preimage (id=1, value=100)
    T->>C: update_postimage (id=1, value=150)
    U->>T: DELETE id=1
    T->>C: delete (id=1, value=150)
```

`MERGE` on a CDF-enabled table produces the same records: matched updates produce a preimage and postimage pair, matched deletes produce `delete`, and inserts produce `insert`.

Reading CDF — batch

```sql
-- Versions 1 to 10 (both inclusive)
SELECT * FROM table_changes('main.sales.orders', 1, 10);

-- Timestamp range (strings in yyyy-MM-dd[ HH:mm:ss[.SSS]])
SELECT * FROM table_changes('main.sales.orders', '2026-10-01', '2026-10-15
 ↪  12:00:00');

-- From version 5 to the latest version
SELECT * FROM table_changes('main.sales.orders', 5);
```

```python
changes_df = (spark.read
    .option("readChangeFeed", "true")
    .option("startingVersion", 1)
    .option("endingVersion", 10)
    .table("main.sales.orders"))

changes_by_time = (spark.read
    .option("readChangeFeed", "true")
    .option("startingTimestamp", "2026-10-01 00:00:00")
    .option("endingTimestamp", "2026-10-15 12:00:00")
    .table("main.sales.orders"))
```

Batch reads **require** a starting version or timestamp. A starting version from before CDF was enabled raises an error. A version past the latest commit raises `timestampGreaterThanLatestCommit`, unless `spark.databricks.delta.changeDataFeed.timestampOutOfRange.enabled = true`, which returns an empty result instead.

Reading CDF — streaming (recommended for incremental ETL)

```
changes_stream = (spark.readStream
    .option("readChangeFeed", "true")
    .table("main.sales.orders"))
```

Use Structured Streaming so Databricks tracks processed versions in the checkpoint. Without `startingVersion`, a new stream first returns the **current snapshot of the table as `insert` records**, then the subsequent changes. If the target already contains everything up to version N, start at `N + 1` with a new checkpoint (`.option("startingVersion", N + 1)`). Rate limits (`maxFilesPerTrigger`, `maxBytesPerTrigger`) apply to whole commits.

Retention and VACUUM interplay CDF is **not a permanent change history**.

- Change records exist only for the table's retention window. Change data files follow the table's data retention and are removed by `VACUUM` (governed by `delta.deletedFileRetentionDuration`, default 7 days). Changes kept in the transaction log follow its checkpoint retention policy; table history is kept for `delta.logRetentionDuration` (default 30 days).
- Some operations (insert-only writes, full-partition deletes) do not write change data files; Databricks computes their changes from the transaction log.
- When a version leaves history, its changes can no longer be read. A stream restarted with a `startingVersion` that is no longer available **fails to start**, and managed tables clean up history automatically.
- For a permanent audit trail, stream the change feed into an archive table (for example, with `trigger(availableNow=True)` on a schedule).
- Never read the change data files directly; always use `table_changes` or `readChangeFeed`.

```
# Archive every change permanently, processed as a scheduled incremental batch
(spark.readStream
    .option("readChangeFeed", "true")
    .table("main.sales.orders")
    .writeStream
    .option("checkpointLocation", "/Volumes/main/chk/orders_cdf_archive")
    .trigger(availableNow=True)
    .toTable("main.audit.orders_changes"))
```

Schema changes and limitations

- With column mapping, a CDF read cannot span a version range that contains a **non-additive** schema change (rename or drop a column, change a type or nullability). Split the read into ranges before and after the change.
- Automatic CDF is not supported on tables with row filters or column masks, and external Iceberg readers cannot query it.

Processing CDF Changes

Choosing change types

Downstream need	Filter on `_change_type`
Latest values for upsert	`insert`, `update_postimage`
Propagate deletes	`delete`

Downstream need	Filter on _change_type
Before/after comparison or audit of what changed	update_preimage together with update_postimage
Count new rows only	insert

```
from pyspark.sql.functions import col

changes = (spark.read
    .option("readChangeFeed", "true")
    .option("startingVersion", 12)
    .table("main.bronze.orders"))

upserts = changes.filter(col("_change_type").isin("insert", "update_postimage"))
deletes = changes.filter(col("_change_type") == "delete")
```

Propagate changes with MERGE A version range can contain **several changes for the same key**. Keep only the latest change per key (by _commit_version) before MERGE, or the merge fails because multiple source rows match one target row.

```
from delta.tables import DeltaTable
from pyspark.sql import Window
from pyspark.sql.functions import col, row_number

def apply_changes_to_silver(batch_df, batch_id):
    latest = (batch_df
        .filter(col("_change_type") != "update_preimage")
        .withColumn("rn", row_number().over(

↪   Window.partitionBy("order_id").orderBy(col("_commit_version").desc())))
        .filter("rn = 1")
        .drop("rn"))

    (DeltaTable.forName(spark, "main.silver.orders").alias("t")
        .merge(latest.alias("s"), "t.order_id = s.order_id")
        .whenMatchedDelete(condition="s._change_type = 'delete'")
        .whenMatchedUpdateAll(condition="s._change_type = 'update_postimage'")
        .whenNotMatchedInsertAll(condition="s._change_type IN ('insert',
↪   'update_postimage')")
        .execute())

(spark.readStream
    .option("readChangeFeed", "true")
    .table("main.bronze.orders")
    .writeStream
    .foreachBatch(apply_changes_to_silver)
    .option("checkpointLocation", "/Volumes/main/chk/bronze_to_silver_orders")
    .start())
```

foreachBatch gives **at-least-once** delivery, so the batch function must be idempotent; a keyed MERGE on the latest change per key is idempotent.

AUTO CDC in Lakeflow Pipelines

AUTO CDC (formerly APPLY CHANGES) replaces the windowing, dedup, and MERGE code above with a declarative flow into a **streaming table**. It handles out-of-order events with the sequence column

and supports SCD Type 1 (default) and Type 2 via `stored_as_scd_type`. Full syntax, arguments, and SCD Type 2 output are in AUTO CDC API.

```python
from pyspark import pipelines as dp
from pyspark.sql.functions import col

# Delta CDF of an upstream table as the CDC source
@dp.temporary_view
def orders_changes():
    return (spark.readStream
        .option("readChangeFeed", "true")
        .table("main.bronze.orders")
        .filter(col("_change_type") != "update_preimage"))

dp.create_streaming_table("orders_silver")

dp.create_auto_cdc_flow(
    target="orders_silver",
    source="orders_changes",
    keys=["order_id"],
    sequence_by=col("_commit_version"),
    apply_as_deletes="_change_type = 'delete'",
    except_column_list=["_change_type", "_commit_version", "_commit_timestamp"],
    stored_as_scd_type=1,
)
```

```sql
CREATE OR REFRESH STREAMING TABLE customers_history;

CREATE FLOW customers_history_cdc AS AUTO CDC INTO customers_history
FROM STREAM(main.cdc.customers_feed)
KEYS (customer_id)
APPLY AS DELETE WHEN operation = 'DELETE'
SEQUENCE BY updated_at
COLUMNS * EXCEPT (operation, updated_at)
STORED AS SCD TYPE 2;
```

Manual Structured Streaming + MERGE	AUTO CDC
You dedupe per key, order by sequence, and handle late events	Sequencing and late events handled by the
You write SCD Type 2 close-and-insert logic	`stored_as_scd_type=2` adds `__START_AT` /
You manage checkpoint and retries in a job	Pipeline manages checkpoints and retries
Any target and format, any custom logic	Target must be a pipeline streaming table (L

SCD Patterns with MERGE (without pipelines)

SCD Type 1 (overwrite)

```sql
MERGE INTO dim_customer AS t
USING stg_customer AS s
ON t.customer_id = s.customer_id
WHEN MATCHED THEN UPDATE SET
  t.name = s.name,
  t.email = s.email,
  t.updated_at = current_timestamp()
WHEN NOT MATCHED THEN INSERT (customer_id, name, email, created_at, updated_at)
  VALUES (s.customer_id, s.name, s.email, current_timestamp(),
   ↪  current_timestamp());
```

SCD Type 2 (history) in one MERGE The common single-statement trick unions the staged changes twice: once keyed (to close the current row) and once with a NULL merge key (so the new version is always inserted).

```
MERGE INTO dim_customer AS t
USING (
  -- Rows that close the current version
  SELECT s.customer_id AS merge_key, s.* FROM stg_customer s
  UNION ALL
  -- Rows inserted as the new current version (NULL key never matches)
  SELECT NULL AS merge_key, s.* FROM stg_customer s
  JOIN dim_customer d
    ON s.customer_id = d.customer_id AND d.is_current = true
  WHERE s.name <> d.name OR s.email <> d.email
) AS src
ON t.customer_id = src.merge_key AND t.is_current = true
WHEN MATCHED AND (t.name <> src.name OR t.email <> src.email) THEN UPDATE SET
  t.is_current = false,
  t.end_date = current_timestamp()
WHEN NOT MATCHED THEN INSERT (customer_id, name, email, start_date, end_date,
 ↪  is_current)
  VALUES (src.customer_id, src.name, src.email, current_timestamp(), NULL, true);
```

New customers arrive through the first branch, find no match, and are inserted. Changed customers close their current row through the first branch and get a new current row through the second. More SCD variants are in SCD Patterns.

```
gantt
    title SCD Type 2 record history
    dateFormat YYYY-MM-DD
    section Customer 123
    Version 1 (John) :v1, 2026-01-01, 2026-03-15
    Version 2 (Johnny) :v2, 2026-03-15, 2026-06-01
    Version 3 (John D.) :active, v3, 2026-06-01, 2026-10-26
```

Continue reading: Part 2 — Delta or Iceberg targets, multi-hop CDC, row tracking, external CDC formats, and exam tips

---

Change Data Capture (CDC) — Part 2

Overview

This part covers what "Delta or Iceberg as the target table format" means for Lakeflow pipeline CDC, how to propagate changes through bronze, silver, and gold, row tracking, external CDC formats such as Debezium, and monitoring. For CDF basics, MERGE patterns, and the AUTO CDC introduction, see Part 1.

Overview

- **Pipeline outputs are Delta tables**: an AUTO CDC target is always a Delta streaming table (USING DELTA is the only option for streaming tables)
- **Iceberg readers** reach a pipeline CDC target through **external data access** with UniForm **Iceberg v3** properties, or through **compatibility mode** (a copy); materialized views can also be created USING ICEBERG (Public Preview)
- **Multi-hop CDC**: CDF (or AUTO CDC) between bronze and silver, and materialized views

for gold aggregates
- **Row tracking** gives stable `_metadata.row_id` values and powers automatic CDF and incremental MV refresh

What the Exam Tests

- Configure incremental CDC pipelines using Lakeflow Pipelines with Delta or Iceberg as the target table format
- Apply Change Data Feed (CDF) to expose row-level changes (updates/deletes) for efficient incremental downstream processing
- Choose the right mechanism when a downstream consumer is an external Iceberg engine
- Keep CDC processing idempotent and ordered across layers

Delta or Iceberg as the CDC Target Format

What the docs support

Need	Supported approach
CDC target for Databricks and Delta consumers	AUTO CDC flow into a **streami**
Same CDC target also readable by external **Iceberg** engines, without a copy	**External data access**: `pipel`
Gold aggregate as a managed Iceberg object	`CREATE OR REFRESH MATERIAL`
Older or path-based Delta/Iceberg clients	**Compatibility mode**
Incremental processing **from** an Iceberg source	MVs refresh incrementally from

Important

The docs do not offer an Iceberg storage format for pipeline **streaming tables**, so AUTO CDC never writes Iceberg natively. For streaming tables the only `USING` option is `DELTA`. "Iceberg as the target" in practice means that the Delta target also publishes **Iceberg metadata** for Iceberg readers. Ordinary Delta tables use UniForm **IcebergCompatV2**, but that **cannot be enabled on materialized views or streaming tables**; pipeline datasets use **IcebergCompatV3** through external data access.

AUTO CDC target readable by Iceberg clients

```sql
-- Pipeline source file (SQL)
CREATE OR REFRESH STREAMING TABLE customers_current
TBLPROPERTIES (
  'delta.columnMapping.mode' = 'name',
  'delta.enableRowTracking' = 'true',
  'delta.enableIcebergCompatV3' = 'true',
  'delta.universalFormat.enabledFormats' = 'iceberg',
  'pipelines.externalMetadata.enabled' = 'true'
);

CREATE FLOW customers_current_cdc AS AUTO CDC INTO customers_current
FROM STREAM(main.cdc.customers_feed)
KEYS (customer_id)
APPLY AS DELETE WHEN operation = 'DELETE'
SEQUENCE BY sequence_num
```

```
COLUMNS * EXCEPT (operation, sequence_num)
STORED AS SCD TYPE 1;

# Same target in Python
from pyspark import pipelines as dp

dp.create_streaming_table(
    name="customers_current",
    table_properties={
        "delta.columnMapping.mode": "name",
        "delta.enableRowTracking": "true",
        "delta.enableIcebergCompatV3": "true",
        "delta.universalFormat.enabledFormats": "iceberg",
        "pipelines.externalMetadata.enabled": "true",
    },
)

dp.create_auto_cdc_flow(
    target="customers_current",
    source="customers_feed",
    keys=["customer_id"],
    sequence_by="sequence_num",
    apply_as_deletes="operation = 'DELETE'",
    except_column_list=["operation", "sequence_num"],
    stored_as_scd_type=1,
)
```

Table property	Purpose
`pipelines.externalMetadata.enabled = true`	Publish external-facing metadata. Can be set for the
`delta.columnMapping.mode = name`	Column mapping is required for Iceberg
`delta.enableRowTracking = true`	Required for Iceberg reads
`delta.enableIcebergCompatV3 = true`	Iceberg v3 compatibility
`delta.universalFormat.enabledFormats = iceberg`	Generate Iceberg metadata

Apply it: run the triggered pipeline once, or stop and restart a continuous pipeline.

Requirements: Unity Catalog, Databricks Runtime 17.3+, the default publishing mode, and external data access enabled for the metastore. The reader needs `EXTERNAL USE SCHEMA` on the schema and `SELECT` on the table.

Limitations: no external writes, no path-based access, row filters and column masks are not enforced for external reads, no time travel through this path, and catalog commits must be disabled first. If external metadata looks stale, a principal with `MODIFY` can run `REPAIR TABLE <table> SYNC METADATA`.

Materialized view stored as Iceberg

```
CREATE OR REFRESH MATERIALIZED VIEW gold_customer_counts
USING ICEBERG
AS SELECT country, COUNT(*) AS customers
FROM customers_current
GROUP BY country;

-- After creation, for example from the SQL editor
REPAIR TABLE main.sales.gold_customer_counts SYNC METADATA;
```

External Iceberg readers can read this MV but not write to it.

Iceberg tables as CDC sources

- **Automatic change data feed** (Databricks Runtime 19+) works on Unity Catalog **managed Iceberg v3** tables as well as Delta tables with row tracking. Read it with `readChangeFeed` / `table_changes`. Only Databricks readers can query it, because change data feed is not part of the Iceberg spec.
- **Legacy CDF** (`delta.enableChangeDataFeed`) supports Delta tables only.
- An MV over a managed Iceberg table refreshes incrementally. For the best incremental refresh support the docs recommend Iceberg v3.

Multi-Hop CDC Propagation

```
flowchart LR
    Src[(OLTP source)] -->|CDC feed| B[bronze.orders_raw<br>append-only, CDF on]
    B -->|CDF or AUTO CDC| S[silver.orders<br>SCD1 / SCD2, CDF on]
    S -->|materialized view| G1[gold.daily_summary]
    S -->|readChangeFeed| G2[downstream replica /<br>external system]
```

Hop	Recommended mechanism
Source → bronze	Append-only streaming table (Auto Loader, Kafka, Lakeflow Connect)
Bronze → silver	AUTO CDC flow (pipelines) or CDF stream + `MERGE` in `foreachBatch`
Silver → gold aggregates	**Materialized view**
Silver → replicas or external systems	Read silver's change feed

```python
# Gold aggregate over a silver table that receives updates and deletes
from pyspark import pipelines as dp
from pyspark.sql.functions import count, sum, to_date

@dp.materialized_view(name="orders_daily_summary")
def orders_daily_summary():
    return (spark.read.table("main.silver.orders")
        .groupBy(to_date("order_date").alias("order_date"))
        .agg(sum("amount").alias("daily_total"), count("*").alias("order_count")))
```

Don’t aggregate CDF rows naively

Summing `insert` + `update_postimage` rows from a change feed double-counts every updated row and ignores deletes. Use a materialized view over the current-state table, or net the changes (subtract preimages and deletes) yourself.

CDC amplification: one source UPDATE becomes two CDF rows (preimage and postimage). A silver `MERGE` can touch many rows. A change in silver can force recomputation of many gold groups. Monitor lag per layer and give every stream its own checkpoint.

Idempotent and Ordered Processing

```python
from delta.tables import DeltaTable
from pyspark.sql import Window
from pyspark.sql.functions import col, row_number

def process_cdc_idempotent(changes_df, target_table, key="id"):
    """Apply only the latest change per key; MERGE makes replays safe."""
```

```
    latest = (changes_df
        .filter(col("_change_type") != "update_preimage")
        .withColumn("rn", row_number().over(
            Window.partitionBy(key).orderBy(col("_commit_version").desc())))
        .filter("rn = 1")
        .drop("rn"))

    (DeltaTable.forName(spark, target_table).alias("t")
        .merge(latest.alias("s"), f"t.{key} = s.{key}")
        .whenMatchedDelete(condition="s._change_type = 'delete'")
        .whenMatchedUpdateAll(condition="s._change_type IN ('insert',
↪   'update_postimage')")
        .whenNotMatchedInsertAll(condition="s._change_type IN ('insert',
↪   'update_postimage')")
        .execute())
```

- **Ordering**: pick the latest event per key by a reliable sequence (`_commit_version` for CDF, LSN or source timestamp for database feeds). With a source timestamp, add a tiebreaker.
- **Idempotency**: `foreachBatch` is at-least-once, so replays must be safe. A keyed `MERGE` of the latest change is idempotent; blind appends are not.
- **Exact duplicates**: `dropDuplicates(["id", "_commit_version"])` removes repeated delivery of the same change.
- **AUTO CDC** does the ordering and dedup for you inside a pipeline.

Row Tracking

Row tracking (Databricks Runtime 14.0+) assigns every row a stable row ID and a row commit version. All Iceberg v3 tables include row tracking; for Delta you enable it explicitly. Automatic CDF and some incremental MV refresh techniques depend on it.

```
CREATE TABLE main.sales.customers (id INT, name STRING)
TBLPROPERTIES (delta.enableRowTracking = true);

-- Existing table: backfills IDs for existing rows (can create several versions
↪  and take time)
ALTER TABLE main.sales.customers SET TBLPROPERTIES (delta.enableRowTracking =
↪  true);
```

Hidden field	Meaning
`_metadata.row_id`	Stable unique row identifier; kept across `MERGE` and `UPDATE`
`_metadata.row_commit_version`	Table version at which the row was last inserted or updated

```
df = spark.table("main.sales.customers")
df.select("*", "_metadata.row_id", "_metadata.row_commit_version").show()

# Rows changed at version 5
df.filter("_metadata.row_commit_version = 5").show()
```

Pause continuous writers (for example, Structured Streaming) before enabling row tracking on an existing table, because the metadata change fails concurrent writes with `MetadataChangedException`. Row IDs and row commit versions **cannot be read while reading the change data feed**. Clones get their own row IDs.

External CDC Integration

Debezium envelope Debezium events carry `before`, `after`, `op` (`c` create, `u` update, `d` delete, `r` snapshot read), and `ts_ms`. For deletes, `after` is null, so take the row (and key) from `before`.

```
from pyspark.sql.functions import col, from_json, when

parsed = (spark.readStream.format("kafka")
    .option("kafka.bootstrap.servers", "broker:9092")
    .option("subscribe", "dbserver1.inventory.orders")
    .load()
    .select(from_json(col("value").cast("string"), debezium_schema).alias("d"))
    .select(
        col("d.op").alias("op"),
        col("d.ts_ms").alias("source_ts_ms"),
        when(col("d.op") == "d",
 ↪  col("d.before")).otherwise(col("d.after")).alias("row"),
    )
    .select("op", "source_ts_ms", "row.*"))
```

Feed `parsed` to AUTO CDC with `apply_as_deletes="op = 'd'"` and `sequence_by="source_ts_ms"` (add a tiebreaker if two events for a key can share a timestamp), or to a `MERGE` in `foreachBatch`.

Normalizing custom operation codes

```
from itertools import chain
from pyspark.sql.functions import coalesce, col, create_map, lit, upper

OP_MAP = {"I": "insert", "INSERT": "insert", "U": "update", "UPDATE": "update",
          "D": "delete", "DELETE": "delete"}
op_lookup = create_map([lit(x) for x in chain(*OP_MAP.items())])

def normalize_ops(df, op_column="operation"):
    return df.withColumn("op_normalized",
                         coalesce(op_lookup[upper(col(op_column))], lit("unknown")))
```

For relational sources (SQL Server, MySQL, PostgreSQL), managed **Lakeflow Connect** connectors handle CDC extraction for you and write to streaming tables, so you don't hand-parse logs.

Monitoring CDC

- **AUTO CDC flows** report `num_upserted_rows` and `num_deleted_rows` as processing metrics in the pipeline event log.
- **Version lag** for CDF consumers: compare the last processed `_commit_version` (store it per batch in `foreachBatch`) with the latest version from `DESCRIBE HISTORY <table> LIMIT 1`.
- **Retention headroom**: alert when lag approaches the table's history retention, because a consumer that falls behind it cannot restart from its last version.

```
-- Per-batch metrics table written from foreachBatch
CREATE TABLE IF NOT EXISTS main.ops.cdc_metrics (
  table_name STRING,
  batch_id BIGINT,
  last_commit_version BIGINT,
  inserts BIGINT,
  updates BIGINT,
  deletes BIGINT,
  processed_at TIMESTAMP
);
```

Use Cases

Scenario	Pattern
Silver customer table must also be queryable from an external Iceberg engine	AUTO CDC into a streaming ta
Audit compliance	SCD Type 2 (AUTO CDC `store`
Delete propagation downstream	CDF `delete` rows + `MERGE ..`
Gold metrics over changing silver	Materialized view
Debugging when a row changed	Row tracking + `DESCRIBE HIS`

Common Issues & Errors

Symptom	Cause
Empty or partial CDF history	CDF enabled after the changes happened
Stream from CDF fails to restart	Requested `startingVersion` no longer in history (VACUUM / log cl
Duplicate downstream rows after a retry	Non-idempotent writes in `foreachBatch`
Updates show old values downstream	Filtering on `update_preimage`
Iceberg client can't see the pipeline table	External metadata not enabled, IcebergCompatV2 used, catalog c
Iceberg readers see stale data	Metadata generation behind
CDF read fails across a column rename	Non-additive schema change in the range (column mapping)

Best Practices

- Enable CDF (or rely on automatic CDF with row tracking) on every table that feeds incremental consumers.
- Prefer AUTO CDC inside pipelines over hand-written `MERGE` for SCD Type 1 and Type 2.
- Use materialized views, not change-feed aggregation, for gold metrics.
- Keep one checkpoint per stream; never share checkpoints across queries.
- Archive the change feed if you need history longer than the table's retention.
- For Iceberg consumers of pipeline tables, use external data access (Iceberg v3) when clients support the REST catalog; otherwise use compatibility mode.

Exam Tips

Tip

- **`delta.enableChangeDataFeed = true`** turns on (legacy) CDF; it is **not retroactive**.
- **`_change_type`**: `insert`, `update_preimage`, `update_postimage`, `delete`. Use **postimage** for new values.
- **SQL** `table_changes('t', start[, end])`; **PySpark** `.option("readChangeFeed", "true")`; start/end are inclusive.
- **VACUUM and log retention limit CDF**: old versions' changes disappear.
- **AUTO CDC target = Delta streaming table**. For Iceberg readers: **external data access + IcebergCompatV3**, or compatibility mode; MVs support `USING ICEBERG` (Preview). IcebergCompatV2 can't be enabled on STs or MVs.
- **Gold aggregates over changing data → materialized view.**

Key Takeaways

- **CDF exposes row-level changes** with `_change_type`, `_commit_version`, `_commit_timestamp`; read them with `table_changes` or `readChangeFeed`, preferably as a stream.

- **Retention bounds CDF**: `VACUUM` removes change data files, and table history expires; archive the feed for a permanent record.
- **Pipeline CDC targets are Delta streaming tables.** Iceberg access is through Iceberg v3 metadata (external data access) or a compatibility copy; only materialized views have a `USING ICEBERG` option, and it is in Public Preview.
- **Iceberg as a source**: automatic CDF covers managed Iceberg v3 tables (DBR 19+, Databricks readers), and MVs refresh incrementally from managed Iceberg tables.
- **Multi-hop**: AUTO CDC or CDF + `MERGE` for bronze → silver, and materialized views for gold.
- **Idempotency and ordering**: latest change per key by sequence, applied with a keyed `MERGE` (or AUTO CDC).
- **Row tracking** (DBR 14.0+) provides `_metadata.row_id` / `_metadata.row_commit_version` and underpins automatic CDF.

Related Topics

- Change Data Capture (CDC) — Part 1 — CDF basics, MERGE patterns
- AUTO CDC API — SCD Type 1 and 2 in pipelines
- Declarative Pipelines — streaming tables vs materialized views
- Delta Lake Operations — Part 1 — MERGE details
- Data Deduplication — dedup before applying changes
- OpenSharing — sharing tables, including CDF, with recipients

Official Documentation

- Use change data feed on Databricks
- Access materialized views and streaming tables using external systems
- Enable external data access to streaming tables and materialized views
- Read Delta Lake tables with Iceberg clients
- What is Apache Iceberg in Databricks?
- CREATE MATERIALIZED VIEW (pipelines)
- CREATE STREAMING TABLE (pipelines)
- Incremental refresh for materialized views
- Advanced AUTO CDC topics
- Row tracking in Databricks
- Pipeline concepts

Lakeflow Connect Database CDC

Overview

Lakeflow Connect managed database connectors replicate inserts, updates, and deletes from relational databases — **SQL Server, MySQL, and PostgreSQL** on this exam — into Unity Catalog streaming tables with no custom CDC code. The connector reads the database's own change log (SQL Server change tracking / CDC, the MySQL binary log, PostgreSQL logical replication), stages the changes, and applies them incrementally through a managed Lakeflow pipeline that is governed by Unity Catalog.

Overview

- **Two architectures**: the standard *gateway-based* design (an always-on **ingestion gateway** on classic compute + a scheduled **ingestion pipeline** on serverless) and the newer single *integrated CDC pipeline* (SQL Server, MySQL)
- **Source prerequisites differ per engine**: SQL Server change tracking or CDC, MySQL

`binlog_format=ROW` + `binlog_row_image=FULL`, PostgreSQL `wal_level=logical` + publication + `pgoutput` replication slot
- **History tracking** is a per-table setting: `SCD_TYPE_1` (overwrite, default) or `SCD_TYPE_2` (keep history with `__START_AT` / `__END_AT`)
- **Credentials live in a Unity Catalog connection**; users need `USE CONNECTION` plus catalog/schema privileges to build pipelines

What the Exam Tests

- "Configure CDC ingestion pipelines from relational database sources, including SQL Server, MySQL, and PostgreSQL, using Lakeflow Connect" — pick the managed connector over JDBC full reloads, file exports, OpenSharing, or federation when the requirement is *continuous insert/update/delete replication with minimal code*
- Know each source's prerequisite (change tracking vs CDC, binlog settings, logical replication slot) and why the gateway must run continuously
- Choose SCD type 1 vs type 2, and know which schema changes evolve automatically and which force a **full refresh**
- Know the governance pieces: UC connection, `CREATE CONNECTION` / `USE CONNECTION`, staging volume, destination streaming tables

---

Where Database Connectors Fit in Lakeflow Connect

Lakeflow Connect groups managed connectors by source type. Only the first two rows below are database ingestion; the others appear here because exam distractors mix them up.

Connector type	What it does
Database connectors (CDC)	MySQL, PostgreSQL, SQL Server, Oracle
Query-based connectors	Oracle, Teradata, SQL Server, MySQL, MariaDB, PostgreSQL (and all Lakehous
SaaS connectors	Salesforce, Workday, HubSpot, Jira, ...
File source connectors	SharePoint, Google Drive, ...
Streaming connectors	Kafka (Beta), RabbitMQ (Beta)

Release status (as stated on the docs pages, Oct 2026)

Connector	Status
Microsoft SQL Server	No preview label on the connector pages
MySQL	**Public Preview** ("Contact your Databricks account team to request acce
PostgreSQL	**Public Preview** ("Reach out to your Databricks account team to enroll")
Oracle (integrated CDC, LogMiner)	**Beta**

Platform update (Oct 2026)

The integrated CDC pipeline for MySQL entered Beta on July 1, 2026 and became generally available on October 15, 2026; it "requires no separate ingestion gateway". Source: Databricks release notes, October 2026

Architecture: Gateway-Based CDC

```
flowchart LR
    subgraph SRC["Source database"]
        DB[(SQL Server / MySQL / PostgreSQL)]
        LOG[[Change log:<br/>CT / CDC / binlog / WAL]]
    end
    subgraph DBX["Databricks workspace"]
        CONN[UC connection<br/>credentials]
        GW["Ingestion gateway<br/>classic compute, continuous"]
        STG[("Staging UC volume<br/>purged after 30 days")]
        PIPE["Ingestion pipeline<br/>serverless, scheduled"]
        ST[("Destination streaming tables<br/>Delta, CDF enabled")]
    end
    DB --> LOG
    CONN -.-> GW
    LOG -- snapshot + changes --> GW
    GW --> STG
    STG --> PIPE
    PIPE -- apply SCD 1 / 2 --> ST
```

Component	Role	Key facts
Connection	UC securable storing source credentials	Created once by an admin; reused by
Ingestion gateway	Extracts snapshots, change logs, and metadata	Runs on **classic compute**, **continuo**
Staging storage	UC volume holding extracted data	Created automatically when the gate
Ingestion pipeline	Moves staged data into destination tables	Runs on **serverless**; runs on your sc
Destination tables	Streaming tables (Delta)	Change data feed is enabled on all ta

Orchestration: every schedule you add creates a Lakeflow Job with the ingestion pipeline as a task; the gateway runs in its own job as a continuous task.

> **Warning**
>
> Do not manually stop the ingestion gateway. If it stops, changes can be dropped by the source's log retention and affected tables need a full refresh. Gateways need classic compute, so they can't run in a serverless-only workspace.

Integrated CDC pipeline (single pipeline) For SQL Server and MySQL you can instead create one pipeline that both extracts and applies changes.

Feature	Standard CDC (gateway-based)	Integrated CDC
Pipelines	Two (gateway + ingestion pipeline)	One
Connection reference	`ingestion_gateway_id`	`connection_name`
Connector type	Implicit	Explicit `connector_type`
Staging volume	Managed by the gateway	Auto-created in the desti
Mode	Gateway continuous, pipeline scheduled	Triggered (scheduled); co
Compute	Classic gateway + serverless pipeline	Classic or serverless
Auto full refresh (MySQL)	Not supported for existing gateway-based MySQL flows	Supported

Each integrated update extracts (initial snapshot on the first run or a full refresh, then binlog/CT/CDC changes), writes to the staging volume, applies the changes with merge semantics on the

primary keys and SCD type, and **stops once it has caught up**. Docs note that each extraction stage runs for at least 10 minutes, so a **60-minute or longer** schedule is a good starting point.

Source Prerequisites per Database

Requirement	SQL Server
Change mechanism	**Change tracking** (needs a primary key; lightweight) or **CDC** (no PK needed; heavier) –
Versions	CT: SQL Server 2012+; CDC: 2012 SP1 CU3+ (Enterprise Edition before 2016)
Server settings	Enable CT or CDC (utility objects script)
Ingestion user	Dedicated user meeting the privilege requirements
Replicas	Primary only — CT/CDC not supported on read replicas or secondaries
Retention gotcha	Expired CT/CDC retention needs full refresh

PostgreSQL source setup (run as admin / table owner)

```
-- Replication user (its credentials go into the UC connection)
CREATE USER databricks_replication WITH PASSWORD 'your_secure_password';
GRANT CONNECT ON DATABASE your_database TO databricks_replication;
GRANT USAGE ON SCHEMA schema_name TO databricks_replication;
GRANT SELECT ON TABLE schema_name.table_name TO databricks_replication;
ALTER USER databricks_replication WITH REPLICATION;

-- Publication first, then the slot (one of each per replicated database)
CREATE PUBLICATION databricks_publication FOR TABLE schema_name.table1,
 ↪  schema_name.table2;

SET ROLE databricks_replication;
-- Databricks supports only the pgoutput plugin
SELECT pg_create_logical_replication_slot('databricks_slot', 'pgoutput');
RESET ROLE;
```

MySQL replication user

```
-- MySQL 8.x (5.7.x uses sha256_password)
CREATE USER 'lakeflow_connect_user'@'%' IDENTIFIED WITH caching_sha2_password BY
 ↪  '<password>';
GRANT REPLICATION SLAVE, REPLICATION CLIENT ON *.* TO 'lakeflow_connect_user'@'%';
```

Also grant `SELECT(plugin, user)` on `mysql.user` and `SELECT` on the databases/tables you ingest.

UC Connection and Permissions

A **connection** is the Unity Catalog securable that stores the source host and credentials; the pipeline definition never contains passwords.

Task	Privileges required
Create a new connection	`CREATE CONNECTION` on the metastore
Build a pipeline on an existing connection	`USE CONNECTION` (or `ALL PRIVILEGES`) on the connection
Write destination tables	`USE CATALOG` on the target catalog; `USE SCHEMA`, `CREATE TABLE`, C
Staging volume	`CREATE VOLUME` on the destination schema, or on the schema in d

- **Personas**: in a *multi-user* setup an admin configures the source and creates the connection in Catalog Explorer; non-admins then create gateway-pipeline pairs with any interface. API/CLI/bundle authoring always needs an existing connection.
- **Creating connections programmatically**: database connectors use API-only authentication, so you can create the connection with the Connections REST API or `databricks connections create --json ...` (for example from a bundle pre-deploy step). The SQL `CREATE CONNECTION` command is documented for Lakehouse Federation connections, not managed ingestion connections.
- **Authentication**: SQL Server supports OAuth U2M/M2M with Microsoft Entra ID and username/password; MySQL and PostgreSQL support username/password only.

Setting Up a Pipeline: UI, API, Bundles

UI wizard (Data Ingestion → *SQL Server*/*MySQL*/*PostgreSQL*): choose or create the connection → name the ingestion pipeline and event-log location → optionally turn on **Auto full refresh for all tables** → name the gateway and pick the **staging location** → select tables (optional destination name and **History tracking**) → choose destination catalog/schema → **Validate** the database setup → add a schedule and notifications.

Declarative Automation Bundles (formerly Databricks Asset Bundles) — gateway-based SQL Server, abridged from the docs example:

```
resources:
  pipelines:
    gw_pipeline:
      name: 'lfc-sqlserver-gateway-${bundle.target}'
      continuous: true                      # gateway pipelines must be continuous
      catalog: ${var.staging_catalog}
      schema: ${var.staging_schema}
      gateway_definition:
        connection_name: ${var.connection_name}
        gateway_storage_catalog: ${var.staging_catalog}
        gateway_storage_schema: ${var.staging_schema}
    mi_pipeline:
      name: 'lfc-sqlserver-ingestion-${bundle.target}'
      continuous: false                      # trigger with a scheduled job
      serverless: true                      # ingestion pipeline is serverless only
      catalog: ${var.dest_catalog}
      schema: ${var.dest_schema}
      ingestion_definition:
        ingestion_gateway_id: ${resources.pipelines.gw_pipeline.id}
        table_configuration:
          scd_type: 'SCD_TYPE_1'
        objects:
          - schema:                          # whole schema; new tables picked up
↪  automatically
              source_catalog: ${var.source_database}
              source_schema: ${var.source_schema}
              destination_catalog: ${var.dest_catalog}
              destination_schema: ${var.dest_schema}
  jobs:
    mi_schedule:
      name: 'lfc-sqlserver-ingestion-schedule-${bundle.target}'
      schedule:
        quartz_cron_expression: '0 */30 * * * ?'
```

```
        timezone_id: 'UTC'
      tasks:
        - task_key: 'run_ingestion'
          pipeline_task:
            pipeline_id: ${resources.pipelines.mi_pipeline.id}
```

PostgreSQL pipelines add `source_type: POSTGRESQL` and a `source_configurations` block naming the slot and publication:

```
source_configurations:
  - catalog:
      source_catalog: your_database
      postgres:
        slot_config:
          slot_name: databricks_slot
          publication_name: databricks_publication
```

Integrated CDC pipeline (MySQL shown) — one pipeline, referenced by connection:

```
resources:
  pipelines:
    integrated_cdc_pipeline:
      name: ${var.pipeline_name}
      catalog: ${var.dest_catalog}
      schema: ${var.dest_schema}
      ingestion_definition:
        connection_name: ${var.connection_name}
        connector_type: CDC
        objects:
          - table:
              source_schema: 'my_database'
              source_table: 'customers'
              destination_catalog: ${var.dest_catalog}
              destination_schema: ${var.dest_schema}
              table_configuration:
                scd_type: 'SCD_TYPE_1'
```

```
databricks bundle deploy
databricks bundle run integrated_cdc_job
```

Other per-table options in `table_configuration`: `include_columns` / `exclude_columns` (column selection), `primary_keys` (override), `sequence_by`, and `auto_full_refresh_policy`. API-based row filtering is **not supported** for the three database connectors.

History Tracking: SCD Type 1 vs Type 2

	SCD_TYPE_1 (history off, default)	SCD_TYPE_2 (history on)
Update in source	Row overwritten in place	Old row kept and marked inactive; new row added
Metadata	None	__START_AT / __END_AT spans driven by the sequence c
Typical use	Current-state replica for BI	Audit / point-in-time dimension history

- Set it in the UI (**History tracking** on the Source page) or with `scd_type` at pipeline, schema, or table level.
- SQL Server: **change tracking does not support `SCD_TYPE_2`** — SCD type 2 requires CDC on the source table.

- Deleting a table or column in the source never deletes it from the destination, even with SCD type 1 (it is marked inactive).
- A full refresh **erases SCD type 2 history**.

These managed SCD semantics match what you would build yourself with the AUTO CDC APIs (formerly APPLY CHANGES); see AUTO CDC API.

Schema Evolution and Full Refresh

Feature availability (connector overview pages)

Behavior	SQL Server	MySQL
New and deleted columns	Automatic	Automatic
Data type changes	Not supported	Not supported
Column renames	Not supported — requires full refresh	Supported — treated as new column
New tables	Automatic if you ingest the whole schema	Automatic if you ingest the whole sch
SCD type 2	Supported (CDC tables)	Supported
Max tables per pipeline	250	250

New columns are ingested on the next run (earlier rows get nulls) unless you pinned an explicit column list. Deleted columns become "inactive"; if a new column later reuses the name, the pipeline fails until you full-refresh or drop the inactive column.

Full refresh behavior for database connectors

1. **Snapshot request** — the gateway starts a fresh snapshot of the table.
2. **Continued availability** — the destination table keeps serving its existing data; no changes are applied meanwhile.
3. **Atomic refresh** — when the snapshot completes, snapshot data plus CDC records accumulated since the request are applied in one update.

Setting	Purpose
`full_refresh_window`	Restrict when snapshot work for full refreshes runs (start hour, days, time zone
`auto_full_refresh_policy`	Automatically full-refresh on table truncate, incompatible schema changes (e.g

Table-level `auto_full_refresh_policy` overrides the pipeline-level setting. You can also trigger a full refresh manually from the pipeline UI, the Pipelines API (`POST /api/2.0/pipelines/{pipeline_id}/update` or `databricks pipelines start-update`.

Monitoring and Cost

- **Gateway event log**: the gateway emits `flow_progress` (row/byte counters, CDC latency) and `operation_progress` (snapshot percentage and ETA) events every 5 minutes per table.

```
SELECT
  timestamp,
  CONCAT(origin.catalog_name, '.', origin.schema_name, '.',
   ↪  origin.dataset_name) AS table_name,
  details:flow_progress:metrics:num_upserted_rows::bigint AS rows_upserted,
```

```
  COALESCE(details:flow_progress:metrics:num_deleted_rows::bigint, 0) AS
    ↪ rows_deleted
FROM event_log('<gateway-pipeline-id>')
WHERE event_type = 'flow_progress'
  AND level = 'METRICS'
  AND origin.pipeline_type = 'INGESTION_GATEWAY'
ORDER BY timestamp DESC;
```

- **Cost**: query `system.billing.usage` where `billing_origin_product = 'LAKEFLOW_CONNECT'`, joining `usage_metadata.dlt_pipeline_id` to `system.lakeflow.pipelines` (the `pipelines` and `pipeline_update_timeline` system tables went GA on Oct 21, 2026). Expect both classic (gateway) and serverless (ingestion pipeline) DBUs.
- **Failure recovery**: connectors retry with exponential backoff and store the last cursor position so the next run resumes; credential expiry needs intervention.
- **Overlapping runs**: if update N is still running when N+1 is due, N+1 is skipped.

Networking

The gateway (or integrated pipeline on classic compute) runs in your workspace VPC/VNet and must reach the database: VPC/VNet peering, Private Link, VPN, AWS Direct Connect, Azure ExpressRoute, or public endpoints all work, and cross-cloud ingestion is supported. Integrated pipelines on serverless need serverless network connectivity configured to the source.

Choosing an Approach

The exam guide's sample Question 3 asks how to "continuously replicate insert, update, and delete change events from an operational PostgreSQL database ... with minimal custom code" — the answer is the Lakeflow Connect managed connector.

Requirement	Best fit
Continuous insert/update/delete replication, minimal code	**Lakeflow Connect database (CDC) c**
Recurring incremental loads but no CDC/binlog access on the source	**Lakeflow Connect query-based con**
One-off or custom load, need writes or native SQL on the source	**Spark JDBC / bundled connector** (sp
Ad hoc / proof-of-concept queries without copying data	**Lakehouse Federation**
Source already exports files to cloud storage	**Auto Loader**
Share data with another organization	**OpenSharing**

Use Cases

- **Operational database → bronze/silver replica** for analytics with SCD type 1 streaming tables
- **Dimension history** (customer, product) with SCD type 2 directly from source CDC
- **Migrating off a nightly JDBC dump** to continuous change capture with lower source load (change tracking / binlog instead of full scans)
- **Multi-database estates** (SQL Server + MySQL + PostgreSQL) governed by one set of UC connections and deployed per environment with bundles

Common Issues & Errors

Symptom	Likely cause
Tables need a full refresh after an outage	Gateway stopped or lagged past log retentio
PostgreSQL disk fills with WAL	Inactive or lagging replication slot; slot left b
MySQL table skipped (`...PARTIAL_JSON`)	`binlog_row_value_options=PARTIAL_JSON`
SCD type 2 rejected for a SQL Server table	Table uses change tracking
Pipeline fails after a column rename (SQL Server / PostgreSQL)	Rename isn't evolved automatically
Initial snapshot fails	Gateway compute undersized
Ingesting from a SQL Server secondary	CT/CDC unsupported on replicas

Best Practices

- Use **change tracking** for SQL Server tables with primary keys (lighter); use **CDC** for tables without a PK or when you need SCD type 2.
- Create a **dedicated replication user** and give non-admins `USE CONNECTION` rather than sharing credentials.
- Deploy gateway, pipeline, and schedule together with **Declarative Automation Bundles** for dev/staging/prod.
- Prefer the **integrated CDC pipeline** (SQL Server, MySQL) when scheduled ingestion is enough and you want one pipeline and no always-on gateway.
- Watch the gateway event log and `system.billing.usage`; the gateway costs money even when the ingestion pipeline is idle.

Exam Tips

Tip

- ”Continuously replicate inserts, updates, **and deletes** from SQL Server / MySQL / PostgreSQL with minimal code” → **Lakeflow Connect managed database connector**, not nightly JDBC, not Auto Loader on CSV exports, not OpenSharing.
- **Gateway = classic compute, always on; ingestion pipeline = serverless, scheduled.** Staging is a UC volume purged after 30 days.
- Source prerequisites: SQL Server **change tracking or CDC** (CT preferred with a PK, CT can't do SCD 2), MySQL **binlog ROW + FULL**, PostgreSQL **logical replication + publication + `pgoutput` slot**.
- `SCD_TYPE_1` is the default; `SCD_TYPE_2` adds `__START_AT` / `__END_AT`; a full refresh wipes SCD 2 history.
- Data type changes always need a full refresh; column renames need one on SQL Server and PostgreSQL but not MySQL.
- Credentials belong in a **UC connection**; `USE CONNECTION` lets non-admins build pipelines.

Key Takeaways

- Lakeflow Connect database connectors turn a source change log into Unity Catalog streaming tables with managed snapshot, CDC apply, retries, and schema evolution.
- The standard design splits capture (continuous gateway on classic compute) from apply (scheduled serverless pipeline) through a staging volume; the integrated CDC pipeline merges them for SQL Server and MySQL.
- Each engine has its own prerequisites: SQL Server CT/CDC, MySQL binlog `ROW`/`FULL`, PostgreSQL `wal_level=logical` with a publication and slot.
- SQL Server has no preview label in the docs, MySQL and PostgreSQL connectors are Public Preview, and the MySQL integrated CDC pipeline went GA on Oct 15, 2026.
- Full refresh for database connectors is snapshot-then-atomic-swap, and `auto_full_refresh_policy` automates recovery from truncates and incompatible DDL.

Related Topics

- Change Data Capture (CDC) — Part 1 — CDF, MERGE, and SCD fundamentals
- AUTO CDC API — hand-built CDC in Lakeflow Pipelines
- Lakehouse Federation — query the same databases without ingesting
- Auto Loader — file-based ingestion
- SCD Patterns — SCD type 1 vs type 2 modeling
- Declarative Automation Bundles — Part 1 — deploying pipelines as code

Official Documentation

- Lakeflow Connect connector concepts
- Managed database connectors
- Microsoft SQL Server ingestion connector
- Configure Microsoft SQL Server for ingestion
- Ingest data from SQL Server
- Create an integrated CDC pipeline for MySQL
- Configure MySQL for ingestion
- Configure PostgreSQL for ingestion
- Ingest data from PostgreSQL
- Enable history tracking (SCD type 2)
- Fully refresh target tables
- Managed connector FAQs
- Query-based connectors
- Monitor ingestion gateway progress with event logs
- Monitor managed ingestion pipeline cost
- Connect to managed ingestion sources
- Release notes — July 2026 and October 2026

OpenSharing

Overview

OpenSharing (formerly Delta Sharing) is the secure data sharing platform in Databricks for sharing data and AI assets with users outside your organization, whether or not they use Databricks. Data is shared live without replication; the provider governs access with Unity Catalog securables (**shares** and **recipients**), and the recipient reads through either the **Databricks-to-Databricks (D2D)** protocol or the **Databricks-to-Open (D2O)** protocol. OpenSharing is also the foundation of Databricks Marketplace and Clean Rooms.

Platform update (Oct 2026)

”Delta Sharing is now OpenSharing” (June 10, 2026). The docs moved to `/opensharing/`, but SQL keywords (`CREATE SHARE`, `CREATE RECIPIENT`), the `deltaSharing` Spark format, the `delta-sharing` Python package, and `deltaSharing*` audit events keep their old names. Source: Databricks release notes, June 2026

Overview

- **Share** = read-only collection of assets from one metastore; **recipient** = the organization allowed to read it; **provider** = the sharing organization
- **D2D**: recipient has a Unity Catalog-enabled workspace; identified by a **sharing identifier**; no tokens to manage; supports notebooks, volumes, models, metric views

- **D2O** ("open sharing"): any platform; authenticates with a **bearer-token credential file** or **OIDC federation**; tabular data only
- `WITH HISTORY` enables time travel, streaming reads, and (with CDF enabled) `table_changes()` for recipients

What the Exam Tests

- "Implement OpenSharing (D2D and Databricks-to-Open) ... for privacy-preserving collaboration" — choose the protocol from the recipient's platform and the assets you need to share
- The provider DDL: `CREATE SHARE`, `ALTER SHARE ... ADD TABLE | VIEW | VOLUME | MODEL`, `CREATE RECIPIENT [USING ID ...]`, `GRANT SELECT ON SHARE ... TO RECIPIENT ...`
- Restricting what a recipient sees: partition specs, recipient properties with `CURRENT_RECIPIENT()`, dynamic views, aliases
- The recipient side: `CREATE CATALOG ... USING SHARE provider.share`, credential files, `deltaSharing` streaming reads, and audit via `system.access.audit`

How OpenSharing Works

```
flowchart LR
    subgraph P["Provider (UC-enabled workspace)"]
        T[(Tables, views, volumes,<br/>models, notebooks)]
        S[Share]
        R1[Recipient: DATABRICKS<br/>sharing identifier]
        R2[Recipient: TOKEN / OIDC]
    end
    subgraph D2D["D2D recipient"]
        C1[Catalog created<br/>USING SHARE]
    end
    subgraph D2O["D2O recipient"]
        C2[Spark / pandas / Power BI /<br/>Iceberg clients]
    end
    T --> S
    S -- GRANT SELECT --> R1
    S -- GRANT SELECT --> R2
    R1 -- managed by Databricks --> C1
    R2 -- credential file or IdP JWT --> C2
```

Ways to share with OpenSharing:

1. **Databricks-to-Databricks protocol** — share data and AI assets with users who have a Unity Catalog-enabled workspace (another account, another cloud, or another metastore in your own account).
2. **Databricks-to-Open protocol** — share tabular data managed in Unity Catalog with users on any computing platform.
3. **Customer-managed open-source OpenSharing server** — share from any platform to any platform (not covered by Databricks docs).
4. **SAP BDC Connector** — zero-copy sharing with SAP Business Data Cloud.

Important

There is no need for OpenSharing between workspaces attached to the **same** Unity Catalog metastore — use normal Unity Catalog grants. D2D sharing between metastores in the same account is enabled by default; sharing outside the account requires an account admin to enable OpenSharing on the metastore.

D2D vs D2O at a glance

Aspect	Databricks-to-Databricks (D2D)
Recipient platform	Unity Catalog-enabled Databricks workspace
Recipient identity	**Sharing identifier** `<cloud>:<region>:<uuid>` of the recipient metastore
Credentials	None for the recipient to manage — Databricks handles authentication
Shareable assets	Tables, streaming tables, managed/foreign Iceberg tables, views and dynamic views, r
Recipient governance	Recipient grants access to the mounted catalog with Unity Catalog
Audit/usage tracking	Both provider and recipient
SQL creation	`CREATE RECIPIENT ... USING ID '<sharing-identifier>'`

Provider Setup and Privileges

Privilege	Scope	Used for
`CREATE SHARE`	Metastore	Create sha
`CREATE RECIPIENT`	Metastore	Create reci
`USE SHARE` + `SET SHARE PERMISSION` (or share owner)	Share	Delegated
`USE RECIPIENT` (or recipient owner)	Recipient	View recipi
`SELECT` / `READ VOLUME` / `EXECUTE`	Each shared table/view, volume, model	The share c
`CREATE PROVIDER`, `USE PROVIDER`	Recipient metastore	Recipient s

Account admins enable OpenSharing on the metastore (for sharing outside the account), set the **default recipient token lifetime** (Databricks recommends tokens expire), and set an **organization name** that recipients see. Databricks recommends a **group** as share owner.

Creating Shares

```
CREATE SHARE IF NOT EXISTS customer_data_share
COMMENT 'Curated customer data for partners';

-- Table, with an alias the recipient must use, and full history
ALTER SHARE customer_data_share
ADD TABLE prod.gold.customers
COMMENT 'Customer master'
AS shared.customers
WITH HISTORY;

-- Whole schema: current and future tables/views/volumes/models (no alias or
 ↪  partition allowed)
ALTER SHARE customer_data_share ADD SCHEMA prod.gold;
```

```
-- Views, materialized views, volumes, models
ALTER SHARE customer_data_share ADD VIEW prod.gold.customer_summary_v;
ALTER SHARE customer_data_share ADD MATERIALIZED VIEW prod.gold.daily_sales_mv;
ALTER SHARE customer_data_share ADD VOLUME prod.raw.contracts AS
 ↪  shared.contracts;  -- D2D only
ALTER SHARE customer_data_share ADD MODEL prod.ml.churn_model;
 ↪  -- D2D only

-- Inspect and maintain
SHOW SHARES;
DESCRIBE SHARE customer_data_share;
SHOW ALL IN SHARE customer_data_share;
ALTER SHARE customer_data_share REMOVE TABLE shared.customers;
```

- **`WITH HISTORY` / `WITHOUT HISTORY`** (DBR 12.2 LTS+): `WITH HISTORY` lets recipients run time travel (`VERSION AS OF`, `TIMESTAMP AS OF`), streaming reads, and transactions; for D2D it also shares the Delta log for better read performance. It is the **default on Databricks SQL and DBR 16.2+** (and always for schema sharing); `WITHOUT HISTORY` is the default on DBR 16.1 and below. `WITH CHANGE DATA FEED` is deprecated.
- To let recipients query **CDF** with `table_changes()`, **enable CDF on the table before** sharing it `WITH HISTORY`.
- **Notebook files** are added through Catalog Explorer (*Manage assets → Add notebook file*), D2D only.
- Streaming tables are added with `ADD TABLE`; D2O recipients can read only their **current snapshot** (no time travel, streaming, or CDF).

What you cannot share

- Tables with **row filters or column masks** (share a dynamic view instead), tables with **collations**, `SHALLOW CLONE` tables, tables using **liquid clustering with partition filtering**, R2 tables with V2 checkpoint
- Metric views, volumes, models, and notebooks over D2O
- Assets from more than one metastore in a single share

Limiting what a recipient sees

```
-- Static partition spec: syntax is PARTITION (...), not WITH PARTITION
ALTER SHARE regional_share
ADD TABLE prod.gold.sales
PARTITION (region = 'US');

-- Several partitions in one spec
ALTER SHARE eu_share
ADD TABLE prod.gold.customers
PARTITION (country = 'DE'), (country = 'FR'), (country = 'GB');

-- One share, many recipients: partition by a recipient property (DBR 12.2 LTS+)
CREATE RECIPIENT acme USING ID
 ↪  'aws:us-west-2:19a84bee-54bc-43a2-87de-023d0ec16016'
PROPERTIES ('country' = 'US');

ALTER SHARE global_share
ADD TABLE prod.gold.sales
PARTITION (country = CURRENT_RECIPIENT().country);
```

```sql
-- Dynamic view with row-level filtering per recipient (DBR 14.2+)
CREATE VIEW prod.gold.sales_by_recipient AS
SELECT * FROM prod.gold.sales
WHERE country = CURRENT_RECIPIENT('country');

-- Column-level redaction
CREATE VIEW prod.gold.customers_masked AS
SELECT
  customer_id,
  CASE WHEN CURRENT_RECIPIENT('country') = 'US' THEN email ELSE 'REDACTED' END AS
  ↪ email
FROM prod.gold.customers;

ALTER SHARE global_share ADD VIEW prod.gold.sales_by_recipient;
```

A provider can't query a `CURRENT_RECIPIENT()` view directly; to test it, share it with yourself and query as a recipient. Shareable views must be defined on Delta tables, foreign tables, other shareable views, or local materialized views and streaming tables.

Managing Recipients

Databricks-to-Databricks recipient The recipient finds their sharing identifier (the output of `current_metastore()` on their side, format `<cloud>:<region>:<uuid>`) and sends it to the provider.

```sql
CREATE RECIPIENT IF NOT EXISTS partner_analytics
USING ID 'aws:eu-west-1:g0c979c8-3e68-4cdf-94af-d05c120ed1ef'
COMMENT 'Partner analytics metastore';
-- authentication_type = DATABRICKS
```

Databricks-to-Open recipient (bearer token)

```sql
CREATE RECIPIENT IF NOT EXISTS external_partner
COMMENT 'Non-Databricks partner';
-- authentication_type = TOKEN

DESCRIBE RECIPIENT external_partner;
-- returns activation_link until the credential file is downloaded
```

- Send the **activation link** over a secure channel; the recipient downloads the credential file **only once**.
- Token lifetime defaults to the metastore setting; tokens are valid for a **maximum of one year**.
- **Rotate** with Catalog Explorer or the CLI — there is no SQL ROTATE clause:

  ```
  databricks recipients rotate-token <recipient-name>
  ↪ <existing-token-expire-in-seconds>
  ```

  A recipient has at most two tokens (active + rotated); `0` expires the old token immediately (use it if the token is compromised).
- Optionally restrict D2O recipients with **IP access lists**.

Databricks-to-Open recipient (OIDC federation) Instead of a long-lived token, the provider creates a recipient with an **OIDC token federation policy** (issuer URL of the recipient's IdP such

as Microsoft Entra ID or Okta, plus allowed users, groups, service principals, or apps). Recipients authenticate with their own IdP — **U2M** (for example Power BI, Tableau) or **M2M** (OpenSharing Python/Spark clients) — and Databricks exchanges the IdP's JWT for short-lived OAuth tokens. MFA and token lifetime are enforced by the IdP; email invitations don't support OIDC.

Grants and recipient maintenance

```
GRANT SELECT ON SHARE customer_data_share TO RECIPIENT partner_analytics;  --
 ↪  SELECT is the only share privilege
SHOW GRANT ON SHARE customer_data_share;
SHOW GRANTS TO RECIPIENT partner_analytics;
REVOKE SELECT ON SHARE customer_data_share FROM RECIPIENT partner_analytics;

ALTER RECIPIENT partner_analytics SET PROPERTIES ('country' = 'DE');
ALTER RECIPIENT partner_analytics OWNER TO `data-sharing-admins`;
DROP RECIPIENT partner_analytics;  -- loses access to every share
DROP SHARE customer_data_share;    -- every recipient loses access
```

OpenSharing by email (Beta): invite a recipient by email address and choose Databricks, open sharing, or either; granting the share sends the activation link. Not supported for OIDC, groups, or service principals, or via the `CREATE RECIPIENT` SQL command.

Consuming Shared Data

D2D recipient

```
-- Requires metastore admin, or CREATE CATALOG + USE PROVIDER (or provider
 ↪  ownership)
SHOW PROVIDERS;
CREATE CATALOG IF NOT EXISTS partner_shared
USING SHARE acme_provider.customer_data_share;

GRANT USE CATALOG ON CATALOG partner_shared TO `analysts`;
SELECT * FROM partner_shared.shared.customers;

-- Shared WITH HISTORY (+ CDF enabled by provider)
SELECT * FROM partner_shared.shared.customers VERSION AS OF 3;
SELECT * FROM table_changes('partner_shared.shared.customers', 0, 3);
```

Objects in a shared catalog are **read-only**; recipients can be granted only read access. Shared notebooks live at the catalog level and can be cloned by anyone with `USE CATALOG`. New shares and share updates are cached for about a minute before they appear.

D2O recipient (credential file)

```
# pip install delta-sharing
import delta_sharing

profile = "/Volumes/main/sharing/creds/config.share"
client = delta_sharing.SharingClient(profile)
print(client.list_all_tables())

pdf = delta_sharing.load_as_pandas(f"{profile}
 ↪  #customer_data_share.shared.customers")
```

```
sdf = delta_sharing.load_as_spark(f"{profile}
↪ #customer_data_share.shared.customers")
```

The credential (profile) file is JSON with `shareCredentialsVersion`, `endpoint`, `bearerToken`, and optional `expirationTime`; Databricks-issued files also carry an `icebergEndpoint` for Iceberg REST Catalog clients. Power BI Desktop uses the **OpenSharing** connector with the endpoint URL and bearer token from the file.

Streaming and CDF reads A table shared **with history** (or a streaming table, for D2D) can be a Structured Streaming source with the `deltaSharing` format:

```
d2d_stream = (spark.readStream
    .format("deltaSharing")
    .option("startingVersion", 0)
    .option("ignoreDeletes", "true")
    .option("maxFilesPerTrigger", 10)
    .table("partner_shared.shared.customers"))

d2o_cdf = (spark.readStream
    .format("deltaSharing")
    .option("readChangeFeed", "true")
    .load(
↪ "/Volumes/main/sharing/creds/config.share#customer_data_share.shared.customers"
↪ ))
```

Supported source options include `startingVersion`, `startingTimestamp`, `ignoreDeletes`, `ignoreChanges`, `maxFilesPerTrigger`, `maxBytesPerTrigger`, and `readChangeFeed`. Tables with deletion vectors or column mapping need `responseFormat=delta` for streaming and CDF reads, and you can't change `responseFormat` on a running stream.

Platform update (Oct 2026)

`Trigger.AvailableNow` on OpenSharing streaming sources became GA on July 14, 2026: it captures the shared table's version at start, processes the backlog in multiple micro-batches (honoring `maxFilesPerTrigger` and `maxVersionsPerRpc`), then stops. It needs DBR 18.0+; on earlier runtimes it is converted to `Trigger.Once`. Source: Databricks release notes, July 2026

Iceberg clients Providers can share Delta tables, managed Iceberg tables, foreign Iceberg tables, views, materialized views, and streaming tables with external engines that speak the **Iceberg REST Catalog API** (Snowflake, Trino, Flink, Spark, PyIceberg). Delta tables need `IcebergCompatV3`; assets in default storage can't be shared to Iceberg clients; sharing views/MVs/streaming tables triggers provider-side materialization. Recipients use the `icebergEndpoint` + bearer token from the credential file, or **OIDC federation** (supported for Iceberg clients since June 15, 2026).

Governance, Cost, and Security

Auditing Provider-side OpenSharing events are logged in `system.access.audit` under `service_name = 'unityCatalog'`; actions performed by recipients start with the `deltaSharing` prefix.

```
SELECT
  event_time,
  request_params.recipient_name,
  request_params.share,
```

```
  request_params.schema,
  request_params.name AS table_name,
  source_ip_address
FROM system.access.audit
WHERE service_name = 'unityCatalog'
  AND action_name IN ('deltaSharingQueryTable', 'deltaSharingQueryTableChanges')
ORDER BY event_time DESC;
```

Who pays, and where data is filtered

Recipient compute	Account relationship	Who pays (views)	Access method
Databricks serverless	Any	Recipient	Direct access to underly
Databricks classic	Same account	Recipient	Direct access
Databricks classic	Different account	Recipient (provider's serverless SKU)	Provider performs filterir
D2O connectors	Any	Provider	Provider performs filterir

- Sharing **within a region** has no egress cost; cross-region/cross-cloud sharing can incur cloud egress fees (Cloudflare R2 avoids them). With **SecureConnect** (Public Preview), providers share storage behind a firewall without allowlisting each recipient, and data transfer is billed by Databricks.
- Files are delivered via short-lived **pre-signed URLs** or scoped **cloud tokens**; for eligible tables shared `WITH HISTORY` and without partition filters, cloud tokens give near-direct-read performance — but they are scoped to the table root, so the recipient can also read the Delta log and not-yet-vacuumed deleted data.
- Revoke at any granularity: revoke a share, restrict IPs, filter rows/columns, rotate tokens, or drop the recipient.

Cross-cloud and regulatory domains D2D works across AWS, Azure, and GCP. Sharing across regulatory domains (commercial ↔ AWS GovCloud, GovCloud DoD, Azure China) is in **Public Preview** since June 15, 2026; Azure Government environments are not supported.

Databricks Marketplace

Marketplace is an open exchange built on OpenSharing for datasets, notebooks, AI models, apps, and MCP servers.

- **Consumers** browse the Open Marketplace or the workspace **Marketplace** page, but must request access from a Databricks workspace. Listings are in the **public marketplace** or a provider's **private exchange**; some are instant, others need provider approval. Datasets are typically delivered as catalogs of tabular data (volumes are also supported); a Unity Catalog-enabled workspace gives the deepest integration, while other consumers can use Power BI, pandas, or Spark.
- **Providers** need a premium, Unity Catalog-enabled workspace and must apply through the Databricks Data Partner Program (or use self-service signup for private exchanges only) and follow the provider policies.

Use Cases

Scenario	Approach
Partner on Databricks (any cloud) needs curated gold tables plus notebooks	D2D share; partner mounts
Regulator or customer without Databricks needs a quarterly extract in Power BI	D2O share with bearer toker
One share serving many regional partners, each seeing only their rows	Recipient properties + PARTI
Downstream partner needs incremental change feed	Enable CDF, share `WITH HIS`
Snowflake team wants to query your tables in place	D2O to an Iceberg REST Cat
Monetize a dataset publicly	Marketplace listing
Two companies need joint results without seeing each other's rows	Not a share — use Clean Roo

Common Issues & Errors

Symptom	Cause
`ALTER SHARE ... WITH PARTITION` syntax error	Wrong keyword
Recipient can't time travel or run `table_changes()`	Shared `WITHOUT HISTORY` (old runtime default) or CDF
Activation link missing from `DESCRIBE RECIPIENT`	Credential file already downloaded (one-time)
D2O integrations break on Dec 8, 2026	D2O tokens issued before Dec 8, 2025 with later or no
Can't add a table with row filters/column masks	Not shareable
Provider can't preview a `CURRENT_RECIPIENT()` view	Sharing context only
New table not visible to recipient yet	Share metadata cached ~1 minute; `information_sche`
Workspace admin can't add a table	Inherited `USE CATALOG`/`USE SCHEMA` from the admin gro
Asset vanished from share after dropping a schema	Cascade deletes bypass share protection

Exam Tips

Tip

- **Recipient on Databricks with UC → D2D** (sharing identifier, no token, notebooks/volumes/models allowed). **Anyone else → D2O** (credential file or OIDC, tables only).
- `CREATE RECIPIENT x USING ID '<cloud>:<region>:<uuid>'` = D2D; `CREATE RECIPIENT x` without `USING ID` = token-based D2O.
- `GRANT SELECT ON SHARE s TO RECIPIENT r` — `SELECT` is the only share privilege.
- `PARTITION (...)` (no `WITH`) shares a slice; `CURRENT_RECIPIENT()` lets one share serve many recipients.
- `WITH HISTORY` (default on DBR 16.2+) enables time travel and streaming; CDF must be enabled on the source **before** sharing.
- Rotate tokens with the CLI/Catalog Explorer, not SQL; max token lifetime one year.
- Same metastore? Don't use OpenSharing — just grant.

Key Takeaways

- OpenSharing (formerly Delta Sharing) shares live data without replication through Unity Catalog shares and recipients.
- D2D uses the recipient metastore's sharing identifier and supports non-tabular assets; D2O uses bearer-token credential files or OIDC federation for any client, including Iceberg REST Catalog engines.
- Providers limit exposure with partitions, recipient properties, dynamic views, and aliases; tables with row filters or column masks can't be shared directly.
- History sharing unlocks time travel, streaming (`deltaSharing` source, `Trigger.AvailableNow` GA on DBR 18.0+), and CDF for recipients.
- Every access is auditable in `system.access.audit` (`deltaSharing*` actions under `unityCatalog`), and Marketplace and Clean Rooms build on the same protocol.

Related Topics

- Clean Rooms — privacy-preserving collaboration on top of OpenSharing
- Lakehouse Federation — query external systems without copying
- Unity Catalog — governance foundation for shares and recipients
- Access Control — UC privileges and dynamic views
- Change Data Capture (CDC) — Part 1 — CDF fundamentals used by history sharing

Official Documentation

- What is OpenSharing? (the old `/delta-sharing/` path redirects here)
- What is the Databricks-to-Databricks protocol?
- What is the Databricks-to-Open sharing protocol?
- Set up OpenSharing for your account
- Create shares
- Create recipients (D2D)
- Create recipients with bearer tokens (D2O)
- Enable OIDC federation for recipients
- Manage access to shares
- Read data shared using D2D
- Read data shared with bearer tokens
- Read shares from an Iceberg client using OIDC
- Audit and monitor data sharing
- ALTER SHARE, CREATE RECIPIENT, GRANT ON SHARE
- What is Databricks Marketplace?
- OpenSharing protocol (open source)

Clean Rooms

Overview

Databricks Clean Rooms uses OpenSharing and serverless compute to give multiple organizations a secure, privacy-protecting environment where they can analyze sensitive data together **without direct access to each other's data**. Each party shares tables, views, volumes, and code into a Databricks-managed *central clean room*; only approved code runs there, and results come back as read-only output tables.

Overview

- A clean room = a securable in **each collaborator's Unity Catalog metastore** + an isolated, ephemeral **central clean room** on Databricks-managed serverless compute
- Assets are shared **to the central clean room only** (via OpenSharing), never directly between collaborators; others see only aliases and column names/types
- **No-trust model**: every notebook needs approval from every collaborator except its uploader; only the designated runner can run it
- Two collaboration models: **approval-based** (default, equal privileges) and **packaged** (provider-consumer, provider code and data hidden — GA Jul 31, 2026)

What the Exam Tests

- ”Implement OpenSharing (D2D and Databricks-to-Open) and Clean Rooms for privacy-preserving collaboration” — decide when a clean room is needed instead of a plain share
- Requirements: Unity Catalog, serverless compute, and OpenSharing enabled on the meta-

store
- The workflow: sharing identifiers → create clean room → add assets with aliases/partitions → add notebook and designate runner → approvals → run → read output tables
- Privileges (`CREATE CLEAN ROOM`, `MODIFY CLEAN ROOM`, `EXECUTE CLEAN ROOM TASK`, `MANAGE`, `BROWSE`) and the audit trail (`system.access.clean_room_events`)

How a Clean Room Works

```
flowchart LR
    subgraph A["Creator metastore"]
        AT[(Tables / views / volumes)]
        AN[Notebooks]
        AS[Clean room securable]
        AO[(Output catalog)]
    end
    subgraph C["Central clean room<br/>Databricks-managed serverless"]
        CC1["catalog: creator"]
        CC2["catalog: collaborator"]
        RUN["Approved notebook / JAR run"]
    end
    subgraph B["Collaborator metastore"]
        BT[(Tables / views / volumes)]
        BS[Clean room securable]
        BO[(Output catalog)]
    end
    AT -- OpenSharing --> CC1
    BT -- OpenSharing --> CC2
    AN --> RUN
    CC1 --> RUN
    CC2 --> RUN
    RUN -- output tables --> AO
    RUN -- output tables --> BO
```

When you create a clean room, Databricks creates three things: a clean room securable in your metastore, the **central clean room** (an isolated ephemeral environment managed by Databricks), and a clean room securable in each collaborator's metastore. Data assets each collaborator adds are registered in the central clean room under that collaborator's catalog (by default **`creator`** and **`collaborator`**), so notebook code references them as `creator.sales.california` or `collaborator.advertiser.profiles`, using any aliases.

What each party can see

Shared with other collaborators	Shar
Clean room name, cloud and region of the central clean room	Ever
Your organization name and Clean Room sharing identifier	**Rea**
Aliases of shared tables, views, volumes	
Column metadata (name or alias, type) — **not data**	
Notebooks (read-only), output tables (read-only, temporary)	
Clean room events system table, run history (notebook name, collaborator that ran it, state, start time)	

Requirements

- A workspace enabled for **serverless compute**
- A workspace enabled for **Unity Catalog**
- **OpenSharing enabled** on the Unity Catalog metastore
- Storage reachable from clean room serverless compute: if a bucket sits behind a storage firewall, use Databricks default storage, allow serverless access, or enable **SecureConnect** for clean rooms (SecureConnect is in Public Preview as of the June 15, 2026 release note)

Built-in safeguards

- After creation the clean room is **locked**: no new collaborators can join.
- At most **ten collaborators** (the creator plus up to nine others).
- The name can't be changed and must be unique in every collaborator's metastore.
- If **any** collaborator deletes the clean room, the central clean room is void and nobody can run tasks.
- Comments you add to your securable are not propagated to others.
- You **cannot share metric views** in a clean room.

Collaborators, Roles, and Privileges

- The **creator** is automatically the owner of the clean room in their account; in each collaborator's account, the **metastore admin** becomes the owner. Ownership can be transferred.
- Notebook roles: the **uploader** adds the notebook and picks exactly one **designated runner** (can be themselves).
- Packaged clean rooms add **provider** and **consumer** roles (see below).

Privilege	Granted on	Lets the principal
`CREATE CLEAN ROOM`	Metastore	Create clean rooms; also needed (or metastore admin) to copy yo
`BROWSE`	Clean room	List it and view comments, owner, assets, run history
`MODIFY CLEAN ROOM`	Clean room	`BROWSE` + add/remove data assets and notebooks, update comme
`EXECUTE CLEAN ROOM TASK`	Clean room	`BROWSE` + run notebooks; also required to create the shared outpu
`MANAGE`	Clean room	`BROWSE` + grant privileges, change owner, delete, manage auto-ap

To add a table or view (if you aren't the owner), you **and** the owner need `SELECT` on it — or `READ VOLUME` for a volume — plus `USE CATALOG` and `USE SCHEMA` on its parents.

Step-by-Step Workflow

```
sequenceDiagram
    participant A as Creator
    participant CR as Central clean room
    participant B as Collaborator
    B->>A: Send Clean Room sharing identifier
    A->>CR: Create clean room (type, cloud/region, collaborators, network policy)
    A->>CR: Add tables / views / volumes (aliases, partitions)
    B->>CR: Add own data assets
    A->>CR: Upload notebook, designate runner = Collaborator
    B->>CR: Review diff, approve latest version
    B->>CR: Run notebook (optionally with parameters)
    CR-->>B: Output tables in runner's output catalog
```

1. **Collect sharing identifiers.** A Clean Room sharing identifier is the organization's **global metastore ID + workspace ID + the contact's email**. Each collaborator copies it from *Catalog → Clean Rooms → (org name) → Copy Clean Room sharing identifier*.
2. **Create the clean room** in Catalog Explorer (*Catalog → Clean Rooms → Create Clean Room*) or via the REST API:
   - **Clean room type**: *Approval-based Clean Room* (default) or *Packaged Clean Room*
   - **Name**: no spaces, periods, or forward slashes; immutable
   - **Cloud and region** of the central clean room: cloud must match your workspace, region can differ (data leaving a region can incur egress costs; pick a region that meets residency rules)
   - **Collaborators**: up to ten, by sharing identifier
   - **Network access policy**: *Full Access* or *Restricted Access* (serverless egress control); **cannot be changed later**
   - Compliance profile (HIPAA, FedRAMP Moderate) is inherited from your workspace; collaborators must use the same profile
3. **Add data assets** (+ *Add data assets*): tables, volumes, or views, each with an optional **alias** (the only name visible in the clean room) and, for tables, optional **partition clauses** to share only part of the table.
4. **Add notebooks** (+ *Add notebooks*), name them, and pick the runner. A notebook shared with results shares those results. JAR analyses and JAR UDFs are also supported (GA Oct 13, 2026).
5. **Approve** — every collaborator except the uploader must approve the **latest version**.
6. **Run** — the designated runner clicks **Run**, optionally passing parameters, or schedules it with the Clean Room notebook task in Lakeflow Jobs.
7. **Consume output** — the runner reads output tables in their output catalog.

To test end-to-end alone, use your own sharing identifier (or another user in your metastore): Databricks creates `test_clean_room` and `test_clean_room_collaborator` in your metastore.

Notebook Approval and Version Control

Rule	Detail
Who approves	All collaborators **except the uploader**
Who runs	Only the **designated runner**, even if everyone approved
Versions	Only the **latest** version can be approved, rejected, or run; changing content or the runner cre
Limits	Max 10 versions per notebook; up to ten reviews per collaborator per version (rejecting is alw
Diff view	Version history shows a difference view between versions
Self-authored	You don't need to approve your own notebook to run it (you can still reject it)

Auto-approval rules (owner or `MANAGE`): auto-approve notebooks authored by another collaborator (or *Anyone* in rooms with more than two parties), each rule tied to one runner. Auto-approval is the default for notebooks uploaded by another collaborator when **you** are the designated runner. Auto-approval rules are not available in packaged clean rooms.

Runtime parameters available to clean room notebooks

Parameter	Meaning
`cr_central_id`	Central clean room ID
`cr_runner_global_metastore_id`, `cr_runner_alias`	Who is running
`cr_<alias>_input_catalog`	Catalog holding a collaborator's shared data (`creator`
`cr_output_catalog` + `cr_output_schema`	Runner-only output location

Parameter	Meaning
`cr_shared_output_schema`	Output every collaborator can read (only when shared

Read them with `dbutils.widgets.get("cr_central_id")` in Python or `:cr_central_id` in SQL.

Output Tables and Shared Output

Output tables are temporary, read-only tables a notebook run creates and shares (via OpenSharing) to the **runner's** metastore.

```
-- Runner-only output: a new schema is created for every run
CREATE TABLE identifier(:cr_output_catalog || '.' || :cr_output_schema ||
 ↪  '.overlapping_users') AS
SELECT collab_profiles.*
FROM collaborator.advertiser.profiles AS collab_profiles
JOIN creator.publisher.profiles AS creator_profiles
  ON collab_profiles.email = creator_profiles.email;

-- Shared output: readable by all collaborators as soon as each table is created
CREATE TABLE identifier(:cr_output_catalog || '.' || :cr_shared_output_schema ||
 ↪  '.audience_overlap') AS
SELECT creator_profiles.segment_id, COUNT(DISTINCT collab_profiles.email) AS
 ↪  overlap_count
FROM collaborator.advertiser.profiles AS collab_profiles
JOIN creator.publisher.profiles AS creator_profiles
  ON collab_profiles.email = creator_profiles.email
GROUP BY creator_profiles.segment_id;
```

Aspect	Runner-only output (`cr_output_schema`)
Who can read	The principal who ran the notebook (and the clean room owner); a metastore admin can gra
When readable	After the run **completes successfully**
Retention	Stored 30 days in the central clean room's default storage — copy elsewhere to keep longer
Setup	Create the output catalog once per clean room (`EXECUTE CLEAN ROOM TASK`; default name <
Status	—

Other output rules: only **tables** are supported (not views or volumes); each run creates a new schema (runs can't append to an earlier output table); queries on output tables require **serverless compute**; readers need `SELECT`, `USE CATALOG`, `USE SCHEMA`; deleting the clean room removes all output tables.

Platform update (Oct 2026)

Shared output in clean rooms became generally available on October 13, 2026. On Enterprise-tier accounts (and Premium on Azure) it is off by default until an account admin turns on **Clean Room Shared Output** on the account console Previews page. Source: Databricks release notes, October 2026

Packaged Clean Rooms and Identity Resolution

A **packaged clean room** is a provider-consumer model: the **provider** supplies notebooks, JARs, and data; the **consumer** adds their own data, triggers runs, and sees the output — without seeing the provider's code or data.

Capability	Approval-based	Packaged — consumer	Packaged — provider
View notebooks and JARs	All code visible	Names only, no code	Own code
Add notebooks and JARs	Yes	No	Yes
View data assets	All visible (metadata)	Own assets only	Own assets only
Trigger runs	Yes	Yes	No
View run output	Yes	Yes	No
Auto-approval rules	Yes	No	No

- Packaged mode is chosen at creation and **cannot be changed**.
- Trust shifts to the provider ("similar to using a private library"), so the consumer should **review the egress network policy** before contributing sensitive data — open egress could let provider code send data out.
- **Identity resolution providers** (Acxiom, Adstra, LiveRamp, The Trade Desk): create a ready-to-use packaged clean room in one step from the Clean Rooms page (GA Oct 13, 2026). You are the consumer, the provider is preconfigured as the package provider, and shared output is enabled. Requires `CREATE CLEAN ROOM`.

Platform update (Oct 2026)

Packaged clean rooms became generally available on July 31, 2026, and one-step creation with identity resolution providers became GA on October 13, 2026. Sources: Release notes, July 2026 and October 2026

Privacy Controls Summary

Control	How it protects data
Central clean room isolation	Code runs in a Databricks-managed serverless plane; neither party has admin
Metadata-only visibility	Collaborators see aliases and column names/types, never rows
Code approval + versioning	Nothing runs until all other parties approve the exact latest version
Designated runner	Only one named collaborator can execute a given notebook
Aliases and partition clauses	Hide real object names; share only a slice of a table
Network access policy	*Restricted Access* blocks egress to unapproved destinations
Output tables	Read-only, temporary (30 days), scoped to the runner unless shared output is u
Audit	`system.access.clean_room_events` + audit log service `clean-room`

Monitoring and Auditing

- **System table** `system.access.clean_room_events` (Public Preview) records `CLEAN_ROOM_CREATED`, `CLEAN_ROOM_DELETED`, `RUN_NOTEBOOK_STARTED`, `RUN_NOTEBOOK_COMPLETED`, `RUN_JAR_STARTED`, `RUN_JAR_COMPLETED`, `CLEAN_ROOM_ASSETS_UPDATED`, `OUTPUT_SCHEMA_DELETED`, `ASSET_REVIEW_CREATED`.
- **Audit log**: clean room actions are logged under service `clean-room` (for example `createCleanRoom`, `createCleanRoomAssetReview`, `createCleanRoomAutoApprovalRule`); permission changes on your securable are logged under `unityCatalog`.

```sql
-- Success rate of every clean room notebook
SELECT
  clean_room_name,
  run_notebook_completed_metadata.notebook_name AS notebook_name,
  ROUND(COUNT(CASE WHEN run_notebook_completed_metadata.state = 'SUCCESS' THEN 1
  ↪  END) * 1.0 / COUNT(*) * 100) AS success_rate
FROM system.access.clean_room_events
WHERE event_type = 'RUN_NOTEBOOK_COMPLETED'
GROUP BY clean_room_name, notebook_name
ORDER BY clean_room_name, notebook_name;
```

Clean Rooms vs Plain OpenSharing

Question	OpenSharing share	Clean room
Does the recipient see rows?	**Yes** — read-only access to the shared data	**No** — only metadata; data i
Direction	One-way, provider → recipient	Multi-party (up to 10), each
Who runs compute	Recipient (on their platform/compute)	Central clean room serverle
Code governance	None — recipient queries freely	Every analysis approved by
Non-Databricks parties	Yes, with Databricks-to-Open sharing	No — every collaborator nee
Typical use	Distribute a dataset, data products, Marketplace	Audience overlap, joint mea

Rule of thumb: if the partner may **see** the data, use a share; if neither side may see the other's raw data but both need a joint result, use a clean room.

Use Cases

- **Advertiser + publisher audience overlap** on hashed emails, returning only aggregate counts
- **Retail media measurement**: a retailer and a brand compute campaign lift without exchanging customer files
- **Identity resolution** with a provider (LiveRamp, The Trade Desk, ...) via a one-step packaged clean room
- **Proprietary model or analytic as a product**: a provider packages notebooks/JARs that consumers run on their own data without seeing the IP
- **Healthcare / financial collaboration** under a HIPAA compliance profile inherited by the central clean room

Common Issues & Errors

Symptom	Cause
Can't copy sharing identifier	Missing `CREATE CLEAN ROOM`
Notebook won't run after an edit	New version reset all approvals
Collaborator can't run an approved notebook	They aren't the designated runner
Notebook fails with table not found	Used the original table name instead of `creator./colla`
”Other collaborators have left the clean room”	Another party deleted the central clean room
Data in a firewalled bucket unreadable	Serverless compute blocked by storage firewall
Can't add a new partner later	Clean rooms are locked after creation
Clean room creation fails with mismatched settings	Shared output / JAR settings differ between collaborator

Exam Tips

Tip

- **Clean Rooms = OpenSharing + serverless compute + approved code.** Requirements: UC, serverless, OpenSharing enabled on the metastore.
- Partners never see each other's rows — only **column names/types and aliases**. If the scenario says "neither party may see the other's raw data", pick a clean room over a share.
- Every notebook needs approval from **all collaborators except the uploader**, and only the **designated runner** can run it; any change creates a new version and resets reviews.
- Max **10 collaborators**, no joining after creation, no renaming; any party deleting it ends it for everyone.
- Output tables: read-only, runner-only by default, kept **30 days**; **shared output** (GA Oct 2026) makes results visible to all collaborators.
- **Packaged clean rooms** (GA Jul 2026) = provider-consumer, provider code hidden, no auto-approval; mode fixed at creation.

Key Takeaways

- A clean room creates a securable in every collaborator's metastore plus an isolated central clean room; assets are shared only into the central room via OpenSharing.
- The no-trust model rests on code approval, versioning, a single designated runner, and a fixed network access policy.
- Privileges map cleanly: `CREATE CLEAN ROOM` (metastore), then `BROWSE`, `MODIFY CLEAN ROOM`, `EXECUTE CLEAN ROOM TASK`, `MANAGE` on the clean room.
- Results flow back as temporary read-only output tables (runner-only or shared output), and every action is auditable in `system.access.clean_room_events`.
- Use OpenSharing to distribute data; use Clean Rooms when collaboration must happen without exposing raw data.

Related Topics

- OpenSharing — the sharing protocol underneath clean rooms
- Unity Catalog — securables and privileges
- Access Control — UC grants and fine-grained access
- Audit, Lineage, and Network Security — audit logs and serverless egress control
- System Tables — querying `system.access.*`

Official Documentation

- What is Databricks Clean Rooms?
- Create clean rooms
- Run notebooks in clean rooms
- Create and work with output tables in Databricks Clean Rooms
- Packaged clean rooms
- Manage clean rooms
- Work with Databricks clean rooms as an invited collaborator
- Run JAR workloads in Databricks clean rooms
- Clean room events system table reference
- Unity Catalog privileges reference
- Release notes — June 2026, July 2026, and October 2026

Lakehouse Federation

Overview

Lakehouse Federation is the Databricks query federation platform: governed, read-only access to data in external systems — MySQL, PostgreSQL, SQL Server, Oracle, Teradata, Redshift, Snowflake, BigQuery, Azure Synapse, Salesforce Data 360, other Databricks workspaces, and external catalogs such as Hive metastore and AWS Glue — **without ETL**. Data stays in the source; Unity Catalog governs access through a **connection** (which holds the credentials) and a **foreign catalog** (which mirrors the external database). The supported-source list grows over time; check the supported data sources page for the current set.

Overview

- **Connection** — a UC securable storing the path and credentials for an external system (`CREATE CONNECTION`); credentials should be passed with the `secret()` function
- **Foreign catalog** — a UC catalog that mirrors an external database or catalog and is queried with three-level names (`CREATE FOREIGN CATALOG ... USING CONNECTION`)
- **Two flavors**: *query federation* (SQL pushed down over JDBC, runs on Databricks and the remote engine) and *catalog federation* (Databricks reads the foreign table's files directly from object storage)
- **Read-only**: federated queries can't write back, except for a federated legacy *internal* Hive metastore
- **Governance** — UC privileges, lineage, search, and audit apply to foreign tables exactly as to native ones

What the Exam Tests

- "Configure Lakehouse Federation with governance across supported source systems, applying UC permissions and connection-level credentials"
- The privilege chain: `CREATE CONNECTION` (metastore) → `CREATE CATALOG` (metastore) + `CREATE FOREIGN CATALOG` (connection) → `USE CATALOG` / `USE SCHEMA` / `SELECT` for readers; `USE CONNECTION` for `remote_query` and ingestion
- Choosing **query federation vs catalog federation vs ingestion** (Lakeflow Connect) for a scenario
- **Pushdown** behaviour and how to confirm it (Query Profile, `EXPLAIN FORMATTED`)

Architecture

```
flowchart LR
    subgraph DBX["Databricks workspace"]
        Q[SQL query / notebook]
        UC["Unity Catalog<br/>privileges, lineage, audit"]
        CONN["Connection<br/>host + credentials"]
        FC["Foreign catalog"]
    end
    subgraph QF["Query federation"]
        DB[(Postgres / SQL Server /<br/>Snowflake / BigQuery ...)]
    end
    subgraph CF["Catalog federation"]
        HMS[[Hive metastore / Glue /<br/>Snowflake Horizon]]
        OBJ[(Object storage files)]
    end
    Q --> UC --> FC
```

```
CONN -.-> FC
FC -- "pushed-down SQL over JDBC" --> DB
DB -- results --> FC
FC -- metadata --> HMS
FC -- "direct file reads<br/>(authorized paths)" --> OBJ
```

All federated traffic goes **directly between Databricks compute and the external system**; neither Unity Catalog nor the control plane sits on the data path (OAuth token exchange is the exception — it runs from the control plane).

Query federation vs catalog federation

	Query federation
How it reads	Pushes the query down to the external database over JDBC; runs on both Databricks and remot
Sources	MySQL, PostgreSQL, Teradata, Oracle, Amazon Redshift, Salesforce Data 360, Snowflake, Micros
Setup	Connection → foreign catalog → grants
Writes	Not supported
Best for	Ad hoc reporting, BI, proof-of-concept access to operational databases

Note

For Snowflake, only Snowflake-managed **Iceberg** tables are eligible for catalog federation; other Snowflake tables are always read through query federation.

Privileges and Connection-Level Credentials

Who needs what

Step	Privilege
Create a connection	`CREATE CONNECTION` (or metastor
Create a foreign catalog	`CREATE CATALOG` **and** (`CREATE FO`
Set authorized paths (catalog federation)	`CREATE FOREIGN SECURABLE`
List / view connection details	`USE CONNECTION`
Run `remote_query` or build Lakeflow ingestion pipelines on the connection	`USE CONNECTION`
Read federated data	`USE CATALOG` + `USE SCHEMA` + SE
Grant on the connection	Connection owner or metastore a

In workspaces enabled for Unity Catalog automatically, workspace admins hold `CREATE CONNECTION` and `CREATE CATALOG` by default. `ALL PRIVILEGES` on a connection implies `CREATE FOREIGN CATALOG` and `USE CONNECTION`.

```
GRANT CREATE FOREIGN CATALOG ON CONNECTION my_postgres TO `data-platform-admins`;
GRANT USE CONNECTION ON CONNECTION my_postgres TO `ingestion-engineers`;

GRANT USE CATALOG ON CATALOG postgres_prod TO `analysts`;
GRANT USE SCHEMA ON SCHEMA postgres_prod.public TO `analysts`;
GRANT SELECT ON TABLE postgres_prod.public.orders TO `analysts`;

REVOKE USE CONNECTION ON CONNECTION my_postgres FROM `ingestion-engineers`;
```

Connection-level credentials

- Credentials are defined **once, on the connection** — not in notebooks, cluster configs, or `spark.conf`. The foreign catalog takes its credentials from the named connection, so `CREATE FOREIGN CATALOG` never contains a password.
- Every query through the foreign catalog reaches the source with the **connection's** credential; Unity Catalog decides *which Databricks users* may run those queries. Give the source account only the database rights you are willing to expose through UC, and use source-side permissions to hide databases or tables from it.
- Provide secrets with the `secret()` function rather than literals.
- Authentication options vary by connector: username/password for most; **AWS IAM** authentication for MySQL and PostgreSQL (`aws_region` + `credential` options); **OAuth** for Snowflake (built-in OAuth, Microsoft Entra ID, Okta, OAuth access token) plus PEM private key or basic auth; OAuth / OAuth M2M with Microsoft Entra ID or basic auth for SQL Server.
- With Snowflake built-in OAuth, the connection creator signs in once; refresh tokens default to a 90-day validity (`OAUTH_REFRESH_TOKEN_VALIDITY`), after which the connection must be re-authenticated.
- **Dedicated access mode** (formerly single user) clusters can use a connection only if the user **owns the connection**.

Creating a Connection and Foreign Catalog

Query federation (PostgreSQL)

```sql
-- Step 1: credentials live in a UC connection
CREATE CONNECTION IF NOT EXISTS my_postgres
TYPE POSTGRESQL
OPTIONS (
  host 'pg.example.com',
  port '5432',
  user secret('pg_scope', 'pg_user'),
  password secret('pg_scope', 'pg_pwd')
);

-- Step 2: mirror one database as a foreign catalog (MySQL omits the database
 ↪  option)
CREATE FOREIGN CATALOG IF NOT EXISTS postgres_prod
USING CONNECTION my_postgres
OPTIONS (database 'prod_db');

-- Step 3: query like any UC catalog
SELECT order_id, amount
FROM postgres_prod.public.orders
WHERE order_date >= current_date() - INTERVAL 7 DAYS;
```

`CREATE CONNECTION` types documented in the SQL reference: `DATABRICKS`, `HTTP`, `MYSQL`, `POSTGRESQL`, `REDSHIFT`, `SNOWFLAKE`, `SQLDW`, `SQLSERVER` (other sources, such as Glue or Hive metastore, have their own connection types on their setup pages). Required options:

Type	Required options
`MYSQL`, `POSTGRESQL`, `REDSHIFT`, `SQLDW`, `SQLSERVER`	`host`, `port`, `user`, `password`
`SNOWFLAKE`	`host`, `port`, `sfWarehouse`, `user`, `password` (basic auth)
`DATABRICKS`	`host`, `httpPath`, `personalAccessToken`

The Catalog Explorer wizard (*Catalog → Add → Create a connection*) creates the connection **and** the foreign catalog in one flow, lets you test the connection, bind the catalog to specific workspaces, set the owner, and grant privilege presets (all account users get `BROWSE` by default; *Data Reader* grants read).

Catalog federation (AWS Glue)

```
CREATE CONNECTION glue_conn TYPE glue
OPTIONS (
  aws_region '<aws-region-ID>',
  aws_account_id '<aws-account-ID>',
  credential '<service-credential-ID>'
);

-- External locations must already cover these paths
CREATE FOREIGN CATALOG IF NOT EXISTS glue_catalog
USING CONNECTION glue_conn
OPTIONS (
  authorized_paths 's3://demo,s3://depts/finance',
  storage_root 's3://my-catalog-metadata'
);
```

Authorized paths are a guardrail: the federated catalog can only read tables whose locations fall under them. Without them, a user who can edit Hive metastore metadata could repoint a table they can SELECT at a sensitive path that UC's storage credential can reach. A `storage_root` is required to read Iceberg tables from a Glue/Hive-federated catalog.

Metadata refresh Unity Catalog refreshes foreign-table metadata automatically at query time. Refresh proactively (for example from a scheduled Lakeflow Job) when external engines change the source or to avoid refresh latency on the first query:

```
REFRESH FOREIGN CATALOG postgres_prod;
REFRESH FOREIGN SCHEMA postgres_prod.public;
REFRESH FOREIGN TABLE postgres_prod.public.orders;
```

Foreign table and column comments can be synced from supported sources (Beta since June 17, 2026; DBR 18.2+ or DBSQL 2026.15+).

Pushdown and Performance

Query federation translates Databricks SQL into the source dialect and pushes down what the source supports. PostgreSQL, for example:

Pushed down
Filters, projections, `LIMIT`, `OFFSET`, aggregates, sorting with limit, `TABLESAMPLE`, arithmetic, boolean and bitwi

- **Confirm pushdown**: click the foreign data source scan node in the Query Profile graph, or run `EXPLAIN FORMATTED`, to see the SQL actually sent to the source.
- **One stream per foreign table**: each referenced foreign table becomes a subquery whose result returns to one executor task over a single stream — huge result sets can run out of memory.
- **No Databricks caching**: Result Cache and Disk Cache don't apply to federated queries.

- **Concurrency**: throttling follows the Databricks SQL concurrent query limit; there is no per-connection limit across warehouses.
- **Repeated or heavy reads**: Databricks recommends loading federated data into a **materialized view** and querying that:

  ```
  CREATE MATERIALIZED VIEW main.silver.orders_mv AS
  SELECT * FROM postgres_prod.public.orders;
  ```

remote_query for native SQL The `remote_query` table-valued function runs SQL written in the **source's own dialect** on a connection (MySQL, PostgreSQL, Teradata, Oracle, Redshift, Snowflake, SQL Server, BigQuery, or a JDBC connection). It needs `USE CONNECTION` (or `MANAGE` on single-user clusters), DBR 18.3+, or a Pro/Serverless SQL warehouse 2025.35+.

```
SELECT * FROM remote_query(
  'my_postgres',
  database => 'prod_db',
  query => 'SELECT region, SUM(amount) AS revenue FROM orders GROUP BY region'
);

-- Delegate access without granting USE CONNECTION: wrap it in a view
CREATE VIEW main.reporting.revenue_by_region AS
SELECT * FROM remote_query('my_postgres', database => 'prod_db', dbtable =>
 ↪  'public.revenue');

GRANT SELECT ON VIEW main.reporting.revenue_by_region TO `analysts`;
```

When users query such a view, the connection check uses the **view owner's** privileges, so the view owner must hold `USE CONNECTION`.

Governance Story

- **Access control**: foreign catalogs, schemas, and tables are UC securables — grant, revoke, set owners, apply tags, and bind catalogs to workspaces like native catalogs.
- **Lineage and audit**: Unity Catalog is the governance layer — it performs the access control checks and auditing and infers lineage for queries on foreign tables, while the read itself uses the source (query federation) or the external catalog's metadata (catalog federation). Query federation is documented as bringing UC fine-grained access control, data lineage, and search to external data.
- **Least privilege at the source**: the connection's database user bounds what *any* Databricks user could ever reach through that connection; UC narrows it per principal.
- **Credential hygiene**: one place to rotate (the connection or the secret it references) instead of per-notebook secrets.
- **Sharing onward**: foreign tables and schemas can be added to OpenSharing shares; results are materialized on the provider side (no `LIMIT` or predicate pushdown for the recipient).

Use Cases

Scenario	Recommendation
Ad-hoc analytics or BI over an operational database	**Query federation** — no pipeline overhea
Proof of concept before building an ETL pipeline	**Query federation**, then move to ingestio
Low-QPS source-of-truth lookups	**Query federation** — always current
Large fact-table transformations or frequent heavy joins	**Ingest** with Lakeflow Connect — Databric

Scenario	Recommendation
Data registered in Glue / Hive metastore during a UC migration	**Catalog federation** (direct file reads, Da
Snowflake-managed Iceberg tables	**Catalog federation**; other Snowflake tab
Recurring incremental loads from a federated source without CDC	Lakeflow Connect **query-based connect**
Need writes to the external system or custom parallelism	Spark data source / JDBC connection, not

Common Issues & Errors

- **Pushdown missing** — a function or construct the source dialect doesn't support (for example a window function) is evaluated in Databricks after pulling rows; check the Query Profile or `EXPLAIN FORMATTED`.
- **Executor out of memory** — a large foreign-table result streams to a single task; filter so pushdown reduces rows, or materialize into Delta.
- **Authentication failures after a password rotation** — the secret or connection wasn't updated; with Snowflake OAuth the refresh token expired and the connection must be re-authenticated.
- **Network reachability** — compute must reach the host: stable egress IP or Private Link for classic compute, Databricks-published serverless outbound IPs for serverless; OAuth connections also need the control plane to reach the IdP endpoint.
- **Dedicated cluster can't use the connection** — dedicated access mode works only for the connection owner; use standard access mode or a SQL warehouse.
- **Missing or colliding tables** — names invalid in UC are skipped; table/schema names are lowercased, so case-only differences collide (Synapse and Redshift case-sensitive identifiers aren't supported).
- **Catalog federation read of a table outside authorized paths fails** — extend the authorized paths (requires `CREATE FOREIGN SECURABLE` on the external location).

Exam Tips

Tip

- ”Query Snowflake / BigQuery / Postgres **without copying data**, governed by UC” → **Lakehouse Federation**. ”Continuously replicate changes” → Lakeflow Connect instead.
- Order of DDL: **`CREATE CONNECTION`** (credentials) → **`CREATE FOREIGN CATALOG ... USING CONNECTION`** → **`GRANT USE CATALOG / USE SCHEMA / SELECT`**.
- `CREATE FOREIGN CATALOG` needs `CREATE CATALOG` on the metastore **plus** `CREATE FOREIGN CATALOG` on the connection (or ownership).
- Credentials live **on the connection** (use `secret()`); readers never see them and only need UC data privileges.
- Query federation pushes SQL down over JDBC; **catalog federation** reads files directly with Databricks compute and needs external locations + authorized paths.
- Federation is **read-only** (except a federated internal Hive metastore) and bypasses Databricks result/disk caching.

Key Takeaways

- Federation = query path, not data path: data stays in the source and Unity Catalog governs access through connections and foreign catalogs.
- Connection-level credentials centralize secrets; UC privileges (`USE CATALOG`, `USE SCHEMA`, `SELECT`) decide who can use them, and `USE CONNECTION` covers `remote_query` and ingestion use.
- Query federation suits ad hoc and BI access; catalog federation suits Hive/Glue/Snowflake-Iceberg migrations and hybrids; Lakeflow Connect suits high-volume, low-latency needs.

- Pushdown is per-source and partial; verify it in the Query Profile and materialize repeated heavy reads.
- Governance (lineage, audit, workspace binding, tags) applies to foreign objects just like native UC objects.

Related Topics

- Lakeflow Connect Database CDC — ingest instead of federate
- OpenSharing — open cross-org sharing (different problem, same domain)
- Unity Catalog — the governance plane for federation
- Access Control — UC privileges and grants
- Secret Management — secrets referenced by connections

Official Documentation

- Connect to external databases and catalogs
- What is query federation?
- What is catalog federation?
- Hive metastore federation
- Enable Hive metastore federation for AWS Glue
- Enable Snowflake catalog federation
- Run federated queries on PostgreSQL
- Run federated queries on Snowflake (OAuth)
- Manage connections for Lakehouse Federation
- Query external databases using `remote_query`
- Networking recommendations for Lakehouse Federation
- Comments on foreign tables
- `CREATE CONNECTION`
- `CREATE CATALOG` / `CREATE FOREIGN CATALOG`
- Unity Catalog privileges reference

Section 3: Data Manipulation (12 % of Exam)

Transforming data once it has landed: window functions, joins and aggregations in Spark SQL and PySpark, Delta DML (MERGE and friends), deduplication, the VARIANT type for semi-structured data, AI functions such as `ai_query` for in-pipeline enrichment, and Lakeflow Pipelines expectations that quarantine, drop or fail bad records. This domain replaces the former *Data Transformation, Cleansing, and Quality* domain.

Topics Overview

```
flowchart LR
    D[Data Manipulation]
    D --> T1["Batch ETL Pipelines"]
    D --> T2["Delta Lake Operations"]
    D --> T3["Data Deduplication"]
    D --> T4["VARIANT and Semi-Structured Data"]
    D --> T5["AI Functions in Data Pipelines"]
    D --> T6["Expectations and Data Quality"]
```

Section Contents

File	Topic
01-batch-etl-pipelines-part1.md	Reads, DataFrame transformations, joins, aggregations, window functio
01-batch-etl-pipelines-part2.md	Batch performance and error handling
02-delta-lake-operations-part1.md	MERGE patterns, OPTIMIZE, VACUUM, time travel, cloning
02-delta-lake-operations-part2.md	Schema operations, table properties, transaction log
03-data-deduplication.md	Batch and streaming deduplication, idempotent writes
04-variant-semi-structured-data.md	VARIANT, `parse_json`, `variant_get`, path access, shredding
05-ai-functions.md	`ai_query` and task-specific AI functions for enrichment and classificatio
06-expectations-data-quality.md	Expectations: warn, drop, fail, and the quarantine pattern

Exam Skill Coverage

Every skill bullet in Section 3 of the Oct 9, 2026 exam guide, with the files that cover it.

Exam guide skill
Apply advanced data transformations, including window functions, joins, and aggregations, using Spark SQL a
Develop a model and query semi-structured data using the VARIANT data type and related functions (e.g., par
Apply AI functions, including ai_query, to perform model inference within data pipelines for enrichment and cl
Implement data quality expectations in Lakeflow Pipelines to quarantine, drop, or fail on bad records.

Key Concepts to Master

Concept	Why it matters
Window functions	`row_number`, `rank`, `lag`/`lead` and running totals over a partitioned, ordered window
VARIANT	Schema-flexible binary encoding of JSON; `parse_json` to load, `:` paths and `variant_ge`
ai_query	Calls a model serving endpoint row by row from SQL or PySpark for enrichment and clas
Expectations	`expect` (warn), `expect_or_drop` (drop), `expect_or_fail` (fail); quarantine by inverting

Related Resources

- Delta Lake cheat sheet
- SQL functions cheat sheet (shared)
- Lakeflow Pipelines cheat sheet (shared)

Batch ETL Pipelines — Part 1

Batch ETL is foundational to data engineering. This part covers DataFrame transformations, read-/write operations, join strategies, aggregations, window functions, and UDFs. Part 2 covers performance optimization, error handling, use cases, and exam tips.

Overview

```
flowchart LR
    subgraph Extract
        S1[Files] --> Read
        S2[Tables] --> Read
        S3[APIs] --> Read
    end

    subgraph Transform
        Read --> Filter
        Filter --> Join
        Join --> Aggregate
        Aggregate --> Clean
    end

    subgraph Load
        Clean --> Write[Write to Delta]
    end
```

Overview

- **Reading data** with explicit schemas and corrupt-record handling
- **Transformations**: column operations, join types, join strategies and hints, aggregations, and pivots
- **Window functions**: ranking, `lead` / `lag`, running totals, and deduplication with `row_number()`
- **Writing data**: write modes, partitioned writes, dynamic partition overwrite, CTAS, and `INSERT`

What the Exam Tests

- Apply advanced data transformations, including window functions, joins, and aggregations, using Spark SQL and PySpark to process large datasets.
- Write the same transformation in Spark SQL and in PySpark
- Choose a join strategy or hint based on the size of each side

ETL vs ELT

Aspect	ETL	ELT
Transform location	Before loading	After loading
Best for	Structured data, compliance	Raw data, data lakes
Databricks approach	Use ELT with medallion architecture	
Flexibility	Less flexible	More flexible

Medallion Architecture (Bronze/Silver/Gold)

```
flowchart LR
    Raw[Raw Data] --> Bronze[Bronze Layer]
    Bronze --> Silver[Silver Layer]
    Silver --> Gold[Gold Layer]

    Bronze -.- B1[Raw ingestion]
    Bronze -.- B2[Minimal transformation]
    Bronze -.- B3[Append-only]

    Silver -.- S1[Cleaned data]
    Silver -.- S2[Deduplicated]
    Silver -.- S3[Schema enforced]

    Gold -.- G1[Business aggregates]
    Gold -.- G2[Ready for BI]
    Gold -.- G3[Optimized for queries]
```

Reading Data

Read from Files

```
# Read Parquet

df = spark.read.format("parquet").load("/path/to/files")

# Read CSV with options

df = (spark.read.format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .option("multiLine", "true")
    .load("/path/to/files/*.csv"))

# Read JSON

df = (spark.read.format("json")
    .option("multiLine", "true")
    .load("/path/to/files"))

# Read Delta

df = spark.read.format("delta").load("/path/to/delta")
df = spark.table("catalog.schema.table_name")
```

Read Options by Format

Format	Key Options
CSV	`header`, `inferSchema`, `delimiter`, `multiLine`, `quote`, `escape`
JSON	`multiLine`, `primitivesAsString`, `allowComments`
Parquet	`mergeSchema`
Delta	`versionAsOf`, `timestampAsOf`

Schema Definition

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType,
 ↪  TimestampType

# Define schema explicitly (recommended for production)

schema = StructType([
    StructField("id", IntegerType(), nullable=False),
    StructField("name", StringType(), nullable=True),
    StructField("created_at", TimestampType(), nullable=True)
])

df = (spark.read.format("json")
    .schema(schema)
    .load("/path/to/files"))
```

Handling Corrupt Records

```
# Mode options for corrupt records

df = (spark.read.format("json")
    # Default: nulls for corrupt fields
    .option("mode", "PERMISSIVE")
    .option("columnNameOfCorruptRecord", "_corrupt_record")
    .load("/path/to/files"))

# DROPMALFORMED: Skip corrupt records

df = (spark.read.format("csv")
    .option("mode", "DROPMALFORMED")
    .load("/path/to/files"))

# FAILFAST: Fail immediately on corrupt record

df = (spark.read.format("csv")
    .option("mode", "FAILFAST")
    .load("/path/to/files"))
```

Mode	Behavior
`PERMISSIVE`	Set corrupt fields to null, store in `_corrupt_record`
`DROPMALFORMED`	Skip corrupt records
`FAILFAST`	Throw exception on first corrupt record

DataFrame Transformations

Select and Column Operations

```python
from pyspark.sql.functions import col, lit, when, coalesce, concat, upper

# Select columns

df.select("col1", "col2")
df.select(col("col1"), col("col2").alias("renamed"))

# Add new column

df.withColumn("new_col", lit("constant"))
df.withColumn("full_name", concat(col("first"), lit(" "), col("last")))

# Conditional column

df.withColumn("status",
    when(col("amount") > 100, "high")
    .when(col("amount") > 50, "medium")
    .otherwise("low")
)

# Handle nulls

df.withColumn("value", coalesce(col("primary"), col("backup"), lit(0)))
```

Filter Operations

```python
# Basic filter

df.filter(col("status") == "active")
df.filter("status = 'active'")  # SQL expression

# Multiple conditions

df.filter((col("amount") > 100) & (col("status") == "active"))
df.filter((col("region") == "US") | (col("region") == "CA"))

# Null handling

df.filter(col("email").isNotNull())
df.filter(col("phone").isNull())

# String operations

df.filter(col("name").like("%Smith%"))
df.filter(col("name").rlike("^[A-Z].*"))  # Regex
df.filter(col("email").contains("@company.com"))
```

Type Casting

```python
from pyspark.sql.functions import col, to_date, to_timestamp

# Cast types

df.withColumn("amount", col("amount").cast("double"))
df.withColumn("id", col("id").cast("integer"))
```

```
# Date/timestamp conversions

df.withColumn("date", to_date(col("date_string"), "yyyy-MM-dd"))
df.withColumn("timestamp", to_timestamp(col("ts_string"), "yyyy-MM-dd HH:mm:ss"))
```

SQL Expressions

```
from pyspark.sql.functions import expr

# Use SQL expressions in DataFrame API

df.withColumn("discount_price", expr("price * (1 - discount_rate)"))
df.selectExpr("*", "price * quantity AS total")
```

Join Operations

Join Types

```
flowchart TD
    subgraph Inner Join
        I1[Left: 1,2,3] --- I2[Right: 2,3,4]
        I2 --> IR[Result: 2,3]
    end

    subgraph Left Join
        L1[Left: 1,2,3] --- L2[Right: 2,3,4]
        L2 --> LR[Result: 1,2,3]
    end

    subgraph Full Outer
        F1[Left: 1,2,3] --- F2[Right: 2,3,4]
        F2 --> FR[Result: 1,2,3,4]
    end
```

```
# Inner join (default)

df1.join(df2, df1.id == df2.id, "inner")

# Left join

df1.join(df2, df1.id == df2.id, "left")

# Right join

df1.join(df2, df1.id == df2.id, "right")

# Full outer join

df1.join(df2, df1.id == df2.id, "full")

# Left anti join (rows in left not in right)

df1.join(df2, df1.id == df2.id, "left_anti")

# Left semi join (rows in left that have match in right)
```

```
df1.join(df2, df1.id == df2.id, "left_semi")

# Cross join

df1.crossJoin(df2)
```

Join on Multiple Columns

```
# Multiple join conditions

df1.join(df2,
    (df1.id == df2.id) & (df1.date == df2.date),
    "inner"
)

# Same column names (simpler syntax)

df1.join(df2, ["id", "date"], "inner")
```

Join Strategies

```
flowchart TD
    Start[Choose Join Strategy] --> Size{Small table < 10MB?}
    Size -->|Yes| Broadcast[Broadcast Join]
    Size -->|No| Keys{Keys sortable?}
    Keys -->|Yes| SortMerge[Sort-Merge Join]
    Keys -->|No| ShuffleHash[Shuffle Hash Join]
```

Strategy	When Used	Performance
Broadcast	Small table (< 10MB default)	Fast, no shuffle
Sort-Merge	Large tables, sorted keys	Good for equi-joins
Shuffle Hash	Large tables, unsorted	Expensive shuffle

Join Hints

```
from pyspark.sql.functions import broadcast

# Force broadcast join

df1.join(broadcast(df2), "id")

# SQL hints

spark.sql("""
    SELECT /*+ BROADCAST(small_table) */ *
    FROM large_table
    JOIN small_table ON large_table.id = small_table.id
""")

# Merge hint (sort-merge join)

spark.sql("""
    SELECT /*+ MERGE(df2) */ *
    FROM df1 JOIN df2 ON df1.id = df2.id
```

```
""")

# Shuffle hash hint

spark.sql("""
    SELECT /*+ SHUFFLE_HASH(df2) */ *
    FROM df1 JOIN df2 ON df1.id = df2.id
""")
```

Broadcast Threshold

```
# Default broadcast threshold is 10MB
# Increase for larger dimension tables

spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "50MB")

# Disable auto broadcast

spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1")
```

Aggregations

Basic Aggregations

```
from pyspark.sql.functions import count, sum, avg, min, max, countDistinct

# Single aggregation

df.agg(count("*").alias("total"))

# Multiple aggregations

df.agg(
    count("*").alias("total_rows"),
    sum("amount").alias("total_amount"),
    avg("amount").alias("avg_amount"),
    countDistinct("customer_id").alias("unique_customers")
)
```

Group By

```
# Group by single column

df.groupBy("region").agg(
    count("*").alias("order_count"),
    sum("amount").alias("total_amount")
)

# Group by multiple columns

df.groupBy("region", "product_category").agg(
    sum("amount").alias("total")
)
```

Pivot

```
# Pivot table

df.groupBy("region").pivot("year").agg(sum("amount"))

# Pivot with specific values (more efficient)

df.groupBy("region").pivot("year", [2022, 2023, 2024]).agg(sum("amount"))
```

Window Functions

Window functions compute values across a set of rows related to the current row.

Window Specification

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, rank, dense_rank, lead, lag, sum

# Basic window

window = Window.partitionBy("customer_id").orderBy("order_date")

# Window with frame

window_frame = (Window.partitionBy("customer_id")
    .orderBy("order_date")
    .rowsBetween(Window.unboundedPreceding, Window.currentRow))
```

Ranking Functions

```
# Row number (unique rank)

df.withColumn("row_num", row_number().over(window))

# Rank (gaps for ties)

df.withColumn("rank", rank().over(window))

# Dense rank (no gaps for ties)

df.withColumn("dense_rank", dense_rank().over(window))
```

Function	Ties Handling	Example: [100, 100, 90]
row_number()	Arbitrary order	1, 2, 3
rank()	Same rank, skip next	1, 1, 3
dense_rank()	Same rank, no skip	1, 1, 2

Lead and Lag

```
# Next value

df.withColumn("next_order_date", lead("order_date", 1).over(window))

# Previous value
```

```
df.withColumn("prev_amount", lag("amount", 1).over(window))

# With default value

df.withColumn("prev_amount", lag("amount", 1, 0).over(window))
```

Running Totals

```
# Cumulative sum

window_running = (Window.partitionBy("customer_id")
    .orderBy("order_date")
    .rowsBetween(Window.unboundedPreceding, Window.currentRow))

df.withColumn("running_total", sum("amount").over(window_running))
```

Deduplication with Window Functions

```
# Keep latest record per customer

window = Window.partitionBy("customer_id").orderBy(col("updated_at").desc())

(df.withColumn("rn", row_number().over(window))
    .filter(col("rn") == 1)
    .drop("rn"))
```

Writing Data

Write Modes

```
# Append - add to existing data

df.write.format("delta").mode("append").save("/path/to/table")

# Overwrite - replace all data

df.write.format("delta").mode("overwrite").save("/path/to/table")

# Error (default) - fail if data exists

df.write.format("delta").mode("error").save("/path/to/table")

# Ignore - skip if data exists

df.write.format("delta").mode("ignore").save("/path/to/table")
```

Mode	Behavior
append	Add new records to existing data
overwrite	Replace all existing data
error / errorifexists	Fail if target exists
ignore	Do nothing if target exists

Write to Tables

```python
# Write to managed table

df.write.format("delta").saveAsTable("catalog.schema.table_name")

# Write to table (append)

df.write.format("delta").mode("append").saveAsTable("catalog.schema.table_name")

# Insert into existing table

df.write.insertInto("catalog.schema.table_name")
```

Partitioning

```python
# Write with partitioning

(df.write.format("delta")
    .partitionBy("year", "month")
    .mode("overwrite")
    .save("/path/to/table"))

# Overwrite specific partitions

(df.write.format("delta")
    .mode("overwrite")
    .option("replaceWhere", "year = 2024 AND month = 1")
    .save("/path/to/table"))
```

Dynamic Partition Overwrite

```python
# Only overwrite partitions that have data in the DataFrame

spark.conf.set("spark.sql.sources.partitionOverwriteMode", "dynamic")

(df.write.format("delta")
    .mode("overwrite")
    .partitionBy("date")
    .save("/path/to/table"))
```

SQL Batch Operations

CTAS (Create Table As Select)

```sql
-- Create new table from query
CREATE TABLE catalog.schema.new_table AS
SELECT * FROM source_table WHERE status = 'active';

-- Create or replace
CREATE OR REPLACE TABLE catalog.schema.new_table AS
SELECT * FROM source_table;
```

INSERT Operations

```sql
-- Insert values
INSERT INTO table_name VALUES (1, 'John', '2024-01-01');
```

```sql
-- Insert from select
INSERT INTO table_name
SELECT * FROM source_table WHERE date = '2024-01-01';

-- Insert overwrite (replace all data)
INSERT OVERWRITE table_name
SELECT * FROM source_table;

-- Insert overwrite partition
INSERT OVERWRITE table_name PARTITION (date = '2024-01-01')
SELECT id, name FROM source_table WHERE date = '2024-01-01';
```

User-Defined Functions (UDFs)

Prefer built-in Spark functions; reach for a UDF only when the logic can't be expressed with them. Python, pandas (vectorized), SQL, and Unity Catalog functions, and when to choose each, are covered in UDFs and Unity Catalog Functions.

> **Continue reading:** Part 2 — Performance Optimization, Error Handling, Use Cases & Exam Tips

---

Batch ETL Pipelines — Part 2

Overview

This part covers performance optimization techniques (Photon, DPP, AQE, partition pruning), error handling patterns, use cases, and exam tips for batch ETL pipelines.

> For DataFrame transformations, joins, aggregations, window functions, and write operations, see Part 1.

Overview

- **Photon** accelerates SQL joins, aggregations, filters, and scans, but not Python or Scala UDFs
- **AQE** (on by default) handles skew, coalesces small partitions, and switches join strategies at runtime; **DPP** pushes dimension filters into partitioned fact tables
- **Partition pruning** needs a direct filter on the partition column; the broadcast join threshold defaults to 10 MB
- **Corrupt-record modes**: `PERMISSIVE` (default), `DROPMALFORMED`, and `FAILFAST`

What the Exam Tests

- Apply advanced data transformations, including window functions, joins, and aggregations, using Spark SQL and PySpark to process large datasets.
- Spot why a batch query does not prune partitions or broadcast a small table
- Choose an error-handling mode for malformed input

Performance Optimization

Photon Engine Photon is Databricks' native vectorized query engine written in C++ for faster SQL and DataFrame operations.

```
# Photon is enabled at cluster level, not via code
# Check if Photon is active

spark.conf.get("spark.databricks.photon.enabled")
```

Workload	Photon Benefit
Joins	High - vectorized hash joins
Aggregations	High - SIMD operations
Filters	High - predicate pushdown
Scans	High - optimized Parquet/Delta reads
Python UDFs	None - not accelerated
Complex nested types	Limited

Photon Limitations:

- Python/Scala UDFs not accelerated (runs in fallback mode)
- Some window functions not fully supported
- Certain complex data types may fall back to Spark

When to use Photon:

- SQL-heavy workloads with joins and aggregations
- Large scan operations on Delta/Parquet
- Filter-heavy queries

Dynamic Partition Pruning (DPP) DPP optimizes joins by pushing partition filters from one side of the join to the other.

```
-- DPP applies automatically in queries like this:
SELECT f.*
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
WHERE d.year = 2024;
-- Spark pushes year=2024 filter to fact_sales scan

# Check DPP configuration

spark.conf.get("spark.sql.optimizer.dynamicPartitionPruning.enabled")  # true by
 ↪  default

# Disable if causing issues (rare)

spark.conf.set("spark.sql.optimizer.dynamicPartitionPruning.enabled", "false")
```

Condition	DPP Applied?
Join on partition column	Yes
Broadcast join	Yes (most effective)
Sort-merge join	Yes (with subquery)
Filter on dimension table	Triggers DPP

Adaptive Query Execution (AQE) AQE optimizes queries at runtime based on actual data statistics.

```python
# AQE is enabled by default in Databricks

spark.conf.get("spark.sql.adaptive.enabled")  # true

# Key AQE features

spark.conf.get("spark.sql.adaptive.coalescePartitions.enabled")  # Reduce
 ↪  partitions
spark.conf.get("spark.sql.adaptive.skewJoin.enabled")  # Handle skew
spark.conf.get("spark.sql.adaptive.localShuffleReader.enabled")  # Optimize
 ↪  shuffles
```

AQE Feature	Behavior
Coalesce partitions	Reduces small partitions after shuffle
Skew join handling	Splits skewed partitions automatically
Join strategy switch	Changes join type based on actual sizes
Local shuffle reader	Avoids shuffle when possible

```python
# Configure shuffle partitions (AQE adjusts automatically)

spark.conf.set("spark.sql.shuffle.partitions", "auto")  # Let AQE decide

# Or set initial value that AQE can reduce

spark.conf.set("spark.sql.shuffle.partitions", "200")
```

Broadcast Join Optimization

```python
from pyspark.sql.functions import broadcast

# Force broadcast for small tables

result = large_df.join(broadcast(small_df), "key")

# Check/set broadcast threshold (default 10MB)

spark.conf.get("spark.sql.autoBroadcastJoinThreshold")  # 10485760 (10MB)

# Increase for larger dimension tables

spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "50MB")

# Disable auto broadcast (force sort-merge)

spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1")
```

```sql
-- SQL broadcast hint
SELECT /*+ BROADCAST(dim_product) */ *
FROM fact_sales f
JOIN dim_product p ON f.product_id = p.product_id;
```

Partition Pruning Best Practices

```python
# Good: Filter on partition column - triggers partition pruning
```

```
df = spark.read.format("delta").load("/path/to/table")
filtered = df.filter(col("date") == "2024-01-15")  # Only reads one partition

# Bad: Filter after load without pushdown

df = spark.read.format("delta").load("/path/to/table")
filtered = df.filter(col("date").cast("string") == "2024-01-15")  # May scan all
 ↪  partitions
```

Pattern	Partition Pruning?
`col("date") == "2024-01-15"`	Yes
`col("date") >= "2024-01-01"`	Yes
`col("date").cast("string") == "2024-01-15"`	No (function on column)
`year(col("date")) == 2024`	No (function on column)

Error Handling

Try-Catch Pattern

```
try:
    df = spark.read.format("delta").load("/path/to/table")
    # Process data
    result = df.filter(col("status") == "active")
    result.write.format("delta").mode("overwrite").save("/output/path")
except AnalysisException as e:
    print(f"Table not found: {e}")
except Exception as e:
    print(f"Processing error: {e}")
    raise
```

Data Quality Checks

```
# Assert row count

row_count = df.count()
assert row_count > 0, "DataFrame is empty"

# Check for nulls in critical columns

null_count = df.filter(col("id").isNull()).count()
assert null_count == 0, f"Found {null_count} null IDs"

# Validate schema

expected_columns = {"id", "name", "amount"}
actual_columns = set(df.columns)
assert expected_columns.issubset(actual_columns), "Missing required columns"
```

Use Cases

Understanding when to apply specific techniques is crucial for the exam and real-world engineering.

Ingestion & Reading

Scenario	Recommended Technique	Why?
Daily Batch Processing	Standard `spark.read` with Schema	Explicit schemas prevent drift ar
Corrupt Data Handling	`PERMISSIVE` mode + `_corrupt_record` column	Allows pipeline to continue proce
Small Reference Data	Cache or Broadcast	faster access during lookups.

Transformations & Performance

Scenario	Recommended Technique	Why?
Complex Math/Logic	Built-in Spark Functions	Significantly faster than UDFs
Custom Complex Logic	Pandas UDF (Vectorized)	Better performance than star
Joins with Small Tables (<10MB)	Broadcast Join	Avoids shuffling large tables a
Joins with Large/Skewed Data	AQE (Adaptive Query Execution)	Automatically handles skew a
Filtering Large Datasets	Partition Pruning (Filter on Partition Key)	Skips scanning irrelevant files

Aggregation & De-duplication

Scenario	Recommended Technique	Why?
Latest Record per Key	Window `row_number()`	Flexible way to pick the ”latest” based on any c
Running Totals	Window `sum()` with `rowsBetween`	Efficiently calculates cumulative sums without s
Pivot Reports	`groupBy().pivot()`	Transforms unique column values into columns

Writing & Loading

Scenario	Recommended Technique	Why?
Full Table Refresh	`overwrite` mode	Completely replaces target table.
Daily Incremental Load	append mode	Adds new records to the end of the table.
Reprocessing Specific Dates	Dynamic Partition Overwrite	Safely overwrites only the partitions being tou

Common Issues & Errors

AnalysisException: Path does not exist **Scenario:** Reading from a path that hasn't been created yet.

Fix: Use `dbutils.fs.ls()` to verify path or use `try-catch` block.

Exam Context: Trick questions often omit the check for file existence before reading.

OutOfMemoryError: Java heap space during specific transformation **Scenario:** Calling `.collect()` on a large DataFrame or broadcasting a table larger than the driver's memory.

Fix: Remove `.collect()` (use `.take()` or `.show()` instead) or increase broadcast threshold/disable broadcast.

Exam Context: Identifying the specific line causing OOM (usually an action that brings data to the driver).

Skew Join (Slow Stages/Stragglers) **Scenario:** One task takes significantly longer than others during a join (e.g., specific keys have millions of records while others have few).

Fix: Enable checking `spark.sql.adaptive.skewJoin.enabled` (default true) or use "salting" technique (manually add a random key).

Exam Context: Identifying "straggler tasks" as a symptom of data skew.

NullPointerException in UDFs **Scenario:** Python UDF fails on null input because it assumes a value exists.

Fix: Explicitly handle `None` in Python code or use `df.na.fill()` before calling the UDF.

Cartesian Product (Cross Join) **Scenario:** Accidental cross join when join conditions are missing or incorrect.

Fix: Ensure join keys are specified correctly. If intentional, set `spark.sql.crossJoin.enabled` to true.

Exam Context: Queries that run forever or produce massive output unexpectedly.

Small File Problem **Scenario:** Writing too many small files (e.g., using `partitionBy` on a high-cardinality column like `timestamp`).

Fix: choosing a lower cardinality partition column (like `date`) or using `OPTIMIZE` / Auto Optimize.

Best Practices

- Define schemas explicitly instead of using `inferSchema`
- Use built-in functions instead of UDFs when possible
- Broadcast small dimension tables for faster joins
- Partition large tables by date or other common filter columns
- Use `replaceWhere` for efficient partition overwrites
- Always handle null values explicitly

Exam Tips

1. **Know all join types** - especially left_anti and left_semi
2. **Window functions** - row_number vs rank vs dense_rank behavior with ties
3. **Write modes** - what happens with each mode when data exists
4. **Broadcast joins** - default threshold is 10MB
5. **Corrupt record handling** - PERMISSIVE, DROPMALFORMED, FAILFAST
6. **Pandas UDFs** are faster than regular Python UDFs
7. **Partition pruning** - filter on partition columns for performance
8. **Photon** accelerates joins/aggregations but not UDFs
9. **AQE** handles skew and optimizes joins at runtime
10. **DPP** pushes filters from dimension to fact tables automatically

Key Takeaways

- **Photon** accelerates SQL joins, aggregations, filters, and scans but provides no speedup for Python/Scala UDFs
- **AQE** (`spark.sql.adaptive.enabled=true` by default) handles skew joins, coalesces small partitions, and switches join strategies at runtime based on actual data statistics
- **DPP** automatically pushes partition filters from a dimension table to a fact table during joins, requiring the join to be on a partition column

- **Broadcast join** threshold default is 10 MB (`spark.sql.autoBroadcastJoinThreshold`); use `broadcast()` hint or increase the threshold for larger dimension tables
- **Partition pruning** only works when filtering directly on a partition column; applying functions (e.g., `year(col("date"))`) prevents pruning
- **Dynamic Partition Overwrite** (`.option("partitionOverwriteMode","dynamic")`) rewrites only touched partitions, preserving untouched partitions in the same table
- **Corrupt record modes**: `PERMISSIVE` (default, stores bad records in `_corrupt_record`), `DROPMALFORMED`, `FAILFAST`
- **Pandas UDFs** (vectorized) outperform standard Python row-at-a-time UDFs and are the recommended choice when custom logic is unavoidable

Related Topics

- Delta Lake Operations - MERGE for upserts
- Data Deduplication - Dedup strategies
- SQL Functions Cheat Sheet

Official Documentation

- Spark SQL Guide
- Databricks DataFrame Guide
- Delta Lake Write Operations

Delta Lake Operations — Part 1

Delta Lake operations are fundamental to the exam. This part covers MERGE, OPTIMIZE, Liquid Clustering, VACUUM, time travel, table cloning, and ANALYZE TABLE. Part 2 covers schema operations, table properties, Delta 3.0+ features, and exam tips.

Overview

```
flowchart TD
    DL[Delta Lake Operations] --> DML[Data Modification]
    DL --> Maint[Table Maintenance]
    DL --> TT[Time Travel]
    DL --> Schema[Schema Management]

    DML --> MERGE
    DML --> UPDATE
    DML --> DELETE

    Maint --> OPTIMIZE
    Maint --> VACUUM
    Maint --> ANALYZE

    TT --> Version[Version Queries]
    TT --> Restore[RESTORE]

    Schema --> Clone[Cloning]
    Schema --> Alter[ALTER TABLE]
```

Overview

- **`MERGE`** for upserts, upserts with deletes, insert-only deduplication, and SCD Type 1
- **File layout**: `OPTIMIZE` with `ZORDER BY`, and liquid clustering
- **`VACUUM`** retention and its effect on time travel; `DESCRIBE HISTORY` and `RESTORE`
- **Shallow vs deep clones**, and `ANALYZE TABLE` statistics

What the Exam Tests

- Apply advanced data transformations, including window functions, joins, and aggregations, using Spark SQL and PySpark to process large datasets.
- Write a `MERGE` for a given upsert, delete, or deduplication requirement
- Predict how `VACUUM` affects time travel

MERGE Operation

MERGE is the most important Delta Lake operation for the exam. It enables upserts (update + insert) in a single atomic transaction.

Basic Syntax

```
MERGE INTO target_table AS t
USING source_table AS s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

MERGE Clauses

```
flowchart TD
    Start[Row from Source] --> Match{Matches Target?}
    Match -->|Yes| Matched[WHEN MATCHED]
    Match -->|No| NotMatched[WHEN NOT MATCHED]

    Matched --> MatchAction{Action?}
    MatchAction --> Update[UPDATE SET]
    MatchAction --> Delete[DELETE]

    NotMatched --> InsertAction[INSERT]

    Target[Target Row] --> BySource{In Source?}
    BySource -->|No| NotBySource[WHEN NOT MATCHED BY SOURCE]
    NotBySource --> BySourceAction{Action?}
    BySourceAction --> UpdateOrphan[UPDATE]
    BySourceAction --> DeleteOrphan[DELETE]
```

Clause	Purpose	When Triggered
`WHEN MATCHED`	Update or delete existing rows	Source row matches target row
`WHEN NOT MATCHED`	Insert new rows	Source row has no match in target
`WHEN NOT MATCHED BY SOURCE`	Handle orphan rows	Target row has no match in source

MERGE with Conditions

```sql
MERGE INTO customers AS t
USING updates AS s
ON t.customer_id = s.customer_id
WHEN MATCHED AND s.is_deleted = true THEN DELETE
WHEN MATCHED AND s.is_deleted = false THEN UPDATE SET
    t.name = s.name,
    t.email = s.email,
    t.updated_at = current_timestamp()
WHEN NOT MATCHED AND s.is_deleted = false THEN INSERT (
    customer_id, name, email, created_at, updated_at
) VALUES (
    s.customer_id, s.name, s.email, current_timestamp(), current_timestamp()
);
```

Star Syntax

```sql
-- UPDATE SET * updates all columns from source
MERGE INTO target AS t
USING source AS s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

MERGE with Schema Evolution

```sql
-- Allow new columns from source to be added to target
MERGE WITH SCHEMA EVOLUTION INTO target AS t
USING source AS s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Python MERGE API

```python
from delta.tables import DeltaTable

delta_table = DeltaTable.forPath(spark, "/path/to/target")

delta_table.alias("t").merge(
    source_df.alias("s"),
    "t.id = s.id"
).whenMatchedUpdateAll(
).whenNotMatchedInsertAll(
).execute()
```

```python
# With conditions

delta_table.alias("t").merge(
    source_df.alias("s"),
    "t.id = s.id"
).whenMatchedUpdate(
    condition="s.is_deleted = false",
    set={"name": "s.name", "email": "s.email"}
).whenMatchedDelete(
    condition="s.is_deleted = true"
).whenNotMatchedInsert(
```

```
    condition="s.is_deleted = false",
    values={"id": "s.id", "name": "s.name", "email": "s.email"}
).execute()
```

Common MERGE Patterns

Simple Upsert

```sql
-- Insert new, update existing
MERGE INTO target AS t
USING source AS s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Upsert with Delete

```sql
-- CDC pattern: handle inserts, updates, and deletes
MERGE INTO target AS t
USING source AS s
ON t.id = s.id
WHEN MATCHED AND s.operation = 'DELETE' THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED AND s.operation != 'DELETE' THEN INSERT *;
```

Insert-Only Merge (Deduplication)

```sql
-- Only insert if not exists (no updates)
MERGE INTO target AS t
USING source AS s
ON t.id = s.id
WHEN NOT MATCHED THEN INSERT *;
```

SCD Type 1 (Overwrite)

```sql
-- Always overwrite with latest values
MERGE INTO dim_customer AS t
USING stg_customer AS s
ON t.customer_id = s.customer_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

OPTIMIZE Command

OPTIMIZE compacts small files into larger ones, improving read performance.

OPTIMIZE Syntax

```sql
-- Optimize entire table
OPTIMIZE catalog.schema.table_name;

-- Optimize specific partition
OPTIMIZE table_name WHERE date = '2024-01-01';

-- Optimize with predicate
OPTIMIZE table_name WHERE region = 'us-west';
```

ZORDER BY ZORDER co-locates related data in the same files for faster filtering.

```
-- Optimize with Z-ordering on frequently filtered columns
OPTIMIZE table_name ZORDER BY (customer_id, order_date);

-- Partition + ZORDER
OPTIMIZE table_name
WHERE date >= '2024-01-01'
ZORDER BY (customer_id);
```

Aspect	OPTIMIZE	ZORDER
Purpose	Compact small files	Co-locate data for queries
When to use	After many small writes	Improve filter performance
Columns	N/A	1-4 columns (practical limit)
Cost	Rewrites files	Higher cost than plain OPTIMIZE

ZORDER Column Selection

```
flowchart TD
    Start[Choose ZORDER Columns] --> Q1{High cardinality?}
    Q1 -->|Yes| Q2{Used in WHERE clauses?}
    Q1 -->|No| Skip[Skip - low cardinality]
    Q2 -->|Yes| Q3{1-4 columns total?}
    Q2 -->|No| Skip2[Skip - not filtered]
    Q3 -->|Yes| Include[Include in ZORDER]
    Q3 -->|No| Prioritize[Keep most selective columns]
```

Auto Optimize

```
-- Enable auto optimize on table
ALTER TABLE table_name SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact' = 'true'
);
```

Property	Behavior
optimizeWrite	Automatically optimize file sizes during writes
autoCompact	Automatically compact small files after writes

Liquid Clustering

Liquid Clustering is an alternative to ZORDER that automatically manages data layout.

```
-- Create table with liquid clustering
CREATE TABLE table_name (
    id INT,
    name STRING,
    region STRING
) USING DELTA
CLUSTER BY (region, id);

-- Add clustering to existing table
ALTER TABLE table_name CLUSTER BY (region, id);
```

```sql
-- Remove clustering
ALTER TABLE table_name CLUSTER BY NONE;
```

Feature	ZORDER	Liquid Clustering
Manual optimization	Required	Automatic
Column changes	Requires full rewrite	Incremental
Best for	Stable query patterns	Evolving patterns
Write overhead	During OPTIMIZE	During writes

VACUUM Command

VACUUM removes old files that are no longer referenced by the Delta table.

VACUUM Syntax

```sql
-- Default retention (7 days / 168 hours)
VACUUM table_name;

-- Custom retention
VACUUM table_name RETAIN 240 HOURS;

-- Dry run (preview files to delete)
VACUUM table_name DRY RUN;
```

Critical VACUUM Facts (Exam Important)

Setting	Value
Default retention	**168 hours (7 days)**
Minimum safe retention	168 hours
Override minimum	`spark.databricks.delta.retentionDurationCheck.enabled = false`

```mermaid
flowchart LR
    Write[Write Data] --> Files[Data Files Created]
    Files --> Update[UPDATE/DELETE/OPTIMIZE]
    Update --> Old[Old Files Marked Obsolete]
    Old --> Retention{Past Retention?}
    Retention -->|No| Keep[Keep for Time Travel]
    Retention -->|Yes| Vacuum[VACUUM Removes]
```

VACUUM Safety

```sql
-- DANGEROUS: Never do this in production
SET spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM table_name RETAIN 0 HOURS;  -- Breaks time travel!

-- Safe: Always use default or higher retention
VACUUM table_name RETAIN 168 HOURS;
```

Warning: Running VACUUM with retention less than 7 days can break:

- Time travel queries
- Concurrent readers
- Streaming queries with old checkpoints

Time Travel

Time travel allows querying previous versions of a Delta table.

Query by Version

```sql
-- Query specific version
SELECT * FROM table_name VERSION AS OF 5;

-- Using @ syntax
SELECT * FROM table_name@v5;
```

Query by Timestamp

```sql
-- Query at specific timestamp
SELECT * FROM table_name TIMESTAMP AS OF '2024-01-15 10:00:00';

-- Using @ syntax
SELECT * FROM table_name@'2024-01-15';
```

Python Time Travel

```python
# By version

df = (spark.read.format("delta")
    .option("versionAsOf", 5)
    .load("/path/to/table"))

# By timestamp

df = (spark.read.format("delta")
    .option("timestampAsOf", "2024-01-15")
    .load("/path/to/table"))
```

DESCRIBE HISTORY

```sql
-- View table history
DESCRIBE HISTORY table_name;

-- Limit results
DESCRIBE HISTORY table_name LIMIT 10;
```

Returns columns:

- `version` - Version number
- `timestamp` - When the version was created
- `operation` - WRITE, MERGE, DELETE, etc.
- `operationParameters` - Details of the operation
- `userIdentity` - Who made the change

RESTORE Command

```sql
-- Restore to previous version
RESTORE TABLE table_name TO VERSION AS OF 5;

-- Restore to timestamp
RESTORE TABLE table_name TO TIMESTAMP AS OF '2024-01-15';
```

Time Travel Retention

```sql
-- Configure log retention (default 30 days)
ALTER TABLE table_name SET TBLPROPERTIES (
    'delta.logRetentionDuration' = 'interval 45 days'
);

-- Configure deleted file retention (default matches VACUUM)
ALTER TABLE table_name SET TBLPROPERTIES (
    'delta.deletedFileRetentionDuration' = 'interval 14 days'
);
```

Table Cloning

Shallow Clone Creates a copy that references the source's data files.

```sql
-- Shallow clone (fast, shares data files)
CREATE TABLE clone_table SHALLOW CLONE source_table;

-- Clone specific version
CREATE TABLE clone_table SHALLOW CLONE source_table VERSION AS OF 10;
```

Deep Clone Creates an independent copy with its own data files.

```sql
-- Deep clone (copies all data)
CREATE TABLE clone_table DEEP CLONE source_table;

-- Clone to specific location
CREATE TABLE clone_table DEEP CLONE source_table
LOCATION 'abfss://container@storage/path';
```

Feature	Shallow Clone	Deep Clone
Data files	Shared (referenced)	Copied
Speed	Fast	Slow (copies data)
Storage	Minimal	Full copy
Independence	Dependent on source	Fully independent
Use case	Testing, experimentation	Backups, migration

ANALYZE TABLE

Compute statistics for the query optimizer.

```sql
-- Compute table statistics
ANALYZE TABLE table_name COMPUTE STATISTICS;

-- Compute column statistics
ANALYZE TABLE table_name COMPUTE STATISTICS FOR COLUMNS col1, col2;

-- Compute all column statistics
ANALYZE TABLE table_name COMPUTE STATISTICS FOR ALL COLUMNS;
```

Continue reading: Part 2 — Schema Operations, Table Properties, Delta 3.0+ Features & Exam Tips

---

Delta Lake Operations — Part 2

Overview

This part covers schema operations, key table properties, transaction log internals, Delta 3.0+ features (deletion vectors, predictive optimization, UniForm, identity columns, VACUUM LITE, row tracking, liquid clustering), use cases, common issues, and exam tips.

For MERGE, OPTIMIZE, VACUUM, time travel, and table cloning, see Part 1.

Overview

- **Schema operations**, including column type widening (safe widening only, such as INT to BIGINT)
- **Delta features**: deletion vectors, liquid clustering, predictive optimization, UniForm, VACUUM LITE, and row tracking
- **Identity columns**: `GENERATED ALWAYS AS IDENTITY` blocks manual inserts; `GENERATED BY DEFAULT AS IDENTITY` allows them
- **Transaction log**: the source of truth for ACID transactions and time travel, with checkpoints every 10 commits

What the Exam Tests

- Choose the appropriate Delta optimization technique (deletion vectors, Liquid Clustering, CLUSTER BY AUTO) for a given table access pattern.
- Know which table property turns on which Delta feature
- Recognise which schema changes Delta allows and which it rejects

Schema Operations

Add Column

```
ALTER TABLE table_name ADD COLUMN new_col STRING;
ALTER TABLE table_name ADD COLUMN new_col STRING AFTER existing_col;
ALTER TABLE table_name ADD COLUMNS (col1 STRING, col2 INT);
```

Rename Column

```
ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
```

Change Column Type

```
-- Only widening conversions allowed (e.g., INT to BIGINT)
ALTER TABLE table_name ALTER COLUMN col_name TYPE BIGINT;
```

Add Constraints

```
-- Primary key (informational, not enforced)
ALTER TABLE table_name ADD CONSTRAINT pk PRIMARY KEY (id);

-- Check constraint (enforced)
ALTER TABLE table_name ADD CONSTRAINT chk_amount CHECK (amount > 0);

-- Not null constraint
ALTER TABLE table_name ALTER COLUMN col_name SET NOT NULL;
```

```sql
-- Drop constraint
ALTER TABLE table_name DROP CONSTRAINT constraint_name;
```

Key Table Properties

Property	Description	Default
`delta.enableChangeDataFeed`	Enable Change Data Feed	false
`delta.autoOptimize.optimizeWrite`	Optimize file sizes on write	false
`delta.autoOptimize.autoCompact`	Auto-compact after writes	false
`delta.logRetentionDuration`	Transaction log retention	30 days
`delta.deletedFileRetentionDuration`	Deleted file retention	7 days
`delta.minReaderVersion`	Minimum reader protocol	varies
`delta.minWriterVersion`	Minimum writer protocol	varies

Transaction Log

The transaction log (`_delta_log/`) is the source of truth for Delta tables.

```
flowchart LR
    subgraph Transaction Log
        J1[00000.json] --> J2[00001.json]
        J2 --> J3[00002.json]
        J3 --> CP[00010.checkpoint.parquet]
        CP --> J4[00011.json]
    end
```

- JSON files record each transaction
- Checkpoint files (every 10 commits) for faster reads
- Enables ACID transactions and time travel

Delta Lake 3.0+ Features

These modern features are increasingly important for the exam.

Deletion Vectors Deletion vectors mark rows as deleted without rewriting entire data files, significantly improving DELETE, UPDATE, and MERGE performance.

```
flowchart LR
    subgraph Without DVs
        D1[DELETE row] --> R1[Rewrite entire file]
        R1 --> S1[Expensive for large files]
    end

    subgraph With DVs
        D2[DELETE row] --> R2[Write small DV file]
        R2 --> S2[Fast, constant time]
    end
```

Aspect	Without DVs	With DVs
DELETE/UPDATE	Rewrite entire file	Write small DV file
Performance	Slow for large files	Fast, constant time
Space reclaim	Immediate	On VACUUM
File count	Fewer files	More small DV files

Enabling Deletion Vectors

```
-- Enable on new table
CREATE TABLE table_name (
    id INT,
    name STRING
) USING DELTA
TBLPROPERTIES ('delta.enableDeletionVectors' = 'true');

-- Enable on existing table
ALTER TABLE table_name
SET TBLPROPERTIES ('delta.enableDeletionVectors' = 'true');
```

```
# Check if DVs are enabled

spark.sql("DESCRIBE DETAIL table_name").select("properties").show(truncate=False)
```

Exam Tip: Deletion vectors are enabled by default on Databricks Runtime 14.x+ for new tables.

Predictive Optimization Databricks automatically analyzes table access patterns and runs OPTIMIZE and VACUUM operations.

```
-- Enable at table level
ALTER TABLE table_name
SET TBLPROPERTIES ('delta.enablePredictiveOptimization' = 'true');
```

Feature	Behavior
Auto OPTIMIZE	Runs based on write patterns
Auto VACUUM	Runs based on file age and access
Workload analysis	Recommends clustering columns
Cost	Included in serverless compute

When to use: Enable for tables with frequent small writes that need regular optimization without manual scheduling.

Delta UniForm (Universal Format) UniForm enables reading Delta tables as Apache Iceberg or Apache Hudi without data conversion.

```
-- Enable UniForm for Iceberg compatibility
CREATE TABLE table_name (
    id INT,
    name STRING
) USING DELTA
TBLPROPERTIES ('delta.universalFormat.enabledFormats' = 'iceberg');

-- Enable on existing table
ALTER TABLE table_name
SET TBLPROPERTIES ('delta.universalFormat.enabledFormats' = 'iceberg');

-- Enable both Iceberg and Hudi
ALTER TABLE table_name
SET TBLPROPERTIES ('delta.universalFormat.enabledFormats' = 'iceberg,hudi');
```

Format	Use Case	Compatibility
Iceberg	Snowflake, Trino, Presto, Spark	Iceberg v2
Hudi	Existing Hudi ecosystems	Hudi 0.14+

Trade-offs:

- Additional metadata storage overhead
- Slight write latency increase
- Enables cross-platform data sharing

Identity Columns Auto-generate unique sequential IDs for new rows (Delta 3.3+).

```
-- GENERATED ALWAYS - system controls all values
CREATE TABLE orders (
    order_id BIGINT GENERATED ALWAYS AS IDENTITY,
    customer_id INT,
    amount DOUBLE
) USING DELTA;

-- GENERATED BY DEFAULT - allows manual override
CREATE TABLE orders (
    order_id BIGINT GENERATED BY DEFAULT AS IDENTITY (START WITH 1000 INCREMENT BY
 ↪  1),
    customer_id INT,
    amount DOUBLE
) USING DELTA;
```

Option	Behavior
`GENERATED ALWAYS`	System generates all values, manual insert fails
`GENERATED BY DEFAULT`	System generates if not provided, allows override
`START WITH`	Initial value
`INCREMENT BY`	Step between values

Use cases: Surrogate keys, audit sequences, replacing sequences from source systems.

VACUUM LITE Faster file cleanup using transaction log analysis (Delta 3.3+).

```
-- VACUUM LITE uses transaction log for faster cleanup
VACUUM table_name LITE;

-- Still supports retention
VACUUM table_name LITE RETAIN 168 HOURS;
```

Aspect	Standard VACUUM	VACUUM LITE
Performance	Slower (file listing)	5-10x faster
Method	Lists all files	Uses transaction log
Best for	Occasional cleanup	Frequent cleanup

Row Tracking Track row-level lineage across table versions (Delta 3.2+).

```sql
-- Enable row tracking on new table
CREATE TABLE table_name (...)
USING DELTA
TBLPROPERTIES ('delta.enableRowTracking' = 'true');

-- Enable on existing table (requires backfill)
ALTER TABLE table_name
SET TBLPROPERTIES ('delta.enableRowTracking' = 'true');
```

Row tracking adds hidden columns:

- `_row_id` - Stable identifier for each row
- `_row_commit_version` - Version when row was last modified

Use cases: Audit trails, CDC verification, debugging data lineage.

Liquid Clustering Deep Dive Expanding on Liquid Clustering for modern workloads:

Migration from ZORDER

```sql
-- Step 1: Check current ZORDER usage
DESCRIBE HISTORY table_name;

-- Step 2: Enable Liquid Clustering (replaces ZORDER)
ALTER TABLE table_name CLUSTER BY (col1, col2);

-- Step 3: Trigger initial clustering
OPTIMIZE table_name;
```

Liquid Clustering vs ZORDER vs Partitioning

Feature	Partitioning	ZORDER	Liquid Clustering
Column cardinality	Low	High	Any
Column changes	Requires rewrite	Requires rewrite	Incremental
Optimization	N/A	Manual OPTIMIZE	Automatic
Skipping	Partition pruning	Data skipping	Data skipping
Best for	Date, region	Customer ID, SKU	Evolving queries

```sql
-- Liquid Clustering with OPTIMIZE still works
OPTIMIZE table_name;  -- Applies clustering

-- Check clustering columns
DESCRIBE DETAIL table_name;
```

Use Cases

Scenario	Recommended Operation	Why?
GDPR Compliance (Right to be Forgotten)	`DELETE + VACUUM`	Removes PII from current tabl
Daily Full Refresh	`overwrite` or `replaceWhere`	Efficiently replaces data witho
Slow Filter Queries	`OPTIMIZE + ZORDER`	Co-locates data to enable effi
Handling Late Arriving Data	`MERGE`	Updates existing records or in

Common Issues & Errors

"Multiple matches" in MERGE **Scenario:** Source table has duplicate keys matching a single target row.

Fix: Deduplicate the source data before running MERGE.

Time Travel Fails ("File not found") **Scenario:** Trying to query an old version after VACUUM has run.

Fix: Increase retention duration if longer history is needed, but accept higher storage costs.

ZORDER Effectiveness Low **Scenario:** Z-ordering on too many columns (e.g., > 5) or low-cardinality columns.

Fix: Limit ZORDER to 1-4 high-cardinality columns frequently used in WHERE clauses.

VACUUM 0 Hours Data Loss **Scenario:** User disabled safety check and ran `VACUUM RETAIN 0 HOURS`.

Fix: Irreversible. Recover from deep clone or backup if available. Never do this in production.

Best Practices

- Run OPTIMIZE on partitions with many small files
- Use ZORDER on 1-4 high-cardinality filter columns
- Schedule VACUUM to run regularly but keep 7+ days retention
- Use shallow clones for testing, deep clones for backups
- Enable auto optimize for streaming or frequent small writes
- Compute statistics after large data loads for better query plans

Exam Tips

1. **VACUUM default is 168 hours (7 days)** - This is frequently tested
2. **MERGE WHEN MATCHED** can have multiple conditions with different actions
3. **ZORDER** columns should be high cardinality and used in WHERE clauses
4. **Time travel** requires files to exist - VACUUM removes old files
5. **Shallow clone** shares data files; **deep clone** copies them
6. **Auto Optimize** has two settings: `optimizeWrite` and `autoCompact`
7. **RESTORE** creates a new version, doesn't delete history
8. **Deletion vectors** are default on DBR 14.x+ - faster DELETE/UPDATE/MERGE
9. **Liquid Clustering** replaces ZORDER for evolving query patterns
10. **UniForm** enables reading Delta as Iceberg/Hudi for cross-platform sharing
11. **Predictive Optimization** auto-schedules OPTIMIZE and VACUUM

Key Takeaways

- **Deletion Vectors** (enabled by default on DBR 14.x+) mark rows as deleted by writing a small DV file instead of rewriting entire data files, significantly improving DELETE, UPDATE, and MERGE performance
- **Liquid Clustering** (`CLUSTER BY`) replaces ZORDER for evolving query patterns; it is incremental, does not require specifying low-cardinality columns, and allows column changes without full rewrites
- **Delta UniForm** (`delta.universalFormat.enabledFormats = 'iceberg'`) allows external engines (Snowflake, Trino, Presto) to read a Delta table as Iceberg without data conversion

- **Predictive Optimization** auto-schedules OPTIMIZE and VACUUM based on table access patterns, eliminating the need for manual maintenance jobs
- **VACUUM LITE** uses the transaction log rather than file listing for cleanup, making it 5–10x faster than standard VACUUM
- **Identity columns** (`GENERATED ALWAYS AS IDENTITY` vs `GENERATED BY DEFAULT AS IDENTITY`) auto-generate sequential surrogate keys; `ALWAYS` prevents manual inserts, `BY DEFAULT` allows overrides
- **Delta transaction log** checkpoints are created every 10 commits and speed up table reads; the log is the source of truth for ACID transactions and time travel
- **Column type widening** via `ALTER TABLE … ALTER COLUMN … TYPE` is limited to safe widening conversions (e.g., INT → BIGINT); narrowing conversions are not allowed

Related Topics

- Change Data Capture - MERGE generates CDF records
- Data Deduplication - MERGE for dedup patterns
- Delta Lake Commands Cheat Sheet

Official Documentation

- Delta Lake MERGE
- Delta Lake OPTIMIZE
- Delta Lake VACUUM
- Delta Lake Time Travel

Data Deduplication

Deduplication ensures data quality by removing duplicate records. Understanding batch and streaming deduplication patterns is important for the exam.

Overview

```
flowchart TD
    subgraph Sources of Duplicates
        S1[Retry logic]
        S2[Multiple producers]
        S3[Reprocessing]
        S4[Source system issues]
    end

    subgraph Dedup Strategies
        S1 & S2 & S3 & S4 --> D1[At-source]
        S1 & S2 & S3 & S4 --> D2[During load]
        S1 & S2 & S3 & S4 --> D3[Post-load]
        S1 & S2 & S3 & S4 --> D4[Continuous streaming]
    end
```

Overview

- **Batch deduplication** with `dropDuplicates` or `row_number()`, which keeps exactly one row per key (`rank()` can keep ties)
- **MERGE is idempotent**; deduplicate the source first, because duplicate source keys make MERGE fail

- **Streaming deduplication** needs a watermark; `dropDuplicatesWithinWatermark` bounds the state
- **Medallion placement**: Silver is the main deduplication layer

What the Exam Tests

- Apply advanced data transformations, including window functions, joins, and aggregations, using Spark SQL and PySpark to process large datasets.
- Pick a deduplication method for batch vs streaming data
- Make streaming writes idempotent with `foreachBatch` and MERGE

Why Duplicates Occur

Source	Cause	Example
Producer retries	At-least-once delivery	Kafka producer retry
Multiple sources	Same data from different paths	Multi-region replication
Pipeline reruns	Failure recovery reprocessing	Job restart after crash
Source systems	Application bugs, sync issues	Double-submitted orders

Deduplication Strategies

```
flowchart TD
    Start[Choose Strategy] --> When{When to Dedup?}

    When -->|Before Load| AtSource[At-Source Dedup]
    When -->|During Load| DuringLoad[Load-Time Dedup]
    When -->|After Load| PostLoad[Post-Load Dedup]
    When -->|Continuously| Streaming[Streaming Dedup]

    AtSource --> A1[Clean at source system]
    DuringLoad --> D1[MERGE / dropDuplicates]
    PostLoad --> P1[Periodic cleanup jobs]
    Streaming --> S1[Watermark-based dedup]
```

Strategy	When	Pros	Cons
At-source	Before ingestion	Cleanest approach	Not always possible
During load	ETL pipeline	Prevents duplicates entering	Processing overhead
Post-load	Scheduled cleanup	Simple implementation	Duplicates exist temporarily
Streaming	Continuous	Near-real-time	Memory for state

Batch Deduplication

dropDuplicates()

```python
# Remove exact duplicates (all columns)

df_deduped = df.dropDuplicates()

# Remove duplicates based on specific columns

df_deduped = df.dropDuplicates(["order_id"])
```

```python
# Remove duplicates on multiple columns

df_deduped = df.dropDuplicates(["customer_id", "order_date", "product_id"])
```

Keep First vs Keep Last By default, `dropDuplicates()` keeps an arbitrary record. To control which record is kept:

```python
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, col

# Keep the LATEST record per key

window = Window.partitionBy("order_id").orderBy(col("updated_at").desc())

df_latest = (df
    .withColumn("rn", row_number().over(window))
    .filter(col("rn") == 1)
    .drop("rn"))

# Keep the FIRST record per key

window = Window.partitionBy("order_id").orderBy(col("created_at").asc())

df_first = (df
    .withColumn("rn", row_number().over(window))
    .filter(col("rn") == 1)
    .drop("rn"))
```

SQL Deduplication

```sql
-- Using ROW_NUMBER
WITH ranked AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY order_id
            ORDER BY updated_at DESC
        ) AS rn
    FROM orders
)
SELECT * FROM ranked WHERE rn = 1;

-- Using DISTINCT
SELECT DISTINCT customer_id, email FROM customers;

-- Using GROUP BY (with aggregation)
SELECT
    customer_id,
    MAX(updated_at) as latest_update,
    FIRST(name) as name
FROM customers
GROUP BY customer_id;
```

rank() vs row_number() vs dense_rank()

```python
from pyspark.sql.functions import rank, row_number, dense_rank

window = Window.partitionBy("customer_id").orderBy(col("amount").desc())

(df.withColumn("row_num", row_number().over(window))
    .withColumn("rank", rank().over(window))
    .withColumn("dense_rank", dense_rank().over(window)))
```

Function	Ties Handling	Values for [100, 100, 90]
`row_number()`	Arbitrary	1, 2, 3
`rank()`	Same rank, skip	1, 1, 3
`dense_rank()`	Same rank, no skip	1, 1, 2

For deduplication, use `row_number()` to guarantee exactly one row per key.

Deduplication with MERGE

MERGE naturally handles duplicates by matching on keys.

Insert-Only MERGE (Dedup Pattern)

```sql
-- Only insert if key doesn't exist
MERGE INTO target AS t
USING source AS s
ON t.id = s.id
WHEN NOT MATCHED THEN INSERT *;
```

MERGE with Duplicate Source When source has duplicates, deduplicate before MERGE:

```python
from delta.tables import DeltaTable

# Source may have duplicates

source_df = spark.table("staging.orders")

# Deduplicate source first

source_deduped = source_df.dropDuplicates(["order_id"])

# Then MERGE

target = DeltaTable.forName(spark, "target.orders")

target.alias("t").merge(
    source_deduped.alias("s"),
    "t.order_id = s.order_id"
).whenMatchedUpdateAll(
).whenNotMatchedInsertAll(
).execute()
```

Handling Multiple Matches in MERGE If MERGE finds multiple matching rows in source, it fails. Always deduplicate source:

```python
# This will fail if source has duplicate order_ids

target.merge(source, "t.order_id = s.order_id")  # Error!

# Fix: Deduplicate source

source_deduped = source.dropDuplicates(["order_id"])
target.merge(source_deduped, "t.order_id = s.order_id")  # Works
```

Streaming Deduplication

dropDuplicates in Streaming

```python
# Basic streaming deduplication

stream_df = spark.readStream.format("delta").load("/source")

deduped_stream = stream_df.dropDuplicates(["id"])

query = (deduped_stream.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoint")
    .start("/target"))
```

Watermark Requirement (Exam Important) For streaming deduplication with unbounded state, you **must** use watermarks:

```python
from pyspark.sql.functions import col

# Streaming dedup WITH watermark - required for production

stream_df = spark.readStream.format("delta").load("/source")

deduped_stream = (stream_df
    .withWatermark("event_time", "10 minutes")
    .dropDuplicates(["id", "event_time"]))

query = (deduped_stream.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoint")
    .start("/target"))
```

Why watermarks are required:

- Without watermark, state grows unbounded
- Watermark enables state cleanup for old data
- Prevents out-of-memory errors

dropDuplicatesWithinWatermark More efficient streaming dedup that only checks within watermark window:

```python
# Only deduplicate within the watermark window

deduped_stream = (stream_df
    .withWatermark("event_time", "10 minutes")
    .dropDuplicatesWithinWatermark(["id"]))
```

Method	State Size	Duplicate Detection
`dropDuplicates()`	Large (all keys)	All time
`dropDuplicatesWithinWatermark()`	Smaller	Within watermark only

Streaming Dedup with Window

```
from pyspark.sql.functions import window

# Deduplicate within time windows

deduped = (stream_df
    .withWatermark("event_time", "1 hour")
    .dropDuplicates(["id", window("event_time", "1 hour")]))
```

Idempotent Writes (Exam Critical)

Idempotent writes ensure the same input always produces the same output, even if reprocessed.

Why Idempotency Matters

Without Idempotency	With Idempotency
Rerun = duplicates	Rerun = same result
Difficult recovery	Easy recovery
Data inconsistency	Data consistency

MERGE for Idempotency MERGE is inherently idempotent:

```
def idempotent_write(batch_df, batch_id):
    """MERGE ensures same result on reprocess."""
    target = DeltaTable.forPath(spark, "/target")

    target.alias("t").merge(
        batch_df.alias("s"),
        "t.id = s.id"
    ).whenMatchedUpdateAll(
    ).whenNotMatchedInsertAll(
    ).execute()
```

foreachBatch Idempotent Pattern

```
def process_batch_idempotent(batch_df, batch_id):
    """Idempotent batch processing."""

    # 1. Deduplicate within batch
    deduped = batch_df.dropDuplicates(["id"])

    # 2. Use MERGE for idempotent upsert
    target = DeltaTable.forName(spark, "catalog.schema.target")

    target.alias("t").merge(
        deduped.alias("s"),
        "t.id = s.id"
    ).whenMatchedUpdateAll(
```

```
    ).whenNotMatchedInsertAll(
    ).execute()

# Apply to streaming

query = (stream_df.writeStream
    .foreachBatch(process_batch_idempotent)
    .option("checkpointLocation", "/checkpoint")
    .start())
```

Transaction ID Pattern Track processed batches to prevent reprocessing:

```
def process_with_txn_tracking(batch_df, batch_id):
    """Skip already processed batches."""

    # Check if batch already processed
    processed = spark.sql(f"""
        SELECT 1 FROM processing_log WHERE batch_id = {batch_id}
    """).count() > 0

    if processed:
        return  # Skip - already done

    # Process the batch
    batch_df.write.format("delta").mode("append").save("/target")

    # Log the processed batch
    spark.sql(f"""
        INSERT INTO processing_log VALUES ({batch_id}, current_timestamp())
    """)
```

Deduplication in Medallion Architecture

```
flowchart LR
    subgraph Bronze
        B[Accept all data<br>including duplicates]
    end

    subgraph Silver
        B --> S[Primary dedup layer<br>dropDuplicates/MERGE]
    end

    subgraph Gold
        S --> G[Aggregations<br>naturally handle dupes]
    end
```

Bronze Layer Accept duplicates, add lineage:

```
# Bronze: Append all data, track source

bronze_df = (raw_df
    .withColumn("_ingestion_time", current_timestamp())
    .withColumn("_source_file", input_file_name()))

bronze_df.write.format("delta").mode("append").save("/bronze/orders")
```

Silver Layer Primary deduplication:

```
# Silver: Deduplicate from Bronze

bronze_df = spark.read.format("delta").load("/bronze/orders")

# Deduplicate - keep latest per order_id

window = Window.partitionBy("order_id").orderBy(col("_ingestion_time").desc())

silver_df = (bronze_df
    .withColumn("rn", row_number().over(window))
    .filter(col("rn") == 1)
    .drop("rn", "_source_file"))

# MERGE to Silver

target = DeltaTable.forPath(spark, "/silver/orders")
target.alias("t").merge(
    silver_df.alias("s"),
    "t.order_id = s.order_id"
).whenMatchedUpdateAll(
).whenNotMatchedInsertAll(
).execute()
```

Gold Layer Aggregations naturally deduplicate:

```
# Gold: Aggregations handle duplicates naturally

gold_df = spark.sql("""
    SELECT
        date,
        COUNT(DISTINCT order_id) as unique_orders,
        SUM(amount) as total_amount
    FROM silver.orders
    GROUP BY date
""")
```

Performance Considerations

Dedup Key Selection

```
# Good: Use minimal unique key

df.dropDuplicates(["order_id"])

# Avoid: Using all columns when key exists

df.dropDuplicates()  # Slower, hashes all columns
```

Partition Alignment

```
# Efficient: Dedup within partitions

(df.repartition("date")
    .dropDuplicates(["order_id"]))
```

```python
# Better for skewed data: Salt the key

from pyspark.sql.functions import concat, lit, floor, rand

(df.withColumn("salt", floor(rand() * 10))
    .repartition("salt")
    .dropDuplicates(["order_id"]))
```

Memory for Streaming State

```python
# Monitor state size

query.lastProgress["stateOperators"]

# Configure state checkpointing

spark.conf.set("spark.sql.streaming.stateStore.stateSchemaCheck", "true")
```

Monitoring Duplicates

Count Duplicates

```python
# Find duplicate counts

duplicate_counts = (df.groupBy("order_id")
    .count()
    .filter(col("count") > 1))

total_duplicates = duplicate_counts.agg(
    sum(col("count") - 1)
).collect()[0][0]

print(f"Total duplicate records: {total_duplicates}")
```

Duplicate Detection Query

```sql
-- Find records with duplicates
SELECT
    order_id,
    COUNT(*) as occurrence_count,
    COLLECT_LIST(updated_at) as timestamps
FROM orders
GROUP BY order_id
HAVING COUNT(*) > 1
ORDER BY occurrence_count DESC;
```

Data Quality Metrics

```python
# Calculate dedup metrics

total_records = df.count()
unique_records = df.dropDuplicates(["id"]).count()
duplicate_records = total_records - unique_records
duplicate_rate = duplicate_records / total_records * 100
```

```
print(f"Duplicate rate: {duplicate_rate:.2f}%")
```

Identity Columns for Deduplication

Delta 3.3 introduces Identity Columns that auto-generate unique IDs:

```
-- Create table with identity column for natural dedup key
CREATE TABLE orders (
    order_id BIGINT GENERATED ALWAYS AS IDENTITY,
    customer_id INT,
    order_date DATE,
    amount DOUBLE
) USING DELTA;

-- Inserts automatically get unique order_id
INSERT INTO orders (customer_id, order_date, amount)
VALUES (100, '2024-01-15', 99.99);
```

Identity columns provide:

- **Guaranteed uniqueness** - system-generated, no application logic needed
- **Natural dedup key** - use as MERGE condition
- **Surrogate key pattern** - no business key collisions

Row Tracking for Dedup Validation

Row Tracking (Delta 3.2+) helps validate deduplication by tracking row lineage:

```
-- Enable row tracking
ALTER TABLE orders
SET TBLPROPERTIES ('delta.enableRowTracking' = 'true');

-- Query includes hidden _metadata columns for lineage
SELECT *, _metadata.row_id, _metadata.row_commit_version
FROM orders;
```

Use Row Tracking to:

- Verify dedup removed correct records
- Audit which rows survived deduplication
- Track row history across MERGE operations

Use Cases

Scenario	Recommended Strategy	Why?
Ingestion from Queue	`dropDuplicates` (Stream)	Removes duplicates caused by at-least-once d
Re-running Failed Job	`MERGE` (Idempotent)	Ensures reprocessed data doesn't create dupli
Silver Layer Cleanup	`row_number()` Window	Deterministically picks the ”best” record (e.g.,
High Volume Stream	`dropDuplicatesWithinWatermark`	Efficiently dedups recent data without unlimite

Common Issues & Errors

State Store OOM (Streaming) **Scenario:** Using `dropDuplicates` on a stream without a watermark.

Fix: Add `.withWatermark()` before deduplication or use `dropDuplicatesWithinWatermark` if acceptable.

Random Record Retention **Scenario:** Using `dropDuplicates()` without specifying order.

Fix: Use `row_number().over(Window.orderBy(...))` to explicitly select which record to keep.

MERGE Failure on Source Duplicates **Scenario:** Running MERGE when source dataset contains duplicate keys.

Fix: Add a deduplication step to the source DataFrame *before* the MERGE operation.

Dedup Not Working Across Partitions **Scenario:** Using `dropDuplicates` after specific shuffle operations that might not preserve global uniqueness context (rare, but conceptual).

Fix: Ensure proper shuffling or standard `dropDuplicates` (which triggers a shuffle) is used globally.

Best Practices

- Use `row_number()` with window functions for deterministic dedup
- Always deduplicate source data before MERGE
- Use watermarks with streaming deduplication
- Implement idempotent writes with MERGE in foreachBatch
- Monitor duplicate rates as a data quality metric
- Choose dedup keys carefully - prefer natural business keys
- Consider `dropDuplicatesWithinWatermark` for large-scale streaming
- Use Identity Columns for auto-generated surrogate keys
- Enable Row Tracking to audit deduplication results

Exam Tips

1. **Streaming dedup requires watermarks** for state cleanup
2. **`dropDuplicatesWithinWatermark`** is more efficient than `dropDuplicates` in streaming
3. **MERGE is naturally idempotent** - same input = same output
4. **Deduplicate source before MERGE** to avoid multiple match errors
5. **`row_number()`** guarantees exactly one record per key (use for dedup)
6. **Bronze accepts duplicates**, Silver deduplicates
7. **foreachBatch + MERGE** is the idempotent streaming pattern
8. **Identity Columns** (Delta 3.3) auto-generate unique IDs, useful as natural dedup keys
9. **Row Tracking** enables validation of dedup by tracking row lineage across operations

Key Takeaways

- **`row_number()`** is the correct window function for deduplication because it guarantees exactly one row per key regardless of ties; `rank()` and `dense_rank()` can produce multiple rows per key when values are tied
- **Streaming deduplication requires watermarks** to bound state size; using `dropDuplicates` without a watermark causes unbounded state growth and eventual OOM
- **`dropDuplicatesWithinWatermark`** is more memory-efficient than `dropDuplicates` because it tracks only keys within the watermark window rather than all keys ever seen
- **MERGE is inherently idempotent**: running the same source data through a MERGE operation multiple times always produces the same target state, making it the safe write pattern for reprocessing
- **Deduplicate source before MERGE**: if the source DataFrame contains duplicate keys, MERGE throws a `DeltaUnsupportedOperationException`; always call `dropDuplicates(["key"])` on the source first
- **Medallion architecture pattern**: Bronze layer accepts duplicates with lineage metadata; Silver is the primary deduplication layer using `row_number()` or MERGE; Gold aggregations naturally handle any remaining duplicates

- **`foreachBatch` + `MERGE`** is the standard idempotent streaming write pattern, combining within-batch dedup (`dropDuplicates`) with cross-batch dedup via MERGE
- **Identity Columns** (`GENERATED ALWAYS AS IDENTITY`) provide system-managed surrogate keys that are guaranteed unique, eliminating application-level key collision concerns

Related Topics

- Batch ETL Pipelines - Window functions
- Structured Streaming - Streaming and watermarks
- Delta Lake Operations - MERGE patterns
- Change Data Capture - Dedup before CDC apply

Official Documentation

- Streaming Deduplication
- Delta Lake MERGE
- Watermarks

VARIANT and Semi-Structured Data

Overview

`VARIANT` is the Databricks data type for semi-structured data (JSON-like objects, arrays, and scalars). It stores values in an optimized binary encoding. Databricks recommends it over JSON strings when the schema is flexible, changing, or unknown. This page covers creating, ingesting, querying, and modeling `VARIANT` data, and when to choose it over `STRING` JSON or `STRUCT`.

Overview

- Create `VARIANT` values with `parse_json` / `try_parse_json`, Auto Loader `singleVariantColumn`, `COPY INTO`, `from_xml(..., 'variant')`, or `from_csv(..., 'v variant')`
- Query with the `:` path syntax plus `::` casts, or with `variant_get` / `try_variant_get`. Inspect shape with `schema_of_variant` / `schema_of_variant_agg`. Flatten with `variant_explode`
- Variant shredding stores commonly occurring fields as separate Parquet columns, which gives semi-structured data structured-like read performance
- `VARIANT` columns can't be partition, clustering, or Z-order keys, and can't be used in comparisons, `GROUP BY`, `ORDER BY`, `DISTINCT`, or set operations

What the Exam Tests

- ”Develop a model and query semi-structured data using the VARIANT data type and related functions (e.g., parse_json, variant_get)”
- Choosing `VARIANT` vs `STRING` JSON vs `STRUCT` for a Bronze or Silver table, given schema volatility and read patterns
- Knowing which functions raise errors (`parse_json`, `variant_get`) and which return `NULL` (`try_parse_json`, `try_variant_get`)
- Promoting frequently filtered fields to typed columns so they can be clustered and skipped

Platform update (Oct 2026)

Variant and Variant shredding became generally available on July 22, 2026. Starting with Databricks Runtime 17.3, shredding is enabled automatically when you create a new table with VARIANT columns. Variant is "coming soon" to workspaces with the compliance security profile enabled. Source: Databricks platform release notes, July 2026

Where VARIANT Fits

The exam's sample question describes a Bronze table holding raw JSON payloads "of varying shape in a single string column". The goal is efficient storage and fast field extraction with no fixed schema declared up front. The answer is to parse the payloads into VARIANT with parse_json and extract fields with variant_get and colon-path access.

```
flowchart LR
    SRC["Raw JSON<br>(Kafka, REST, files)"] --> ING["Auto
↪   Loader<br>singleVariantColumn<br>or parse_json"]
    ING --> BRZ["Bronze<br>payload VARIANT"]
    BRZ --> SLV["Silver<br>hot fields as typed columns<br>+ remaining payload
↪   VARIANT"]
    SLV --> GLD["Gold<br>aggregates on typed columns"]
```

Runtime and Table Requirements

Item	Requirement (per docs)
VARIANT type and functions	Function reference pages list Databricks SQL and Databricks Runtime
Read/write tables with variant support	Databricks Runtime 15.4 LTS or above
Delta tables	Variant support must be enabled. Creating a table with a VARIANT col
Apache Iceberg tables	All Iceberg v3 tables support VARIANT. Iceberg v2 tables don't
Variant shredding	Databricks Runtime 17.3+ to read and write shredded tables. 18.1+ r
File formats ingested as VARIANT	JSON (15.3+), XML (16.4+), CSV (16.4+)

```
-- New table: a VARIANT column enables the feature
CREATE TABLE main.bronze.events_raw (payload VARIANT);

-- Existing Delta table: enable variant support explicitly
ALTER TABLE main.bronze.events_raw
SET TBLPROPERTIES ('delta.feature.variantType' = 'supported');
```

Creating VARIANT Values

parse_json(jsonStr) returns a VARIANT. If the string isn't valid JSON, it raises MALFORMED_RECORD_IN_PARSING.
try_parse_json(jsonStr) returns NULL instead.

```
-- CTAS: keep one typed key column, store the whole document as VARIANT
CREATE TABLE main.bronze.orders_raw AS
SELECT
  json_string:id AS id,              -- ':' on a JSON STRING returns STRING
  parse_json(json_string) AS payload
FROM main.landing.orders_json;

-- Append into an existing VARIANT column
```

```sql
INSERT INTO main.bronze.orders_raw (payload)
SELECT parse_json(json_string) FROM main.landing.orders_json;

-- Tolerate malformed records: NULL instead of a failed query
SELECT try_parse_json('{"a" : invalid, "b" : 2}');   -- NULL

-- Other constructors
SELECT from_xml(xml_string, 'variant') FROM main.landing.orders_xml;
SELECT from_csv(csv_string, 'v variant').v FROM main.landing.orders_csv;
SELECT CAST(123.456 AS VARIANT);                      -- typed literal to VARIANT
SELECT to_variant_object(named_struct('k', 'v'));     -- STRUCT or MAP to VARIANT
 ↪  object
```

Important

VARIANT natively holds OBJECT, ARRAY, and scalar types. To store a STRUCT or MAP as a variant object, use to_variant_object. MAP keys must be STRING.

```python
from pyspark.sql.functions import col, parse_json

(spark.read.table("main.landing.orders_json")
    .select(parse_json(col("json_string")).alias("payload"))
    .write
    .mode("append")
    .saveAsTable("main.bronze.orders_raw"))
```

Ingesting into VARIANT

Auto Loader Auto Loader can load each whole record into a single VARIANT column with singleVariantColumn. Because the full record lands in one column, no schema evolution happens during ingestion, and rescuedDataColumn isn't supported.

```python
(spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("singleVariantColumn", "payload")
    .load("/Volumes/main/landing/orders/")
    .writeStream
    .option("checkpointLocation", checkpoint_path)
    .toTable("main.bronze.orders_raw"))
```

You can also declare individual columns as VARIANT in a schema or in schema hints:

```python
# Typed column plus a VARIANT column
(spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .schema("name STRING, address VARIANT")
    .load("/Volumes/main/landing/customers/"))

# Infer everything else; force one column to VARIANT
(spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaHints", "address VARIANT")
    .load("/Volumes/main/landing/customers/"))
```

Caveats from the ingestion docs:

- Malformed records can't be encoded as VARIANT.

- Overly large records are treated like corrupt records. In the default `PERMISSIVE` mode, they're captured in the `corruptRecordColumn`.

COPY INTO and Kafka

```sql
CREATE TABLE main.bronze.orders_raw (payload VARIANT);

COPY INTO main.bronze.orders_raw
  FROM '/Volumes/main/landing/orders/'
  FILEFORMAT = JSON
  FORMAT_OPTIONS ('singleVariantColumn' = 'payload');
```

```python
from pyspark.sql.functions import col, parse_json

(spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "host1:port1")
    .option("subscribe", "orders")
    .load()
    .select(
        col("key").cast("string"),
        parse_json(col("value").cast("string")).alias("payload"))
    .writeStream
    .option("checkpointLocation", checkpoint_path)
    .toTable("main.bronze.orders_kafka"))
```

Databricks recommends Auto Loader over `COPY INTO` when available.

Querying VARIANT with Path Syntax

The same SQL path operators used for JSON strings work on `VARIANT`.

Syntax	Selects	Example
`col:field`	Top-level field	`payload:owner`
`col:a.b` or `col:a['b']`	Nested field with a named key	`payload:store.bicycle`
`col:arr[n]`	Array element (0-based)	`payload:store.fruit[0]`
`col:['field name']`	Field with special characters (space, `.`, `:`, `[ ]`)	`payload:['zip code']`
`expr::TYPE`	Cast (synonym for `cast`)	`payload:price::DOUBLE`
`expr ?:: TYPE`	Error-tolerant cast (synonym for `try_cast`, DBR 15.3+)	`payload:price ?::INT`

```sql
SELECT
  payload:owner::STRING                    AS owner,
  payload:store.bicycle.price::DOUBLE      AS bike_price,
  payload:store.fruit[0].type::STRING      AS first_fruit,
  payload:['zip code']::STRING             AS zip,
  payload:store.bicycle::STRUCT<price DOUBLE, color STRING> AS bicycle
FROM main.bronze.store_data;
```

Rules to remember:

- **Field names are case-sensitive.** `payload:Owner` and `payload:owner` look for different fields. JSON strings match case-insensitively, so migrations can break here.
- **`:` on a `VARIANT` returns a `VARIANT`.** Cast with `::` before comparing, grouping, or sorting.
- **Missing path returns `NULL`** (a `NULL` of type `VARIANT`). An out-of-bounds array index also returns `NULL`.
- **`[*]` isn't supported** on `VARIANT` for unpacking all array elements. Use `variant_explode` instead.

- **Escaping:** `['<field>']` escapes every special character. Backticks escape spaces and colons but not periods or brackets. A backtick-escaped name that contains `.` or `[ ]` returns `NULL`.

variant_get and try_variant_get

```
variant_get ( variantExpr, path, type )
try_variant_get ( variantExpr, path, type )
```

- `path` is a `STRING` literal JSON path expression such as `'$.data[1].a'`.
- `type` is a `STRING` literal such as `'string'`, `'double'`, or `'array<int>'`.

Situation	variant_get	try_variant_get
Path not found	`NULL`	`NULL`
Found, castable to `type`	Typed value	Typed value
Found, not castable to `type`	Error `INVALID_VARIANT_CAST`	`NULL`

```
SELECT
  variant_get(payload, '$.owner', 'string')                AS owner,
  try_variant_get(payload, '$.store.bicycle.price', 'double') AS price,
  try_variant_get(payload, '$.key', 'boolean')             AS bad_cast   -- NULL,
 ↪  not an error
FROM main.bronze.store_data;

-- try_variant_get is equivalent to path + ?:: cast
SELECT payload:store.bicycle.price ?::DOUBLE FROM main.bronze.store_data;
```

```
from pyspark.sql.functions import col, variant_get, try_variant_get

df = spark.read.table("main.bronze.store_data")
df.select(
    variant_get(col("payload"), "$.owner", "string").alias("owner"),
    try_variant_get(col("payload"), "$.store.bicycle.price",
 ↪  "double").alias("price"),
    variant_get(col("payload"), "$['zip.code']", "string").alias("zip"))
```

NULL Semantics

Variants can contain two kinds of null:

Kind	Meaning	Test
SQL `NULL`	The value is missing (for example `parse_json(NULL)`, or a path that doesn't exist)	`IS NULL`
Variant null	The document explicitly stores JSON `null`	`is_varian`

```
SELECT
  is_variant_null(parse_json(NULL))                    AS sql_null,
 ↪  -- false
  is_variant_null(parse_json('null'))                  AS variant_null,
 ↪  -- true
  is_variant_null(parse_json('{"a": null}'):a)         AS null_value,
 ↪  -- true
  is_variant_null(parse_json('{"a": null}'):missing)   AS missing_path;
 ↪  -- false
```

Discovering the Schema

Function	Kind	Returns
`schema_of_variant(v)`	Scalar	DDL-format schema of one value, for ex
`schema_of_variant_agg(v) [FILTER (WHERE cond)]`	Aggregate	Combined schema of all values in a gro

```
-- What shapes are arriving in Bronze?
SELECT schema_of_variant_agg(payload) AS combined_schema
FROM main.bronze.events_raw
WHERE ingest_date = current_date();
```

Use the combined schema to decide which fields to promote to typed Silver columns, or which STRUCT to cast to.

Flattening Arrays and Objects

`variant_explode(input)` is a table-valued generator that returns `pos INT`, `key STRING`, and `value VARIANT`. Use it in the `FROM` clause with `LATERAL`.

- **Object:** one row per key/value pair.
- **Array:** one row per element, and key is always `NULL`.
- **`NULL` or scalar input:** no rows. Use `variant_explode_outer` to keep a row of `NULL`s.

```
-- One row per line item
SELECT o.id, li.pos, li.value:sku::STRING AS sku, li.value:qty::INT AS qty
FROM main.bronze.orders_raw AS o,
LATERAL variant_explode(o.payload:line_items) AS li;
```

```
from pyspark.sql.functions import col, variant_get

items = (spark.read.table("main.bronze.orders_raw")
    .select(variant_get(col("payload"), "$.line_items",
 ↪  "variant").alias("items")))

exploded = (items
    .lateralJoin(spark.tvf.variant_explode(col("items").outer()))
    .drop("items"))
```

Converting Back to JSON

`to_json(variant)` returns a JSON `STRING`, and any options are ignored for `VARIANT` input. It's the *logical* inverse of `parse_json`, but not an exact one. Whitespace isn't preserved, key order is arbitrary, and trailing zeros in numbers might be dropped.

```
SELECT to_json(payload) AS payload_json FROM main.bronze.orders_raw;
```

Variant Shredding

Shredding stores commonly occurring fields of a `VARIANT` column as separate columns in the underlying Parquet files. This cuts the I/O needed to read those fields and improves compression compared with a single binary blob.

Topic	Behavior
Automatic enablement	From Databricks Runtime 17.3, `CREATE TABLE` with a `VARIANT` column enables shredd
Not automatic	`CREATE OR REPLACE TABLE` and `ALTER TABLE` don't enable it, so existing workloads d

Topic	Behavior
Opt in / out	`delta.enableVariantShredding` (Delta) or `iceberg.enableVariantShredding` (Ice
Existing data	Enabling applies only to new writes. Rewrite old data with `REORG TABLE ... APPLY`
Remove completely	`ALTER TABLE ... DROP FEATURE "variantShredding"` rewrites data unshredded a
Scope	Top-level `VARIANT` columns and `VARIANT` fields in structs. Not `VARIANT` inside arrays
Cost	Some overhead on writes

```sql
ALTER TABLE main.bronze.events_raw
SET TBLPROPERTIES ('delta.enableVariantShredding' = 'true');

-- Shred data written before the property was set
REORG TABLE main.bronze.events_raw APPLY (SHRED VARIANT);
```

Limitations

- No partitioning by a `VARIANT` column, and a `VARIANT` column can't be a clustering key or Z-order key.
- No `GROUP BY`, `ORDER BY`, or `DISTINCT` on a `VARIANT` column, and no comparisons or set operators (`INTERSECT`, `UNION`, `EXCEPT`).
- A generated column can't produce a `VARIANT`.
- `VARIANT` columns don't collect `minValues` / `maxValues` statistics.
- A single value can't exceed 128 MiB (16 MiB in Databricks Runtime 17.1 and below).

Modeling Choice: VARIANT vs STRING JSON vs STRUCT

Approach	Best for
`VARIANT`	Semi-structured JSON whose fields or types change, or where an optimized encoding is worth
`STRING` (JSON)	Capturing a complete, exact copy of raw source data
`STRUCT`	Well-known schemas where you want validated data
`MAP` / `ARRAY`	Semi-structured data with around 500 fields

Scenario	Choose
Bronze landing of JSON payloads whose shape varies by event type	`VARIANT`
Exact raw copy is required for replay or audit, and nobody queries the field values	`STRING`
Stable, contract-governed schema queried heavily in Silver/Gold	`STRUCT` or flat typed colu
Frequently filtered field inside a variant payload	Extract it to a typed top-l
Need to cluster or skip on a nested field	`STRUCT` field (liquid cluste

Tip

Databricks recommends extracting frequently queried fields into non-variant columns ”to accelerate queries and optimize storage layout”. A common Silver pattern is: typed columns for keys, timestamps, and filters, plus the remaining payload kept as `VARIANT`.

```sql
-- CREATE TABLE (not CREATE OR REPLACE) so DBR 17.3+ enables shredding
 ↪  automatically
CREATE TABLE main.silver.events (
  event_id   STRING,
  event_type STRING,
  event_ts   TIMESTAMP,
```

```
  payload    VARIANT
)
CLUSTER BY (event_type, event_ts);

INSERT INTO main.silver.events
SELECT
  payload:id::STRING,
  payload:type::STRING,
  payload:ts::TIMESTAMP,
  payload
FROM main.bronze.events_raw;
```

SQL to PySpark Reference

Import the functions from `pyspark.sql.functions`.

SQL	PySpark
`parse_json(s)`	`parse_json(col("s"))`
`try_parse_json(s)`	`try_parse_json(col("s"))`
`variant_get(v, '$.a', 'int')`	`variant_get(col("v"), "$.a", "int")`
`try_variant_get(v, '$.a', 'int')`	`try_variant_get(col("v"), "$.a", "int")`
`schema_of_variant(v)`	`schema_of_variant(col("v"))`
`schema_of_variant_agg(v)`	`schema_of_variant_agg(col("v"))`
`is_variant_null(v)`	`is_variant_null(col("v"))`
`to_json(v)`	`to_json(col("v"))`
`LATERAL variant_explode(v)`	`df.lateralJoin(spark.tvf.variant_explode(col("v").outer()))`
`v:a.b::INT`	`df.selectExpr("v:a.b::INT")`

Collected values arrive in Python as `VariantVal`. Call `.toPython()` to get a Python object or `.toJson()` to get a JSON string. You can also build one with `VariantVal.parseJson('{"a": 1}')`.

Use Cases

- **Bronze ingestion of event streams.** Kafka, Kinesis, Event Hub, or webhook payloads land as `VARIANT` and stay robust to new or changed fields.
- **REST API responses and schemaless databases.** Store heterogeneous documents without per-source schemas, then promote stable fields downstream.
- **Multi-tenant or multi-event-type tables.** Different event types share one table, and `schema_of_variant_agg` profiles what arrives.
- **Replacing JSON-string columns.** Migrate `get_json_object` or `:` queries on strings to `VARIANT` for better read and write performance.

Common Issues & Errors

Symptom	Cause
`MALFORMED_RECORD_IN_PARSING`	`parse_json` hit invalid JSON
`INVALID_VARIANT_CAST`	`variant_get` found the field but couldn't cast it
Field unexpectedly `NULL`	Case mismatch (paths are case-sensitive), wrong path, or a ba
Error grouping or sorting by `payload:status`	The result of `:` is still `VARIANT`
`[*]` query returns an error	Wildcard array access isn't supported on `VARIANT`
No speed-up after enabling shredding	Shredding applies only to new writes
`CLUSTER BY (payload)` fails	`VARIANT` can't be a clustering or partition key

Symptom	Cause
External Delta client can't write the table	Enabling variant upgrades the writer protocol

Best Practices

- Land raw semi-structured data as `VARIANT` in Bronze. Use Auto Loader with `singleVariantColumn` when the whole record should stay intact.
- Use `try_parse_json` / `try_variant_get` in pipelines that must not fail on bad records. Pair them with expectations to quarantine failures.
- Promote fields used for joins, filters, and clustering to typed top-level columns in Silver.
- Run `schema_of_variant_agg` periodically to detect drift before it reaches typed layers.
- Use Databricks Runtime 18.1+ for shredded tables when you need variant statistics and data skipping.

Exam Tips

Tip

- **Flexible schema plus efficient storage plus fast extraction means `VARIANT`.** Distractors usually include keeping JSON as `STRING` with `regexp_extract`, or forcing a rigid schema with STRUCT or Parquet.
- **`try_` means `NULL`, not an error.** `parse_json` and `variant_get` raise errors. `try_parse_json`, `try_variant_get`, and `::` return `NULL`.
- **`:` returns `VARIANT`.** Cast with `::` (or `variant_get` with a type) to get a typed value.
- **Paths are case-sensitive** on `VARIANT`, unlike JSON strings.
- **`VARIANT` can't be a partition, clustering, or Z-order key**, and it can't be grouped, sorted, or compared. Extract a typed column instead.
- **`variant_explode` goes in `FROM` with `LATERAL`.** It returns `pos`, `key`, and `value`. For arrays, `key` is `NULL`.
- **Shredding** is automatic for new tables on DBR 17.3+. Use `REORG TABLE ... APPLY (SHRED VARIANT)` for existing data.

Key Takeaways

- **`VARIANT`** is the recommended type for semi-structured data with changing or unknown schemas. It outperforms JSON strings on reads and writes.
- **Creation:** `parse_json` (errors on bad input), `try_parse_json` (`NULL` on bad input), Auto Loader / `COPY INTO` with `singleVariantColumn`, `from_xml(..., 'variant')`, and `from_csv(..., 'v variant')`.
- **Extraction:** `col:path` with `::` casts, `variant_get(v, '$.path', 'type')`, or the error-tolerant `try_variant_get`. Array indexes are 0-based, and missing paths return `NULL`.
- **Schema tools:** `schema_of_variant` describes one value. `schema_of_variant_agg` merges a group using least common types and falls back to `VARIANT` on conflicts.
- **Nulls:** SQL `NULL` (missing) is different from variant null (stored JSON `null`). Test the latter with `is_variant_null`.
- **Shredding** (GA Jul 22, 2026) gives structured-like performance. It's enabled automatically by `CREATE TABLE` on DBR 17.3+, but not by `CREATE OR REPLACE` or `ALTER`.
- **Modeling:** `VARIANT` for flexibility, `STRING` for an exact raw copy, `STRUCT` for validated, well-known schemas. Promote hot fields to typed columns for clustering and data skipping.

Related Topics

- Auto Loader - schema inference, schema hints, and rescued data

- Streaming Ingestion from Message Buses - Kafka payloads
- Schema Management - enforcement and evolution
- Expectations and Data Quality - quarantining records that fail parsing or casting
- AI Functions in Data Pipelines - task-specific AI functions return `VARIANT`
- Partitioning Strategies - clustering keys for promoted columns

Official Documentation

- Query variant data
- How is variant different than JSON strings?
- Model semi-structured data
- Ingest data as semi-structured variant type
- Variant type support for Apache Iceberg and Delta Lake
- Use variant shredding to optimize performance
- VARIANT type
- parse_json function
- try_parse_json function
- variant_get function
- try_variant_get function
- schema_of_variant_agg function
- variant_explode table-valued function
- `?::` (question double colon sign) operator
- Databricks platform release notes, July 2026

AI Functions in Data Pipelines

Overview

AI Functions are built-in SQL functions that apply large language models (LLMs) and other models to data stored on Databricks. You can call them from Databricks SQL, notebooks, Lakeflow pipelines, and jobs. For data engineers, they turn enrichment and classification (sentiment, categories, extracted fields, masking, summaries) into ordinary SQL expressions inside batch or streaming pipelines.

Overview

- **Task-specific AI Functions** such as `ai_classify`, `ai_extract`, `ai_analyze_sentiment`, `ai_summarize`, `ai_mask`, `ai_translate`, and `ai_parse_document` are purpose-built, Databricks-managed, and need no prompt engineering
- **`ai_query`** is the general-purpose function. It calls a model serving endpoint (Databricks-hosted `system.ai` models, provisioned throughput, external, or custom models) with your own prompt, `returnType` / `responseFormat`, `modelParameters`, and `failOnError`
- AI Functions manage parallelization, retries, and scaling. Submit the whole dataset in one query and set `failOnError => false` for large jobs
- Governance runs through Unity Catalog and Unity Gateway privileges, and cost shows up in `system.billing.usage`

What the Exam Tests

- ”Apply AI functions, including ai_query, to perform model inference within data pipelines for enrichment and classification tasks”
- Choosing a task-specific function vs `ai_query` for a given enrichment requirement
- Using `responseFormat` for structured output and `failOnError => false` so one bad row

doesn't fail a batch
- Placing inference in a Lakeflow pipeline, a scheduled job, or Structured Streaming, and accounting for cost and access control

Platform update (Oct 2026)

Docs now name Databricks-hosted models as `system.ai.<model>` model services. `ai_query` requests to `system.ai` model services are routed automatically through **Unity Gateway**, which provides usage tracking and permission checks. Gateway rate limits, guardrails, inference tables, and fallbacks don't apply to `ai_query` batch workloads. Source: Use ai_query

---

Requirements

Requirement	Detail
Compute	Not available on Classic SQL warehouses (Databricks SQL Classic)
Runtime	Databricks Runtime 15.4 LTS or above. 18.2+ is recommended for performance and th
Region	The workspace must be in a supported Model Serving / AI Functions region
Data access	Query permission on the Unity Catalog table that holds the input data
Endpoint (`ai_query`)	An existing serving endpoint. For Databricks-hosted foundation models, the endpoint a

Task-Specific Functions vs ai_query

Databricks recommends starting with a task-specific function when one matches the objective. Switch to `ai_query` when you need control over the prompt, the model, parameters, or output format, or when you need to call a custom, fine-tuned, or external model.

```
flowchart TD
    Need[Enrichment need] --> Match{Task-specific function<br>matches the task?}
    Match -->|Yes| TS["ai_classify / ai_extract /<br>ai_analyze_sentiment /
↪   ai_summarize /<br>ai_mask / ai_translate / ai_parse_document"]
    Match -->|No| Q{Need a custom prompt,<br>specific model, custom ML
↪   model,<br>or exact output schema?}
    Q -->|Yes| AQ["ai_query(endpoint, request, ...)"]
    Q -->|No| TS
```

Task-specific functions

Task	Function	Signature
Classification / routing	`ai_classify`	`ai_classify(content, labels [, options])`
Structured field extraction	`ai_extract`	`ai_extract(content, schema [, options])`
Sentiment	`ai_analyze_sentiment`	`ai_analyze_sentiment(content)`
Summary	`ai_summarize`	`ai_summarize(content [, max_words])`
PII masking	`ai_mask`	`ai_mask(content, labels)` with labels an ARRAY<ST
Translation	`ai_translate`	`ai_translate(content, to_lang)`
Grammar	`ai_fix_grammar`	`ai_fix_grammar(content)`
Semantic similarity	`ai_similarity`	Compares two strings
Document parsing	`ai_parse_document`	Parses binary documents (text, tables, figures, layout)
Free-form prompt	`ai_gen`	Answers a user-provided prompt using a state-of-the-ar
Forecasting	`ai_forecast`	Table-valued function that extrapolates time series
Vector retrieval	`vector_search`	Queries an AI Search index

The docs also list Beta functions such as `ai_enrich`, `ai_decide`, `ai_prep_search`, `ai_search`, `ai_transcribe`, `ai_predict_class`, `ai_predict_value`, `ai_detect_anomalies`, and `ai_top_drivers`.

```
SELECT
  review_id,
  ai_analyze_sentiment(review_text)                       AS sentiment,
  ai_summarize(review_text, 30)                           AS summary,
  ai_translate(review_text, 'en')                         AS review_en,
  ai_mask(review_text, array('person', 'email', 'phone')) AS review_masked
FROM main.silver.product_reviews;
```

ai_classify

- `labels` is a `STRING`. It can be a JSON array of names (`'["urgent", "not_urgent"]'`) or a JSON object mapping names to descriptions. You need 2–500 labels in versions 2.x and 2–20 in version 1.0.
- `options` is a `MAP<STRING, STRING>`. Keys include `version` (`1.0`, `2.0`, `2.1`), `instructions`, `multilabel`, and, in version 2.1 only, `enableConfidenceScores` and `enableRationales`.
- `content` can be a `STRING` or a `VARIANT` produced by another AI function, such as `ai_parse_document`.
- Version 2.1 is recommended. It returns `{"response": [{"value": ...}], "metadata": {...}, "error_message": null}`. Version 2.1 isn't output-compatible with 2.0, because each label is now an object with a `value` key. Pin the version explicitly.

```
SELECT
  ticket_id,
  ai_classify(
    body,
    '{"billing_issue": "Payment, invoice, or refund problems",
      "product_defect": "Malfunction, bug, or breakage",
      "account_issue": "Login failures or password resets"}',
    map('version', '2.1', 'instructions', 'Customer support ticket routing.')
  ) AS c
FROM main.silver.support_tickets;

-- The result is VARIANT, so read it with path syntax and casts
SELECT ticket_id, c:response[0].value::STRING AS category,
 ↪  c:error_message::STRING AS err
FROM classified_tickets;
```

ai_extract `schema` is either a JSON array of field names (all treated as strings) or a JSON object with types (`string`, `integer`, `number`, `boolean`, `enum`), descriptions, nested `objects`, and `arrays`. Types are validated. In versions 2.x the result is a `VARIANT`, and each scalar field comes back as an object with a `value` key. Version 2.1 adds optional citations and confidence scores. `options` can set `mode` = `precision` for long or complex documents.

```
SELECT
  invoice_id,
  ai_extract(
    invoice_text,
    '{"vendor_name": {"type": "string"},
      "total_amount": {"type": "number", "description": "Invoice total"},
      "invoice_date": {"type": "string", "description": "YYYY-MM-DD"}}',
    map('version', '2.1')
  ) AS x
FROM main.bronze.invoices;
-- x:response.total_amount.value::DECIMAL(18,2)
```

Tip

Task-specific functions that return `VARIANT` compose with `VARIANT` path syntax. See VARIANT and Semi-Structured Data.

ai_query

Syntax

```
-- Foundation model endpoint, or custom model endpoint that has a model schema
ai_query(endpoint, request)

-- Custom model endpoint without a model schema
ai_query(endpoint, request, returnType, failOnError)

-- Optional named arguments
ai_query(endpoint => ..., request => ..., returnType => ..., failOnError => ...,
         modelParameters => ..., responseFormat => ..., files => ...)
```

Argument	Meaning
`endpoint`	STRING literal naming a Databricks foundation model, external model, or custom model se
`request`	STRING for foundation and external models. For custom models, a column or STRUCT whos
`returnType`	Expected return type, similar to the `from_json` schema. Optional on DBR 15.2+, where it
`failOnError`	Boolean literal, default `true` (DBR 15.3+). When `false`, the result is a STRUCT with respo
`modelParameters`	Constant STRUCT of model parameters such as `max_tokens` and `temperature` (DBR 15.3+
`responseFormat`	Structured output for chat models (DBR 15.4 LTS+). Accepts a DDL-style string or a JSON
`files`	Multimodal image input (JPEG or PNG), for example `files => content from read_files(`

Enrichment with structured output

```
SELECT
  article_id,
  ai_query(
    'system.ai.meta-llama-3-3-70b-instruct',
    'Extract the category of the following news article: ' || body,
    responseFormat => '{
      "type": "json_schema",
      "json_schema": {
        "name": "news_extraction",
        "schema": {
          "type": "object",
          "properties": {
            "title": {"type": "string"},
            "category": {"type": "string",
              "enum": ["Politics", "Sports", "Technology", "Business"]}
          }
        },
        "strict": true
      }
    }',
    modelParameters => named_struct('max_tokens', 200, 'temperature', 0.0),
    failOnError => false
  ) AS result
FROM main.silver.news;
```

With `failOnError => false`:

- A successful row has `result.errorMessage` set to `NULL`.
- A row that hits a model endpoint error has `result.response` set to `NULL` and `result.errorMessage` populated.
- Errors that aren't model endpoint errors still fail the whole query.

A DDL-style `responseFormat` is also accepted, for example `responseFormat => 'STRUCT<extraction:STRUCT<keywords:ARRAY<STRING>>>'`.

Classification with a custom or traditional ML model

```
SELECT
  message_id,
  ai_query(
    endpoint   => 'spam-classification-endpoint',
    request    => named_struct('timestamp', ts, 'sender', from_number, 'text',
↪  text),
    returnType => 'BOOLEAN'
  ) AS is_spam
FROM main.silver.messages;
```

Reusable, governed wrapper Wrap `ai_query` in a Unity Catalog SQL function so consumers call a governed function instead of a raw endpoint:

```
CREATE FUNCTION main.ai.correct_grammar(text STRING)
  RETURNS STRING
  RETURN ai_query('system.ai.meta-llama-3-3-70b-instruct',
                  CONCAT('Correct this to standard English:\n', text));

GRANT EXECUTE ON FUNCTION main.ai.correct_grammar TO `data-scientists`;
```

PySpark There's no separate DataFrame API in the docs examples. Call AI Functions through `expr` / `selectExpr`:

```
from pyspark.sql.functions import expr

df_out = (spark.read.table("main.silver.articles")
    .withColumn(
        "summary",
        expr(
            "ai_query('system.ai.gpt-oss-120b', "
            "CONCAT('Summarize: ', text), "
            "modelParameters => named_struct('max_tokens', 100, 'temperature',
             ↪  0.7))"
        )))

df_out.write.mode("overwrite").saveAsTable("main.gold.article_summaries")
```

Using AI Functions in Pipelines

The docs list three production deployment targets for batch inference pipelines: **Lakeflow pipelines**, scheduled **jobs**, and **Structured Streaming**.

```
flowchart LR
    RAW["Volume / Kafka"] --> ST["Streaming table<br>news_raw"]
```

```
    ST --> MV1["Materialized view<br>news_categorized<br>(ai_query +
 ↪  responseFormat)"]
    MV1 --> MV2["Materialized view<br>news_validated<br>(expectations)"]
    MV2 --> MV3["Materialized view<br>news_summarized<br>(ai_query)"]
```

Lakeflow pipeline (SQL)

```
CREATE OR REFRESH STREAMING TABLE news_raw
AS SELECT * FROM STREAM(read_files(
  '/Volumes/main/news/raw_csv', format => 'csv', header => true));

CREATE OR REFRESH MATERIALIZED VIEW news_categorized
AS SELECT
  inputs,
  ai_query(
    'system.ai.meta-llama-3-3-70b-instruct',
    'Extract the category of the following news article: ' || inputs,
    responseFormat => '{"type": "json_schema", "json_schema": {"name":
 ↪  "news_extraction",
      "schema": {"type": "object", "properties": {"title": {"type": "string"},
      "category": {"type": "string"}}}, "strict": true}}'
  ) AS meta_data
FROM news_raw;

-- Validate LLM output before downstream use
CREATE OR REFRESH MATERIALIZED VIEW news_validated (
  CONSTRAINT valid_category EXPECT (
    get_json_object(meta_data, '$.category') IN ('Politics', 'Sports',
 ↪  'Technology', 'Business'))
  ON VIOLATION DROP ROW
)
AS SELECT * FROM news_categorized;
```

Lakeflow pipeline (Python)

```
from pyspark import pipelines as dp
from pyspark.sql.functions import expr

@dp.materialized_view(comment="LLM-categorized news")
def news_categorized():
    return (spark.read.table("news_raw")
        .withColumn(
            "meta_data",
            expr(
                "ai_query('system.ai.meta-llama-3-3-70b-instruct', "
                "concat('Extract the category of the following news article: ',
                 ↪  inputs))"
            )))

@dp.materialized_view()
@dp.expect_or_drop("has_output", "meta_data IS NOT NULL")
def news_validated():
    return spark.read.table("news_categorized")
```

Structured Streaming and jobs

- **Structured Streaming:** apply `ai_query` in a `select` / `selectExpr` on a streaming DataFrame and write with a checkpoint. Each micro-batch infers only the new rows.
- **Lakeflow Jobs:** schedule a SQL or notebook task that runs the `ai_query` statement on a cadence. Refresh standalone materialized views with the SQL task type.

Incremental design consideration

The materialized view incremental-refresh support table lists only *deterministic* built-in functions and immutable UDFs as incrementalizable, and says other non-deterministic functions aren't supported. If a materialized view that calls an AI function falls back to a full recompute, every row is re-inferred and billed again. Check with `EXPLAIN CREATE MATERIALIZED VIEW` or the pipeline's incrementalization status. For append-only inputs, run inference in a streaming table or a Structured Streaming query, which processes each new row once. Source: Incremental refresh for materialized views

Batch Inference at Scale

Documented best practices:

- **Submit the full dataset in a single query.** AI Functions handle parallelization, retries, and scaling. Splitting data into small manual batches can reduce throughput.
- **Use Databricks-hosted `system.ai` models,** not provisioned throughput endpoints, for batch work. They're fully managed and scale automatically.
- **Pick a batch-optimized model.** The docs list models optimized for high-throughput batch. Non-optimized models give lower throughput.
- **Set `failOnError => false` for large workloads** so the job completes, keeps successful rows, and records per-row errors.
- **Monitor progress** in the query profile. The **AI Query** node shows completed and failed inferences and total time.

For provisioned throughput models (for example fine-tuned foundation models), AI Functions don't use the compute provisioned to the endpoint. AI Functions manage the scaling used for batch inference.

Cost

Your query compute (SQL warehouse, job cluster, or pipeline) is always billed. Inference cost depends on the function:

Function group	Inference b
Task-specific functions (for example `ai_classify`, `ai_summarize`, `ai_translate`)	Databricks-
`ai_query`	Billed for th
`ai_forecast`, `ai_detect_anomalies`, `ai_predict_class`, `ai_predict_value`, `ai_top_drivers`	Run entirely

In `system.billing.usage`:

- `ai_similarity`, `ai_analyze_sentiment`, `ai_summarize`, `ai_translate`, `ai_fix_grammar`, and `ai_mask` are recorded under `billing_origin_product = 'MODEL_SERVING'` with `product_features.model_serving.offering_type = 'BATCH_INFERENCE'`. `ai_query` batch workloads also appear under this product.
- Other inference-billed task functions, such as `ai_classify`, `ai_extract`, `ai_parse_document`, and `ai_search`, are recorded under `billing_origin_product = 'AI_FUNCTIONS'`.

```
-- Batch inference usage attributed to Lakeflow pipelines
SELECT u.usage_metadata.dlt_pipeline_id AS pipeline_id,
```

```
      SUM(u.usage_quantity)            AS usage_quantity
FROM system.billing.usage u
WHERE u.billing_origin_product = 'MODEL_SERVING'
  AND u.product_features.model_serving.offering_type = 'BATCH_INFERENCE'
  AND u.usage_metadata.dlt_pipeline_id IS NOT NULL
GROUP BY ALL;
```

`ai_query` calls to `system.ai` model services are also captured in the Unity Gateway usage table `system.ai_gateway.usage`.

Governance: Who Can Call What

Control	What it governs
Endpoint permission	`ai_query` against a serving endp
Unity Gateway model services (`system.ai`)	Databricks-hosted models querie
AI Functions Unity Catalog permissions (Public Preview, needs enablement)	Task-specific AI Functions in syst
UC SQL function wrapper	Who can invoke a specific promp
Compute and workspace settings	Whether custom or external end

Platform update (Oct 2026)

Foundation model Unity Catalog permissions no longer accept new enrollments. To restrict access to Databricks-hosted models, docs direct you to Unity Gateway model APIs. Source: Foundation model Unity Catalog permissions

Data security: for task-specific functions such as `ai_classify` and `ai_extract`, documents are processed inside the Databricks security perimeter. Databricks doesn't store the parameters passed to the call, but it does retain run metadata.

Failure Handling Patterns

Failure	Behavior
Model endpoint error on one row (`ai_query`)	Default `failOnError => true` fails the query
Non-endpoint error (for example a bad argument)	Whole query fails even with `failOnError => false`
Task function can't produce a result	Version 2.x functions return a `VARIANT` with `error_mes`
Output doesn't match the expected schema or labels	LLM output varies
Permission error on a custom or external endpoint	Admin turned off the feature, or the definer lacks `CAN`

Use Cases

- **Classification and routing:** `ai_classify` support tickets into queues, with label descriptions for accuracy.
- **Entity and field extraction:** `ai_extract` invoice or contract fields into typed Silver columns. Chain it after `ai_parse_document` for PDFs.
- **Sentiment and summarization:** `ai_analyze_sentiment` and `ai_summarize` on reviews or call transcripts for Gold dashboards.
- **PII protection:** `ai_mask` free text before sharing it downstream.
- **Custom models in SQL:** `ai_query` to score rows with a custom ML model (spam, churn) served on Model Serving.

Common Issues & Errors

- **"Function not available" on a SQL warehouse:** AI Functions aren't available on Classic SQL warehouses.
- **One malformed response kills a nightly job:** `failOnError` defaults to `true`. Set it to `false` for batch.
- **Throughput lower than expected:** manual micro-batching, a non-batch-optimized model, or a provisioned throughput endpoint. Submit one query against a batch-optimized `system.ai` model.
- **Unexpected cost spikes:** a materialized view with inference may be recomputing fully on each refresh. Check the refresh technique in the event log (`planning_information`) or move inference to a streaming table.
- **`ai_classify` results changed shape after an upgrade:** version 2.1 returns label objects (`value`). Pin `map('version', '2.1')` and read `response[0].value`.
- **Rate limits or guardrails configured in Unity Gateway don't apply:** gateway rate limits, guardrails, inference tables, and fallbacks aren't applied to `ai_query` batch workloads.

Best Practices

- Prefer task-specific functions. Use `ai_query` only when you need prompt, model, or output-format control.
- Always constrain output, with `responseFormat` for `ai_query`, `enum` / typed schemas for `ai_extract`, and explicit labels for `ai_classify`. Then validate with pipeline expectations.
- Pin function versions (`map('version', '2.1')`) for production stability.
- Attribute cost with `system.billing.usage` and `system.ai_gateway.usage`. Restrict who can call which model or function through Unity Catalog grants.

Exam Tips

Tip

- **"Classify / extract / sentiment / summarize / mask / translate, with no custom model"** points to the task-specific function. **"Specific model, custom prompt, custom ML endpoint, or exact JSON schema"** points to `ai_query`.
- **`failOnError => false`** returns a STRUCT with `response` and `errorMessage`, so a batch completes despite bad rows.
- **`responseFormat`** enforces structured output for chat models. **`returnType`** declares the type for custom model endpoints. **`modelParameters`** passes constants such as `max_tokens` and `temperature`.
- **Batch at scale:** use one query over the full dataset, Databricks-hosted `system.ai` models, and no manual batching. AI Functions handle retries and scaling.
- **Where it runs:** Lakeflow pipelines (MV or streaming table definitions), scheduled jobs, or Structured Streaming. Not Classic SQL warehouses.
- **Governance:** `CAN QUERY` on the endpoint for `ai_query`, `EXECUTE` on `system.ai` objects, and AI Functions UC permissions for task functions only (not `ai_query`).

Key Takeaways

- **Two families:** task-specific AI Functions (managed, no prompt) and `ai_query` (general-purpose, any supported endpoint).
- **`ai_query` arguments:** `endpoint`, `request`, plus optional `returnType`, `failOnError` (default `true`), `modelParameters`, `responseFormat`, and `files`.
- **Structured outputs:** `ai_classify` and `ai_extract` (versions 2.x) return `VARIANT` with `response` / `error_message`. Read them with `VARIANT` path syntax.

- **Pipelines:** call AI Functions inside Lakeflow pipeline datasets, validate outputs with expectations, and watch materialized view refresh behavior because inference is expensive to recompute.
- **Cost:** query compute plus inference. Track it in `system.billing.usage` (`MODEL_SERVING` / `BATCH_INFERENCE` and `AI_FUNCTIONS`).
- **Governance:** Unity Catalog and Unity Gateway privileges decide who can call which model or function.

Related Topics

- VARIANT and Semi-Structured Data - parsing `VARIANT` results from AI Functions
- Expectations and Data Quality - quarantining invalid LLM outputs
- Lakeflow Spark Declarative Pipelines - streaming tables and materialized views
- System Tables - `system.billing.usage` cost attribution
- Cost Optimization - serverless and inference cost control
- Access Control - Unity Catalog grants

Official Documentation

- Transform unstructured data using AI Functions
- Use ai_query
- ai_query function
- Deploy batch inference pipelines
- ai_classify function
- ai_extract function
- ai_analyze_sentiment function
- ai_summarize function
- ai_mask function
- ai_translate function
- AI Functions Unity Catalog permissions
- Discover and govern access to model services
- Query model services (Unity Gateway)
- Foundation model Unity Catalog permissions
- Incremental refresh for materialized views
- Databricks platform release notes, July 2026

Expectations and Data Quality

Overview

Expectations are SQL Boolean constraints attached to streaming tables, materialized views, and temporary views in a Lakeflow pipeline. Each one has a name, a constraint, and an action (**warn**, **drop**, or **fail**), and its pass/fail counts are recorded in the pipeline event log. A **quarantine** keeps invalid records for review instead of silently losing them.

Platform update (Oct 2026)

The Python decorators moved from the `dlt` module to `pyspark.pipelines`: write `@dp.expect`, `@dp.expect_or_drop`, `@dp.expect_or_fail` after `from pyspark import pipelines as dp`. `import dlt` / `@dlt.expect` still work. The exam's "Lakeflow Pipelines" is the docs' "Lakeflow Spark Declarative Pipelines". Expectations are a Lakeflow feature and are not part of open-source Apache Spark Declarative Pipelines. Sources: Expectations (Python reference), Lakeflow pipelines Python language reference

Overview

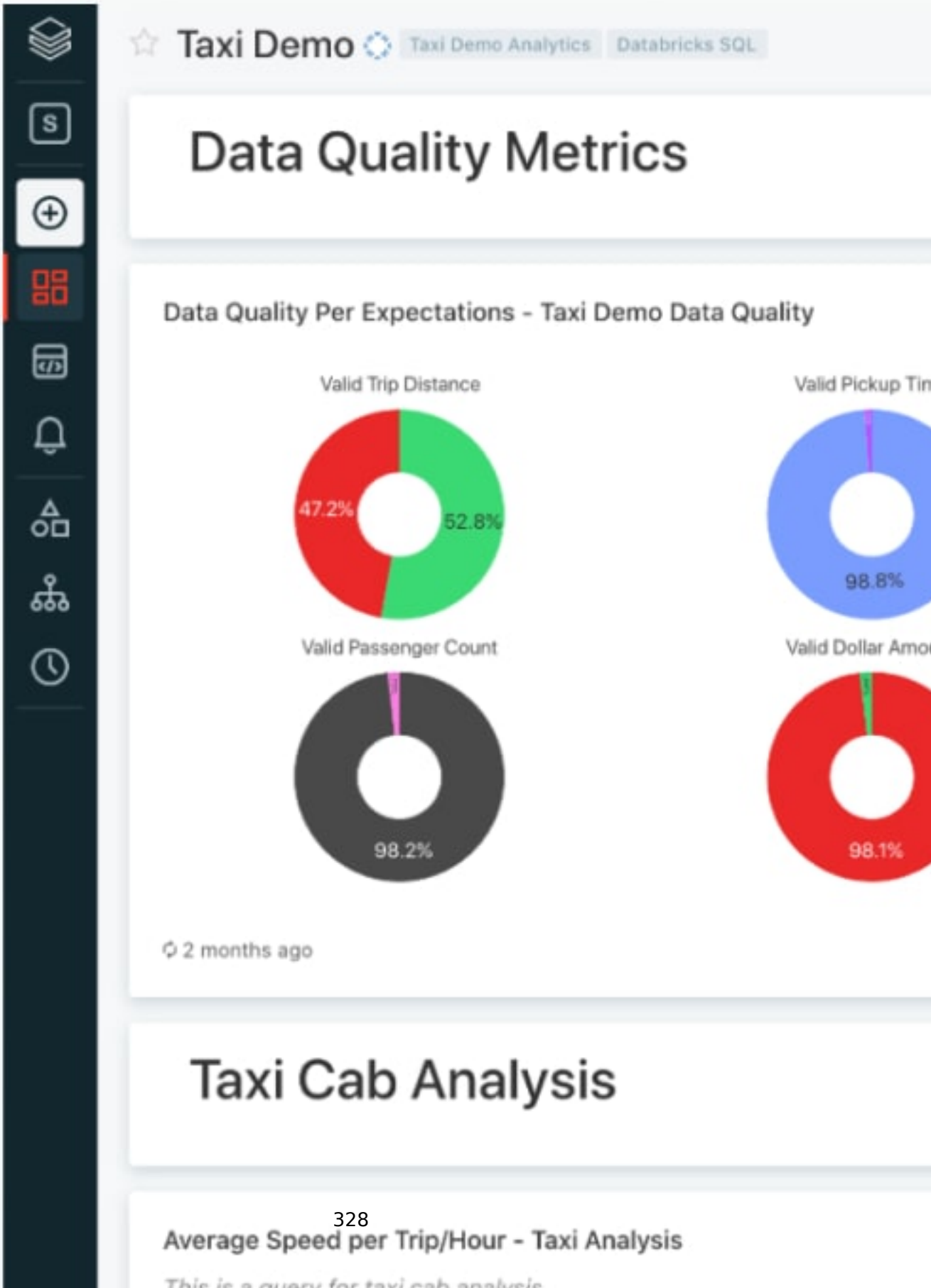
- **Warn** (default, `EXPECT` / `@dp.expect`): invalid rows are written to the target and counted
- **Drop** (`ON VIOLATION DROP ROW` / `@dp.expect_or_drop`): invalid rows are dropped before the write and counted as dropped
- **Fail** (`ON VIOLATION FAIL UPDATE` / `@dp.expect_or_fail`): an invalid row fails the flow's update, which is rolled back; manual fixing is required
- **Quarantine**: flag invalid rows with warn-level expectations and route them to a separate path, instead of dropping or failing

What the Exam Tests

- Implement data quality expectations in Lakeflow Pipelines to quarantine, drop, or fail on bad records
- Choose the action for a business rule (keep and monitor vs remove vs stop the update)
- Write the SQL `CONSTRAINT ... EXPECT ... ON VIOLATION` and the Python `@dp.expect*` / `expect_all*` forms
- Find expectation metrics in the event log (`flow_progress` → `data_quality`)

How Expectations Work

```
flowchart LR
    In[Input rows] --> E{Constraint true?}
    E -->|Yes| Out[Target dataset]
    E -->|No, warn| Out
    E -->|No, drop| X[Dropped<br>count logged]
    E -->|No, fail| F[Flow update fails<br>and rolls back]
    Out --> M[Pass / fail metrics<br>in event log]
    X --> M
```


Taxi Demo
Taxi Demo Analytics
Databricks SQL
Data Quality Metrics
Data Quality Per Expectations - Taxi Demo Data Quality
Valid Trip Distance
47.2%
52.8%
Valid Pickup Tim
98.8%
Valid Passenger Count
98.2%
Valid Dollar Amou
98.1%
2 months ago
Taxi Cab Analysis
Average Speed per Trip/Hour - Taxi Analysis

Lakeflow pipeline graph showing expectation pass, fail, and drop row counts per table.

Each expectation has three parts:

1. **Name**: identifies the expectation in metrics and errors. It must be unique within a dataset, and you can reuse it across datasets.
2. **Constraint**: a SQL Boolean expression evaluated per record. It must not contain custom Python functions, calls to external services, or subqueries that reference other tables.
3. **Action**: warn (default), drop, or fail.

Choosing the Action

Action	SQL	Python	Invalid rows
Warn	`CONSTRAINT c EXPECT (cond)`	`@dp.expect("c", "cond")`	Written to the target
Drop	`... ON VIOLATION DROP ROW`	`@dp.expect_or_drop("c", "cond")`	Dropped before the write
Fail	`... ON VIOLATION FAIL UPDATE`	`@dp.expect_or_fail("c", "cond")`	Stop execution; table upd

Scenario	Action
Track a soft rule (missing optional email) without losing rows	Warn
Rows missing a required key must never reach silver	Drop (or quarantine if someo
Bad rows need investigation and possible repair	Quarantine (warn + flag + se
A violation means the batch is corrupt (negative balances in a financial report)	Fail
Validate across tables (row counts match, no missing keys)	Fail on a validation materializ

Fail scope

In a **triggered** pipeline, a failed expectation fails and rolls back only **that flow**; other parallel flows keep updating. In a **continuous** pipeline it stops the flow **and all dependent flows**. Expectations enforce data quality, not orchestration. A validation table does not stop downstream tables from updating. To gate downstream work on a validation, split them into separate pipelines and chain the pipeline tasks in a job.

SQL Syntax

```sql
-- Warn: keep the row, record the violation
CREATE OR REFRESH STREAMING TABLE silver_events (
  CONSTRAINT valid_event_type EXPECT (event_type IN ('click', 'view', 'purchase'))
)
AS SELECT * FROM STREAM bronze_events;

-- Drop: several constraints, separated by commas
CREATE OR REFRESH STREAMING TABLE silver_transactions (
  CONSTRAINT valid_transaction_id EXPECT (transaction_id IS NOT NULL) ON
   ↪ VIOLATION DROP ROW,
  CONSTRAINT valid_amount EXPECT (amount IS NOT NULL AND amount > 0) ON VIOLATION
   ↪ DROP ROW,
  CONSTRAINT valid_currency EXPECT (currency IN ('USD', 'EUR', 'GBP')) ON
   ↪ VIOLATION DROP ROW
)
AS SELECT * FROM STREAM bronze_transactions;

-- Fail: stop the update on any violation
```

```sql
CREATE OR REFRESH MATERIALIZED VIEW gold_account_balances (
  CONSTRAINT non_negative_balance EXPECT (balance >= 0) ON VIOLATION FAIL UPDATE
)
AS SELECT account_id, SUM(amount) AS balance
FROM silver_transactions
GROUP BY account_id;
```

The expectation expression may use literals, the dataset's columns, and deterministic built-in functions. It **cannot** contain aggregate functions, window functions, table-valued generator functions, or subqueries. Aggregate checks therefore go on a materialized view whose **output columns** hold the aggregates, as in `gold_account_balances` above.

Python Syntax

Expectation decorators go **after** the dataset decorator (`@dp.table`, `@dp.materialized_view`, `@dp.temporary_view`) and before the function.

```python
from pyspark import pipelines as dp
from pyspark.sql.functions import col

@dp.table(name="silver_orders")
@dp.expect("valid_date", "order_date <= current_date()")                # warn
@dp.expect_or_drop("valid_order_id", "order_id IS NOT NULL")            # drop
@dp.expect_or_fail("valid_currency", "currency IN ('USD', 'EUR', 'GBP')")  # fail
def silver_orders():
    return spark.readStream.table("bronze_orders")
```

Groups of expectations (Python only) Only Python can group expectations and apply **one collective action** with `expect_all`, `expect_all_or_drop`, or `expect_all_or_fail`. Each takes a dictionary of `{name: constraint}`. In SQL you list several `CONSTRAINT` clauses, each with its own action.

```python
order_rules = {
    "valid_order_id": "order_id IS NOT NULL",
    "valid_customer": "customer_id IS NOT NULL",
    "valid_amount": "amount IS NOT NULL AND amount > 0",
}

@dp.table(name="orders_monitored")
@dp.expect_all(order_rules)          # warn on each rule
def orders_monitored():
    return spark.readStream.table("bronze_orders")

@dp.table(name="orders_clean")
@dp.expect_all_or_drop(order_rules)  # drop a row that fails any rule
def orders_clean():
    return spark.readStream.table("bronze_orders")

@dp.materialized_view(name="orders_for_finance")
@dp.expect_all_or_fail(order_rules)  # fail the update if any rule fails
def orders_for_finance():
    return spark.read.table("orders_clean")
```

Expectations on AUTO CDC targets A target created with `dp.create_streaming_table()` takes expectations as **parameters** (`expect_all`, `expect_all_or_drop`, `expect_all_or_fail`),

because no decorated function is involved. In SQL, use the `CONSTRAINT` clause on the target. Expectations are **not supported** with AUTO CDC FROM SNAPSHOT.

```
dp.create_streaming_table(
    name="customers_current",
    expect_all_or_drop={"valid_id": "customer_id IS NOT NULL"},
)
```

Reusable rules Databricks recommends keeping rule definitions separate from pipeline code (a Delta table of rules or a Python module), tagging related rules, and loading them with a helper that returns a dictionary. Loading rules dynamically is supported in Python, not SQL.

```
def get_rules(tag):
    rows = spark.read.table("main.dq.rules").filter(col("tag") == tag).collect()
    return {row["name"]: row["constraint"] for row in rows}

@dp.table(name="raw_markets")
@dp.expect_all_or_drop(get_rules("validity"))
def raw_markets():
    return spark.readStream.table("bronze_markets")
```

Quarantine Pattern

Use quarantine when invalid records must be kept for review or repair, rather than dropped (lost from the main flow) or failed (blocking everyone).

```
flowchart LR
    Raw[Raw rows] --> Q["Private table with warn expectations<br>+ is_quarantined
 ↪  flag"]
    Q -->|is_quarantined = false| Valid[valid view / silver table]
    Q -->|is_quarantined = true| Bad[quarantine view / table]
    Bad --> Review[Review and repair]
```

Documented pattern: flag, then split Warn-level expectations record metrics for every rule. A computed `is_quarantined` column (the negation of all rules combined) splits the data into valid and invalid paths. Nothing is dropped, so the counts reconcile.

```
from pyspark import pipelines as dp
from pyspark.sql.functions import expr

rules = {
    "valid_pickup_zip": "(pickup_zip IS NOT NULL)",
    "valid_dropoff_zip": "(dropoff_zip IS NOT NULL)",
}
quarantine_rules = "NOT({0})".format(" AND ".join(rules.values()))

@dp.temporary_view
def raw_trips_data():
    return spark.readStream.table("samples.nyctaxi.trips")

@dp.table(private=True, partition_cols=["is_quarantined"])
@dp.expect_all(rules)                          # warn: metrics, no rows lost
def trips_data_quarantine():
    return (
        spark.readStream.table("raw_trips_data")
        .withColumn("is_quarantined", expr(quarantine_rules))
```

```
    )

@dp.temporary_view
def valid_trips_data():
    return spark.read.table("trips_data_quarantine").filter("is_quarantined =
     ↪  false")

@dp.temporary_view
def invalid_trips_data():
    return spark.read.table("trips_data_quarantine").filter("is_quarantined =
     ↪  true")
```

```
-- SQL equivalent
CREATE OR REFRESH PRIVATE STREAMING TABLE trips_data_quarantine (
  CONSTRAINT valid_pickup_zip EXPECT (pickup_zip IS NOT NULL),
  CONSTRAINT valid_dropoff_zip EXPECT (dropoff_zip IS NOT NULL)
)
PARTITIONED BY (is_quarantined)
AS SELECT
  *,
  NOT ((pickup_zip IS NOT NULL) AND (dropoff_zip IS NOT NULL)) AS is_quarantined
FROM STREAM(samples.nyctaxi.trips);

CREATE TEMPORARY VIEW valid_trips_data AS
SELECT * FROM trips_data_quarantine WHERE is_quarantined = FALSE;

CREATE TEMPORARY VIEW invalid_trips_data AS
SELECT * FROM trips_data_quarantine WHERE is_quarantined = TRUE;
```

To publish the bad rows for a review team, make the invalid path a published table instead of a temporary view.

Variant: drop from silver, mirror into a quarantine table

```
VALID = "order_id IS NOT NULL AND amount IS NOT NULL AND amount > 0"

@dp.table(name="silver_orders")
@dp.expect_or_drop("valid_order", VALID)
def silver_orders():
    return spark.readStream.table("bronze_orders")

@dp.table(name="quarantine_orders")
def quarantine_orders():
    return spark.readStream.table("bronze_orders").filter(f"NOT ({VALID})")
```

The two filters must be **exact complements**. Write constraints with explicit `IS NOT NULL` guards so that they never evaluate to `NULL`. Otherwise a row can fall out of both paths, because `NOT (NULL)` is still `NULL` and `WHERE` keeps only `TRUE`.

Pattern	Rows lost?	Metrics
Drop only	Yes (counted, not kept)	Dropped count
Flag and split (documented quarantine)	No	Per-rule pass/fail
Drop + mirror table	No, if conditions are exact complements	Dropped count on silver
Fail	Update rolled back	None for the failing updat

Cross-Table Validation

For checks across datasets, define a validation materialized view with a **fail** expectation:

```
@dp.materialized_view(name="count_verification")
@dp.expect_or_fail("no_rows_dropped", "a_count == b_count")
def count_verification():
    return spark.sql("""
        SELECT * FROM
          (SELECT COUNT(*) AS a_count FROM table_a),
          (SELECT COUNT(*) AS b_count FROM table_b)
    """)

@dp.materialized_view(name="report_pk_tests")
@dp.expect_or_fail("unique_pk", "num_entries = 1")
def report_pk_tests():
    return spark.read.table("report").groupBy("pk").count().withColumnRenamed(
     ↪  "count",
     ↪  "num_entries")
```

Downstream tables do not wait for these checks. Use separate pipelines chained in a job if a failed check must block publication.

Monitoring Expectations

The pipeline UI shows warn and drop metrics in the dataset's **Data quality** tab. In the event log, `flow_progress` events store per-expectation `passed_records` and `failed_records` in `details:flow_progress.data_quality.expectations`, and `dropped_records` in `details:flow_progress.data_quality`.

```
SELECT
  row_expectations.dataset AS dataset,
  row_expectations.name AS expectation,
  SUM(row_expectations.passed_records) AS passing_records,
  SUM(row_expectations.failed_records) AS failing_records
FROM (
  SELECT explode(
    from_json(
      details:flow_progress:data_quality:expectations,
      "array<struct<name: string, dataset: string, passed_records: int,
       ↪  failed_records: int>>"
    )
  ) AS row_expectations
  FROM event_log(<pipeline-id>)
  WHERE event_type = 'flow_progress'
)
GROUP BY row_expectations.dataset, row_expectations.name;
```

Metrics are **missing** when no expectations are defined, when the flow type does not support expectations (for example, sinks), when there was no update to the dataset in that run, or when the update failed on a fail expectation. Views are computed only when queried, so a view's metrics may be missing or appear several times. More event log queries are in Lakeflow Event Logs.

Requirements and Limitations

- Expectations require the **ADVANCED** product edition, which is the default `edition` (`CORE` covers streaming ingest, `PRO` adds CDC).

- Supported on streaming tables, materialized views, and temporary views only, so data quality metrics exist only for those.
- Not supported on AUTO CDC FROM SNAPSHOT flows or sinks.
- You can also add expectations to standalone streaming tables and materialized views in Databricks SQL with the same `CONSTRAINT` clause; the Data quality tab isn't available there, so query the event log instead.

Use Cases

- **Silver order gate**: `expect_all_or_drop` removes rows without `order_id` or with non-positive amounts. A warn expectation tracks missing emails without losing orders.
- **Repairable bad data**: a flag-and-split quarantine keeps rows with invalid zip codes in a separate path for a data-steward review job, and the metrics show the failure rate per rule.
- **Financial close**: `expect_or_fail` on a gold balance view stops the update if any balance is negative, so the previous good version stays published.

Common Issues & Errors

Symptom	Cause
Every row fails a string check	Unquoted literal (`status = active`) or c
Rows with NULLs slip through or vanish from both quarantine paths	Constraint evaluates to `NULL` for null colu
`EXPECTATION_VIOLATION` error stops a flow	`expect_or_fail` triggered
No metrics for a fail expectation	Update failed before metrics were record
No metrics for a dataset	It is a view, a sink, unchanged in the run
Downstream table updated despite failed validation table	Expectations do not orchestrate
Aggregate inside the constraint rejected	Aggregates and subqueries are not allow
Pipeline rejects a dataset definition with repeated expectations	Two expectations share a name on one d

Note

The docs disagree on whether expectations stop a materialized view from refreshing incrementally. The CREATE MATERIALIZED VIEW reference says an MV with expectations is fully refreshed on every update. The incremental refresh page lists expectations as supported, except when the MV reads from a view that has expectations, or has a DROP expectation together with NOT NULL columns. Check the refresh type of the update (Incrementalization column or the `planning_information` event) instead of assuming.

Best Practices

- **Name expectations descriptively** (`valid_order_id`, not `c1`); the name appears in metrics and errors.
- **Layer the rules**: in bronze, keep everything and add warn-only expectations for observability. In silver, drop or quarantine on required fields and business rules. In gold, fail on critical invariants and cross-table checks.
- **Keep rules in one place** (a rules table or Python module) and apply them with `expect_all*` helpers.
- **Prefer quarantine over drop** when someone must be able to explain or repair rejected rows.

Exam Tips

Tip

- **Three actions**: `EXPECT` = warn (keep row), `ON VIOLATION DROP ROW` = drop, `ON VIOLATION FAIL UPDATE` = fail.
- **Python**: `@dp.expect`, `@dp.expect_or_drop`, `@dp.expect_or_fail`; grouped `@dp.expect_all`, `@dp.expect_all_or_drop`, `@dp.expect_all_or_fail` take a dict (Python only).
- **Quarantine** = keep invalid rows separately (flag with warn expectations and split, or drop plus a complementary table). It is not an `ON VIOLATION` option.
- **Fail** rolls back the flow's update; triggered pipelines keep other flows running, continuous pipelines stop dependent flows.
- **Metrics** live in `flow_progress` events under `data_quality`; there are no metrics for a failed update.
- Expectations are **Lakeflow-only** (not in open-source SDP) and need the ADVANCED edition (the default).

Key Takeaways

- **Warn, drop, fail**: warn writes invalid rows and counts them, drop removes them before the write, and fail stops the flow's update and rolls it back.
- **SQL**: `CONSTRAINT name EXPECT (cond) [ON VIOLATION DROP ROW | FAIL UPDATE]`. **Python**: `@dp.expect*` decorators placed after `@dp.table` / `@dp.materialized_view` / `@dp.temporary_view`.
- **Group actions with `expect_all*`** (Python only); for AUTO CDC targets pass `expect_all*` to `dp.create_streaming_table()`.
- **Quarantine** keeps bad rows reviewable. The documented pattern uses warn expectations plus an `is_quarantined` flag and two views; drop plus a mirror table also works if the conditions are exact complements.
- **Constraints** are per-row deterministic SQL: no UDFs, external calls, subqueries, or aggregates. Guard NULLs explicitly.
- **Expectations do not orchestrate**: to block downstream work on a validation, use separate pipelines and job dependencies.

Related Topics

- Declarative Pipelines — dataset types and run modes
- AUTO CDC API — CDC targets and their expectations
- Data Deduplication — removing duplicates before validation
- Lakeflow Event Logs — querying data quality metrics

Official Documentation

- Manage data quality with pipeline expectations
- Expectation recommendations and advanced patterns
- Expectations (Python reference)
- CREATE STREAMING TABLE (pipelines)
- CREATE MATERIALIZED VIEW (pipelines)
- create_streaming_table
- Monitor pipelines with the event log
- Incremental refresh for materialized views
- Pipeline properties reference (editions)

Section 4: Monitoring and Alerting (10 % of Exam)

Watching production workloads: system tables (billing, compute, access, lakeflow), the pipeline event log, Databricks REST APIs / CLI / SDK for jobs and pipelines, Lakehouse alerts across business, quality, cost, query, security and AI-agent signals, and Lakeflow Jobs run monitoring.

Topics Overview

```
flowchart LR
    D[Monitoring]
    D --> T1["System Tables"]
    D --> T2["Lakeflow/Lakeflow Pipelines Event Logs"]
    D --> T3["Lakehouse Alerts"]
    D --> T4["Databricks CLI"]
    D --> T5["Databricks REST API — Part 1: Jobs, Clusters, DBFS & Workspace APIs"]
    D --> T6["Streaming Monitoring & Optimization"]
```

Section Contents

File	Topic
01-system-tables.md	`system.billing`, `system.compute`, `system.access`, `system.lake`
02-lakeflow-event-logs.md	Pipeline event log: health, flow progress, expectation metrics
03-lakehouse-alerts.md	Alerts on metrics, quality, cost, query health, audit events, agent
04-databricks-cli-part1.md	CLI install, auth, workspace and jobs commands
04-databricks-cli-part2.md	Clusters, secrets, bundles, UC commands, output formats
05-rest-api-part1.md	REST API basics, auth, Jobs API
05-rest-api-part2.md	Permissions, SQL Statement Execution, Databricks SDK for Python
06-streaming-monitoring-optimization.md	Streaming metrics, back-pressure, state store monitoring

Exam Skill Coverage

Every skill bullet in Section 4 of the Oct 9, 2026 exam guide, with the files that cover it.

Exam guide skill
Use Databricks system tables (billing, compute, access, lakeflow) for cost analysis, auditing, and workload mo
Use Databricks REST APIs / CLI and SDK for monitoring and analyzing jobs and pipelines.
Use Pipeline event logs to monitor pipeline health and data quality.
Use Databricks Lakehouse alerts to monitor governed business metrics, data quality and anomaly signals, usa
Use Lakeflow Jobs UI and Jobs API to monitor job status and performance metrics.

Key Concepts to Master

Concept	Why it matters
`system.billing.usage`	DBU consumption per workload; join with `system.billing.list_prices` for cost at
`system.lakeflow.*`	Job and pipeline metadata and run timelines for workload monitoring
Pipeline event log	Per-pipeline Delta log of updates, flow progress and expectation results
Alerts	A scheduled query evaluated against a condition that notifies destinations — and ca
Databricks SDK	Typed Python client (`WorkspaceClient`) over the REST API for scripted monitoring

Related Resources

- Unity Catalog cheat sheet
- Performance troubleshooting (shared)

System Tables

Overview

System tables are a Databricks-hosted analytical store of your account's operational data, exposed as read-only Delta tables in the `system` catalog of every Unity Catalog metastore. For the exam, four schemas matter most. `system.billing` covers cost, `system.compute` covers clusters and warehouses, `system.access` covers audit and lineage, and `system.lakeflow` covers jobs and pipelines. `system.query.history` is the usual companion for SQL workload monitoring.

Overview

- `system.billing.usage` holds DBU usage per record. `system.billing.list_prices` turns it into list cost through `pricing.default` over a price validity window.
- `usage_metadata` (`job_id`, `job_run_id`, `warehouse_id`, `cluster_id`, `dlt_pipeline_id`, ...) and `identity_metadata.run_as` attribute spend to workloads and identities.
- `system.compute.*` and `system.lakeflow.*` are mostly SCD2 dimension tables (latest row per entity by `change_time`). They also include immutable timelines sliced into rows of at most one hour.
- `system.access.audit` records who did what (`service_name`, `action_name`, `user_identity`, `request_params`, `response`). Lineage lives in `system.access.table_lineage` and `column_lineage`.

What the Exam Tests

- "Use Databricks system tables (billing, compute, access, lakeflow) for cost analysis, auditing, and workload monitoring."
- Attributing DBU spend to specific jobs, pipelines, and SQL warehouses by joining `system.billing.usage` with `system.billing.list_prices` and the compute and lakeflow tables. The exam guide's sample question tests exactly this.
- Choosing the right table for a question (audit trail vs lineage vs job runs vs warehouse events vs query history) and filtering it correctly.
- Knowing the access model (UC privileges, admins by default), retention, and record latency, so you know what system tables can and cannot answer.

Platform update (Oct 2026)

The `system.lakeflow` tables `pipelines` and `pipeline_update_timeline` became generally available on October 21, 2026. The `lakeflow` schema was previously named `workflow`, and both contain identical content. The `system.lineage` and `system.operational_data` schemas are deprecated and contain empty tables. Query lineage from `system.access.table_lineage` and `system.access.column_lineage`. Sources: October 2026 release notes, Jobs system table reference, System tables reference

---

The System Catalog at a Glance

```
flowchart TB
    subgraph SYS["system catalog"]
        B["billing<br/>usage, list_prices"]
        C["compute<br/>clusters, node_types, node_timeline,<br/>warehouses,
↪  warehouse_events,<br/>instance_events, instance_pools"]
        A["access<br/>audit, table_lineage,
↪  column_lineage,<br/>workspaces_latest, ..."]
        L["lakeflow<br/>jobs, job_tasks,
↪  job_run_timeline,<br/>job_task_run_timeline,<br/>pipelines,
↪  pipeline_update_timeline"]
        Q["query<br/>history"]
    end
    B -->|"usage_metadata.cluster_id / warehouse_id"| C
    B -->|"usage_metadata.job_id / job_run_id / dlt_pipeline_id"| L
    L -->|"compute_ids"| C
```

Table	What it records
`system.billing.usage`	Every billable usage record in the account
`system.billing.list_prices`	SKU price history (a row each time a price
`system.compute.clusters`	SCD history of classic compute configurati
`system.compute.node_timeline`	Per-minute node utilization of classic comp
`system.compute.warehouses`	History of SQL warehouse configurations
`system.compute.warehouse_events`	Warehouse start, stop, and scale events
`system.access.audit` (Public Preview)	Audit events from workspaces in the regio
`system.access.table_lineage` / `column_lineage`	Read and write events on UC tables and c
`system.lakeflow.jobs` / `job_tasks`	SCD history of job and task definitions
`system.lakeflow.job_run_timeline` / `job_task_run_timeline`	Job runs and task runs, sliced by hour
`system.lakeflow.pipelines` / `pipeline_update_timeline`	SCD history of pipelines, and pipeline upd
`system.query.history`	Queries on SQL warehouses, serverless nc

Other schemas you may meet in monitoring scenarios include `system.storage` (predictive optimization history), `system.serving` (endpoint usage), `system.alert` (alert configuration and evaluations), and `system.data_quality_monitoring` (anomaly detection results).

Access Requirements and Behavior

- **Unity Catalog required:** the account needs at least one UC-enabled workspace. The tables include data from all workspaces in the account, including non-UC workspaces, but you can only query them from a UC-enabled workspace. The metastore must be on Privilege Model Version 1.0.
- **Who can read by default:** users who hold **both** the account admin and metastore admin roles. Everyone else needs `USE CATALOG` on `system`, plus `USE SCHEMA` and `SELECT` on the schemas.
- **Read-only:** system tables cannot be modified.
- **Region scope:** regional tables only show workspaces in the same cloud region as the workspace you query from. Global tables (`billing.usage`, `billing.list_prices`) show the whole account.
- **Not real-time:** for example, `billing.usage` records typically arrive within 12 hours, while lakeflow, compute, and query history records typically arrive within an hour.

- **Schema drift:** Databricks can add new columns and struct fields at any time, but never changes or removes existing ones. Enable schema evolution if you copy system tables into your own tables.
- **Over-broad queries fail:** a query that isn't selective enough returns `System Table query returned too much data`. Add predicates, especially on date columns.

```
-- Grant a cost team read access to billing
GRANT USE CATALOG ON CATALOG system TO `cost-analysts`;
GRANT USE SCHEMA  ON SCHEMA  system.billing TO `cost-analysts`;
GRANT SELECT      ON SCHEMA  system.billing TO `cost-analysts`;
```

Streaming from system tables: system tables are shared with you through OpenSharing. Streaming reads need Databricks Runtime 16.4 or above, and reading the change data feed needs 17.3 or above. Set `skipChangeCommits` so that deletes in the source don't break the stream. Because the tables use a 7-day VACUUM retention, a stream that lags more than 7 days can break.

```
usage_stream = (spark.readStream
    .option("skipChangeCommits", "true")
    .table("system.billing.usage"))
```

Configurable retention (Beta): an account admin can opt in and set one retention period, from 30 to 3,650 days, for all supported system tables. While the Beta is enabled, the free period becomes 395 days. The `data_classification`, `data_quality_monitoring`, and `lakebase` schemas are excluded.

Billing: `system.billing`

`system.billing.usage` key columns

Column	Type	Notes
`record_id`	string	Unique ID of the usage record
`workspace_id`, `account_id`	string	Where the usage happened
`sku_name`	string	For example `STANDARD_ALL_PURPOSE_COMPUTE`. Joins
`cloud`	string	`AWS`, `AZURE`, or `GCP`. Also a `list_prices` join key.
`usage_start_time`, `usage_end_time`	timestamp	The usage interval (UTC)
`usage_date`	date	Use this for fast date filtering and aggregation
`usage_quantity`, `usage_unit`	decimal, string	Amount consumed and its unit (for example `DBU`)
`custom_tags`	map	Tags on the compute or usage policy. Use `custom_ta`
`usage_metadata`	struct	Resource IDs: `job_id`, `job_run_id`, `job_name`, `wareh`
`identity_metadata`	struct	`run_as` (who ran the workload), `owned_by` (SQL warel
`billing_origin_product`	string	The product that generated the usage: `JOBS`, `DLT` (La
`product_features`	struct	For example `is_serverless`, `is_photon`, `jobs_tier`,
`usage_type`	string	`COMPUTE_TIME`, `STORAGE_SPACE`, `NETWORK_BYTE`, `TOKE`
`record_type`	string	`ORIGINAL`, `RETRACTION`, or `RESTATEMENT`

Which `usage_metadata` field is populated depends on the workload:

Field	Populated for
`job_id`, `job_run_id`	Serverless jobs and jobs on **job compute**. Not populated for jobs on all-p
`warehouse_id`	Workloads run on a SQL warehouse
`cluster_id`	Non-serverless compute (notebooks, jobs, Lakeflow pipelines, legacy mo
`dlt_pipeline_id`, `dlt_update_id`	Lakeflow pipelines and features built on them (materialized views, Lakefl
`notebook_id`, `notebook_path`	Serverless notebooks

Corrections: a correction adds a RETRACTION row (negative usage_quantity) and usually a RESTATEMENT row. Always SUM(usage_quantity) so the retraction cancels the original. Never filter to record_type = 'ORIGINAL' when you report totals.

system.billing.list_prices

Column	Notes
price_start_time, price_end_time	Validity window of the price (UTC). price_end_time is NULL f
sku_name, cloud, currency_code, usage_unit	Join and unit keys
pricing	struct. pricing.default is a single list price for simple estir

The table only stores price **changes**, so the join must match the price window. Joining on sku_name alone duplicates rows whenever a price changed.

```
-- Correct list-cost join (the pattern used throughout the docs)
SELECT
  u.usage_date,
  u.billing_origin_product,
  SUM(u.usage_quantity)                       AS dbus,
  SUM(u.usage_quantity * p.pricing.default)   AS list_cost
FROM system.billing.usage u
JOIN system.billing.list_prices p
  ON  u.cloud    = p.cloud
  AND u.sku_name = p.sku_name
  AND u.usage_start_time >= p.price_start_time
  AND (u.usage_end_time <= p.price_end_time OR p.price_end_time IS NULL)
WHERE u.usage_date >= current_date() - INTERVAL 30 DAYS
GROUP BY ALL
ORDER BY u.usage_date DESC, list_cost DESC;

-- Chargeback by custom tag (tags come from compute, jobs, or serverless usage
 ↪  policies)
SELECT
  custom_tags['team']  AS team,
  sku_name,
  SUM(usage_quantity)  AS dbus
FROM system.billing.usage
WHERE usage_date >= date_trunc('MONTH', current_date())
  AND custom_tags['team'] IS NOT NULL
GROUP BY ALL
ORDER BY dbus DESC;
```

Cost Attribution: Billing + Pricing + Compute + Lakeflow

The exam guide's sample question asks how to attribute last month's DBU spend to specific jobs and SQL warehouses using governed data already in Unity Catalog. The answer is system.billing.usage joined with the pricing table and the compute and lakeflow tables. The Spark UI, driver logs, and PDF billing exports are all wrong answers.

```
flowchart LR
    U["billing.usage"] -- "cloud, sku_name,<br/>usage_start_time in price window"
 ↪  --> P["billing.list_prices<br/>pricing.default"]
    U -- "workspace_id +<br/>usage_metadata.job_id" -->
 ↪  J["lakeflow.jobs<br/>(latest row: name)"]
```

```
    U -- "workspace_id +<br/>usage_metadata.job_id + job_run_id" -->
 ↪  R["lakeflow.job_run_timeline<br/>(result_state, period_*)"]
    U -- "workspace_id +<br/>usage_metadata.warehouse_id" -->
 ↪  W["compute.warehouses<br/>(latest row: warehouse_name)"]
    U -- "workspace_id +<br/>usage_metadata.cluster_id" -->
 ↪  C["compute.clusters<br/>(owned_by, cluster_source)"]
    U -- "workspace_id +<br/>usage_metadata.dlt_pipeline_id" -->
 ↪  PL["lakeflow.pipelines<br/>(latest row: name)"]
```

```sql
-- Top 20 jobs and SQL warehouses by list cost, last calendar month
WITH priced AS (
  SELECT
    u.workspace_id,
    u.usage_metadata.job_id        AS job_id,
    u.usage_metadata.warehouse_id  AS warehouse_id,
    u.usage_quantity               AS dbus,
    u.usage_quantity * p.pricing.default AS list_cost
  FROM system.billing.usage u
  JOIN system.billing.list_prices p
    ON  u.cloud = p.cloud
    AND u.sku_name = p.sku_name
    AND u.usage_start_time >= p.price_start_time
    AND (u.usage_end_time <= p.price_end_time OR p.price_end_time IS NULL)
  WHERE u.usage_date BETWEEN date_trunc('MONTH', add_months(current_date(), -1))
                         AND last_day(add_months(current_date(), -1))
    AND u.usage_unit = 'DBU'
),
latest_jobs AS (            -- SCD2: keep the latest row per job
  SELECT workspace_id, job_id, name
  FROM system.lakeflow.jobs
  QUALIFY ROW_NUMBER() OVER (PARTITION BY workspace_id, job_id ORDER BY
 ↪  change_time DESC) = 1
),
latest_warehouses AS (      -- configuration history: keep the latest row per
 ↪  warehouse
  SELECT workspace_id, warehouse_id, warehouse_name
  FROM system.compute.warehouses
  QUALIFY ROW_NUMBER() OVER (PARTITION BY workspace_id, warehouse_id ORDER BY
 ↪  change_time DESC) = 1
)
SELECT
  CASE WHEN p.job_id IS NOT NULL THEN 'JOB' ELSE 'SQL_WAREHOUSE' END AS
   ↪  workload_type,
  COALESCE(j.name, w.warehouse_name)  AS workload_name,
  COALESCE(p.job_id, p.warehouse_id)  AS workload_id,
  SUM(p.dbus)      AS dbus,
  SUM(p.list_cost) AS list_cost
FROM priced p
LEFT JOIN latest_jobs j
  ON p.workspace_id = j.workspace_id AND p.job_id = j.job_id
LEFT JOIN latest_warehouses w
  ON p.workspace_id = w.workspace_id AND p.warehouse_id = w.warehouse_id
WHERE p.job_id IS NOT NULL OR p.warehouse_id IS NOT NULL
GROUP BY ALL
ORDER BY list_cost DESC
LIMIT 20;
```

```sql
-- Pipeline spend: billing enriched with pipeline names
WITH latest_pipelines AS (
  SELECT *
  FROM system.lakeflow.pipelines
  QUALIFY ROW_NUMBER() OVER (PARTITION BY workspace_id, pipeline_id ORDER BY
 ↪  change_time DESC) = 1
)
SELECT pl.name, u.usage_metadata.dlt_pipeline_id AS pipeline_id,
 ↪  SUM(u.usage_quantity) AS dbus
FROM system.billing.usage u
LEFT JOIN latest_pipelines pl
  ON u.workspace_id = pl.workspace_id
 AND u.usage_metadata.dlt_pipeline_id = pl.pipeline_id
WHERE u.usage_metadata.dlt_pipeline_id IS NOT NULL
  AND u.usage_date >= current_date() - INTERVAL 30 DAYS
GROUP BY ALL
ORDER BY dbus DESC;
```

Attribution caveats:

- **Jobs on all-purpose compute** carry no `usage_metadata.job_id`, because several workloads share that compute. You can only approximate their cost through `usage_metadata.cluster_id`. For precise attribution, run jobs on job compute or serverless.
- **Classic all-purpose usage:** `identity_metadata.run_as` is the identity that created the compute, not necessarily its owner. To attribute to the owner, join `usage_metadata.cluster_id` to `system.compute.clusters.owned_by`.
- **Notebook workflow runs** (`WORKFLOW_RUN`, for example `dbutils.notebook.run`) are billed to the parent notebook's run. They have no `job_id` or `job_run_id` of their own.
- **Region:** billing is global, but compute and lakeflow are regional. A join only matches rows for workspaces in the region you query from.

Compute: `system.compute`

Table	Type	Key columns
`clusters`	SCD (history of configs)	`cluster_id`, `cluster_name`, `owned_by`, `cluste`
`node_types`	Reference	`node_type`, `core_count`, `memory_mb`, `gpu_coun`
`node_timeline`	Per-minute metrics	`cluster_id`, `instance_id`, `start_time`, `end_t`
`warehouses`	History of configs	`warehouse_id`, `warehouse_name`, `warehouse_t`
`warehouse_events`	Events	`warehouse_id`, `event_type` (STARTING, RUNNIN
`instance_events` (Public Preview)	Events	`instance_id`, `event_type`, `state` (INSTANCE_L

`cluster_source` values: `UI` or `API` (all-purpose compute), `JOB` (all job compute), and `PIPELINE` or `PIPELINE_MAINTENANCE`. `data_security_mode`: `USER_ISOLATION` = Standard and `SINGLE_USER` = Dedicated.

Warning

`clusters` and `node_timeline` cover classic all-purpose, jobs, and pipeline compute only. They contain no records for **serverless** compute or **SQL warehouses**. Use `warehouses` and `warehouse_events` for warehouses, and `billing.usage` for serverless. Nodes that ran for less than 10 minutes might not appear in `node_timeline`.

```sql
-- Jobs running on all-purpose compute in the last 30 days (a cost anti-pattern)
WITH ap_clusters AS (
  SELECT workspace_id, cluster_id, cluster_name, owned_by
  FROM system.compute.clusters
  WHERE cluster_source IN ('UI', 'API')
  QUALIFY ROW_NUMBER() OVER (PARTITION BY workspace_id, cluster_id ORDER BY
 ↪  change_time DESC) = 1
),
task_compute AS (
  SELECT DISTINCT workspace_id, job_id, EXPLODE(compute_ids) AS cluster_id
  FROM system.lakeflow.job_task_run_timeline
  WHERE period_start_time >= current_date() - INTERVAL 30 DAYS
)
SELECT t.workspace_id, t.job_id, c.cluster_name, c.owned_by
FROM task_compute t
JOIN ap_clusters c USING (workspace_id, cluster_id);

-- Average and peak utilization per classic cluster over the last day
SELECT
  cluster_id,
  driver,
  AVG(cpu_user_percent + cpu_system_percent) AS avg_cpu_pct,
  MAX(cpu_user_percent + cpu_system_percent) AS peak_cpu_pct,
  AVG(mem_used_percent)                      AS avg_mem_pct
FROM system.compute.node_timeline
WHERE start_time >= current_timestamp() - INTERVAL 1 DAY
GROUP BY cluster_id, driver
ORDER BY avg_cpu_pct DESC;

-- Warehouse activity timeline for one warehouse this month (explains its billing)
SELECT event_time, event_type, cluster_count
FROM system.compute.warehouse_events
WHERE warehouse_id = '<warehouse-id>'
  AND event_time >= date_trunc('MONTH', current_date())
ORDER BY event_time DESC;
```

Access: `system.access`

Audit log: `system.access.audit` (Public Preview)

Column	Notes
`event_time`, `event_date`	Filter on `event_date` f
`workspace_id`	`0` for account-level ev
`service_name`, `action_name`	For example `unityCat`
`user_identity`	struct, for example `us`
`identity_metadata`	struct, with `run_by` (w
`request_params`	map, for example `req`
`response`	struct, for example re
`source_ip_address`, `user_agent`, `session_id`, `request_id`, `event_id`, `audit_level`	Request context

`service_name`	Example `action_name` values
`accounts`	`login`, `tokenLogin`, `samlLogin`, `oidcBrowserLogin`

service_name	Example action_name values
unityCatalog	getTable, createTable, deleteTable, updatePermissions, generateTemporaryTableCrede
clusters	create, edit, start, restart, delete, permanentDelete
jobs	create, delete, runNow, cancel, repairRun, runFailed
secrets	createScope, putSecret, getSecret, deleteSecret, listSecrets

Masked parameters: some request_params keys that hold SQL definitions (view_definition, function_info, definition_json, managed_definition) are omitted unless you are an account admin or a member of the databricks_pii_access group.

```
-- Who changed Unity Catalog permissions this week, and on what
SELECT
  event_time,
  user_identity.email                    AS changed_by,
  request_params['securable_type']       AS securable_type,
  request_params['securable_full_name']  AS securable,
  request_params['changes']              AS changes
FROM system.access.audit
WHERE event_date >= current_date() - 7
  AND service_name = 'unityCatalog'
  AND action_name  = 'updatePermissions'
ORDER BY event_time DESC;

-- Who read a sensitive table in the last 30 days
SELECT user_identity.email, COUNT(*) AS reads, MAX(event_time) AS last_read
FROM system.access.audit
WHERE event_date >= current_date() - 30
  AND service_name = 'unityCatalog'
  AND action_name  = 'getTable'
  AND request_params['full_name_arg'] = 'prod.hr.salaries'
GROUP BY user_identity.email
ORDER BY reads DESC;

-- Denied or failed requests by service in the last day
SELECT service_name, action_name, response.status_code, COUNT(*) AS events
FROM system.access.audit
WHERE event_date >= current_date() - 1
  AND response.status_code >= 400
GROUP BY ALL
ORDER BY events DESC;
```

Lineage: system.access.table_lineage and column_lineage Each row is a read or write event. Key columns are source_table_full_name, target_table_full_name (plus source_path and target_path for path-based access), source_type, target_type, entity_type and entity_id (the notebook, job, pipeline, and so on that performed it), created_by, event_time, and event_date. column_lineage adds source_column_name and target_column_name.

```
-- Downstream impact: everything written from prod.silver.customers in the last
 ↪  90 days
SELECT DISTINCT target_table_full_name, entity_type, entity_id
FROM system.access.table_lineage
WHERE source_table_full_name = 'prod.silver.customers'
  AND target_table_full_name IS NOT NULL
  AND event_date >= current_date() - 90;
```

Lakeflow: `system.lakeflow`

Table	Type	Entity key	Purpose
`jobs`	SCD2	`workspace_id`, `job_id`	Job definitions: name, r
`job_tasks`	SCD2	`workspace_id`, `job_id`, `task_key`	Task definitions: deper
`job_run_timeline`	Immutable	`workspace_id`, `job_id`, `run_id`	Job runs: `period_star`
`job_task_run_timeline`	Immutable	`workspace_id`, `run_id` (task run)	Task runs: `job_run_id`
`pipelines`	SCD2	`workspace_id`, `pipeline_id`	Pipeline definitions: pi
`pipeline_update_timeline`	Immutable	`workspace_id`, `pipeline_id`, `update_id`	Pipeline updates: upda

Reading SCD2 tables A change emits a **new row**, and a deletion is a new row with `delete_time` set. Take the latest row per entity **first**, then drop deleted entities.

```sql
-- Current, non-deleted jobs
WITH latest AS (
  SELECT *,
         ROW_NUMBER() OVER (PARTITION BY workspace_id, job_id ORDER BY change_time
          ↪  DESC) AS rn
  FROM system.lakeflow.jobs
  QUALIFY rn = 1
)
SELECT workspace_id, job_id, name, run_as, trigger_type
FROM latest
WHERE delete_time IS NULL;
```

Since October 19, 2026, saving a job without changing it no longer adds rows to `jobs` or `job_tasks`.

Timeline slicing

- Each timeline row covers **at most one hour**, so a run longer than an hour becomes several rows with the same `run_id`. Since January 19, 2026, new rows are aligned to clock hours (a run from 01:25 to 03:40 becomes rows ending at 02:00, 03:00, and 03:40).
- `result_state` and `termination_code` are populated **only on the final row** of a run. Filter `result_state IS NOT NULL` to count outcomes, and `SUM(period_end_time - period_start_time)` to get durations.
- A run that never started has `period_start_time = period_end_time`. Check `termination_code` to see why.
- The `*_duration_seconds` columns in `job_run_timeline` are populated only for single-task jobs created with legacy Jobs APIs. Use `job_task_run_timeline` for durations of multi-task jobs.

Reference values	
`run_type`	`JOB_RUN` (normal job run, in all job tables), `SUBMIT_RUN` (one-time `runs/submit`, in bo
Job `result_state`	`SUCCEEDED`, `FAILED`, `SKIPPED`, `CANCELLED`, `TIMED_OUT`, `ERROR`, `BLOCKED`, `NULL` (interme
`trigger_type`	`CRON`, `PERIODIC`, `CONTINUOUS`, `FILE_ARRIVAL`, `TABLE`, `MODEL`, `ONETIME`, `ONETIME_RETR`
Pipeline `result_state`	`COMPLETED`, `FAILED`, `CANCELED`
Pipeline `update_type`	`FULL_REFRESH`, `REFRESH`, `VALIDATE`
`pipeline_type`	`ETL_PIPELINE`, `MATERIALIZED_VIEW`, `STREAMING_TABLE`, `INGESTION_PIPELINE`, `INGES`

Workload monitoring queries

```sql
-- Failure rate per job over the last 7 days (terminal rows only)
WITH latest_jobs AS (
  SELECT workspace_id, job_id, name
```

```
  FROM system.lakeflow.jobs
  QUALIFY ROW_NUMBER() OVER (PARTITION BY workspace_id, job_id ORDER BY
 ↪  change_time DESC) = 1
),
terminal AS (
  SELECT workspace_id, job_id, run_id, result_state
  FROM system.lakeflow.job_run_timeline
  WHERE period_end_time >= current_timestamp() - INTERVAL 7 DAYS
    AND result_state IS NOT NULL
)
SELECT
  j.name,
  t.workspace_id,
  t.job_id,
  COUNT(*)                                                    AS terminal_runs,
  COUNT_IF(t.result_state IN ('FAILED', 'ERROR', 'TIMED_OUT')) AS failures,
  ROUND(100 * COUNT_IF(t.result_state IN ('FAILED', 'ERROR', 'TIMED_OUT')) /
   ↪  COUNT(*), 1) AS failure_pct
FROM terminal t
LEFT JOIN latest_jobs j USING (workspace_id, job_id)
GROUP BY ALL
ORDER BY failures DESC;

-- p50 / p90 duration per task over the last 7 days (sum hourly slices per task
 ↪  run first)
WITH task_runs AS (
  SELECT workspace_id, job_id, task_key, run_id,
         CAST(SUM(period_end_time - period_start_time) AS LONG) AS duration_seconds
  FROM system.lakeflow.job_task_run_timeline
  WHERE period_start_time >= current_timestamp() - INTERVAL 7 DAYS
  GROUP BY ALL
)
SELECT workspace_id, job_id, task_key,
       COUNT(*)                              AS task_runs,
       PERCENTILE(duration_seconds, 0.5)     AS p50_seconds,
       PERCENTILE(duration_seconds, 0.9)     AS p90_seconds
FROM task_runs
GROUP BY ALL
ORDER BY p90_seconds DESC
LIMIT 50;

-- Failed pipeline updates in the last 7 days, with pipeline names
WITH latest_pipelines AS (
  SELECT workspace_id, pipeline_id, name, pipeline_type
  FROM system.lakeflow.pipelines
  QUALIFY ROW_NUMBER() OVER (PARTITION BY workspace_id, pipeline_id ORDER BY
 ↪  change_time DESC) = 1
)
SELECT p.name, p.pipeline_type, u.pipeline_id, u.update_id, u.update_type,
       u.trigger_type, u.period_end_time
FROM system.lakeflow.pipeline_update_timeline u
LEFT JOIN latest_pipelines p USING (workspace_id, pipeline_id)
WHERE u.period_end_time >= current_timestamp() - INTERVAL 7 DAYS
  AND u.result_state = 'FAILED'
ORDER BY u.period_end_time DESC;

-- Jobs repaired or retried most often (more than one terminal row per run)
```

```sql
SELECT workspace_id, job_id, run_id, COUNT(*) - 1 AS retries_count
FROM system.lakeflow.job_run_timeline
WHERE result_state IS NOT NULL
GROUP BY ALL
HAVING retries_count > 0
ORDER BY retries_count DESC
LIMIT 10;
```

Query History: `system.query.history`

This table records statements on SQL warehouses, serverless notebooks and jobs, and Lakeflow pipelines. By default only admins can read it, and Databricks recommends sharing it through dynamic views.

Column
statement_id, session_id
execution_status
compute
executed_by, executed_as
statement_text, statement_type, error_message
total_duration_ms, execution_duration_ms, compilation_duration_ms, waiting_for_compute_duration
read_bytes, read_rows, produced_rows, read_files, pruned_files, spilled_local_bytes, from_result_
query_source
start_time, end_time, update_time

```sql
-- Slowest and failed statements on one warehouse in the last day
SELECT statement_id, executed_by, execution_status,
       total_duration_ms, waiting_at_capacity_duration_ms, produced_rows,
 ↪  error_message
FROM system.query.history
WHERE compute.warehouse_id = '<warehouse-id>'
  AND start_time >= current_timestamp() - INTERVAL 1 DAY
ORDER BY total_duration_ms DESC
LIMIT 50;
```

Which Table Answers the Question?

Scenario	Table(s)
Which jobs and warehouses cost the most last month?	billing.usage + billing.list_prices + lakefl
What did a pipeline cost?	billing.usage (usage_metadata.dlt_pipeline_
Who granted SELECT on a table, or who read it?	access.audit (unityCatalog / updatePermissior
Which downstream tables break if I change this table?	access.table_lineage / column_lineage
Which jobs fail most, or got slower this week?	lakeflow.job_run_timeline, lakeflow.job_tasl
Which pipeline updates failed, and what triggered them?	lakeflow.pipeline_update_timeline + lakeflo
Which jobs run on all-purpose clusters?	lakeflow.job_task_run_timeline.compute_ids
Is a cluster over-provisioned?	compute.node_timeline
When did a warehouse scale up, or why was it running?	compute.warehouse_events
Slow or failed SQL statements	query.history

Use Cases

- **Monthly chargeback:** attribute DBUs and list cost to teams with `custom_tags['team']`, and to workloads through `usage_metadata.job_id` / `warehouse_id` / `dlt_pipeline_id`. Publish the result as a governed view.
- **Security investigation:** trace `updatePermissions` events and denied (`response.status_code` 403) requests in `system.access.audit`, filtering on `event_date`.
- **Reliability review:** rank jobs by failure rate and p90 task duration from the lakeflow timelines, and flag jobs that run on all-purpose compute.
- **Change impact:** before deprecating a silver table, list its downstream targets from `system.access.table_lineage`.

Common Issues & Errors

Cost numbers are inflated after a join **Cause:** joining `list_prices` on `sku_name` only matches every historical price row.

Fix: also join on `cloud`, and require `usage_start_time >= price_start_time AND (usage_end_time <= price_end_time OR price_end_time IS NULL)`.

A job's cost is missing from `usage_metadata.job_id` **Cause:** the job ran on all-purpose compute, or it was a `WORKFLOW_RUN` launched from a notebook.

Fix: move the job to job compute or serverless. For shared compute, attribute approximately by `usage_metadata.cluster_id`.

Job counts or durations look wrong **Cause:** long runs are split into hourly rows, and `result_state` is only set on the last row.

Fix: count outcomes with `result_state IS NOT NULL`, and sum `period_end_time - period_start_time` per `run_id`.

Duplicate or deleted jobs appear in results **Cause:** `jobs`, `job_tasks`, and `pipelines` are SCD2 tables.

Fix: take `ROW_NUMBER() ... ORDER BY change_time DESC = 1` per entity key, then filter `delete_time IS NULL`.

Permission denied, or tables not visible **Cause:** by default only users who are both account admin and metastore admin can read system tables. The workspace may also not be UC-enabled.

Fix: grant `USE CATALOG` on `system`, plus `USE SCHEMA` and `SELECT` on the schema, and query from a UC-enabled workspace.

Workspaces are missing **Cause:** regional tables (compute, lakeflow, access, query) only show workspaces in the region you query from.

Fix: run the query from a workspace in each region. `billing.usage` is global.

Query fails with "returned too much data" **Cause:** the predicates are not selective enough.

Fix: add date filters (`usage_date`, `event_date`, `period_start_time`) and narrow the workspace or entity.

Exam Tips

Tip

- **Cost attribution** = `system.billing.usage` JOIN `system.billing.list_prices` (price window + `cloud` + `sku_name`, `pricing.default`) JOIN `lakeflow.jobs` / `compute.warehouses` / `lakeflow.pipelines` through `usage_metadata`. Never the Spark UI, driver logs, or a PDF export.
- **`usage_metadata.job_id`** exists only for jobs on job compute or serverless. **`warehouse_id`** is for SQL warehouses, **`dlt_pipeline_id`** for pipelines, and **`cluster_id`** for classic compute.
- **History tables** (SCD2 `jobs`, `job_tasks`, `pipelines`, the SCD `clusters` table, and `warehouses`, which adds a snapshot whenever properties change): take the latest `change_time` per key.
- **Timelines are sliced hourly.** `result_state` appears only on the final row.
- **Audit:** `service_name` + `action_name`, `user_identity.email`, `request_params['...']`, `response.status_code`, and filter on `event_date`. Account-level events have `workspace_id = 0`.
- **Lineage** is in `system.access`. `system.lineage` is deprecated and empty.
- **Access:** account admin + metastore admin by default. Otherwise grant `USE CATALOG`, `USE SCHEMA`, and `SELECT`. The tables are read-only.
- **Retention:** most tables keep 365 days, `node_timeline` 90 days, and `list_prices` indefinitely. Records are not real-time.
- **Serverless and SQL warehouses** are not in `compute.clusters` or `node_timeline`.

Key Takeaways

- **Four exam schemas:** `billing` (usage, list_prices), `compute` (clusters, node_timeline, warehouses, warehouse_events, ...), `access` (audit, table/column lineage), and `lakeflow` (jobs, job_tasks, job_run_timeline, job_task_run_timeline, pipelines, pipeline_update_timeline).
- **List cost** is `usage_quantity * pricing.default` from the price row whose validity window covers the usage. Sum across `RETRACTION` and `RESTATEMENT` rows, because corrections cancel out.
- **Attribution keys** are in `usage_metadata`. Identity attribution is `identity_metadata.run_as` (and `owned_by` for warehouses).
- **Lakeflow tables** give account-wide job and pipeline history for 365 days. They complement the Jobs UI and API, which keep run history for 60 days.
- **Access model:** UC-governed and read-only. Data is regional or global, typically delivered within an hour (or 12 hours for billing), and streamable with `skipChangeCommits`.

Related Topics

- Lakeflow Event Logs - Per-pipeline event log (expectations, flow progress)
- Lakehouse Alerts - Alerting on system-table queries
- Databricks REST API — Part 2 - Monitoring runs through the SDK, REST, and CLI
- Lakeflow Jobs — Part 2 - Jobs UI monitoring and notifications
- Cost Optimization - Acting on cost findings
- Query Profiler - Drilling into a statement from query history
- Audit, Lineage and Network Security - Audit and lineage for compliance

Official Documentation

- System tables reference
- Billable usage system table reference
- Pricing system table reference

- Monitor job costs and performance with system tables
- Monitor the cost of serverless compute
- Compute system tables reference
- SQL warehouses system table reference
- Warehouse events system table reference
- Audit log system table reference
- Audit log reference (services and actions)
- Lineage system tables reference
- Jobs system table reference (lakeflow schema)
- Query history system table reference
- October 2026 release notes

Lakeflow/Lakeflow Pipelines Event Logs

Lakeflow Pipelines (formerly DLT) / Lakeflow pipelines generate detailed event logs that provide observability into pipeline execution, data quality, and maintenance operations. Understanding these logs is essential for monitoring and debugging production pipelines.

Overview

```
flowchart TB
    Pipeline[Lakeflow Declarative Pipeline] --> EventLog[Event Log]

    EventLog --> Definition[flow_definition]
    EventLog --> Progress[flow_progress]
    EventLog --> Quality[data_quality]
    EventLog --> Maintenance[maintenance_actions]
    EventLog --> UserAction[user_action]

    Definition --> |Pipeline structure| Graph[Dataset dependencies]
    Progress --> |Execution metrics| Metrics[Rows, duration, status]
    Quality --> |Expectations| Expectations[Pass/fail counts]
```

Overview

- **Access**: query the event log with `event_log(TABLE(...))`, or by path for pipelines without a named target
- **Event types**: `flow_definition` (structure), `flow_progress` (metrics, expectation results, errors), and `maintenance_actions` (OPTIMIZE / VACUUM)
- **Extracting fields**: use `:` paths on the `details` column, for example `details:flow_progress.metrics.num_output_rows`
- **Health signals**: `seconds_of_backlog` shows streaming lag; filter `level` for `ERROR` and `WARN`

What the Exam Tests

- Use Pipeline event logs to monitor pipeline health and data quality.
- Find expectation pass and fail counts (inside `flow_progress` events, not a separate event type)
- Diagnose a failed update from the error fields

Event Log Location

Accessing Event Logs

```
-- Query event log using event_log() function
SELECT *
FROM event_log(TABLE(main.default.my_pipeline_target_table))
ORDER BY timestamp DESC;

-- For pipelines without target table, use pipeline storage location
SELECT *
FROM delta.`/pipelines/<pipeline-id>/system/events`
ORDER BY timestamp DESC;
```

Event Log Schema

Column	Type	Description
id	STRING	Unique event ID
timestamp	TIMESTAMP	Event timestamp
origin	STRUCT	Pipeline and cluster info
event_type	STRING	Type of event
level	STRING	INFO, WARN, ERROR
maturity_level	STRING	STABLE, EVOLVING
message	STRING	Human-readable message
details	STRING	JSON with event details
error	STRUCT	Error information if failed

Event Types

flow_definition Captures pipeline graph structure and dataset definitions.

```
SELECT
    timestamp,
    details:flow_definition.name AS dataset_name,
    details:flow_definition.schema AS schema_name,
    details:flow_definition.target_type AS target_type,
    details:flow_definition.flow_type AS flow_type
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_definition'
ORDER BY timestamp DESC;
```

flow_progress Tracks execution progress for each dataset update.

```
SELECT
    timestamp,
    details:flow_progress.data_quality.dataset AS dataset,
    details:flow_progress.metrics.num_output_rows AS rows_written,
    details:flow_progress.status AS status,
    details:flow_progress.metrics.seconds_of_processing AS processing_time
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_progress'
ORDER BY timestamp DESC;
```

Metrics in flow_progress

Metric	Description
num_output_rows	Rows written to target
num_upserted_rows	Rows upserted (AUTO CDC flows)
num_deleted_rows	Rows deleted
seconds_of_processing	Processing duration
seconds_of_backlog	Streaming backlog time

data_quality Captures expectation results for data quality checks.

```
SELECT
    timestamp,
    details:expectations.dataset AS dataset,
    details:expectations.name AS expectation_name,
    details:expectations.passed_records AS passed,
    details:expectations.failed_records AS failed
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_progress'
    AND details:expectations IS NOT NULL
ORDER BY timestamp DESC;
```

maintenance_actions Records OPTIMIZE, VACUUM, and other maintenance operations.

```
SELECT
    timestamp,
    details:maintenance_actions.dataset AS dataset,
    details:maintenance_actions.action AS action,
    details:maintenance_actions.files_removed AS files_removed,
    details:maintenance_actions.bytes_removed AS bytes_removed
FROM event_log(TABLE(my_table))
WHERE event_type = 'maintenance_actions'
ORDER BY timestamp DESC;
```

user_action Captures manual interventions like full refresh or stop.

```
SELECT
    timestamp,
    details:user_action.action AS action,
    details:user_action.user_id AS user_id
FROM event_log(TABLE(my_table))
WHERE event_type = 'user_action'
ORDER BY timestamp DESC;
```

Common Monitoring Queries

Pipeline Execution Summary

```
-- Overall pipeline run summary
SELECT
    DATE_TRUNC('hour', timestamp) AS hour,
    COUNT(CASE WHEN event_type = 'flow_progress' THEN 1 END) AS dataset_updates,
    SUM(CAST(details:flow_progress.metrics.num_output_rows AS BIGINT)) AS
     ↪  total_rows,
```

```
    SUM(CAST(details:flow_progress.metrics.seconds_of_processing AS DOUBLE)) AS
     ↪  total_processing_seconds
FROM event_log(TABLE(my_table))
WHERE timestamp >= current_timestamp() - INTERVAL 24 HOURS
    AND event_type = 'flow_progress'
GROUP BY DATE_TRUNC('hour', timestamp)
ORDER BY hour DESC;
```

Dataset Processing Metrics

```
-- Per-dataset metrics
SELECT
    details:flow_progress.data_quality.dataset AS dataset,
    COUNT(*) AS update_count,
    SUM(CAST(details:flow_progress.metrics.num_output_rows AS BIGINT)) AS
     ↪  total_rows,
    AVG(CAST(details:flow_progress.metrics.seconds_of_processing AS DOUBLE)) AS
     ↪  avg_processing_seconds,
    MAX(CAST(details:flow_progress.metrics.seconds_of_processing AS DOUBLE)) AS
     ↪  max_processing_seconds
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_progress'
    AND timestamp >= current_timestamp() - INTERVAL 7 DAYS
GROUP BY details:flow_progress.data_quality.dataset
ORDER BY total_rows DESC;
```

Data Quality Monitoring

```
-- Expectation pass/fail rates
SELECT
    details:expectations.dataset AS dataset,
    details:expectations.name AS expectation,
    SUM(CAST(details:expectations.passed_records AS BIGINT)) AS total_passed,
    SUM(CAST(details:expectations.failed_records AS BIGINT)) AS total_failed,
    ROUND(
        SUM(CAST(details:expectations.passed_records AS BIGINT)) * 100.0 /
        NULLIF(SUM(CAST(details:expectations.passed_records AS BIGINT)) +
               SUM(CAST(details:expectations.failed_records AS BIGINT)), 0),
        2
    ) AS pass_rate
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_progress'
    AND details:expectations IS NOT NULL
    AND timestamp >= current_timestamp() - INTERVAL 24 HOURS
GROUP BY details:expectations.dataset, details:expectations.name
ORDER BY pass_rate ASC;
```

Error Analysis

```
-- Failed updates and errors
SELECT
    timestamp,
    details:flow_progress.data_quality.dataset AS dataset,
    details:flow_progress.status AS status,
    error.message AS error_message,
    error.exception AS exception
```

```
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_progress'
    AND (details:flow_progress.status = 'FAILED' OR error IS NOT NULL)
ORDER BY timestamp DESC
LIMIT 50;
```

Streaming Lag Monitoring

```
-- Monitor streaming backlog
SELECT
    timestamp,
    details:flow_progress.data_quality.dataset AS dataset,
    CAST(details:flow_progress.metrics.seconds_of_backlog AS DOUBLE) AS
     ↪  backlog_seconds,
    CAST(details:flow_progress.metrics.num_output_rows AS BIGINT) AS
     ↪  rows_processed
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_progress'
    AND details:flow_progress.metrics.seconds_of_backlog IS NOT NULL
    AND timestamp >= current_timestamp() - INTERVAL 1 HOUR
ORDER BY timestamp DESC;
```

Building Monitoring Dashboards

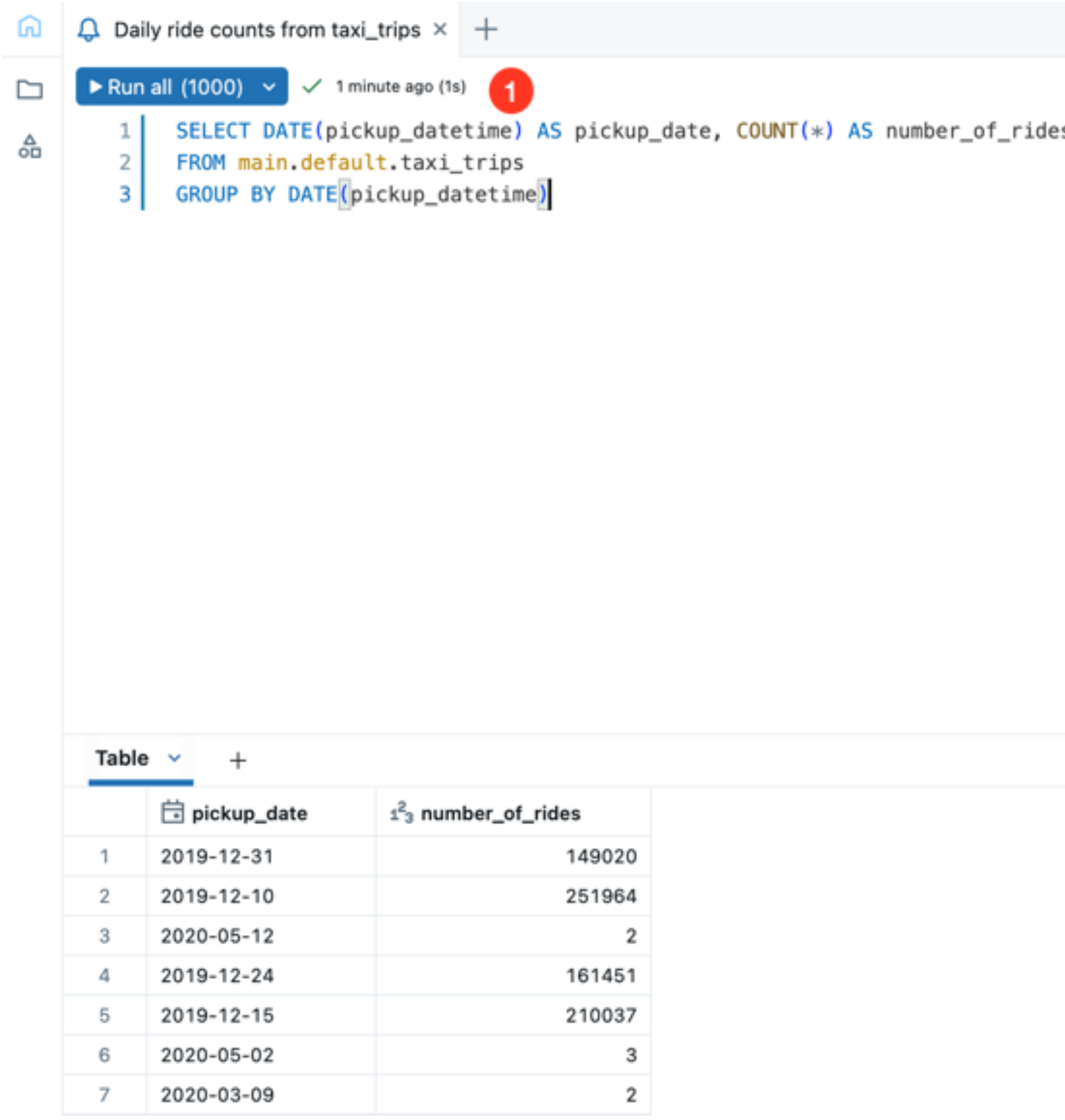
Pipeline Health Dashboard

```
-- Create materialized view for dashboard
CREATE OR REPLACE VIEW monitoring.pipeline_health AS
SELECT
    DATE_TRUNC('hour', timestamp) AS hour,
    origin.pipeline_id,
    COUNT(DISTINCT origin.update_id) AS pipeline_runs,
    SUM(CASE WHEN level = 'ERROR' THEN 1 ELSE 0 END) AS errors,
    SUM(CASE WHEN level = 'WARN' THEN 1 ELSE 0 END) AS warnings,
    SUM(CAST(details:flow_progress.metrics.num_output_rows AS BIGINT)) AS
     ↪  rows_processed
FROM event_log(TABLE(my_table))
WHERE timestamp >= current_timestamp() - INTERVAL 7 DAYS
GROUP BY DATE_TRUNC('hour', timestamp), origin.pipeline_id;
```

Data Quality Trends

```
-- Track quality over time
CREATE OR REPLACE VIEW monitoring.quality_trends AS
SELECT
    DATE(timestamp) AS date,
    details:expectations.dataset AS dataset,
    details:expectations.name AS expectation,
    SUM(CAST(details:expectations.passed_records AS BIGINT)) AS passed,
    SUM(CAST(details:expectations.failed_records AS BIGINT)) AS failed
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_progress'
    AND details:expectations IS NOT NULL
    AND timestamp >= current_timestamp() - INTERVAL 30 DAYS
GROUP BY DATE(timestamp), details:expectations.dataset,
 ↪  details:expectations.name;
```

Alerting Patterns



	pickup_date	number_of_rides
1	2019-12-31	149020
2	2019-12-10	251964
3	2020-05-12	2
4	2019-12-24	161451
5	2019-12-15	210037
6	2020-05-02	3
7	2020-03-09	2

SQL alert configuration showing threshold condition and notification channel settings.

Alert on Data Quality Failures

```python
# Databricks notebook scheduled job

from pyspark.sql.functions import col, sum as spark_sum

# Query recent quality failures

failures_df = spark.sql("""
    SELECT
        details:expectations.dataset AS dataset,
        details:expectations.name AS expectation,
        SUM(CAST(details:expectations.failed_records AS BIGINT)) AS failed_count
    FROM event_log(TABLE(my_table))
    WHERE event_type = 'flow_progress'
        AND details:expectations IS NOT NULL
        AND timestamp >= current_timestamp() - INTERVAL 1 HOUR
    GROUP BY details:expectations.dataset, details:expectations.name
    HAVING SUM(CAST(details:expectations.failed_records AS BIGINT)) > 0
""")

if failures_df.count() > 0:
    # Send alert (webhook, email, etc.)
    failures = failures_df.collect()
    alert_message = f"Data quality failures detected: {len(failures)}
 ↪  expectations failed"
    # send_alert(alert_message)
```

Alert on Processing Delays

```python
# Check for streaming lag

lag_df = spark.sql("""
    SELECT
        details:flow_progress.data_quality.dataset AS dataset,
        MAX(CAST(details:flow_progress.metrics.seconds_of_backlog AS DOUBLE)) AS
 ↪  max_backlog
    FROM event_log(TABLE(my_table))
    WHERE event_type = 'flow_progress'
        AND details:flow_progress.metrics.seconds_of_backlog IS NOT NULL
        AND timestamp >= current_timestamp() - INTERVAL 15 MINUTES
    GROUP BY details:flow_progress.data_quality.dataset
    HAVING MAX(CAST(details:flow_progress.metrics.seconds_of_backlog AS DOUBLE))
 ↪  > 300
""")

if lag_df.count() > 0:
    # Alert on significant backlog (> 5 minutes)
    pass
```

Event Log Retention

Configuring Retention

```
# DLT pipeline configuration

{
    "configuration": {
```

```
        "pipelines.eventLog.retentionDays": "30"
    }
}
```

Manual Cleanup

```
-- Event logs are Delta tables, can use VACUUM
VACUUM delta.`/pipelines/<pipeline-id>/system/events` RETAIN 168 HOURS;
```

Troubleshooting Common Issues

Pipeline Stuck or Slow

```
-- Check recent processing times
SELECT
    timestamp,
    details:flow_progress.data_quality.dataset AS dataset,
    CAST(details:flow_progress.metrics.seconds_of_processing AS DOUBLE) AS
     ↪  processing_time,
    details:flow_progress.status AS status
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_progress'
    AND timestamp >= current_timestamp() - INTERVAL 1 HOUR
ORDER BY processing_time DESC
LIMIT 20;

-- Check for pending datasets
SELECT
    timestamp,
    details:flow_progress.data_quality.dataset AS dataset,
    details:flow_progress.status AS status
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_progress'
    AND details:flow_progress.status IN ('PENDING', 'RUNNING')
ORDER BY timestamp DESC;
```

Data Quality Investigation

```
-- Find records that failed expectations
SELECT
    timestamp,
    details:expectations.dataset AS dataset,
    details:expectations.name AS expectation,
    details:expectations.failed_records AS failed_count
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_progress'
    AND details:expectations IS NOT NULL
    AND CAST(details:expectations.failed_records AS BIGINT) > 0
ORDER BY timestamp DESC;
```

Pipeline Dependency Analysis

```
-- View pipeline graph
SELECT
    details:flow_definition.name AS dataset_name,
    details:flow_definition.target_type AS target_type,
```

```
    details:flow_definition.schema AS schema
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_definition'
ORDER BY details:flow_definition.name;
```

Integration with External Monitoring

Export to External Systems

```
# Export event logs to external monitoring

event_log_df = spark.sql("""
    SELECT
        timestamp,
        event_type,
        level,
        message,
        details
    FROM event_log(TABLE(my_table))
    WHERE timestamp >= current_timestamp() - INTERVAL 1 HOUR
""")

# Write to external sink

(event_log_df.write
    .format("json")
    .mode("append")
    .save("/Volumes/monitoring/dlt_events/"))

# Or stream to Kafka/Event Hub for real-time monitoring
```

Metrics to External Dashboard

```
-- Prepare metrics for Grafana/Datadog
SELECT
    timestamp,
    'dlt_rows_processed' AS metric_name,
    origin.pipeline_id AS pipeline,
    details:flow_progress.data_quality.dataset AS dataset,
    CAST(details:flow_progress.metrics.num_output_rows AS DOUBLE) AS value
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_progress'
    AND timestamp >= current_timestamp() - INTERVAL 5 MINUTES;
```

Use Cases

- **Data Quality Alerting**: Querying the `data_quality` events in the log to trigger a PagerDuty or Slack alert if the expectation pass rate for a critical Gold dataset drops below a required 99% threshold.
- **Pipeline SLA Monitoring**: Tracking `flow_progress` events to measure the `seconds_of_processing` trends over time, ensuring the overall pipeline consistently meets its required delivery SLAs.
- **Streaming Backlog Analysis**: Using the `seconds_of_backlog` metric to monitor streaming ingestion health and proactively scale up the underlying compute cluster when event consumption falls too far behind production.

Common Issues & Errors

Event Log Not Found **Scenario:** Can't query event log.

Fix: Verify pipeline has run and target table exists:

```
-- Check pipeline storage location directly
SELECT * FROM delta.`/pipelines/<pipeline-id>/system/events` LIMIT 10;
```

Missing Expectation Results **Scenario:** No expectation data in logs.

Fix: Verify expectations are defined in pipeline:

```
@dlt.expect("valid_id", "id IS NOT NULL")  # Must be defined
def my_table():
    return ...
```

Stale Metrics **Scenario:** Metrics not updating.

Fix: Check pipeline is running and not paused:

```
SELECT
    MAX(timestamp) AS last_event
FROM event_log(TABLE(my_table))
WHERE event_type = 'flow_progress';
```

Exam Tips

1. **Event log function** - Use `event_log(TABLE(target_table))` to query
2. **Event types** - flow_definition, flow_progress, user_action, maintenance_actions
3. **flow_progress metrics** - num_output_rows, seconds_of_processing, seconds_of_backlog
4. **Expectations in logs** - Pass/fail counts in flow_progress events
5. **JSON details** - Use `:` notation to extract from details column
6. **Streaming backlog** - seconds_of_backlog indicates lag
7. **Error tracking** - Check error struct for failure details
8. **Retention** - Configure with pipelines.eventLog.retentionDays
9. **Storage location** - `/pipelines/<id>/system/events` Delta table
10. **Level column** - INFO, WARN, ERROR for filtering

Key Takeaways

- **Event log access**: Query the event log using `event_log(TABLE(target_table))` or directly via `delta.\/pipelines//system/events“` for pipelines without a named target.
- **Key event types**: `flow_definition` captures pipeline structure; `flow_progress` captures per-dataset execution metrics and expectation results; `maintenance_actions` records OPTIMIZE/VACUUM runs.
- **JSON extraction syntax**: Use the colon (`:`) notation to extract fields from the `details` STRING column (e.g., `details:flow_progress.metrics.num_output_rows`).
- **Streaming backlog metric**: `seconds_of_backlog` in `flow_progress` events indicates how far behind a streaming dataset is; a growing value signals consumer lag.
- **Expectation results location**: Pass/fail counts for data quality constraints appear inside `flow_progress` events when `details:expectations` is not NULL — not in a separate event type.
- **Error diagnosis**: Check the `error.message` and `error.exception` fields within `flow_progress` events to diagnose failed dataset updates.
- **Retention configuration**: Set event log retention with the pipeline property `pipelines.eventLog.reten` the event log is a Delta table and can be vacuumed with `VACUUM`.

- **Level column filtering**: Use `level = 'ERROR'` or `level = 'WARN'` to quickly filter the event log for operational issues.

Related Topics

- Lakeflow Pipelines - Lakeflow Pipelines basics
- Expectations - Data quality
- System Tables - Account-level monitoring

Official Documentation

- Lakeflow Pipelines Event Logs
- Monitoring Lakeflow Pipelines
- Event Log Schema

Lakehouse Alerts

Overview

A Databricks alert runs a SQL query on a schedule, checks a condition against the result, and notifies users or notification destinations when the condition is met. Because the alert is just a query, it can watch anything you can express in SQL: governed metric views, data quality results, billing, query and warehouse history, audit logs, agent traces, or a gate inside a Lakeflow Job.

Overview

- An alert has a query (owned by the alert), a SQL warehouse, a **condition** (column or aggregation, operator, static or column threshold), a **schedule** (Quartz cron), and **notifications** (users or destinations, optional repeat, optional custom template).
- Evaluations resolve to `OK`, `TRIGGERED`, or `ERROR`. Legacy alerts also had `UNKNOWN`.
- The signals the exam lists map to sources: metric views, `system.data_quality_monitoring.table_results` and profile metric tables, `system.billing.*`, `system.query.history` and `system.compute.warehouse_events`, `system.access.audit`, MLflow production monitoring on agent traces, and the **SQL alert task** inside Lakeflow Jobs.
- A **SQL alert task** evaluates an alert as a job step. An **If/else condition** task can then branch on `{{tasks.<task>.output.alert_state}}`.

What the Exam Tests

- ”Use Databricks Lakehouse alerts to monitor governed business metrics, data quality and anomaly signals, usage and cost, SQL warehouse/query health, audit and security events, AI agent quality, and Lakeflow Job branching.”
- Picking the right source table or feature for each signal type, and writing the alert query so a simple condition (for example `> 0`) fires.
- Choosing between a scheduled alert, an account budget, a Data Quality Monitoring alert rule, and a SQL alert task in a job.
- Wiring an alert into a job so downstream tasks run only when the alert is (or is not) `TRIGGERED`.

Platform update (Oct 2026)

The exam calls this feature "Databricks Lakehouse alerts". The docs call it **Databricks SQL alerts**. The earlier version is documented as **legacy alerts**, and both can be used side by side. Since July 9, 2026, Databricks SQL alerts are also available by default in workspaces with the compliance security profile enabled. Sources: Databricks SQL alerts, July 2026 release notes

---

How an Alert Works

```
flowchart LR
    Sched["Schedule<br/>(Quartz cron)"] --> Run["Run alert query<br/>on SQL
 ↪  warehouse"]
    Job["SQL alert task<br/>in a Lakeflow Job"] --> Run
    Run --> Eval{"Condition<br/>vs threshold"}
    Eval -->|met| Trig[TRIGGERED]
    Eval -->|not met| Ok[OK]
    Eval -->|query or warehouse error| Err[ERROR]
    Trig --> Notify["Notify users and<br/>destinations"]
    Ok -.->|"if Notify on OK"| Notify
    Notify --> Dest["Email, Slack, Teams,<br/>PagerDuty, webhook"]
```

Alert editor components

Component	What you set
Query editor	The SQL the alert evaluates. Each alert owns its query; an existing saved query cannot be re
Compute	The SQL warehouse that runs the alert query
Schedule	A periodic schedule, optionally edited as Quartz cron syntax
Share	Permissions (`CAN RUN`, `CAN MANAGE`)
Condition	The value to check, the operator, and the threshold. **Test condition** previews the result.
Notifications	Users or notification destinations, plus an optional notification frequency that repeats until th
Advanced	**Notify on OK**, **Empty result state**, and the notification **Template**

Important

Alerts do not support queries with parameters. If the logic needs runtime values, compute them inside the query (for example `current_date() - 1`).

Condition options

- **Value:** the first value of a column in the result, or an aggregation over all rows of one column. The Alerts API defines `SUM`, `AVG`, `COUNT`, `COUNT_DISTINCT`, `MIN`, `MAX`, `MEDIAN`, and `STDDEV`.
- **Operator:** `EQUAL`, `NOT_EQUAL`, `GREATER_THAN`, `GREATER_THAN_OR_EQUAL`, `LESS_THAN`, `LESS_THAN_OR_EQUAL`, `IS_NULL`, `IS_NOT_NULL`.
- **Threshold:** a **static value** (default) or **another column** of the result.
- **Aggregations wrap your SQL:** a `SUM` condition on `SELECT 1 AS c` runs `WITH q AS (SELECT 1 AS c) SELECT SUM(c) FROM q`. In that case `QUERY_RESULT_ROWS` and `QUERY_RESULT_COLS` show only the aggregated result.
- **Multi-column logic:** return a single flag from the query and alert on it.

```sql
-- Fire only when BOTH conditions hold: alert when the value = 1
SELECT CASE WHEN drafts_count > 10000 AND archived_count > 5000 THEN 1 ELSE 0 END
 ↪  AS breach
```

```
FROM (
  SELECT
    SUM(CASE WHEN is_archived THEN 1 ELSE 0 END) AS archived_count,
    SUM(CASE WHEN is_draft THEN 1 ELSE 0 END)    AS drafts_count
  FROM queries
) data;
```

Notifications and templates

- **Destinations:** workspace admins create them (Email, Slack, Webhook, Microsoft Teams, PagerDuty). After that, every user can pick them. Webhook endpoints must use HTTPS.
- **Template variables (standard editor):** `{{ALERT_NAME}}`, `{{ALERT_STATUS}}`, `{{ALERT_CONDITION}}`, `{{ALERT_THRESHOLD}}`, `{{ALERT_COLUMN}}`, `{{ALERT_URL}}`, `{{QUERY_RESULT_VALUE}}`, `{{QUERY_RESULT_ROWS}}`, `{{QUERY_RESULT_COLS}}`, `{{QUERY_RESULT_TABLE}}`. `QUERY_RESULT_TABLE` is limited to the first 100 rows, and only email destinations render HTML.
- **Markdown editor:** references variables as `@ALERT_NAME` and supports a subset of those variables.
- **Webhook payload:** after a state change the alert sends a JSON body with `"event": "alert_state_change"` and the alert's definition and evaluation details.

```
Subject: Alert "{{ALERT_NAME}}" changed status to {{ALERT_STATUS}}
Body:    {{ALERT_COLUMN}} {{ALERT_CONDITION}} {{ALERT_THRESHOLD}}
         Current value: {{QUERY_RESULT_VALUE}}
         Open: {{ALERT_URL}}
```

Operational limits worth knowing

- Each workspace has a default limit of **250 simultaneous active alert runs**. When every slot is busy, a scheduled run is **skipped**, and the alert does not evaluate. Stagger schedules (01:30, 01:32, 01:34 instead of all at 01:30).
- **Run now** on the alert page runs the query immediately and shows the result next to the evaluation history.
- You can track alerts in Databricks Git folders. Newly cloned alerts start with their **schedules paused**.

Signal 1: Governed Business Metrics (Metric Views)

Alerts on Unity Catalog metric views work the same as alerts on any other dataset. Reference the metric view by its **fully qualified name**, and wrap each measure in `MEASURE()`.

```
-- Alert: TRIGGERED when yesterday's governed revenue is below plan
SELECT MEASURE(`Total Revenue`) AS total_revenue
FROM main.finance.orders_metric_view
WHERE `Order Date` = current_date() - 1;
-- Condition: total_revenue  LESS_THAN  static value 250000
```

Why use a metric view rather than raw tables: the measure's definition (filters, joins, aggregation) is governed in Unity Catalog in one place. The alert therefore computes ”Total Revenue” the same way as every dashboard or query that reads the same metric view.

Signal 2: Data Quality and Anomaly Signals

You can get data quality alerts in three ways:

Need	Feature
”Tell me when any table in schema X goes stale or is missing rows”	**Anomaly detection** (Public Preview) wi
Custom threshold on anomaly results	SQL alert on `system.data_quality_mon`
Statistic out of range or drift (null fraction, distribution shift)	**Profile alerts**: a SQL alert on `{output_`

Anomaly detection checks **freshness** (is a commit unusually late?) and **completeness** (did the last 24 hours write fewer rows than predicted?). It stores results in `system.data_quality_monitoring.table_r` By default only account admins can read that table.

```
-- Alert on anomaly detection: count tables currently Unhealthy in main.sales
-- Condition: unhealthy_tables GREATER_THAN 0
WITH latest AS (
  SELECT *,
         ROW_NUMBER() OVER (PARTITION BY table_id ORDER BY event_time DESC) AS rn
  FROM system.data_quality_monitoring.table_results
  WHERE catalog_name = 'main' AND schema_name = 'sales'
)
SELECT COUNT(*) AS unhealthy_tables
FROM latest
WHERE rn = 1 AND status = 'Unhealthy';
```

```
-- Profile alert on a profiled table "orders" whose metrics go to main.monitoring
-- Latest null percentage of customer_id; Condition: pct_null GREATER_THAN <your
 ↪  threshold>
SELECT percent_null AS pct_null
FROM main.monitoring.orders_profile_metrics
WHERE column_name = 'customer_id'
  AND slice_key IS NULL          -- whole table, not a slice
  AND log_type = 'INPUT'
ORDER BY window.start DESC
LIMIT 1;
```

Tip

To catch failures in a **pipeline's own expectations**, query the Lakeflow pipeline event log (`flow_progress` events). See Lakeflow Event Logs. You can put an alert on that query too.

Signal 3: Usage and Cost

Cost alerts are plain SQL over `system.billing.usage` joined to `system.billing.list_prices`. The query returns a row only when spend is over budget, so the condition can be ”row count > 0” (or the value of `list_cost`).

```
-- Alert when any workspace's serverless jobs/notebooks list cost in the last 30
 ↪  days exceeds 5,000
SELECT
  u.workspace_id,
  SUM(u.usage_quantity * p.pricing.default) AS list_cost
FROM system.billing.usage u
JOIN system.billing.list_prices p
  ON  u.cloud = p.cloud
  AND u.sku_name = p.sku_name
  AND u.usage_start_time >= p.price_start_time
```

```
  AND (u.usage_end_time <= p.price_end_time OR p.price_end_time IS NULL)
WHERE u.sku_name LIKE '%SERVERLESS%'
  AND u.billing_origin_product IN ('JOBS', 'INTERACTIVE')
  AND u.usage_date >= current_date() - INTERVAL 30 DAYS
GROUP BY u.workspace_id
HAVING list_cost > 5000;
```

Swap the `GROUP BY` for `identity_metadata.run_as` (per user) or `usage_metadata.job_id` (per job). For Lakeflow Connect ingestion, filter `billing_origin_product = 'LAKEFLOW_CONNECT'`.

SQL alert vs budget

	SQL alert on `system.billing`	Account **budget**
Who creates it	Anyone with access to the system tables	Account admins (or w
Scope	Any SQL you can write (per job, per user, per tag, trend vs last week)	Workspaces, resource
Thresholds	One condition per alert	Up to **four** monthly U
Delivery	Any notification destination	Email addresses
Latency	`system.billing.usage` records typically arrive within 12 hours	Up to 24 hours betwe
Money basis	List price from `list_prices`	List price of the SKU;

Signal 4: SQL Warehouse and Query Health

```
-- Alert: queries on a warehouse that failed or ran > 10 minutes in the last hour
-- Condition: bad_queries GREATER_THAN 0
SELECT COUNT(*) AS bad_queries
FROM system.query.history
WHERE compute.warehouse_id = '<warehouse-id>'
  AND start_time >= current_timestamp() - INTERVAL 1 HOUR
  AND (execution_status = 'FAILED' OR total_duration_ms > 600000);

-- Alert: warehouses scaled up to 2+ clusters and still upscaled (hours since
 ↪  scale-up)
-- Condition: MAX(upscaled_hours) GREATER_THAN 4
SELECT
  we.warehouse_id,
  TIMESTAMPDIFF(MINUTE, we.event_time, current_timestamp()) / 60.0 AS
 ↪  upscaled_hours
FROM system.compute.warehouse_events we
WHERE we.event_type = 'SCALED_UP'
  AND we.cluster_count >= 2
  AND NOT EXISTS (
    SELECT 1 FROM system.compute.warehouse_events we2
    WHERE we2.warehouse_id = we.warehouse_id
      AND we2.event_time > we.event_time
      AND (we2.event_type = 'SCALED_DOWN'
           OR (we2.event_type = 'SCALED_UP' AND we2.cluster_count < 2))
  );
```

Useful `system.query.history` columns for health alerts: `execution_status` (`FINISHED`, `FAILED`, `CANCELED`), `total_duration_ms`, `waiting_at_capacity_duration_ms` (time queued for available compute capacity), `waiting_for_compute_duration_ms` (time waiting for compute to be provisioned), `spilled_local_bytes`, and `compute.warehouse_id`. Warehouse event types are `STARTING`, `RUNNING`, `SCALED_UP`, `SCALED_DOWN`, `STOPPING`, and `STOPPED`.

Signal 5: Audit and Security Events

```sql
-- Alert: Unity Catalog permission changes in the last hour
-- Condition: row count GREATER_THAN 0 (or COUNT(*) as the value)
SELECT
  event_time,
  user_identity.email                    AS changed_by,
  request_params['securable_full_name']  AS securable,
  request_params['changes']              AS changes
FROM system.access.audit
WHERE event_date >= current_date() - 1          -- partition filter
  AND event_time >= current_timestamp() - INTERVAL 1 HOUR
  AND service_name = 'unityCatalog'
  AND action_name = 'updatePermissions';

-- Alert: spike of denied requests (HTTP 403) per user in the last hour
-- Condition: MAX(denied) GREATER_THAN 50
SELECT user_identity.email AS user_email, COUNT(*) AS denied
FROM system.access.audit
WHERE event_date >= current_date() - 1
  AND event_time >= current_timestamp() - INTERVAL 1 HOUR
  AND response.status_code = 403
GROUP BY user_identity.email;
```

The docs' Genie Agents monitoring page follows the same pattern: query `system.access.audit` with `service_name = 'aibiGenie'`, for example grouped by `response.status_code`, and attach an alert.

Signal 6: AI Agent Quality

There is no "agent quality" system table. The documented approach has two parts:

1. **Production monitoring (Beta)** runs MLflow 3 scorers continuously on a sample of an agent's traces. You register a scorer and start it with a sampling rate, and the results are attached to each trace as feedback.
2. **Traces stored in Unity Catalog** land in Delta tables (`<prefix>_otel_spans`, `<prefix>_otel_annotations`, `<prefix>_otel_logs`, `<prefix>_otel_metrics`). Because they are SQL-native, they can "drive alerts and Lakeflow pipelines". The alerts docs list "catch failures in agents" as a use case that pairs with production monitoring.

```python
from mlflow.genai.scorers import Safety, ScorerSamplingConfig

# Score 100 % of production traces for safety; results attach to traces as
 ↪  feedback
safety = Safety().register(name="safety")
safety = safety.start(sampling_config=ScorerSamplingConfig(sample_rate=1.0))
```

Exam framing: to alert on agent quality, first get the quality signal (scorers through production monitoring), then put a SQL alert on the query over the stored results. Monitoring UC-stored traces needs a SQL warehouse ID set on the experiment with `set_databricks_monitoring_sql_warehouse_id()`.

Signal 7: Lakeflow Job Branching

A **SQL alert task** evaluates an existing Databricks SQL alert as a job step. The task runs the alert query on the chosen warehouse (or the alert's own warehouse), evaluates the condition, and notifies the task's subscribers (or the alert's own).

Key behaviors:

- **Task status is Succeeded whether or not the alert triggered.** The task reports **Failed** only when evaluation errors (query error, warehouse problem). A triggered alert therefore does **not** stop the job by itself. You branch on it.
- **Branch with an If/else condition task** on the dynamic value `{{tasks.<alert_task_key>.output.aler` whose value is `UNKNOWN`, `OK`, or `TRIGGERED`.
- **The task ignores the alert's own schedule.** That schedule keeps running independently.
- **Requirements:** at least `CAN RUN` on the alert, and a **serverless or pro** SQL warehouse.
- **Limitations:** no parameters (use a SQL task with a query if you need them), and only Databricks SQL alerts. Legacy alerts are not supported.

```
flowchart LR
    A[refresh_silver<br/>pipeline task] --> B[check_nulls<br/>SQL alert task]
    B --> C{"is_breached<br/>If/else:<br/>alert_state == TRIGGERED"}
    C -->|true| D[quarantine_and_page]
    C -->|false| E[publish_gold]
```

```yaml
resources:
  jobs:
    silver_quality_gate:
      name: silver_quality_gate
      tasks:
        - task_key: refresh_silver
          pipeline_task:
            pipeline_id: <pipeline-id>

        - task_key: check_nulls
          depends_on:
            - task_key: refresh_silver
          alert_task:                       # SQL alert task (Alert v2), CLI 0.296.0+
            alert_id: <alert-id>              # or workspace_path:
↪  ./nulls.dbalert.json
            warehouse_id: <warehouse-id>    # serverless or pro

        - task_key: is_breached
          depends_on:
            - task_key: check_nulls
          condition_task:
            op: EQUAL_TO
            left: "{{tasks.check_nulls.output.alert_state}}"
            right: TRIGGERED

        - task_key: quarantine_and_page
          depends_on:
            - task_key: is_breached
              outcome: "true"
          notebook_task:
            notebook_path: ../src/quarantine.py

        - task_key: publish_gold
          depends_on:
```

```
      - task_key: is_breached
        outcome: "false"
    notebook_task:
      notebook_path: ../src/publish_gold.py
```

The **SQL task** type also has an **Alert** option (`sql_task.alert.alert_id`, with `pause_subscriptions` and `subscriptions`). The Databricks SQL alerts docs point to the dedicated **SQL Alert** task type (`alert_task`) for running an alert as a job step.

Branch on...	Use
Whether an alert condition is met	SQL alert task → If/else on `output.alert_state`
A value a query returns	SQL task (query) → If/else on `{{tasks.<t>.output.first_row.<col>}}`
A value a notebook computed	Notebook task sets a task value → If/else on `{{tasks.<t>.values.<key>}`
Whether an upstream task failed	`run_if: AT_LEAST_ONE_FAILED` (no condition task needed)

Managing Alerts Programmatically

- **REST (Alerts API, latest alerts):** `GET /api/2.0/alerts`, `GET /api/2.0/alerts/{id}`, `POST /api/2.0/alerts`, `PATCH /api/2.0/alerts/{id}`, `DELETE /api/2.0/alerts/{id}` (moves the alert to trash).
- **Python SDK:** `w.alerts_v2.list_alerts()`, `get_alert(id)`, `create_alert(alert)`, `update_alert(id, alert, update_mask)`, `trash_alert(id)`.
- **System tables (Public Preview):** `system.alert.alerts` (SCD history of each alert's configuration) and `system.alert.alert_evaluation_history` (one row per evaluation: `evaluated_state`, `error_code`, `error_message`, `notified_subscriptions`, `failed_subscriptions`, and `job_id`/`task_key` when the alert ran inside a job).

```sql
-- Which alerts fired most in the last 7 days (alert on alerts / alert hygiene)
SELECT alert_id, COUNT(*) AS trigger_count_7d
FROM system.alert.alert_evaluation_history
WHERE start_time >= current_date() - INTERVAL 7 DAY
  AND evaluated_state = 'TRIGGERED'
GROUP BY alert_id
ORDER BY trigger_count_7d DESC;
```

Use Cases

- **Revenue guardrail:** a daily alert on a governed metric view pages finance when yesterday's `Total Revenue` falls below plan. Every consumer uses the same measure definition.
- **Schema-wide freshness:** a Data Quality Monitoring alert rule on `main.sales` posts to Slack when any table is marked stale or incomplete, without writing SQL.
- **Runaway serverless spend:** an hourly alert on `system.billing.usage` joined with `list_prices` returns rows only for workspaces above budget, and notifies a PagerDuty destination.
- **Quality gate in a pipeline job:** after the silver refresh, a SQL alert task checks null rates. An If/else task routes to a quarantine notebook when the alert is `TRIGGERED`, and to the gold publish otherwise.

Common Issues & Errors

Alert never triggers inside a job, but the job "succeeds" **Cause:** the SQL alert task reports **Succeeded** whenever the evaluation completes, even when the state is `TRIGGERED`.

Fix: add an If/else condition task on {{tasks.<alert_task>.output.alert_state}} and make downstream tasks depend on its true/false outcome.

Alert runs are skipped at a busy time **Cause:** the workspace hit its limit of 250 simultaneous active alert runs because many alerts share the same schedule.

Fix: stagger schedules. If the limit is consistently reached, request a quota increase from Databricks Support.

Alert shows ERROR or the task fails **Cause:** the query fails, the warehouse is unavailable, or the job uses a classic warehouse (SQL alert tasks need serverless or pro).

Fix: use **Test condition** or **Run now**, fix the query, and pick a serverless or pro warehouse.

Query uses parameters **Cause:** alerts and SQL alert tasks don't support parameterized queries.

Fix: hard-code or compute the values in SQL, or use a SQL task with a parameterized query.

Cost alert fires on "yesterday" data that is still changing **Cause:** billing records typically arrive within 12 hours, and corrections add RETRACTION and RESTATEMENT rows.

Fix: aggregate with SUM(usage_quantity) (retractions cancel originals) and evaluate a closed window, such as the last full day.

Exam Tips

Tip

- **One query, one condition:** put the business logic in SQL and leave the alert condition trivial (> 0, = 1, < threshold).
- **States:** latest alerts use OK / TRIGGERED / ERROR. UNKNOWN belongs to legacy alerts, and it also appears in the alert_state dynamic value.
- **Job branching:** SQL alert task + If/else on {{tasks.<t>.output.alert_state}}. Remember the task is **Succeeded** even when triggered.
- **Metric views:** reference by fully qualified name and use MEASURE(). Nothing else is special.
- **Schema-wide data quality without SQL:** Data Quality Monitoring alert rules. **Statistic or drift thresholds:** profile alerts on _profile_metrics / _drift_metrics.
- **Cost:** system.billing.usage × system.billing.list_prices (pricing.default). **Monthly account-level guardrail with email:** budgets.
- **Warehouse and query health:** system.query.history (execution_status, total_duration_ms, compute.warehouse_id) and system.compute.warehouse_events.
- **Security:** system.access.audit filtered on service_name / action_name. Always filter event_date.
- **Agents:** production monitoring scorers → feedback on traces → alert on the stored results.

Key Takeaways

- **Databricks SQL alerts** (the exam's "Lakehouse alerts") schedule a query, evaluate a condition, and notify users or destinations. Each alert owns its query, and parameters aren't supported.
- **Conditions** use a column value or an aggregation (SUM, AVG, COUNT, ...), an operator, and a static or column threshold. Aggregations wrap the query in a CTE.

- **Notifications** go to users or admin-managed destinations (Email, Slack, Teams, PagerDuty, webhook). They support **Notify on OK**, repeat frequency, and custom templates with `{{ALERT_*}}` / `{{QUERY_RESULT_*}}` variables.
- **Signal sources:** metric views, `system.data_quality_monitoring.table_results` and profile metric tables, `system.billing.*`, `system.query.history` and `system.compute.warehouse_events`, `system.access.audit`, and MLflow production monitoring results.
- **Job branching** uses a SQL alert task (`alert_task`) followed by an If/else condition task (`condition_task`) whose dependents use `outcome: "true"` / `"false"`.
- **Alert operations** can be audited through `system.alert.alerts` and `system.alert.alert_evaluation_` (Public Preview).

Related Topics

- System Tables - Schemas and columns used by alert queries
- Lakeflow Event Logs - Pipeline health and expectation metrics to alert on
- Databricks REST API — Part 2 - SDK, REST, and CLI monitoring of jobs and pipelines
- Lakeflow Jobs — Part 1 - Task values and If/else tasks
- Lakeflow Jobs — Part 2 - Job notifications and health rules
- Expectations and Data Quality - Pipeline expectations
- Cost Optimization - Acting on cost signals
- Audit, Lineage and Network Security - Audit log events

Official Documentation

- Databricks SQL alerts
- Create an alert
- Alert query patterns
- Manage alerts
- SQL alert webhook payload
- What are legacy alerts?
- SQL alert task for jobs
- Add branching logic to a job with the If/else task
- What is a dynamic value reference?
- Job task types in bundles (alert_task, condition_task, sql_task)
- Query metric views
- Anomaly detection
- Alerts for anomaly detection
- Data quality monitoring results system table
- Profile alerts
- Profile metric tables
- Monitor the cost of serverless compute
- Create and monitor budgets
- Example queries for monitoring SQL warehouse activity
- Warehouse events system table
- Query history system table
- Audit log system table
- Monitor Genie Agents usage with audit logs and alerts
- Set up production monitoring
- Store OpenTelemetry traces in Unity Catalog
- Alert system tables
- Manage notification destinations
- Access control lists (Alerts ACLs)

Databricks CLI — Part 1

The Databricks CLI provides command-line access to Databricks workspaces for automation, CI/CD pipelines, and administrative tasks.

Overview

```
flowchart TB
    CLI[Databricks CLI] --> Auth[Authentication]
    CLI --> Commands[Command Groups]

    Auth --> Token[Personal Access Token]
    Auth --> OAuth[OAuth U2M/M2M]
    Auth --> Profile[Config Profiles]

    Commands --> Workspace[workspace]
    Commands --> Jobs[jobs]
    Commands --> Clusters[clusters]
    Commands --> DBFS[fs]
    Commands --> Secrets[secrets]
    Commands --> Bundle[bundle]
```

Overview

- **Installation and authentication**: personal access tokens, OAuth M2M, and profiles in `~/.databrickscfg`
- **Authentication precedence**: environment variables, then `--profile`, then the default profile
- **Workspace and file commands**: import and export notebooks; `fs` commands with `dbfs:/` paths
- **Jobs commands** to list, create, run, and manage job runs

What the Exam Tests

- Use Databricks REST APIs / CLI and SDK for monitoring and analyzing jobs and pipelines.
- Create and automate ETL workloads using Lakeflow Jobs via the UI, API, and CLI.
- Script CLI calls for automation and CI/CD

Installation

Install Methods

```
# macOS with Homebrew

brew tap databricks/tap
brew install databricks

# Linux/macOS with curl

curl -fsSL https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh
 ↪  | sh

# Windows with winget

winget install Databricks.DatabricksCLI
```

```
# Python pip (legacy CLI v0.x - not recommended)

pip install databricks-cli
```

Verify Installation

```
# Check version

databricks --version

# Show help

databricks --help
```

CLI Versions

Version	Status	Features
CLI v0.x	Legacy	Basic commands, PAT only
CLI v2.x	Current	OAuth, Bundles, enhanced commands

Authentication

Authentication Methods

```
flowchart LR
    Auth[Authentication] --> PAT[Personal Access Token]
    Auth --> OAuth[OAuth]
    Auth --> SP[Service Principal]

    PAT --> Config["~/.databrickscfg"]
    OAuth --> Browser["Browser flow"]
    SP --> Secret["Client secret"]
```

Personal Access Token (PAT)

```
# Configure with PAT

databricks configure --token

# Enter workspace URL and token when prompted
# Databricks Host: https://adb-1234567890.12.azuredatabricks.net
# Personal Access Token: dapi1234567890abcdef
```

Configuration File The CLI stores credentials in `~/.databrickscfg`:

```
# ~/.databrickscfg

[DEFAULT]
host = https://adb-1234567890.12.azuredatabricks.net
token = dapi1234567890abcdef

[production]
host = https://adb-0987654321.21.azuredatabricks.net
```

```
token = dapi0987654321fedcba

[development]
host = https://adb-1111111111.11.azuredatabricks.net
token = dapi1111111111aaaaaa
```

Using Profiles

```
# Use specific profile

databricks workspace list --profile production

# Set default profile via environment variable

export DATABRICKS_CONFIG_PROFILE=production
databricks workspace list

# Override with environment variables

export DATABRICKS_HOST=https://adb-xxx.azuredatabricks.net
export DATABRICKS_TOKEN=dapi123456789
```

OAuth Authentication (M2M) For service principals and automated workflows:

```
# Configure OAuth with service principal

databricks configure --oauth

# Environment variables for service principal

export DATABRICKS_HOST=https://adb-xxx.azuredatabricks.net
export DATABRICKS_CLIENT_ID=your-client-id
export DATABRICKS_CLIENT_SECRET=your-client-secret
```

Authentication Precedence

Priority	Method	Source
1	Environment variables	DATABRICKS_HOST, DATABRICKS_TOKEN
2	Profile flag	--profile production
3	Config profile env	DATABRICKS_CONFIG_PROFILE
4	DEFAULT profile	~/.databrickscfg [DEFAULT]

Workspace Commands

List Workspace Contents

```
# List root workspace

databricks workspace list /

# List user folder

databricks workspace list /Users/user@company.com/
```

```
# List with details (long format)

databricks workspace list / --output json
```

Export Notebooks

```
# Export single notebook as source

databricks workspace export /Users/user/notebook ./local/notebook.py --format
 ↪  SOURCE

# Export as DBC archive

databricks workspace export /Users/user/notebook ./local/notebook.dbc --format
 ↪  DBC

# Export entire folder recursively

databricks workspace export-dir /Users/user/project/ ./local/project/ --overwrite
```

Import Notebooks

```
# Import Python notebook

databricks workspace import ./local/notebook.py /Users/user/notebook --language
 ↪  PYTHON

# Import SQL notebook

databricks workspace import ./local/query.sql /Users/user/query --language SQL

# Import folder recursively

databricks workspace import-dir ./local/project/ /Users/user/project/ --overwrite
```

Workspace Export Formats

Format	Flag	Extension	Use Case
SOURCE	`--format SOURCE`	`.py`, `.sql`, `.scala`	Version control
HTML	`--format HTML`	`.html`	Sharing
JUPYTER	`--format JUPYTER`	`.ipynb`	Jupyter
DBC	`--format DBC`	`.dbc`	Archive

Delete Workspace Items

```
# Delete notebook

databricks workspace delete /Users/user/old_notebook

# Delete folder recursively

databricks workspace delete /Users/user/old_folder --recursive
```

Create Directory

```
databricks workspace mkdirs /Users/user/new_project/notebooks
```

File System (DBFS) Commands

List Files

```
# List DBFS root

databricks fs ls dbfs:/

# List with details

databricks fs ls dbfs:/data/bronze/ --long

# Output as JSON

databricks fs ls dbfs:/data/ --output json
```

Copy Files

```
# Upload local file to DBFS

databricks fs cp ./local/data.csv dbfs:/data/input/data.csv

# Upload directory recursively

databricks fs cp ./local/folder/ dbfs:/data/folder/ --recursive --overwrite

# Download from DBFS

databricks fs cp dbfs:/data/output.csv ./local/output.csv

# Download directory

databricks fs cp dbfs:/data/results/ ./local/results/ --recursive
```

Move and Delete

```
# Move/rename file

databricks fs mv dbfs:/old/path.csv dbfs:/new/path.csv

# Delete file

databricks fs rm dbfs:/data/temp.csv

# Delete directory recursively

databricks fs rm dbfs:/data/temp_folder/ --recursive
```

Create Directory

```
databricks fs mkdirs dbfs:/data/new_folder/
```

View File Contents

```
# View file (first 64KB)

databricks fs cat dbfs:/data/sample.txt
```

Jobs Commands

List Jobs

```
# List all jobs

databricks jobs list

# List with pagination

databricks jobs list --limit 50 --offset 0

# Output as JSON

databricks jobs list --output json
```

Get Job Details

```
# Get job by ID

databricks jobs get --job-id 123456

# Get job by name

databricks jobs get --job-id $(databricks jobs list --output json | jq -r
 ↪  '.jobs[] | select(.settings.name=="my-job") | .job_id')
```

Create Job

```
# Create job from JSON file

databricks jobs create --json-file job_config.json

# Create job from inline JSON

databricks jobs create --json '{
  "name": "My ETL Job",
  "tasks": [{
    "task_key": "etl_task",
    "notebook_task": {
      "notebook_path": "/Users/user/etl_notebook"
    },
    "existing_cluster_id": "1234-567890-abcdef"
  }]
}'
```

Job Configuration Example

```
{
  "name": "Daily ETL Pipeline",
```

```
  "tasks": [
    {
      "task_key": "extract",
      "notebook_task": {
        "notebook_path": "/Workspace/Jobs/extract",
        "base_parameters": {
          "date": "{{job.start_time.iso_date}}"
        }
      },
      "new_cluster": {
        "spark_version": "14.3.x-scala2.12",
        "num_workers": 2,
        "node_type_id": "Standard_DS3_v2"
      }
    },
    {
      "task_key": "transform",
      "depends_on": [{"task_key": "extract"}],
      "notebook_task": {
        "notebook_path": "/Workspace/Jobs/transform"
      },
      "existing_cluster_id": "1234-567890-abcdef"
    }
  ],
  "schedule": {
    "quartz_cron_expression": "0 0 8 * * ?",
    "timezone_id": "America/New_York"
  },
  "email_notifications": {
    "on_failure": ["team@company.com"]
  }
}
```

Run Job

```
# Run job immediately

databricks jobs run-now --job-id 123456

# Run with parameter overrides

databricks jobs run-now --job-id 123456 --notebook-params '{"date": "2024-01-15"}'

# Submit one-time run (no saved job)

databricks jobs submit --json-file run_config.json
```

Manage Job Runs

```
# List runs for a job

databricks runs list --job-id 123456

# Get run details

databricks runs get --run-id 789012
```

```
# Cancel run

databricks runs cancel --run-id 789012

# Get run output

databricks runs get-output --run-id 789012
```

Update and Delete Jobs

```
# Update job (reset entire config)

databricks jobs reset --job-id 123456 --json-file updated_config.json

# Update job (partial update)

databricks jobs update --job-id 123456 --json '{"name": "New Job Name"}'

# Delete job

databricks jobs delete --job-id 123456
```

Continue reading: Part 2 — Cluster, Secrets, Bundle Commands & Exam Prep

---

Databricks CLI — Part 2

Overview

This file covers cluster management, secrets, Declarative Automation Bundles (DABs), Unity Catalog CLI commands, output formats, common scripting patterns, and exam preparation.

Overview

- **Cluster, secrets, and Unity Catalog commands**
- **Bundle workflow**: `bundle validate`, then `bundle deploy --target <env>`, then `bundle run <resource>`
- **Secrets**: values can never be read back through the CLI; only key names are listed
- **Output formats**: `--output json` for scripting with tools such as `jq`

What the Exam Tests

- Use Databricks REST APIs / CLI and SDK for monitoring and analyzing jobs and pipelines.
- Choose between `jobs run-now` (existing job) and `jobs submit` (one-time run)
- Know what bundle `mode: development` does to deployed resource names

Cluster Commands

List Clusters

```
# List all clusters

databricks clusters list
```

```
# Output as JSON

databricks clusters list --output json
```

Cluster Operations

```
# Get cluster details

databricks clusters get --cluster-id 1234-567890-abcdef

# Start cluster

databricks clusters start --cluster-id 1234-567890-abcdef

# Restart cluster

databricks clusters restart --cluster-id 1234-567890-abcdef

# Terminate cluster

databricks clusters delete --cluster-id 1234-567890-abcdef

# Permanently delete cluster

databricks clusters permanent-delete --cluster-id 1234-567890-abcdef
```

Create Cluster

```
# Create cluster from JSON

databricks clusters create --json '{
  "cluster_name": "my-cluster",
  "spark_version": "14.3.x-scala2.12",
  "node_type_id": "Standard_DS3_v2",
  "num_workers": 2,
  "autotermination_minutes": 60
}'
```

Edit Cluster

```
# Edit existing cluster

databricks clusters edit --json '{
  "cluster_id": "1234-567890-abcdef",
  "num_workers": 4,
  "autotermination_minutes": 120
}'
```

Secrets Commands

Secret Scopes

```
# List all secret scopes

databricks secrets list-scopes
```

```
# Create Databricks-backed scope

databricks secrets create-scope --scope my-scope

# Create scope with specific ACL

databricks secrets create-scope --scope my-scope --initial-manage-principal users

# Delete scope

databricks secrets delete-scope --scope my-scope
```

Manage Secrets

```
# List secrets in scope (names only, not values)

databricks secrets list --scope my-scope

# Create/update secret

databricks secrets put --scope my-scope --key db-password --string-value
 ↪  "secret123"

# Create secret from file

databricks secrets put --scope my-scope --key ssh-key --binary-file ./id_rsa

# Delete secret

databricks secrets delete --scope my-scope --key db-password
```

Secret ACLs

```
# List ACLs for scope

databricks secrets list-acls --scope my-scope

# Grant access

databricks secrets put-acl --scope my-scope --principal user@company.com
 ↪  --permission READ

# Revoke access

databricks secrets delete-acl --scope my-scope --principal user@company.com
```

Permission	Capabilities
READ	Read secrets
WRITE	Read and write secrets
MANAGE	Full control including ACLs

Bundle Commands (DAB)

Declarative Automation Bundles (DABs) enable infrastructure-as-code for Databricks resources.

Initialize Bundle

```
# Create new bundle from template

databricks bundle init

# Initialize in existing directory

databricks bundle init --template default-python ./my-project
```

Bundle Workflow

```
# Validate bundle configuration

databricks bundle validate

# Deploy to target environment

databricks bundle deploy

# Deploy to specific target

databricks bundle deploy --target production

# Run a resource from bundle

databricks bundle run my-job

# Destroy deployed resources

databricks bundle destroy
```

Bundle Configuration (databricks.yml)

```
bundle:
  name: my-etl-project

workspace:
  host: https://adb-1234567890.12.azuredatabricks.net

resources:
  jobs:
    daily_etl:
      name: "Daily ETL Job"
      tasks:
        - task_key: extract
          notebook_task:
            notebook_path: ./notebooks/extract.py

targets:
  development:
    mode: development
    default: true
    workspace:
      host: https://dev.azuredatabricks.net
```

```yaml
  production:
    mode: production
    workspace:
      host: https://prod.azuredatabricks.net
    resources:
      jobs:
        daily_etl:
          schedule:
            quartz_cron_expression: "0 0 8 * * ?"
            timezone_id: "America/New_York"
```

Bundle Commands Summary

Command	Purpose
`bundle init`	Create new bundle
`bundle validate`	Check configuration
`bundle deploy`	Deploy resources
`bundle run`	Execute resource
`bundle destroy`	Remove deployed resources
`bundle sync`	Sync files to workspace

Unity Catalog Commands

List Catalogs and Schemas

```
# List catalogs

databricks unity-catalog catalogs list

# List schemas in catalog

databricks unity-catalog schemas list --catalog-name main

# List tables in schema

databricks unity-catalog tables list --catalog-name main --schema-name default
```

Manage Permissions

```
# Get table permissions

databricks unity-catalog permissions tables get --full-name main.default.my_table

# Grant permissions

databricks unity-catalog permissions tables update --full-name
 ↪  main.default.my_table --json '{
  "changes": [{
    "principal": "user@company.com",
    "add": ["SELECT", "MODIFY"]
  }]
}'
```

Output Formats

JSON Output

```
# Get JSON output for parsing

databricks jobs list --output json | jq '.jobs[].settings.name'

# Pretty print JSON

databricks clusters get --cluster-id xxx --output json | jq '.'
```

Table Output (Default)

```
# Default table format

databricks jobs list
```

Common Patterns

CI/CD Pipeline Integration

```
#!/bin/bash
# Deploy pipeline script

set -e

# Configure authentication

export DATABRICKS_HOST=${{ secrets.DATABRICKS_HOST }}
export DATABRICKS_TOKEN=${{ secrets.DATABRICKS_TOKEN }}

# Validate bundle

databricks bundle validate

# Deploy to staging

databricks bundle deploy --target staging

# Run tests

databricks bundle run integration-tests --target staging

# Deploy to production (manual approval required)

databricks bundle deploy --target production
```

Backup Workspace

```
#!/bin/bash
# Backup all notebooks

BACKUP_DIR="./backup/$(date +%Y%m%d)"
mkdir -p $BACKUP_DIR

# Export shared notebooks
```

```
databricks workspace export-dir /Shared/ $BACKUP_DIR/Shared/ --overwrite

# Export user notebooks

for user in $(databricks workspace list /Users/ --output json | jq -r
 ↪  '.[].path'); do
    databricks workspace export-dir "$user" "$BACKUP_DIR$user" --overwrite
done
```

Migrate Jobs Between Workspaces

```
#!/bin/bash
# Export job from source, import to target

# Export from source

DATABRICKS_CONFIG_PROFILE=source databricks jobs get --job-id 123 --output json >
 ↪  job.json

# Remove job_id and created_time for import

jq 'del(.job_id, .created_time)' job.json > job_clean.json

# Import to target

DATABRICKS_CONFIG_PROFILE=target databricks jobs create --json-file
 ↪  job_clean.json
```

Use Cases

- **CI/CD Automation**: Integrating Declarative Automation Bundles (DABs) deployment into GitHub Actions or Azure DevOps pipelines.
- **Bulk Operations**: Managing hundreds of secrets, creating cluster policies, or syncing workspace directories automatically.
- **Local IDE Integration**: Interacting with the remote Databricks workspace directly from a local terminal or IDE.

Common Issues & Errors

Authentication Failed **Scenario:** Invalid or expired token.

```
# Error: INVALID_PARAMETER_VALUE: Invalid access token
```

Fix: Regenerate PAT and reconfigure:

```
databricks configure --token
```

Profile Not Found **Scenario:** Specified profile doesn't exist in config.

```
# Error: cannot find profile "nonexistent" in ~/.databrickscfg
```

Fix: Check profile name matches config file or create profile.

Permission Denied on Workspace **Scenario:** User lacks permission for workspace operation.

```
# Error: PERMISSION_DENIED: User does not have permission
```

Fix: Request appropriate workspace permissions from admin.

Cluster Not Found **Scenario:** Cluster ID is invalid or cluster was deleted.

```
# Error: RESOURCE_DOES_NOT_EXIST: Cluster xxx does not exist
```

Fix: Verify cluster ID with `databricks clusters list`.

Rate Limiting **Scenario:** Too many API requests.

```
# Error: 429 Too Many Requests
```

Fix: Add delays between requests or batch operations.

Exam Tips

1. **Authentication order** - Environment variables > profile flag > config file
2. **Profile syntax** - Use `--profile name` or `DATABRICKS_CONFIG_PROFILE`
3. **Workspace paths** - Always start with `/` for absolute paths
4. **Export formats** - SOURCE for version control, DBC for archives
5. **Jobs API** - `run-now` triggers existing job, `submit` creates one-time run
6. **Secret scopes** - `create-scope` for new, `put` to add secrets
7. **Bundle commands** - validate → deploy → run workflow
8. **DBFS paths** - Use `dbfs:/` prefix for fs commands
9. **JSON output** - Use `--output json` for scripting and automation
10. **Cluster states** - Know PENDING, RUNNING, TERMINATED, ERROR states

Key Takeaways

- **Authentication precedence**: environment variables (`DATABRICKS_HOST`, `DATABRICKS_TOKEN`) override profile flag (`--profile`), which overrides the default profile in `~/.databrickscfg`
- **Declarative Automation Bundle (DAB) workflow**: `bundle validate` → `bundle deploy --target <env>` → `bundle run <resource>`; `databricks.yml` defines resources, targets, and workspace settings
- **`jobs run-now`** triggers an existing job by ID; **`jobs submit`** creates a one-time run without a persistent job definition
- **Secret scope management**: `databricks secrets create-scope` creates the scope; `databricks secrets put --scope ... --key ... --string-value ...` adds or updates a secret; values can never be retrieved via CLI, only key names are listed
- **DBFS paths** in CLI commands require the `dbfs:/` prefix (e.g., `databricks fs ls dbfs:/data/`)
- **Output format**: default is human-readable table; `--output json` enables scripting with tools like `jq`
- **Cluster lifecycle states**: PENDING → RUNNING → TERMINATED (or ERROR); use `databricks clusters list --output json` to check state programmatically
- **Bundle `mode: development`** applies a prefix to deployed resource names to prevent collision with production resources; `mode: production` deploys without prefix

Related Topics

- REST API — Part 1 - Programmatic API access
- Declarative Automation Bundles - Infrastructure as code
- CI/CD Integration - Pipeline automation

Official Documentation

- Databricks CLI Reference
- CLI Authentication
- Declarative Automation Bundles

Databricks REST API — Part 1: Jobs, Clusters, DBFS & Workspace APIs

The Databricks REST API provides programmatic access to workspace resources for automation, integration, and custom tooling. This part covers API basics, authentication, Jobs API, Clusters API, DBFS API, Workspace API, and Secrets API.

Overview

```
flowchart TB
    Client[API Client] --> Auth[Authentication]
    Auth --> API[REST API]

    API --> Jobs["Jobs API 2.1"]
    API --> Clusters["Clusters API 2.0"]
    API --> DBFS["DBFS API 2.0"]
    API --> Workspace["Workspace API 2.0"]
    API --> Secrets["Secrets API 2.0"]
    API --> UC["Unity Catalog API"]

    Jobs --> Create[Create/Update Jobs]
    Jobs --> Run[Run Jobs]
    Jobs --> Monitor[Monitor Runs]
```

Overview

- **API basics**: versioned REST endpoints, authenticated with personal access tokens, OAuth service principals, or the Python SDK
- **Jobs API 2.1**: create jobs, `run-now` vs `runs/submit`, run states, and cancelling runs
- **Other APIs**: Clusters, DBFS, Workspace, and Secrets

What the Exam Tests

- Use Lakeflow Jobs UI and Jobs API to monitor job status and performance metrics.
- Create and automate ETL workloads using Lakeflow Jobs via the UI, API, and CLI.
- Read run states to tell a failed run from a cancelled or still-running one

API Basics

Base URL

```
https://<workspace-url>/api/<version>/<endpoint>

Examples:
https://adb-1234567890.12.azuredatabricks.net/api/2.1/jobs/list
https://adb-1234567890.12.azuredatabricks.net/api/2.0/clusters/list
```

API Versions

Version	Endpoints	Status
2.0	Clusters, DBFS, Workspace, Secrets	Stable
2.1	Jobs (enhanced)	Current
2.2	SQL Statements	Current

Authentication

Personal Access Token (PAT)

```
# Bearer token authentication

curl -X GET \
  'https://adb-xxx.azuredatabricks.net/api/2.1/jobs/list' \
  -H 'Authorization: Bearer dapi1234567890abcdef'
```

Python SDK Authentication

```
from databricks.sdk import WorkspaceClient

# Using environment variables (recommended)
# DATABRICKS_HOST and DATABRICKS_TOKEN

w = WorkspaceClient()

# Explicit configuration

w = WorkspaceClient(
    host="https://adb-xxx.azuredatabricks.net",
    token="dapi1234567890abcdef"
)

# Using config profile

w = WorkspaceClient(profile="production")
```

OAuth (Service Principal)

```
from databricks.sdk import WorkspaceClient

# Service principal authentication

w = WorkspaceClient(
    host="https://adb-xxx.azuredatabricks.net",
    client_id="your-client-id",
    client_secret="your-client-secret"
)
```

Jobs API 2.1

The Jobs API manages job definitions and runs.

List Jobs

```
# List all jobs

curl -X GET \
  'https://adb-xxx.azuredatabricks.net/api/2.1/jobs/list' \
  -H 'Authorization: Bearer $TOKEN'

# List with pagination

curl -X GET \
  'https://adb-xxx.azuredatabricks.net/api/2.1/jobs/list?limit=25&offset=0' \
  -H 'Authorization: Bearer $TOKEN'
```

```
# Python SDK

from databricks.sdk import WorkspaceClient

w = WorkspaceClient()
for job in w.jobs.list():
    print(f"{job.job_id}: {job.settings.name}")
```

Get Job Details

```
curl -X GET \
  'https://adb-xxx.azuredatabricks.net/api/2.1/jobs/get?job_id=123456' \
  -H 'Authorization: Bearer $TOKEN'
```

```
job = w.jobs.get(job_id=123456)
print(job.settings.name)
print(job.settings.tasks)
```

Create Job

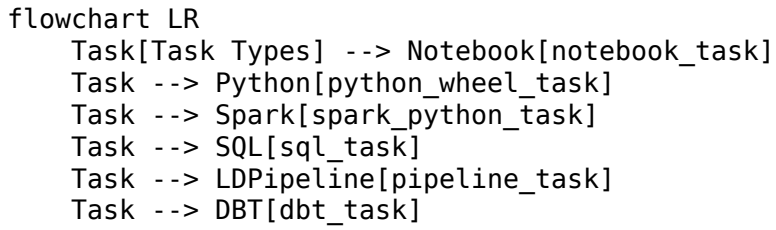
```
curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.1/jobs/create' \
  -H 'Authorization: Bearer $TOKEN' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "My ETL Job",
    "tasks": [{
      "task_key": "extract_task",
      "notebook_task": {
        "notebook_path": "/Users/user/etl/extract",
        "base_parameters": {
          "date": "2024-01-01"
        }
      },
      "new_cluster": {
        "spark_version": "14.3.x-scala2.12",
        "num_workers": 2,
        "node_type_id": "Standard_DS3_v2"
      }
    }],
    "schedule": {
      "quartz_cron_expression": "0 0 8 * * ?",
      "timezone_id": "America/New_York"
```

```
    }
  }'
```

```
from databricks.sdk.service.jobs import Task, NotebookTask, JobCluster

job = w.jobs.create(
    name="My ETL Job",
    tasks=[
        Task(
            task_key="extract_task",
            notebook_task=NotebookTask(
                notebook_path="/Users/user/etl/extract",
                base_parameters={"date": "2024-01-01"}
            ),
            new_cluster={
                "spark_version": "14.3.x-scala2.12",
                "num_workers": 2,
                "node_type_id": "Standard_DS3_v2"
            }
        )
    ]
)
print(f"Created job: {job.job_id}")
```

Job Task Types



Task Type	Use Case	Key Fields
notebook_task	Run notebook	notebook_path, base_parameters
spark_python_task	Run Python file	python_file, parameters
python_wheel_task	Run Python wheel	package_name, entry_point
sql_task	Run SQL query	query, warehouse_id
pipeline_task	Run Lakeflow pipeline	pipeline_id
dbt_task	Run dbt project	project_directory, commands

Run Job (run-now) Trigger an existing job immediately:

```
curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.1/jobs/run-now' \
  -H 'Authorization: Bearer $TOKEN' \
  -H 'Content-Type: application/json' \
  -d '{
    "job_id": 123456,
    "notebook_params": {
      "date": "2024-01-15",
      "env": "prod"
    }
  }'
```

```
run = w.jobs.run_now(
    job_id=123456,
    notebook_params={"date": "2024-01-15", "env": "prod"}
)
print(f"Run ID: {run.run_id}")
```

Submit One-Time Run (runs/submit) Create and run a job without saving the job definition:

```
curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.1/jobs/runs/submit' \
  -H 'Authorization: Bearer $TOKEN' \
  -H 'Content-Type: application/json' \
  -d '{
    "run_name": "One-time ETL",
    "tasks": [{
      "task_key": "main",
      "notebook_task": {
        "notebook_path": "/Users/user/one_time_job"
      },
      "new_cluster": {
        "spark_version": "14.3.x-scala2.12",
        "num_workers": 1,
        "node_type_id": "Standard_DS3_v2"
      }
    }]
  }'
```

run-now vs runs/submit

Feature	run-now	runs/submit
Requires existing job	Yes	No
Job definition saved	Yes	No
Use case	Production scheduled jobs	Ad-hoc runs, testing
Parameter override	Yes	Define inline
Appears in job run history	Yes	No (separate runs list)

Get Run Status

```
curl -X GET \
  'https://adb-xxx.azuredatabricks.net/api/2.1/jobs/runs/get?run_id=789012' \
  -H 'Authorization: Bearer $TOKEN'
```

```
run = w.jobs.get_run(run_id=789012)
print(f"State: {run.state.life_cycle_state}")
print(f"Result: {run.state.result_state}")
```

Run States

```
stateDiagram-v2
    [*] --> PENDING
    PENDING --> RUNNING
    RUNNING --> TERMINATING
    TERMINATING --> TERMINATED
    TERMINATED --> [*]
```

```
    RUNNING --> INTERNAL_ERROR
    INTERNAL_ERROR --> [*]

    PENDING --> SKIPPED
    SKIPPED --> [*]
```

Life Cycle State	Meaning
PENDING	Run queued, waiting for resources
RUNNING	Actively executing
TERMINATING	Finishing up
TERMINATED	Completed
SKIPPED	Skipped due to conditions
INTERNAL_ERROR	Platform error

Result State	Meaning
SUCCESS	Completed successfully
FAILED	Task failed
TIMEDOUT	Exceeded timeout
CANCELED	Manually canceled

Cancel Run

```
curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.1/jobs/runs/cancel' \
  -H 'Authorization: Bearer $TOKEN' \
  -H 'Content-Type: application/json' \
  -d '{"run_id": 789012}'
```

```
w.jobs.cancel_run(run_id=789012)
```

Update Job

```
# Reset entire job configuration

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.1/jobs/reset' \
  -H 'Authorization: Bearer $TOKEN' \
  -H 'Content-Type: application/json' \
  -d '{
    "job_id": 123456,
    "new_settings": {
      "name": "Updated Job Name",
      "tasks": [...]
    }
  }'

# Partial update

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.1/jobs/update' \
  -H 'Authorization: Bearer $TOKEN' \
  -H 'Content-Type: application/json' \
  -d '{
```

```
    "job_id": 123456,
    "new_settings": {
      "name": "New Name Only"
    }
  }'
```

Delete Job

```
curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.1/jobs/delete' \
  -H 'Authorization: Bearer $TOKEN' \
  -H 'Content-Type: application/json' \
  -d '{"job_id": 123456}'
```

Clusters API 2.0

List Clusters

```
curl -X GET \
  'https://adb-xxx.azuredatabricks.net/api/2.0/clusters/list' \
  -H 'Authorization: Bearer $TOKEN'
```

```
for cluster in w.clusters.list():
    print(f"{cluster.cluster_id}: {cluster.cluster_name} ({cluster.state})")
```

Get Cluster Details

```
curl -X GET \
  'https://adb-xxx.azuredatabricks.net/api/2.0/clusters/get?cluster_id=1234-
  ↪ 567890-abc'
  ↪ \
  -H 'Authorization: Bearer $TOKEN'
```

Create Cluster

```
curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/clusters/create' \
  -H 'Authorization: Bearer $TOKEN' \
  -H 'Content-Type: application/json' \
  -d '{
    "cluster_name": "my-cluster",
    "spark_version": "14.3.x-scala2.12",
    "node_type_id": "Standard_DS3_v2",
    "num_workers": 2,
    "autotermination_minutes": 60,
    "spark_conf": {
      "spark.speculation": "true"
    },
    "custom_tags": {
      "team": "data-engineering"
    }
  }'
```

```
cluster = w.clusters.create(
    cluster_name="my-cluster",
    spark_version="14.3.x-scala2.12",
    node_type_id="Standard_DS3_v2",
```

```
    num_workers=2,
    autotermination_minutes=60
).result()  # Wait for creation
print(f"Cluster ID: {cluster.cluster_id}")
```

Cluster Operations

```
# Start cluster

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/clusters/start' \
  -H 'Authorization: Bearer $TOKEN' \
  -d '{"cluster_id": "1234-567890-abc"}'

# Restart cluster

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/clusters/restart' \
  -H 'Authorization: Bearer $TOKEN' \
  -d '{"cluster_id": "1234-567890-abc"}'

# Terminate cluster

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/clusters/delete' \
  -H 'Authorization: Bearer $TOKEN' \
  -d '{"cluster_id": "1234-567890-abc"}'

# Permanently delete

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/clusters/permanent-delete' \
  -H 'Authorization: Bearer $TOKEN' \
  -d '{"cluster_id": "1234-567890-abc"}'
```

Cluster States

State	Description
PENDING	Being created
RUNNING	Ready for use
RESTARTING	Restarting
RESIZING	Changing size
TERMINATING	Shutting down
TERMINATED	Stopped
ERROR	Failed to start

DBFS API 2.0

List Files

```
curl -X GET \
  'https://adb-xxx.azuredatabricks.net/api/2.0/dbfs/list?path=/data/' \
  -H 'Authorization: Bearer $TOKEN'
```

Read File

```
# Read file (base64 encoded, max 1MB)

curl -X GET \
  'https://adb-
   ↪  xxx.azuredatabricks.net/api/2.0/dbfs/read?path=/data/sample.txt&offset=0&length=1000'
   ↪  \
  -H 'Authorization: Bearer $TOKEN'
```

Upload File For files larger than 1MB, use the streaming upload API:

```
# Create upload handle

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/dbfs/create' \
  -H 'Authorization: Bearer $TOKEN' \
  -d '{"path": "/data/large_file.csv", "overwrite": true}'

# Response: {"handle": 123456789}

# Add blocks (repeat for each chunk, max 1MB per block)

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/dbfs/add-block' \
  -H 'Authorization: Bearer $TOKEN' \
  -d '{"handle": 123456789, "data": "base64_encoded_data"}'

# Close handle

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/dbfs/close' \
  -H 'Authorization: Bearer $TOKEN' \
  -d '{"handle": 123456789}'
```

File Operations

```
# Create directory

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/dbfs/mkdirs' \
  -H 'Authorization: Bearer $TOKEN' \
  -d '{"path": "/data/new_folder/"}'

# Delete file/directory

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/dbfs/delete' \
  -H 'Authorization: Bearer $TOKEN' \
  -d '{"path": "/data/temp/", "recursive": true}'

# Move file

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/dbfs/move' \
```

```
  -H 'Authorization: Bearer $TOKEN' \
  -d '{"source_path": "/old/path.csv", "destination_path": "/new/path.csv"}'
```

Workspace API 2.0

List Workspace

```
curl -X GET \
  'https://adb-xxx.azuredatabricks.net/api/2.0/workspace/list?path=/Users/user/'
   ↪  \
  -H 'Authorization: Bearer $TOKEN'
```

Export Notebook

```
# Export as SOURCE format

curl -X GET \
  'https://adb-
   ↪  xxx.azuredatabricks.net/api/2.0/workspace/export?path=/Users/user/notebook&format=SOUR
   ↪  \
  -H 'Authorization: Bearer $TOKEN'

# Response contains base64 encoded content
```

Import Notebook

```
curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/workspace/import' \
  -H 'Authorization: Bearer $TOKEN' \
  -d '{
    "path": "/Users/user/new_notebook",
    "format": "SOURCE",
    "language": "PYTHON",
    "content": "base64_encoded_content",
    "overwrite": true
  }'
```

Delete Workspace Object

```
curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/workspace/delete' \
  -H 'Authorization: Bearer $TOKEN' \
  -d '{"path": "/Users/user/old_notebook", "recursive": false}'
```

Secrets API 2.0

List Scopes

```
curl -X GET \
  'https://adb-xxx.azuredatabricks.net/api/2.0/secrets/scopes/list' \
  -H 'Authorization: Bearer $TOKEN'
```

Create Scope

```
curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/secrets/scopes/create' \
```

```
  -H 'Authorization: Bearer $TOKEN' \
  -d '{"scope": "my-scope"}'
```

Manage Secrets

```
# List secrets (names only)

curl -X GET \
  'https://adb-xxx.azuredatabricks.net/api/2.0/secrets/list?scope=my-scope' \
  -H 'Authorization: Bearer $TOKEN'

# Put secret

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/secrets/put' \
  -H 'Authorization: Bearer $TOKEN' \
  -d '{
    "scope": "my-scope",
    "key": "db-password",
    "string_value": "secret123"
  }'

# Delete secret

curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/secrets/delete' \
  -H 'Authorization: Bearer $TOKEN' \
  -d '{"scope": "my-scope", "key": "db-password"}'
```

Continue reading: Part 2 — Permissions, SQL, Error Handling & Use Cases

---

Databricks REST API — Part 2: Monitoring with SDK, REST & CLI; Permissions, SQL & Error Handling

Overview

This part shows how to monitor and analyze Lakeflow Jobs and Lakeflow Pipelines programmatically with the Databricks SDK for Python, the REST API, and the Databricks CLI. It then covers the Permissions API, the SQL Statement Execution API, error handling, and rate limits. For API basics, authentication, and the Jobs, Clusters, DBFS, and Workspace APIs, see Part 1.

Overview

- The same operations exist in three interfaces: SDK method (`w.jobs.list_runs`), REST endpoint (`GET /api/2.2/jobs/runs/list`), and CLI command (`databricks jobs list-runs`).
- Jobs monitoring uses `list_runs`, `get_run`, and `get_run_output`. Pipelines monitoring uses `list_pipelines`, `get`, `list_updates`, `get_update`, and `list_pipeline_events`.
- Run health comes from `status.state` and `status.termination_details` (the `state` object is deprecated), plus the duration fields (`queue_duration`, `setup_duration`, `execution_duration`, `cleanup_duration`, `run_duration`, all in milliseconds).
- The SDK handles authentication, pagination (`next_page_token`), and waiting (`run_now_and_wait`, `.result()`) for you.

What the Exam Tests

- "Use Databricks REST APIs / CLI and SDK for monitoring and analyzing jobs and pipelines."
- Mapping a monitoring need (active runs, failed runs since yesterday, why a run failed, pipeline errors, update history) to the right method, endpoint, or command and its filter parameters.
- Knowing the current Jobs API version (2.2), its pagination behavior, and the per-endpoint rate limits that affect polling scripts.

Platform update (Oct 2026)

Databricks recommends **Jobs API 2.2** for new and existing clients. Version 2.2 enables queueing by default, removes `has_more` from list responses (check for `next_page_token` instead), and paginates arrays longer than 100 elements in `jobs/get` and `jobs/runs/get`. Part 1 shows `/api/2.1/` paths for the Jobs API. Source: Updating from Jobs API 2.1 to 2.2

Monitoring Jobs and Pipelines: SDK, REST, and CLI

Interface map

Need	Python SDK (`w = WorkspaceClient()`)
List runs (filters: job, active/completed, time range, run type)	`w.jobs.list_runs(...)`
One run with task details and repair history	`w.jobs.get_run(run_id, include_history=`
Output or error of one **task** run	`w.jobs.get_run_output(run_id)`
Trigger and wait	`w.jobs.run_now_and_wait(job_id, timeout`
Find pipelines	`w.pipelines.list_pipelines(filter="name`
Pipeline state and health	`w.pipelines.get(pipeline_id)`
Pipeline update history	`w.pipelines.list_updates(pipeline_id)`
One pipeline update	`w.pipelines.get_update(pipeline_id, upd`
Pipeline event log entries	`w.pipelines.list_pipeline_events(pipeli`

Databricks SDK for Python: jobs The SDK (`pip install databricks-sdk`) is in Beta and is fine to use in production. Databricks recommends pinning the minor version. `WorkspaceClient()` with no arguments uses Databricks unified authentication: environment variables, a `.databrickscfg` profile, or default notebook authentication when the code runs inside a Databricks notebook.

```
from datetime import datetime, timedelta, timezone

from databricks.sdk import WorkspaceClient

w = WorkspaceClient()
job_id = 123456
since_ms = int((datetime.now(timezone.utc) - timedelta(days=1)).timestamp() *
 ↪  1000)

# Completed runs of one job in the last 24 hours (the iterator follows
 ↪  next_page_token)
for run in w.jobs.list_runs(job_id=job_id, completed_only=True,
 ↪  start_time_from=since_ms):
    details = run.status.termination_details if run.status else None
    print(
```

```
        run.run_id,
        run.status.state.value if run.status else None,      # e.g. TERMINATED
        details.code.value if details and details.code else None,  # e.g.
↪  RUN_EXECUTION_ERROR
        (run.run_duration or 0) / 1000,                      # seconds, includes
↪  repairs
        run.run_page_url,
    )

# Runs still in flight (QUEUED, PENDING, RUNNING, or TERMINATING)
active = list(w.jobs.list_runs(job_id=job_id, active_only=True))

# Drill into one run: per-task status and time breakdown
run = w.jobs.get_run(run_id=987654321, include_history=True)
print("queued ms:", run.queue_duration, "total ms:", run.run_duration)

for task in run.tasks or []:
    print(
        task.task_key,
        task.status.state.value if task.status else None,
       task.setup_duration, task.execution_duration, task.cleanup_duration,  # ms
    )

# Why did a task fail? get_run_output takes a TASK run_id
failed = [t for t in run.tasks or []
          if t.status and t.status.termination_details
          and t.status.termination_details.type
          and t.status.termination_details.type.value != "SUCCESS"]
for t in failed:
    out = w.jobs.get_run_output(run_id=t.run_id)
    print(t.task_key, out.error, out.error_trace)

# Trigger a job and block until it finishes (default timeout is 20 minutes)
finished = w.jobs.run_now_and_wait(job_id=job_id, timeout=timedelta(minutes=60))
print(finished.status.state, finished.status.termination_details)

# Equivalent: run_now returns a Wait object, and .result() blocks
finished = w.jobs.run_now(job_id=job_id).result(timeout=timedelta(minutes=60))
```

Key Run fields for monitoring:

Field	Meaning
`status.state`	QUEUED, PENDING, BLOCKED, WA
`status.termination_details.code`	Why it ended: SUCCESS, SUCC
`status.termination_details.type`	SUCCESS, CLIENT_ERROR, INTE
`state` (**deprecated**)	`life_cycle_state` (PENDING,
`start_time`, `end_time`	Epoch milliseconds. `end_time`
`queue_duration`, `setup_duration`, `execution_duration`, `cleanup_duration`	Milliseconds. For multitask ru
`run_duration`	Milliseconds for the job run **ar**
`tasks[]`	Per-task `task_key`, `run_id`, s
`repair_history`	Returned when you request i
`run_page_url`, `trigger`, `run_type`	Link to the UI, what triggered

`get_run_output` returns the first 5 MB of a **task** run's output: `notebook_output.result` (the `dbutils.notebook.exit()` value), `error`, `error_trace`, `logs`, and `sql_output`. For a SQL alert task, `alert_output.alert_state` gives the evaluated alert state. Runs are removed after **60**

days, so export or copy anything you need longer. For account-wide history up to 365 days, query `system.lakeflow.job_run_timeline` instead.

Databricks SDK for Python: pipelines

```
w = WorkspaceClient()

# Find a pipeline by name (filters: name LIKE '...' or notebook='<path>')
pipeline = next(iter(w.pipelines.list_pipelines(filter="name LIKE '%orders%'")))
pid = pipeline.pipeline_id

# Current state and health
p = w.pipelines.get(pipeline_id=pid)
print(p.state, p.health)            # e.g. IDLE / RUNNING / FAILED, HEALTHY /
 ↪  UNHEALTHY
for u in p.latest_updates or []:
    print(u.update_id, u.state, u.creation_time)

# Update history: list_updates returns ListUpdatesResponse (not an iterator)
resp = w.pipelines.list_updates(pipeline_id=pid, max_results=25)
for u in resp.updates or []:
    print(u.update_id, u.state.value if u.state else None, u.cause,
     ↪  u.full_refresh)

# Errors and warnings from the event log
for ev in w.pipelines.list_pipeline_events(
    pipeline_id=pid,
    filter="level in ('ERROR', 'WARN') AND timestamp > '2026-10-25T00:00:00Z'",
    order_by=["timestamp desc"],
):
    print(ev.timestamp, ev.level, ev.event_type, ev.message)
```

Pipeline object	Values to know
`GetPipelineResponse.state`	`IDLE`, `DEPLOYING`, `STARTING`, `RUNNING`, `STOPPING`, `RECOVERING`, `RESETTING`, `FA`
`GetPipelineResponse.health`	`HEALTHY`, `UNHEALTHY`
`UpdateInfo.state`	`QUEUED`, `CREATED`, `WAITING_FOR_RESOURCES`, `INITIALIZING`, `RESETTING`, `SETT`
`PipelineEvent`	`id`, `timestamp`, `level` (`INFO`, `WARN`, `ERROR`, `METRICS`), `event_type`, `message`, `e`
`list_pipeline_events` filter	`level='ERROR'`, `level in ('INFO','WARN')`, `id='<event-id>'`, `timestamp`

Tip

The events API returns the same records as the pipeline **event log**. For SQL analytics over many updates (expectation pass/fail counts, flow progress, lineage), query the event log table instead. See Lakeflow Event Logs.

REST API equivalents

```
HOST="https://<workspace-url>"

# Active runs of a job (limit is 1-25, default 20; page with page_token)
curl -s -H "Authorization: Bearer $TOKEN" \
  "$HOST/api/2.2/jobs/runs/list?job_id=123456&active_only=true&limit=25"
```

```
# Completed runs since a UTC epoch-millisecond timestamp
curl -s -H "Authorization: Bearer $TOKEN" \
  "$HOST
   ↪  /api/2.2/jobs/runs/list?job_id=123456&completed_only=true&start_time_from=176143680000

# One run with repair history
curl -s -H "Authorization: Bearer $TOKEN" \
  "$HOST/api/2.2/jobs/runs/get?run_id=987654321&include_history=true"

# Output and error of one task run
curl -s -H "Authorization: Bearer $TOKEN" \
  "$HOST/api/2.2/jobs/runs/get-output?run_id=987654322"

# Pipeline updates and ERROR events (URL-encode the filter)
curl -s -H "Authorization: Bearer $TOKEN" \
  "$HOST/api/2.0/pipelines/<pipeline-id>/updates?max_results=25"
curl -s -G -H "Authorization: Bearer $TOKEN" \
  --data-urlencode "filter=level='ERROR'" --data-urlencode "max_results=50" \
  "$HOST/api/2.0/pipelines/<pipeline-id>/events"
```

Pagination: list endpoints return `next_page_token` (and `prev_page_token` for jobs). Pass it back as `page_token`. In Jobs API 2.2 there is no `has_more` field, so "no `next_page_token`" means "no more results". The `offset` parameter is deprecated in favor of `page_token`.

Databricks CLI equivalents

```
# Jobs
databricks jobs list-runs --job-id 123456 --active-only
databricks jobs list-runs --job-id 123456 --completed-only --start-time-from
 ↪  1761436800000
databricks jobs get-run 987654321 --include-history
databricks jobs get-run-output 987654322

# Pipelines (resource commands)
databricks pipelines list-pipelines --filter "name LIKE '%orders%'"
databricks pipelines get <pipeline-id>
databricks pipelines list-updates <pipeline-id> --max-results 25
databricks pipelines get-update <pipeline-id> <update-id>
databricks pipelines list-pipeline-events <pipeline-id> --filter "level='ERROR'"
 ↪  --max-results 50

# Pipelines (project commands, using the pipeline KEY from the bundle YAML)
databricks pipelines history my_pipeline --start-time 2026-10-25T00:00:00Z
databricks pipelines logs my_pipeline --level ERROR,METRICS --event-type
 ↪  update_progress
```

`databricks pipelines logs` shows the events of the most recent update by default. Use `--update-id`, `--level`, `--event-type`, `--start-time`/`--end-time`, and `-n` to narrow it.

Choosing the interface

Situation	Best fit
Python automation, CI checks, or polling until done	SDK (typed objects, auto-pagination, bui
Shell scripts, ad hoc checks, CI steps without Python	CLI (one command per operation, scripta
Non-Python clients or tools that only speak HTTP	REST API

Situation	Best fit
Account-wide history, trends, or cost (beyond 60 days, across jobs)	System tables (`system.lakeflow.*`), no

---

Permissions API 2.0

Get permissions

```
# Get job permissions
curl -X GET \
  'https://adb-xxx.azuredatabricks.net/api/2.0/permissions/jobs/123456' \
  -H "Authorization: Bearer $TOKEN"

# Get cluster permissions
curl -X GET \
  'https://adb-xxx.azuredatabricks.net/api/2.0/permissions/clusters/1234-567890-
  ↪  abc'
  ↪  \
  -H "Authorization: Bearer $TOKEN"
```

Set permissions

```
curl -X PATCH \
  'https://adb-xxx.azuredatabricks.net/api/2.0/permissions/jobs/123456' \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "access_control_list": [
      {
        "user_name": "user@company.com",
        "permission_level": "CAN_MANAGE_RUN"
      },
      {
        "group_name": "data-engineers",
        "permission_level": "CAN_VIEW"
      }
    ]
  }'
```

Permission levels

Resource	Levels
Jobs	`CAN_VIEW`, `CAN_MANAGE_RUN`, `IS_OWNER`, `CAN_MANAGE`
Lakeflow pipelines	`CAN_VIEW`, `CAN_RUN`, `CAN_MANAGE`, `IS_OWNER`
Clusters	`CAN_ATTACH_TO`, `CAN_RESTART`, `CAN_MANAGE` (`CAN_VIEW` is in Beta)
Notebooks	`CAN_READ` (shown as CAN VIEW in the UI), `CAN_RUN`, `CAN_EDIT`, `CAN_MANAGE`

To monitor a job you need at least `CAN_VIEW`. To see a run's Spark UI and logs, or to cancel or run it, you need `CAN_MANAGE_RUN`.

SQL Statement Execution API

Execute SQL statements on SQL warehouses:

```
curl -X POST \
  'https://adb-xxx.azuredatabricks.net/api/2.0/sql/statements' \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "warehouse_id": "abc123def456",
    "statement": "SELECT * FROM main.default.my_table LIMIT 100",
    "wait_timeout": "30s"
  }'
```

```
# Using the SDK
statement = w.statement_execution.execute_statement(
    warehouse_id="abc123def456",
    statement="SELECT * FROM main.default.my_table LIMIT 100",
    wait_timeout="30s",
)

for row in statement.result.data_array:
    print(row)
```

This is also how an external tool can query `system.lakeflow.*` or `system.billing.*` for monitoring data without a notebook.

Error Handling

HTTP status codes

Code	Meaning
200	Success
400	Bad request (invalid parameters)
401	Unauthorized (invalid token)
403	Forbidden (insufficient permissions)
404	Resource not found
429	Rate limited
500	Internal server error

Error response format

```
{
  "error_code": "RESOURCE_DOES_NOT_EXIST",
  "message": "Job 123456 does not exist"
}
```

Common error codes

Error Code	Meaning
`INVALID_PARAMETER_VALUE`	Invalid request parameter
`RESOURCE_DOES_NOT_EXIST`	Job, cluster, or notebook not found
`RESOURCE_ALREADY_EXISTS`	Name conflict
`PERMISSION_DENIED`	Insufficient access
`QUOTA_EXCEEDED`	Resource limits reached
`TEMPORARILY_UNAVAILABLE`	Service temporarily down

For 500-level errors from the Jobs API, Databricks recommends retrying for up to 10 minutes, with at least 30 seconds between retries.

Python error handling

```python
from databricks.sdk import WorkspaceClient
from databricks.sdk.errors import NotFound, PermissionDenied

w = WorkspaceClient()

try:
    job = w.jobs.get(job_id=999999)
except NotFound:
    print("Job not found")
except PermissionDenied:
    print("Access denied")
except Exception as e:
    print(f"Unexpected error: {e}")
```

Rate Limiting

Jobs and Pipelines API limits (per workspace, fixed)

Endpoint	Limit
/jobs/runs/get	100 requests per second
/jobs/runs/list	30 requests per second
/jobs/runs/get-output	20 requests per second
/jobs/run-now, /jobs/create, /jobs/get, /jobs/list	20 requests per second each
/jobs/runs/submit	35 requests per second
/jobs/runs/repair	5 requests per second
Pipelines API GET	150 requests per second
Pipelines API POST / PUT / DELETE	50 requests per second

These limits are marked **Fixed**, which means Databricks cannot raise them on request. A dashboard that polls runs/get for hundreds of runs every second can hit them. Use backoff, poll less often, or move to system tables for history.

Handling rate limits

```python
import time
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()

def call_with_retry(func, max_retries=3):
    for attempt in range(max_retries):
        try:
            return func()
        except Exception as e:
            if "429" in str(e):  # Rate limited
                wait_time = 2 ** attempt  # Exponential backoff
                time.sleep(wait_time)
            else:
                raise
```

```
    raise Exception("Max retries exceeded")

# Usage
result = call_with_retry(lambda: list(w.jobs.list()))
```

Use Cases

Automated job deployment

```
from databricks.sdk import WorkspaceClient

def deploy_job(w, job_config):
    """Deploy or update a job based on configuration."""
    existing_jobs = list(w.jobs.list(name=job_config["name"]))

    if existing_jobs:
        job_id = existing_jobs[0].job_id
        w.jobs.reset(job_id=job_id, new_settings=job_config)
        return job_id
    job = w.jobs.create(**job_config)
    return job.job_id
```

Failure report for the last 24 hours

```
from datetime import datetime, timedelta, timezone

def failed_runs_last_day(w, job_id):
    """Return (run_id, termination code, URL) for runs that did not succeed."""
    since_ms = int((datetime.now(timezone.utc) - timedelta(days=1)).timestamp() *
 ↪  1000)
    report = []
    for run in w.jobs.list_runs(job_id=job_id, completed_only=True,
     ↪  start_time_from=since_ms):
        details = run.status.termination_details if run.status else None
        if details and details.code and details.code.value != "SUCCESS":
            report.append((run.run_id, details.code.value, run.run_page_url))
    return report
```

Pipeline health check in CI

```
def assert_pipeline_healthy(w, pipeline_id):
    p = w.pipelines.get(pipeline_id=pipeline_id)
    errors = list(w.pipelines.list_pipeline_events(
        pipeline_id=pipeline_id, filter="level='ERROR'", max_results=10))
    if (p.health and p.health.value == "UNHEALTHY") or errors:
        raise RuntimeError(f"{p.name}: {len(errors)} recent ERROR events")
```

Common Issues & Errors

Polling a run shows state but the code expects status **Cause:** `state` (`life_cycle_state` / `result_state`) is deprecated in favor of `status` (`state` plus `termination_details`).

Fix: read `run.status.state` and `run.status.termination_details.code`. Keep any `state` handling only for older clients.

`get_run_output` fails on a multitask job run **Cause:** the endpoint takes a **task** run ID, not the parent job run ID.

Fix: call `get_run` first, then pass each `tasks[i].run_id` to `get_run_output`.

Script only sees 20 runs, or stops paging **Cause:** `limit` defaults to 20 (maximum 25), and Jobs API 2.2 removed `has_more`.

Fix: follow `next_page_token` until it is absent, or use the SDK iterator, which pages for you.

Old runs return 400 or are missing **Cause:** runs are automatically removed after 60 days.

Fix: export run results, or query `system.lakeflow.job_run_timeline` (365 days).

Invalid token format **Cause:** the token doesn't start with `dapi`, or it is malformed.

Fix: regenerate the personal access token under Settings > Developer > Access tokens, or use OAuth through a CLI profile.

Workspace URL with a trailing slash **Cause:** a doubled slash breaks the API path.

```
Wrong:   https://adb-xxx.azuredatabricks.net//api/2.2/jobs/list
Correct: https://adb-xxx.azuredatabricks.net/api/2.2/jobs/list
```

Exam Tips

Tip

- **Jobs API is 2.2** (Databricks recommends it for new and existing clients). The Pipelines API is `/api/2.0/pipelines`.
- **Monitor runs:** `list_runs` / `runs/list` / `jobs list-runs` with `job_id`, `active_only` or `completed_only`, `start_time_from` and `start_time_to` (epoch ms), and `run_type`.
- **Diagnose a run:** `get_run` (per-task `status`, durations, `include_history`) and then `get_run_output` with a **task** `run_id` (`error`, `error_trace`, `notebook_output.result`).
- **`status` replaces `state`.** `termination_details.code` tells you why the run ended.
- **Pipelines:** `get` (state, health, latest updates), `list_updates` / `get_update` (update states), and `list_pipeline_events` with `filter="level='ERROR'"`.
- **Pagination:** `page_token` / `next_page_token`. `limit` for runs/list is 1–25. The SDK auto-paginates.
- **Retention:** API run history lasts 60 days. Use `system.lakeflow` tables for 365-day account-wide analysis.
- **Authentication:** `Authorization: Bearer <token>`. The SDK and CLI use unified auth (profiles, env vars, notebook context).

Key Takeaways

- **Three interfaces, one API:** every SDK method maps to a REST endpoint and a CLI command (`w.jobs.list_runs` = `GET /api/2.2/jobs/runs/list` = `databricks jobs list-runs`).
- **Jobs monitoring fields:** `status.state`, `status.termination_details.{code,type,message}`, `start_time` and `end_time` (epoch ms), the duration fields (ms), `tasks[]`, and `run_page_url`.
- **Pipelines monitoring:** pipeline `state` and `health`, update `state` (`COMPLETED`, `FAILED`, `CANCELED`, ...), and events with `level`, `event_type`, and `message`.
- **The SDK** handles authentication, pagination, retries, and waiting (`run_now_and_wait`, `Wait.result()`), so it's the preferred way to script monitoring.

- **Rate limits are per endpoint and fixed** (for example, `runs/get` 100/s, `runs/list` 30/s, Pipelines GET 150/s). Back off on 429 responses.
- **Permission levels:** jobs use `CAN_VIEW` / `CAN_MANAGE_RUN` / `IS_OWNER` / `CAN_MANAGE`, and pipelines use `CAN_VIEW` / `CAN_RUN` / `CAN_MANAGE` / `IS_OWNER`.

Related Topics

- Databricks REST API — Part 1 - Authentication and the core Jobs, Clusters, DBFS, and Workspace APIs
- Databricks CLI — Part 1 - CLI installation, profiles, and jobs commands
- Databricks CLI — Part 2 - Output formats and scripting patterns
- System Tables - Account-wide job and pipeline history in `system.lakeflow`
- Lakeflow Event Logs - Event log schema behind `list_pipeline_events`
- Lakeflow Jobs — Part 2 - Jobs UI monitoring and notifications
- CI/CD Integration — Part 1 - Using the API in pipelines

Official Documentation

- Databricks REST API reference
- Jobs API: List job runs
- Jobs API: Get a single job run
- Updating from Jobs API 2.1 to 2.2
- Pipelines API: List pipeline events
- Pipelines API: List pipeline updates
- Databricks SDK for Python
- SDK reference: `w.jobs`
- SDK reference: `w.pipelines`
- CLI reference: jobs command group
- CLI reference: pipelines command group
- Resource limits (API rate limits)
- Access control lists
- Monitor Lakeflow Jobs

Streaming Monitoring & Optimization

Overview

This guide covers production streaming operations: back-pressure and rate limiting, monitoring and troubleshooting, state store internals, and performance tuning.

> For streaming join patterns and stateful processing, see Streaming Joins & Stateful Operations.

Overview

- **Back-pressure**: input rate above processing rate, or batch duration above the trigger interval; limit intake with `maxFilesPerTrigger` or `maxOffsetsPerTrigger`
- **Monitoring**: `StreamingQueryListener`, `query.lastProgress`, and `query.recentProgress`
- **Checkpoints**: compatibility rules and corruption recovery
- **State stores**: HDFS-backed vs RocksDB, and when to switch

What the Exam Tests

- Use Lakeflow Jobs UI and Jobs API to monitor job status and performance metrics.
- Read progress metrics such as `numRowsDroppedByWatermark` and state row counts to diagnose a stream
- Choose rate limits and a state store for a lagging stream

Back-Pressure and Rate Limiting

Rate Limiting Configuration Control how much data each micro-batch processes to prevent overwhelming downstream systems or running out of memory.

```
# Delta / Auto Loader rate limiting

stream = (
    spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema/events")
    # File-level rate limiting
    .option("maxFilesPerTrigger", 100)
    # Byte-level rate limiting
    .option("maxBytesPerTrigger", "10g")
    .load("/data/raw/events")
)

# Kafka rate limiting

kafka_stream = (
    spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "broker1:9092")
    .option("subscribe", "events")
    # Offset-level rate limiting
    .option("maxOffsetsPerTrigger", 500000)
    # Min partitions for parallelism
    .option("minPartitions", 10)
    .load()
)

# Delta table source rate limiting

delta_stream = (
    spark.readStream
    .format("delta")
    .option("maxFilesPerTrigger", 1000)
    .option("maxBytesPerTrigger", "5g")
    .load("/data/source_table")
)
```

Rate Limiting Parameters by Source

Source	Parameter	Default	Effect
Delta / Auto Loader	`maxFilesPerTrigger`	1000	Max files per micro-batch

Source	Parameter	Default	Effect
Delta / Auto Loader	`maxBytesPerTrigger`	None	Max bytes per micro-batch
Kafka	`maxOffsetsPerTrigger`	None (all)	Max records per micro-batch
Kafka	`minPartitions`	Kafka partitions	Min read partitions
Rate source	`rowsPerSecond`	1	Rows generated per second

Detecting Back-Pressure Back-pressure occurs when processing takes longer than the trigger interval, causing a growing backlog.

```
# Monitor back-pressure indicators

def check_backpressure(query):
    progress = query.lastProgress
    if progress is None:
        return

    input_rate = progress.get("inputRowsPerSecond", 0)
    process_rate = progress.get("processedRowsPerSecond", 0)
    batch_duration = progress.get("batchDuration", 0)
    trigger_ms = progress.get("triggerExecution", {}).get(
        "triggerIntervalMs", 0
    )

    print(f"Input rate:    {input_rate:.0f} rows/sec")
    print(f"Process rate:  {process_rate:.0f} rows/sec")
    print(f"Batch duration: {batch_duration} ms")

    if process_rate > 0 and input_rate > process_rate:
        print("WARNING: Back-pressure detected!")
        print(f"  Ratio: {input_rate / process_rate:.2f}x")

    if trigger_ms > 0 and batch_duration > trigger_ms:
        print("WARNING: Batch duration exceeds trigger interval!")
```

Back-Pressure Indicators

Metric	Healthy	Back-Pressure	Action
`inputRowsPerSecond` vs `processedRowsPerSecond`	Input <= Process	Input > Process	Reduce ba
Batch duration vs trigger interval	Duration < Interval	Duration > Interval	Increase t
`numInputRows` trend	Stable	Growing	Check sou
State size trend	Stable or bounded	Monotonically growing	Check wa

StreamingQueryListener for Monitoring

```
from pyspark.sql.streaming import StreamingQueryListener

class BackpressureListener(StreamingQueryListener):
    def onQueryStarted(self, event):
        print(f"Query started: {event.id}")

    def onQueryProgress(self, event):
        progress = event.progress
        input_rate = progress.inputRowsPerSecond
```

```
        process_rate = progress.processedRowsPerSecond

        # Log metrics to monitoring system
        if process_rate > 0 and input_rate > process_rate * 1.5:
            print(
                f"ALERT: Query {event.progress.name} "
                f"backpressure ratio: {input_rate / process_rate:.2f}"
            )

        # Check state operator metrics
        for op in progress.stateOperators:
            if op.numRowsTotal > 1000000:
                print(
                    f"ALERT: State size {op.numRowsTotal} rows "
                    f"for operator {op.operatorName}"
                )

    def onQueryTerminated(self, event):
        print(f"Query terminated: {event.id}")
        if event.exception:
            print(f"Exception: {event.exception}")

# Register the listener

spark.streams.addListener(BackpressureListener())
```

Streaming Monitoring and Troubleshooting

query.recentProgress Analysis The `recentProgress` attribute returns the last 100 progress reports, enabling trend analysis.

```
# Analyze recent progress for trends

progress_list = query.recentProgress

if progress_list:
    # Extract key metrics over time
    for p in progress_list[-5:]:  # Last 5 batches
        batch_id = p.get("batchId", "N/A")
        num_rows = p.get("numInputRows", 0)
        duration = p.get("batchDuration", 0)
        input_rate = p.get("inputRowsPerSecond", 0)
        process_rate = p.get("processedRowsPerSecond", 0)

        state_info = ""
        if p.get("stateOperators"):
            state_op = p["stateOperators"][0]
            state_rows = state_op.get("numRowsTotal", 0)
            state_mem = state_op.get("memoryUsedBytes", 0)
            dropped = state_op.get("numRowsDroppedByWatermark", 0)
            state_info = (
                f"  State: {state_rows} rows, "
                f"{state_mem / 1024 / 1024:.1f} MB, "
                f"{dropped} dropped"
            )
```

```
        print(
            f"Batch {batch_id}: {num_rows} rows, "
            f"{duration}ms, "
            f"in={input_rate:.0f}/s, "
            f"proc={process_rate:.0f}/s"
            f"{state_info}"
        )
```

Key Progress Metrics Reference

Metric Path	Description	What to Watch
numInputRows	Rows in this micro-batch	Sudden spikes or drops
inputRowsPerSecond	Ingestion rate	Should match source rate
processedRowsPerSecond	Processing throughput	Must keep up with input
batchDuration	Total batch time (ms)	Should be < trigger interv
stateOperators[].numRowsTotal	Total rows in state store	Should not grow unbound
stateOperators[].memoryUsedBytes	State memory usage	Approaching executor me
stateOperators[].numRowsDroppedByWatermark	Late events dropped	High = adjust watermark
sources[].startOffset	Where batch started reading	Verify expected offsets
sources[].endOffset	Where batch finished reading	Verify progress
sink.numOutputRows	Rows written to sink	Should match expected o

Common Failure Patterns

```
flowchart TD
    Failure[Streaming Failure] --> OOM[OutOfMemoryError]
    Failure --> Checkpoint[Checkpoint Issues]
    Failure --> Schema[Schema Mismatch]
    Failure --> Source[Source Unavailable]

    OOM --> OOM1["Unbounded state growth<br>Fix: Add watermarks"]
    OOM --> OOM2["Large micro-batches<br>Fix: Add rate limits"]
    OOM --> OOM3["Executor memory too small<br>Fix: Use RocksDB or larger nodes"]

    Checkpoint --> CP1["Checkpoint corruption<br>Fix: Start with new checkpoint"]
    Checkpoint --> CP2["Incompatible schema change<br>Fix: New checkpoint
↪   location"]
   Checkpoint --> CP3["Storage permission error<br>Fix: Check IAM / credentials"]

    Schema --> SC1["Source schema evolved<br>Fix: Use schema evolution options"]
    Schema --> SC2["Cast errors<br>Fix: Handle nulls, validate types"]

    Source --> SR1["Kafka broker down<br>Fix: Retry with failOnDataLoss=false"]
    Source --> SR2["File source empty<br>Fix: Check path, permissions"]
```

Checkpoint Corruption Recovery

```
# Step 1: Check checkpoint contents

dbutils.fs.ls("/checkpoints/my_query/")

# Expected: commits/, offsets/, sources/, state/, metadata

# Step 2: Check for corrupted state
```

```
try:
    state_df = spark.read.format("statestore").load(
        "/checkpoints/my_query/state/0"
    )
    print(f"State rows: {state_df.count()}")
except Exception as e:
    print(f"State corrupted: {e}")

# Step 3: Recovery options

# Option A: Resume from last valid checkpoint (automatic)
# Spark will attempt this by default on restart

# Option B: Start fresh (loses exactly-once guarantee during transition)
# Use only if checkpoint is unrecoverable

dbutils.fs.rm("/checkpoints/my_query/", recurse=True)

# Restart query with same checkpoint path

# Option C: Start from specific offset (Kafka)

query = (
    df.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoints/my_query_v2/")
    .start("/output/table")
)
```

Checkpoint Compatibility Rules

Change Type	Compatible?	Action Required
Add new column to select	Yes	Resume from existing checkpoint
Remove column from select	Yes	Resume from existing checkpoint
Change filter condition	Yes	Resume from existing checkpoint
Change groupBy keys	**No**	New checkpoint location required
Change watermark delay	**No**	New checkpoint location required
Change join condition	**No**	New checkpoint location required
Change state schema	**No**	New checkpoint location required
Add new stateful operator	**No**	New checkpoint location required
Change output mode	**No**	New checkpoint location required

Monitoring Dashboard Checklist

```
# Production monitoring script

def streaming_health_check():
    """Run periodic health checks on all active streams."""
    for query in spark.streams.active:
        progress = query.lastProgress
        if not progress:
            continue
```

```
        name = query.name or query.id

        # Check 1: Is batch duration reasonable?
        duration = progress.get("batchDuration", 0)
        if duration > 60000:  # > 60 seconds
            print(f"WARN [{name}]: Batch duration {duration}ms > 60s")

        # Check 2: Is state growing unboundedly?
        for op in progress.get("stateOperators", []):
            total_rows = op.get("numRowsTotal", 0)
            if total_rows > 5000000:
                print(
                    f"ALERT [{name}]: State has {total_rows} rows"
                )

        # Check 3: Are we keeping up with input?
        input_rate = progress.get("inputRowsPerSecond", 0)
        process_rate = progress.get("processedRowsPerSecond", 0)
        if process_rate > 0 and input_rate > process_rate * 2:
            print(
                f"ALERT [{name}]: Input rate {input_rate:.0f}/s "
                f">> process rate {process_rate:.0f}/s"
            )

        # Check 4: Any exceptions?
        exc = query.exception()
        if exc:
            print(f"ERROR [{name}]: {exc}")

streaming_health_check()
```

State Store Deep Dive

HDFS vs RocksDB State Store

```
flowchart TD
    subgraph HDFS["HDFS State Store (Default)"]
        H1[All state in JVM heap]
        H2[Versioned as Delta-like files]
        H3[Fast for small state]
        H4[OOM risk with large state]
    end

    subgraph RocksDB["RocksDB State Store"]
        R1[State on local SSD]
        R2[Only hot data in memory]
        R3[Handles GB+ of state]
        R4[Slight latency overhead]
    end

    Decision{State Size?} -->|< 100 MB| HDFS
    Decision -->|> 100 MB| RocksDB
```

RocksDB Configuration

```
# Enable RocksDB state store

spark.conf.set(
    "spark.sql.streaming.stateStore.providerClass",
    "com.databricks.sql.streaming.state.RocksDBStateStoreProvider"
)

# RocksDB tuning options

spark.conf.set(
    "spark.sql.streaming.stateStore.rocksdb.compactOnCommit", "true"
)
spark.conf.set(
    "spark.sql.streaming.stateStore.rocksdb.changelogCheckpointing.enabled",
    "true"
)

# Changelog checkpointing: reduces checkpoint time by writing
# only changes (deltas) instead of full snapshots
# Recommended for large state stores
```

State Store Selection Guide

Factor	HDFS (Default)	RocksDB
State size	< 100 MB	100 MB to 100+ GB
Memory usage	All in heap	Spills to disk
Checkpoint speed	Fast (small state)	Fast with changelog
Recovery time	Fast	Slightly slower
Key count	< 1M keys	Millions of keys
Recommended for	Simple aggregations	Stream-stream joins, large dedup

Use Cases

- **Large Kafka or file backlog**: cap intake per micro-batch with `maxOffsetsPerTrigger` (Kafka) or `maxFilesPerTrigger` (files) so the stream catches up without overloading executors
- **Production stream alerting**: register a `StreamingQueryListener` before starting the query to push progress and termination events to monitoring
- **Large stateful aggregation**: switch to the RocksDB state store, with changelog checkpointing, when state exceeds about 100 MB or has millions of keys
- **Incompatible query change**: start a new, versioned checkpoint location (for example `/checkpoints/query_v2/`) when changing groupBy keys, watermark delay, or output mode

Common Issues & Errors

Stream-Stream Join State Explosion **Scenario:** Two streams joined without time range condition. State grows without bound even with watermarks defined.

Fix: Always include a time range condition in the join predicate that bounds how long events from each side need to be retained.

Exam Context: Questions often test whether you know that watermarks alone are not sufficient for stream-stream joins -- a time range condition in the join predicate is also required for effective state cleanup.

Outer Join Producing No NULL Results **Scenario:** Left outer stream-stream join never emits rows with NULL right-side columns. The watermark on the right stream is too short, or the time range condition is too narrow.

Fix: Ensure the right-side watermark delay is large enough to allow reasonable matching time. NULL-padded rows emit only after the watermark guarantees no match can arrive.

mapGroupsWithState Timeout Not Firing **Scenario:** State timeout configured with `ProcessingTimeTimeout` but `hasTimedOut` never returns true.

Fix: Timeouts only fire when there is data to process. If no new events arrive for a group, the timeout callback will not be invoked until the next micro-batch that has at least some data for any group. Ensure the stream receives regular input.

Global Watermark Too Conservative **Scenario:** Using "min" watermark policy with two streams where one is much slower. State grows excessively because the effective watermark is held back by the slow stream.

Fix: Consider switching to "max" policy if some late data loss is acceptable. Alternatively, ensure both streams have similar data arrival rates.

Dedup State OOM After Hours of Running **Scenario:** Using `dropDuplicates` without watermark or with a very wide watermark. State grows until executor runs out of memory.

Fix: Use `dropDuplicatesWithinWatermark` with an appropriately sized watermark, or switch to RocksDB state store for large state.

Checkpoint Incompatible After Adding Aggregation **Scenario:** Adding a `groupBy` aggregation to an existing streaming query and trying to resume from the old checkpoint.

Fix: Any change to stateful operators requires a new checkpoint location. Start the modified query with a fresh checkpoint path.

StreamingQueryListener Not Receiving Events **Scenario:** Registered listener does not receive `onQueryProgress` events.

Fix: Ensure the listener is registered before the query starts. Verify the listener class extends `StreamingQueryListener` and implements all three methods (`onQueryStarted`, `onQueryProgress`, `onQueryTerminated`).

Best Practices

- Always define watermarks on both sides of stream-stream joins with tight time range conditions
- Use `dropDuplicatesWithinWatermark` instead of `dropDuplicates` for streaming deduplication
- Choose RocksDB state store when state exceeds 100 MB or has high key cardinality
- Enable changelog checkpointing for RocksDB to reduce checkpoint overhead
- Implement `StreamingQueryListener` for production monitoring and alerting
- Set appropriate rate limits (`maxFilesPerTrigger`, `maxOffsetsPerTrigger`) to prevent backpressure
- Monitor `stateOperators.numRowsTotal` to detect unbounded state growth
- Use `flatMapGroupsWithState` only when you need zero-or-many output semantics; prefer built-in aggregations when possible
- Test stateful logic with small datasets and the `rate` source before connecting production sources

- Keep checkpoint locations versioned (e.g., `/checkpoints/query_v2/`) when making incompatible query changes

Exam Tips

1. **Stream-stream joins require watermarks on both sides** for inner joins, and on the opposite side for outer joins (left outer needs right watermark, right outer needs left watermark)
2. **Time range conditions are essential** in stream-stream join predicates for bounding state; watermarks alone are not sufficient for state cleanup
3. **Stream-static joins re-read the static side** each micro-batch; no watermark is needed on the static side
4. **`mapGroupsWithState`** emits exactly one output per group; **`flatMapGroupsWithState`** emits zero or more
5. **ProcessingTimeTimeout** is based on wall clock time; **EventTimeTimeout** is based on the watermark advancing past a set timestamp
6. **Global watermark policy "min"** (default) is conservative and retains more state; **"max"** is aggressive and drops more late data
7. **Events at exactly the watermark boundary are included** (not dropped); the condition is `event_time >= watermark`
8. **`dropDuplicatesWithinWatermark`** is more memory-efficient than `dropDuplicates` because it only tracks keys within the watermark window
9. **Checkpoint compatibility**: changing groupBy keys, watermark delay, join conditions, state schema, or output mode requires a **new checkpoint location**
10. **Back-pressure** is indicated when `inputRowsPerSecond` exceeds `processedRowsPerSecond` or batch duration exceeds the trigger interval; use `maxOffsetsPerTrigger` (Kafka) or `maxFilesPerTrigger` (files) to control intake

Key Takeaways

- **Back-pressure detection**: `inputRowsPerSecond > processedRowsPerSecond` or batch duration exceeding the trigger interval are the primary signals; use `maxFilesPerTrigger` (files) or `maxOffsetsPerTrigger` (Kafka) to limit intake
- **`StreamingQueryListener`** provides event-driven monitoring for production streams; implement all three methods (`onQueryStarted`, `onQueryProgress`, `onQueryTerminated`) and register before starting the query
- **Checkpoint compatibility**: changes to groupBy keys, watermark delay, join conditions, state schema, or output mode are incompatible with an existing checkpoint and require a new checkpoint location
- **RocksDB state store** is recommended when state exceeds ~100 MB or has millions of keys; it spills to local SSD instead of keeping all state in JVM heap; enable changelog checkpointing to reduce checkpoint write overhead
- **Stream-stream join state explosion** occurs when no time range condition is included in the join predicate; watermarks alone do not bound state — a time range condition is also required
- **`numRowsDroppedByWatermark`** in `stateOperators` progress metrics indicates late events being discarded; a high value suggests the watermark delay should be increased
- **Global watermark policy `min`** (default) can cause excessive state retention when one stream is much slower than another; switching to `max` reduces state but increases late-data loss
- **`query.lastProgress`** and `query.recentProgress` are the primary APIs for monitoring batch metrics, state size, throughput, and lag in production streaming queries

Related Topics

- Structured Streaming - Streaming fundamentals, triggers, output modes, basic watermarking

- Auto Loader - File ingestion with schema inference and evolution
- Performance Optimization - Spark tuning for streaming workloads
- Data Deduplication - Batch and streaming dedup patterns
- Streaming Optimization - File sizing, trigger tuning, checkpoint optimization

Official Documentation

- Stream-Stream Joins
- Stateful Streaming
- Watermarks
- RocksDB State Store
- Streaming Deduplication
- StreamingQueryListener
- Structured Streaming Guide

Section 5: Cost & Performance Optimization (15 % of Exam)

Making tables and queries cheaper and faster: Unity Catalog managed tables with Predictive Optimization, Liquid Clustering and `CLUSTER BY AUTO`, deletion vectors, file sizing, the disk (Delta) cache, Change Data Feed for incremental downstream work, the query profile, and Spark tuning. Up from 13 % to 15 % in the Oct 9, 2026 guide.

Topics Overview

```
flowchart LR
    D[Cost & Performance]
    D --> T1["Managed Tables, Predictive Optimization, and Automatic Clustering"]
    D --> T2["File Sizing and Compaction"]
    D --> T3["Data Skipping: Liquid Clustering vs Partitioning and Z-ORDER"]
    D --> T4["Disk Caching"]
    D --> T5["Query Profile"]
    D --> T6["Spark Tuning"]
    D --> T7["EXPLAIN Plans & Adaptive Query Execution"]
    D --> T8["Photon, Diagnostics & Query Optimization"]
    D --> T9["Streaming Performance Optimization"]
    D --> T10["Cost Optimization"]
```

Section Contents

File	Topic
01-managed-tables-predictive-optimization.md	UC managed tables, Predictive Optimization, Liquid Clusterin
02-file-sizing.md	Small-file problem, OPTIMIZE, auto compaction, optimized w
03-zorder-indexing.md	Data skipping: Liquid Clustering vs partitioning and Z-ORDE
04-disk-caching.md	Disk cache (Delta cache) vs Spark cache; SQL warehouse ca
05-query-profiler.md	Query profile: pruning, join strategies, shuffle and spill
06-spark-tuning.md	Shuffle partitions, AQE, join optimization, memory
07-explain-plans-aqe.md	Reading EXPLAIN plans; Adaptive Query Execution
08-photon-diagnostics-optimization-part1.md	Photon, memory and spill diagnostics
08-photon-diagnostics-optimization-part2.md	Query optimization strategies
09-streaming-optimization.md	Trigger intervals, state, checkpoint and source tuning
10-cost-optimization.md	Compute cost levers: job vs all-purpose, spot, autoscaling, p

Exam Skill Coverage

Every skill bullet in Section 5 of the Oct 9, 2026 exam guide, with the files that cover it.

Exam guide skill
Explain how Unity Catalog managed tables, Predictive Optimization, and Liquid Clustering reduce operational
Choose the appropriate Delta optimization technique (deletion vectors, Liquid Clustering, CLUSTER BY AUTO) f
Understand how caching (Delta cache) improves query performance for repeated reads of the same data.
Apply Change Data Feed (CDF) to expose row-level changes (updates/deletes) for efficient incremental downs
Use the query profile to identify performance bottlenecks, such as data-skipping inefficiencies, join strategies,
Compare Liquid Clustering vs partitioning/ZORDER for a given table size and query pattern.

Key Concepts to Master

Concept	Why it matters
Predictive Optimization	Databricks runs OPTIMIZE, VACUUM and statistics collection on UC m
Liquid Clustering / `CLUSTER BY AUTO`	Incremental, changeable clustering keys; with AUTO, Databricks pick
Deletion vectors	Mark deleted / updated rows without rewriting whole files — cuts wri
Disk cache	Local-SSD copies of remote Parquet / Delta files that speed up repea
Change Data Feed	Row-level inserts / updates / deletes readable with `table_changes` o

Related Resources

- Performance optimization cheat sheet (shared)
- Spark configurations cheat sheet (shared)
- Delta Lake cheat sheet

Managed Tables, Predictive Optimization, and Automatic Clustering

Overview

Unity Catalog managed tables hand storage, layout, and maintenance to the platform. Predictive optimization then runs `OPTIMIZE`, `VACUUM`, and `ANALYZE` for you. Liquid clustering (manual keys or `CLUSTER BY AUTO`) and deletion vectors decide how data is laid out and how cheaply rows can change. Together they remove most of the hand-written maintenance jobs that older Delta pipelines needed.

Overview

- **UC managed tables** are the default and recommended table type; some features (predictive optimization, automatic liquid clustering, metadata caching, auto file deletion after `DROP`) only exist for managed tables
- **Predictive optimization** runs `OPTIMIZE`, `VACUUM`, and `ANALYZE` on serverless jobs compute, controlled by an account → catalog → schema → table inheritance model
- **Liquid clustering** (`CLUSTER BY`) replaces partitioning and `ZORDER`; **`CLUSTER BY AUTO`** lets predictive optimization pick and evolve the keys from the query workload
- **Deletion vectors** mark changed rows in metadata instead of rewriting whole Parquet files, which cuts write amplification from frequent small `MERGE` / `UPDATE` / `DELETE`

What the Exam Tests

- Explain how Unity Catalog managed tables, Predictive Optimization, and Liquid Clustering reduce operational and maintenance overhead for a given workload
- Choose the appropriate Delta optimization technique (deletion vectors, Liquid Clustering, `CLUSTER BY AUTO`) for a given table access pattern
- Know the requirements and defaults (managed tables only, predictive optimization required for `CLUSTER BY AUTO`, deletion vectors need compatible clients)

The Maintenance Work These Features Remove

Manual task on an older Delta table	Who does it on a UC managed table
Scheduled `OPTIMIZE` jobs for compaction	Predictive optimization (`OPTIMIZE`,
Scheduled `VACUUM` jobs	Predictive optimization (`VACUUM`, ho

Manual task on an older Delta table	Who does it on a UC managed table
`ANALYZE TABLE` for statistics	Predictive optimization (`ANALYZE`, p
Choosing partition / `ZORDER` columns and rewriting when queries change	Liquid clustering; with `CLUSTER BY`
Rewriting whole files for a handful of updated rows	Deletion vectors (soft-deletes resol
Deleting files from cloud storage after `DROP TABLE`	Automatic for managed tables after

```
flowchart LR
    MT[UC managed table] --> PO[Predictive optimization]
    PO --> OPT[OPTIMIZE<br/>compaction + incremental clustering]
    PO --> VAC[VACUUM<br/>remove unreferenced files]
    PO --> ANA[ANALYZE<br/>statistics]
    PO --> AUTO[CLUSTER BY AUTO<br/>key selection]
    MT --> DV[Deletion vectors<br/>cheap row-level changes]
```

Unity Catalog Managed Tables

Unity Catalog managed tables are the default and recommended table type for Delta Lake and Apache Iceberg. Unity Catalog manages all read, write, storage, and optimization responsibilities, and data files live in the managed storage location of the containing schema or catalog.

Features Only Managed Tables Get

Feature	What it does
Predictive optimization	Automatically runs maintenance operations
Automatic liquid clustering	Selects and updates clustering keys as query patterns change (nee
Metadata caching	In-memory caching of transaction metadata, fewer requests to the
Automatic file deletion after `DROP TABLE`	Data files are deleted after the recovery period expires
Catalog commits	Commit coordination in Unity Catalog: multi-table transactions, fas
Multi-statement transactions	Multiple statements as one atomic commit

External tables do not get these features. For external tables you also have to delete files from your storage bucket yourself after a drop.

Access by Name, Not Path All reads and writes to managed tables use the three-level name (`catalog.schema.table`). Path-based access to managed tables isn't supported (except in Compatibility Mode), because it bypasses Unity Catalog access controls.

```
-- Managed Delta table (Delta is the default format)
CREATE TABLE main.sales.orders (
  order_id BIGINT,
  customer_id BIGINT,
  order_ts TIMESTAMP,
  amount DECIMAL(10, 2)
);

-- Managed Iceberg table: you must say USING iceberg
CREATE TABLE main.sales.orders_iceberg (order_id BIGINT) USING iceberg;
```

Dropping and Recovering

- `UNDROP TABLE` recovers a dropped managed table; by default tables are recoverable for 7 days.
- After the recovery period, an asynchronous purge deletes the underlying data files.
- A configurable recovery period (Public Preview) can be set at catalog or schema level: 0 hours (no recovery) or 7 to 30 days; the schema setting wins if both are set.

Converting External Tables to Managed Databricks recommends `ALTER TABLE ... SET MANAGED` over CTAS: it keeps the table name, settings, permissions, views, and history, and keeps reader/writer downtime short.

```
-- External Delta table -> UC managed table
-- Requires Databricks Runtime 17.3 LTS+ or serverless compute
ALTER TABLE main.legacy.events SET MANAGED;
```

Note

Foreign tables use a different form (`SET MANAGED {MOVE | COPY}`). Adding `MOVE` or `COPY` when converting an external table makes the command fail.

Predictive Optimization

Predictive optimization automatically runs `OPTIMIZE`, `VACUUM`, and `ANALYZE` on Unity Catalog managed tables (Delta Lake and Iceberg). It finds tables that would benefit from maintenance, queues the operations, and also collects statistics when data is written. Databricks recommends it for all UC managed tables.

What It Runs

Operation	Effect
`OPTIMIZE`	Triggers incremental clustering for clustered tables; optimizes file sizes
`VACUUM`	Deletes data files no longer referenced by the table (storage cost)
`ANALYZE`	Collects statistics to improve query performance
Key selection	With automatic liquid clustering, may select new clustering keys before clustering data

Important

`OPTIMIZE` run by predictive optimization **does not run `ZORDER`**, and it ignores Z-ordered files. Tables that rely on Z-order do not get their Z-order maintained automatically, which is one more reason to move them to liquid clustering.

Compute, Billing, and Prerequisites

- Runs on **serverless compute for jobs**; billed under a **serverless jobs SKU**.
- Workspace on the **Premium plan or above** in a supported region.
- Use **SQL warehouses or Databricks Runtime 12.2 LTS or above**.
- **Only Unity Catalog managed tables** are supported. It does not run on **external tables** or on **tables loaded as OpenSharing recipients**.

Enablement Hierarchy You can enable predictive optimization for an account, a catalog, a schema, or a table. All UC managed tables inherit the account value by default, and each lower level can override it.

```
flowchart TD
    A[Account<br/>account console: Settings > Feature enablement] -->
 ↪  C[Catalog<br/>ENABLE / DISABLE / INHERIT]
    C --> S[Schema<br/>ENABLE / DISABLE / INHERIT]
    S --> T[Table<br/>ENABLE / DISABLE / INHERIT]
```

```
-- Override at catalog, schema, or table level
ALTER CATALOG main ENABLE PREDICTIVE OPTIMIZATION;
ALTER SCHEMA main.staging DISABLE PREDICTIVE OPTIMIZATION;
ALTER TABLE main.sales.orders INHERIT PREDICTIVE OPTIMIZATION;  -- go back to the
 ↪  parent's setting

-- Check the effective setting (the "Predictive Optimization" field shows if it
 ↪  is inherited)
DESCRIBE SCHEMA EXTENDED main.staging;
DESCRIBE TABLE EXTENDED main.sales.orders;
```

Level	Who can change it
Account	Account admin
Catalog	Catalog owner, or `MANAGE` on the catalog
Schema	Schema owner, or `MANAGE` on the schema
Table	Table owner, or `MANAGE` on the table

Two inheritance rules that show up in scenarios:

- Disabling at the account level does **not** disable it for catalogs or schemas that explicitly enabled it.
- An object that explicitly disabled it stays blocked even if the account is later enabled.

Observing What It Did

```
-- Operation history, cost (ESTIMATED_DBU), and status per table
SELECT catalog_name, schema_name, table_name, operation_type,
       SUM(usage_quantity) AS estimated_dbus,
       COUNT(*) AS operations
FROM system.storage.predictive_optimization_operations_history
WHERE start_time >= current_date() - INTERVAL 30 DAYS
  AND operation_status = 'SUCCESSFUL'
GROUP BY ALL
ORDER BY estimated_dbus DESC;
```

- `operation_type` values include `COMPACTION`, `VACUUM`, `ANALYZE`, `CLUSTERING`, and `AUTO_CLUSTERING_COLUM`
- The system table is in Public Preview; records are typically available within 12 hours.
- On Databricks Runtime 18 LTS and above, `DESCRIBE TABLE EXTENDED ... AS JSON` returns a `predictive_optimization_evaluations` field with the latest evaluation (and skip reason) per operation type. The Catalog Explorer **History** tab marks skipped operations as **Not applied**.

VACUUM Retention Still Matters The `VACUUM` retention window comes from `delta.deletedFileRetentionI` (default 7 days). To keep longer time travel, set it **before** enabling predictive optimization:

```sql
ALTER TABLE main.sales.orders
SET TBLPROPERTIES ('delta.deletedFileRetentionDuration' = '30 days');
```

If you set it below 7 days, predictive optimization still keeps files for at least 7 days when it runs `VACUUM FULL`.

Liquid Clustering

Liquid clustering is a data layout technique that **replaces table partitioning and ZORDER**. It organizes data by clustering keys, and you can redefine keys without rewriting existing data. Databricks recommends liquid clustering for **all new tables, including streaming tables and materialized views**. It is not compatible with partitioning or ZORDER on the same table.

Liquid clustering is generally available for Delta Lake tables with Databricks Runtime 15.4 LTS and above (Public Preview for Iceberg tables on 16.4 LTS and above). Triggering clustering needs Databricks Runtime 13.3 LTS or above.

When It Helps Most

- Queries that filter on high-cardinality columns
- Tables with heavy data skew
- Fast-growing tables that would otherwise need maintenance and tuning
- Tables with concurrent write requirements
- Tables with varied or changing access patterns
- Tables where a typical partition key would give too many or too few partitions

Syntax

```sql
-- New table
CREATE TABLE main.sales.orders (
  order_id BIGINT, customer_id BIGINT, order_date DATE, amount DECIMAL(10, 2)
) CLUSTER BY (customer_id, order_date);

-- CTAS: CLUSTER BY goes after the table name, not in the SELECT
CREATE TABLE main.sales.orders_copy CLUSTER BY (customer_id)
AS SELECT * FROM main.sales.orders;

-- Existing unpartitioned table
ALTER TABLE main.sales.orders CLUSTER BY (customer_id, order_date);

-- Change keys at any time (existing data is not rewritten)
ALTER TABLE main.sales.orders CLUSTER BY (order_date);

-- Turn clustering off (clustered data is not rewritten)
ALTER TABLE main.sales.orders CLUSTER BY NONE;

-- See the current keys
DESCRIBE DETAIL main.sales.orders;
```

```python
# DataFrame API: clustering keys can only be set on create / overwrite, not on
 ↪  append
(spark.read.table("main.sales.orders")
    .write.clusterBy("customer_id")
    .saveAsTable("main.sales.orders_by_customer"))
```

Choosing Keys

- Up to **four** clustering keys; pick the columns most often used in query filters. Order does not matter.
- If two columns are highly correlated, include only one of them.
- Keys must have **statistics collected** (by default the first 32 columns of the schema).
- For tables under about 10 TB, more keys can make single-column filters slower (four keys performs worse than two); the gap becomes negligible as tables grow.
- Supported key types include date, timestamp, string, integer types, float, double, and decimal; struct fields work with dot notation, but map/array elements and complex types do not.

How Data Actually Gets Clustered Clustering happens in two places: on write (for some operations, above a size threshold) and during `OPTIMIZE`.

Clustering keys	Clustering-on-write threshold (UC managed)	Threshold (other Delta tables)
1	64 MB	256 MB
2	256 MB	1 GB
3	512 MB	2 GB
4	1 GB	4 GB

- Operations that cluster on write include `INSERT INTO`, CTAS/RTAS, `COPY INTO` from Parquet, and `spark.write.mode("append")`. Not all operations cluster on write, so `OPTIMIZE` still matters.
- `OPTIMIZE` on a clustered table is **incremental**: it only rewrites data that needs clustering, so most runs are quick.
- With predictive optimization, Databricks recommends **disabling scheduled `OPTIMIZE` jobs**. Without it, schedule `OPTIMIZE`; for tables with many updates or inserts, every one or two hours.
- After you **enable clustering for the first time or change keys**, run `OPTIMIZE <table> FULL` to recluster existing data (Databricks Runtime 16.4 LTS and above per the liquid clustering page). On large tables that were never clustered on those keys, this can take hours.

```
OPTIMIZE main.sales.orders;        -- incremental clustering
OPTIMIZE main.sales.orders FULL;   -- recluster everything after a key change
```

Protocol and Concurrency Side Effects

- Tables with liquid clustering use Delta **writer version 7 and reader version 3**; clients that don't support these protocols cannot read them, and the protocol cannot be downgraded.
- Enabling liquid clustering turns on deletion vectors and row tracking by default. Together they give **row-level concurrency**, so concurrent writes conflict less. Turning either off also turns off row-level concurrency, and `DELETE`, `MERGE`, and `UPDATE` might run slower.

```
-- Optional: opt out of deletion vectors before enabling clustering (for old
 ↪  clients)
ALTER TABLE main.sales.orders SET TBLPROPERTIES ('delta.enableDeletionVectors' =
 ↪  false);
ALTER TABLE main.sales.orders CLUSTER BY (customer_id);
```

Automatic Liquid Clustering (`CLUSTER BY AUTO`)

With `CLUSTER BY AUTO`, Databricks chooses the clustering keys for you. It is available for **Unity Catalog managed Delta tables on Databricks Runtime 15.4 LTS and above** (managed Ice-

berg v3 tables need 18.0 and above; not available for managed Iceberg v2). It also works for materialized views and streaming tables in Lakeflow Pipelines.

How Keys Are Chosen Automatic liquid clustering **requires predictive optimization** for key selection and clustering, and runs asynchronously:

- **Analyzes the query workload**: looks at the table's historical queries to find the best candidate columns.
- **Adapts to changes**: selects new keys if query patterns or data distributions change.
- **Cost-aware**: changes keys only when the predicted savings from better data skipping outweigh the cost of reclustering.

It might not select keys when the table is too small, already has an effective layout (from earlier manual keys or a natural insertion order that matches queries), isn't queried often, or the workloads aren't on Databricks Runtime 15.4 LTS or above (key selection relies on metadata introduced in 15.4 LTS).

Syntax

```
-- New table
CREATE OR REPLACE TABLE main.sales.events (event_id BIGINT, event_ts TIMESTAMP,
 ↪  user_id BIGINT)
CLUSTER BY AUTO;

-- Existing table, including one with manual keys
ALTER TABLE main.sales.orders CLUSTER BY AUTO;

-- Give hints first, then switch to AUTO
ALTER TABLE main.sales.orders CLUSTER BY (customer_id, order_date);
ALTER TABLE main.sales.orders CLUSTER BY AUTO;

-- Convert a partitioned table (DBR 18.1+); starts from the partition columns
ALTER TABLE main.sales.legacy_orders REPLACE PARTITIONED BY WITH CLUSTER BY AUTO;

-- Turn it off (or set explicit keys instead)
ALTER TABLE main.sales.orders CLUSTER BY NONE;
```

```
# DataFrameWriter (DBR 16.4+): only when creating or replacing a table
(df.write
    .format("delta")
    .option("clusterByAuto", "true")
    .saveAsTable("main.sales.events"))
```

```
# Lakeflow Pipelines (pyspark.pipelines)
from pyspark import pipelines as dp

@dp.table(cluster_by_auto=True)
def events_bronze():
    return spark.readStream.table("main.raw.events")
```

Check the result with `DESCRIBE TABLE` or `SHOW TBLPROPERTIES`: `clusterByAuto` is `true`, and `clusteringColumns` shows the keys currently selected.

Warning

Running `CREATE OR REPLACE TABLE` **without** `CLUSTER BY AUTO` on a table that had automatic clustering turns it off and does not keep the selected columns. Include `CLUSTER BY AUTO` in

the replace statement to keep it.

Deletion Vectors

Without deletion vectors, changing one row means rewriting the whole Parquet file that contains it. Deletion vectors mark rows as modified in metadata instead, and **reads apply the deletion vector entries at query time** to resolve the current table state (a merge-on-read pattern). Updated rows are written to new files; the old versions are marked as soft-deleted.

```
flowchart LR
    subgraph Without["Without deletion vectors"]
        W1[MERGE updates 3 rows] --> W2[Rewrite every touched<br/>Parquet file in full]
    end
    subgraph With["With deletion vectors"]
        D1[MERGE updates 3 rows] --> D2[Write small DV marking<br/>old rows + new rows file]
        D2 --> D3[Readers skip marked rows]
        D3 --> D4[OPTIMIZE / REORG PURGE<br/>rewrites files later]
    end
```

When Deletion Vectors Help

Pattern	Effect
Frequent small MERGE, UPDATE, or DELETE on large tables (CDC upserts, GDPR deletes)	Big win: avoids rewrit
Concurrent writers on the same table	Row-level concurrency
Append-only tables	Little benefit; nothing
Tables read by external clients without deletion-vector support	Avoid, or drop the fea

On Photon compute (serverless and pro SQL warehouses, and clusters on DBR 14.0+), **predictive I/O for updates** uses deletion vectors automatically to speed up DELETE, MERGE, and UPDATE.

Enabling Them

```
CREATE TABLE main.cdc.customers (id BIGINT, name STRING)
TBLPROPERTIES ('delta.enableDeletionVectors' = true);

ALTER TABLE main.cdc.customers SET TBLPROPERTIES ('delta.enableDeletionVectors' =
 ↪  true);
```

- The **Auto-Enable Deletion Vectors** workspace setting controls whether new Delta tables get them by default (options: *New UC managed and Databricks SQL tables*, *All new tables*, *Disabled*, or *Default*, which varies by workspace/region). It applies to tables created with SQL warehouses and DBR 14.0+.
- You can't use ALTER to enable or remove deletion vectors on a materialized view or streaming table; use CREATE.
- Enabling them **upgrades the table protocol**: clients without deletion-vector support can no longer read the table.

Client	Write support	Read support
Databricks Runtime with Photon	MERGE, UPDATE, DELETE on 12.2 LTS+	12.2 LTS+

Client	Write support	Read support
Databricks Runtime without Photon	DELETE 12.2 LTS+, UPDATE 14.1+, MERGE 14.3 LTS+	12.2 LTS+
OSS Spark + OSS Delta Lake	DELETE on Delta 2.4.0+, UPDATE on 3.0.0+	Delta 2.3.0+
OpenSharing recipients	Not supported	DBR 14.1+, or del

Physically Applying the Changes Soft-deleted rows stay in the old Parquet files until something rewrites them:

```
OPTIMIZE main.cdc.customers;                    -- compaction rewrites files with
 ↪  DVs (no strict guarantee for every file)
REORG TABLE main.cdc.customers APPLY (PURGE);   -- rewrites all files that
 ↪  contain soft-deleted rows
-- Auto compaction on write also rewrites files that carry deletion vectors
```

To **physically remove** deleted data (storage cost, GDPR), run `REORG TABLE ... APPLY (PURGE)` and then `VACUUM` so the old files from earlier versions are deleted.

To restore compatibility with other clients, drop the feature with `ALTER TABLE <table> DROP FEATURE deletionVectors` (Databricks recommends Databricks Runtime 16.3 and above for `DROP FEATURE`).

Choosing a Technique for an Access Pattern

Access pattern	Best fit
Large table, frequent small `MERGE` / `UPDATE` / `DELETE` of individual rows	**Deletion vectors**
Filters on a few known columns (high or low cardinality), stable query patterns	**Liquid clustering** C
UC managed table, many consumers with shifting or unknown filters	**`CLUSTER BY AUTO` +**
Fast-growing or skewed table where a partition key would create tiny or huge partitions	**Liquid clustering** (
Many concurrent writers hitting the same table	Liquid clustering (de
Team spends time on scheduled `OPTIMIZE` / `VACUUM` / `ANALYZE` jobs	**UC managed table**
External table or table read by clients without DV / writer v7 support	Keep it simple: avoi

```
flowchart TD
    Start[Table access pattern] --> W{Frequent small<br/>row-level changes?}
    W -->|Yes| DV[Enable deletion vectors]
    W -->|No| R{Mostly filtered reads?}
    DV --> R
    R -->|Yes| M{UC managed + predictive<br/>optimization on?}
    M -->|Yes, patterns vary or unknown| AUTO[CLUSTER BY AUTO]
    M -->|Known, stable filter columns| LC[CLUSTER BY cols]
    M -->|No, external table| LC2[CLUSTER BY cols<br/>+ scheduled OPTIMIZE]
    R -->|No, append-only scans| PO[Managed table +<br/>predictive optimization]
```

Use Cases

- **CDC upserts into a large silver table**: hourly `MERGE` touches a few thousand rows spread across many files. Deletion vectors stop each `MERGE` from rewriting those files in full; predictive optimization compacts and purges later.
- **Shared gold table with changing consumers**: analysts filter by region one quarter and by product the next. `CLUSTER BY AUTO` on a managed table lets predictive optimization change keys when it pays off, so no one has to re-plan partitioning.

- **Retiring maintenance jobs**: a team migrates external tables with `ALTER TABLE ... SET MANAGED`, confirms predictive optimization is inherited from the catalog, and deletes their nightly `OPTIMIZE`/`VACUUM` notebooks. They then use `system.storage.predictive_optimization_operat` for cost reporting.
- **GDPR erasure**: `DELETE` is fast with deletion vectors, but compliance needs the bytes gone, so the job follows with `REORG TABLE ... APPLY (PURGE)` and `VACUUM`.

Common Issues & Errors

Predictive Optimization Not Running on a Table **Cause:** the table is external, shared in through OpenSharing, in a schema/catalog with `DISABLE`, or the workspace is not on Premium / a supported region.

Fix: check `DESCRIBE TABLE EXTENDED` for the inherited value, convert external tables with `SET MANAGED`, and check skip reasons (Catalog Explorer History tab, **Not applied** rows).

Z-Order Degrades After Enabling Predictive Optimization **Cause:** predictive optimization's `OPTIMIZE` does not run `ZORDER` and ignores Z-ordered files.

Fix: move the table to liquid clustering (use the `ZORDER BY` columns as clustering keys, or `CLUSTER BY AUTO`).

`CLUSTER BY AUTO` Never Picks Keys **Cause:** table too small, already well laid out, rarely queried, predictive optimization disabled, or queries come from runtimes older than 15.4 LTS.

Fix: confirm predictive optimization is enabled; check the `AUTO LIQUID` row in the History tab for the skip reason; optionally seed hints with `CLUSTER BY (cols)` before `CLUSTER BY AUTO`.

Old Data Still Not Clustered After Changing Keys **Cause:** changing keys does not rewrite existing data; plain `OPTIMIZE` is incremental.

Fix: run `OPTIMIZE <table> FULL` once after enabling clustering or changing keys.

External Reader Fails After Enabling Deletion Vectors **Cause:** the protocol upgrade makes the table unreadable to clients without deletion-vector support (for example OSS Delta below 2.3.0).

Fix: upgrade the client, or `ALTER TABLE ... DROP FEATURE deletionVectors`; set the workspace auto-enable option to *Disabled* if such clients read newly created tables.

Deleted Data Still Present in Storage **Cause:** deletion vectors only soft-delete; old files remain until rewritten and vacuumed.

Fix: `REORG TABLE ... APPLY (PURGE)`, then `VACUUM`.

Best Practices

- Create new tables as **UC managed tables** and keep predictive optimization inherited from the account unless you have a reason to override.
- Use **`CLUSTER BY AUTO`** for managed tables when query patterns are mixed or unknown; use explicit `CLUSTER BY` when you know the filter columns or the table is external.
- **Disable scheduled `OPTIMIZE` jobs** on tables covered by predictive optimization.
- Set `delta.deletedFileRetentionDuration` before enabling predictive optimization if you need more than 7 days of time travel.
- Keep deletion vectors on for update-heavy tables unless an external or old client must read them.

Exam Tips

Tip

- **Frequent small `MERGE/UPDATE/DELETE` → deletion vectors.** They mark rows instead of rewriting files (sample exam Q6). Partitioning by primary key, `VACUUM` with zero retention, or disabling predictive optimization are distractors.
- **"Reduce maintenance overhead" → UC managed tables + predictive optimization.** It runs `OPTIMIZE`, `VACUUM`, `ANALYZE` on serverless jobs compute; external tables are not covered.
- **`CLUSTER BY AUTO` requires predictive optimization** and a UC managed table; keys come from the table's query history and change only when it is cost-beneficial.
- **Liquid clustering is incompatible with partitioning and `ZORDER`** on the same table; change keys with `ALTER TABLE ... CLUSTER BY`, recluster old data with `OPTIMIZE ... FULL`.
- **Enablement hierarchy:** account → catalog → schema → table, with `ENABLE | DISABLE | INHERIT PREDICTIVE OPTIMIZATION`; history lives in `system.storage.predictive_optimization_operations_history`.
- **Deletion vectors are soft deletes:** `REORG TABLE ... APPLY (PURGE)` + `VACUUM` to physically remove data.

Key Takeaways

- **Managed tables are the base layer**: predictive optimization, automatic liquid clustering, metadata caching, and automatic file cleanup after `DROP` only exist for UC managed tables.
- **Predictive optimization replaces maintenance jobs**: it runs `OPTIMIZE` (compaction and incremental clustering, never `ZORDER`), `VACUUM`, and `ANALYZE`, billed as serverless jobs.
- **Inheritance**: account default flows to catalogs, schemas, and tables; explicit `ENABLE`/`DISABLE` at a lower level overrides it, and `INHERIT` resets it.
- **Liquid clustering is the recommended layout for new tables**, including streaming tables and materialized views; up to four keys; keys can change without rewriting data.
- **`CLUSTER BY AUTO`** lets predictive optimization pick and evolve keys from real queries; it needs a managed Delta table on DBR 15.4 LTS+ and predictive optimization.
- **Deletion vectors cut write amplification** for row-level changes; reads apply them at query time, and `OPTIMIZE`, auto compaction, or `REORG ... APPLY (PURGE)` rewrite files later.
- **Compatibility costs**: liquid clustering needs writer v7/reader v3, and deletion vectors upgrade the protocol, so old or external clients may lose access.

Related Topics

- File Sizing and Compaction - `OPTIMIZE`, auto compaction, `VACUUM`
- Liquid Clustering vs Partitioning and Z-ORDER - choosing a layout by table size and query pattern
- Disk Caching - faster repeated reads of the same data
- Query Profile - confirm data skipping after clustering
- Cost Optimization - compute and serverless cost levers
- Partitioning Strategies - when partitioning still makes sense
- Unity Catalog - managed vs external tables, privileges
- System Tables - querying system tables for cost and operations

Official Documentation

- Unity Catalog managed tables for Delta Lake and Apache Iceberg
- Convert external or foreign tables to managed tables
- Predictive optimization for Unity Catalog managed tables

- Predictive optimization system table reference
- Use liquid clustering for tables
- Deletion vectors in Databricks
- Auto-enable deletion vectors
- What is predictive I/O?
- REORG TABLE
- Drop a Delta Lake table feature
- Catalog commits
- Lakeflow pipelines Python `table` reference

File Sizing and Compaction

Proper file sizing is critical for Delta Lake performance. The small file problem is one of the most common causes of poor query performance and excessive costs in data lakes.

Overview

```
flowchart TB
    subgraph Problem["Small File Problem"]
        direction TB
        S1[File 1<br>1 MB]
        S2[File 2<br>2 MB]
        S3[File 3<br>500 KB]
        S4[File 4<br>1.5 MB]
        S5[File N<br>...]
    end

    subgraph Impact["Impact"]
        Overhead[Metadata Overhead]
        Latency[High Read Latency]
        Cost[Increased Cost]
    end

    subgraph Solution["Solution"]
        Optimize[OPTIMIZE Command]
        AutoOpt[Auto Optimize]
        Compact[Compacted Files]
    end

    Problem --> Impact
    Impact --> Solution
    Solution --> Optimal[Optimal 1GB Files]
```

Overview

- **Small files slow reads**; `OPTIMIZE` compacts them and accepts `ZORDER BY` and a `WHERE` predicate
- **`optimizeWrite`** bin-packs files during a write; **`autoCompact`** compacts after the write
- **`VACUUM`** deletes files the Delta log no longer references (default retention 7 days)
- **Diagnosis**: `DESCRIBE DETAIL` returns `numFiles` and `sizeInBytes`; over-partitioning is a common cause of small files

What the Exam Tests

- Design scalable Delta/Iceberg table layouts for large data assets by mapping partitioning to data grain, aligning clustering to relationship access patterns, and maintaining balanced file sizes through compaction.
- Pick a compaction strategy for batch and for streaming writes
- Diagnose a small-file problem from table metadata

Why File Size Matters

Small Files Impact

Issue	Description	Impact
Metadata overhead	Each file has metadata to track	Slow table operations
Read amplification	Many small reads instead of few large	High I/O cost
Task overhead	Spark creates task per file	Excessive parallelism
Cloud costs	Per-request charges	Higher storage costs
Memory pressure	File listing consumes memory	Driver OOM

Optimal File Sizes

Scenario	Target Size	Reasoning
General workloads	1 GB	Balances read efficiency and parallelism
Streaming writes	128 MB	Faster micro-batch commits
Small tables	32-128 MB	Avoid over-consolidation
Partitioned tables	256 MB - 1 GB	Per partition target

OPTIMIZE Command

Basic Usage

```
-- Optimize entire table
OPTIMIZE catalog.schema.table_name;

-- Optimize specific partition
OPTIMIZE catalog.schema.orders
WHERE order_date >= '2024-01-01';

-- Optimize with Z-ORDER
OPTIMIZE catalog.schema.orders
ZORDER BY (customer_id, order_date);
```

OPTIMIZE Behavior

```
flowchart LR
    subgraph Before["Before OPTIMIZE"]
        F1[1 MB]
        F2[2 MB]
        F3[500 KB]
        F4[1.5 MB]
        F5[3 MB]
    end
```

```
    subgraph Process["OPTIMIZE Process"]
        Read[Read Small Files]
        Combine[Combine Data]
        Write[Write Optimized Files]
    end

    subgraph After["After OPTIMIZE"]
        O1[~1 GB File]
    end

    Before --> Read
    Read --> Combine
    Combine --> Write
    Write --> After
```

OPTIMIZE Output

```
OPTIMIZE my_table;

-- Returns metrics:
-- numFilesAdded: 5
-- numFilesRemoved: 150
-- numFilesReadInPredicates: 150
-- sizeOfFilesAdded: 5368709120 (5 GB)
-- sizeOfFilesRemoved: 5368709120 (5 GB)
-- operationMetrics: {...}
```

Monitoring OPTIMIZE Results

```
-- Check table history for OPTIMIZE operations
DESCRIBE HISTORY catalog.schema.orders
WHERE operation = 'OPTIMIZE'
ORDER BY timestamp DESC
LIMIT 10;

-- Query Delta log for OPTIMIZE metrics
SELECT
    version,
    timestamp,
    operation,
    operationMetrics.numFilesAdded,
    operationMetrics.numFilesRemoved
FROM (
    DESCRIBE HISTORY catalog.schema.orders
)
WHERE operation = 'OPTIMIZE';
```

Auto Optimize

Optimized Writes

```
-- Enable optimized writes for a table
ALTER TABLE catalog.schema.orders
SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true'
```

```
);

-- Optimized writes bin-pack data during writes
-- Reduces small file creation at write time
```

```
# Spark configuration for optimized writes

spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true")

# Per-write option

(df.write
    .format("delta")
    .option("optimizeWrite", "true")
    .mode("append")
    .saveAsTable("catalog.schema.table"))
```

Auto Compaction

```
-- Enable auto compaction
ALTER TABLE catalog.schema.orders
SET TBLPROPERTIES (
    'delta.autoOptimize.autoCompact' = 'true'
);

-- Auto compaction runs after writes
-- Compacts small files automatically
-- Less aggressive than manual OPTIMIZE
```

Configuration Options

Property	Default	Description
delta.autoOptimize.optimizeWrite	false	Bin-pack during writes
delta.autoOptimize.autoCompact	false	Auto-compact after writes
delta.autoOptimize.minFileSize	128 MB	Files smaller than this are candidates
delta.autoOptimize.maxFileSize	1 GB	Target output file size

Workspace-Level Configuration

```
# Set defaults for all tables in session

spark.conf.set("spark.databricks.delta.autoCompact.enabled", "true")
spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true")

# File size configuration

spark.conf.set("spark.databricks.delta.optimizeWrite.fileSize", "128mb")
```

Streaming Considerations

Streaming File Size Challenges

```
Streaming writes create small files because:
1. Micro-batches commit frequently
2. Each micro-batch creates new files
```

```
3. Watermark advances create new commits

Without optimization:
- Hundreds of small files per hour
- Query performance degrades over time
- Maintenance overhead increases
```

Optimized Writes for Streaming

```python
# Enable for streaming writes

(stream_df.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoint/path")
    .option("optimizeWrite", "true")
    .trigger(processingTime="10 seconds")
    .table("catalog.schema.streaming_table"))
```

Auto Compaction for Streaming

```sql
-- Best configuration for streaming tables
ALTER TABLE catalog.schema.streaming_events
SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact' = 'true'
);
```

Trigger OPTIMIZE After Streaming

```python
# Option 1: Scheduled maintenance job
# Run OPTIMIZE periodically (hourly/daily)

# Option 2: After streaming batch completion

def optimize_after_batch(batch_df, batch_id):
    if batch_id % 100 == 0:  # Every 100 batches
        spark.sql("OPTIMIZE catalog.schema.streaming_table")

(stream_df.writeStream
    .foreachBatch(optimize_after_batch)
    .start())
```

Partitioning Impact on File Size

Partition Considerations

```
flowchart TD
    Table[Table Data] --> Partition{Partitioned?}
    Partition -->|No| Single[All Files Together]
    Partition -->|Yes| Multiple[Files Per Partition]

    Multiple --> Check{Data Per Partition?}
    Check -->|Large| Good[Good File Sizes]
    Check -->|Small| Bad[Small Files Per Partition]
```

Over-Partitioning Problem

```sql
-- Over-partitioned: Creates many small files
CREATE TABLE orders
PARTITIONED BY (order_date, hour, region, store_id)
-- 365 days × 24 hours × 10 regions × 100 stores = 8.76M partitions!

-- Better: Partition by date only
CREATE TABLE orders
PARTITIONED BY (order_date)
-- 365 partitions, each with good file sizes
```

File Size Per Partition Target

```
Guideline:
- Target at least 1 GB per partition
- If partition has < 1 GB, consider coarser partitioning

Example:
- Daily data: 500 MB → Don't partition by day
- Daily data: 10 GB → Partition by day is fine
- Daily data: 100 GB → Consider partitioning by day + hour
```

VACUUM and File Management

VACUUM Command

```sql
-- Remove old files (default 7 days retention)
VACUUM catalog.schema.orders;

-- Specify retention period
VACUUM catalog.schema.orders RETAIN 168 HOURS;

-- Dry run to see files that would be deleted
VACUUM catalog.schema.orders DRY RUN;
```

VACUUM Behavior

```
VACUUM removes:
- Files no longer referenced by Delta table
- Old checkpoint files
- Old commit files

VACUUM does NOT:
- Compact files (use OPTIMIZE)
- Remove current files
- Remove files within retention period
```

VACUUM Best Practices

```sql
-- Standard weekly vacuum
VACUUM catalog.schema.orders RETAIN 168 HOURS;

-- More aggressive for cost savings (use with caution)
-- Requires disabling safety check
SET spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM catalog.schema.orders RETAIN 24 HOURS;
```

Monitoring File Sizes

Table Metrics

```sql
-- Check table details
DESCRIBE DETAIL catalog.schema.orders;

-- Returns:
-- numFiles: 150
-- sizeInBytes: 5368709120
-- numPartitions: 30
```

File Size Distribution

```sql
-- Query Delta log for file sizes
SELECT
    add.path,
    add.size / 1024 / 1024 AS size_mb,
    add.partitionValues
FROM (
    SELECT explode(add) AS add
    FROM delta.`/path/to/table/_delta_log/*.json`
)
WHERE add IS NOT NULL
ORDER BY size_mb DESC
LIMIT 100;
```

Identify Small File Tables

```sql
-- Find tables with small file issues
SELECT
    table_catalog,
    table_schema,
    table_name,
    num_files,
    ROUND(size_in_bytes / 1024 / 1024 / 1024, 2) AS size_gb,
    ROUND(size_in_bytes / num_files / 1024 / 1024, 2) AS avg_file_mb
FROM system.information_schema.tables
WHERE table_type = 'MANAGED'
    AND num_files > 0
    AND (size_in_bytes / num_files) < 128 * 1024 * 1024  -- < 128 MB
ORDER BY num_files DESC
LIMIT 20;
```

Compaction Strategies

Manual Compaction Schedule

```python
# Scheduled maintenance notebook

from datetime import datetime, timedelta

# Tables to optimize

tables = [
    "catalog.schema.orders",
    "catalog.schema.customers",
```

```
    "catalog.schema.events"
]

for table in tables:
    # Optimize recent partitions only
    spark.sql(f"""
        OPTIMIZE {table}
        WHERE _update_date >= date_sub(current_date(), 7)
    """)

    # Vacuum older data
    spark.sql(f"VACUUM {table} RETAIN 168 HOURS")
```

Incremental Compaction

```
-- Only compact partitions with small files
-- Use predicate to target specific partitions
OPTIMIZE catalog.schema.orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-01-07';
```

Compaction with Z-ORDER

```
-- Combine compaction with clustering
OPTIMIZE catalog.schema.orders
WHERE order_date >= '2024-01-01'
ZORDER BY (customer_id);
```

Lakeflow Pipelines and File Sizing

Lakeflow Pipelines Auto Optimize

```
Lakeflow Pipelines automatically:
- Enables optimized writes
- Manages file sizes
- Handles streaming compaction

No manual configuration needed for basic use.
```

Lakeflow Pipelines Table Properties

```
@dlt.table(
    name="optimized_table",
    table_properties={
        "delta.autoOptimize.optimizeWrite": "true",
        "delta.autoOptimize.autoCompact": "true",
        "delta.targetFileSize": "134217728"  # 128 MB
    }
)
def optimized_table():
    return dlt.read_stream("source")
```

Use Cases

- **Optimizing Write-Heavy Micro-batch Streaming**: Enabling `optimizeWrite` and `autoCompact` on a Delta table for a high-volume event stream to systematically bin-pack output files, keeping Delta scan latencies low downstream.

- **Cost Reduction via Manual Compaction**: Scheduling a weekly OPTIMIZE command via a Databricks Job to consolidate thousands of small 5MB JSON files ingested daily into ideal 1GB Parquet files, reducing object storage GET request costs by a factor of 200.

Common Issues & Errors

OPTIMIZE Takes Too Long **Scenario:** OPTIMIZE runs for hours.

Fix: Optimize in smaller chunks:

```
-- Instead of full table
-- OPTIMIZE catalog.schema.orders;

-- Optimize by partition
OPTIMIZE catalog.schema.orders
WHERE order_date = '2024-01-15';

-- Or by date range
OPTIMIZE catalog.schema.orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-01-07';
```

Files Grow Back After OPTIMIZE **Scenario:** Small files return after optimization.

Fix: Enable auto optimization:

```
ALTER TABLE catalog.schema.orders
SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact' = 'true'
);
```

VACUUM Removes Too Many Files **Scenario:** Time travel fails after VACUUM.

Fix: Maintain proper retention:

```
-- Don't go below 7 days without understanding implications
VACUUM catalog.schema.orders RETAIN 168 HOURS;

-- For time travel needs, increase retention
ALTER TABLE catalog.schema.orders
SET TBLPROPERTIES (
    'delta.logRetentionDuration' = 'interval 30 days',
    'delta.deletedFileRetentionDuration' = 'interval 30 days'
);
```

Out of Memory During OPTIMIZE **Scenario:** Driver OOM during OPTIMIZE.

Fix: Reduce parallelism or optimize partitions:

```
# Reduce concurrent files

spark.conf.set("spark.databricks.delta.optimize.maxFileSize", "512mb")
spark.conf.set("spark.databricks.delta.optimize.minFileSize", "64mb")

# Optimize smaller partitions

spark.sql("""
```

```
    OPTIMIZE table WHERE partition_col = 'specific_value'
""")
```

Exam Tips

1. **Target file size** - 1 GB for general workloads, 128 MB for streaming
2. **OPTIMIZE** - Compacts small files, can combine with ZORDER
3. **Auto optimize** - optimizeWrite (during write) vs autoCompact (after write)
4. **VACUUM** - Removes old files, default 7 day retention
5. **Small file problem** - Metadata overhead, read amplification
6. **Streaming** - Enable optimized writes, schedule periodic OPTIMIZE
7. **Partitioning** - Target at least 1 GB per partition
8. **OPTIMIZE predicate** - Use WHERE clause to limit scope
9. **Lakeflow Pipelines** - Auto-manages file sizes, no manual OPTIMIZE needed
10. **Monitoring** - DESCRIBE DETAIL shows numFiles and sizeInBytes

Key Takeaways

- **Target file size**: Aim for ~1 GB per file for general workloads and ~128 MB for streaming micro-batch outputs to balance read efficiency with write overhead.
- **OPTIMIZE command**: Consolidates small files into larger ones; can be combined with `ZORDER BY` in the same command and supports a `WHERE` predicate to limit scope to specific partitions.
- **optimizeWrite vs autoCompact**: `optimizeWrite` bin-packs files during the write operation; `autoCompact` runs a background compaction job after writes complete — both are disabled by default.
- **VACUUM removes old files**: `VACUUM` deletes files no longer referenced by the Delta log; the default retention is 7 days (168 hours) — never vacuum below this unless time travel is not needed.
- **Over-partitioning causes small files**: Partitioning by too many high-cardinality columns (e.g., date + hour + region + store) creates millions of tiny partitions, each with very small files.
- **Lakeflow Pipelines auto-manages file sizes**: Lakeflow Pipelines automatically enables `optimizeWrite` and manages file sizing — manual `OPTIMIZE` commands are not needed for Lakeflow Pipelines-managed tables.
- **Streaming compaction strategy**: Enable both `optimizeWrite` and `autoCompact` on streaming output tables, or schedule a periodic `OPTIMIZE` job every N batches using `foreachBatch`.
- **DESCRIBE DETAIL for diagnosis**: `DESCRIBE DETAIL table_name` returns `numFiles` and `sizeInBytes`, making it easy to calculate average file size and identify tables needing compaction.

Related Topics

- Z-ORDER Indexing - Combine with OPTIMIZE
- Spark Tuning - Write performance
- Delta Lake Operations - Table maintenance

Official Documentation

- OPTIMIZE
- Auto Optimize
- VACUUM
- File Size Tuning

Data Skipping: Liquid Clustering vs Partitioning and Z-ORDER

Overview

Every layout technique on Databricks exists to let queries skip files. Hive-style partitioning skips whole directories, Z-ORDER co-locates values inside files so per-file statistics can skip them, and liquid clustering does the same job incrementally with keys you can change. The exam asks you to pick the right one for a given table size and query pattern; the docs' current answer for new tables is liquid clustering.

Overview

- **Data skipping** uses per-file min/max/null-count/record-count statistics collected on write
- **Partitioning** = one directory per value; only for low/known-cardinality columns and very large tables (Databricks: don't partition tables under 1 TB)
- **Z-ORDER** = multi-column co-location applied by `OPTIMIZE ... ZORDER BY`; manual, not maintained by predictive optimization, not combinable with liquid clustering
- **Liquid clustering** (`CLUSTER BY` / `CLUSTER BY AUTO`) replaces both, works for low and high cardinality, clusters incrementally, and lets keys change without a rewrite

What the Exam Tests

- Compare Liquid Clustering vs partitioning/ZORDER for a given table size and query pattern
- Know Databricks' size guidance (< 1 TB, 1–100 TB, ≥ 100 TB) and that liquid clustering is recommended for all new tables
- Know how to migrate a partitioned or Z-ordered table to liquid clustering and how to verify skipping

---

Data Skipping Fundamentals

```
flowchart TB
    subgraph Without["Unclustered layout"]
        direction LR
        F1[File 1: ids 1-9000]
        F2[File 2: ids 3-9800]
        F3[File 3: ids 5-9900]
    end

    subgraph With["Clustered layout (Z-ORDER or liquid)"]
        direction LR
        Z1[File 1: ids 1-3000]
        Z2[File 2: ids 3001-6000]
        Z3[File 3: ids 6001-9900]
    end

    Query["Query: WHERE id = 4200"] --> Without
    Query --> With
    Without --> Scan[Min/max overlap: read all files]
    With --> Skip[Only File 2 can match: skip 2 files]
```

Data skipping statistics are collected **automatically when you write** to a Delta Lake or managed Iceberg table. At query time Databricks uses per-file **minimum and maximum values, null counts, and total records** to skip files that can't match the filter. Layout techniques make those min/max ranges narrow; without them, every file's range overlaps the filter value.

Which Columns Have Statistics

Table type	Default statistics columns
UC external tables	First 32 columns of the schema
UC managed tables with predictive optimization	First 32 columns, plus additional columns that predictive op

If you aren't using predictive optimization, change the defaults with table properties:

```
-- Number of leading columns to collect stats on (depends on column order)
ALTER TABLE main.sales.orders SET TBLPROPERTIES
 ↪  ('delta.dataSkippingNumIndexedCols' = '40');

-- Explicit list (DBR 13.3 LTS+); supersedes dataSkippingNumIndexedCols
ALTER TABLE main.sales.orders SET TBLPROPERTIES ('delta.dataSkippingStatsColumns'
 ↪  = 'customer_id, order_date, region');

-- Changing the properties only affects future writes; recompute for existing
 ↪  data (DBR 14.3 LTS+)
ANALYZE TABLE main.sales.orders COMPUTE DELTA STATISTICS;
```

- Statistics aren't used for columns outside the current statistics column list.
- Long strings are truncated during statistics collection; consider excluding long string columns that are rarely filtered.
- Z-ORDER and liquid clustering keys **must** be columns with statistics; Z-ordering a column without stats is ineffective and wastes compute.

Hive-Style Partitioning

Partitioning splits a table into one directory per partition-column value, so filters on that column prune whole directories.

```
CREATE TABLE main.sales.orders_by_date (
  order_id BIGINT, customer_id BIGINT, amount DECIMAL(10, 2), order_date DATE
) PARTITIONED BY (order_date);
```

What the docs say about it today:

- Most tables **under 100 TB don't need partitioning**. Unpartitioned Delta tables automatically use **ingestion time clustering**, which gives partitioning-like performance for datetime filters with no tuning.
- Partitioning works well only for **low or known cardinality** fields (dates, physical locations), not high-cardinality fields such as timestamps.
- Each partition should hold **at least 1 GB**; fewer, larger partitions tend to outperform many small ones.
- A bad partitioning choice can need a **full rewrite** to fix, which is slow and expensive for large tables.
- Partition columns must be top-level columns; you can't partition by struct fields or complex types (liquid clustering can cluster on struct fields).

Size Guidance from the Docs

Table size	Guidance
Less than 1 TB	Don't partition
More than 1 TB to 100 TB	Use liquid clustering instead of partitioning; partitioning more often hurts than hel

Table size	Guidance
100 TB or more	Partitioning might improve performance, but use liquid clustering first and verify a

---

Z-ORDER

Z-ordering co-locates related values in the same set of files so data-skipping statistics become selective. It is applied by `OPTIMIZE`:

```
OPTIMIZE main.sales.events ZORDER BY (event_type);

-- Multiple columns (effectiveness drops with each extra column)
OPTIMIZE main.sales.events ZORDER BY (user_id, event_type);

-- Limit the rewrite with a partition predicate (WHERE accepts partition columns
 ↪  only)
OPTIMIZE main.sales.events
WHERE event_date >= current_timestamp() - INTERVAL 1 day
ZORDER BY (event_type);
```

Rules to know:

- Use `ZORDER BY` on columns that are commonly used in predicates and have **high cardinality**.
- You **cannot Z-order on partition columns**, and Z-order can only co-locate data **within a partition** (files are never combined across partition boundaries).
- Z-ordering is **not idempotent** but aims to be incremental; re-running is not guaranteed to get faster. If no new data was added to a partition that was just Z-ordered, Z-ordering it again has no effect.
- It balances files by number of rows, not bytes, so `OPTIMIZE` task times and file sizes can be skewed when recent rows are much wider.
- Data written after the last `OPTIMIZE ... ZORDER BY` is not Z-ordered until the next run, so Z-order needs a scheduled job.
- **Predictive optimization never runs `ZORDER`** and ignores Z-ordered files.
- You **can't use `ZORDER` on a liquid-clustered table**.

Liquid Clustering in Brief

Liquid clustering replaces partitioning and Z-ORDER. It is recommended for all new tables, including streaming tables and materialized views. Full details (syntax, thresholds, `CLUSTER BY AUTO`, deletion vectors) are in Managed Tables, Predictive Optimization, and Automatic Clustering.

```
CREATE TABLE main.sales.orders (
  order_id BIGINT, customer_id BIGINT, order_date DATE, amount DECIMAL(10, 2)
) CLUSTER BY (customer_id, order_date);

ALTER TABLE main.sales.orders CLUSTER BY AUTO;    -- UC managed + predictive
 ↪  optimization
OPTIMIZE main.sales.orders;                        -- incremental clustering
OPTIMIZE main.sales.orders FULL;                   -- recluster everything after a
 ↪  key change
```

- Up to **four** keys; for tables under about 10 TB, more keys can slow single-column filters (four keys is worse than two).

- Keys can change any time with `ALTER TABLE ... CLUSTER BY`; existing data is not rewritten until `OPTIMIZE ... FULL`.
- Works for both low- and high-cardinality columns and avoids fixed partition boundaries and the small-file issues of static partitioning.
- Liquid clustering tables allow **row-level concurrency** (with deletion vectors and row tracking), so concurrent writes conflict less.

Comparison: Liquid Clustering vs Partitioning vs Z-ORDER

Aspect	Hive-style partitioning	Z-ORDER
Mechanism	Separate directory per value; prunes directories	Co-locates va
Column cardinality	Low or known only (date, region)	High (ids, tim
Columns	One or a few low-cardinality columns	Several, but e
Changing the columns	Full table rewrite	Re-run OPTIM
New data	Lands in the right partition	Unordered un
Maintained by predictive optimization	Compaction within partitions only	**No** (ZORDER n
Automatic key selection	No	No
Skewed data	Partition key can give too many or too few partitions	Balances row
Combine with the others	Can combine with Z-ORDER	Can combine
Docs recommendation	Only for 100 TB+ tables, after testing liquid clustering	Use liquid clu

---

Decision by Table Size and Query Pattern

Table size	Query pattern
Under 1 TB	Any
Under ~10 TB	Mostly single-column filters
1–100 TB	Filters on high-cardinality columns (customer_id
1–100 TB	Filters on a date plus an id
Any (UC managed)	Mixed, changing, or unknown filter patterns
Any	Heavy skew, or a partition key would create too
Any	Many concurrent writers
100 TB+	Every query filters on a low-cardinality column w
Existing partitioned / Z-ordered table	Poor skipping, over-partitioning, or rising OPTIMI
Table read by clients that can't handle writer v7 / reader v3	Any

```
flowchart TD
    Start[Choose a layout] --> New{New table?}
    New -->|Yes| Managed{UC managed with<br/>predictive optimization?}
    Managed -->|Yes, filters vary| AUTO[CLUSTER BY AUTO]
    Managed -->|Known filter columns| LC[CLUSTER BY up to 4 keys]
    Managed -->|No| LC
    New -->|No, existing| Old{Partitioned or<br/>Z-ordered today?}
    Old -->|Yes| Conv[REPLACE PARTITIONED BY<br/>WITH CLUSTER BY ... then
↪   OPTIMIZE]
    Old -->|Unpartitioned| Alter[ALTER TABLE CLUSTER BY<br/>then OPTIMIZE FULL]
    Start --> Huge{100 TB+ and every query<br/>filters one low-cardinality
↪   column?}
    Huge -->|Yes| Test[Test liquid clustering first;<br/>partition only if proven
↪   faster]
```

Migrating to Liquid Clustering

Convert a Partitioned Table In Place (Recommended) Databricks recommends the `REPLACE PARTITIONED BY WITH CLUSTER BY` conversion (Databricks Runtime 18.1 and above; external and managed Delta tables). It minimizes reader and writer downtime; afterwards the table supports reads from DBR 13.3 LTS and above.

```
-- Keep the partition columns as clustering keys
ALTER TABLE main.sales.orders_by_date REPLACE PARTITIONED BY WITH CLUSTER BY;

-- Cluster on different/additional columns, then run OPTIMIZE to benefit
ALTER TABLE main.sales.orders_by_date REPLACE PARTITIONED BY WITH CLUSTER BY
 ↪  (order_date, customer_id);
OPTIMIZE main.sales.orders_by_date;

-- Start from the partition columns and let predictive optimization adapt (UC
 ↪  managed only)
ALTER TABLE main.sales.orders_by_date REPLACE PARTITIONED BY WITH CLUSTER BY AUTO;
```

- On conversion, the old partition columns become **hierarchical clustering keys** (prioritized ahead of the other keys).
- Verify with `DESCRIBE EXTENDED` (new clustering columns) and `DESCRIBE HISTORY` (REORG, UPGRADE PROTOCOL, and REPLACE PARTITIONED BY WITH CLUSTER BY operations).
- **Not supported** for streaming tables and materialized views created in Lakeflow Pipelines: change the pipeline definition from `PARTITIONED BY` to `CLUSTER BY` instead.
- Converting a table partitioned by a **timestamp** column needs `spark.databricks.delta.liquidConversi = false` and then `ANALYZE TABLE ... COMPUTE DELTA STATISTICS`.

Picking Keys When You Migrate

Current technique	Clustering keys to use
Hive-style partitioning	The partition columns
Z-order indexing	The `ZORDER BY` columns
Partitioning + Z-order	Both the partition columns and the ZO
Generated column to reduce cardinality (e.g. `date` from a timestamp)	The original column; don't create the

Unpartitioned, Z-Ordered Table

```
ALTER TABLE main.sales.events CLUSTER BY (user_id, event_type);  -- former ZORDER
 ↪  BY columns
OPTIMIZE main.sales.events FULL;                                 -- recluster
 ↪  existing data once
```

A CTAS into a new `CLUSTER BY` table also works, but it rewrites the whole table and doesn't keep all table metadata.

Bloom Filter Indexes (Deprecated)

Platform update (Oct 2026)

Bloom filter indexes are **deprecated**: Databricks says not to use them and to remove existing ones. Predictive I/O (Photon) performs file skipping on all columns automatically and fully supersedes them; liquid clustering improves skipping for frequently filtered columns. Source: Bloom filter indexes (deprecated)

```
-- Remove an existing Bloom filter index, then VACUUM to clean up the
 ↪  _delta_index files
DROP BLOOMFILTER INDEX ON TABLE main.sales.orders;
```

Verifying That Skipping Works

- **Query profile / query history**: the query details panel shows the percentage of data pruned during scanning; see Query Profile.
- **`system.query.history`**: compare `pruned_files` vs `read_files` and `pruned_files_bytes` vs `read_files_bytes`.
- **Performance insights** flag layout problems: `COVERAGE_FILTER_KEYS_CLUSTERING` / `COVERAGE_FILTER_KEYS_PARTITIONING` (table clustered/partitioned on keys the query doesn't filter on), `COVERAGE_STATS_DELTA` (missing statistics for the filter columns), `DATA_FILE_SIZE` (many small files), and `MANUAL_DATA_LAYOUT` (manually optimized table that could use automatic liquid clustering).
- **`EXPLAIN`**: `PartitionFilters` shows partition pruning; see EXPLAIN Plans & AQE.

```
SELECT statement_id, read_files, pruned_files,
       ROUND(pruned_files / NULLIF(read_files + pruned_files, 0) * 100, 1) AS
        ↪  pct_files_pruned
FROM system.query.history
WHERE statement_text ILIKE '%main.sales.orders%'
  AND start_time >= current_timestamp() - INTERVAL 1 DAY
ORDER BY start_time DESC;
```

Use Cases

- **200 GB dimension table partitioned by `country`**: under 1 TB, so remove partitioning; liquid clustering on `country` (or `CLUSTER BY AUTO`) gives the same skipping without small files.
- **20 TB event table filtered by `user_id` and `event_date`**: liquid clustering on both columns instead of partition-by-date plus nightly `ZORDER BY (user_id)`; predictive optimization keeps it clustered.
- **Shared gold table whose filters change every quarter**: `CLUSTER BY AUTO` on the managed table instead of re-planning partitions.
- **Legacy 50 TB table partitioned by `(year, month, day)` and Z-ordered by `customer_id`**: `REPLACE PARTITIONED BY WITH CLUSTER BY (day, customer_id)` (or with the partition and Z-order columns), then `OPTIMIZE`.

Common Issues & Errors

Z-ORDER Stops Paying Off **Cause:** new files arrive unordered between runs, the filter columns don't match the Z-order columns, or too many columns are listed.

Fix: move to liquid clustering using the `ZORDER BY` columns as keys; predictive optimization will then cluster new data incrementally.

`ZORDER` Fails on a Clustered Table **Cause:** liquid clustering is not compatible with `ZORDER` (or partitioning).

Fix: use `OPTIMIZE` without `ZORDER BY`, or change keys with `ALTER TABLE ... CLUSTER BY`.

Thousands of Tiny Partitions **Cause:** partitioned on a high-cardinality column, or a table far below 1 TB.

Fix: convert with `REPLACE PARTITIONED BY WITH CLUSTER BY`; partitions should hold at least 1 GB each.

Filters on a Column Still Scan Everything **Cause:** the column has no statistics (beyond the first 32 columns), or the filter wraps the column in a type conversion.

Fix: add it with `delta.dataSkippingStatsColumns` and `ANALYZE TABLE ... COMPUTE DELTA STATISTICS`, or enable predictive optimization on a managed table; check the `COVERAGE_STATS_DELTA` insight.

Old Data Not Clustered After Changing Keys **Cause:** key changes don't rewrite existing data; `OPTIMIZE` is incremental.

Fix: run `OPTIMIZE <table> FULL`.

Exam Tips

Tip

- **New table → liquid clustering** (Databricks recommends it for all new tables, including streaming tables and materialized views). Unknown or changing filters on a UC managed table → `CLUSTER BY AUTO`.
- **Size rules:** < 1 TB don't partition; 1–100 TB use liquid clustering instead; ≥ 100 TB partitioning *might* help but test liquid clustering first; ≥ 1 GB per partition.
- **Partitioning = low/known cardinality; Z-ORDER = high cardinality; liquid = both.**
- **Liquid clustering can't be combined with partitioning or `ZORDER`**; predictive optimization never runs `ZORDER`.
- **Changing layout columns:** partitioning needs a rewrite; Z-order needs a new `OPTIMIZE ... ZORDER BY`; liquid clustering just needs `ALTER TABLE ... CLUSTER BY` (+ `OPTIMIZE FULL` for old data).
- **Migration:** `ALTER TABLE ... REPLACE PARTITIONED BY WITH CLUSTER BY`; use partition + Z-order columns as keys.
- Bloom filter indexes are deprecated; don't pick them as the modern answer.

Key Takeaways

- **Data skipping** relies on per-file min/max/null/record statistics, collected on write for the first 32 columns by default (managed tables with predictive optimization add frequently filtered columns).
- **Partitioning** suits only very large tables filtered on a low-cardinality column with ≥ 1 GB per partition; tables under 1 TB shouldn't be partitioned, and unpartitioned tables get ingestion time clustering for free.
- **Z-ORDER** co-locates high-cardinality values within files but is manual, not maintained by predictive optimization, works only within partitions, and loses effectiveness with each added column.
- **Liquid clustering** handles low and high cardinality, up to four keys, clusters incrementally, supports row-level concurrency, and lets you change keys without an immediate rewrite.

- **`CLUSTER BY AUTO`** is the low-maintenance choice for UC managed tables with mixed or changing query patterns.
- **Migrate** partitioned or Z-ordered tables with `REPLACE PARTITIONED BY WITH CLUSTER BY`, then verify pruning in the query profile or `system.query.history`.

Related Topics

- Managed Tables, Predictive Optimization, and Automatic Clustering - liquid clustering, `CLUSTER BY AUTO`, deletion vectors in depth
- File Sizing and Compaction - `OPTIMIZE` and small files
- Query Profile - confirm files and bytes pruned
- EXPLAIN Plans & AQE - `PartitionFilters`, pushed filters
- Partitioning Strategies - partition design in data modeling
- Delta Lake Fundamentals - transaction log and file statistics

Official Documentation

- Use liquid clustering for tables
- When to partition tables on Databricks
- Data skipping (statistics and Z-ordering)
- Optimize data file layout (OPTIMIZE)
- Predictive optimization for Unity Catalog managed tables
- Bloom filter indexes (deprecated)
- Query performance insights
- Query history system table reference

Disk Caching

Overview

The exam calls it the **Delta cache**: Databricks keeps local copies of remote Parquet and Delta data files on worker SSDs, so repeated reads of the same data skip the trip to cloud object storage. You also need to know how it differs from the Apache Spark cache (`cache()` / `persist()`), the result caches in Databricks SQL, and what changes on serverless compute.

Platform update (Oct 2026)

The Delta cache is now called the **disk cache** (it was also called the DBIO cache). It is a proprietary Databricks feature, not part of the Delta Lake protocol; the rename avoids that confusion. Source: Optimize performance with caching on Databricks

Overview

- **Disk (Delta) cache**: automatic, local-SSD copies of Parquet data files (including Delta tables) in a fast intermediate format, filled on first read and kept consistent automatically
- **Spark cache**: manual `df.cache()` / `persist()` of any DataFrame or RDD, stored as blocks according to the storage level
- **Databricks SQL** adds a UI cache, local and remote **result caches**, and the disk cache on warehouse nodes
- **Serverless**: disk-cache Spark configs aren't settable; DataFrame caching on serverless (Public Preview, Oct 23, 2026) stores a session-private snapshot

What the Exam Tests

- Understand how caching (Delta cache) improves query performance for repeated reads of the same data
- Choose between the disk cache and the Spark cache for a given workload
- Recognise when caching does not help (single reads, cold starts, frequently changing data, wrong instance type)

How the Disk Cache (Delta Cache) Works

Reading from cloud object storage is slower than reading from local disk. Without caching, every query that reads the same Parquet files fetches them from object storage again.

When a node fetches a Parquet data file from remote storage, it **automatically stores a copy on its local storage in a fast intermediate format**. Later reads of the same data run locally and are significantly faster. The cache works for all Parquet data files, including Delta Lake tables.

```
sequenceDiagram
    participant Q as Query (task on worker)
    participant C as Local SSD disk cache
    participant S as Cloud object storage
    Q->>C: Need file part-0001.parquet
    C-->>Q: Miss (first read)
    Q->>S: Fetch file
    S-->>Q: Parquet data
    Q->>C: Store copy (fast intermediate format)
    Note over Q,C: Later queries on the same data
    Q->>C: Need part-0001.parquet
    C-->>Q: Hit: read locally, no object-storage round trip
```

Key Properties

Property	Behaviour
What is cached	Parquet data files (Delta Lake tables included) read from S3, ABFS, and other file systems
When	Automatically, on the first read (if the cache is enabled)
Where	Local files on the worker node (local SSD)
Consistency	Detects files that are created, deleted, modified, or overwritten and updates itself; stale entr
Eviction	Automatically in LRU fashion or on any file change; cleared when the cluster restarts
Size	By default uses at most half of the space on the workers' local SSDs
Availability	Enabled by default on worker instance types that support it (workers with SSD volumes)

Important

The disk cache stores **files**, not query results. Any query that reads the same files benefits, even if its filters, joins, or aggregations are different from the query that first loaded them.

Enabling and Configuring the Disk Cache

The recommended (and easiest) setup is to **choose a worker instance type with SSD volumes**; those workers are enabled and configured for disk caching automatically.

```
# Check and toggle at runtime (classic compute)
spark.conf.get("spark.databricks.io.cache.enabled")
spark.conf.set("spark.databricks.io.cache.enabled", "true")
```

Disabling the cache does **not** drop data already on local storage. It stops queries from adding data to the cache and from reading from it.

To control how much local storage it uses, set these Spark configurations **when you create the cluster**:

Setting	Meaning
`spark.databricks.io.cache.maxDiskUsage`	Disk space per node reserved for cached data, in byte
`spark.databricks.io.cache.maxMetaDataCache`	Disk space per node reserved for cached metadata, ir
`spark.databricks.io.cache.compression.enabled`	Whether cached data is stored compressed

```
spark.databricks.io.cache.maxDiskUsage 50g
spark.databricks.io.cache.maxMetaDataCache 1g
spark.databricks.io.cache.compression.enabled false
```

Note

In SQL warehouses and Databricks Runtime 14.2 and above, the `CACHE SELECT` command (used to pre-warm the disk cache) is **ignored**; an enhanced disk caching algorithm is used instead.

When a worker is decommissioned (for example, when autoscaling removes it), the data cached on that worker is lost and has to be reread from the source, so heavy autoscaling makes the cache less stable.

Disk Cache vs Spark Cache

Feature	Disk cache (Delta cache)	Apache Spark
Stored as	Local files on a worker node	In-memory bl
Applied to	Any Parquet table on S3, ABFS, and other file systems	Any DataFram
Triggered	Automatically, on the first read (if enabled)	Manually; req
Evaluated	Lazily	Lazily (on clas
Availability	Enabled by default on certain node types; toggle with config flags	Always availa
Evicted	Automatically (LRU) or on any file change; manually by restarting the cluster	Automatically
Sees new data	Yes, invalidates changed files automatically	Varies: refresl

```
# Spark cache: you choose what to cache, typically an expensive intermediate
 ↪  result
enriched = (spark.table("main.sales.orders")
    .join(spark.table("main.sales.customers"), "customer_id")
    .filter("order_date >= '2026-01-01'"))

enriched.cache()          # lazy on classic compute: filled by the first action
enriched.count()          # materializes the cache
by_region = enriched.groupBy("region").sum("amount")
enriched.unpersist()      # release when done
```

Which One to Use

Scenario	Choose
Dashboards or analysts repeatedly querying the same Delta tables	**Disk cache** (automatic; not
Iterative queries on the same tables in a notebook	**Disk cache**
An expensive join/aggregation result reused several times in one job or session	**Spark cache** (`df.cache()`)
Data must always reflect the latest table version	Disk cache (consistent); avo
Non-Parquet intermediate data (any DataFrame)	Spark cache

Databricks recommends using automatic disk caching, and using the disk cache instead of the Spark cache for repeated reads of Parquet and Delta Lake data.

Caching Layers in Databricks SQL

Databricks SQL warehouses stack several caches. A query first checks result caches; only if there is no cached result does it execute, and execution reads files through the disk cache.

```
flowchart TD
    Q[Query on SQL warehouse] --> L{Local result cache<br/>in memory, per cluster}
    L -->|Hit| R[Return cached result]
    L -->|Miss| RR{Remote result cache<br/>serverless only, workspace-wide}
    RR -->|Hit| R
    RR -->|Miss| X[Execute query]
    X --> D{Disk cache<br/>local SSD}
    D -->|Hit| F[Read file locally]
    D -->|Miss| O[Read from object storage<br/>then cache the file]
```

Cache	Scope and storage
Databricks SQL UI cache	Per user; query results and SQL editor visualizations (not AI/BI dashboards)
Local result cache	Per cluster, in memory
Remote result cache	**Serverless only**; persisted as workspace system data, shared by all warehouses i
Disk cache	Local SSD on warehouse nodes; data files, not results

```
-- Turn off result caching for testing or benchmarking only
SET use_cached_result = false;
```

Two consequences to remember:

- A **query profile is not available** for a query served from the query cache. To get one, make a trivial change to the query (for example, change or remove the `LIMIT`).
- `system.query.history` records cache behaviour: `from_result_cache` (result came from the cache), `cache_origin_statement_id`, and `read_io_cache_percent` (share of persistent data bytes read from the IO cache).

```
-- How much of each warehouse's reading came from cache in the last day
SELECT compute.warehouse_id,
       COUNT(*) AS statements,
       AVG(CASE WHEN from_result_cache THEN 1 ELSE 0 END) AS
        ↪  result_cache_hit_ratio,
       AVG(read_io_cache_percent) AS avg_io_cache_percent
FROM system.query.history
WHERE start_time >= current_timestamp() - INTERVAL 1 DAY
GROUP BY ALL;
```

Caching on Serverless Compute

On serverless notebooks and jobs, Databricks manages the compute, and only a short list of Spark properties can be set (such as `spark.sql.shuffle.partitions` and `spark.sql.files.maxPartitionBytes`). The `spark.databricks.io.cache.*` settings are not among them, so you can't configure the disk cache there.

DataFrame Caching on Serverless (Public Preview)

Platform update (Oct 2026)

DataFrame caching on serverless compute entered Public Preview on October 23, 2026. `df.cache()` or `df.persist()` reuses an expensive result for the rest of the session. It requires serverless environment version 5 or above, Databricks Connect for Python 18.1.8+ or Scala 18.1.5+, and is enabled by default for most workspaces (admins can disable it from the Previews page). Source: Databricks platform release notes, October 2026

Behaviour	Serverless compute	Classic
Materialization	**Eager**: computed when you call `df.cache()`	Lazy: c
Cache matching	Only the cached DataFrame and DataFrames derived from it	Any qu
Data freshness	Fixed snapshot taken at `df.cache()`	Varies
Sharing scope	Private to your session	Shared
`df.unpersist()`	Destroys the DataFrame and releases the snapshot	Releas
Storage level	`persist(level)` accepts and ignores any level; `storageLevel` reports `DISK_ONLY`	Honou

```
from pyspark.sql import functions as F

daily = (spark.table("main.sales.orders")
    .filter(F.col("order_date") == "2026-10-15")
    .join(spark.table("main.sales.customers"), "customer_id"))

daily.cache()                       # runs the join now and stores a snapshot

total = daily.agg(F.sum("amount").alias("total")).first()["total"]
by_region = daily.groupBy("region").agg(F.sum("amount").alias("revenue"))   #
 ↪ reads the snapshot

daily.unpersist()                   # daily and DataFrames derived from it are now
 ↪ unusable
```

Other serverless rules:

- The snapshot lasts until the session ends, **at most seven days**; only that session can read it.
- Permission changes, row filters, and column masks applied to source tables **after** caching don't apply to the cached data.
- A new DataFrame that repeats the same query does **not** use the cache; derive from the cached DataFrame instead.
- Not available in workspaces with the compliance security profile, or in AWS GovCloud.
- Not supported on serverless (they raise an exception): `CACHE TABLE`, `UNCACHE TABLE`, `CLEAR CACHE`, `REFRESH TABLE`, `spark.catalog.cacheTable()`, `spark.catalog.uncacheTable()`, `spark.catalog.clearCache()`, and `df.checkpoint()`. To reuse a result in SQL, use a session-local temporary table.

When Caching Does Not Help

Situation	Why caching doesn't pay off
Data read only once (single-pass ETL, one-off backfill)	First read always comes from object storage
Cold start: new cluster, restarted cluster, or stopped warehouse	Local caches are empty after a restart
Workers without local SSDs	Disk cache isn't enabled by default on those
Data that isn't Parquet	The disk cache applies to Parquet data files
Tables rewritten constantly	Changed files are evicted; result caches inv
Heavy autoscaling churn	Decommissioned workers lose their cached
Spark-caching a plain table read	Later reads still scan a full copy of the table
Write-heavy jobs	Caches speed up reads, not writes

Use Cases

- **BI dashboards on a SQL warehouse**: the same gold tables are queried all day. The disk cache keeps the files on local SSD; identical queries hit the result cache; on serverless warehouses the remote result cache survives a warehouse restart.
- **Iterative notebook analysis**: an analyst runs many different queries against the same Delta table on a cluster with SSD workers; after the first query, scans read locally.
- **Reusing an expensive join in one job**: a job joins a large fact table with customers once, then writes three different aggregates. `df.cache()` on the joined DataFrame (or the serverless DataFrame cache) avoids recomputing the join three times.
- **Benchmarking a query change**: set `use_cached_result = false` so the second run actually executes, then compare query profiles.

Common Issues & Errors

Cache Doesn't Seem to Help **Cause:** worker type without local SSD, a cold cluster or warehouse, or data read only once.

Fix: pick an SSD-backed worker type; check `read_io_cache_percent` in `system.query.history`; accept that the first read always hits object storage.

`CACHE SELECT` Has No Effect **Cause:** in SQL warehouses and DBR 14.2+, `CACHE SELECT` is ignored.

Fix: remove it; the enhanced disk caching algorithm fills the cache on read.

`CACHE TABLE` Fails on Serverless **Cause:** SQL and catalog cache APIs aren't supported on serverless compute.

Fix: use `df.cache()` (serverless DataFrame caching) or a session-local temporary table.

Stale Results After Writing to the Source Table **Cause:** a cached DataFrame (Spark cache, or the serverless snapshot) does not reliably pick up new writes; serverless snapshots never do.

Fix: rebuild the DataFrame from the source tables after writing; don't use `unpersist()` as a refresh.

"Query Profile Is Not Available" **Cause:** the result was served from the query cache.

Fix: make a trivial change to the query (such as the `LIMIT`) or `SET use_cached_result = false` while testing.

Exam Tips

Tip

- **"Repeated reads of the same data are slow" → disk (Delta) cache.** It is automatic on SSD-backed workers and caches Parquet/Delta files locally after the first read.
- **Disk cache = files on local SSD, automatic, always consistent.** Spark cache = DataFrame blocks, manual (`cache()`/`persist()`), may be stale.
- **No manual invalidation**: the disk cache detects changed files; result caches invalidate when tables are updated.
- **SQL warehouse layers**: UI cache (per user, ≤ 7 days), local result cache, remote result cache (serverless only, 24 hours, survives restarts), disk cache.
- **Serverless**: `CACHE TABLE` and friends throw errors; `df.cache()` creates an eager, session-private snapshot (Public Preview).
- The cache only helps **reads that repeat**; it doesn't speed up the first scan or writes.

Key Takeaways

- **Disk cache (formerly Delta cache / DBIO cache)** copies remote Parquet data files, including Delta tables, to local SSD in a fast intermediate format on first read; later reads are local.
- **Enabled by default** on worker types with SSD volumes; uses at most half the local SSD by default; toggled with `spark.databricks.io.cache.enabled` and sized with `maxDiskUsage` / `maxMetaDataCache` at cluster creation.
- **Consistency is automatic**: changed, deleted, or overwritten files are detected and evicted.
- **Spark cache is different**: manual, any DataFrame/RDD, stored per storage level, released with `unpersist()`; Databricks recommends the disk cache for repeated reads of Parquet and Delta data.
- **Databricks SQL** adds result caching: a local in-memory cache per cluster and a serverless-only remote result cache shared across warehouses, both with a 24-hour life cycle.
- **Serverless** doesn't expose disk-cache configs; DataFrame caching on serverless (Public Preview, Oct 2026) is eager, snapshot-based, and session-private.

Related Topics

- Managed Tables, Predictive Optimization, and Automatic Clustering - reduce what has to be read in the first place
- File Sizing and Compaction - fewer, right-sized files to scan and cache
- Query Profile - read IO and cache metrics for a query
- Spark Tuning - memory, shuffle, and other runtime settings
- Photon Diagnostics and Optimization (Part 1) - Photon and fast repeat access
- Databricks Compute - choosing instance types and serverless

Official Documentation

- Optimize performance with caching on Databricks
- Query caching (Databricks SQL)
- Cache DataFrames on serverless compute
- Serverless compute limitations
- Configure Spark properties (serverless supported list)
- Query history system table reference
- Databricks platform release notes, October 2026

Query Profile

Overview

The query profile visualizes how a query actually executed: every operator, its time, rows, and memory, plus Databricks' own performance insights. It is the main tool for finding bottlenecks in queries run on SQL warehouses and serverless compute, and this topic shows how to read it for three exam-tested problems: poor data skipping, the wrong join strategy, and expensive shuffles (with spill and skew).

Overview

- **Where**: Query History, the SQL editor, notebooks on a SQL warehouse or serverless compute, the Lakeflow pipelines UI, and the Jobs UI; classic clusters use the Spark UI instead
- **What**: a query details summary (metrics, pruning, wall-clock breakdown, IO) and a detailed DAG of operators with *Time spent*, *Memory peak*, and *Rows*
- **Performance insights** name the problem for you (`COVERAGE_STATS_DELTA`, `EXPLODING_JOIN`, `DATA_SPILL`, `DATA_SKEW`, ...)
- The same metrics are queryable in `system.query.history` (`pruned_files`, `read_files`, `spilled_local_bytes`, `shuffle_read_bytes`, ...)

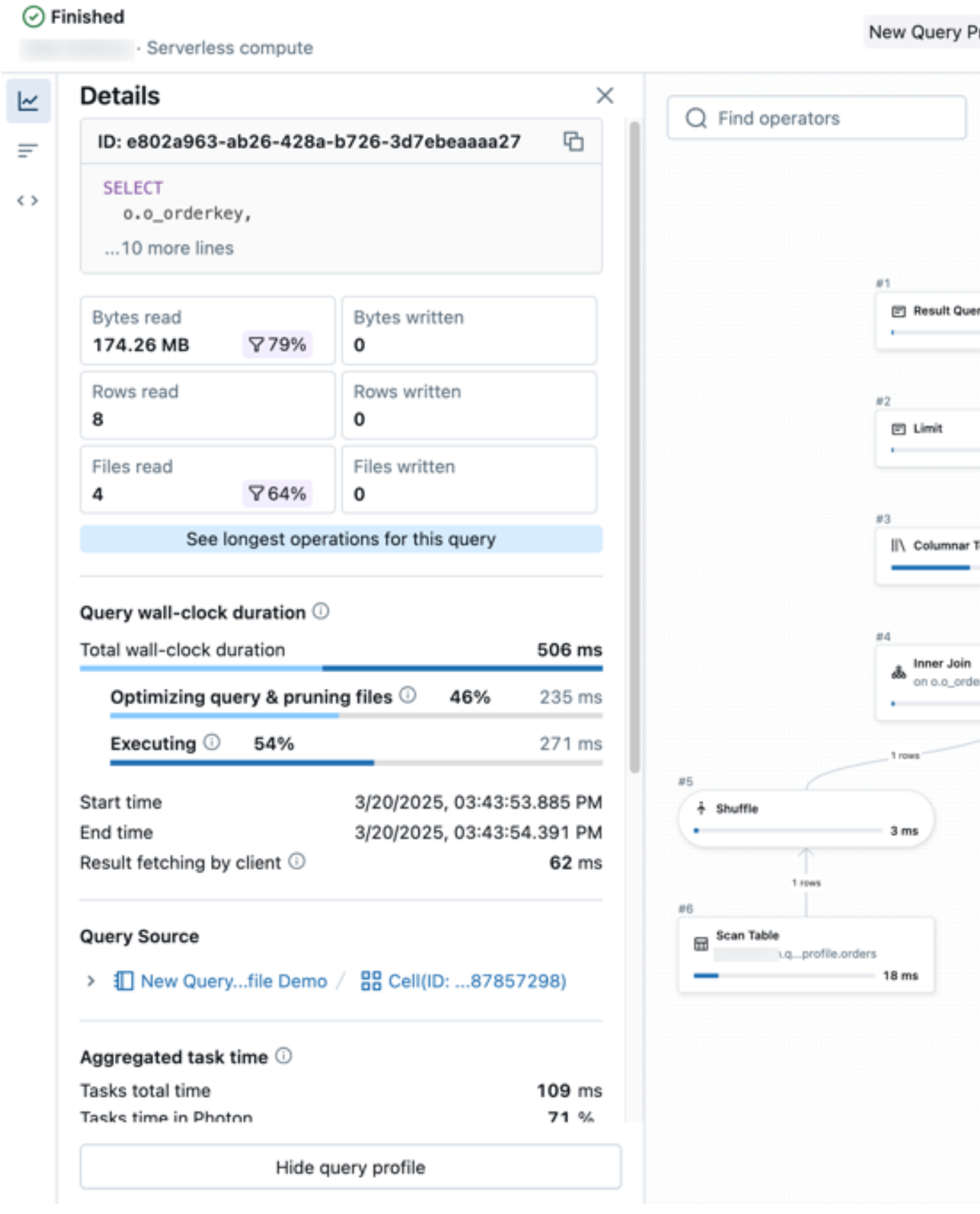
What the Exam Tests

- Use the query profile to identify performance bottlenecks, such as data-skipping inefficiencies, join strategies, and shuffle operations
- Map a symptom in the profile (low pruning, exploding join, large shuffle, spill, skew) to the right fix
- Know where the profile is available and when it isn't (queries served from the result cache)

Where to Open the Query Profile

Starting point	How
Query History (sidebar)	Click a query → query details panel → **See**
SQL editor	During or after execution, click the elapse
Notebook attached to a SQL warehouse or **serverless compute**	Click **See performance** under the cell →
Lakeflow pipelines UI	**Query History** tab of the pipeline
Jobs UI	Query details for job runs on SQL wareho
Classic all-purpose / jobs compute	Use the Spark UI (SQL/DataFrame tab); se

- To view a profile you must **own the query** or have at least **CAN MONITOR** on the SQL warehouse that ran it.
- If your workspace is enabled for serverless compute, Query History also contains SQL and Python queries run on **serverless notebooks and jobs**.
- **No profile is available for queries served from the query cache.** Make a trivial change (for example, change or remove the `LIMIT`) to force execution.
- For Databricks SQL queries you can also open the profile in the Spark UI: kebab menu → **Open in Spark UI**.


Finished
· Serverless compute
New Query P
Details
ID: e802a963-ab26-428a-b726-3d7ebeaaaa27
SELECT
o.o_orderkey,
...10 more lines
Bytes read
174.26 MB
79%
Bytes written
0
Rows read
8
Rows written
0
Files read
4
64%
Files written
0
See longest operations for this query
Query wall-clock duration
Total wall-clock duration
506 ms
Optimizing query & pruning files
46%
235 ms
Executing
54%
271 ms
Start time
3/20/2025, 03:43:53.885 PM
End time
3/20/2025, 03:43:54.391 PM
Result fetching by client
62 ms
Query Source
New Query...file Demo
Cell(ID: ...87857298)
Aggregated task time
Tasks total time
109 ms
Tasks time in Photon
71 %
Hide query profile
Find operators
#1
Result Quer
#2
Limit
#3
Columnar T
#4
Inner Join
on o.o_orde
1 rows
#5
Shuffle
3 ms
1 rows
#6
Scan Table
i.q...profile.orders
18 ms

Query Profile in the Databricks SQL editor visualizing per-operator execution details.

Anatomy of the Query Profile

Query Details Panel (Summary)

Item	Use it to
Query status	Queued, Running, Finished, Failed, or Cancelled
Query metrics	Popular metrics; the **filter icons show the percent of data pruned** during
Query wall-clock duration	Total elapsed time, broken down into **scheduling**, **query optimization and**
Aggregated task time	Combined task time across all cores; much longer than wall-clock means goo
Input/Output (IO)	Data read and written
See longest operators	Opens **Top operators**: the most expensive operators
Performance insights summary	The most impactful insights, ranked by projected effect on total task duration

Detailed Profile

- **Left side** tabs: **Details** (summary metrics), **Top operators**, **Query text**, and **Performance insights** (full detail for each insight).
- **Right side**: the operator **DAG**. Switch the displayed metric between **Time spent**, **Memory peak**, and **Rows**; search for operators or columns; click an operator for detailed metrics and its description.
- Some operations are hidden by default because they're unlikely to be bottlenecks. Use **Enable verbose mode** to see all operations and extra metrics.
- Some non-Photon operations run as a group and share the parent operator's metrics.
- **Photon**: the Execution Details view shows the percentage of task time spent in Photon, and the plan colours **Photon operators purple** and standard operators **grey**.

Common Operators

Operator	Meaning
Scan	Data read from a data source and output as rows
Filter	Rows filtered by a criterion such as a `WHERE` clause
Join	Rows from multiple relations combined into one set
Shuffle	Data redistributed or repartitioned between executors; **expensive** because it moves data acro
Hash / Sort	Rows grouped by a key and aggregated (`SUM`, `COUNT`, `MAX`)
Union	Rows from relations with the same schema concatenated

A Reading Workflow

```
flowchart TD
    Start[Slow query] --> Sum[Query details:<br/>wall-clock breakdown, pruning %,
↪   IO]
    Sum --> Ins{Performance<br/>insights?}
    Ins -->|Yes| Fix[Follow the insight's recommendation]
    Ins -->|No| Top[Top operators / DAG by Time spent]
    Top --> Scan{Scan dominates?}
    Scan -->|Yes| Skip[Data-skipping problem:<br/>layout, stats, filters]
    Scan -->|No| Join{Join or Shuffle dominates?}
```

```
    Join -->|Join rows explode or shrink| JoinFix[Fix join condition / filter
↪   earlier /<br/>broadcast the small side]
    Join -->|Shuffle, high Memory peak| Spill[Spill or skew:<br/>bigger
↪   warehouse, salting, AQE]
```

Symptom → Cause → Fix

What you see in the profile	Likely cause
Scan is the top operator; low pruning percentage	Filters don't hit the layout or statistics
Many small files read (`DATA_FILE_SIZE`)	Small-file problem
Join outputs far more rows than it reads (`EXPLODING_JOIN`)	Missing or wrong join condition, duplicate keys
Join outputs far fewer rows than it reads (`SELECTIVE_JOIN`)	Filtering happens after the join
Large Shuffle feeding a join of a big and a small table	Shuffled join where a broadcast would do
High *Memory peak* and `DATA_SPILL`	Operator state doesn't fit in memory
A few tasks far slower than the rest (`DATA_SKEW`)	Uneven key distribution
Long scheduling portion, `EXCESSIVE_QUEUE_TIME`	Warehouse at capacity
`COVERAGE_PHOTON`	Operation not supported by Photon

Identifying Data-Skipping Inefficiencies

Data skipping is efficient when the scan reads only the files that can contain matching rows. In the profile:

1. **Query metrics** in the details panel show the **percent of data pruned** (filter icons).
2. The **wall-clock breakdown** includes *query optimization and file pruning* time.
3. In the DAG, a **Scan** with most of the *Time spent* and *Rows* far above what the query returns points to weak skipping.
4. **Performance insights** name the layout problem:

Insight	Meaning
`COVERAGE_FILTER_KEYS_CLUSTERING`	Table is clustered by keys the scan doesn't filter on
`COVERAGE_FILTER_KEYS_PARTITIONING`	Table is partitioned by keys the scan doesn't filter on
`COVERAGE_STATS_DELTA`	Delta skipping statistics missing or incomplete for the filter
`DATA_FILE_SIZE`	Scan reads many small files
`MANUAL_DATA_LAYOUT`	Manually optimized table that could benefit from automatic
`WIDE_PROJECTION`	Query projects all columns
`AUTO_LIQUID_CLUSTERING` (applied acceleration)	Query read less data thanks to automatic liquid clustering

```sql
-- Stats "Unused": the filter converts the column's type, so min/max can't be used
SELECT * FROM main.sales.orders WHERE CAST(order_date AS STRING) = '2026-10-15';

-- Skippable: compare the column directly
SELECT order_id, amount FROM main.sales.orders WHERE order_date =
↪  DATE'2026-10-15';
```

Confirm pruning at scale with `system.query.history`:

Column	Meaning
`read_files` / `pruned_files`	Files read after pruning / files pruned

Column	Meaning
`read_files_bytes` / `pruned_files_bytes`	Bytes of files read / pruned after partition and file pruning
`read_rows`, `produced_rows`	Rows read / rows returned
`read_io_cache_percent`	Percent of persistent data bytes read from the IO (disk) cache

---

Identifying Join Strategies

Strategy	How it works
Broadcast hash join	The small side is broadcast to every executor; the large
Shuffled hash join	Both sides are shuffled by the join key; a hash table is b
Sort-merge join	Both sides are shuffled and sorted, then merged
Nested loop join (broadcast or shuffle-and-replicate)	Every row compared with every row

How to read it in the profile:

- Click the **Join** operator for its details and description. If **Shuffle** operators feed **both** join inputs, it is a shuffled join (hash or sort-merge). If the small input reaches the join without a shuffle of the big side, it is a broadcast join.
- For the exact physical operator names (`BroadcastHashJoin`, `ShuffledHashJoin`, `SortMergeJoin`), use **Open in Spark UI** or `EXPLAIN`; see EXPLAIN Plans & AQE.
- Compare *Rows* in vs out of the join: `EXPLODING_JOIN` and `SELECTIVE_JOIN` insights flag both extremes; `REDUNDANT_JOIN` flags an outer join that changed nothing.
- `HISTORY_BASED_JOIN_STRATEGY` (applied acceleration) means Databricks chose a broadcast join instead of shuffling, based on measurements from earlier runs of similar queries.
- On Photon, sort-merge joins are replaced with hash joins.

Levers when the strategy is wrong:

```
-- Hints (priority if both sides are hinted: BROADCAST > MERGE > SHUFFLE_HASH >
 ↪  SHUFFLE_REPLICATE_NL)
SELECT /*+ BROADCAST(c) */ o.order_id, c.segment
FROM main.sales.orders o
JOIN main.sales.customers c ON o.customer_id = c.customer_id;

SELECT /*+ SHUFFLE_HASH(c) */ * FROM main.sales.orders o JOIN
 ↪  main.sales.customers c USING (customer_id);
SELECT /*+ MERGE(c) */ * FROM main.sales.orders o JOIN main.sales.customers c
 ↪  USING (customer_id);

-- Give the optimizer statistics (COVERAGE_STATS_OPTIMIZER insight)
ANALYZE TABLE main.sales.customers COMPUTE STATISTICS FOR ALL COLUMNS;
```

Setting	Default	Effect
`spark.sql.autoBroadcastJoinThreshold`	10 MB (Apache Spark)	Max table size broad
`spark.databricks.adaptive.autoBroadcastJoinThreshold`	30MB	AQE threshold for sw

A `BROADCAST` hint broadcasts the hinted side regardless of `autoBroadcastJoinThreshold`, but Databricks doesn't guarantee a hinted strategy if it doesn't support the join type.

Identifying Shuffle, Spill, and Skew

Shuffle operators redistribute data between executors and are expensive. In the DAG, sort by *Time spent* and *Memory peak* and look at shuffles feeding joins, aggregations, and windows.

Signal	Where
Large Shuffle operators	DAG, Top operators; `shuffle_read_bytes` in `system.qu`
`DATA_SPILL`	Performance insights; `spilled_local_bytes`
`DATA_SKEW`	Performance insights
Aggregated task time ≫ wall-clock	Query details
Aggregated task time < wall-clock, long scheduling	Query details; `EXCESSIVE_QUEUE_TIME`

Adaptive query execution is on by default and re-optimizes at each shuffle or broadcast exchange: it can switch sort-merge joins to broadcast hash joins, coalesce small post-shuffle partitions, split skewed partitions, and propagate empty relations. The profile shows the plan that actually ran, unlike SQL EXPLAIN, which shows only the initial plan. On classic compute, `spark.sql.shuffle.partitions` defaults to 200 (set it to `auto` for auto-optimized shuffle); on serverless it defaults to `auto`.

Find Bottlenecks Across Many Queries

```sql
-- Worst offenders in the last day: spill, shuffle, and poor pruning
SELECT statement_id,
       executed_by,
       total_duration_ms,
       spilled_local_bytes,
       shuffle_read_bytes,
       read_files,
       pruned_files,
       from_result_cache
FROM system.query.history
WHERE start_time >= current_timestamp() - INTERVAL 1 DAY
  AND execution_status = 'FINISHED'
ORDER BY spilled_local_bytes DESC, shuffle_read_bytes DESC
LIMIT 20;
```

Then open the profile for a `statement_id` from Query History to see which operator caused it.

Query Profile vs EXPLAIN vs Spark UI

Tool	Shows
Query profile	Executed plan with per-operator time, rows, memory; insights; pruning %
EXPLAIN / `df.explain()`	Planned physical plan (SQL EXPLAIN doesn't run the query, so no AQE change
Spark UI (SQL/DataFrame tab)	Executed plan DAG, stages, tasks, executors

Sharing a Profile

- **Share** copies a URL; the recipient needs **CAN MANAGE** on the query.
- Otherwise **Download** the profile as JSON; others **Import query profile (JSON)** from the Query History kebab menu (it loads only into their browser session).

Use Cases

- **Dashboard query got slower after a schema change**: the profile shows the Scan dominating and `COVERAGE_STATS_DELTA` with status *Unavailable*: the new filter column is beyond the first 32 columns. Adding it to `delta.dataSkippingStatsColumns` and recomputing statistics restores skipping.
- **Fact-dimension join taking minutes**: Shuffle operators feed both inputs of the join although the dimension is small; a `BROADCAST` hint (or fresh statistics) removes the shuffle of the fact table.
- **Nightly aggregation on a Small warehouse**: `DATA_SPILL` and high *Memory peak* on the Hash aggregate; scaling the warehouse up removes the spill.
- **Pipeline review**: the Query History tab in the Lakeflow pipelines UI shows a flow flagged `FLOW_FULL_RECOMPUTE`; rewriting the query for incremental support cuts bytes read.

Common Issues & Errors

"Query profile is not available" **Cause:** the result came from the query cache.

Fix: change the query trivially (for example the `LIMIT`) or `SET use_cached_result = false` while testing.

Can't See Another User's Query Profile **Cause:** you need to own the query or have **CAN MONITOR** on the warehouse.

Fix: ask for CAN MONITOR, or have the owner download the profile JSON for you to import.

Profile Shows Low Pruning Even with Filters **Cause:** filters on columns that aren't clustering/partition keys, missing statistics, or a type conversion in the filter.

Fix: follow `COVERAGE_FILTER_KEYS_*` / `COVERAGE_STATS_DELTA`; consider `CLUSTER BY AUTO`.

Expected a Broadcast Join but Got a Shuffled Join **Cause:** the small side's estimated size is above the threshold or statistics are missing.

Fix: `ANALYZE TABLE ... COMPUTE STATISTICS`, a `BROADCAST` hint, or rely on AQE's runtime switch (30MB threshold).

Cartesian-Like Join **Cause:** missing or non-equi join condition (nested loop join, `EXPLODING_JOIN`).

Fix: add the equality condition; filter inputs first.

Exam Tips

Tip

- **Data-skipping problem** → look at the **pruning %** in query details, a dominant **Scan**, and insights like `COVERAGE_STATS_DELTA` / `COVERAGE_FILTER_KEYS_CLUSTERING`; fix with layout (liquid clustering), statistics, and direct column comparisons.
- **Join problem** → inspect the **Join** node and whether **Shuffle** operators feed both inputs; broadcast the small side; `EXPLODING_JOIN` = output rows ≫ input rows.
- **Shuffle problem** → Shuffle operators with high *Time spent* / *Memory peak*; `DATA_SPILL` → bigger warehouse; `DATA_SKEW` → salting or pre-aggregation.

- **Where**: Query History, SQL editor, notebooks on SQL warehouse/serverless (**See performance**), pipelines and Jobs UI; classic clusters → Spark UI.
- **Not available for cached results**; needs ownership or **CAN MONITOR** on the warehouse.

Key Takeaways

- **The query profile shows what actually ran**: an operator DAG with *Time spent*, *Memory peak*, and *Rows*, plus Top operators and Photon coverage.
- **Query details** give the fast triage view: pruning percentage, wall-clock breakdown (scheduling, optimization and file pruning, execution), aggregated task time, and IO.
- **Performance insights** turn common problems into named findings with recommendations, from data layout (`COVERAGE_STATS_DELTA`, `DATA_FILE_SIZE`) to joins (`EXPLODING_JOIN`, `SELECTIVE_JOIN`) to resources (`DATA_SPILL`, `DATA_SKEW`, `EXCESSIVE_QUEUE_TIME`).
- **Join strategy**: Shuffle operators on both join inputs mean a shuffled join; a broadcast join avoids shuffling the large side. Hints, statistics, and AQE (30MB runtime threshold) steer it.
- **Shuffle is the expensive operator**; spill and skew show up as insights and in `system.query.history` (`spilled_local_bytes`, `shuffle_read_bytes`).
- **`system.query.history`** holds the same metrics for every statement (`pruned_files`, `read_files`, `read_io_cache_percent`, `from_result_cache`), so you can find bad queries in bulk.

Related Topics

- EXPLAIN Plans & AQE - physical plan operators and runtime re-optimization
- Spark Tuning - shuffle partitions, broadcast, memory
- Data Skipping: Liquid Clustering vs Partitioning and Z-ORDER - fixing poor pruning
- Managed Tables, Predictive Optimization, and Automatic Clustering - automatic layout maintenance
- Disk Caching - IO cache and result cache
- Photon Diagnostics and Optimization (Part 1) - Photon coverage in the profile
- Spark UI Debugging - classic-compute equivalent
- System Tables - `system.query.history` and others

Official Documentation

- Query profile
- Query performance insights
- Query history
- Query history system table reference
- Adaptive query execution
- Hints (join and partitioning hints)
- What is Photon?
- Apache Spark SQL performance tuning (broadcast threshold)

Spark Tuning

Understanding Spark configuration and tuning is essential for optimizing performance of data engineering workloads. This guide covers key configurations, Adaptive Query Execution (AQE), and optimization strategies.

Overview

```
flowchart TB
    subgraph Tuning["Spark Tuning Areas"]
        Shuffle[Shuffle Optimization]
        Memory[Memory Management]
        Parallelism[Parallelism]
        AQE[Adaptive Query Execution]
        Joins[Join Optimization]
    end

    subgraph Outcome["Performance Outcomes"]
        Faster[Faster Queries]
        Lower[Lower Cost]
        Stable[Stable Execution]
    end

    Tuning --> Outcome
```

Overview

- **AQE** is on by default: it coalesces partitions, handles skew joins, and switches join strategies at runtime
- **Shuffle and joins**: size `spark.sql.shuffle.partitions` to about 128 MB per partition; the broadcast threshold defaults to 10 MB
- **`coalesce` vs `repartition`**: `coalesce` reduces partitions without a shuffle; `repartition` shuffles to rebalance
- **Memory and skew**: `spark.memory.fraction`, salting for extreme skew, and predicate and column pushdown

What the Exam Tests

- Use the query profile to identify performance bottlenecks, such as data-skipping inefficiencies, join strategies, and shuffle operations.
- Pick the configuration change for a shuffle, join, or skew symptom
- Know which tuning AQE already does without manual settings

Key Configurations

Shuffle Configuration

Configuration	Default	Description
`spark.sql.shuffle.partitions`	200	Number of partitions for shuffles
`spark.sql.adaptive.enabled`	true	Enable AQE
`spark.sql.adaptive.coalescePartitions.enabled`	true	Auto-coalesce shuffle partitions
`spark.sql.adaptive.coalescePartitions.minPartitionSize`	1MB	Minimum partition size

Memory Configuration

Configuration	Default	Description
`spark.executor.memory`	Varies	Executor heap memory
`spark.executor.memoryOverhead`	10%	Off-heap memory

Configuration	Default	Description
`spark.memory.fraction`	0.6	Fraction of heap for execution/storage
`spark.memory.storageFraction`	0.5	Fraction of memory.fraction for storage

Join Configuration

Configuration	Default	Description
`spark.sql.autoBroadcastJoinThreshold`	10MB	Threshold for broadcast join
`spark.sql.adaptive.autoBroadcastJoinThreshold`	Same	AQE broadcast threshold
`spark.sql.broadcastTimeout`	300s	Broadcast timeout

Adaptive Query Execution (AQE)

AQE Benefits

```
flowchart LR
    subgraph Without["Without AQE"]
        Static[Static Plan]
        Fixed[Fixed Partitions]
    end

    subgraph With["With AQE"]
        Dynamic[Dynamic Plan]
        Adaptive[Adaptive Partitions]
        Skew[Skew Handling]
    end

    Query[Query] --> Without
    Query --> With
    With --> Better[Better Performance]
```

AQE Features

Feature	Description	Benefit
Coalesce Partitions	Combine small shuffle partitions	Reduce overhead
Skew Join	Split skewed partitions	Avoid stragglers
Local Shuffle Reader	Skip shuffle for small data	Reduce I/O
Dynamic Join Strategy	Switch join type at runtime	Optimal join

Enabling AQE

```
# AQE is enabled by default in Databricks
# Verify it's enabled

spark.conf.get("spark.sql.adaptive.enabled")  # Should be "true"

# Enable specific AQE features

spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
```

```
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.adaptive.localShuffleReader.enabled", "true")
```

AQE Coalesce Partitions

```
# Before AQE: 200 partitions regardless of data size
# After AQE: Partitions automatically coalesced

# Configure minimum partition size

spark.conf.set("spark.sql.adaptive.coalescePartitions.minPartitionSize", "64MB")

# Configure initial partitions (AQE will coalesce if needed)

spark.conf.set("spark.sql.adaptive.coalescePartitions.initialPartitionNum",
 ↪  "200")
```

AQE Skew Join Handling

```
# Enable skew join optimization

spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")

# Configure skew thresholds

spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionFactor", "5")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes",
 ↪  "256MB")

# A partition is skewed if:
# size > skewedPartitionFactor * median_size AND
# size > skewedPartitionThresholdInBytes
```

Shuffle Optimization

Understanding Shuffle

```
flowchart LR
    subgraph Stage1["Stage 1"]
        M1[Map Task 1]
        M2[Map Task 2]
        M3[Map Task 3]
    end

    subgraph Shuffle["Shuffle"]
        S[Network Transfer]
    end

    subgraph Stage2["Stage 2"]
        R1[Reduce Task 1]
        R2[Reduce Task 2]
    end

    M1 --> S
    M2 --> S
```

```
    M3 --> S
    S --> R1
    S --> R2
```

Reducing Shuffles

```
# Bad: Multiple shuffles

(df.groupBy("a").agg(sum("b"))
  .join(other_df, "a")
  .groupBy("a").agg(count("*")))

# Better: Combine operations

(df.join(other_df, "a")
  .groupBy("a").agg(sum("b"), count("*")))
```

Shuffle Partition Tuning

```
# Rule of thumb: 128MB per partition
# Total data size / 128MB = number of partitions

# For 100GB data:
# 100GB / 128MB ≈ 800 partitions

spark.conf.set("spark.sql.shuffle.partitions", "800")

# With AQE, start high and let it coalesce

spark.conf.set("spark.sql.shuffle.partitions", "1000")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
```

Pre-Shuffle Operations

```
# Filter before shuffle

(df.filter(col("date") >= "2024-01-01")
  .groupBy("customer_id").agg(sum("amount")))

# Select only needed columns before shuffle

(df.select("customer_id", "amount")
  .groupBy("customer_id").agg(sum("amount")))
```

Join Optimization

Join Types Performance

Join Type	Performance	When Used
Broadcast Hash	Best	Small table < threshold
Sort Merge	Good	Large tables, sorted
Shuffle Hash	Medium	Unsorted, medium tables
Broadcast Nested Loop	Poor	Cross joins, non-equi
Cartesian	Worst	No join condition

Join Type	Performance	When Used

Broadcast Join

```
# Automatic broadcast (table < 10MB)

small_df.join(large_df, "key")  # Broadcasts small_df

# Increase threshold for larger broadcasts

spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "100MB")

# Force broadcast with hint

from pyspark.sql.functions import broadcast

result = large_df.join(broadcast(medium_df), "key")
```

```
-- SQL broadcast hint
SELECT /*+ BROADCAST(small_table) */ *
FROM large_table l
JOIN small_table s ON l.id = s.id;
```

Sort Merge Join Optimization

```
# Pre-sort data for repeated joins

df_sorted = df.sortWithinPartitions("key")
df_sorted.write.bucketBy(100, "key").saveAsTable("bucketed_table")

# Bucketed tables skip shuffle for joins

t1 = spark.table("bucketed_table_1")
t2 = spark.table("bucketed_table_2")
result = t1.join(t2, "key")  # No shuffle needed
```

Join Hints

```
-- Force broadcast
SELECT /*+ BROADCAST(t2) */ * FROM t1 JOIN t2 ON t1.id = t2.id;

-- Force sort merge
SELECT /*+ MERGE(t2) */ * FROM t1 JOIN t2 ON t1.id = t2.id;

-- Force shuffle hash
SELECT /*+ SHUFFLE_HASH(t2) */ * FROM t1 JOIN t2 ON t1.id = t2.id;

-- Force shuffle replicate nested loop
SELECT /*+ SHUFFLE_REPLICATE_NL(t2) */ * FROM t1 JOIN t2 ON t1.id = t2.id;
```

Memory Tuning

Memory Layout

```
flowchart TB
    subgraph Executor["Executor Memory"]
        subgraph Heap["JVM Heap"]
            Reserved[Reserved 300MB]
            User[User Memory 40%]
            Unified[Unified Memory 60%]
        end
        Overhead[Memory Overhead 10%]
    end

    subgraph Unified["Unified Memory"]
        Execution[Execution Memory]
        Storage[Storage/Cache Memory]
    end
```

Memory Configuration

```
# Executor memory settings

spark.conf.set("spark.executor.memory", "8g")
spark.conf.set("spark.executor.memoryOverhead", "2g")

# Memory fraction settings

spark.conf.set("spark.memory.fraction", "0.6")  # 60% for execution+storage
spark.conf.set("spark.memory.storageFraction", "0.5")  # 50% of that for storage

# For memory-intensive operations

spark.conf.set("spark.memory.fraction", "0.8")  # More for execution
```

Avoiding OOM

```
# Increase memory overhead for Python UDFs

spark.conf.set("spark.executor.memoryOverhead", "4g")

# Reduce parallelism if tasks are too memory-intensive

spark.conf.set("spark.sql.shuffle.partitions", "400")

# Enable off-heap memory

spark.conf.set("spark.memory.offHeap.enabled", "true")
spark.conf.set("spark.memory.offHeap.size", "4g")
```

Parallelism Tuning

Task Parallelism

```
# Default parallelism for RDD operations

spark.conf.set("spark.default.parallelism", "200")

# Shuffle parallelism for DataFrame operations
```

```
spark.conf.set("spark.sql.shuffle.partitions", "200")

# Rule: 2-3 tasks per core
# cores → 200-300 partitions
```

Repartitioning

```
# Increase parallelism

df = df.repartition(500)

# Decrease parallelism (preserves order)

df = df.coalesce(100)

# Repartition by column (for joins)

df = df.repartition("join_key")

# Repartition with both count and column

df = df.repartition(100, "join_key")
```

Coalesce vs Repartition

Operation	Shuffle	Use Case
coalesce(n)	No	Reduce partitions
repartition(n)	Yes	Increase or rebalance
repartition(n, col)	Yes	Partition by column

Delta Lake Optimization

Write Optimization

```
# Optimize write file sizes

spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true")

# Auto-compact after writes

spark.conf.set("spark.databricks.delta.autoCompact.enabled", "true")

# Target file size

spark.conf.set("spark.databricks.delta.optimizeWrite.fileSize", "128mb")
```

Read Optimization

```
# Enable data skipping

spark.conf.set("spark.databricks.delta.stats.skipping", "true")

# Optimize parquet reading
```

```python
spark.conf.set("spark.sql.parquet.filterPushdown", "true")
spark.conf.set("spark.sql.parquet.enableVectorizedReader", "true")
```

Photon Engine

Tungsten: Spark's Native Execution Layer Tungsten is Apache Spark's built-in execution engine (Spark 1.5+) that provides off-heap memory management via `UnsafeRow`, Whole-Stage Code Generation (WSCG), and vectorized Parquet/ORC reading. Photon replaces Tungsten for supported operations with a C++ implementation that removes JVM overhead entirely; for unsupported operations, Spark falls back to Tungsten automatically.

See Spark Fundamentals — Tungsten for full details.

What is Photon

```
Photon is Databricks' native vectorized query engine:
- Written in C++ for better performance
- Vectorized processing (SIMD)
- Better memory management
- Faster than Spark for supported operations
```

Photon Benefits

Operation	Improvement
Aggregations	2-8x faster
Joins	2-5x faster
Filters	2-4x faster
String operations	3-10x faster

Enabling Photon

```
Photon is enabled by:
1. Selecting Photon-enabled cluster type
2. Choosing Photon runtime
3. No code changes needed

Check if Photon is active:
- Spark UI shows "Photon" in query plans
- Query profile shows Photon operators
```

Photon-Optimized Operations

```sql
-- These benefit most from Photon:
-- Hash aggregations
SELECT customer_id, SUM(amount), COUNT(*)
FROM orders
GROUP BY customer_id;

-- Hash joins
SELECT * FROM orders o
JOIN customers c ON o.customer_id = c.id;

-- Filters with string operations
SELECT * FROM events
WHERE event_type LIKE '%purchase%';
```

Query Optimization

Predicate Pushdown

```
# Filters pushed to scan

df = (spark.read.parquet("data/")
    .filter(col("date") == "2024-01-15"))  # Pushed to file scan

# Check query plan

df.explain()
# Should show PushedFilters in FileScan
```

Column Pruning

```
# Only read needed columns

df = (spark.read.parquet("data/")
    .select("id", "name", "amount"))  # Only 3 columns read

# Avoid SELECT *
# Bad: df.select("*")
# Good: df.select("col1", "col2", "col3")
```

Cache Strategies

```
# Cache for repeated access

df = spark.table("orders").cache()

# Use MEMORY_AND_DISK for large datasets

from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_AND_DISK)

# Unpersist when done

df.unpersist()
```

```
-- SQL caching
CACHE TABLE orders;
UNCACHE TABLE orders;

-- With eager caching
CACHE TABLE orders SELECT * FROM orders WHERE date >= '2024-01-01';
```

Use Cases

- **Skew Join Mitigation via AQE**: Ensuring Adaptive Query Execution's `skewJoin` property is enabled to automatically identify and split massive keys during a heavy fact-to-dimension join, stopping straggler tasks from destroying cluster utilization.
- **Interactive Query Acceleration**: Turning on the Photon native vectorized engine for a Databricks SQL Warehouse to aggressively accelerate heavily grouped hash aggregations and string manipulation filters for BI analysts without altering any underlying SQL.

Common Issues & Errors

Shuffle Spill to Disk **Scenario:** Tasks spilling to disk, slow performance.

Fix: Increase memory or reduce partition size:

```
# Increase memory per partition

spark.conf.set("spark.sql.shuffle.partitions", "1000")  # Smaller partitions

# Or increase executor memory
# In cluster config: spark.executor.memory = 16g
```

Skewed Data in Joins **Scenario:** One task takes much longer than others.

Fix: Enable AQE skew handling or salt keys:

```
# Enable skew join

spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")

# Or manually salt skewed keys

from pyspark.sql.functions import rand, concat, lit

# Add salt to skewed key

salted_df = df.withColumn("salted_key",
    concat(col("key"), lit("_"), (rand() * 10).cast("int")))
```

Broadcast Timeout **Scenario:** BroadcastExchangeExec timeout error.

Fix: Increase timeout or reduce table size:

```
# Increase timeout

spark.conf.set("spark.sql.broadcastTimeout", "600")

# Or disable broadcast for this query

spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1")
```

Out of Memory **Scenario:** ExecutorLostFailure or OutOfMemoryError.

Fix: Tune memory settings:

```
# Increase overhead for Python/UDFs

spark.conf.set("spark.executor.memoryOverhead", "4g")

# Reduce parallelism

spark.conf.set("spark.sql.shuffle.partitions", "100")

# Enable off-heap

spark.conf.set("spark.memory.offHeap.enabled", "true")
spark.conf.set("spark.memory.offHeap.size", "4g")
```

Exam Tips

1. **AQE** - Enabled by default, handles coalesce, skew, dynamic joins
2. **shuffle.partitions** - Default 200, tune based on data size
3. **Broadcast threshold** - Default 10MB, increase for larger broadcasts
4. **Coalesce vs Repartition** - Coalesce no shuffle, repartition has shuffle
5. **Photon** - Native engine, 2-8x faster for supported operations
6. **Skew handling** - AQE or salting keys
7. **Memory fraction** - 60% for execution+storage by default
8. **Predicate pushdown** - Filters pushed to file scan level
9. **Column pruning** - Only read needed columns
10. **Cache** - Use for repeated access, unpersist when done

Key Takeaways

- **AQE is on by default**: Adaptive Query Execution (`spark.sql.adaptive.enabled = true`) is enabled by default in Databricks and automatically handles partition coalescing, skew joins, and dynamic join strategy switching.
- **shuffle.partitions rule of thumb**: Target ~128 MB per shuffle partition; divide total shuffle data size by 128 MB to determine the right `spark.sql.shuffle.partitions` value.
- **Broadcast threshold**: The default broadcast threshold is 10 MB (`spark.sql.autoBroadcastJoinThresho` increase it or use `broadcast()` hints to force broadcast for tables up to a few hundred MB.
- **Coalesce vs Repartition**: `coalesce(n)` reduces partitions without a shuffle; `repartition(n)` triggers a full shuffle but evenly distributes data — use coalesce to reduce, repartition to increase or rebalance.
- **Photon acceleration**: Photon is Databricks' native C++ vectorized engine providing 2–8x speedups for aggregations, joins, and filters — enabled by selecting a Photon-compatible cluster runtime, no code changes required.
- **Memory fraction**: By default, 60% of executor heap (`spark.memory.fraction = 0.6`) is allocated for execution and storage combined; increase to 0.8 for memory-intensive shuffle or join workloads.
- **Salting for skew**: When AQE cannot fully resolve extreme data skew, manually add a random salt column to the skewed join key, join with the salt applied, then strip the salt after the join.
- **Predicate and column pushdown**: Spark automatically pushes predicates to the file scan level and reads only the columns requested — avoid SELECT * and put filters before joins to maximize these optimizations.

Related Topics

- File Sizing - Write optimization
- Z-ORDER Indexing - Read optimization
- Query Profiler - Performance analysis

Official Documentation

- Spark Configuration
- Adaptive Query Execution
- Photon
- Performance Tuning

EXPLAIN Plans & Adaptive Query Execution

This guide covers how to read and interpret Spark query plans and how Adaptive Query Execution (AQE) optimizes queries at runtime. Understanding these tools is essential for diagnosing

performance bottlenecks.

Overview

```
flowchart TB
    subgraph Analysis["Query Plan Analysis"]
        Explain[EXPLAIN Plans]
        AQE[Adaptive Query Execution]
        SparkUI[Spark UI Deep Dive]
        Memory[Memory & Spill Diagnostics]
    end

    subgraph Acceleration["Acceleration"]
        Photon[Photon Engine]
        Optimization[Query Optimization Strategies]
    end

    subgraph Outcome["Outcomes"]
        Faster[Faster Queries]
        Lower[Lower Resource Usage]
        Debug[Effective Debugging]
    end

    Analysis --> Outcome
    Acceleration --> Outcome
```

Overview

- **Reading plans**: physical plans run bottom-up from `FileScan`; `PartitionFilters` skip directories, `PushedFilters` skip data inside files
- **`EXPLAIN EXTENDED`** shows the parsed, analyzed, optimized logical, and physical plans
- **AQE** re-optimizes at shuffle boundaries, for example switching `SortMergeJoin` to `BroadcastHashJoin`; `AQEShuffleRead` / `CustomShuffleReaderExec` confirms it acted
- **Python UDFs block pushdown**; `ANALYZE TABLE` statistics feed the cost-based optimizer

What the Exam Tests

- Use the query profile to identify performance bottlenecks, such as data-skipping inefficiencies, join strategies, and shuffle operations.
- Tell from a plan whether partition pruning or predicate pushdown happened
- Confirm from a plan that AQE changed a join or split a skewed partition

Reading EXPLAIN Plans

EXPLAIN Variants

Command	Description	Detail Level
`EXPLAIN`	Physical plan only	Basic
`EXPLAIN EXTENDED`	Parsed, Analyzed, Optimized, and Physical plans	Full
`EXPLAIN FORMATTED`	Physical plan with formatted output and codegen details	Readable
`EXPLAIN COST`	Optimized plan with statistics and cost estimates	Cost-based

Using EXPLAIN in SQL

```sql
-- Basic physical plan
EXPLAIN
SELECT customer_id, SUM(amount) AS total
FROM orders
WHERE order_date >= '2024-01-01'
GROUP BY customer_id;

-- Extended plan showing all stages
EXPLAIN EXTENDED
SELECT customer_id, SUM(amount) AS total
FROM orders
WHERE order_date >= '2024-01-01'
GROUP BY customer_id;

-- Formatted plan with codegen details
EXPLAIN FORMATTED
SELECT o.customer_id, c.name, SUM(o.amount) AS total
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.order_date >= '2024-01-01'
GROUP BY o.customer_id, c.name;

-- Cost-based plan with statistics
EXPLAIN COST
SELECT customer_id, SUM(amount) AS total
FROM orders
WHERE order_date >= '2024-01-01'
GROUP BY customer_id;
```

Using EXPLAIN in Python

```python
# DataFrame explain

df = (spark.table("orders")
    .filter(col("order_date") >= "2024-01-01")
    .groupBy("customer_id")
    .agg(sum("amount").alias("total")))

# Simple physical plan

df.explain()

# Extended plan (all stages)

df.explain(mode="extended")

# Formatted output

df.explain(mode="formatted")

# Cost-based plan

df.explain(mode="cost")
```

Plan Structure (Extended)

```
== Parsed Logical Plan ==
  Raw SQL/DataFrame operations as parsed by Spark

== Analyzed Logical Plan ==
  Resolved table names, column types, and references

== Optimized Logical Plan ==
  After Catalyst optimizer applies rules (predicate pushdown, etc.)

== Physical Plan ==
  Actual execution strategy chosen by the planner
```

Key Operators in Physical Plans

```
flowchart TB
    subgraph ScanOps["Scan Operators"]
        FileScan[FileScan parquet / delta]
        DataSourceScan[DataSourceV2Scan]
    end

    subgraph TransformOps["Transform Operators"]
        Filter[Filter]
        Project[Project - Column Selection]
        Window[Window]
        Sort[Sort]
    end

    subgraph AggOps["Aggregation Operators"]
        HashAgg[HashAggregate]
        SortAgg[SortAggregate]
    end

    subgraph JoinOps["Join Operators"]
        BHJ[BroadcastHashJoin]
        SMJ[SortMergeJoin]
        SHJ[ShuffledHashJoin]
        CP[CartesianProduct]
    end

    subgraph ShuffleOps["Shuffle Operators"]
        Exchange[Exchange - hashpartitioning]
        BroadcastExchange[BroadcastExchange]
    end

    ScanOps --> TransformOps
    TransformOps --> AggOps
    TransformOps --> JoinOps
    AggOps --> ShuffleOps
    JoinOps --> ShuffleOps
```

Join Operator Selection

Join Operator	When Chosen	Performance	Shuffle Required
BroadcastHashJoin	One side < broadcast threshold (10MB)	Best	No (broadcast)

Join Operator	When Chosen	Performance	Shuffle Required
SortMergeJoin	Both sides large, equi-join	Good	Yes
ShuffledHashJoin	Medium tables, hash hint or config	Medium	Yes
BroadcastNestedLoopJoin	Non-equi join with small table	Poor	No (broadcast)
CartesianProduct	No join condition (cross join)	Worst	Yes

Reading Plans Bottom-Up

```
Physical plans are read from BOTTOM to TOP:

*(5) HashAggregate(keys=[customer_id], functions=[sum(amount)])     <-- Final
 ↪  aggregation
+- Exchange hashpartitioning(customer_id, 200)                      <-- Shuffle
   +- *(4) HashAggregate(keys=[customer_id], functions=[partial_sum(amount)])
 ↪  <-- Partial agg
      +- *(3) Project [customer_id, amount]                         <-- Column
 ↪  selection
         +- *(2) Filter (order_date >= 2024-01-01)                  <-- Filter
 ↪  applied
           +- *(1) FileScan parquet [customer_id,amount,order_date] <-- Start here

Read order: FileScan -> Filter -> Project -> Partial Agg -> Shuffle -> Final Agg

The asterisk (*) with a number indicates a whole-stage code generation stage.
```

Identifying Partition Pruning

```
Look for PartitionFilters in the FileScan operator:

FileScan parquet default.orders[customer_id,amount,order_date]
  Batched: true
  DataFilters: []
  Format: Parquet
  PartitionFilters: [isnotnull(order_date), (order_date >= 2024-01-01)]  <--
 ↪  PRUNING!
  PushedFilters: []
  ReadSchema: struct<customer_id:string,amount:double>

PartitionFilters: Spark reads ONLY matching partitions (skips others entirely)
DataFilters: Applied after reading data from files
```

Identifying Predicate Pushdown

```
Look for PushedFilters in the FileScan operator:

FileScan parquet default.events[event_id,event_type,user_id,timestamp]
  Batched: true
  DataFilters: [isnotnull(event_type), (event_type = purchase)]
  Format: Parquet
  PartitionFilters: []
  PushedFilters: [IsNotNull(event_type), EqualTo(event_type,purchase)]  <--
 ↪  PUSHDOWN!
  ReadSchema: struct<event_id:string,event_type:string,user_id:string>
```

```
PushedFilters: Filter pushed to the data source (Parquet/Delta reader)
  - Filters rows during scan (before data enters Spark)
  - Reduces I/O significantly
```

Data Skipping Indicators

```
For Delta Lake tables, data skipping uses column statistics:

FileScan parquet delta.`/data/orders`
  ...
  PushedFilters: [GreaterThanOrEqual(amount, 1000)]
  ...
  delta.skipping.stats: {numFilesSkipped: 45, numFilesRead: 5}  <-- DATA SKIPPING

Key indicators:
- numFilesSkipped: Files skipped due to min/max stats
- numFilesRead: Files actually scanned
- High skip ratio = effective data skipping
```

```python
# Verify data skipping with metrics

df = spark.read.format("delta").load("/data/orders")
filtered = df.filter(col("amount") >= 1000)

# Check the plan for PushedFilters

filtered.explain(mode="formatted")

# Collect statistics to improve data skipping

spark.sql("ANALYZE TABLE orders COMPUTE STATISTICS FOR COLUMNS amount,
 ↪  order_date")
```

Adaptive Query Execution (AQE) Deep Dive

How AQE Changes Plans at Runtime

```
flowchart LR
    subgraph Compile["Compile Time"]
        StaticPlan[Static Physical Plan]
        Estimate[Estimated Statistics]
    end

    subgraph Runtime["Runtime"]
        Stage1[Execute Stage 1]
        Stats[Collect Real Statistics]
        Reoptimize[Re-optimize Remaining Plan]
        Stage2[Execute Stage 2 - Optimized]
    end

    Compile --> Stage1
    Stage1 --> Stats
    Stats --> Reoptimize
    Reoptimize --> Stage2

AQE operates at shuffle boundaries:
1. Execute stages up to the first shuffle
```

```
2. Collect actual runtime statistics (partition sizes, row counts)
3. Re-optimize the remaining plan using real data
4. Repeat for each subsequent stage

This is why AQE can:
- Convert SortMergeJoin to BroadcastHashJoin if one side is small
- Coalesce small partitions after shuffle
- Split skewed partitions for better load balancing
```

AQE Key Configurations

```
# Core AQE settings

spark.conf.set("spark.sql.adaptive.enabled", "true")

# Coalescing partitions

spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.minPartitionSize", "64MB")
spark.conf.set("spark.sql.adaptive.coalescePartitions.initialPartitionNum",
 ↪  "200")
spark.conf.set("spark.sql.adaptive.advisoryPartitionSizeInBytes", "128MB")

# Skew join

spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionFactor", "5")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes",
 ↪  "256MB")

# Dynamic join strategy

spark.conf.set("spark.sql.adaptive.autoBroadcastJoinThreshold", "10MB")
spark.conf.set("spark.sql.adaptive.localShuffleReader.enabled", "true")
```

CustomShuffleReaderExec

```
CustomShuffleReaderExec (or AQEShuffleRead in newer versions):
- Appears in plans when AQE modifies the shuffle reader
- Indicates AQE is actively optimizing

Before AQE:
  Exchange hashpartitioning(id, 200)
  +- ShuffleQueryStage ...

After AQE:
  CustomShuffleReaderExec
    coalesced partitions: 200 -> 15        <-- Partitions reduced!
  +- ShuffleQueryStage ...

Look for:
- "coalesced" = partitions were merged
- "skewed" = skewed partitions were split
- "local" = local shuffle reader optimization applied
```

AQE Skew Join Optimization

```
A partition is considered skewed when BOTH conditions are met:
  1. size > skewedPartitionFactor * median_partition_size
  2. size > skewedPartitionThresholdInBytes (default 256MB)

What AQE does with skewed partitions:
  - Splits the large partition into smaller sub-partitions
  - Replicates the other side of the join for each sub-partition
  - Processes sub-partitions in parallel

Plan indicator:
  SortMergeJoin [id], Inner
  :- SortMergeJoin-SkewedPartition [id]     <-- Skew handling active
  :  :- ...
```

```
# Simulate and observe AQE skew handling
# Create a skewed dataset

skewed_df = spark.range(1000000).withColumn(
    "key",
    when(col("id") < 900000, lit("hot_key"))  # 90% goes to one key
    .otherwise(col("id").cast("string"))
)

other_df = spark.range(100).withColumn(
    "key", col("id").cast("string")
).withColumn("value", lit("data"))

# Join - AQE will detect and handle skew

result = skewed_df.join(other_df, "key")
result.explain(mode="formatted")  # Look for skew indicators
```

AQE Coalescing Behavior

```
Coalescing example:

Initial shuffle: 200 partitions
Partition sizes (MB): [50, 2, 1, 80, 3, 1, 120, 5, ...]

AQE advisory partition size: 128MB

After coalescing:
  Partition 1: [50 + 2 + 1 + 80] = 133 MB  (combined 4 small partitions)
  Partition 2: [3 + 1 + 120] = 124 MB       (combined 3 partitions)
  Partition 3: [5 + ...] = ...

Result: 200 partitions -> ~15 partitions (actual data-dependent)
```

Verifying AQE Is Working

```
# Method 1: Check the final plan after execution

df = (spark.table("orders")
    .join(spark.table("customers"), "customer_id")
```

```
    .groupBy("region")
    .agg(sum("amount")))

# Force execution

df.collect()

# Now check the executed plan (shows AQE changes)

df.explain(mode="formatted")
# Look for: AdaptiveSparkPlan, CustomShuffleReaderExec, isFinalPlan=true
```

```
-- Method 2: Check AQE status in SQL
SET spark.sql.adaptive.enabled;
-- Result: true

-- Run query and check Spark UI SQL tab
-- The plan will show "AdaptiveSparkPlan isFinalPlan=true"
-- Compare initial vs final plan to see AQE changes
```

Use Cases

- **Query Bottleneck Identification**: Using `EXPLAIN FORMATTED` to dissect a sluggish aggregation pipeline and discovering a `SortMergeJoin` without partition pruning, prompting the engineer to add date filters and drastically speed up the query.
- **Adaptive Skew Handling Validation**: Reviewing the Spark physical plan to confirm the presence of `CustomShuffleReaderExec` with "skewed" partitions, proving that AQE successfully detected and split a massively skewed client ID during a critical data join.

Common Issues & Errors

Hard-to-Read Physical Plans **Scenario:** Raw `EXPLAIN` output is dense and difficult to interpret. **Fix:** Always use `EXPLAIN FORMATTED` in SQL or `.explain(mode="formatted")` in Python to get a structured, easier-to-read layout.

Missing Statistics Leading to Bad Plans **Scenario:** Optimizer chooses slow operations (e.g., SortMergeJoin over BroadcastHashJoin) for small tables. **Fix:** Run `ANALYZE TABLE <name> COMPUTE STATISTICS` so Catalyst has accurate file and row counts to inform Cost-Based Optimization.

Exam Tips

1. **Read physical plans bottom-up** - start at `FileScan` and trace up to the output operator
2. **`PartitionFilters` vs `PushedFilters`** - partition filters skip whole directories; pushed filters skip row groups inside files
3. **`EXPLAIN EXTENDED`** - shows the parsed, analyzed, optimized logical, and physical plans; use `EXPLAIN FORMATTED` for a readable layout
4. **AQE re-optimizes at shuffle boundaries** - it can switch `SortMergeJoin` to `BroadcastHashJoin` at runtime
5. **`AQEShuffleRead` / `CustomShuffleReaderExec`** - confirms AQE coalesced or split shuffle partitions
6. **Skew** - `SortMergeJoin-SkewedPartition` in the plan shows AQE split a skewed partition
7. **Python UDFs block predicate pushdown** - their filters never appear in `PushedFilters`
8. **`ANALYZE TABLE ... COMPUTE STATISTICS`** - gives the cost-based optimizer accurate statistics for join decisions

Key Takeaways

- **Read plans bottom-up**: Physical plans execute from bottom to top — start reading at `FileScan` (the data source) and trace upward to the final output operator.
- **PartitionFilters prune directories**: `PartitionFilters` in `FileScan` means entire partition directories are skipped before any data is read; `PushedFilters` filters row groups inside files — both reduce I/O but PartitionFilters has the greater impact.
- **AQE re-optimizes at shuffle boundaries**: AQE collects actual runtime statistics after each stage completes, then re-optimizes the remaining plan — enabling dynamic switching from `SortMergeJoin` to `BroadcastHashJoin` at runtime.
- **CustomShuffleReaderExec confirms AQE**: Seeing `CustomShuffleReaderExec` (or `AQEShuffleRead`) in the physical plan confirms that AQE has actively modified the shuffle reader by coalescing or splitting partitions.
- **EXPLAIN EXTENDED shows all four plans**: Parsed, Analyzed, Optimized Logical, and Physical — comparing Analyzed vs Optimized reveals exactly which Catalyst optimizer rules fired (predicate pushdown, column pruning, etc.).
- **UDFs block predicate pushdown**: Python UDFs are opaque to the Catalyst optimizer — filters applied via UDFs will never appear in `PushedFilters` and cannot benefit from data source skipping.
- **ANALYZE TABLE enables CBO**: Running `ANALYZE TABLE ... COMPUTE STATISTICS FOR COLUMNS` provides the Cost-Based Optimizer accurate column histograms and row counts for better join order and strategy decisions.
- **Skew indicator in plan**: The presence of `SortMergeJoin-SkewedPartition` in the physical plan confirms AQE's skew join optimization has detected and split a heavily skewed partition.

Related Topics

- Spark Tuning - AQE configuration, shuffle, and join tuning
- Query Profile - Find bottlenecks in the Databricks SQL query profile
- Photon, Diagnostics & Query Optimization — Part 1 - Photon acceleration, memory and spill diagnostics, Spark UI deep dive
- Photon, Diagnostics & Query Optimization — Part 2 - Query optimization strategies

Official Documentation

- EXPLAIN Statement
- Adaptive query execution
- Cost-Based Optimizer

Photon, Diagnostics & Query Optimization — Part 1

Overview

This part covers Photon engine acceleration, memory and spill diagnostics, and Spark UI analysis for Databricks workloads.

> For EXPLAIN plans and AQE internals, see EXPLAIN Plans & Adaptive Query Execution.

Overview

- **Photon**: which operations it accelerates, which fall back to Spark, how to verify it ran, and what it costs
- **Memory and spill**: executor memory areas, finding spill in the Spark UI, and fixing it
- **Spark UI deep dive**: jobs, stages, and tasks; the SQL tab plan; DAG visualization; task

metrics; straggler tasks from skew

What the Exam Tests

- Identify pertinent diagnostic information using Spark UI, cluster logs, system tables, and query profiles to troubleshoot errors.
- Use the query profile to identify performance bottlenecks, such as data-skipping inefficiencies, join strategies, and shuffle operations.
- Decide when Photon helps a workload and spot when it falls back

Photon Acceleration

What Photon Is

```
Photon is Databricks' native vectorized query engine:
- Written in C++ (not JVM-based like Spark)
- Uses columnar memory layout for SIMD processing
- Operates on batches of data (vectorized execution)
- Replaces Spark's Whole-Stage Code Generation for supported operations
- Compatible with Spark APIs (no code changes needed)
```

```
flowchart LR
    subgraph Traditional["Traditional Spark"]
        JVM[JVM Execution]
        RowBased[Row-at-a-time]
        CodeGen[Whole-Stage CodeGen]
    end

    subgraph PhotonEngine["Photon Engine"]
        Native[C++ Native Code]
        Vectorized[Vectorized / Columnar]
        SIMD[SIMD Instructions]
    end

    Query[Query] --> Traditional
    Query --> PhotonEngine
    PhotonEngine --> Faster["2-8x Faster"]
    Traditional --> Baseline[Baseline]
```

Operations Photon Accelerates

Operation Category	Specific Operations	Speedup
Scans	Parquet/Delta reads, column pruning	2-4x
Filters	Predicate evaluation, null checks	2-4x
Aggregations	SUM, COUNT, AVG, MIN, MAX, GROUP BY	2-8x
Joins	Hash joins, broadcast joins	2-5x
Writes	Parquet/Delta writes, insert operations	2-3x
String operations	LIKE, CONTAINS, string functions	3-10x
Expressions	Arithmetic, comparisons, CASE WHEN	2-4x

Operations That Fall Back to Spark

```
Photon does NOT accelerate these (falls back to Spark JVM):
- Python UDFs (regular UDFs, pandas UDFs)
```

```
- Scala/Java UDFs
- Complex nested types (deeply nested structs, maps of arrays)
- Some window functions with complex frames
- RDD operations
- Non-Delta/non-Parquet formats (CSV, JSON scanning)
- Some regex operations with advanced patterns
- User-defined aggregation functions (UDAFs)

Important: Fallback is automatic and transparent. The query
still runs; it just uses Spark for those specific operators.
```

Verifying Photon Is Being Used

```
In query plans, look for Photon-specific operators:

WITH Photon:
  PhotonGroupingAgg(keys=[customer_id], functions=[sum(amount)])
  +- PhotonBroadcastHashJoin [customer_id], [id], Inner
     :- PhotonScan parquet [customer_id, amount]
     +- PhotonBroadcastExchange
        +- PhotonScan parquet [id, name]

WITHOUT Photon (standard Spark):
  HashAggregate(keys=[customer_id], functions=[sum(amount)])
  +- BroadcastHashJoin [customer_id], [id], Inner
     :- FileScan parquet [customer_id, amount]
     +- BroadcastExchange
        +- FileScan parquet [id, name]

Key differences:
- "Photon" prefix on operators (PhotonScan, PhotonGroupingAgg, etc.)
- Spark UI shows Photon operators in green
- Query profile tab labels Photon-executed nodes
```

```
# Check if Photon is enabled on current cluster

spark.conf.get("spark.databricks.photon.enabled")

# Returns "true" if Photon is active

# Run a query and verify Photon usage

df = (spark.table("orders")
    .groupBy("customer_id")
    .agg(sum("amount").alias("total")))

df.explain(mode="formatted")
# Look for "Photon" prefix in operator names
```

Photon-Compatible Cluster Types

```
Photon is available on:
- Photon-enabled runtimes (Databricks Runtime with Photon)
- Cluster types that support Photon:
  - All-Purpose clusters (Photon runtime selected)
  - Job clusters (Photon runtime selected)
```

```
  - SQL Warehouses (Photon enabled by default)

Runtime selection:
  Standard:  "14.3.x-scala2.12"           (No Photon)
  Photon:    "14.3.x-photon-scala2.12"     (Photon enabled)

SQL Warehouses:
  - Classic SQL Warehouse: Photon enabled by default
  - Serverless SQL Warehouse: Photon enabled by default
```

Cost Implications

Compute Type	Standard DBU Rate	Photon DBU Rate	Price/Performance
All-Purpose	1.0x	~1.5-2.0x per DBU	Better if 2x+ speedup
Jobs Compute	1.0x	~1.5-2.0x per DBU	Better if 2x+ speedup
SQL Warehouse	Included	Included	Always beneficial

```
Cost-benefit analysis:
- Photon clusters have higher per-DBU rates
- But queries finish faster (often 2-8x)
- Net savings when: speedup factor > DBU rate increase

Example:
  Standard: 10 DBUs x 2 hours x $0.15 = $3.00
  Photon:   15 DBUs x 0.5 hours x $0.15 = $1.13  (62% cheaper)

Best ROI scenarios:
  - Aggregation-heavy workloads
  - Large join operations
  - String-heavy processing
  - SQL analytics dashboards
```

When Photon Provides the Most Benefit

```
HIGH benefit:
  - SQL analytics and BI queries
  - Aggregation-heavy ETL pipelines
  - Large table joins (hash joins)
  - String-heavy transformations
  - Parquet/Delta scan-heavy workloads
  - Dashboards with many concurrent queries

LOW benefit:
  - UDF-heavy workloads (Python/Scala UDFs)
  - ML training workloads
  - Streaming with minimal transformations
  - Workloads on non-Parquet formats
  - Simple pass-through pipelines
```

Memory and Spill Diagnostics

Understanding Memory Areas

```
flowchart TB
    subgraph Executor["Executor JVM Memory"]
```

```
        Reserved["Reserved Memory<br>300 MB (fixed)"]

        subgraph Usable["Usable Memory = (Total - 300MB)"]
            subgraph Unified["Unified Memory (spark.memory.fraction = 0.6)"]
                Execution["Execution Memory<br>Shuffles, joins, sorts,
↪  aggregations"]
                Storage["Storage Memory<br>Cached data, broadcast variables"]
            end
            User["User Memory (0.4)<br>Internal metadata, UDF data structures"]
        end
    end

    subgraph OffHeap["Off-Heap Memory"]
        Overhead["Memory Overhead (10%)<br>JVM internals, interned strings"]
        PythonMem["Python Worker Memory<br>pandas UDFs, Python processes"]
    end

    note1["Execution and Storage share<br>the Unified pool
↪  dynamically.<br>Execution can evict Storage<br>but not vice versa."]
```

Identifying Spill in Spark UI

```
Where to find spill metrics:

1. Spark UI -> Stages tab -> Select a stage
2. Look at "Summary Metrics" section:
   - Shuffle Spill (Memory): Data serialized to memory before spill
   - Shuffle Spill (Disk): Data actually written to disk

   If Spill (Disk) > 0, you have a spill problem.

3. Task-level details:
   - Click "Show Additional Metrics"
   - Check per-task spill values
   - Large variance indicates data skew causing spill

4. SQL tab -> Query plan:
   - Sort or HashAggregate nodes may show spill metrics
   - Look for "spill size" in operator statistics
```

Spill Metrics Explained

Metric	Meaning	Impact
Spill (Memory)	Data size before serialization	Indicates memory pressure
Spill (Disk)	Data written to local disk	Severe performance hit
Input Size	Data read by the task	Helps identify data skew
Shuffle Write	Data written for shuffle	Network + disk I/O

```
Spill severity levels:
- No spill: Ideal
- Spill (Memory) only: Minor impact (data serialized but fits in memory)
- Spill (Disk) < 1 GB: Moderate impact
- Spill (Disk) > 1 GB: Severe impact - needs immediate attention
- Spill (Disk) > 10 GB: Critical - query may fail or run extremely slowly
```

Solving Spill Problems

```
# Solution 1: Increase partitions (reduce data per partition)

spark.conf.set("spark.sql.shuffle.partitions", "2000")  # Was 200

# Solution 2: Increase executor memory
# In cluster configuration:
# spark.executor.memory = 16g  (was 8g)
# spark.executor.memoryOverhead = 4g

# Solution 3: Use broadcast joins to avoid shuffle

from pyspark.sql.functions import broadcast
result = large_df.join(broadcast(small_df), "key")

# Solution 4: Filter data earlier in the pipeline

df = (spark.table("orders")
    .filter(col("order_date") >= "2024-01-01")  # Reduce data FIRST
    .groupBy("customer_id")
    .agg(sum("amount")))

# Solution 5: Increase memory fraction for execution

spark.conf.set("spark.memory.fraction", "0.8")  # More for execution
spark.conf.set("spark.memory.storageFraction", "0.3")  # Less for cache
```

```
-- SQL: Check if a specific query spills
-- Run query, then check Spark UI SQL tab for the query
SELECT customer_id, region, SUM(amount) AS total
FROM orders
WHERE order_date >= '2024-01-01'
GROUP BY customer_id, region
ORDER BY total DESC;

-- If spilling, try reducing data volume
-- or use broadcast hint for join-heavy queries
SELECT /*+ BROADCAST(regions) */
    o.customer_id, r.region_name, SUM(o.amount) AS total
FROM orders o
JOIN regions r ON o.region_id = r.id
GROUP BY o.customer_id, r.region_name;
```

Garbage Collection Tuning

```
GC indicators of problems (Spark UI -> Executors tab):
- GC Time > 10% of task time: Tune GC
- Frequent Full GC pauses: Memory pressure

Common GC settings for Spark:
  spark.executor.extraJavaOptions:
    -XX:+UseG1GC
    -XX:InitiatingHeapOccupancyPercent=35
    -XX:G1HeapRegionSize=16m
    -XX:ConcGCThreads=4
```

```
Note: On Databricks, GC is pre-tuned. Adjust only if
monitoring shows GC is a bottleneck.
```

Spark UI Deep Dive

Jobs, Stages, Tasks Hierarchy

```
flowchart TB
    subgraph Application["Spark Application"]
        subgraph Job1["Job 1 (triggered by action)"]
            subgraph Stage1["Stage 1"]
                T1[Task 1]
                T2[Task 2]
                T3[Task 3]
            end
            subgraph Stage2["Stage 2"]
                T4[Task 4]
                T5[Task 5]
            end
        end
        subgraph Job2["Job 2"]
            subgraph Stage3["Stage 3"]
                T6[Task 6]
                T7[Task 7]
            end
        end
    end

    note["Job = triggered by an action (collect, save, count)<br>Stage = separated
↪  by shuffles<br>Task = one partition processed on one core"]
```

Spark UI Tabs

Tab	What It Shows	Key Metrics
Jobs	All jobs in the application	Duration, stages, status
Stages	Stage details and tasks	Input/output, shuffle, spill
Storage	Cached DataFrames/tables	Memory used, partitions cached
Environment	Spark configuration	All active settings
Executors	Executor health and metrics	Memory, GC, task counts
SQL	Query plans and execution	Physical plan, metrics per node

SQL Tab: Physical Plan Visualization

```
The SQL tab is the most important for query analysis:

1. Click on a query description to see its plan
2. The DAG shows operators as nodes with metrics:
   - Rows output
   - Time spent
   - Spill bytes (if any)
3. Photon operators are labeled differently
4. Hover over nodes for detailed metrics:
   - number of output rows
   - scan time
```

```
   - bytes read
   - spill metrics
```

Reading the DAG Visualization

```
DAG reading guide:

         WholeStageCodegen (3)                    [Top = Final output]
            |
        HashAggregate                             [Aggregation]
        (keys: customer_id)
        (rows: 10,000)
            |
        Exchange hashpartitioning                  [Shuffle boundary]
        (customer_id, 200)
        (data size: 500 MB)
            |
        WholeStageCodegen (2)
            |
        HashAggregate (partial)                    [Partial aggregation]
        (rows: 500,000)
            |
        Project                                    [Column selection]
        (customer_id, amount)
            |
        Filter                                     [Predicate]
        (order_date >= 2024-01-01)
        (rows in: 10M, rows out: 500K)
            |
        FileScan parquet                           [Data source scan]
        (files read: 5, files skipped: 45)

Key observations:
- Row counts DECREASE as you go up (filtering effect)
- Exchange nodes indicate shuffles (expensive)
- Partial -> Final aggregation pattern is normal (map-side combine)
- Files skipped indicates effective data skipping
```

Task Metrics to Watch

Metric	Healthy Range	Warning Sign
Duration	Uniform across tasks	10x+ variance = data skew
Input Size	Uniform across tasks	Large variance = partition skew
Shuffle Read	Proportional to data	Very large = expensive shuffle
Shuffle Write	Proportional to data	Large = needs more partitions
Spill (Disk)	0	Any value = memory pressure
GC Time	< 5% of task time	> 10% = GC problem
Peak Execution Memory	< executor memory	Near limit = risk of OOM

Identifying Straggler Tasks (Data Skew)

```
In the Stages tab, look at Summary Metrics:

Metric         Min      25th      Median    75th      Max
```

```
Duration       2s       3s       4s       5s       120s     <-- Straggler!
Input Size     50 MB    60 MB    65 MB    70 MB    5 GB     <-- Skewed!
Shuffle Read   10 MB    15 MB    20 MB    25 MB    2 GB     <-- Skewed!

When Max >> Median (more than 5x), you have data skew.

Solutions:
1. Enable AQE skew join optimization
2. Salt the skewed key
3. Pre-filter/pre-aggregate to reduce skew
4. Use broadcast join if one side is small enough
```

> **Continue reading:** Part 2 — Query Optimization Strategies, Common Issues & Exam Tips

---

Photon, Diagnostics & Query Optimization — Part 2

Overview

This part covers query optimization strategies, common issues, exam tips, and practice questions for Photon and query diagnostics.

> For Photon acceleration, memory diagnostics, and Spark UI analysis, see Part 1.

Overview

- **Catalyst optimizations**: predicate pushdown, column pruning, constant folding, join reordering, and subquery elimination
- **Statistics**: `ANALYZE TABLE ... COMPUTE STATISTICS FOR COLUMNS` with the cost-based optimizer settings
- **Photon in plans**: Photon operators carry a `Photon` prefix; Python UDFs force a fallback to Spark
- **Plan red flags**: `CartesianProduct` or `BroadcastNestedLoopJoin` usually means a missing join condition

What the Exam Tests

- Use the query profile to identify performance bottlenecks, such as data-skipping inefficiencies, join strategies, and shuffle operations.
- Fix a pushdown, spill, or broadcast problem from the evidence in a plan

Query Optimization Strategies

Predicate Pushdown

```python
# Predicate pushdown moves filters to the scan level
# This reduces data read from storage

# Good: Filter can be pushed down

df = (spark.table("orders")
    .filter(col("order_date") == "2024-01-15")   # Pushed to scan
    .filter(col("status") == "completed")         # Pushed to scan
    .select("customer_id", "amount"))
```

```
df.explain()
# Should show PushedFilters: [EqualTo(order_date,...), EqualTo(status,...)]
```

```
-- Verify pushdown in SQL
EXPLAIN
SELECT customer_id, amount
FROM orders
WHERE order_date = '2024-01-15'
  AND status = 'completed';

-- Look for PushedFilters in the output
```

Column Pruning

```
# Column pruning reads only needed columns from Parquet
# Parquet is columnar, so this skips entire column chunks

# Bad: Reads ALL columns then selects

df = spark.table("orders").select("*")

# Good: Reads only needed columns

df = spark.table("orders").select("customer_id", "amount", "order_date")

# The physical plan shows ReadSchema with only requested columns:
# FileScan parquet [customer_id,amount,order_date]
# ReadSchema: struct<customer_id:string,amount:double,order_date:date>
```

Constant Folding

```
Catalyst optimizer evaluates constant expressions at compile time:

Before optimization:
  Filter (amount > 100 * 10)

After constant folding:
  Filter (amount > 1000)

Also applies to:
  - String concatenation of literals
  - Date arithmetic with constants
  - Boolean simplification (true AND x -> x)
```

Join Reordering

```
# Catalyst can reorder joins for better performance
# based on table statistics

# The optimizer may reorder these joins:

result = (table_a
    .join(table_b, "key_ab")    # table_b is 1TB
```

```
    .join(table_c, "key_ac"))   # table_c is 10MB

# Optimizer may choose to join table_a with table_c first
# (broadcast table_c) then join with table_b

# Force specific order with hints if needed:

result = (table_a
    .join(broadcast(table_c), "key_ac")
    .join(table_b, "key_ab"))
```

Statistics Collection: ANALYZE TABLE

```
-- Collect table-level statistics
ANALYZE TABLE orders COMPUTE STATISTICS;

-- Collect column-level statistics (more useful for CBO)
ANALYZE TABLE orders COMPUTE STATISTICS FOR COLUMNS
    customer_id, amount, order_date, status;

-- Check collected statistics
DESCRIBE EXTENDED orders;

-- Column statistics include:
--   distinct count, min, max, null count, avg length, max length, histogram
```

```
# Verify statistics are being used

df = spark.table("orders").filter(col("amount") > 1000)
df.explain(mode="cost")
# Look for "Statistics(sizeInBytes=..., rowCount=...)" in the plan
```

Histogram-Based Optimization

```
-- Equi-height histograms improve join and filter estimates
ANALYZE TABLE orders COMPUTE STATISTICS FOR COLUMNS amount;

-- Histograms help the optimizer with:
--   - Skewed data distributions
--   - Range predicates (amount BETWEEN 100 AND 500)
--   - Join cardinality estimation

-- Enable histogram-based optimization
SET spark.sql.cbo.enabled = true;
SET spark.sql.cbo.joinReorder.enabled = true;
SET spark.sql.statistics.histogram.enabled = true;
```

Subquery Elimination

```
Catalyst optimizer eliminates redundant subqueries:

Before:
  SELECT *
  FROM orders
  WHERE customer_id IN (SELECT id FROM customers WHERE region = 'US')
    AND order_date IN (SELECT id FROM customers WHERE region = 'US')
```

```
                    ^-- Same subquery referenced twice

After optimization:
  - Subquery computed once and reused
  - May be converted to a join

Tip: Use EXISTS instead of IN for large subqueries:
  WHERE EXISTS (SELECT 1 FROM customers c WHERE c.id = o.customer_id AND c.region
 ↪  = 'US')
```

Practice Questions

Question 1 An engineer runs `EXPLAIN FORMATTED` on a query joining two Delta tables and sees the following in the physical plan:

```
+- SortMergeJoin [customer_id], [id], Inner
   :- Exchange hashpartitioning(customer_id, 200)
   :  +- FileScan parquet [customer_id, amount]
   +- Exchange hashpartitioning(id, 200)
      +- FileScan parquet [id, name, region]
```

The `customers` table has only 5 MB of data. What should the engineer do to improve performance?

A) Increase `spark.sql.shuffle.partitions` to 500 B) Use a broadcast hint or increase `spark.sql.autoBroadcastJoinThreshold` to broadcast the customers table C) Add a `SORT BY` clause before the join D) Disable Adaptive Query Execution

Answer

Correct Answer: B
The customers table is only 5 MB, which is below the default broadcast threshold of 10 MB but is using SortMergeJoin with two shuffles. This suggests statistics may not be available. Using `broadcast(customers)` hint or verifying that `autoBroadcastJoinThreshold >= 5MB` would eliminate both shuffles by broadcasting the small table. AQE may also detect this at runtime, but the explicit hint guarantees it.

Question 2 An engineer notices that a query plan shows `PushedFilters: []` (empty) even though the query has a `WHERE` clause filtering on a column. Which of the following is the most likely cause?

A) The table does not have statistics collected B) The filter uses a Python UDF on the column C) The table is stored in Delta format D) The cluster does not have Photon enabled

Answer

Correct Answer: B
Python UDFs are opaque to the Catalyst optimizer and cannot be pushed down to the data source. The optimizer cannot evaluate a UDF at scan time because UDF logic runs in the Python process. Statistics (A) affect cost-based optimization but not predicate pushdown. Delta tables (C) support pushdown well. Photon (D) does not affect pushdown behavior.

Question 3 In the Spark UI Stages tab, an engineer observes the following task summary for a shuffle stage:

```
Duration:     Min=2s   Median=4s   Max=180s
Input Size:   Min=50MB Median=65MB Max=8GB
```

```
Spill (Disk): Min=0    Median=0    Max=6GB
```

What does this indicate and what is the best solution?

A) Network latency - increase `spark.sql.broadcastTimeout` B) Insufficient executor memory - increase `spark.executor.memory` C) Data skew - enable AQE skew join optimization D) Too few partitions - decrease `spark.sql.shuffle.partitions`

Answer

Correct Answer: C
The extreme variance between median and max values (duration: 4s vs 180s, input: 65MB vs 8GB) is a classic indicator of data skew. One partition has vastly more data than others, causing a straggler task that also spills 6GB to disk. Enabling AQE skew join optimization (`spark.sql.adaptive.skewJoin.enabled = true`) will automatically split the large partition into smaller sub-partitions. Option D would make the problem worse by creating fewer, larger partitions.

Question 4 An engineer is running a query on a Photon-enabled cluster but the `EXPLAIN FORMATTED` output shows `HashAggregate` and `FileScan` operators without the "Photon" prefix. What is the most likely reason?

A) The query result set is too large for Photon B) The query uses a Python UDF in one of the transformations C) The Delta table has not been optimized recently D) The cluster is using too many executors

Answer

Correct Answer: B
Python UDFs force Photon to fall back to standard Spark JVM execution. When any operator in the pipeline requires Spark JVM execution (like a Python UDF), operators adjacent to it may also fall back. The solution is to replace Python UDFs with built-in Spark SQL functions that are Photon-compatible. Table optimization (C) and cluster sizing (D) do not affect whether Photon is used.

Question 5 Which EXPLAIN mode should an engineer use to see how the Catalyst optimizer has transformed the logical plan, including predicate pushdown and column pruning optimizations?

A) `EXPLAIN` B) `EXPLAIN EXTENDED` C) `EXPLAIN FORMATTED` D) `EXPLAIN COST`

Answer

Correct Answer: B
`EXPLAIN EXTENDED` shows all four plan stages: Parsed Logical Plan, Analyzed Logical Plan, Optimized Logical Plan, and Physical Plan. By comparing the Analyzed and Optimized logical plans, an engineer can see exactly which optimizer rules were applied, including predicate pushdown and column pruning. `EXPLAIN` (A) shows only the physical plan. `EXPLAIN FORMATTED` (C) shows a formatted physical plan. `EXPLAIN COST` (D) shows cost estimates but not the full optimization stages.

Use Cases

- **Cost-Based Optimizer Enhancement**: Running `ANALYZE TABLE COMPUTE STATISTICS FOR COLUMNS` on a multi-terabyte fact table so the Catalyst optimizer can exploit column histograms to accurately estimate filter selectivity and choose a broadcast join over a costly sort-merge.

- **Predicate Pushdown Verification**: Using `EXPLAIN FORMATTED` to definitively prove that a newly added filter on a massive Delta table is successfully populating the `PushedFilters` array, drastically reducing the volume of data pulled from object storage into Spark memory.

Common Issues & Errors

Query Plan Shows CartesianProduct **Scenario:** Unexpectedly slow query with CartesianProduct in the plan.

Cause: Missing or incorrect join condition.

Fix: Verify join conditions:

```
-- Bad: Missing join condition creates CartesianProduct
SELECT * FROM orders o, customers c
WHERE o.amount > 100;

-- Good: Explicit join condition
SELECT * FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.amount > 100;
```

Predicate Pushdown Not Working **Scenario:** PushedFilters is empty when you expect pushdown.

Cause: UDF in filter, non-deterministic expression, or filter after transformation.

Fix: Move simple filters before complex operations:

```
# Bad: Filter after UDF prevents pushdown

df = (spark.table("orders")
    .withColumn("processed", my_udf(col("data")))
    .filter(col("order_date") == "2024-01-15"))

# Good: Simple filters first

df = (spark.table("orders")
    .filter(col("order_date") == "2024-01-15")  # Pushed down
    .withColumn("processed", my_udf(col("data"))))
```

AQE Not Converting to Broadcast Join **Scenario:** AQE does not convert SortMergeJoin to BroadcastHashJoin despite small data.

Cause: Runtime table size exceeds adaptive broadcast threshold, or AQE broadcast is disabled.

Fix: Adjust AQE broadcast threshold:

```
# Increase the adaptive broadcast threshold

spark.conf.set("spark.sql.adaptive.autoBroadcastJoinThreshold", "100MB")

# Verify AQE is enabled

spark.conf.get("spark.sql.adaptive.enabled")  # Should be "true"
```

Photon Fallback to Spark **Scenario:** Expected Photon acceleration but plan shows standard Spark operators.

Cause: UDFs, unsupported types, or non-Photon runtime.

Fix: Check runtime and avoid UDFs:

```
# Check if Photon is enabled

print(spark.conf.get("spark.databricks.photon.enabled"))

# Replace UDFs with built-in functions
# Bad: UDF prevents Photon

from pyspark.sql.functions import udf
clean_udf = udf(lambda x: x.strip().lower())
df = df.withColumn("clean_name", clean_udf(col("name")))

# Good: Built-in functions use Photon

df = df.withColumn("clean_name", lower(trim(col("name"))))
```

Excessive Shuffle in Multi-Join Queries **Scenario:** Plan shows multiple Exchange operators (shuffles) across joins.

Fix: Repartition once on the join key, or use bucketed tables:

```
# Strategy 1: Pre-repartition on join key

orders = spark.table("orders").repartition("customer_id")
payments = spark.table("payments").repartition("customer_id")
result = orders.join(payments, "customer_id")

# Strategy 2: Bucketed tables (eliminates shuffle for repeated joins)

spark.sql("""
    CREATE TABLE orders_bucketed
    USING delta
    CLUSTERED BY (customer_id) INTO 100 BUCKETS
    AS SELECT * FROM orders
""")
```

Spill Causing Slow Aggregations **Scenario:** HashAggregate shows large spill metrics in Spark UI.

Fix: Increase partitions and memory:

```
# More partitions = less data per partition = less spill

spark.conf.set("spark.sql.shuffle.partitions", "2000")

# Or increase the memory fraction for execution

spark.conf.set("spark.memory.fraction", "0.8")

# For extreme cases, use sort-based aggregation (spills more gracefully)

spark.conf.set("spark.sql.execution.useObjectHashAggregateExec", "false")
```

Exam Tips

1. **Read plans bottom-up** - Start at FileScan (data source) and work up to the final output; the bottom operator executes first
2. **PartitionFilters vs PushedFilters** - PartitionFilters prune entire partitions (directory-level); PushedFilters push predicates into the file reader (row-group level)
3. **AQE operates at shuffle boundaries** - It collects real statistics after each stage completes and re-optimizes the remaining plan using actual data sizes
4. **Photon prefix in operators** - When Photon is active, operators are prefixed with "Photon" (e.g., PhotonGroupingAgg, PhotonScan); absence means Spark fallback
5. **Exchange = Shuffle** - Every Exchange operator in the plan means a full shuffle across the network; minimize these for better performance
6. **BroadcastHashJoin vs SortMergeJoin** - Broadcast is chosen when one side is smaller than `autoBroadcastJoinThreshold` (default 10MB); AQE can switch to broadcast at runtime if actual size is small
7. **Spill to disk is a red flag** - Any non-zero Shuffle Spill (Disk) in Spark UI means memory pressure; fix by increasing partitions, memory, or using broadcast joins
8. **ANALYZE TABLE enables CBO** - Column statistics allow the Cost-Based Optimizer to make better decisions about join order and strategy
9. **Photon does not accelerate UDFs** - Python/Scala UDFs always fall back to Spark JVM execution; replace with built-in functions to benefit from Photon
10. **CustomShuffleReaderExec confirms AQE** - Seeing this operator (or AQEShuffleRead) in the plan confirms that AQE has actively modified the execution strategy

Key Takeaways

- **Photon prefix in operators**: When Photon is active, plan operators are named `PhotonGroupingAgg`, `PhotonScan`, etc.; absence of the prefix means the operator fell back to standard Spark JVM execution.
- **UDFs disable Photon**: Python UDFs always execute in the Spark JVM and cause adjacent operators to fall back from Photon — replace UDFs with built-in Spark SQL functions to restore Photon acceleration.
- **Predicate pushdown order matters**: Place simple equality or range filters before UDFs and complex transformations so the Catalyst optimizer can push them into `PushedFilters` at the scan level.
- **ANALYZE TABLE for CBO**: `ANALYZE TABLE ... COMPUTE STATISTICS FOR COLUMNS` collects column histograms; enable `spark.sql.cbo.enabled = true` and `spark.sql.cbo.joinReorder.enabled = true` to leverage them.
- **Subquery elimination**: Catalyst automatically detects and evaluates duplicate subqueries only once — prefer `EXISTS` over `IN` for large correlated subqueries to allow the optimizer to convert them to joins.
- **Spill fix for aggregations**: If `HashAggregate` spills to disk, increase `spark.sql.shuffle.partitions` (more, smaller partitions) or raise `spark.memory.fraction` to give execution more heap space.
- **CartesianProduct is a bug signal**: Seeing `CartesianProduct` or `BroadcastNestedLoopJoin` without an explicit cross join intent almost always indicates a missing or incorrect join condition in the query.
- **AQE broadcast conversion**: If AQE does not convert `SortMergeJoin` to `BroadcastHashJoin` despite small runtime table size, increase `spark.sql.adaptive.autoBroadcastJoinThreshold` or use an explicit `broadcast()` hint.

Related Topics

- Spark Tuning - Core Spark configurations, AQE, and shuffle optimization
- File Sizing - File compaction and OPTIMIZE for scan performance
- Z-ORDER Indexing - Data layout optimization and data skipping

- Cost Optimization - Cluster selection, Photon cost/benefit analysis

Official Documentation

- EXPLAIN Statement
- Adaptive Query Execution
- Photon Runtime
- Query Performance Tuning
- Spark UI Guide
- Cost-Based Optimizer

Streaming Performance Optimization

Optimizing Structured Streaming workloads requires balancing latency, throughput, resource utilization, and cost. This guide covers key tuning strategies for streaming pipelines on Databricks.

Overview

```
flowchart TB
    subgraph Tuning["Streaming Optimization Areas"]
        Trigger[Trigger Intervals]
        State[State Management]
        Checkpoint[Checkpointing]
        Files[File Sizing]
        Source[Source Tuning]
    end

    subgraph Goals["Performance Goals"]
        Latency[Low Latency]
        Throughput[High Throughput]
        Cost[Cost Efficiency]
        Stability[Stability]
    end

    Tuning --> Goals
```

Overview

- **Trigger choice** trades latency for cost: `processingTime` for near real time, `availableNow` to clear a backlog and stop
- **State**: watermarks bound state; RocksDB handles large state; deleting a checkpoint forces reprocessing
- **Rate limits**: `maxOffsetsPerTrigger` (Kafka) and `cloudFiles.maxBytesPerTrigger` (Auto Loader); notification mode for large directories
- **Streaming file sizes**: `optimizeWrite` and `autoCompact` prevent tiny micro-batch files

What the Exam Tests

- Apply Structured Streaming stateful-processing semantics, including watermarks, output modes, foreachBatch, and checkpoints for fault-tolerant exactly-once state recovery.
- Choose a trigger for a latency and cost target
- Detect back-pressure and pick the fix (rate limits or more compute)

Trigger Interval Optimization

Trigger Types

Trigger	Use Case	Latency	Throughput
`processingTime("10 seconds")`	Near real-time	Low	Medium
`processingTime("1 minute")`	Balanced	Medium	High
`availableNow()`	Catch-up processing	N/A	Maximum
`once()`	One-time batch	N/A	Maximum

Choosing the Right Trigger

```
flowchart TD
    Start[Select Trigger] --> Q1{Real-time needed?}
    Q1 --> |< 1 min latency| PT10[processingTime 10s]
    Q1 --> |1-5 min OK| PT60[processingTime 1 min]
    Q1 --> |No| Q2{Catch-up scenario?}
    Q2 --> |Yes| AN[availableNow]
    Q2 --> |One-time| Once[once]
```

Trigger Configuration

```
# Near real-time (high resource usage)

query = (df.writeStream
    .trigger(processingTime="10 seconds")
    .format("delta")
    .option("checkpointLocation", checkpoint_path)
    .start(output_path))

# Balanced (recommended for most use cases)

query = (df.writeStream
    .trigger(processingTime="1 minute")
    .format("delta")
    .option("checkpointLocation", checkpoint_path)
    .start(output_path))

# Catch-up mode (process all available, then stop)

query = (df.writeStream
    .trigger(availableNow=True)
    .format("delta")
    .option("checkpointLocation", checkpoint_path)
    .start(output_path))
```

Trigger Interval Trade-offs

Short Intervals (< 30s)	Long Intervals (> 5 min)
Lower latency	Higher latency
More overhead	Less overhead
More small files	Larger batches
Higher cost	Lower cost

Short Intervals (< 30s)	Long Intervals (> 5 min)
Better for alerts	Better for analytics

State Management

Watermarks for State Cleanup Watermarks tell Spark when to discard old state, preventing unbounded memory growth.

```
# Define watermark for late data handling

df_with_watermark = (df
    .withWatermark("event_time", "10 minutes")
    .groupBy(
        window("event_time", "5 minutes"),
        "device_id"
    )
    .agg(avg("temperature").alias("avg_temp")))
```

Watermark Strategy

Late Data Tolerance	Watermark Setting	State Size
None	”0 seconds”	Minimal
Low (dashboards)	”5 minutes”	Small
Medium (reports)	”1 hour”	Medium
High (compliance)	”24 hours”	Large

```
flowchart LR
    subgraph Timeline
        W[Watermark<br>10:00] --> |Late data accepted| Accept[Events > 10:00]
        W --> |Dropped| Drop[Events < 10:00]
    end
```

State Store Configuration

```
# Configure RocksDB state store for large state

spark.conf.set(
    "spark.sql.streaming.stateStore.providerClass",
    "com.databricks.sql.streaming.state.RocksDBStateStoreProvider"
)

# Tune state store memory

spark.conf.set(
    "spark.sql.streaming.stateStore.rocksdb.blockCacheSizeMB",
    "256"
)
```

Stateful Operation Best Practices

Operation	Memory Impact	Recommendation
`groupBy with window`	High	Always use watermark
`dropDuplicates`	High	Set watermark, limit columns
Stream-stream join	Very High	Use watermarks on both sides
`flatMapGroupsWithState`	Variable	Implement state timeout

Checkpoint Optimization

Checkpoint Location

```
# Always specify checkpoint location

query = (df.writeStream
    .option("checkpointLocation", "/mnt/checkpoints/my_stream")
    .format("delta")
    .start(output_path))
```

Checkpoint Best Practices

Practice	Reason
Use cloud storage (S3/ADLS/GCS)	Durability and recovery
Unique path per query	Avoid conflicts
Don't delete manually	Breaks exactly-once
Monitor checkpoint size	Detect state growth

Recovery and Restart

```
# Stream automatically recovers from checkpoint
# If query logic changes, you may need to:

# Option 1: Start fresh (loses exactly-once for transition)

dbutils.fs.rm(checkpoint_path, recurse=True)

# Option 2: Use checkpoint version compatibility
# (only works for compatible changes)
```

File Sizing for Streaming

Target File Sizes

Scenario	Target Size	Configuration
Streaming writes	128 MB	`optimizeWrite.fileSize`
High-frequency triggers	64-128 MB	Smaller for faster commits
Low-frequency triggers	256 MB - 1 GB	Larger for efficiency

Auto Compaction Configuration

```
# Enable auto-optimize for streaming tables

spark.conf.set(
```

```
    "spark.databricks.delta.optimizeWrite.enabled",
    "true"
)
spark.conf.set(
    "spark.databricks.delta.autoCompact.enabled",
    "true"
)

# Or set on table

spark.sql("""
ALTER TABLE streaming_output
SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact' = 'true'
)
""")
```

Streaming File Management

```
flowchart TB
    subgraph Writes["Streaming Writes"]
        W1[Micro-batch 1<br>10 MB]
        W2[Micro-batch 2<br>15 MB]
        W3[Micro-batch 3<br>12 MB]
    end

    subgraph OptWrite["Optimized Writes"]
        OW[Combined<br>128 MB target]
    end

    subgraph AutoCompact["Auto Compaction"]
        AC[Background<br>Compaction]
    end

    Writes --> |optimizeWrite| OptWrite
    OptWrite --> |autoCompact| AC
```

Source-Specific Tuning

Kafka Source

```
# Control batch size from Kafka

df = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", servers)
    .option("subscribe", topic)
    # Limit records per trigger
    .option("maxOffsetsPerTrigger", 100000)
    # Start from specific offset
    .option("startingOffsets", "earliest")
    # Fetch size tuning
    .option("kafka.fetch.max.bytes", 52428800)  # 50 MB
    .load())
```

Kafka Tuning Parameters

Parameter	Default	Description
`maxOffsetsPerTrigger`	None	Max records per micro-batch
`minPartitions`	None	Min Kafka partitions to read
`kafka.fetch.max.bytes`	1 MB	Max data per fetch request
`kafka.max.partition.fetch.bytes`	1 MB	Max per partition

Auto Loader Tuning

```
# Optimized Auto Loader configuration

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", schema_path)
    # Performance options
    .option("cloudFiles.maxFilesPerTrigger", 1000)
    .option("cloudFiles.maxBytesPerTrigger", "10g")
    # Use notifications for faster discovery
    .option("cloudFiles.useNotifications", "true")
    .load(input_path))
```

Auto Loader Parameters

Parameter	Description	When to Use
`maxFilesPerTrigger`	Files per micro-batch	Limit processing rate
`maxBytesPerTrigger`	Bytes per micro-batch	Control memory usage
`useNotifications`	Event-based discovery	Large directories (>10K files)
`cloudFiles.backfillInterval`	Periodic full listing	Ensure no missed files

Backpressure and Rate Limiting

Controlling Processing Rate

```
# Kafka: Limit offsets per trigger

.option("maxOffsetsPerTrigger", 50000)

# Auto Loader: Limit files and bytes

.option("cloudFiles.maxFilesPerTrigger", 100)
.option("cloudFiles.maxBytesPerTrigger", "1g")

# Rate source (testing): Control rate

.option("rowsPerSecond", 1000)
```

Detecting Backpressure

```
# Check streaming query progress

for q in spark.streams.active:
```

```
    print(f"Query: {q.name}")
    print(f"  Input rows: {q.lastProgress['numInputRows']}")
    print(f"  Processing rate: {q.lastProgress['processedRowsPerSecond']}")
    print(f"  Batch duration: {q.lastProgress['batchDuration']}")
```

Backpressure Indicators

Metric	Healthy	Backpressure
Batch duration	< Trigger interval	> Trigger interval
Input rate	Stable	Growing backlog
Processing rate	>= Input rate	< Input rate

Monitoring Streaming Performance

Key Metrics to Watch

```
# Access streaming query metrics

query = df.writeStream.start()

# Get latest progress

progress = query.lastProgress

# Key metrics

print(f"Input rows: {progress['numInputRows']}")
print(f"Input rate: {progress['inputRowsPerSecond']}")
print(f"Process rate: {progress['processedRowsPerSecond']}")
print(f"Batch duration: {progress['batchDuration']} ms")
```

Streaming Metrics Dashboard

Metric	Description	Alert Threshold
`numInputRows`	Rows in batch	Unusual spikes
`inputRowsPerSecond`	Ingestion rate	Sudden drops
`processedRowsPerSecond`	Processing rate	< Input rate
`batchDuration`	Processing time	> Trigger interval
`stateOperators.numRowsTotal`	State size	Unbounded growth

Using Spark UI for Streaming

1. **Streaming tab**: Overview of active queries
2. **Query details**: Per-batch metrics
3. **State information**: Memory usage, state size
4. **Input rate graph**: Throughput over time

Use Cases

- **Catch-up Processing Post-Outage**: Utilizing the `availableNow()` trigger in a Structured Streaming query to swiftly power through a backlog of millions of queued Kafka events left after a maintenance window, cleanly spinning down the cluster when complete.

- **Handling Out-of-Order Sensor Data**: Configuring a generous 24-hour watermark on an IoT sensor stateful aggregation stream using RocksDB, allowing delayed telemetry records to be gracefully incorporated into yesterday's daily averages without running out of JVM heap space.

Common Issues & Errors

Issue	Symptom	Solution
Unbounded state	OOM, slow batches	Add watermark
Small files	Many tiny files	Enable auto-compact
High latency	Batches take too long	Reduce trigger interval scope
Missed data	Data not processed	Check `useNotifications`, backfill
Checkpoint corruption	Stream won't start	Start fresh (carefully)

Exam Tips

1. **Trigger selection**: Match trigger interval to latency requirements
2. **Watermarks**: Required for stateful operations to prevent unbounded state
3. **Checkpoints**: Never delete - essential for exactly-once semantics
4. **Auto Loader**: Use `useNotifications` for directories with many files
5. **Kafka tuning**: `maxOffsetsPerTrigger` controls micro-batch size
6. **State stores**: RocksDB for large state, default for small state
7. **File sizing**: 128 MB target for streaming, enable auto-compact

Key Numbers to Remember

Metric	Recommended Value
Streaming file size	128 MB
Typical watermark	5-30 minutes
Max state store size	Monitor, no fixed limit
Kafka `maxOffsetsPerTrigger`	10K-1M depending on message size
Auto Loader `maxFilesPerTrigger`	100-10K depending on file size

Key Takeaways

- **Trigger selection governs latency and cost**: `processingTime("10 seconds")` gives near real-time latency at higher cost; `availableNow()` processes all backlog then stops, ideal for catch-up after outages.
- **Watermarks prevent unbounded state**: Any stateful operation (`groupBy` with window, `dropDuplicates`, stream-stream join) must use `withWatermark()` to bound how long late data is accepted and state is retained.
- **RocksDB for large state**: Switch to `RocksDBStateStoreProvider` for stateful streaming queries with large state (millions of keys) to avoid JVM heap pressure from the default in-memory state store.
- **Never delete checkpoints**: Deleting a checkpoint breaks exactly-once semantics and forces reprocessing from the beginning — only delete deliberately when changing incompatible query logic.
- **Auto Loader notification mode**: Set `cloudFiles.useNotifications = true` for directories with many files (> 10,000) to use cloud event notifications instead of periodic full directory listings for file discovery.
- **maxOffsetsPerTrigger for Kafka**: Control micro-batch size and prevent consumer lag from overloading executors by setting `maxOffsetsPerTrigger`; similarly use `cloudFiles.maxBytesPerTrigger` for Auto Loader.

- **128 MB streaming file target**: Enable `delta.autoOptimize.optimizeWrite = true` and `delta.autoOptimize.autoCompact = true` on streaming output Delta tables to prevent accumulation of tiny micro-batch files.
- **Backpressure detection**: If `batchDuration` consistently exceeds the trigger interval or `processedRowsPerSecond` falls below `inputRowsPerSecond`, the stream has a backpressure problem requiring source rate limiting or cluster scaling.

Related Topics

- Structured Streaming — Part 1 - Triggers, output modes, and watermarks
- Streaming Monitoring & Optimization - Back-pressure, progress metrics, and the state store deep dive
- Auto Loader - File notification mode and rate limits
- File Sizing and Compaction - `optimizeWrite`, `autoCompact`, and `OPTIMIZE`

Official Documentation

- Configure Structured Streaming trigger intervals
- Apply watermarks to control data processing thresholds
- RocksDB State Store
- Production considerations for Structured Streaming
- Connect to Apache Kafka
- Auto Loader options

Cost Optimization

Understanding cost drivers and optimization strategies is essential for running efficient Databricks workloads. This guide covers compute selection, instance types, autoscaling, and cost-saving best practices.

Overview

```
flowchart TB
    subgraph Costs["Cost Components"]
        Compute[Compute - DBUs]
        Cloud[Cloud Infrastructure]
        Storage[Storage]
        Network[Data Transfer]
    end

    subgraph Strategies["Optimization Strategies"]
        RightSize[Right-sizing]
        Spot[Spot Instances]
        AutoScale[Autoscaling]
        JobCluster[Job Clusters]
        Serverless[Serverless]
    end

    Costs --> Strategies
    Strategies --> Savings[Cost Savings]
```

Overview

- **Job clusters** have a lower DBU rate than all-purpose clusters and terminate after each run
- **Spot and pools**: spot workers with an on-demand driver (`SPOT_WITH_FALLBACK`); instance pools cut startup time
- **Serverless** has a higher per-DBU rate but bills only for execution, with no cluster management
- **Cost attribution**: `system.billing.usage` joined with `system.billing.list_prices`, grouped by `custom_tags`

What the Exam Tests

- Use Databricks system tables (billing, compute, access, lakeflow) for cost analysis, auditing, and workload monitoring.
- Choose appropriate compute and configuration for environments and dependencies — including serverless compute (serverless environments, dependency management, performance mode), high-memory notebook tasks, and auto-optimization settings (e.g., disallowing retries).
- Pick the cheapest compute option that still meets a workload's needs

Cost Components

Understanding DBUs

```
DBU (Databricks Unit):
- Unit of processing capability
- Charged per hour
- Different rates for:
  - All-Purpose Compute
  - Jobs Compute
  - SQL Compute
  - Lakeflow Pipelines

Pricing tiers (example):
- All-Purpose: ~0.40 DBU/hour
- Jobs: ~0.15 DBU/hour (62% cheaper)
- SQL Serverless: ~0.70 DBU/hour
```

Cloud Infrastructure Costs

Component	Cost Driver
VM instances	Instance type, duration
Storage	Amount stored, tier
Data transfer	Egress between regions
Managed services	Additional cloud services

Cluster Types Comparison

All-Purpose vs Job Clusters

Aspect	All-Purpose	Job Cluster
Use case	Development, exploration	Production jobs
Lifecycle	Always available	Created per job
DBU rate	Higher	Lower (60% less)
Startup time	Instant (if running)	Cold start
Cost model	Pay for uptime	Pay for execution

Cost Comparison Example

```
Scenario: Daily ETL job running 2 hours

All-Purpose Cluster (running 24/7):
- 24 hours × 10 DBUs × $0.40 = $96/day
- Monthly: ~$2,880

Job Cluster (2 hours/day):
- 2 hours × 10 DBUs × $0.15 = $3/day
- Monthly: ~$90

Savings: 97%
```

When to Use Each

```
flowchart TD
    Start[Workload Type] --> Dev{Development?}
    Dev -->|Yes| Interactive[All-Purpose Cluster]
    Dev -->|No| Prod{Production Job?}
    Prod -->|Yes| Freq{Frequency?}
    Freq -->|Continuous| AllPurpose[All-Purpose with Pool]
    Freq -->|Scheduled| JobCluster[Job Cluster]
    Prod -->|No| AdHoc{Ad-hoc Query?}
    AdHoc -->|Yes| SQL[SQL Warehouse]
    AdHoc -->|No| Serverless[Serverless Compute]
```

Spot Instances

Understanding Spot Pricing

```
Spot Instances:
- Spare cloud capacity at discount
- Up to 90% cheaper than on-demand
- Can be interrupted with 2-min warning
- Best for fault-tolerant workloads
```

Spot Configuration

```yaml
# databricks.yml job cluster configuration

job_clusters:
  - job_cluster_key: etl_cluster
    new_cluster:
      spark_version: "14.3.x-scala2.12"
      node_type_id: "Standard_DS3_v2"
      num_workers: 4
```

```yaml
    # Spot configuration
    aws_attributes:
      availability: SPOT_WITH_FALLBACK
      zone_id: auto
      spot_bid_price_percent: 100

    # Azure spot configuration
    azure_attributes:
      availability: SPOT_WITH_FALLBACK_AZURE
      spot_bid_max_price: -1  # Pay up to on-demand price

    # GCP spot configuration
    gcp_attributes:
      availability: PREEMPTIBLE_WITH_FALLBACK_GCP
```

Spot Instance Strategies

Strategy	Driver	Workers	Use Case
Full Spot	Spot	Spot	Development, non-critical
Spot Workers	On-Demand	Spot	Production with reliability
Fallback	On-Demand	Spot with fallback	Critical jobs

Spot Best Practices

```
1. Use SPOT_WITH_FALLBACK for workers
2. Keep driver on-demand for stability
3. Enable checkpointing for streaming
4. Choose diverse instance types
5. Set reasonable spot bid price
```

Autoscaling

Autoscaling Configuration

```yaml
# Autoscaling cluster configuration

new_cluster:
  autoscale:
    min_workers: 2
    max_workers: 10
  spark_version: "14.3.x-scala2.12"
  node_type_id: "Standard_DS3_v2"
```

Autoscaling Behavior

```
flowchart LR
    subgraph Load["Workload"]
        Low[Low Load]
        Medium[Medium Load]
        High[High Load]
    end

    subgraph Cluster["Cluster Size"]
        Min[Min Workers]
```

```
        Mid[Mid Workers]
        Max[Max Workers]
    end

    Low --> Min
    Medium --> Mid
    High --> Max
```

Autoscaling Best Practices

```
1. Set appropriate min/max based on workload
2. Use min_workers > 0 to avoid cold starts
3. Consider autoscale vs fixed for predictable workloads
4. Monitor scaling events for tuning
```

Enhanced Autoscaling

```python
# Optimized autoscaling settings

spark.conf.set("spark.databricks.autoscale.enabled", "true")
spark.conf.set("spark.databricks.autoscale.downscaling.rate", "1.0")  # Normal
 ↪  downscale
spark.conf.set("spark.databricks.autoscale.minWorkers", "2")
spark.conf.set("spark.databricks.autoscale.maxWorkers", "10")
```

Instance Pools

What Are Instance Pools

```
Instance Pools:
- Pre-provisioned VM instances
- Reduce cluster startup time
- Share resources across clusters
- Keep instances warm during idle periods
```

Pool Configuration

```yaml
# Instance pool configuration

instance_pools:
  etl_pool:
    instance_pool_name: "ETL Pool"
    node_type_id: "Standard_DS3_v2"
    min_idle_instances: 2
    max_capacity: 20
    idle_instance_autotermination_minutes: 30
    preloaded_spark_versions:
      - "14.3.x-scala2.12"
```

Using Pools in Jobs

```yaml
# Reference pool in job configuration

job_clusters:
  - job_cluster_key: pooled_cluster
    new_cluster:
```

```
      spark_version: "14.3.x-scala2.12"
      instance_pool_id: ${resources.instance_pools.etl_pool.instance_pool_id}
      num_workers: 4
```

Pool Sizing Strategy

```
Pool sizing:
- min_idle_instances: Covers typical concurrent jobs
- max_capacity: Handles peak load
- idle_termination: Balance cost vs startup time

Example:
- 5 jobs typically run concurrently
- Peak: 10 concurrent jobs
- min_idle = 5, max = 20
```

Serverless Compute

Serverless Options

Service	Use Case	Billing
Serverless SQL	Analytics queries	Per query DBU
Serverless Compute	Notebooks, jobs	Per execution DBU
Serverless Lakeflow Pipelines	Pipeline execution	Per pipeline DBU

Serverless Benefits

```
Advantages:
- Zero cluster management
- Instant startup
- Auto-scaling built-in
- Pay only for execution

Considerations:
- Higher per-DBU rate
- Less configuration control
- Best for variable workloads
```

Serverless Configuration

```
# Serverless job configuration

tasks:
  - task_key: serverless_task
    notebook_task:
      notebook_path: /path/to/notebook
    # No cluster configuration = serverless
```

SQL Warehouse Optimization

Warehouse Sizing

Size	Use Case	Cost
2X-Small	Development, light queries	Lowest
Small	Standard reporting	Low
Medium	Complex analytics	Medium
Large	Heavy concurrent workloads	Higher

Warehouse Configuration

```
Cost optimization settings:
1. Auto-stop: 10-30 minutes idle
2. Scaling: Min 1, Max based on concurrency
3. Spot: Enable for cost savings
4. Channel: Current (stable) vs Preview (features)
```

Serverless vs Classic SQL

Aspect	Serverless	Classic
Startup	Instant	Minutes
Scaling	Automatic	Configured
Cost model	Per query	Per hour
Best for	Variable load	Steady load

Cost Monitoring

System Tables for Cost Analysis

```sql
-- Query billing usage
SELECT
    workspace_id,
    sku_name,
    usage_date,
    usage_unit,
    SUM(usage_quantity) AS total_usage
FROM system.billing.usage
WHERE usage_date >= date_sub(current_date(), 30)
GROUP BY workspace_id, sku_name, usage_date, usage_unit
ORDER BY usage_date DESC, total_usage DESC;
```

Cost by Cluster

```sql
-- Cost breakdown by cluster
SELECT
    cluster_id,
    cluster_name,
    SUM(usage_quantity) AS total_dbus,
    COUNT(DISTINCT usage_date) AS active_days
FROM system.billing.usage
WHERE usage_date >= date_sub(current_date(), 30)
    AND sku_name LIKE '%COMPUTE%'
GROUP BY cluster_id, cluster_name
ORDER BY total_dbus DESC
LIMIT 20;
```

Cost by Job

```sql
-- Cost by job
SELECT
    j.job_id,
    j.job_name,
    COUNT(r.run_id) AS run_count,
    SUM(r.run_duration / 1000 / 3600) AS total_hours,
    AVG(r.run_duration / 1000 / 60) AS avg_minutes
FROM system.lakeflow.jobs j
JOIN system.lakeflow.job_run_timeline r ON j.job_id = r.job_id
WHERE r.start_time >= date_sub(current_date(), 30)
GROUP BY j.job_id, j.job_name
ORDER BY total_hours DESC
LIMIT 20;
```

Optimization Strategies

Right-Sizing Clusters

```
Right-sizing process:
1. Monitor actual resource usage
2. Check Spark UI for executor utilization
3. Review task duration and data shuffle
4. Adjust worker count and instance type
5. Test and validate changes
```

Memory-Optimized vs Compute-Optimized

Workload	Instance Type	Example
Shuffle-heavy	Memory-optimized	Large joins, aggregations
CPU-heavy	Compute-optimized	ML training, compression
Balanced	General purpose	Standard ETL
I/O-heavy	Storage-optimized	Large data scans

Job Scheduling Optimization

```yaml
# Schedule jobs during off-peak

schedule:
  quartz_cron_expression: "0 0 2 * * ?"  # 2 AM
  timezone_id: "America/New_York"

# Batch multiple small jobs

tasks:
  - task_key: combined_etl
    notebook_task:
      notebook_path: /combined_etl
    # Single cluster for multiple tables
```

Cost-Saving Patterns

Pattern 1: Development Environment

```yaml
# Cost-effective dev setup

targets:
  dev:
    resources:
      clusters:
        dev_cluster:
          num_workers: 1
          node_type_id: "Standard_DS3_v2"
          autotermination_minutes: 30
          # Single node for development
```

Pattern 2: Production ETL

```yaml
# Cost-effective production

targets:
  prod:
    resources:
      jobs:
        daily_etl:
          job_clusters:
            - job_cluster_key: etl
              new_cluster:
                num_workers: 4
                node_type_id: "Standard_DS3_v2"
                # Spot for workers
                aws_attributes:
                  availability: SPOT_WITH_FALLBACK
                  first_on_demand: 1  # Driver on-demand
```

Pattern 3: Variable Workload

```yaml
# Autoscaling for variable load

new_cluster:
  autoscale:
    min_workers: 1
    max_workers: 10
  # Start small, scale as needed
```

Pattern 4: Shared Resources

```yaml
# Use instance pools for multiple jobs

instance_pools:
  shared_pool:
    min_idle_instances: 2
    max_capacity: 20

# Multiple jobs share the pool

jobs:
  job1:
    job_clusters:
      - new_cluster:
```

```
        instance_pool_id: ${pool_id}
  job2:
    job_clusters:
      - new_cluster:
          instance_pool_id: ${pool_id}
```

Use Cases

- **Separating Development from Production**: Providing data engineers with autoterminating All-Purpose clusters for interactive notebook development while running their scheduled daily ETL pipelines on cheaper automated Job clusters, yielding massive DBU savings.
- **Spot Instance Fallback for Batch Workloads**: Configuring a large data reprocessing job to execute on a cluster using cloud Spot instances for its workers with a fallback to On-Demand, significantly reducing costs without sacrificing reliability if Spot capacity drops.

Common Issues & Errors

Clusters Running Idle **Scenario:** All-purpose clusters running 24/7.

Fix: Configure auto-termination:

```
# Auto-terminate after 30 minutes idle

new_cluster:
  autotermination_minutes: 30
```

Over-Provisioned Clusters **Scenario:** Large clusters for small workloads.

Fix: Right-size based on actual usage:

```
1. Check Spark UI executor utilization
2. If < 50% utilized, reduce workers
3. Consider autoscaling
```

Spot Instance Interruptions **Scenario:** Jobs fail due to spot interruption.

Fix: Use fallback configuration:

```
aws_attributes:
  availability: SPOT_WITH_FALLBACK
  spot_bid_price_percent: 100
```

High Development Costs **Scenario:** Expensive clusters for development.

Fix: Use smaller instances and auto-termination:

```
dev_cluster:
  num_workers: 0  # Single node
  node_type_id: "Standard_DS3_v2"  # Smaller instance
  autotermination_minutes: 15
```

Exam Tips

1. **Job vs All-Purpose** - Job clusters ~60% cheaper
2. **Spot instances** - Up to 90% savings, use with fallback
3. **Autoscaling** - Set min/max based on workload patterns
4. **Instance pools** - Reduce startup time, share resources
5. **Serverless** - Higher per-DBU but pay only for execution

6. **SQL warehouses** - Auto-stop, right-size for concurrency
7. **System tables** - system.billing.usage for cost analysis
8. **Right-sizing** - Monitor utilization, adjust accordingly
9. **Spot configuration** - Driver on-demand, workers spot
10. **Schedule optimization** - Run during off-peak hours

Key Takeaways

- **Job clusters are ~60% cheaper**: Automated Job clusters charge a lower DBU rate than All-Purpose clusters and terminate automatically after each run — always use them for scheduled production workloads.
- **Spot instances save up to 90%**: Use `SPOT_WITH_FALLBACK` for worker nodes and keep the driver node On-Demand to balance cost savings with reliability against spot interruptions.
- **system.billing.usage for cost analysis**: Query `system.billing.usage` joined with `system.billing.list_prices` to calculate actual estimated costs per workspace, SKU, or custom tag.
- **Auto-termination for dev clusters**: Set `autotermination_minutes: 30` on All-Purpose clusters used for development to prevent idle clusters from accumulating costs overnight.
- **Instance pools reduce startup**: Pre-provisioned instance pools eliminate the cold-start delay for job clusters while keeping instances warm, reducing startup time from minutes to seconds.
- **Serverless trade-off**: Serverless compute has a higher per-DBU rate but charges only for actual execution time with zero cluster management — cost-effective for highly variable or infrequent workloads.
- **Right-sizing process**: Check Spark UI executor utilization; if executors are consistently below 50% utilized, reduce worker count or switch to a smaller instance type.
- **Custom tags for chargebacks**: Apply `custom_tags` to clusters and workspaces so `system.billing.usage` can be queried by team or project for automated departmental chargeback reports.

Related Topics

- Databricks Compute - Cluster types
- System Tables - Usage monitoring
- Declarative Automation Bundles - Job configuration

Official Documentation

- Cluster Pricing
- Instance Pools
- Spot Instances
- Serverless Compute
- Cost Management

Section 6: Ensuring Data Security and Compliance (8 % of Exam)

Protecting data: least-privilege ACLs on Unity Catalog securables and workspace objects, ABAC policies driven by governed tags, anonymization and pseudonymization through column masks, PII detection and masking in batch and streaming pipelines, and data purging for retention and GDPR erasure.

Topics Overview

```
flowchart LR
    D[Security & Compliance]
    D --> T1["Access Control"]
    D --> T2["ABAC Policies and Governed Tags"]
    D --> T3["Anonymization, PII Controls, and Data Purging"]
    D --> T4["Data Classification, Compliance & Permissions"]
    D --> T5["Secret Management"]
    D --> T6["Audit Logging, Data Lineage & Network Security"]
```

Section Contents

File	Topic
01-access-control.md	UC grants and workspace-object ACLs, row filters and column m
02-abac-governed-tags.md	Governed tags and ABAC row-filter / column-mask policies at sc
03-anonymization-pii-and-data-purging.md	Hashing, tokenization, suppression, generalization; PII detectio
04-classification-compliance-permissions.md	Data classification, compliance patterns, advanced permission
05-secret-management.md	Secret scopes and secret access in code and jobs
06-audit-lineage-network-security.md	Audit logs, lineage, network security

Exam Skill Coverage

Every skill bullet in Section 6 of the Oct 9, 2026 exam guide, with the files that cover it.

Exam guide skill
Apply least-privilege access control lists (ACLs) to secure Unity Catalog securable objects and workspace reso
Apply attribute-based access control (ABAC) policies with governed tags to enforce row filters and column mas
Apply anonymization and pseudonymization methods — such as hashing, tokenization, suppression, and gene
Implement a compliant data pipeline that enforces PII detection and masking controls across batch and strean
Implement a data purging strategy that satisfies data retention and deletion policies (e.g., GDPR right-to-eras

Key Concepts to Master

Concept	Why it matters
Least privilege	Grant the narrowest privilege on the narrowest securable to a grou
ABAC policy	One central rule that applies a row filter or column mask wherever
Pseudonymization vs anonymization	Pseudonymized data can be re-linked with a key or lookup; anonym
Right to erasure	DELETE the rows, purge deletion vectors, then VACUUM so old files

Related Resources

- Unity Catalog cheat sheet
- Unity Catalog quick reference (shared)

Access Control

Access control in Databricks encompasses table-level permissions, row-level security, column-level security, and data masking. Understanding these patterns is essential for implementing secure data platforms.

Overview

```
flowchart TB
    AccessControl[Access Control] --> TableLevel[Table-Level]
    AccessControl --> RowLevel[Row-Level Security]
    AccessControl --> ColumnLevel[Column-Level Security]
    AccessControl --> Masking[Data Masking]

    TableLevel --> Grants[GRANT/REVOKE]
    RowLevel --> DynamicViews[Dynamic Views]
    ColumnLevel --> ColumnMask[Column Masking]
    Masking --> Functions[Masking Functions]
    AccessControl --> ABAC[ABAC Policies on Governed Tags]
    AccessControl --> WS[Workspace Object ACLs]
```

Overview

- **Unity Catalog grants** secure data securables (catalogs, schemas, tables, volumes, functions); grant to groups, at the narrowest level that works
- **Workspace object ACLs** secure resources such as jobs, compute, notebooks, folders, SQL warehouses and pipelines
- **Row filters and column masks** restrict rows and values per table; **ABAC policies** do the same centrally from governed tags

What the Exam Tests

- Apply least-privilege access control lists (ACLs) to secure Unity Catalog securable objects and workspace resources.
- Combine Unity Catalog grants (data) with workspace ACLs on jobs, compute, notebooks, SQL warehouses, and pipelines (resources).
- Choose between dynamic views, table-level row filters / column masks, and ABAC policies.

Table-Level Access Control

Privilege Types

Privilege	Description	Applies To
SELECT	Read data	Tables, Views
MODIFY	Insert, Update, Delete	Tables
CREATE TABLE	Create tables	Schemas
CREATE VIEW	Create views	Schemas

Privilege	Description	Applies To
ALL PRIVILEGES	Full access	Any securable

Granting Privileges

```
-- Grant SELECT on specific table
GRANT SELECT ON TABLE prod.gold.customers TO `analysts`;

-- Grant SELECT on all current AND future tables in the schema (inheritance)
GRANT SELECT ON SCHEMA prod.gold TO `analysts`;

-- Grant multiple privileges
GRANT SELECT, MODIFY ON TABLE prod.silver.orders TO `data-engineers`;

-- Grant to user
GRANT SELECT ON TABLE prod.gold.revenue TO `user@company.com`;

-- Grant to group
GRANT SELECT ON TABLE prod.gold.revenue TO `finance-team`;

-- Grant to service principal
GRANT SELECT ON TABLE prod.gold.revenue TO `sp-etl-pipeline`;
```

Revoking Privileges

```
-- Revoke specific privilege
REVOKE SELECT ON TABLE prod.gold.customers FROM `analysts`;

-- Revoke all privileges
REVOKE ALL PRIVILEGES ON TABLE prod.gold.customers FROM `analysts`;

-- Revoke from schema
REVOKE SELECT ON SCHEMA prod.gold FROM `analysts`;
```

Viewing Privileges

```
-- Show grants on table
SHOW GRANTS ON TABLE prod.gold.customers;

-- Show grants on schema
SHOW GRANTS ON SCHEMA prod.gold;

-- Show grants for one principal (an ON clause is always required)
SHOW GRANTS `analysts` ON SCHEMA prod.gold;
```

Grant Hierarchy

```
flowchart TD
    Catalog[CATALOG: USE CATALOG] --> Schema[SCHEMA: USE SCHEMA]
    Schema --> Table[TABLE: SELECT, MODIFY]
    Schema --> View[VIEW: SELECT]
    Schema --> Function[FUNCTION: EXECUTE]
```

Users need access at each level to reach objects:

```sql
-- Full access chain for analysts
GRANT USE CATALOG ON CATALOG prod TO `analysts`;
GRANT USE SCHEMA ON SCHEMA prod.gold TO `analysts`;
GRANT SELECT ON TABLE prod.gold.customers TO `analysts`;
```

Row-Level Security

Row-level security restricts which rows users can access.

Dynamic Views Pattern

```sql
-- Create view that filters based on user
CREATE OR REPLACE VIEW prod.secure.my_customers AS
SELECT *
FROM prod.gold.customers
WHERE region = (
    SELECT region FROM prod.admin.user_regions
    WHERE username = current_user()
);

-- Grant access to view only
GRANT SELECT ON VIEW prod.secure.my_customers TO `regional-analysts`;
-- Do NOT grant access to underlying table
```

Using current_user()

```sql
-- Filter by user's assigned data
CREATE OR REPLACE VIEW prod.secure.my_sales AS
SELECT *
FROM prod.gold.sales
WHERE sales_rep_email = current_user();

-- Filter by manager hierarchy
CREATE OR REPLACE VIEW prod.secure.team_sales AS
SELECT s.*
FROM prod.gold.sales s
JOIN prod.admin.manager_hierarchy h
    ON s.sales_rep_id = h.employee_id
WHERE h.manager_email = current_user()
   OR s.sales_rep_email = current_user();
```

Using is_account_group_member()

```sql
-- Different access based on group membership
CREATE OR REPLACE VIEW prod.secure.orders AS
SELECT
    order_id,
    customer_id,
    order_date,
    CASE
        WHEN is_account_group_member('executives') THEN amount
        WHEN is_account_group_member('managers') THEN
            CASE WHEN amount > 10000 THEN NULL ELSE amount END
        ELSE NULL
    END AS amount,
    region
```

```
FROM prod.gold.orders
WHERE
    is_account_group_member('executives')
    OR (is_account_group_member('regional-managers') AND region = 'US')
    OR (is_account_group_member('analysts') AND order_date >= current_date() -
     ↪  365);
```

Row-Level Security with Mapping Table

```
-- Create mapping table for user-data access
CREATE TABLE prod.admin.user_data_access (
    username STRING,
    allowed_region STRING,
    allowed_department STRING
);

-- Insert access rules
INSERT INTO prod.admin.user_data_access VALUES
    ('alice@company.com', 'US', 'Sales'),
    ('alice@company.com', 'US', 'Marketing'),
    ('bob@company.com', 'EU', 'Sales');

-- Create secure view
CREATE OR REPLACE VIEW prod.secure.department_data AS
SELECT d.*
FROM prod.gold.department_data d
JOIN prod.admin.user_data_access a
    ON d.region = a.allowed_region
    AND d.department = a.allowed_department
WHERE a.username = current_user();
```

Column-Level Security

Column-level security controls which columns users can see.

Column Masking in Views

```
-- Hide sensitive columns
CREATE OR REPLACE VIEW prod.secure.customers AS
SELECT
    customer_id,
    name,
    -- Mask email for non-privileged users
    CASE
        WHEN is_account_group_member('customer-support') THEN email
        ELSE CONCAT(LEFT(email, 2), '***@***.com')
    END AS email,
    -- Completely hide SSN
    CASE
        WHEN is_account_group_member('compliance') THEN ssn
        ELSE NULL
    END AS ssn,
    city,
    state
FROM prod.gold.customers;
```

Column Masking Functions

```sql
-- Create reusable masking function
CREATE OR REPLACE FUNCTION prod.functions.mask_email(email STRING)
RETURNS STRING
RETURN CASE
    WHEN is_account_group_member('full-access') THEN email
    ELSE REGEXP_REPLACE(email, '(.{2}).*@', '$1***@')
END;

-- Use function in view
CREATE OR REPLACE VIEW prod.secure.contacts AS
SELECT
    contact_id,
    name,
    prod.functions.mask_email(email) AS email,
    phone
FROM prod.gold.contacts;
```

Common Masking Patterns

```sql
-- Email masking: john.doe@company.com -> jo***@***.com
REGEXP_REPLACE(email, '(.{2}).*@.*', '$1***@***.com')

-- Phone masking: 555-123-4567 -> ***-***-4567
CONCAT('***-***-', RIGHT(phone, 4))

-- SSN masking: 123-45-6789 -> ***-**-6789
CONCAT('***-**-', RIGHT(ssn, 4))

-- Credit card: 4111-1111-1111-1111 -> ****-****-****-1111
CONCAT('****-****-****-', RIGHT(card_number, 4))

-- Full name: John Doe -> J*** D**
CONCAT(LEFT(first_name, 1), '*** ', LEFT(last_name, 1), '**')

-- Null sensitive columns
CASE WHEN is_account_group_member('authorized') THEN sensitive_col ELSE NULL END
```

Row and Column Filters (Unity Catalog)

Unity Catalog supports declarative row and column filters on tables.

Row Filters

```sql
-- Add row filter to table
ALTER TABLE prod.gold.orders
SET ROW FILTER prod.functions.region_filter ON (region);

-- Create the filter function
CREATE OR REPLACE FUNCTION prod.functions.region_filter(region STRING)
RETURNS BOOLEAN
RETURN (
    is_account_group_member('global-access')
    OR region IN (
        SELECT allowed_region
```

```
        FROM prod.admin.user_regions
        WHERE username = current_user()
    )
);

-- Remove row filter
ALTER TABLE prod.gold.orders DROP ROW FILTER;
```

Column Masks

```
-- Add column mask to table
ALTER TABLE prod.gold.customers
ALTER COLUMN ssn SET MASK prod.functions.mask_ssn;

-- Create the mask function
CREATE OR REPLACE FUNCTION prod.functions.mask_ssn(ssn STRING)
RETURNS STRING
RETURN CASE
    WHEN is_account_group_member('compliance') THEN ssn
    ELSE CONCAT('***-**-', RIGHT(ssn, 4))
END;

-- Remove column mask
ALTER TABLE prod.gold.customers ALTER COLUMN ssn DROP MASK;
```

Benefits of Declarative Filters

Aspect	Views	Row/Column Filters
Applied at	Query time	Table level
Bypassing	Users query underlying table	Cannot bypass
Maintenance	Multiple views needed	Single definition
Performance	Full optimization and pushdown	Engine favors security over some optimizations

Workspace Object ACLs

Unity Catalog grants protect **data**; workspace access control lists (ACLs) protect **workspace resources** such as jobs, compute, notebooks, folders, SQL warehouses, and pipelines. Workspace admins have CAN MANAGE on all workspace objects, and users automatically get CAN MANAGE on objects they create. Objects in a folder inherit the folder's permissions.

Resource	Permission levels (lowest → highest)	Key distinctions
Jobs	CAN VIEW, CAN MANAGE RUN, IS OWNER, CAN MANAGE	CAN MANAGE RUN ca
Compute (clusters)	CAN VIEW (Beta), CAN ATTACH TO, CAN RESTART, CAN MANAGE	CAN ATTACH TO = att
Notebooks	CAN VIEW, CAN RUN, CAN EDIT, CAN MANAGE	CAN RUN = attach an
Folders	CAN VIEW, CAN EDIT, CAN RUN, CAN MANAGE	Contents inherit folde
SQL warehouses	CAN VIEW, CAN MONITOR, CAN USE, IS OWNER, CAN MANAGE	CAN USE = start and
Lakeflow pipelines	CAN VIEW, CAN RUN, CAN MANAGE, IS OWNER	CAN RUN = start/stop
Secret scopes	READ, WRITE, MANAGE	READ = read and list

Note

The UI says CAN VIEW where the Permissions API uses `CAN_READ` for notebooks and folders. API and bundle level names use underscores, for example `CAN_MANAGE_RUN`.

Job identity: owner vs Run as A job runs with the privileges of its **Run as** identity (by default the creator, who is also IS OWNER). Whoever clicks **Run now**, the job still uses the Run as identity, so CAN MANAGE RUN can let users trigger a job over data they cannot read themselves. Databricks recommends setting **Run as** to a service principal.

Least-privilege example: production pipeline

```
# databricks.yml (Declarative Automation Bundles)
run_as:
  service_principal_name: "6f0c...-sp-etl"     # job identity, not a person

resources:
  jobs:
    nightly_etl:
      name: nightly_etl
      permissions:
        - group_name: data-engineers
          level: CAN_MANAGE_RUN                # can run and repair, cannot edit
        - group_name: analysts
          level: CAN_VIEW                      # can see results only
```

```
-- The service principal gets exactly the data access the job needs
GRANT USE CATALOG ON CATALOG prod TO `sp-etl`;
GRANT USE SCHEMA, SELECT ON SCHEMA prod.bronze TO `sp-etl`;
GRANT USE SCHEMA, SELECT, MODIFY, CREATE TABLE ON SCHEMA prod.silver TO `sp-etl`;
-- Analysts read gold only; data is masked further by ABAC policies
GRANT USE CATALOG ON CATALOG prod TO `analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA prod.gold TO `analysts`;
```

Least-privilege checklist: grant to groups, not users; grant `CREATE TABLE` and `EXECUTE` at the narrowest level (schema / individual function); give jobs a service-principal Run as identity; give operators CAN MANAGE RUN instead of CAN MANAGE; give warehouse consumers CAN USE; reserve `MANAGE` and ownership for a governance group. For tag-driven row filters and column masks across many tables, see ABAC Policies and Governed Tags.

Access Control Patterns

Environment-Based Access

```
-- Development: full access
GRANT ALL PRIVILEGES ON CATALOG dev TO `developers`;

-- Staging: read + limited write
GRANT USE CATALOG ON CATALOG staging TO `developers`;
GRANT SELECT ON SCHEMA staging.bronze TO `developers`;
GRANT SELECT, MODIFY ON SCHEMA staging.silver TO `developers`;

-- Production: read-only for most
GRANT USE CATALOG ON CATALOG prod TO `developers`;
GRANT SELECT ON SCHEMA prod.gold TO `developers`;
```

```
-- Production write access only for ETL
GRANT MODIFY ON SCHEMA prod.silver TO `etl-service-principal`;
```

Role-Based Access Control (RBAC)

```
-- Define roles as groups
-- analysts, data-engineers, data-scientists, admins

-- Analysts: read gold layer
GRANT USE CATALOG ON CATALOG prod TO `analysts`;
GRANT USE SCHEMA ON SCHEMA prod.gold TO `analysts`;
GRANT SELECT ON SCHEMA prod.gold TO `analysts`;

-- Data Engineers: full silver/gold access
GRANT USE CATALOG ON CATALOG prod TO `data-engineers`;
GRANT ALL PRIVILEGES ON SCHEMA prod.bronze TO `data-engineers`;
GRANT ALL PRIVILEGES ON SCHEMA prod.silver TO `data-engineers`;
GRANT SELECT ON SCHEMA prod.gold TO `data-engineers`;

-- Data Scientists: read + create in sandbox
GRANT USE CATALOG ON CATALOG prod TO `data-scientists`;
GRANT SELECT ON SCHEMA prod.gold TO `data-scientists`;
GRANT ALL PRIVILEGES ON SCHEMA prod.sandbox TO `data-scientists`;
```

Least Privilege Pattern

```
-- Step 1: Revoke default access
REVOKE ALL PRIVILEGES ON CATALOG prod FROM `users`;

-- Step 2: Grant minimum required access
GRANT USE CATALOG ON CATALOG prod TO `app-users`;
GRANT USE SCHEMA ON SCHEMA prod.gold TO `app-users`;
GRANT SELECT ON TABLE prod.gold.products TO `app-users`;
-- Only specific tables, not entire schema
```

Data Access Auditing

Audit Logs Unity Catalog automatically logs all access.

```
-- Query audit logs (system table)
SELECT
    event_time,
    user_identity.email AS user_email,
    action_name,
    request_params.full_name_arg AS object_name
FROM system.access.audit
WHERE service_name = 'unityCatalog'
    AND action_name = 'getTable'
    AND event_date >= current_date() - 7
ORDER BY event_time DESC;
```

Common Audit Queries

```
-- Who accessed a specific table?
SELECT DISTINCT
    user_identity.email,
```

```
    COUNT(*) as access_count
FROM system.access.audit
WHERE request_params.full_name_arg = 'prod.gold.customers'
    AND action_name = 'getTable'
    AND event_date >= current_date() - 30
GROUP BY user_identity.email;

-- What tables did a user access?
SELECT DISTINCT
    request_params.full_name_arg AS table_name,
    MIN(event_time) AS first_access,
    MAX(event_time) AS last_access,
    COUNT(*) AS access_count
FROM system.access.audit
WHERE user_identity.email = 'analyst@company.com'
    AND action_name = 'getTable'
    AND event_date >= current_date() - 30
GROUP BY request_params.full_name_arg;

-- Permission changes
SELECT
    event_time,
    user_identity.email AS changed_by,
    action_name,
    request_params
FROM system.access.audit
WHERE service_name = 'unityCatalog'
    AND action_name = 'updatePermissions'
ORDER BY event_time DESC;
```

Service Principal Access

Creating Service Principal Access

```
-- Grant access to service principal
GRANT USE CATALOG ON CATALOG prod TO `sp-etl-pipeline`;
GRANT USE SCHEMA ON SCHEMA prod.bronze TO `sp-etl-pipeline`;
GRANT SELECT, MODIFY ON SCHEMA prod.bronze TO `sp-etl-pipeline`;

-- Service principal for read-only access
GRANT USE CATALOG ON CATALOG prod TO `sp-bi-dashboard`;
GRANT USE SCHEMA ON SCHEMA prod.gold TO `sp-bi-dashboard`;
GRANT SELECT ON SCHEMA prod.gold TO `sp-bi-dashboard`;
```

Service Principal Best Practices

Practice	Description
Separate principals	Different SP for different purposes
Minimal privileges	Only grant what's needed
Regular rotation	Rotate secrets periodically
Audit access	Monitor SP activity in audit logs

Use Cases

- **Multi-Tenant Data Platforms**: Isolating department data using declarative row-level filters (e.g., Sales can only see their designated region's rows) without duplicating data into multiple tables.
- **Regulatory Compliance**: Hiding PII columns like emails or SSNs from standard analysts using column masking, while still allowing compliance officers to view the unmasked data.
- **Delegated Administration**: Using the `MANAGE` privilege to allow team leads to manage access to their own schemas without granting them full ownership rights.

Common Issues & Errors

User Can't See Tables **Scenario:** User has SELECT but can't see tables.

Fix: Grant USE CATALOG and USE SCHEMA:

```
GRANT USE CATALOG ON CATALOG prod TO `user@company.com`;
GRANT USE SCHEMA ON SCHEMA prod.gold TO `user@company.com`;
```

View Security Bypass **Scenario:** User queries underlying table directly.

Fix: Only grant access to view, revoke table access:

```
REVOKE SELECT ON TABLE prod.gold.customers FROM `analysts`;
GRANT SELECT ON VIEW prod.secure.customers TO `analysts`;
```

Function Not Executing **Scenario:** Cannot attach a masking/filter function, or a view's function fails.

Fix: The principal assigning a row filter or column mask needs `EXECUTE` on the function (plus `USE CATALOG` / `USE SCHEMA`):

```
GRANT EXECUTE ON FUNCTION prod.functions.mask_email TO `analysts`;
```

Row Filter Performance **Scenario:** Row filter causes slow queries.

Fix: Use simple, deterministic SQL UDFs with few arguments, prefer `CASE` logic, and keep lookup tables small so they are broadcast.

Exam Tips

1. **Privilege hierarchy** - Need USE CATALOG → USE SCHEMA → table privilege
2. **current_user()** - Returns current user's email for row filtering
3. **is_account_group_member()** - Check group membership in views/functions
4. **Dynamic views** - Best for curated/joined data; for many tables prefer ABAC policies
5. **Declarative filters** - ALTER TABLE with ROW FILTER/MASK
6. **Audit logs** - system.access.audit for access tracking
7. **Grant inheritance** - Schema/catalog grants apply to all current and future objects within
8. **View security** - Grant view access, revoke table access
9. **Service principals** - Use for automated pipelines
10. **Masking functions** - Reusable column masking logic
11. **Workspace ACLs** - Jobs: CAN VIEW / CAN MANAGE RUN / IS OWNER / CAN MANAGE; warehouses: CAN USE for query users; least privilege = smallest level that works

Key Takeaways

- **Privilege access chain**: a user needs `USE CATALOG` → `USE SCHEMA` → table privilege (e.g., `SELECT`) to query a table; granting `SELECT` without `USE CATALOG`/`USE SCHEMA` results in a permission denied error
- **Dynamic views for RLS**: create a view with a `WHERE` clause using `current_user()` or `is_account_group_member()`; grant `SELECT` on the view and revoke direct access to the underlying table
- **current_user() vs is_account_group_member()**: `current_user()` returns the user's email for per-user filtering; `is_account_group_member('group')` checks group membership for role-based filtering inside views and masking functions
- **Declarative row/column filters**: `ALTER TABLE ... SET ROW FILTER <function> ON (<col>)` and `ALTER TABLE ... ALTER COLUMN <col> SET MASK <function>` attach filters directly to the table — cannot be bypassed by querying the table directly (unlike views)
- **Column masking patterns**: use `REGEXP_REPLACE`, `CONCAT`, `LEFT`, `RIGHT`, and `NULL` to reveal only last-4-digits, initials, or nothing depending on group membership
- **MANAGE privilege**: grants the ability to grant/revoke permissions on an object without being the owner; more powerful than data privileges for governance delegation
- **Audit log table**: `system.access.audit` records all UC access events; filter by `service_name = 'unityCatalog'`, `action_name`, and `event_date` for compliance queries
- **Schema-level grants**: `GRANT SELECT ON SCHEMA prod.gold TO analysts` applies to all current tables **and to tables created later** (privilege inheritance)
- **Workspace ACLs**: jobs, compute, notebooks, folders, SQL warehouses, and pipelines have their own permission levels; give operators CAN MANAGE RUN, warehouse users CAN USE, and run jobs as a service principal

Related Topics

- Unity Catalog - Governance foundation
- Data Sharing - Sharing with external parties
- Secret Management - Credential security
- ABAC Policies and Governed Tags - Row filters and column masks at scale
- Anonymization, PII Controls, and Data Purging - Masking techniques

Official Documentation

- Unity Catalog permissions model concepts
- Row filters and column masks
- Manually apply row filters and column masks
- Access control lists
- Manage identities, permissions, and privileges for Lakeflow Jobs
- Set permissions for resources in Declarative Automation Bundles
- SHOW GRANTS
- Audit log reference

ABAC Policies and Governed Tags

Overview

Attribute-based access control (ABAC) in Unity Catalog replaces per-table row filters and column masks with a small number of centrally managed **policies** that match tables and columns by **governed tags**. One policy attached to a catalog or schema protects every current and future table whose tags match, and table owners cannot remove or bypass it.

Overview

- **Governed tags** are account-level tag keys with a tag policy: an allowed-value list and permissions (`CREATE`, `MANAGE`, `ASSIGN`) that control who can define and apply them.
- **ABAC policies** (`CREATE POLICY ... ON CATALOG | SCHEMA | TABLE`) apply a row filter or column mask UDF to every table in scope whose tags match `WHEN` / `MATCH COLUMNS` conditions, for the principals in `TO` minus `EXCEPT`.
- Policies are evaluated by Unity Catalog at query time and enforced by the Databricks Runtime, so new tagged tables are covered automatically; policies "act like future grants".
- ABAC needs serverless, or standard/dedicated compute on Databricks Runtime 16.4+; older runtimes fail closed.

What the Exam Tests

- Apply attribute-based access control (ABAC) policies with governed tags to enforce row filters and column masks at scale.
- Choose ABAC over per-table `ALTER TABLE ... SET MASK` / `SET ROW FILTER` or dynamic views when one central rule must cover hundreds of current and future tables (exam guide sample Question 7).
- Read a `CREATE POLICY` statement and predict which tables, columns, and users it affects.
- Know the permissions involved: `ASSIGN` on the governed tag plus `APPLY TAG` on the object to tag; `MANAGE` (or ownership) on the scope plus `EXECUTE` on the UDF to create a policy.

Why ABAC

With table-level controls, every table needs its own `ALTER TABLE ... ALTER COLUMN ... SET MASK` statement, and each table owner can change or drop it. With hundreds of PII tables this does not scale and drifts as new tables arrive. ABAC moves the rule up the hierarchy and keys it on metadata:

```
flowchart LR
    T[Define governed tag taxonomy<br/>pii = ssn, email, phone] --> A[Tag columns
↪   and tables<br/>manually, by Data Classification,<br/>or by tag propagation]
    A --> P[Create one policy on a catalog or schema<br/>MATCH COLUMNS has_tag pii]
    P --> Q[User queries any table in scope]
    Q --> E{UC evaluates policy:<br/>user in TO and not in EXCEPT?<br/>table and
↪   columns match tags?}
    E -->|yes| M[Runtime applies the UDF<br/>as a secure view over the scan]
    E -->|no| R[Data returned unmasked / unfiltered]
```

Governed Tags

Governed tags are account-level tags with built-in rules. When you create a governed tag you also define its **tag policy**: the allowed values and who can assign it. They can be applied to Unity Catalog objects (catalogs, schemas, tables, columns, volumes, models, and more) and to workspace objects such as dashboards, Genie Agents, apps, and notebooks, but **not** to compute resources such as SQL warehouses or to jobs, which use a separate tagging mechanism.

ABAC policies use **governed tags only**, not ungoverned (free-form) tags.

Governed tag permissions

Permission	What it allows	Scope
`CREATE`	Create new governed tags	Account
`MANAGE`	Edit, delete, and manage permissions on governed tags	Account or individual governed tag
`ASSIGN`	Assign the governed tag to Unity Catalog objects	Account or individual governed tag

- Account admins hold `CREATE`, `MANAGE`, and `ASSIGN` on the account by default; workspace admins hold `CREATE` by default (account admins can revoke this).
- A user with `CREATE` automatically gets `MANAGE` on each governed tag they create.
- `ASSIGN` controls who can use the tag; assigning a governed tag to a Unity Catalog object **also** requires `APPLY TAG` on that object (plus `USE CATALOG` / `USE SCHEMA` on its parents), or ownership of the object.

Warning

Tagging is a security boundary. Anyone who can change tags on a data asset can change which policies apply to it. Restrict `ASSIGN` to data stewards and audit tag changes.

Create, edit, and apply governed tags Governed tags can be created in Catalog Explorer (**Catalog** > **Govern** > **Governed Tags**) or with SQL (`CREATE GOVERNED TAG` requires Databricks Runtime 18.1 or above):

```
-- Key-only governed tag
CREATE GOVERNED TAG isPii;

-- Governed tag with a description and an allowed-value list
CREATE GOVERNED TAG pii
  DESCRIPTION 'Kind of personally identifiable information the asset contains'
  VALUES ('ssn', 'email', 'phone');

-- SET VALUES is declarative: the list REPLACES all existing allowed values
ALTER GOVERNED TAG pii SET VALUES ('ssn', 'email', 'phone', 'dob');

DROP GOVERNED TAG isPii;
```

Apply the tag to objects (requires `ASSIGN` on the governed tag and `APPLY TAG` on the object):

```
-- Databricks Runtime 13.3+: ALTER ... SET TAGS
ALTER TABLE prod.crm.customers ALTER COLUMN ssn SET TAGS ('pii' = 'ssn');
ALTER SCHEMA prod.hr SET TAGS ('sensitivity' = 'high');

-- Databricks Runtime 16.1+: SET TAG / UNSET TAG
SET TAG ON COLUMN prod.crm.customers.email pii = email;
UNSET TAG ON COLUMN prod.crm.customers.email pii;
```

Rules that matter for ABAC

Rule	Detail
Inheritance for policy evaluation	Tags on a catalog or schema are implicitly inherited by the schemas and tal
Governing existing tags	Creating a governed tag with the same key as tags already in use makes al
Deleting a governed tag	Assignments stay on objects but become ungoverned. Any policy that refer
Dropping a tagged column	Fails until the governed tag is removed from the column (blocks pipelines th
System governed tags	Predefined by Databricks and not editable, for example `class.*` (applied by
Limits	Up to 1,000 governed tags per account and 500 allowed values per governe
Propagation delay	After assigning or modifying a tag, it can take a few minutes to take effect.

ABAC Policy Types

Policy type	Effect
Row filter policy	Hides rows where the UDF returns FALSE
Column mask policy	Replaces the column value with the UDF result
GRANT policy	Dynamically grants a privilege on matching securables (models, model services, MCP
DENY policy (Beta)	Denies `MANAGE ACCESS CONTROL`; always wins over any grant

For this exam objective, focus on **row filter and column mask policies** — the ABAC equivalents of table-level row filters and column masks.

CREATE POLICY Syntax

```
CREATE [ OR REPLACE ] POLICY policy_name
ON { METASTORE | CATALOG catalog_name | SCHEMA schema_name | TABLE table_name }
[ COMMENT description ]
{ row_filter_body | column_mask_body | grant_policy_body }

row_filter_body
  ROW FILTER function_name
  TO principal [, ...]
  [ EXCEPT principal [, ...] ]
  FOR TABLES
  [ WHEN condition ]
  [ MATCH COLUMNS condition [ [ AS ] alias ] [, ...] ]
  [ USING COLUMNS ( function_arg [, ...] ) ]

column_mask_body
  COLUMN MASK function_name
  TO principal [, ...]
  [ EXCEPT principal [, ...] ]
  FOR TABLES
  [ WHEN condition ]
  [ MATCH COLUMNS condition [ [ AS ] alias ] [, ...] ]
  ON COLUMN alias
  [ USING COLUMNS ( function_arg [, ...] ) ]
```

Clause	Meaning
`ON CATALOG / SCHEMA / TABLE`	**Scope.** The policy is evaluated for every table (including streaming t
`TO ... EXCEPT ...`	**Principals.** Users, groups, or service principals the policy applies to,
`FOR TABLES`	Securable type the row filter / mask targets.
`WHEN condition`	**Table condition** on the table's tags (direct or inherited from the cata
`MATCH COLUMNS condition AS alias`	**Column condition** on column tags (direct only). Row filter policies a
`ON COLUMN alias`	Column mask only: the matched column that is masked. Its value is b
`USING COLUMNS (...)`	Extra UDF arguments, in order: `MATCH COLUMNS` aliases, constants, or

Condition functions

Function	Where	Behavior
`has_tag('key')`	`WHEN` and `MATCH COLUMNS`	`WHEN`: tag on the table or inhe
`has_tag_value('key', 'value')`	`WHEN` and `MATCH COLUMNS`	Same context rules, also che

Function	Where	Behavior
`get_tag_value('key')`	`USING COLUMNS` only	Passes the table's (or inherite
`get_column_tag_value(alias, 'key')`	`USING COLUMNS` only	Passes a matched column's t
`has_identity_attribute_value(...)` (Beta)	`WHEN` of column mask policies	Evaluates the querying user's

> **Note**
>
> The snake_case names (`has_tag`, `has_tag_value`) are current. The older camelCase forms `hasTag` and `hasTagValue` still work for backward compatibility but are not recommended; you may still see them in older exam material.

Worked Examples

Column mask on every `pii` column in a catalog (sample Question 7)

```
-- 1. Masking UDF: SQL UDFs are recommended (the optimizer can inline them)
CREATE FUNCTION prod.governance.mask_pii(val STRING)
  RETURNS STRING
  DETERMINISTIC
  RETURN '***';

-- 2. One policy for all current AND future tables in the catalog
CREATE POLICY mask_pii_columns
ON CATALOG prod
COMMENT 'Mask any column tagged pii for everyone except the privacy team'
COLUMN MASK prod.governance.mask_pii
TO `account users`
EXCEPT `privacy_officers`, `sp-etl-pipeline`
FOR TABLES
MATCH COLUMNS has_tag('pii') AS pii_col
ON COLUMN pii_col;
```

No table is altered. A new table created next month with a column tagged `pii` is masked the moment the tag is applied.

Row filter on tables tagged `sensitivity = high`

```
CREATE FUNCTION prod.governance.non_eu_region(geo_region STRING)
  RETURNS BOOLEAN
  RETURN geo_region <> 'eu';

CREATE POLICY hide_eu_customers
ON SCHEMA prod.customers
ROW FILTER prod.governance.non_eu_region
TO `analysts`
FOR TABLES
WHEN has_tag_value('sensitivity', 'high')      -- table condition
MATCH COLUMNS has_tag('geo_region') AS region  -- column condition
USING COLUMNS (region);                        -- column value passed to the UDF
```

One policy and one UDF for all PII types

```
CREATE FUNCTION main.governance.mask_by_type(col STRING, pii_type STRING)
  RETURNS STRING
```

```
  RETURN CASE
    WHEN pii_type = 'email' THEN regexp_replace(col, '(^[^@]+)', '***')
    WHEN pii_type = 'ssn'   THEN '***-**-' || right(col, 4)
    WHEN pii_type = 'phone' THEN '***-***-' || right(col, 4)
    ELSE '***'
  END;

CREATE POLICY mask_all_pii
ON CATALOG main
COLUMN MASK main.governance.mask_by_type
TO `account users`
FOR TABLES
MATCH COLUMNS has_tag('pii') AS col
ON COLUMN col
USING COLUMNS (get_column_tag_value(col, 'pii'));
```

A new PII type only needs a new CASE branch, not a new policy.

UDF requirements

- Row filter UDFs return BOOLEAN; column mask UDFs return a type that matches or can be cast to the column type (Databricks auto-casts input and output; incompatible casts raise a runtime error).
- The policy creator needs EXECUTE on the UDF; querying users need only their normal table grants for the policy to apply to them.
- Prefer SQL UDFs, simple CASE logic, DETERMINISTIC, few arguments, and error-safe expressions (`try_divide`); Python UDFs are supported but cannot be inlined.
- Prefer TO / EXCEPT over `is_account_group_member()` inside the UDF to target principals; use identity functions only for genuinely conditional logic.

Manage and inspect policies

```
SHOW POLICIES ON SCHEMA prod.customers;            -- policies attached directly
 ↪ here
SHOW EFFECTIVE POLICIES ON TABLE prod.crm.orders;  -- plus policies inherited
 ↪ from parents
DESCRIBE POLICY hide_eu_customers ON SCHEMA prod.customers;
DROP POLICY hide_eu_customers ON SCHEMA prod.customers;
```

Action	Required permission
CREATE / DROP a policy	MANAGE on the target securable or ownership (metastore admin for ON METAST
SHOW [EFFECTIVE] POLICIES	READ METADATA or MANAGE on the target, or ownership

Scope, Inheritance, and Future Tables

- A policy on a **catalog** evaluates every table in every schema of that catalog; on a **schema**, every table in that schema; on a **table**, only that table. Databricks recommends the highest applicable level, usually the catalog; table-level policies should be the exception.
- Policies are dynamic: when a new table is created in scope and tagged (directly, by inheritance from its schema/catalog, by Data Classification, or by tag propagation), existing policies apply with no extra configuration — the docs describe policies as acting **like future grants**.
- Policies **do not grant access.** Users still need USE CATALOG, USE SCHEMA, and SELECT through normal grants; row filter and column mask policies only restrict what those users see.
- A policy set at the catalog or schema level cannot be modified, removed, or bypassed by individual table owners.

Policy Evaluation at Query Time

```
sequenceDiagram
    participant U as Querying user
    participant UC as Unity Catalog (control plane)
    participant RT as Databricks Runtime
    U->>UC: SELECT * FROM prod.crm.customers
    UC->>UC: Find policies whose scope covers the table
    UC->>UC: Is user in TO and not in EXCEPT?
    UC->>UC: Do WHEN / MATCH COLUMNS tag conditions match?
    UC->>RT: Effective row filter / column masks in table metadata
    RT->>RT: Wrap scan in a secure view and apply UDFs
    RT-->>U: Filtered / masked result
```

Behavior	Detail
Views and functions	Access to underlying tables is checked against the **view/functior
One filter per table, one mask per column	Only one distinct row filter can resolve per user and table, and on
Fail-closed	If Databricks cannot enforce every applicable policy (unsupported
Deleted UDF	Queries in scope fail with `UC_DEPENDENCY_DOES_NOT_EXIST` until t
Time travel	Time travel on ABAC-protected tables is in Beta for qualifying tabl

Separation of Duties

Step	Typical owner	Required permissi
1. Define tag taxonomy	Governance team	Account admin, o
2. Tag data assets	Data stewards, data creators, or Data Classification	`ASSIGN` on the tag
3. Write policies	Governance admins	`MANAGE` (or owners
4. Create tables in governed scopes	Data engineers	`CREATE TABLE` on
5. Query	Consumers	Normal grants (US

Compute Requirements and Limitations

Area	Requirement or limitation
Compute	Serverless; **standard** compute on Databricks Runtime 16.4+; **dedi**
Policy quotas	100 policies per catalog, 100 per schema, 50 per table, 10,000 per
Materialized views and streaming tables	Pipelines refresh them as the pipeline owner / run-as identity. If tha
Cloning	Deep and shallow clones of ABAC-protected tables are blocked unle
OpenSharing	ABAC-protected tables can be shared only if the share owner is exe
AI Search indexes	Source-table policies are not enforced on the index; exclude maske
Views	ABAC on views is Beta (workspace preview, Databricks Runtime 19-

Important

Principals in EXCEPT see unfiltered, unmasked data. Exempt only trusted identities such as the service principals that run ETL, cloning, or sharing workloads.

Cross-engine ABAC and governed tag propagation

Platform update (Oct 2026)

Cross-engine ABAC is generally available (Oct 1, 2026): supported external engines can read Unity Catalog **managed** tables with ABAC policies, row filters, and column masks enforced by Databricks serverless compute. It requires external data access on the metastore, `EXTERNAL USE SCHEMA` for the querying principal, managed tables with catalog commits, and OAuth M2M or PAT authentication. Source: Cross-engine attribute-based access controls (ABAC)

Platform update (Oct 2026)

Governed tag propagation is in Beta (Oct 12, 2026). When enabled per governed tag, Unity Catalog uses lineage to copy table- and column-level governed tags to tables and views derived through `CREATE TABLE AS SELECT`, `CREATE VIEW`, `INSERT INTO ... SELECT`, or `MERGE` on serverless compute, so ABAC policies follow the data downstream. It is asynchronous, does not apply to ungoverned tags, and is not supported for streaming tables or materialized views created by Lakeflow pipelines. Source: Tag propagation

ABAC Policy vs Table-Level Row Filter / Column Mask vs Dynamic View

Consideration	ABAC policy	Ta
Syntax	`CREATE POLICY ... ON CATALOG/SCHEMA/TABLE`	AL
Scope	All tables in the catalog/schema whose tags match, **including future tables**	Or
Matching	Dynamic, via governed tags	Bo
Who controls it	Catalog/schema owner or `MANAGE` holder; table owners cannot override	Ta
Principal targeting	`TO` / `EXCEPT` plus optional identity functions in the UDF	Id
New object needed	No	No
Inspect	`SHOW EFFECTIVE POLICIES`, `DESCRIBE POLICY`	Ta
Performance and probing	Secure-view barrier; some optimizations restricted	Se
Best for	Hundreds of tables, centralized rules, separation of duties	A

Use Cases

- **Enterprise PII masking:** one catalog-level column mask policy masks every column tagged `pii` in hundreds of tables, including tables created later by any team.
- **Regional row-level security:** a schema-level row filter policy restricts each regional team to its own rows for every table that has a column tagged `region`.
- **Quarantine until classified:** apply a default `classification = unverified` tag at the catalog level and a row filter policy that blocks those tables until a steward reviews and retags them.
- **Pipeline exemptions:** list the ETL service principal in EXCEPT so materialized views and streaming tables refresh with unmasked source data, while consumers see masked results.

Common Issues & Errors

Symptom	Likely cause
Policy created but data is not masked	Column tag missing (table tags do not flow t
`UC_ABAC_MULTIPLE_ROW_FILTERS` / `MULTIPLE_MASKS` error	Two different UDFs resolve for the same user
All queries in a catalog fail with `UC_ABAC_UNKNOWN_TAG_POLICY`	A governed tag referenced by a policy was d
Job on Databricks Runtime 15.4 cannot read the table	Runtime below 16.4 (fail-closed)
Materialized view contains masked values for everyone	Pipeline owner / run-as identity was subject t
`ALTER TABLE ... DROP COLUMN` fails	Column carries a governed tag

Symptom	Likely cause
`CREATE POLICY` permission denied	Missing `MANAGE`/ownership on the scope or E

Best Practices

- Design a small, controlled taxonomy (for example one `sensitivity` tag with fixed values) before writing policies.
- Attach policies at the catalog or schema level; avoid policy sprawl and review overlapping policies.
- Restrict `ASSIGN` on sensitive governed tags and audit tag and policy changes in `system.access.audit`.
- Use simple, deterministic SQL UDFs and `TO` / `EXCEPT` rather than complex identity logic.
- Never delete a governed tag that a policy references.

Exam Tips

Tip

- "Single, centrally managed rule ... all current and future tables ... without altering each table" → **ABAC policy with governed tags**, not per-table `SET MASK`, schema row filters, or dynamic views.
- `MATCH COLUMNS has_tag(...)` checks **column** tags only; `WHEN has_tag(...)` checks **table** tags, including tags inherited from the schema or catalog.
- Policies restrict data; they never grant `SELECT`. Users still need `USE CATALOG` + `USE SCHEMA` + `SELECT`.
- `EXCEPT` = fully exempt (sees raw data). Pipeline owners, cloning principals, and share owners must be exempt for those operations.
- Tagging permission = `ASSIGN` on the governed tag **and** `APPLY TAG` on the object. Policy permission = `MANAGE` (or ownership) on the scope **and** `EXECUTE` on the UDF.
- ABAC requires Databricks Runtime 16.4+ or serverless; older compute fails closed.
- Only one distinct row filter per table and one distinct mask per column can resolve for a user; conflicting UDFs return an error.

Key Takeaways

- **Governed tags** are account-level tags with allowed values and `CREATE` / `MANAGE` / `ASSIGN` permissions; ABAC policies only work with governed tags.
- **`CREATE POLICY`** attaches a row filter or column mask UDF to a catalog, schema, or table scope; `TO` / `EXCEPT` pick principals; `WHEN` matches table tags; `MATCH COLUMNS` matches column tags; `ON COLUMN` picks the masked column; `USING COLUMNS` passes extra arguments.
- **Inheritance:** policies cover every current and future table in scope; tags on catalogs and schemas are inherited by tables for policy evaluation, but never by columns.
- **Table owners cannot override** catalog- or schema-level policies, unlike table-level filters and masks that the owner manages.
- **Enforcement** happens at query time as a secure view, evaluated for the querying user, and fails closed on unsupported compute or missing dependencies.
- **Limitations** to remember: Databricks Runtime 16.4+, one distinct filter/mask per user, pipeline identities in `EXCEPT` for MV/ST refreshes, cloning and OpenSharing require exempt principals.
- **Oct 2026 updates:** cross-engine ABAC is GA (Oct 1); governed tag propagation is Beta (Oct 12).

Related Topics

- Access Control — least-privilege grants, table-level row filters and column masks, dynamic views
- Anonymization, PII Controls, and Data Purging — masking UDF techniques, Data Classification, erasure workflow
- Data Classification, Compliance & Permissions — tag-based classification and compliance patterns
- Tags, Comments, and Discoverability — tags and comments as metadata
- Unity Catalog — privilege inheritance model
- OpenSharing — sharing tables protected by policies

Official Documentation

- Attribute-based access control in Unity Catalog
- Core concepts for ABAC
- CREATE POLICY
- SHOW POLICIES
- When to use ABAC vs table-level row filters and column masks
- Requirements, quotas, and limitations for row filter and column mask policies
- Row filter and column mask policy evaluation and runtime behavior
- Best practices for ABAC policies
- Common patterns for row filtering and column masking
- Governed tags
- Create and manage governed tags
- Manage permissions on governed tags
- Apply tags to Unity Catalog securable objects
- Tag propagation
- Cross-engine attribute-based access controls (ABAC)
- October 2026 release notes

Anonymization, PII Controls, and Data Purging

Overview

This topic covers the three privacy controls a professional data engineer builds on Databricks: transforming confidential values (hashing, tokenization, suppression, generalization), detecting and masking PII consistently across batch and streaming pipelines, and physically purging personal data to satisfy retention and right-to-erasure requests with Delta Lake and Unity Catalog.

Overview

- **Pseudonymization** (hashing, tokenization, encryption) keeps records linkable and is reversible with extra information; **anonymization** (suppression, generalization, aggregation) removes the link.
- Techniques can run at **write time** in the pipeline (silver/gold store transformed values) or at **query time** through column masks and ABAC policies (storage keeps raw values).
- **Data Classification** scans Unity Catalog tables, streaming tables, and materialized views, tags sensitive columns with `class.*` system tags, and feeds ABAC masking policies.
- **Erasure** = `DELETE` / `MERGE` → `REORG TABLE ... APPLY (PURGE)` for deletion vectors → `VACUUM` after the retention window, plus downstream propagation, CDF, and upstream sources.

What the Exam Tests

- Apply anonymization and pseudonymization methods — hashing, tokenization, suppression, and generalization — to confidential data, for example with column masks and related Unity Catalog features.
- Implement a compliant data pipeline that enforces PII detection and masking controls across batch and streaming workloads using Unity Catalog features.
- Implement a data purging strategy that satisfies retention and deletion policies (GDPR right-to-erasure / right-to-be-forgotten) using Delta Lake and Unity Catalog features.
- Know why DELETE alone does not erase data: old files, deletion vectors, time travel, and change data feed still hold it.

Anonymization vs Pseudonymization

Aspect	Pseudonymization	Anonymization
Goal	Replace identifiers with substitutes while keeping records joinable	Remove or coarsen
Reversible?	Yes, with additional information (salt, key, token vault)	No (by design)
Typical techniques	Salted hashing, tokenization, reversible encryption	Suppression, genera
Joinability	Same input → same pseudonym across tables (deterministic)	Lost or reduced
Still personal data?	Generally treated as personal data because it can be re-linked	Intended to fall outs

Important

Databricks guidance for GDPR/CCPA is that **complete deletion is preferable to obfuscation**: pseudonymization and masking reduce exposure, but eliminating re-identification risk often requires deleting the PII. Use the techniques below to minimize who sees PII, and the purge workflow to erase it.

Technique Catalog

Technique	Reversible	Databricks building blocks
Hashing	No (but guessable for low-entropy values)	`sha2(expr, bitLength)` with a secret salt from
Tokenization	Yes, via a token vault	Token lookup table in a restricted schema, `uuid(`
Encryption	Yes, with the key	`aes_encrypt` / `aes_decrypt` / `try_aes_decrypt`,
Suppression	No	`CASE ... THEN NULL`, `mask()`, dropping the colu
Generalization	No	`date_trunc`, `year`, `floor` buckets, `left()` trunca

Hashing (deterministic pseudonymization) `sha2(expr, bitLength)` returns a hex string; `bitLength` can be `0` (equivalent to 256), 224, 256, 384, or 512.

```
-- Write-time: salted, normalized hash stored in silver
CREATE OR REPLACE TABLE prod.silver.customers AS
SELECT
  sha2(concat(secret('pii-scope', 'hash-salt'), lower(trim(email))), 256) AS
 ↪  customer_key,
  country,
  signup_date
FROM prod.bronze.customers;
```

An unsalted hash of a low-entropy value (phone number, SSN) can be reversed by hashing every candidate value, so keep the salt in a secret scope and never in code. The ABAC documentation's **consistent hashing** pattern adds a version parameter so the hash can be rotated through the policy's `USING COLUMNS`:

```
CREATE FUNCTION prod.governance.pseudonymize(val STRING, version INT)
  RETURNS STRING
  DETERMINISTIC
  RETURN sha2(concat(val, CAST(version AS STRING)), 256);
```

Note

The `secret()` SQL function requires Databricks Runtime 11.3 LTS or above and is in preview in Databricks SQL; Databricks makes a best-effort attempt to redact secret values from output.

Tokenization with a token vault

```
-- Token vault in a schema only the privacy service principal can read
CREATE TABLE IF NOT EXISTS prod.vault.email_tokens (token STRING, email STRING);

MERGE INTO prod.vault.email_tokens AS v
USING (SELECT DISTINCT email FROM prod.bronze.customers) AS s
ON v.email = s.email
WHEN NOT MATCHED THEN INSERT (token, email) VALUES (uuid(), s.email);

-- Downstream tables store only the token
CREATE OR REPLACE TABLE prod.silver.orders AS
SELECT t.token AS customer_token, o.order_id, o.amount, o.order_date
FROM prod.bronze.orders AS o
JOIN prod.vault.email_tokens AS t ON o.email = t.email;
```

Re-identification is a join back to the vault, controlled by Unity Catalog grants on `prod.vault`. This also supports erasure: the GDPR guidance notes that when non-PII data is keyed by a non-natural key such as a `user_id` instead of a natural key like email, you can delete the PII and leave the non-PII data in place — a vault token plays the same role, so deleting the vault row unlinks every downstream record.

Reversible encryption

```
-- Key: 16, 24, or 32 bytes (AES-128/192/256); GCM is the default mode
SELECT base64(aes_encrypt(email, secret('pii-scope', 'aes-key'))) AS email_enc
FROM prod.bronze.customers;

SELECT CAST(try_aes_decrypt(unbase64(email_enc), secret('pii-scope', 'aes-key'))
 ↪  AS STRING) AS email
FROM prod.silver.customers_enc;
```

Suppression and partial masking

```
SELECT
  customer_id,
  CASE WHEN is_account_group_member('privacy_officers') THEN ssn END AS ssn,  --
   ↪  NULL for others
  concat('***-**-', right(ssn, 4))                                    AS ssn_last4,
```

```
  mask(phone)                                                    AS
 ↪  phone_masked  -- digits become 'n'
FROM prod.silver.customers;
```

`mask(str [, upperChar [, lowerChar [, digitChar [, otherChar]]]])` replaces upper-case letters with X, lower-case with x, and digits with n by default.

Generalization

```
SELECT
  concat(CAST(floor(age / 10) * 10 AS STRING), '-', CAST(floor(age / 10) * 10 + 9
   ↪  AS STRING)) AS age_band,
  date_trunc('MONTH', birth_date)  AS birth_month,
  concat(left(zip_code, 3), '**')  AS zip3,
  country
FROM prod.silver.patients;
```

Applying Techniques with Column Masks

A column mask is a SQL UDF attached to a column; it runs at query time, so storage keeps the raw value and each user sees the version their group allows.

```
CREATE OR REPLACE FUNCTION prod.governance.mask_email(email STRING)
  RETURNS STRING
  RETURN CASE
    WHEN is_account_group_member('privacy_officers') THEN email
    ELSE sha2(lower(email), 256)   -- pseudonymized but still joinable
  END;

-- Table-level: one column on one table
ALTER TABLE prod.silver.customers ALTER COLUMN email SET MASK
 ↪  prod.governance.mask_email;

-- At scale: one ABAC policy for every column tagged by Data Classification
CREATE POLICY mask_detected_emails
ON CATALOG prod
COLUMN MASK prod.governance.mask_email
TO `account users`
EXCEPT `sp-etl-pipeline`
FOR TABLES
MATCH COLUMNS has_tag('class.email_address') AS e
ON COLUMN e;
```

Write-time vs query-time protection

Choose	When
Write-time transformation (hash, tokenize, generalize in the pipeline)	Downstream consumers never need
Query-time column mask / ABAC policy	Some users must see raw values; rul
Both	Raw PII isolated in a restricted bronze

PII Detection with Data Classification

Databricks **Data Classification** uses an agentic AI system (an LLM plus regular-expression detectors) to classify and tag sensitive columns in Unity Catalog.

Aspect	Behavior
What is scanned	Tables, streaming tables, and materialized views in enabled catalogs (all or selected sc
How	Incremental "intelligent scanning" on serverless compute; new tables and columns are
Output	Detections per class (for example `class.email_address`, `class.us_ssn`, `class.credi`
Auto-tagging	When enabled for a class (metastore or catalog level), existing and future detections ar
Enable on a catalog	Own the catalog or have `MANAGE` on it; workspace must have serverless compute
Enable auto-tagging	`USE CATALOG` and `APPLY TAG` on the catalog plus `ASSIGN` on the tag (metastore admin f
Wrong detection	Exclude it in the review panel: removes the tag, prevents re-tagging, and improves futu
Governance	From the results page, **New policy** creates an ABAC column mask policy pre-filled for t

Platform update (Oct 2026)

Data Classification became generally available on Apr 20, 2026. The Data Classification **API and Terraform resource** became GA on Oct 14, 2026 (auto-tagging configuration `auto_tag_configs` remains Public Preview), and on Oct 15, 2026 it added 50 new classes (payment card, personal/employee data, software secrets, regional identifiers). Source: Data Classification release notes

Warning

AI-generated comments are not a PII detector: Databricks states the comment model should not be relied on for data classification tasks such as detecting PII columns. Use Data Classification (or custom classifiers) instead.

Compliant Pipeline: Detection and Masking Across Batch and Streaming

```
flowchart LR
    SRC[Kafka / files / CDC] --> B[bronze schema<br/>raw PII, ETL principal only]
    B -->|stream| S[silver streaming tables<br/>hash / tokenize keys]
    S --> G[gold materialized views]
    DC[Data Classification<br/>auto-tags class.* columns] -.-> B
    DC -.-> S
    DC -.-> G
    P[ABAC column mask policy<br/>ON CATALOG prod<br/>EXCEPT pipeline identity]
↪   -.-> S
    P -.-> G
    G --> C[Dashboards, SQL, jobs<br/>see masked values]
```

1. **Isolate raw PII**: land raw data in a bronze schema granted only to the pipeline's service principal; keep PII in clearly named schemas and columns so grants and audits stay simple.
2. **Detect**: enable Data Classification with auto-tagging on the catalog so new PII columns in tables, streaming tables, and materialized views get `class.*` tags without manual work.
3. **Mask centrally**: create ABAC column mask (and row filter) policies on the catalog keyed on those tags. Policies apply to every table type in scope, so batch tables, streaming tables, and materialized views are protected identically for every reader.
4. **Exempt the pipeline identity**: pipelines refresh materialized views and streaming tables with the pipeline owner's or run-as identity; if that identity is masked, the target permanently stores masked data. Put it in EXCEPT.
5. **Pseudonymize at write** for columns consumers never need in clear text (hash or tokenize in the silver transformation).
6. **Audit**: query `system.access.audit` for tag and policy changes and use the classification results page to see unmasked access.

Databricks recommends applying Unity Catalog column masks and row filters to the tables a

pipeline produces rather than writing custom masking logic in pipeline code, because table-level controls protect every consumer, not just the pipeline.

Pipeline-defined masks and filters For a single streaming table or materialized view you can declare the controls in the Lakeflow pipeline definition; add, change, or drop them through the `CREATE OR REFRESH` statement:

```
CREATE OR REFRESH STREAMING TABLE customers_silver (
  id INT COMMENT 'Customer ID',
  name STRING,
  region STRING,
  ssn STRING MASK prod.governance.ssn_mask_fn COMMENT 'SSN masked for privacy'
)
WITH ROW FILTER prod.governance.us_filter_fn ON (region)
AS SELECT * FROM STREAM(customers_bronze);
```

Behavior	Detail
Refresh	Filter and mask functions run with the **pipeline owner's** rights; `CURRENT_USER` / `IS_`
Query	When users query the streaming table or materialized view, identity functions evalua
Audit	`DESCRIBE EXTENDED`, `INFORMATION_SCHEMA`, or Catalog Explorer show the filters and
Dedicated compute	Tables with row filters or column masks don't support streaming workloads on dedica
Tag propagation (Beta)	Not supported for streaming tables or materialized views created by Lakeflow pipelin

Data Purging and the Right to Erasure

Why DELETE is not enough

Where the data still lives after `DELETE`	Removed by
Current files, when **deletion vectors** are enabled (rows only marked deleted)	`REORG TABLE ... APPLY (PU`
Old data files referenced by earlier table versions (time travel)	`VACUUM` once the files are olde
Change data feed records (include the deleted row's data)	Expire with table history; lega
Downstream silver/gold tables and materialized views	Propagated deletes, MV refres
Upstream sources (Kafka topics, landing files, source databases)	Deletion at the source; Auto L
Disk cache on running clusters	Cluster restart

Erasure workflow

```
flowchart TD
    R[Request logged in<br/>gdpr_requests control table] --> D[Scheduled job:
↪   MERGE ... WHEN MATCHED THEN DELETE<br/>on bronze, then every table with PII]
    D --> MV[Materialized views: refresh<br/>deletes handled automatically]
    D --> ST[Streaming tables: DELETE via DML +<br/>skipChangeCommits on readers,
↪   or full refresh]
    MV --> P[REORG TABLE ... APPLY PURGE<br/>tables with deletion vectors]
    ST --> P
    P --> V[VACUUM after the retention
↪   window<br/>delta.deletedFileRetentionDuration, default 7 days]
    V --> U[Delete upstream copies<br/>Kafka, landing files, source systems]
    U --> A[Mark request complete<br/>evidence from DESCRIBE HISTORY]
```

1. Delete in bronze first, driven by a control table

```
MERGE INTO prod.bronze.users AS target
USING (SELECT user_id FROM prod.compliance.gdpr_requests WHERE status =
 ↪  'pending') AS source
ON target.user_id = source.user_id
WHEN MATCHED THEN DELETE;
```

Databricks recommends MERGE against a control table when deleting many subjects at once, starting in bronze and then propagating to silver and gold.

2. Propagate to downstream tables

- **Materialized views** handle source deletions automatically; refresh them (deleting source data can trigger a full recompute) and run maintenance.
- **Streaming tables** process append-only sources, so a delete in the source breaks the stream. Recommended: delete from the source with DML, delete from the streaming table with DML, then set skipChangeCommits on the streaming read. Alternative: full refresh, which clears the streaming state and reprocesses everything — risky if the upstream (for example a Kafka topic with 7-day retention) no longer has the history.

```
from pyspark import pipelines as dp

@dp.table(name="prod.bronze.users_bronze")
def users_bronze():
    return (spark.readStream
        .option("skipChangeCommits", "true")
        .table("prod.raw.source_users"))
```

Note

Streaming state also matters: Structured Streaming keeps state across micro-batches until watermarks remove it, so an identifier used as a key in a stateful operator persists in the checkpoint's state until it is evicted or the stream is fully refreshed.

3. Purge soft-deleted rows (deletion vectors) With deletion vectors, DELETE, UPDATE, and MERGE only mark rows as removed. For GDPR, run REORG TABLE ... APPLY (PURGE) after deleting — this applies to Delta tables, materialized views, and streaming tables.

```
REORG TABLE prod.bronze.users APPLY (PURGE);
-- Only partition-column predicates are supported in WHERE
REORG TABLE prod.bronze.events WHERE event_date >= '2026-01-01' APPLY (PURGE);
```

APPLY (PURGE) rewrites only files that contain soft-deleted data and is idempotent. The same purge is needed after ALTER TABLE ... DROP COLUMN on tables with column mapping, because dropping a column is also a soft delete. For large tables, spark.databricks.delta.reorg.purgeMode = rows limits the purge to files with soft-deleted rows.

4. VACUUM after the retention window REORG creates a new table version; the old files still serve time travel until VACUUM removes them once they **expire** under the retention threshold, so VACUUM must run with a delay after REORG.

```
-- Retention is controlled by table properties
ALTER TABLE prod.bronze.users SET TBLPROPERTIES (
  'delta.deletedFileRetentionDuration' = 'interval 7 days',   -- default; VACUUM
   ↪  threshold
  'delta.logRetentionDuration'         = 'interval 30 days'   -- default; table
   ↪  history
);
```

```
VACUUM prod.bronze.users DRY RUN;   -- list up to 1000 files that would be deleted
VACUUM prod.bronze.users;           -- FULL mode by default; LITE uses the
 ↪ transaction log
```

Setting	Default	Notes
`delta.deletedFileRetentionDuration`	`interval 7 days`	Threshold VACUUM uses
`delta.logRetentionDuration`	`interval 30 days`	How long table history
`spark.databricks.delta.retentionDurationCheck.enabled`	`true`	Safety check that bloc

Important

In Databricks Runtime 18.0 and above, `VACUUM` ignores the retention-duration argument except when it is `0 HOURS`, time travel beyond `deletedFileRetentionDuration` is blocked for all tables, and `deletedFileRetentionDuration` cannot exceed `logRetentionDuration` (or vice versa). Set retention with the `delta.deletedFileRetentionDuration` table property instead of `RETAIN n HOURS`; for Unity Catalog managed tables the table property governs retention from Databricks Runtime 13.3 LTS. Source: Databricks Runtime 18.0 release notes

Predictive optimization automatically runs `VACUUM` on Unity Catalog managed tables, and Lakeflow pipelines maintain streaming tables and materialized views (within 24 hours of an update when predictive optimization is not enabled). Manual `VACUUM` is still needed elsewhere.

5. Change data feed and time travel

- Each CDF change record includes the row data plus whether it was inserted, updated, or deleted, so the delete itself is visible in the feed until those versions leave the retention window. Legacy CDF stores change files under `_change_data`, which `VACUUM` removes.
- Anything that copied CDF output (audit tables, replicas, caches) must be purged too; archive tables built from CDF are ordinary tables under your own retention.
- After `VACUUM`, versions older than the retention period can no longer be queried with time travel.

6. Upstream raw data GDPR and CCPA apply to data outside Delta Lake too: delete from Kafka and other queues, source databases, and cloud storage. For files ingested by Auto Loader, `cloudFiles.cleanSource` (`OFF` default, `DELETE`, or `MOVE`; Databricks Runtime 16.4+) removes or archives processed files after `cloudFiles.cleanSource.retentionDuration` (default 30 days).

Retention-policy purging The same mechanics enforce time-based retention:

```
DELETE FROM prod.silver.web_events WHERE event_date < current_date() - INTERVAL
 ↪ 365 DAYS;
REORG TABLE prod.silver.web_events APPLY (PURGE);  -- if deletion vectors are
 ↪ enabled
-- VACUUM (or predictive optimization) removes the old files after the retention
 ↪ window
```

`VACUUM` commits audit information to the transaction log, so `DESCRIBE HISTORY` shows the `DELETE`, `REORG`, and `VACUUM` operations as evidence that a request was completed.

Use Cases

- **Pseudonymized analytics:** silver stores a salted sha2 customer key so analysts can join orders, clicks, and support tickets without ever seeing an email address.

- **Right-to-be-forgotten service:** a nightly job reads `gdpr_requests`, runs `MERGE ... DELETE` from bronze through gold, purges deletion vectors, and records completion once `VACUUM` has run past the retention window.
- **Automatic PII protection for new feeds:** Data Classification tags a newly ingested phone column with a `class.*` tag, and an existing catalog-level ABAC policy masks it the same day without any engineer action.
- **Research data release:** gold tables expose age bands and ZIP prefixes (generalization) with direct identifiers suppressed, for sharing with a partner.

Common Issues & Errors

Symptom	Cause
Deleted customer still visible with `VERSION AS OF`	Old files not yet vacuumed
Data still physically present after `DELETE` and `VACUUM`	Deletion vectors only marked
`VACUUM ... RETAIN 24 HOURS` seems to do nothing on Databricks Runtime 18	Retention argument ignored
Streaming table fails after a GDPR delete in its source	Streaming from Delta expect
Materialized view refreshed with masked values for everyone	Pipeline identity subject to th
`ALTER TABLE ... DROP COLUMN` of a PII column fails	Column has a governed tag
Hash values reversed by an attacker	Unsalted hash of a small valu

Best Practices

- Model PII so it can be deleted without destroying non-PII facts: key facts by a surrogate `user_id` or token, keep identifiers in one table.
- Keep raw PII in as few tables as possible; pseudonymize at write for silver and gold.
- Combine Data Classification auto-tagging with catalog-level ABAC policies so detection and masking scale together.
- Schedule erasure as a job with a control table, and record completion only after `VACUUM` has passed the retention window.
- Enable predictive optimization on managed tables so `VACUUM` actually runs.

Exam Tips

Tip

- Reversible with a key or lookup → **pseudonymization** (hashing with salt, tokenization, encryption). Not reversible → **anonymization** (suppression, generalization).
- "Same masked value everywhere so joins still work" → deterministic **hashing** (sha2) in a mask UDF or at write time.
- "Detect PII automatically across new tables" → **Data Classification** with auto-tagging (`class.*` tags) + **ABAC** policy on those tags.
- GDPR erasure on a table with deletion vectors: `DELETE`/`MERGE` → `REORG TABLE ... APPLY (PURGE)` → `VACUUM`. `OPTIMIZE` alone does not guarantee the purge.
- Default `VACUUM` threshold is 7 days (`delta.deletedFileRetentionDuration`); default history is 30 days (`delta.logRetentionDuration`).
- `VACUUM RETAIN 0 HOURS` after every `MERGE` is a trap: it breaks time travel and can delete files of in-flight writers.
- Streaming tables need `skipChangeCommits` (or a full refresh) after source deletes; materialized views handle deletes on refresh.

Key Takeaways

- **Hashing, tokenization, and encryption pseudonymize** (linkable, reversible with extra information); **suppression and generalization anonymize**.
- **Column masks and ABAC policies** apply these techniques at query time per group; write-time transformations remove PII from downstream storage.
- **Data Classification** scans tables, streaming tables, and materialized views, applies `class.*` system tags, and pairs with ABAC policies for scalable masking; it is GA, with its API GA since Oct 14, 2026.
- **Pipelines** should publish tables protected by Unity Catalog masks/filters, and the pipeline identity must be exempt from policies on its sources.
- **Erasure** requires delete → purge deletion vectors → `VACUUM` after retention, plus propagation to downstream tables, CDF consumers, and upstream sources.
- **Databricks Runtime 18.0+** ignores `VACUUM` retention arguments other than `0 HOURS`; manage retention with table properties.

Related Topics

- ABAC Policies and Governed Tags — policy syntax for tag-driven masking at scale
- Data Classification, Compliance & Permissions — tagging basics, CCPA export, deletion tracking table
- Access Control — table-level row filters, column masks, dynamic views
- Secret Management — secret scopes for salts and encryption keys
- Tags, Comments, and Discoverability — tags as metadata
- Auto Loader — ingesting the raw files that may also need purging

Official Documentation

- Prepare your data for GDPR compliance
- Deletion vectors in Databricks
- REORG TABLE
- Remove unused data files with vacuum
- VACUUM
- Work with table history
- Use change data feed on Databricks
- Databricks Runtime 18.0 release notes
- Data Classification
- Data Classification release notes
- Row filters and column masks
- Common patterns for row filtering and column masking
- Manage identities, permissions, and privileges for pipelines
- Use Unity Catalog with pipelines
- CREATE STREAMING TABLE (pipelines)
- Fine-grained access control on dedicated compute
- sha2 function
- secret function
- aes_encrypt function
- mask function
- Spark API options reference (Auto Loader `cloudFiles.cleanSource`)
- Apply watermarks to control data processing thresholds

Data Classification, Compliance & Permissions

Overview

This guide covers advanced data protection patterns: data classification & tagging, GDPR/CCPA compliance implementation, and advanced Unity Catalog permission models.

> For audit logging, lineage tracking, and network security, see Audit Logging, Data Lineage & Network Security.

Overview

- **Tags** on tables and columns (`SET TAGS`), queried through `information_schema.table_tags` and `column_tags`
- **GDPR and CCPA**: erasure is `DELETE` followed by `VACUUM`; subject access requests export a subject's records across tables
- **Permissions**: ownership vs privileges; `MANAGE` lets a principal grant without owning; grants inherit to future objects
- **Group-based access**: grant to groups synced from an identity provider through SCIM

What the Exam Tests

- Implement a compliant data pipeline that enforces PII detection and masking controls across batch and streaming workloads using Unity Catalog features.
- Implement a data purging strategy that satisfies data retention and deletion policies (e.g., GDPR right-to-erasure, GDPR right-to-be-forgotten deletion) using Delta Lake and Unity Catalog features.
- Explain why `DELETE` alone does not physically erase data

Data Classification and Tagging

Tags on Unity Catalog objects provide a mechanism for data classification, policy enforcement, and discovery. Tags are key-value pairs attached to catalogs, schemas, tables, and columns.

Setting Tags on Unity Catalog Objects

```
-- Tag a catalog
ALTER CATALOG prod SET TAGS ('environment' = 'production', 'owner_team' =
 ↪  'platform');

-- Tag a schema
ALTER SCHEMA prod.sensitive SET TAGS ('data_classification' = 'confidential');

-- Tag a table
ALTER TABLE prod.gold.customers SET TAGS (
    'contains_pii' = 'true',
    'data_owner' = 'customer-team',
    'retention_days' = '730'
);

-- Tag a column
ALTER TABLE prod.gold.customers ALTER COLUMN email SET TAGS (
    'pii_type' = 'email',
    'sensitivity' = 'high'
);
```

```
ALTER TABLE prod.gold.customers ALTER COLUMN ssn SET TAGS (
    'pii_type' = 'ssn',
    'sensitivity' = 'critical',
    'mask_required' = 'true'
);

-- Remove tags
ALTER TABLE prod.gold.customers UNSET TAGS ('retention_days');
```

Querying Tags for Compliance

```
-- Find all tables tagged as containing PII
SELECT
    catalog_name,
    schema_name,
    table_name,
    tag_name,
    tag_value
FROM system.information_schema.table_tags
WHERE tag_name = 'contains_pii'
    AND tag_value = 'true';

-- Find all columns tagged as sensitive
SELECT
    catalog_name,
    schema_name,
    table_name,
    column_name,
    tag_name,
    tag_value
FROM system.information_schema.column_tags
WHERE tag_name = 'sensitivity'
    AND tag_value IN ('high', 'critical');
```

PII Classification Patterns

```
flowchart TB
    subgraph Classification["Data Classification Levels"]
        Public[Public]
        Internal[Internal]
        Confidential[Confidential]
        Restricted[Restricted / Critical]
    end

    Public --> |"No restrictions"| Open[Open Access]
    Internal --> |"Employees only"| EmployeeAccess[Employee Groups]
    Confidential --> |"Need-to-know"| RoleAccess[Role-Based Access]
    Restricted --> |"Strict controls"| StrictAccess[Approved Users + Audit]
```

```
-- Implement classification hierarchy with tags
ALTER TABLE prod.public.product_catalog SET TAGS ('classification' = 'public');
ALTER TABLE prod.internal.employee_directory SET TAGS ('classification' =
 ↪  'internal');
ALTER TABLE prod.sensitive.customer_pii SET TAGS ('classification' =
 ↪  'confidential');
```

```
ALTER TABLE prod.restricted.payment_cards SET TAGS ('classification' =
 ↪  'restricted');
```

Sensitivity Labels for Columns

```
-- Classify columns by PII type
ALTER TABLE prod.gold.customers ALTER COLUMN first_name SET TAGS ('pii_type' =
 ↪  'name');
ALTER TABLE prod.gold.customers ALTER COLUMN last_name SET TAGS ('pii_type' =
 ↪  'name');
ALTER TABLE prod.gold.customers ALTER COLUMN email SET TAGS ('pii_type' =
 ↪  'email');
ALTER TABLE prod.gold.customers ALTER COLUMN phone SET TAGS ('pii_type' =
 ↪  'phone');
ALTER TABLE prod.gold.customers ALTER COLUMN ssn SET TAGS ('pii_type' =
 ↪  'government_id');
ALTER TABLE prod.gold.customers ALTER COLUMN date_of_birth SET TAGS ('pii_type' =
 ↪  'dob');
ALTER TABLE prod.gold.customers ALTER COLUMN ip_address SET TAGS ('pii_type' =
 ↪  'ip_address');
```

Automated Data Classification Approach

```
# Automated PII detection and tagging

from pyspark.sql import functions as F

PII_PATTERNS = {
    "email": r"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z]{2,}$",
    "ssn": r"^\d{3}-\d{2}-\d{4}$",
    "phone": r"^\+?1?\d{10,14}$",
    "credit_card": r"^\d{4}[-\s]?\d{4}[-\s]?\d{4}[-\s]?\d{4}$",
    "ip_address": r"^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$",
}

def classify_columns(table_name, sample_size=1000):
    """Scan a table's data to detect potential PII columns."""
    df = spark.table(table_name).limit(sample_size)
    results = {}

    for col_name in df.columns:
        for pii_type, pattern in PII_PATTERNS.items():
            match_count = df.filter(
                F.col(col_name).cast("string").rlike(pattern)
            ).count()
            match_ratio = match_count / sample_size if sample_size > 0 else 0

            if match_ratio > 0.5:
                results[col_name] = pii_type
                # Apply tag
                spark.sql(f"""
                    ALTER TABLE {table_name}
                    ALTER COLUMN `{col_name}`
                  SET TAGS ('pii_type' = '{pii_type}', 'auto_classified' = 'true')
                """)
```

```
    return results

# Run classification

detected = classify_columns("prod.gold.customers")
print(f"Detected PII columns: {detected}")
```

Compliance Patterns

GDPR: Right to Be Forgotten The General Data Protection Regulation (GDPR) requires organizations to delete personal data when a data subject requests it. With Delta Lake, this involves DELETE followed by VACUUM to permanently remove data from storage.

```
-- Step 1: Delete the data subject's records from all tables
DELETE FROM prod.gold.customers
WHERE customer_id = 'CUST-12345';

DELETE FROM prod.gold.orders
WHERE customer_id = 'CUST-12345';

DELETE FROM prod.gold.interactions
WHERE customer_id = 'CUST-12345';

-- Step 2: VACUUM to permanently remove deleted files
-- Default retention is 7 days; for GDPR you may need to reduce it
VACUUM prod.gold.customers RETAIN 0 HOURS;
VACUUM prod.gold.orders RETAIN 0 HOURS;
VACUUM prod.gold.interactions RETAIN 0 HOURS;
```

Why is VACUUM needed after DELETE?

Delta Lake DELETE creates new data files without the deleted rows but keeps the old files for time travel. **VACUUM** permanently removes old files from storage. Without VACUUM, the data subject's records still exist in older file versions.

```
-- IMPORTANT: You must disable the safety check for RETAIN 0 HOURS
SET spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM prod.gold.customers RETAIN 0 HOURS;
SET spark.databricks.delta.retentionDurationCheck.enabled = true;

-- WARNING: RETAIN 0 HOURS breaks time travel for all data in the table
-- Best practice: schedule GDPR deletions and VACUUM with 7-day retention
```

Platform update (Oct 2026)

On tables with deletion vectors, run `REORG TABLE ... APPLY (PURGE)` before `VACUUM` so deleted rows are rewritten out of the data files, and on Databricks Runtime 18.0+ `VACUUM` uses the `delta.deletedFileRetentionDuration` table property rather than the `RETAIN` argument (except `RETAIN 0 HOURS`). The full erasure workflow is in Anonymization, PII Controls, and Data Purging. Sources: GDPR compliance with Delta Lake, VACUUM

GDPR Deletion Tracking

```sql
-- Track deletion requests for compliance evidence
CREATE TABLE IF NOT EXISTS prod.compliance.deletion_requests (
    request_id STRING,
    data_subject_id STRING,
    request_date DATE,
    tables_affected ARRAY<STRING>,
    deletion_completed_at TIMESTAMP,
    vacuum_completed_at TIMESTAMP,
    processed_by STRING,
    status STRING
);

-- Log a completed deletion
INSERT INTO prod.compliance.deletion_requests VALUES (
    'DEL-2025-001',
    'CUST-12345',
    '2025-06-15',
    ARRAY('prod.gold.customers', 'prod.gold.orders', 'prod.gold.interactions'),
    current_timestamp(),
    NULL,
    'gdpr-service-principal',
    'deleted_pending_vacuum'
);
```

CCPA Data Subject Requests California Consumer Privacy Act (CCPA) requires the ability to provide all data held about a consumer and to delete it upon request.

```sql
-- CCPA: Data Subject Access Request (DSAR) - export all data for a user
CREATE OR REPLACE TEMPORARY VIEW ccpa_export AS
SELECT 'customers' AS source_table, to_json(struct(*)) AS record_data
FROM prod.gold.customers WHERE customer_id = 'CUST-12345'
UNION ALL
SELECT 'orders', to_json(struct(*))
FROM prod.gold.orders WHERE customer_id = 'CUST-12345'
UNION ALL
SELECT 'interactions', to_json(struct(*))
FROM prod.gold.interactions WHERE customer_id = 'CUST-12345';

-- Export to a file for delivery to the data subject
-- Write to a volume for secure access
SELECT * FROM ccpa_export;
```

Data Retention Policies with Delta Lake

```sql
-- Enforce retention by deleting data older than the retention period
DELETE FROM prod.gold.event_logs
WHERE event_date < current_date() - INTERVAL 365 DAYS;

-- Reclaim storage after deletion
VACUUM prod.gold.event_logs;

-- Set Delta table properties for automatic retention management
ALTER TABLE prod.gold.event_logs SET TBLPROPERTIES (
    'delta.logRetentionDuration' = 'interval 30 days',
    'delta.deletedFileRetentionDuration' = 'interval 7 days'
);
```

Regulatory Compliance Architecture

```
flowchart TB
    subgraph Ingestion["Data Ingestion"]
        Raw[Raw Data]
        Classify[Classify & Tag PII]
    end

    subgraph Processing["Data Processing"]
        Bronze[Bronze - Raw with PII tags]
        Silver[Silver - Masked / Pseudonymized]
        Gold[Gold - Aggregated / Anonymized]
    end

    subgraph Governance["Governance Controls"]
        Tags[Data Classification Tags]
        RLS[Row-Level Security]
        Masking[Column Masking]
        Audit[Audit Logging]
        Lineage[Data Lineage]
    end

    subgraph Compliance["Compliance Operations"]
        DSAR[Data Subject Requests]
        Deletion[Right to be Forgotten]
        Retention[Retention Policies]
        Reporting[Compliance Reporting]
    end

    Raw --> Classify --> Bronze --> Silver --> Gold
    Governance --> Processing
    Compliance --> Processing
```

Cross-Region Data Governance

```
-- Ensure data residency by using region-specific catalogs
CREATE CATALOG eu_data
MANAGED LOCATION 's3://eu-west-1-bucket/eu-data/';

CREATE CATALOG us_data
MANAGED LOCATION 's3://us-east-1-bucket/us-data/';

-- Restrict who can create tables in each region
GRANT CREATE TABLE ON CATALOG eu_data TO `eu-data-engineers`;
GRANT CREATE TABLE ON CATALOG us_data TO `us-data-engineers`;

-- Views can span regions for authorized users
CREATE VIEW prod.global.all_customers AS
SELECT *, 'EU' AS data_region FROM eu_data.gold.customers
UNION ALL
SELECT *, 'US' AS data_region FROM us_data.gold.customers;
```

Advanced Permission Patterns

Permission Inheritance Model

```
flowchart TD
    Metastore["Metastore Admin<br/>(full control over all objects)"]
    CatalogOwner["Catalog Owner<br/>(full control over catalog)"]
    SchemaOwner["Schema Owner<br/>(full control over schema)"]
    TableOwner["Table Owner<br/>(full control over table)"]

    Metastore --> |"implicit"| CatalogOwner
    CatalogOwner --> |"implicit"| SchemaOwner
    SchemaOwner --> |"implicit"| TableOwner

    style Metastore fill:#e6ccff
    style CatalogOwner fill:#cce5ff
    style SchemaOwner fill:#ccffcc
    style TableOwner fill:#ffe6cc
```

Key inheritance rules:

Rule	Description
Ownership cascades	Object owner has full control over all children
USE privilege required	Must have USE CATALOG and USE SCHEMA to access tables
Grants inherit downward	A privilege granted on a catalog or schema applies to all current **and future** objec
Usage still required	Inherited SELECT is usable only with USE CATALOG and USE SCHEMA on the parents

Ownership vs Privileges

```
-- Ownership: full control, can grant/revoke, alter, drop
ALTER TABLE prod.gold.revenue SET OWNER TO `data-engineering-team`;

-- Privileges: specific permissions, granted by owner or admin
GRANT SELECT ON TABLE prod.gold.revenue TO `analysts`;

-- Key differences
-- Owner can do everything; privilege holders can only do what's granted
-- Only one owner per object; many principals can have privileges
-- Ownership is transferred; privileges are granted/revoked
```

Capability	Owner	Privilege Holder
Read data	Yes	Only with SELECT
Modify data	Yes	Only with MODIFY
Grant to others	Yes	Only with GRANT OPTION
Drop object	Yes	No
Alter object	Yes	No
Transfer ownership	Yes	No

MANAGE Permission The MANAGE privilege allows a principal to manage grants on an object without being the owner. This is useful for delegating governance administration.

```
-- Grant MANAGE on a schema (can manage grants within the schema)
GRANT MANAGE ON SCHEMA prod.gold TO `governance-admins`;

-- Now governance-admins can grant/revoke within prod.gold
-- without being the schema owner

-- MANAGE is powerful: treat it like owner-level access for permissions
```

External Location Permissions

```
-- External location permission hierarchy
-- 1. Storage credential: how to authenticate
-- 2. External location: which paths are allowed
-- 3. Table/volume: the actual objects

-- Grant ability to create external tables using a location
GRANT CREATE EXTERNAL TABLE ON EXTERNAL LOCATION my_landing TO `data-engineers`;

-- Grant ability to read/write files directly
GRANT READ FILES ON EXTERNAL LOCATION my_landing TO `data-engineers`;
GRANT WRITE FILES ON EXTERNAL LOCATION my_landing TO `data-engineers`;

-- Grant ability to create external volumes
GRANT CREATE EXTERNAL VOLUME ON EXTERNAL LOCATION my_landing TO `data-engineers`;
```

Storage Credential Security

```
-- Storage credentials are the most privileged objects
-- Only metastore admins should create them
CREATE STORAGE CREDENTIAL prod_s3_cred
WITH (AWS_IAM_ROLE = 'arn:aws:iam::123456789:role/databricks-prod');

-- Grant CREATE EXTERNAL LOCATION to allow teams to create
-- their own external locations using this credential
GRANT CREATE EXTERNAL LOCATION ON STORAGE CREDENTIAL prod_s3_cred
TO `platform-team`;

-- List all storage credentials
SHOW STORAGE CREDENTIALS;

-- Audit who uses a storage credential
SELECT *
FROM system.access.audit
WHERE action_name IN ('createStorageCredential', 'getStorageCredential')
    AND event_date >= current_date() - 30;
```

Service Principal Patterns for Data Access

```
flowchart LR
    subgraph Jobs["Automated Pipelines"]
        ETL[ETL Job]
        ML[ML Training]
        BI[BI Refresh]
    end

    subgraph ServicePrincipals["Service Principals"]
        SP_ETL[sp-etl-pipeline]
        SP_ML[sp-ml-training]
        SP_BI[sp-bi-reader]
    end

    subgraph AccessLevels["Access Levels"]
        Full[Read + Write Bronze/Silver]
        ReadGold[Read Gold + Write ML Schema]
```

```
        ReadOnly[Read Gold Only]
    end

    ETL --> SP_ETL --> Full
    ML --> SP_ML --> ReadGold
    BI --> SP_BI --> ReadOnly
```

```
-- ETL service principal: broad write access
GRANT USE CATALOG ON CATALOG prod TO `sp-etl-pipeline`;
GRANT USE SCHEMA ON SCHEMA prod.bronze TO `sp-etl-pipeline`;
GRANT USE SCHEMA ON SCHEMA prod.silver TO `sp-etl-pipeline`;
GRANT SELECT, MODIFY, CREATE TABLE ON SCHEMA prod.bronze TO `sp-etl-pipeline`;
GRANT SELECT, MODIFY, CREATE TABLE ON SCHEMA prod.silver TO `sp-etl-pipeline`;

-- ML service principal: read gold, write to ML schema
GRANT USE CATALOG ON CATALOG prod TO `sp-ml-training`;
GRANT USE SCHEMA ON SCHEMA prod.gold TO `sp-ml-training`;
GRANT SELECT ON SCHEMA prod.gold TO `sp-ml-training`;
GRANT USE SCHEMA ON SCHEMA prod.ml TO `sp-ml-training`;
GRANT SELECT, MODIFY, CREATE TABLE ON SCHEMA prod.ml TO `sp-ml-training`;

-- BI service principal: read-only on gold
GRANT USE CATALOG ON CATALOG prod TO `sp-bi-reader`;
GRANT USE SCHEMA ON SCHEMA prod.gold TO `sp-bi-reader`;
GRANT SELECT ON SCHEMA prod.gold TO `sp-bi-reader`;
```

Group-Based Access Best Practices

```
-- Best practice: NEVER grant directly to individual users
-- Always use groups for scalable access management

-- Anti-pattern (avoid):
GRANT SELECT ON TABLE prod.gold.revenue TO `alice@company.com`;
GRANT SELECT ON TABLE prod.gold.revenue TO `bob@company.com`;

-- Best practice (use groups):
GRANT SELECT ON TABLE prod.gold.revenue TO `finance-analysts`;
-- Add alice and bob to the finance-analysts group via SCIM/Identity Provider

-- Layered group strategy
-- Layer 1: Role groups (what they do)
--   analysts, engineers, scientists
-- Layer 2: Team groups (which team)
--   sales-team, marketing-team, finance-team
-- Layer 3: Access-level groups (how much access)
--   gold-readers, silver-writers, admin-access

-- Example: implementing layered group access
-- Sales analysts get read access to sales gold tables
GRANT USE CATALOG ON CATALOG prod TO `sales-analysts`;
GRANT USE SCHEMA ON SCHEMA prod.gold TO `sales-analysts`;
GRANT SELECT ON TABLE prod.gold.sales_summary TO `sales-analysts`;
GRANT SELECT ON TABLE prod.gold.sales_by_region TO `sales-analysts`;

-- Sales engineers get write access to sales silver tables
GRANT USE CATALOG ON CATALOG prod TO `sales-engineers`;
GRANT USE SCHEMA ON SCHEMA prod.silver TO `sales-engineers`;
```

```
GRANT SELECT, MODIFY, CREATE TABLE ON SCHEMA prod.silver TO `sales-engineers`;
GRANT USE SCHEMA ON SCHEMA prod.gold TO `sales-engineers`;
GRANT SELECT ON SCHEMA prod.gold TO `sales-engineers`;
```

Practice Questions

Question 1: A data engineer needs to determine which downstream tables and dashboards will be affected by renaming a column in `prod.silver.orders`. Which Unity Catalog feature should they use?

A) Information Schema B) Data Lineage and Impact Analysis C) Audit Logging D) OpenSharing

Answer

Correct Answer: B
Data Lineage in Unity Catalog tracks column-level dependencies. By querying the lineage API or using Catalog Explorer, the engineer can see all downstream tables and notebooks that reference the column, enabling proper impact analysis before making the change.

Question 2: An organization must retain audit logs for 3 years for SOC 2 compliance. The default retention for `system.access.audit` is 365 days. What is the recommended approach?

A) Change the system table retention setting to 3 years B) Create a scheduled job to archive audit logs to a separate Delta table C) Export audit logs to CSV files monthly D) Enable extended retention via the account console

Answer

Correct Answer: B
The `system.access.audit` table retention cannot be changed directly. The recommended approach is to create a scheduled job that copies audit records to your own Delta table (e.g., `prod.compliance.audit_archive`) where you control the retention policy.

Question 3: Which SQL command correctly applies a tag to indicate a column contains personally identifiable information?

A) `ALTER TABLE prod.gold.customers SET TAGS ('pii' = 'true')` B) `ALTER TABLE prod.gold.customers ALTER COLUMN email SET TAGS ('pii_type' = 'email')` C) `TAG COLUMN prod.gold.customers.email AS 'pii'` D) `UPDATE METADATA SET tag = 'pii' WHERE column = 'email'`

Answer

Correct Answer: B
Column-level tags use `ALTER TABLE ... ALTER COLUMN ... SET TAGS (...)` syntax. Option A tags the table, not the column. Options C and D use invalid syntax.

Question 4: A company needs to comply with GDPR right-to-be-forgotten. After running `DELETE FROM prod.gold.customers WHERE customer_id = 'X'`, what additional step is required to permanently remove the data from storage?

A) Run `OPTIMIZE` on the table B) Run `VACUUM` on the table C) Run `PURGE` on the table D) The `DELETE` command permanently removes the data

Answer

Correct Answer: B
Delta Lake DELETE creates new data files without the deleted rows but retains old files for time travel. VACUUM permanently removes old files from storage. Without VACUUM, the data subject's records still exist in older file versions and can be recovered via time travel.

Question 5: Which permission allows a principal to manage grants on a Unity Catalog object without being the owner?

A) ALL PRIVILEGES B) MANAGE C) ADMIN D) GRANT OPTION

Answer

Correct Answer: B
The `MANAGE` privilege allows a principal to grant and revoke permissions on an object without being the owner. `ALL PRIVILEGES` grants all data access but not necessarily grant management. `GRANT OPTION` allows passing along one's own privileges but is not a standalone permission for managing all grants.

Use Cases

- **GDPR 'Right to be Forgotten' Fulfillment**: Automating the deletion of specific customer IDs across all Gold tables and immediately running `VACUUM` with 0 retention to permanently purge the files.
- **Automated PII Tagging**: Running a daily script that scans new tables for patterns (like emails or SSNs) and automatically applies Unity Catalog tags to classify their sensitivity for governance tracking.
- **Service Principal Pipelines**: Using service principals with minimal privileges inherited from a group to securely run automated ETL jobs without relying on an individual's personal access token.

Common Issues & Errors

Lineage Not Showing **Scenario:** Table lineage is empty in Catalog Explorer.

Cause: Lineage is only captured for queries run on Unity Catalog-managed tables. Tables in `hive_metastore` do not generate lineage.

Fix:

```
-- Migrate tables to Unity Catalog
CREATE TABLE prod.silver.orders AS
SELECT * FROM hive_metastore.default.orders;

-- Run the pipeline again; lineage will now be captured
```

Audit Logs Missing Events **Scenario:** Cannot find expected events in `system.access.audit`.

Fix: Verify the correct filters:

```
-- Check available service names
SELECT DISTINCT service_name
FROM system.access.audit
```

```
WHERE event_date >= current_date() - 1;

-- Check available action names for a service
SELECT DISTINCT action_name
FROM system.access.audit
WHERE service_name = 'unityCatalog'
    AND event_date >= current_date() - 1;
```

Information Schema Returns Empty Results **Scenario:** Query returns no rows.

Cause: The user only sees objects they have permission to access.

Fix: Grant appropriate privileges or query as an admin:

```
-- information_schema respects permissions
-- User must have USE CATALOG / USE SCHEMA to see objects
GRANT USE CATALOG ON CATALOG prod TO `compliance-team`;
GRANT USE SCHEMA ON SCHEMA prod.gold TO `compliance-team`;
```

Tags Not Appearing **Scenario:** Tags set on objects are not queryable.

Fix: Ensure you are querying the correct system table:

```
-- Table tags
SELECT * FROM system.information_schema.table_tags
WHERE catalog_name = 'prod';

-- Column tags
SELECT * FROM system.information_schema.column_tags
WHERE catalog_name = 'prod';
```

VACUUM With RETAIN 0 HOURS Fails **Scenario:** Cannot run VACUUM with 0-hour retention.

Fix: Disable the safety check (understand the impact first):

```
-- This disables time travel for affected data
SET spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM prod.gold.customers RETAIN 0 HOURS;
SET spark.databricks.delta.retentionDurationCheck.enabled = true;
```

Permission Denied on External Location **Scenario:** User cannot create an external table.

Fix: Grant the required external location privilege:

```
-- Check existing grants
SHOW GRANTS ON EXTERNAL LOCATION my_landing;

-- Grant access
GRANT CREATE EXTERNAL TABLE ON EXTERNAL LOCATION my_landing TO `data-engineers`;
```

Exam Tips

1. **Lineage is automatic** -- Unity Catalog captures table and column lineage for all queries against UC-managed tables (not hive_metastore)
2. **Audit table** -- `system.access.audit` is the primary table for compliance monitoring; filter by `service_name`, `action_name`, and `event_date`

3. **Information schema** -- `system.information_schema` tables follow SQL standard; visibility is filtered by user permissions
4. **VACUUM for GDPR** -- DELETE alone is insufficient; VACUUM is required to permanently remove data from Delta Lake storage
5. **Tags for classification** -- Use `ALTER TABLE/COLUMN SET TAGS` for data classification; query tags via `system.information_schema.table_tags` and `column_tags`
6. **Private Link vs SCC** -- Private Link secures control-plane-to-data-plane traffic; SCC removes public IPs from cluster nodes; both are recommended for regulated workloads
7. **MANAGE permission** -- Allows granting/revoking on an object without ownership; note that `ALL PRIVILEGES` does not include `MANAGE`
8. **Group-based access** -- Always use groups rather than individual user grants for scalable permission management
9. **Service principals** -- Use separate service principals per pipeline/application with least-privilege access
10. **Ownership vs privileges** -- Owners have full control (including DROP and ALTER); privilege holders are limited to specific operations

---

Key Takeaways

- **Tag syntax**: table tags use `ALTER TABLE ... SET TAGS ('key' = 'value')`; column tags use `ALTER TABLE ... ALTER COLUMN <col> SET TAGS ('key' = 'value')`; tags are queried via `system.information_schema.table_tags` and `system.information_schema.column_tags`
- **GDPR right-to-be-forgotten**: `DELETE` alone is not sufficient — Delta Lake keeps old data files for time travel; `VACUUM` must also be run to permanently remove deleted files from storage; `RETAIN 0 HOURS` requires disabling the retention safety check
- **CCPA data subject requests**: export all records for a data subject using `UNION ALL` across tables filtered by `customer_id`, then deliver to the consumer; deletion follows the same DELETE + VACUUM pattern as GDPR
- **MANAGE permission**: allows a principal to grant/revoke privileges on an object without being the owner; `ALL PRIVILEGES` does not include it
- **Ownership vs privileges**: owners have full implicit control (SELECT, MODIFY, DROP, ALTER, GRANT, transfer ownership); privilege holders are limited to the specific operations granted to them; only owners can drop objects
- **Grants inherit to future objects**: granting `SELECT ON SCHEMA prod.gold TO analysts` covers the tables in that schema now and any created later (analysts still need `USE CATALOG` and `USE SCHEMA`) — see Unity Catalog permission inheritance
- **Group-based access best practice**: always grant to groups rather than individual users; use layered group strategies (role groups + team groups + access-level groups) synced from an identity provider via SCIM
- **Lineage only works for UC-managed tables**: hive_metastore tables do not generate automatic lineage; migration to Unity Catalog is required to enable lineage tracking

Related Topics

- Unity Catalog - UC basics, hierarchy, metastore, permissions
- Access Control - Row/column security, dynamic views
- Data Sharing - OpenSharing
- Secret Management - Secrets, Key Vault

Official Documentation

- Unity Catalog Data Lineage

- Audit Logs (System Tables)
- Information Schema
- Tags in Unity Catalog
- Private Link
- Secure Cluster Connectivity
- IP Access Lists
- GDPR and Databricks
- VACUUM

Secret Management

Secure credential management is essential for production data pipelines. Databricks provides secret scopes to store and access sensitive information like passwords, API keys, and connection strings without exposing them in code.

Overview

```
flowchart TB
    Code[Notebook/Job] --> Dbutils[dbutils.secrets]
    Dbutils --> Scope[Secret Scope]

    subgraph Backends["Secret Backends"]
        DBBacked[Databricks-backed]
        AzureKV[Azure Key Vault]
        AWSSecrets[AWS Secrets Manager]
    end

    Scope --> Backends
```

Overview

- **Scope types**: Databricks-backed and Azure Key Vault-backed; scopes are created with the CLI or REST API, not SQL
- **`dbutils.secrets.get()`** values are redacted in notebook output, but transforming the string can bypass redaction
- **Scope ACLs**: `READ`, `WRITE`, and `MANAGE`
- **Secrets vs Unity Catalog credentials**: secrets for application credentials, storage credentials for cloud storage; jobs reference secrets as `{{secrets/scope/key}}`

What the Exam Tests

- Apply least-privilege access control lists (ACLs) to secure Unity Catalog securable objects and workspace resources.
- Choose between a secret scope and a Unity Catalog storage credential
- Audit secret access in `system.access.audit`

Secret Scope Types

Type	Backend	Management	Best For
Databricks-backed	Databricks internal storage	Databricks CLI/API	Quick setup, simple use c
Azure Key Vault-backed	Azure Key Vault	Azure portal + Databricks	Enterprise Azure, centraliz

Type	Backend	Management	Best For
AWS Secrets Manager	AWS Secrets Manager	AWS console + Databricks	Enterprise AWS, centralize

Databricks-Backed Scopes

Creating Scopes

```
# Create scope via CLI

databricks secrets create-scope --scope my-scope

# Create scope with initial manage principal

databricks secrets create-scope --scope team-secrets --initial-manage-principal
 ↪  users
# Via REST API

import requests

response = requests.post(
    f"{workspace_url}/api/2.0/secrets/scopes/create",
    headers={"Authorization": f"Bearer {token}"},
    json={"scope": "my-scope"}
)
```

Managing Secrets

```
# Add/update secret

databricks secrets put --scope my-scope --key db-password --string-value
 ↪  "secret123"

# Add secret from file

databricks secrets put --scope my-scope --key ssh-key --binary-file ./id_rsa

# List secrets in scope (shows keys only, not values)

databricks secrets list --scope my-scope

# Delete secret

databricks secrets delete --scope my-scope --key db-password

# Delete scope

databricks secrets delete-scope --scope my-scope
```

Scope ACLs

```
# List ACLs for scope

databricks secrets list-acls --scope my-scope
```

```
# Grant access

databricks secrets put-acl --scope my-scope --principal user@company.com
 ↪  --permission READ
databricks secrets put-acl --scope my-scope --principal data-engineers
 ↪  --permission WRITE

# Revoke access

databricks secrets delete-acl --scope my-scope --principal user@company.com
```

Permission	Capabilities
READ	Get secret values
WRITE	Create/update secrets + READ
MANAGE	Full control including ACLs

Azure Key Vault-Backed Scopes

Setup Requirements

1. Azure Key Vault with secrets
2. Databricks service principal with Key Vault access
3. Key Vault-backed scope in Databricks

Creating Key Vault Scope

```
# Via CLI

databricks secrets create-scope \
    --scope azure-kv-scope \
    --scope-backend-type AZURE_KEYVAULT \
    --resource-id /subscriptions/<subscription-id>/resourceGroups/<rg>
     ↪  /providers/Microsoft.KeyVault/vaults/<vault-name>
     ↪  \
    --dns-name https://<vault-name>.vault.azure.net/

# The scope automatically syncs with Key Vault
```

Key Vault Configuration

```
Azure Key Vault Setup:
1. Create Key Vault in Azure portal
2. Add secrets to Key Vault
3. Grant Databricks managed identity or service principal:
   - Key Vault Secrets User role (read secrets)
   - Or configure Access Policies with Get secret permission
4. Create scope in Databricks pointing to Key Vault
```

AWS Secrets Manager

Integration Pattern

```
# AWS Secrets Manager integration via IAM role
```

```python
# Option 1: Instance profile with Secrets Manager access
# Cluster has IAM role that can access Secrets Manager

import boto3
import json

def get_secret(secret_name, region_name="us-east-1"):
    client = boto3.client("secretsmanager", region_name=region_name)
    response = client.get_secret_value(SecretId=secret_name)

    if "SecretString" in response:
        return json.loads(response["SecretString"])
    else:
        return response["SecretBinary"]

# Usage

db_creds = get_secret("prod/database/credentials")
password = db_creds["password"]
```

Databricks-backed for AWS Secrets

```python
# Create Databricks-backed scope (recommended for Databricks-native approach)

databricks secrets create-scope --scope aws-creds

# Store AWS credentials or other secrets

databricks secrets put --scope aws-creds --key api-key
```

Accessing Secrets in Code

dbutils.secrets API

```python
# List available scopes

scopes = dbutils.secrets.listScopes()
for scope in scopes:
    print(scope.name)

# List secrets in scope (names only)

secrets = dbutils.secrets.list("my-scope")
for secret in secrets:
    print(secret.key)

# Get secret value

password = dbutils.secrets.get(scope="my-scope", key="db-password")
api_key = dbutils.secrets.get(scope="azure-kv", key="api-key")

# Get secret as bytes (for binary secrets)

ssh_key = dbutils.secrets.getBytes(scope="my-scope", key="ssh-key")
```

Secret Redaction Secrets are automatically redacted in notebook output:

```
# Secret values are redacted

password = dbutils.secrets.get("my-scope", "db-password")
print(password)  # Output: [REDACTED]

# Even in f-strings

print(f"Password is: {password}")  # Output: Password is: [REDACTED]

# In DataFrames

df = spark.createDataFrame([(password,)], ["secret"])
df.show()  # Shows [REDACTED]
```

When Redaction Doesn't Work

```
# Redaction doesn't work if you transform the secret

password = dbutils.secrets.get("my-scope", "db-password")

# These may expose the secret - AVOID!

print(password[0:5])  # Partial string - NOT redacted
print(list(password))  # Converted to list - NOT redacted
print(password.encode())  # Encoded - NOT redacted
```

Secrets in Job Parameters

Referencing Secrets in Jobs

```
# In job configuration, reference secrets
# Notebook parameter: {{secrets/my-scope/db-password}}

# In notebook, access via widget or directly

password = dbutils.widgets.get("password")
# or

password = dbutils.secrets.get("my-scope", "db-password")
```

Job Configuration Example

```
{
  "name": "ETL Job",
  "tasks": [{
    "task_key": "extract",
    "notebook_task": {
      "notebook_path": "/Jobs/extract",
      "base_parameters": {
        "db_host": "{{secrets/db-scope/host}}",
        "db_password": "{{secrets/db-scope/password}}"
      }
    }
  }]
}
```

Integration with Unity Catalog

Secrets vs Unity Catalog Credentials

Aspect	Secrets	UC Storage Credentials
Scope	Workspace-level	Account-level (Unity Catalog)
Use case	Application credentials	Cloud storage access
Management	dbutils.secrets	Unity Catalog SQL
Governance	Scope ACLs	UC permissions

When to Use Each

```python
# Use secrets for:
# - Database passwords
# - API keys
# - Custom application credentials

password = dbutils.secrets.get("app-scope", "password")

# Use Unity Catalog for:
# - Cloud storage access (external locations)
# - Governed data access

# Storage access via UC (no secrets needed)

df = spark.read.format("delta").load("/Volumes/catalog/schema/volume/data/")
```

Monitoring and Auditing

Audit Secret Access

```sql
-- Query audit logs for secret access
SELECT
    event_time,
    user_identity.email,
    action_name,
    request_params.scope AS scope,
    request_params.key AS secret_key,
    response.status_code
FROM system.access.audit
WHERE service_name = 'secrets'
ORDER BY event_time DESC
LIMIT 100;

-- Track who accessed specific scope
SELECT
    user_identity.email,
    COUNT(*) AS access_count,
    MAX(event_time) AS last_access
FROM system.access.audit
WHERE service_name = 'secrets'
    AND request_params.scope = 'prod-database'
    AND action_name = 'getSecret'
GROUP BY user_identity.email;
```

Use Cases

Database Connections

```
# Get database credentials

host = dbutils.secrets.get("db-scope", "host")
port = dbutils.secrets.get("db-scope", "port")
database = dbutils.secrets.get("db-scope", "database")
username = dbutils.secrets.get("db-scope", "username")
password = dbutils.secrets.get("db-scope", "password")

# JDBC connection

jdbc_url = f"jdbc:postgresql://{host}:{port}/{database}"

df = (spark.read
    .format("jdbc")
    .option("url", jdbc_url)
    .option("dbtable", "public.customers")
    .option("user", username)
    .option("password", password)
    .load())
```

API Authentication

```
import requests

# Get API key from secrets

api_key = dbutils.secrets.get("api-scope", "external-api-key")

# Use in API call

headers = {"Authorization": f"Bearer {api_key}"}
response = requests.get("https://api.example.com/data", headers=headers)
```

Cloud Storage Credentials

```
# Set Spark config with secrets

storage_account = "mystorageaccount"
access_key = dbutils.secrets.get("azure-scope", "storage-access-key")

spark.conf.set(
    f"fs.azure.account.key.{storage_account}.blob.core.windows.net",
    access_key
)

# Now can access storage

df = spark.read.format("parquet").load(
    f"wasbs://container@{storage_account}.blob.core.windows.net/data/"
)
```

Service Principal Credentials

```python
# Azure service principal authentication

client_id = dbutils.secrets.get("azure-scope", "sp-client-id")
client_secret = dbutils.secrets.get("azure-scope", "sp-client-secret")
tenant_id = dbutils.secrets.get("azure-scope", "tenant-id")

# Configure for ADLS Gen2

spark.conf.set("fs.azure.account.auth.type", "OAuth")
spark.conf.set("fs.azure.account.oauth.provider.type",
    "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
spark.conf.set("fs.azure.account.oauth2.client.id", client_id)
spark.conf.set("fs.azure.account.oauth2.client.secret", client_secret)
spark.conf.set("fs.azure.account.oauth2.client.endpoint",
    f"https://login.microsoftonline.com/{tenant_id}/oauth2/token")
```

Common Issues & Errors

Secret Scope Not Found **Scenario:** Scope doesn't exist or user lacks access.

```
# Error: Secret scope 'nonexistent' not found
```

Fix: Verify scope exists and user has access:

```
databricks secrets list-scopes
databricks secrets list-acls --scope my-scope
```

Permission Denied **Scenario:** User can't read secrets.

Fix: Grant READ permission:

```
databricks secrets put-acl --scope my-scope --principal user@company.com
 ↪  --permission READ
```

Secret Not Found in Scope **Scenario:** Key doesn't exist in scope.

```
# Error: Secret 'missing-key' not found in scope 'my-scope'
```

Fix: Verify secret exists:

```
databricks secrets list --scope my-scope
# Add if missing

databricks secrets put --scope my-scope --key missing-key
```

Azure Key Vault Connection Failed **Scenario:** Can't access Key Vault-backed scope.

Fix: Verify:

1. Key Vault exists and DNS name is correct
2. Service principal has Key Vault access
3. Network connectivity (private endpoints if applicable)

Secret Value Accidentally Exposed **Scenario:** Secret leaked in logs or output.

Fix:

1. Immediately rotate the secret
2. Review code for transformation of secret values
3. Ensure no `print(secret[0:n])` patterns

Best Practices

Scope Organization

```
Recommended scope structure:
├── prod-database           # Production database credentials
├── dev-database            # Development database credentials
├── azure-storage           # Azure storage keys
├── api-keys               # External API keys
├── service-principals     # SP credentials
└── team-secrets           # Team-specific secrets
```

Access Control Pattern

```
# Production: Limited access

databricks secrets put-acl --scope prod-database --principal
 ↪  etl-service-principal --permission READ
databricks secrets put-acl --scope prod-database --principal platform-admins
 ↪  --permission MANAGE

# Development: Broader access

databricks secrets put-acl --scope dev-database --principal developers
 ↪  --permission READ
databricks secrets put-acl --scope dev-database --principal developers
 ↪  --permission WRITE
```

Secret Rotation

```
def rotate_database_password(scope, key, new_password):
    """Rotate database password and update secret."""
    # 1. Update password in target system
    # (database, API, etc.)

    # 2. Update secret in Databricks
    # Via CLI: databricks secrets put --scope {scope} --key {key}

    # 3. Verify new password works
    # Test connection

    # 4. Log rotation event
    print(f"Rotated secret {scope}/{key}")
```

Exam Tips

1. **Scope types** - Databricks-backed, Azure Key Vault, AWS Secrets Manager
2. **dbutils.secrets** - listScopes(), list(), get(), getBytes()
3. **Redaction** - Automatic in output, doesn't work on transformed values
4. **ACL permissions** - READ, WRITE, MANAGE
5. **Scope creation** - Via CLI or API, not SQL
6. **Key Vault integration** - Requires resource ID and DNS name
7. **Secret reference in jobs** - {{secrets/scope/key}} syntax
8. **Never hardcode** - Always use secrets for credentials
9. **Audit logs** - system.access.audit for secret access tracking
10. **UC vs Secrets** - UC for storage credentials, secrets for app credentials

Key Takeaways

- **Secret scope types**: Databricks-backed scopes store secrets in Databricks internal storage (managed via CLI/API); Azure Key Vault-backed scopes proxy to Azure Key Vault (requires resource ID + DNS name); secret values can never be retrieved via CLI or SQL — only key names are listed
- **dbutils.secrets API**: `dbutils.secrets.listScopes()`, `.list(scope)`, `.get(scope, key)`, `.getBytes(scope, key)` — always call `.get()` in code, never hardcode credentials
- **Automatic redaction**: secret values retrieved via `dbutils.secrets.get()` are automatically displayed as `[REDACTED]` in notebook output; transformations on the secret string (slicing, encoding, joining to list) bypass redaction and may expose the value
- **Scope ACL permissions**: `READ` (get values), `WRITE` (create/update secrets + READ), `MANAGE` (full control including ACLs); created with `databricks secrets put-acl`
- **Scope creation**: scopes can only be created via the Databricks CLI (`databricks secrets create-scope`) or REST API — not via SQL
- **Job secret references**: secrets can be injected as job task parameters using `{{secrets/scope/key}}` syntax in the job configuration JSON
- **Secrets vs Unity Catalog credentials**: use Databricks secrets for application credentials (passwords, API keys); use UC storage credentials for cloud storage access paths — these are separate systems with different management
- **Audit trail**: secret access is logged in `system.access.audit` with `service_name = 'secrets'` and `action_name = 'getSecret'`; monitor for unusual patterns

Related Topics

- Unity Catalog - Storage credentials
- Access Control - Permission patterns
- DBFS and Mounts - Storage integration

Official Documentation

- Secret Management
- Secret Scopes
- Azure Key Vault-backed Scopes
- Databricks CLI Secrets

Audit Logging, Data Lineage & Network Security

This guide covers advanced Databricks governance topics focused on observability and infrastructure security: data lineage & impact analysis, audit logging, information schema metadata queries, and network security (Private Link, Secure Cluster Connectivity).

Overview

```
flowchart TB
    AdvGov[Advanced Governance] --> Lineage[Data Lineage]
    AdvGov --> Audit[Audit Logging]
    AdvGov --> InfoSchema[Information Schema]
    AdvGov --> Network[Network Security]
    AdvGov --> Classification[Data Classification]
    AdvGov --> Compliance[Compliance Patterns]
    AdvGov --> Permissions[Advanced Permissions]

    Lineage --> LineageAPI[Lineage API]
```

```
    Lineage --> ImpactAnalysis[Impact Analysis]
    Audit --> SystemTables[System Tables]
    Audit --> Alerting[Alerting]
    InfoSchema --> MetadataQueries[Metadata Queries]
    Network --> PrivateLink[Private Link]
    Network --> SCC[Secure Cluster Connectivity]
    Classification --> Tags[Tags & Labels]
    Compliance --> GDPR[GDPR / CCPA]
    Permissions --> Inheritance[Permission Inheritance]
```

Overview

- **Lineage**: Unity Catalog records table, column, and notebook or job lineage for UC tables (not `hive_metastore`); the lineage API supports impact analysis
- **Audit logs**: `system.access.audit` key columns and 365-day default retention; archive by copying to your own table
- **Information schema** results are filtered by the caller's permissions
- **Network security**: Secure Cluster Connectivity removes public IPs from cluster nodes; Private Link keeps traffic on the cloud provider's private network

What the Exam Tests

- Use Databricks system tables (billing, compute, access, lakeflow) for cost analysis, auditing, and workload monitoring.
- Use lineage to assess the impact of a schema change
- Distinguish Secure Cluster Connectivity from Private Link

Data Lineage and Impact Analysis

Unity Catalog automatically captures data lineage for tables, columns, notebooks, and jobs. Lineage records are created whenever data flows from one object to another through Spark queries or pipeline operations.

How Lineage Is Captured

```
flowchart LR
    subgraph Sources["Source Objects"]
        S1[Table A]
        S2[Table B]
    end

    subgraph Processing["Processing"]
        NB[Notebook / Job]
    end

    subgraph Targets["Target Objects"]
        T1[Table C]
    end

    S1 --> NB
    S2 --> NB
    NB --> T1
```

```
    subgraph LineageStore["Unity Catalog Lineage"]
        L1["Table A --> Table C"]
        L2["Table B --> Table C"]
        L3["Column-level mappings"]
    end
```

Lineage Type	What Is Tracked	Automatically Captured?
Table-to-table	Which tables feed into other tables	Yes
Column-level	Which source columns map to target columns	Yes
Notebook/job	Which notebook or job produced the data	Yes
Cross-workspace	Lineage across workspaces sharing the same metastore	Yes
Dashboard	Queries powering dashboard visualizations	Yes

Viewing Lineage in Catalog Explorer Lineage is visualized in the Catalog Explorer UI under the **Lineage** tab for any table or column. The graph shows upstream (sources) and downstream (consumers) in an interactive diagram.

Lineage API (Programmatic Access)

```
import requests

# Table lineage: get upstream and downstream tables

response = requests.get(
    f"{workspace_url}/api/2.1/unity-catalog/lineage/table-lineage",
    headers={"Authorization": f"Bearer {token}"},
    json={
        "table_name": "prod.gold.revenue"
    }
)

upstream_tables = response.json().get("upstreams", [])
downstream_tables = response.json().get("downstreams", [])

for table in upstream_tables:
    print(f"Upstream: {table['tableInfo']['name']}")

for table in downstream_tables:
    print(f"Downstream: {table['tableInfo']['name']}")

# Column lineage: get column-level dependencies

response = requests.get(
    f"{workspace_url}/api/2.1/unity-catalog/lineage/column-lineage",
    headers={"Authorization": f"Bearer {token}"},
    json={
        "table_name": "prod.gold.revenue",
        "column_name": "total_amount"
    }
)

for col in response.json().get("upstream_cols", []):
    print(f"Source:
     ↪  {col['catalog']}.{col['schema']}.{col['table']}.{col['name']}")
```

Impact Analysis Impact analysis uses downstream lineage to understand the blast radius of a schema change (rename column, drop table, change data type).

```
def assess_impact(workspace_url, token, table_name):
    """Identify all downstream objects affected by changes to a table."""
    response = requests.get(
        f"{workspace_url}/api/2.1/unity-catalog/lineage/table-lineage",
        headers={"Authorization": f"Bearer {token}"},
        json={"table_name": table_name}
    )

    downstreams = response.json().get("downstreams", [])
    print(f"Changing '{table_name}' affects {len(downstreams)} downstream
     ↪  objects:")

    for ds in downstreams:
        info = ds.get("tableInfo", {})
        print(f"  - {info.get('catalog_name')}.{info.get('schema_name')}.{info.
         ↪  get('name')}")
        # Also check notebook/job dependencies
        if ds.get("notebookInfos"):
            for nb in ds["notebookInfos"]:
                print(f"    Notebook: {nb.get('notebook_path')}")

# Usage

assess_impact(workspace_url, token, "prod.silver.orders")
```

Cross-Workspace Lineage When multiple workspaces share the same Unity Catalog metastore, lineage is tracked across workspace boundaries automatically. A pipeline in Workspace A that reads from `prod.bronze.events` and writes to `prod.silver.events_cleaned` is captured the same way regardless of which workspace runs the job.

Column-Level Lineage Column-level lineage tracks how individual columns are derived:

```
-- Example: column lineage is captured automatically
CREATE OR REPLACE TABLE prod.gold.revenue AS
SELECT
    o.order_date,
    c.region,
    SUM(o.amount) AS total_revenue  -- lineage: amount -> total_revenue
FROM prod.silver.orders o
JOIN prod.silver.customers c
    ON o.customer_id = c.customer_id
GROUP BY o.order_date, c.region;

-- In Catalog Explorer, the column "total_revenue" will show
-- lineage back to prod.silver.orders.amount
-- The column "region" will show lineage back to prod.silver.customers.region
```

Audit Logging Deep Dive

The `system.access.audit` table captures all access events across workspaces and Unity Catalog. This is the primary tool for compliance monitoring and forensic investigation.

Audit Table Structure

Column	Type	Description
`event_time`	TIMESTAMP	When the event occurred
`event_date`	DATE	Partition column for efficient queries
`workspace_id`	LONG	Workspace where event happened
`service_name`	STRING	Service category (e.g., `unityCatalog`, `clusters`, `secrets`)
`action_name`	STRING	Specific action (e.g., `getTable`, `createTable`, `grant`)
`user_identity`	STRUCT	Contains `email` and other identity fields
`request_params`	MAP	Parameters of the request (table name, etc.)
`response`	STRUCT	Contains `status_code` and `error_message`
`audit_level`	STRING	`ACCOUNT_LEVEL` or `WORKSPACE_LEVEL`
`account_id`	STRING	Databricks account ID
`source_ip_address`	STRING	IP address of the requester

Audit Event Categories

Category	Service Name	Example Actions
Workspace events	`workspace`, `clusters`, `jobs`	Login, cluster create, job run
Account events	`accounts`, `iam`	User provisioning, group management
Unity Catalog events	`unityCatalog`	getTable, createTable, grant, revoke
Secret events	`secrets`	getSecret, createScope
SQL Warehouse events	`databrickssql`	Query execution, warehouse start/stop

Common Audit Queries

Who Accessed PII Tables?

```
-- Identify all users who accessed tables tagged or known to contain PII
SELECT
    event_date,
    user_identity.email AS user_email,
    action_name,
    request_params.full_name_arg AS table_name,
    source_ip_address,
    response.status_code
FROM system.access.audit
WHERE action_name IN ('getTable', 'selectFromTable', 'commandSubmit')
    AND request_params.full_name_arg IN (
        'prod.sensitive.customers_pii',
        'prod.sensitive.employee_records',
        'prod.sensitive.payment_info'
    )
    AND event_date >= current_date() - 30
ORDER BY event_time DESC;
```

Which Users Created or Deleted Tables?

```
-- Track table creation and deletion events
SELECT
    event_time,
    user_identity.email AS user_email,
```

```
    action_name,
    request_params.full_name_arg AS table_name,
    request_params.table_type AS table_type,
    response.status_code
FROM system.access.audit
WHERE action_name IN ('createTable', 'deleteTable', 'dropTable')
    AND event_date >= current_date() - 90
ORDER BY event_time DESC;
```

Failed Access Attempts

```
-- Detect potential unauthorized access attempts
SELECT
    event_date,
    user_identity.email AS user_email,
    action_name,
    request_params.full_name_arg AS target_object,
    response.status_code,
    response.error_message,
    source_ip_address,
    COUNT(*) AS failure_count
FROM system.access.audit
WHERE response.status_code >= 400
    AND service_name = 'unityCatalog'
    AND event_date >= current_date() - 7
GROUP BY ALL
HAVING failure_count >= 3
ORDER BY failure_count DESC;
```

Permission Changes Over Time

```
-- Audit all GRANT and REVOKE operations
SELECT
    event_time,
    user_identity.email AS changed_by,
    action_name,
    request_params.securable_type AS object_type,
    request_params.full_name_arg AS object_name,
    request_params.principal AS grantee,
    request_params.privilege AS privilege,
    response.status_code
FROM system.access.audit
WHERE action_name IN ('grant', 'revoke', 'updatePermissions')
    AND event_date >= current_date() - 30
ORDER BY event_time DESC;
```

Audit Log Retention and Archival

Aspect	Details
Default retention	365 days in `system.access.audit`
Storage	Stored in system tables, managed by Databricks
Archival strategy	Copy to your own Delta table for longer retention
Partitioning	By `event_date` for efficient queries

```sql
-- Archive audit logs to your own table for extended retention
CREATE TABLE IF NOT EXISTS prod.compliance.audit_archive
USING DELTA
PARTITIONED BY (event_date)
AS SELECT * FROM system.access.audit WHERE 1=0;

-- Daily archival job
INSERT INTO prod.compliance.audit_archive
SELECT *
FROM system.access.audit
WHERE event_date = current_date() - 1
    AND event_date NOT IN (
        SELECT DISTINCT event_date FROM prod.compliance.audit_archive
    );
```

Creating Alerts on Audit Events

```sql
-- Create a view for suspicious activity (use with Databricks SQL Alerts)
CREATE OR REPLACE VIEW prod.compliance.suspicious_activity AS
SELECT
    event_time,
    user_identity.email AS user_email,
    action_name,
    request_params.full_name_arg AS object_name,
    source_ip_address,
    response.error_message
FROM system.access.audit
WHERE event_date >= current_date() - 1
    AND (
        -- Multiple failed access attempts
        (response.status_code >= 400 AND action_name LIKE '%Table%')
        -- Permission escalation
        OR action_name IN ('grant', 'updatePermissions')
        -- Sensitive table access outside business hours
        OR (
            request_params.full_name_arg LIKE '%pii%'
            AND (HOUR(event_time) < 6 OR HOUR(event_time) > 22)
        )
    );

-- Set up a Databricks SQL Alert on this view
-- Alert condition: COUNT(*) > 0
-- Notification: email or Slack webhook
```

Information Schema

The `system.information_schema` provides metadata about all Unity Catalog objects. It follows the SQL standard and enables compliance reporting and programmatic governance.

Key Information Schema Tables

Table	Description
`catalogs`	All catalogs visible to the current user
`schemata`	All schemas (databases)
`tables`	All tables and views
`columns`	All columns with data types
`table_privileges`	Grants on tables
`schema_privileges`	Grants on schemas
`catalog_privileges`	Grants on catalogs
`views`	View definitions
`routines`	Functions and procedures

Querying Catalogs and Schemas

```
-- List all catalogs with metadata
SELECT
    catalog_name,
    catalog_owner,
    comment,
    created,
    last_altered
FROM system.information_schema.catalogs
ORDER BY catalog_name;

-- List all schemas in a catalog
SELECT
    catalog_name,
    schema_name,
    schema_owner,
    comment,
    created
FROM system.information_schema.schemata
WHERE catalog_name = 'prod'
ORDER BY schema_name;
```

Querying Tables and Columns

```
-- Find all tables in the gold layer
SELECT
    table_catalog,
    table_schema,
    table_name,
    table_type,
    table_owner,
    comment,
    created,
    last_altered
FROM system.information_schema.tables
WHERE table_schema = 'gold'
    AND table_catalog = 'prod'
ORDER BY table_name;

-- Find all columns with their data types
SELECT
    table_catalog,
```

```
    table_schema,
    table_name,
    column_name,
    data_type,
    is_nullable,
    ordinal_position,
    comment
FROM system.information_schema.columns
WHERE table_catalog = 'prod'
    AND table_schema = 'gold'
    AND table_name = 'customers'
ORDER BY ordinal_position;
```

Querying Privileges for Compliance

```
-- Report: Who has access to what tables?
SELECT
    grantor,
    grantee,
    table_catalog,
    table_schema,
    table_name,
    privilege_type,
    is_grantable
FROM system.information_schema.table_privileges
WHERE table_catalog = 'prod'
ORDER BY grantee, table_name;

-- Report: All schema-level privileges
SELECT
    grantor,
    grantee,
    catalog_name,
    schema_name,
    privilege_type
FROM system.information_schema.schema_privileges
WHERE catalog_name = 'prod'
ORDER BY grantee, schema_name;

-- Find tables with no explicit grants (potential orphans)
SELECT
    t.table_catalog,
    t.table_schema,
    t.table_name,
    t.table_owner
FROM system.information_schema.tables t
LEFT JOIN system.information_schema.table_privileges p
    ON t.table_catalog = p.table_catalog
    AND t.table_schema = p.table_schema
    AND t.table_name = p.table_name
WHERE p.grantee IS NULL
    AND t.table_catalog = 'prod';
```

Object Ownership Report

```sql
-- Report: All objects and their owners
SELECT
    table_catalog AS catalog,
    table_schema AS schema,
    table_name AS object_name,
    table_type AS object_type,
    table_owner AS owner,
    created,
    last_altered
FROM system.information_schema.tables
WHERE table_catalog = 'prod'
ORDER BY table_owner, table_schema, table_name;
```

DESCRIBE and SHOW Commands

```sql
-- DESCRIBE commands for governance
DESCRIBE CATALOG prod;
DESCRIBE SCHEMA prod.gold;
DESCRIBE TABLE EXTENDED prod.gold.customers;
DESCRIBE FUNCTION prod.functions.mask_email;

-- SHOW commands for discovery
SHOW CATALOGS;
SHOW SCHEMAS IN prod;
SHOW TABLES IN prod.gold;
SHOW VIEWS IN prod.gold;
SHOW FUNCTIONS IN prod.functions;
SHOW GRANTS ON TABLE prod.gold.customers;
SHOW GRANTS TO `data-analysts`;
```

Network Security

Network security controls how data flows between Databricks, cloud provider infrastructure, and external networks. These configurations are critical for meeting compliance requirements.

Network Architecture Overview

```mermaid
flowchart TB
    subgraph Internet["Public Internet"]
        Users[Users / BI Tools]
    end

    subgraph CloudVPC["Customer VPC / VNet"]
        subgraph DataPlane["Data Plane"]
            Clusters[Clusters]
            SQLWarehouse[SQL Warehouses]
        end
        Storage[(Cloud Storage)]
        PrivateEP[Private Endpoint]
    end

    subgraph Databricks["Databricks Control Plane"]
        WebApp[Web Application]
        API[REST API]
```

```
        ControlServices[Control Services]
    end

    Users --> |IP Access Lists| WebApp
    WebApp --> |Private Link| PrivateEP
    PrivateEP --> DataPlane
    DataPlane --> |Private Endpoint| Storage
    Clusters --> |No Public IP / SCC| ControlServices
```

Private Link / Private Endpoints Private Link ensures traffic between the control plane and data plane travels over the cloud provider's backbone network, never crossing the public internet.

Cloud	Service	Purpose
AWS	AWS PrivateLink	Control plane to data plane connectivity
Azure	Azure Private Link	Control plane to data plane, storage access
GCP	Private Service Connect	Control plane to data plane

```
Private Link Setup (High-Level):
1. Create VPC/VNet endpoint for Databricks control plane
2. Create VPC/VNet endpoint for storage (S3/ADLS/GCS)
3. Configure Databricks workspace to use private endpoints
4. Disable public network access (optional but recommended)
5. Validate connectivity from clusters to control plane
```

Secure Cluster Connectivity (SCC / No Public IP) SCC ensures that cluster nodes have no public IP addresses. All communication between clusters and the Databricks control plane is initiated outbound through a secure tunnel.

Aspect	With SCC	Without SCC
Public IP on nodes	No	Yes
Control plane communication	Outbound tunnel	Inbound SSH
Network exposure	Minimal	Higher
Recommended for production	Yes	No

```
Enabling SCC:
- AWS: Enable "No Public IP" in workspace deployment
- Azure: Enabled by default on new workspaces
- GCP: Enable secure cluster connectivity in workspace settings
```

IP Access Lists IP access lists restrict which IP addresses can access the Databricks workspace (UI and API).

```python
# Configure IP access lists via REST API

import requests

# Add allowed IP range

response = requests.post(
    f"{account_url}/api/2.0/ip-access-lists",
    headers={"Authorization": f"Bearer {token}"},
    json={
```

```
        "label": "Corporate VPN",
        "list_type": "ALLOW",
        "ip_addresses": [
            "10.0.0.0/8",
            "172.16.0.0/12",
            "203.0.113.0/24"
        ]
    }
)

# Block specific IPs

response = requests.post(
    f"{account_url}/api/2.0/ip-access-lists",
    headers={"Authorization": f"Bearer {token}"},
    json={
        "label": "Blocked IPs",
        "list_type": "BLOCK",
        "ip_addresses": ["198.51.100.0/24"]
    }
)
```

VPC/VNet Peering Patterns VPC (AWS) or VNet (Azure) peering connects the Databricks data plane VPC to your own network for accessing on-premises data sources or other cloud services.

```
VPC Peering Setup:
1. Create peering between Databricks VPC and your VPC
2. Update route tables in both VPCs
3. Configure security groups/NSGs to allow traffic
4. Validate connectivity (e.g., JDBC to on-prem database)
```

Comparison: Network Connectivity Options

Feature	Private Link	VPC Peering	Public Access
Traffic path	Cloud backbone	Cloud backbone	Internet
Control plane access	Private	Public (UI/API)	Public
Storage access	Private	Private	Public
Setup complexity	High	Medium	Low
Cost	Higher	Moderate	Lowest
Compliance (HIPAA/PCI)	Required	Acceptable	Not recommended
Data exfiltration risk	Lowest	Low	Highest

Network Architecture for Compliance

Requirement	HIPAA	PCI-DSS	SOC 2
Private Link	Required	Required	Recommended
SCC / No Public IP	Required	Required	Recommended
IP Access Lists	Required	Required	Required
Encryption at rest	Required	Required	Required
Encryption in transit	Required	Required	Required
Audit logging	Required	Required	Required
VPC isolation	Required	Required	Recommended

Use Cases

- **Forensic Investigation**: Using `system.access.audit` to determine exactly who deleted a critical table, transferred ownership, or changed permissions on a sensitive dataset.
- **Automated Compliance Reporting**: Querying the `system.information_schema.table_privileges` daily to ensure no unauthorized users have been granted `MODIFY` access to production tables.
- **Secure Enterprise Connectivity**: Implementing Secure Cluster Connectivity (SCC) and Private Link to ensure all data processing traffic remains on the cloud provider's private backbone, satisfying strict InfoSec requirements.

Common Issues & Errors

Missing Audit Logs for Long-Term Compliance **Scenario:** An auditor requests access logs from 2 years ago, but the system table doesn't have them because the default retention is only 365 days. **Fix:** Create a scheduled job that incrementally copies `system.access.audit` records to your own Delta table for long-term archival.

Lineage Blind Spots **Scenario:** Data lineage is not showing up in the Catalog Explorer for a specific set of tables. **Fix:** Tables stored in the legacy `hive_metastore` do not support automatic lineage capture. Migrate the tables to Unity Catalog to enable lineage.

Exam Tips

1. **Lineage scope** - captured automatically for Unity Catalog tables; `hive_metastore` tables generate no lineage
2. **Column-level lineage** - visible in Catalog Explorer and through the lineage REST API
3. **Impact analysis** - use downstream lineage to see which tables, views, notebooks, and dashboards a change affects
4. **Audit retention** - `system.access.audit` keeps 365 days by default; copy records to your own Delta table for longer retention
5. **Information schema** - results only include objects the caller can access
6. **SCC vs Private Link** - Secure Cluster Connectivity removes public IPs from cluster nodes; Private Link keeps traffic on the cloud provider's private network

Key Takeaways

- **Automatic lineage capture**: Unity Catalog records table-to-table, column-level, and notebook/job lineage for all queries run against UC-managed tables; tables in `hive_metastore` do NOT generate lineage
- **Column-level lineage**: tracked automatically when Spark queries create or overwrite UC tables; visible in Catalog Explorer under the Lineage tab and queryable via the REST API (`/api/2.1/unity-catalog/lineage/column-lineage`)
- **Impact analysis**: downstream lineage from the table lineage API shows the ”blast radius” before a schema change — which tables, views, notebooks, and dashboards depend on a given table or column
- **Audit table structure**: `system.access.audit` is partitioned by `event_date` for performance; key columns are `service_name`, `action_name`, `user_identity.email`, `request_params`, and `response.status_code`; default retention is 365 days
- **Audit long-term archival**: `system.access.audit` retention cannot be extended in-place; create a scheduled job to copy records to your own Delta table for multi-year compliance retention
- **Information schema visibility**: `system.information_schema` tables are permission-filtered — users only see objects they have `USE CATALOG`/`USE SCHEMA` access to; compliance team needs explicit grants to query the full object inventory

- **Secure Cluster Connectivity (SCC)**: removes public IP addresses from cluster nodes; all control-plane communication is initiated outbound from the cluster through a secure tunnel; recommended for all production workloads
- **Private Link vs SCC**: Private Link routes traffic between the Databricks control plane and cloud storage over the cloud provider's private backbone (not the public internet); SCC removes public IPs from cluster nodes — both address different network threat surfaces

Related Topics

- Unity Catalog - Governance foundation
- Access Control - Row/column security
- Secret Management - Audit of secret access
- Data Classification, Compliance & Permissions - GDPR, tagging, advanced permissions

Official Documentation

- View data lineage using Unity Catalog
- Audit log system table reference
- Information schema
- Secure Cluster Connectivity
- Private Link
- IP Access Lists

Section 7: Data Governance (5 % of Exam)

Organising and describing data in Unity Catalog: the object hierarchy and permission inheritance model, tags and comments for discoverability, and Volumes vs legacy DBFS.

Topics Overview

```
flowchart LR
    D[Data Governance]
    D --> T1["Unity Catalog"]
    D --> T2["Tags, Comments, and Discoverability"]
    D --> T3["DBFS and Cloud Storage"]
```

Section Contents

File	Topic	Pri
01-unity-catalog.md	UC hierarchy, securables, permission inheritance, ownership	Hig
02-tags-comments-and-discoverability.md	Tags and comments on securables; search and discovery	Hig
03-dbfs-and-uc-volumes.md	UC Volumes vs DBFS and mounts	Me

Exam Skill Coverage

Every skill bullet in Section 7 of the Oct 9, 2026 exam guide, with the files that cover it.

Exam guide skill
Demonstrate understanding of Unity Catalog tags and comments as mechanisms for adding metadata to secu
Demonstrate understanding of the Unity Catalog permission inheritance model as a mechanism for managing

Key Concepts to Master

Concept	Why it matters
Permission inheritance	A privilege granted on a catalog or schema applies to every current and future o
`USE CATALOG` / `USE SCHEMA`	Required on each parent before a child privilege such as SELECT can be used
Tags and comments	Metadata on securables that drives search, discovery and (with governed tags) p

Related Resources

- Unity Catalog cheat sheet
- Unity Catalog Basics (shared)

Unity Catalog

Unity Catalog is Databricks' unified governance solution for data and AI assets. It provides centralized access control, auditing, lineage, and data discovery across workspaces.

Overview

```
flowchart TB
    subgraph Account["Databricks Account"]
        Metastore[Unity Catalog Metastore]
    end

    subgraph Workspaces["Workspaces"]
        WS1[Workspace 1]
        WS2[Workspace 2]
        WS3[Workspace 3]
    end

    Metastore --> WS1
    Metastore --> WS2
    Metastore --> WS3

    subgraph Objects["Governed Objects"]
        Catalogs[Catalogs]
        Schemas[Schemas]
        Tables[Tables/Views]
        Volumes[Volumes]
        Functions[Functions]
        Models[ML Models]
    end

    Metastore --> Objects
```

Overview

- One metastore per region per account; objects are addressed as `catalog.schema.object`.
- Privileges are granted on securable objects and **inherit downward** to all current and future child objects; `USE CATALOG` and `USE SCHEMA` gate access to everything below them.
- Every object has one owner; ownership does not inherit, but owners (and `MANAGE` holders) can manage child objects.

What the Exam Tests

- Demonstrate understanding of the Unity Catalog permission inheritance model as a mechanism for managing access control across catalogs, schemas, and objects.
- Predict effective access from a set of grants (exam guide sample Question 8: `SELECT` on a catalog covers schemas and tables created later).
- Distinguish ownership, `MANAGE`, `BROWSE`, `ALL PRIVILEGES`, and usage privileges.

Unity Catalog Architecture

Hierarchy

```
erDiagram
    METASTORE ||--o{ CATALOG : contains
    CATALOG ||--o{ SCHEMA : contains
    SCHEMA ||--o{ TABLE : contains
    SCHEMA ||--o{ VIEW : contains
    SCHEMA ||--o{ VOLUME : contains
    SCHEMA ||--o{ FUNCTION : contains
```

```
    SCHEMA ||--o{ MODEL : contains
    TABLE ||--o{ COLUMN : has
```

Three-Level Namespace All objects are referenced using a three-part name:

```
catalog_name.schema_name.object_name
```

```
-- Examples
main.default.customers
prod.sales.orders
dev.analytics.daily_metrics
```

Level	Purpose	Examples
Catalog	Top-level container, often by environment or domain	`prod`, `dev`, `staging`, `sales`, `marketing`
Schema	Logical grouping (like database)	`raw`, `bronze`, `silver`, `gold`, `default`
Object	Tables, views, volumes, functions, models	`customers`, `orders`, `daily_revenue`

Metastore

The metastore is the top-level container for all Unity Catalog objects.

Metastore Characteristics

Aspect	Description
Scope	One per region per account
Storage	Managed storage location in cloud
Attachment	Attached to one or more workspaces
Admin	Metastore admin role has full control

Creating and Managing Metastore

```
-- Metastore is created via account console or API
-- Cannot be created via SQL

-- View metastore information
SELECT * FROM system.information_schema.metastores;

-- View current metastore
SELECT current_metastore();
```

Metastore Admin Capabilities

- Create and manage catalogs
- Assign catalog ownership
- Create storage credentials and external locations
- Manage metastore-level settings
- Assign workspace access to catalogs

Catalogs

Catalogs are the primary container for organizing data assets.

Creating Catalogs

```
-- Create catalog
CREATE CATALOG IF NOT EXISTS prod;

-- Create catalog with comment
CREATE CATALOG dev
COMMENT 'Development environment catalog';

-- Create catalog with managed location
CREATE CATALOG staging
MANAGED LOCATION 's3://my-bucket/staging/';

-- View catalogs
SHOW CATALOGS;

-- Describe catalog
DESCRIBE CATALOG prod;

-- Drop catalog (must be empty)
DROP CATALOG IF EXISTS temp_catalog;

-- Force drop catalog with contents
DROP CATALOG IF EXISTS temp_catalog CASCADE;
```

Catalog Best Practices

Pattern	Example	Use Case
By environment	`prod`, `dev`, `staging`	Environment isolation
By domain	`sales`, `marketing`, `finance`	Business domain separation
By team	`data_engineering`, `data_science`	Team ownership
Hybrid	`prod_sales`, `dev_marketing`	Combined approach

Schemas

Schemas (also called databases) group related tables and objects.

Creating Schemas

```
-- Set current catalog
USE CATALOG prod;

-- Create schema
CREATE SCHEMA IF NOT EXISTS sales;

-- Create schema with comment
CREATE SCHEMA bronze
COMMENT 'Raw data landing zone';

-- Create schema with managed location
CREATE SCHEMA gold
MANAGED LOCATION 's3://my-bucket/prod/gold/';

-- View schemas in current catalog
```

```
SHOW SCHEMAS;

-- View schemas in specific catalog
SHOW SCHEMAS IN prod;

-- Describe schema
DESCRIBE SCHEMA prod.sales;

-- Drop schema
DROP SCHEMA IF EXISTS temp_schema;
DROP SCHEMA IF EXISTS temp_schema CASCADE;
```

Default Schema Each catalog has a `default` schema created automatically:

```
-- These are equivalent
SELECT * FROM prod.default.customers;

USE CATALOG prod;
USE SCHEMA default;
SELECT * FROM customers;
```

Tables

Table Types

```
flowchart TB
    Tables[Tables] --> Managed[Managed Tables]
    Tables --> External[External Tables]

    Managed --> M1["Data stored in managed location"]
    Managed --> M2["UC manages lifecycle"]
    Managed --> M3["DROP deletes data"]

    External --> E1["Data in external location"]
    External --> E2["User manages storage"]
    External --> E3["DROP keeps data"]
```

Creating Tables

```
-- Managed table (default)
CREATE TABLE prod.sales.orders (
    order_id STRING,
    customer_id INT,
    order_date DATE,
    amount DECIMAL(18,2)
)
USING DELTA;

-- External table
CREATE TABLE prod.raw.external_data (
    id INT,
    data STRING
)
USING DELTA
LOCATION 's3://external-bucket/data/';
```

```sql
-- Create table from query
CREATE TABLE prod.gold.daily_revenue AS
SELECT
    order_date,
    SUM(amount) as total_revenue
FROM prod.silver.orders
GROUP BY order_date;
```

Table Properties

```sql
-- View table details
DESCRIBE TABLE EXTENDED prod.sales.orders;

-- View table properties
SHOW TBLPROPERTIES prod.sales.orders;

-- Set table properties
ALTER TABLE prod.sales.orders SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true'
);

-- Add table comment
COMMENT ON TABLE prod.sales.orders IS 'Sales orders fact table';
```

Managed vs External Tables

Aspect	Managed Table	External Table
Storage location	Unity Catalog managed	User-specified location
Data lifecycle	Managed by UC	User managed
DROP behavior	Deletes data	Keeps data
Creation	No LOCATION clause	LOCATION clause required
Best for	Most use cases	Existing data, shared storage

Views

Creating Views

```sql
-- Standard view
CREATE VIEW prod.gold.customer_summary AS
SELECT
    customer_id,
    COUNT(*) as order_count,
    SUM(amount) as total_spent
FROM prod.silver.orders
GROUP BY customer_id;

-- Replace existing view
CREATE OR REPLACE VIEW prod.gold.customer_summary AS
SELECT
    customer_id,
    COUNT(*) as order_count,
    SUM(amount) as total_spent,
    MAX(order_date) as last_order
FROM prod.silver.orders
```

```sql
GROUP BY customer_id;

-- Materialized views: CREATE MATERIALIZED VIEW on a Pro or Serverless SQL
 ↪  warehouse,
-- or inside a Lakeflow pipeline (refreshes run on serverless pipelines)
```

Dynamic Views for Security

```sql
-- Row-level security using current_user()
CREATE VIEW prod.sales.my_orders AS
SELECT *
FROM prod.sales.orders
WHERE sales_rep = current_user();

-- Column masking
CREATE VIEW prod.sales.orders_masked AS
SELECT
    order_id,
    customer_id,
    order_date,
    CASE
        WHEN is_account_group_member('finance') THEN amount
        ELSE NULL
    END as amount
FROM prod.sales.orders;
```

Volumes

Volumes provide governed access to non-tabular data (files).

Volume Types

Type	Storage	Use Case
Managed	UC managed location	Generated files, outputs
External	User cloud storage	Landing zones, existing files

Creating Volumes

```sql
-- Managed volume
CREATE VOLUME prod.raw.landing_zone;

-- External volume
CREATE EXTERNAL VOLUME prod.raw.external_files
LOCATION 's3://my-bucket/external/';

-- List volumes
SHOW VOLUMES IN prod.raw;

-- Describe volume
DESCRIBE VOLUME prod.raw.landing_zone;
```

Using Volumes

```python
# Volume paths

volume_path = "/Volumes/prod/raw/landing_zone/"

# Read from volume

df = spark.read.format("csv").load(f"{volume_path}data.csv")

# Write to volume

df.write.format("parquet").save(f"{volume_path}output/")

# File operations

dbutils.fs.ls(volume_path)
dbutils.fs.cp(f"{volume_path}file.csv", f"{volume_path}archive/file.csv")
```

Storage Credentials and External Locations

Storage Credentials Storage credentials define how to authenticate to cloud storage (for example an IAM role for S3). They are created in Catalog Explorer (**Catalog** > **Connect** > **Credentials**), the Databricks CLI, or Terraform — there is no `CREATE STORAGE CREDENTIAL` SQL statement (SQL can list, describe, alter, and grant on them). Creating one requires the `CREATE STORAGE CREDENTIAL` privilege on the metastore (metastore admins have it by default).

```sql
-- Inspect storage credentials with SQL
SHOW STORAGE CREDENTIALS;
DESCRIBE STORAGE CREDENTIAL my_s3_credential;
```

External Locations External locations map cloud storage paths to Unity Catalog.

```sql
-- Create external location
CREATE EXTERNAL LOCATION my_landing_zone
URL 's3://my-bucket/landing/'
WITH (STORAGE CREDENTIAL my_s3_credential);

-- Grant access to external location
GRANT READ FILES, WRITE FILES ON EXTERNAL LOCATION my_landing_zone TO
 ↪  `data-engineers`;

-- List external locations
SHOW EXTERNAL LOCATIONS;
```

Permission Model

Unity Catalog access control works by granting **privileges** on **securable objects**. Catalogs and schemas are **container objects**: a privilege granted on a container applies to its children through inheritance.

Privilege Inheritance

```
flowchart TD
    MS["Metastore<br/>grants do NOT inherit to children<br/>(CREATE CATALOG,
 ↪  CREATE EXTERNAL LOCATION ...)"]
    CAT["Catalog<br/>USE CATALOG, BROWSE, USE SCHEMA, SELECT,<br/>MODIFY, CREATE
 ↪  SCHEMA, CREATE TABLE, MANAGE ..."]
```

```
    SCH["Schema<br/>USE SCHEMA, SELECT, MODIFY,<br/>CREATE TABLE, EXECUTE, MANAGE
↪   ..."]
    OBJ["Tables, views, volumes, functions, models<br/>SELECT, MODIFY, READ
↪   VOLUME, EXECUTE ..."]
    MS --> CAT
    CAT -->|"inherits to ALL current and future<br/>schemas and objects"| SCH
    SCH -->|"inherits to ALL current and future<br/>objects in the schema"| OBJ
```

Rule	What it means
Privileges inherit downward	A privilege granted on a catalog applies to all schemas in it and all tabl
Usage privileges gate access	To read a table you need `USE CATALOG` on the catalog, `USE SCHEMA` on t
Usage privileges are a boundary	Only catalog/schema owners or `MANAGE` holders can grant `USE CATALOG`
Metastore grants don't inherit	Metastore-level privileges control metastore-scoped operations (`CREATE`
Denials inherit too	ABAC DENY policies (Beta, currently only `MANAGE ACCESS CONTROL`) app
ABAC follows the same rules	Inheritance applies whether a privilege is granted manually or through

Note

Metastores created during the Unity Catalog public preview (before August 25, 2022) may still run an earlier privilege model without inheritance; they can be upgraded to Privilege Model version 1.0.

Inheritance in Practice (sample Question 8)

```
-- One statement: access the catalog, every schema, and every current AND future
↪  table
GRANT USE CATALOG, USE SCHEMA, SELECT ON CATALOG sales TO `finance_team`;

-- Engineers can create tables in any current or future schema of the catalog
GRANT USE CATALOG, USE SCHEMA, CREATE TABLE ON CATALOG analytics TO
↪  `data_engineers`;

-- Narrower: read every current and future table in one schema only
GRANT USE CATALOG ON CATALOG prod TO `analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA prod.gold TO `analysts`;
```

If `SELECT` is granted on a catalog and new schemas and tables are created later, the grantee can read the new tables (given the usage privileges) without any new grant. There is no need to re-grant per table, and no "future grants" syntax is required.

Common Privileges

Privilege	Lowest securable it applies to	Allows
`USE CATALOG`	Catalog	Prerequisite to interact w
`USE SCHEMA`	Schema	Prerequisite to interact w
`BROWSE`	Catalog (granted at catalog level)	See objects and their me
`SELECT`	Table, view	Read data
`MODIFY`	Table	Insert, update, and delet
`CREATE SCHEMA`	Catalog	Create schemas
`CREATE TABLE`	Schema	Create tables and views
`CREATE VOLUME` / `READ VOLUME` / `WRITE VOLUME`	Schema / volume	Create volumes; read or
`EXECUTE`	Function, model	Call a function or use a r

Privilege	Lowest securable it applies to	Allows
APPLY TAG	Object	Add and edit tags on the
READ METADATA	Any	Read-only visibility into r
MANAGE	Any	Manage privileges, trans
ALL PRIVILEGES	Any	All applicable privileges,

Any privilege in the table can be granted on a parent container (for example SELECT on a schema or catalog) to cover the children by inheritance.

Granting, Revoking, and Viewing Permissions

```
GRANT SELECT ON TABLE prod.gold.revenue TO `data-analysts`;
GRANT SELECT, MODIFY ON TABLE prod.silver.orders TO `data-engineers`;
GRANT ALL PRIVILEGES ON SCHEMA prod.silver TO `data-engineers`;   -- does not
 ↪  include MANAGE
GRANT MANAGE ON SCHEMA prod.gold TO `governance-admins`;          -- delegate
 ↪  grant management
GRANT BROWSE ON CATALOG prod TO `account users`;                  --
 ↪  discoverability without data access

REVOKE SELECT ON TABLE prod.gold.revenue FROM `data-analysts`;
REVOKE ALL PRIVILEGES ON SCHEMA prod.silver FROM `data-engineers`;

-- SHOW GRANTS [principal] ON securable_object (inherited, denied, and granted
 ↪  privileges)
SHOW GRANTS ON TABLE prod.gold.revenue;
SHOW GRANTS `data-analysts` ON SCHEMA prod.gold;
```

Warning

Unity Catalog GRANT has no WITH GRANT OPTION. To let someone else administer grants on an object, grant MANAGE or transfer ownership. SHOW GRANTS always needs an ON <securable> clause.

Ownership

Every securable object has exactly one owner (a user, service principal, or group); the principal that creates the object is the initial owner.

```
DESCRIBE TABLE EXTENDED prod.sales.orders;                    -- shows the owner
ALTER TABLE prod.sales.orders SET OWNER TO `data-engineering-team`;
ALTER SCHEMA prod.sales SET OWNER TO `sales-team`;
ALTER CATALOG prod SET OWNER TO `platform-team`;
```

Capability	Owner
All capabilities on the object	Yes, implied (not shown as ALL PRIVILEGES in SHOW G
Grant and revoke privileges, transfer ownership, drop	Yes
Manage child objects	Yes, implied for all current and future children
Usage privileges needed	Only on the containers above the object
Number of principals	Exactly one

- **Ownership does not inherit.** Owning a catalog does not make you the owner of its schemas or tables, but you can manage all of them.

- Data access still needs usage privileges: `MANAGE` reduces usage requirements only for metadata and management actions, not for `SELECT` or `MODIFY`.
- Best practice: assign ownership of production objects to **groups** (the creator is the first owner and should reassign it), so administration does not depend on one person.

Groups and Users

Identity Management

```
-- Create group (account admin)
-- Done via account console or SCIM

-- Add user to group (account admin)
-- Done via account console or SCIM

-- Use groups in grants
GRANT SELECT ON TABLE prod.gold.revenue TO `data-analysts`;

-- Check group membership
SELECT is_account_group_member('data-analysts');

-- Get current user
SELECT current_user();
```

Group-Based Access Pattern

```
-- Create role-based access
GRANT USE CATALOG ON CATALOG prod TO `analysts`;
GRANT USE SCHEMA ON SCHEMA prod.gold TO `analysts`;
GRANT SELECT ON SCHEMA prod.gold TO `analysts`;

GRANT USE CATALOG ON CATALOG prod TO `engineers`;
GRANT USE SCHEMA ON SCHEMA prod.silver TO `engineers`;
GRANT SELECT, MODIFY ON SCHEMA prod.silver TO `engineers`;
GRANT CREATE TABLE ON SCHEMA prod.silver TO `engineers`;
```

Information Schema

Unity Catalog provides system tables for metadata queries.

```
-- List all catalogs
SELECT * FROM system.information_schema.catalogs;

-- List all schemas
SELECT * FROM system.information_schema.schemata
WHERE catalog_name = 'prod';

-- List all tables
SELECT * FROM system.information_schema.tables
WHERE table_schema = 'gold';

-- List all columns
SELECT * FROM system.information_schema.columns
WHERE table_name = 'orders';

-- List all privileges
```

```sql
SELECT * FROM system.information_schema.table_privileges
WHERE table_name = 'orders';
```

Lineage

Unity Catalog automatically tracks data lineage.

Lineage is captured for queries run on Databricks at table and column level and is visible in Catalog Explorer, through the lineage API, and in the lineage system tables (rolling 1-year window):

```sql
SELECT source_table_full_name, target_table_full_name, entity_type, event_time
FROM system.access.table_lineage
WHERE target_table_full_name = 'prod.gold.daily_revenue'
ORDER BY event_time DESC;
-- Column-level lineage: system.access.column_lineage
```

Migration from Hive Metastore

Comparison

Aspect	Hive Metastore	Unity Catalog
Scope	Workspace-level	Account-level
Governance	Limited	Comprehensive
Lineage	None	Automatic
Access control	Basic ACLs	Fine-grained
Data sharing	Not supported	OpenSharing

Migration Approaches

```sql
-- External (or externally stored managed) Hive tables -> UC external tables,
 ↪  metadata only
SYNC TABLE prod.bronze.my_table FROM hive_metastore.default.my_table DRY RUN;
SYNC TABLE prod.bronze.my_table FROM hive_metastore.default.my_table;
SYNC SCHEMA prod.bronze FROM hive_metastore.default SET OWNER
 ↪  `data-engineering-team`;

-- Hive managed tables stored in workspace storage (DBFS root) -> copy into a UC
 ↪  managed table
CREATE TABLE prod.bronze.my_managed_table DEEP CLONE
 ↪  hive_metastore.default.my_managed_table;
```

SYNC records the target in an upgraded_to table property on the source table for bookkeeping; it cannot upgrade Hive managed tables stored in DBFS root.

Use Cases

Multi-Environment Setup

```sql
-- Create environment catalogs
CREATE CATALOG dev;
CREATE CATALOG staging;
CREATE CATALOG prod;

-- Same schema structure across environments
CREATE SCHEMA dev.sales;
CREATE SCHEMA staging.sales;
```

```
CREATE SCHEMA prod.sales;

-- Environment-specific permissions
GRANT ALL PRIVILEGES ON CATALOG dev TO `developers`;
GRANT USE CATALOG, USE SCHEMA, SELECT ON CATALOG staging TO `developers`;  --
 ↪  read all, incl. future tables
GRANT USE CATALOG ON CATALOG prod TO `analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA prod.gold TO `analysts`;
```

Data Mesh Pattern

```
-- Domain-based catalogs
CREATE CATALOG sales_domain;
CREATE CATALOG marketing_domain;
CREATE CATALOG finance_domain;

-- Each domain manages their own data
GRANT ALL PRIVILEGES ON CATALOG sales_domain TO `sales-data-team`;
GRANT ALL PRIVILEGES ON CATALOG marketing_domain TO `marketing-data-team`;

-- Cross-domain read access
GRANT USE CATALOG, USE SCHEMA, SELECT ON CATALOG sales_domain TO
 ↪  `marketing-data-team`;
```

Common Issues & Errors

Permission Denied **Scenario:** User cannot access table.

Fix: Grant required privileges:

```
-- Check what's missing (SHOW GRANTS needs an ON clause)
SHOW GRANTS `user@company.com` ON TABLE prod.gold.revenue;
SHOW GRANTS `user@company.com` ON SCHEMA prod.gold;

-- Grant catalog access
GRANT USE CATALOG ON CATALOG prod TO `user@company.com`;

-- Grant schema access
GRANT USE SCHEMA ON SCHEMA prod.gold TO `user@company.com`;

-- Grant table access
GRANT SELECT ON TABLE prod.gold.revenue TO `user@company.com`;
```

Cannot Create External Table **Scenario:** External location not configured.

Fix: Create external location first:

```
-- Create storage credential and external location
-- Then create table
CREATE TABLE prod.raw.external_data
USING DELTA
LOCATION 's3://bucket/path/';
```

Catalog Not Visible **Scenario:** User cannot see catalog.

Fix: Grant USE CATALOG:

```
GRANT USE CATALOG ON CATALOG prod TO `data-team`;
```

Exam Tips

1. **Three-level namespace** - Always `catalog.schema.object`
2. **Managed vs External** - Managed tables have UC-managed storage and lifecycle
3. **Permission inheritance** - Privileges granted on a catalog or schema apply to all current AND future children; still need USE CATALOG and USE SCHEMA
4. **Metastore scope** - One per region per account
5. **Ownership** - One owner per object; ownership does not inherit, but owners can manage all child objects
6. **Storage credentials** - Required for external locations
7. **Volumes** - For non-tabular file access with governance
8. **Groups** - Use groups for scalable access management
9. **Information schema** - Query metadata via `system.information_schema`
10. **Lineage** - Automatic, visible in Catalog Explorer and `system.access.table_lineage`
11. **No `WITH GRANT OPTION`** - Delegate with `MANAGE`; `ALL PRIVILEGES` excludes `MANAGE`

Key Takeaways

- **Three-level namespace**: every Unity Catalog object is referenced as `catalog.schema.object`; the metastore is one level above the catalog and is scoped to one per cloud region per account
- **Managed vs external tables**: managed tables store data in the UC-controlled location and `DROP TABLE` deletes the data; external tables store data at a user-specified `LOCATION` and `DROP TABLE` keeps the data
- **Permission hierarchy**: to access a table, a user must have `USE CATALOG` on the catalog, `USE SCHEMA` on the schema, and then the relevant privilege (e.g., `SELECT`) on the table — missing any level causes permission denied
- **Volume types**: managed volumes store files in the UC-managed location; external volumes point to a user-controlled cloud path; both are accessed via `/Volumes/<catalog>/<schema>/<volume>/` paths
- **Storage credentials and external locations**: a storage credential holds the cloud authentication (IAM role / managed identity); an external location maps a cloud storage URL to a credential; external tables and volumes require a matching external location
- **Dynamic views for row/column security**: `current_user()` and `is_account_group_member()` in view `WHERE` / `CASE` clauses enforce user-specific data access without modifying the underlying table
- **Privilege inheritance**: a grant on a catalog or schema applies to all current and future child objects, so `SELECT` on a catalog covers schemas and tables created later (with `USE CATALOG` / `USE SCHEMA`); metastore-level grants do not inherit
- **Ownership model**: every UC object has exactly one owner with full control (grant, alter, drop, transfer); ownership does not inherit downward but owners can manage all child objects; `MANAGE` delegates grant management without ownership; `BROWSE` exposes metadata without data access
- **Information schema**: `system.information_schema.tables`, `.columns`, `.table_privileges`, `.schemata` provide SQL-standard metadata and are permission-filtered (users only see what they have access to)

Related Topics

- Access Control - Row/column level security
- Data Sharing - OpenSharing
- Secret Management - Secure credentials
- Tags, Comments, and Discoverability - Metadata for discovery
- ABAC Policies and Governed Tags - Tag-driven row filters and column masks

Official Documentation

- Unity Catalog permissions model concepts
- Unity Catalog privileges reference
- Unity Catalog best practices
- GRANT
- SHOW GRANTS
- Manage storage credentials
- SYNC
- Data lineage
- CREATE MATERIALIZED VIEW

Tags, Comments, and Discoverability

Overview

Unity Catalog lets you attach two kinds of descriptive metadata to securable objects: **comments** (free-text descriptions) and **tags** (key/optional-value attributes). Both feed workspace search, Catalog Explorer, and the Discover page, and tags additionally drive classification, certification, domains, and ABAC policies.

Overview

- **Comments** describe catalogs, schemas, tables, views, columns, and volumes; they are set with `COMMENT ON`, `COMMENT` clauses in `CREATE` / `ALTER`, or AI-generated suggestions in Catalog Explorer.
- **Tags** are key/optional-value pairs set with `ALTER ... SET TAGS` or `SET TAG ON`; **governed tags** add an account-level allowed-value list and `ASSIGN` permission; **system tags** are predefined by Databricks.
- Metadata is queryable through `INFORMATION_SCHEMA` (`*_TAGS` views, `COMMENT` columns) and searchable from the workspace search bar.
- Discovery surfaces: workspace search, Catalog Explorer, and the curated **Discover** page organized by **domains** (built on governed tags).

What the Exam Tests

- Demonstrate understanding of Unity Catalog tags and comments as mechanisms for adding metadata to securable objects to improve data discoverability.
- Pick the right mechanism: comment (human description) vs ungoverned tag (flexible label) vs governed tag (controlled vocabulary, policy-ready) vs certification system tag (trust signal).
- Know the SQL to set and query tags and comments, and the privileges required (`APPLY TAG`, `MODIFY`, ownership, `ASSIGN`, `BROWSE`).

Metadata Mechanisms Compared

Mechanism	What it is
Comment	Free-text description of an object or column
Ungoverned tag	Key with optional value, any user with `APPLY TAG` can set any value
Governed tag	Account-level tag key with a tag policy (allowed values, who may assign)
System governed tag	Governed tag predefined by Databricks (`system.certification_status`, `class.*`,

Mechanism	What it is
Domain	Business-aligned grouping on the Discover page, built on a governed tag

```
flowchart LR
    C[Comments] --> S[Workspace search<br/>names, comments, columns, tag keys]
    T[Tags and governed tags] --> S
    T --> D[Discover page<br/>domains = governed tags]
    T --> A[ABAC policies<br/>has_tag / has_tag_value]
    CERT[system.certification_status] --> S
    CERT --> D
    C --> CE[Catalog Explorer]
    T --> CE
    IS[INFORMATION_SCHEMA<br/>*_TAGS, COMMENT columns] -.-> T
    IS -.-> C
```

Comments

Setting comments

```
-- At creation time
CREATE CATALOG sales COMMENT 'Sales domain data, owned by revenue engineering';
CREATE SCHEMA sales.gold COMMENT 'Curated, business-ready sales tables';
CREATE TABLE sales.gold.orders (
  order_id BIGINT COMMENT 'Surrogate key from the order system',
  amount   DECIMAL(18, 2) COMMENT 'Order total in USD'
) COMMENT 'One row per completed order';

-- Afterwards with COMMENT ON
COMMENT ON TABLE sales.gold.orders IS 'One row per completed order, refreshed
 ↪  hourly';
COMMENT ON COLUMN sales.gold.orders.amount IS 'Order total in USD, tax included';
 ↪  -- DBR 16.1+
COMMENT ON SCHEMA sales.gold IS 'Curated sales layer';
COMMENT ON VOLUME sales.raw.landing IS 'Raw CSV drops from the POS vendor';
COMMENT ON TABLE sales.gold.orders IS NULL;   -- NULL removes the comment

-- Or with ALTER TABLE
ALTER TABLE sales.gold.orders ALTER COLUMN amount COMMENT 'Order total in USD';
```

`COMMENT ON` supports catalogs, schemas, tables (including streaming tables, views, and materialized views), columns, volumes, connections, shares, recipients, and providers.

Privileges to set comments

Object	Required
Catalog, schema, volume	Owner or `MANAGE` on the object
Table or column of a table	`MODIFY` on the table
View or column of a view	Owner of the view
Materialized view	`COMMENT ON` requires Databricks Runtime 16.3+

AI-generated comments In Catalog Explorer, **AI generate** suggests a description for catalogs, schemas, tables, views, materialized views, table columns, functions, models, and volumes, based

on metadata such as the table schema and column names. You review it and **Accept** or **Edit** before saving.

Fact	Detail
Who can add them	Object owner or a user with `MODIFY` (views and materialized views: owner only)
Toggle	No workspace- or catalog-level switch; generated per object on request
Side effect	Saving runs an `ALTER` statement, which can disrupt running pipelines and jobs
Review	Human review is strongly recommended; the model should **not** be relied on to detect PII
Viewing	`BROWSE` on the catalog is enough to view comments without `USE CATALOG` / `USE SCHEMA`

Tags

Tags are attributes with a key and an optional value. Unity Catalog supports tags on catalogs, schemas, tables, table columns, volumes, views, functions, registered models, model versions, secrets, external metadata objects, and services; dashboards, Genie Agents, apps, and notebooks can be tagged too.

Setting tags

```
-- Databricks Runtime 13.3+: ALTER <object> ... SET TAGS / UNSET TAGS
ALTER CATALOG sales SET TAGS ('domain' = 'revenue');
ALTER SCHEMA sales.gold SET TAGS ('layer' = 'gold', 'owner_team' = 'revenue_eng');
ALTER TABLE sales.gold.orders SET TAGS ('refresh' = 'hourly');
ALTER TABLE sales.gold.orders ALTER COLUMN customer_email SET TAGS ('pii' =
 ↪  'email');
ALTER TABLE sales.gold.orders UNSET TAGS ('refresh');

-- Databricks Runtime 16.1+: SET TAG / UNSET TAG (one tag per statement)
SET TAG ON TABLE sales.gold.orders cost_center = revenue;
SET TAG ON COLUMN sales.gold.orders.customer_email pii = email;
UNSET TAG ON COLUMN sales.gold.orders.customer_email pii;
```

Requirements and constraints

- Own the object, or have `APPLY TAG` on the object plus `USE SCHEMA` on its schema and `USE CATALOG` on its catalog.
- For a **governed** tag you also need `ASSIGN` on that governed tag, and the value must be in its allowed list.
- Up to 50 tags per securable object and 1,000 column tags per table; keys and values up to 256 characters; keys are case-sensitive (`Sales` and `sales` are different tags), some special characters are not allowed, and leading or trailing spaces are rejected.
- One column per `ALTER TABLE ... SET TAGS` statement (unlike `COMMENT`, which can change several columns at once).
- A column that carries a governed tag cannot be dropped until the tag is removed.

Warning

Tag data is stored as plain text and may be replicated globally. Never put personal or sensitive information in tag names or values.

Inheritance Tags applied to a catalog or schema are implicitly inherited by objects beneath it **only when ABAC policies are evaluated**; columns never inherit table tags. Outside ABAC evaluation, tags are not copied down the hierarchy. The separate **tag propagation** feature (Beta, Oct 12, 2026) copies governed tags along **lineage** to derived tables and views.

Governed vs ungoverned tags

Aspect	Ungoverned tag	Governed tag
Defined	Ad hoc on any object	At the account level with a tag policy
Values	Any value	Only values from the allowed list (max 500 p
Who can assign	Anyone with `APPLY TAG` on the object	Requires `ASSIGN` on the governed tag **and** A
Usable in ABAC policies	No	Yes
Basis for domains	No	Yes
UI marker	Listed under **Other**	Lock icon, listed under **Governed**

Creating a governed tag with an existing key turns all existing assignments of that key into governed assignments; deleting a governed tag leaves the assignments in place but ungoverned.

Certification and deprecation The system governed tag `system.certification_status` has two values: `certified` (check mark in the workspace) and `deprecated` (restricted icon). It applies to catalogs, schemas, tables, views, volumes, functions, registered models, dashboards, Genie Agents, apps, and notebooks, and requires `ASSIGN` on the tag plus the usual tagging privileges.

```
SET TAG ON TABLE sales.gold.orders `system.certification_status` = `certified`;
SET TAG ON TABLE sales.gold.orders_v1 `system.certification_status` =
 ↪  `deprecated`;
```

Certification can also be automated at scale with automated tag assignment rules (Beta), for example deprecating tables with no reads or writes in 30 days.

Querying Metadata with SQL

Each catalog's `INFORMATION_SCHEMA` exposes tags and comments; rows are filtered to objects the user has permission to interact with.

View	Contains
`INFORMATION_SCHEMA.CATALOG_TAGS`	`catalog_name`, `tag_name`, `tag_value`
`INFORMATION_SCHEMA.SCHEMA_TAGS`	Tags on schemas
`INFORMATION_SCHEMA.TABLE_TAGS`	Tags on tables, views, and materialized views (`catalog_name`, `sche`
`INFORMATION_SCHEMA.COLUMN_TAGS`	Column tags, adds `column_name`
`INFORMATION_SCHEMA.VOLUME_TAGS`	Tags on volumes
`INFORMATION_SCHEMA.TABLES` / `COLUMNS`	`COMMENT` column holds the table / column comment

```
-- Every column tagged pii in the sales catalog
SELECT schema_name, table_name, column_name, tag_value
FROM sales.information_schema.column_tags
WHERE tag_name = 'pii';

-- Metastore-wide: certified tables (system catalog sees all catalogs you can
 ↪  access)
SELECT catalog_name, schema_name, table_name
FROM system.information_schema.table_tags
WHERE tag_name = 'system.certification_status' AND tag_value = 'certified';

-- Tables with no description (documentation gaps)
SELECT table_catalog, table_schema, table_name
FROM system.information_schema.tables
WHERE comment IS NULL AND table_schema <> 'information_schema';
```

```
-- Comments and tags for one table
DESCRIBE TABLE EXTENDED sales.gold.orders;
```

Note

The `*_TAGS` views use `catalog_name` / `schema_name` columns, while `INFORMATION_SCHEMA.TABLES` and `COLUMNS` use the SQL-standard `table_catalog` / `table_schema`.

Discoverability Surfaces

Workspace search

- Searches table, view, and model **names and comments**, **column names and comments**, and **tag keys** for Unity Catalog objects (tables must be registered in Unity Catalog; `hive_metastore` tables are not returned).
- Tag syntax: `tag:<tag_key>` or `tag:<tag_key>:<tag_value>` (exact term match). Tag search covers tables, views, models, volumes, functions, dashboards, Genie Agents, and notebooks — not catalogs, schemas, or columns.
- Filters such as `type:table owner:me` and `type:table certificationStatus:certified`.
- AI-generated comments make search aware of company jargon; popularity signals rank frequently used tables higher; semantic natural-language search over tables is in Public Preview.
- Results are permission-filtered: tag search requires at least `BROWSE` on the object or its parent catalog.

The BROWSE privilege `BROWSE` lets users discover objects and see their name, description, and tags, and request access, **without** `USE CATALOG`, `USE SCHEMA`, or any data access. For data objects it is granted at the catalog level and covers everything in the catalog. Databricks recommends granting `BROWSE` on catalogs to the `All account users` group.

```
GRANT BROWSE ON CATALOG sales TO `account users`;
```

Catalog Explorer Catalog Explorer shows everything in the catalog hierarchy: object details, comments, tags, permissions, sample data, and lineage. It is where tags, comments, and AI-generated descriptions are edited, and where certification badges appear next to object names.

Discover page and domains The **Discover** page is a curated browsing experience: consumers find assets organized by **domains** and **subdomains** or by asset type, see certified and AI-recommended assets, and search, without needing to know catalog hierarchies.

Concept	Detail
Domain	Account-level grouping of assets (tables, dashboards, notebooks, Genie Agents) **built on a gov**
Subdomain	One level of nesting, named `{parentDomainTag}/{subdomainName}` (for example `Finance/Tax`)
Curators	Need `MANAGE DISCOVER` at the account level or `MANAGE` on a domain/subdomain; account and wo
Consumers	Browse domains, asset types, certified assets, and Genie-recommended domains (published by
Lifecycle	New domains start as drafts visible only to curators until published

Platform update (Oct 2026)

The Discover page and domains became generally available on Oct 8, 2026. Source: October 2026 release notes

Use Cases

- **Self-service discovery:** comments on every gold table and column, `BROWSE` granted to all account users, and certified tags let analysts find the authoritative `orders` table through search without filing access tickets.
- **Data-mesh navigation:** governed `domain` tags power Discover page domains, so marketing users browse "Marketing" assets across many catalogs.
- **Documentation audits:** a scheduled query on `information_schema.tables` flags tables with `comment IS NULL` for owners to fix, using AI-generated comments as a starting point.
- **Deprecation:** tagging an old table `system.certification_status = deprecated` warns consumers in search and the SQL editor before the table is dropped.

Common Issues & Errors

Symptom	Cause
`SET TAGS` on a column fails with a permission error	Missing `APPLY TAG` (or `USE CATALOG` / `USE SCHEMA`), or m
Governed tag value rejected	Value not in the tag's allowed list
Tag query returns no rows	Querying the wrong view or columns (`table_catalog` ins
`COMMENT ON COLUMN` fails	Databricks Runtime below 16.1, or no `MODIFY` on the tabl
Saving an AI comment broke a streaming job	Saving issues an `ALTER` on the table
Table not found by tag search	User lacks `BROWSE`, the object type doesn't support tag s

Exam Tips

Tip

- Tags and comments are **metadata on securable objects**; they never grant or restrict access on their own (ABAC policies use governed tags to do that).
- `COMMENT ON <object> ... IS '...'` sets a comment; `IS NULL` removes it. Column comments: `COMMENT ON COLUMN` (DBR 16.1+) or `ALTER TABLE ... ALTER COLUMN ... COMMENT`.
- Tags: `ALTER <object> SET TAGS ('k' = 'v')` (DBR 13.3+) or `SET TAG ON <object> k = v` (DBR 16.1+). Query with `INFORMATION_SCHEMA.TABLE_TAGS` / `COLUMN_TAGS`.
- Tagging needs `APPLY TAG` on the object (or ownership); governed tags also need `ASSIGN`.
- `BROWSE` = see that data exists (names, comments, tags) without reading it — the discoverability privilege.
- "Controlled vocabulary that only stewards can apply" → **governed tag**. "Mark the trusted table" → `system.certification_status = certified`.

Key Takeaways

- **Comments** document objects and columns for humans and search; set them with `COMMENT ON`, `COMMENT` clauses, or AI-generated suggestions that require human review.
- **Tags** are key/optional-value attributes on catalogs, schemas, tables, columns, volumes, views, functions, models, and more; they make assets searchable and filterable.
- **Governed tags** add account-level allowed values and an `ASSIGN` permission, enabling consistent classification, domains, and ABAC; **system tags** such as `system.certification_status` and `class.*` are predefined.
- **INFORMATION_SCHEMA** exposes `CATALOG_TAGS`, `SCHEMA_TAGS`, `TABLE_TAGS`, `COLUMN_TAGS`, `VOLUME_TAGS`, and `COMMENT` columns, filtered by the caller's permissions.
- **Discovery** happens through workspace search (names, comments, columns, tags), Catalog Explorer, and the Discover page with domains (GA Oct 8, 2026); `BROWSE` makes metadata visible without data access.

Related Topics

- Unity Catalog — object hierarchy and permission inheritance
- ABAC Policies and Governed Tags — governed tags as policy attributes
- Data Classification, Compliance & Permissions — tag-based classification patterns
- Anonymization, PII Controls, and Data Purging — Data Classification `class.*` tags

Official Documentation

- Apply tags to Unity Catalog securable objects
- SET TAG
- COMMENT ON
- ALTER TABLE COLUMN clause
- Add AI-generated comments to Unity Catalog objects
- Governed tags
- Flag data as certified or deprecated
- TABLE_TAGS
- COLUMN_TAGS
- Information schema
- Search for workspace objects
- Data discovery in Unity Catalog
- Discover page
- Domains and subdomains
- Unity Catalog permissions model concepts (BROWSE)
- Tag propagation

DBFS and Cloud Storage

Understanding Databricks File System (DBFS) and cloud storage integration is essential for managing data in Databricks pipelines.

Overview

```
flowchart TB
    subgraph Storage["Storage Options"]
        DBFS[DBFS Root]
        Mounts[Mount Points]
        Volumes[Unity Catalog Volumes]
        Direct[Direct Cloud Access]
    end

    subgraph Cloud["Cloud Storage"]
        S3[AWS S3]
        ADLS[Azure Data Lake]
        GCS[Google Cloud Storage]
    end

    Storage --> Cloud
```

Overview

- **DBFS paths**: `dbfs:/...` with Spark APIs, `/dbfs/...` with Python file I/O
- **Mounts are legacy**; Unity Catalog volumes (`/Volumes/<catalog>/<schema>/<volume>/`)

are the governed replacement
- **Managed vs external volumes**: managed volume files are removed on drop; external volume files are not
- **Direct cloud access**: storage URI schemes and credential precedence, with Unity Catalog external locations first

What the Exam Tests

- Apply least-privilege access control lists (ACLs) to secure Unity Catalog securable objects and workspace resources.
- Pick the right path format for the API in use
- Replace mount-based access with Unity Catalog volumes

DBFS (Databricks File System)

DBFS is a distributed file system mounted into a Databricks workspace.

DBFS Architecture

```
flowchart LR
    User[User/Notebook] --> DBFS[DBFS Layer]
    DBFS --> Root[DBFS Root]
    DBFS --> Mounts[Mount Points]
    DBFS --> FileStore[FileStore]

    Root --> Cloud[Cloud Storage]
    Mounts --> Cloud
```

DBFS Path Formats

Format	Context	Example
`dbfs:/`	Spark APIs	`dbfs:/data/input.csv`
`/dbfs/`	Local file APIs	`/dbfs/data/input.csv`
`file:/`	Local driver filesystem	`file:/tmp/local.csv`

```python
# Spark APIs use dbfs:/ prefix

df = spark.read.format("csv").load("dbfs:/data/input.csv")

# Python file APIs use /dbfs/ path

with open("/dbfs/data/config.json", "r") as f:
    config = json.load(f)

# Equivalent operations
# These read the same file:

spark.read.text("dbfs:/data/sample.txt")  # Spark
open("/dbfs/data/sample.txt", "r")         # Python
```

DBFS Root Storage DBFS root is the default storage location for workspace data.

```
flowchart TB
    DBFS[DBFS Root] --> User[/user/]
    DBFS --> FileStore[/FileStore/]
    DBFS --> MLflow[/mlflow/]
    DBFS --> Data[/data/]

    User --> Hive[Hive metastore data]
    FileStore --> Uploads[Uploaded files, images]
    MLflow --> Artifacts[MLflow artifacts]
```

Path	Purpose	Notes
/user/hive/warehouse/	Hive metastore tables	Legacy location
/FileStore/	Uploaded files, images	Accessible via URL
/FileStore/tables/	Uploaded data files	For small files
/mlflow/	MLflow artifacts	Experiment tracking
/tmp/	Temporary files	Auto-cleaned

DBFS Operations

```
# List files

files = dbutils.fs.ls("dbfs:/data/")
for f in files:
    print(f"{f.name} - {f.size} bytes - isDir: {f.isDir()}")

# Check if path exists

def path_exists(path):
    try:
        dbutils.fs.ls(path)
        return True
    except Exception:
        return False

# Create directory

dbutils.fs.mkdirs("dbfs:/data/new_folder/")

# Copy files

dbutils.fs.cp("dbfs:/source/file.csv", "dbfs:/dest/file.csv")
dbutils.fs.cp("dbfs:/source/", "dbfs:/dest/", recurse=True)

# Move files

dbutils.fs.mv("dbfs:/old/path.csv", "dbfs:/new/path.csv")

# Delete files

dbutils.fs.rm("dbfs:/data/temp.csv")
dbutils.fs.rm("dbfs:/data/temp_folder/", recurse=True)

# Read file content (first 64KB)
```

```
content = dbutils.fs.head("dbfs:/data/sample.txt", maxBytes=1024)

# Write small text file

dbutils.fs.put("dbfs:/data/output.txt", "Hello World", overwrite=True)
```

FileStore FileStore is a special DBFS location accessible via web URLs.

```
# Upload file to FileStore

dbutils.fs.cp("dbfs:/data/image.png", "dbfs:/FileStore/images/image.png")

# Access via URL
# https://<workspace-url>/files/images/image.png

# Display image in notebook

displayHTML('<img src="/files/images/image.png">')
```

FileStore Path	URL Path
dbfs:/FileStore/images/logo.png	/files/images/logo.png
dbfs:/FileStore/tables/data.csv	/files/tables/data.csv

Mount Points (Legacy)

Mount points map cloud storage to DBFS paths. **Note:** Mounts are considered legacy; use Unity Catalog Volumes for new workloads.

Mount Architecture

```
flowchart LR
    Code[Spark Code] --> Mount["/mnt/data"]
    Mount --> DBFS[DBFS Layer]
    DBFS --> Cloud["s3://bucket/data or abfss://container@storage.dfs"]
```

AWS S3 Mounts

```
# Mount S3 bucket with instance profile (recommended)

dbutils.fs.mount(
    source="s3a://my-bucket/data",
    mount_point="/mnt/s3-data"
)

# Mount with access keys (not recommended for production)

access_key = dbutils.secrets.get("aws", "access-key")
secret_key = dbutils.secrets.get("aws", "secret-key")

dbutils.fs.mount(
    source="s3a://my-bucket/data",
    mount_point="/mnt/s3-data",
    extra_configs={
        "fs.s3a.access.key": access_key,
```

```
        "fs.s3a.secret.key": secret_key
    }
)
```

Azure Data Lake Storage Mounts

```
# Mount ADLS Gen2 with service principal

configs = {
    "fs.azure.account.auth.type": "OAuth",
    "fs.azure.account.oauth.provider.type":
     ↪  "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider",
    "fs.azure.account.oauth2.client.id": dbutils.secrets.get("azure",
     ↪  "client-id"),
    "fs.azure.account.oauth2.client.secret": dbutils.secrets.get("azure",
     ↪  "client-secret"),
    "fs.azure.account.oauth2.client.endpoint":
     ↪  "https://login.microsoftonline.com/<tenant-id>/oauth2/token"
}

dbutils.fs.mount(
    source="abfss://container@storageaccount.dfs.core.windows.net/",
    mount_point="/mnt/adls-data",
    extra_configs=configs
)

# Mount with SAS token

configs = {
    "fs.azure.sas.<container>.<storage-account>.blob.core.windows.net":
        dbutils.secrets.get("azure", "sas-token")
}

dbutils.fs.mount(
    source="wasbs://container@storageaccount.blob.core.windows.net/",
    mount_point="/mnt/blob-data",
    extra_configs=configs
)
```

GCS Mounts

```
# Mount Google Cloud Storage

dbutils.fs.mount(
    source="gs://my-bucket/data",
    mount_point="/mnt/gcs-data"
)
```

Managing Mounts

```
# List all mounts

mounts = dbutils.fs.mounts()
for mount in mounts:
    print(f"{mount.mountPoint} -> {mount.source}")
```

```python
# Check if mount exists

def mount_exists(mount_point):
    return any(m.mountPoint == mount_point for m in dbutils.fs.mounts())

# Unmount

dbutils.fs.unmount("/mnt/s3-data")

# Refresh mounts (after external changes)

dbutils.fs.refreshMounts()
```

Mount Best Practices

Do	Don't
Use secrets for credentials	Hardcode credentials
Use instance profiles/managed identity	Use access keys in code
Document mount points	Create ad-hoc mounts
Consider Unity Catalog Volumes	Create new mounts in 2024+

Unity Catalog Volumes (Recommended)

Unity Catalog Volumes provide governed access to cloud storage.

Volume Types

```mermaid
flowchart TB
    Volumes[Unity Catalog Volumes] --> Managed[Managed Volumes]
    Volumes --> External[External Volumes]

    Managed --> M1["Databricks manages storage"]
    Managed --> M2["Automatic cleanup"]

    External --> E1["User provides storage location"]
    External --> E2["Bring your own data"]
```

Type	Storage	Governance	Use Case
Managed	Databricks-managed	Full	Generated data, outputs
External	Your cloud storage	Full	Existing data, landing zones

Creating Volumes

```sql
-- Create managed volume
CREATE VOLUME main.default.my_volume;

-- Create external volume
CREATE EXTERNAL VOLUME main.default.landing_zone
LOCATION 's3://my-bucket/landing/';

-- Create external volume (Azure)
CREATE EXTERNAL VOLUME main.default.landing_zone
LOCATION 'abfss://container@storage.dfs.core.windows.net/landing/';
```

Volume Paths

```
# Volume paths use /Volumes/ prefix
# Format: /Volumes/<catalog>/<schema>/<volume>/

# Read from volume

df = spark.read.format("csv").load("/Volumes/main/default/landing_zone/data.csv")

# Write to volume

df.write.format("parquet").save("/Volumes/main/default/my_volume/output/")

# Python file operations

with open("/Volumes/main/default/my_volume/config.json", "r") as f:
    config = json.load(f)

# List volume contents

files = dbutils.fs.ls("/Volumes/main/default/my_volume/")
```

Volume Operations

```
-- List volumes in schema
SHOW VOLUMES IN main.default;

-- Describe volume
DESCRIBE VOLUME main.default.my_volume;

-- Drop volume
DROP VOLUME main.default.my_volume;
```

Volume vs Mount Comparison

Feature	Mounts	Unity Catalog Volumes
Governance	No	Yes (UC permissions)
Audit logging	Limited	Full
Access control	Workspace-level	Fine-grained
Path format	`/mnt/...`	`/Volumes/catalog/schema/volume/`
Recommended	Legacy	New workloads

Direct Cloud Access

Access cloud storage directly without mounts or volumes.

AWS S3 Direct Access

```
# With instance profile (attached to cluster)

df = spark.read.format("parquet").load("s3://bucket/path/")

# With temporary credentials
```

```
spark.conf.set("fs.s3a.access.key", access_key)
spark.conf.set("fs.s3a.secret.key", secret_key)
df = spark.read.format("parquet").load("s3a://bucket/path/")

# Using assume role

spark.conf.set("fs.s3a.aws.credentials.provider",
    "org.apache.hadoop.fs.s3a.auth.AssumedRoleCredentialProvider")
spark.conf.set("fs.s3a.assumed.role.arn", "arn:aws:iam::123456789:role/my-role")
```

Azure Direct Access

```
# With service principal

spark.conf.set(f"fs.azure.account.auth.type.{storage_account}
 ↪  .dfs.core.windows.net",
 ↪  "OAuth")
spark.conf.set(f"fs.azure.account.oauth.provider.type.{storage_account}
 ↪  .dfs.core.windows.net",
    "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
spark.conf.set(f"fs.azure.account.oauth2.client.id.{storage_account}
 ↪  .dfs.core.windows.net",
    client_id)
spark.conf.set(f"fs.azure.account.oauth2.client.secret.{storage_account}
 ↪  .dfs.core.windows.net",
    dbutils.secrets.get("azure", "client-secret"))
spark.conf.set(f"fs.azure.account.oauth2.client.endpoint.{storage_account}
 ↪  .dfs.core.windows.net",
    f"https://login.microsoftonline.com/{tenant_id}/oauth2/token")

df = spark.read.format("parquet").load(f"abfss://container@{storage_account}
 ↪  .dfs.core.windows.net/path/")

# With SAS token

spark.conf.set(f"fs.azure.sas.{container}.{storage_account}
 ↪  .blob.core.windows.net",
    dbutils.secrets.get("azure", "sas-token"))
df = spark.read.format("parquet").load(f"wasbs://{container}@{storage_account}
 ↪  .blob.core.windows.net/path/")
```

GCS Direct Access

```
# With service account (attached to cluster)

df = spark.read.format("parquet").load("gs://bucket/path/")

# With service account key file

spark.conf.set("google.cloud.auth.service.account.enable", "true")
spark.conf.set("google.cloud.auth.service.account.json.keyfile",
 ↪  "/path/to/keyfile.json")
```

Storage Protocols

Protocol Comparison

Protocol	Cloud	Format	Use Case
`s3://`	AWS	Legacy S3	Older compatibility
`s3a://`	AWS	Optimized S3	Recommended for AWS
`wasb://`	Azure Blob	Legacy	Blob storage
`wasbs://`	Azure Blob	Legacy + SSL	Blob storage
`abfs://`	Azure ADLS Gen2	Optimized	Recommended for Azure
`abfss://`	Azure ADLS Gen2	Optimized + SSL	Recommended for Azure
`gs://`	GCP	GCS	Google Cloud

Choosing the Right Protocol

```
flowchart TD
    Cloud{Cloud Provider?}
    Cloud --> |AWS| S3[Use s3a://]
    Cloud --> |Azure| Azure{Storage Type?}
    Cloud --> |GCP| GCS[Use gs://]

    Azure --> |ADLS Gen2| ABFSS[Use abfss://]
    Azure --> |Blob| WASBS[Use wasbs://]
```

Access Patterns

Credential Hierarchy

```
flowchart TD
    Access[Storage Access] --> UC{Unity Catalog?}
    UC --> |Yes| UCAccess[UC manages credentials]
    UC --> |No| Mount{Using Mount?}

    Mount --> |Yes| MountCreds[Mount configuration]
    Mount --> |No| Direct{Direct Access?}

    Direct --> |Yes| SparkConf[Spark conf settings]
    Direct --> |No| Instance[Instance profile/Managed identity]
```

Recommended Access Pattern by Use Case

Use Case	Recommended Approach
New workloads	Unity Catalog Volumes
Legacy pipelines	Existing mounts
External data	External Volumes
Cross-cloud	External Volumes with storage credentials
Ad-hoc access	Direct access with secrets

Data Movement Patterns

Copy Between Locations

```
# Copy between DBFS locations

dbutils.fs.cp("dbfs:/source/", "dbfs:/dest/", recurse=True)

# Copy from cloud to DBFS

dbutils.fs.cp("s3://bucket/data/", "dbfs:/local-copy/", recurse=True)

# Copy using Spark (for large data)

df = spark.read.format("parquet").load("s3://source-bucket/data/")
df.write.format("parquet").save("abfss://container@storage/dest/")
```

Upload/Download with REST API

```
# For small files, use REST API

import requests
import base64

# Upload to DBFS

url = f"{workspace_url}/api/2.0/dbfs/put"
data = base64.b64encode(file_content).decode()
response = requests.post(url, headers=headers, json={
    "path": "/data/file.txt",
    "contents": data,
    "overwrite": True
})
```

Use Cases

Landing Zone Pattern

```
flowchart LR
    Source[External Source] --> Landing["/Volumes/main/raw/landing/"]
    Landing --> Bronze["/Volumes/main/bronze/tables/"]
    Bronze --> Silver["/Volumes/main/silver/tables/"]
    Silver --> Gold["/Volumes/main/gold/tables/"]
```

```
# Landing zone with external volume

landing_path = "/Volumes/main/raw/landing/"

# Process new files

new_files = dbutils.fs.ls(landing_path)
for file in new_files:
    if file.name.endswith(".csv"):
        df = spark.read.format("csv").option("header", "true").load(file.path)
 ↪  df.write.format("delta").mode("append").saveAsTable("main.bronze.raw_data")

        # Archive processed file
        dbutils.fs.mv(file.path, f"/Volumes/main/raw/archive/{file.name}")
```

Multi-Cloud Data Access

```
# Read from AWS

aws_df = spark.read.format("parquet").load("/Volumes/aws/raw/data/")

# Read from Azure

azure_df = spark.read.format("parquet").load("/Volumes/azure/raw/data/")

# Combine data

combined = aws_df.unionByName(azure_df)
combined.write.format("delta").saveAsTable("main.combined.all_data")
```

Common Issues & Errors

Permission Denied on DBFS **Scenario:** Cannot access DBFS path.

Fix: Check cluster IAM role or storage permissions.

Mount Point Already Exists **Scenario:** Mount fails because path exists.

```
# Error: Directory already mounted
```

Fix: Unmount first or use different mount point:

```
if mount_exists("/mnt/data"):
    dbutils.fs.unmount("/mnt/data")
dbutils.fs.mount(source="...", mount_point="/mnt/data")
```

Credential Not Found **Scenario:** Direct access fails due to missing credentials.

Fix: Set Spark configuration or use Unity Catalog:

```
# Check if config is set

spark.conf.get("fs.s3a.access.key", "NOT_SET")
```

Path Format Mismatch **Scenario:** Mixing `/dbfs/` and `dbfs:/` incorrectly.

```
# Wrong: Using /dbfs/ with Spark

df = spark.read.csv("/dbfs/data/file.csv")  # Error

# Correct: Use dbfs:/ with Spark

df = spark.read.csv("dbfs:/data/file.csv")

# Correct: Use /dbfs/ with Python

with open("/dbfs/data/file.csv") as f: ...
```

Volume Not Found **Scenario:** Unity Catalog volume path not accessible.

Fix: Check catalog/schema permissions and volume existence:

```
SHOW VOLUMES IN main.default;
DESCRIBE VOLUME main.default.my_volume;
```

Exam Tips

1. **DBFS paths** - `dbfs:/` for Spark, `/dbfs/` for Python file APIs
2. **Mount deprecation** - Mounts are legacy; volumes are recommended
3. **Volume types** - Managed (Databricks stores) vs External (your storage)
4. **Volume path format** - `/Volumes/<catalog>/<schema>/<volume>/`
5. **FileStore** - Special DBFS location accessible via web URLs
6. **Protocol choice** - s3a:// for AWS, abfss:// for Azure ADLS Gen2
7. **Credential precedence** - UC > Instance profile > Spark conf > Mount config
8. **DBFS root** - Contains /user/hive/warehouse, /FileStore, /tmp
9. **refreshMounts()** - Needed after external mount changes
10. **Direct access** - Requires explicit credential configuration

Key Takeaways

- **DBFS path formats**: use `dbfs:/...` with Spark APIs (`spark.read`, `df.write`); use `/dbfs/...` with Python file I/O (`open()`); never mix these — using `/dbfs/` with Spark or `dbfs:/` with Python file APIs causes errors
- **Mount points are legacy**: `dbutils.fs.mount()` is deprecated for new workloads; Unity Catalog Volumes (`/Volumes/catalog/schema/volume/`) are the recommended replacement with full governance and audit logging
- **Volume types**: Managed volumes are stored in Databricks-controlled storage and auto-cleaned on drop; External volumes point to user-owned cloud storage and are not cleaned on drop
- **Volume path format**: `/Volumes/<catalog>/<schema>/<volume>/path` — works for both Spark APIs and Python file I/O
- **FileStore** (`dbfs:/FileStore/`) is a special DBFS location accessible via HTTP URL (`/files/...`); use it for images and small files to embed in notebooks
- **Cloud storage protocols**: prefer `s3a://` (AWS), `abfss://` (Azure ADLS Gen2 with SSL), `gs://` (GCP); avoid legacy `s3://`, `wasb://`, `abfs://`
- **DBFS root key directories**: `/user/hive/warehouse/` (Hive metastore tables), `/FileStore/` (web-accessible files), `/mlflow/` (MLflow artifacts), `/tmp/` (auto-cleaned temp files)
- **Credential precedence** for cloud storage access: Unity Catalog external location credentials > cluster instance profile / managed identity > Spark configuration > mount configuration

Related Topics

- Unity Catalog - Governance for volumes
- Secret Management - Storing credentials
- Auto Loader - File ingestion from cloud storage

Official Documentation

- DBFS
- Unity Catalog Volumes
- Mount Cloud Storage
- External Locations

Section 8: Debugging and Deploying (10 % of Exam)

Finding and fixing failures, then shipping code: diagnostics from the Spark UI, compute logs, system tables and query profiles; job repair with parameter overrides; deploying with Declarative Automation Bundles; and Git-based CI/CD with Databricks Git folders.

Topics Overview

```
flowchart LR
    D[Debugging & Deploying]
    D --> T1["Spark UI Debugging"]
    D --> T2["Job Repair and Parameter Overrides"]
    D --> T3["Declarative Automation Bundles"]
    D --> T4["Bundle Deployment Strategies — Part 1: Bundle Patterns & CI/CD
↪   Pipelines"]
    D --> T5["CI/CD Integration — Part 1: Platforms & Configuration"]
    D --> T6["Git Folders"]
```

Section Contents

File	Topic
01-spark-ui-debugging.md	Spark UI, compute logs, and choosing a diagnostic source
02-job-repair-and-parameter-overrides.md	Repair runs, re-running failed tasks, parameter overrides
03-declarative-automation-bundles-part1.md	Bundle structure, resources, variables, commands, targets
03-declarative-automation-bundles-part2.md	State, CI/CD integration, templates, patterns
04-bundle-deployment-strategies-part1.md	Advanced bundle and CI/CD pipeline patterns
04-bundle-deployment-strategies-part2.md	Rollback strategies, OIDC federation
05-cicd-integration-part1.md	Authentication; GitHub Actions, Azure DevOps, GitLab, Jenkins
05-cicd-integration-part2.md	Testing and deployment strategies in CI/CD
06-git-folders.md	Databricks Git folders (formerly Repos) for Git-based workflows

Exam Skill Coverage

Every skill bullet in Section 8 of the Oct 9, 2026 exam guide, with the files that cover it.

Exam guide skill
Identify pertinent diagnostic information using Spark UI, cluster logs, system tables, and query profiles to trou
Analyze the errors and remediate the failed job runs with job repairs and parameter overrides.
Deploy Databricks resources using Declarative Automation Bundles (formerly Databricks Asset Bundles).
Configure Git-based CI/CD workflows using Databricks Git folders (formerly Repos) to deploy notebooks and co

Key Concepts to Master

Concept	Why it matters
Repair run	Re-runs only the failed tasks (and their dependents) of a job run, option
Bundle targets	Per-environment overrides in `databricks.yml`; `databricks bundle de`
`mode: development` vs `production`	Development prefixes and pauses resources for safe iteration; producti
Git folders	Workspace folders backed by a Git repo; CI updates a production folder

Related Resources

- Spark configurations cheat sheet (shared)
- Error messages appendix (shared)

Spark UI Debugging

Overview

The Spark UI shows how a Spark application executed: jobs, stages, tasks, executors, and SQL plans. That makes it the main tool for finding skew, spill, and resource problems on classic compute. It is one of four diagnostic sources the exam names, alongside **cluster (compute) logs**, **system tables**, and **query profiles**, and you need to know which one answers which question.

Overview

- Spark execution hierarchy: an action creates a job, shuffle boundaries split it into stages, and each partition becomes a task
- Spark UI tabs (Jobs, Stages, Executors, SQL/DataFrame) expose skew, spill, GC pressure, and join strategy
- Compute logs: driver logs (stdout, stderr, Log4j), worker logs, the compute event log, and log delivery to a Unity Catalog volume
- Serverless compute has no Spark UI or Spark logs: use the query profile, query history, and the job run timeline instead

What the Exam Tests

- ”Identify pertinent diagnostic information using Spark UI, cluster logs, system tables, and query profiles to troubleshoot errors.”
- Reading stage and task metrics to tell skew, spill, small files, and failed broadcasts apart
- Picking the right diagnostic source for a symptom, including what is (and is not) available on serverless

---

Spark UI Tabs at a Glance

```
flowchart TB
    Application[Spark Application] --> Jobs[Jobs]
    Jobs --> Stages[Stages]
    Stages --> Tasks[Tasks]

    subgraph SparkUI["Spark UI Tabs"]
        JobsTab[Jobs]
        StagesTab[Stages]
        StorageTab[Storage]
        EnvironmentTab[Environment]
        ExecutorsTab[Executors]
        SQLTab[SQL/DataFrame]
    end
```

Spark Execution Hierarchy

Jobs, Stages, and Tasks

```
flowchart TD
    Action["Action (collect, save, count)"] --> Job[Job]
    Job --> Stage1[Stage 1]
    Job --> Stage2[Stage 2]
    Job --> Stage3[Stage 3]

    Stage1 --> |Shuffle| Stage2
    Stage2 --> |Shuffle| Stage3

    Stage1 --> T1[Task 1.1]
    Stage1 --> T2[Task 1.2]
    Stage1 --> T3[Task 1.3]
```

Level	Created By	Characteristics
Job	Each action (save, collect, count)	Contains multiple stages
Stage	Shuffle boundaries	Contains multiple tasks
Task	Partitions	Smallest unit of work

What Triggers Shuffles

Operation	Causes Shuffle
`groupBy`, `reduceByKey`	Yes
`join` (most types)	Yes
`repartition`	Yes
`distinct`	Yes
`orderBy`, `sort`	Yes
`map`, `filter`	No
`coalesce` (reduce only)	No
`broadcast` join	No (small table)

Jobs Tab

Job List View The Jobs tab shows all jobs in the application:

Column	Description
Job Id	Unique identifier
Description	Action that triggered job
Submitted	Start timestamp
Duration	Total execution time
Stages	Succeeded/Total stages
Tasks	Succeeded/Total tasks

Identifying Issues from Jobs

```
Signs of Problems:
- Long duration compared to similar jobs
- Failed stages/tasks
- Many retried tasks
- Significant time spent in specific stages
```

Stages Tab

Stage Details Each stage shows:

Metric	Description	What to Look For
Duration	Wall clock time	Compare to expected
Input	Data read	Size vs expected
Output	Data written	Compare to input
Shuffle Read	Data from previous stages	Large = potential bottleneck
Shuffle Write	Data for next stages	Large = expensive

DAG Visualization

```
flowchart LR
    Scan1["Scan (parquet)"] --> Filter1["Filter"]
    Scan2["Scan (parquet)"] --> Project["Project"]
    Filter1 --> Exchange1["Exchange (shuffle)"]
    Project --> Exchange2["Exchange (shuffle)"]
    Exchange1 --> Join["SortMergeJoin"]
    Exchange2 --> Join
    Join --> Aggregate["HashAggregate"]
    Aggregate --> Exchange3["Exchange"]
    Exchange3 --> FinalAgg["HashAggregate"]
```

Stage Metrics to Monitor

```
Key Metrics:
├── Duration
│   ├── Scheduler Delay (waiting for resources)
│   ├── Task Deserialization
│   ├── GC Time (garbage collection)
│   ├── Result Serialization
│   └── Getting Result Time
├── Shuffle Read
│   ├── Read Size
│   ├── Records Read
│   └── Fetch Wait Time
├── Shuffle Write
│   ├── Write Size
│   └── Records Written
└── Spill
    ├── Spill (Memory) - deserialized size of the spilled data in memory
    └── Spill (Disk) - serialized size of the spilled data on disk
```

Tasks Tab

Task-Level Analysis Individual task view reveals:

Column	Description	Issues Indicated
Duration	Task execution time	Variance indicates skew
GC Time	Garbage collection	High = memory pressure
Input Size	Data read by task	Uneven = skewed partitions

Column	Description	Issues Indicated
Shuffle Read	From other executors	High = shuffle-heavy
Shuffle Write	To other executors	High = expensive shuffle
Errors	Error messages	Failed tasks

Task Duration Distribution

```
Normal Distribution:
Task 1: ████████ 8s
Task 2: ████████ 8s
Task 3: ████████ 9s
Task 4: ████████ 8s

Skewed Distribution (Problem!):
Task 1: ██       2s
Task 2: ██       2s
Task 3: ██       2s
Task 4: ████████████████████████ 60s  ← Straggler
```

Executors Tab

Executor Metrics

Metric	Description	Healthy Range
RDD Blocks	Cached data blocks	Based on caching strategy
Storage Memory	Memory for caching	< 60% typically
Disk Used	Spilled data on disk	Should be minimal
Active Tasks	Currently running	Even distribution
Failed Tasks	Task failures	Should be 0 or low
Total GC Time	Garbage collection	< 10% of total time

Executor Health Indicators

```
Healthy Executor:
├── GC Time: 5% of total
├── Storage Memory: 40% used
├── Disk Used: 0 MB
├── Failed Tasks: 0
└── Active Tasks: 4 (= cores)

Unhealthy Executor:
├── GC Time: 40% of total ← Memory pressure
├── Storage Memory: 95% used ← Near limit
├── Disk Used: 10 GB ← Spilling to disk
├── Failed Tasks: 5 ← Task failures
└── Active Tasks: 1 ← Underutilized
```

SQL/DataFrame Tab

Query Execution Plans The SQL tab shows:

- Executed SQL queries and DataFrame operations

- Query plans (logical and physical)
- Per-operator metrics

Reading Query Plans

```
== Physical Plan ==
*(5) HashAggregate(keys=[customer_id], functions=[sum(amount)])
+- Exchange hashpartitioning(customer_id, 200), ENSURE_REQUIREMENTS
   +- *(4) HashAggregate(keys=[customer_id], functions=[partial_sum(amount)])
      +- *(4) Project [customer_id, amount]
         +- *(4) BroadcastHashJoin [order_id], [order_id], Inner
            :- *(2) Filter isnotnull(order_id)
            :  +- *(2) Scan parquet [order_id, customer_id, amount]
            +- BroadcastExchange HashedRelationBroadcastMode
               +- *(3) Scan parquet [order_id]
```

Plan Components

Component	Symbol	Description
Scan	`Scan parquet`	Read from storage
Filter	`Filter`	Apply predicates
Project	`Project`	Select columns
Exchange	`Exchange`	Shuffle data
HashAggregate	`HashAggregate`	Aggregation
BroadcastHashJoin	`BroadcastHashJoin`	Broadcast join
SortMergeJoin	`SortMergeJoin`	Sort-merge join
Sort	`Sort`	Order data

Common Performance Issues

Data Skew Symptoms:

- One task much longer than others
- Straggler tasks in stage
- Uneven input sizes across tasks

In Spark UI:

```
Stage 5 Task Summary:
Min Duration: 2s
25th percentile: 3s
Median: 4s
75th percentile: 5s
Max Duration: 180s  ← Significant skew!
```

Solutions:

```
# Enable AQE skew handling

spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")

# Salting technique for severe skew

from pyspark.sql.functions import col, concat, lit, rand
```

```
# Add salt to skewed key

df_salted = df.withColumn(
    "salted_key",
    concat(col("skewed_key"), lit("_"), (rand() * 10).cast("int"))
)

# Join with salted keys
# Then remove salt after join
```

Shuffle Spill Symptoms:

- Spill (Disk) > 0 in stage details
- Slow shuffle operations
- High GC time

In Spark UI:

```
Shuffle Spill (Memory): 10 GB
Shuffle Spill (Disk): 5 GB  ← Problem!
```

Solutions:

```
# Increase memory fraction for execution

spark.conf.set("spark.memory.fraction", "0.8")

# Increase shuffle partitions

spark.conf.set("spark.sql.shuffle.partitions", "400")

# Use more/larger executors
# Cluster configuration change
```

Small File Problem Symptoms:

- Many small tasks
- High task overhead
- Slow stage startup

In Spark UI:

```
Stage 1: 10000 tasks  ← Too many small tasks
Input: 1 GB
Average task input: 100 KB  ← Very small
```

Solutions:

```
# Coalesce before writing

df.coalesce(100).write.format("delta").save("/path")

# Repartition for better distribution

df.repartition(100).write.format("delta").save("/path")

# Use OPTIMIZE on Delta tables

spark.sql("OPTIMIZE my_table")
```

Broadcast Join Failure Symptoms:

- Sort-merge join when broadcast expected
- Large shuffle read/write
- Long join stages

In Spark UI:

```
Expected: BroadcastHashJoin
Actual: SortMergeJoin
Shuffle Read: 50 GB  ← Large shuffle
```

Solutions:

```
from pyspark.sql.functions import broadcast

# Force broadcast

result = large_df.join(broadcast(small_df), "key")

# Increase broadcast threshold (default 10MB)

spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "100MB")
```

Excessive GC Symptoms:

- High GC Time in executors
- Task duration variance
- Executor failures

In Spark UI:

```
Executor 1 GC Time: 45% of total  ← Too high!
```

Solutions:

```
# Increase executor memory
# --executor-memory 16g

# Tune G1GC
# --conf spark.executor.extraJavaOptions="-XX:+UseG1GC -XX:G1HeapRegionSize=16m"

# Reduce data cached in memory

df.unpersist()

# Use Kryo serialization

spark.conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
```

OOM (Out of Memory) Symptoms:

- Executor lost errors
- Task failures with OOM
- Out-of-memory stack traces in the driver logs or worker logs (see Compute Logs)

Solutions:

```
# Increase driver/executor memory
```

```
# --driver-memory 8g
# --executor-memory 16g

# Increase memory overhead

spark.conf.set("spark.executor.memoryOverhead", "2g")

# Avoid collecting large results to driver
# Bad: df.collect()
# Good: df.write.save() or df.take(100)

# Increase partitions to reduce per-partition memory

spark.conf.set("spark.sql.shuffle.partitions", "400")
```

Interpreting Metrics

Healthy vs Unhealthy Indicators

Metric	Healthy	Concerning	Critical
GC Time	<5%	10-20%	>20%
Spill (Disk)	0	>0	Large
Task Duration CV	<0.5	0.5-1.0	>1.0
Failed Tasks	0	1-5	>5
Shuffle Read Wait	<10%	10-30%	>30%

Performance Checklist

```
□ Check job duration against baseline
□ Look for failed stages/tasks
□ Check task duration distribution for skew
□ Verify shuffle sizes are reasonable
□ Check for spill (memory/disk)
□ Review executor GC times
□ Examine query plan for unexpected operations
□ Verify broadcast joins are working
□ Check partition counts and sizes
```

Accessing Spark UI

```
All-purpose or jobs compute (classic):
1. Compute → select the compute → Spark UI tab
2. For a job run: open the task run details page and follow the
   link to the compute that ran the task
3. Worker (executor) logs are reached from the Spark UI tab

Serverless compute:
   No Spark UI → use the query profile (see "Diagnostics on Serverless")
```

- The Jobs API `spark_context_id` field notes that the Spark UI "continues to be available after the run has completed."
- After a **terminated compute is restarted**, the Spark UI shows the restarted compute only, not the history from before termination.
- A **Streaming** tab appears only while a streaming query runs on the compute. Ideally each micro-batch finishes within 80 % of the batch interval.

Compute Logs (Cluster Logs)

The exam calls these ”cluster logs”. Current docs call them **compute logs**.

Log	What it contains
Driver logs	`print` and log statements from notebooks, jobs, and libraries, in three outputs: **st**
Worker (executor) logs	Executor-side `stdout`, `stderr`, and Log4j output
Compute event log	Lifecycle events such as creation, termination, and configuration edits, triggered
Metrics	Compute metrics, collected every minute, with a date picker for history

Driver Logs

- Start here for driver-side exceptions and for your own `print` and logging output.
- By default only users with **CAN MANAGE** on the compute can view driver logs. Setting `spark.databricks.acl.needAdminPermissionToViewLogs` to `false` in the compute's Spark config lets CAN ATTACH TO and CAN RESTART users view them.
- **Secrets are not redacted** from driver stdout and stderr, so never print credentials.
- Executor logs are not supported on **Standard** access mode compute. On **Dedicated** access mode, the dedicated user or group and workspace admins can read them.

Compute Event Log

- Shows what happened to the compute itself, not to your code: use it for ”the cluster terminated”, ”the cluster could not start”, or resize questions.
- Events are kept for **60 days**. **Download** exports the filtered rows to CSV or Excel, capped at 10,000 rows.

Compute Log Delivery Driver, worker, and event logs can be delivered to long-term storage:

Destination	Notes
Unity Catalog volume (recommended)	Most secure option on Unity Catalog-enabled compute. Needs **Sta**
Amazon S3	Needs an instance profile with `PutObject` and `PutObjectAcl`
DBFS (legacy)	Only if DBFS root and mounts are not disabled for the workspace

- Logs are delivered **every five minutes** and archived hourly, until the compute terminates.
- Databricks creates a subfolder named after the `cluster_id`. The Clusters API places driver logs under `$destination/$clusterId/driver` and executor logs under `$destination/$clusterId/executor`.
- Only one destination per compute. For job clusters, set it in the job's compute settings or in `new_cluster.cluster_log_conf`.

```
{
  "new_cluster": {
    "spark_version": "<runtime-version>",
    "node_type_id": "<node-type>",
    "num_workers": 4,
    "cluster_log_conf": {
      "volumes": {
        "destination": "/Volumes/main/ops/compute_logs"
      }
    }
  }
}
```

Diagnostics on Serverless

Serverless compute removes most compute-level tools. Know what replaces each one.

Classic tool	Serverless
Spark UI	**Not available**
Spark driver and worker logs	**Not available**: only client-side application logs
Per-task log isolation (jobs)	Task logs are **not isolated per task run**
Compute event log	Not supported
Most Spark configurations	Not supported (only a documented subset)
Compute-scoped init scripts	Not supported
Query history	**Available**: includes all SQL and Python queries run on serverless notebooks ar

- In a serverless job run, the **Timeline** view lists each query statement under its task. Click a statement for its insights or query profile. A lightbulb marks tasks and queries with **performance insights**.
- In a notebook on serverless or a SQL warehouse, click **See performance** under a cell, then a statement, to open its query profile.
- The query profile shows each operator's **time spent**, **memory peak**, and **rows**, plus pruning, I/O, and ranked performance insights. Viewing it requires owning the query or at least **CAN MONITOR** on the SQL warehouse; a profile can be downloaded as JSON and imported elsewhere.
- Queries served from the **query cache** have no profile. Change the query slightly to get one.

Which Diagnostic Source for Which Symptom

Symptom or question	Go to first
One task much slower than its peers (straggler)	Spark UI → Stages → task summary
`Spill (Disk)` > 0, high GC	Spark UI → Stages / Executors
Join did not broadcast	Spark UI → SQL/DataFrame plan, or the
Slow SQL on a warehouse or serverless	**Query profile** (from query history)
Python exception, `print` output, driver OOM	**Driver logs** (stderr, Log4j)
Executor failures, executor-side exceptions	**Worker logs** via Spark UI → Executors
Cluster failed to start or terminated unexpectedly	**Compute event log**
Logs needed after the compute is gone	**Log delivery** location (UC volume)
Which jobs failed this week, and why	**System tables**: `system.lakeflow.jo`
What did this workload cost	**System tables**: `system.billing.usa`
Which queries were slow or failed across warehouses and serverless	**System tables**: `system.query.histo`
A failed task inside a job run	Task run details: output and error

```sql
-- Most common failure reasons for job tasks in the last 7 days
SELECT termination_code, result_state, count(*) AS task_runs
FROM system.lakeflow.job_task_run_timeline
WHERE result_state IN ('FAILED', 'ERROR', 'TIMED_OUT')
  AND period_start_time > current_timestamp() - INTERVAL 7 DAYS
GROUP BY termination_code, result_state
ORDER BY task_runs DESC;
```

Platform update (Oct 2026)

The Lakeflow pipelines system tables `system.lakeflow.pipelines` and `system.lakeflow.pipeline_update_timeline` became generally available on Oct 21, 2026, so pipeline update failures can be queried the same way. Source: October 2026 release notes

Use Cases

Debugging Slow Job

```
1. Open Spark UI for job
2. Identify slowest stage
3. Check stage metrics:
   - Large shuffle read/write?
   - Spill to disk?
   - Skewed task durations?
4. Drill into tasks:
   - Find straggler tasks
   - Check input sizes
   - Review error messages
5. Apply appropriate fix
```

Optimizing Joins

```
1. Find join stage in SQL tab
2. Check join type:
   - BroadcastHashJoin (ideal for small table)
   - SortMergeJoin (large tables)
   - ShuffledHashJoin
3. If SortMergeJoin with small table:
   - Force broadcast
   - Increase threshold
4. Check shuffle metrics
5. Look for skew in join keys
```

Common Issues & Errors

Symptom	Likely cause
Spark UI tab missing for a notebook or job	The workload ran on **serverless** compute
Spark UI shows nothing from before an incident	The compute was **restarted** after termination
Cannot open driver logs	Default log ACL is CAN MANAGE
No executor logs on shared compute	**Standard** access mode does not support executor logs
Log delivery to a volume is ignored	Dedicated access mode assigned to a **group**, or missing WR
Query profile not available for a query	Result served from the **query cache**
`result_state` is `NULL` in job timeline tables	Long run split into hourly slices; only the last slice has a stat

Exam Tips

Tip

- **Job-Stage-Task hierarchy**: actions create jobs, shuffles create stages, partitions create tasks; `Exchange` = shuffle.
- **Skew** = one straggler task with much larger input; **spill** = `Spill (Disk)` > 0; high GC time = memory pressure.
- **Broadcast vs SortMerge**: a `SortMergeJoin` on a small table means the broadcast did not happen.
- **Driver logs** (stdout, stderr, Log4j) for driver-side errors and `print` output; **worker logs** through the Spark UI; **event log** for compute lifecycle (60-day retention).
- **Log delivery**: a Unity Catalog volume is the recommended destination; logs arrive every five minutes under a `cluster_id` folder.

- **Serverless**: no Spark UI, no Spark logs. Use the **query profile**, query history, and the job run timeline.
- **System tables** answer fleet-wide questions: failed tasks (`system.lakeflow.job_task_run_timeline`), cost (`system.billing.usage`), slow queries (`system.query.history`).

Key Takeaways

- **Job-Stage-Task hierarchy**: Each Spark action creates a Job; shuffle boundaries create Stage splits; each partition creates one Task.
- **Exchange = Shuffle**: Every Exchange node in a physical plan represents a full network shuffle and is a primary performance cost.
- **Skew detection**: A straggler task with a much longer duration than its peers and a much larger input size indicates data skew in that partition.
- **Spill is a red flag**: Any non-zero `Spill (Disk)` in stage details indicates memory pressure and usually requires increasing partitions or executor memory.
- **GC overhead threshold**: GC time above 10% of total executor time indicates memory pressure; above 20% is critical and likely causes instability.
- **AQE helps at runtime**: Adaptive Query Execution can automatically handle skew joins, coalesce small shuffle partitions, and switch join strategies based on actual data sizes.
- **Broadcast vs SortMergeJoin**: `BroadcastHashJoin` eliminates the shuffle for the small side; `SortMergeJoin` requires a shuffle on both sides — always prefer broadcast for small tables.
- **OOM types differ**: Driver OOM is caused by collecting too much data (avoid `df.collect()`); Executor OOM requires increasing `spark.executor.memoryOverhead` or reducing per-partition data.
- **Compute logs**: driver logs, worker logs, and the compute event log cover code errors, executor errors, and compute lifecycle; deliver them to a UC volume to keep them after the compute terminates.
- **Pick the source by symptom**: Spark UI for classic execution detail, query profile for SQL and serverless statements, system tables for cross-job history and cost.

Related Topics

- Job Repair and Parameter Overrides - Remediating the failed runs you diagnose here
- Query Profiler - Query profile operators and metrics
- System Tables - `system.lakeflow`, `system.billing`, and `system.query` tables
- Performance Optimization - Tuning strategies
- Databricks Compute - Access modes and serverless
- Structured Streaming - Streaming UI

Official Documentation

- Debugging with the Spark UI
- Diagnose cost and performance issues using the Spark UI
- Manage compute (event log, driver and worker logs, metrics)
- Compute configuration reference (compute log delivery)
- Serverless compute limitations
- Query profile
- Query history
- Monitoring and observability for Lakeflow Jobs
- Jobs system table reference
- Apache Spark Web UI
- Spark SQL performance tuning

Job Repair and Parameter Overrides

Overview

When a multi-task Lakeflow Jobs run fails, you rarely want to start the whole run again. You read the failed task's output and error, fix the cause, and then **repair** the run. A repair re-runs only the unsuccessful tasks and the tasks that depend on them, inside the original run. Parameter overrides let that repair, or a fresh **Run now with different settings**, use different values without editing the job.

Overview

- Triage starts on the job **Runs** tab (matrix view), then the **Task run details** page (output, error message, metadata, links to compute logs and query history)
- **Repair run** re-runs unsuccessful tasks plus their dependents as part of the *original* run, using the *current* job and task settings; successful tasks are not re-run
- **Job parameters** are pushed down to key-value tasks and win over task parameters with the same key; you can override them at trigger time and at repair time
- Automation: `POST /api/2.2/jobs/runs/repair`, `databricks jobs repair-run`, and `w.jobs.repair_run()` in the Python SDK

What the Exam Tests

- "Analyze the errors and remediate the failed job runs with job repairs and parameter overrides."
- Choosing repair vs. **Run now** vs. automatic retries for a given failure
- Knowing which tasks a repair re-runs, which settings it uses, and why idempotent writes matter
- Overriding job parameters on a repair or a manual run, and referencing them with `{{job.parameters.<name>}}`

Failure-to-Fix Workflow

```
flowchart TD
    Fail[Job run failed] --> Matrix[Runs tab: matrix view]
    Matrix --> Task[Task run details: output, error, metadata]
    Task --> Cause{Root cause?}
    Cause -->|Code or task config| Edit[Edit task / fix code]
    Cause -->|Compute too small| Compute[Change workers, instance type, or swap
↪   compute]
    Cause -->|Wrong input value| Params[Plan a parameter override]
    Cause -->|Upstream source down| Wait[Wait until the source recovers]
    Edit --> Repair[Repair run]
    Compute --> Repair
    Params --> Repair
    Wait --> Repair
    Repair --> Check{All tasks green?}
    Check -->|No| Task
    Check -->|Yes| Done[Run succeeded]
```

Reading a Failed Run

Where to Look in the UI

Location	What it shows
Jobs & Pipelines > job > **Runs** tab, matrix view	One row per run, one cell per task. Green = success, red =
Hover a failed cell	Start/end time, status, duration, cluster details, sometime
Runs tab, list view	Start time, run ID, trigger type, duration, status, **error co**
Job run details page	Graph, Timeline, and List views of the run
Task run details page	Task **output**, **error message**, metadata, source code, the
Serverless task, Timeline view	Each query statement and its duration; click a statement f

Note

A task can be unsuccessful because it **failed** or because it was **skipped** after an upstream task failed. Fix the first red task in the dependency chain; the pink (skipped) tasks downstream are symptoms. For a failed job, the Jobs UI also offers **Diagnose Error**, which uses Genie Code to analyze the failure.

Result States You Will See The Jobs API reports task and run outcomes in `status` (the older `state.result_state` field is deprecated). The values below come from the Jobs API Run object.

`result_state`	Meaning for triage
`FAILED`	The task completed with an error: read its output and error
`TIMEDOUT`	The task hit its timeout (duration threshold)
`CANCELED`	Canceled by a user or the platform
`UPSTREAM_FAILED`	Not run because a dependency failed: fix the upstream task
`UPSTREAM_CANCELED`	Not run because a dependency was canceled
`MAXIMUM_CONCURRENT_RUNS_REACHED`	The job was already at its concurrent-run limit

The full enum also includes values such as `SUCCESS`, `SUCCESS_WITH_FAILURES`, `EXCLUDED`, and `DISABLED`.

Reading a Run Programmatically

```
# Run metadata, including every repair attempt (repair_history)
databricks jobs get-run 455644833 --include-history --include-resolved-values -o
 ↪  json

# Output and metadata of ONE task run: error, error_trace,
 ↪  dbutils.notebook.exit() value
# (output is capped at the first 5 MB)
databricks jobs get-run-output 455644834 -o json
```

- `get-run --include-history` adds `repair_history`: one `ORIGINAL` item and one `REPAIR` item per repair, each with an `id` and the `task_run_ids` that ran in it.
- `get-run-output` takes a **task** run ID, not the parent job run ID.
- Runs are removed automatically after **60 days**, so export anything you need to keep (the run details page offers **Export to HTML** for notebook results).

Finding Failures Across Many Jobs (System Tables) The `system.lakeflow` schema keeps job and task run history for **365 days**. Note that system tables use their own value spellings (`SUCCEEDED`, `FAILED`, `TIMED_OUT`, `CANCELLED`, `SKIPPED`, `ERROR`, `BLOCKED`).

```
-- Failed or timed-out task runs in the last 7 days, newest first
SELECT
  job_id,
  job_run_id,
  run_id AS task_run_id,
  task_key,
  result_state,
  termination_code,
  compute_ids,
  period_start_time,
  period_end_time
FROM system.lakeflow.job_task_run_timeline
WHERE result_state IN ('FAILED', 'ERROR', 'TIMED_OUT')
  AND period_start_time > current_timestamp() - INTERVAL 7 DAYS
ORDER BY period_start_time DESC;
```

`termination_code`	Typical next step
`RUN_EXECUTION_ERROR`	Task code failed: read the task error, fix code or input, repair
`CLUSTER_ERROR`, `DRIVER_ERROR`	Compute problem: check the compute event log and driver logs
`LIBRARY_INSTALLATION_ERROR`	Fix the library or environment spec, then repair
`UNAUTHORIZED_ERROR`	Grant the run-as identity the missing permission
`STORAGE_ACCESS_ERROR`	Fix the storage credential or external location
`REPOSITORY_CHECKOUT_FAILED`	Git source unreachable: fix the Git credential or URL
`WORKSPACE_RUN_LIMIT_EXCEEDED`	Too many concurrent active runs: spread the schedule

Important

For runs longer than one hour, the timeline tables split the run into hourly slices, and `result_state` and `termination_code` are filled in only on the final slice. Filter on non-null values when you count failures.

Remediating the Cause Before You Repair

Repaired tasks run with the **current** job and task settings. Fix first, repair second.

Root cause	Remediation
Wrong task configuration (path, parameters, dependencies)	**Edit task**, change the settings, **Save task**
Bug in the notebook or script	Fix the code the task points at, then repair
Not enough compute	Under **Job details** > **Compute** > **Configure**, c
Run blocked by **Maximum concurrent runs**	Wait for other runs to finish, or raise the limit
External source was unavailable	Repair after the source recovers
Wrong runtime value (date, catalog, flag)	Repair with an overridden job parameter

---

Repair Run Semantics

What a Repair Does

Aspect	Behavior
Tasks re-run	Only the **unsuccessful** tasks and their **dependent** tasks
Successful tasks	**Not** re-run
Settings used	The **current** job and task settings, so edits made before the repair apply
Where it is recorded	Inside the **original run**: the matrix view gets a new column for the repair, and `repair_`
Task restart point	Each re-run task starts **from the beginning**
Shared job cluster	The repair creates a new job cluster with a `_v1` suffix (for example `my_job_cluster_v1`
Minimum job size	Two or more tasks. For a failed **single-task** job, use **Run now**
Run duration	Measured from the first run's start to the latest repair's finish
Disabled tasks	Chosen by run state, not disabled state. To force a disabled task to run, name it in `reru`
Run state	The run must **not** be in progress when you repair it

Repairs are not idempotent by themselves

Lakeflow Jobs does not make tasks idempotent. If a task wrote part of its output before failing, a repair can **duplicate** that data. Write with `MERGE`, overwrite, or other idempotent patterns wherever duplicates matter.

Example: Which Tasks a Repair Re-runs

```
flowchart LR
    I["ingest OK"] --> A["transform_a OK"]
    I --> B["transform_b X FAILED"]
    A --> P["publish → UPSTREAM_FAILED"]
    B --> P
```

Task	Original run	Re-run by default repair?	Why
`ingest`	Succeeded	No	Successful tasks are kept
`transform_a`	Succeeded	No	Successful tasks are kept
`transform_b`	Failed	**Yes**	Unsuccessful
`publish`	Skipped (upstream failed)	**Yes**	Unsuccessful dependent of `transform_`

Repair in the UI

1. Open the **Job run details** page (from the **Start time** column or the matrix view).
2. Click **Repair run**. The **Repair job run** dialog lists the tasks that will re-run.
3. Optionally add or edit **parameters**. These override the existing values for this repair.
4. Click **Repair run** to confirm.
5. Check the new matrix column. Repaired tasks that were red should turn green.

To restore an original value in a later repair, clear that parameter's key and value in the dialog.

Continuous Jobs Continuous jobs do not use repair. After consecutive failures pass a threshold, Lakeflow Jobs retries them with exponential backoff. **Restart run** cancels the active run, resets the retry period, and starts a new run.

Parameter Overrides

Job Parameters vs. Task Parameters vs. Bundle Variables

	Job parameters
Defined on	The job
Resolved	At **run time**
Overridable per run	Yes: **Run now with different settings**, **Repair run**, API `job_parameters`
Precedence	**Wins** over a task parameter with the same key
Reaching tasks	Pushed down automatically to key-value tasks; JSON-array tasks need `{{job.parameter`

- Job parameter keys may contain only alphanumeric characters, `_`, `-`, and `.`. Values can be strings or dynamic value references, and any valid JSON, including arrays.
- Bundle variables are deployment-time values. Use **job parameters** for values that change from run to run.
- If a job has only task parameters, the run dialog offers **Switch to legacy parameters**. Databricks no longer recommends this.

Reading Parameters Inside a Task

Task type	How the code reads parameters
Notebook (Python)	`dbutils.widgets.get("run_date")`
Notebook (SQL)	Named parameter marker `:run_date`
Python script	Command-line arguments (`sys.argv` or `argparse`)
Python wheel	Keyword arguments to the entry point
JAR / Spark Submit	Arguments to the main class's `main` method
SQL task	`:param_name` in the SQL; use `{{job.parameters.<name>}}` in the task configuration
Pipeline	Pipeline task **Parameters** box, read with named parameters

```python
# Notebook task: read a job parameter that is pushed down by name
dbutils.widgets.text("run_date", "")  # default for interactive runs only
run_date = dbutils.widgets.get("run_date")

if not run_date:
    raise ValueError("run_date job parameter is required")

df = spark.table("main.silver.orders").where(f"order_date = '{run_date}'")
```

Note

A standalone notebook run does not receive job parameters, so a widget default keeps interactive runs working. The docs warn that a default can also **mask** a misconfigured parameter in a job, so validate required values explicitly.

Dynamic Value References Useful in Repairs

Reference	Value
`{{job.parameters.<name>}}`	Current value of job parameter <name>, including an overri
`{{job.run_id}}`	ID of the job run (unchanged across repairs)
`{{job.repair_count}}`	Number of repair attempts on the current run
`{{task.execution_count}}`	Times this task has run, counting retries **and** repairs

Reference	Value
{{tasks.<task_name>.result_state}}	Upstream task outcome, such as success or failed
{{tasks.<task_name>.error_code}}	Upstream error code, such as RunExecutionError (empty
{{tasks.<task_name>.values.<value_name>}}	Task value set by an upstream task

Old forms such as {{run_id}}, {{task_retry_count}}, and {{parent_run_id}} are deprecated. Use {{task.run_id}}, {{task.execution_count}}, and {{job.run_id}}. A malformed reference is treated as a literal string, which is an easy silent bug.

Run Now with Different Settings Use this for a **new** run, for example a backfill for one date or a re-run of a single-task job.

1. Click the caret next to **Run now** and select **Run now with different settings**.
2. Optionally select or clear tasks in the graph (affects only this run).
3. Optionally enter job parameters as key-value pairs. These override the configured values or add new ones.
4. Optionally change **Performance optimized** (serverless).
5. Click **Run**.

Subsets of tasks use the only field with + expansion: my_task (only this task), +my_task (plus upstream), my_task+ (plus downstream), +my_task+ (both). The + expansion skips disabled tasks unless you name them.

```
# Jobs CLI: new run with an overridden job parameter.
# With --json, job_id goes in the body and no positional argument is allowed.
databricks jobs run-now --json '{
  "job_id": 11223344,
  "job_parameters": {"run_date": "2026-10-25"},
  "only": ["+publish"]
}'

# Bundle-deployed job: --params takes comma-separated key=value pairs
databricks bundle run nightly_etl -t prod --params run_date=2026-10-25 --only
 ↪  +publish
```

Retries, Timeouts, and Repair

Mechanism	Triggered by
Retry policy (max_retries, min_retry_interval_millis, retry_on_timeout)	Automatic, when a task fail
Timeout (timeout_seconds, or a **Run duration** metric threshold)	Automatic, when time runs
Repair run	You (UI, CLI, API, SDK) after
Run now (optionally different settings)	You
Restart run	You

Retry facts from the Jobs API and task docs:

- max_retries: 0 = never retry (the default), -1 = retry indefinitely. A run counts as unsuccessful if it ends FAILED or INTERNAL_ERROR.
- min_retry_interval_millis: time from the **start** of the failed run to the retry. By default, unsuccessful runs are retried immediately.
- retry_on_timeout: default false, so timeouts are not retried unless you enable it.
- With both a timeout and retries, the timeout applies to **each** retry.
- timeout_seconds: 0 means no timeout.

- Serverless jobs auto-optimize retries by default. Continuous jobs use exponential backoff.

```
flowchart TD
    E[Task failed] --> T{Transient?}
    T -->|Yes| R[Retry policy handles it automatically]
    T -->|No| F[Fix code, config, compute, or input]
    F --> N{How many tasks in the job?}
    N -->|One| RN[Run now, with different settings if needed]
    N -->|Two or more| RP[Repair run, with parameter overrides if needed]
```

Jobs API: `runs/repair`

`POST /api/2.2/jobs/runs/repair` re-runs one or more tasks as part of the original job run. The tasks use the current job and task settings, and the repair appears in the original run's history.

Field	Type	Meaning
`run_id`	int64, **required**	Job run to repair. Must not be in progress
`latest_repair_id`	int64	ID of the latest repair. Not needed for the first repair; **r**
`rerun_tasks`	array of string	Task keys to repair
`rerun_all_failed_tasks`	boolean (default `false`)	Repair all failed tasks. Use **either** this **or** `rerun_tasks`
`rerun_dependent_tasks`	boolean (default `false`)	Also repair every task that depends on the tasks being
`job_parameters`	object	Job-level parameter overrides for the repair, for examp
`pipeline_params`	object	For pipeline tasks: `full_refresh`, `refresh_selection`
`performance_target`	string	Serverless performance mode for this repair (`PERFORMA`

The response is `{"repair_id": <int64>}`. Pass it as `latest_repair_id` in the next repair so repairs stay sequential.

```
{
  "run_id": 455644833,
  "latest_repair_id": 734650698524280,
  "rerun_tasks": ["transform_b"],
  "rerun_dependent_tasks": true,
  "job_parameters": {"run_date": "2026-10-25"}
}
```

Note

The Python SDK's `repair_run` still accepts the legacy per-task-type fields (`notebook_params`, `python_params`, `jar_params`, `sql_params`, `spark_submit_params`, `python_named_params`, `dbt_commands`). The 2.2 REST reference documents `job_parameters` for repairs. Use job parameters for new jobs.

CLI: `databricks jobs repair-run`

```
# 1. Inspect the run and its repair history
databricks jobs get-run 455644833 --include-history -o json

# 2. First repair: re-run every failed task and wait (default --timeout 20m)
databricks jobs repair-run 455644833 --rerun-all-failed-tasks

# 3. Later repair with specific tasks and a parameter override.
#    With --json, run_id goes in the body and no positional argument is allowed.
```

```
databricks jobs repair-run --json '{
  "run_id": 455644833,
  "latest_repair_id": 734650698524280,
  "rerun_tasks": ["transform_b"],
  "rerun_dependent_tasks": true,
  "job_parameters": {"run_date": "2026-10-25"}
}' --no-wait
```

Flag	Purpose
`--rerun-all-failed-tasks`	Repair all failed tasks
`--rerun-dependent-tasks`	Also repair tasks that depend on the repaired ones
`--latest-repair-id`	Required on second and later repairs
`--performance-target`	`PERFORMANCE_OPTIMIZED` or `STANDARD` (serverless)
`--json`	Full request body: the only way to pass `rerun_tasks` and `job_parameters`
`--no-wait` / `--timeout`	Return immediately, or wait up to the timeout for `TERMINATED` or `SKIPPED`

Python SDK

```python
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()
run_id = 455644833

# Find the latest repair ID (None if the run was never repaired)
run = w.jobs.get_run(run_id=run_id, include_history=True)
repair_ids = [
    item.id
    for item in (run.repair_history or [])
    if item.type is not None and item.type.value == "REPAIR"
]
latest_repair_id = repair_ids[-1] if repair_ids else None

# Repair failed tasks with a parameter override, then block until it finishes
repaired = w.jobs.repair_run(
    run_id=run_id,
    latest_repair_id=latest_repair_id,
    rerun_all_failed_tasks=True,
    job_parameters={"run_date": "2026-10-25"},
).result()

print(repaired.status)
```

`repair_run` returns a waiter (`Wait[Run]`). `.result()` blocks until the run reaches `TERMINATED` or `SKIPPED`. The SDK also offers `repair_run_and_wait(..., timeout=...)`.

Use Cases

- **Bad input partition**: A nightly job fails in `transform_b` because one day's file was late. After the file lands, a **repair** re-runs `transform_b` and `publish` without re-ingesting or re-transforming the successful branches.
- **Wrong date in a scheduled run**: The run used yesterday's default `run_date`. Repair with a `run_date` job parameter override instead of editing the job.

- **Undersized compute**: A task fails with out-of-memory errors. Increase workers or swap compute in the job, then repair. The shared job cluster comes back as <key>_v1 with the new settings.
- **Automated remediation**: An orchestrator watches `system.lakeflow.job_task_run_timeline` (or a failure notification) and calls `runs/repair` with `rerun_all_failed_tasks` once the upstream system reports healthy.

Common Issues & Errors

Symptom	Cause
No **Repair run** button for a failed job	Job has a single task
Repair API call rejected on the second attempt	Missing or stale `latest_repair_id`
Repair rejected immediately	Run is still in progress
Duplicate rows after a repair	Failed task had appended partial output
Override "did not apply" in a script task	Script takes a JSON array of args, so job params a
Task parameter value ignored	A job parameter with the same key wins
`rerun_tasks` and `rerun_all_failed_tasks` together fails	They are mutually exclusive
A disabled task is not picked up by the repair	Repair selects tasks by run state, not disabled sta
Notebook works interactively, fails in the job	Widget default masked a missing parameter

Best Practices

- Make every task safe to re-run (idempotent writes), because repair restarts tasks from the beginning.
- Use retry policies for transient failures and repairs for deterministic ones; do not retry indefinitely (`-1`) on code bugs.
- Prefer job parameters over legacy task parameters, and give required parameters explicit validation.
- Set a **Run duration** warning threshold so you are notified before a task times out.
- Use system tables for fleet-wide failure analysis and the run details page for a single run.

Exam Tips

Tip

- **Repair = unsuccessful tasks + their dependents, same run ID, current settings.** Successful tasks are not re-run.
- **Fix first, then repair**: edits to task settings, compute, or code are picked up by the repair.
- **Single-task job?** No repair. Use **Run now** (with different settings if you need overrides).
- **Overrides**: job parameters can be overridden on **Run now with different settings** and on **Repair run**; a job parameter beats a task parameter with the same key.
- **API fields**: `rerun_tasks` *or* `rerun_all_failed_tasks`; `rerun_dependent_tasks` adds downstream tasks even if they succeeded; `latest_repair_id` is required after the first repair; `job_parameters` carries overrides.
- **Retries vs repair**: retries are automatic and for transient errors (`max_retries` default `0`; `retry_on_timeout` default `false`); repair is a deliberate re-run after a fix.
- **Duplicates after repair** point to non-idempotent writes, not to a platform bug.

Key Takeaways

- **Triage path**: matrix view, then the first red task's **Task run details** (output, error, metadata, compute logs), and `get-run-output` for programmatic access.

- **Repair scope**: only unsuccessful tasks and their dependents re-run, inside the original run, using the current job and task settings.
- **Repair history**: each repair adds a matrix column and a REPAIR entry in `repair_history`; the next API repair must send `latest_repair_id`.
- **Idempotency**: each repaired task restarts from scratch, so partial writes can duplicate unless the task uses idempotent operations.
- **Parameter overrides**: job parameters are run-time values, pushed down to key-value tasks, overridable at run and repair time; bundle variables are deployment-time values.
- **Right tool**: retries for transient errors, repair for fixed deterministic errors in multi-task jobs, Run now for single-task jobs and fresh runs, Restart run for continuous jobs.
- **Automation**: `POST /api/2.2/jobs/runs/repair`, `databricks jobs repair-run`, or `w.jobs.repair_run(...).result()`.

Related Topics

- Spark UI Debugging - Diagnostic sources for the errors you find here
- Lakeflow Jobs — Part 2 - Job parameters, retries, and notifications
- System Tables - Querying `system.lakeflow` run history
- Databricks CLI — Part 1 - CLI setup and command groups
- REST API — Part 1 - Calling the Jobs API
- Declarative Automation Bundles — Part 1 - `databricks bundle run --params`

Official Documentation

- Troubleshoot and repair job failures
- Jobs API: Repair a job run
- Configure job parameters
- Access parameter values from a task
- What is a dynamic value reference?
- Trigger a single job run
- Configure and edit tasks (retries, duration thresholds)
- Monitoring and observability for Lakeflow Jobs
- Jobs system table reference
- `jobs` command group (CLI)
- Databricks SDK for Python: `w.jobs`

Declarative Automation Bundles — Part 1

Overview

Declarative Automation Bundles (formerly Databricks Asset Bundles) describe Databricks resources such as jobs, pipelines, and clusters as code in a `databricks.yml` project, then validate and deploy them with the Databricks CLI. One bundle can target dev, staging, and prod with per-target overrides. This part covers bundle structure, resource definitions, variables, commands, targets, deployment modes, presets, permissions, run_as, and artifacts.

Overview

- A bundle = `databricks.yml` (plus `include`d YAML files) + source code, deployed with `databricks bundle deploy -t <target>`
- `targets` hold per-environment overrides; `mode: development` and `mode: production` apply opinionated defaults, and `presets` customize them
- `variables` (`${var.name}`) and substitutions (`${bundle.target}`,

`${workspace.current_user.short_name}`) are resolved at **deployment** time
- `run_as` and `permissions` separate the identity that runs a job from the one that deployed it

What the Exam Tests

- "Deploy Databricks resources using Declarative Automation Bundles (formerly Databricks Asset Bundles)."
- Deploying the same job and pipeline definitions identically to dev, staging, and prod from source control with per-target overrides
- Knowing the core commands (`init`, `validate`, `deploy`, `run`, `destroy`, `summary`) and what development vs. production mode change
- Choosing `run_as` (service principal) and `permissions` for production targets

Platform update (Oct 2026)

Databricks Asset Bundles was renamed **Declarative Automation Bundles** on March 16, 2026. The CLI command group is unchanged: `databricks bundle ...`. Source: Declarative Automation Bundles feature release notes

---

Bundle Lifecycle

```
flowchart TB
    subgraph Development["Source control"]
        Code[Source code]
        Config[databricks.yml]
        Resources[resources/*.yml]
    end

    subgraph Bundle["databricks bundle ..."]
        Validate[validate]
        Plan[plan]
        Deploy[deploy -t target]
        Run[run]
        Destroy[destroy]
    end

    subgraph Targets["Targets"]
        Dev[dev: mode development]
        Staging[staging]
        Prod[prod: mode production]
    end

    Development --> Validate
    Validate --> Plan
    Plan --> Deploy
    Deploy --> Targets
    Deploy --> Run
```

Bundle Structure

Standard Project Layout

```
my-databricks-project/
├── databricks.yml               # Main bundle configuration (bundle root)
├── resources/
│   ├── jobs.yml                # Job definitions
│   ├── pipelines.yml           # Lakeflow pipeline definitions
│   └── clusters.yml            # Cluster definitions
├── src/
│   ├── notebooks/              # Notebook source files
│   │   ├── bronze/
│   │   ├── silver/
│   │   └── gold/
│   ├── pipelines/              # Pipeline source files
│   └── python/                 # Python package built as a wheel
├── tests/
│   ├── unit/
│   └── integration/
└── .databricks/                # Local CLI working files: keep out of Git
```

Top-Level Mappings

Mapping	Purpose
`bundle`	Bundle name (required), optional `git`, `databricks_cli_version`, deployment settings
`include`	Globs of extra YAML files, relative to the file that lists them
`variables`	Custom variables, referenced as `${var.<name>}`
`workspace`	`host`, `profile`, `root_path`, `artifact_path`, `file_path`, `state_path`
`artifacts`	Local builds to run and upload on deploy (`whl`, `jar`, `tgz`)
`resources`	Jobs, pipelines, clusters, schemas, volumes, dashboards, apps, and more
`permissions`	Permissions applied to every resource in the bundle
`run_as`	Identity that jobs and pipelines run as
`sync`	`include` / `exclude` / `paths` for files uploaded to the workspace
`scripts`	Commands you can run with `bundle run`
`python`	Load resources defined in Python (`databricks-bundles` package)
`targets`	Per-environment overrides; exactly one may set `default: true`

Default workspace paths: `root_path` is `/Workspace/Users/${workspace.current_user.userName}/.bundle`. Under it, `artifact_path` defaults to `${workspace.root}/artifacts`, `file_path` to `${workspace.root}/files` and `state_path` to `${workspace.root}/state`.

Core Configuration File

```yaml
# databricks.yml

bundle:
  name: my-etl-project

include:
  - resources/*.yml

variables:
  environment:
    description: "Deployment environment"
    default: dev
```

```yaml
  catalog:
    description: "Unity Catalog name"
    default: dev_catalog

workspace:
  host: https://adb-1234567890.1.azuredatabricks.net

artifacts:
  etl_package:
    type: whl
    path: ./src/python
    build: poetry build

# Applied to every resource in the bundle
permissions:
  - level: CAN_VIEW
    group_name: data-analysts
  - level: CAN_MANAGE
    group_name: data-engineers

targets:
  dev:
    mode: development
    default: true
    variables:
      environment: dev
      catalog: dev_catalog

  staging:
    variables:
      environment: staging
      catalog: staging_catalog
    workspace:
      root_path: /Workspace/Shared/.bundle/${bundle.name}/${bundle.target}
    presets:
      name_prefix: "staging_"
      trigger_pause_status: PAUSED

  prod:
    mode: production
    variables:
      environment: prod
      catalog: prod_catalog
    workspace:
      root_path: /Workspace/Shared/.bundle/${bundle.name}/${bundle.target}
    run_as:
      # Service principal application ID
      service_principal_name: "00000000-0000-0000-0000-000000000000"
```

Resource Definitions

Job Configuration

```yaml
# resources/jobs.yml

resources:
```

```yaml
jobs:
  etl_daily_job:
    name: "ETL Daily Pipeline - ${var.environment}"
    schedule:
      quartz_cron_expression: "0 0 6 * * ?"
      timezone_id: "UTC"

    email_notifications:
      on_failure:
        - data-team@company.com

    tags:
      environment: ${var.environment}
      team: data-engineering

    # Job parameters: pushed down to every notebook task, and
    # overridable on Run now and Repair run without redeploying
    parameters:
      - name: catalog
        default: ${var.catalog}            # deploy-time default
      - name: run_date
        default: "{{job.start_time.iso_date}}"

    tasks:
      - task_key: bronze_ingestion
        notebook_task:
          # Relative to THIS file (resources/jobs.yml)
          notebook_path: ../src/notebooks/bronze/ingest.py
        job_cluster_key: etl_cluster

      - task_key: silver_transform
        depends_on:
          - task_key: bronze_ingestion
        notebook_task:
          notebook_path: ../src/notebooks/silver/transform.py
        job_cluster_key: etl_cluster

      - task_key: gold_aggregate
        depends_on:
          - task_key: silver_transform
        notebook_task:
          notebook_path: ../src/notebooks/gold/aggregate.py
        job_cluster_key: etl_cluster

    job_clusters:
      - job_cluster_key: etl_cluster
        new_cluster:
          spark_version: "15.4.x-scala2.12"
          node_type_id: "Standard_DS3_v2"
          num_workers: 2
```

Relative paths and parameters

Paths in a resource file resolve relative to the configuration file **that declares them**, not the bundle root. The old fallback lookup was removed in CLI 0.266.0, so a wrong relative path is now

an error. Bundle validation does not allow job-level `parameters` and task `base_parameters` in the same job: once a job has job parameters, move task values up to the job level.

Lakeflow Pipeline Configuration

```yaml
# resources/pipelines.yml

resources:
  pipelines:
    streaming_pipeline:
      name: "Streaming ETL - ${var.environment}"
      catalog: ${var.catalog}
      schema: ${var.environment}_streaming   # `target` is the deprecated legacy
↪  field
      serverless: true                       # recommended for new pipelines; omit
↪  `clusters`

      configuration:
        environment: ${var.environment}

      libraries:
        - glob:
            include: ../src/pipelines/streaming/**

      continuous: false

      notifications:
        - email_recipients:
            - data-team@company.com
          alerts:
            - on-update-failure
            - on-flow-failure
```

You do not set `development: ` per environment yourself. `mode: development` marks pipelines as development pipelines, and `mode: production` checks that they are not.

Cluster Configuration

```yaml
# resources/clusters.yml

resources:
  clusters:
    shared_analytics:
      cluster_name: "Analytics Cluster - ${var.environment}"
      spark_version: "15.4.x-scala2.12"
      node_type_id: "Standard_DS4_v2"
      autoscale:
        min_workers: 1
        max_workers: 8
      autotermination_minutes: 30
      custom_tags:
        environment: ${var.environment}
```

Other resource types include `schemas`, `volumes`, `dashboards`, `apps`, `model_serving_endpoints`, `registered_models`, `quality_monitors`, `sql_warehouses`, and more (see the resources reference). A resource-level `lifecycle.prevent_destroy: true` protects it from `bundle destroy`.

Variables and Substitutions

Custom Variables

```yaml
# databricks.yml

variables:
  # Simple variable with default
  environment:
    default: dev

  # No default: must be supplied at deploy time
  notification_email:
    description: "Alert recipient"

  # Lookup: resolve an object's ID from its name
  etl_pool_id:
    lookup:
      instance_pool: etl-pool

  # Complex variable (maps or lists)
  default_cluster:
    type: complex
    default:
      spark_version: "15.4.x-scala2.12"
      node_type_id: "Standard_DS3_v2"
      num_workers: 2
```

Lookup types include `alert`, `cluster_policy`, `cluster`, `dashboard`, `instance_pool`, `job`, `metastore`, `notification_destination`, `pipeline`, `query`, `service_principal`, and `warehouse`. A lookup fails if the name matches no object or more than one.

Where a Variable's Value Comes From (first match wins)

1. `--var="<name>=<value>"` on the `bundle` command
2. An environment variable `BUNDLE_VAR_<name>`
3. `.databricks/bundle/<target>/variable-overrides.json`
4. The `variables` mapping inside the selected target
5. The variable's top-level `default`

Note

Bundle variables are **deployment-time** values. For values that change per run, use **job parameters**, which can be overridden on **Run now** and **Repair run**.

Built-in Substitutions

Substitution	Value
`${bundle.name}`	Bundle name
`${bundle.target}`	Selected target (`dev`, `prod`, ...)
`${workspace.current_user.userName}`	Deploying user's user name (email)
`${workspace.current_user.short_name}`	Short form of the user name (used in the dev-mode prefix)
`${workspace.root_path}`	Bundle root folder in the workspace
`${workspace.file_path}`	Where synced source files are deployed
`${resources.jobs.<job-name>.id}`	ID of a job defined in the bundle

Substitution	Value
`${resources.pipelines.<name>.<field>}`	Any field of a bundle resource, e.g. `.schema`

Variable Substitution Example

```yaml
resources:
  jobs:
    my_job:
      name: "Job - ${var.environment}"
      tasks:
        - task_key: main
          notebook_task:
            notebook_path: ../src/notebooks/main.py
            base_parameters:
              catalog: ${var.catalog}
              config_path: ${workspace.file_path}/config/${bundle.target}.yml
              bundle_name: ${bundle.name}
```

Bundle Commands

Core Commands

```
# Initialize a new bundle from a template (prompts if no template given)
databricks bundle init default-python

# Check the configuration is syntactically valid (-o json for JSON output)
databricks bundle validate -t prod -o json

# Preview what deploy would create, update, or delete
databricks bundle plan -t prod

# Deploy to the default target, or to a named one
databricks bundle deploy
databricks bundle deploy -t prod

# Run a job (or pipeline, app, or script) defined in the bundle
databricks bundle run -t prod etl_daily_job

# Run with job parameters: comma-separated key=value pairs
databricks bundle run -t prod etl_daily_job --params run_date=2026-10-25

# Show deployed resources and their workspace URLs
databricks bundle summary -t prod

# Delete everything this bundle deployed (cannot be undone)
databricks bundle destroy -t dev
```

Command	Notes
`init [TEMPLATE]`	Default templates: `default-python`, `default-minimal`, `default`
`validate`	Syntax check; warns on properties not in the resource schema
`plan`	Shows planned changes; `deploy --plan` replays a saved plan (di
`deploy`	Builds artifacts, uploads files, creates or updates resources. Flags
`run [KEY]`	Jobs: `--params`, `--only`, `--restart`, `--no-wait`. Pipelines: `--ful`
`destroy`	`--auto-approve`, `--force-lock`; skips resources with `lifecycle`

Command	Notes
`summary`	Lists deployed resources for the target
`deployment bind KEY RESOURCE_ID`	Attach an existing workspace resource (for example a job) to a bu
`generate job --existing-job-id <id>`	Write bundle YAML for an existing job (`pipeline`, `dashboard`, app

Validation and Debugging

```
# Supply a variable without a default
databricks bundle validate -t prod --var="notification_email=ops@company.com"

# Verbose logs for a failing deploy (global flag)
databricks bundle deploy -t prod --debug
```

Target Configuration

Development vs. Production Modes

Behavior	`mode: development`	`mode:`
Resource names	Prefixed with `[dev ${workspace.current_user.short_name}]`	Uncha
Tags	Adds a dev tag to jobs and pipelines	None a
Schedules and triggers	**Paused** (set `pause_status: UNPAUSED` to override)	As con
Pipelines	Marked `development: true`	Deploy
Concurrent job runs	Enabled	As con
Cluster override (`--cluster-id`)	Allowed	**Not al**
Deployment lock	Disabled (enable with `bundle.deployment.lock.enabled: true`)	Enable
Git branch check	None	Curren
Identity	Deploying user	Servic

Presets `presets` in a target customize behavior with or without a `mode`. Presets override mode defaults, and settings on an individual resource override presets.

Preset	Effect
`name_prefix`	Prefix for resource names
`pipelines_development`	Lock pipeline deployments in development mode
`trigger_pause_status`	Pause status for all triggers and schedules (always `PAUSED` in development mo
`jobs_max_concurrent_runs`	Max concurrent runs for all jobs
`tags`	Tags applied to all resources that support tags
`source_linked_deployment`	Resources point at workspace source files instead of deployed copies
`artifacts_dynamic_version`	Patch `whl` versions dynamically on each deploy

Environment-Specific Overrides

```
targets:
  dev:
    resources:
      jobs:
        etl_daily_job:
          job_clusters:
            - job_cluster_key: etl_cluster
              new_cluster:
                num_workers: 1
```

```yaml
  prod:
    resources:
      jobs:
        etl_daily_job:
          job_clusters:
            - job_cluster_key: etl_cluster
              new_cluster:
                num_workers: 8
                driver_node_type_id: "Standard_DS5_v2"
```

Target-level `resources` merge into the top-level definitions by join key (`job_cluster_key` for job clusters, `label` for pipeline clusters, `task_key` for tasks). Conflicting settings take the target's value; everything else is kept, so you restate only what changes.

`run_as` and `permissions`

```yaml
targets:
  prod:
    mode: production
    run_as:
      service_principal_name: "00000000-0000-0000-0000-000000000000"  #
↪  application ID
    permissions:
      - level: CAN_MANAGE
        group_name: data-engineers
      - level: CAN_RUN
        service_principal_name: "00000000-0000-0000-0000-000000000000"
```

- `run_as` takes `user_name` or `service_principal_name`. Non-admins can set only their own user name.
- If the deployer differs from the `run_as` identity, only **jobs and pipelines** are supported in the bundle. `run_as` is not supported with model serving endpoints.
- To deploy with a service principal as `run_as`, deploy as that service principal or hold **CAN_USE** on it.
- Top-level and target `permissions` allow `CAN_VIEW`, `CAN_MANAGE`, and `CAN_RUN`, each with `user_name`, `group_name`, or `service_principal_name`.

Artifacts and Libraries

Python Wheel Artifacts

```yaml
# databricks.yml
artifacts:
  etl_package:
    type: whl
    path: ./src/python       # relative to databricks.yml
    build: poetry build      # default when omitted: python setup.py bdist_wheel

# resources/jobs.yml
resources:
  jobs:
    wheel_job:
      tasks:
        - task_key: main
          job_cluster_key: etl_cluster
          python_wheel_task:
```

```
      package_name: etl_package
      entry_point: main
    libraries:
      - whl: ../src/python/dist/*.whl   # relative to this file
```

On serverless compute, Python wheel tasks need an `environment_key` that points at a job `environments` entry instead of `libraries`.

JAR Artifacts

```
artifacts:
  spark_extensions:
    type: jar
    path: ./libs
    build: mvn package -DskipTests
    files:
      - source: ./libs/target/spark-extensions-1.0.jar

resources:
  jobs:
    jar_job:
      tasks:
        - task_key: main
          job_cluster_key: etl_cluster
          spark_jar_task:
            main_class_name: com.company.Main
          libraries:
            - jar: ../libs/target/spark-extensions-1.0.jar
```

> **Continue reading:** Part 2 — Sync, State, CI/CD, Templates, Common Patterns & Exam Tips

---

Declarative Automation Bundles — Part 2

Overview

This part covers file sync and deployment state, running bundles from CI/CD, bundle templates, common patterns, and the errors you hit most often when deploying Declarative Automation Bundles.

> For bundle structure, resource definitions, variables, commands, targets, modes, presets, `run_as`, and artifacts, see Part 1.

Overview

- `sync` decides which local files reach the workspace (`.gitignore` syntax, applied on top of the bundle's `.gitignore`)
- Deployment state lives in the workspace under `workspace.state_path`; the **direct deployment engine** replaced Terraform as the default
- CI/CD runs `databricks bundle validate` → `deploy -t <target>` → `run`, authenticated as a service principal (ideally with workload identity federation)
- `databricks bundle init` starts from a default or custom template

What the Exam Tests

- Deploying the same bundle to dev, staging, and prod from source control with CI/CD
- Recognizing the right fix for common deploy failures (missing variables, permissions, existing resources, stale locks)
- Knowing that bundles, not manual UI recreation or workspace imports, give reproducible multi-environment deployment

---

Sync and State Management

Sync Configuration

```yaml
# databricks.yml

sync:
  include:
    - "src/**"
    - "resources/**"
  exclude:
    - "**/__pycache__"
    - "*.pyc"
    - "tests/**"
```

- `include` and `exclude` use standard `.gitignore` pattern syntax.
- `exclude` is applied **on top of** the bundle root's `.gitignore`, so files Git ignores are not synced by default.
- Patterns only filter files inside folders covered by `sync.paths`. They cannot pull in files from outside those paths.
- For one-way, local-to-workspace syncing during development, use `databricks bundle sync --watch`.

Deployment State

```
Workspace (source of truth for deployments):
${workspace.root_path}/
├── artifacts/     # artifact_path: built wheels and JARs
├── files/         # file_path: synced source files
└── state/         # state_path: deployment state for this target

Local working directory:
.databricks/       # CLI cache and per-target files: add to .gitignore
```

- Each target has its own `root_path`, so dev, staging, and prod keep separate state.
- Because state is stored in the workspace, a different machine (for example a CI runner) can deploy the same target. Use `databricks bundle summary --force-pull` to refresh from the remote state.
- `bundle deploy` takes a **deployment lock** so concurrent deploys do not interfere (development mode disables it). `--force-lock` should be used only when a crashed or interrupted deploy left a stale lock.

Deployment engine

The **direct deployment engine** (built on the Databricks Go SDK, no Terraform) became GA on June 10, 2026 and is the default for bundles created with CLI 1.3.0 and above. CLI 1.14+

auto-migrates Terraform-based bundles on deploy, and the Terraform engine was removed in CLI 1.20.0. One visible change: removing a field from YAML reverts it to the resource default on the next deploy. Sources: Declarative Automation Bundles feature release notes, Direct deployment engine

Integration with CI/CD

GitHub Actions Workflow (Workload Identity Federation)

```yaml
# .github/workflows/deploy.yml

name: Deploy Bundle

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  id-token: write   # lets the job request a GitHub OIDC token
  contents: read

env:
  DATABRICKS_AUTH_TYPE: github-oidc
  DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
  DATABRICKS_CLIENT_ID: ${{ vars.DATABRICKS_SP_CLIENT_ID }}   # service principal
 ↪ application ID

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - name: Validate bundle
        run: databricks bundle validate -t staging

  deploy-staging:
    needs: validate
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - name: Deploy to staging
        run: databricks bundle deploy -t staging
      - name: Run integration tests
        run: databricks bundle run -t staging integration_tests

  deploy-prod:
    needs: deploy-staging
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
```

```
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - name: Deploy to production
        run: databricks bundle deploy -t prod
```

- With `DATABRICKS_AUTH_TYPE: github-oidc`, the CLI exchanges the GitHub workload identity token for a Databricks OAuth token. No PAT or client secret is stored in GitHub.
- The service principal needs a **federation policy** whose issuer is `https://token.actions.githubusercon` and whose subject matches the token, for example `repo:my-org/my-repo:environment:production`. Databricks recommends the GitHub **environment** entity type over a branch.
- Use a separate service principal (and policy) per environment for least privilege. See Bundle Deployment Strategies — Part 2 for the policy setup.

Azure DevOps Pipeline (OAuth M2M)

```
# azure-pipelines.yml

trigger:
  branches:
    include:
      - main

pool:
  vmImage: 'ubuntu-latest'

stages:
  - stage: Validate
    jobs:
      - job: ValidateBundle
        steps:
          - script: |
              curl -fsSL
↪   https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
            displayName: 'Install Databricks CLI'

          - script: databricks bundle validate -t staging
            displayName: 'Validate Bundle'
            env:
              DATABRICKS_HOST: $(DATABRICKS_HOST)
              DATABRICKS_CLIENT_ID: $(DATABRICKS_CLIENT_ID)
              DATABRICKS_CLIENT_SECRET: $(DATABRICKS_CLIENT_SECRET)

  - stage: DeployProd
    dependsOn: Validate
    condition: and(succeeded(), eq(variables['Build.SourceBranch'],
↪   'refs/heads/main'))
    jobs:
      - deployment: DeployProduction
        environment: production
        strategy:
          runOnce:
            deploy:
              steps:
                - script: databricks bundle deploy -t prod
```

```
    env:
      DATABRICKS_HOST: $(PROD_DATABRICKS_HOST)
      DATABRICKS_CLIENT_ID: $(PROD_DATABRICKS_CLIENT_ID)
      DATABRICKS_CLIENT_SECRET: $(PROD_DATABRICKS_CLIENT_SECRET)
```

`DATABRICKS_CLIENT_ID` + `DATABRICKS_CLIENT_SECRET` select OAuth machine-to-machine (`oauth-m2m`) authentication for a service principal. `DATABRICKS_TOKEN` (a personal access token) still works, but the docs mark PAT authentication as legacy. Azure DevOps can also use workload identity federation (`DATABRICKS_AUTH_TYPE: azure-devops-oidc`).

Bundle Templates

Default and Custom Templates

```
# Interactive: choose from the default templates
databricks bundle init

# A specific default template
databricks bundle init default-python

# A custom template from Git or a local path
databricks bundle init https://github.com/company/bundle-template

# Custom template whose schema file is in a subfolder of the repo
databricks bundle init https://github.com/company/templates --template-dir
 ↪  dab-container-template

# Non-interactive (CI): answer the prompts from a JSON file
databricks bundle init default-python --config-file ./init-params.json
 ↪  --output-dir ./new-project
```

Template	What you get
`default-python`	A job and an ETL pipeline (requires uv)
`default-minimal`	Only the required files and catalog variables, no sample code
`default-sql`	A job that runs SQL on a SQL warehouse
`default-scala`	A Scala JAR set up for serverless
`dbt-sql`	dbt-core locally, deployed as a job with a dbt task
`mlops-stacks`	Full-stack MLOps Stacks project
`pydabs`	Like `default-python`, but bundle configuration written in Python

Custom Template Structure

```
my-template/
├── databricks_template_schema.json   # prompts shown to the user
├── library/                          # optional template helpers
└── template/
    ├── databricks.yml.tmpl
    ├── resources/
    │   └── {{.project_name}}_job.yml.tmpl
    └── src/
        └── {{.project_name}}/
            └── main.py.tmpl
```

Templates use Go `text/template` syntax (`{{.project_name}}`), including in file and folder names.

Template Schema

```
{
  "properties": {
    "project_name": {
      "type": "string",
      "description": "Project name",
      "default": "my_project"
    },
    "include_pipeline": {
      "type": "string",
      "description": "Include a Lakeflow pipeline (yes/no)",
      "default": "no"
    }
  }
}
```

Common Patterns

Multi-Environment Deployment

```
# databricks.yml

variables:
  environment:
    default: dev

targets:
  dev:
    mode: development
    default: true
    variables:
      environment: dev
    workspace:
      host: https://dev.cloud.databricks.com

  staging:
    variables:
      environment: staging
    workspace:
      host: https://staging.cloud.databricks.com

  prod:
    mode: production
    variables:
      environment: prod
    workspace:
      host: https://prod.cloud.databricks.com
    run_as:
      service_principal_name: "00000000-0000-0000-0000-000000000000"
```

The same resources deploy to every target; only the target overrides differ. This is the pattern behind ”deploy identically to dev, staging, and prod with environment-specific overrides.”

Shared Resources Across Teams

```yaml
# Include shared configurations (globs are relative to this file)

include:
  - ./shared/common-clusters.yml
  - ./shared/notification-settings.yml
  - ./team-specific/*.yml
```

Parameterized Notebooks

```python
# Notebook task: read a job parameter defined in the bundle's job `parameters`
dbutils.widgets.text("catalog", "dev_catalog")  # default for interactive runs
 ↪  only
catalog = dbutils.widgets.get("catalog")

spark.sql(f"USE CATALOG {catalog}")
```

Use a job parameter whose default is `${var.catalog}` so the value is set per target at deploy time but can still be overridden per run.

Use Cases

- **Environment promotion**: define dev, `staging`, and `prod` targets in `databricks.yml` and deploy the same pipeline configuration to each workspace with `databricks bundle deploy -t <target>`.
- **Infrastructure as code**: keep job schedules, cluster definitions, and Lakeflow pipeline settings in the same Git repository as the code, reviewed through pull requests.
- **Adopting existing jobs**: run `databricks bundle generate job --existing-job-id <id>` to write YAML for a job built in the UI, then `deployment bind` it so the bundle manages it.
- **Per-developer sandboxes**: `mode: development` gives every engineer a prefixed, paused copy of the same jobs and pipelines.

Common Issues & Errors

Bundle Validation Fails **Scenario:** `databricks bundle validate` reports that a variable has no value.

```bash
# Provide the variable on the command line...
databricks bundle validate -t prod --var="notification_email=ops@company.com"

# ...or as an environment variable (BUNDLE_VAR_<name>)
export BUNDLE_VAR_notification_email=ops@company.com
databricks bundle validate -t prod
```

Do not use bundle variables for secrets. Reference secrets from a secret scope in the code instead.

Permission Denied on Deploy **Scenario:** Deploy fails with a permission error.

Fix: Check the deploying identity's workspace permissions, the `run_as` identity (the deployer needs CAN_USE on a `run_as` service principal), and the bundle `permissions`:

```yaml
targets:
  prod:
    run_as:
      service_principal_name: "00000000-0000-0000-0000-000000000000"
    permissions:
      - level: CAN_MANAGE
        group_name: data-engineers
```

Resource Already Exists **Scenario:** A job or pipeline with the same name already exists in the workspace, created outside the bundle.

Fix: Bind the existing resource to the bundle instead of creating a duplicate:

```
# Bind workspace job 123456789 to the bundle key etl_daily_job, then deploy
databricks bundle deployment bind etl_daily_job 123456789 -t prod
databricks bundle deploy -t prod

# Stop managing it from the bundle (the workspace job is kept)
databricks bundle deployment unbind etl_daily_job -t prod
```

Stale Deployment Lock **Scenario:** Deploy fails because a lock is held, after a previous CI run was killed.

Fix: Confirm nothing else is deploying, then:

```
databricks bundle deploy -t prod --force-lock
```

In CI, prevent the problem by serializing deploys per target (for example a GitHub Actions `concurrency` group).

Artifact Build Failure **Scenario:** The wheel build fails during `bundle deploy`.

Fix: Run the same build command locally first, and check the artifact's `path` and `build`:

```
cd src/python
poetry build
```

```
artifacts:
  etl_package:
    type: whl
    path: ./src/python   # relative to databricks.yml
    build: poetry build  # must succeed on the CI runner too
```

Exam Tips

Tip

- **Same definitions, many environments** → one `databricks.yml` with `targets` and per-target overrides, deployed with `databricks bundle deploy -t <target>`. Manually recreating jobs, workspace imports, or one shared workspace are the wrong answers.
- **Commands**: `validate` (syntax), `plan` (preview), `deploy` (create/update), `run` (trigger), `destroy` (delete), `summary` (list). The command group is still `databricks bundle` after the rename.
- **Development mode**: `[dev <short_name>]` prefix, dev tag, paused schedules and triggers, development pipelines, cluster override allowed, no deployment lock.
- **Production mode**: pipelines must not be in development mode, optional Git branch check, no cluster override, and a service principal in `run_as` is recommended.
- **Variables** (`${var.x}`) resolve at deploy time; per-run values belong in **job parameters**.
- **CI auth**: service principal via workload identity federation (`github-oidc`) or OAuth M2M; PATs are legacy.
- **Existing resources** → `bundle deployment bind` (or `bundle generate ... --bind`).

Key Takeaways

- **Development mode behavior**: `mode: development` prefixes names with `[dev ${workspace.current_u` adds a dev tag, pauses schedules and triggers, marks pipelines as development, and disables the deployment lock.
- **Variable syntax**: custom variables are `${var.name}`; context substitutions include `${bundle.target}`, `${workspace.current_user.userName}`, and `${workspace.file_path}`; values come from `--var`, `BUNDLE_VAR_<name>`, target variables, then defaults.
- **Core CLI commands**: `validate` checks configuration; `deploy` creates or updates resources; `run` triggers a job or pipeline; `destroy` removes what the bundle deployed.
- **`run_as` for production**: a service principal in `run_as` separates the run identity from the deployer; without one, production mode requires `run_as` and `permissions` and rejects user-specific paths.
- **State**: deployment state is stored per target in the workspace (`state_path`); keep the local `.databricks/` folder out of Git. The direct deployment engine is now the default and the Terraform engine is gone in current CLIs.
- **Sync**: `sync` include/exclude use `.gitignore` syntax and build on the bundle's `.gitignore`.
- **Artifacts**: `bundle deploy` builds and uploads `whl` and `jar` artifacts declared in `artifacts:`.
- **Composition**: `include:` merges shared YAML (clusters, notifications, permissions) into one bundle.

Related Topics

- Declarative Automation Bundles — Part 1 - Structure, targets, modes, presets, `run_as`
- Bundle Deployment Strategies — Part 1 - Advanced bundle patterns, blue/green, canary
- CI/CD Integration — Part 1 - Pipeline workflows on CI platforms
- Git Folders - Git folders vs bundles
- Job Repair and Parameter Overrides - Overriding job parameters at run time

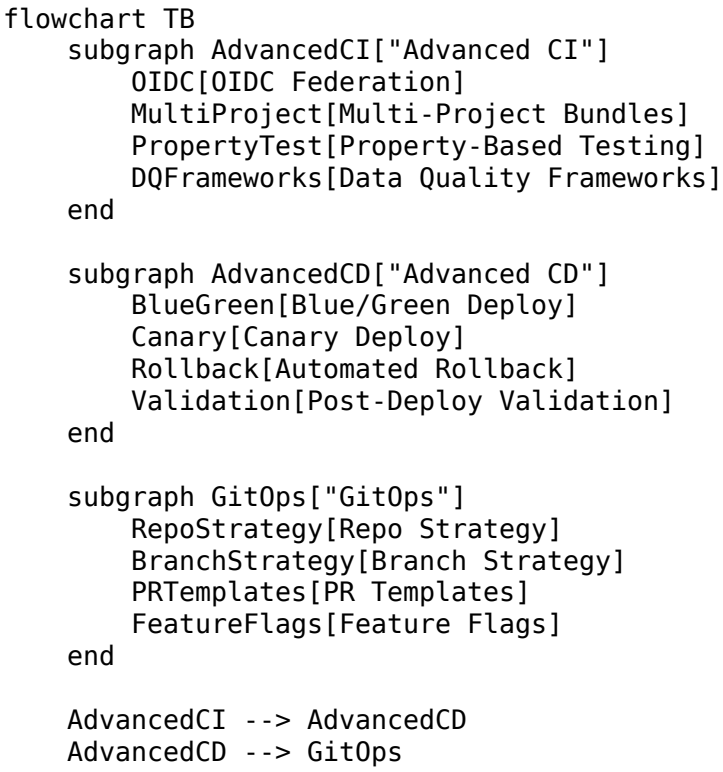
Official Documentation

- What are Declarative Automation Bundles?
- Declarative Automation Bundles feature release notes
- Bundle configuration
- Configuration reference
- Bundle resources
- Deployment modes
- Substitutions and variables
- Specify a run identity (`run_as`)
- Override cluster settings
- Configure job parameters in bundles
- Bundle project templates
- `bundle` command group
- Direct deployment engine
- Enable workload identity federation for GitHub Actions
- Environment variables for unified authentication

Bundle Deployment Strategies — Part 1: Bundle Patterns & CI/CD Pipelines

This part covers advanced Declarative Automation Bundle patterns, CI/CD pipeline DAGs, and blue/-green and canary deployment strategies. For rollback, feature flags, schema migration, and OIDC federation see Part 2.

Overview



Overview

- **Bundle patterns**: multi-project organization, `include` for shared configuration, custom variables, and resource permissions in `databricks.yml`
- **Artifacts and targets**: artifact management and per-target overrides
- **CI/CD pipelines**: a full pipeline DAG and multi-environment promotion
- **Release strategies**: blue/green and canary deployments for data pipelines

What the Exam Tests

- Deploy Databricks resources using Declarative Automation Bundles (formerly Databricks Asset Bundles).
- Structure one bundle for several environments with target overrides
- Choose a promotion and release strategy for a pipeline change

Advanced Declarative Automation Bundle Patterns

Multi-Project Bundle Organization Large organizations often manage multiple data projects. Bundles support includes and overrides to share configuration across projects.

```
databricks-platform/
├── shared/
│   ├── common-clusters.yml        # Shared cluster definitions
│   ├── notification-defaults.yml  # Shared notification settings
│   └── permissions.yml            # Common permission grants
├── projects/
│   ├── ingestion/
│   │   ├── databricks.yml
```

```
│   │   └── resources/
│   │       ├── jobs.yml
│   │       └── pipelines.yml
│   ├── analytics/
│   │   ├── databricks.yml
│   │   └── resources/
│   │       └── jobs.yml
│   └── ml-serving/
│       ├── databricks.yml
│       └── resources/
│           ├── jobs.yml
│           └── model-serving.yml
└── ci/
    ├── deploy-all.sh
    └── validate-all.sh
```

Bundle Inheritance with Includes

```yaml
# projects/ingestion/databricks.yml

bundle:
  name: ingestion-pipelines

include:
  # Inherit shared cluster definitions
  - ../../shared/common-clusters.yml
  # Inherit permission grants
  - ../../shared/permissions.yml
  # Include project-specific resources
  - resources/*.yml

variables:
  project_name:
    default: ingestion
  catalog:
    default: dev_catalog
  owner_team:
    default: data-engineering
```

```yaml
# shared/common-clusters.yml

resources:
  jobs:
    _shared_job_template:
      job_clusters:
        - job_cluster_key: standard_etl
          new_cluster:
            spark_version: "15.4.x-scala2.12"
            node_type_id: "Standard_DS3_v2"
            num_workers: 2
            spark_conf:
              spark.databricks.delta.preview.enabled: "true"
              spark.sql.shuffle.partitions: "auto"
            custom_tags:
              team: ${var.owner_team}
```

```
            project: ${var.project_name}

        - job_cluster_key: heavy_etl
          new_cluster:
            spark_version: "15.4.x-scala2.12"
            node_type_id: "Standard_DS5_v2"
            autoscale:
              min_workers: 2
              max_workers: 16
```

Custom Variable Interpolation Declarative Automation Bundles support several variable interpolation patterns.

Variable Pattern	Description	Example Value
`${var.name}`	User-defined variable	`prod`, `my_catalog`
`${bundle.name}`	Bundle name from config	`ingestion-pipelines`
`${bundle.target}`	Current deployment target	`dev`, `staging`, `prod`
`${workspace.host}`	Workspace URL	`https://adb-123.azuredatabricks`
`${workspace.current_user.userName}`	Deploying user	`user@company.com`
`${workspace.root_path}`	Workspace root for bundle	`/Workspace/Shared/.bundle/...`
`${resources.jobs.<name>.id}`	Resource ID after deploy	`987654321`
`${resources.pipelines.<name>.id}`	Pipeline ID after deploy	`abc-123-def`

```
# Advanced variable interpolation examples

variables:
  environment:
    default: dev
  catalog:
    default: dev_catalog
  alert_email:
    default: "data-team-dev@company.com"

  # Variable with lookup - resolves at deploy time
  warehouse_id:
    lookup:
      warehouse: shared-sql-warehouse

  # Variable with lookup for cluster policy
  cluster_policy_id:
    lookup:
      cluster_policy: data-engineering-policy

resources:
  jobs:
    orchestrator_job:
      name: "${var.project_name}-orchestrator-${bundle.target}"
      tags:
        bundle: ${bundle.name}
        target: ${bundle.target}
        deployed_by: ${workspace.current_user.userName}

      tasks:
        - task_key: run_pipeline
```

```
          pipeline_task:
            # Reference another resource by lookup
            pipeline_id: ${resources.pipelines.main_pipeline.id}

        - task_key: notify_downstream
          depends_on:
            - task_key: run_pipeline
          notebook_task:
            notebook_path: ../src/notebooks/notify.py
            base_parameters:
              # Reference the job's own ID
              orchestrator_job_id: ${resources.jobs.orchestrator_job.id}
              workspace_host: ${workspace.host}
```

Resource Permissions in databricks.yml

```
# Granular permissions for deployed resources

resources:
  jobs:
    production_etl:
      name: "Production ETL - ${var.environment}"
      permissions:
        - level: CAN_VIEW
          group_name: data-analysts
        - level: CAN_MANAGE_RUN
          group_name: data-engineers
        - level: CAN_MANAGE
          group_name: platform-admins
        - level: IS_OWNER
          service_principal_name: prod-etl-sp

  pipelines:
    streaming_pipeline:
      name: "Streaming Pipeline - ${var.environment}"
      permissions:
        - level: CAN_VIEW
          group_name: data-analysts
        - level: CAN_RUN
          group_name: data-engineers
        - level: CAN_MANAGE
          service_principal_name: prod-etl-sp
```

Artifact Management

```
# databricks.yml - Multiple artifact types

artifacts:
  # Python wheel from Poetry project
  etl_core:
    type: whl
    path: ./libs/etl-core
    build: poetry build

  # Python wheel from setup.py
  data_quality:
```

```
    type: whl
    path: ./libs/data-quality
    build: python setup.py bdist_wheel

resources:
  jobs:
    pipeline_with_libs:
      tasks:
        - task_key: transform
          python_wheel_task:
            package_name: etl_core
            entry_point: run_transform
            parameters:
              - "--catalog"
              - "${var.catalog}"
          libraries:
            - whl: ../dist/etl_core/*.whl
            - whl: ../dist/data_quality/*.whl
            # External PyPI dependencies
            - pypi:
                package: great-expectations==0.18.0
```

Complex Target Configurations with Overrides

```
# databricks.yml

targets:
  dev:
    mode: development
    default: true
    variables:
      environment: dev
      catalog: dev_catalog
      alert_email: "dev-team@company.com"
    workspace:
      host: https://adb-dev.1.azuredatabricks.net
      root_path:
↪  /Workspace/Users/${workspace.current_user.userName}/.bundle/${bundle.name}/${bundle.targ
    resources:
      jobs:
        production_etl:
          # Override schedule in dev - paused by development mode
          schedule:
            quartz_cron_expression: "0 0 */2 * * ?"
            timezone_id: "UTC"
          job_clusters:
            - job_cluster_key: standard_etl
              new_cluster:
                num_workers: 1  # Smaller cluster in dev

  staging:
    variables:
      environment: staging
      catalog: staging_catalog
      alert_email: "staging-alerts@company.com"
    workspace:
```

```yaml
    host: https://adb-staging.1.azuredatabricks.net
    root_path: /Workspace/Shared/.bundle/${bundle.name}/${bundle.target}
  run_as:
    service_principal_name: staging-deploy-sp

prod:
  mode: production
  variables:
    environment: prod
    catalog: prod_catalog
    alert_email: "prod-alerts@company.com"
  workspace:
    host: https://adb-prod.1.azuredatabricks.net
    root_path: /Workspace/Shared/.bundle/${bundle.name}/${bundle.target}
  run_as:
    service_principal_name: prod-deploy-sp
  resources:
    jobs:
      production_etl:
        job_clusters:
          - job_cluster_key: standard_etl
            new_cluster:
              num_workers: 8  # Full cluster in prod
              driver_node_type_id: "Standard_DS5_v2"
        # Production notifications
        email_notifications:
          on_failure:
            - prod-oncall@company.com
            - data-engineering-leads@company.com
          on_success:
            - data-engineering-metrics@company.com
```

Advanced CI/CD Pipeline Patterns

Full Pipeline DAG

```
flowchart LR
    subgraph CI["Continuous Integration"]
        Lint[Lint & Format]
        Validate[Bundle Validate]
        UnitTest[Unit Tests]
        Coverage[Coverage Check]
        Build[Build Artifacts]
    end

    subgraph CDStaging["CD - Staging"]
        DeployStage[Deploy to Staging]
        IntTest[Integration Tests]
        DQCheck[Data Quality Gates]
        PerfTest[Performance Tests]
    end

    subgraph CDProd["CD - Production"]
        Approval[Manual Approval]
        DeployProd[Deploy to Prod]
        Smoke[Smoke Tests]
```

```
        Monitor[Health Monitoring]
    end

    Lint --> Validate
    Validate --> UnitTest
    UnitTest --> Coverage
    Coverage --> Build
    Build --> DeployStage
    DeployStage --> IntTest
    IntTest --> DQCheck
    DQCheck --> PerfTest
    PerfTest --> Approval
    Approval --> DeployProd
    DeployProd --> Smoke
    Smoke --> Monitor
```

Multi-Environment Promotion Strategy

```yaml
# .github/workflows/full-pipeline.yml

name: Full CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: false

jobs:
  # Stage 1: Lint and validate
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.10"
      - run: |
          pip install ruff mypy
          ruff check src/
          ruff format --check src/
          mypy src/ --ignore-missing-imports

  # Stage 2: Unit tests with coverage gate
  unit-tests:
    runs-on: ubuntu-latest
    needs: lint
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.10"
```

```yaml
      - run: pip install -r requirements-dev.txt
      - run: |
          pytest tests/unit/ \
            --cov=src \
            --cov-report=xml \
            --cov-fail-under=80 \
            --junitxml=test-results.xml \
            -v
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: test-results
          path: test-results.xml

  # Stage 3: Bundle validate and build
  build:
    runs-on: ubuntu-latest
    needs: unit-tests
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - run: databricks bundle validate -t staging
        env:
          DATABRICKS_HOST: ${{ secrets.STAGING_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.STAGING_TOKEN }}
      - run: |
          cd libs/etl-core && poetry build
          cd ../data-quality && poetry build

  # Stage 4: Deploy to staging + integration tests
  deploy-staging:
    runs-on: ubuntu-latest
    needs: build
    if: github.ref == 'refs/heads/main'
    environment: staging
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - name: Deploy to staging
        run: databricks bundle deploy -t staging
        env:
          DATABRICKS_HOST: ${{ secrets.STAGING_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.STAGING_TOKEN }}

      - name: Run integration tests
        run: |
          databricks bundle run integration_test_job -t staging
        env:
          DATABRICKS_HOST: ${{ secrets.STAGING_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.STAGING_TOKEN }}

      - name: Run data quality validation
        run: |
          databricks bundle run data_quality_check -t staging
        env:
```

```
          DATABRICKS_HOST: ${{ secrets.STAGING_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.STAGING_TOKEN }}

  # Stage 5: Deploy to production with approval
  deploy-production:
    runs-on: ubuntu-latest
    needs: deploy-staging
    if: github.ref == 'refs/heads/main'
    environment: production  # Requires manual approval
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main

      - name: Deploy to production
        run: databricks bundle deploy -t prod
        env:
          DATABRICKS_HOST: ${{ secrets.PROD_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.PROD_TOKEN }}

      - name: Run smoke tests
        run: |
          databricks bundle run smoke_test_job -t prod
        env:
          DATABRICKS_HOST: ${{ secrets.PROD_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.PROD_TOKEN }}

      - name: Notify deployment success
        if: success()
        uses: slackapi/slack-github-action@v1
        with:
          channel-id: deployments
          slack-message: "Production deployment succeeded for ${{ github.sha }}"
        env:
          SLACK_BOT_TOKEN: ${{ secrets.SLACK_BOT_TOKEN }}
```

Blue/Green Deployment for Data Pipelines

```
flowchart TB
    subgraph Active["Active (Blue)"]
        BlueJob[ETL Job - Blue]
        BlueTable[(prod.blue_events)]
    end

    subgraph Standby["Standby (Green)"]
        GreenJob[ETL Job - Green]
        GreenTable[(prod.green_events)]
    end

    subgraph Router["Traffic Router"]
        View[prod.events VIEW]
    end

    BlueJob --> BlueTable
    GreenJob --> GreenTable
    View -->|Points to active| BlueTable
    View -.->|Switch after validation| GreenTable
```

```yaml
# Blue/green bundle targets

targets:
  prod-blue:
    mode: production
    variables:
      environment: prod
      slot: blue
      target_schema: prod_blue
    run_as:
      service_principal_name: prod-sp

  prod-green:
    mode: production
    variables:
      environment: prod
      slot: green
      target_schema: prod_green
    run_as:
      service_principal_name: prod-sp
```

```python
# scripts/blue_green_switch.py

"""Blue/green switch script for data pipelines."""
import sys
from databricks.sdk import WorkspaceClient

def switch_active_slot(catalog: str, current_slot: str):
    """Switch the active slot by updating views."""
    new_slot = "green" if current_slot == "blue" else "blue"
    w = WorkspaceClient()

    # Update all views to point to the new slot
    tables = w.tables.list(catalog_name=catalog, schema_name=f"prod_{new_slot}")
    for table in tables:
        table_name = table.name
        w.statement_execution.execute_statement(
            warehouse_id="your-warehouse-id",
            statement=f"""
                CREATE OR REPLACE VIEW {catalog}.prod.{table_name} AS
                SELECT * FROM {catalog}.prod_{new_slot}.{table_name}
            """
        )
    print(f"Switched active slot from {current_slot} to {new_slot}")

if __name__ == "__main__":
    switch_active_slot(sys.argv[1], sys.argv[2])
```

Canary Deployment Pattern

```python
# scripts/canary_deploy.py

"""Canary deployment for Databricks jobs."""
import time
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.jobs import RunLifeCycleState, RunResultState
```

```python
def canary_deploy(
    job_name: str,
    canary_job_name: str,
    monitor_minutes: int = 30,
    max_error_rate: float = 0.05
):
    """
    Deploy canary version and monitor before full rollout.

    1. Deploy the canary job (processes subset of data)
    2. Monitor for errors over the monitoring window
    3. If healthy, proceed with full deployment
    4. If unhealthy, rollback canary and alert
    """
    w = WorkspaceClient()

    # Run canary job
    print(f"Starting canary job: {canary_job_name}")
    canary_run = w.jobs.run_now(
        job_id=get_job_id(w, canary_job_name)
    )

    # Monitor canary
    print(f"Monitoring canary for {monitor_minutes} minutes...")
    run_result = canary_run.result(timeout=timedelta(minutes=monitor_minutes))

    if run_result.state.result_state == RunResultState.SUCCESS:
        # Validate canary output data quality
        quality_check = validate_canary_output(w, canary_job_name)
        if quality_check["error_rate"] < max_error_rate:
            print("Canary passed. Proceeding with full deployment.")
            return True
        else:
            print(f"Canary data quality failed: {quality_check}")
            return False
    else:
        print(f"Canary job failed: {run_result.state}")
        return False

def get_job_id(client, job_name):
    """Look up job ID by name."""
    jobs = client.jobs.list(name=job_name)
    for job in jobs:
        return job.job_id
    raise ValueError(f"Job not found: {job_name}")
```

Continue reading: Part 2 — Rollback, Feature Flags & OIDC

Bundle Deployment Strategies — Part 2: Rollback, Feature Flags & OIDC

Overview

For advanced bundle patterns, CI/CD pipeline DAGs, blue/green, and canary deployments, see Part 1.

This part covers rollback strategies, feature flags in data pipelines, schema migration patterns, and OIDC federation for secure CI/CD authentication.

Overview

- **Rollback**: redeploy the previous commit with `databricks bundle deploy`; undo bad data with `RESTORE TABLE`
- **Feature flags** in a Unity Catalog Delta table, and tracked, idempotent schema migrations
- **OIDC federation** with GitHub Actions replaces stored PATs and client secrets
- **Credential hygiene**: separate service principals per environment, and protection rules on production

What the Exam Tests

- Deploy Databricks resources using Declarative Automation Bundles (formerly Databricks Asset Bundles).
- Plan a rollback that covers both the deployed code and the data it wrote

Rollback Strategies

```
flowchart TD
    Deploy[Deploy New Version] --> Monitor{Health Check}
    Monitor -->|Healthy| Done[Deployment Complete]
    Monitor -->|Unhealthy| RollbackType{Rollback Type}

    RollbackType --> BundleRollback[Bundle Rollback<br>Redeploy previous version]
    RollbackType --> DataRollback[Data Rollback<br>Delta Time Travel]
    RollbackType --> ConfigRollback[Config Rollback<br>Git revert + deploy]

    BundleRollback --> Verify[Verify Rollback]
    DataRollback --> Verify
    ConfigRollback --> Verify
```

```
#!/bin/bash
# scripts/rollback.sh - Automated rollback script

set -euo pipefail

ENVIRONMENT=${1:-staging}
PREVIOUS_COMMIT=${2:-HEAD~1}

echo "Rolling back ${ENVIRONMENT} to commit ${PREVIOUS_COMMIT}"

# Checkout previous known-good version

git checkout "${PREVIOUS_COMMIT}"

# Deploy the previous version

databricks bundle deploy -t "${ENVIRONMENT}"

# Verify deployment health

databricks bundle run smoke_test_job -t "${ENVIRONMENT}"
```

```
echo "Rollback to ${PREVIOUS_COMMIT} completed successfully"
```

```
-- Data rollback using Delta Time Travel
-- Restore table to a previous version after bad deployment

-- Check table history
DESCRIBE HISTORY prod_catalog.gold.daily_metrics;

-- Restore to a specific version
RESTORE TABLE prod_catalog.gold.daily_metrics TO VERSION AS OF 42;

-- Or restore to a timestamp
RESTORE TABLE prod_catalog.gold.daily_metrics
TO TIMESTAMP AS OF '2025-12-01T00:00:00Z';
```

Feature Flags in Data Pipelines

```
# src/feature_flags.py

"""Feature flag management for data pipelines."""
from databricks.sdk import WorkspaceClient

class FeatureFlags:
    """Read feature flags from a Unity Catalog table."""

    def __init__(self, catalog: str, schema: str = "config"):
        self.table = f"{catalog}.{schema}.feature_flags"

    def is_enabled(self, flag_name: str, environment: str) -> bool:
        """Check if a feature flag is enabled for the given environment."""
        result = spark.sql(f"""
            SELECT enabled
            FROM {self.table}
            WHERE flag_name = '{flag_name}'
              AND environment = '{environment}'
        """).collect()

        if result:
            return result[0]["enabled"]
        return False

# Usage in pipeline notebooks

flags = FeatureFlags(catalog="prod_catalog")

if flags.is_enabled("use_new_transform_v2", environment="prod"):
    df = apply_transform_v2(df)
else:
    df = apply_transform_v1(df)
```

```
-- Feature flags table schema
CREATE TABLE IF NOT EXISTS config.feature_flags (
    flag_name STRING NOT NULL,
    environment STRING NOT NULL,
    enabled BOOLEAN DEFAULT false,
    description STRING,
    updated_by STRING,
```

```
    updated_at TIMESTAMP DEFAULT current_timestamp()
)
USING DELTA
TBLPROPERTIES ('delta.enableChangeDataFeed' = 'true');

-- Example flags
INSERT INTO config.feature_flags VALUES
  ('use_new_transform_v2', 'dev', true, 'New transform logic', 'user@co.com',
 ↪  current_timestamp()),
  ('use_new_transform_v2', 'staging', true, 'New transform logic', 'user@co.com',
 ↪  current_timestamp()),
  ('use_new_transform_v2', 'prod', false, 'New transform logic', 'user@co.com',
 ↪  current_timestamp()),
  ('enable_streaming_ingest', 'prod', true, 'Switch to streaming', 'user@co.com',
 ↪  current_timestamp());
```

Schema Migration Patterns with Unity Catalog

```
# src/migrations/schema_manager.py

"""Schema migration management for CI/CD pipelines."""

class SchemaMigrator:
    """Apply schema migrations in order during deployment."""

    def __init__(self, catalog: str, schema: str, migrations_path: str):
        self.catalog = catalog
        self.schema = schema
        self.migrations_path = migrations_path
        self.tracking_table = f"{catalog}.{schema}._schema_migrations"

    def initialize(self):
        """Create migration tracking table if it does not exist."""
        spark.sql(f"CREATE SCHEMA IF NOT EXISTS {self.catalog}.{self.schema}")
        spark.sql(f"""
            CREATE TABLE IF NOT EXISTS {self.tracking_table} (
                version INT,
                description STRING,
                applied_at TIMESTAMP,
                applied_by STRING
            ) USING DELTA
        """)

    def get_applied_versions(self):
        """Get list of already-applied migration versions."""
        return [
            row.version for row in
            spark.sql(f"SELECT version FROM {self.tracking_table}").collect()
        ]

    def apply_pending(self):
        """Apply all pending migrations in order."""
        applied = set(self.get_applied_versions())
        migrations = self._load_migrations()

        for version, description, sql_statement in sorted(migrations):
```

```
            if version not in applied:
                print(f"Applying migration {version}: {description}")
                spark.sql(sql_statement)
                spark.sql(f"""
                    INSERT INTO {self.tracking_table}
                    VALUES ({version}, '{description}', current_timestamp(),
                            current_user())
                """)
                print(f"Migration {version} applied successfully")

    def _load_migrations(self):
        """Load migration files from the migrations directory."""
        import os
        migrations = []
        for filename in sorted(os.listdir(self.migrations_path)):
            if filename.endswith('.sql'):
                version = int(filename.split('_')[0])
                description = filename.replace('.sql', '').split('_', 1)[1]
                with open(os.path.join(self.migrations_path, filename)) as f:
                    sql = f.read()
                migrations.append((version, description, sql))
        return migrations
```

```
migrations/
├── 001_create_bronze_events.sql
├── 002_add_user_agent_column.sql
├── 003_create_silver_sessions.sql
└── 004_add_gold_daily_metrics.sql
```

```
-- migrations/002_add_user_agent_column.sql
ALTER TABLE bronze.events ADD COLUMN user_agent STRING;
```

OIDC Federation Deep Dive

GitHub Actions OIDC with Databricks Databricks calls this **workload identity federation** (a form of OAuth token federation): the CI runner's OIDC token is exchanged for a short-lived Databricks OAuth token for a service principal, so no PAT or client secret is stored in CI. Source: Enable workload identity federation for GitHub Actions

```
sequenceDiagram
    participant GH as GitHub Actions
    participant OIDC as OIDC Provider
    participant DB as Databricks
    participant SP as Service Principal

    GH->>OIDC: Request OIDC Token
    OIDC->>GH: JWT Token (short-lived)
    GH->>DB: Authenticate with JWT
    DB->>SP: Validate against SP config
    SP->>DB: Grant access
    DB->>GH: Access Token
    GH->>DB: Deploy bundle
```

```
# .github/workflows/oidc-deploy.yml

name: Deploy with OIDC
```

```yaml
on:
  push:
    branches: [main]

permissions:
  id-token: write   # Required for OIDC
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4

      - uses: databricks/setup-cli@main

      # No token or client secret needed - workload identity federation
      - name: Deploy to production
        run: databricks bundle deploy -t prod
        env:
          DATABRICKS_AUTH_TYPE: github-oidc
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
          DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_SP_CLIENT_ID }}  # SP
 ↪  application ID


# Create a service principal federation policy (account level), e.g. with
#   databricks account service-principal-federation-policy create <SP_NUMERIC_ID>
 ↪  --json '{"oidc_policy": {...}}'
#    - Issuer:    https://token.actions.githubusercontent.com
#    - Subject:   repo:my-org/my-repo:environment:prod  (environment entity type
 ↪  recommended)
#    - Audiences: your Databricks account ID (the default if omitted)

# Alternative on Azure: Microsoft Entra federated credential for GitHub Actions

az ad app federated-credential create \
    --id <app-object-id> \
    --parameters '{
        "name": "github-actions-deploy",
        "issuer": "https://token.actions.githubusercontent.com",
        "subject": "repo:my-org/my-repo:ref:refs/heads/main",
        "audiences": ["api://AzureADTokenExchange"]
    }'
```

Security Best Practices for CI/CD Credentials

Practice	Description	Priority
Use workload identity federation (OIDC)	No long-lived secrets in CI	High
Scope service principals	Minimum required permissions	High
Avoid PATs (legacy); if used, rotate	Set expiration, rotate regularly	High
Environment protection	Require approval for prod deploy	High
Audit CI/CD access	Review service principal activity	Medium
IP allowlisting	Restrict CI runner IP ranges	Medium

Practice	Description	Priority
Separate SPs per environment	Different SPs for dev/staging/prod	Medium

```
# Environment protection rules in GitHub
# Settings → Environments → production
# - Required reviewers: 2 approvers
# - Wait timer: 5 minutes
# - Deployment branches: main only
# - Environment secrets: PROD_HOST, PROD_SP_CLIENT_ID

# Branch protection for main
# Settings → Branches → main
# - Require pull request reviews
# - Require status checks to pass
# - Require CODEOWNERS review
# - No force pushes
```

Use Cases

- **Failed production release**: redeploy the previous known-good commit with `databricks bundle deploy -t prod`, then `RESTORE TABLE` any data the bad release wrote
- **Gradual feature rollout**: read a feature flag from a Unity Catalog Delta table at pipeline runtime to turn new logic on per environment without redeploying
- **Repeatable schema changes**: track applied migration versions in a Delta table so each numbered migration script runs once in every environment
- **Secretless deployments**: authenticate GitHub Actions to Databricks with workload identity federation (OIDC) instead of a stored PAT or client secret

Common Issues & Errors

Bundle Rollback Leaves Bad Data Behind **Scenario:** Redeploying the previous commit restored the job definitions, but tables still hold rows written by the failed release. **Fix:** A bundle rollback only covers resources. Roll back data separately with Delta time travel: check `DESCRIBE HISTORY`, then `RESTORE TABLE ... TO VERSION AS OF N` (or `TO TIMESTAMP AS OF`).

Schema Migration Applied Twice **Scenario:** A migration script runs again in an environment where it was already applied. **Fix:** Record each applied migration version in a dedicated Delta table and skip versions already recorded, so every script runs exactly once.

CI Depends on a Long-Lived Secret **Scenario:** The deployment workflow stores a PAT or client secret that can leak or expire. **Fix:** Use workload identity federation: grant `id-token: write` in the GitHub Actions workflow, set `DATABRICKS_AUTH_TYPE: github-oidc`, and create a federation policy for the service principal.

One Service Principal Can Deploy to Every Environment **Scenario:** A single service principal has access to dev, staging, and prod, so a mistake in a lower environment can change production. **Fix:** Use a separate service principal per environment with minimum permissions, and add environment protection rules (required reviewers) for production.

Exam Tips

1. **Multi-project bundles** - Use `include:` to share cluster and permission config across projects

2. **Variable interpolation** - Know `${var.name}`, `${bundle.target}`, `${workspace.host}` syntax
3. **Blue/green with views** - SQL views act as the traffic router between slots
4. **Canary pattern** - Deploy to subset, monitor, then promote or rollback
5. **Delta Time Travel for rollback** - `RESTORE TABLE ... TO VERSION AS OF N`
6. **Feature flags** - Store in Unity Catalog Delta table, read at pipeline runtime
7. **Schema migrations** - Track applied versions in a Delta table to ensure idempotency
8. **Workload identity federation (OIDC)** - Eliminates long-lived secrets; requires `id-token: write` in GitHub Actions and `DATABRICKS_AUTH_TYPE: github-oidc`
9. **Environment protection rules** - Enforce manual approval and reviewer gates for production
10. **Separate service principals** - Use different SPs per environment for least-privilege access

Key Takeaways

- **Bundle rollback procedure**: The fastest rollback with DAB is `git checkout <previous-commit>` then `databricks bundle deploy -t prod` — this redeploys the previous configuration without destroying and recreating resources.
- **Delta Time Travel for data rollback**: Use `RESTORE TABLE ... TO VERSION AS OF N` or `TO TIMESTAMP AS OF '...'` to undo bad data written by a failed deployment, independently of the infrastructure rollback.
- **Feature flags in Unity Catalog**: Store feature flags in a Delta table in Unity Catalog and read them at pipeline runtime to toggle behavior per environment without redeployment.
- **Schema migrations tracking**: Track applied migration versions in a dedicated Delta table to ensure each migration SQL script is idempotent and runs exactly once across all environments.
- **OIDC federation eliminates secrets**: Configure OIDC with `permissions: id-token: write` in GitHub Actions to exchange short-lived JWT tokens instead of storing long-lived PATs or client secrets.
- **Separate service principals per env**: Use different service principals for dev, staging, and prod to enforce least-privilege access and prevent accidental cross-environment operations.
- **Environment protection rules**: Require manual approvers and a wait timer on GitHub production environments to add a human gate before all production bundle deployments.
- **Blue-green via SQL views**: Implement blue-green slot switching by replacing a SQL view that points from the `prod_blue` schema to `prod_green`, making the switchover instantaneous for consumers.

Related Topics

- CI/CD Integration - Platform-specific CI/CD configuration
- CI/CD Integration Part 2 - Testing and secret management patterns
- Declarative Automation Bundles - Core DAB configuration reference
- Advanced Testing & Operations - Property-based and Lakeflow Pipelines testing

Official Documentation

- Declarative Automation Bundles
- Bundle Configuration Reference
- OAuth token federation
- Enable workload identity federation for GitHub Actions
- Delta Time Travel

CI/CD Integration — Part 1: Platforms & Configuration

Continuous Integration and Continuous Deployment (CI/CD) practices are essential for maintaining quality and reliability in data engineering projects. This part covers authentication and CI/CD platform configuration; for testing strategies, secret management, and deployment patterns see Part 2.

Overview

```
flowchart TB
    subgraph Development["Development"]
        Code[Code Changes]
        PR[Pull Request]
    end

    subgraph CI["Continuous Integration"]
        Lint[Lint & Format]
        Validate[Bundle Validate]
        UnitTest[Unit Tests]
        Build[Build Artifacts]
    end

    subgraph CD["Continuous Deployment"]
        DeployDev[Deploy Dev]
        IntTest[Integration Tests]
        DeployStaging[Deploy Staging]
        DeployProd[Deploy Production]
    end

    Code --> PR
    PR --> Lint
    Lint --> Validate
    Validate --> UnitTest
    UnitTest --> Build
    Build --> DeployDev
    DeployDev --> IntTest
    IntTest --> DeployStaging
    DeployStaging --> DeployProd
```

Overview

- **Authentication**: service principals for CI/CD on Azure and AWS
- **GitHub Actions**: a complete workflow and reusable workflows
- **Other platforms**: Azure DevOps pipelines and variable groups, GitLab CI/CD, and Jenkins

What the Exam Tests

- Configure Git-based CI/CD workflows using Databricks Git folders (formerly Repos) to deploy notebooks and code.
- Deploy Databricks resources using Declarative Automation Bundles (formerly Databricks Asset Bundles).
- Authenticate pipelines as service principals rather than as individual users

Authentication Methods

Service Principal Authentication

```
# Environment variables for service principal

export DATABRICKS_HOST="https://adb-1234567890.1.azuredatabricks.net"
export DATABRICKS_CLIENT_ID="00000000-0000-0000-0000-000000000000"
export DATABRICKS_CLIENT_SECRET="your-client-secret"

# Or, in GitHub Actions, workload identity federation (no secret stored)

export DATABRICKS_AUTH_TYPE="github-oidc"
export DATABRICKS_HOST="https://adb-1234567890.1.azuredatabricks.net"
export DATABRICKS_CLIENT_ID="00000000-0000-0000-0000-000000000000"
```

DATABRICKS_CLIENT_ID + DATABRICKS_CLIENT_SECRET select OAuth M2M (oauth-m2m); DATABRICKS_TOKEN is a personal access token, which the docs mark as legacy. Source: Environment variables for unified authentication

Azure Service Principal Setup

```
# Create service principal

az ad sp create-for-rbac --name "databricks-cicd-sp" --role Contributor \
    --scopes /subscriptions/{subscription-id}/resourceGroups/{resource-group}

# Grant Databricks workspace access
# In Databricks Admin Console:
# Add service principal to workspace
# Grant appropriate permissions (workspace admin or specific ACLs)
```

AWS Authentication

```
# The Databricks CLI does not authenticate with AWS instance profiles or AWS keys.
# Use a Databricks service principal (OAuth M2M or workload identity federation):

export DATABRICKS_HOST="https://my-workspace.cloud.databricks.com"
export DATABRICKS_CLIENT_ID="00000000-0000-0000-0000-000000000000"
export DATABRICKS_CLIENT_SECRET="your-client-secret"

# Legacy: personal access token
# export DATABRICKS_TOKEN="dapi..."
```

GitHub Actions

Complete Workflow Example

```
# .github/workflows/databricks-cicd.yml

name: Databricks CI/CD

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
```

```yaml
env:
  PYTHON_VERSION: '3.10'

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ env.PYTHON_VERSION }}

      - name: Install dependencies
        run: |
          pip install -r requirements-dev.txt
          pip install ruff pytest

      - name: Lint with ruff
        run: ruff check src/

      - name: Run unit tests
        run: pytest tests/unit/ -v --junitxml=test-results.xml

      - name: Upload test results
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: test-results
          path: test-results.xml

  validate-bundle:
    runs-on: ubuntu-latest
    needs: lint-and-test
    steps:
      - uses: actions/checkout@v4

      - name: Setup Databricks CLI
        uses: databricks/setup-cli@main

      - name: Validate bundle
        run: databricks bundle validate
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}

  deploy-dev:
    runs-on: ubuntu-latest
    needs: validate-bundle
    if: github.event_name == 'push' && github.ref == 'refs/heads/develop'
    environment: development
    steps:
      - uses: actions/checkout@v4
```

```
      - uses: databricks/setup-cli@main

      - name: Deploy to development
        run: databricks bundle deploy -t dev
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}

  deploy-staging:
    runs-on: ubuntu-latest
    needs: validate-bundle
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    environment: staging
    steps:
      - uses: actions/checkout@v4

      - uses: databricks/setup-cli@main

      - name: Deploy to staging
        run: databricks bundle deploy -t staging
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}

      - name: Run integration tests
        run: |
          databricks bundle run integration_test_job -t staging
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}

  deploy-production:
    runs-on: ubuntu-latest
    needs: deploy-staging
    if: github.ref == 'refs/heads/main'
    environment: production
    steps:
      - uses: actions/checkout@v4

      - uses: databricks/setup-cli@main

      - name: Deploy to production
        run: databricks bundle deploy -t prod
        env:
          DATABRICKS_HOST: ${{ secrets.PROD_DATABRICKS_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.PROD_DATABRICKS_TOKEN }}
```

Reusable Workflow

```
# .github/workflows/deploy-template.yml

name: Deploy Template

on:
  workflow_call:
```

```yaml
    inputs:
      environment:
        required: true
        type: string
      target:
        required: true
        type: string
    secrets:
      DATABRICKS_HOST:
        required: true
      DATABRICKS_TOKEN:
        required: true

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}
    steps:
      - uses: actions/checkout@v4

      - uses: databricks/setup-cli@main

      - name: Deploy bundle
        run: databricks bundle deploy -t ${{ inputs.target }}
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}
```

```yaml
# .github/workflows/main.yml - Using the template

name: Main Pipeline

on:
  push:
    branches: [main]

jobs:
  deploy-staging:
    uses: ./.github/workflows/deploy-template.yml
    with:
      environment: staging
      target: staging
    secrets:
      DATABRICKS_HOST: ${{ secrets.STAGING_HOST }}
      DATABRICKS_TOKEN: ${{ secrets.STAGING_TOKEN }}

  deploy-prod:
    needs: deploy-staging
    uses: ./.github/workflows/deploy-template.yml
    with:
      environment: production
      target: prod
    secrets:
      DATABRICKS_HOST: ${{ secrets.PROD_HOST }}
      DATABRICKS_TOKEN: ${{ secrets.PROD_TOKEN }}
```

Azure DevOps

Pipeline Configuration

```yaml
# azure-pipelines.yml

trigger:
  branches:
    include:
      - main
      - develop

pr:
  branches:
    include:
      - main

pool:
  vmImage: 'ubuntu-latest'

variables:
  pythonVersion: '3.10'

stages:
  - stage: Build
    displayName: 'Build and Test'
    jobs:
      - job: BuildAndTest
        steps:
          - task: UsePythonVersion@0
            inputs:
              versionSpec: '$(pythonVersion)'

          - script: |
              pip install -r requirements-dev.txt
              pip install pytest ruff
            displayName: 'Install dependencies'

          - script: ruff check src/
            displayName: 'Lint code'

          - script: pytest tests/unit/ --junitxml=test-results.xml
            displayName: 'Run unit tests'

          - task: PublishTestResults@2
            inputs:
              testResultsFiles: 'test-results.xml'
              testRunTitle: 'Unit Tests'

  - stage: ValidateBundle
    displayName: 'Validate Bundle'
    dependsOn: Build
    jobs:
      - job: Validate
        steps:
          - script: |
```

```
            curl -fsSL
↪ https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
          displayName: 'Install Databricks CLI'

        - script: databricks bundle validate
          displayName: 'Validate bundle'
          env:
            DATABRICKS_HOST: $(DATABRICKS_HOST)
            DATABRICKS_TOKEN: $(DATABRICKS_TOKEN)

- stage: DeployDev
  displayName: 'Deploy to Development'
  dependsOn: ValidateBundle
  condition: and(succeeded(), eq(variables['Build.SourceBranch'],
↪ 'refs/heads/develop'))
  jobs:
    - deployment: DeployDev
      environment: development
      strategy:
        runOnce:
          deploy:
            steps:
              - checkout: self

              - script: |
                  curl -fsSL
↪ https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
                displayName: 'Install Databricks CLI'

              - script: databricks bundle deploy -t dev
                displayName: 'Deploy to dev'
                env:
                  DATABRICKS_HOST: $(DEV_DATABRICKS_HOST)
                  DATABRICKS_TOKEN: $(DEV_DATABRICKS_TOKEN)

- stage: DeployStaging
  displayName: 'Deploy to Staging'
  dependsOn: ValidateBundle
  condition: and(succeeded(), eq(variables['Build.SourceBranch'],
↪ 'refs/heads/main'))
  jobs:
    - deployment: DeployStaging
      environment: staging
      strategy:
        runOnce:
          deploy:
            steps:
              - checkout: self

              - script: |
                  curl -fsSL
↪ https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
                displayName: 'Install Databricks CLI'

              - script: databricks bundle deploy -t staging
```

```
                displayName: 'Deploy to staging'
                env:
                  DATABRICKS_HOST: $(STAGING_DATABRICKS_HOST)
                  DATABRICKS_TOKEN: $(STAGING_DATABRICKS_TOKEN)

  - stage: DeployProd
    displayName: 'Deploy to Production'
    dependsOn: DeployStaging
    condition: succeeded()
    jobs:
      - deployment: DeployProd
        environment: production
        strategy:
          runOnce:
            deploy:
              steps:
                - checkout: self

                - script: |
                    curl -fsSL
↪ https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
                  displayName: 'Install Databricks CLI'

                - script: databricks bundle deploy -t prod
                  displayName: 'Deploy to production'
                  env:
                    DATABRICKS_HOST: $(PROD_DATABRICKS_HOST)
                    DATABRICKS_TOKEN: $(PROD_DATABRICKS_TOKEN)
```

Variable Groups

```
# Reference variable groups in pipeline

variables:
  - group: databricks-dev-credentials
  - group: databricks-prod-credentials

# Variable group contents (configured in Azure DevOps):
# databricks-dev-credentials:
#   - DEV_DATABRICKS_HOST
#   - DEV_DATABRICKS_TOKEN
```

GitLab CI/CD

Pipeline Configuration

```
# .gitlab-ci.yml

stages:
  - test
  - validate
  - deploy-dev
  - deploy-staging
  - deploy-prod

variables:
```

```yaml
  PYTHON_VERSION: "3.10"

.databricks_setup: &databricks_setup
  before_script:
    - curl -fsSL
↪  https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
    - export PATH="$HOME/.databricks:$PATH"

test:
  stage: test
  image: python:${PYTHON_VERSION}
  script:
    - pip install -r requirements-dev.txt
    - ruff check src/
    - pytest tests/unit/ --junitxml=report.xml
  artifacts:
    reports:
      junit: report.xml

validate:
  stage: validate
  image: python:${PYTHON_VERSION}
  <<: *databricks_setup
  script:
    - databricks bundle validate
  variables:
    DATABRICKS_HOST: $DATABRICKS_HOST
    DATABRICKS_TOKEN: $DATABRICKS_TOKEN

deploy-dev:
  stage: deploy-dev
  image: python:${PYTHON_VERSION}
  <<: *databricks_setup
  script:
    - databricks bundle deploy -t dev
  environment:
    name: development
  only:
    - develop
  variables:
    DATABRICKS_HOST: $DEV_DATABRICKS_HOST
    DATABRICKS_TOKEN: $DEV_DATABRICKS_TOKEN

deploy-staging:
  stage: deploy-staging
  image: python:${PYTHON_VERSION}
  <<: *databricks_setup
  script:
    - databricks bundle deploy -t staging
    - databricks bundle run integration_tests -t staging
  environment:
    name: staging
  only:
    - main
  variables:
```

```
    DATABRICKS_HOST: $STAGING_DATABRICKS_HOST
    DATABRICKS_TOKEN: $STAGING_DATABRICKS_TOKEN

deploy-prod:
  stage: deploy-prod
  image: python:${PYTHON_VERSION}
  <<: *databricks_setup
  script:
    - databricks bundle deploy -t prod
  environment:
    name: production
  when: manual
  only:
    - main
  variables:
    DATABRICKS_HOST: $PROD_DATABRICKS_HOST
    DATABRICKS_TOKEN: $PROD_DATABRICKS_TOKEN
```

Jenkins

Jenkinsfile

```
// Jenkinsfile
pipeline {
    agent any

    environment {
        PYTHON_VERSION = '3.10'
    }

    stages {
        stage('Setup') {
            steps {
                sh '''
                    curl -fsSL
 ↪  https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
                    pip install -r requirements-dev.txt
                '''
            }
        }

        stage('Test') {
            steps {
                sh 'ruff check src/'
                sh 'pytest tests/unit/ --junitxml=test-results.xml'
            }
            post {
                always {
                    junit 'test-results.xml'
                }
            }
        }

        stage('Validate') {
            steps {
                withCredentials([
```

```
                    string(credentialsId: 'databricks-host', variable:
                     ↪  'DATABRICKS_HOST'),
                    string(credentialsId: 'databricks-token', variable:
                     ↪  'DATABRICKS_TOKEN')
                ]) {
                    sh 'databricks bundle validate'
                }
            }
        }

        stage('Deploy Staging') {
            when {
                branch 'main'
            }
            steps {
                withCredentials([
                    string(credentialsId: 'staging-host', variable:
                     ↪  'DATABRICKS_HOST'),
                    string(credentialsId: 'staging-token', variable:
                     ↪  'DATABRICKS_TOKEN')
                ]) {
                    sh 'databricks bundle deploy -t staging'
                }
            }
        }

        stage('Deploy Production') {
            when {
                branch 'main'
            }
            input {
                message "Deploy to production?"
                ok "Deploy"
            }
            steps {
                withCredentials([
                    string(credentialsId: 'prod-host', variable:
                     ↪  'DATABRICKS_HOST'),
                    string(credentialsId: 'prod-token', variable:
                     ↪  'DATABRICKS_TOKEN')
                ]) {
                    sh 'databricks bundle deploy -t prod'
                }
            }
        }
    }

    post {
        failure {
            mail to: 'team@company.com',
                 subject: "Pipeline Failed: ${currentBuild.fullDisplayName}",
                 body: "Check console output at ${env.BUILD_URL}"
        }
    }
}
```

Continue reading: Part 2 — Testing, Secrets & Monitoring

CI/CD Integration — Part 2: Testing, Secrets & Monitoring

Overview

For CI/CD platform configuration (GitHub Actions, Azure DevOps, GitLab, Jenkins), see Part 1.

This part covers testing strategies within CI/CD pipelines, secret management, deployment strategies, monitoring, and common issues.

Overview

- **Testing in CI**: unit tests first, then integration tests, then data quality tests
- **Deployment strategies**: blue-green targets and canary rollouts with bundles
- **CI secrets** live in GitHub Secrets, Azure DevOps variable groups, or HashiCorp Vault, never in workflow YAML
- **Guardrails**: `concurrency:` groups serialize deployments; environment gates add manual approval

What the Exam Tests

- Configure Git-based CI/CD workflows using Databricks Git folders (formerly Repos) to deploy notebooks and code.
- Deploy Databricks resources using Declarative Automation Bundles (formerly Databricks Asset Bundles).
- Order tests and approvals before a production deployment

Testing Strategies

Unit Testing in CI

```
# GitHub Actions example

- name: Run unit tests
  run: |
    pytest tests/unit/ \
      --cov=src \
      --cov-report=xml \
      --cov-report=html \
      -v

- name: Upload coverage
  uses: codecov/codecov-action@v3
  with:
    files: coverage.xml
```

Integration Testing

```
# Run Databricks job as integration test

- name: Run integration tests
  run: |
```

```
    # Deploy test resources
    databricks bundle deploy -t test

    # Run test job and wait for completion
    RUN_ID=$(databricks bundle run integration_test_job -t test --output json |
↪   jq -r '.run_id')

    # Poll for completion
    while true; do
      STATE=$(databricks runs get --run-id $RUN_ID --output json | jq -r
↪   '.state.life_cycle_state')
      if [ "$STATE" == "TERMINATED" ]; then
        RESULT=$(databricks runs get --run-id $RUN_ID --output json | jq -r
↪   '.state.result_state')
        if [ "$RESULT" != "SUCCESS" ]; then
          echo "Integration tests failed"
          exit 1
        fi
        break
      fi
      sleep 30
    done
  env:
    DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
    DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}
```

Data Quality Testing

```python
# tests/integration/test_data_quality.py

def test_bronze_table_schema():
    """Verify bronze table has expected schema."""
    df = spark.table("catalog.bronze.events")
    expected_columns = ["event_id", "timestamp", "user_id", "event_type"]
    assert all(col in df.columns for col in expected_columns)

def test_silver_no_nulls():
    """Verify silver table has no null primary keys."""
    df = spark.table("catalog.silver.events")
    null_count = df.filter(df.event_id.isNull()).count()
    assert null_count == 0, f"Found {null_count} null event_ids"

def test_gold_aggregation():
    """Verify gold aggregation produces expected results."""
    df = spark.table("catalog.gold.daily_summary")
    assert df.count() > 0, "Gold table is empty"
```

Deployment Strategies

Blue-Green Deployment

```
flowchart LR
    subgraph Production
        Blue[Blue Environment<br>Current]
        Green[Green Environment<br>New Version]
    end
```

```
    Traffic[User Traffic] --> Blue
    Deploy[Deploy] --> Green
    Test[Test Green] --> Switch[Switch Traffic]
    Switch --> Green
```

```yaml
# Bundle configuration for blue-green

targets:
  prod-blue:
    variables:
      environment: prod
      slot: blue
    resources:
      jobs:
        etl_job:
          name: "ETL Pipeline - Blue"

  prod-green:
    variables:
      environment: prod
      slot: green
    resources:
      jobs:
        etl_job:
          name: "ETL Pipeline - Green"
```

Canary Deployment

```python
# Canary deployment script

import time

def canary_deploy(job_name, canary_percentage=10):
    """Deploy to canary cluster first, then full rollout."""

    # Deploy to canary
    print(f"Deploying to canary ({canary_percentage}% traffic)")
    deploy_canary(job_name)

    # Monitor canary for issues
    print("Monitoring canary deployment...")
    time.sleep(300)  # 5 minutes

    if check_canary_health(job_name):
        # Full rollout
        print("Canary healthy, proceeding with full rollout")
        deploy_full(job_name)
    else:
        # Rollback
        print("Canary unhealthy, rolling back")
        rollback_canary(job_name)
        raise Exception("Canary deployment failed")
```

Secret Management in CI/CD

GitHub Secrets

```yaml
# Reference secrets in workflow

env:
  DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
  DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}

# Environment-specific secrets

jobs:
  deploy-prod:
    environment: production
    env:
      DATABRICKS_HOST: ${{ secrets.PROD_DATABRICKS_HOST }}
```

Azure DevOps Variable Groups

```yaml
# Reference variable groups

variables:
  - group: databricks-credentials

# Or use Azure Key Vault

variables:
  - group: databricks-keyvault-secrets
```

HashiCorp Vault Integration

```yaml
# GitHub Actions with Vault

- name: Import Secrets
  uses: hashicorp/vault-action@v2
  with:
    url: ${{ secrets.VAULT_URL }}
    token: ${{ secrets.VAULT_TOKEN }}
    secrets: |
      secret/data/databricks/prod host | DATABRICKS_HOST ;
      secret/data/databricks/prod token | DATABRICKS_TOKEN

- name: Deploy
  run: databricks bundle deploy -t prod
```

Monitoring and Notifications

Slack Notifications

```yaml
# GitHub Actions

- name: Notify Slack on failure
  if: failure()
  uses: slackapi/slack-github-action@v1
  with:
    channel-id: 'deployments'
```

```
    slack-message: |
      :x: Deployment failed!
      Workflow: ${{ github.workflow }}
      Branch: ${{ github.ref }}
      Commit: ${{ github.sha }}
  env:
    SLACK_BOT_TOKEN: ${{ secrets.SLACK_BOT_TOKEN }}
```

Email Notifications

```
# Azure DevOps

- task: SendEmail@1
  condition: failed()
  inputs:
    To: 'data-team@company.com'
    Subject: 'Pipeline Failed: $(Build.DefinitionName)'
    Body: |
      Pipeline $(Build.DefinitionName) failed.
      Branch: $(Build.SourceBranch)
      Build: $(Build.BuildNumber)
```

Use Cases

- **Automated Pull Request Validation**: Triggering a GitHub Actions workflow on every PR that deploys a temporary bundle to a staging workspace, runs integration tests, and reports the result back to GitHub.
- **Secure Production Deployment**: Configuring an Azure DevOps pipeline to use a service principal (with Azure Key Vault secrets) to deploy a Databricks job to production only when code is merged to the `main` branch.

Common Issues & Errors

Authentication Failures **Scenario:** CI/CD fails with authentication error.

Fix: Verify credentials and permissions:

```
# Test authentication locally

export DATABRICKS_HOST="https://..."
export DATABRICKS_TOKEN="dapi..."
databricks workspace list /

# Check token expiration (PATs are legacy and have a configurable lifetime);
# prefer a service principal with OAuth M2M or workload identity federation
```

Bundle Validation Errors **Scenario:** Validation fails in CI but works locally.

Fix: Check environment variables and paths:

```
# Ensure all required variables are set

- name: Debug variables
  run: |
    echo "Host: $DATABRICKS_HOST"
    databricks bundle validate --debug
```

Deployment Timeout **Scenario:** Deployment takes too long and times out.

Fix: Increase timeout or optimize deployment:

```
# GitHub Actions

- name: Deploy
  run: databricks bundle deploy -t prod
  timeout-minutes: 30

# There is no per-resource-type deploy flag; `--select` exists for plan/deploy
# but the docs say it is not intended for production. Split large bundles instead.
```

Deployment Lock Conflicts **Scenario:** Multiple CI runs deploy the same target at once and fail on the deployment lock.

Fix: Serialize deployments per target (use `--force-lock` only to clear a stale lock from a crashed run):

```
# GitHub Actions - use concurrency

concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: false
```

Missing Dependencies **Scenario:** Tests fail due to missing Spark in CI.

Fix: Mock Spark or use appropriate test configuration:

```
# conftest.py for unit tests

import pytest
from unittest.mock import MagicMock

@pytest.fixture
def mock_spark():
    spark = MagicMock()
    return spark
```

Exam Tips

1. **Authentication** - Know service principal vs PAT token authentication
2. **GitHub Actions** - Understand workflow syntax and secrets management
3. **Azure DevOps** - Know stages, jobs, and deployment environments
4. **Bundle commands** - validate, deploy, run in CI context
5. **Environment separation** - Use targets for dev/staging/prod
6. **Testing levels** - Unit, integration, data quality testing approaches
7. **Secret management** - Never hardcode credentials in pipelines
8. **Deployment strategies** - Blue-green, canary, rolling deployments
9. **Notifications** - Slack, email, webhook integrations
10. **Concurrency** - Handle parallel deployments and state conflicts

Key Takeaways

- **Service principal over PAT**: Production CI/CD pipelines should authenticate using service principals rather than personal access tokens (PATs) to avoid dependency on individual user credentials.

- **Concurrency gating**: Use GitHub Actions `concurrency:` groups to serialize deployments to a target, so parallel merges do not collide on the bundle's deployment lock and state.
- **Testing pyramid in CI**: Unit tests (local, fast, no Spark) run first; integration tests (real Spark/Databricks) run after; data quality tests (schema and null checks) validate production correctness.
- **Blue-green with bundles**: Implement blue-green deployments by defining two separate `targets` (e.g., `prod-blue`, `prod-green`) in `databricks.yml` with slot-specific resource names.
- **Canary pattern**: Deploy to a small subset first, monitor for a fixed period, then either promote to full rollout or trigger rollback — never deploy directly to 100% of traffic.
- **Secret management**: Never hardcode credentials in workflow YAML; use GitHub Secrets, Azure DevOps variable groups, or HashiCorp Vault for all credential references.
- **Environment gates**: Require manual approvers and wait timers on GitHub Actions production environments to enforce a human-in-the-loop before critical deployments.
- **Deployment timeout handling**: Set explicit `timeout-minutes` on deploy steps; split very large bundles rather than relying on partial deploys.

Related Topics

- Declarative Automation Bundles - DAB configuration
- Git Folders - Version control
- Unit Testing - Testing practices
- Databricks CLI — Part 1 - CLI commands

Official Documentation

- CI/CD with Databricks
- GitHub Actions Integration
- Azure DevOps Integration
- Service Principal Authentication

Git Folders

Overview

Databricks Git folders (formerly Repos) are a visual Git client and API inside the workspace: you clone a remote repository, branch, commit, push, pull, merge, and rebase notebooks and files. For CI/CD, automation can update a production Git folder to a branch or tag through the Repos API or `databricks repos update`, while Declarative Automation Bundles deploy jobs and pipelines as code.

Overview

- Git folders = Repos renamed: same functionality, can live anywhere in the workspace, and support more asset types
- UI operations: clone, branch, commit & push, pull, merge, rebase, reset, resolve conflicts
- CI/CD: a **production Git folder** updated only by automation (`PATCH /api/2.0/repos/{id}` or `databricks repos update <id> --branch|--tag`)
- Databricks recommends bundles to deploy jobs and pipelines; Git folders hold the code

What the Exam Tests

- "Configure Git-based CI/CD workflows using Databricks Git folders (formerly Repos) to deploy notebooks and code."

- Updating a production Git folder from CI to a branch or tag, with a service principal's Git credential
- Choosing Git folders vs. bundles vs. a job's remote Git source (`git_source`)

Architecture

```
flowchart TB
    subgraph GitProvider["Git Provider"]
        GitHub[GitHub]
        GitLab[GitLab]
        Azure[Azure DevOps]
        Bitbucket[Bitbucket]
    end

    subgraph Databricks["Databricks Workspace"]
        GitFolder[Git Folder]
        Notebooks[Notebooks]
        Files[Python/SQL Files]
    end

    GitProvider <-->|Clone/Pull/Push| GitFolder
    GitFolder --> Notebooks
    GitFolder --> Files
```

Git Folders vs Legacy Repos

Feature	Git folders
Location	Any workspace folder, at any depth under your user folder, several per user
Asset types	Files, notebooks, folders, plus DBSQL queries, AI/BI dashboard drafts, and alerts (Publi
Remote URL	**Required**
Existing /Repos paths	Still work: /Repos and /Workspace/Repos are the same folder, so job, dbutils.noteb
Recommendation	Create new Git folders

Platform update (Oct 2026)

Repos are now called **Git folders**: "The core functionality hasn't changed, only the terminology." The API and CLI still use the repos name. Source: What happened to Databricks Repos?

Setting Up Git Folders

Creating a Git Folder

```
UI Steps:
1. Navigate to Workspace (you need CAN MANAGE on the parent folder)
2. New → Git folder
3. Enter repository URL
4. Select Git provider
5. Optionally enable sparse checkout mode
6. Click Create
```

Git Provider Configuration

```
Cloud providers:
- GitHub, GitHub Advanced Enterprise, GitHub Enterprise Cloud
- Atlassian Bitbucket Cloud
- GitLab and GitLab Enterprise Edition
- Microsoft Azure DevOps (Azure Repos)
- AWS CodeCommit

Self-hosted (must be internet-accessible, or use the Git server proxy):
- GitHub Enterprise Server, Bitbucket Server / Data Center,
  GitLab Self-Managed, Azure DevOps Server
```

Git Credential Setup

```
In Databricks: user icon → Settings → Linked accounts → Add Git credential

For GitHub:
- Recommended: the Databricks GitHub app (OAuth 2.0, tokens renewed
  automatically, can be limited to specific repositories)
- Or a fine-grained PAT (Contents: Read and write), or a classic PAT
  with the repo scope (plus workflow to edit GitHub Actions files)

For Azure DevOps:
1. Go to Azure DevOps → User Settings → Personal Access Tokens
2. Create token with:
   - Code (Read & Write)
3. In Databricks: User Settings → Git Integration
4. Select Azure DevOps Services, enter token

For jobs and CI/CD: configure a Git credential for a service principal
```

Working with Git Folders

Basic Git Operations

```
Pull:
- Git dialog → Pull
- Updates the folder with remote changes
- Conflicts with local changes are resolved as merge conflicts

Commit & Push:
- Right-click folder → Git → Commit & Push
- Enter commit message
- Select files to include
- Pushes directly to remote branch

Create Branch:
- Right-click folder → Git → Create Branch
- Enter branch name
- Optionally checkout new branch

Switch Branch:
- Select from available branches
- Discard changes you do not want to carry over first
```

```
Merge / Rebase / Reset:
- Merge another branch, rebase onto one, or reset
- Reset = hard reset plus force push of the branch
```

With the **Git CLI in Git folders** preview, standard `git` commands also run from the web terminal (serverless environment version 5+ or Databricks Runtime 17.0+).

Resolving Conflicts

```
flowchart TD
    Pull[Pull from Remote] --> Conflict{Conflict?}
    Conflict -->|No| Success[Update Complete]
    Conflict -->|Yes| Options[Resolution Options]
    Options --> Keep[Keep Local]
    Options --> Take[Take Remote]
    Options --> Manual[Manual Merge]
    Manual --> Edit[Edit Files]
    Edit --> Commit[Commit Resolution]
```

```
Conflict Resolution (merge, rebase, or pull):
1. The Git folders UI lists the conflicting files
2. Choose per file:
   - Keep all current changes
   - Take all incoming changes
   - Edit the file manually
3. Or abort the operation to discard the conflicting changes
4. Commit and push the resolution
```

File Organization

Supported File Types New notebooks default to **IPYNB** format. Git folders recognize a notebook by the `.ipynb` extension or by a marker such as `# Databricks notebook source` at the top of a source file. Outputs of `.ipynb` notebooks are committed only if a workspace admin allows it (controlled per repo by `.databricks/commit_outputs`). MLflow experiments, legacy alerts, and Genie Agents cannot be committed.

File Type	Extension	Notes
Jupyter Notebook (default)	.ipynb	Keeps dashboards and visualizations
Python Notebook	.py	With # Databricks notebook source header
SQL Notebook	.sql	With -- Databricks notebook source header
Scala Notebook	.scala	With // Databricks notebook source header
R Notebook	.r	With # Databricks notebook source header
Python File	.py	Regular Python module
SQL File	.sql	SQL scripts
YAML	.yml, .yaml	Configuration files
JSON	.json	Configuration files
Markdown	.md	Documentation

Project Structure Best Practices

```
my-data-project/
├── databricks.yml              # Bundle configuration
├── README.md                   # Project documentation
├── src/
│   ├── notebooks/
```

```
│   │   ├── bronze/
│   │   │   └── ingest.py      # Databricks notebook
│   │   ├── silver/
│   │   │   └── transform.py
│   │   └── gold/
│   │       └── aggregate.py
│   └── python/
│       ├── __init__.py
│       ├── utils.py           # Regular Python module
│       └── transformations.py
├── tests/
│   ├── unit/
│   │   └── test_utils.py
│   └── integration/
│       └── test_pipeline.py
├── config/
│   ├── dev.yml
│   └── prod.yml
└── resources/
    ├── jobs.yml
    └── pipelines.yml
```

Notebook Source Format

```python
# Databricks notebook source
# MAGIC %md
# MAGIC # Bronze Ingestion Notebook
# MAGIC This notebook ingests raw data from source systems.

# COMMAND ----------

# MAGIC %md
# MAGIC ## Configuration

# COMMAND ----------

dbutils.widgets.text("source_path", "")
dbutils.widgets.text("target_table", "")

source_path = dbutils.widgets.get("source_path")
target_table = dbutils.widgets.get("target_table")

# COMMAND ----------

# MAGIC %md
# MAGIC ## Read Source Data

# COMMAND ----------

df = spark.read.format("json").load(source_path)

# COMMAND ----------
```

```
# MAGIC %md
# MAGIC ## Write to Bronze Table

# COMMAND ----------

df.write.format("delta").mode("append").saveAsTable(target_table)
```

Branching Strategies

GitFlow for Data Projects

```
gitGraph
    commit id: "initial"
    branch develop
    commit id: "feature-base"
    branch feature/new-pipeline
    commit id: "add-bronze"
    commit id: "add-silver"
    checkout develop
    merge feature/new-pipeline
    branch release/1.0
    commit id: "version-bump"
    checkout main
    merge release/1.0 tag: "v1.0"
    checkout develop
    merge release/1.0
```

Environment Branches

```
Branch Strategy:
├── main (production)
│   └── Protected, requires PR approval
├── staging
│   └── Pre-production testing
├── develop
│   └── Integration branch
└── feature/*
    └── Individual feature branches
```

PR Workflow

```
sequenceDiagram
    participant Dev as Developer
    participant Feature as feature/branch
    participant Develop as develop
    participant Main as main

    Dev->>Feature: Create branch
    Dev->>Feature: Commit changes
    Dev->>Develop: Open PR
    Develop->>Develop: CI tests run
    Develop->>Develop: Code review
    Develop->>Develop: Merge PR
    Develop->>Main: Open release PR
    Main->>Main: Deploy to production
```

Integration with Workflows

Running Notebooks from Git Folders

```
# In a job task configuration

{
    "notebook_task": {
        "notebook_path": "/Workspace/Users/user@company.com/my-
        ↪  project/src/notebooks/bronze/ingest",
        "base_parameters": {
            "source_path": "/mnt/raw/events",
            "target_table": "bronze.events"
        }
    }
}
```

%run with Git Folders

```
# Databricks notebook source

# Run a notebook from the same Git folder
# COMMAND ----------

# Relative path from current notebook

%run ./utils/common_functions

# COMMAND ----------

# Or absolute workspace path

%run /Workspace/Users/user@company.com/my-project/src/notebooks/utils/
 ↪  common_functions
```

Importing Python Modules On Databricks Runtime 11.3 LTS and above, the notebook's working directory and the **Git folder root** are added to `sys.path` automatically, so modules at the repo root import directly. Use `sys.path.append` only for other directories (on serverless the working directory is not guaranteed, so use absolute paths).

```
# Databricks notebook source

# Only needed because src/python is not the Git folder root

import sys
sys.path.append("/Workspace/Users/user@company.com/my-project/src/python")

# COMMAND ----------

# Import project modules

from utils import data_validation
from transformations import clean_data
```

```
# COMMAND ----------

# Use imported functions

df = spark.table("bronze.events")
df_cleaned = clean_data(df)
data_validation.check_nulls(df_cleaned)
```

Automating Git Operations

Production Git Folder (CI/CD for Code) Databricks recommends **Declarative Automation Bundles** for CI/CD. If you deploy only code, use a **production Git folder**:

1. Choose the repository and deployment branch.
2. Give a service principal a Git credential for that repository.
3. Create the Git folder for that repo and branch in a subfolder under `/Workspace` (not under a user folder).
4. Share it: **Can Run** for project users and automation service principals; optionally **Can View** for everyone. Only admins and service principals get edit rights.
5. Keep it in sync with automation only: external CI/CD after a PR merges into the deployment branch, or a scheduled job that calls the API. Never edit it by hand.

```
flowchart LR
    PR[PR merged to main] --> CI[CI runner as service principal]
    CI --> Update["databricks repos update /Workspace/Production/my-project
↪   --branch main"]
    Update --> Folder[Production Git folder]
    Folder --> Job[Jobs run notebooks from the folder]
```

Repos API for Git Folders

Operation	Endpoint
Create (clone)	`POST /api/2.0/repos` with `url`, `provider`, `pat`
Get	`GET /api/2.0/repos/{id}`
Update: checkout a branch or tag, or pull the latest commit	`PATCH /api/2.0/repos/{id}` with `branch` **or** `ta`
Delete	`DELETE /api/2.0/repos/{id}`
List	`GET /api/2.0/repos` (deprecated: omits repos

Updating to a **tag** leaves the Git folder in a **detached HEAD** state, which suits release deployments.

```
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()  # authenticates as the CI service principal

# Pull the latest commit of main into the production Git folder
w.repos.update(repo_id=12345, branch="main")

# Or pin a release tag (detached HEAD)
w.repos.update(repo_id=12345, tag="v1.4.0")
```

CLI Operations

```
# Get a Git folder by ID or workspace path
databricks repos get 12345
```

```
# Checkout + pull the latest commit of a branch (ID or path)
databricks repos update 12345 --branch main
databricks repos update /Workspace/Production/my-project --branch main

# Checkout a release tag
databricks repos update 12345 --tag v1.4.0

# Create (clone) a Git folder
databricks repos create https://github.com/company/project.git gitHub \
    --path /Workspace/Production/my-project
```

Remote Git Source in Jobs (Alternative) A job can skip the workspace copy entirely: set `git_source` (`git_url`, `git_provider`, and one of `git_branch`, `git_tag`, `git_commit`). Notebook, Python script, dbt, and SQL file tasks then read files from the remote repository at run time.

Security and Permissions

Git Folder Permissions

```
Permission Levels (Git folder ACLs):
- CAN READ: View contents
- CAN RUN: Run notebooks
- CAN EDIT: Edit and rename assets
- CAN MANAGE: Create/switch branches, pull and push, create/import/
  delete/move assets, change permissions

Most Git operations require CAN MANAGE.
```

Credential Management

```
Git Credentials Storage:
1. Personal: User Settings → Git Integration
   - Stored per user
   - Used for that user's operations

2. Service principal:
   - Configure a Git credential for the service principal
   - Recommended for jobs and CI/CD automation
   - No shared "workspace" Git credential: each identity has its own
```

Branch Protection

```
Enforce in Git Provider:
1. Require PR reviews before merge
2. Require status checks (CI)
3. Require signed commits
4. Restrict who can push to protected branches

Databricks follows Git provider rules
```

Comparison: Git Folders vs Declarative Automation Bundles

Aspect	Git Folders	Declarative Automation Bund
Purpose	Version control UI	Deployment automation
Workflow	Interactive development	CI/CD deployment
Configuration	Git provider settings	databricks.yml
Deployment	Production Git folder updated by `repos update` / Repos API	`databricks bundle deploy`
What is deployed	Code only (notebooks, files)	Code **plus** jobs, pipelines, ar
Best for	Development; code-only deployments	Production deployment (Data

When to Use Each

```
Git Folders:
- Interactive notebook development
- Collaborative editing
- Quick prototyping
- Viewing version history

Declarative Automation Bundles:
- Automated deployments
- Environment promotion
- Resource configuration as code
- CI/CD pipelines
```

Combined Workflow

```
flowchart TB
    subgraph Development
        GitFolder[Git Folder in Workspace]
        Edit[Edit Notebooks]
        Commit[Commit & Push]
    end

    subgraph CI/CD
        PR[Pull Request]
        Test[Run Tests]
        Bundle[Bundle Validate]
    end

    subgraph Deployment
        Deploy[Bundle Deploy]
        Prod[Production]
    end

    GitFolder --> Edit
    Edit --> Commit
    Commit --> PR
    PR --> Test
    Test --> Bundle
    Bundle --> Deploy
    Deploy --> Prod
```

Use Cases

- **Collaborative Notebook Development**: Multiple data scientists working in their own personal Git Folders, cloning a shared GitHub repository, developing in isolated branches, and using the Databricks UI to commit and push changes.

- **Lightweight Code Execution**: Using `%run` or Python `import` statements to execute modular utility scripts stored in the Git Folder repository without the overhead of building and deploying a Python wheel.

Common Issues & Errors

Authentication Failed **Scenario:** Cannot clone or push to repository.

Fix:

```
1. Verify Git credentials in User Settings
2. Check token permissions (repo scope for GitHub)
3. Ensure token hasn't expired
4. For SSO: Re-authenticate with Git provider
```

Merge Conflicts **Scenario:** Pull fails due to conflicts.

Fix:

```
1. Save local changes externally if needed
2. Discard local changes: Right-click → Git → Discard Changes
3. Pull again
4. Reapply changes manually
5. Commit and push
```

Notebook Format Issues **Scenario:** Notebook doesn't render correctly.

Fix:

```
# Ensure notebook has correct header
# For Python:
# Databricks notebook source

# For SQL:

-- Databricks notebook source

# For Scala:

// Databricks notebook source
```

Missing Files After Pull **Scenario:** Files missing after Git operation.

Fix:

```
1. Check branch: Ensure correct branch selected
2. Check .gitignore: File might be ignored
3. Refresh workspace: Close and reopen folder
4. Check permissions: Ensure read access to files
```

Large Repository Performance **Scenario:** Operations slow on large repos.

Fix:

```
1. Use sparse checkout mode (cone patterns) for large repos
2. Keep total workspace assets and files under 20,000; avoid monorepos
3. Remember the limits: working branches up to 1 GB, each Git operation
```

```
   up to 2 GB memory and 4 GB disk writes, files over 10 MB not viewable in the UI
4. Split into smaller repositories
```

Best Practices

Development Workflow

```
1. Create feature branch for each change
2. Make small, focused commits
3. Write descriptive commit messages
4. Pull frequently to stay updated
5. Use PR reviews for quality control
6. Delete merged branches
```

Repository Organization

```
1. One repository per project/domain
2. Clear folder structure (src/, tests/, config/)
3. Include README with setup instructions
4. Use .gitignore for generated files
5. Keep notebooks focused and modular
6. Separate code from configuration
```

Collaboration

```
1. Establish branching conventions
2. Document code review process
3. Use consistent naming conventions
4. Communicate before major changes
5. Keep main branch always deployable
```

Exam Tips

Tip

- **Git folders = Repos renamed**. Exam text says ”Databricks Git folders (formerly Repos)”; API and CLI still say `repos`.
- **CI/CD with Git folders**: a production Git folder, updated only by automation with `databricks repos update <id-or-path> --branch <b>` or `--tag <t>` (PATCH `/api/2.0/repos/{id}`), authenticated as a service principal with a Git credential.
- **Tag = detached HEAD**: pinning a release tag is the safe way to deploy a fixed version.
- **Bundles vs Git folders**: bundles deploy code plus jobs and pipelines and are Databricks' recommended CI/CD path; Git folders alone deploy code only.
- **Permissions**: pulling, pushing, and switching branches need **CAN MANAGE**; production folders give most users **Can Run**.
- **Imports**: the Git folder root is on `sys.path` automatically (DBR 11.3+).
- **Credentials** are per identity (user or service principal); the Databricks GitHub app is recommended for GitHub.

Key Takeaways

- **Git folders supersede the Repos name**: same functionality, can live anywhere in the workspace, require a remote URL; existing `/Repos` paths keep working.

- **Supported providers**: GitHub, Bitbucket Cloud, GitLab, Azure DevOps, and AWS CodeCommit in the cloud, plus self-hosted servers that are internet-accessible or reached through the Git server proxy.
- **Notebook formats**: new notebooks default to IPYNB; source-format notebooks are recognized by the `# Databricks notebook source` header.
- **Permission levels**: CAN READ, CAN RUN, CAN EDIT, CAN MANAGE; most Git operations need CAN MANAGE.
- **Conflict resolution**: keep current changes, take incoming changes, edit manually, or abort.
- **Production Git folder**: admin-created, run-only for users, updated only by automation after merges.
- **CLI/API**: `databricks repos update <id-or-path> --branch main` checks out and pulls the latest commit; `--tag v1.0.0` pins a release.
- **Combined workflow**: develop in personal Git folders, review through PRs, deploy with bundles (or a production Git folder for code-only deployments).

Related Topics

- Declarative Automation Bundles — Part 1 - Deployment automation
- CI/CD Integration — Part 1 - Pipeline workflows
- Workspace and Notebooks - Notebook features

Official Documentation

- Databricks Git folders
- What happened to Databricks Repos?
- Git folders concepts
- Create and manage Git folders
- CI/CD with Databricks Git folders
- Connect your Git provider to Databricks
- Git folder limits and reference
- Supported asset types in Git folders
- Notebook formats
- Work with Python modules
- Access control lists (Git folder ACLs)
- Repos API
- `repos` command group (CLI)

Section 9: Data Modeling (5 % of Exam)

Designing tables that stay fast at scale: the medallion architecture, Delta fundamentals, table layouts (partitioning by grain, clustering by access pattern, compaction), schema management, SCD patterns, and dimensional models that use materialized views and Unity Catalog metric views.

Topics Overview

```
flowchart LR
    D[Data Modeling]
    D --> T1["Medallion Architecture"]
    D --> T2["Delta Lake Fundamentals"]
    D --> T3["Partitioning Strategies"]
    D --> T4["Schema Management"]
    D --> T5["Slowly Changing Dimension (SCD) Patterns"]
    D --> T6["Dimensional Modeling and Metric Views"]
```

Section Contents

File	Topic
01-medallion-architecture.md	Bronze / silver / gold layers and multi-hop processing
02-delta-lake-fundamentals.md	Delta architecture, ACID, constraints, generated and identity
03-partitioning-strategies.md	Table layouts for large Delta / Iceberg tables: partitioning, cl
04-schema-management.md	Schema enforcement and evolution, column mapping
05-scd-patterns.md	SCD Types 1, 2, 3, 4, 6 and AUTO CDC for SCD Type 2
06-dimensional-modeling-and-metric-views.md	Star schemas, materialized views, Unity Catalog metric view

Exam Skill Coverage

Every skill bullet in Section 9 of the Oct 9, 2026 exam guide, with the files that cover it.

Exam guide skill
Design scalable Delta/Iceberg table layouts for large data assets by mapping partitioning to data grain, alignin
Design dimensional models for analytical workloads, leveraging Materialized Views for pre-computed aggrega

Key Concepts to Master

Concept	Why it matters
Partition by grain	Partition only large tables, on a low-cardinality column such as date; otherw
Clustering by access pattern	Choose clustering keys from the columns used in filters and joins
Metric view	A governed, reusable metric definition (dimensions + measures) queried wi
Materialized view	Pre-computed aggregate that dashboards read directly, refreshed incremen

Related Resources

- Medallion Architecture (shared)
- Delta Lake cheat sheet

Medallion Architecture

The medallion architecture (also known as multi-hop architecture) is a data design pattern for organizing data in a lakehouse. It progressively improves data quality across Bronze, Silver, and Gold layers.

Overview

```
flowchart LR
    Sources[Data Sources] --> Bronze[Bronze Layer]
    Bronze --> Silver[Silver Layer]
    Silver --> Gold[Gold Layer]
    Gold --> Consumers[Consumers]

    Bronze -.- B1["Raw data"]
    Silver -.- S1["Cleansed data"]
    Gold -.- G1["Business-ready"]

    Consumers --> BI[BI/Analytics]
    Consumers --> ML[ML/AI]
    Consumers --> Apps[Applications]
```

Overview

- **Bronze**: append-only raw data with metadata columns; **Silver**: enforced schema, deduplication, and MERGE; **Gold**: business aggregates and star schemas
- **Quarantine pattern**: route invalid rows to a separate table with an `_error_reason` column
- **Lakeflow Pipelines** maps cleanly to the three layers
- **Unity Catalog organization**: catalogs per environment with schemas per layer, or domain-based schemas

What the Exam Tests

- Design dimensional models for analytical workloads, leveraging Materialized Views for pre-computed aggregation and Unity Catalog Metric Views for governed, reusable metric definitions, to ensure efficient querying and aggregation.
- Place each transformation, check, and deduplication step in the right layer
- Organize medallion layers in Unity Catalog

Layer Characteristics

Comparison Table

Aspect	Bronze	Silver	Gold
Data Quality	Raw, as-is	Cleansed, validated	Curated, business-ready
Schema	Flexible (schema-on-read)	Enforced	Strict, documented
Update Pattern	Append-only	Merge/Upsert	Aggregated
Deduplication	None	Yes	Yes
Data Types	Often strings	Properly typed	Optimized
Transformations	Minimal	Cleaning, standardization	Business logic
Retention	Long (audit)	Medium-Long	Based on requirements
Users	Data engineers	Data engineers, analysts	Analysts, scientists, apps

Data Quality Progression

```
flowchart TB
    subgraph Quality["Data Quality"]
        direction LR
        Q1[Low] --> Q2[Medium] --> Q3[High]
    end

    subgraph Layers["Layers"]
        direction LR
        Bronze --> Silver --> Gold
    end

    Quality -.-> Layers
```

Bronze Layer

The Bronze layer captures raw data exactly as received from source systems.

Bronze Layer Principles

```
flowchart TB
    Sources[Multiple Sources] --> Ingest[Ingestion]
    Ingest --> Bronze[Bronze Tables]

    subgraph BronzePrinciples["Bronze Principles"]
        A1[Append-only writes]
        A2[Preserve source schema]
        A3[Add metadata columns]
        A4[No business transformations]
        A5[Full data lineage]
    end

    Bronze --> BronzePrinciples
```

Bronze Table Design

```
from pyspark.sql.functions import current_timestamp, input_file_name, lit

# Read raw data with minimal transformation

raw_df = spark.read.format("json").load("/Volumes/raw/landing/customers/")

# Add ingestion metadata (standard Bronze practice)

bronze_df = (raw_df
    .withColumn("_ingested_at", current_timestamp())
    .withColumn("_source_file", input_file_name())
    .withColumn("_source_system", lit("crm_api")))

# Write to Bronze layer (append-only)

(bronze_df.write
    .format("delta")
    .mode("append")
```

```
    .option("mergeSchema", "true")
    .saveAsTable("bronze.customers_raw"))
```

```
-- Bronze table with metadata columns
CREATE TABLE bronze.customers_raw (
    -- Source columns (preserved as-is)
    customer_id STRING,
    name STRING,
    email STRING,
    raw_data STRING,  -- Store entire payload for complex sources

    -- Metadata columns
    _ingested_at TIMESTAMP,
    _source_file STRING,
    _source_system STRING,
    _batch_id STRING
)
USING DELTA
PARTITIONED BY (_ingested_at::date);
```

Auto Loader for Bronze Ingestion

```
# Auto Loader for incremental Bronze ingestion

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/checkpoints/customers/_schema")
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load("/Volumes/raw/landing/customers/"))

# Add metadata and write to Bronze

(df.withColumn("_ingested_at", current_timestamp())
  .withColumn("_source_file", input_file_name())
  .writeStream
  .format("delta")
  .option("checkpointLocation", "/checkpoints/customers/_checkpoint")
  .option("mergeSchema", "true")
  .trigger(availableNow=True)
  .toTable("bronze.customers_raw"))
```

Bronze Layer Best Practices

Practice	Rationale
Append-only writes	Preserves complete history for replay
Schema-on-read	Handle source schema changes gracefully
Add ingestion metadata	Enable debugging and lineage tracking
Store raw payloads	Allow reprocessing with different logic
Enable schema evolution	Accommodate new source fields
Partition by ingestion date	Efficient retention management

Silver Layer

The Silver layer contains cleansed, validated, and deduplicated data.

Silver Layer Principles

```
flowchart TB
    Bronze[Bronze Layer] --> Transform[Transformations]
    Transform --> Silver[Silver Tables]

    subgraph SilverOps["Silver Operations"]
        C1[Data cleansing]
        C2[Schema enforcement]
        C3[Deduplication]
        C4[Data type casting]
        C5[Standardization]
        C6[Data validation]
    end

    Transform --> SilverOps
```

Silver Table Design

```
from pyspark.sql.functions import col, when, regexp_replace, lower, trim

# Read from Bronze

bronze_df = spark.read.table("bronze.customers_raw")

# Clean and transform to Silver

silver_df = (bronze_df
    .dropDuplicates(["customer_id"])
    .withColumn("customer_id", col("customer_id").cast("int"))
    .withColumn("email", lower(trim(col("email"))))
    .withColumn("name", trim(col("name")))
    .withColumn("is_valid_email",
        col("email").rlike("^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$"))
    .filter(col("customer_id").isNotNull())
    .select(
        "customer_id",
        "name",
        "email",
        "is_valid_email",
        "_source_system",
        "_ingested_at"
    ))

# Write to Silver with MERGE for idempotency

silver_df.createOrReplaceTempView("bronze_updates")

spark.sql("""
    MERGE INTO silver.customers AS target
    USING bronze_updates AS source
    ON target.customer_id = source.customer_id
```

```
    WHEN MATCHED THEN UPDATE SET *
    WHEN NOT MATCHED THEN INSERT *
""")
```

```
-- Silver table with enforced schema and constraints
CREATE TABLE silver.customers (
    customer_id INT NOT NULL,
    name STRING NOT NULL,
    email STRING,
    is_valid_email BOOLEAN,
    _source_system STRING,
    _ingested_at TIMESTAMP,

    -- Constraints for data quality
    CONSTRAINT valid_customer_id CHECK (customer_id > 0)
)
USING DELTA
TBLPROPERTIES (
    'delta.columnMapping.mode' = 'name',
    'delta.minReaderVersion' = '2',
    'delta.minWriterVersion' = '5'
);
```

Silver MERGE Pattern

```
from delta.tables import DeltaTable

# Incremental Silver update with MERGE

def update_silver_customers(bronze_df):
    # Clean and prepare data
    updates_df = (bronze_df
        .dropDuplicates(["customer_id"])
        .withColumn("customer_id", col("customer_id").cast("int"))
        .filter(col("customer_id").isNotNull()))

    # Get Silver table
    silver_table = DeltaTable.forName(spark, "silver.customers")

    # MERGE updates
    (silver_table.alias("target")
        .merge(
            updates_df.alias("source"),
            "target.customer_id = source.customer_id"
        )
        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute())
```

Silver Layer Best Practices

Practice	Rationale
Enforce schema	Catch data quality issues early
Use MERGE for updates	Ensure idempotent processing
Add constraints	Validate data at write time

Practice	Rationale
Deduplicate records	Single source of truth
Proper data types	Enable efficient queries
Maintain audit columns	Track data lineage

Gold Layer

The Gold layer contains business-level aggregates and consumption-ready datasets.

Gold Layer Principles

```
flowchart TB
    Silver[Silver Layer] --> Aggregate[Aggregation & Business Logic]
    Aggregate --> Gold[Gold Tables]

    subgraph GoldOps["Gold Operations"]
        G1[Business aggregates]
        G2[KPIs and metrics]
        G3[Feature engineering]
        G4[Dimensional modeling]
        G5[Pre-computed joins]
    end

    Aggregate --> GoldOps

    Gold --> BI[BI Dashboards]
    Gold --> ML[ML Models]
    Gold --> Reports[Reports]
```

Gold Table Design

```
from pyspark.sql.functions import sum, count, avg, max, date_trunc

# Read from Silver

customers = spark.read.table("silver.customers")
orders = spark.read.table("silver.orders")

# Create Gold aggregate table

customer_metrics = (customers.join(orders, "customer_id")
    .groupBy("customer_id", "name", "email")
    .agg(
        count("order_id").alias("total_orders"),
        sum("order_amount").alias("lifetime_value"),
        avg("order_amount").alias("avg_order_value"),
        max("order_date").alias("last_order_date")
    ))

# Write to Gold layer

(customer_metrics.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("gold.customer_metrics"))
```

```sql
-- Gold table for customer metrics
CREATE OR REPLACE TABLE gold.customer_metrics AS
SELECT
    c.customer_id,
    c.name,
    c.email,
    COUNT(o.order_id) AS total_orders,
    SUM(o.order_amount) AS lifetime_value,
    AVG(o.order_amount) AS avg_order_value,
    MAX(o.order_date) AS last_order_date,
    CASE
        WHEN SUM(o.order_amount) > 10000 THEN 'Platinum'
        WHEN SUM(o.order_amount) > 5000 THEN 'Gold'
        WHEN SUM(o.order_amount) > 1000 THEN 'Silver'
        ELSE 'Bronze'
    END AS customer_tier
FROM silver.customers c
LEFT JOIN silver.orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.email;

-- Daily sales metrics (fact table pattern)
CREATE OR REPLACE TABLE gold.daily_sales_metrics AS
SELECT
    DATE_TRUNC('day', order_date) AS order_date,
    region,
    product_category,
    COUNT(DISTINCT customer_id) AS unique_customers,
    COUNT(order_id) AS total_orders,
    SUM(order_amount) AS total_revenue,
    AVG(order_amount) AS avg_order_value
FROM silver.orders
GROUP BY 1, 2, 3;
```

Dimensional Modeling in Gold

```sql
-- Dimension table
CREATE TABLE gold.dim_customer (
    customer_key BIGINT GENERATED ALWAYS AS IDENTITY,
    customer_id INT,
    name STRING,
    email STRING,
    customer_tier STRING,
    is_current BOOLEAN,
    effective_date DATE,
    end_date DATE
)
USING DELTA;

-- Fact table
CREATE TABLE gold.fact_orders (
    order_key BIGINT GENERATED ALWAYS AS IDENTITY,
    customer_key BIGINT,
    product_key BIGINT,
    date_key INT,
    order_id STRING,
    order_amount DECIMAL(18,2),
```

```
    quantity INT
)
USING DELTA;

-- Date dimension
CREATE TABLE gold.dim_date (
    date_key INT,
    full_date DATE,
    day_of_week INT,
    day_name STRING,
    month INT,
    month_name STRING,
    quarter INT,
    year INT,
    is_weekend BOOLEAN,
    is_holiday BOOLEAN
)
USING DELTA;
```

Gold Layer Best Practices

Practice	Rationale
Pre-aggregate data	Fast query performance
Apply business logic	Consistent business rules
Create star schemas	Efficient BI queries
Optimize for reads	Primary use is consumption
Document calculations	Business user transparency
Schedule refreshes	Data freshness requirements

Multi-Hop Processing with Lakeflow Pipelines

Lakeflow Pipelines (formerly DLT) simplifies medallion architecture implementation.

```
import dlt
from pyspark.sql.functions import col, current_timestamp, input_file_name

# Bronze Layer

@dlt.table(
    comment="Raw customer data from CRM"
)
def bronze_customers():
    return (
        spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.schemaLocation", "/checkpoints/customers/_schema")
        .load("/Volumes/raw/landing/customers/")
        .withColumn("_ingested_at", current_timestamp())
        .withColumn("_source_file", input_file_name())
    )

# Silver Layer
```

```python
@dlt.table(
    comment="Cleansed customer data"
)
@dlt.expect("valid_customer_id", "customer_id IS NOT NULL")
@dlt.expect_or_drop("valid_email", "email RLIKE '^[a-zA-Z0-9._%+-]+@'")
def silver_customers():
    return (
        dlt.read_stream("bronze_customers")
        .withColumn("customer_id", col("customer_id").cast("int"))
        .withColumn("email", lower(trim(col("email"))))
        .dropDuplicates(["customer_id"])
    )

# Gold Layer

@dlt.table(
    comment="Customer metrics for analytics"
)
def gold_customer_metrics():
    customers = dlt.read("silver_customers")
    orders = dlt.read("silver_orders")

    return (
        customers.join(orders, "customer_id")
        .groupBy("customer_id", "name", "email")
        .agg(
            count("order_id").alias("total_orders"),
            sum("order_amount").alias("lifetime_value")
        )
    )
```

Unity Catalog Organization

Recommended Catalog Structure

```
flowchart TB
    subgraph Catalogs["Catalogs"]
        Dev[dev]
        Staging[staging]
        Prod[prod]
    end

    subgraph Schemas["Schemas (in each catalog)"]
        Bronze[bronze]
        Silver[silver]
        Gold[gold]
    end

    Catalogs --> Schemas
```

```sql
-- Create catalog structure
CREATE CATALOG IF NOT EXISTS prod;

CREATE SCHEMA IF NOT EXISTS prod.bronze;
CREATE SCHEMA IF NOT EXISTS prod.silver;
CREATE SCHEMA IF NOT EXISTS prod.gold;
```

```sql
-- Create Bronze table
CREATE TABLE prod.bronze.customers_raw (...)
USING DELTA;

-- Create Silver table
CREATE TABLE prod.silver.customers (...)
USING DELTA;

-- Create Gold table
CREATE TABLE prod.gold.customer_metrics (...)
USING DELTA;
```

Alternative: Domain-Based Organization

```sql
-- Domain-based organization
CREATE CATALOG IF NOT EXISTS prod;

-- By domain
CREATE SCHEMA IF NOT EXISTS prod.sales_bronze;
CREATE SCHEMA IF NOT EXISTS prod.sales_silver;
CREATE SCHEMA IF NOT EXISTS prod.sales_gold;

CREATE SCHEMA IF NOT EXISTS prod.marketing_bronze;
CREATE SCHEMA IF NOT EXISTS prod.marketing_silver;
CREATE SCHEMA IF NOT EXISTS prod.marketing_gold;
```

Error Handling Patterns

Quarantine Pattern

```
flowchart LR
    Source[Source Data] --> Validate{Valid?}
    Validate --> |Yes| Silver[Silver Table]
    Validate --> |No| Quarantine[Quarantine Table]
    Quarantine --> Review[Manual Review]
    Review --> |Fixed| Silver
```

```python
# Separate valid and invalid records

bronze_df = spark.read.table("bronze.customers_raw")

# Define validation rules

valid_df = bronze_df.filter(
    (col("customer_id").isNotNull()) &
    (col("email").rlike("^[a-zA-Z0-9._%+-]+@"))
)

invalid_df = bronze_df.filter(
    (col("customer_id").isNull()) |
    (~col("email").rlike("^[a-zA-Z0-9._%+-]+@"))
).withColumn("_error_reason",
    when(col("customer_id").isNull(), "null_customer_id")
    .when(~col("email").rlike("^[a-zA-Z0-9._%+-]+@"), "invalid_email")
    .otherwise("unknown")
```

```
)

# Write valid records to Silver

valid_df.write.mode("append").saveAsTable("silver.customers")

# Write invalid records to quarantine

invalid_df.write.mode("append").saveAsTable("silver.customers_quarantine")
```

Data Lineage

Tracking Lineage Across Layers

```
# Add lineage tracking columns

def add_lineage(df, source_table, transform_name):
    return (df
        .withColumn("_source_table", lit(source_table))
        .withColumn("_transform_name", lit(transform_name))
        .withColumn("_transform_timestamp", current_timestamp()))

# Bronze to Silver

bronze_df = spark.read.table("bronze.customers_raw")
silver_df = clean_customers(bronze_df)
silver_df = add_lineage(silver_df, "bronze.customers_raw", "clean_customers")
silver_df.write.mode("append").saveAsTable("silver.customers")

# Silver to Gold

customers = spark.read.table("silver.customers")
gold_df = create_customer_metrics(customers)
gold_df = add_lineage(gold_df, "silver.customers", "create_customer_metrics")
gold_df.write.mode("overwrite").saveAsTable("gold.customer_metrics")
```

Use Cases

Retail Analytics

Layer	Tables	Purpose
Bronze	`transactions_raw`, `inventory_raw`, `customers_raw`	Raw POS, inventory feeds
Silver	`transactions`, `inventory`, `customers`	Cleansed, deduplicated
Gold	`daily_sales`, `customer_segments`, `inventory_alerts`	KPIs, ML features

IoT Data Pipeline

Layer	Tables	Purpose
Bronze	`sensor_readings_raw`	All sensor events
Silver	`sensor_readings`	Validated, normalized readings
Gold	`device_health_scores`, `anomaly_alerts`	Aggregates, ML predictions

Common Issues & Errors

Schema Drift in Bronze **Scenario:** New columns appear in source data.

Fix: Enable schema evolution:

```
.option("mergeSchema", "true")
```

Duplicate Records in Silver **Scenario:** MERGE creates duplicates due to late-arriving data.

Fix: Use proper deduplication window:

```
window = Window.partitionBy("customer_id").orderBy(col("_ingested_at").desc())
deduped = df.withColumn("rn", row_number().over(window)).filter("rn = 1")
```

Stale Gold Aggregates **Scenario:** Gold tables not reflecting latest Silver data.

Fix: Schedule Gold refreshes appropriately or use streaming tables.

Cross-Layer Inconsistency **Scenario:** Silver and Gold tables out of sync.

Fix: Use Lakeflow Pipelines for dependency management or implement proper orchestration.

Exam Tips

1. **Layer purposes** - Bronze (raw), Silver (cleansed), Gold (aggregated)
2. **Bronze patterns** - Append-only, preserve source schema, add metadata
3. **Silver patterns** - MERGE for updates, enforce schema, deduplicate
4. **Gold patterns** - Pre-aggregate, business logic, optimize for queries
5. **Lakeflow Pipelines alignment** - Bronze → Silver → Gold maps to streaming/materialized tables
6. **Quarantine pattern** - Separate invalid records for later processing
7. **Schema evolution** - Use `mergeSchema` in Bronze, enforce in Silver
8. **Partition strategy** - By date in Bronze, by business key in Silver/Gold
9. **Catalog organization** - Consider layer-based or domain-based schemas
10. **Lineage tracking** - Add metadata columns for debugging

Key Takeaways

- **Bronze layer principles**: append-only writes, preserve source schema, add metadata columns (`_ingested_at`, `_source_file`, `_source_system`), no business transformations, enable schema evolution with `mergeSchema=true`
- **Silver layer principles**: enforce schema with NOT NULL constraints and CHECK constraints, deduplicate records using `row_number()` or MERGE, use MERGE for idempotent upserts from Bronze
- **Gold layer patterns**: pre-aggregate for query performance, apply consistent business logic, build star schemas (fact + dimension tables) optimized for read-heavy consumption
- **Quarantine pattern**: separate valid from invalid records during Silver processing; route invalid rows to a quarantine table with an `_error_reason` column for later remediation
- **Lakeflow Pipelines maps cleanly to medallion**: Bronze = streaming ingestion with Auto Loader, Silver = expectations + dedup + MERGE, Gold = aggregation tables
- **Unity Catalog organization**: use catalogs per environment (dev/staging/prod) with schemas per layer (bronze/silver/gold), or domain-based schemas (sales_bronze, sales_silver, etc.)
- **Schema drift in Bronze**: handled with `mergeSchema=true` and Auto Loader `schemaEvolutionMode=addNewColumns`; schema enforcement belongs in Silver

- **Partition strategy by layer**: Bronze partitioned by ingestion date for retention management; Silver/Gold partitioned by business date or business key

Related Topics

- Delta Lake Fundamentals - Underlying technology
- Schema Management - Schema evolution across layers
- Lakeflow Pipelines - Lakeflow Pipelines for medallion

Official Documentation

- Medallion Architecture
- Lakeflow Pipelines
- Data Engineering Best Practices

Delta Lake Fundamentals

Delta Lake is an open-source storage layer that brings ACID transactions, scalable metadata handling, and data versioning to data lakes.

Overview

```
flowchart TB
    subgraph DeltaLake["Delta Lake"]
        Transaction[Transaction Log]
        Data[Parquet Data Files]
        Checkpoint[Checkpoints]
    end

    subgraph Features["Key Features"]
        ACID[ACID Transactions]
        TimeTravel[Time Travel]
        Schema[Schema Enforcement]
        Versioning[Data Versioning]
        CDC[Change Data Feed]
    end

    DeltaLake --> Features
```

Overview

- **Transaction log**: JSON commits in `_delta_log/` plus Parquet checkpoints every 10 versions; optimistic concurrency control for writers
- **Time travel** with `VERSION AS OF` / `TIMESTAMP AS OF`; `VACUUM` default retention is 7 days
- **Clones**: shallow clones share the source files, deep clones copy them
- **Table features**: enforced `NOT NULL` / `CHECK` constraints vs informational keys, column mapping, and identity and generated columns

What the Exam Tests

- Design scalable Delta/Iceberg table layouts for large data assets by mapping partitioning to data grain, aligning clustering to relationship access patterns, and maintaining balanced file sizes through compaction.

- Predict the effect of concurrent writes, VACUUM, and clones on a table
- Know which constraints Delta enforces and which are informational

Delta Lake Architecture

Table Structure

```
delta_table/
├── _delta_log/                 # Transaction log directory
│   ├── 00000000000000000000.json
│   ├── 00000000000000000001.json
│   ├── 00000000000000000002.json
│   ├── ...
│   └── 00000000000000000010.checkpoint.parquet
├── part-00000-xxx.snappy.parquet
├── part-00001-xxx.snappy.parquet
└── part-00002-xxx.snappy.parquet
```

Transaction Log

```
flowchart LR
    subgraph TransactionLog["_delta_log/"]
        J0["0.json"]
        J1["1.json"]
        J2["2.json"]
        CP["10.checkpoint.parquet"]
    end

    J0 --> |Add files| State0[Version 0 State]
    J1 --> |Modify| State1[Version 1 State]
    J2 --> |Delete| State2[Version 2 State]
    CP --> |Snapshot| State10[Version 10 State]
```

Component	Purpose
JSON log files	Individual transaction records
Checkpoint files	Aggregated state snapshots (every 10 versions)
Parquet data files	Actual data storage

Creating Delta Tables

SQL Syntax

```
-- Create managed Delta table
CREATE TABLE main.default.customers (
    customer_id INT,
    name STRING,
    email STRING,
    created_at TIMESTAMP
)
USING DELTA;

-- Create table with partitioning
CREATE TABLE main.default.orders (
```

```
    order_id STRING,
    customer_id INT,
    order_date DATE,
    amount DECIMAL(18,2)
)
USING DELTA
PARTITIONED BY (order_date);

-- Create external Delta table
CREATE TABLE main.default.external_data
USING DELTA
LOCATION 's3://bucket/path/to/delta/';

-- Create table with properties
CREATE TABLE main.default.events (
    event_id STRING,
    event_type STRING,
    event_data STRING,
    event_time TIMESTAMP
)
USING DELTA
TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact' = 'true',
    'delta.logRetentionDuration' = 'interval 30 days',
    'delta.deletedFileRetentionDuration' = 'interval 7 days'
);
```

DataFrame API

```
# Create Delta table from DataFrame

df.write.format("delta").saveAsTable("main.default.customers")

# Create with options

(df.write
    .format("delta")
    .mode("overwrite")
    .option("overwriteSchema", "true")
    .partitionBy("order_date")
    .saveAsTable("main.default.orders"))

# Save to path (external table)

(df.write
    .format("delta")
    .mode("overwrite")
    .save("/Volumes/main/default/data/customers"))
```

ACID Transactions

Delta Lake provides full ACID transaction support.

ACID Properties

Property	Description	Delta Implementation
Atomicity	All or nothing	Transaction log ensures complete writes
Consistency	Valid state transitions	Schema enforcement, constraints
Isolation	Concurrent transactions don't interfere	Optimistic concurrency control
Durability	Committed changes are permanent	Cloud storage + transaction log

Transaction Isolation

```
# Serializable isolation level (default)
# Concurrent writes are handled with optimistic concurrency

# Writer 1

df1.write.format("delta").mode("append").save("/path/to/table")

# Writer 2 (concurrent)

df2.write.format("delta").mode("append").save("/path/to/table")

# Both succeed if no conflicts, or one retries
```

Conflict Resolution

```
sequenceDiagram
    participant W1 as Writer 1
    participant Log as Transaction Log
    participant W2 as Writer 2

    W1->>Log: Read version 5
    W2->>Log: Read version 5
    W1->>Log: Write version 6 OK
    W2->>Log: Write version 6 (conflict!)
    W2->>Log: Retry: Read version 6
    W2->>Log: Write version 7 OK
```

Time Travel

Query and restore previous versions of data.

Query by Version

```
-- Query specific version
SELECT * FROM main.default.customers VERSION AS OF 5;

-- Query specific timestamp
SELECT * FROM main.default.customers TIMESTAMP AS OF '2024-01-15 10:00:00';

-- Using @ syntax
SELECT * FROM main.default.customers@v5;
SELECT * FROM main.default.customers@20240115100000;
```

DataFrame Time Travel

```python
# Read specific version

df = (spark.read.format("delta")
    .option("versionAsOf", 5)
    .load("/path/to/table"))

# Read specific timestamp

df = (spark.read.format("delta")
    .option("timestampAsOf", "2024-01-15 10:00:00")
    .load("/path/to/table"))
```

View Table History

```sql
-- View all table history
DESCRIBE HISTORY main.default.customers;

-- View recent history
DESCRIBE HISTORY main.default.customers LIMIT 10;
```

```python
from delta.tables import DeltaTable

delta_table = DeltaTable.forName(spark, "main.default.customers")
history_df = delta_table.history()
history_df.show()
```

History Output Columns

Column	Description
version	Version number
timestamp	When operation occurred
operation	Type of operation (WRITE, MERGE, DELETE, etc.)
operationParameters	Parameters used in operation
operationMetrics	Metrics like rows affected
userIdentity	Who performed the operation

Restore Table

```sql
-- Restore to specific version
RESTORE TABLE main.default.customers TO VERSION AS OF 5;

-- Restore to specific timestamp
RESTORE TABLE main.default.customers TO TIMESTAMP AS OF '2024-01-15 10:00:00';
```

```python
# Restore using DeltaTable

delta_table = DeltaTable.forName(spark, "main.default.customers")
delta_table.restoreToVersion(5)
# or

delta_table.restoreToTimestamp("2024-01-15 10:00:00")
```

Table Cloning

Shallow Clone Creates metadata copy that references original data files.

```sql
-- Shallow clone (metadata only, shares data files)
CREATE TABLE main.default.customers_clone
SHALLOW CLONE main.default.customers;

-- Clone specific version
CREATE TABLE main.default.customers_snapshot
SHALLOW CLONE main.default.customers VERSION AS OF 10;
```

```python
from delta.tables import DeltaTable

# Shallow clone

(DeltaTable.forName(spark, "main.default.customers")
    .clone("/path/to/clone", isShallow=True))
```

Deep Clone Creates full copy of data and metadata.

```sql
-- Deep clone (full copy)
CREATE TABLE main.default.customers_backup
DEEP CLONE main.default.customers;

-- Deep clone to external location
CREATE TABLE main.default.customers_archive
DEEP CLONE main.default.customers
LOCATION 's3://archive-bucket/customers/';
```

```python
# Deep clone

(DeltaTable.forName(spark, "main.default.customers")
    .clone("/path/to/clone", isShallow=False))
```

Clone Comparison

Feature	Shallow Clone	Deep Clone
Data copied	No (references original)	Yes (full copy)
Storage cost	Minimal	Same as original
Independence	Dependent on source files	Fully independent
Use case	Testing, quick snapshots	Backups, migration
Time to create	Fast	Depends on data size
Source changes	May affect clone	No effect

```mermaid
flowchart TB
    subgraph Shallow["Shallow Clone"]
        S1[Clone Metadata] --> SD[Original Data Files]
        S2[Original Metadata] --> SD
    end

    subgraph Deep["Deep Clone"]
        D1[Clone Metadata] --> DD1[Copied Data Files]
        D2[Original Metadata] --> DD2[Original Data Files]
    end
```

Delta Table Properties

Common Properties

```
-- Set table properties
ALTER TABLE main.default.customers SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact' = 'true'
);

-- View table properties
SHOW TBLPROPERTIES main.default.customers;

-- Remove property
ALTER TABLE main.default.customers UNSET TBLPROPERTIES
 ↪  ('delta.autoOptimize.optimizeWrite');
```

Key Properties Reference

Property	Default	Description
delta.autoOptimize.optimizeWrite	false	Coalesce small files on write
delta.autoOptimize.autoCompact	false	Run OPTIMIZE after writes
delta.logRetentionDuration	30 days	How long to keep transaction log
delta.deletedFileRetentionDuration	7 days	How long to keep deleted files
delta.enableChangeDataFeed	false	Enable Change Data Feed
delta.columnMapping.mode	none	Enable column mapping (name/id)
delta.minReaderVersion	1	Minimum reader protocol version
delta.minWriterVersion	2	Minimum writer protocol version

DML Operations

INSERT

```
-- Insert values
INSERT INTO main.default.customers VALUES
    (1, 'John Doe', 'john@example.com', current_timestamp()),
    (2, 'Jane Smith', 'jane@example.com', current_timestamp());

-- Insert from query
INSERT INTO main.default.customers
SELECT * FROM staging.customers WHERE is_new = true;

-- Insert overwrite (replace all data)
INSERT OVERWRITE main.default.customers
SELECT * FROM source_customers;

-- Insert overwrite specific partitions
INSERT OVERWRITE main.default.orders
PARTITION (order_date = '2024-01-15')
SELECT order_id, customer_id, amount
FROM staging.orders
WHERE order_date = '2024-01-15';
```

UPDATE

```sql
-- Update with condition
UPDATE main.default.customers
SET email = 'updated@example.com'
WHERE customer_id = 1;

-- Update with subquery
UPDATE main.default.customers
SET status = 'premium'
WHERE customer_id IN (
    SELECT customer_id FROM main.default.orders
    WHERE total_amount > 10000
);
```

DELETE

```sql
-- Delete with condition
DELETE FROM main.default.customers
WHERE customer_id = 1;

-- Delete based on join
DELETE FROM main.default.customers
WHERE customer_id IN (
    SELECT customer_id FROM main.default.inactive_customers
);
```

MERGE (Upsert)

```sql
-- MERGE for upsert
MERGE INTO main.default.customers AS target
USING staging.customers AS source
ON target.customer_id = source.customer_id
WHEN MATCHED AND source.is_deleted = true THEN DELETE
WHEN MATCHED THEN UPDATE SET
    target.name = source.name,
    target.email = source.email,
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN INSERT (customer_id, name, email, created_at)
VALUES (source.customer_id, source.name, source.email, current_timestamp());
```

```python
from delta.tables import DeltaTable

delta_table = DeltaTable.forName(spark, "main.default.customers")

(delta_table.alias("target")
    .merge(
        source_df.alias("source"),
        "target.customer_id = source.customer_id"
    )
    .whenMatchedDelete(condition="source.is_deleted = true")
    .whenMatchedUpdate(set={
        "name": "source.name",
        "email": "source.email",
        "updated_at": "current_timestamp()"
    })
    .whenNotMatchedInsert(values={
        "customer_id": "source.customer_id",
```

```
        "name": "source.name",
        "email": "source.email",
        "created_at": "current_timestamp()"
    })
    .execute())
```

Table Maintenance

OPTIMIZE Compacts small files into larger ones.

```
-- Optimize entire table
OPTIMIZE main.default.customers;

-- Optimize specific partitions
OPTIMIZE main.default.orders WHERE order_date >= '2024-01-01';

-- Optimize with ZORDER (data skipping)
OPTIMIZE main.default.orders
ZORDER BY (customer_id);

-- Optimize with multiple ZORDER columns
OPTIMIZE main.default.events
ZORDER BY (event_type, user_id);
```

VACUUM Removes old files no longer referenced by table.

```
-- Vacuum with default retention (7 days)
VACUUM main.default.customers;

-- Vacuum with specific retention
VACUUM main.default.customers RETAIN 168 HOURS;  -- 7 days

-- Dry run (shows files that would be deleted)
VACUUM main.default.customers DRY RUN;
```

```
from delta.tables import DeltaTable

delta_table = DeltaTable.forName(spark, "main.default.customers")

# Vacuum with 7 days retention

delta_table.vacuum(168)  # hours

# Vacuum with retention less than default (requires override)

spark.conf.set("spark.databricks.delta.retentionDurationCheck.enabled", "false")
delta_table.vacuum(0)  # Be careful - breaks time travel!
```

Maintenance Best Practices

Operation	Frequency	Notes
OPTIMIZE	Daily/Weekly	Based on write frequency
VACUUM	Weekly	After OPTIMIZE
ZORDER	With OPTIMIZE	On frequently filtered columns
ANALYZE	After major loads	Update statistics

Constraints

NOT NULL Constraint

```sql
-- Add NOT NULL during creation
CREATE TABLE main.default.customers (
    customer_id INT NOT NULL,
    name STRING NOT NULL,
    email STRING
);

-- Alter existing column
ALTER TABLE main.default.customers ALTER COLUMN email SET NOT NULL;
```

CHECK Constraints

```sql
-- Add check constraint
ALTER TABLE main.default.customers
ADD CONSTRAINT valid_email CHECK (email LIKE '%@%');

-- Add multiple constraints
ALTER TABLE main.default.orders
ADD CONSTRAINT positive_amount CHECK (amount > 0);

ALTER TABLE main.default.orders
ADD CONSTRAINT valid_status CHECK (status IN ('pending', 'completed',
 ↪  'cancelled'));

-- Drop constraint
ALTER TABLE main.default.customers DROP CONSTRAINT valid_email;

-- Show constraints
DESCRIBE DETAIL main.default.customers;
```

Primary Key and Foreign Key (Informational)

```sql
-- Primary key (informational, not enforced)
ALTER TABLE main.default.customers
ADD CONSTRAINT pk_customers PRIMARY KEY (customer_id);

-- Foreign key (informational, not enforced)
ALTER TABLE main.default.orders
ADD CONSTRAINT fk_orders_customers
FOREIGN KEY (customer_id) REFERENCES main.default.customers(customer_id);
```

Note: Primary key and foreign key constraints are informational only in Delta Lake. They're used by query optimizers but not enforced on write.

Generated Columns

```sql
-- Create table with generated column
CREATE TABLE main.default.orders (
    order_id STRING,
    order_date DATE,
    order_year INT GENERATED ALWAYS AS (YEAR(order_date)),
    order_month INT GENERATED ALWAYS AS (MONTH(order_date)),
    price DECIMAL(10,2),
```

```
    quantity INT,
    total DECIMAL(12,2) GENERATED ALWAYS AS (price * quantity)
);

-- Generated columns are computed automatically
INSERT INTO main.default.orders (order_id, order_date, price, quantity)
VALUES ('ORD001', '2024-01-15', 99.99, 5);

-- Query includes generated values
SELECT * FROM main.default.orders;
-- order_year = 2024, order_month = 1, total = 499.95
```

Identity Columns

```
-- Auto-incrementing ID column
CREATE TABLE main.default.events (
    event_id BIGINT GENERATED ALWAYS AS IDENTITY,
    event_type STRING,
    event_data STRING
);

-- With custom start and increment
CREATE TABLE main.default.transactions (
    txn_id BIGINT GENERATED ALWAYS AS IDENTITY (START WITH 1000 INCREMENT BY 1),
    amount DECIMAL(18,2)
);

-- GENERATED BY DEFAULT allows manual override
CREATE TABLE main.default.customers (
    customer_id BIGINT GENERATED BY DEFAULT AS IDENTITY,
    name STRING
);

-- Can insert specific ID
INSERT INTO main.default.customers (customer_id, name) VALUES (999, 'Manual ID');
```

Column Mapping

Enable renaming/dropping columns without rewriting data.

```
-- Enable column mapping
ALTER TABLE main.default.customers SET TBLPROPERTIES (
    'delta.columnMapping.mode' = 'name',
    'delta.minReaderVersion' = '2',
    'delta.minWriterVersion' = '5'
);

-- Now can rename columns
ALTER TABLE main.default.customers RENAME COLUMN name TO full_name;

-- Can drop columns
ALTER TABLE main.default.customers DROP COLUMN temp_column;
```

Mode	Description
none	Default, no mapping
name	Map by column name (enables rename/drop)
id	Map by internal ID (most flexible)

Use Cases

Data Quality Enforcement

```sql
-- Table with comprehensive constraints
CREATE TABLE main.default.validated_orders (
    order_id STRING NOT NULL,
    customer_id INT NOT NULL,
    order_date DATE NOT NULL,
    status STRING NOT NULL,
    amount DECIMAL(18,2) NOT NULL,

    -- Constraints
    CONSTRAINT pk_order PRIMARY KEY (order_id),
    CONSTRAINT fk_customer FOREIGN KEY (customer_id) REFERENCES
     ↪  customers(customer_id),
    CONSTRAINT valid_amount CHECK (amount > 0),
    CONSTRAINT valid_status CHECK (status IN ('pending', 'processing', 'shipped',
     ↪  'delivered'))
)
USING DELTA;
```

Audit and Compliance

```python
# Query for audit trail

history = spark.sql("DESCRIBE HISTORY main.default.sensitive_data")

# Find who made changes

changes = history.filter(
    (col("operation") == "UPDATE") &
    (col("timestamp") >= "2024-01-01")
).select("version", "timestamp", "userIdentity", "operationMetrics")
```

Common Issues & Errors

Time Travel Version Not Found **Scenario:** Requested version no longer available.

Fix: Check retention settings and adjust:

```sql
ALTER TABLE main.default.customers SET TBLPROPERTIES (
    'delta.logRetentionDuration' = 'interval 60 days'
);
```

VACUUM Breaks Time Travel **Scenario:** Time travel fails after VACUUM.

Fix: Never vacuum below `deletedFileRetentionDuration`:

```python
# Safe vacuum (respects retention)
```

```
delta_table.vacuum()  # Uses default 168 hours

# Never do this in production:
# spark.conf.set("spark.databricks.delta.retentionDurationCheck.enabled",
 ↪  "false")
# delta_table.vacuum(0)
```

Shallow Clone Source Modified **Scenario:** Shallow clone fails because source files were vacuumed.

Fix: Use deep clone for long-term snapshots, or ensure source retention exceeds clone lifetime.

Constraint Violation **Scenario:** Write fails due to constraint.

```
-- Error: CHECK constraint valid_amount violated
INSERT INTO orders VALUES ('ORD001', 1, -50);
```

Fix: Validate data before write or handle constraint errors.

Exam Tips

1. **ACID** - Delta provides serializable isolation by default
2. **Time travel** - Use VERSION AS OF or TIMESTAMP AS OF syntax
3. **Shallow vs Deep clone** - Shallow shares files, deep copies everything
4. **OPTIMIZE** - Compacts files, ZORDER improves data skipping
5. **VACUUM** - Default 7-day retention, breaks time travel if too aggressive
6. **Constraints** - CHECK enforced, PK/FK informational only
7. **Column mapping** - Required for rename/drop column operations
8. **Generated columns** - Computed automatically, stored physically
9. **Identity columns** - ALWAYS vs BY DEFAULT for auto-increment
10. **Transaction log** - JSON files + checkpoints every 10 versions

Key Takeaways

- **Transaction log structure**: the `_delta_log/` directory stores JSON commit files (one per transaction) plus checkpoint Parquet files every 10 versions; readers replay the log to reconstruct current table state
- **ACID isolation**: Delta Lake uses optimistic concurrency control — concurrent writers both attempt to commit; the second writer detects a conflict, retries by reading the updated version, and then commits
- **Time travel syntax**: `VERSION AS OF <n>` or `TIMESTAMP AS OF '<ts>'` (SQL) and `.option("versionAsOf", n)` / `.option("timestampAsOf", "<ts>")` (Python); `@v<n>` shorthand also works
- **Shallow vs deep clone**: `SHALLOW CLONE` copies metadata only and shares original Parquet files (fast, dependent on source retention); `DEEP CLONE` copies both metadata and data files (independent, safe for backups)
- **VACUUM default retention**: 7 days (168 hours); running VACUUM below the `deletedFileRetentionDuration` breaks time travel for those versions; override requires `retentionDurationCheck.enabled = false`
- **Constraints**: `NOT NULL` and `CHECK` constraints are enforced on every write; `PRIMARY KEY` and `FOREIGN KEY` are informational only (used by the optimizer, not enforced)
- **Column mapping**: required to `RENAME COLUMN` or `DROP COLUMN` without rewriting data; enable with `TBLPROPERTIES ('delta.columnMapping.mode' = 'name')` plus reader/writer version upgrades

- **Identity vs generated columns**: `GENERATED ALWAYS AS IDENTITY` auto-increments and disallows manual inserts; `GENERATED BY DEFAULT AS IDENTITY` allows manual value override; generated expression columns are computed and stored physically

Related Topics

- Schema Management - Schema evolution details
- SCD Patterns - Using MERGE for slowly changing dimensions
- Delta Lake Operations - Advanced operations

Official Documentation

- Delta Lake Documentation
- Databricks Delta Lake Guide
- Delta Lake Table Properties
- Delta Lake Constraints

Partitioning Strategies

Effective partitioning is critical for query performance and data management in Delta Lake. Understanding when to partition, how to choose partition columns, and modern alternatives like liquid clustering is essential.

Overview

```
flowchart TB
    Partitioning[Data Organization] --> Traditional[Traditional Partitioning]
    Partitioning --> ZOrder[Z-ORDER Indexing]
    Partitioning --> Liquid[Liquid Clustering]

    Traditional --> Hive[Hive-style partitions]
    ZOrder --> DataSkipping[Data skipping optimization]
    Liquid --> Automatic[Automatic clustering]
```

Overview

- **Partition pruning** needs a direct filter on the partition column; a function on the column defeats it
- **When to partition**: only very large tables (100 TB or more), on a low-cardinality grain column with at least 1 GB per partition; otherwise use liquid clustering
- **Z-ORDER vs liquid clustering**: clustering keys change with `ALTER TABLE ... CLUSTER BY`, then `OPTIMIZE FULL` reclusters existing data
- **Overwrites and file size**: dynamic partition overwrite, `replaceWhere`, and autotuned target file sizes

What the Exam Tests

- Design scalable Delta/Iceberg table layouts for large data assets by mapping partitioning to data grain, aligning clustering to relationship access patterns, and maintaining balanced file sizes through compaction.
- Compare Liquid Clustering vs partitioning/ZORDER for a given table size and query pattern.
- Choose partition and clustering keys from table size, grain, and query filters

Traditional Partitioning

How Partitioning Works

```
flowchart TB
    Table[Delta Table] --> Part1["partition=2024-01-01/"]
    Table --> Part2["partition=2024-01-02/"]
    Table --> Part3["partition=2024-01-03/"]

    Part1 --> Files1["part-00000.parquet<br>part-00001.parquet"]
    Part2 --> Files2["part-00000.parquet<br>part-00001.parquet"]
    Part3 --> Files3["part-00000.parquet"]
```

Directory Structure

```
delta_table/
├── _delta_log/
├── date=2024-01-01/
│   ├── part-00000-xxx.snappy.parquet
│   └── part-00001-xxx.snappy.parquet
├── date=2024-01-02/
│   ├── part-00000-xxx.snappy.parquet
│   └── part-00001-xxx.snappy.parquet
└── date=2024-01-03/
    └── part-00000-xxx.snappy.parquet
```

Creating Partitioned Tables

```
-- SQL: Create partitioned table
CREATE TABLE main.default.orders (
    order_id STRING,
    customer_id INT,
    order_date DATE,
    amount DECIMAL(18,2)
)
USING DELTA
PARTITIONED BY (order_date);

-- Multiple partition columns
CREATE TABLE main.default.events (
    event_id STRING,
    event_type STRING,
    user_id INT,
    event_data STRING,
    event_date DATE,
    region STRING
)
USING DELTA
PARTITIONED BY (event_date, region);
```

```
# DataFrame API: Write with partitioning

(df.write
    .format("delta")
    .partitionBy("order_date")
    .mode("overwrite")
```

```
    .saveAsTable("main.default.orders"))

# Multiple partition columns

(df.write
    .format("delta")
    .partitionBy("event_date", "region")
    .mode("overwrite")
    .saveAsTable("main.default.events"))
```

Partition Pruning

Partition pruning automatically skips irrelevant partitions during query execution.

How Pruning Works

```
flowchart LR
    Query["SELECT * FROM orders<br>WHERE order_date = '2024-01-15'"] -->
↪   Prune{Partition Pruning}
    Prune --> |Skip| P1["2024-01-01 X"]
    Prune --> |Skip| P2["2024-01-14 X"]
    Prune --> |Read| P3["2024-01-15 OK"]
    Prune --> |Skip| P4["2024-01-16 X"]
```

Effective Pruning Patterns

```
-- Equality filter - PRUNING OCCURS OK
SELECT * FROM orders WHERE order_date = '2024-01-15';

-- Range filter - PRUNING OCCURS OK
SELECT * FROM orders WHERE order_date BETWEEN '2024-01-01' AND '2024-01-31';

-- IN list - PRUNING OCCURS OK
SELECT * FROM orders WHERE order_date IN ('2024-01-15', '2024-01-16');

-- Function on partition column - NO PRUNING X
SELECT * FROM orders WHERE YEAR(order_date) = 2024;

-- Expression on partition column - NO PRUNING X
SELECT * FROM orders WHERE order_date + INTERVAL 1 DAY = '2024-01-16';
```

Pruning Patterns Summary

Filter Pattern	Pruning?	Example
Equality =	Yes	`date = '2024-01-15'`
Comparison <, >, <=, >=	Yes	`date >= '2024-01-01'`
BETWEEN	Yes	`date BETWEEN '2024-01-01' AND '2024-01-31'`
IN list	Yes	`date IN ('2024-01-15', '2024-01-16')`
Function on column	No	`YEAR(date) = 2024`
Expression	No	`date + 1 = '2024-01-16'`
OR with non-partition	Partial	`date = '2024-01-15' OR amount > 100`

Choosing Partition Columns

Selection Criteria

Criterion	Good Partition Column	Bad Partition Column
Cardinality	Low to medium (100-10000)	Very high (millions)
Query patterns	Frequently filtered	Rarely filtered
Data distribution	Even across values	Highly skewed
Granularity	Date, region, category	Timestamp, user_id

Decision Flow

```
flowchart TD
    Start[Choose Partition Column] --> Cardinality{Cardinality?}
    Cardinality --> |< 10000 values| Query{Filtered in queries?}
    Cardinality --> |> 10000 values| NoPartition[Don't partition by this]

    Query --> |Yes, frequently| Distribution{Even distribution?}
    Query --> |No| NoPartition

    Distribution --> |Yes| GoodPartition[Good partition candidate]
    Distribution --> |No, skewed| Consider[Consider but monitor]
```

Good Partition Column Examples

```
-- Date-based partitioning (most common)
PARTITIONED BY (order_date)  -- Daily
PARTITIONED BY (order_month) -- Monthly (derived column)

-- Region/geography
PARTITIONED BY (country_code)
PARTITIONED BY (region)

-- Category/type
PARTITIONED BY (product_category)
PARTITIONED BY (event_type)
```

Bad Partition Column Examples

```
-- Too high cardinality (millions of small files)
PARTITIONED BY (user_id)        -- Avoid
PARTITIONED BY (transaction_id) -- Avoid
PARTITIONED BY (event_timestamp) -- Avoid

-- Rarely filtered
PARTITIONED BY (internal_flag)  -- Avoid if not queried
```

Generated Partition Columns

Create derived columns for partitioning.

```
-- Generate partition column from timestamp
CREATE TABLE main.default.events (
    event_id STRING,
    event_time TIMESTAMP,
```

```
    event_data STRING,
    event_date DATE GENERATED ALWAYS AS (CAST(event_time AS DATE))
)
USING DELTA
PARTITIONED BY (event_date);

-- Generate year/month columns
CREATE TABLE main.default.orders (
    order_id STRING,
    order_timestamp TIMESTAMP,
    amount DECIMAL(18,2),
    order_year INT GENERATED ALWAYS AS (YEAR(order_timestamp)),
    order_month INT GENERATED ALWAYS AS (MONTH(order_timestamp))
)
USING DELTA
PARTITIONED BY (order_year, order_month);
```

```
# Add partition column before writing

from pyspark.sql.functions import year, month, to_date

df_with_partition = (df
    .withColumn("order_date", to_date("order_timestamp"))
    .withColumn("order_year", year("order_timestamp")))

(df_with_partition.write
    .format("delta")
    .partitionBy("order_year", "order_month")
    .saveAsTable("main.default.orders"))
```

Dynamic Partition Overwrite

Overwrite only partitions present in the DataFrame.

```
# Enable dynamic partition overwrite

spark.conf.set("spark.sql.sources.partitionOverwriteMode", "dynamic")

# Only overwrites partitions in the DataFrame

(df.write
    .format("delta")
    .mode("overwrite")
    .partitionBy("order_date")
    .saveAsTable("main.default.orders"))

# Using replaceWhere for explicit partition overwrite

(df.write
    .format("delta")
    .mode("overwrite")
    .option("replaceWhere", "order_date >= '2024-01-01' AND order_date <
 ↪  '2024-02-01'")
    .saveAsTable("main.default.orders"))
```

```
-- SQL equivalent
INSERT OVERWRITE main.default.orders
```

```
PARTITION (order_date)
SELECT * FROM staging_orders;

-- Explicit partition overwrite
INSERT OVERWRITE main.default.orders
PARTITION (order_date = '2024-01-15')
SELECT order_id, customer_id, amount
FROM staging_orders
WHERE order_date = '2024-01-15';
```

Z-ORDER Optimization

Z-ORDER co-locates related data for efficient data skipping.

How Z-ORDER Works

```
flowchart TB
    subgraph Before["Before Z-ORDER"]
        B1["File 1: customer 1-1000, dates mixed"]
        B2["File 2: customer 1-1000, dates mixed"]
        B3["File 3: customer 1-1000, dates mixed"]
    end

    subgraph After["After Z-ORDER BY (customer_id)"]
        A1["File 1: customer 1-333"]
        A2["File 2: customer 334-666"]
        A3["File 3: customer 667-1000"]
    end

    Before --> |OPTIMIZE ZORDER| After
```

Z-ORDER Syntax

```
-- Z-ORDER by single column
OPTIMIZE main.default.orders
ZORDER BY (customer_id);

-- Z-ORDER by multiple columns
OPTIMIZE main.default.events
ZORDER BY (user_id, event_type);

-- Z-ORDER specific partitions
OPTIMIZE main.default.orders
WHERE order_date >= '2024-01-01'
ZORDER BY (customer_id);
```

Z-ORDER Best Practices

Guideline	Recommendation
Number of columns	1-4 columns (effectiveness decreases)
Column selection	High cardinality columns used in filters
Order matters	Put most selective column first
With partitioning	Use Z-ORDER on non-partition filter columns
Frequency	Run after significant data changes

Z-ORDER Column Selection

```
-- Good: High cardinality, frequently filtered
OPTIMIZE orders ZORDER BY (customer_id);  -- Often filtered by customer
OPTIMIZE events ZORDER BY (user_id, session_id);  -- User analysis queries

-- Less effective: Low cardinality
OPTIMIZE orders ZORDER BY (status);  -- Only a few distinct values

-- Combine with partitioning
-- Table partitioned by order_date
-- Z-ORDER by customer_id for customer-based queries
OPTIMIZE orders WHERE order_date = '2024-01-15' ZORDER BY (customer_id);
```

Liquid Clustering

Liquid clustering is the modern replacement for partitioning + Z-ORDER.

Liquid Clustering Benefits

Feature	Partitioning + Z-ORDER	Liquid Clustering
Initial setup	Choose at table creation	Choose at table creation
Change columns	Requires table rebuild	Just ALTER TABLE
Incremental clustering	Manual OPTIMIZE	Automatic
Small file problem	Can occur	Handled automatically
Write performance	Manual optimization	Optimized writes

Enable Liquid Clustering

```
-- Create new table with liquid clustering
CREATE TABLE main.default.orders (
    order_id STRING,
    customer_id INT,
    order_date DATE,
    amount DECIMAL(18,2)
)
USING DELTA
CLUSTER BY (customer_id, order_date);

-- Convert existing table to liquid clustering
ALTER TABLE main.default.orders
CLUSTER BY (customer_id, order_date);

-- Remove clustering
ALTER TABLE main.default.orders
CLUSTER BY NONE;

-- Change clustering columns (easy with liquid clustering!)
ALTER TABLE main.default.orders
CLUSTER BY (region, order_date);
```

Liquid Clustering with Auto-Compaction

```sql
-- Enable auto-optimization for liquid clustering
ALTER TABLE main.default.orders SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact' = 'true'
);
```

Triggering Clustering

```sql
-- Incremental clustering of data that still needs it
OPTIMIZE main.default.orders;

-- Recluster all existing data, e.g. after changing keys (DBR 16.4 LTS+)
OPTIMIZE main.default.orders FULL;

-- Clustering also happens:
-- 1. On write (INSERT INTO, CTAS/RTAS, COPY INTO from Parquet, append) once the
--    transaction exceeds a size threshold (64 MB for 1 key on UC managed tables)
-- 2. Through predictive optimization, which runs OPTIMIZE on UC managed tables
```

When to Use Liquid Clustering

Scenario	Recommendation
New tables	Use liquid clustering
Uncertain query patterns	Use liquid clustering (easy to change)
Tables of 100 TB or more	Liquid clustering first. Partition only if it measurably helps
Need exact partition control	Use traditional partitioning
Streaming workloads	Liquid clustering works well

Partitioning vs Z-ORDER vs Liquid Clustering

Comparison Matrix

Aspect	Partitioning	Z-ORDER	Liquid Clustering
Data organization	Physical directories	Within files	Within files
Filter efficiency	Partition pruning	Data skipping	Data skipping
Cardinality support	Low-medium	High	High
Column changes	Table rebuild	Re-OPTIMIZE	ALTER TABLE
Maintenance	Manual	Manual OPTIMIZE	OPTIMIZE (run by predictive optimization or
Streaming support	Good	Manual maintenance	Good

Decision Guide

```
flowchart TD
    Start[Choose Strategy] --> New{New table?}

    New --> |Yes| Liquid{High cardinality<br>filter columns?}
    Liquid --> |Yes| UseLiquid[Use Liquid Clustering]
    Liquid --> |No| LowCard{Low cardinality<br>frequently filtered?}
    LowCard --> |Yes| UsePartition[Use Partitioning]
    LowCard --> |No| UseLiquid

    New --> |No, existing| Migrate{Can migrate?}
```

```
    Migrate --> |Yes| UseLiquid
    Migrate --> |No| UseZOrder[Use Z-ORDER on<br>existing partitions]
```

Small File Problem

Problem Description

```
flowchart LR
    Streaming[Streaming Writes] --> Many["1000s of small files<br>(1-10 MB each)"]
    Many --> Slow[Slow queries]
    Many --> Metadata[Metadata overhead]
```

Solutions

```
-- Solution 1: Auto-optimize (recommended)
ALTER TABLE main.default.orders SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact' = 'true'
);

-- Solution 2: Periodic OPTIMIZE
OPTIMIZE main.default.orders;

-- Solution 3: Repartition before write
-- (manual control over file count)
```

```
# Control file size with repartition
# (Docs: don't repartition/coalesce just before a write when optimized writes is
 ↪  on)

(df.repartition(10)
    .write
    .format("delta")
    .mode("append")
    .saveAsTable("main.default.orders"))
```

```
-- Pin a fixed target file size (otherwise Databricks autotunes by table size)
ALTER TABLE main.default.orders SET TBLPROPERTIES ('delta.targetFileSize' =
 ↪  '256mb');
```

Target File Sizes Databricks autotunes target file size from table size. See File size targets below. On Unity Catalog managed tables, only `OPTIMIZE` respects a manual `delta.targetFileSize`.

Partition Evolution

Changing Partition Scheme Traditional partitioning cannot be changed without recreating the table:

```
# Cannot change partition columns directly
# Must recreate table

# Step 1: Create new table with desired partitioning

spark.sql("""
    CREATE TABLE main.default.orders_new
    USING DELTA
```

```
    PARTITIONED BY (order_month)
    AS SELECT
        *,
        DATE_TRUNC('month', order_date) AS order_month
    FROM main.default.orders
""")

# Step 2: Rename tables

spark.sql("ALTER TABLE main.default.orders RENAME TO main.default.orders_old")
spark.sql("ALTER TABLE main.default.orders_new RENAME TO main.default.orders")

# Step 3: Drop old table (after validation)
# spark.sql("DROP TABLE main.default.orders_old")
```

Liquid Clustering Evolution

```
-- Easy with liquid clustering!
ALTER TABLE main.default.orders
CLUSTER BY (region, order_month);

-- New keys apply to subsequent writes and OPTIMIZE runs.
-- Existing data is NOT rewritten until you run:
OPTIMIZE main.default.orders FULL;
```

> **Note**
>
> In Databricks Runtime 18.1+, `ALTER TABLE ... REPLACE PARTITIONED BY WITH CLUSTER BY` converts a partitioned Delta table to liquid clustering in place, without the CTAS rebuild shown above.

Designing Layouts for Large Delta and Iceberg Tables

> **What the Exam Tests**
>
> "Design scalable Delta/Iceberg table layouts for large data assets by mapping partitioning to data grain, aligning clustering to relationship access patterns, and maintaining balanced file sizes through compaction."

Exam phrase	Documented guidance
Map partitioning to data grain	Partition only large tables, only on low or known cardinality c
Align clustering to relationship access patterns	Pick clustering keys from the columns most used in filters **an**
Balanced file sizes through compaction	`OPTIMIZE` compacts and incrementally clusters. Run `OPTIMIZ`

Size thresholds for partitioning

Table size	Guidance
Less than 1 TB	Don't partition
1 TB to 100 TB	Use liquid clustering instead. Partitioning likely hurts more often than it helps
100 TB or more	Partitioning might help, but use liquid clustering first and verify the gain
Any partitioned table	Each partition should contain at least 1 GB. Fewer, larger partitions outperform many s

```sql
-- Large fact table: no partitions; cluster on the time grain and the main join key
CREATE TABLE main.gold.fact_events (
  event_date  DATE,
  customer_id BIGINT,
  event_type  STRING,
  amount      DECIMAL(18,2)
)
CLUSTER BY (event_date, customer_id)
TBLPROPERTIES ('delta.liquid.hierarchicalClusteringColumns' = 'event_date');

-- Convert an over-partitioned table in place (DBR 18.1+); partition columns
-- become hierarchical clustering keys. Then cluster the data.
ALTER TABLE main.gold.fact_events_legacy
  REPLACE PARTITIONED BY WITH CLUSTER BY (event_date, customer_id);
OPTIMIZE main.gold.fact_events_legacy;
```

When migrating keys, use partition columns and `ZORDER BY` columns as clustering keys. For a generated low-cardinality column (such as a date derived from a timestamp), cluster on the original column instead. Streaming tables and materialized views in Lakeflow pipelines can't use the conversion command. Change their definitions to `CLUSTER BY`.

Managed Iceberg tables

- Unity Catalog managed Iceberg tables support **only liquid clustering**. `PARTITIONED BY` columns are interpreted as clustering keys, so the conversion command isn't needed (running it raises an error).
- Liquid clustering on Iceberg is Public Preview (DBR 16.4 LTS+). Automatic liquid clustering needs Iceberg v3 and DBR 18.0+.
- Partition evolution (`ADD` / `DROP` / `REPLACE PARTITION FIELD`) works only from external Iceberg engines through the Iceberg REST Catalog, not from Databricks SQL. Expression transforms such as `years()` or `bucket()` aren't supported. With `bucket(c1, n)`, Unity Catalog drops the transform and clusters on `c1`.
- Managed Iceberg tables require predictive optimization.

File size targets Unity Catalog managed tables tune file size automatically. The autotuned target is 256 MB for tables under 2.56 TB, grows linearly to 1 GB between 2.56 TB and 10 TB, and stays at 1 GB above 10 TB. Larger targets don't rewrite existing smaller files. To force a size, set `delta.targetFileSize` (Delta) or `iceberg.targetFileSize` (Iceberg). For tables over 1 TB without predictive optimization, run `OPTIMIZE` on a schedule. Auto compaction and optimized writes reduce small files but don't replace `OPTIMIZE`.

Use Cases

Time-Series Data

```sql
-- Partition by date, Z-ORDER by device
CREATE TABLE main.default.sensor_data (
    device_id STRING,
    reading_time TIMESTAMP,
    value DOUBLE,
    reading_date DATE GENERATED ALWAYS AS (CAST(reading_time AS DATE))
)
USING DELTA
PARTITIONED BY (reading_date);
```

```sql
-- Queries filter by date range and device
OPTIMIZE sensor_data ZORDER BY (device_id);
```

E-commerce Orders

```sql
-- Liquid clustering for flexible queries
CREATE TABLE main.default.orders (
    order_id STRING,
    customer_id INT,
    order_date DATE,
    region STRING,
    amount DECIMAL(18,2)
)
USING DELTA
CLUSTER BY (order_date, customer_id, region);

-- Supports queries by date, customer, or region
```

Log Analytics

```sql
-- High volume, filter by time and service
CREATE TABLE main.default.application_logs (
    log_id STRING,
    service_name STRING,
    log_level STRING,
    message STRING,
    log_time TIMESTAMP,
    log_date DATE GENERATED ALWAYS AS (CAST(log_time AS DATE))
)
USING DELTA
PARTITIONED BY (log_date)
TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true'
);

OPTIMIZE application_logs ZORDER BY (service_name, log_level);
```

Common Issues & Errors

Too Many Small Partitions **Scenario:** Partitioning by high-cardinality column creates millions of small files.

Fix: Choose lower cardinality partition column or use liquid clustering:

```sql
-- Instead of partitioning by timestamp
-- Partition by date or month
PARTITIONED BY (event_date)

-- Or use liquid clustering
CLUSTER BY (event_timestamp)
```

Partition Pruning Not Working **Scenario:** Query scans all partitions despite filter.

Fix: Check filter pushdown:

```sql
-- Doesn't prune
SELECT * FROM orders WHERE YEAR(order_date) = 2024;
```

```sql
-- Does prune
SELECT * FROM orders WHERE order_date >= '2024-01-01' AND order_date <
 ↪  '2025-01-01';
```

Skewed Partitions **Scenario:** One partition has much more data than others.

Fix: Repartition or handle hot partitions:

```python
# Adaptive write for skewed data

spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

Z-ORDER Not Effective **Scenario:** Z-ORDER doesn't improve query performance.

Fix: Verify column selection and run OPTIMIZE:

```sql
-- Check if OPTIMIZE has been run
DESCRIBE HISTORY main.default.orders;

-- Ensure Z-ORDER columns match query filters
OPTIMIZE orders ZORDER BY (customer_id);  -- If filtering by customer
```

Exam Tips

1. **Partition pruning** - Only works with direct column filters, not functions
2. **Z-ORDER** - Use for high cardinality columns, 1-4 columns max
3. **Liquid clustering** - Modern replacement, easy to change columns
4. **Small file problem** - Use auto-optimize or periodic OPTIMIZE
5. **Dynamic partition overwrite** - Only overwrites partitions in DataFrame
6. **Generated columns** - Use for derived partition columns
7. **Cardinality** - Low-medium for partitioning, high for Z-ORDER/clustering
8. **replaceWhere** - Explicit partition overwrite option
9. **File size target** - Autotuned: 256 MB below 2.56 TB, growing to 1 GB above 10 TB. Pin with `delta.targetFileSize`
10. **Partition evolution** - Traditional requires a rebuild (or `REPLACE PARTITIONED BY WITH CLUSTER BY` on DBR 18.1+). Liquid uses `ALTER TABLE ... CLUSTER BY` and then `OPTIMIZE FULL`
11. **When to partition** - Under 1 TB never. 1–100 TB use liquid clustering. At least 1 GB per partition
12. **Clustering for joins** - Cluster fact tables on frequently joined or filtered keys so dynamic file pruning skips files. Managed Iceberg tables support only liquid clustering

Key Takeaways

- **Partition pruning conditions**: equality (=), range (<, >, `BETWEEN`), and `IN` filters on the partition column enable pruning; applying a function to the column (e.g., `YEAR(date)`) defeats pruning entirely
- **Partition column cardinality**: low to medium cardinality (hundreds to ~10,000 distinct values) is ideal; high-cardinality columns (user_id, transaction_id, timestamp) create millions of small files and hurt performance
- **Z-ORDER mechanics**: `OPTIMIZE ... ZORDER BY (col)` co-locates rows with similar values in the same files; this enables Delta data skipping using per-file min/max statistics; effectiveness diminishes beyond 4 columns
- **Liquid clustering vs partitioning**: Liquid Clustering (`CLUSTER BY`) replaces partitioning + Z-ORDER; clustering columns can be changed with a simple `ALTER TABLE ... CLUSTER BY` without rebuilding the table

- **Dynamic partition overwrite**: `spark.sql.sources.partitionOverwriteMode = dynamic` causes `mode("overwrite")` to replace only the partitions present in the DataFrame, leaving other partitions untouched
- **replaceWhere**: `.option("replaceWhere", "<condition>")` on a write restricts the overwrite to matching partitions; a more explicit alternative to dynamic partition overwrite
- **Small file problem**: streaming and frequent small writes create many small files; solutions are `optimizeWrite=true` (coalesce on write), `autoCompact=true` (compact after write), periodic `OPTIMIZE`, or switching to Liquid Clustering
- **Target file size**: Databricks autotunes by table size, from 256 MB (under 2.56 TB) to 1 GB (over 10 TB). Unity Catalog managed tables tune automatically, and predictive optimization runs `OPTIMIZE`, `VACUUM`, and `ANALYZE` for them
- **Layout for large assets**: partition only tables of 100 TB or more on a low-cardinality grain column with at least 1 GB per partition. Otherwise use liquid clustering keyed on filter and join columns (up to 4), and compact with `OPTIMIZE`

Related Topics

- Delta Lake Fundamentals - OPTIMIZE and VACUUM
- Performance Optimization - Query tuning
- Z-Order Indexing - Deep dive
- File Sizing - compaction and file size tuning
- Dimensional Modeling and Metric Views - star-schema keys and joins

Official Documentation

- When to partition tables on Databricks
- Use liquid clustering for tables
- Control data file size
- Predictive optimization for Unity Catalog managed tables
- Dynamic file pruning
- What is Apache Iceberg in Databricks? (partition evolution)
- Data skipping (Z-order)

Schema Management

Schema management in Delta Lake encompasses schema enforcement, schema evolution, and schema changes. Understanding these concepts is critical for building robust data pipelines.

Overview

```
flowchart TB
    Schema[Schema Management] --> Enforcement[Schema Enforcement]
    Schema --> Evolution[Schema Evolution]
    Schema --> Changes[Schema Changes]

    Enforcement --> Reject[Reject incompatible writes]
    Evolution --> Add[Add new columns]
    Changes --> Alter[Alter existing schema]
```

Overview

- **Schema enforcement** rejects extra columns and wrong types; missing nullable columns are filled with NULL

- **Schema evolution**: `mergeSchema` adds columns and widens compatible types; `overwriteSchema` replaces the schema
- **Column mapping** (`delta.columnMapping.mode = 'name'`) is required for `RENAME COLUMN` and `DROP COLUMN`
- **Streaming and drift**: Auto Loader evolution modes with `cloudFiles.schemaLocation`, and VARIANT for unknown schemas

What the Exam Tests

- Develop data ingestion pipelines to ingest a variety of data formats, including Delta Lake, Parquet, JSON, Iceberg, CSV, and Binary from diverse sources such as message buses (Kafka, Kinesis, Pub/Sub) and cloud storage.
- Predict which writes schema enforcement rejects
- Choose the evolution option for a given schema change

Schema Enforcement

Schema enforcement prevents writes that don't match the table schema.

How Schema Enforcement Works

```
flowchart LR
    Data[Incoming Data] --> Check{Schema Match?}
    Check --> |Yes| Write[Write to Table]
    Check --> |No| Reject[Reject Write]
```

Enforcement Rules

Scenario	Result
Extra columns in data	Rejected (unless evolution enabled)
Missing columns in data	Filled with NULL (if nullable)
Wrong data types	Rejected
Correct schema	Accepted

Schema Enforcement Example

```
# Create table with specific schema

spark.sql("""
    CREATE TABLE main.default.customers (
        customer_id INT,
        name STRING,
        email STRING
    ) USING DELTA
""")

# This succeeds - exact match

df_valid = spark.createDataFrame([
    (1, "John", "john@example.com")
], ["customer_id", "name", "email"])
```

```
df_valid.write.format("delta").mode("append").saveAsTable(
    "main.default.customers")

# This fails - extra column

df_extra = spark.createDataFrame([
    (2, "Jane", "jane@example.com", "555-1234")
], ["customer_id", "name", "email", "phone"])

# Error: A schema mismatch detected when writing to the Delta table

df_extra.write.format("delta").mode("append").saveAsTable(
    "main.default.customers")

# This succeeds - missing nullable column fills with NULL

df_missing = spark.createDataFrame([
    (3, "Bob")
], ["customer_id", "name"])

df_missing.write.format("delta").mode("append").saveAsTable(
    "main.default.customers")
# email column will be NULL
```

Schema Evolution

Schema evolution allows automatic schema changes when writing data.

Enable Schema Evolution

```
# Option 1: Per-write option

(df.write
    .format("delta")
    .mode("append")
    .option("mergeSchema", "true")
    .saveAsTable("main.default.customers"))

# Option 2: Spark configuration (session-wide)

spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")
df.write.format("delta").mode("append").saveAsTable("main.default.customers")
```

```
-- SQL equivalent
SET spark.databricks.delta.schema.autoMerge.enabled = true;

INSERT INTO main.default.customers
SELECT * FROM new_data_with_extra_columns;
```

Schema Evolution Operations

Operation	mergeSchema	overwriteSchema
Add new column	✓	✓
Widen data type (e.g., INT→LONG)	✓	✓

Operation	mergeSchema	overwriteSchema
Change column type	✗	✓
Remove column	✗	✓
Rename column	✗	✓ (loses data)
Reorder columns	✓	✓

mergeSchema Examples

```
# Original schema: customer_id INT, name STRING, email STRING

# Add new column - succeeds with mergeSchema

df_new_column = spark.createDataFrame([
    (4, "Alice", "alice@example.com", "555-9876")
], ["customer_id", "name", "email", "phone"])

(df_new_column.write
    .format("delta")
    .mode("append")
    .option("mergeSchema", "true")
    .saveAsTable("main.default.customers"))

# Table now has: customer_id, name, email, phone
# Existing rows have NULL for phone
```

overwriteSchema Use overwriteSchema for complete schema replacement:

```
# Completely replace schema (dangerous - loses data!)

df_new_schema = spark.createDataFrame([
    ("C001", "John Doe", "Premium")
], ["customer_code", "full_name", "tier"])

(df_new_schema.write
    .format("delta")
    .mode("overwrite")
    .option("overwriteSchema", "true")
    .saveAsTable("main.default.customers"))

# Table now has completely different schema
# All previous data is replaced
```

Auto Loader Schema Evolution

```
# Auto Loader with schema evolution

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/checkpoints/_schema")
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load("/landing/data/"))
```

```python
# Write with mergeSchema

(df.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoints/_checkpoint")
    .option("mergeSchema", "true")
    .toTable("bronze.events"))
```

Schema Evolution Modes for Auto Loader

Mode	Behavior
`addNewColumns`	Add new columns, fail on other changes
`failOnNewColumns`	Fail on any schema change
`rescue`	Add new columns, put unexpected data in `_rescued_data`
`none`	Use fixed schema, ignore changes

Column Mapping

Column mapping enables advanced schema changes without rewriting data.

Enable Column Mapping

```sql
-- Enable column mapping
ALTER TABLE main.default.customers SET TBLPROPERTIES (
    'delta.columnMapping.mode' = 'name',
    'delta.minReaderVersion' = '2',
    'delta.minWriterVersion' = '5'
);
```

Column Mapping Modes

Mode	Description	Use Case
`none`	Default, no mapping	Simple tables
`name`	Map by column name	Enable rename/drop
`id`	Map by internal ID	Maximum flexibility

Operations Enabled by Column Mapping

```sql
-- With column mapping enabled:

-- Rename column (preserves data)
ALTER TABLE main.default.customers
RENAME COLUMN name TO full_name;

-- Drop column (marks as deleted, doesn't rewrite)
ALTER TABLE main.default.customers
DROP COLUMN temp_column;

-- Change column type (some changes)
ALTER TABLE main.default.customers
ALTER COLUMN customer_id TYPE BIGINT;
```

Without Column Mapping

```
-- Without column mapping, these operations fail:
-- Error: Cannot rename column
ALTER TABLE main.default.customers RENAME COLUMN name TO full_name;

-- Workaround: Create new table with desired schema
CREATE TABLE main.default.customers_v2 AS
SELECT customer_id, name AS full_name, email FROM main.default.customers;
```

ALTER TABLE Operations

Add Columns

```
-- Add single column
ALTER TABLE main.default.customers
ADD COLUMN phone STRING;

-- Add multiple columns
ALTER TABLE main.default.customers
ADD COLUMNS (
    phone STRING,
    address STRING,
    city STRING
);

-- Add column with comment
ALTER TABLE main.default.customers
ADD COLUMN phone STRING COMMENT 'Customer phone number';

-- Add column with position
ALTER TABLE main.default.customers
ADD COLUMN phone STRING AFTER email;

ALTER TABLE main.default.customers
ADD COLUMN priority INT FIRST;
```

Change Column

```
-- Change data type (with column mapping or compatible types)
ALTER TABLE main.default.customers
ALTER COLUMN customer_id TYPE BIGINT;

-- Change nullability
ALTER TABLE main.default.customers
ALTER COLUMN email SET NOT NULL;

ALTER TABLE main.default.customers
ALTER COLUMN email DROP NOT NULL;

-- Add/change column comment
ALTER TABLE main.default.customers
ALTER COLUMN email COMMENT 'Primary email address';
```

Rename Column

```sql
-- Requires column mapping enabled
ALTER TABLE main.default.customers
RENAME COLUMN name TO full_name;
```

Drop Column

```sql
-- Requires column mapping enabled
ALTER TABLE main.default.customers
DROP COLUMN temp_column;

-- Drop multiple columns
ALTER TABLE main.default.customers
DROP COLUMNS (temp1, temp2);
```

Type Widening Delta Lake allows widening numeric types:

From	To (Allowed)
BYTE	SHORT, INT, LONG, DECIMAL
SHORT	INT, LONG, DECIMAL
INT	LONG, DECIMAL
FLOAT	DOUBLE

```sql
-- Valid type widening
ALTER TABLE main.default.orders
ALTER COLUMN quantity TYPE BIGINT;  -- INT to BIGINT

-- Invalid type narrowing (fails)
ALTER TABLE main.default.orders
ALTER COLUMN quantity TYPE TINYINT;  -- BIGINT to TINYINT fails
```

Schema Validation Patterns

Pre-Write Validation

```python
from pyspark.sql.types import StructType, StructField, IntegerType, StringType

# Define expected schema

expected_schema = StructType([
    StructField("customer_id", IntegerType(), nullable=False),
    StructField("name", StringType(), nullable=False),
    StructField("email", StringType(), nullable=True)
])

# Validate incoming data

def validate_schema(df, expected):
    """Validate DataFrame schema matches expected schema."""
    actual_fields = {f.name: f for f in df.schema.fields}
    expected_fields = {f.name: f for f in expected.fields}

    errors = []

    # Check for missing required columns
```

```
    for name, field in expected_fields.items():
        if name not in actual_fields:
            if not field.nullable:
                errors.append(f"Missing required column: {name}")

    # Check data types
    for name, field in expected_fields.items():
        if name in actual_fields:
            if actual_fields[name].dataType != field.dataType:
                errors.append(f"Type mismatch for {name}: expected
↪   {field.dataType}, got {actual_fields[name].dataType}")

    return errors

# Usage

errors = validate_schema(incoming_df, expected_schema)
if errors:
    raise ValueError(f"Schema validation failed: {errors}")
```

Schema Comparison

```
def compare_schemas(source_df, target_table):
    """Compare source DataFrame schema with target table."""
    target_df = spark.table(target_table)

    source_cols = set(source_df.columns)
    target_cols = set(target_df.columns)

    added = source_cols - target_cols
    removed = target_cols - source_cols
    common = source_cols & target_cols

    type_changes = []
    for col in common:
        source_type = source_df.schema[col].dataType
        target_type = target_df.schema[col].dataType
        if source_type != target_type:
            type_changes.append((col, target_type, source_type))

    return {
        "added_columns": added,
        "removed_columns": removed,
        "type_changes": type_changes
    }
```

Schema in Streaming

Streaming Schema Enforcement

```
# Streaming with schema enforcement

df = (spark.readStream
    .schema(expected_schema)  # Enforce schema on read
    .format("json")
    .load("/landing/data/"))
```

```
# Write without mergeSchema - strict enforcement

(df.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoints/")
    .toTable("bronze.events"))
```

Handling Schema Changes in Streaming

```
# Option 1: Use rescue column for unexpected data

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/checkpoints/_schema")
    .option("cloudFiles.schemaEvolutionMode", "rescue")
    .load("/landing/data/"))

# Unexpected fields go to _rescued_data column

(df.writeStream
    .format("delta")
    .option("checkpointLocation", "/checkpoints/_cp")
    .option("mergeSchema", "true")
    .toTable("bronze.events"))

# Option 2: Fail on schema changes and alert

df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/checkpoints/_schema")
    .option("cloudFiles.schemaEvolutionMode", "failOnNewColumns")
    .load("/landing/data/"))
```

Schema Drift Handling

Pattern: Schema Drift Detection

```
def detect_schema_drift(current_df, reference_table):
    """Detect if incoming data has schema drift from reference table."""
    reference_df = spark.table(reference_table)

    current_cols = set(current_df.columns)
    reference_cols = set(reference_df.columns)

    new_columns = current_cols - reference_cols
    missing_columns = reference_cols - current_cols

    if new_columns:
        print(f"New columns detected: {new_columns}")
        # Log, alert, or handle as needed

    if missing_columns:
        print(f"Missing columns: {missing_columns}")
```

```
        # Log, alert, or handle as needed

    return len(new_columns) > 0 or len(missing_columns) > 0

# Usage

if detect_schema_drift(incoming_df, "silver.customers"):
    # Handle drift - merge schema, reject, or alert
    pass
```

Pattern: Flexible Schema with Variant Type

```
-- Store semi-structured data with flexible schema
CREATE TABLE bronze.events_raw (
    event_id STRING,
    event_time TIMESTAMP,
    event_data VARIANT  -- Semi-structured data type
);

-- Query VARIANT column
SELECT
    event_id,
    event_data:user_id::INT AS user_id,
    event_data:action::STRING AS action,
    event_data:metadata.browser::STRING AS browser
FROM bronze.events_raw;
```

Lakeflow Pipelines Schema Management

Schema Hints in Lakeflow Pipelines

```
import dlt
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

# Explicit schema

@dlt.table(
    comment="Customers with enforced schema",
    schema=StructType([
        StructField("customer_id", IntegerType(), nullable=False),
        StructField("name", StringType(), nullable=False),
        StructField("email", StringType(), nullable=True)
    ])
)
def silver_customers():
    return dlt.read_stream("bronze_customers")

# Or use schema hints

@dlt.table(
    comment="Customers with inferred schema"
)
def silver_customers():
    return (
        dlt.read_stream("bronze_customers")
        .withColumn("customer_id", col("customer_id").cast("int"))
```

```
        .withColumn("name", col("name").cast("string"))
    )
```

Use Cases

Handling Upstream Schema Changes

```
def handle_upstream_changes(source_path, target_table):
    """Handle upstream schema changes gracefully."""

    # Read new data
    new_df = spark.read.format("parquet").load(source_path)

    # Get target schema
    target_df = spark.table(target_table)
    target_cols = set(target_df.columns)

    # Find new columns
    new_cols = set(new_df.columns) - target_cols

    if new_cols:
        # Log schema change
        print(f"Schema change detected: new columns {new_cols}")

        # Option 1: Add columns to target table
        for col_name in new_cols:
            col_type = new_df.schema[col_name].dataType.simpleString()
            spark.sql(f"ALTER TABLE {target_table} ADD COLUMN {col_name}
↪   {col_type}")

    # Write with mergeSchema for safety
    (new_df.write
        .format("delta")
        .mode("append")
        .option("mergeSchema", "true")
        .saveAsTable(target_table))
```

Migration to New Schema

```
def migrate_table_schema(old_table, new_table, transformations):
    """Migrate data from old schema to new schema."""

    # Read old table
    df = spark.table(old_table)

    # Apply transformations
    for old_col, new_col, transform in transformations:
        if transform:
            df = df.withColumn(new_col, transform(col(old_col)))
        else:
            df = df.withColumnRenamed(old_col, new_col)

    # Write to new table
    df.write.format("delta").saveAsTable(new_table)

# Usage
```

```python
transformations = [
    ("name", "full_name", None),  # Simple rename
    ("customer_id", "customer_id", lambda c: c.cast("bigint")),  # Type change
    ("created_at", "created_date", lambda c: to_date(c))  # Transform
]

migrate_table_schema("old.customers", "new.customers", transformations)
```

Common Issues & Errors

Schema Mismatch on Write **Scenario:** Write fails due to schema mismatch.

```
# Error: A schema mismatch detected when writing to the Delta table
```

Fix: Use `mergeSchema` or adjust data to match:

```
df.write.option("mergeSchema", "true").saveAsTable("table")
```

Cannot Change Column Type **Scenario:** ALTER COLUMN TYPE fails.

```
-- Error: Cannot change data type from STRING to INT
```

Fix: Enable column mapping or create new table:

```
ALTER TABLE table SET TBLPROPERTIES ('delta.columnMapping.mode' = 'name');
```

Cannot Rename/Drop Column **Scenario:** Rename or drop operation fails.

Fix: Enable column mapping first:

```
ALTER TABLE main.default.customers SET TBLPROPERTIES (
    'delta.columnMapping.mode' = 'name',
    'delta.minReaderVersion' = '2',
    'delta.minWriterVersion' = '5'
);
```

Streaming Schema Change **Scenario:** Streaming job fails on schema change.

Fix: Use `rescue` mode or restart with new schema:

```
.option("cloudFiles.schemaEvolutionMode", "rescue")
```

Exam Tips

1. **mergeSchema vs overwriteSchema** - merge adds columns, overwrite replaces schema
2. **Schema enforcement** - Extra columns rejected, missing nullable columns become NULL
3. **Column mapping** - Required for rename/drop column operations
4. **Type widening** - INT→LONG allowed, LONG→INT not allowed
5. **Auto Loader modes** - `addNewColumns`, `failOnNewColumns`, `rescue`, `none`
6. **ALTER TABLE ADD** - Can specify position with FIRST or AFTER
7. **NOT NULL** - Can be set/dropped with ALTER COLUMN
8. **Streaming schema** - Use checkpointLocation for schema tracking
9. **VARIANT type** - For flexible semi-structured data
10. **Schema validation** - Compare schemas before write for safety

Key Takeaways

- **Schema enforcement rules**: extra columns in the incoming DataFrame are rejected; missing nullable columns are silently filled with NULL; wrong data types are rejected — schema enforcement is always on by default in Delta Lake
- **mergeSchema vs overwriteSchema**: `mergeSchema=true` adds new columns and widens compatible types but cannot remove or rename columns; `overwriteSchema=true` completely replaces the schema (and data) on an overwrite write
- **Column mapping requirement**: `RENAME COLUMN` and `DROP COLUMN` require `delta.columnMapping.mode = 'name'` (plus `minReaderVersion=2`, `minWriterVersion=5`); without it, those DDL statements fail
- **Type widening rules**: only safe numeric widenings are allowed (e.g., INT → LONG, FLOAT → DOUBLE); narrowing (LONG → INT) is always rejected
- **Auto Loader schema evolution modes**: `addNewColumns` silently adds new columns; `failOnNewColumns` stops the stream on any change; `rescue` routes unexpected fields to `_rescued_data`; `none` ignores source schema changes entirely
- **ALTER TABLE ADD column positioning**: `ADD COLUMN <name> <type> FIRST` or `ADD COLUMN <name> <type> AFTER <other_col>` controls insertion order
- **Streaming schema tracking**: Auto Loader requires a `cloudFiles.schemaLocation` checkpoint path to persist inferred schemas across restarts; omitting it forces re-inference on every restart
- **VARIANT type**: stores arbitrary semi-structured JSON in a single column; queried with colon-path notation (`event_data:user_id::INT`); useful when upstream schema is completely unknown

Related Topics

- Delta Lake Fundamentals - Core Delta features
- Auto Loader - Schema inference and evolution
- Structured Streaming - Streaming schema handling

Official Documentation

- Delta Lake Schema Enforcement
- Delta Lake Schema Evolution
- Column Mapping
- ALTER TABLE

Slowly Changing Dimension (SCD) Patterns

Slowly Changing Dimensions (SCDs) are data warehousing techniques for managing dimension data that changes over time. Understanding SCD patterns is essential for building accurate historical analytics.

Overview

```
flowchart TB
    SCD[SCD Patterns] --> Type1[Type 1: Overwrite]
    SCD --> Type2[Type 2: History]
    SCD --> Type3[Type 3: Previous]
    SCD --> Type4[Type 4: Mini-Dimension]
    SCD --> Type6[Type 6: Hybrid]

    Type1 --> T1D["No history"]
```

```
    Type2 --> T2D["Full history"]
    Type3 --> T3D["Limited history"]
```

Overview

- **SCD Type 1** overwrites; **Type 2** adds a history row per change with a surrogate key, `is_current`, and effective and end dates
- **Types 3, 4, and 6**: previous-value columns, mini-dimensions, and the hybrid of Types 1, 2, and 3
- **AUTO CDC** with `stored_as_scd_type=2` builds Type 2 history automatically, adding `__START_AT` and `__END_AT`
- **MERGE** implements SCD in Delta Lake; Type 2 often uses a two-step MERGE

What the Exam Tests

- Implement CDC pipelines using AUTO CDC APIs (formerly APPLY CHANGES) in Lakeflow Pipelines, including SCD Type 1 and Type 2 via stored_as_scd_type.
- Design dimensional models for analytical workloads, leveraging Materialized Views for pre-computed aggregation and Unity Catalog Metric Views for governed, reusable metric definitions, to ensure efficient querying and aggregation.
- Write a point-in-time query against a Type 2 dimension

SCD Type Comparison

Type	History Preserved	Storage	Complexity	Use Case
Type 1	None	Low	Simple	Corrections, non-critical changes
Type 2	Full	High	Medium	Complete audit trail
Type 3	Limited (1 previous)	Medium	Low	Track only previous value
Type 4	Full (separate table)	High	High	Rapidly changing attributes
Type 6	Full + Current	Highest	Highest	Blend of Type 1, 2, 3

SCD Type 1: Overwrite

Type 1 simply overwrites the existing value with the new value. No history is maintained.

Type 1 Schema

```
CREATE TABLE dim_customer (
    customer_id INT PRIMARY KEY,
    name STRING,
    email STRING,
    city STRING,
    tier STRING,
    updated_at TIMESTAMP
)
USING DELTA;
```

Type 1 Implementation

```
-- Type 1 using MERGE
MERGE INTO dim_customer AS target
USING staging_customer AS source
ON target.customer_id = source.customer_id
```

```
WHEN MATCHED THEN UPDATE SET
    target.name = source.name,
    target.email = source.email,
    target.city = source.city,
    target.tier = source.tier,
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN INSERT (
    customer_id, name, email, city, tier, updated_at
) VALUES (
    source.customer_id, source.name, source.email,
    source.city, source.tier, current_timestamp()
);
```

```
from delta.tables import DeltaTable

delta_table = DeltaTable.forName(spark, "dim_customer")

(delta_table.alias("target")
    .merge(
        staging_df.alias("source"),
        "target.customer_id = source.customer_id"
    )
    .whenMatchedUpdate(set={
        "name": "source.name",
        "email": "source.email",
        "city": "source.city",
        "tier": "source.tier",
        "updated_at": "current_timestamp()"
    })
    .whenNotMatchedInsertAll()
    .execute())
```

Type 1 - Before and After

```
Before:
| customer_id | name     | city       | tier   |
|-------------|----------|------------|--------|
| 1           | John Doe | New York   | Silver |

Change: John moves to Boston

After (Type 1):
| customer_id | name     | city       | tier   |
|-------------|----------|------------|--------|
| 1           | John Doe | Boston     | Silver |

History of "New York" is LOST
```

When to Use Type 1

- Data corrections (fixing typos)
- Non-critical attribute changes
- When history isn't required for reporting
- Source system doesn't provide history

SCD Type 2: Full History

Type 2 maintains complete history by inserting new rows for each change.

Type 2 Schema

```
CREATE TABLE dim_customer_type2 (
    surrogate_key BIGINT GENERATED ALWAYS AS IDENTITY,
    customer_id INT,
    name STRING,
    email STRING,
    city STRING,
    tier STRING,
    is_current BOOLEAN,
    effective_date DATE,
    end_date DATE,
    updated_at TIMESTAMP
)
USING DELTA;

-- Add index on business key + is_current for performance
-- CREATE INDEX idx_customer_current ON dim_customer_type2(customer_id,
 ↪  is_current);
```

Type 2 Implementation with MERGE

```
-- Step 1: Identify changed records and expire them
MERGE INTO dim_customer_type2 AS target
USING (
    SELECT
        s.customer_id,
        s.name,
        s.email,
        s.city,
        s.tier
    FROM staging_customer s
    INNER JOIN dim_customer_type2 d
        ON s.customer_id = d.customer_id
        AND d.is_current = true
    WHERE s.name != d.name
       OR s.email != d.email
       OR s.city != d.city
       OR s.tier != d.tier
) AS changes
ON target.customer_id = changes.customer_id
    AND target.is_current = true
WHEN MATCHED THEN UPDATE SET
    target.is_current = false,
    target.end_date = current_date() - 1,
    target.updated_at = current_timestamp();

-- Step 2: Insert new current records
INSERT INTO dim_customer_type2 (
    customer_id, name, email, city, tier,
    is_current, effective_date, end_date, updated_at
)
```

```sql
SELECT
    s.customer_id,
    s.name,
    s.email,
    s.city,
    s.tier,
    true AS is_current,
    current_date() AS effective_date,
    '9999-12-31' AS end_date,
    current_timestamp() AS updated_at
FROM staging_customer s
LEFT JOIN dim_customer_type2 d
    ON s.customer_id = d.customer_id
    AND d.is_current = true
WHERE d.customer_id IS NULL  -- New customers
   OR s.name != d.name  -- Changed customers
   OR s.email != d.email
   OR s.city != d.city
   OR s.tier != d.tier;
```

Type 2 - Single MERGE Pattern

```python
from delta.tables import DeltaTable
from pyspark.sql.functions import col, lit, current_date, current_timestamp, when

def scd_type2_merge(spark, source_df, target_table, business_key,
 ↪  tracked_columns):
    """
    Implement SCD Type 2 using MERGE.

    Args:
        source_df: DataFrame with new/changed records
        target_table: Target Delta table name
        business_key: Column name for business key
        tracked_columns: List of columns to track for changes
    """
    delta_table = DeltaTable.forName(spark, target_table)

    # Build change detection condition
    change_conditions = " OR ".join([
        f"source.{col} != target.{col}" for col in tracked_columns
    ])

    # Create staging view with flags
    source_df.createOrReplaceTempView("source_data")

    staged = spark.sql(f"""
        SELECT
            s.*,
            CASE
                WHEN t.{business_key} IS NULL THEN 'INSERT'
                WHEN {change_conditions.replace('source.',
 ↪  's.').replace('target.', 't.')} THEN 'UPDATE'
                ELSE 'NO_CHANGE'
            END AS merge_action
        FROM source_data s
```

```
        LEFT JOIN {target_table} t
            ON s.{business_key} = t.{business_key}
            AND t.is_current = true
    """)

    # Expire changed records
    spark.sql(f"""
        MERGE INTO {target_table} AS target
        USING (
            SELECT {business_key} FROM source_data
            WHERE merge_action = 'UPDATE'
        ) AS expired
        ON target.{business_key} = expired.{business_key}
            AND target.is_current = true
        WHEN MATCHED THEN UPDATE SET
            is_current = false,
            end_date = current_date() - interval 1 day,
            updated_at = current_timestamp()
    """)

    # Insert new and changed records
    columns = ", ".join(tracked_columns)
    spark.sql(f"""
        INSERT INTO {target_table}
        ({business_key}, {columns}, is_current, effective_date, end_date,
↪   updated_at)
        SELECT
            {business_key}, {columns},
            true AS is_current,
            current_date() AS effective_date,
            DATE'9999-12-31' AS end_date,
            current_timestamp() AS updated_at
        FROM staged
        WHERE merge_action IN ('INSERT', 'UPDATE')
    """)
```

Type 2 - Before and After

```
Before:
| sk | customer_id | city     | is_current | effective | end_date   |
|----|-------------|----------|------------|-----------|------------|
| 1  | 1           | New York | true       | 2023-01-01| 9999-12-31 |

Change: John moves to Boston on 2024-01-15

After (Type 2):
| sk | customer_id | city     | is_current | effective | end_date   |
|----|-------------|----------|------------|-----------|------------|
| 1  | 1           | New York | false      | 2023-01-01| 2024-01-14 |
| 2  | 1           | Boston   | true       | 2024-01-15| 9999-12-31 |

Full history is PRESERVED
```

Querying Type 2 Dimensions

```sql
-- Current state
SELECT * FROM dim_customer_type2
WHERE is_current = true;

-- Historical point-in-time
SELECT * FROM dim_customer_type2
WHERE effective_date <= '2023-06-15'
  AND end_date >= '2023-06-15';

-- Full history for a customer
SELECT * FROM dim_customer_type2
WHERE customer_id = 1
ORDER BY effective_date;

-- Join fact table with dimension at transaction time
SELECT
    f.order_id,
    f.order_date,
    f.amount,
    d.city AS city_at_order_time
FROM fact_orders f
JOIN dim_customer_type2 d
    ON f.customer_id = d.customer_id
    AND f.order_date BETWEEN d.effective_date AND d.end_date;
```

SCD Type 3: Previous Value

Type 3 adds columns to track the previous value of specific attributes.

Type 3 Schema

```sql
CREATE TABLE dim_customer_type3 (
    customer_id INT PRIMARY KEY,
    name STRING,
    email STRING,
    current_city STRING,
    previous_city STRING,
    city_change_date DATE,
    current_tier STRING,
    previous_tier STRING,
    tier_change_date DATE,
    updated_at TIMESTAMP
)
USING DELTA;
```

Type 3 Implementation

```sql
MERGE INTO dim_customer_type3 AS target
USING staging_customer AS source
ON target.customer_id = source.customer_id
WHEN MATCHED AND target.current_city != source.city THEN UPDATE SET
    target.previous_city = target.current_city,
    target.current_city = source.city,
    target.city_change_date = current_date(),
    target.updated_at = current_timestamp()
```

```
WHEN MATCHED AND target.current_tier != source.tier THEN UPDATE SET
    target.previous_tier = target.current_tier,
    target.current_tier = source.tier,
    target.tier_change_date = current_date(),
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN INSERT (
    customer_id, name, email, current_city, current_tier, updated_at
) VALUES (
    source.customer_id, source.name, source.email,
    source.city, source.tier, current_timestamp()
);
```

Type 3 - Before and After

```
Before:
| customer_id | current_city | previous_city | city_change_date |
|-------------|--------------|---------------|------------------|
| 1           | New York     | NULL          | NULL             |

Change: John moves to Boston

After (Type 3):
| customer_id | current_city | previous_city | city_change_date |
|-------------|--------------|---------------|------------------|
| 1           | Boston       | New York      | 2024-01-15       |

Only ONE previous value is tracked
```

SCD Type 4: Mini-Dimension

Type 4 separates rapidly changing attributes into a separate dimension table.

Type 4 Schema

```
-- Main dimension (slowly changing attributes)
CREATE TABLE dim_customer (
    customer_id INT PRIMARY KEY,
    name STRING,
    email STRING,
    registration_date DATE
)
USING DELTA;

-- Mini-dimension (rapidly changing attributes)
CREATE TABLE dim_customer_profile (
    profile_key BIGINT GENERATED ALWAYS AS IDENTITY,
    customer_id INT,
    city STRING,
    tier STRING,
    loyalty_points INT,
    is_current BOOLEAN,
    effective_date DATE,
    end_date DATE
)
USING DELTA;
```

```sql
-- Fact table references both
CREATE TABLE fact_orders (
    order_id STRING,
    customer_id INT,
    profile_key BIGINT,  -- Links to mini-dimension
    order_date DATE,
    amount DECIMAL(18,2)
)
USING DELTA;
```

Type 4 - Diagram

```mermaid
flowchart LR
    Fact[fact_orders] --> DimCustomer[dim_customer]
    Fact --> MiniDim[dim_customer_profile]

    DimCustomer --> |customer_id| Slow[Slow attributes: name, email]
    MiniDim --> |profile_key| Rapid[Rapid attributes: tier, points]
```

SCD Type 6: Hybrid (1 + 2 + 3)

Type 6 combines Types 1, 2, and 3 for maximum flexibility.

Type 6 Schema

```sql
CREATE TABLE dim_customer_type6 (
    surrogate_key BIGINT GENERATED ALWAYS AS IDENTITY,
    customer_id INT,
    name STRING,

    -- Type 3 columns (current + previous)
    current_city STRING,
    previous_city STRING,

    -- Additional Type 2 historical tracking
    is_current BOOLEAN,
    effective_date DATE,
    end_date DATE,

    updated_at TIMESTAMP
)
USING DELTA;
```

Type 6 Implementation

```sql
-- Type 6: Maintain both history rows AND current value columns

-- Step 1: Update all historical rows with new current values (Type 1 aspect)
UPDATE dim_customer_type6
SET current_city = (
    SELECT city FROM staging_customer s
    WHERE s.customer_id = dim_customer_type6.customer_id
)
WHERE customer_id IN (SELECT customer_id FROM staging_customer);
```

```sql
-- Step 2: Expire current rows for changed records (Type 2 aspect)
UPDATE dim_customer_type6
SET
    is_current = false,
    end_date = current_date() - 1
WHERE is_current = true
  AND customer_id IN (
    SELECT s.customer_id
    FROM staging_customer s
    JOIN dim_customer_type6 d
      ON s.customer_id = d.customer_id AND d.is_current = true
    WHERE s.city != d.current_city
  );

-- Step 3: Insert new rows (Type 2 aspect)
INSERT INTO dim_customer_type6 (
    customer_id, name, current_city, previous_city,
    is_current, effective_date, end_date
)
SELECT
    s.customer_id,
    s.name,
    s.city AS current_city,
    d.current_city AS previous_city,  -- Type 3 aspect
    true,
    current_date(),
    '9999-12-31'
FROM staging_customer s
JOIN dim_customer_type6 d
    ON s.customer_id = d.customer_id
WHERE s.city != d.current_city
  AND d.effective_date = (
    SELECT MAX(effective_date)
    FROM dim_customer_type6
    WHERE customer_id = s.customer_id
  );
```

Lakeflow Pipelines AUTO CDC for SCD Type 2

Lakeflow Pipelines provide built-in SCD Type 1 and Type 2 through the AUTO CDC APIs (formerly APPLY CHANGES). The target must be a streaming table; `stored_as_scd_type` selects the SCD type (default 1).

Platform update (Oct 2026)

AUTO CDC replaces APPLY CHANGES with the same syntax: `dlt.apply_changes()` is now `dp.create_auto_cdc_flow()` (same signature) and the Python module is `from pyspark import pipelines as dp` (`import dlt` still works). Source: The AUTO CDC APIs

```python
from pyspark import pipelines as dp
from pyspark.sql.functions import col

# Source data
@dp.temporary_view
def customers_cdc():
    return (spark.readStream.format("cloudFiles")
```

```
        .option("cloudFiles.format", "json")
        .load("/Volumes/main/landing/customers_cdc/"))

# SCD Type 2 with AUTO CDC
dp.create_streaming_table("dim_customer_scd2")

dp.create_auto_cdc_flow(
    target="dim_customer_scd2",
    source="customers_cdc",
    keys=["customer_id"],
    sequence_by=col("updated_at"),
    stored_as_scd_type=2,
    column_list=["customer_id", "name", "email", "city", "tier", "updated_at"],
    track_history_column_list=["city", "tier"],
)

# Resulting table has:
# - __START_AT / __END_AT: the sequence values (same type as updated_at) that
 ↪  opened and closed each version
# - __END_AT IS NULL marks the current version
# - Changes to columns outside track_history_column_list update the current
 ↪  version in place
```

```
-- Same flow in SQL
CREATE OR REFRESH STREAMING TABLE dim_customer_scd2;

CREATE FLOW dim_customer_scd2_cdc AS AUTO CDC INTO dim_customer_scd2
FROM STREAM(customers_cdc)
KEYS (customer_id)
SEQUENCE BY updated_at
COLUMNS (customer_id, name, email, city, tier, updated_at)
STORED AS SCD TYPE 2
TRACK HISTORY ON * EXCEPT (name, email, updated_at);
```

Lakeflow Pipelines SCD Type 1

```
dp.create_streaming_table("dim_customer_scd1")

dp.create_auto_cdc_flow(
    target="dim_customer_scd1",
    source="customers_cdc",
    keys=["customer_id"],
    sequence_by=col("updated_at"),
    stored_as_scd_type=1,  # Type 1: overwrite (the default)
)
```

Handling Deletes in SCD

Soft Deletes in Type 2

```
-- Mark as deleted instead of removing
MERGE INTO dim_customer_type2 AS target
USING deleted_customers AS source
ON target.customer_id = source.customer_id
    AND target.is_current = true
WHEN MATCHED THEN UPDATE SET
```

```
    target.is_current = false,
    target.end_date = current_date() - 1,
    target.is_deleted = true,
    target.deleted_at = current_timestamp();
```

Lakeflow Pipelines with Deletes

```
dp.create_auto_cdc_flow(
    target="dim_customer_scd2",
    source="customers_cdc",
    keys=["customer_id"],
    sequence_by=col("updated_at"),
    stored_as_scd_type=2,
    apply_as_deletes="operation = 'DELETE'",
    except_column_list=["operation"],
)
# For SCD Type 2, a delete closes the current version (sets __END_AT).
# apply_as_truncates is supported only for SCD Type 1.
```

Performance Considerations

Indexing Type 2 Tables

```
-- Optimize for common query patterns
-- Note: Delta Lake doesn't have traditional indexes, use ZORDER instead

OPTIMIZE dim_customer_type2
ZORDER BY (customer_id, is_current);

-- Or use liquid clustering
ALTER TABLE dim_customer_type2
CLUSTER BY (customer_id, is_current);
```

Partitioning Type 2 Tables

```
-- Partition by is_current for current-state queries
CREATE TABLE dim_customer_type2 (
    ...
)
USING DELTA
PARTITIONED BY (is_current);

-- Or partition by effective date for time-based queries
CREATE TABLE dim_customer_type2 (
    ...
    effective_year INT GENERATED ALWAYS AS (YEAR(effective_date))
)
USING DELTA
PARTITIONED BY (effective_year);
```

Use Cases

Choosing SCD Type

Scenario	Recommended Type	Rationale
Correcting data errors	Type 1	No need to preserve mistakes
Regulatory compliance	Type 2	Full audit trail required
”What was the value last month?”	Type 3	Simple previous value tracking
Customer tier changes	Type 2	Analyze tier progression
Rapidly changing attributes	Type 4	Separate from stable dimensions
Complex analytics needs	Type 6	Maximum flexibility

Common Issues & Errors

Duplicate Surrogate Keys **Scenario:** Multiple current records for same business key.

Fix: Add constraint or validate:

```
-- Check for duplicates
SELECT customer_id, COUNT(*) as cnt
FROM dim_customer_type2
WHERE is_current = true
GROUP BY customer_id
HAVING COUNT(*) > 1;

-- Fix: Keep latest, expire others
WITH ranked AS (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY customer_id
        ORDER BY effective_date DESC
    ) as rn
    FROM dim_customer_type2
    WHERE is_current = true
)
UPDATE dim_customer_type2
SET is_current = false, end_date = current_date() - 1
WHERE surrogate_key IN (
    SELECT surrogate_key FROM ranked WHERE rn > 1
);
```

Incorrect End Dates **Scenario:** Gap or overlap in effective date ranges.

Fix: Validate date ranges:

```
-- Check for gaps
SELECT t1.customer_id, t1.end_date, t2.effective_date
FROM dim_customer_type2 t1
JOIN dim_customer_type2 t2
    ON t1.customer_id = t2.customer_id
    AND t1.end_date + 1 != t2.effective_date
    AND t1.is_current = false
    AND t2.surrogate_key > t1.surrogate_key;
```

Performance Issues on Large Type 2 Tables **Scenario:** Queries on Type 2 dimension are slow.

Fix: Optimize query patterns:

```
# Filter early on is_current
```

```python
current_df = spark.table("dim_customer_type2").filter("is_current = true")

# Use broadcast for small dimensions

from pyspark.sql.functions import broadcast
result = fact_df.join(broadcast(current_df), "customer_id")
```

Exam Tips

1. **Type 1** - Overwrites, no history, simple UPDATE
2. **Type 2** - Full history, surrogate key, is_current flag + effective dates
3. **Type 3** - Previous value column, limited history
4. **Type 4** - Separate mini-dimension for rapidly changing attributes
5. **Type 6** - Hybrid of 1, 2, 3 for maximum flexibility
6. **MERGE** - Primary mechanism for SCD implementation in Delta Lake
7. **Lakeflow Pipelines AUTO CDC (formerly APPLY CHANGES)** - Built-in SCD support with `stored_as_scd_type`
8. **End date convention** - Often `9999-12-31` for current records
9. **Point-in-time query** - `effective_date <= date AND end_date >= date`
10. **Surrogate vs Natural key** - Type 2 requires surrogate key for uniqueness

Key Takeaways

- **SCD Type 1**: overwrites the existing row with new values; no history is preserved; implemented with a MERGE that only has `WHEN MATCHED THEN UPDATE` and `WHEN NOT MATCHED THEN INSERT`
- **SCD Type 2**: preserves full history by inserting a new row on each change; requires a surrogate key (often an identity column), `is_current BOOLEAN`, `effective_date`, and `end_date` (convention: `9999-12-31` for active rows)
- **SCD Type 2 point-in-time query**: `WHERE effective_date <= '<as_of_date>' AND end_date >= '<as_of_date>'` retrieves the dimension value that was active at a given point in time
- **SCD Type 3**: adds `previous_<attr>` columns to track only the immediately prior value; limited to one change of history per tracked attribute
- **SCD Type 4**: extracts rapidly changing attributes into a separate mini-dimension table; the fact table holds a foreign key to the mini-dimension at transaction time
- **SCD Type 6**: hybrid of Types 1 + 2 + 3; stores full Type 2 history rows AND carries a `current_<attr>` column (Type 1 update) and a `previous_<attr>` column (Type 3) on every row
- **Lakeflow Pipelines AUTO CDC (formerly APPLY CHANGES)**: `dp.create_auto_cdc_flow(..., stored_as_scd_type=2)` (formerly `dlt.apply_changes`) generates Type 2 history automatically; the resulting table adds `__START_AT` and `__END_AT` columns holding sequence values; `stored_as_scd_type=1` (the default) does a simple upsert
- **MERGE as primary SCD mechanism**: MERGE is the standard DML for all SCD types in Delta Lake; two-step MERGE (expire then insert) is common for Type 2 to avoid ambiguous row matching

Related Topics

- Delta Lake Fundamentals - MERGE operations
- Medallion Architecture - Where dimensions fit
- AUTO CDC API - Lakeflow Pipelines SCD support

Official Documentation

- Delta Lake MERGE

- The AUTO CDC APIs
- create_auto_cdc_flow
- Change data capture and snapshots (SCD Type 1 and 2)

Dimensional Modeling and Metric Views

Overview

Analytical (Gold) layers on Databricks are usually modeled as star or snowflake schemas: fact tables at a declared grain, joined to dimension tables through surrogate keys. On top of that model, **materialized views** pre-compute expensive joins and aggregations, and **Unity Catalog metric views** define each business metric once as governed, reusable logic that any tool can query at any grouping.

Overview

- Star and snowflake schemas fit Databricks better than heavily normalized (3NF) models. They need fewer joins, and wider tables give file-level data skipping more to work with
- Surrogate keys come from identity columns. Primary key and foreign key constraints are informational, and with RELY they let the optimizer eliminate joins and aggregations
- Materialized views store query results and refresh incrementally when possible (manually, on a schedule, or `TRIGGER ON UPDATE`)
- Metric views (`CREATE VIEW ... WITH METRICS LANGUAGE YAML`) declare source, joins, fields (dimensions), and measures in YAML. Consumers query them with `MEASURE()`. Window measures are GA (Oct 6, 2026)

What the Exam Tests

- ”Design dimensional models for analytical workloads, leveraging Materialized Views for pre-computed aggregation and Unity Catalog Metric Views for governed, reusable metric definitions, to ensure efficient querying and aggregation”
- Recognizing that inconsistent metric SQL across dashboards calls for a metric view, and that slow dashboards recomputing big joins call for materialized views
- Picking between a metric view, a materialized view, and a standard view for a given requirement
- Knowing how keys, informational constraints, and clustering support star-schema queries

Platform update (Oct 2026)

Metric view docs moved under **Unity Catalog semantics** and now use the keyword `fields`. `dimensions` is still accepted as a synonym for backward compatibility. Window measures in metric views became GA on October 6, 2026. Sources: Metric view YAML syntax reference, Databricks platform release notes, October 2026

Star Schema on Databricks

Databricks recommends against heavily normalized models such as third normal form for new lakehouse datasets. Star and snowflake schemas ”perform well on Databricks, as there are fewer joins present in standard queries and fewer keys to keep in sync”. More fields in a single table also let the optimizer skip more data using file-level statistics.

```
flowchart LR
    F["fact_sales<br>grain: one row per order line<br>date_key, customer_key,
 ↪  product_key,<br>quantity, gross_amount, discount_amount"]
    D1["dim_date<br>date_key (PK)"]
    D2["dim_customer<br>customer_key (PK, identity)<br>SCD Type 2"]
    D3["dim_product<br>product_key (PK)"]
    D4["dim_category<br>(snowflaked)"]
    F --> D1
    F --> D2
    F --> D3
    D3 --> D4
```

Concept	Fact table	Dimension table
Holds	Measurable events (amounts, quantities) at a declared grain	Descriptive attri
Keys	Foreign keys to each dimension, plus degenerate keys such as `order_id`	Surrogate prima
Size and change	Large, append-heavy	Smaller. Change
Typical layout	Liquid clustering on frequently filtered or joined keys (date, customer)	Liquid clustering
Built with	Streaming tables or `MERGE`; aggregates as materialized views	AUTO CDC (SCD

Declare the grain first. Every measure in a fact table must be valid at that grain, for example "one row per order line". Mixed grains in one table cause double counting when the table is joined.

Keys: Surrogate Keys and Informational Constraints

Identity columns for surrogate keys

```
CREATE TABLE main.gold.dim_customer (
  customer_key   BIGINT NOT NULL GENERATED ALWAYS AS IDENTITY,
  customer_id    STRING NOT NULL,          -- natural / business key
  customer_name  STRING,
  segment        STRING,
  valid_from     TIMESTAMP,
  valid_to       TIMESTAMP,
  is_current     BOOLEAN,
  CONSTRAINT dim_customer_pk PRIMARY KEY (customer_key)
);
```

Identity column rules from the `CREATE TABLE` reference:

- The column must be `BIGINT`. `START WITH` and `INCREMENT BY` default to 1, and `step` can't be 0.
- Values are unique and statistically increasing, but **not guaranteed contiguous**.
- With `GENERATED ALWAYS`, you can't supply your own values. `GENERATED BY DEFAULT` allows explicit values.
- You can't `PARTITIONED BY` or `UPDATE` an identity column.
- **Declaring an identity column disables concurrent transactions** on that table. Use it only where concurrent writes to the table aren't needed.
- **Materialized views don't support identity columns or surrogate keys.** Generate keys in a Delta table (or the pipeline target) and keep materialized views for aggregates.

Primary and foreign keys are informational Primary key, foreign key, and unique constraints aren't enforced. Primary and foreign keys are GA in Databricks Runtime 15.2+. Unique constraints are Public Preview in Databricks Runtime 18.2+. Validate key integrity in the pipeline, for example with expectations. With `RELY`, the optimizer can use a constraint to rewrite queries.

```sql
CREATE TABLE main.gold.fact_sales (
  order_id        STRING,
  date_key        INT,
  customer_key    BIGINT,
  product_key     BIGINT,
  quantity        INT,
  gross_amount    DECIMAL(18,2),
  discount_amount DECIMAL(18,2),
  CONSTRAINT fact_sales_customer_fk FOREIGN KEY (customer_key)
    REFERENCES main.gold.dim_customer
)
CLUSTER BY (date_key, customer_key);

-- Mark a verified primary key as reliable so the optimizer can use it
ALTER TABLE main.gold.dim_customer DROP PRIMARY KEY;
ALTER TABLE main.gold.dim_customer ADD PRIMARY KEY (customer_key) RELY;
```

Optimization with a RELY primary key or unique constraint	Effect
Eliminate unnecessary aggregations	`SELECT DISTINCT` pk skips the `DISTINCT`
Eliminate unnecessary joins	A `LEFT JOIN` to a dimension on its key is removed

- RELY optimizations require Photon-enabled compute.
- Relying on a constraint that isn't actually true can return incorrect results.
- CTAS doesn't support the constraint clause. Add constraints with `ALTER TABLE ... ADD CONSTRAINT`.
- Catalog Explorer can show an Entity Relationship Diagram from declared primary and foreign keys.

Layout for star-schema joins Dynamic file pruning pushes filter results from a filtered dimension down to the fact table scan at runtime. Its effectiveness depends on clustering. The docs recommend liquid clustering on the columns you frequently join on, so matching rows sit in fewer files. By default, dynamic file pruning triggers when the large table is at least 10 GB with at least 10 files. See Partitioning Strategies for layout design.

Materialized Views for Pre-Computed Aggregation

A materialized view is a Unity Catalog managed table that physically stores a query's results and updates them as the sources change. Typical uses are serving pre-computed results to dashboards and building layered Gold transformations.

Where defined	Syntax
Standalone (Databricks SQL warehouse or serverless notebook)	`CREATE [OR REPLACE] MATERIALIZED VIEW`
Lakeflow pipeline	`CREATE OR REFRESH MATERIALIZED VIEW` o

```sql
-- Daily revenue by region, refreshed on a schedule
CREATE OR REPLACE MATERIALIZED VIEW main.gold.daily_revenue_by_region
  SCHEDULE CRON '0 30 3 * * ?' AT TIME ZONE 'UTC'
AS SELECT
  d.calendar_date,
  c.region,
  SUM(f.gross_amount - f.discount_amount) AS net_revenue,
  COUNT(*)                                AS order_lines
FROM main.gold.fact_sales f
```

```
JOIN main.gold.dim_date d     ON f.date_key = d.date_key
JOIN main.gold.dim_customer c ON f.customer_key = c.customer_key
GROUP BY d.calendar_date, c.region;
```

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, sum as sum_

@dp.materialized_view(comment="Net revenue by region and day")
def daily_revenue_by_region():
    f = spark.read.table("main.gold.fact_sales")
    c = spark.read.table("main.gold.dim_customer")
    return (f.join(c, "customer_key")
        .groupBy("date_key", "region")
        .agg(sum_(col("gross_amount") -
 ↪  col("discount_amount")).alias("net_revenue")))
```

Refresh options

Option	Syntax	Use when
Manual	`REFRESH MATERIALIZED VIEW mv [FULL]`, add `ASYNC` to return immediately	Ad hoc, dev
Schedule	`SCHEDULE EVERY 1 HOUR` or `SCHEDULE CRON '...' AT TIME ZONE '...'`	Predictable
Trigger on update	`TRIGGER ON UPDATE [AT MOST EVERY INTERVAL 5 MINUTES]`	Production
Job SQL task	Lakeflow Jobs SQL task running `REFRESH`	Cross-syste

The owner, and any user granted `REFRESH` on the view, can refresh it.

Incremental vs full refresh

- **Incremental refresh** processes only changes since the last update. **Full refresh** reruns the query. Results are identical. By default a cost model picks the cheaper method.
- Incremental refresh **requires serverless** and works best with sources that have **row tracking** enabled. Deletion vectors and change data feed are also recommended.
- Supported incrementally: deterministic `SELECT` expressions, `GROUP BY`, `WITH`, `UNION ALL`, `WHERE` / `HAVING`, inner and outer joins, window functions with `PARTITION BY`, `QUALIFY`, and expectations.
- Forces a full recompute: recursive CTEs, non-deterministic functions (except time functions in `WHERE`), and `SUM` / `AVG` over `FLOAT` / `DOUBLE`. Cast those columns to `DECIMAL` to keep incremental refresh.
- Control the behavior with `REFRESH POLICY AUTO | INCREMENTAL | INCREMENTAL STRICT | FULL`. `INCREMENTAL STRICT` fails rather than fall back to a full refresh.
- Check eligibility with `EXPLAIN CREATE MATERIALIZED VIEW`, and check the technique used (`ROW_BASED`, `GROUP_AGGREGATE`, `FULL_RECOMPUTE`, and so on) in the event log, `event_type = 'planning_information'`.

```
ALTER TABLE main.gold.fact_sales SET TBLPROPERTIES (
  delta.enableDeletionVectors = true,
  delta.enableRowTracking = true,
  delta.enableChangeDataFeed = true);

CREATE MATERIALIZED VIEW main.gold.revenue_by_product
REFRESH POLICY INCREMENTAL
AS SELECT product_key, SUM(gross_amount) AS gross_revenue
FROM main.gold.fact_sales GROUP BY product_key;
```

Other materialized view limitations: no time travel, and the change data feed can't be read unless enabled on the view (Beta). Underlying files can contain upstream values that aren't in the view's schema, so don't share the storage with untrusted consumers.

Unity Catalog Metric Views

A metric view separates **measure definitions** from the **fields** (dimensions) used to group and filter them. A standard view locks in its groupings when you create it. A metric view lets consumers group by any available field, and the engine generates the correct aggregation. Metric views are Unity Catalog securables, so one definition serves SQL, AI/BI dashboards, Genie Agents, alerts, JDBC/ODBC, and external BI tools.

YAML structure

Top-level key	Required	Purpose
`version`	Yes	YAML specification version, for example `1.1`
`source`	Yes	Table, view, metric view, or SQL query, usually the fa
`joins`	No	Star joins (`name`, `source`, `'on'` or `using`). Nest `joins`
`filter`	No	SQL boolean expression applied to every query
`fields` (alias `dimensions`)	One of fields or measures	Scalar expressions used in `SELECT`, `WHERE`, and `GROUP`
`measures`	One of fields or measures	Aggregate expressions, queried with `MEASURE()`. The
`parameters`	No	Named values passed when the view is queried as a
`materialization`	No	Pre-computed materialized views with automatic que
`comment`	No	Description. Fields and measures can also carry `disp`

Example: one governed "net revenue" This is the scenario from the exam's sample question 10: one authoritative definition replaces SQL copied into each dashboard.

```
CREATE OR REPLACE VIEW main.gold.sales_metrics
WITH METRICS
LANGUAGE YAML
AS $$
version: 1.1
comment: "Governed sales KPIs"
source: main.gold.fact_sales
joins:
  - name: customer
    source: main.gold.dim_customer
    'on': source.customer_key = customer.customer_key
    rely:
      at_most_one_match: true
  - name: dt
    source: main.gold.dim_date
    'on': source.date_key = dt.date_key
fields:
  - name: order_month
    expr: DATE_TRUNC('MONTH', dt.calendar_date)
  - name: region
    expr: customer.region
  - name: segment
    expr: customer.segment
measures:
  - name: gross_revenue
    expr: SUM(source.gross_amount)
```

```
  - name: discounts
    expr: SUM(source.discount_amount)
  - name: net_revenue
    expr: MEASURE(gross_revenue) - MEASURE(discounts)
    comment: "Gross revenue minus discounts - the single company definition"
  - name: order_count
    expr: COUNT(DISTINCT source.order_id)
  - name: avg_order_value
    expr: MEASURE(net_revenue) / MEASURE(order_count)
$$;
```

Modeling notes from the docs:

- **Star joins** are `LEFT OUTER JOIN`s from the source (fact) to dimensions and must be many-to-one. In a many-to-many case, the first matching row is selected. Only the joins a query needs are performed.
- **`rely: at_most_one_match: true`** (DBR 18.1+) lets the planner optimize many-to-one joins. It isn't validated, so a fan-out returns wrong `SUM` / `COUNT` results.
- **Snowflake schemas** nest `joins` inside a join (DBR 17.3+). Reference columns by path, such as `customer.nation.n_name`.
- **`cardinality: one_to_many`** (DBR 18.1+, spec 1.1) treats a joined table as a separate fact source aggregated at the source grain.
- **Composed measures** reference earlier measures with `MEASURE(name)`, so a change to an atomic measure flows through to everything built on it.
- When a SQL query with a `JOIN` is the `source`, declare primary and foreign key constraints with `RELY` on the underlying tables.

Querying a metric view

```
SELECT
  order_month,
  region,
  MEASURE(net_revenue)     AS net_revenue,
  MEASURE(avg_order_value) AS aov
FROM main.gold.sales_metrics
WHERE segment = 'Enterprise'
GROUP BY ALL
ORDER BY order_month;
```

- Every measure must be wrapped in `MEASURE()`. `agg()` is a synonym in DBR 18.1+. `SELECT *` doesn't work, so list fields and wrap measures explicitly.
- `MEASURE()` can't be used as a window function with `OVER`. In DBR 18.1+, `MEASURE(m) FILTER (WHERE ...)` is allowed.
- A metric view can't be joined directly at query time. Aggregate it in a CTE, then join the CTE.
- `DESCRIBE TABLE EXTENDED <metric_view> AS JSON` returns the YAML definition in the **View Text** field.

Window measures (GA Oct 6, 2026) Window measures define rolling, cumulative, period-over-period, and semiadditive aggregations.

Key	Required	Values
`order`	Yes	Field that orders the window. It must be deterministic
`range`	Yes	`current`, `cumulative`, `trailing <n> <unit>`, `leading <n> <unit>`, or `all`. The
`semiadditive`	Yes	`first` or `last`: how to collapse when `order` isn't in the `GROUP BY`
`offset`	No	Shift the frame, for example `-12 month` (dated, DBR 18.1+) or `-1` (numeric index,

```yaml
measures:
  - name: monthly_sales
    expr: SUM(sales)
    window:
      - order: month
        range: current
        semiadditive: last
  - name: monthly_sales_py
    expr: SUM(sales)
    window:
      - order: month
        range: current
        semiadditive: last
        offset: -12 month
  - name: yoy_growth_pct
    expr: (MEASURE(monthly_sales) - MEASURE(monthly_sales_py))
      / NULLIF(MEASURE(monthly_sales_py), 0)
  - name: t7d_customers
    expr: COUNT(DISTINCT customer_key)
    window:
      - order: order_date
        range: trailing 7 day
        semiadditive: last
```

Materialization for metric views Metric views can pre-compute aggregations themselves. A `materialization` block makes Databricks create a managed Lakeflow pipeline that maintains materialized views. The optimizer then **rewrites queries automatically**: exact match first, then rollup match, then unaggregated match, and finally the source tables.

```yaml
materialization:
  schedule: every 6 hours
  mode: relaxed
  materialized_views:
    - name: baseline
      type: unaggregated
    - name: revenue_by_month_region
      type: aggregated
      dimensions:
        - order_month
        - region
      measures:
        - gross_revenue
        - discounts
```

- Requirements: serverless compute enabled, and DBR 17.3+ for the querying compute.
- `mode` must be `relaxed`. Freshness isn't verified, so matched queries return the last refresh while unmatched queries read live data. Align the schedule with the upstream pipeline.
- The block lists fields under `dimensions:` even when the top level uses `fields:`.
- Non-additive measures such as `COUNT(DISTINCT)` and percentiles need an **exact** match. They can't be rolled up.
- You can't materialize a metric view whose source tables use row filters, column masks, or ABAC policies, or one with invoker-dependent expressions (`current_user()`) or parameters.

Governance and lifecycle

- **Privileges:** `GRANT SELECT ON <metric_view> TO <group>`. Standard Unity Catalog inheritance applies.
- **Editing:** only the owner can edit. `ALTER VIEW ... AS $$ <complete YAML> $$` replaces the whole definition. For collaborative editing, transfer ownership to a group that has SELECT on all sources. That isn't supported for materialized metric views.
- **Runtime:** metric views need DBR 16.4+. Joins with snowflake schemas, materialization, and agent metadata need 17.3+.
- **Sharing:** metric views can be added to an OpenSharing share (Beta).

Choosing: Metric View vs Materialized View vs Standard View

Requirement	Metric view	M
One governed definition of a KPI reused by every tool	**Best fit**	N
Consumers choose grouping and filters at query time	**Yes** (any field)	N
Pre-computed results, so dashboards don't recompute joins	Yes, with `materialization`	**B**
Stores data / incremental refresh	Only through materialization	**Y**
Ratio and non-additive metrics stay correct at any grain	**Yes**. Measures re-aggregate from base rows	O
Simple reusable SELECT logic or column-level abstraction	Overkill	O
Query syntax	`MEASURE(m)` with `GROUP BY`	P

Sample question 10 pattern: ”net revenue” is computed differently in each dashboard, and the dashboards are slow because they recompute large joins and aggregations. The answer is to define net revenue once in a Unity Catalog metric view, and use materialized views (standalone, or the metric view's own `materialization`) for the pre-computed aggregates the dashboards read.

Use Cases

- **Enterprise KPI layer:** finance, sales, and operations metrics defined once as metric views and consumed by AI/BI dashboards, Genie Agents, alerts, and Power BI or Tableau.
- **Dashboard acceleration:** Gold materialized views at the dashboard grain, with `TRIGGER ON UPDATE` or a schedule aligned to upstream pipelines.
- **Period-over-period reporting:** window measures for trailing 7-day users, month-to-date, or year-over-year growth, without hand-written window SQL in every report.
- **Retail star schema:** a `fact_sales` table clustered on date and customer keys, an SCD Type 2 customer dimension with identity surrogate keys, and RELY primary keys for join elimination.

Common Issues & Errors

Symptom	Cause
`SELECT *` on a metric view fails or omits measures	Measures must be evaluated with `MEASURE()`
Measures inflated after adding a join	Dimension join fans out (not many-to-one), or `at_most_`
Joining a metric view to another table fails	Direct joins aren't supported
Materialized view refresh is always a full recompute	Classic compute, row tracking missing on sources, or un
Identity column rejected in a materialized view	Materialized views don't support identity columns or sur
Concurrent writers fail on a dimension table	An identity column disables concurrent transactions
Metric view materialization creation fails	Source uses row filters, column masks, or ABAC, or uses
Dashboards show stale numbers for some queries	`relaxed` mode serves the last refresh when a query ma

Exam Tips

Tip

- **"Inconsistent metric definitions across dashboards"** means a Unity Catalog metric view. **"Slow dashboards recomputing joins and aggregations"** means materialized views. When both appear, combine them (sample question 10).
- **Metric view syntax:** `CREATE VIEW ... WITH METRICS LANGUAGE YAML AS $$ ... $$`. Query with `MEASURE()` and `GROUP BY`.
- **YAML keys:** `version`, `source`, `joins`, `filter`, `fields` / `dimensions`, `measures`, optional `materialization`. Window measures need `order`, `range`, and `semiadditive`.
- **Primary and foreign keys aren't enforced.** `RELY` enables join and aggregation elimination on Photon.
- **Identity columns** provide surrogate keys (`BIGINT GENERATED ALWAYS AS IDENTITY`). They're not contiguous, they disable concurrent transactions, and they aren't supported in materialized views.
- **Materialized view incremental refresh** needs serverless and works best with row tracking. `REFRESH POLICY INCREMENTAL STRICT` fails instead of doing a full recompute.
- **Star or snowflake over 3NF** on Databricks: fewer joins and better data skipping.

Key Takeaways

- **Dimensional model:** declare the fact grain, connect facts to dimensions with surrogate keys, and track dimension history with SCD Type 2 (AUTO CDC).
- **Keys:** identity columns generate surrogate keys. Primary and foreign keys are informational, and RELY unlocks optimizer rewrites.
- **Materialized views:** store results, refresh manually, on a schedule, or on upstream change, and refresh incrementally on serverless when the query and sources allow it.
- **Metric views:** YAML-defined, governed, reusable measures over a fact table and its joins. Consumers pick any grouping and call `MEASURE()`.
- **Window measures** (GA Oct 6, 2026) cover rolling, cumulative, and period-over-period metrics with `order`, `range`, `semiadditive`, and optional `offset`.
- **Metric view materialization** combines both ideas: governed definitions plus pre-computed aggregates, with automatic query rewrite.

Related Topics

- Medallion Architecture - where the Gold dimensional layer sits
- SCD Patterns - Type 1 and Type 2 dimensions
- AUTO CDC API - declarative SCD in Lakeflow Pipelines
- Partitioning Strategies - clustering facts on join and filter keys
- Lakeflow Spark Declarative Pipelines - materialized views and streaming tables in pipelines
- Expectations and Data Quality - validating key integrity

Official Documentation

- Data modeling
- Constraints on Databricks
- Query optimization using primary key and unique constraints
- CREATE TABLE [USING] (identity columns)
- Dynamic file pruning
- Use standalone materialized views
- Incremental refresh for materialized views
- Schedule refreshes
- Unity Catalog metric views

- Create a metric view
- Metric view YAML syntax reference
- Query metric views
- Advanced techniques for metric views
- Materialization for metric views
- Manage metric views
- Metric view feature availability
- measure aggregate function
- Databricks platform release notes, October 2026

Data Engineer Professional - Exam Resources

Preparation materials for the Databricks Certified Data Engineer Professional exam.

Available Resources

Resource	Description
Practice Questions	Practice questions for each of the 9 Oct 9, 2026 exam sections
Mock Exam	Full-length practice exam (63 questions, labelled by 2026 section)
Mock Exam 2	Alternative practice exam (60 questions, labelled by 2026 section)
Cheat Sheets	Quick reference cards for key topics
Exam Tips	Strategies, time management, common pitfalls
Final Review	20-minute exam-morning cram scan
Official Links	Databricks documentation and registration

Exam Tips and Strategies

Exam Format (Oct 9, 2026 exam guide)

- **Questions**: 60 scored multiple-choice, plus possible unscored items (extra time is already included)
- **Duration**: 120 minutes (about 2 minutes per question)
- **Passing score**: not published — Databricks reports pass / fail only
- **Language**: English (per the PDF); code in stems is Python and SQL
- **Delivery**: online proctored or test center; no test aides
- **Official samples**: the exam guide PDF ends with 10 sample questions, one per new-style objective — work them first

Time Management

Recommended Pacing

Time	Activity
0-90 min	First pass (1.5 min/question avg)
90-110 min	Review flagged questions
110-120 min	Final review

Question Strategy

1. **Read carefully** - Watch for words like ”BEST”, ”MOST”, ”LEAST”
2. **Flag and move** - Don't spend more than 2-3 minutes on difficult questions
3. **Eliminate wrong answers** - Narrow down to 2 choices
4. **Trust first instinct** - Don't change answers unless certain

Topic Priority by Weight

Priority	Section	Weight	Study time
1	Developing Code for Data Processing using Python and SQL	23 %	~23 %

Priority	Section	Weight	Study time
2	Cost & Performance Optimization	15 %	~15 %
3	Data Ingestion & Acquisition	12 %	~12 %
3	Data Manipulation	12 %	~12 %
5	Monitoring and Alerting	10 %	~10 %
5	Debugging and Deploying	10 %	~10 %
7	Ensuring Data Security and Compliance	8 %	~8 %
8	Data Governance	5 %	~5 %
8	Data Modeling	5 %	~5 %

Key Topics to Master

Must Know (named in the Oct 9, 2026 guide)

- ☐ Streaming table vs materialized view — which one reflects late updates to old aggregates
- ☐ AUTO CDC (formerly APPLY CHANGES): `keys`, `sequence_by`, `stored_as_scd_type` 1 vs 2
- ☐ Structured Streaming state: watermarks, output modes, `foreachBatch`, checkpoints
- ☐ Lakeflow Jobs: If/else and For each tasks; repair runs with parameter overrides
- ☐ Deletion vectors vs Liquid Clustering vs `CLUSTER BY AUTO`; Liquid Clustering vs partitioning / Z-ORDER
- ☐ Predictive Optimization on Unity Catalog managed tables
- ☐ System tables: `system.billing.usage + list_prices` for cost; `system.access.audit`; `system.lakeflow.*`
- ☐ ABAC policies on governed tags; column masks and row filters
- ☐ Unity Catalog permission inheritance (grants flow to future objects)
- ☐ Declarative Automation Bundles: `databricks.yml`, targets, `databricks bundle deploy -t <target>`

Should Know (new or reworded skills)

- ☐ VARIANT: `parse_json`, `:` paths, `variant_get`
- ☐ `ai_query` and task-specific AI functions in pipelines
- ☐ Lakeflow Connect CDC from SQL Server, MySQL, PostgreSQL
- ☐ OpenSharing D2D vs Databricks-to-Open; Clean Rooms
- ☐ Lakehouse Federation: connections, foreign catalogs, `USE CONNECTION`
- ☐ Unity Catalog metric views vs materialized views
- ☐ Lakehouse (SQL) alerts and the SQL alert task for job branching
- ☐ Python project layout for bundles; dependency troubleshooting on serverless
- ☐ `assertDataFrameEqual`, `assertSchemaEqual`, `DataFrame.transform`
- ☐ Disk cache (Delta cache) vs Spark cache

Good to Know

- ☐ Databricks CLI and SDK commands for jobs and pipelines
- ☐ Spark UI and compute logs for diagnosis
- ☐ Specific Spark configurations
- ☐ Git folders (formerly Repos) CI/CD

Common Question Patterns

Scenario-Based Questions

```
"A data engineer needs to process files as they arrive in cloud storage.
The solution must handle schema changes automatically.
Which approach should they use?"
```

Strategy: Identify requirements, match to feature capabilities.

Best Practice Questions

```
"What is the recommended approach for handling late data in streaming?"
```

Strategy: Know official best practices, not just what ”works.”

Troubleshooting Questions

```
"A streaming job is failing with OutOfMemoryError.
What should the engineer check FIRST?"
```

Strategy: Know debugging steps and common causes.

Configuration Questions

```
"Which configuration enables automatic schema evolution for Delta writes?"
```

Strategy: Memorize key configuration names and their effects.

Common Exam Traps

Watch For These

1. **”BEST” vs ”correct”** - Multiple answers may work, pick optimal
2. **Deprecated features** - Old ways may be in wrong answers
3. **Default values** - Know what's enabled/disabled by default
4. **Similar options** - `once=True` vs `availableNow=True`
5. **Order of operations** - Some steps must come before others

Specific Gotchas

Topic	Trap	Correct Answer
VACUUM	Any retention works	Default retention 7 d
Schema evolution	Always automatic	Must enable with opt
Streaming checkpoints	Optional	Required for exactly-
Unity Catalog	Workspace-level	Metastore is per regi
Clone	Deep always better	Shallow for testing
Late updates to an aggregate	Streaming table	Materialized view
Grant on a catalog	Only existing tables	Inherited by current
PII column masking at scale	Per-table `SET MASK`	ABAC policy on a gov
Frequent small MERGE / DELETE	Partition by key	Deletion vectors
Product names	”APPLY CHANGES”, ”Asset Bundles”, ”Delta Sharing”	AUTO CDC, Declarat

Day Before Exam

Do

- ☐ Review cheat sheets
- ☐ Get good sleep
- ☐ Prepare testing environment
- ☐ Test internet connection
- ☐ Have ID ready

Don't

- ☐ Cram new material
- ☐ Stay up late
- ☐ Skip meals
- ☐ Stress about specific questions

During the Exam

Technical Setup

- Stable internet connection
- Quiet environment
- Clear desk (proctored)
- Valid government ID
- Close unnecessary applications

Mental Approach

- Stay calm, you've prepared
- Each question is independent
- Wrong answers don't cascade
- Use full time available
- Flag uncertain questions

If You Don't Pass

- Review score report by section
- Focus on weak areas
- Check the Certification FAQ for the current retake policy
- Practice more questions in weak areas
- Re-take with confidence

Study Resources Priority

Essential (Do These)

1. Databricks Academy - Advanced Data Engineering
2. Official documentation for weak areas
3. Practice exams — check they target the Oct 9, 2026 guide
4. Hands-on labs in Databricks Free Edition

Helpful (If Time Permits)

1. Community posts and forums
2. YouTube tutorials
3. Blog posts from certified engineers
4. Additional practice questions

Skip

- Content written before the Oct 9, 2026 guide that still uses 10 domains or 59 questions
- Non-Databricks specific Spark content
- Deep dives on rarely tested topics

Quick Reference Numbers

Item	Value
VACUUM default retention	168 hours (7 days)
Default shuffle partitions	200
Broadcast join threshold	10 MB
Checkpoint retention	Indefinite (manual cleanup)
Transaction log retention (`delta.logRetentionDuration`)	30 days
Streaming micro-batch	Default processing mode

Final Checklist

Before clicking "Submit":

- ☐ All questions answered
- ☐ Flagged questions reviewed
- ☐ No obvious mistakes
- ☐ Time remaining checked
- ☐ Ready to submit

Final Review — DE Professional (20-minute exam-morning scan)

Important

Cram scan for exam morning. Each line is a fact the exam can test — if any look unfamiliar, hit the topic folder on the way home.

2-minute facts that show up *most often*

- **Streaming table vs materialized view**: an aggregate that must reflect **late updates to prior days** is a materialized view; append-only ingestion processed once is a streaming table
- **AUTO CDC (formerly `APPLY CHANGES INTO`)** is the declarative SCD inside Lakeflow Pipelines — keys + `sequence_by` required, `stored_as_scd_type` 1 (default) or 2
- **Watermark + checkpoint**: the watermark bounds state for late events; the checkpoint restores offsets and state so a restart resumes exactly once
- **`MERGE INTO`** is the atomic upsert on Delta; **deletion vectors** cut the file rewrites of frequent small MERGE / UPDATE / DELETE
- **Liquid clustering** (`CLUSTER BY`, or `CLUSTER BY AUTO` to let Databricks pick keys) is the recommended layout for new tables; `OPTIMIZE ... FULL` re-clusters existing data
- **Predictive Optimization** runs OPTIMIZE, VACUUM and statistics on Unity Catalog managed tables for you
- **Cost attribution** = `system.billing.usage` joined with `system.billing.list_prices` (and `system.lakeflow.jobs` / compute tables)
- **ABAC policy on a governed tag** masks every tagged column in current and future tables — no per-table `SET MASK`
- **UC grants inherit downward** to current and future objects; `USE CATALOG` + `USE SCHEMA` are still required
- **Declarative Automation Bundles** (formerly Asset Bundles): `databricks.yml` + targets; `databricks bundle deploy -t <target>`

5-minute per-section quick-fire (9 sections)

01 — Developing Code for Data Processing using Python and SQL (23 %)

- Bundle project: `databricks.yml`, `resources/`, `src/<package>/`, `tests/`; build a wheel as an artifact
- UDFs: prefer built-ins → SQL UDF → pandas UDF → Python UDF; Unity Catalog functions for governed reuse (EXECUTE)
- Pipelines Python API: `from pyspark import pipelines as dp`; `@dp.table`, `@dp.materialized_view`, `@dp.expect_or_drop`
- Lakeflow Jobs: If/else condition and For each tasks; serverless performance-optimized vs standard mode
- Tests: `assertDataFrameEqual`, `assertSchemaEqual`, small functions composed with `DataFrame.transform`

02 — Data Ingestion & Acquisition (12 %)

- Auto Loader (`cloudFiles`) for continuous file ingest, incl. `binaryFile`; `COPY INTO` for idempotent batch
- Kafka / Kinesis / Pub/Sub streaming sources; exactly-once into Delta via checkpoints
- Lakeflow Connect managed connectors for SQL Server, MySQL, PostgreSQL CDC
- OpenSharing (formerly Delta Sharing): D2D uses a sharing identifier; Databricks-to-Open uses bearer tokens or OIDC
- Clean Rooms: collaborators run approved notebooks without seeing each other's raw data
- Lakehouse Federation: `CREATE CONNECTION` + `CREATE FOREIGN CATALOG`; credentials live on the connection

03 — Data Manipulation (12 %)

- Window functions (`row_number`, `lag`, running totals), joins, aggregations
- VARIANT: `parse_json`, `col:path::type`, `variant_get`, `try_variant_get`
- `ai_query(endpoint, request)` and task functions (`ai_classify`, `ai_extract`, ...) for enrichment
- Expectations: warn (`expect`), drop (`_or_drop`), fail (`_or_fail` — the flow update fails and rolls back); quarantine = flag + separate table

04 — Monitoring and Alerting (10 %)

- System tables: `system.billing`, `system.compute`, `system.access`, `system.lakeflow`
- Pipeline event log: `event_log(<pipeline-id>)`, `flow_progress` events carry expectation metrics
- Alerts: a scheduled query + condition + notification; the SQL alert task can branch a job with If/else
- REST API / CLI / SDK: `w.jobs.list_runs`, `w.pipelines.list_pipeline_events`, `databricks jobs list-runs`

05 — Cost & Performance Optimization (15 %)

- Managed tables + Predictive Optimization + liquid clustering = less maintenance work
- Disk cache (Delta cache) speeds repeated reads from local SSD; Spark `cache()` is different
- Change Data Feed: `delta.enableChangeDataFeed`, `table_changes`, `readChangeFeed`; `_change_type` values
- Query profile: files / bytes pruned vs read, join strategy, shuffle and spill

06 — Ensuring Data Security and Compliance (8 %)

- Least privilege: grant to groups, on the narrowest securable; workspace-object ACLs (CAN VIEW / CAN RUN / CAN MANAGE)
- Pseudonymization (salted hash, tokens) is reversible with a key; anonymization (suppression, generalization) is not
- Erasure: DELETE → `REORG TABLE ... APPLY (PURGE)` (deletion vectors) → VACUUM after retention

07 — Data Governance (5 %)

- Grants on a catalog or schema are inherited by current and future objects
- Tags and comments on securables drive search and discovery; governed tags also drive ABAC

08 — Debugging and Deploying (10 %)

- Diagnose with the Spark UI, driver / compute logs, system tables and the query profile
- **Repair run** re-runs only failed tasks (and dependents), optionally with new parameter values
- Bundles: `validate` → `deploy -t` → `run`; `mode: development` prefixes names and pauses schedules
- Git folders (formerly Repos): CI updates a production Git folder to a branch or tag

09 — Data Modeling (5 %)

- Partition only large tables, by low-cardinality grain (date); cluster on filter / join keys; compact with OPTIMIZE
- Star schema: facts + dimensions; materialized views for pre-aggregation; **metric views** for governed, reusable metrics (`MEASURE()`)
- SCD Type 1 = overwrite; Type 2 = new row with an effective range (full history)

Common-trap reminders

Trap	Right answer
Stem says ”DLT”, ”Delta Live Tables” or ”APPLY CHANGES”	Read as Lakeflow Pipelines / AUTO CDC
”Asset Bundles” / ”DABs”	Declarative Automation Bundles — same `databr`
”Delta Sharing”	OpenSharing
”Aggregate must include late updates to prior days”	Materialized view
”Replicate PostgreSQL changes with minimal code”	Lakeflow Connect, not JDBC or OpenSharing
”Query Snowflake without ETL”	Lakehouse Federation
”One rule to mask every `pii` column”	ABAC policy on a governed tag
”Same metric computed differently in each dashboard”	Unity Catalog metric view
”Re-run only the failed task with a new date”	Repair run with a parameter override
”Most-managed governance answer”	Unity Catalog

Today's exam — 120-minute time budget

- **60 scored questions ÷ 120 minutes ≈ 2 min/question** — flag long scenarios on first read, return at end
- Multi-paragraph scenarios: read the *question* first, then the scenario — finds the relevant facts faster
- ”Which is the LEAST” / ”EXCEPT” stems: read twice; mark answer to your *original* reading then verify
- Default to the **most-managed** option: Lakeflow Jobs > custom Airflow; Lakeflow Pipelines > hand-written Spark stream; UC > workspace ACLs

Eat. Hydrate. Breathe.

You've put in the hours. Trust the prep. Go pass it.

Official Documentation Links

Organised by the 9 sections of the Oct 9, 2026 exam guide. Each topic file also ends with its own **Official Documentation** list.

Certification Resources

- Data Engineer Professional certification page
- Oct 9, 2026 exam guide (PDF)
- Certification FAQ
- Databricks Academy
- Databricks Free Edition (hands-on practice)

Section 1 — Developing Code for Data Processing using Python and SQL

- Declarative Automation Bundles
- Bundle library dependencies
- Serverless dependencies and environments
- User-defined functions
- Unity Catalog UDFs
- Lakeflow pipelines concepts
- Lakeflow pipelines Python reference (`pyspark.pipelines`)
- AUTO CDC APIs (formerly APPLY CHANGES)
- Structured Streaming watermarks
- Structured Streaming in production
- If/else condition task
- Run serverless jobs (performance mode, auto-optimization)
- Unit testing notebooks

Section 2 — Data Ingestion & Acquisition

- Auto Loader
- Kafka
- Kinesis
- Pub/Sub
- Change data feed
- Lakeflow Connect
- OpenSharing
- Clean Rooms
- Lakehouse Federation

Section 3 — Data Manipulation

- MERGE INTO
- VARIANT
- AI functions
- `ai_query`
- Expectations

Section 4 — Monitoring and Alerting

- System tables
- Jobs system tables
- Pipeline event log
- Databricks SQL alerts
- Databricks SDK for Python
- Jobs CLI commands
- Monitor Lakeflow Jobs

Section 5 — Cost & Performance Optimization

- Unity Catalog managed tables
- Predictive optimization
- Liquid clustering
- Deletion vectors
- Disk cache
- Query profile
- Partitions

Section 6 — Ensuring Data Security and Compliance

- Access control lists
- ABAC
- Governed tags
- Row filters and column masks
- Data Classification
- GDPR compliance with Delta
- VACUUM

Section 7 — Data Governance

- Unity Catalog privileges and inheritance
- Privileges reference
- Tags
- COMMENT ON

Section 8 — Debugging and Deploying

- Debugging with the Spark UI
- Repair job failures
- Job parameters
- Bundle deployment modes
- Git folders
- CI/CD with Git folders

Section 9 — Data Modeling

- Data modeling
- Constraints
- Materialized views
- Unity Catalog metric views

Release Notes

- Platform release notes
- Lakeflow pipelines release notes
- Bundles release notes

Community Resources

- Databricks Community
- Delta Lake GitHub

Cheat Sheets - Data Engineer Professional

Quick reference cards for key exam topics. Product names follow the Oct 9, 2026 exam guide (Lakeflow Pipelines, AUTO CDC, Declarative Automation Bundles, OpenSharing).

Available Cheat Sheets

Cheat Sheet	Description
Delta Lake Quick Reference	Delta Lake commands, optimization, and maintenance
Streaming Quick Reference	Structured Streaming, triggers, and checkpointing
Auto Loader Quick Reference	Cloud Files, schema inference, and file notifications
Lakeflow Pipelines Quick Reference	Streaming tables vs materialized views, `@dp.*` decorators, AUTO CDC (fo
Unity Catalog Quick Reference	Namespaces, grants, and data governance

Shared Cheat Sheets

See also the shared cheat sheets for content applicable across certifications:

- Delta Lake Commands
- Lakeflow Pipelines (shared, cross-cert)
- Spark Configurations
- SQL Functions
- Unity Catalog Quick Reference

Auto Loader Quick Reference

Basic Syntax

```
# Basic Auto Loader read

df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/path/to/schema")
    .load("/path/to/source"))

# Write to Delta table

(df.writeStream
    .option("checkpointLocation", "/path/to/checkpoint")
    .trigger(availableNow=True)
    .toTable("catalog.schema.target_table"))
```

Essential Options

Option	Description	Required
`cloudFiles.format`	Source file format (json, csv, parquet, avro, text)	Yes
`cloudFiles.schemaLocation`	Path to store inferred schema	Yes (for inference)
`cloudFiles.schemaHints`	Override inferred column types	No
`cloudFiles.inferColumnTypes`	Infer types (default: strings for JSON/CSV)	No

```
# With schema hints and type inference

df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaHints", "id INT, timestamp TIMESTAMP")
    .load("/data/source"))
```

Schema Evolution Modes

Mode	Behavior	Use Case
`addNewColumns`	Add new columns, rescue unparseable	Production default
`rescue`	All unexpected data to `_rescued_data`	Debug, preserve data
`failOnNewColumns`	Fail stream on new columns	Strict schema control
`none`	Ignore new columns	Fixed schema

```
# Schema evolution with addNewColumns

df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load("/data/source"))
```

Rescued Data Column

```
# Always include rescued data column

df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("cloudFiles.schemaEvolutionMode", "rescue")
    .load("/data/source"))

# _rescued_data column contains:
# - Unparseable records
# - Unexpected columns
# - Type mismatches
```

Directory Listing vs File Notification

Mode	Option	Description
Directory Listing	`cloudFiles.useNotifications = false`	Polls directory (default)
File Notification	`cloudFiles.useNotifications = true`	Uses cloud events

```
# File notification mode (scalable for large directories)

df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("cloudFiles.useNotifications", "true")
    .load("/data/source"))
```

Aspect	Directory Listing	File Notification
Setup	Automatic	Requires cloud config
Scalability	Slows with many files	Constant time
Latency	Depends on list time	Near real-time
Cost	Free	Cloud notification costs
Best for	< 10K files/day	> 10K files/day

File Format Options

JSON

```
df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("multiLine", "true")
    .load("/data/json/"))
```

CSV

```
df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("header", "true")
    .option("sep", ",")
    .option("inferSchema", "false")
    .load("/data/csv/"))
```

Parquet

```
df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "parquet")
    .load("/data/parquet/"))
# No schema location needed - schema embedded in files
```

Common Options Reference

Option	Default	Description
`cloudFiles.format`	-	Source format (required)
`cloudFiles.schemaLocation`	-	Schema storage path

Option	Default	Description
`cloudFiles.inferColumnTypes`	false	Infer types vs all strings
`cloudFiles.schemaEvolutionMode`	addNewColumns	How to handle changes
`cloudFiles.useNotifications`	false	Use cloud notifications
`cloudFiles.includeExistingFiles`	true	Process existing files
`cloudFiles.maxFilesPerTrigger`	1000	Files per micro-batch
`cloudFiles.maxBytesPerTrigger`	-	Bytes per micro-batch
`pathGlobFilter`	-	Filter files by pattern
`recursiveFileLookup`	false	Search subdirectories

File Filtering

```
# Only process JSON files

df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("pathGlobFilter", "*.json")
    .load("/data/source"))

# Recursive file lookup

df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "parquet")
    .option("recursiveFileLookup", "true")
    .load("/data/source"))
```

Metadata Columns

```
# Access file metadata

df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .load("/data/source")
    .select(
        "*",
        "_metadata.file_path",
        "_metadata.file_name",
        "_metadata.file_size",
        "_metadata.file_modification_time"
    ))
```

Metadata Column	Description
`_metadata.file_path`	Full file path
`_metadata.file_name`	File name only
`_metadata.file_size`	Size in bytes
`_metadata.file_modification_time`	Last modified timestamp

Complete Example

```
# Production Auto Loader pipeline
```

```python
df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/mnt/checkpoints/schema")
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .option("cloudFiles.useNotifications", "true")
    .option("maxFilesPerTrigger", "1000")
    .load("/mnt/landing/events/"))

# Add metadata and write

(df.select(
    "*",
    "_metadata.file_path".alias("source_file"),
    current_timestamp().alias("ingestion_time")
).writeStream
    .format("delta")
    .option("checkpointLocation", "/mnt/checkpoints/events")
    .option("mergeSchema", "true")
    .trigger(availableNow=True)
    .toTable("bronze.events"))
```

Common Exam Tips

1. `cloudFiles.schemaLocation` required for schema inference (JSON, CSV)
2. `addNewColumns` is default schema evolution mode
3. `_rescued_data` column captures unparseable data
4. Use file notification for > 10K files/day
5. `inferColumnTypes=true` for typed inference (otherwise strings)
6. Parquet/Avro don't need schema location (embedded)
7. `_metadata` column provides file-level information
8. `trigger(availableNow=True)` processes all available, then stops
9. Schema stored at schemaLocation persists across restarts
10. `maxFilesPerTrigger` controls throughput/latency tradeoff

Delta Lake Quick Reference

MERGE (Upsert)

```sql
-- Basic upsert
MERGE INTO target AS t
USING source AS s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

-- Conditional update
MERGE INTO target AS t
USING source AS s
ON t.id = s.id
WHEN MATCHED AND s.updated_at > t.updated_at
  THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

-- With delete
MERGE INTO target AS t
```

```sql
USING source AS s
ON t.id = s.id
WHEN MATCHED AND s.is_deleted = true THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

```python
# PySpark MERGE

from delta.tables import DeltaTable

target = DeltaTable.forName(spark, "catalog.schema.target")

target.alias("t").merge(
    source.alias("s"),
    "t.id = s.id"
).whenMatchedUpdateAll(
).whenNotMatchedInsertAll(
).execute()
```

OPTIMIZE

```sql
-- Basic optimize (bin-packing)
OPTIMIZE catalog.schema.table;

-- Optimize with Z-ORDER
OPTIMIZE catalog.schema.table
ZORDER BY (column1, column2);

-- Optimize specific partitions
OPTIMIZE catalog.schema.table
WHERE date = '2024-01-15';
```

Setting	Default	Description
Target file size	1 GB	Optimal file size for queries
Auto optimize	Enabled (managed tables)	Automatic compaction
Auto compact	After write	Triggers when small files accumulate

VACUUM

```sql
-- Vacuum with default retention (7 days)
VACUUM catalog.schema.table;

-- Vacuum with specific retention
VACUUM catalog.schema.table RETAIN 168 HOURS;

-- Dry run (show files to delete)
VACUUM catalog.schema.table DRY RUN;
```

Parameter	Default	Notes
Minimum retention	168 hours (7 days)	Cannot go below unless safety disabled
Safety check	Enabled	`spark.databricks.delta.retentionDurationCheck.enabled`

Warning: VACUUM removes ability to time travel beyond retention period.

Z-ORDER vs Liquid Clustering

Feature	Z-ORDER	Liquid Clustering
Syntax	`OPTIMIZE ... ZORDER BY`	`CLUSTER BY` in table definition
Automatic	No (manual)	Yes (incremental)
Column limit	~4 columns	~4 columns
When to use	Existing tables	New tables (recommended)
Rewrite	Full table	Incremental

```
-- Z-ORDER (existing tables)
OPTIMIZE my_table ZORDER BY (country, city);

-- Liquid Clustering (new tables)
CREATE TABLE my_table (
  id INT,
  country STRING,
  city STRING
) CLUSTER BY (country, city);

-- Change clustering columns
ALTER TABLE my_table CLUSTER BY (region, country);
```

Time Travel

```
-- Query by version
SELECT * FROM my_table VERSION AS OF 5;
SELECT * FROM my_table@v5;

-- Query by timestamp
SELECT * FROM my_table TIMESTAMP AS OF '2024-01-15T10:00:00';

-- View history
DESCRIBE HISTORY my_table;

-- Restore to version
RESTORE TABLE my_table TO VERSION AS OF 5;
RESTORE TABLE my_table TO TIMESTAMP AS OF '2024-01-15';
```

Property	Default	Description
Log retention	30 days	How long transaction log kept
Data retention	7 days	VACUUM default

Table Constraints

```
-- NOT NULL
ALTER TABLE my_table ALTER COLUMN id SET NOT NULL;

-- CHECK constraint
ALTER TABLE my_table ADD CONSTRAINT valid_price CHECK (price > 0);

-- Drop constraint
ALTER TABLE my_table DROP CONSTRAINT valid_price;
```

```sql
-- View constraints
DESCRIBE TABLE EXTENDED my_table;
```

Key Table Properties

```sql
-- Enable Change Data Feed
ALTER TABLE my_table SET TBLPROPERTIES (
  'delta.enableChangeDataFeed' = 'true'
);

-- Enable auto optimize
ALTER TABLE my_table SET TBLPROPERTIES (
  'delta.autoOptimize.optimizeWrite' = 'true',
  'delta.autoOptimize.autoCompact' = 'true'
);

-- Set log retention
ALTER TABLE my_table SET TBLPROPERTIES (
  'delta.logRetentionDuration' = 'interval 45 days'
);

-- View all properties
SHOW TBLPROPERTIES my_table;
```

Property	Default	Description
`delta.enableChangeDataFeed`	false	Enable CDF
`delta.autoOptimize.optimizeWrite`	true (managed)	Auto file sizing
`delta.autoOptimize.autoCompact`	true (managed)	Auto compaction
`delta.logRetentionDuration`	30 days	Transaction log retention
`delta.deletedFileRetentionDuration`	7 days	Minimum VACUUM retention

Schema Evolution

```sql
-- Add column
ALTER TABLE my_table ADD COLUMN new_col STRING;

-- Add column with position
ALTER TABLE my_table ADD COLUMN new_col STRING AFTER existing_col;

-- Rename column
ALTER TABLE my_table RENAME COLUMN old_name TO new_name;

-- Change column type (limited)
ALTER TABLE my_table ALTER COLUMN col_name TYPE BIGINT;
```

```python
# Auto merge schema on write

(df.write.format("delta")
    .option("mergeSchema", "true")
    .mode("append")
    .saveAsTable("my_table"))
```

Cloning

```sql
-- Shallow clone (metadata only, references source files)
CREATE TABLE my_clone SHALLOW CLONE source_table;

-- Deep clone (full data copy)
CREATE TABLE my_clone DEEP CLONE source_table;

-- Clone to specific version
CREATE TABLE my_clone CLONE source_table VERSION AS OF 5;
```

Clone Type	Use Case	Data Copied
Shallow	Testing, experimentation	No (references)
Deep	Production copy, archival	Yes (full copy)

Common Exam Tips

1. VACUUM default is 168 hours (7 days) - cannot reduce without disabling safety
2. Z-ORDER is manual; Liquid Clustering is automatic and incremental
3. Time travel limited by VACUUM retention, not log retention
4. `mergeSchema=true` for automatic schema evolution on writes
5. Change Data Feed must be explicitly enabled per table
6. Shallow clones don't copy data; they reference source files
7. MERGE can handle INSERT, UPDATE, and DELETE in single statement
8. Constraints are validated on write, not retroactively

Lakeflow Pipelines Quick Reference

Platform update (Oct 2026)

The exam says **Lakeflow Pipelines** and **AUTO CDC APIs (formerly APPLY CHANGES)**. The docs call the product **Lakeflow Spark Declarative Pipelines** (renamed from Lakeflow Declarative Pipelines on Nov 12, 2025; formerly DLT) and also use "Lakeflow pipelines". The Python `dlt` module has been replaced by `pyspark.pipelines` — `import dlt` still works, but `from pyspark import pipelines as dp` is recommended. Sources: November 2025 release notes, Python language reference

Full study notes: Declarative Pipelines · AUTO CDC API · Expectations

Dataset Types

Type	SQL	Python
Streaming table	`CREATE OR REFRESH STREAMING TABLE`	`@dp.table` returning a streaming DataFrame
Materialized view	`CREATE OR REFRESH MATERIALIZED VIEW`	`@dp.materialized_view` returning a batch Da
Temporary view	`CREATE TEMPORARY VIEW`	`@dp.temporary_view`

```sql
-- Streaming table: Auto Loader via read_files with STREAM
CREATE OR REFRESH STREAMING TABLE bronze_orders
AS SELECT * FROM STREAM read_files('/Volumes/main/landing/orders/', format =>
 ↪  'json');

-- Materialized view: refreshed incrementally on serverless when the query allows
CREATE OR REFRESH MATERIALIZED VIEW gold_daily_totals
AS SELECT order_date, SUM(amount) AS total_amount
```

```sql
FROM silver_orders
GROUP BY order_date;
```

```python
from pyspark import pipelines as dp

@dp.table
def bronze_orders():
    return (spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .load("/Volumes/main/landing/orders/"))

@dp.materialized_view
def gold_daily_totals():
    return (spark.read.table("silver_orders")
        .groupBy("order_date")
        .sum("amount"))
```

Streaming table or materialized view?

Scenario	Choose
Ingest files or Kafka events exactly once	Streaming table
Aggregate must reflect **late-arriving updates to prior days** or upstream deletes	Materialized view
Join where dimension changes must update old results	Materialized view
Sub-minute latency in a continuous pipeline	Streaming table
Apply inserts / updates / deletes from a CDC feed	Streaming table + AUTO

AUTO CDC (formerly APPLY CHANGES)

Older material	Current
`APPLY CHANGES INTO LIVE.target FROM STREAM(LIVE.src) ...`	`CREATE FLOW f AS AUTO CDC INTO tar`
`dlt.apply_changes(...)`	`dp.create_auto_cdc_flow(...)` (same
`dlt.apply_changes_from_snapshot(...)`	`dp.create_auto_cdc_from_snapshot_f`
`dlt.create_streaming_table(...)`	`dp.create_streaming_table(...)`

```sql
CREATE OR REFRESH STREAMING TABLE customers_current;

CREATE FLOW customers_current_cdc AS AUTO CDC INTO customers_current
FROM STREAM(main.cdc.customers_feed)
KEYS (customer_id)
APPLY AS DELETE WHEN operation = 'DELETE'
SEQUENCE BY event_ts
COLUMNS * EXCEPT (operation, event_ts)
STORED AS SCD TYPE 1;
```

```python
from pyspark import pipelines as dp

dp.create_streaming_table("customers_history")

dp.create_auto_cdc_flow(
    target="customers_history",
    source="customers_cdc",
    keys=["customer_id"],
    sequence_by="event_ts",
```

```
    apply_as_deletes="operation = 'DELETE'",
    except_column_list=["operation", "event_ts"],
    stored_as_scd_type=2,
    track_history_column_list=["address", "email"],
)
```

Argument / clause	Meaning
keys / KEYS	Columns that identify a row (required)
sequence_by / SEQUENCE BY	Logical order of events; resolves out-of-order arrivals (required)
stored_as_scd_type / STORED AS	1 (default, latest value) or 2 (history with __START_AT / __END_AT
apply_as_deletes	Condition that turns an event into a delete
apply_as_truncates	Condition that truncates the target — **SCD Type 1 only**
track_history_column_list	SCD Type 2: only changes to these columns open a new history r
except_column_list / COLUMNS * EXCEPT	Drop operational columns from the target

Current rows in an SCD Type 2 target have `__END_AT IS NULL`. `__START_AT` / `__END_AT` hold the `SEQUENCE BY` value, with the same data type.

Expectations

Action	SQL	Python	Invalid rows
Warn (default)	`CONSTRAINT c EXPECT (cond)`	`@dp.expect("c", "cond")`	Kept, counted
Drop	`... ON VIOLATION DROP ROW`	`@dp.expect_or_drop("c", "cond")`	Dropped, counted
Fail	`... ON VIOLATION FAIL UPDATE`	`@dp.expect_or_fail("c", "cond")`	—

Several rules at once: `@dp.expect_all({...})`, `@dp.expect_all_or_drop({...})`, `@dp.expect_all_or_fail`
Quarantine = keep bad rows with a warn expectation, flag them, and route them to a separate table for review.

```
CREATE OR REFRESH STREAMING TABLE silver_orders (
  CONSTRAINT valid_id EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT positive_amount EXPECT (amount > 0) ON VIOLATION FAIL UPDATE
)
AS SELECT * FROM STREAM(bronze_orders);
```

Event Log

```
-- Expectation pass / fail counts per dataset
SELECT
  row_expectations.dataset AS dataset,
  row_expectations.name AS expectation,
  SUM(row_expectations.passed_records) AS passing_records,
  SUM(row_expectations.failed_records) AS failing_records
FROM (
  SELECT explode(
    from_json(
      details:flow_progress:data_quality:expectations,
      "array<struct<name: string, dataset: string, passed_records: int,
       ↪  failed_records: int>>"
    )
  ) AS row_expectations
  FROM event_log(<pipeline-id>)
```

```
  WHERE event_type = 'flow_progress'
)
GROUP BY row_expectations.dataset, row_expectations.name;
```

More queries: Lakeflow Event Logs.

Common Exam Tips

1. Streaming table = each appended row processed once; materialized view = always equals the query, incremental on serverless
2. Late updates to already-aggregated days → materialized view
3. EXPECT warns, `DROP ROW` removes, `FAIL UPDATE` stops and rolls back that flow's update
4. AUTO CDC (formerly APPLY CHANGES): KEYS + `SEQUENCE BY` are required; `STORED AS` defaults to SCD Type 1
5. SCD Type 2 current rows: `__END_AT IS NULL`; `apply_as_truncates` is SCD Type 1 only
6. Reference tables by name (`spark.read.table("silver_orders")`); the `LIVE.` prefix is legacy
7. Expectation metrics live in `flow_progress` events of the pipeline event log
8. Triggered mode processes available data and stops; continuous mode keeps running for low latency
9. Python dataset functions return DataFrames — never call `writeStream`, `save()` or `collect()` inside them

---

Structured Streaming Quick Reference

Basic Streaming Read

```python
# Read from Delta

df = spark.readStream.format("delta").table("source_table")

# Read from Auto Loader

df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema/path")
    .load("/data/path"))

# Read from Kafka

df = (spark.readStream.format("kafka")
    .option("kafka.bootstrap.servers", "host:port")
    .option("subscribe", "topic")
    .load())
```

Basic Streaming Write

```python
# Write to Delta

(df.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "/checkpoint/path")
    .toTable("target_table"))
```

```python
# Write to Delta with trigger

(df.writeStream
    .format("delta")
    .outputMode("append")
    .trigger(availableNow=True)
    .option("checkpointLocation", "/checkpoint/path")
    .toTable("target_table"))
```

Trigger Types

Trigger	Syntax	Behavior
Default (micro-batch)	None	Process as fast as possible
Processing time	`trigger(processingTime='10 seconds')`	Fixed interval
Once	`trigger(once=True)`	Single batch, then stop
Available now	`trigger(availableNow=True)`	All available data, then stop
Continuous	`trigger(continuous='1 second')`	Low-latency (experimental)

```python
# Processing time trigger

.trigger(processingTime='30 seconds')

# Once trigger (deprecated, use availableNow)

.trigger(once=True)

# Available now (preferred for batch-like streaming)

.trigger(availableNow=True)
```

Output Modes

Mode	Description	Use Case
append	Only new rows	Immutable data
complete	Entire result table	Aggregations
update	Only changed rows	Stateful operations

Watermarking

```python
# Define watermark for late data

(df.withWatermark("event_time", "10 minutes")
    .groupBy(window("event_time", "5 minutes"))
    .count())
```

Watermark	Meaning
"10 minutes"	Accept data up to 10 min late
"1 hour"	Accept data up to 1 hour late
"0 seconds"	No late data allowed

Window Functions

```
# Tumbling window (non-overlapping)

df.groupBy(window("timestamp", "10 minutes"))

# Sliding window (overlapping)

df.groupBy(window("timestamp", "10 minutes", "5 minutes"))

# Session window

df.groupBy(session_window("timestamp", "10 minutes"))
```

Auto Loader Options

Option	Description
cloudFiles.format	Source file format (json, csv, parquet)
cloudFiles.schemaLocation	Schema inference storage
cloudFiles.inferColumnTypes	Infer column types
cloudFiles.schemaEvolutionMode	addNewColumns, rescue, failOnNewColumns
cloudFiles.useNotifications	Use file notifications (vs directory listing)

```
(spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/schema")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .option("cloudFiles.inferColumnTypes", "true")
    .load("/data/path"))
```

Stream-Stream Joins

```
# Inner join with watermarks

ordersWithWatermark = orders.withWatermark("orderTime", "10 minutes")
paymentsWithWatermark = payments.withWatermark("paymentTime", "10 minutes")

ordersWithWatermark.join(
    paymentsWithWatermark,
    expr("""
        orderId = paymentOrderId AND
        paymentTime >= orderTime AND
        paymentTime <= orderTime + interval 1 hour
    """),
    "inner"
)
```

Checkpoint Management

```
# Checkpoint location (required for exactly-once)

.option("checkpointLocation", "/mnt/checkpoint/stream_name")
```

Important: Never change checkpoint location for a running stream. To restart fresh, delete the checkpoint.

foreachBatch

```python
def process_batch(batch_df, batch_id):
    # Custom processing per micro-batch
    batch_df.write.format("delta").mode("append").save("/path")

(df.writeStream
    .foreachBatch(process_batch)
    .option("checkpointLocation", "/checkpoint")
    .start())
```

Monitoring

```python
# Get current stream status

query = df.writeStream...start()
query.status
query.lastProgress
query.recentProgress

# Stop stream

query.stop()

# Await termination

query.awaitTermination()
```

Key Concepts

Concept	Description
Micro-batch	Default processing model, batches of data
Exactly-once	Guaranteed with checkpointing
Idempotent	Same input produces same output
State store	Persists aggregation state

Unity Catalog — DE Professional Supplement

> For the full Unity Catalog reference (object creation, permissions, external locations, OpenSharing), see shared/cheat-sheets/unity-catalog-quick-ref.md.

This file covers DE Professional-specific patterns not in the shared reference.

Permission Inheritance

```
CATALOG (USE CATALOG)
    │
    └── SCHEMA (USE SCHEMA)  ← Requires USE CATALOG on parent
            │
            └── TABLE (SELECT)  ← Requires USE SCHEMA on parent
```

Rule: Accessing a table always requires `USE CATALOG` + `USE SCHEMA` + `SELECT`.

Bulk Grant on Schema

```sql
-- Grant SELECT on all tables in a schema at once
GRANT SELECT ON ALL TABLES IN SCHEMA my_catalog.my_schema TO `analysts`;

-- Grant to groups (best practice — not individuals)
GRANT SELECT ON TABLE my_table TO `data_analysts_group`;
```

System Tables for Audit & Governance

```sql
-- Audit log: who accessed what and when
SELECT * FROM system.access.audit
WHERE action_name = 'getTable'
  AND request_params.full_name_arg = 'my_catalog.my_schema.my_table';

-- Catalog metadata
SELECT * FROM system.information_schema.catalogs;

-- Table metadata
SELECT * FROM system.information_schema.tables
WHERE table_catalog = 'my_catalog';

-- Column metadata
SELECT * FROM system.information_schema.columns
WHERE table_catalog = 'my_catalog'
  AND table_schema = 'my_schema';
```

System Table	Description
system.access.audit	Audit logs for all UC actions
system.information_schema.catalogs	Catalog metadata
system.information_schema.tables	Table metadata
system.information_schema.columns	Column metadata

Volume Access Path

```python
# Volumes are accessed via /Volumes/<catalog>/<schema>/<volume>/<path>

df = spark.read.csv("/Volumes/my_catalog/my_schema/my_volume/data.csv")
dbutils.fs.ls("/Volumes/my_catalog/my_schema/my_volume/")
```

Security Functions

Function	Description
current_user()	Current user's email
is_account_group_member('group')	Check account-level group membership
is_member('group')	Check workspace-level group membership
current_user_attribute('attr')	Get a custom user attribute value

---

DE Professional Exam Tips

1. Three-level namespace: `catalog.schema.table`
2. `USE CATALOG` + `USE SCHEMA` both required before SELECT resolves
3. Metastore is **account-level**, not workspace-level
4. Grant to groups, not individuals — best practice for exam questions
5. Dynamic views enable row/column security without data duplication
6. `current_user()` and `is_account_group_member()` are the key security filter functions
7. Volume path format: `/Volumes/catalog/schema/volume/path`
8. Owners can grant any privilege on their objects (no explicit grant needed for owner)
9. External locations require a storage credential to exist first
10. `system.access.audit` is the primary table for UC audit logging questions

Practice Questions

Scenario-based practice questions organized by the 9 sections of the Oct 9, 2026 exam guide. 82 of the 155 questions are marked *(2026 blueprint)* — they target skills that are new or reworded in that guide (AUTO CDC, VARIANT, `ai_query`, ABAC, OpenSharing, Clean Rooms, Lakeflow Connect, metric views, `CLUSTER BY AUTO`, alerts, and more).

> **Note**
>
> The question files were renumbered when they were regrouped into the 9 sections, so progress saved in the practice quiz under the old 8-file layout does not carry over.

Sections

Section	Questions	New for 2026	Exam Weight	Li
1: Developing Code for Data Processing using Python and SQL	39	18	23 %	St
2: Data Ingestion & Acquisition	14	12	12 %	St
3: Data Manipulation	9	6	12 %	St
4: Monitoring and Alerting	10	8	10 %	St
5: Cost & Performance Optimization	29	12	15 %	St
6: Ensuring Data Security and Compliance	17	10	8 %	St
7: Data Governance	9	4	5 %	St
8: Debugging and Deploying	20	8	10 %	St
9: Data Modeling	8	4	5 %	St
Total	**155**	**82**	**100 %**	

Scoring Guide

Score	Assessment
≥ 80 % correct	Excellent — ready for the exam
70–79 %	Good — review weak sections
60–69 %	Fair — more study needed
< 60 %	Work through the topic folders again

Databricks does not publish a numeric passing score for the Oct 9, 2026 exam guide; the bands above are study guidance only.

Additional Practice Resources

- Databricks Academy — official training
- Oct 9, 2026 exam guide (PDF) — 10 official sample questions
- Udemy practice exams — third-party; check it targets the Oct 9, 2026 guide
- Mock Exam 1 and Mock Exam 2

Section 1: Developing Code for Data Processing using Python and SQL (23 %)

Practice questions for Section 1 of the Oct 9, 2026 exam guide. Questions marked *(2026 blueprint)* target skills that are new or reworded in that guide. Study material: Section 1 folder.

Question 1: Streaming Triggers **Question** *(Medium)*: A streaming job processes data from a message queue. The business requires the job to process all available data once per hour and then stop. Which trigger configuration should be used?

A) `trigger(processingTime='1 hour')`
B) `trigger(once=True)`
C) `trigger(availableNow=True)`
D) `trigger(continuous='1 hour')`

Answer

Correct Answer: C
`availableNow=True` processes all available data and stops, making it ideal for scheduled batch-like streaming. `processingTime` runs continuously. `once=True` is deprecated and only processes one batch. `continuous` is for low-latency and doesn't stop.

Question 2: Watermarking in Streaming **Question** *(Medium)*: A streaming aggregation job groups events by a 10-minute window. Events can arrive up to 30 minutes late. Which watermark configuration is correct?

A) `withWatermark("event_time", "10 minutes")`
B) `withWatermark("processing_time", "30 minutes")`
C) `withWatermark("event_time", "40 minutes")`
D) `withWatermark("event_time", "30 minutes")`

Answer

Correct Answer: D
The watermark should match the maximum expected lateness (30 minutes). This tells Spark to wait 30 minutes before finalizing windows. Using event_time (not processing_time) is required for event-time processing.

Question 3: Incremental Processing with readStream **Question** *(Medium)*: A pipeline reads from a Delta table that receives both inserts and updates. Only new and updated records should be processed. Which configuration enables this?

A) `spark.readStream.option("ignoreChanges", "true").table("source")`
B) `spark.readStream.option("readChangeData", "true").table("source")`
C) `spark.readStream.option("readChangeFeed", "true").table("source")`
D) `spark.readStream.option("skipChangeCommits", "true").table("source")`

Answer

Correct Answer: C
`readChangeFeed = true` (with CDF enabled on the table) reads the change data feed which includes inserts, updates, and deletes. Option A ignores updates/deletes. Option B uses the wrong option name (`readChangeData` is not valid; the correct name is `readChangeFeed`). Option D is not a valid option.

Question 4: Stream-Stream Join Question *(Medium)*: A data engineer needs to join two streaming sources - clickstream events and ad impressions - to calculate ad conversion rates. Both streams have event timestamps. What is required when performing a stream-stream join?

A) Both streams must use the same checkpoint location
B) Both streams must have identical schemas
C) A time-range condition (watermark) must be defined on both streams
D) One stream must be converted to a static DataFrame first

Answer

Correct Answer: C
Stream-stream joins require watermarks on both streams to define a time-range condition. This allows Spark to know when old state can be discarded. Checkpoint locations are per-stream. Schemas don't need to match. Both sides remain streaming.

Question 5: Stream-Static Join Question *(Medium)*: A streaming pipeline enriches real-time transaction data with customer details from a slowly-changing dimension table. What happens to the static DataFrame in a stream-static join when it's updated?

A) The static side is re-read for each micro-batch, picking up changes
B) The static side is cached at query start and never refreshed
C) The query fails if the static table changes during execution
D) Updates to the static table require restarting the streaming query

Answer

Correct Answer: A
In a stream-static join, the static DataFrame is re-read for each micro-batch. This means updates to the dimension table are automatically picked up without restarting the query. This makes it ideal for slowly-changing dimensions.

Question 6: Stateful Streaming Operations Question *(Hard)*: A real-time fraud detection system needs to maintain custom state per user session, tracking cumulative transaction amounts and flagging when thresholds are exceeded within a rolling time window. Which Spark streaming operation provides the most flexibility for this use case?

A) Window aggregation with `groupBy(window(...)).sum()`
B) `dropDuplicatesWithinWatermark()` with session tracking
C) `foreachBatch()` with manual state management
D) `flatMapGroupsWithState()` with GroupStateTimeout

Answer

Correct Answer: D
`flatMapGroupsWithState()` provides arbitrary stateful processing with full control over state shape, timeout behavior, and output. Window aggregations are limited to predefined aggregation functions. `foreachBatch()` doesn't maintain state across batches. `dropDuplicatesWithinWatermark()` is for dedup only.

Question 7: Advanced Watermarking Question *(Medium)*: A streaming aggregation uses a 10-minute watermark. An event arrives with a timestamp 15 minutes behind the current watermark. How does Spark handle this late event?

A) The event is added to the appropriate window and triggers a recomputation
B) The event is placed in a special "late data" output stream
C) The event is silently dropped and not included in any aggregation
D) The query throws a StreamingQueryException for the late data

Answer

Correct Answer: C
Events arriving after the watermark threshold has passed are silently dropped. Spark does not re-open completed windows. There is no "late data" output stream or exception. The watermark defines the boundary for acceptable lateness.

Question 8: Streaming Deduplication Question *(Hard)*: An IoT pipeline receives duplicate sensor readings due to at-least-once delivery. Each reading has a `device_id`, `reading_id`, and `event_time`. The pipeline uses a 1-hour watermark. Which approach correctly deduplicates the stream while managing state?

A) `dropDuplicatesWithinWatermark("device_id", "reading_id")` with the existing watermark
B) `dropDuplicates("device_id", "reading_id")` without any watermark
C) `MERGE INTO target USING stream ON target.reading_id = stream.reading_id`
D) `groupBy("device_id", "reading_id").count().filter("count = 1")`

Answer

Correct Answer: A
`dropDuplicatesWithinWatermark()` uses the existing watermark to bound state size - it only tracks duplicates within the watermark window. `dropDuplicates()` without watermark keeps all keys in state forever, causing unbounded state growth. MERGE is for batch. GroupBy-count doesn't actually deduplicate the output.

Question 9: State Store Configuration Question *(Hard)*: A stateful streaming query tracking millions of user sessions is running out of executor memory due to the state store size. Which state store backend should be used for large state volumes?

A) HDFS state store with increased `spark.executor.memory`
B) Redis-backed state store for distributed state management
C) Delta Lake state store for ACID guarantees on state
D) RocksDB state store backend with `spark.sql.streaming.stateStore.providerClass`

Answer

Correct Answer: D
RocksDB state store uses local disk (SSD) instead of JVM heap memory, allowing much larger state without OOM errors. Configure via `spark.sql.streaming.stateStore.providerClass=com.databricks.sql.streaming.state.RocksDBSta`
The default HDFS backend stores state in JVM memory. Delta Lake and Redis state stores are not available in Spark.

Question 10: dbutils.widgets **Question** *(Easy)*: A notebook needs to accept a date parameter that defaults to yesterday's date when not provided. Which code correctly creates this widget?

A) `dbutils.widgets.text("date", str(date.today() - timedelta(days=1)))`
B) `dbutils.widgets.dropdown("date", str(date.today() - timedelta(days=1)), [])`
C) `dbutils.widgets.create("date", default=str(date.today() - timedelta(days=1)))`
D) `dbutils.widgets.parameter("date", str(date.today() - timedelta(days=1)))`

Answer

Correct Answer: A
`dbutils.widgets.text(name, defaultValue)` creates a text input widget with a default value. Option B requires a list of choices. Options C and D are not valid methods.

Question 11: %run Magic Command **Question** *(Easy)*: A main notebook needs to call a utility notebook and use variables defined in it. Which statement is true about `%run`?

A) Variables from the called notebook are not accessible in the calling notebook
B) `%run` executes the notebook asynchronously
C) Variables from the called notebook are available in the calling notebook's scope
D) `%run` can only be used with notebooks in the same folder

Answer

Correct Answer: C
`%run` executes the notebook in the current notebook's context, making all variables and functions available. It runs synchronously and can reference notebooks using relative or absolute paths.

Question 12: Jobs API Run Now **Question** *(Easy)*: A data engineer needs to trigger a job via API with custom parameters. Which API endpoint and method should be used?

A) `POST /api/2.1/jobs/run-now`
B) `GET /api/2.1/jobs/trigger`
C) `POST /api/2.0/jobs/start`
D) `PUT /api/2.1/jobs/execute`

Answer

Correct Answer: A
`POST /api/2.1/jobs/run-now` triggers a job run with optional parameter overrides. This is the current Jobs API version. The other endpoints don't exist.

Question 13: Cluster Types **Question** *(Medium)*: A production ETL job runs daily for 2 hours. Cost optimization is a priority. Which cluster configuration is most cost-effective?

A) Job cluster created for each run
B) All-purpose cluster running 24/7
C) All-purpose cluster with auto-termination
D) Serverless cluster

Answer

Correct Answer: A
Job clusters are created for the job run and terminated after completion. They have lower DBU rates (~60% less than all-purpose) and only incur costs during execution. All-purpose clusters cost more and may have idle time even with auto-termination.

Question 14: Unit Testing with Nutter **Question** *(Medium)*: A team needs to run tests inside Databricks notebooks as part of CI/CD. Which Nutter pattern is correct?

A) `def test_my_function():` followed by assertions
B) `TestCase` class inheritance
C) `@test` decorator on test methods
D) `before_`, `run_`, and `assertion_` method prefixes for each test

Answer

Correct Answer: D
Nutter uses a specific naming convention: `before_<test>` for setup, `run_<test>` for execution, and `assertion_<test>` for verification. This pattern allows tests to run inside Databricks notebooks.

Question 15: Integration Testing Lakeflow Pipelines **Question** *(Hard)*: A team needs to test a Lakeflow pipeline end-to-end as part of CI/CD, validating that expectations pass and output data is correct. What is the recommended approach for integration testing Lakeflow pipelines?

A) Deploy the pipeline to a test target with a test catalog/schema, trigger a full refresh, then validate the output tables and event log
B) Mock the Lakeflow Pipelines framework locally using pytest and validate transformation logic without deploying
C) Run the pipeline in `development` mode which automatically validates all expectations and reports results
D) Use the Lakeflow Pipelines testing API `dlt.test_pipeline()` to run the pipeline in a sandbox environment

Answer

Correct Answer: A
Lakeflow pipelines must run on Databricks infrastructure. Deploy to a test target (using a test catalog/schema for isolation), trigger a full refresh via the API, then query the output tables and event log for expectation results. Local mocking doesn't test Lakeflow Pipelines-specific behavior. Development mode runs the pipeline but doesn't automatically validate outputs. There is no `dlt.test_pipeline()` API.

Question 16: Testing Strategies - Unit vs Integration **Question** *(Medium)*: A data engineer writes a Python function that transforms a DataFrame (adding calculated columns, filtering invalid rows). The function is used inside a Lakeflow pipeline. What is the correct testing strategy for this function?

A) Only test within the Lakeflow pipeline since the function depends on the Lakeflow Pipelines runtime
B) Write integration tests that deploy the full pipeline and verify end-to-end output
C) Use `unittest.mock` to mock the entire SparkSession and DataFrame API
D) Extract the function to a separate module, unit test it with a local SparkSession and sample data, then integration test the full pipeline

Answer

Correct Answer: D
Pure transformation functions should be extracted to testable modules and unit tested locally with a SparkSession (no Lakeflow Pipelines dependency needed). This gives fast feedback. Integration tests run the full pipeline on Databricks for end-to-end validation. Mocking the entire DataFrame API is brittle and doesn't test actual Spark behavior. Testing only in Lakeflow Pipelines is slow and expensive.

Question 17: Streaming Table vs Materialized View **Question** *(Medium)*: A Lakeflow pipeline needs a table that aggregates daily sales totals from a streaming source. Which table type is most appropriate?

A) Streaming table
B) Temporary view
C) Live table
D) Materialized view

Answer

Correct Answer: D
Materialized views are appropriate for aggregations as they fully recompute results. Streaming tables are for append-only incremental processing and don't support aggregations well. ”Live table” is deprecated terminology.

Question 18: AUTO CDC SCD Type **Question** *(Medium)*: A CDC pipeline receives insert, update, and delete events. The target table should reflect the current state only. Which AUTO CDC (formerly APPLY CHANGES) configuration is correct?

A) `STORED AS SCD TYPE 1`
B) `STORED AS SCD TYPE 2`
C) `STORED AS SNAPSHOT`
D) `STORED AS CURRENT`

Answer

Correct Answer: A
SCD Type 1 maintains only the current state by overwriting on updates. SCD Type 2 maintains full history with start/end dates. Options C and D are not valid AUTO CDC CHANGES syntax.

Question 19: Lakeflow Pipelines Table Reference **Question** *(Easy)*: A Lakeflow Pipelines SQL query needs to read from another table defined in the same pipeline. How should the source table be referenced?

A) `SELECT * FROM source_table`
B) `SELECT * FROM delta.source_table`
C) `SELECT * FROM LIVE.source_table`
D) `SELECT * FROM dlt.source_table`

Answer

Correct Answer: C
In Lakeflow Pipelines SQL, tables within the pipeline are referenced using the `LIVE.` prefix. This tells Lakeflow Pipelines that the table is defined within the same pipeline and creates the proper dependency.

Question 20: Pipeline Modes **Question** *(Easy)*: A Lakeflow pipeline processes data continuously as it arrives with minimal latency. Which pipeline configuration enables this?

A) `triggered: true`
B) `streaming: true`
C) `development: false`
D) `continuous: true`

Answer

Correct Answer: D
`continuous: true` keeps the pipeline running continuously, processing data as it arrives. Triggered mode (default) processes data in batches when manually started or scheduled. `development` affects cluster size, not processing mode.

Question 21: Full Refresh **Question** *(Easy)*: A schema change requires reprocessing all historical data in a Lakeflow pipeline. How should the pipeline be refreshed?

A) Use `databricks pipelines start --full-refresh`
B) Delete the target tables and restart
C) Set `reset: true` in pipeline configuration
D) Drop and recreate the pipeline

Answer

Correct Answer: A
`--full-refresh` flag clears checkpoints and reprocesses all data from the beginning. This preserves pipeline configuration while resetting state. Manually deleting tables or recreating pipelines is error-prone.

Question 22: Bundle Resource Not Deployed *(2026 blueprint)* **Question** *(Easy)*: A team keeps its Declarative Automation Bundle modular: `databricks.yml` at the root, one YAML file per job under `resources/`, and shared code in `src/my_project/`. A developer adds `resources/ingest.job.yml`, runs `databricks bundle deploy -t dev`, and the deployment succeeds, but the new job does not appear in the workspace. What is the most likely fix?

A) Rename the file to `databricks.yml` so the CLI reads it
B) Run `databricks bundle init default-python` again to regenerate the project
C) Move the job definition into `src/my_project/__init__.py`
D) Add the file, or a glob such as `resources/*.yml`, to the top-level `include` mapping in `databricks.yml`

Answer

Correct Answer: D
The resource file must be listed in the top-level `include` mapping.
A bundle has exactly one `databricks.yml`, and every other configuration file must be listed in `include` (globs relative to the declaring file). A is wrong because a bundle can have only one `databricks.yml`. C is wrong because `src/` holds Python source, not bundle configuration. B regenerates a template and doesn't fix the missing include.

Question 23: Building a Wheel as a Bundle Artifact *(2026 blueprint)* **Question** *(Medium)*: A bundle's `src/my_project/` package must be built into a wheel on every deployment and run by a `python_wheel_task` on serverless compute. Which configuration is correct?

A) An `artifacts` entry with `type: whl` and `build: uv build --wheel`, and a job `environments` spec whose `dependencies` list the built wheel, referenced from the task by `environment_key`
B) An init script that runs `pip install` on the serverless compute at startup
C) A compute-scoped library attached to the serverless compute through the Libraries tab
D) A task-level `libraries` entry only, with no environment, because serverless Python wheel tasks don't need one

Answer

Correct Answer: A
Build the wheel with an `artifacts` entry and install it through a job environment referenced by `environment_key`.
`artifacts` with `type: whl` builds the package locally before upload, and on serverless, Python wheel and Python script tasks require an environment via `environment_key`. B and C are wrong because serverless doesn't support init scripts or compute-scoped libraries. D is wrong because an environment is required for Python wheel tasks on serverless.

Question 24: Stale Custom Package on Serverless *(2026 blueprint)* **Question** *(Medium)*: A data engineer fixes a bug in an internal Python package, rebuilds the wheel with the same version number `1.4.0`, and redeploys the bundle. The serverless job still behaves as if the old code is installed. What should the engineer do?

A) Add `dbutils.library.restartPython()` at the end of the job's entry point
B) Switch the job to standard performance mode
C) Increase the package version (or set `dynamic_version: true` on the bundle artifact) so the cached environment is not reused
D) Install the package from the DBFS root instead of a Unity Catalog volume

Answer

Correct Answer: C
Change the package version, manually or with `dynamic_version: true`.
Serverless caches job environments, and Databricks documents that when you change a custom package's implementation you must also update its version number. `dynamic_version: true` patches the wheel version on each deploy. A restarts Python but doesn't change what is installed. B affects startup latency and cost, not dependencies. D is wrong because DBFS-root library storage is deprecated and wouldn't bypass the cache.

Question 25: Private Package Fails Only on Serverless *(2026 blueprint)* **Question** *(Hard)*: A package hosted in an internal Artifactory repository installs correctly on classic clusters, where an init script sets proxy environment variables. When the same notebook runs on serverless compute, installing the package from the Environment side pane fails. Which fix follows Databricks guidance?

A) Set `HTTP_PROXY` with `os.environ` in the first notebook cell before the dependency installs
B) Install the package as a Maven coordinate
C) Attach the init script to the serverless compute through a compute policy
D) Specify `--index-url` or `--extra-index-url` in the dependency specification, or have an admin configure the repository as a default package repository

Answer

Correct Answer: D
Point the dependency at the private index, or configure a workspace default package repository. On serverless, dependencies are resolved before any code runs and compute-scoped environment variables aren't supported, so runtime proxy settings have no effect. A fails for that reason. C fails because serverless doesn't support init scripts or compute policies. B is wrong because the package is a Python package and Maven coordinates aren't supported on serverless.

Question 26: Governed Python Function for SQL Warehouses *(2026 blueprint)* **Question** *(Medium)*: A team wrote a session-scoped `@udf` that masks email addresses in a notebook. Analysts in several workspaces attached to the same metastore must call it from dashboards on a serverless SQL warehouse, and access must be controlled per group. What should the team do?

A) Register the function with `spark.udf.register` in a shared notebook
B) Convert it to a pandas UDF so it runs faster
C) Create it with `CREATE FUNCTION catalog.schema.mask_email(...) RETURNS STRING LANGUAGE PYTHON AS $$ ... $$` and grant EXECUTE to the analyst group
D) Package it in a wheel and install it as a compute-scoped library on the SQL warehouse

Answer

Correct Answer: C
Register it as a Unity Catalog Python UDF and grant EXECUTE.
Unity Catalog functions are persisted, governed with EXECUTE, callable from pro and serverless SQL warehouses, and reachable from all workspaces attached to the metastore. A and B create session-scoped functions that exist only in the defining SparkSession. D is wrong because SQL warehouses don't take compute-scoped libraries.

Question 27: Vectorized Governed UDF *(2026 blueprint)* **Question** *(Hard)*: A governed Unity Catalog Python UDF computes a score row by row and is too slow on large tables. The function must stay in Unity Catalog and return exactly one value per input row, and an expensive lookup table should be loaded only once. Which definition fits?

A) `CREATE FUNCTION ... RETURNS TABLE (...) LANGUAGE PYTHON HANDLER 'Scorer'` with an `eval` method
B) A session-scoped `groupBy().applyInPandas()` call in each notebook
C) `CREATE FUNCTION ... RETURNS DOUBLE LANGUAGE PYTHON PARAMETER STYLE PANDAS HANDLER 'handler'`, with the lookup loaded outside the handler that iterates over `pandas.Series` batches
D) `CREATE TEMPORARY FUNCTION ... LANGUAGE SQL` that calls the Python UDF

Answer

Correct Answer: C
Use a Batch Unity Catalog Python UDF with `PARAMETER STYLE PANDAS` and a `HANDLER`.
Batch Unity Catalog Python UDFs process iterators of `pandas.Series`, must yield the same number of rows as the input, and code outside the handler runs once. A defines a UDTF, which returns tables rather than one value per row. B is session-scoped, not governed, and loads whole groups into memory. D creates a temporary, ungoverned wrapper that still calls the slow row-at-a-time UDF.

Question 28: Cheapest Serverless Nightly Job *(2026 blueprint)* **Question** *(Easy)*: A nightly serverless job has a six-hour SLA and the team wants to minimize DBU consumption. Startup delays of several minutes are acceptable. Which setting should they change?

A) Enable high memory on the job's notebook task
B) Attach an instance pool to the serverless job
C) Uncheck **Enable serverless auto-optimization** on every task
D) Turn off **Performance optimized** so the job uses standard performance mode (`performance_target: STANDARD`)

Answer

Correct Answer: D
Use standard performance mode.
Standard mode uses less compute and consumes fewer DBUs on the same SKU, with a typical startup of 4 to 6 minutes, which suits batch jobs that tolerate latency. A increases the DBU emission rate. C changes retry behaviour, not cost profile. B is wrong because serverless doesn't support instance pools.

Question 29: Non-Idempotent Serverless Task *(2026 blueprint)* **Question** *(Medium)*: A serverless job task posts invoices to an external billing API and is not idempotent. After a transient failure, invoices were sent twice even though the task's `max_retries` is 0. What should the engineer configure?

A) In the task's retry policy, uncheck **Enable serverless auto-optimization (may include additional retries)** (`disable_auto_optimization: true`)
B) Set `max_retries` to `-1`
C) Switch the task to high memory
D) Pin the serverless environment version

Answer

Correct Answer: A
Disable serverless auto-optimization for the task.
Serverless auto-optimization is on by default and retries failed tasks; Databricks recommends turning it off for workloads that must run at most once, such as non-idempotent jobs. B retries indefinitely, the opposite of the goal. C and D don't affect retries.

Question 30: Testing a Composed Transformation *(2026 blueprint)* **Question** *(Medium)*: A silver pipeline is written as `df.transform(clean_names).transform(add_net_amount, tax_rate=0.1)`. A `pytest` test compares its output, which contains floating-point amounts and has no defined row order, with an expected DataFrame. Which assertion is most appropriate?

A) `assert actual.collect() == expected.collect()`
B) `assertSchemaEqual(actual, expected)`
C) `assertDataFrameEqual(actual, expected, checkRowOrder=True)`
D) `assertDataFrameEqual(actual, expected, rtol=1e-5)`

Answer

Correct Answer: D
`assertDataFrameEqual` with a relative tolerance and the default row-order handling.
`assertDataFrameEqual` compares schema and data, ignores row order by default, and treats floats as equal within `rtol`/`atol`. A depends on row order and exact float equality. B compares only schemas and expects `StructType` arguments, not DataFrames. C would fail because the output isn't sorted.

Question 31: Schema Contract Test *(2026 blueprint)* **Question** *(Easy)*: A downstream team relies on the output columns and types of a `build_gold(df)` function, but the data values change daily. Which test checks only that contract?

A) `assertSchemaEqual(build_gold(df).schema, expected_schema)` where `expected_schema` is a `StructType`
B) `assertDataFrameEqual(build_gold(df), expected_df)`
C) `assert build_gold(df).count() > 0`
D) `assertDataFrameEqual(build_gold(df), [], checkRowOrder=True)`

Answer

Correct Answer: A
Compare the schemas with `assertSchemaEqual`.

`assertSchemaEqual` compares two `StructType` schemas and raises `[DIFFERENT_SCHEMA]` on mismatch without looking at data. B also compares row values, which change daily. C checks only that rows exist. D compares against an empty list of rows and would fail on any data.

Question 32: Auto Loader Streaming Table *(2026 blueprint)* **Question** *(Medium)*: JSON files land continuously in a Unity Catalog volume. A data engineer must ingest them into a bronze table in a Lakeflow pipeline so that each file is processed exactly once, new files are picked up incrementally, and schema inference is handled automatically. Which SQL definition meets the requirement?

A) `CREATE OR REFRESH MATERIALIZED VIEW bronze AS SELECT * FROM read_files('/Volumes/main/landing format => 'json')`
B) `CREATE OR REFRESH STREAMING TABLE bronze AS SELECT * FROM read_files('/Volumes/main/landing/o format => 'json')` with `cloudFiles.schemaLocation` set in the pipeline configuration
C) A scheduled `COPY INTO bronze` job with `COPY_OPTIONS ('force' = 'true')`
D) `CREATE OR REFRESH STREAMING TABLE bronze AS SELECT * FROM STREAM read_files('/Volumes/main/la format => 'json')`

Answer

Correct Answer: D
A streaming table that reads with `STREAM read_files(...)` uses Auto Loader for incremental, exactly-once file ingestion.
Databricks recommends streaming tables with Auto Loader for file ingestion, and `read_files` invokes Auto Loader only with the `STREAM` keyword. A is wrong because materialized views should not be defined over Auto Loader ingest; they are recomputed rather than processing each file once. C is wrong because `force = true` reloads every file and creates duplicates. B is wrong because a streaming table's query must be a streaming query, so `STREAM` is required, and pipelines manage the schema location automatically.

Question 33: Auto Loader Schema Evolution Mode *(2026 blueprint)* **Question** *(Medium)*: A bronze streaming table ingests JSON events with Auto Loader in a Lakeflow pipeline. Front-end teams add fields without notice. The team wants the stream never to fail because of schema changes, and wants unexpected fields preserved for later review rather than added as columns. Which `cloudFiles.schemaEvolutionMode` should they set?

A) `addNewColumns`
B) `failOnNewColumns`
C) `rescue`
D) `none`

Answer

Correct Answer: C
`rescue` never evolves the schema, never fails the stream, and records new columns in the rescued data column.
A is wrong because `addNewColumns` deliberately fails the stream with `UnknownFieldException` after adding the columns and needs a restart. B is wrong because `failOnNewColumns` stops the stream until the schema or the data file is fixed. D is wrong because `none` ignores new columns and does not rescue their data unless a rescued data column is configured.

Question 34: Gold Table Over Updated Silver Data *(2026 blueprint)* **Question** *(Medium)*: A silver orders table is regularly updated with `MERGE` when refunds adjust past orders. A gold table must report revenue per customer that always reflects those adjustments. It is refreshed hourly by a serverless Lakeflow pipeline, and the team wants to avoid recomputing everything when only a few customers change. Which object should the engineer define?

A) A materialized view over the silver table
B) A streaming table that reads the silver table with `skipChangeCommits`
C) A temporary view over the silver table
D) A streaming table that reads the silver table without any options

Answer

Correct Answer: A
A materialized view always matches its query's result over the current source data, and on serverless it refreshes incrementally when the query supports it.
B is wrong because `skipChangeCommits` ignores update and delete commits, so the refunds would never reach gold. C is wrong because a temporary view is not persisted or published; it is recomputed whenever it is queried and is usable only inside the pipeline. D is wrong because a streaming read fails when its source contains update or delete commits, and a streaming table never revisits rows it already processed.

Question 35: Ingesting from Kafka *(2026 blueprint)* **Question** *(Easy)*: A Kafka topic keeps messages for 7 days. The business needs every event kept permanently in a bronze table with latency of a few seconds. Which Lakeflow pipeline design fits?

A) A materialized view that reads the topic with `read_kafka`, refreshed every minute
B) A materialized view over `read_kafka` with `REFRESH POLICY INCREMENTAL STRICT`
C) A temporary view over `read_kafka` queried by downstream tables
D) A streaming table that reads the topic with `STREAM read_kafka(...)` in a continuously running pipeline

Answer

Correct Answer: D
A streaming table processes each Kafka record once and keeps it after the topic's retention expires; continuous execution gives low latency.
A and B are wrong because the docs warn against defining materialized views on sources that don't retain history, such as Kafka. A recompute would lose events older than the retention period, and MVs are not designed for second-level latency. C is wrong because a view is not persisted, so nothing is kept permanently.

Question 36: Selective SCD Type 2 History *(2026 blueprint)* **Question** *(Medium)*: An AUTO CDC flow maintains an employee dimension. Auditors need a new history row only when `salary` or `department` changes. Frequent changes to `last_login` must update the current row without creating history. Which `dp.create_auto_cdc_flow` arguments meet the requirement?

A) `stored_as_scd_type=1, except_column_list=["last_login"]`
B) `stored_as_scd_type=2, apply_as_truncates=expr("operation = 'T'")`
C) `stored_as_scd_type=2, except_column_list=["last_login"]`
D) `stored_as_scd_type=2, track_history_column_list=["salary", "department"]`

Answer

Correct Answer: D
SCD Type 2 with `track_history_column_list` creates versions only for the listed columns; changes to other columns update the current version in place.
A is wrong because SCD Type 1 keeps no history at all. C is wrong because `except_column_list` removes `last_login` from the target entirely and every remaining column still creates history. B is wrong because `apply_as_truncates` is supported only for SCD Type 1 and has nothing to do with history tracking.

Question 37: Migrating apply_changes *(2026 blueprint)* **Question** *(Medium)*: A pipeline contains `dlt.apply_changes(target="dim_customer", source="customer_cdc", keys=["id"], sequence_by=col("ts"), stored_as_scd_type=2)`. The engineer is moving it to the current `pyspark.pipelines` API without changing behavior. Which code is correct?

A) `dp.create_streaming_table("dim_customer")` followed by `dp.create_auto_cdc_flow(target="dim_cus source="customer_cdc", keys=["id"], sequence_by=col("ts"), stored_as_scd_type=2)`
B) An `@dp.materialized_view` named `dim_customer` that runs a `MERGE` against the source
C) `dp.create_auto_cdc_from_snapshot_flow(target="dim_customer", source="customer_cdc", keys=["id"], stored_as_scd_type=2)`
D) A managed table created with `CREATE TABLE dim_customer ... FLOW`, then `dp.create_auto_cdc_flow(...` with the same arguments

Answer

Correct Answer: A
`create_auto_cdc_flow` replaces `apply_changes` with the same signature, and its target must be a streaming table created first.
B is wrong because dataset functions must return a DataFrame and must not run writes such as `MERGE`. C is wrong because the snapshot API compares full snapshots and is meant for sources without a change feed; it has no `sequence_by`. D is wrong because managed tables created with `CREATE TABLE ... FLOW` do not support CDC flows and fail with `MANAGED_TABLE_DOES_NOT_SUPPORT_CDC`.

Question 38: When Structured Streaming Fits Better *(2026 blueprint)* **Question** *(Hard)*: A team needs a streaming aggregation emitted in **complete** output mode to a custom sink. They also want explicit control over the trigger interval and the checkpoint location, and plan to run it on jobs compute. Which approach fits these operational constraints?

A) A streaming table in a continuous Lakeflow pipeline with its flow set to complete mode
B) A materialized view refreshed every minute by a scheduled pipeline
C) Spark Structured Streaming code scheduled as a Lakeflow Jobs task
D) An AUTO CDC flow with `stored_as_scd_type=1`

Answer

Correct Answer: C
Structured Streaming gives full control over output mode, trigger, sink, and checkpoint location.
A is wrong because pipelines currently expose only append and update flows, and they manage checkpoints themselves. B is wrong because a materialized view is a batch flow, not a streaming query writing to a custom sink. D is wrong because AUTO CDC applies change events into

a streaming table; it is not a complete-mode aggregation to a custom sink.

Question 39: When Declarative Pipelines Fit Better *(2026 blueprint)* **Question** *(Medium)*: A SQL-focused team is building a bronze-silver-gold pipeline. They need automatic dependency ordering, automatic retries of transient failures, data quality rules with pass/fail metrics, lineage, and CDC handling, all with minimal custom code on Databricks. Which approach best fits?

A) Structured Streaming notebooks chained as job tasks, with custom metrics tables
B) Open-source Apache Spark Declarative Pipelines on a self-managed Spark 4.1 cluster
C) Scheduled `COPY INTO` statements in a SQL warehouse
D) Lakeflow Pipelines with streaming tables, materialized views, expectations, and AUTO CDC

Answer

Correct Answer: D
Lakeflow Pipelines infer dependencies, retry failures (task, then flow, then pipeline), and add expectations, AUTO CDC, and a queryable event log on top of SDP.
A is wrong because the team would have to write and maintain the orchestration, retries, checkpoints, and quality metrics themselves. B is wrong because open-source SDP does not include expectations, AUTO CDC, or the event log; those are Lakeflow additions. C is wrong because `COPY INTO` is batch file loading with no dependency graph, expectations, or CDC handling.

Section 2: Data Ingestion & Acquisition (12 %)

Practice questions for Section 2 of the Oct 9, 2026 exam guide. Questions marked *(2026 blueprint)* target skills that are new or reworded in that guide. Study material: Section 2 folder.

Question 1: Auto Loader Schema Evolution **Question** *(Easy)*: A data engineering team ingests JSON files from cloud storage using Auto Loader. New fields are occasionally added to the source data. Which configuration ensures new columns are automatically added to the target table schema?

A) `cloudFiles.schemaEvolutionMode = "addNewColumns"`
B) `cloudFiles.schemaEvolutionMode = "rescue"`
C) `cloudFiles.inferColumnTypes = "true"`
D) `cloudFiles.mergeSchema = "true"`

Answer

Correct Answer: A
`schemaEvolutionMode = "addNewColumns"` automatically adds new columns to the schema. Option B rescues unexpected data to a separate column. Option C enables type inference but not schema evolution. Option D is not a valid Auto Loader option (that's for Delta writes).

Question 2: OpenSharing **Question** *(Easy)*: A company needs to share data with an external partner who uses Snowflake. Which statement about OpenSharing is correct?

A) The recipient must have a Databricks workspace
B) OpenSharing only works within the same cloud provider
C) Data is physically copied to the recipient's storage
D) The recipient can read shared data using any OpenSharing client

Answer

Correct Answer: D
OpenSharing is an open protocol. Recipients can use any compatible client (Spark, pandas, Power BI, Snowflake, etc.) without needing Databricks. Data is read in place, not copied. It works across cloud providers.

Question 3: Iceberg Readers for a CDC Target *(2026 blueprint)* **Question** *(Hard)*: An AUTO CDC flow in a Lakeflow pipeline maintains an SCD Type 1 streaming table. An external engine using an Iceberg v3 REST catalog client must read this table with read-after-write freshness and **without** a separate data copy. What should the engineer do?

A) Add the table properties `delta.columnMapping.mode = 'name'`, `delta.enableRowTracking = 'true'`, `delta.enableIcebergCompatV3 = 'true'`, `delta.universalFormat.enabledFormats = 'iceberg'`, and `pipelines.externalMetadata.enabled = 'true'` to the streaming table, then run the pipeline
B) Declare the target as `CREATE OR REFRESH STREAMING TABLE ... USING ICEBERG`
C) Set `delta.enableIcebergCompatV2 = 'true'` on the streaming table
D) Enable compatibility mode for the streaming table

Answer

Correct Answer: A
External data access with UniForm Iceberg v3 properties publishes Iceberg metadata for pipeline datasets without copying data.
B is wrong because for pipeline streaming tables the only `USING` option is `DELTA`. C is wrong because Iceberg reads with IcebergCompatV2 cannot be enabled on streaming tables or materialized views. D is wrong because compatibility mode creates a copy that refreshes on a schedule (hourly by default), which breaks both requirements.

Question 4: Managed Iceberg Gold Object *(2026 blueprint)* **Question** *(Medium)*: In a Lakeflow pipeline, a gold aggregate computed from a CDC-maintained silver table must be created as an object that external Iceberg readers can read. The workspace has the managed Iceberg materialized view preview enabled. Which definition fits?

A) `STORED AS ICEBERG` added to the AUTO CDC flow that writes the silver table
B) A sink created with `dp.create_sink` using an Iceberg format
C) `CREATE OR REFRESH MATERIALIZED VIEW gold_summary USING ICEBERG AS SELECT ...`, followed by `REPAIR TABLE gold_summary SYNC METADATA`
D) Legacy change data feed on the silver table so Iceberg readers can consume its changes

Answer

Correct Answer: C
Materialized views accept `USING ICEBERG` (managed Iceberg MVs, Public Preview); after creation you run `REPAIR TABLE ... SYNC METADATA`.

A is wrong because STORED AS only accepts SCD TYPE 1, SCD TYPE 2, or BITEMPORAL. B is wrong because pipeline sinks support Delta, Kafka, Event Hubs, and custom Python data sources, not Iceberg. D is wrong because legacy CDF supports only Delta tables, and change data feed is not part of the Iceberg spec.

Question 5: Kafka Starting Offsets *(2026 blueprint)* **Question** *(Medium)*: A new Structured Streaming query reads an existing Kafka topic with `spark.readStream.format("kafka")` and writes to a Delta table, but the first run ingests none of the messages already in the topic. The engineer then sets `startingOffsets` to `earliest` and restarts the query with the same checkpoint, yet historical messages are still missing. What explains both behaviors?

A) Kafka streaming reads default to `earliest`, so the topic must have been compacted
B) `startingOffsets` is ignored unless `maxOffsetsPerTrigger` is also set
C) Streaming reads default to `latest`, and `startingOffsets` only applies when a query starts with a new checkpoint
D) Delta sinks discard records older than the table's creation time

Answer

Correct Answer: C
Kafka streaming reads default to `startingOffsets = latest`, and the option is honored only for a new query, not one resuming from a checkpoint.
The Spark options reference states the default is `latest` for streaming (`earliest` for batch) and that resumed queries always use the checkpoint, so a fresh checkpoint location is needed. A is wrong because the streaming default is `latest`. B is wrong because `maxOffsetsPerTrigger` only caps batch size. D is wrong because Delta sinks have no such filter.

Question 6: Binary Files with Auto Loader *(2026 blueprint)* **Question** *(Easy)*: A team must incrementally ingest PNG images that land in a Unity Catalog volume alongside other file types, storing each image's path, size, and raw bytes in a Delta table. Which configuration fits?

A) Auto Loader with `cloudFiles.format` set to `binaryFile` and `pathGlobFilter` set to `*.png`
B) Auto Loader with `cloudFiles.format` set to `parquet` and `cloudFiles.inferColumnTypes` set to `true`
C) `spark.readStream.format("delta")` pointed at the volume path
D) `COPY INTO` with `FILEFORMAT = ICEBERG`

Answer

Correct Answer: A
Auto Loader supports `binaryFile`, which returns `path`, `modificationTime`, `length`, and `content` per file, and `pathGlobFilter` selects the suffix.
The Auto Loader patterns docs show exactly this `binaryFile` + `pathGlobFilter` combination because the path is only a prefix filter. B is wrong because images are not Parquet. C is wrong because the volume contains raw files, not a Delta table. D is wrong because Iceberg is a table format, not a file format for `COPY INTO`.

Question 7: Choosing a CDC Ingestion Approach *(2026 blueprint)* **Question** *(Medium)*: An operational SQL Server database must be continuously replicated into Unity Catalog, including

updates and deletes, with minimal custom code and full row history kept for a customer dimension table. Which approach meets the requirement?

A) A nightly JDBC full-table read that overwrites the target table
B) A Lakehouse Federation foreign catalog queried by a materialized view
C) A Lakeflow Connect SQL Server connector with CDC enabled on the source table and `scd_type` set to `SCD_TYPE_2`
D) A Lakeflow Connect SQL Server connector using change tracking with `scd_type` set to `SCD_TYPE_2`

Answer

Correct Answer: C
The managed SQL Server connector with source CDC enabled and `SCD_TYPE_2` replicates changes continuously and keeps history.
Lakeflow Connect docs note that SCD type 2 requires CDC on the source and that change tracking does not support `SCD_TYPE_2`, which rules out D. A misses changes between runs and keeps no history. B queries the source live without replicating changes or history.

Question 8: Ingestion Gateway Behavior *(2026 blueprint)* **Question** *(Hard)*: A team uses a gateway-based Lakeflow Connect PostgreSQL pipeline. To save money, an engineer stops the ingestion gateway each night and restarts it in the morning, while the ingestion pipeline runs hourly on serverless. What is the main risk?

A) The serverless ingestion pipeline cannot start while the gateway is stopped
B) The destination streaming tables lose their change data feed
C) The staging volume is deleted whenever the gateway stops
D) Changes can be lost to source log retention, forcing a full refresh of affected tables

Answer

Correct Answer: D
The gateway must run continuously to capture changes before the source truncates its change logs; stopping it can drop changes and require full refreshes.
The docs warn not to stop the gateway, which runs on classic compute as a continuous pipeline. A is wrong because the ingestion pipeline can run on its own schedule from staging data. C is wrong because staged data is purged after 30 days, not on stop. B is wrong because CDF is enabled on all target tables regardless.

Question 9: D2D Recipient Setup *(2026 blueprint)* **Question** *(Medium)*: A provider wants to share curated tables, a Unity Catalog volume, and a notebook with a partner whose Databricks workspace is enabled for Unity Catalog in another account, without managing any tokens. What should the provider do?

A) Run `CREATE RECIPIENT partner USING ID '<cloud>:<region>:<uuid>'` with the partner's sharing identifier, then `GRANT SELECT ON SHARE s TO RECIPIENT partner`
B) Run `CREATE RECIPIENT partner` and send the activation link for the credential file
C) Grant `USE CATALOG` on the source catalog directly to the partner's account users
D) Create an OIDC federation recipient and share the volume through an Iceberg REST client

Answer

Correct Answer: A
A Databricks-to-Databricks recipient identified by the partner metastore's sharing identifier needs no tokens and supports volumes and notebooks.
The OpenSharing docs say D2D supports notebook, volume, and model sharing, which Databricks-to-Open does not. B creates a token-based Databricks-to-Open recipient that cannot receive volumes or notebooks. C does not work across metastores in different accounts. D is a Databricks-to-Open method and still cannot carry volumes.

Question 10: Sharing a Table Subset *(2026 blueprint)* **Question** *(Medium)*: A provider has one share for many regional partners and wants each recipient to see only rows for its own country from a partitioned table, without creating a separate share per partner. Which approach works?

A) `ALTER SHARE s ADD TABLE t WITH PARTITION (country = 'US')` for each partner
B) Share the table `WITHOUT HISTORY` so recipients only see current rows
C) Add row filters to the source table and share it directly
D) Set a `country` recipient property and add the table with `PARTITION (country = CURRENT_RECIPIENT().country)`

Answer

Correct Answer: D
Recipient properties referenced with `CURRENT_RECIPIENT()` in the partition spec let one share filter partitions per recipient.
The `ALTER SHARE` reference documents `PARTITION (column = CURRENT_RECIPIENT().<property-key>)`. A uses invalid `WITH PARTITION` syntax and still needs per-partner shares. C fails because tables with row filters or column masks cannot be shared. B controls history, not row visibility.

Question 11: Clean Room vs Share *(2026 blueprint)* **Question** *(Medium)*: An advertiser and a publisher, both on Databricks with Unity Catalog, want to compute audience overlap on hashed emails. Neither may see the other's rows, and both want to review any code before it touches their data. What should they use?

A) A Databricks-to-Databricks OpenSharing share in each direction
B) A Databricks Marketplace private exchange
C) Lakehouse Federation from each workspace into the other
D) A Databricks clean room with notebooks approved by the other collaborator and run by a designated runner

Answer

Correct Answer: D
Clean Rooms share data only into a central clean room, expose only metadata to collaborators, and run only notebooks approved by every other collaborator.
The Clean Rooms docs describe this no-trust model. A gives each recipient read access to the other's rows. C also exposes the data for direct querying. B distributes data products and gives no code approval or row protection.

Question 12: Clean Room Output *(2026 blueprint)* **Question** *(Hard)*: In an approval-based clean room, a notebook run writes results with `cr_output_catalog` and `cr_output_schema`. The other collaborator complains they cannot read the results. What explains this, and what fixes it for future runs?

A) Output tables are readable only by the runner by default; create a new clean room with shared output enabled and write to `cr_shared_output_schema`
B) Output tables expire after one hour; rerun the notebook more often
C) The collaborator lacks `MODIFY CLEAN ROOM`; grant it on the clean room
D) Output tables are views; convert them to tables with `CREATE TABLE AS SELECT`

Answer

Correct Answer: A
By default only the runner can read output tables; shared output, set when the clean room is created, lets all collaborators read tables in `cr_shared_output_schema`.
The output-tables docs state shared output can't be changed after creation, so a new clean room is required. B is wrong because outputs are kept 30 days. C is wrong because `MODIFY CLEAN ROOM` manages assets, not output access. D is wrong because only tables are supported as output.

Question 13: Federation Privileges *(2026 blueprint)* **Question** *(Medium)*: An admin created a Unity Catalog connection to a PostgreSQL database with credentials stored through `secret()`. A data engineer must now create a foreign catalog from it, and analysts must only query one table. Which grants are needed?

A) Engineer: `USE CONNECTION` on the connection; analysts: the PostgreSQL password
B) Engineer: `ALL PRIVILEGES` on the metastore; analysts: `BROWSE` on the connection
C) Engineer: `CREATE CONNECTION` on the metastore; analysts: `USE CONNECTION` on the connection
D) Engineer: `CREATE CATALOG` on the metastore and `CREATE FOREIGN CATALOG` on the connection; analysts: `USE CATALOG`, `USE SCHEMA`, and `SELECT` on the foreign objects

Answer

Correct Answer: D
Creating a foreign catalog requires `CREATE CATALOG` on the metastore plus `CREATE FOREIGN CATALOG` on the connection, and readers need normal UC data privileges.
Query federation docs list these requirements, and credentials come from the connection so readers never need them. A exposes credentials and `USE CONNECTION` doesn't allow catalog creation. C's `CREATE CONNECTION` creates connections, not catalogs, and `USE CONNECTION` doesn't grant table reads. B over-grants and `BROWSE` doesn't allow querying.

Question 14: Query vs Catalog Federation *(2026 blueprint)* **Question** *(Medium)*: A company is migrating tables registered in AWS Glue to Unity Catalog incrementally. During the migration, users must query Glue tables through Unity Catalog with UC access control, running only on Databricks compute and reading files directly from S3. Which approach fits?

A) Query federation over JDBC to Glue
B) A Lakeflow Connect database connector for Glue
C) Catalog federation with a Glue connection, external locations, and a foreign catalog with authorized paths
D) An OpenSharing share from Glue

Answer

Correct Answer: C
Catalog federation reads foreign tables directly from object storage on Databricks compute and is documented for Glue-based incremental migrations.
Setup needs a connection, external locations for the table paths, and a foreign catalog with authorized paths. A describes query federation, which pushes SQL to a database engine and doesn't apply to Glue. B is wrong because Lakeflow Connect database connectors read relational change logs. D is wrong because Glue is not an OpenSharing provider.

Section 3: Data Manipulation (12 %)

Practice questions for Section 3 of the Oct 9, 2026 exam guide. Questions marked *(2026 blueprint)* target skills that are new or reworded in that guide. Study material: Section 3 folder.

Question 1: MERGE Operation **Question** *(Easy)*: A data engineer needs to update existing records and insert new ones from a source table into a target Delta table. Which MERGE clause handles records that exist in the source but not in the target?

A) `WHEN MATCHED THEN UPDATE`
B) `WHEN NOT MATCHED BY SOURCE THEN DELETE`
C) `WHEN MATCHED THEN INSERT`
D) `WHEN NOT MATCHED THEN INSERT`

Answer

Correct Answer: D
`WHEN NOT MATCHED THEN INSERT` handles source records that don't have a matching key in the target. `WHEN MATCHED` handles existing records. Option C is invalid syntax. Option B handles records in target not in source.

Question 2: Delta Lake Time Travel **Question** *(Medium)*: A user accidentally deleted important records 3 days ago. The table has default retention settings. Which command restores the data?

A) `RESTORE TABLE orders TO VERSION AS OF 100`
B) `SELECT * FROM orders TIMESTAMP AS OF '2024-01-12'`
C) `ROLLBACK TABLE orders TO 3 DAYS AGO`
D) Both A and B can be used to access historical data

Answer

Correct Answer: D
Both version-based (`VERSION AS OF`) and timestamp-based (`TIMESTAMP AS OF`) time travel work. RESTORE actually reverts the table, while SELECT reads historical data. Default retention is 7 days, so 3-day-old data is available.

Question 3: Expectations **Question** *(Easy)*: A Lakeflow pipeline requires that all records have a non-null customer_id. Invalid records should be removed but not fail the pipeline. Which expectation syntax achieves this?

A) `CONSTRAINT valid_id EXPECT (customer_id IS NOT NULL)`
B) `@dlt.expect_all("valid_id", "customer_id IS NOT NULL")`
C) `CONSTRAINT valid_id EXPECT (customer_id IS NOT NULL) ON VIOLATION FAIL UPDATE`
D) `CONSTRAINT valid_id EXPECT (customer_id IS NOT NULL) ON VIOLATION DROP ROW`

Answer

Correct Answer: D
`ON VIOLATION DROP ROW` removes invalid records without failing the pipeline. Option A only logs warnings but keeps invalid rows. Option C fails the entire pipeline. Option B has incorrect syntax.

Question 4: Quarantining Bad Records *(2026 blueprint)* **Question** *(Medium)*: Records that fail two validation rules must not reach the silver path but must be kept for review. The team wants per-rule pass/fail metrics, and the pipeline must keep running. Which implementation fits?

A) `@dp.expect_all_or_drop(rules)` on the silver table only
B) `@dp.expect_all_or_fail(rules)` on the silver table
C) A private table with `@dp.expect_all(rules)` that adds `is_quarantined = NOT(<all rules>)`, plus valid and invalid views that filter on that flag
D) `CONSTRAINT valid EXPECT (...) ON VIOLATION QUARANTINE ROW`

Answer

Correct Answer: C
The documented quarantine pattern uses warn-level expectations for metrics and a computed flag to split valid and invalid rows without losing any.
A is wrong because dropped rows are only counted, not kept for review. B is wrong because a fail expectation stops the flow's update instead of routing the rows. D is wrong because the only `ON VIOLATION` actions are `DROP ROW` and `FAIL UPDATE`.

Question 5: Fail Expectation Behavior *(2026 blueprint)* **Question** *(Hard)*: A **triggered** Lakeflow pipeline has a materialized view `gold_balances` with `@dp.expect_or_fail("non_negative", "balance >= 0")`. An independent streaming table in the same pipeline ingests clickstream data. During an update, one balance is negative. What happens?

A) The violating row is dropped and counted as a dropped record in the event log
B) Every flow in the pipeline, including the clickstream table, fails and rolls back
C) The row is written to `gold_balances` and a warning metric is recorded
D) The `gold_balances` flow fails and its update is rolled back, while the independent clickstream flow continues to update

Answer

Correct Answer: D
In a triggered pipeline, a fail expectation fails and atomically rolls back only that flow's update; other parallel flows are not failed.
A describes `expect_or_drop`. B is wrong because only in continuous mode does a failure also

stop dependent flows, and an unrelated flow is not affected in triggered mode. C describes the default warn action. Note also that no expectation metrics are recorded for an update that fails.

Question 6: Tolerant JSON Parsing into VARIANT *(2026 blueprint)* **Question** *(Medium)*: A Bronze table receives webhook payloads as JSON strings in a column named `raw`. About 0.1 % of payloads are malformed. The Silver job must convert every payload to a `VARIANT` column without failing the whole batch, and it must keep a `NULL` for unparseable payloads so they can be quarantined later. Which expression should the engineer use?

A) `parse_json(raw)`
B) `try_parse_json(raw)`
C) `to_json(raw)`
D) `variant_get(raw, '$', 'variant')`

Answer

Correct Answer: B
`try_parse_json(raw)` returns a `VARIANT` for valid JSON and `NULL` for malformed input, so the batch keeps running.
`try_parse_json` is the error-tolerant version of `parse_json`, and its `NULL` result can be routed to a quarantine table. `parse_json` (A) raises `MALFORMED_RECORD_IN_PARSING` on invalid JSON and would fail the batch. `to_json` (C) goes the other way, turning a `STRUCT` or `VARIANT` into a JSON string. `variant_get` (D) extracts typed values from an existing `VARIANT` expression and can't parse a string.

Question 7: Clustering on a Field Inside VARIANT *(2026 blueprint)* **Question** *(Hard)*: A Silver table stores clickstream events in a single `payload VARIANT` column. Most queries filter on `payload:customer_id` and `payload:event_ts`, and scans are slow. The team wants file-level data skipping on those two attributes while keeping the rest of the payload flexible. Which design should the engineer choose?

A) Run `ALTER TABLE events CLUSTER BY (payload)` and then `OPTIMIZE`
B) Recreate the table with `PARTITIONED BY (payload:customer_id)`
C) Extract `payload:customer_id::BIGINT` and `payload:event_ts::TIMESTAMP` into typed top-level columns, keep `payload` as `VARIANT`, and cluster on the new columns
D) Run `OPTIMIZE events ZORDER BY (payload)`

Answer

Correct Answer: C
Promote the frequently filtered fields to typed columns, keep the rest as `VARIANT`, and liquid-cluster on the typed columns.
`VARIANT` columns can't be partition, clustering, or Z-order keys, and they don't collect min/-max statistics. Databricks recommends extracting frequently queried fields into non-variant columns to speed up queries and improve layout. Options A, B, and D all try to use the `VARIANT` column, or an expression on it, as a layout key, which isn't supported. Partition columns must also be top-level columns rather than expressions.

Question 8: Keeping Large ai_query Batches Running *(2026 blueprint)* **Question** *(Medium)*: A nightly job enriches 40 million product reviews with `ai_query('system.ai.meta-llama-3-3-70b-instruct', prompt || review_text)`. A handful of rows hit model endpoint errors each night, the whole query fails, and all completed inferences are lost. What should the engineer change to keep the successful results and still identify the failed rows?

A) Add `failOnError => false` and write rows where `result.errorMessage IS NOT NULL` to a retry table
B) Split the input into 1,000-row batches and run them in a Python loop
C) Wrap the call in `try_cast(... AS STRING)`
D) Add `modelParameters => named_struct('temperature', 0.0)`

Answer

Correct Answer: A
With `failOnError => false`, `ai_query` returns a STRUCT with `response` and `errorMessage`, so the job completes and failed rows can be found.
The docs recommend `failOnError => false` for large workloads. A row with a model endpoint error gets a `NULL response` and a populated `errorMessage`, and successful rows are kept. Manual micro-batching (B) is discouraged because AI Functions already handle parallelization, retries, and scaling, and smaller batches can lower throughput. `try_cast` (C) doesn't intercept inference errors. Model parameters (D) change generation behavior, not error handling. `temperature` already defaults to 0.0 in `ai_query`.

Question 9: Ticket Routing with Minimal Prompt Engineering *(2026 blueprint)* **Question** *(Medium)*: A Lakeflow pipeline must route incoming support tickets to one of five known queues (billing, defect, account, feature request, other). The team has no custom model, wants no prompt engineering, and needs the output to be exactly one of the five label names so downstream routing logic doesn't break. Which approach fits best?

A) `ai_summarize(body, 5)` followed by `regexp_extract` to find a queue name
B) `ai_gen('Which queue should this ticket go to? ' || body)`
C) `ai_query` against a chat model with a free-form prompt and no `responseFormat`
D) `ai_classify(body, '["billing", "defect", "account", "feature_request", "other"]', map('version', '2.1'))`

Answer

Correct Answer: D
`ai_classify` is the task-specific function for classifying text into labels you provide, and it returns label names that exactly match the `labels` argument.
Databricks recommends starting with a task-specific AI Function when one matches the task. `ai_classify` needs no prompt, accepts label descriptions and instructions, and in version 2.x returns a `VARIANT` whose `response[0].value` is one of the supplied labels. Option A is a fragile workaround on a summary. `ai_gen` (B) and an unconstrained `ai_query` (C) produce free text that might not match a queue name. `ai_query` is meant for cases that need a specific model, prompt, or output schema.

Section 4: Monitoring and Alerting (10 %)

Practice questions for Section 4 of the Oct 9, 2026 exam guide. Questions marked *(2026 blueprint)* target skills that are new or reworded in that guide. Study material: Section 4 folder.

Question 1: System Tables **Question** *(Easy)*: A data engineer needs to track compute costs by team over the past month. Which system table provides this information?

A) `system.billing.usage`
B) `system.access.audit`
C) `system.compute.clusters`
D) `system.query.history`

Answer

Correct Answer: A
`system.billing.usage` contains cost and usage data including DBUs consumed. `audit` tracks access events. `clusters` contains cluster metadata. `query.history` tracks SQL queries, not costs.

Question 2: Lakeflow Pipelines Event Log **Question** *(Easy)*: A Lakeflow pipeline ran with data quality issues. The team needs to find how many records failed expectations. How can this information be retrieved?

A) Query the Delta table directly
B) Check the Spark UI
C) Query the pipeline's event log
D) Review the cluster logs

Answer

Correct Answer: C
The Lakeflow Pipelines event log contains expectation metrics including passed/failed record counts. Query it using `event_log(TABLE(pipeline))` or access via the system tables. The event log provides detailed pipeline observability.

Question 3: Attribute DBU Spend to Jobs *(2026 blueprint)* **Question** *(Medium)*: A data engineer must report last month's list cost per job. They join `system.billing.usage` to `system.billing.list_prices` only on `sku_name` and the totals come out several times higher than the invoice. What change fixes the join?

A) Filter `system.billing.usage` to `record_type = 'ORIGINAL'` only
B) Also join on `cloud` and require `usage_start_time` to fall between `price_start_time` and `price_end_time` (or `price_end_time IS NULL`)
C) Replace `pricing.default` with `usage_quantity` from `list_prices`
D) Join to `system.compute.clusters` on `sku_name` instead

Answer

Correct Answer: B
Join on `cloud`, `sku_name`, and the price validity window.
`list_prices` stores one row per price change, so joining on `sku_name` alone multiplies each usage row by every historical price. The docs join on `cloud`, `sku_name`, and `usage_start_time >= price_start_time AND (usage_end_time <= price_end_time OR price_end_time IS NULL)`, then multiply by `pricing.default`. Filtering to ORIGINAL rows (A) would drop correc-

tions that should be summed. `list_prices` has no `usage_quantity` (C). `compute.clusters` has no prices and no `sku_name` (D).

Question 4: Missing Job Cost in Billing *(2026 blueprint)* **Question** *(Hard)*: A team notices that several scheduled jobs never appear when they group `system.billing.usage` by `usage_metadata.job_id`, even though `system.lakeflow.job_run_timeline` shows the jobs ran successfully every day. What is the most likely explanation?

A) The jobs ran on all-purpose compute, where `usage_metadata.job_id` is not populated
B) Billing records for jobs are only written after 365 days
C) `usage_metadata.job_id` is only populated for jobs triggered by the REST API
D) The jobs ran in a different region, so `system.billing.usage` cannot see them

Answer

Correct Answer: A
`job_id` is populated only for jobs on job compute or serverless compute.
The billable usage reference states that `usage_metadata.job_id` and `job_run_id` are populated for serverless jobs and jobs on job compute, but not for jobs run on all-purpose compute. That usage can only be approximated through `cluster_id`. Billing arrives within hours, not after 365 days (B). The trigger type doesn't matter (C). `system.billing.usage` is a global table that covers every region (D).

Question 5: Monitor Failed Pipeline Events from a Script *(2026 blueprint)* **Question** *(Medium)*: A data engineer writes a Python CI check that must fail when a Lakeflow pipeline logged any ERROR events. They use the Databricks SDK for Python with `w = WorkspaceClient()`. Which call returns those events?

A) `w.jobs.get_run_output(run_id=pipeline_id)`
B) `w.pipelines.list_updates(pipeline_id=pid, filter="level='ERROR'")`
C) `w.pipelines.list_pipeline_events(pipeline_id=pid, filter="level='ERROR'")`
D) `w.pipelines.get(pipeline_id=pid).events`

Answer

Correct Answer: C
Use `list_pipeline_events` with `filter="level='ERROR'"`.
`list_pipeline_events` (REST `GET /api/2.0/pipelines/{pipeline_id}/events`, CLI `databricks pipelines list-pipeline-events`) supports SQL-like filters such as `level='ERROR'` and `timestamp > '...'`. `list_updates` lists updates and takes no level filter (B). `get_run_output` is for job task runs (A). `GetPipelineResponse` has `state`, `health`, and `latest_updates`, but no `events` attribute (D).

Question 6: Why Did a Task Fail via the Jobs API *(2026 blueprint)* **Question** *(Medium)*: A multitask job run failed. An engineer calls the Jobs API `runs/get-output` with the job run's `run_id` and gets an error instead of the task's stack trace. What should they do?

A) Use Jobs API 2.0, because 2.2 removed run output
B) Call `runs/get` for the job run, then call `runs/get-output` with the failed task's `run_id` from

tasks[]
C) Call runs/list with expand_tasks=true, which returns error_trace
D) Read state.result_state from runs/get, which contains the stack trace

Answer

Correct Answer: B
Pass the task run ID from runs/get to runs/get-output.
runs/get-output (SDK get_run_output) returns the output, error, and error_trace of a single task run, so you first read tasks[].run_id from runs/get. Jobs API 2.2 still provides run output (A). runs/list returns run metadata, not error traces (C). state is deprecated, and its result_state is only an enum (D).

Question 7: Branch a Job on an Alert *(2026 blueprint)* **Question** *(Hard)*: A job refreshes a silver table and then must run a quarantine notebook only when a Databricks SQL alert on null rates is TRIGGERED, and otherwise publish to gold. The engineer adds a SQL alert task, but the quarantine step never runs, even when the alert triggers, because the alert task always shows Succeeded. What should they add?

A) Set run_if: AT_LEAST_ONE_FAILED on the quarantine task
B) An If/else condition task comparing {{tasks.<alert_task>.output.alert_state}} to TRIGGERED, with quarantine depending on the true outcome and publish on the false outcome
C) A schedule on the underlying alert so it runs before the job
D) Increase the alert task's retries so it fails when triggered

Answer

Correct Answer: B
Add an If/else condition task on output.alert_state.
A SQL alert task reports Succeeded whenever evaluation completes, whether or not the alert triggered. It fails only on evaluation errors. The dynamic value {{tasks.<task>.output.alert_state}} (UNKNOWN, OK, or TRIGGERED) feeds an If/else task whose dependents use the true and false outcomes. AT_LEAST_ONE_FAILED never fires because the task succeeds (A). The alert's own schedule is independent of the job (C). Retries don't change a successful evaluation (D).

Question 8: Schema-Wide Freshness Alerts *(2026 blueprint)* **Question** *(Medium)*: A platform team wants Slack notifications whenever any table in the main.sales schema becomes stale or receives far fewer rows than usual, without writing or maintaining SQL for each table. Which approach fits best?

A) Create one Databricks SQL alert per table on DESCRIBE HISTORY
B) Enable anomaly detection on the schema and create a Data Quality Monitoring alert rule scoped to main.sales with a Slack notification destination
C) Configure a budget in the account console with the schema as a tag filter
D) Add a duration warning health rule to every job that writes the schema

Answer

Correct Answer: B
Use anomaly detection with a Data Quality Monitoring alert rule.

Anomaly detection checks freshness and completeness for every table in a schema. Alert rules created in the Data Quality Monitoring UI are scoped to a catalog or schema and notify users or destinations such as Slack, with no SQL to write. A SQL alert per table (A) means manual work for each table. Budgets track spend, not data quality (C). Job duration warnings track run time, not table freshness (D).

Question 9: Interpret Job Run Status *(2026 blueprint)* **Question** *(Easy)*: In the Lakeflow Jobs UI, a multitask job run shows Succeeded with failures. What does this mean?

A) All tasks succeeded after at least one retry
B) Some tasks failed, but all leaf tasks succeeded
C) The run exceeded its duration warning threshold
D) The run was skipped because of max concurrent runs

Answer

Correct Answer: B
Some tasks failed, but every leaf task succeeded.
The run outcome is determined by the leaf tasks, the tasks with no downstream dependencies. Succeeded with failures means some tasks failed but all leaf tasks succeeded, and notifications treat it as a success. Retries that eventually succeed show as Succeeded (A). Exceeding a duration threshold triggers a warning, not a status (C). A run blocked by concurrency limits is Skipped (D).

Question 10: Find the Slow Phase of a Job Run *(2026 blueprint)* **Question** *(Medium)*: A scheduled job on classic compute has recently started finishing 15 minutes later than usual, even though its code and data volume are unchanged. In the Jobs UI, where can an engineer quickly see whether the extra time is spent queueing, waiting for compute, installing libraries, or running?

A) The run breakdown shown when hovering the run's Duration on the job run details page
B) The Spark UI event timeline of the driver
C) The Top 5 error types table on the Runs tab
D) The `system.billing.usage` table filtered by `job_run_id`

Answer

Correct Answer: A
Hover the run's Duration to see the run breakdown by phase.
The job run details page breaks Duration into Queued, Waiting for resources, Library installation, and Running, for both job runs and task runs. The Spark UI (B) only covers execution inside Spark. The Top 5 error types table (C) summarizes failures. Billing (D) shows DBUs, not a timing breakdown.

Section 5: Cost & Performance Optimization (15 %)

Practice questions for Section 5 of the Oct 9, 2026 exam guide. Questions marked *(2026 blueprint)* target skills that are new or reworded in that guide. Study material: Section 5 folder.

Question 1: Change Data Feed **Question** *(Easy)*: A data engineer needs to track all changes (inserts, updates, deletes) to a Delta table for downstream CDC processing. Which statement correctly enables this capability?

A) `ALTER TABLE orders SET TBLPROPERTIES ('delta.enableChangeDataFeed' = true)`
B) `ALTER TABLE orders SET TBLPROPERTIES ('delta.enableChangeDataCapture' = true)`
C) `ALTER TABLE orders ENABLE CHANGE TRACKING`
D) `CREATE TABLE orders WITH (CDC = ENABLED)`

Answer

Correct Answer: A
`delta.enableChangeDataFeed = true` enables Change Data Feed (CDF) on a Delta table. This allows reading changes using `table_changes()` function or `readChangeData` option. The other options use incorrect syntax.

Question 2: Back-Pressure and Rate Limiting **Question** *(Medium)*: A streaming query reads from Kafka and the processing time per micro-batch is consistently exceeding the trigger interval, causing increasing latency. Which configuration limits the amount of data processed per micro-batch?

A) `trigger(processingTime="10 seconds")` to increase batch frequency
B) `spark.sql.streaming.stateStore.maintenanceInterval` to reduce state overhead
C) `spark.sql.shuffle.partitions` to increase parallelism
D) `maxOffsetsPerTrigger` to limit records consumed from Kafka per batch

Answer

Correct Answer: D
`maxOffsetsPerTrigger` controls the maximum number of offsets (records) read from Kafka per trigger, providing back-pressure control. Changing the trigger interval doesn't reduce per-batch data. State maintenance affects state cleanup, not ingestion. Shuffle partitions affect parallelism, not input volume.

Question 3: Liquid Clustering vs Z-ORDER **Question** *(Medium)*: A table is frequently filtered by `customer_id` and `order_date`. The team wants automatic optimization. Which approach provides automatic maintenance?

A) `ALTER TABLE table CLUSTER BY (customer_id, order_date)`
B) `OPTIMIZE table ZORDER BY (customer_id, order_date)` scheduled daily
C) `CREATE TABLE table PARTITIONED BY (customer_id, order_date)`
D) Both A and B provide automatic maintenance

Answer

Correct Answer: A
Liquid Clustering (`CLUSTER BY`) provides automatic, incremental clustering without manual OPTIMIZE commands. Z-ORDER requires scheduled OPTIMIZE runs. Partitioning is for low-cardinality columns and doesn't cluster data within partitions.

Question 4: Query Profiler Question *(Medium)*: A SQL query is running slowly. The data engineer wants to see if partition pruning is working. What should they look for in EXPLAIN output?

A) `PartitionFilters` showing filter conditions
B) `DataFilters` showing filter conditions
C) `PushedFilters` showing filter conditions
D) All of the above indicate different types of filtering

Answer

Correct Answer: D
`PartitionFilters` shows partition pruning (best). `DataFilters` shows filters applied after scan. `PushedFilters` shows filters pushed to the file format. Ideally, filters should appear in PartitionFilters for best performance.

Question 5: Identifying Data Skew Question *(Medium)*: A job has 99 tasks completing in 1 minute but one task takes 30 minutes. What is the most likely cause?

A) Insufficient cluster memory
B) Data skew in partition keys
C) Network timeout
D) Corrupt data file

Answer

Correct Answer: B
One task taking much longer than others typically indicates data skew - one partition has significantly more data. Solutions include salting keys, using AQE skew handling, or repartitioning data more evenly.

Question 6: OPTIMIZE and ZORDER Question *(Easy)*: A table frequently filters on `region` and `date` columns. Small files are causing performance issues. Which command best optimizes query performance?

A) `OPTIMIZE table_name`
B) `OPTIMIZE table_name ZORDER BY (region)`
C) `OPTIMIZE table_name ZORDER BY (region, date)`
D) `VACUUM table_name`

Answer

Correct Answer: C
ZORDER on both frequently filtered columns provides the best data skipping. Option A compacts files but doesn't optimize data layout. Option B only optimizes for one column. VACUUM removes old files but doesn't improve layout.

Question 7: Shuffle Partitions Question *(Medium)*: A Spark job processing 10GB of data is running slowly with default settings. What is the default value of `spark.sql.shuffle.partitions` and how should it be adjusted?

A) 100; increase for large data
B) 200; decrease for small data
C) 200; increase for large data
D) 500; keep default for all data

Answer

Correct Answer: B
Default is 200. For 10GB (relatively small), reducing partitions can improve performance by reducing overhead. Rule of thumb: target ~128MB per partition. 10GB / 128MB = 80 partitions.

Question 8: Broadcast Join **Question** *(Easy)*: A large fact table (100GB) is joined with a small dimension table (50MB). How can the join be optimized?

A) Increase shuffle partitions
B) Use a broadcast hint on the large table
C) Use a broadcast hint on the small table
D) Partition both tables by the join key

Answer

Correct Answer: C
Broadcasting the small table avoids shuffling the large table. The small table is sent to all executors for a local join. Default broadcast threshold is 10MB, so a hint may be needed for 50MB: `/*+ BROADCAST(small_table) */`.

Question 9: AQE (Adaptive Query Execution) **Question** *(Medium)*: A query has skewed data causing one task to run much longer than others. Which AQE feature helps with this?

A) Coalesce partitions
B) Skew join optimization
C) Dynamic broadcast join
D) Local shuffle reader

Answer

Correct Answer: B
AQE's skew join optimization automatically detects and splits skewed partitions to distribute work more evenly. Coalesce combines small partitions. Dynamic broadcast switches join strategies. Local shuffle reader reduces I/O.

Question 10: File Size Optimization **Question** *(Easy)*: A streaming job creates many small files, degrading query performance. Which configuration helps?

A) `spark.sql.shuffle.partitions = 1`
B) `delta.autoOptimize.optimizeWrite = true`
C) `spark.databricks.delta.vacuum.enabled = true`
D) `delta.checkpoint.writeStatsAsJson = false`

Answer

Correct Answer: B
`optimizeWrite` bins data during writes to create better-sized files. This is especially helpful for streaming which naturally creates small files. Option A would create single-file outputs (bad for parallelism). Vacuum removes old files but doesn't compact.

Question 11: Cost Optimization **Question** *(Easy)*: A daily ETL job runs for 3 hours. The team wants to reduce costs. Which change provides the biggest cost savings?

A) Switch from all-purpose to job cluster
B) Increase cluster size to finish faster
C) Run during business hours
D) Disable auto-scaling

Answer

Correct Answer: A
Job clusters have ~60% lower DBU rates than all-purpose clusters. They're created for the job and terminated after, eliminating idle time costs. Larger clusters may cost more overall. Auto-scaling typically helps with costs.

Question 12: EXPLAIN Plan Analysis **Question** *(Hard)*: A data engineer runs `EXPLAIN EXTENDED` on a slow query and sees `SortMergeJoin` in the physical plan where one table is only 5MB. What does this indicate and how should it be fixed?

A) The join is optimal; SortMergeJoin is always the fastest strategy
B) Add a Z-ORDER on the join key to improve data locality
C) The broadcast threshold is too low; increase `spark.sql.autoBroadcastJoinThreshold` or add a broadcast hint
D) The query needs more shuffle partitions to distribute the sort

Answer

Correct Answer: C
A 5MB table should be broadcast-joined (avoiding shuffle), not sort-merge-joined. The default threshold is 10MB, but if statistics are inaccurate, Spark may choose SortMergeJoin. Increasing the threshold or using `/*+ BROADCAST(t) */` forces the more efficient BroadcastHashJoin.

Question 13: Photon Engine **Question** *(Medium)*: A team enables Photon on their cluster expecting all queries to run faster. Some queries show improvement while others show no change. Which workload type benefits MOST from Photon acceleration?

A) Machine learning model training with MLlib
B) Scan-heavy queries with aggregations and joins on large Delta tables
C) Python UDF-heavy transformations
D) Streaming micro-batches with very small data volumes

Answer

Correct Answer: B
Photon is a C++ vectorized execution engine optimized for scan-heavy SQL/DataFrame operations including aggregations, joins, and filters. It doesn't accelerate Python UDFs (which run in Python), MLlib (which uses different execution paths), or very small data volumes (where overhead exceeds benefit).

Question 14: Memory and Spill Diagnostics Question *(Medium)*: A Spark job shows "spill to disk" warnings in the Spark UI. The job completes but is much slower than expected. What is the primary cause of disk spill and the best remediation?

A) Disk spill is caused by too many tasks; reduce `spark.sql.shuffle.partitions`
B) Disk spill is caused by insufficient CPU cores; increase cluster size
C) Disk spill indicates corrupted data blocks; run OPTIMIZE on source tables
D) Disk spill occurs when partition data exceeds available memory; increase `spark.sql.shuffle.partitions` or executor memory

Answer

Correct Answer: D
Spill to disk happens when a partition's data cannot fit in memory. Either increase shuffle partitions (smaller partitions) or executor memory (more room per partition). Reducing partitions makes the problem worse. CPU cores don't affect memory spill. OPTIMIZE affects file sizes, not in-memory partition sizes.

Question 15: Liquid Clustering Question *(Hard)*: A table currently uses `OPTIMIZE ... ZORDER BY (region, date)` which must be run manually after each batch load. The team wants automatic incremental data layout optimization. What is the correct approach to migrate to liquid clustering?

A) `ALTER TABLE t CLUSTER BY (region, date)` and future writes/OPTIMIZE runs apply clustering incrementally
B) `ALTER TABLE t SET TBLPROPERTIES ('delta.liquidClustering' = true, 'clusterColumns' = 'region,date')`
C) Drop and recreate the table with `CREATE TABLE t CLUSTER BY (region, date)` using CTAS
D) Liquid clustering and Z-ORDER cannot coexist; the table must be fully rewritten first

Answer

Correct Answer: A
`ALTER TABLE ... CLUSTER BY` enables liquid clustering on an existing table. Future writes and OPTIMIZE runs will incrementally cluster data without rewriting the entire table. Option B uses incorrect syntax. Option C loses table history unnecessarily. Liquid clustering replaces Z-ORDER; they don't need to coexist since liquid clustering is incremental.

Question 16: Query Optimization with Predicate Pushdown Question *(Hard)*: A query reads from a Delta table partitioned by `date` and filtered by `region`. The EXPLAIN plan shows a

`Filter` node above the Scan node for the `region` predicate. How can predicate pushdown for the `region` column be improved?

A) Add a WHERE clause earlier in the SQL query to force pushdown
B) Enable `spark.sql.optimizer.enablePredicatePushDown = true` (disabled by default)
C) Apply Z-ORDER or liquid clustering on `region` to enable data skipping via file-level statistics
D) Convert the `region` column to a partition column for physical-level pushdown

Answer

Correct Answer: C
Delta Lake uses file-level min/max statistics for data skipping. Z-ORDER or liquid clustering by `region` co-locates similar values, making file statistics effective for skipping irrelevant files. Predicate pushdown is enabled by default. Query order doesn't affect pushdown. While partitioning by `region` works, it's not always advisable if cardinality is very high. Data skipping with Z-ORDER/clustering is the better approach for non-partition columns.

Question 17: Auto Optimize Settings **Question** *(Medium)*: A streaming pipeline writes to a Delta table using `foreachBatch`. The table accumulates many small files between manual OPTIMIZE runs. The team wants to automate file compaction. Which combination of settings prevents small file accumulation for this use case?

A) Set `spark.databricks.delta.autoCompact.enabled = true` only
B) Set `spark.databricks.delta.optimizeWrite.enabled = true` only
C) Enable both auto compaction and optimize write on the SparkSession
D) Set both `delta.autoOptimize.optimizeWrite` and `delta.autoOptimize.autoCompact` as table properties

Answer

Correct Answer: D
Setting both as table properties ensures they persist regardless of cluster configuration. `optimizeWrite` bins data during writes for better sizes. `autoCompact` runs a lightweight OPTIMIZE after writes. Using table properties (`ALTER TABLE SET TBLPROPERTIES`) is more reliable than SparkSession configs which must be set on every cluster/session.

Question 18: Reading Row-Level Changes *(2026 blueprint)* **Question** *(Medium)*: A Delta table has `delta.enableChangeDataFeed = true`. A downstream process must incrementally handle only rows that were updated or deleted since its last run, using the new values for updated rows, without scanning the whole table. Which approach is most efficient?

A) Read with `.option("readChangeFeed", "true")` from the last processed version and keep rows where `_change_type` is `update_postimage` or `delete`
B) Read with `readChangeFeed` and keep rows where `_change_type` is `update_preimage`
C) Compare the current table with `VERSION AS OF` the previous version using a full anti-join
D) Run `DESCRIBE HISTORY` and reprocess the full table whenever a new version exists

Answer

Correct Answer: A
The change data feed returns only the changed rows; `update_postimage` carries the values after an update and `delete` marks removed rows.

B is wrong because `update_preimage` holds the values before the update. C works but scans both versions in full, which defeats incremental processing. D reprocesses everything and gives no row-level change information.

Question 19: CDF and Retention *(2026 blueprint)* **Question** *(Hard)*: A stream that reads a Delta table's change data feed was paused for six weeks. The table uses default retention settings and is vacuumed regularly. When restarted from a new checkpoint with `startingVersion` set to the last version it processed, the stream fails to start. What is the most likely cause?

A) Change data feed is disabled automatically whenever `VACUUM` runs
B) `readChangeFeed` works only for batch reads, not streaming reads
C) The requested version and its change data are no longer in the table history; `VACUUM` removed the change files and log retention (default 30 days) expired the version
D) `OPTIMIZE` renamed the `_change_type` column, breaking the stream schema

Answer

Correct Answer: C
CDF is not a permanent record: change data follows the table's retention, and a stream cannot start from a version that is no longer in history.
A is wrong because `VACUUM` deletes old files but does not turn off the table property. B is wrong because CDF supports streaming reads, and streaming is the recommended way to consume it. D is wrong because `OPTIMIZE` does not rename CDF metadata columns. The fix is to restart from an available version, and to archive the change feed if a long history is required.

Question 20: Retiring Maintenance Jobs *(2026 blueprint)* **Question** *(Medium)*: A team owns 40 external Delta tables registered in Unity Catalog. Every night an all-purpose cluster runs notebooks that execute `OPTIMIZE`, `VACUUM`, and `ANALYZE TABLE` on each table, and the team spends hours tuning the schedule. Predictive optimization is enabled at the account level. They want the platform to handle this maintenance automatically with the least ongoing effort. What should they do?

A) Run `ALTER TABLE ... ENABLE PREDICTIVE OPTIMIZATION` on each external table and keep the nightly notebooks as a fallback
B) Enable auto compaction on every table and move the notebooks to a job cluster to lower the DBU rate
C) Convert the tables to Unity Catalog managed tables with `ALTER TABLE ... SET MANAGED`, let them inherit predictive optimization, and remove the scheduled maintenance jobs
D) Add `ZORDER BY` to the nightly `OPTIMIZE` statements so that fewer runs are needed

Answer

Correct Answer: C
Convert the tables to UC managed tables and let predictive optimization run `OPTIMIZE`, `VACUUM`, and `ANALYZE` automatically.
Predictive optimization only supports Unity Catalog managed tables; it runs maintenance on serverless jobs compute when it is cost-effective, and Databricks recommends disabling scheduled `OPTIMIZE` jobs once it is on. `SET MANAGED` keeps the table name, permissions, and history. A is wrong because predictive optimization does not run on external tables, whatever the table-level setting. B still leaves the team owning `VACUUM` and statistics jobs. D adds work: Z-order must be re-applied manually and predictive optimization never runs `ZORDER`.

Question 21: Predictive Optimization Inheritance *(2026 blueprint)* **Question** *(Hard)*: Predictive optimization is enabled for the account. An admin runs `ALTER CATALOG finance DISABLE PREDICTIVE OPTIMIZATION;` and later `ALTER SCHEMA finance.reporting ENABLE PREDICTIVE OPTIMIZATION;`. No table-level settings exist, and all tables are Unity Catalog managed tables. Which statement describes the result?

A) Managed tables in `finance.reporting` are maintained by predictive optimization, while managed tables in other `finance` schemas are not
B) No tables in the `finance` catalog are maintained, because a catalog-level `DISABLE` overrides every schema below it
C) All tables in `finance` are maintained, because the account-level setting always takes precedence
D) Only tables created after the schema-level `ENABLE` are maintained; existing tables must be re-created

Answer

Correct Answer: A
The schema-level `ENABLE` overrides the catalog's `DISABLE` for `finance.reporting`, and the other schemas inherit the catalog's `DISABLE`.
Predictive optimization uses an inheritance model: account → catalog → schema → table, where an explicit `ENABLE` or `DISABLE` at a lower level overrides its parent and `INHERIT` resets it. B is wrong because lower levels can override the catalog. C is wrong because the account value is only the default that objects inherit. D is wrong because the setting applies to existing managed tables in the schema; no re-creation is needed.

Question 22: Changing Filter Patterns *(2026 blueprint)* **Question** *(Medium)*: A 30 TB Unity Catalog managed gold table is queried by hundreds of analysts. One quarter most queries filter on `region`, the next on `product_id` and `order_date`, and the platform team has no time to keep re-tuning the layout. Predictive optimization is enabled for the catalog. Which approach best fits this access pattern?

A) Partition the table by `region` and run a nightly `OPTIMIZE ... ZORDER BY (product_id, order_date)`
B) Enable deletion vectors so that layout changes are applied at read time
C) Define liquid clustering with all five frequently filtered columns as keys and run `OPTIMIZE FULL` every night
D) Run `ALTER TABLE ... CLUSTER BY AUTO` so that predictive optimization selects and evolves the clustering keys from the query workload

Answer

Correct Answer: D
`CLUSTER BY AUTO` lets predictive optimization choose clustering keys from the table's query history and change them as patterns shift.
Automatic liquid clustering is available for UC managed tables with predictive optimization; it analyzes the historical query workload and changes keys only when the expected data-skipping savings outweigh the clustering cost. A hard-codes one pattern, needs manual Z-order runs, and partitioning is not recommended under 100 TB. B is wrong because deletion vectors reduce write amplification for row-level changes; they do not cluster data. C is invalid because liquid clustering allows at most four keys, and nightly `OPTIMIZE FULL` is unnecessary since clustering

is incremental.

Question 23: Physically Removing Deleted Rows *(2026 blueprint)* **Question** *(Hard)*: A customer table has deletion vectors enabled because it receives frequent small `MERGE` and `DELETE` operations. For a GDPR request, the engineer runs `DELETE FROM main.crm.customers WHERE customer_id = 42`. The compliance team requires that the deleted record's bytes no longer exist in any data file in storage. What should the engineer do next?

A) Run `VACUUM` with the default retention; it removes soft-deleted rows from current data files
B) Run `REORG TABLE main.crm.customers APPLY (PURGE)`, then run `VACUUM` so the old files from earlier table versions are removed
C) Run `OPTIMIZE main.crm.customers ZORDER BY (customer_id)`, which guarantees every deletion vector is applied
D) Disable deletion vectors with `ALTER TABLE ... SET TBLPROPERTIES ('delta.enableDeletionVectors' = false)`, which rewrites the affected files immediately

Answer

Correct Answer: B
`REORG TABLE ... APPLY (PURGE)` rewrites files that contain soft-deleted rows, and `VACUUM` then deletes the older files that still hold the data.
With deletion vectors, a `DELETE` only marks the row in metadata, so the bytes stay in the existing Parquet file. The docs say to run `REORG TABLE ... APPLY (PURGE)` and then `VACUUM` to physically remove old data. A is wrong because `VACUUM` only deletes files no longer referenced by the table; the current file with the soft-deleted row is still referenced. C is wrong because compaction gives no strict guarantee that every deletion-vector change is applied. D is wrong because changing the property affects future writes; it does not purge existing soft-deleted data.

Question 24: Speeding Up Repeated Reads *(2026 blueprint)* **Question** *(Easy)*: Analysts on a classic all-purpose cluster run many different queries against the same few Delta tables throughout the day. Each query reads the data again from cloud object storage, and the team wants repeated reads to be faster without changing any notebook code. What should they do?

A) Add `df.cache()` after every `spark.read.table()` call in the notebooks
B) Run `CACHE SELECT * FROM` each table at the start of the day
C) Choose a worker instance type with local SSDs so the disk (Delta) cache is enabled and stores copies of the Parquet files locally after the first read
D) Increase `spark.sql.shuffle.partitions` so that more tasks read from storage in parallel

Answer

Correct Answer: C
Use SSD-backed workers so the disk cache automatically keeps local copies of the data files.
The disk cache (formerly Delta cache) is enabled by default on worker types that support it; when a node fetches a Parquet file, it stores a copy on local storage, so later reads of the same data, from any query, run locally. A requires code changes and caches DataFrames manually in memory; Databricks recommends the disk cache instead of the Spark cache for repeated reads of Parquet and Delta data. B is wrong because `CACHE SELECT` is ignored on Databricks Runtime 14.2 and above and on SQL warehouses. D affects shuffle parallelism, not how often files are fetched from object storage.

Question 25: Disk Cache Consistency *(2026 blueprint)* **Question** *(Medium)*: A cluster with the disk cache enabled has been serving dashboard queries on a Delta table for hours. A job then runs a `MERGE` that rewrites several of the table's files. A data engineer worries that the dashboards will keep returning old data from the cache. What happens?

A) The cache keeps serving the old files until someone runs `CLEAR CACHE` on the cluster
B) The cluster must be restarted to clear the local SSD cache
C) The disk cache only stores query results, so the dashboards show old results until the 24-hour cache life cycle ends
D) The disk cache detects that the data files were modified, invalidates the stale entries, and later reads return the current data with no manual action

Answer

Correct Answer: D
The disk cache keeps itself consistent by detecting created, deleted, modified, or overwritten files.
The docs state that you can write, modify, and delete table data with no need to explicitly invalidate cached data; stale entries are invalidated and evicted automatically. A and B describe manual invalidation that the disk cache does not require (restarting only clears it). C confuses the disk cache, which stores data files, with the Databricks SQL result cache; the result cache has a 24-hour life cycle but is also invalidated when the underlying tables are updated.

Question 26: Stats Unused in the Query Profile *(2026 blueprint)* **Question** *(Medium)*: A SQL warehouse query filters a large liquid-clustered table with `WHERE CAST(event_date AS STRING) = '2026-10-01'`, and `event_date` is a clustering key. The query profile shows the Scan operator taking most of the time, a low percentage of data pruned, and a `COVERAGE_STATS_DELTA` insight with the status Unused. What is the best fix?

A) Increase the warehouse size so the full scan finishes faster
B) Rewrite the predicate to compare the column directly, for example `WHERE event_date = DATE'2026-10-01'`
C) Add a `/*+ BROADCAST */` hint to the query
D) Run `SET use_cached_result = false` so the profile reflects a fresh execution

Answer

Correct Answer: B
Compare `event_date` directly so its min/max statistics can be used for file skipping.
The `COVERAGE_STATS_DELTA` insight reports Unused when statistics can't be used because the filter converts the data type; data skipping relies on per-file min/max values of the column itself. A treats the symptom and costs more without restoring skipping. C is about join strategy and the query has no join problem. D only bypasses the result cache; it doesn't change how many files are read.

Question 27: Missed Broadcast Join *(2026 blueprint)* **Question** *(Hard)*: A query joins a 2 TB fact table to a 5 MB country dimension on `country_code`. In the query profile, Shuffle operators feed both inputs of the Join, and the Shuffle on the fact side has the highest Time spent and Memory peak. The dimension table has no optimizer statistics. What change most directly removes the expensive shuffle of the fact table?

A) Increase `spark.sql.shuffle.partitions` so each shuffle partition is smaller
B) Add a `/*+ MERGE(d) */` hint so the join uses a sort-merge join
C) Add a `/*+ BROADCAST(d) */` hint on the dimension (or collect statistics with `ANALYZE TABLE`) so the small side is broadcast and the fact table is not shuffled
D) Z-order the fact table by `country_code`

Answer

Correct Answer: C
Broadcasting the small dimension avoids shuffling the 2 TB fact table.
Shuffle operators on both join inputs mean a shuffled join; in a broadcast hash join the small side is sent to every executor and the large side isn't shuffled. The `BROADCAST` hint broadcasts the hinted side regardless of `autoBroadcastJoinThreshold`, and missing or stale statistics (the `COVERAGE_STATS_OPTIMIZER` insight) lead to wrong size estimates and a slow join strategy. A only resizes the shuffle; the fact table is still moved across the network. B forces a strategy that also shuffles (and sorts) both sides. D improves file skipping for filters, not the join's data movement.

Question 28: Small Table, Many Partitions *(2026 blueprint)* **Question** *(Medium)*: A 400 GB orders table is partitioned by `order_date`, which has produced thousands of small partitions. Most queries filter on `customer_id` (millions of distinct values) and a date range. The team wants the layout Databricks currently recommends for this size and query pattern. What should they choose?

A) Re-partition the table by `customer_id` to match the most common filter
B) Keep the date partitioning and schedule a nightly `OPTIMIZE ... ZORDER BY (customer_id)`
C) Keep the date partitioning and add a Bloom filter index on `customer_id`
D) Remove the partitioning and use liquid clustering on `order_date` and `customer_id`

Answer

Correct Answer: D
Drop partitioning and cluster the table on the filter columns with liquid clustering.
Databricks says tables under 1 TB should not be partitioned, recommends liquid clustering for all new tables, and notes that liquid clustering works for both low- and high-cardinality columns without the small-file issues of static partitioning. A is worse: partitioning only suits low or known cardinality, and millions of `customer_id` values would create millions of tiny partitions. B keeps the over-partitioned layout and adds a manual Z-order job that predictive optimization never maintains. C is wrong because Bloom filter indexes are deprecated and superseded by predictive I/O and liquid clustering.

Question 29: Migrating a Partitioned, Z-Ordered Table *(2026 blueprint)* **Question** *(Hard)*: A 60 TB Delta table is partitioned by `event_date` and Z-ordered nightly by `device_id`. The team wants to move to liquid clustering with minimal reader and writer downtime while keeping data skipping on both columns, using Databricks Runtime 18.1. Which approach should they use?

A) Run `ALTER TABLE events REPLACE PARTITIONED BY WITH CLUSTER BY (event_date, device_id)` and then run `OPTIMIZE`
B) Run `ALTER TABLE events CLUSTER BY (device_id)` on the partitioned table and keep the nightly Z-order job
C) Create a new table with both `PARTITIONED BY (event_date)` and `CLUSTER BY (device_id)` using CTAS

D) Run `OPTIMIZE events ZORDER BY (event_date, device_id)` once, which converts the table to liquid clustering

Answer

Correct Answer: A
Use the in-place `REPLACE PARTITIONED BY WITH CLUSTER BY` conversion with the partition and Z-order columns as keys, then `OPTIMIZE`.
Databricks recommends this conversion (DBR 18.1+) because it minimizes reader and writer downtime, and its migration guidance says to use both the partition columns and the `ZORDER BY` columns as clustering keys; running `OPTIMIZE` afterwards applies the new clustering. B is wrong because `ALTER TABLE ... CLUSTER BY` enables liquid clustering only on unpartitioned tables, and liquid clustering is incompatible with `ZORDER`. C is invalid because clustering is not compatible with partitioning. D only rewrites files with Z-order; it never enables liquid clustering.

Section 6: Ensuring Data Security and Compliance (8 %)

Practice questions for Section 6 of the Oct 9, 2026 exam guide. Questions marked *(2026 blueprint)* target skills that are new or reworded in that guide. Study material: Section 6 folder.

Question 1: Row-Level Security **Question** *(Medium)*: Different sales teams should only see data for their assigned regions. Which approach implements row-level security in Unity Catalog?

A) Create separate tables for each region
B) Use dynamic views with `current_user()` function
C) Apply column masking policies
D) Use secret scopes for each region

Answer

Correct Answer: B
Dynamic views using `current_user()` or `is_member()` functions filter rows based on the querying user's identity or group membership. Column masking hides column values. Separate tables don't scale. Secret scopes are for credentials.

Question 2: Secret Management **Question** *(Easy)*: A notebook needs to access a database password without exposing it in code. Which approach correctly retrieves the secret?

A) `dbutils.secrets.get(scope="db-scope", key="password")`
B) `spark.conf.get("db.password")`
C) `dbutils.credentials.get("db-scope", "password")`
D) `os.environ.get("DB_PASSWORD")`

Answer

Correct Answer: A
`dbutils.secrets.get(scope, key)` retrieves secrets from Databricks secret scopes. The value is automatically redacted in notebook output. The other options either don't exist or expose credentials.

Question 3: Data Lineage **Question** *(Medium)*: An audit team needs to understand how a Gold-layer table is derived from raw data sources, including all intermediate transformations. How does Unity Catalog capture this lineage information?

A) Lineage is automatically captured from Spark jobs, Lakeflow pipelines, and SQL queries without additional configuration
B) Engineers must manually register lineage by calling `dbutils.lineage.register()` after each transformation
C) Lineage is only available for Lakeflow pipelines, not for general Spark jobs
D) Lineage requires enabling a separate "Lineage Tracking" feature at the metastore level

Answer

Correct Answer: A
Unity Catalog automatically captures column-level and table-level lineage from Spark operations, Lakeflow pipelines, and SQL queries. No manual registration is needed. The lineage is available in the Catalog Explorer UI and via REST API. It works across notebooks, jobs, and Lakeflow pipelines.

Question 4: Audit Logging **Question** *(Medium)*: A security team needs to investigate who accessed a sensitive table in the last 30 days and what queries they ran. Which system table provides this information?

A) `system.information_schema.tables` - contains table metadata including last access time
B) `system.billing.usage` - tracks DBU consumption per user and table
C) `system.access.audit` - records all access events including user, action, and resource
D) `system.compute.clusters` - logs cluster activity and associated table access

Answer

Correct Answer: C
`system.access.audit` is the system table that records all access events in Unity Catalog, including who accessed which resources, what actions were performed, and when. Information schema has metadata, not access history. Billing tracks costs. Compute tables track cluster lifecycle.

Question 5: Column Masking **Question** *(Hard)*: A table contains PII (email, phone number). Analysts need to query the table but should only see masked versions of PII columns unless they are in the `pii_readers` group. Which Unity Catalog feature implements this requirement?

A) Create a dynamic view that checks `is_member('pii_readers')` and conditionally shows full or masked values
B) Create separate tables with and without PII, granting access to the appropriate one
C) Use `DENY SELECT` on PII columns for non-PII readers
D) Apply column mask functions using `ALTER TABLE ... ALTER COLUMN ... SET MASK`

Answer

Correct Answer: D
Unity Catalog column masking (`SET MASK`) applies a function that controls what value each user sees based on their identity or group membership. This is applied directly to the table, not requiring separate views or tables. `DENY SELECT` on individual columns is not supported in Unity Catalog. Dynamic views work but column masking is the more direct, recommended approach.

Question 6: Network Security **Question** *(Hard)*: A financial services company requires that all Databricks workspace traffic stays within their private network and never traverses the public internet. Which network configuration achieves this?

A) Enable IP access lists to restrict source IP addresses
B) Configure VPN peering between the corporate network and Databricks control plane
C) Deploy a workspace with Private Link / Private Endpoints for both front-end and back-end connectivity
D) Use a firewall to block all outbound traffic from the Databricks data plane

Answer

Correct Answer: C
Private Link (AWS) / Private Endpoints (Azure) ensure both the front-end (UI/API) and back-end (data plane to control plane) connections remain on the private network backbone without traversing the public internet. IP access lists restrict who can access but still use public internet. VPN alone doesn't cover all connection paths. Blocking outbound breaks Databricks functionality.

Question 7: GDPR Compliance - Right to Erasure **Question** *(Hard)*: Under GDPR, a customer requests deletion of all their personal data. The customer's data exists in Bronze, Silver, and Gold Delta tables across the lakehouse. What is the correct approach to fulfill this deletion request?

A) Run `VACUUM` on all tables to remove files containing the customer's data
B) Use time travel to revert each table to a version before the customer's data was ingested
C) Delete the customer's records only from Gold tables since Bronze/Silver are considered raw storage exempt from GDPR
D) Execute `DELETE FROM` on all tables containing the customer's data, then `VACUUM` to physically remove the old files

Answer

Correct Answer: D
GDPR right to erasure requires physical deletion. First, `DELETE FROM` marks records as removed in Delta's transaction log. Then `VACUUM` removes the old files that still contain the deleted data. All tables (including Bronze/Silver) must be processed - GDPR applies regardless of data tier. Time travel would lose all other changes. VACUUM alone doesn't target specific records.

Question 8: Least-Privilege Job Operators *(2026 blueprint)* **Question** *(Medium)*: An on-call support group must be able to trigger, cancel, and repair runs of a production Lakeflow job and

read its run logs, but must not be able to edit the job's tasks or change its permissions. Which job permission level should the group receive?

A) CAN VIEW
B) CAN MANAGE
C) CAN MANAGE RUN
D) IS OWNER

Answer

Correct Answer: C
CAN MANAGE RUN is the least-privilege level that allows running, cancelling, and viewing run logs without editing the job.
CAN MANAGE RUN allows Run now, Run now with different parameters, cancel, and viewing Spark UI and logs, but not editing job settings, deleting, or modifying permissions. CAN VIEW only shows job details and results. CAN MANAGE and IS OWNER both add editing, deleting, and permission changes, which violates least privilege, and a group cannot be IS OWNER of a job.

Question 9: Data Grants vs Workspace ACLs *(2026 blueprint)* **Question** *(Medium)*: Analysts must run ad hoc queries on the `prod.gold` schema through a shared SQL warehouse, with no ability to change the warehouse or modify data. Which combination follows least privilege?

A) CAN MANAGE on the warehouse and `ALL PRIVILEGES` on `prod.gold`
B) CAN USE on the warehouse, plus `USE CATALOG` on `prod` and `USE SCHEMA`, `SELECT` on `prod.gold`
C) CAN USE on the warehouse and `SELECT` on `prod.gold` only
D) IS OWNER on the warehouse and `MODIFY` on `prod.gold`

Answer

Correct Answer: B
CAN USE on the warehouse plus `USE CATALOG`, `USE SCHEMA`, and `SELECT` is the minimal combination.
Workspace ACLs control the warehouse (CAN USE lets users start it and run queries) while Unity Catalog grants control data; reading a table requires the usage privileges on the parent catalog and schema plus `SELECT`. Option C fails because `SELECT` without `USE CATALOG` causes permission denied. Options A and D grant editing rights on the warehouse and write access to data.

Question 10: Choosing ABAC for Future Tables *(2026 blueprint)* **Question** *(Medium)*: A governance team must mask every column tagged `pii : ssn` in all current and future tables of the `hr` catalog, and table owners must not be able to remove the masking. What should they implement?

A) A column mask policy created with `CREATE POLICY ... ON CATALOG hr COLUMN MASK ... MATCH COLUMNS has_tag_value('pii', 'ssn')`
B) `ALTER TABLE ... ALTER COLUMN ssn SET MASK` on every existing table
C) A dynamic view per table that hashes the SSN column
D) A row filter function attached to each schema in the catalog

Answer

Correct Answer: A
An ABAC column mask policy at the catalog level, matching the governed tag, covers current and future tables centrally.
ABAC policies are evaluated dynamically for every table in scope whose columns carry the matching governed tag, and table owners cannot modify or bypass a policy set at a higher level. Per-table SET MASK does not cover future tables and table owners can remove it. Dynamic views require new objects and users could still query base tables if granted. A row filter hides rows, not column values.

Question 11: ABAC Permissions and Tag Scope *(2026 blueprint)* **Question** *(Hard)*: A schema-level ABAC column mask policy uses `MATCH COLUMNS has_tag('pii') AS c ON COLUMN c`. A steward tags the table `customers` with `pii` but not its columns, and queries still return unmasked values. What is the cause?

A) The policy creator lacks SELECT on the table
B) Column conditions in `MATCH COLUMNS` match only tags set directly on columns; table tags do not propagate to columns
C) ABAC policies only apply to tables created after the policy
D) The `pii` tag must be ungoverned for ABAC to use it

Answer

Correct Answer: B
`MATCH COLUMNS` checks column-level tags only, so the column itself must be tagged. `has_tag()` in `WHEN` considers table tags and tags inherited from the schema or catalog, but in `MATCH COLUMNS` it checks tags applied directly on the column; tags never inherit to columns. Policies apply to existing and future tables, and ABAC works only with governed tags. Creating a policy requires `MANAGE` on the scope and `EXECUTE` on the UDF, not `SELECT`.

Question 12: Pseudonymization for Joins *(2026 blueprint)* **Question** *(Medium)*: Analysts need to join orders, support tickets, and web events by customer without ever seeing customer email addresses, and the same customer must map to the same value in every table. Which technique fits best?

A) Suppression: replace email with `NULL` in all tables
B) Generalization: keep only the email domain
C) Pseudonymization: store `sha2(concat(<secret salt>, lower(email)), 256)` as the join key
D) Random tokens generated with `uuid()` independently in each table

Answer

Correct Answer: C
A salted deterministic hash pseudonymizes the email while keeping it joinable across tables. `sha2` produces the same output for the same input, so the hashed key joins across tables, and the secret salt prevents dictionary reversal of low-entropy values. Suppression and generalization remove the ability to join individual customers. Independent `uuid()` values differ per table unless managed through a single token vault.

Question 13: Anonymization vs Pseudonymization *(2026 blueprint)* **Question** *(Easy)*: Which transformation is an anonymization technique rather than a pseudonymization technique?

A) Replacing exact ages with ten-year age bands and ZIP codes with their first three digits
B) Replacing emails with tokens stored in a restricted lookup table
C) Encrypting SSNs with `aes_encrypt` using a key from a secret scope
D) Hashing customer IDs with a salted `sha2`

Answer

Correct Answer: A
Generalization (age bands, truncated ZIP codes) coarsens data irreversibly, which is anonymization.
Tokenization, encryption, and salted hashing keep records linkable and can be reversed or re-linked with additional information (vault, key, or salt), so they are pseudonymization techniques.

Question 14: Automated PII Detection and Masking *(2026 blueprint)* **Question** *(Medium)*: New source feeds land in a catalog every week through batch jobs and Lakeflow streaming tables. The company wants sensitive columns detected and masked automatically without engineers tagging each new column. Which approach uses Unity Catalog features to achieve this?

A) Ask engineers to add AI-generated comments to every new table and mask columns whose comment mentions PII
B) Enable Data Classification with auto-tagging on the catalog and create an ABAC column mask policy on the catalog that matches the `class.*` tags
C) Write a regex UDF in each pipeline and apply it before writing every table
D) Create a dynamic view over each new table after it is published

Answer

Correct Answer: B
Data Classification auto-tags detected PII columns with `class.*` system tags, and a catalog-level ABAC policy masks them for all table types.
Data Classification incrementally scans tables, streaming tables, and materialized views, and ABAC policies apply to all tables in scope, so new feeds are covered automatically for batch and streaming consumers. Databricks states AI-generated comments should not be relied on to detect PII. Per-pipeline regex and dynamic views require manual work for each new table.

Question 15: Pipeline Identity and Masked Refreshes *(2026 blueprint)* **Question** *(Hard)*: After a catalog-level ABAC column mask policy `TO account users` is created, a Lakeflow pipeline refreshes a gold materialized view from masked silver tables, and every user, including privacy officers, now sees masked values in the materialized view. What is the correct fix?

A) Grant SELECT on the materialized view to the privacy officers
B) Add the pipeline owner or run-as service principal to the policy's EXCEPT clause and fully refresh the materialized view
C) Disable deletion vectors on the materialized view
D) Move the policy from the catalog to each table

Answer

Correct Answer: B
The pipeline identity must be exempt so the refresh reads unmasked source data.
Pipelines evaluate policies with the pipeline owner or run-as identity during refresh, so a masked identity permanently stores masked data in the materialized view; exempting it via EXCEPT and refreshing fixes the stored data while consumers remain covered by policies. Extra grants do not unmask stored values, deletion vectors are unrelated, and table-level policies would reproduce the same problem.

Question 16: Erasure with Deletion Vectors *(2026 blueprint)* **Question** *(Medium)*: A table has deletion vectors enabled. To satisfy a right-to-erasure request, an engineer runs `DELETE FROM users WHERE user_id = 42`. Which additional steps physically remove the user's data from storage?

A) `OPTIMIZE users` immediately, which guarantees removal
B) `REORG TABLE users APPLY (PURGE)`, then `VACUUM users` after the old files exceed the retention threshold
C) `VACUUM users` immediately; deletion vectors are removed by `VACUUM`
D) `RESTORE TABLE users` to the version before the delete

Answer

Correct Answer: B
`REORG TABLE ... APPLY (PURGE)` rewrites files containing soft-deleted rows, and `VACUUM` later deletes the old files.
With deletion vectors, `DELETE` only marks rows as removed; `APPLY (PURGE)` rewrites the affected files, but older files remain for time travel until `VACUUM` removes them after they expire under `delta.deletedFileRetentionDuration`. `OPTIMIZE` compaction gives no strict guarantee of applying deletion vectors, `VACUUM` alone does not rewrite current files, and `RESTORE` brings the data back.

Question 17: Retention on Databricks Runtime 18 *(2026 blueprint)* **Question** *(Hard)*: On Databricks Runtime 18.0, an engineer runs `VACUUM events RETAIN 72 HOURS` to shorten the purge window for a compliance requirement, but files older than 72 hours are not removed. What should they do?

A) Run `VACUUM events RETAIN 72 HOURS DRY RUN` first
B) Set `delta.deletedFileRetentionDuration` on the table to the required interval and run `VACUUM`
C) Increase `delta.logRetentionDuration` to 365 days
D) Disable predictive optimization

Answer

Correct Answer: B
In Databricks Runtime 18.0+, retention is controlled by the `delta.deletedFileRetentionDuration` table property.
The Runtime 18.0 release notes state that `VACUUM` ignores the retention argument except for `0 HOURS`, and that `deletedFileRetentionDuration` and `logRetentionDuration` must be consistent. `DRY RUN` only lists files, increasing log retention lengthens history, and disabling predictive

optimization stops automatic `VACUUM` instead of purging sooner.

Section 7: Data Governance (5 %)

Practice questions for Section 7 of the Oct 9, 2026 exam guide. Questions marked *(2026 blueprint)* target skills that are new or reworded in that guide. Study material: Section 7 folder.

Question 1: Unity Catalog Volumes vs DBFS **Question** *(Easy)*: A team needs to store raw data files that will be governed by Unity Catalog. Which storage option should they use?

A) DBFS root storage
B) Mounted cloud storage
C) Unity Catalog Volume
D) Workspace files

Answer

Correct Answer: C
Unity Catalog Volumes provide governed file storage with access controls, auditing, and lineage. DBFS and mounts are legacy approaches without UC governance. Workspace files are for notebooks and small files, not data.

Question 2: Unity Catalog Permission Inheritance **Question** *(Easy)*: A user is granted SELECT on a schema in Unity Catalog. What happens to tables created in that schema after the grant?

A) User automatically has SELECT on new tables
B) User must be granted SELECT on each new table
C) User has no access to new tables
D) User has MODIFY access to new tables

Answer

Correct Answer: A
Permissions in Unity Catalog inherit downward. A SELECT grant on a schema applies to all current and future tables in that schema. This simplifies access management compared to table-level grants.

Question 3: Managed vs External Tables **Question** *(Easy)*: A team needs to register an existing data lake path in Unity Catalog without moving the data. What type of table should they create?

A) Managed table
B) External table
C) View
D) Materialized view

Answer

Correct Answer: B
External tables point to data at a user-specified location. Managed tables store data in Unity Catalog-managed storage (data would be moved/copied). Views don't store data. Materialized views persist computed results.

Question 4: Information Schema Question *(Medium)*: A data governance team needs to programmatically inventory all tables across all catalogs that a service principal has access to, including their column types and descriptions. Which query approach provides this information?

A) Query `system.information_schema.columns` which shows metadata for all accessible objects
B) Run `SHOW TABLES IN *` to list all tables across catalogs
C) Use `dbutils.fs.ls()` to scan the metastore storage location
D) Call the Unity Catalog REST API endpoint `/api/2.1/unity-catalog/tables`

Answer

Correct Answer: A
`system.information_schema.columns` provides metadata about all columns in all tables the current user/principal can access. It includes column names, types, descriptions, and table ownership. `SHOW TABLES` works per-schema only. `dbutils.fs.ls()` shows files, not metadata. The REST API works but requires pagination and more code.

Question 5: Data Classification Tags Question *(Medium)*: An organization needs to tag tables and columns containing sensitive data (PII, financial, health) to enforce governance policies and improve discoverability. How should data classification be implemented in Unity Catalog?

A) Store classification metadata in a separate governance database and cross-reference during queries
B) Use Unity Catalog tags (`ALTER TABLE ... SET TAGS`) to apply classification labels at table and column level
C) Add `COMMENT` on each column with classification keywords and search comments for enforcement
D) Create a naming convention (e.g., `pii_email`, `sensitive_ssn`) and enforce via CI/CD checks

Answer

Correct Answer: B
Unity Catalog tags provide first-class support for classifying tables and columns. Tags are searchable in Catalog Explorer, can be queried via information schema, and integrate with governance workflows. Comments are free-text and harder to enforce programmatically. Naming conventions are fragile. Separate databases add complexity.

Question 6: Tags vs Comments *(2026 blueprint)* Question *(Easy)*: A data steward wants analysts to find all tables belonging to the finance team through workspace search filters and also give each table a human-readable description. Which combination is appropriate?

A) Comments for both purposes
B) A tag such as `team = finance` for filtering, and `COMMENT ON TABLE` for the description

C) Table properties set with TBLPROPERTIES for both purposes
D) A row filter that returns only finance rows

Answer

Correct Answer: B
Tags are key-value attributes suited to filtering and searching, while comments hold free-text descriptions.
Workspace search supports `tag:<key>:<value>` and tag filters, and also indexes table and column comments. Comments alone do not give structured filtering, table properties are not the discovery mechanism, and row filters restrict data access rather than add metadata.

Question 7: Discoverability Without Data Access *(2026 blueprint)* **Question** *(Medium)*: All employees should be able to see that tables in the `sales` catalog exist, including their names, comments, and tags, and request access, without being able to read any data. Which grant achieves this?

A) `GRANT USE CATALOG ON CATALOG sales` to the account users group
B) `GRANT SELECT ON CATALOG sales` to the account users group
C) `GRANT BROWSE ON CATALOG sales` to the account users group
D) `GRANT APPLY TAG ON CATALOG sales` to the account users group

Answer

Correct Answer: C
`BROWSE` exposes object metadata and access requests without data access.
`BROWSE` is granted at the catalog level and lets users discover objects and view names, descriptions, and tags without `USE CATALOG`, `USE SCHEMA`, or `SELECT`. `SELECT` grants data access, `USE CATALOG` alone is only a prerequisite, and `APPLY TAG` lets users change tags.

Question 8: Inheritance to New Tables *(2026 blueprint)* **Question** *(Easy)*: An administrator grants `USE CATALOG, USE SCHEMA, SELECT` on catalog `sales` to the `finance_team` group. A week later, a new schema `sales.forecast` and a table `sales.forecast.q4` are created. Assuming no other grants, what can `finance_team` do with `sales.forecast.q4`?

A) Nothing until `SELECT` is granted on the new table
B) Query it, because privileges granted on a catalog are inherited by current and future schemas and tables
C) Query it only if they also own the new schema
D) Modify and drop it, because catalog grants imply ownership

Answer

Correct Answer: B
Catalog-level privileges inherit downward to all current and future child objects.
Because `USE CATALOG`, `USE SCHEMA`, and `SELECT` were granted on the catalog, they apply to every schema and table in it, including ones created later. No per-table grant is needed, ownership is unrelated to inherited privileges, and `SELECT` does not include `MODIFY` or drop rights.

Question 9: Ownership vs MANAGE *(2026 blueprint)* **Question** *(Medium)*: A platform team wants a governance group to grant and revoke privileges on every table in `prod.gold`, including future tables, without making the group the owner and without giving it automatic read access to the data. What should they grant?

A) `ALL PRIVILEGES ON SCHEMA prod.gold`
B) `SELECT ON SCHEMA prod.gold WITH GRANT OPTION`
C) `MANAGE ON SCHEMA prod.gold`
D) Ownership of every table in `prod.gold`

Answer

Correct Answer: C
`MANAGE` on the schema delegates grant management for the schema and all its current and future objects.
`MANAGE` allows managing privileges, transferring ownership, and dropping objects without ownership and is inherited by child objects, but does not itself include data privileges. `ALL PRIVILEGES` grants data access and excludes `MANAGE`, Unity Catalog has no `WITH GRANT OPTION`, and transferring ownership of each table is not what was asked.

Section 8: Debugging and Deploying (10 %)

Practice questions for Section 8 of the Oct 9, 2026 exam guide. Questions marked *(2026 blueprint)* target skills that are new or reworded in that guide. Study material: Section 8 folder.

Question 1: Databricks CLI Authentication **Question** *(Medium)*: A CI/CD pipeline needs to authenticate with Databricks using a service principal. Which environment variables should be set?

A) `DATABRICKS_HOST` and `DATABRICKS_TOKEN`
B) `DATABRICKS_HOST`, `DATABRICKS_CLIENT_ID`, and `DATABRICKS_CLIENT_SECRET`
C) `DATABRICKS_URL` and `DATABRICKS_API_KEY`
D) `DATABRICKS_WORKSPACE` and `DATABRICKS_PAT`

Answer

Correct Answer: B
Service principal authentication requires the workspace host URL, client ID (application ID), and client secret. Option A is for PAT token authentication. Options C and D use invalid variable names.

Question 2: Spark UI Debugging **Question** *(Medium)*: A Spark job is experiencing long task times with high "Shuffle Read Blocked Time." What does this indicate?

A) Too few partitions
B) Network congestion or slow shuffle fetch
C) Out of memory errors
D) Insufficient disk space

Answer

Correct Answer: B
Shuffle Read Blocked Time indicates tasks waiting for shuffle data from other executors, typically due to network issues or executors being slow to serve shuffle blocks. Solutions include increasing shuffle retry settings or investigating network.

Question 3: Declarative Automation Bundles **Question** *(Medium)*: A team needs to deploy the same pipeline to dev, staging, and production environments. How should environments be configured in Declarative Automation Bundles?

A) Create separate bundle files for each environment
B) Use targets in databricks.yml with environment-specific settings
C) Use Git branches for each environment
D) Manually modify settings before each deployment

Answer

Correct Answer: B
DAB uses targets to define environment-specific configurations in a single databricks.yml. This is the recommended approach for multi-environment deployments. Separate files or manual changes lead to drift and errors.

Question 4: Bundle Deployment Mode **Question** *(Easy)*: A developer wants to test their bundle without affecting shared resources. Which target mode should they use?

A) `mode: production`
B) `mode: development`
C) `mode: testing`
D) `mode: sandbox`

Answer

Correct Answer: B
Development mode prefixes resource names with the username, pauses schedules, and uses personal workspace paths. This prevents conflicts with other developers or production resources.

Question 5: Git Folders **Question** *(Easy)*: A team wants to version control their notebooks with automatic sync to Git. Which statement about Git Folders is correct?

A) Git Folders require manual sync after each change
B) Git Folders can only connect to GitHub
C) Git Folders replace the need for Declarative Automation Bundles
D) Git Folders provide native Git operations like pull, commit, and push

Answer

Correct Answer: D
Git Folders provide native Git operations directly in the Databricks UI. They support multiple Git providers (GitHub, GitLab, Azure DevOps, etc.). They complement DAB (which handles deployment), not replace it.

Question 6: CI/CD Pipeline **Question** *(Easy)*: A GitHub Actions workflow needs to deploy a bundle to Databricks. Which step is required before running `databricks bundle deploy`?

A) Install Python
B) Run `databricks bundle init`
C) Set up Databricks CLI with authentication
D) Create the workspace manually

Answer

Correct Answer: C
The Databricks CLI must be installed and authenticated (via `databricks/setup-cli` action or environment variables) before deployment commands work. The bundle already exists in the repo. Python is helpful but CLI is essential.

Question 7: OIDC Federation for CI/CD **Question** *(Hard)*: A security team requires that CI/CD pipelines authenticate to Databricks without storing long-lived tokens in GitHub Actions secrets. What is the recommended authentication approach?

A) Use OIDC federation with GitHub's identity provider so the pipeline exchanges a short-lived GitHub token for a Databricks access token
B) Generate a personal access token (PAT) and store it as an encrypted GitHub secret with automatic rotation
C) Use a service principal with a client secret stored in a vault and injected at runtime
D) Configure SSH key-based authentication between GitHub Actions runners and the Databricks workspace

Answer

Correct Answer: A
OIDC (OpenID Connect) federation allows GitHub Actions to authenticate to Databricks without storing any secrets. The pipeline receives a short-lived OIDC token from GitHub's identity provider and exchanges it for a Databricks token. This eliminates secret management entirely. PATs and client secrets still require secret storage and rotation. SSH authentication is not supported for Databricks API calls.

Question 8: Blue/Green Deployment **Question** *(Hard)*: A production pipeline processes financial data and cannot tolerate downtime during updates. The team needs a zero-downtime deployment strategy. How should a blue/green deployment be implemented with Declarative Automation Bundles?

A) Deploy the new version to the same target and rely on Databricks to handle the transition automatically

B) Maintain two separate DAB targets (blue and green), deploy the new version to the inactive target, run validation, then switch traffic
C) Use `databricks bundle deploy --rolling` to gradually replace running tasks
D) Deploy to a staging target, run all tests, then use `databricks bundle promote` to move to production

Answer

Correct Answer: B
Blue/green deployment uses two parallel environments. Deploy to the inactive one, validate, then switch traffic (e.g., by updating a job alias or routing). This provides instant rollback by switching back. Databricks doesn't auto-handle transitions. There is no `--rolling` or `promote` command in the DAB CLI.

Question 9: Bundle Variable Substitution **Question** *(Medium)*: A DAB project needs different cluster sizes, catalog names, and notification emails for dev vs. production targets. Which approach correctly parameterizes these differences?

A) Use shell environment variables like `$CLUSTER_SIZE` directly in databricks.yml
B) Create separate databricks.yml files for each environment (databricks-dev.yml, databricks-prod.yml)
C) Use Jinja2 templating syntax `{{ env.CLUSTER_SIZE }}` in the bundle configuration
D) Define variables in the bundle and override them per target using `variables:` blocks with `${var.variable_name}` syntax

Answer

Correct Answer: D
DAB supports native variable substitution with `${var.variable_name}` syntax. Variables are defined at the bundle level and can be overridden per target in the `targets:` section. Shell variables, separate files, and Jinja2 are not supported in DAB configuration syntax.

Question 10: Deployment Validation **Question** *(Easy)*: A deployment pipeline needs to verify that a bundle is correctly configured before deploying to production, checking for missing permissions, invalid references, and configuration errors. Which command performs this validation?

A) `databricks bundle test -t production`
B) `databricks bundle lint -t production`
C) `databricks bundle validate -t production`
D) `databricks bundle check -t production`

Answer

Correct Answer: C
`databricks bundle validate` checks the bundle configuration for errors including missing resources, invalid references, schema validation, and permission issues. It should always run before `databricks bundle deploy`. There is no `test`, `lint`, or `check` subcommand in the DAB CLI.

Question 11: GitOps Workflow **Question** *(Medium)*: A team implements GitOps where the Git repository is the single source of truth. Changes to production should only occur through merged pull requests. How should the CI/CD pipeline enforce this GitOps principle?

A) Allow direct deployments from any branch but require admin approval in the Databricks workspace
B) Configure the deployment pipeline to only trigger on merges to the main branch, with PR reviews and bundle validation as required checks
C) Use Git Folders to auto-sync the main branch to the production workspace
D) Lock the production workspace to read-only and require API-only deployments

Answer

Correct Answer: B
GitOps requires that all changes flow through Git. The CI/CD pipeline should only deploy from main branch (after PR merge), with validation and tests as required GitHub/GitLab checks before merge is allowed. Git Folders sync notebooks but don't handle full bundle deployment. Read-only workspaces break operational needs. Direct deployment from any branch bypasses review.

Question 12: Spark UI Stage Analysis **Question** *(Medium)*: A data engineer examines the Spark UI and notices one stage has 200 tasks where 199 complete in 2 seconds but 1 task takes 15 minutes. What is this pattern called and which AQE feature addresses it?

A) A straggler task; enable speculative execution with `spark.speculation = true`
B) Data skew; AQE's `spark.sql.adaptive.skewJoin.enabled` splits the skewed partition
C) A scheduling delay; increase `spark.executor.cores` for faster task scheduling
D) Garbage collection overhead; tune JVM GC with `spark.executor.extraJavaOptions`

Answer

Correct Answer: B
One task taking much longer than others with the same data is classic data skew. AQE's skew join optimization detects skewed partitions and splits them into smaller sub-partitions. Speculative execution re-runs slow tasks but doesn't fix the root cause. This isn't a scheduling or GC issue.

Question 13: Diagnosing on Serverless *(2026 blueprint)* **Question** *(Medium)*: A notebook task in a Lakeflow Jobs run on serverless compute became much slower after a code change, and the data engineer wants operator-level detail (time spent, rows, memory, pruning) for the slow statement. The compute page shows no Spark UI tab. Where should the engineer look?

A) The Driver logs tab on the serverless compute
B) The compute event log for the serverless compute
C) The query profile for the statement, opened from the task's timeline in the job run (or from query history)
D) Log delivery output in a Unity Catalog volume configured on the serverless compute

Answer

Correct Answer: C
The query profile replaces the Spark UI for serverless workloads.

Serverless compute does not provide the Spark UI or Spark logs; Databricks says to use the query profile, which shows per-operator time spent, rows, memory peak, and pruning, and serverless job runs list each statement in the task timeline with a link to its profile. A and D are wrong because Spark driver logs and compute log delivery are not available on serverless (only client-side application logs). B is wrong because compute event logs are not supported on serverless and only describe compute lifecycle, not query execution.

Question 14: Fleet-Wide Failure Analysis *(2026 blueprint)* **Question** *(Medium)*: A platform team must find every job task that failed or timed out across all workspaces in the last week, grouped by failure reason, without opening each run in the UI. Which source gives this most directly?

A) The Spark UI Jobs tab of each job cluster
B) The `system.lakeflow.job_task_run_timeline` system table, filtered on `result_state` and grouped by `termination_code`
C) The driver logs of each job cluster delivered to a volume
D) The `system.billing.usage` system table filtered on failed SKUs

Answer

Correct Answer: B
The `system.lakeflow.job_task_run_timeline` table records each task run's `result_state` and `termination_code` across the account.
System tables are governed, queryable records with 365-day retention, so one SQL query can count `FAILED`, `ERROR`, and `TIMED_OUT` task runs by `termination_code` (for example `RUN_EXECUTION_ERROR` or `CLUSTER_ERROR`). A is per-application and disappears when compute is restarted. C requires parsing raw log files per cluster and does not give outcomes. D tracks usage and cost, not run outcomes.

Question 15: What a Repair Re-runs *(2026 blueprint)* **Question** *(Medium)*: A job has tasks `ingest`, then `clean_a` and `clean_b` in parallel, then `publish` depending on both. In last night's run `ingest` and `clean_a` succeeded, `clean_b` failed, and `publish` was skipped. The engineer fixes the bug in `clean_b`'s notebook and clicks **Repair run** with default settings. What happens?

A) All four tasks run again in a new job run with a new run ID
B) Only `clean_b` runs again; `publish` must be triggered manually
C) `clean_b` and `publish` run again as part of the original run, using the current job and task settings
D) `ingest`, `clean_b`, and `publish` run again because repairs always restart from the first task

Answer

Correct Answer: C
A repair re-runs only the unsuccessful tasks and their dependents, inside the original run.
Repair re-runs the failed `clean_b` and the skipped dependent `publish`, does not re-run successful tasks (`ingest`, `clean_a`), uses the current job and task settings so the fix is picked up, and records the repair in the original run's history. A describes Run now, not repair. B ignores dependent tasks. D is wrong because successful tasks are not re-run.

Question 16: Repair with a Parameter Override via API *(2026 blueprint)* **Question** *(Hard)*: A run that was already repaired once failed again because its `run_date` job parameter was wrong. An orchestration script must repair all failed tasks again, this time with `run_date` set to `2026-10-25`, through `POST /api/2.2/jobs/runs/repair`. Which request body is correct?

A) `{"run_id": 455644833, "rerun_all_failed_tasks": true, "job_parameters": {"run_date": "2026-10-25"}}`
B) `{"run_id": 455644833, "latest_repair_id": 734650698524280, "rerun_all_failed_tasks": true, "job_parameters": {"run_date": "2026-10-25"}}`
C) `{"run_id": 455644833, "latest_repair_id": 734650698524280, "rerun_tasks": ["clean_b"], "rerun_all_failed_tasks": true}`
D) `{"job_id": 11223344, "job_parameters": {"run_date": "2026-10-25"}, "only": ["clean_b"]}`

Answer

Correct Answer: B
Option B supplies `latest_repair_id`, `rerun_all_failed_tasks`, and the `job_parameters` override.
`latest_repair_id` is not needed for the first repair but must be provided on later repairs of the same run, and `job_parameters` carries the override. A omits `latest_repair_id` for a second repair. C sets both `rerun_tasks` and `rerun_all_failed_tasks`, which are mutually exclusive, and has no override. D is a `run-now` request that starts a new run instead of repairing the existing one.

Question 17: Same Definitions in Every Environment *(2026 blueprint)* **Question** *(Easy)*: A team wants one job and one Lakeflow pipeline, stored in Git, deployed to dev, staging, and prod workspaces with a different catalog per environment and a service principal running production. Which approach fits?

A) Build the job in the dev UI, export its JSON, and import it into staging and prod
B) Define the resources once in a `databricks.yml` bundle with `targets` that override a `catalog` variable and set `run_as` on `prod`, then run `databricks bundle deploy -t <target>`
C) Keep the job in a Git folder and copy the folder to each workspace with the workspace import CLI
D) Use one shared workspace for all three environments to avoid configuration drift

Answer

Correct Answer: B
A Declarative Automation Bundle with per-target overrides deploys identical definitions to each environment.
Bundles declare resources as code, `targets` override variables per environment, `run_as` sets a production service principal, and `databricks bundle deploy -t <target>` deploys reproducibly from CI/CD. A is manual and drifts. C moves code only, not job or pipeline definitions. D removes environment isolation instead of managing it.

Question 18: Development Mode Behavior *(2026 blueprint)* **Question** *(Medium)*: An engineer deploys a bundle with `databricks bundle deploy -t dev`, where the dev target sets `mode: development`. The bundle's job has a daily schedule. What does the deployed job look like?

A) Same name as production, schedule active
B) Name prefixed with `[dev <short_name>]`, a dev tag added, and the schedule paused

C) Same name as production, schedule paused, and the job runs as a service principal automatically
D) Deployment fails because development mode requires `run_as`

Answer

Correct Answer: B
Development mode prefixes names, tags resources dev, and pauses schedules and triggers. `mode: development` prepends `[dev ${workspace.current_user.short_name}]` to resource names, adds a dev tag, pauses all schedules and triggers, marks pipelines as development, and disables the deployment lock. A and C keep the production name, which development mode changes. D confuses it with production mode, which is where `run_as` (a service principal) is recommended or required.

Question 19: Updating a Production Git Folder from CI *(2026 blueprint)* **Question** *(Medium)*: A team deploys only notebooks and Python files through a production Git folder at `/Workspace/Production/etl`. After each pull request merges into `main`, their GitHub Actions workflow must bring that folder to the latest commit of `main`. Which command should the workflow run, authenticated as a service principal that has a Git credential?

A) `databricks repos update /Workspace/Production/etl --branch main`
B) `databricks workspace import-dir ./src /Workspace/Production/etl --overwrite`
C) `databricks repos create https://github.com/org/etl.git gitHub --path /Workspace/Production/etl`
D) `databricks bundle destroy -t prod` followed by `databricks bundle deploy -t prod`

Answer

Correct Answer: A
`databricks repos update ... --branch main` checks out `main` and pulls its latest commit into the Git folder.
The Repos API update (`PATCH /api/2.0/repos/{id}`) switches a Git folder to a branch or tag or updates it to the latest commit on the same branch, which is the documented automation for a production Git folder. B bypasses Git and turns the folder's contents into untracked copies. C clones a new Git folder and fails if the path already exists. D tears down bundle resources and is unrelated to updating a Git folder.

Question 20: Git Folders vs Bundles *(2026 blueprint)* **Question** *(Medium)*: A team uses personal Git folders for development. They now need CI/CD that deploys both the notebook code and the job definitions (schedules, clusters, task graph) to production. What does Databricks recommend?

A) A production Git folder only, because pulling the branch also updates job definitions
B) Declarative Automation Bundles, with the jobs defined in source and deployed by `databricks bundle deploy`
C) Editing the production jobs in the UI after each Git folder pull
D) Storing job JSON in the Git folder and relying on Databricks to apply it automatically

Answer

Correct Answer: B
Databricks recommends Declarative Automation Bundles to package and deploy CI/CD workflows.
Bundles deploy code plus jobs and pipelines as code, while a production Git folder deploys code only. A is wrong because updating a Git folder does not change job definitions. C is manual and error-prone. D is wrong because Databricks does not apply job JSON files from a Git folder automatically.

Section 9: Data Modeling (5 %)

Practice questions for Section 9 of the Oct 9, 2026 exam guide. Questions marked *(2026 blueprint)* target skills that are new or reworded in that guide. Study material: Section 9 folder.

Question 1: Medallion Architecture **Question** *(Easy)*: A data architect is designing a lakehouse. Raw JSON data needs to be ingested, cleaned, and then aggregated. In which layer should data type casting and null handling occur?

A) Bronze layer
B) Silver layer
C) Gold layer
D) Landing zone

Answer

Correct Answer: B
The Silver layer is for cleaned, conformed data including type casting, null handling, deduplication, and standardization. Bronze stores raw data as-is. Gold is for business-level aggregations. Landing zone is temporary staging.

Question 2: Schema Evolution **Question** *(Easy)*: A streaming pipeline receives data with a new column. The target Delta table should automatically accept this change. Which option enables this?

A) `.option("mergeSchema", "true")`
B) `.option("overwriteSchema", "true")`
C) `.option("schemaEvolution", "true")`
D) Schema evolution is automatic in Delta

Answer

Correct Answer: A
`mergeSchema = true` allows adding new columns during writes. `overwriteSchema` replaces the entire schema (dangerous). Option C is not a valid option. Schema evolution must be explicitly enabled.

Question 3: SCD Type 2 **Question** *(Easy)*: A dimension table needs to track full history of changes with effective dates. Which columns are typically added for SCD Type 2?

A) `is_current`, `version`
B) `start_date`, `end_date`, `is_current`
C) `created_at`, `updated_at`
D) `previous_value`, `current_value`

Answer

Correct Answer: B
SCD Type 2 tracks history using `start_date` (when version became active), `end_date` (when superseded, NULL for current), and often `is_current` flag. This allows point-in-time queries and current state queries.

Question 4: Partitioning Best Practices **Question** *(Hard)*: A table has 100 million rows with a `status` column containing 5 distinct values and a `customer_id` column with 1 million distinct values. Which partitioning strategy is correct?

A) Partition by `customer_id`
B) Partition by `status`
C) Partition by both `customer_id` and `status`
D) Don't partition, use Z-ORDER on `customer_id`

Answer

Correct Answer: D
Partitioning works best for low-cardinality columns with ~1000 or fewer values. `customer_id` has too many values (would create 1M partitions). `status` has too few values (only 5 partitions, may not help much). Z-ORDER or liquid clustering on high-cardinality columns is better.

Question 5: One Definition, Any Grouping *(2026 blueprint)* **Question** *(Medium)*: Finance defines average order value as net revenue divided by the number of distinct orders. Analysts query it from AI/BI dashboards, Genie Agents, and Power BI, grouped by region in one report and by month and customer segment in another. Some reports currently average pre-aggregated daily ratios and get wrong numbers. Which design gives one governed definition that stays correct at any grouping?

A) A Unity Catalog metric view over `fact_sales` with atomic measures for net revenue and order count, and `avg_order_value` defined as `MEASURE(net_revenue) / MEASURE(order_count)`
B) A materialized view that stores average order value by day, region, and segment
C) A standard view that computes `AVG(order_value)` grouped by region
D) A Python UDF that each notebook imports to compute the ratio

Answer

Correct Answer: A
A metric view defines measures once and lets consumers group by any field. The engine computes the ratio from the atomic measures at the requested grain.
Metric views separate measure definitions from fields, so a composed measure like `MEASURE(net_revenue) / MEASURE(order_count)` re-aggregates correctly for any `GROUP BY` and is governed in Unity Catalog for every tool. A materialized view (B) fixes the grain, and aver-

aging stored ratios across rows gives the wrong answer. A standard view (C) fixes its grouping and uses a different formula. A UDF copied into notebooks (D) isn't governed and doesn't reach dashboards or BI tools.

Question 6: Enabling Incremental Materialized View Refresh *(2026 blueprint)* **Question** *(Hard)*: A Gold materialized view, refreshed on serverless, pre-aggregates the Delta tables `fact_sales` and `dim_product` for dashboards. Neither source table has row tracking enabled. Every refresh shows a `FULL_RECOMPUTE` technique in the event log, and refresh cost keeps growing with table size. The query uses only `SUM` over `DECIMAL` columns, `GROUP BY`, and an inner join between the two tables. Which change most directly allows incremental refresh?

A) Add a `customer_key BIGINT GENERATED ALWAYS AS IDENTITY` column to the materialized view
B) Cast the `DECIMAL` amounts to `DOUBLE` before summing
C) Enable row tracking on the source tables and set `REFRESH POLICY INCREMENTAL STRICT` so a non-incremental plan fails instead of recomputing
D) Convert the materialized view to a standard view so it's computed at query time

Answer

Correct Answer: C
Incremental refresh needs supported operations, serverless refresh, and row tracking on the Delta sources. `INCREMENTAL STRICT` makes any remaining blocker fail loudly instead of silently recomputing.
The docs say source Delta tables should have row tracking enabled to support incremental refresh, and `REFRESH POLICY INCREMENTAL STRICT` fails the refresh rather than falling back to a full recompute. That makes the cause visible and stops the cost growth. Materialized views don't support identity columns or surrogate keys (A). `SUM` over `FLOAT` or `DOUBLE` forces a full refresh, so casting to `DOUBLE` (B) makes things worse. A standard view (D) removes the pre-computation the dashboards depend on.

Question 7: Fixing an Over-Partitioned Table *(2026 blueprint)* **Question** *(Medium)*: A 600 GB Delta table is partitioned by `customer_id` and has millions of small files. Queries filter on `customer_id` and on `event_date`. The workspace runs Databricks Runtime 18.1. The engineer wants better data skipping and balanced file sizes without a manual CTAS rebuild. What should the engineer do?

A) Add a generated `event_month` column and repartition the table by `(event_month, customer_id)`
B) Keep the partitions and run `OPTIMIZE ... ZORDER BY (event_date)` in each partition
C) Lower `delta.targetFileSize` to 16 MB so each partition writes smaller files
D) Run `ALTER TABLE ... REPLACE PARTITIONED BY WITH CLUSTER BY (customer_id, event_date)` and then `OPTIMIZE`

Answer

Correct Answer: D
Converting in place to liquid clustering on the filter columns, then running `OPTIMIZE`, fixes both data skipping and file sizes.
The docs say not to partition tables under 1 TB at all, and partitioning by a high-cardinality column creates the small-file problem described here. In DBR 18.1+, `REPLACE PARTITIONED BY WITH CLUSTER BY` converts the table without a full manual rebuild, and `OPTIMIZE` then

clusters and compacts the data. Option A keeps a partitioned layout below the recommended size and still partitions on a high-cardinality column. Option B leaves millions of small partitions in place, because Z-order can't combine files across partition boundaries. Option C makes the small-file problem worse.

Question 8: Clustering a Fact Table for Star-Schema Joins *(2026 blueprint)* **Question** *(Hard)*: A 30 TB unpartitioned fact table, `fact_sales`, is joined to a small filtered `dim_product` table on `product_key`, and most queries also filter on `sale_date`. Query profiles show that dynamic file pruning is applied but still scans most fact files, because `product_key` values are spread across every file. Which layout change best aligns the table with this access pattern?

A) Partition `fact_sales` by `product_key`
B) Enable liquid clustering on `fact_sales` with `CLUSTER BY (sale_date, product_key)`, then run `OPTIMIZE ... FULL`
C) Set `delta.targetFileSize` to 32 MB so there are more files to skip
D) Disable predictive optimization and schedule `VACUUM` every hour

Answer

Correct Answer: B
Liquid clustering on the filter column and the join key puts matching rows in fewer files, so dynamic file pruning can skip more of them. `OPTIMIZE FULL` reclusters the existing data.
The dynamic file pruning docs say its effect depends on how data is clustered, and recommend liquid clustering on frequently joined columns. Clustering keys should come from the most common filter and join columns, up to four keys. Partitioning on a high-cardinality join key (A) causes over-partitioning and small files, and the docs prefer liquid clustering below 100 TB anyway. Shrinking files (C) works against the autotuned 1 GB target for a table this size and adds metadata overhead. `VACUUM` (D) removes unreferenced files and doesn't change data layout.

Mock Exam 1 — Databricks Data Engineer Professional

Simulate the real exam under timed conditions before your test date.

Important

Relabelled for the Oct 9, 2026 exam guide. Every question now carries its section from the 9-section guide, and product names follow the guide (Lakeflow Pipelines, AUTO CDC, Declarative Automation Bundles, OpenSharing). The questions themselves were written for earlier guides, so the section mix below is not the official weighting — the real exam has 60 scored questions. Skills new in 2026 are drilled in the practice questions.

Exam Instructions

Detail	Value
Total Questions	63
Time Limit	120 minutes
Result	Pass / fail (the Oct 9, 2026 exam guide publishes no numeric passing score)
Format	Scenario-based, single correct answer

Domain Distribution (labelled to the Oct 9, 2026 exam guide)

Exam section	Official weight	Questions in this mock
Developing Code for Data Processing using Python and SQL	23 %	14
Data Ingestion & Acquisition	12 %	5
Data Manipulation	12 %	5
Monitoring and Alerting	10 %	6
Cost & Performance Optimization	15 %	10
Ensuring Data Security and Compliance	8 %	5
Data Governance	5 %	4
Debugging and Deploying	10 %	7
Data Modeling	5 %	7

Scoring Guide

Score	Result
50–63 (≥ 80 %)	Excellent
44–49 (70–79 %)	Likely passing
38–43 (60–69 %)	Close — focused study needed
Below 38 (< 60 %)	Comprehensive review needed

> **Note**
>
> Databricks does not publish a numeric passing threshold for the current exam guide.

After the exam — debrief

When you've finished the timed sitting, see `debrief.md` for the domain-to-resource map, the per-section question breakdown, and a study plan tailored to your miss count.

Mock Exam — Databricks Data Engineer Professional

63 scenario-based questions, each labelled with its Oct 9, 2026 exam section. Attempt all questions before revealing answers.

Total: 63 questions | Time: 120 minutes | Self-check target: 70 % (44/63) — Databricks publishes no passing score

← Back to Mock Exam

Data Ingestion & Acquisition (Question 1)

Question 1 ***(Easy)*** **Scenario**: A data engineering team is building an ingestion pipeline that reads JSON files from cloud storage. The source system occasionally adds new fields to the JSON schema without notice. The team needs to automatically capture all fields while maintaining schema consistency in the Delta table.

Question: Which Auto Loader configuration will automatically detect and add new columns to the target Delta table?

A) `cloudFiles.inferColumnTypes = true` B) `cloudFiles.format = json` with `rescuedDataColumn` C) `cloudFiles.schemaLocation` with `mergeSchema = true` D) `cloudFiles.schemaEvolutionMode = addNewColumns`

Answer

Correct Answer: D
`cloudFiles.schemaEvolutionMode = addNewColumns` is the Auto Loader configuration that automatically detects and adds new columns to the target schema. Option A only controls type inference, not schema evolution. Option C uses `schemaLocation` for storing schema but `mergeSchema` is a Delta write option, not Auto Loader. Option B captures unparseable data but doesn't handle schema evolution.

Cost & Performance Optimization (Question 2)

Question 2 ***(Medium)*** **Scenario**: A streaming pipeline processes clickstream data with an average of 50,000 events per second. The pipeline writes to a Delta table that downstream analysts query frequently. The team notices query performance degrading due to many small files.

Question: Which trigger configuration balances latency and file size for this workload?

A) `trigger(availableNow=True)` B) `trigger(processingTime="1 second")` C) `trigger(processingTime="1 minute")` D) `trigger(once=True)`

Answer

Correct Answer: C
A 1-minute processing time trigger allows sufficient data to accumulate for larger file writes while maintaining reasonable latency. Option A processes all available data at once (batch-like). Option B writes too frequently, creating small files. Option D is for one-time batch processing, not continuous streaming.

Data Modeling (Question 3)

Question 3 ***(Hard)*** **Scenario**: A data engineer needs to implement a pipeline that captures both current and historical versions of customer records. When a customer's address changes, the old record should be marked as inactive with an end date, and a new active record should be inserted.

Question: Which approach correctly implements this SCD Type 2 pattern using Delta Lake?

A) Use `MERGE INTO` with `WHEN MATCHED AND target.is_current = true THEN UPDATE` to close the old record, then `INSERT` the new record B) Use `MERGE INTO` with `WHEN MATCHED THEN UPDATE` to update the existing record C) Use `INSERT OVERWRITE` to replace all records for the customer D) Use `DELETE` followed by `INSERT` to replace the customer record

Answer

Correct Answer: A
SCD Type 2 requires closing the current record (setting end_date and is_current=false) and inserting a new record. The MERGE statement with a condition on `is_current = true` ensures only the active record is updated. Options B and D implement SCD Type 1 (overwrite). Option C would lose history.

Data Ingestion & Acquisition (Question 4)

Question 4 ***(Medium)*** **Scenario**: A pipeline uses Auto Loader to ingest JSON files. The data engineer wants to ensure that the inferred schema is persisted and reused across stream restarts, and that any data not matching the schema is captured for later analysis rather than causing failures.

Question: Which Auto Loader configuration achieves both requirements?

A) Set `cloudFiles.schemaLocation` and `cloudFiles.schemaEvolutionMode = rescue` B) Set `cloudFiles.inferColumnTypes = true` and `badRecordsPath` C) Set `cloudFiles.schemaHints` with all expected columns D) Set `cloudFiles.format = json` with `mode = PERMISSIVE`

Answer

Correct Answer: A
`cloudFiles.schemaLocation` persists the inferred schema to a directory so it's reused across restarts. `schemaEvolutionMode = rescue` captures data that doesn't match the schema into a `_rescued_data` column instead of failing. Option B's `badRecordsPath` is for batch reads. Option C provides hints but doesn't persist schema or rescue data. Option D uses invalid configuration syntax.

Developing Code for Data Processing using Python and SQL (Question 5)

Question 5 ***(Medium)*** **Scenario**: A streaming pipeline processes transaction data and must ensure exactly-once semantics when writing to a Delta table. The pipeline occasionally fails and restarts from checkpoints.

Question: Which combination ensures exactly-once processing guarantees?

A) `outputMode("append")` with checkpointing enabled B) `outputMode("complete")` with `foreachBatch()` and manual deduplication C) `outputMode("append")` with checkpointing and idempotent Delta writes D) `outputMode("update")` with watermarking

Answer

Correct Answer: C
Exactly-once semantics require checkpointing (for tracking progress) combined with idempotent writes (Delta Lake's transaction log ensures writes are atomic and can be retried safely). Option A lacks idempotency mention. Option B requires manual handling. Option D is for stateful aggregations.

Cost & Performance Optimization (Questions 6–7)

Question 6 ***(Medium)*** **Scenario**: A data engineer is building a Change Data Feed (CDF) pipeline to propagate changes from a bronze table to silver. The bronze table has CDF enabled. The engineer needs to read only the changes since the last pipeline run.

Question: Which code correctly reads incremental changes from the CDF-enabled table?

A) `spark.read.format("delta").option("readChangeFeed", "true").table("bronze")` B) `spark.readStream.format("delta").option("ignoreChanges", "true").table("bronze")` C) `spark.read.format("delta").option("startingVersion", last_version).table("bronze")` D) `spark.readStream.format("delta").option("readChangeFeed", "true").table("bronze")`

Answer

Correct Answer: D
Using `readStream` with `readChangeFeed = true` creates a streaming source that automatically tracks progress and reads only new changes. Option A is batch read (would need version parameters). Option C reads snapshots, not change records. Option B ignores changes rather than reading them.

Question 7 ***(Medium)*** **Scenario**: A MERGE operation is taking longer than expected when updating a large Delta table with 500 million rows. The merge condition uses `customer_id` which is not the partition column. Analysis shows the merge scans nearly all files in the table.

Question: What is the most effective optimization for this MERGE operation?

A) Increase `spark.sql.shuffle.partitions` to 1000 B) Add a Z-ORDER on `customer_id` C) Partition the table by `customer_id` D) Enable `spark.databricks.delta.merge.enableLowShuffle`

Answer

Correct Answer: B
Z-ORDER on `customer_id` co-locates data with similar customer_id values, enabling file pruning during MERGE and dramatically reducing files scanned. Option A adds parallelism but doesn't reduce data scanned. Option C is impractical for high-cardinality columns. Option D is not a valid configuration.

Developing Code for Data Processing using Python and SQL (Question 8)

Question 8 ***(Medium)*** **Scenario**: A streaming pipeline uses watermarking to handle late-arriving data. The watermark is set to 10 minutes. A data quality check shows that some events arriving 15 minutes late are being dropped.

Question: What is the expected behavior and how should the engineer address this?

A) Increase the watermark to 20 minutes to capture more late data B) This is expected behavior; events beyond the watermark are intentionally dropped C) Set `spark.sql.streaming.stateStore.providerCl` to handle late data D) Use `outputMode("complete")` to include all data

Answer

Correct Answer: B
Watermarking intentionally drops data arriving after the threshold to bound state store size. Events 15 minutes late with a 10-minute watermark will be dropped by design. Option A could work but increases state size. Options C and D don't address late data handling. The engineer should evaluate if 10 minutes is the right threshold for their use case.

Data Ingestion & Acquisition (Question 9)

Question 9 *(Medium)* **Scenario**: A data engineer needs to implement a pipeline that reads from multiple Kafka topics with different schemas. Each topic should be written to its own Delta table while maintaining streaming progress.

Question: Which approach correctly handles multiple topics with independent schemas?

A) Read all topics in one stream and use `filter()` to split by topic before writing B) Use `foreachBatch()` to route records to different tables based on topic C) Create separate streaming queries for each topic with independent checkpoints D) Use `readStream.format("kafka").option("subscribe", "topic1,topic2")` with schema registry

Answer

Correct Answer: C
Separate streaming queries with independent checkpoints provide isolation for different schemas and independent failure handling. Option A requires a common schema. Option B complicates checkpoint management. Option D subscribes to multiple topics but doesn't handle different schemas well.

Ensuring Data Security and Compliance (Question 10)

Question 10 *(Easy)* **Scenario**: A table uses `DELETE` operations to remove records that are older than 90 days for compliance. After running deletes for several weeks, the table's storage size hasn't decreased despite fewer rows.

Question: What must be done to reclaim storage space after DELETE operations?

A) Run `OPTIMIZE` to compact files B) Recreate the table using `CREATE TABLE AS SELECT` C) Run `FSCK REPAIR TABLE` to fix storage D) Run `VACUUM` with an appropriate retention period

Answer

Correct Answer: D
DELETE in Delta Lake marks records as removed but doesn't physically delete files (to support time travel). VACUUM removes files older than the retention period that are no longer referenced. Option A compacts but doesn't remove old files. Option C checks table integrity. Option B is inefficient.

Developing Code for Data Processing using Python and SQL (Question 11)

Question 11 *(Medium)* **Scenario**: A streaming pipeline aggregates hourly sales totals. The source data sometimes arrives out of order, with events from previous hours arriving in the current batch. The pipeline uses `groupBy(window(timestamp, "1 hour"))`.

Question: Which configuration ensures accurate aggregations while bounding state store growth?

A) Set `outputMode("complete")` to recompute all windows B) Add `withWatermark("timestamp", "2 hours")` before the aggregation C) Use `outputMode("append")` without watermarking D) Set `spark.sql.streaming.stateStore.maintenanceInterval` to ”1 hour”

Answer

Correct Answer: B
Watermarking allows Spark to handle late data within the threshold (2 hours) while cleaning up state for windows that can no longer receive updates. Option A recomputes but doesn't bound state. Option C would drop late data immediately. Option D controls maintenance frequency, not late data handling.

Debugging and Deploying (Question 12)

Question 12 *(Hard)* **Scenario**: A data engineer is debugging a streaming pipeline that stopped processing data. The query shows `ACTIVE` status but the `processedRowsPerSecond` metric shows 0. New files are continuously arriving in the source location.

Question: What is the most likely cause of this issue?

A) The checkpoint location is full B) Auto Loader's `cloudFiles.maxFilesPerTrigger` is set too low C) The source files are being written to a different path than configured D) The stream is waiting for the trigger interval

Answer

Correct Answer: C
A stream showing ACTIVE with 0 rows processed while files are arriving typically indicates the configured source path doesn't match where files are being written. Option A would cause errors, not zero processing. Option B would limit throughput but not stop it entirely. Option D wouldn't show persistent zero processing.

Data Manipulation (Question 13)

Question 13 *(Hard)* **Scenario**: A MERGE operation needs to handle duplicate keys in the source data. The business rule states that when duplicates exist, the record with the latest timestamp should be used.

Question: Which approach correctly handles this deduplication requirement?

A) Use `dropDuplicates()` on the source DataFrame before MERGE B) Use multiple MERGE statements with different conditions C) Set `spark.databricks.delta.merge.repartitionBeforeWrite.enabled = true` D) Deduplicate within the MERGE using `ROW_NUMBER()` in a subquery

Answer

Correct Answer: D
Using `ROW_NUMBER()` partitioned by key and ordered by timestamp DESC in a subquery, then filtering for row_number = 1, deduplicates source data correctly within the MERGE while ensuring the latest record is kept. Option A (`dropDuplicates()`) is non-deterministic and doesn't guarantee keeping the record with the latest timestamp. Option C affects write performance, not deduplication. Option B is unnecessarily complex.

Data Ingestion & Acquisition (Question 14)

Question 14 *(Easy)* **Scenario**: A pipeline processes IoT sensor data using Auto Loader. The sensors occasionally send malformed JSON that causes parsing errors. The team wants to capture bad records for analysis without failing the pipeline.

Question: Which configuration captures malformed records while allowing the pipeline to continue?

A) Set `cloudFiles.ignoreCorruptFiles = true` B) Set `badRecordsPath` to a storage location C) Use `cloudFiles.rescuedDataColumn` to capture unparseable data D) Wrap the read in a try-catch block

Answer

Correct Answer: C
`rescuedDataColumn` creates a column containing data that couldn't be parsed according to the schema, allowing analysis of bad records. Option A ignores corrupt files entirely. Option B is for batch operations. Option D doesn't work with streaming transformations.

Cost & Performance Optimization (Question 15)

Question 15 *(Medium)* **Scenario**: A batch ETL job reads from a Delta table and writes to another Delta table. The job runs daily and should only process records that changed since the last run. The source table has Change Data Feed enabled.

Question: Which code pattern correctly implements incremental processing using CDF?

A) Read with `option("readChangeFeed", "true").option("startingVersion", last_processed_version)` B) Read with `option("versionAsOf", last_processed_version)` and compare with current version C) Use `CHANGES` table-valued function in SQL D) Read the `_delta_log` directly to find changed files

Answer

Correct Answer: A
Reading CDF with `startingVersion` returns all changes (inserts, updates, deletes) since that version, including `_change_type` column. Option B reads snapshots, not changes. Option C is not valid syntax. Option D is fragile and not recommended.

Data Manipulation (Questions 16–17)

Question 16 *(Easy)* **Scenario**: A streaming pipeline writes to a Delta table that is also queried by a BI dashboard. Users report seeing inconsistent results during pipeline runs, with some queries returning partial batches.

Question: How does Delta Lake ensure consistent reads during concurrent writes?

A) Delta uses table-level locks to prevent reads during writes B) Delta uses snapshot isolation; readers see consistent snapshots regardless of concurrent writes C) Delta queues read requests until writes complete D) Configure `delta.isolationLevel = serializable` to ensure consistency

Answer

Correct Answer: B
Delta Lake provides snapshot isolation by default. Readers see a consistent snapshot based on the transaction log at query start time, regardless of concurrent writes. Option A is incorrect; Delta doesn't use table locks. Option C is incorrect; reads aren't queued. Option D is not a valid setting.

Question 17 *(Medium)* **Scenario**: A data engineer needs to implement conditional updates in a MERGE operation. Records should only be updated if the source timestamp is newer than the target timestamp. Otherwise, the source record should be ignored.

Question: Which MERGE clause correctly implements this conditional update logic?

A) `WHEN MATCHED AND source.timestamp > target.timestamp THEN UPDATE SET *` B) `WHEN MATCHED THEN UPDATE SET * WHERE source.timestamp > target.timestamp` C) `WHEN MATCHED THEN UPDATE SET target.* = source.* IF source.timestamp > target.timestamp` D) `WHEN MATCHED THEN UPDATE SET * HAVING source.timestamp > target.timestamp`

Answer

Correct Answer: A
The condition in `WHEN MATCHED AND condition` filters which matched rows receive the update. Rows not meeting the condition are skipped. Option B has invalid syntax (WHERE after UPDATE SET). Options C and D use invalid syntax.

Developing Code for Data Processing using Python and SQL (Questions 18–20)

Question 18 *(Hard)* **Scenario**: A streaming pipeline uses `foreachBatch()` to write to multiple Delta tables. The engineer wants to ensure atomicity across all writes within each batch--either all tables are updated or none are.

Question: How can the engineer achieve atomic writes across multiple tables in `foreachBatch()`?

A) Wrap all writes in a single Spark transaction using `spark.sql("BEGIN TRANSACTION")` B) Use Delta Lake's multi-table transactions with `spark.databricks.delta.multiTableTransaction.enabled` C) This cannot be achieved directly; implement idempotent writes with manual rollback logic D) Use `foreachPartition()` instead for transactional guarantees

Answer

Correct Answer: C
Delta Lake doesn't support multi-table transactions. The recommended approach is to make each write idempotent (using merge keys or batch IDs) so failed batches can be safely reprocessed. Options A and B describe non-existent features. Option D doesn't provide multi-table atomicity.

Question 19 *(Medium)* **Scenario**: A data engineer is creating a notebook that should accept runtime parameters for the environment (dev, staging, prod) and a date range. The notebook will be called both interactively during development and by a scheduled job.

Question: Which approach correctly implements parameterization for both use cases?

A) Use `dbutils.widgets.text()` with defaults that can be overridden by job parameters B) Use environment variables set at cluster startup C) Use `spark.conf.get()` to read parameters from cluster configuration D) Hard-code values and create separate notebooks for each environment

Answer

Correct Answer: A
`dbutils.widgets` creates parameters with defaults for interactive use that can be overridden when called from jobs using the notebook task parameters. Option B requires cluster configuration. Option C requires cluster-level settings. Option D is unmaintainable.

Question 20 ***(Medium)*** **Scenario**: A notebook needs to execute SQL queries and store the results in a Python variable for further processing. The data engineer wants to use the most appropriate magic command.

Question: Which approach correctly captures SQL results in a Python variable?

A) Use `%sql` and reference the `_sqldf` variable B) Use `%sql` with `INTO :variable_name` syntax C) Use `spark.sql()` and assign to a variable D) Use `%run` to execute a SQL notebook and capture results

Answer

Correct Answer: C
`spark.sql("SELECT ...")` returns a DataFrame that can be assigned to a variable for further processing in Python. This is the recommended programmatic approach. Option A's `_sqldf` does exist in Databricks notebooks (it captures the last %sql result) but is implicit and less reliable for production code. Option B has invalid syntax. Option D doesn't capture SQL results.

Ensuring Data Security and Compliance (Question 21)

Question 21 ***(Easy)*** **Scenario**: A data engineer needs to securely access a third-party API from a Databricks notebook. The API requires an authentication token that should not be stored in code or version control.

Question: Which approach correctly implements secure credential management?

A) Store the token in a notebook cell and delete it after running B) Store the token in a Delta table with restricted access C) Use environment variables on the cluster D) Use `dbutils.secrets.get(scope="api_scope", key="api_token")`

Answer

Correct Answer: D
Databricks Secrets provides secure credential storage with access control. Secrets are encrypted at rest and redacted in logs. Option A exposes secrets in notebook history. Option B is not designed for secrets. Option C requires cluster admin access and isn't as secure.

Developing Code for Data Processing using Python and SQL (Question 22)

Question 22 *(Easy)* **Scenario**: A team maintains a library of shared utility functions used across multiple notebooks. When the library is updated, all notebooks using it should automatically get the latest version without modification.

Question: Which approach provides centralized code sharing with automatic updates?

A) Copy utility functions into each notebook B) Use `%run /Workspace/Shared/utilities` at the start of each notebook C) Package utilities as a wheel file and install via `%pip install` D) Store utilities in a Delta table and read them at runtime

Answer

Correct Answer: B
`%run` executes another notebook in the current notebook's context, making all its functions available. Changes to the shared notebook are immediately available to all callers. Option A requires manual updates. Option C requires reinstallation for updates. Option D is impractical for code.

Debugging and Deploying (Question 23)

Question 23 *(Medium)* **Scenario**: A data engineer is configuring the Databricks CLI for a CI/CD pipeline. The pipeline runs in a containerized environment where storing credentials in `~/.databrickscfg` is not practical.

Question: Which authentication method is most appropriate for this CI/CD scenario?

A) OAuth user-to-machine authentication B) Personal access token stored in `DATABRICKS_TOKEN` environment variable C) Service principal with `DATABRICKS_CLIENT_ID` and `DATABRICKS_CLIENT_SECRET` D) Username and password authentication

Answer

Correct Answer: C
Service principals with OAuth are the recommended approach for CI/CD automation. They provide machine identity without personal credentials and can be scoped with specific permissions. Option A requires user interaction. Option B uses personal tokens that expire. Option D is deprecated.

Monitoring and Alerting (Question 24)

Question 24 *(Medium)* **Scenario**: A REST API call to create a cluster returns successfully with a cluster_id. However, a subsequent API call to run a job on that cluster fails with ”Cluster not found.”

Question: What is the most likely cause and solution?

A) The cluster creation is asynchronous; poll the cluster status until it's RUNNING B) The API token doesn't have permission to access the cluster C) The cluster_id was not properly formatted in the job request D) Rate limiting is preventing the second API call

Answer

Correct Answer: A
Cluster creation via API returns immediately with a cluster_id, but the cluster takes time to provision. The job API call failed because the cluster wasn't ready yet. The solution is to poll `GET /api/2.0/clusters/get` until state is RUNNING. Options B-D are possible but less likely given the described sequence.

Developing Code for Data Processing using Python and SQL (Question 25)

Question 25 ***(Medium)*** **Scenario**: A team is deciding between all-purpose clusters and job clusters for their production ETL workloads that run on a schedule four times daily.

Question: Which recommendation optimizes for cost while maintaining reliability?

A) Use all-purpose clusters with auto-termination to save costs B) Use a shared all-purpose cluster across all jobs to maximize utilization C) Use serverless compute for all jobs regardless of workload characteristics D) Use job clusters which are automatically created and terminated per job run

Answer

Correct Answer: D
Job clusters are created for each job run and terminated afterward, providing cost savings (lower DBU rates) and isolation between runs. Option A still has higher DBU rates. Option B risks resource contention and failures affecting multiple jobs. Option C may not be cost-effective for predictable scheduled workloads.

Data Governance (Question 26)

Question 26 ***(Medium)*** **Scenario**: A data engineer needs to list all files in a Unity Catalog volume and filter for Parquet files created in the last 24 hours.

Question: Which approach correctly accesses Unity Catalog volumes?

A) `dbutils.fs.ls("dbfs:/Volumes/catalog/schema/volume")` B) `dbutils.fs.ls("/Volumes/catalog/sch`
C) `spark.read.format("binaryFile").load("dbfs:/Volumes/catalog/schema/volume")` D) `os.listdir("/dbfs/Volumes/catalog/schema/volume")`

Answer

Correct Answer: B
Unity Catalog volumes are accessed via the `/Volumes` path without the `dbfs:` prefix. Option A uses the wrong path format. Option C reads file content, not metadata. Option D may work but `dbutils.fs` is the recommended approach for Databricks.

Developing Code for Data Processing using Python and SQL (Questions 27–28)

Question 27 *(Easy)* **Scenario**: A notebook widget is created with `dbutils.widgets.dropdown("env", "dev", ["dev", "staging", "prod"])`. When the notebook runs as part of a job, the job needs to override this value to ”prod”.

Question: How should the job task be configured to pass the parameter?

A) In the notebook task configuration, set `base_parameters: {"env": "prod"}` B) Set cluster environment variable `ENV=prod` C) Use `spark.conf.set("env", "prod")` in the job configuration D) Create a separate notebook for production without widgets

Answer

Correct Answer: A
The notebook task's `base_parameters` map directly to widget values, overriding defaults when the notebook runs as a job. Option B doesn't connect to widgets. Option C sets Spark config, not widget values. Option D is unmaintainable.

Question 28 *(Medium)* **Scenario**: A data engineer is using the Jobs API 2.1 to submit a job run. The job includes a notebook task that accepts parameters. The engineer wants to pass dynamic values based on the current date.

Question: Which API endpoint and request body correctly submits this job run?

A) POST `/api/2.1/jobs/create` with `notebook_params` in the request B) PUT `/api/2.1/jobs/update` with `notebook_params` in the settings C) POST `/api/2.1/jobs/runs/submit` with `notebook_params` in the task configuration D) POST `/api/2.1/jobs/run-now` with `notebook_params` in the request

Answer

Correct Answer: D
`run-now` triggers an existing job with parameter overrides via `notebook_params`. Option A creates a job definition but doesn't run it. Option C submits a one-time run (requires full job spec). Option B updates job definition, doesn't run it.

Data Governance (Question 29)

Question 29 *(Medium)* **Scenario**: A team is migrating from DBFS mounts to Unity Catalog external locations. Their existing code uses paths like `dbfs:/mnt/data/bronze/`.

Question: What is the recommended migration approach for accessing cloud storage with Unity Catalog?

A) Update all paths to use `abfss://` or `s3://` URLs directly B) Create external locations in Unity Catalog and use `/Volumes/` paths C) Keep using mounts but register them as external locations D) Use storage credentials to access cloud storage via `dbutils.fs`

Answer

Correct Answer: B
Unity Catalog volumes provide governed access to cloud storage with paths like `/Volumes/catalog/schema/volume/`. External locations and volumes replace the ungoverned mount pattern. Option A bypasses governance. Option C mixes patterns incorrectly. Option D doesn't

leverage Unity Catalog.

Monitoring and Alerting (Question 30)

Question 30 *(Medium)* **Scenario**: A data engineer needs to run a SQL query from the Databricks CLI and output the results as JSON for processing by another script.

Question: Which CLI command correctly executes a SQL query and formats output as JSON?

A) `databricks sql execute --query "SELECT * FROM table" --format JSON` B) `databricks workspace execute-sql --statement "SELECT * FROM table" --output json` C) `databricks sql-cli -e "SELECT * FROM table" -o json` D) `databricks api post /api/2.0/sql/statements -d '{"statement": "SELECT * FROM table"}'`

Answer

Correct Answer: D
The Databricks CLI can make arbitrary API calls. The SQL Statement Execution API (`/api/2.0/sql/statements`) executes SQL queries programmatically. Options A-C use non-existent CLI commands. The actual implementation may vary by CLI version, but the API approach always works.

Data Modeling (Questions 31–34)

Question 31 *(Easy)* **Scenario**: A retail company is implementing the medallion architecture. Raw point-of-sale data arrives as JSON files with nested structures containing transaction details and line items.

Question: Which layer should handle flattening the nested JSON structure into a relational format?

A) Bronze layer - ingest and flatten immediately B) Gold layer - create denormalized tables for analytics C) Silver layer - transform raw data into cleaned, conformed structures D) A separate staging layer before bronze

Answer

Correct Answer: C
The silver layer is responsible for data cleansing and conforming, including flattening nested structures. Bronze should preserve raw data as-is for auditability. Gold focuses on business aggregations. Option D adds unnecessary complexity.

Question 32 *(Easy)* **Scenario**: A Delta table has a column `email` defined as `STRING NOT NULL`. A new data file arrives containing records with null email values.

Question: What happens when this data is written to the table?

A) Null values are converted to empty strings B) The write fails with a constraint violation error C) Records with null emails are silently dropped D) The NOT NULL constraint is automatically removed

Answer

Correct Answer: B
Delta Lake enforces NOT NULL constraints at write time. Attempting to write null values to a NOT NULL column causes the entire write operation to fail with an error. Options A, C, and D describe behaviors that don't occur.

Question 33 ***(Medium)*** **Scenario**: A data engineer needs to add a new column `loyalty_tier` to an existing Delta table with millions of records. The column should be nullable and positioned after the `customer_id` column.

Question: Which statement correctly adds this column?

A) `ALTER TABLE customers ADD COLUMN loyalty_tier STRING AFTER customer_id` B) `ALTER TABLE customers ADD COLUMNS (loyalty_tier STRING AFTER customer_id)` C) `ALTER TABLE customers ADD COLUMN loyalty_tier STRING` (column position cannot be specified) D) Table must be recreated to add a column at a specific position

Answer

Correct Answer: A
Delta Lake supports column positioning with FIRST or AFTER clauses in ADD COLUMN statements. The syntax `ALTER TABLE ADD COLUMN column_name TYPE AFTER existing_column` correctly positions the new column. Option B uses incorrect syntax (COLUMNS instead of COLUMN with positioning). Option C is incorrect because position CAN be specified. Option D is unnecessary since ADD COLUMN with AFTER is supported.

Question 34 ***(Easy)*** **Scenario**: A slowly changing dimension table tracks customer addresses with SCD Type 2 implementation. The table has columns: customer_id, address, start_date, end_date, is_current. A customer changes their address.

Question: How many rows should be affected by this single address change?

A) 1 row (update the existing record) B) 2 rows (close old record, insert new record) C) 3 rows (update old, insert new, update audit table) D) Depends on the number of historical records for that customer

Answer

Correct Answer: B
SCD Type 2 handles changes by: (1) closing the current record (setting end_date and is_current=false) and (2) inserting a new record with the new address (is_current=true). This affects exactly 2 rows. Option A describes SCD Type 1. Options C and D describe non-standard implementations.

Cost & Performance Optimization (Question 35)

Question 35 ***(Hard)*** **Scenario**: A fact table containing 5 years of transaction data is partitioned by `transaction_date`. Most queries filter on `customer_id` and `product_id`, rarely filtering on date. Query performance is poor.

Question: What modification would most improve query performance?

A) Add more partitions by also partitioning on `customer_id` B) Remove date partitioning and use Z-ORDER on `customer_id, product_id` C) Keep date partitioning and add Z-ORDER on `customer_id, product_id` D) Convert to Liquid Clustering on `customer_id, product_id`

Answer

Correct Answer: D
Liquid Clustering on the frequently filtered columns provides automatic data organization and maintenance. Since queries rarely filter on date, date partitioning provides little benefit. Option B loses time-based data management benefits. Option C combines both but has more maintenance overhead than liquid clustering. Option A creates too many small partitions.

Data Manipulation (Question 36)

Question 36 ***(Medium)*** **Scenario**: A data engineer is implementing a Delta table for product catalog data. Products are frequently updated, and analysts need to see what the catalog looked like at any point in the past 30 days.

Question: Which Delta Lake feature and configuration supports this requirement?

A) Enable Change Data Feed to track all changes B) Create a separate history table with trigger-based logging C) Time travel with default retention (30 days log retention) D) Use Delta Lake versioning with `delta.logRetentionDuration = 30 days`

Answer

Correct Answer: C
Delta Lake's time travel allows querying historical versions using `VERSION AS OF` or `TIMESTAMP AS OF`. The default 30-day log retention supports this requirement. Option A tracks changes but doesn't provide point-in-time queries. Option B is unnecessary given Delta's built-in capabilities. Option D describes the feature partially but misses that it's the default behavior.

Data Modeling (Question 37)

Question 37 ***(Medium)*** **Scenario**: A table has schema evolution enabled with `mergeSchema = true`. The source data occasionally includes columns with names that differ from existing columns only by case (e.g., ”CustomerId” vs ”customerid”).

Question: What is the default behavior in Delta Lake and how can it be controlled?

A) Delta is case-insensitive by default; both names map to the same column B) Delta is case-sensitive by default; they become separate columns C) Delta throws an error on case conflicts D) Delta automatically renames conflicting columns with suffixes

Answer

Correct Answer: A
By default, Delta Lake (following Spark SQL) treats column names as case-insensitive. ”CustomerId” and ”customerid” refer to the same column. This behavior can be changed with `spark.sql.caseSensitive = true`. Options B, C, and D describe non-default behaviors.

Data Manipulation (Question 38)

Question 38 *(Easy)* **Scenario**: A deep clone of a production Delta table is created for a testing environment. After the clone, updates are made to both the production table and the clone.

Question: How do changes to the production table affect the cloned table?

A) Changes automatically propagate to the clone B) Changes propagate until the clone is modified, then it becomes independent C) Only schema changes propagate; data changes don't D) The clone is completely independent; changes don't propagate

Answer

Correct Answer: D
Deep clone creates a completely independent copy with its own data files. After cloning, both tables evolve independently. No changes propagate between them. Option A describes linked tables, not clones. Options B and C describe behaviors that don't exist.

Data Modeling (Question 39)

Question 39 *(Medium)* **Scenario**: A dimension table needs to track corrections to historical records. When an error is discovered in a past record, the correction should be applied, but there must also be an audit trail showing the original incorrect value.

Question: Which SCD type best supports this requirement?

A) SCD Type 1 with a separate audit log table B) SCD Type 2 with effective dates C) SCD Type 3 with previous value columns D) SCD Type 6 (hybrid) with Type 1, 2, and 3 elements

Answer

Correct Answer: A
For corrections (not legitimate changes), SCD Type 1 updates the current value while a separate audit log table preserves the history of corrections. Type 2 is for legitimate business changes, not error corrections. Type 3 only tracks one previous value. Type 6 is complex and not specifically designed for corrections.

Data Governance (Question 40)

Question 40 *(Medium)* **Scenario**: A data engineering team needs to grant a group of analysts SELECT access to all tables in a schema, including tables that will be created in the future.

Question: Which Unity Catalog permission model correctly implements this requirement?

A) Grant SELECT on each table individually as they're created B) Create a view for each table and grant SELECT on views only C) Grant SELECT on the catalog to give access to all schemas and tables D) Grant USE SCHEMA and SELECT on SCHEMA to give access to all current and future tables

Answer

Correct Answer: D
Granting SELECT at the schema level provides access to all current and future tables in that schema. USE SCHEMA is required to access objects within the schema. Option A requires ongoing maintenance. Option C is too broad. Option B is unnecessarily complex.

Ensuring Data Security and Compliance (Question 41)

Question 41 ***(Medium)*** **Scenario**: A table contains employee data including salary information. The HR department should see all columns, but managers should only see employee name, department, and title--not salary.

Question: Which Unity Catalog feature implements this column-level security requirement?

A) Create separate tables for HR and managers with different columns B) Use dynamic views with `is_account_group_member()` to filter columns C) Use column masks to hide salary for non-HR users D) Implement application-level security in the BI tool

Answer

Correct Answer: C
Column masks in Unity Catalog apply functions to column values based on the querying user's permissions, showing the real value to authorized users and masked/null values to others. Option A creates data duplication. Option B requires view maintenance. Option D doesn't protect data at the source.

Data Ingestion & Acquisition (Question 42)

Question 42 ***(Easy)*** **Scenario**: An organization wants to share a curated dataset with an external partner who uses Snowflake. The partner should receive daily updates but should not have direct access to the Databricks workspace.

Question: Which feature enables secure data sharing with external organizations?

A) Create a service principal for the partner to access via API B) Use OpenSharing to share data via open protocol C) Export data daily to a shared cloud storage location D) Create a Databricks workspace for the partner with access to the dataset

Answer

Correct Answer: B
OpenSharing provides an open protocol for secure data sharing across organizations and platforms. Partners can use compatible clients (including Snowflake, Spark, pandas) without Databricks access. Option A requires workspace access. Option C requires file transfer management. Option D is costly and requires Databricks licenses.

Ensuring Data Security and Compliance (Question 43)

Question 43 *(Easy)* **Scenario**: A data engineer is building a notebook that needs to access credentials for an external database. The credentials are stored in Azure Key Vault, which has been configured as a secret backend.

Question: Which code correctly retrieves the database password?

A) `dbutils.secrets.get(scope="keyvault-scope", key="db-password")` B) `spark.conf.get("azure.keyv password")` C) `dbutils.credentials.get("keyvault-scope", "db-password")` D) `%keyvault get db-password`

Answer

Correct Answer: A
When Azure Key Vault is configured as a secret scope backend, secrets are accessed through `dbutils.secrets.get()` using the scope name and key name. The Key Vault integration is transparent to the code. Options B, C, and D use invalid syntax.

Data Governance (Question 44)

Question 44 *(Easy)* **Scenario**: A Unity Catalog metastore is shared across multiple workspaces. A user has SELECT permission on a table granted in Workspace A. The same user accesses Workspace B.

Question: What permissions does the user have on that table in Workspace B?

A) No permissions; grants are workspace-specific B) Same SELECT permission; Unity Catalog permissions are centralized C) Read-only permissions with a warning about cross-workspace access D) Permissions depend on workspace-level settings

Answer

Correct Answer: B
Unity Catalog provides centralized governance across all workspaces attached to the same metastore. Permissions granted on objects apply regardless of which workspace is used to access them. This is a key benefit of Unity Catalog over legacy Hive metastore.

Ensuring Data Security and Compliance (Question 45)

Question 45 *(Hard)* **Scenario**: A healthcare organization needs to implement row-level security so that doctors can only see records for patients assigned to them. The patient_assignments table maps doctors to patients.

Question: Which approach correctly implements this row-level security in Unity Catalog?

A) Add a row filter using `ROW FILTER filter_function ON (patient_id)` B) Create dynamic views with `WHERE doctor_id = current_user()` C) Use column masks to hide rows from unauthorized users D) Implement filtering in the application layer only

Answer

Correct Answer: A
Unity Catalog row filters apply a function that determines which rows each user can access based on their identity. The filter function can join with assignment tables to implement com-

plex rules. Option B works but requires view maintenance. Option C is for column security, not row security. Option D doesn't protect data at the source.

Monitoring and Alerting (Question 46)

Question 46 *(Easy)* **Scenario**: A cost analyst needs to identify which jobs consumed the most DBUs last month to optimize spending.

Question: Which system table provides this billing information?

A) `system.compute.clusters` B) `system.access.audit` C) `system.billing.usage` D) `system.workflow.jobs`

Answer

Correct Answer: C
The `system.billing.usage` table contains detailed DBU consumption data by workspace, cluster, and SKU. It's the primary source for cost analysis. Option A has cluster metadata. Option B has security audit logs. Option D has job definitions, not cost data.

Debugging and Deploying (Question 47)

Question 47 *(Medium)* **Scenario**: A streaming job is running slower than expected. The Spark UI shows that one task in a stage takes 10x longer than the others, while most executors sit idle.

Question: What does this pattern indicate and what is the recommended solution?

A) Insufficient executors; increase cluster size B) Data skew; enable AQE skew join handling or salt the skewed key C) Memory pressure; increase executor memory D) Network bottleneck; check cluster networking

Answer

Correct Answer: B
One slow task with idle executors is the classic sign of data skew--one partition has much more data than others. AQE can automatically split skewed partitions, or you can manually salt keys. Option A wouldn't help if executors are idle. Options C and D show different symptoms.

Monitoring and Alerting (Question 48)

Question 48 *(Medium)* **Scenario**: A Lakeflow (DLT) pipeline shows ”Update Failed” status. The data engineer needs to identify which specific expectation caused the failure.

Question: Which approach retrieves the expectation violation details?

A) Check the cluster driver logs B) Review the pipeline settings in the UI C) Use `dbutils.pipelines.getLatestUpdate()` API D) Query the pipeline's event log table for expectation metrics

Answer

Correct Answer: D
Lakeflow Pipelines stores detailed metrics including expectation results in the event log. Querying the event log with filters for the update ID shows which expectations failed and the row counts. Option A has limited detail. Option C isn't a valid API. Option B shows configuration, not runtime details.

Cost & Performance Optimization (Question 49)

Question 49 ***(Medium)*** **Scenario**: A data engineer needs to understand why a SQL query is performing a full table scan despite having filters that should enable partition pruning.

Question: Which tool reveals whether partition pruning is occurring?

A) Run `ANALYZE TABLE` to update statistics B) Check the Query History for execution time breakdown C) Run `EXPLAIN` and check for `PartitionFilters` in the plan D) Review the table's `DESCRIBE DETAIL` output

Answer

Correct Answer: C
`EXPLAIN` shows the query execution plan including `PartitionFilters` (pruned partitions) and `PushedFilters` (filters pushed to scan). Empty `PartitionFilters` indicates no pruning occurred. Option A updates statistics but doesn't show the plan. Options B and D don't show execution details.

Monitoring and Alerting (Question 50)

Question 50 ***(Easy)*** **Scenario**: The security team requests a report of all users who accessed a specific table containing PII data in the last 7 days.

Question: Which system table contains this access information?

A) `system.billing.usage` B) `system.access.audit` C) `system.information_schema.tables` D) `system.compute.clusters`

Answer

Correct Answer: B
The `system.access.audit` table contains audit logs of actions including table reads, with user identity and timestamp. Filter by `action_name` and `request_params` for table access events. Option A has billing data. Option C has table metadata. Option D has cluster info.

Cost & Performance Optimization (Question 51)

Question 51 *(Medium)* **Scenario**: A batch job's execution time has gradually increased from 30 minutes to 2 hours over the past month. The code hasn't changed.

Question: Which factors should be investigated first?

A) Data volume growth and partition sizing B) Cluster hardware degradation C) Changes to Databricks Runtime D) Network latency to cloud storage

Answer

Correct Answer: A
Gradual performance degradation with unchanged code typically indicates data volume growth or small file accumulation (affecting read performance). Check table sizes, file counts, and partition strategies. Option B is unlikely to be gradual. Options C and D would cause immediate changes, not gradual.

Debugging and Deploying (Question 52)

Question 52 *(Easy)* **Scenario**: A data engineering team is setting up Declarative Automation Bundles (DABs) for their project. They need separate configurations for development, staging, and production environments.

Question: Which `databricks.yml` structure correctly defines these environments?

A) Create separate `databricks-dev.yml`, `databricks-staging.yml`, `databricks-prod.yml` files B) Use `environments:` section with runtime variable substitution C) Use `targets:` section with `dev:`, `staging:`, and `prod:` subsections D) Create separate bundles for each environment in different directories

Answer

Correct Answer: C
DAB uses a `targets:` section to define environment-specific configurations. Each target can override bundle settings like workspace host, cluster configs, and variables. Option A doesn't support DAB's inheritance model. Option B uses invalid syntax. Option D duplicates code.

Developing Code for Data Processing using Python and SQL (Question 53)

Question 53 *(Medium)* **Scenario**: A CI/CD pipeline runs unit tests on PySpark code that transforms DataFrames. The tests should run quickly without requiring a Databricks cluster.

Question: Which testing approach is most appropriate?

A) Deploy code to a Databricks cluster and run tests remotely B) Skip unit tests and rely on integration tests only C) Use Nutter framework with Databricks Connect D) Use local Spark session in pytest with mocked data

Answer

Correct Answer: D
Local Spark sessions in pytest allow fast unit tests without cluster costs or dependencies. Mock DataFrames test transformation logic efficiently. Option A is slow and costly for unit tests. Option C still requires cluster resources. Option B misses the value of unit testing.

Debugging and Deploying (Questions 54–55)

Question 54 ***(Easy)*** **Scenario**: A Git folder in Databricks is configured with main branch protection. A developer tries to push directly to main and receives an error.

Question: What is the recommended workflow for making changes?

A) Create a feature branch, commit changes, and create a pull request B) Disable branch protection temporarily to push changes C) Use `%git` magic commands to force push to main D) Make changes in a separate notebook outside Git folders

Answer

Correct Answer: A
The standard Git workflow with branch protection: create a feature branch, make changes, push, create PR for review, then merge to main after approval. This ensures code review and testing. Options B and C bypass protections. Option D loses version control benefits.

Question 55 ***(Medium)*** **Scenario**: A GitHub Actions workflow deploys Declarative Automation Bundles to a staging environment. The workflow needs to authenticate with Databricks.

Question: Which authentication method is recommended for GitHub Actions?

A) Store personal access token in GitHub Secrets B) Configure OIDC federation between GitHub and Databricks with service principal C) Use username and password stored in GitHub Secrets D) Create a dedicated user account for GitHub Actions

Answer

Correct Answer: B
OIDC federation with service principals is the recommended approach for CI/CD authentication. GitHub Actions can authenticate using short-lived tokens without storing long-lived credentials. Option A uses personal tokens that expire. Option C is deprecated. Option D creates management overhead.

Developing Code for Data Processing using Python and SQL (Question 56)

Question 56 ***(Medium)*** **Scenario**: A Nutter test notebook tests a function that reads from a Delta table. The test should verify the function handles empty tables correctly.

Question: Which Nutter pattern correctly implements this test case?

A) Create an empty table in `before_all()` and clean up in `after_all()` B) Use `assertion_empty_table()` with the expected result C) Mock the Delta table read using `unittest.mock` D) Use Nutter's built-in empty table generator

Answer

Correct Answer: A
Nutter's `before_all()` runs setup before tests (create empty test table), and `after_all()`

runs cleanup (drop table). The actual test verifies behavior with the empty table. Option B isn't a Nutter method. Option C is complex within Nutter. Option D doesn't exist.

Debugging and Deploying (Question 57)

Question 57 *(Medium)* **Scenario**: A bundle deployment fails with the error: ”Resource already exists”. The data engineer confirms the resource was created by a previous partial deployment.

Question: What is the correct approach to resolve this and complete the deployment?

A) Manually delete the resource and redeploy B) Update the resource name in databricks.yml to avoid conflict C) Run `databricks bundle destroy` then `databricks bundle deploy` D) Run `databricks bundle deploy --force` to overwrite

Answer

Correct Answer: D
The `--force` flag allows DAB to take ownership of and update existing resources that match the bundle definition. This handles partial deployments gracefully. Option A risks missing dependent resources. Option C destroys everything unnecessarily. Option B changes the deployment, not fixing it.

Developing Code for Data Processing using Python and SQL (Questions 58–59)

Question 58 *(Medium)* **Scenario**: A Lakeflow (DLT) pipeline has a streaming table that ingests data and a downstream materialized view that aggregates it. The materialized view is showing stale data even though the streaming table is updating.

Question: What is the most likely cause?

A) The pipeline is configured for triggered execution, not continuous B) Materialized views don't automatically refresh from streaming tables C) There's a data type mismatch between the tables D) The materialized view requires a manual REFRESH command

Answer

Correct Answer: A
In triggered execution mode, materialized views only refresh when the pipeline runs. For near real-time updates, use continuous mode or schedule frequent pipeline runs. In continuous mode, materialized views refresh as streaming tables update. Options B and D are incorrect; Lakeflow Pipelines handles refreshes automatically within pipeline runs. Option C would cause errors, not stale data.

Question 59 *(Medium)* **Scenario**: A Lakeflow pipeline implements CDC from a source database using AUTO CDC (formerly APPLY CHANGES). Some records are arriving with out-of-order timestamps due to source system behavior.

Question: How does AUTO CDC handle out-of-order records by default?

A) Records are processed in arrival order, ignoring timestamps B) Records are processed by sequence column; out-of-order records are dropped C) Records are processed by sequence column; out-of-order records update if sequence is higher D) The pipeline fails on out-of-order detection

Answer

Correct Answer: C
AUTO CDC uses the SEQUENCE BY column to determine record order. If a later-arriving record has a higher sequence value than the current record, it updates the target. Records with lower sequence values than existing records are ignored (as they represent older data). This handles out-of-order arrival correctly.

Monitoring and Alerting (Question 60)

Question 60 ***(Medium)*** **Scenario**: A Lakeflow pipeline has an expectation `CONSTRAINT valid_amount EXPECT (amount > 0) ON VIOLATION DROP ROW`. After processing, the engineer wants to know how many rows were dropped.

Question: How can the dropped row count be retrieved?

A) Check the `dropped_records` column in the target table B) Dropped rows are not tracked; add manual logging C) Run `DESCRIBE HISTORY` on the target table D) Query the pipeline event log for expectation metrics

Answer

Correct Answer: D
Lakeflow Pipelines stores detailed expectation metrics in the event log including `num_dropped_records` for each expectation. Query the event log filtering for expectation events. Option A doesn't exist. Option C shows Delta operations, not expectation details. Option B is incorrect; Lakeflow Pipelines tracks this automatically.

Cost & Performance Optimization (Questions 61–63)

Question 61 ***(Medium)*** **Scenario**: A Delta table with 2TB of data has 50,000 small files averaging 40MB each. Queries scanning the table are slow due to file listing overhead.

Question: What is the optimal compaction strategy?

A) Run `OPTIMIZE` to compact files to the default 1GB target size B) Run `VACUUM` to remove small files C) Enable auto-compaction for future writes and run `OPTIMIZE` for existing files D) Repartition the table with `spark.sql.shuffle.partitions = 2000`

Answer

Correct Answer: C
`OPTIMIZE` compacts existing small files into larger ones (target ~1GB). Enabling auto-compaction (`spark.databricks.delta.autoCompact.enabled`) prevents small files in future writes. Option B removes old files, not compaction. Option D affects partitions in processing, not file sizes.

Question 62 *(Medium)* **Scenario**: A query joins a 500GB fact table with a 50MB dimension table. The join is running slowly with significant shuffle.

Question: Which optimization would most improve this join performance?

A) Increase `spark.sql.shuffle.partitions` to 1000 B) Enable broadcast join by setting `spark.sql.autoBroadcastJoinThreshold = 100MB` C) Z-ORDER the fact table by the join key D) Partition both tables by the join key

Answer

Correct Answer: B
A 50MB dimension table is a good candidate for broadcast join (eliminates shuffle of the large fact table). Increasing the threshold from 10MB default to 100MB enables automatic broadcast. Option A doesn't reduce shuffle. Option C helps filters, not joins. Option D requires data redistribution.

Question 63 *(Medium)* **Scenario**: A data engineer is tuning a complex aggregation query that processes 100GB of data. The query runs out of memory during the shuffle phase.

Question: Which configuration change most likely resolves this issue?

A) Increase `spark.sql.shuffle.partitions` to create smaller partitions B) Decrease `spark.memory.fraction` to leave more room for user objects C) Enable `spark.sql.adaptive.enabled` for dynamic partition coalescing D) Set `spark.executor.memoryOverhead` to 0 for maximum heap space

Answer

Correct Answer: A
OOM during shuffle typically means individual partitions are too large. Increasing shuffle partitions creates smaller partitions that fit in memory. Option B reduces available memory. Option C helps with small partitions, not OOM. Option D can cause off-heap OOM errors.

← Back to Mock Exam

Supplementary questions (not in the timed set) These five questions were added for the Nov 30, 2025 guide and are not part of the timed set or the practice-quiz bank. Each is labelled with its Oct 9, 2026 exam section — the former *Data Sharing and Federation* questions now belong to **Data Ingestion & Acquisition** (12 %).

Question DSF-1 *(Medium — Data Ingestion & Acquisition)* **Scenario**: A data engineering team needs to give a partner company read-only access to a Delta table that's updated every hour. The partner uses Databricks too, but is in a different cloud account. The team wants to avoid copying data and wants UC to enforce all access.

Question: Which pattern fits best?

A) Export the table to S3 as Parquet and email the partner a presigned URL
B) Create a Databricks-to-Databricks (D2D) OpenSharing share with the partner's UC sharing identifier and grant the share to a `RECIPIENT`
C) Replicate the table to the partner's account using Lakeflow Pipelines
D) Set up a foreign catalog in the partner's workspace pointing at the team's table

Answer

Correct Answer: B
Databricks-to-Databricks OpenSharing is purpose-built for this: it shares the live Delta table (the partner sees updates as they land), uses UC identities on both sides, and the provider grants a `SHARE` to a `RECIPIENT` identified by the partner's UC sharing identifier. No data copy, full UC audit trail. Option A loses governance and freshness. Option C (Lakeflow Pipelines replication) creates a parallel pipeline you have to maintain AND loses freshness — pipelines run on a schedule, while D2D Share is live. Option D is Lakehouse Federation, which is for the *consumer* to query *external* (non-Delta) sources — not for cross-Databricks data sharing.

Question DSF-2 ***(Medium — Data Ingestion & Acquisition)*** **Scenario**: An analytics team wants to query a Snowflake `orders` table for ad-hoc analysis without ETL'ing it into Delta. They need filter pushdown to keep performance reasonable, and credentials must be governed centrally.

Question: Which combination is correct?

A) Create a Databricks `JDBC` connection in `spark.conf` and read with `spark.read.format("jdbc")`
B) Create a Unity Catalog `CONNECTION` of type `SNOWFLAKE`, then `CREATE FOREIGN CATALOG` over it; query as `catalog.schema.table`
C) Use OpenSharing — Snowflake supports being a OpenSharing share recipient
D) Use Auto Loader with the Snowflake source connector

Answer

Correct Answer: B
Lakehouse Federation is the documented path for federated, read-mostly access to external databases. The two-step DDL — `CREATE CONNECTION ... TYPE SNOWFLAKE ...` followed by `CREATE FOREIGN CATALOG ... USING CONNECTION ...` — stores the credential in UC (auditable, rotatable), exposes the Snowflake database as a UC catalog, and supports filter / projection / aggregation pushdown to Snowflake where the dialect allows. Option A scatters credentials in cluster config and bypasses UC. Option C is wrong direction — Snowflake is not a OpenSharing share recipient platform. Option D is for object-store file ingestion, not relational sources.

Question PERF-1 ***(Hard — Cost & Performance Optimization)*** **Scenario**: A 2 TB Delta table is partitioned by `event_date` and Z-ORDERed by `customer_id`. Queries filter on both `event_date` (high selectivity) and `region` (low cardinality, 5 distinct values). The team is considering migrating to liquid clustering.

Question: Which migration approach gives the best query performance with the least operational disruption?

A) Run `OPTIMIZE table ZORDER BY (customer_id, region, event_date)` to add `region` to the Z-ORDER
B) Drop the table and recreate it with `CLUSTER BY (customer_id, event_date, region)` and a one-time `OPTIMIZE` to materialise clustering
C) Run `ALTER TABLE table CLUSTER BY (customer_id, event_date, region)` and let liquid clustering organise data incrementally on subsequent writes; back-fill with `OPTIMIZE table FULL`
D) Switch to bucketing on `customer_id` with 100 buckets

Answer

Correct Answer: C
Liquid clustering supports **`ALTER TABLE ... CLUSTER BY`** for migration without recreating the table. New writes adopt the clustering; an explicit `OPTIMIZE table FULL` triggers a one-time reorganisation of existing data. This avoids the disruption of B (recreate) and the partitioning lock-in of A (Z-ORDER is non-adaptive and partition columns can't be Z-ORDERed). D is a Hive-era pattern not relevant to Delta on Databricks.

Question COST-1 ***(Medium — Cost & Performance Optimization)*** **Scenario**: A nightly Lakeflow Job runs a Spark batch pipeline that takes 45 minutes on a Standard (DBR) all-purpose cluster of 8 i3.xlarge workers. The team wants to halve the cost.

Question: Which two changes typically yield the biggest cost reduction with the least risk?

A) Switch to a Job cluster (cheaper DBU rate per node) AND enable autoscaling 2–8 workers
B) Reduce worker count to 4 and keep the cluster as all-purpose
C) Move to Photon-enabled instance type AND enable single-user access mode
D) Switch to spot instances AND disable autoscaling

Answer

Correct Answer: A
Two independent levers, both safe: (1) Job clusters cost ~50 % less per DBU than all-purpose clusters because they're not designed for shared interactive use; (2) autoscaling drops idle nodes so you only pay for what you use during low-throughput phases of the job. B caps capacity (may slow the job → cost trade-off neutralised). C may help but Photon depends on workload shape and single-user mode is unrelated to cost. D introduces spot-eviction risk and removes elasticity.

Question MON-1 ***(Medium — Monitoring and Alerting)*** **Scenario**: A Lakeflow Job that processes daily orders has been failing intermittently for the past week. The team needs both proactive alerting (so on-call is paged before downstream teams notice) AND a way to query historical failure patterns to identify the root cause.

Question: Which combination satisfies both requirements?

A) Configure Lakeflow Jobs notifications (email + webhook to PagerDuty) for failures AND query `system.lakeflow.job_run_timeline` for historical failure analytics
B) Add a Slack webhook inside the job's last task that fires on success; absence of message indicates failure
C) Set up a SQL Alert that polls `SHOW JOBS` every 5 minutes
D) Rely on Databricks support to surface failures automatically

Answer

Correct Answer: A
Lakeflow Jobs notifications fire on success/failure/duration thresholds and route via email or webhook (PagerDuty, Slack, custom HTTP). For historical analytics, system tables under `sys-`

`tem.lakeflow.*` expose job run history queryable in SQL. B is anti-pattern ("absence of signal" alerts are notoriously unreliable). C `SHOW JOBS` doesn't surface run-level state; the system tables do. D is not how Databricks works.

Mock Exam 1 Debrief — Databricks Data Engineer Professional

After timing yourself through the 63 questions, use this debrief to:

1. **Identify weak sections** with the per-question map below
2. **Find the topic folder** that covers each missed question
3. **Follow the study plan** matched to your miss count

Tip

The number of questions you missed matters less than *which sections* they came from. Three misses in the 5 % Data Governance section hit very differently than three misses in the 23 % Developing Code section.

Note

Questions are labelled with the **9 sections of the Oct 9, 2026 exam guide**. The mock was written for earlier guides, so its mix does not match the official weights exactly — use the counts below when you score yourself, and use the practice questions for the skills that are new in 2026.

Section → study-resource mapping

Exam section	Topic folder
1. Developing Code for Data Processing using Python and SQL (23 %)	01-developing-code-for-data-processir
2. Data Ingestion & Acquisition (12 %)	02-data-ingestion-and-acquisition
3. Data Manipulation (12 %)	03-data-manipulation
4. Monitoring and Alerting (10 %)	04-monitoring-and-alerting
5. Cost & Performance Optimization (15 %)	05-cost-and-performance-optimizatior
6. Ensuring Data Security and Compliance (8 %)	06-ensuring-data-security-and-compli
7. Data Governance (5 %)	07-data-governance
8. Debugging and Deploying (10 %)	08-debugging-and-deploying
9. Data Modeling (5 %)	09-data-modeling

Per-question quick map Each question is mapped to its primary exam section. Questions that touch two sections are listed under the one their correct answer depends on. The supplementary questions at the end of `questions.md` are listed last.

Q#	Difficulty	Section	Topic folder	Cheat sheet
1	Easy	Data Ingestion	Section 2 folder	Auto Loader quick ref
2	Medium	Cost & Performance	Section 5 folder	Performance optimization
3	Hard	Data Modeling	Section 9 folder	Delta Lake commands
4	Medium	Data Ingestion	Section 2 folder	Auto Loader quick ref
5	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
6	Medium	Cost & Performance	Section 5 folder	Performance optimization
7	Medium	Cost & Performance	Section 5 folder	Performance optimization

Q#	Difficulty	Section	Topic folder	Cheat sheet
8	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
9	Medium	Data Ingestion	Section 2 folder	Auto Loader quick ref
10	Easy	Security & Compliance	Section 6 folder	Unity Catalog quick ref
11	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
12	Hard	Debugging & Deploying	Section 8 folder	Spark configurations
13	Hard	Data Manipulation	Section 3 folder	Delta Lake commands
14	Easy	Data Ingestion	Section 2 folder	Auto Loader quick ref
15	Medium	Cost & Performance	Section 5 folder	Performance optimization
16	Easy	Data Manipulation	Section 3 folder	Delta Lake commands
17	Medium	Data Manipulation	Section 3 folder	Delta Lake commands
18	Hard	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
19	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
20	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
21	Easy	Security & Compliance	Section 6 folder	Unity Catalog quick ref
22	Easy	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
23	Medium	Debugging & Deploying	Section 8 folder	Spark configurations
24	Medium	Monitoring	Section 4 folder	Unity Catalog quick ref
25	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
26	Medium	Data Governance	Section 7 folder	Unity Catalog quick ref
27	Easy	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
28	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
29	Medium	Data Governance	Section 7 folder	Unity Catalog quick ref
30	Medium	Monitoring	Section 4 folder	Unity Catalog quick ref
31	Easy	Data Modeling	Section 9 folder	Delta Lake commands
32	Easy	Data Modeling	Section 9 folder	Delta Lake commands
33	Medium	Data Modeling	Section 9 folder	Delta Lake commands
34	Easy	Data Modeling	Section 9 folder	Delta Lake commands
35	Hard	Cost & Performance	Section 5 folder	Performance optimization
36	Medium	Data Manipulation	Section 3 folder	Delta Lake commands
37	Medium	Data Modeling	Section 9 folder	Delta Lake commands
38	Easy	Data Manipulation	Section 3 folder	Delta Lake commands
39	Medium	Data Modeling	Section 9 folder	Delta Lake commands
40	Medium	Data Governance	Section 7 folder	Unity Catalog quick ref
41	Medium	Security & Compliance	Section 6 folder	Unity Catalog quick ref
42	Easy	Data Ingestion	Section 2 folder	Auto Loader quick ref
43	Easy	Security & Compliance	Section 6 folder	Unity Catalog quick ref
44	Easy	Data Governance	Section 7 folder	Unity Catalog quick ref
45	Hard	Security & Compliance	Section 6 folder	Unity Catalog quick ref
46	Easy	Monitoring	Section 4 folder	Unity Catalog quick ref
47	Medium	Debugging & Deploying	Section 8 folder	Spark configurations
48	Medium	Monitoring	Section 4 folder	Unity Catalog quick ref
49	Medium	Cost & Performance	Section 5 folder	Performance optimization
50	Easy	Monitoring	Section 4 folder	Unity Catalog quick ref
51	Medium	Cost & Performance	Section 5 folder	Performance optimization
52	Easy	Debugging & Deploying	Section 8 folder	Spark configurations
53	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
54	Easy	Debugging & Deploying	Section 8 folder	Spark configurations
55	Medium	Debugging & Deploying	Section 8 folder	Spark configurations
56	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
57	Medium	Debugging & Deploying	Section 8 folder	Spark configurations
58	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
59	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
60	Medium	Monitoring	Section 4 folder	Unity Catalog quick ref
61	Medium	Cost & Performance	Section 5 folder	Performance optimization

Q#	Difficulty	Section	Topic folder	Cheat sheet
62	Medium	Cost & Performance	Section 5 folder	Performance optimization
63	Medium	Cost & Performance	Section 5 folder	Performance optimization
DSF-1	—	Data Ingestion	Section 2 folder	Auto Loader quick ref
DSF-2	—	Data Ingestion	Section 2 folder	Auto Loader quick ref
PERF-1	—	Cost & Performance	Section 5 folder	Performance optimization
COST-1	—	Cost & Performance	Section 5 folder	Performance optimization
MON-1	—	Monitoring	Section 4 folder	Unity Catalog quick ref

Per-section question counts (for self-scoring)

Exam section	Official weight	Questions in this mock (of 63
1. Developing Code for Data Processing using Python and SQL	23 %	14
2. Data Ingestion & Acquisition	12 %	5
3. Data Manipulation	12 %	5
4. Monitoring and Alerting	10 %	6
5. Cost & Performance Optimization	15 %	10
6. Ensuring Data Security and Compliance	8 %	5
7. Data Governance	5 %	4
8. Debugging and Deploying	10 %	7
9. Data Modeling	5 %	7

Study plan by miss count

Missed	Status	Study plan
0–6 (≥ 90 %)	Excellent	Quick review of any missed topic; you're exam-ready
7–13 (78–89 %)	Likely passing	Identify the 2-3 weakest domains; spend 1-2 hours per domain re-reading
14–19 (70–77 %)	On the bubble	Block 1-2 weeks for focused study on the weakest 3-4 domains. Read eve
20–25 (60–68 %)	Not ready yet	Take 2-3 weeks for comprehensive review. Work through each topic folde
26+ (< 60 %)	Significant gap	Restart from shared/fundamentals/. Plan a 4-6 week study cycle. Conside

When most misses cluster in one section

Section with the misses	Top study action
Developing Code	Read `01-developing-code-for-data-processing/` end to end; build a small pipeli
Data Ingestion	Run Auto Loader, a Kafka read and a Lakeflow Connect pipeline side by side; create
Data Manipulation	Write window-function and MERGE queries by hand; load JSON into a VARIANT colun
Monitoring	Query `system.billing.usage` and `system.lakeflow.job_run_timeline`; create a
Cost & Performance	Read `05-cost-and-performance-optimization/01-managed-tables-predictive`
Security & Compliance	Write a column mask and an ABAC policy on a governed tag; walk through the DELE
Data Governance	Grant on a catalog, create a new table, and confirm the grant is inherited; add tags
Debugging & Deploying	Deploy a bundle to two targets; fail a job task on purpose and repair the run with a
Data Modeling	Design a star schema and define one metric view over it

---

Mock Exam 2 — Databricks Data Engineer Professional

A second timed mock with different scenarios — take after Mock Exam 1.

Important

Relabelled for the Oct 9, 2026 exam guide. Every question now carries its section from the 9-section guide, and product names follow the guide (Lakeflow Pipelines, AUTO CDC, Declarative Automation Bundles, OpenSharing). The questions themselves were written for earlier guides, so the section mix below is not the official weighting — the real exam has 60 scored questions. Skills new in 2026 are drilled in the practice questions.

Exam Instructions

Detail	Value
Total Questions	60
Time Limit	120 minutes
Result	Pass / fail (the Oct 9, 2026 exam guide publishes no numeric passing score)
Format	Scenario-based, single correct answer

Domain Distribution (labelled to the Oct 9, 2026 exam guide)

Exam section	Official weight	Questions in this mock
Developing Code for Data Processing using Python and SQL	23 %	16
Data Ingestion & Acquisition	12 %	5
Data Manipulation	12 %	5
Monitoring and Alerting	10 %	4
Cost & Performance Optimization	15 %	7
Ensuring Data Security and Compliance	8 %	4
Data Governance	5 %	2
Debugging and Deploying	10 %	8
Data Modeling	5 %	9

Scoring Guide

Score	Result
48–60 (≥ 80 %)	Excellent
42–47 (70–79 %)	Likely passing
36–41 (60–69 %)	Close — focused study needed
Below 36 (< 60 %)	Comprehensive review needed

Note

Databricks does not publish a numeric passing threshold for the current exam guide.

After the exam — debrief

When you've finished the timed sitting, see `debrief.md` for the domain-to-resource map, the per-section question breakdown, and a study plan tailored to your miss count.

Mock Exam 2 — Databricks Data Engineer Professional

60 scenario-based questions focusing on advanced topics and scenario-based reasoning. Attempt all questions before revealing answers.

Total: 60 questions | Time: 120 minutes | Self-check target: 70 % (42/60) — Databricks publishes no passing score

← Back to Mock Exam 2

Data Ingestion & Acquisition (Questions 1–2)

Question 1 ***(Medium)*** **Scenario**: A data engineering team ingests millions of small JSON files per day from a cloud storage landing zone. The files are uploaded by thousands of IoT devices. Using Auto Loader with directory listing mode, the pipeline takes over 40 minutes just to discover new files before any processing begins.

Question: Which Auto Loader configuration change will most effectively reduce the file discovery time for this pipeline?

A) Set `cloudFiles.maxFilesPerTrigger` to a higher value to process more files per batch B) Switch to `cloudFiles.useIncrementalListing = true` to use sorted directory listing C) Switch to `cloudFiles.useNotifications = true` to use file notification mode D) Set `cloudFiles.schemaLocation` to a faster storage tier for schema caching

Answer

Correct Answer: C
File notification mode uses cloud-native event notifications (e.g., AWS SNS/SQS, Azure Event Grid) to detect new files, eliminating the need to list the entire directory. This is dramatically faster than directory listing mode when dealing with millions of files. Option A increases batch size but does not speed up file discovery. Option B improves directory listing but still requires listing operations that are slow at scale.

Question 2 ***(Hard)*** **Scenario**: A streaming pipeline uses Auto Loader to ingest Parquet files with a defined schema stored at a checkpoint location. After a deployment, the upstream system adds two new required columns to the Parquet files. The pipeline is restarted and immediately fails with an `AnalysisException` indicating the new columns are missing from the existing schema.

Question: What happens when Auto Loader encounters schema changes after recovering from a checkpoint?

A) Auto Loader uses the previously inferred schema from the schema location and rejects files that do not match, causing a `UnknownFieldException` that stops the stream unless schema evolution is configured B) Auto Loader automatically adds the new columns and backfills existing data with null values C) Auto Loader ignores the new columns and continues processing with the original schema only D) Auto Loader restarts from the beginning of the source directory, re-reading all files with the new schema

Answer

Correct Answer: A
Auto Loader persists the inferred schema to the schema location and reuses it across restarts. When new columns appear, the behavior depends on `cloudFiles.schemaEvolutionMode`.

Without explicit configuration, unexpected schema changes cause the stream to fail. Options B and C describe behaviors that only occur with specific `schemaEvolutionMode` settings (`addNewColumns` or `none` respectively).

Data Modeling (Question 3)

Question 3 ***(Hard)*** **Scenario**: A retail company needs to implement SCD Type 2 on a `dim_customer` table. When a customer updates their address, the current record must be closed (set `end_date` and `is_current = false`) and a new record must be inserted with `is_current = true`. The source data arrives as a staging table with only the latest state per customer.

Question: Which MERGE implementation correctly handles SCD Type 2 for this use case?

A) A single MERGE with `WHEN MATCHED THEN UPDATE SET end_date = current_date(), is_current = false` and `WHEN NOT MATCHED THEN INSERT` -- this handles both closing old records and inserting new ones B) Two separate SQL statements: first an UPDATE to close changed records, then an INSERT for the new versions, wrapped in a transaction C) A single MERGE with `WHEN MATCHED AND target.is_current = true AND target.address <> source.address THEN UPDATE SET *` and `WHEN NOT MATCHED THEN INSERT *` D) A single MERGE with `WHEN MATCHED AND target.is_current = true AND target.address <> source.address THEN UPDATE SET end_date = current_date(), is_current = false` combined with a separate INSERT statement for new active records using a UNION of the matched changes

Answer

Correct Answer: D
SCD Type 2 requires two operations for changed records: closing the existing record and inserting a new active version. A single MERGE cannot both update and insert for the same matched key, so the correct pattern uses MERGE to close the old record and a separate INSERT for the new version. Option A only closes old records but never inserts the new active version. Option C overwrites the record entirely (SCD Type 1 behavior).

Developing Code for Data Processing using Python and SQL (Questions 4–5)

Question 4 ***(Hard)*** **Scenario**: A data engineer needs to join two streaming DataFrames: `orders` (with `order_time`) and `shipments` (with `ship_time`). The business rule states that a shipment must occur within 7 days of the order to be considered valid. Both streams have watermarks defined.

Question: Which join condition correctly implements this time-bounded stream-stream join?

A) `orders.order_id == shipments.order_id` with a post-join filter `shipments.ship_time - orders.order_time <= interval 7 days` B) `orders.order_id == shipments.order_id AND shipments.ship_time BETWEEN orders.order_time AND orders.order_time + interval 7 days` C) `orders.order_id == shipments.order_id` with watermarks set to 7 days on both streams D) `orders.order_id == shipments.order_id AND shipments.ship_time - orders.order_time <= 7` with implicit day conversion

Answer

Correct Answer: B
The time-range condition must be part of the join condition (not a post-join filter) so that Spark

can use it along with watermarks to determine when old state can be discarded. The BETWEEN clause clearly expresses the valid time window. Option A applies the filter after the join, which prevents Spark from optimizing state cleanup. Option C defines watermarks but lacks the required time-range join condition. Option D uses an integer comparison rather than an interval expression.

Question 5 *(Hard)* **Scenario**: A data engineer uses `foreachBatch()` to write streaming micro-batches to both a Delta table and an external PostgreSQL database. If the pipeline fails mid-batch and restarts, some records may be written to PostgreSQL twice.

Question: Which pattern ensures exactly-once semantics for the PostgreSQL writes within `foreachBatch()`?

A) Use the `batchId` parameter to track which batches have been committed in a metadata table in PostgreSQL, and skip re-processing of already-committed batch IDs B) Use `outputMode("append")` with checkpointing, which guarantees exactly-once writes to all sinks automatically C) Deduplicate the PostgreSQL table after each batch using a `DELETE` of duplicate rows based on primary key D) Write to PostgreSQL first and Delta second so that the checkpoint only advances after both succeed

Answer

Correct Answer: A
The `batchId` provided to the `foreachBatch` function is guaranteed to be the same for a replayed batch after failure recovery. By recording the committed `batchId` in PostgreSQL and checking it before writing, the pipeline achieves idempotent writes and exactly-once semantics. Option B only guarantees exactly-once for Delta sinks, not external databases. Option C creates a window of inconsistency and is not truly idempotent.

Cost & Performance Optimization (Questions 6–7)

Question 6 *(Medium)* **Scenario**: A data engineer reads the Change Data Feed from a Delta table and needs to process the different types of row-level changes separately. The engineer writes a filter condition to isolate updated records but is unsure of the exact column values.

Question: Which set of values does the _change_type column contain in a Change Data Feed?

A) `insert`, `update`, `delete` B) `added`, `modified`, `removed` C) `insert`, `update_preimage`, `update_postimage`, `delete` D) `create`, `update_before`, `update_after`, `drop`

Answer

Correct Answer: C
The _change_type column in CDF contains four possible values: `insert` for new rows, `update_preimage` for the row state before an update, `update_postimage` for the row state after an update, and `delete` for removed rows. The pre/post image distinction for updates is critical for CDC pipelines that need to detect exactly which columns changed.

Question 7 *(Medium)* **Scenario**: A data engineering team manages hundreds of Delta tables across multiple schemas. They spend significant time manually running `OPTIMIZE` and `VACUUM` operations and tuning Z-ORDER columns. The team wants to automate these maintenance tasks.

Question: What does Databricks predictive optimization automate when enabled on a Unity Catalog metastore?

A) It automates only `VACUUM` operations based on storage usage thresholds B) It automates only `OPTIMIZE` operations by monitoring query patterns C) It automates `OPTIMIZE` and `VACUUM` but requires manual Z-ORDER column specification D) It automates `OPTIMIZE`, `VACUUM`, and `ZORDERING` by analyzing query patterns and table statistics to determine when and how to run maintenance

Answer

Correct Answer: D
Predictive optimization is a Unity Catalog feature that automatically runs `OPTIMIZE` (including file compaction), `VACUUM` (file cleanup), and `ZORDERING` (based on observed query predicates) without manual intervention. It uses table access patterns and statistics to determine the optimal schedule and configuration. Options A through C each describe only a subset of what predictive optimization handles.

Developing Code for Data Processing using Python and SQL (Question 8)

Question 8 ***(Hard)*** **Scenario**: An IoT streaming pipeline receives sensor readings that occasionally include duplicates due to at-least-once delivery. The pipeline uses a 2-hour watermark on `event_time`. A data engineer uses `dropDuplicates("sensor_id", "reading_id")` to remove duplicates but notices that state size grows unboundedly over days of continuous operation.

Question: Which approach resolves the unbounded state growth while still deduplicating records?

A) Reduce the watermark to 30 minutes so state expires faster B) Replace `dropDuplicates()` with `dropDuplicatesWithinWatermark("sensor_id", "reading_id")` which uses the existing watermark to bound deduplication state C) Add a `DISTINCT` clause in a downstream SQL query instead of deduplicating in the stream D) Increase executor memory to accommodate the growing state

Answer

Correct Answer: B
`dropDuplicatesWithinWatermark()` leverages the existing watermark to automatically expire deduplication state for events older than the watermark threshold. In contrast, `dropDuplicates()` retains all seen keys in state indefinitely because it has no time bound. Option A changes the late-data tolerance and does not fix the root cause since `dropDuplicates` ignores the watermark entirely.

Data Ingestion & Acquisition (Question 9)

Question 9 ***(Medium)*** **Scenario**: A data engineer configures Auto Loader with `cloudFiles.schemaEvolutionMode = "rescue"` to ingest CSV files. After several weeks, the team notices a `_rescued_data` column in the target table contains non-null JSON strings for some rows.

Question: What does the `_rescued_data` column contain?

A) A JSON string containing column values that did not conform to the expected schema, including columns with unexpected names or data that failed type casting B) The complete original raw record as a single string for any row that had at least one parsing warning C) Only the column

names that were added since the schema was first inferred D) A binary-encoded copy of the entire source file that contained the mismatched record

Answer

Correct Answer: A
The `_rescued_data` column captures a JSON map of any data that could not be parsed into the expected schema. This includes columns with unexpected names not in the schema and values that failed type casting (e.g., a string in an integer column). It does not store the entire raw record or file, only the specific fields that did not fit the defined schema.

Data Manipulation (Question 10)

Question 10 ***(Medium)*** **Scenario**: A data analyst needs to query a Delta table as it existed at a specific point in time for a compliance audit. The analyst knows the exact timestamp of the end-of-day snapshot they need, but does not know which Delta version corresponds to that timestamp. Another analyst needs to reproduce a specific report that was generated from a known Delta version.

Question: When is it more appropriate to use `TIMESTAMP AS OF` versus `VERSION AS OF` for Delta time travel?

A) `TIMESTAMP AS OF` is for streaming queries; `VERSION AS OF` is for batch queries B) `VERSION AS OF` should always be preferred because timestamps can be ambiguous across time zones C) `TIMESTAMP AS OF` works only with dates, not datetime values; `VERSION AS OF` supports both D) `TIMESTAMP AS OF` is ideal when querying by a business point-in-time (e.g., end-of-day), while `VERSION AS OF` is ideal for reproducibility when the exact commit version is known

Answer

Correct Answer: D
`TIMESTAMP AS OF` maps a datetime to the latest Delta version committed at or before that timestamp, making it natural for business-time queries like audits. `VERSION AS OF` uses the exact commit version number, ensuring deterministic reproducibility when the specific version is tracked (e.g., in a report metadata table). Both work in batch and streaming contexts, and timestamps support full datetime precision.

Developing Code for Data Processing using Python and SQL (Question 11)

Question 11 ***(Hard)*** **Scenario**: A data engineer implements a custom stateful streaming operation using `flatMapGroupsWithState()` to track user sessions. Sessions should expire if no event is received within 30 minutes. The engineer needs to choose between `ProcessingTimeTimeout` and `EventTimeTimeout`.

Question: What is the key difference between `ProcessingTimeTimeout` and `EventTimeTimeout` in stateful streaming?

A) `ProcessingTimeTimeout` triggers based on the event timestamp column, while `EventTimeTimeout` triggers based on wall-clock time B) `ProcessingTimeTimeout` is more accurate for out-of-order data because it accounts for late-arriving events C) `EventTimeTimeout` triggers when the watermark advances past the timeout timestamp, providing deterministic timeout behavior based

on data timestamps rather than wall-clock time D) There is no functional difference; they are aliases for the same timeout mechanism

Answer

Correct Answer: C
`EventTimeTimeout` is driven by the watermark, so the timeout fires when the watermark (derived from event timestamps across all partitions) advances past the configured timeout timestamp. This makes it deterministic and reproducible during reprocessing. `ProcessingTimeTimeout` uses the system clock, so timeouts depend on when micro-batches execute, which is non-deterministic and not reproducible.

Data Manipulation (Question 12)

Question 12 ***(Medium)*** **Scenario**: A data engineer is tasked with running `VACUUM` on a large Delta table to reclaim storage. The engineer considers setting the retention period to 0 hours to remove all historical files immediately.

Question: Why is running `VACUUM` with a retention period less than 7 days considered dangerous?

A) It will permanently delete the Delta transaction log, making the table unreadable B) It may remove files that are still needed by concurrent readers or writers that started before the VACUUM, leading to query failures with `FileNotFoundException` C) It converts the Delta table to a plain Parquet table by removing all metadata files D) It triggers a full table rewrite that doubles the storage usage temporarily

Answer

Correct Answer: B
VACUUM with a short retention period can delete data files that are still referenced by long-running queries or streaming jobs that hold an older table snapshot. These queries will fail with `FileNotFoundException` when they attempt to read the removed files. The 7-day default retention provides a safety buffer for concurrent operations. The transaction log (`_delta_log`) is not affected by VACUUM.

Developing Code for Data Processing using Python and SQL (Question 13)

Question 13 ***(Medium)*** **Scenario**: A data engineer builds a streaming pipeline that computes a running count of events per product category using `groupBy("category").count()`. The engineer tries to use `outputMode("append")` but the query fails at runtime.

Question: Which output mode correctly supports streaming aggregations with `groupBy`?

A) `outputMode("complete")` outputs the entire aggregation result table after each micro-batch, replacing all previous output B) `outputMode("append")` writes only new rows and is the only mode that supports aggregations C) `outputMode("update")` is required because it is the only mode that works with stateful operations D) Both `outputMode("append")` and `outputMode("update")` work equally well for aggregations without watermarks

Answer

Correct Answer: A
For streaming aggregations without a watermark, `complete` mode is required because the aggregation results change with every micro-batch and must be fully rewritten. append mode fails because Spark cannot determine when an aggregation result is finalized without a watermark. With a watermark defined, append and update modes also become available, but without one, only `complete` works.

Data Ingestion & Acquisition (Question 14)

Question 14 ***(Medium)*** **Scenario**: A data engineer configures Auto Loader to read JSON files using `format("cloudFiles")` and sets both `cloudFiles.format` and the top-level `.format()` in the DataStreamReader chain. The engineer is confused about which setting takes precedence.

Question: How does the configuration hierarchy work for Auto Loader's file format setting?

A) The top-level `.format("json")` call on the DataStreamReader overrides `cloudFiles.format` B) The two settings are independent: `.format()` sets the stream source type and `cloudFiles.format` sets the file parsing format C) Setting both causes an `IllegalArgumentException` at runtime because they conflict D) `.format("cloudFiles")` is required to activate Auto Loader as the stream source, and `cloudFiles.format` separately specifies the data file format (e.g., JSON, CSV, Parquet) to parse within Auto Loader

Answer

Correct Answer: D
Auto Loader requires `.format("cloudFiles")` on the DataStreamReader to identify the source as Auto Loader. The `cloudFiles.format` option then tells Auto Loader which file format parser to use for the actual data files. These are not competing settings but operate at different levels: one selects the streaming source, the other configures it. Using `.format("json")` directly would bypass Auto Loader entirely and use the standard file stream source.

Data Manipulation (Question 15)

Question 15 ***(Medium)*** **Scenario**: A multi-hop (Bronze/Silver/Gold) architecture ingests raw event data into Bronze, then applies transformations and quality checks into Silver. Some records arrive with missing required fields, malformed timestamps, or unexpected data types. The team wants to avoid data loss while maintaining data quality in downstream layers.

Question: Where in the multi-hop architecture should malformed data be handled to balance data quality and completeness?

A) At the Gold layer, since business logic determines what is valid B) At the Bronze-to-Silver transition, where data quality rules quarantine bad records to a separate ”quarantine” table while valid records proceed to Silver C) At ingestion into Bronze, rejecting any records that do not pass schema validation D) Malformed data should be silently dropped at every layer to maintain clean tables

Answer

Correct Answer: B
The Bronze-to-Silver transition is the standard point for data quality enforcement in a multi-hop architecture. Bronze stores raw data as-is for auditability and replay. At the Silver layer, quality rules validate, clean, and route malformed records to a quarantine table for investigation. Option C risks data loss at ingestion, and Option D provides no visibility into data quality issues.

Data Modeling (Question 16)

Question 16 ***(Hard)*** **Scenario**: A data engineer adds a `CHECK` constraint to an existing Delta table with 500 million rows: `ALTER TABLE orders ADD CONSTRAINT positive_amount CHECK (amount > 0)`. The table currently contains 200 rows where `amount` is 0 or negative.

Question: What happens when this `ALTER TABLE ADD CONSTRAINT` statement is executed?

A) The constraint is added and the 200 violating rows are automatically deleted B) The constraint is added successfully but only enforced on future writes; existing rows are not validated C) The statement fails because existing rows violate the constraint, and the constraint is not added until all violating data is removed or corrected D) The constraint is added and the 200 violating rows are quarantined to a separate error table

Answer

Correct Answer: C
Delta Lake validates all existing data against the CHECK constraint before adding it. If any rows violate the constraint, the `ALTER TABLE` statement fails and the constraint is not created. The engineer must first clean or remove the violating rows before the constraint can be added. This prevents the table from entering an inconsistent state with a constraint that existing data already violates.

Developing Code for Data Processing using Python and SQL (Question 17)

Question 17 ***(Medium)*** **Scenario**: A data engineer is designing a streaming pipeline that reads from Kafka and writes to a Delta table. The team needs to understand the exactly-once guarantees provided by Structured Streaming with Delta Lake as the sink.

Question: What level of delivery guarantee does Structured Streaming provide with checkpointing and a Delta Lake sink?

A) Exactly-once end-to-end: Structured Streaming's checkpointing tracks offsets, and Delta Lake's transaction log ensures each micro-batch is committed atomically and idempotently, so replayed batches produce no duplicates B) At-least-once: checkpointing tracks offsets but Delta Lake may write duplicate rows if a batch is replayed after a partial commit C) At-most-once: if a failure occurs, the uncommitted batch is lost and processing resumes from the next offset D) Exactly-once only if `foreachBatch()` is used with manual deduplication logic

Answer

Correct Answer: A
Structured Streaming with checkpointing and a Delta Lake sink provides exactly-once end-to-

end guarantees. The checkpoint records which offsets have been processed, and Delta Lake's transaction log ensures that write operations are atomic and idempotent. If a micro-batch is replayed after failure, Delta detects the duplicate commit and does not write the data again.

Data Modeling (Question 18)

Question 18 ***(Hard)*** **Scenario**: A daily batch pipeline uses `MERGE` to upsert fact records from a staging table into a target fact table. The fact table joins to a `dim_product` dimension table. Occasionally, new products appear in the fact staging data before they have been added to the dimension table, causing foreign key mismatches.

Question: What is the recommended pattern for handling late-arriving dimension data in an incremental batch pipeline?

A) Delay the fact table MERGE until all dimension tables are fully loaded, using a job dependency that blocks the fact pipeline B) Reject fact records with unknown dimension keys by filtering them out before the MERGE, accepting the data loss C) Use a surrogate key of -1 for unknown dimensions and update the fact records later when the dimension arrives, but this requires a second pass over the data D) Insert fact records with the unknown dimension key, place those records into an ”early-arriving facts” holding table, and reprocess them in the next pipeline run after the dimension has been loaded

Answer

Correct Answer: D
The early-arriving facts pattern captures fact records whose dimension keys are not yet available in a holding (or error) table, then reprocesses them in subsequent runs once the dimension data has arrived. This avoids data loss (Option B), avoids tight pipeline coupling and delays (Option A), and avoids the complexity of placeholder surrogate keys that must be corrected later (Option C).

Developing Code for Data Processing using Python and SQL (Question 19)

Question 19 ***(Medium)*** **Scenario**: A data engineering team builds a notebook that generates reports for different business units. The notebook must allow users to select a business unit from a predefined list when running interactively, and the same notebook must accept the business unit as a parameter when triggered by a scheduled job. The engineer uses `dbutils.widgets.dropdown("business_unit", "finance", ["finance", "marketing", "operations", "engineering"])` to create the parameter.

Question: When the scheduled job needs to generate the report for the ”operations” business unit, how should the job task be configured to override the widget default?

A) Set a cluster environment variable `BUSINESS_UNIT=operations` and modify the notebook to read from `os.environ` B) In the notebook task configuration, set `base_parameters: {"business_unit": "operations"}` to override the widget default C) Create a separate copy of the notebook with the dropdown default changed to ”operations” D) Use `spark.conf.set("business_unit", "operations")` in an init script attached to the job cluster

Answer

Correct Answer: B
The `base_parameters` field in a notebook task directly maps to widget names and overrides their default values at runtime. This is the intended mechanism for parameterizing notebooks across interactive and job contexts. Options A and D bypass the widget system entirely, and Option C creates maintenance overhead with duplicated notebooks.

Debugging and Deploying (Question 20)

Question 20 *(Medium)* **Scenario**: A platform team is migrating their deployment pipelines from the legacy Databricks CLI (version 0.x) to Declarative Automation Bundles (DABs) using the new Databricks CLI (version 0.200+). They currently use JSON template files and shell scripts to deploy jobs, clusters, and notebooks. A team member asks what key difference they should expect during the migration.

Question: Which statement correctly describes a key difference between Declarative Automation Bundles and the legacy CLI approach?

A) Declarative Automation Bundles require all resources to be defined in Python rather than YAML or JSON B) Declarative Automation Bundles only support job deployments and cannot manage clusters or permissions C) Declarative Automation Bundles use declarative YAML configuration files (`databricks.yml`) that define resources, environments, and deployment targets in a single project structure D) Declarative Automation Bundles replace the REST API entirely and cannot be used alongside direct API calls

Answer

Correct Answer: C
Declarative Automation Bundles introduce a declarative, project-based approach using `databricks.yml` files where resources (jobs, pipelines, etc.), environment-specific overrides, and deployment targets are defined together. This replaces the imperative scripting approach of the legacy CLI. Option A is wrong because DABs use YAML, Option B understates their capabilities, and Option D is incorrect since the API and bundles can coexist.

Developing Code for Data Processing using Python and SQL (Questions 21–24)

Question 21 *(Medium)* **Scenario**: A data engineer needs to create a multi-task job using the REST API (Jobs 2.1) where Task B and Task C both depend on Task A completing successfully, and Task D depends on both Task B and Task C completing. The engineer constructs the JSON payload for the `POST /api/2.1/jobs/create` endpoint.

Question: Which JSON structure correctly defines the task dependency graph for this job?

A) Define each task with a `depends_on` array referencing parent task keys, such as Task B having `"depends_on": [{"task_key": "task_a"}]` and Task D having `"depends_on": [{"task_key": "task_b"}, {"task_key": "task_c"}]` B) Set a single `execution_order` array at the job level listing tasks in sequential order: `["task_a", "task_b", "task_c", "task_d"]` C) Use `run_after` fields on each task with a single string value referencing one parent, requiring a separate mechanism for multi-parent dependencies D) Define a `dag` object at the job level with an adjacency list of task edges separate from the task definitions

Answer

Correct Answer: A
The Jobs API 2.1 uses a depends_on array within each task definition to specify upstream dependencies by `task_key`. This allows expressing complex DAGs including fan-out and fan-in patterns directly within the task list. Option B would enforce purely sequential execution, Option C incorrectly limits dependencies to a single parent, and Option D describes a structure not used by the Jobs API.

Question 22 ***(Hard)*** **Scenario**: A startup runs ad hoc data exploration notebooks during business hours and nightly batch ETL pipelines. The exploration workloads are unpredictable in timing and size, often sitting idle between queries. The nightly ETL pipelines are well-characterized with stable resource requirements and run for approximately 2 hours each night. The team wants to minimize cost while maintaining fast startup times for interactive work.

Question: Which compute configuration best balances cost and responsiveness for these two workload types?

A) Use a single large all-purpose cluster for both workloads with auto-scaling enabled B) Use job clusters for interactive exploration and serverless compute for nightly ETL C) Use all-purpose clusters with auto-termination for both workloads to ensure resources are freed D) Use serverless compute for ad hoc exploration notebooks and job clusters for the nightly ETL pipelines

Answer

Correct Answer: D
Serverless compute eliminates startup latency and charges only for active usage, making it ideal for unpredictable, bursty interactive workloads. Job clusters are cost-optimized for scheduled batch work since they offer lower DBU rates and auto-terminate after each run. Option A wastes resources during idle periods, Option B reverses the optimal assignment, and Option C still incurs higher DBU rates for production workloads.

Question 23 ***(Medium)*** **Scenario**: A data engineering team runs 50 scheduled jobs throughout the day, each creating a new job cluster. They notice that cluster startup time averages 5-7 minutes, which delays pipeline SLAs. The clusters all use the same instance type (`i3.xlarge`) and Databricks Runtime version. The team wants to reduce startup time without changing the job definitions significantly.

Question: Which approach most effectively reduces cluster startup time for these jobs?

A) Switch all jobs to use a single shared all-purpose cluster with auto-scaling to avoid repeated cluster creation B) Create an instance pool with `i3.xlarge` instances and configure each job cluster to use the pool, keeping warm instances available for immediate allocation C) Increase the cluster size for each job so that Spark initialization is parallelized across more nodes D) Attach init scripts to pre-install all libraries at the cluster level so startup only involves launching the runtime

Answer

Correct Answer: B
Instance pools maintain a set of warm (pre-provisioned) cloud instances that can be allocated to clusters instantly, reducing startup time from minutes to seconds. Since all jobs use the same instance type, a shared pool maximizes reuse. Option A sacrifices job isolation and cost

optimization, Option C does not reduce provisioning time, and Option D addresses library install time but not the dominant cost of instance provisioning.

Question 24 *(Medium)* **Scenario**: A governance team needs to enforce that all data engineering clusters in the workspace use specific Databricks Runtime versions (13.3 LTS or 14.3 LTS only), have a maximum of 8 worker nodes, are tagged with a `cost_center` value, and cannot enable credential passthrough. They want these rules applied automatically when any engineer creates a cluster.

Question: Which Databricks feature allows the governance team to enforce these constraints?

A) Cluster policies that define allowed attribute values, maximum limits, and required tags, then assign the policy to data engineering groups B) Workspace-level admin settings that globally restrict all cluster configurations to the approved runtime versions C) Unity Catalog metastore-level compute restrictions that govern runtime versions and node counts D) Custom init scripts that validate cluster configuration at startup and terminate non-compliant clusters

Answer

Correct Answer: A
Cluster policies provide fine-grained, declarative constraints on cluster attributes including runtime versions (allowlist), maximum nodes (range), required tags (fixed values), and disabled features. Policies are assigned to groups so engineers can only create compliant clusters. Option B lacks the granularity needed, Option C does not manage compute configurations, and Option D is reactive rather than preventive.

Ensuring Data Security and Compliance (Question 25)

Question 25 *(Medium)* **Scenario**: A workspace administrator creates a folder structure `/Workspace/Projects/TeamAlpha/` and grants the `data_engineers` group CAN MANAGE permission on the `TeamAlpha` folder. A team member creates a new notebook inside this folder. Another engineer from the `data_analysts` group, who has no explicit permissions on the folder, reports that they cannot view the notebook.

Question: Which statement correctly explains the permission behavior in this scenario?

A) The analyst cannot view the notebook because notebook-level permissions always override folder-level permissions B) The analyst should be able to view the notebook because all authenticated users have implicit read access to all workspace objects C) The analyst cannot view the notebook because Databricks workspace permissions are additive and do not grant implicit access D) The analyst cannot view the notebook because folder-level ACLs are inherited by child objects, and without explicit permissions on the folder or notebook, the analyst has no access

Answer

Correct Answer: D
Workspace folder permissions use inheritance: child objects (notebooks, sub-folders) inherit the ACLs of their parent folder. Since the `data_analysts` group was not granted any permission on the `TeamAlpha` folder, they inherit no access to objects within it. Option A incorrectly states that notebook permissions override folder permissions (they supplement them), Option B is false, and Option C is partially correct but misses the inheritance mechanism.

Developing Code for Data Processing using Python and SQL (Question 26)

Question 26 ***(Medium)*** **Scenario**: A data engineer has a main orchestration notebook that needs to call a utility notebook. When using `%run ./utils`, all variables and functions defined in `utils` become available in the main notebook's scope. The engineer considers switching to `dbutils.notebook.run("./utils", timeout_seconds=120)` instead. A colleague warns that the behavior will be different.

Question: Which statement correctly describes a key behavioral difference between `%run` and `dbutils.notebook.run()`?

A) `%run` executes the child notebook in a separate process while `dbutils.notebook.run()` executes in the same process B) `dbutils.notebook.run()` executes the child notebook in an isolated context and returns only a string result, whereas `%run` merges the child's execution context (variables, functions, imports) into the calling notebook C) `dbutils.notebook.run()` supports passing DataFrames as parameters while `%run` only supports string parameters D) `%run` is asynchronous and returns immediately while `dbutils.notebook.run()` blocks until completion

Answer

Correct Answer: B
`%run` is a compile-time inclusion that merges the child notebook's entire execution context (all variables, functions, and imports) into the parent, as if the code were pasted inline. `dbutils.notebook.run()` launches the child in an isolated context and can only return a single string via `dbutils.notebook.exit()`. Option A reverses the isolation model, Option C is incorrect since neither method passes DataFrames directly, and Option D is incorrect since `%run` also blocks.

Cost & Performance Optimization (Question 27)

Question 27 ***(Medium)*** **Scenario**: A data analytics team is evaluating Databricks SQL warehouse tiers. They need to run interactive dashboards for executives, support concurrent BI tool connections from 20 analysts, and use query federation to join Databricks tables with data in an external PostgreSQL database. Cost is a consideration but not the primary driver.

Question: Which Databricks SQL warehouse type meets all of these requirements?

A) Serverless SQL warehouse, which provides instant scaling, high concurrency, built-in query federation, and eliminates infrastructure management overhead B) Pro SQL warehouse, which supports query federation and high concurrency but requires manual cluster management C) Classic SQL warehouse, which provides the lowest cost and supports all the required features including query federation D) Any warehouse type works since query federation, concurrency, and dashboards are available on all tiers

Answer

Correct Answer: A
Serverless SQL warehouses provide instant elastic scaling for high concurrency, built-in support for query federation (including external databases like PostgreSQL), and require no infrastructure management. Pro warehouses support federation but lack serverless scaling benefits. Classic warehouses do not support query federation, making Option C incorrect. Option D is wrong because feature availability varies across tiers.

Data Governance (Question 28)

Question 28 ***(Medium)*** **Scenario**: A data engineer needs to store raw CSV and JSON files uploaded by external partners in Unity Catalog for governed access. The files should be accessible via SQL and Python, and the team wants Unity Catalog to manage the lifecycle and permissions of the stored files. The files should not be converted to Delta format since downstream systems require the original file formats.

Question: Which Unity Catalog feature is most appropriate for this requirement?

A) Create an external table with `STORED AS CSV` format to register the files in Unity Catalog B) Create an external location pointing to the cloud storage path where partner files are uploaded C) Create a managed volume within a Unity Catalog schema so that files are stored under Unity Catalog's managed storage with full lifecycle governance D) Create a Delta table and use `COPY INTO` to ingest the CSV/JSON files, then export them back to the original format when needed

Answer

Correct Answer: C
Managed volumes in Unity Catalog store files in the catalog's managed storage location, providing full lifecycle management (files are deleted when the volume is dropped) and governed access through Unity Catalog permissions. Files retain their original format and are accessible via `/Volumes/` paths. Option A does not apply since the requirement is for file storage, not tabular data. Option B provides governance over a location but not lifecycle management. Option D adds unnecessary conversion steps.

Developing Code for Data Processing using Python and SQL (Questions 29–30)

Question 29 ***(Hard)*** **Scenario**: A data engineer needs to schedule a job that runs Monday through Friday at 6:00 AM, 12:00 PM, and 6:00 PM UTC. The job should also be configured with a retry policy of up to 3 retries with a 5-minute interval between attempts if any task fails.

Question: Which configuration correctly implements both the schedule and retry policy?

A) Use three separate jobs each with a simple daily cron schedule (`0 6 * * 1-5`, `0 12 * * 1-5`, `0 18 * * 1-5`) and configure retries at the job level with `max_retries: 3` B) Use a single cron expression `0 6,12,18 * * MON-FRI` with a task-level `retry_policy` of `max_retries: 3, min_retry_interval_millis: 300000` C) Use a continuous trigger with a conditional check on the current hour and day, and implement retry logic within the notebook code using try/except blocks D) Use a single cron expression `0 6,12,18 * * 1-5` with a task-level `retry_policy` specifying `max_retries: 3` and `min_retry_interval_millis: 300000`

Answer

Correct Answer: D
The cron expression `0 6,12,18 * * 1-5` correctly triggers at 6:00, 12:00, and 18:00 on weekdays (1-5 represents Monday-Friday in the Databricks/Quartz cron format). The task-level `retry_policy` with `max_retries` and `min_retry_interval_millis` (300000ms = 5 minutes) configures automatic retries. Option A unnecessarily creates three separate jobs. Option B uses `MON-FRI` which is not valid in the Quartz cron format used by Databricks. Option C is overly complex and loses built-in retry tracking.

Question 30 ***(Medium)*** **Scenario**: A data engineering team needs to install a custom monitoring agent on every cluster in the workspace. They also need to install a team-specific Python library only on clusters used by the data science team. An engineer proposes using init scripts but is unsure about the execution model.

Question: Which statement correctly describes the behavior of global and cluster-scoped init scripts in Databricks?

A) Global init scripts and cluster-scoped init scripts run in parallel during cluster startup, with no guaranteed ordering between them B) Global init scripts run first on every cluster in the workspace, followed by cluster-scoped init scripts, allowing the monitoring agent to be installed globally while team-specific libraries are added via cluster-scoped scripts C) Cluster-scoped init scripts run before global init scripts, so team-specific configurations take precedence over workspace-wide settings D) Global init scripts can only be configured by metastore admins through Unity Catalog, not through workspace admin settings

Answer

Correct Answer: B
Databricks executes global init scripts first (in the order specified by the admin), followed by cluster-scoped init scripts. This means the monitoring agent can be deployed as a global init script that runs on every cluster, while the team-specific Python library is added as a cluster-scoped init script on data science clusters only. Option A incorrectly states they run in parallel, Option C reverses the execution order, and Option D is wrong since global init scripts are managed through workspace admin settings.

Data Modeling (Questions 31–34)

Question 31 ***(Hard)*** **Scenario**: A logistics company ingests GPS telemetry data from delivery trucks into a Bronze table. The data arrives as raw JSON with minimal structure issues. The analytics team needs a Gold table that provides real-time aggregated metrics such as average speed per route and delivery time percentiles. The Silver layer would only perform minor formatting (e.g., timestamp parsing) with no significant cleansing or joins required.

Question: What is the most appropriate approach for building this pipeline in the medallion architecture?

A) Write directly from Bronze to Gold, skipping Silver, since the intermediate transformations add no meaningful value B) Always create a Silver layer to maintain architectural consistency, even if transformations are trivial C) Write the raw data directly to Gold and skip both Bronze and Silver D) Create a Silver table but only materialize it as a view to reduce storage costs

Answer

Correct Answer: A
When the Silver layer would only perform trivial transformations that add no real cleansing or conforming value, it is acceptable to bypass it and write directly from Bronze to Gold. The medallion architecture is a guideline, not a rigid rule, and unnecessary layers add latency and storage overhead without benefit. Option B wastes resources on an empty pass-through, and option C loses the raw data auditability that Bronze provides.

Question 32 *(Hard)* **Scenario**: A data engineer is implementing SCD Type 2 on a `dim_customer` table using Delta Lake's MERGE statement. The table tracks historical changes with `effective_start_date`, `effective_end_date`, and `is_current` flag columns. When a matching customer record has changed attributes, the engineer needs to expire the old row and insert the new row.

Question: Which MERGE approach correctly implements SCD Type 2 with proper current/expired flag management?

A) Use MERGE with `WHEN MATCHED THEN UPDATE SET *` to overwrite the existing row with the new values B) Use MERGE with `WHEN MATCHED THEN DELETE` followed by a separate INSERT of both the expired and new rows C) Use MERGE with `WHEN MATCHED AND source.hash <> target.hash THEN UPDATE SET is_current = false, effective_end_date = current_date()` combined with a `WHEN NOT MATCHED THEN INSERT` clause, then run a second INSERT for the new current rows D) Use MERGE with `WHEN MATCHED THEN UPDATE SET is_current = false` and include the new row in the WHEN NOT MATCHED clause, which automatically handles both operations in a single pass

Answer

Correct Answer: C
SCD Type 2 requires two operations for changed records: expiring the old row (updating `is_current` to false and setting `effective_end_date`) and inserting the new current row. The correct approach uses a MERGE to expire matched rows with changed attributes, then a separate INSERT statement to add the new current versions. Option D is incorrect because the `WHEN NOT MATCHED` clause only fires for source rows with no match in the target, not for rows that were just updated.

Question 33 *(Hard)* **Scenario**: A data engineering team has a production Delta table with a column named `txn_amt` that business users find confusing. They want to rename it to `transaction_amount`. They also need to drop an obsolete column called `legacy_flag`. The table has 500 million rows and they cannot afford to rewrite the entire table.

Question: Which approach allows the team to rename and drop columns on the Delta table without rewriting data files?

A) Use `ALTER TABLE RENAME COLUMN txn_amt TO transaction_amount` and `ALTER TABLE DROP COLUMN legacy_flag` directly, as Delta Lake supports this by default B) Create a new table with the desired schema and use `INSERT INTO ... SELECT` to copy the data C) Create a view on top of the existing table with column aliases to simulate the rename D) Enable Delta column mapping by setting `delta.columnMapping.mode = 'name'` and then use `ALTER TABLE RENAME COLUMN` and `ALTER TABLE DROP COLUMN`

Answer

Correct Answer: D
Delta column mapping (`delta.columnMapping.mode = 'name'`) decouples logical column names from physical column names in Parquet files, enabling column rename and drop operations without rewriting data files. Without column mapping enabled, Delta Lake does not support column rename or drop operations. Option A would fail because the default column mapping mode (none) does not permit these operations. Option B requires a full data rewrite, which violates the constraint.

Question 34 *(Medium)* **Scenario**: A financial services company is building a lakehouse for regulatory reporting. They have complex queries that join customer, account, transaction, and product data. Reports require filtering by customer segment, product type, and time period. Query performance is critical because reports are generated on tight regulatory deadlines, and the data team includes SQL analysts accustomed to relational modeling.

Question: Which data modeling approach is most appropriate for the Gold layer of this lakehouse?

A) A fully denormalized single wide table containing all customer, account, transaction, and product attributes B) A star schema with a central fact table for transactions and dimension tables for customer, account, and product C) A normalized third normal form (3NF) model to minimize data redundancy D) A Data Vault 2.0 model with hubs, links, and satellites for maximum flexibility

Answer

Correct Answer: B
A star schema is ideal for the Gold layer when queries involve multiple filter dimensions and the team includes SQL analysts familiar with relational patterns. It provides a balance of query performance (through reduced joins compared to 3NF) and manageability (compared to a single denormalized table that would be unwieldy with this many entities). Option A would create an extremely wide table with massive redundancy, while option C optimizes for storage at the cost of query performance due to excessive joins.

Data Manipulation (Question 35)

Question 35 *(Medium)* **Scenario**: A data engineer needs to create a copy of a 2 TB production Delta table for a development environment. The dev team will run experimental transformations that may alter or delete data, but they primarily need to read the table for testing queries. Storage costs are a concern, and the copy needs to be created quickly during a short maintenance window.

Question: Which cloning strategy best meets these requirements?

A) Use SHALLOW CLONE, which copies only the metadata and transaction log while referencing the original data files, providing a fast and storage-efficient copy B) Use DEEP CLONE, which copies all data files and metadata to create a fully independent copy C) Use CTAS (`CREATE TABLE AS SELECT`) to create a complete physical copy of the data D) Use SHALLOW CLONE and then immediately run OPTIMIZE on the clone to consolidate files

Answer

Correct Answer: A
SHALLOW CLONE creates a copy by referencing the source table's data files without physically duplicating them, making it both fast and storage-efficient. This is ideal for development and testing environments where cost and speed matter. Option B (DEEP CLONE) would require copying all 2 TB of data, which is slow and expensive. Note that if the dev team writes to a shallow clone, only the modified files are stored separately while unchanged data still references the source.

Data Modeling (Question 36)

Question 36 *(Hard)* **Scenario**: A data engineer is designing a Delta table to store IoT sensor readings. The table will hold 10 billion rows and receive 50 million new rows daily. Queries almost

always filter by `device_region` (5 distinct values) and `reading_date`. Occasional queries filter by `sensor_id` (500,000 distinct values). The team is using Hive-style partitioning.

Question: Which partitioning strategy is most appropriate for this table?

A) Partition by `sensor_id` to ensure queries filtering on individual sensors are fast B) Partition by both `device_region` and `reading_date` to align with the two most common filter columns C) Partition by `device_region` only, and use Z-ORDER BY on `reading_date, sensor_id` for additional data skipping D) Partition by `reading_date` only, since it is the most granular time-based column

Answer

Correct Answer: C
With Hive-style partitioning, partition columns should have low cardinality to avoid creating too many small files. `device_region` with 5 distinct values is an excellent partition column. Adding `reading_date` as a partition would create thousands of sub-partitions (5 regions x 365+ days), leading to a small-files problem. Z-ORDER on `reading_date` and `sensor_id` provides efficient data skipping within each partition. Option A is wrong because 500,000 partitions would cause severe performance degradation.

Cost & Performance Optimization (Question 37)

Question 37 ***(Medium)*** **Scenario**: A data platform team is migrating an existing Hive-style partitioned Delta table (`PARTITIONED BY (year, month, day)`) to a more modern layout strategy. The table suffers from partition skew (some days have 100x more data than others), requires frequent partition management, and analysts occasionally need to query by non-partition columns like `customer_segment` and `product_category`.

Question: What is the primary advantage of converting this table to use Liquid Clustering?

A) Liquid Clustering compresses data files more efficiently than Hive-style partitioning, reducing storage costs B) Liquid Clustering incrementally reorganizes data without requiring full table rewrites and supports flexible, multi-column clustering that adapts to evolving query patterns C) Liquid Clustering eliminates the need for the OPTIMIZE command entirely since data is always optimally organized at write time D) Liquid Clustering automatically creates materialized views for the most frequently queried column combinations

Answer

Correct Answer: B
Liquid Clustering's key advantages over Hive-style partitioning are incremental data reorganization (no full rewrites needed), support for multiple clustering columns without the small-files problem of multi-column partitioning, and the ability to change clustering keys without rewriting all data. Option C is incorrect because OPTIMIZE is still used to trigger clustering; it just works incrementally. Option A is misleading because compression is not the primary differentiator.

Data Manipulation (Question 38)

Question 38 ***(Medium)*** **Scenario**: A new data engineer joins a team that uses the medallion architecture. They are asked to document the data quality expectations at each layer. The pipeline ingests raw e-commerce event data and produces analytics-ready tables for business intelligence.

Question: Which description correctly characterizes the transformations applied at each layer of the medallion architecture?

A) Bronze applies schema enforcement and deduplication; Silver adds business aggregations; Gold provides filtered views for consumers B) Bronze performs data type casting and validation; Silver joins reference data; Gold stores raw backup copies C) Bronze ingests raw data with minimal transformation (e.g., adding metadata columns and enforcing schema-on-read); Silver cleanses, deduplicates, and conforms data with schema enforcement; Gold applies business-level aggregations, joins, and curations for specific use cases D) Bronze and Silver perform identical cleansing operations for redundancy; Gold handles all business logic and aggregation

Answer

Correct Answer: C
The medallion architecture defines clear responsibilities: Bronze preserves raw data with minimal changes (append metadata, track ingestion), Silver handles data quality (dedup, null handling, type casting, schema enforcement, conforming), and Gold delivers business-ready datasets (aggregations, dimensional models, KPI tables). Option A incorrectly places schema enforcement and deduplication at the Bronze layer, which should preserve raw data fidelity.

Data Modeling (Question 39)

Question 39 ***(Hard)*** **Scenario**: A data engineer adds both a `NOT NULL` constraint and a `CHECK` constraint (`CHECK (quantity > 0)`) to a Delta table. During a pipeline run, a batch of 10,000 records is written where 5 records have null quantities and 3 records have `quantity = -1`.

Question: How does Delta Lake enforce these constraints during the write operation?

A) The NOT NULL constraint is enforced but CHECK constraints are only evaluated during reads, so only the 5 null records cause failures B) Both constraints are treated as warnings, and violating records are logged but still written to the table C) The CHECK constraint catches all 8 violating records (nulls and negatives) since CHECK implicitly covers nulls, and the write fails for the entire batch D) Both constraints are enforced independently at write time, and the entire write operation fails if any record violates either constraint, with an error indicating which constraint was violated

Answer

Correct Answer: D
Delta Lake enforces both NOT NULL and CHECK constraints at write time. If any record in the batch violates either constraint, the entire transaction is rejected with an error message identifying the violated constraint. These are independent checks: NOT NULL catches the 5 null records and CHECK catches the 3 negative-quantity records. Option C is incorrect because CHECK constraints do not implicitly handle nulls; SQL CHECK constraints evaluate to UNKNOWN (not false) for nulls, so the NOT NULL constraint is needed separately.

Data Governance (Question 40)

Question 40 ***(Medium)*** **Scenario**: Your organization recently purchased a second Databricks workspace for a newly acquired subsidiary. The platform team attaches this new workspace to the existing Unity Catalog metastore that the parent company already uses. Before the attachment, the subsidiary's workspace relied on a legacy Hive metastore with several hundred tables.

Question: What happens to the existing Hive metastore tables in the subsidiary workspace once it is attached to the Unity Catalog metastore?

A) The legacy Hive metastore tables are automatically migrated into Unity Catalog as managed tables B) The legacy Hive metastore tables remain accessible through the `hive_metastore` catalog but are not governed by Unity Catalog C) The legacy Hive metastore tables are deleted and must be recreated in Unity Catalog D) The legacy Hive metastore tables are copied into Unity Catalog as external tables with identical permissions

Answer

Correct Answer: B
When a workspace is attached to a Unity Catalog metastore, the legacy Hive metastore does not disappear. Its tables remain accessible under the built-in `hive_metastore` catalog, but they are not governed by Unity Catalog's centralized access controls. To bring those tables under Unity Catalog governance, teams must explicitly migrate them using tools like the UCX migration utility or `CREATE TABLE` statements.

Ensuring Data Security and Compliance (Question 41)

Question 41 *(Hard)* **Scenario**: A financial services company stores transaction data in a single `transactions` table containing records from all regional offices. Compliance requires that analysts in each regional group (e.g., `region_us`, `region_eu`, `region_apac`) can only query rows that belong to their own region. The security team wants this enforced at the data layer, not in the BI tool.

Question: Which approach correctly implements this row-level security requirement in Unity Catalog?

A) Create a dynamic view that filters rows using `SELECT * FROM transactions WHERE region = CASE WHEN is_account_group_member('region_us') THEN 'US' WHEN is_account_group_member('region_eu') THEN 'EU' WHEN is_account_group_member('region_apac') THEN 'APAC' END` B) Apply a column mask on the `region` column so that unauthorized analysts see NULL instead of the actual region value C) Create three separate tables partitioned by region and grant each group SELECT on only their table D) Use a cluster-level Spark configuration to set `spark.sql.region.filter` to the user's assigned region

Answer

Correct Answer: A
Dynamic views with `is_account_group_member()` provide row-level security by evaluating the querying user's group membership at runtime and filtering rows accordingly. This approach enforces security centrally at the data layer without requiring data duplication. Option B hides column values but still returns all rows, Option C creates data management overhead with duplicated tables, and Option D is not a real Spark configuration parameter.

Data Ingestion & Acquisition (Question 42)

Question 42 *(Hard)* **Scenario**: A pharmaceutical company wants to share de-identified clinical trial results with three external research institutions. Two of the institutions use Databricks, but the third institution uses only Apache Spark on their own Hadoop cluster. The data must be shared securely without giving any institution direct access to the company's workspace.

Question: How does the third institution (the non-Databricks recipient) authenticate and access data shared via OpenSharing?

A) They receive a Databricks personal access token that grants read-only access to the shared tables B) They connect via OAuth using the pharmaceutical company's identity provider C) They install a Databricks Connect client that tunnels into the provider's workspace D) They download an activation link that provisions a bearer token, which the open-source OpenSharing client uses to authenticate with the sharing server

Answer

Correct Answer: D
OpenSharing uses an open protocol where recipients receive an activation link to generate a bearer token stored in a credential file. The open-source OpenSharing connector (available for Spark, pandas, and other platforms) uses this token to authenticate directly with the sharing server without requiring a Databricks account. Options A and C incorrectly assume the recipient needs Databricks credentials or workspace connectivity, and Option B is not how OpenSharing authenticates external recipients.

Monitoring and Alerting (Question 43)

Question 43 ***(Hard)*** **Scenario**: The security operations team has detected suspicious behavior -- a service principal that normally runs nightly ETL pipelines has been issuing `GRANT` statements during business hours. The team needs to query the audit logs to identify all privilege escalation events performed by this service principal in the last 7 days.

Question: Which query correctly retrieves the relevant audit log entries from the Unity Catalog system tables?

A) `SELECT * FROM system.information_schema.grants WHERE grantee = 'etl-service-principal' AND grant_date > current_date() - INTERVAL 7 DAYS` B) `SELECT * FROM system.access.audit WHERE user_identity = 'etl-service-principal' AND event_type = 'SECURITY_ALERT'` C) `SELECT * FROM system.access.audit WHERE user_identity.email = 'etl-service-principal' AND action_name IN ('updatePermissions', 'grantPermission') AND event_time > current_date() - INTERVAL 7 DAYS` D) `SELECT * FROM system.billing.usage WHERE identity.email = 'etl-service-principal' AND usage_type = 'GRANT_OPERATION'`

Answer

Correct Answer: C
The `system.access.audit` table records all governance-related actions in Unity Catalog, including permission changes. Filtering on `user_identity.email` for the service principal and `action_name` for grant-related operations isolates privilege escalation events. Option A queries the information schema which shows current grants but not the audit trail of who made changes, Option B uses a non-existent `event_type` value, and Option D queries billing data which does not track security operations.

Ensuring Data Security and Compliance (Questions 44–45)

Question 44 *(Medium)* **Scenario**: A data engineering team currently uses a senior engineer's personal access token (PAT) embedded in an Airflow DAG to trigger Databricks jobs in production. The engineer is planning to leave the company in two weeks, and management is concerned about continuity and security.

Question: What is the recommended approach to address this situation and prevent similar issues going forward?

A) Create a service principal with a client secret, assign it only the permissions needed to run production jobs, and configure Airflow to authenticate using the service principal's credentials B) Transfer the engineer's personal access token to a shared team account so that multiple engineers can maintain it C) Generate a new personal access token under the engineering manager's account and update Airflow before the engineer departs D) Disable token-based authentication entirely and switch to interactive SSO login for all Airflow-triggered jobs

Answer

Correct Answer: A
Service principals are the recommended approach for production workloads because their lifecycle is independent of any individual user. They support least-privilege access through dedicated role assignments and their credentials can be rotated without impacting a person's account. Option B creates a shared credential anti-pattern with no individual accountability, Option C merely shifts the same problem to a different person, and Option D is impractical since orchestrators like Airflow require non-interactive authentication.

Question 45 *(Hard)* **Scenario**: A government agency stores classified datasets in a Databricks workspace deployed in an isolated VNet with no public internet access. Despite these network controls, a recent internal review discovered that a user with SELECT access on a sensitive table was able to copy data to an external cloud storage account by running `COPY INTO 'abfss://external-container@external-account.dfs.core.windows.net/'` from a notebook.

Question: Which combination of controls should be implemented to prevent this type of data exfiltration while still allowing legitimate workloads?

A) Revoke all SELECT permissions and require users to submit data access requests for each query B) Enable IP access lists on the workspace and restrict notebook execution to read-only mode C) Disable the `COPY INTO` command at the cluster level using a Spark configuration override D) Configure storage firewall rules to restrict egress to only approved storage accounts, and apply table ACLs in Unity Catalog to limit write permissions on external locations to authorized service principals only

Answer

Correct Answer: D
Defense-in-depth requires both network-level and data-level controls working together. Storage firewall rules and NSG/egress restrictions block connectivity to unapproved storage accounts, while Unity Catalog's external location permissions ensure that only authorized service principals can write to sanctioned destinations. Option A is operationally impractical, Option B does not prevent data writes to external storage, and Option C does not address other exfiltration methods such as direct Spark writes via DataFrames.

Debugging and Deploying (Question 46)

Question 46 *(Hard)* **Scenario**: A production Spark job processing customer transaction data is intermittently failing with `java.lang.OutOfMemoryError: Java heap space` on executor nodes. The data engineer opens the Spark UI to investigate and notices that the job has multiple `.cache()` calls on large DataFrames that persist across several stages. The Storage tab shows several cached RDDs consuming the majority of available executor memory.

Question: Based on the Spark UI Storage tab observations, what is the most likely root cause and the recommended corrective action?

A) The shuffle partitions are too large; increase `spark.sql.shuffle.partitions` to reduce partition size B) Cached DataFrames are consuming excessive executor memory; unpersist DataFrames that are no longer needed downstream C) The executor JVM garbage collection is misconfigured; switch to G1GC with larger heap regions D) The driver node is running out of memory from collecting results; increase driver memory allocation

Answer

Correct Answer: B
When the Spark UI Storage tab shows multiple large cached DataFrames consuming most executor memory, it creates memory pressure that leaves insufficient heap space for active computation. The fix is to call `.unpersist()` on DataFrames once they are no longer needed in subsequent stages, freeing executor memory for processing. Options A, C, and D address different symptoms and would not resolve memory consumed by unnecessary cached data.

Monitoring and Alerting (Questions 47–48)

Question 47 *(Hard)* **Scenario**: A Lakeflow Pipelines (formerly DLT) pipeline ingests clickstream events from Kafka and materializes several streaming tables. The pipeline has been running for several hours but the gold-layer aggregation table is lagging significantly behind the bronze ingestion layer. The data engineer needs to quantify the exact backlog at each stage to identify which flow is the bottleneck.

Question: What is the most effective method to identify backlog metrics across individual flows in the Lakeflow pipeline?

A) Check the Lakeflow pipeline UI graph view and look for red-highlighted tables indicating failures B) Query `system.billing.usage` filtered by the pipeline ID to find the slowest compute phase C) Query the pipeline's event log for `flow_progress` events and examine the `metrics.backlog_bytes` field for each flow D) Examine the Spark UI Streaming tab for each cluster to view micro-batch processing rates

Answer

Correct Answer: C
The Lakeflow Pipelines event log stores `flow_progress` event types that include detailed metrics such as `backlog_bytes` and `num_output_rows` for each individual flow in the pipeline. Querying these events lets you pinpoint exactly which flow has the largest backlog and is causing downstream lag. Option A shows status but not quantitative backlog metrics, Option B provides cost data not pipeline throughput, and Option D is not directly accessible for DLT-managed clusters.

Question 48 *(Medium)* **Scenario**: A finance team reports that the monthly Databricks bill has increased by 40% compared to the previous month, but no new jobs or clusters were added. The

platform administrator needs to identify which specific workloads or SKUs are responsible for the cost increase and compare usage patterns between the two months.

Question: Which approach most efficiently identifies the source of the cost anomaly?

A) Export the cluster activity logs from the Admin Console and aggregate compute hours per workspace B) Review the Databricks account billing dashboard to compare high-level totals by workspace C) Query `system.billing.usage` to aggregate DBU consumption by `sku_name`, `workspace_id`, and `usage_date`, then compare month-over-month D) Check each job's run history in the Lakeflow Jobs UI and manually calculate DBU usage from cluster uptime

Answer

Correct Answer: C
The `system.billing.usage` system table provides granular, queryable billing records that include SKU name, workspace ID, usage quantities, and dates, enabling precise month-over-month comparisons with SQL aggregations. This is far more efficient than manual approaches. Option B gives only high-level summaries without SKU-level drill-down, Option A lacks DBU cost attribution, and Option D is impractical at scale.

Debugging and Deploying (Question 49)

Question 49 ***(Hard)*** **Scenario**: A long-running Spark batch job on an autoscaling cluster frequently triggers autoscale-up events, but the added executors are quickly lost with `ExecutorLostFailure` errors. The Ganglia metrics dashboard for the cluster shows that executor memory utilization spikes above 95% shortly before each executor loss, while CPU utilization remains around 40%.

Question: Based on the Ganglia metrics pattern, what is the most likely cause of the executor failures?

A) Executors are running out of memory due to large shuffle spills or oversized partitions; increase `spark.executor.memory` or repartition the data B) The cluster network bandwidth is saturated during shuffle operations; switch to a compute-optimized instance type C) The autoscaler is removing executors prematurely due to idle timeout; increase the autoscale idle timeout threshold D) The YARN resource manager is preempting executors for higher-priority jobs; configure the job with maximum priority

Answer

Correct Answer: A
Ganglia showing memory utilization spiking to 95% with low CPU usage before executor loss is a strong indicator of out-of-memory conditions, typically caused by large shuffle operations or skewed partitions that exceed the executor memory limit. Increasing executor memory or repartitioning to reduce per-partition size resolves this. Option B would show network metrics spiking rather than memory, Option C applies when executors go idle (not when they crash), and Option D is not applicable in Databricks-managed YARN environments.

Monitoring and Alerting (Question 50)

Question 50 *(Hard)* **Scenario**: A data engineering team runs hundreds of Structured Streaming jobs across multiple workspaces. They need a centralized alerting system that triggers PagerDuty notifications when any streaming query's processing rate drops below a threshold or when a query falls behind by more than 10 minutes of event-time lag.

Question: What is the recommended approach to implement custom monitoring and alerting for Structured Streaming queries at this scale?

A) Schedule a notebook to periodically call `spark.streams.active` and check `lastProgress` for each query, then send alerts via webhook B) Enable Databricks SQL Alerts on the `system.billing.usage` table to detect when streaming cluster costs increase unexpectedly C) Implement a `StreamingQueryListener` that captures `onQueryProgress` events, extracts processing rate and watermark lag metrics, and publishes them to an external monitoring system D) Configure Ganglia metric thresholds on each cluster to trigger email alerts when CPU utilization drops below expected levels

Answer

Correct Answer: C
`StreamingQueryListener` is the official Spark API for programmatically monitoring streaming queries, providing `onQueryProgress` callbacks with detailed metrics including `inputRowsPerSecond`, `processedRowsPerSecond`, and watermark information. Publishing these to an external system like PagerDuty enables centralized alerting across all jobs. Option A requires polling and is less reliable, Option B monitors cost not streaming health, and Option D uses infrastructure metrics that do not directly reflect streaming processing rates.

Cost & Performance Optimization (Question 51)

Question 51 *(Hard)* **Scenario**: A Databricks SQL query that joins a 500-million-row fact table with three dimension tables is taking over 15 minutes to execute. The data analyst has verified that all tables have up-to-date statistics and the warehouse is properly sized. They open the Query Profile in the Databricks SQL UI to investigate.

Question: How should the analyst use the Query Profile to identify the specific bottleneck operator causing the slow performance?

A) Check the ”Results” tab to see which rows are returned most frequently and identify if the output is too large B) Examine the operator tree in the Query Profile to find the node with the highest time duration and spill-to-disk metrics, then optimize the corresponding join or aggregation C) Review the ”Query History” page to compare this query's execution time with previous runs and detect regression D) Look at the warehouse's ”Monitoring” tab to check if other concurrent queries are competing for resources

Answer

Correct Answer: B
The Query Profile's operator tree visualizes each physical operator (scan, join, aggregate, exchange) with time spent and data processed, and highlights operators that spill to disk. Identifying the node with the longest duration and largest spill reveals the exact bottleneck, allowing targeted optimization such as adding filters, changing join order, or adjusting the join strategy. Options A and C do not pinpoint the specific operator, and Option D checks for resource contention rather than query-level optimization issues.

Debugging and Deploying (Questions 52–54)

Question 52 *(Medium)* **Scenario**: A data engineering team maintains a `databricks.yml` bundle configuration. The root-level resources section defines a job with `max_concurrent_runs: 1` and a cluster with `num_workers: 2`. The `targets.production` section redefines the same job with `max_concurrent_runs: 4` but does not specify `num_workers`.

Question: When the bundle is deployed to the production target, what is the effective configuration for `max_concurrent_runs` and `num_workers`?

A) `max_concurrent_runs: 1`, `num_workers: 2` -- root-level defaults always take precedence B) `max_concurrent_runs: 4`, `num_workers: unset` -- the target completely replaces the root-level job definition C) The deployment fails because the target cannot override root-level resource settings D) `max_concurrent_runs: 4`, `num_workers: 2` -- the target overrides only the settings it explicitly specifies

Answer

Correct Answer: D
DAB uses a merge strategy where target-level settings are deep-merged with root-level defaults. Properties explicitly set in the target override the corresponding root-level values, while unspecified properties inherit the root-level defaults. This allows teams to define common baseline configurations at the root level and only override what differs per environment.

Question 53 *(Hard)* **Scenario**: A platform team configures OIDC-based authentication between GitHub Actions and Databricks using a service principal. During a workflow run, GitHub Actions requests an OIDC token from GitHub's cloud provider and sends it to the Databricks token endpoint.

Question: What happens during the OIDC token exchange with Databricks?

A) Databricks validates the GitHub OIDC token against the configured identity federation policy and issues a short-lived OAuth access token for the service principal B) Databricks stores the GitHub OIDC token as a persistent secret and uses it for all future API calls from that service principal C) GitHub Actions receives a Databricks personal access token that is valid for 90 days and cached in the workflow runner D) Databricks converts the GitHub OIDC token into a workspace-level API key that must be manually rotated every 30 days

Answer

Correct Answer: A
In the OIDC token exchange flow, Databricks validates the incoming GitHub OIDC token by checking its claims against the identity federation policy configured on the service principal. If validation succeeds, Databricks issues a short-lived OAuth token scoped to the service principal's permissions. This eliminates the need to store long-lived credentials in GitHub Secrets and provides automatic token rotation per workflow run.

Question 54 *(Medium)* **Scenario**: A team is deploying a new version of a production Lakeflow pipeline that processes 500 million records per day. They need to validate the new pipeline version with real production data before fully switching over, while ensuring zero downtime and the ability to instantly revert if issues are detected.

Question: Which deployment strategy best meets these requirements?

A) Blue-green deployment: deploy the new pipeline version alongside the old one, then switch all traffic instantly by updating the job definition B) Rolling deployment: gradually update each cluster node to run the new pipeline code one at a time C) Canary deployment: route a small percentage of incoming data to the new pipeline version, monitor its metrics, and gradually increase the percentage if results are healthy D) Feature flag deployment: wrap all new transformation logic in feature flags and toggle them on simultaneously across the existing pipeline

Answer

Correct Answer: C
A canary deployment allows the team to validate the new pipeline version with a small subset of real production data while keeping the existing pipeline handling the majority of traffic. This provides production-grade validation with minimal blast radius -- if the canary pipeline shows errors or data quality issues, the team can instantly stop routing data to it. Unlike blue-green, canary gives gradual confidence building rather than an all-or-nothing switchover.

Developing Code for Data Processing using Python and SQL (Question 55)

Question 55 ***(Hard)*** **Scenario**: A CI/CD pipeline runs automated tests against a Lakeflow pipeline that uses expectations such as `CONSTRAINT valid_amount EXPECT (amount > 0) ON VIOLATION DROP ROW`. After the test pipeline run completes, the team needs to programmatically verify that no more than 0.1% of rows were dropped by expectations before promoting the code to production.

Question: How should the team validate the expectation results in their CI/CD pipeline?

A) Parse the Lakeflow pipeline logs from the cluster driver output to count dropped rows B) Query the Lakeflow Pipelines event log for expectation metrics using `expectations.dropped_records` and compare the drop rate against the 0.1% threshold C) Add a downstream notebook that reads the target table's row count and compares it to the source table's row count D) Configure a Lakeflow Pipelines alert that sends an email notification if the drop rate exceeds 0.1%

Answer

Correct Answer: B
The Lakeflow Pipelines event log captures detailed metrics for each expectation, including the number of records that passed, failed, and were dropped. By querying the event log for `expectations` events, the CI/CD pipeline can programmatically extract the drop counts, calculate the drop rate, and fail the pipeline promotion if the rate exceeds the defined threshold. This approach is precise, automatable, and uses the purpose-built observability layer rather than indirect row-count comparisons or non-programmatic alerts.

Debugging and Deploying (Questions 56–57)

Question 56 ***(Medium)*** **Scenario**: A data engineering team deploys a production job using Declarative Automation Bundles. The job should be runnable by members of the `data-ops` group but not editable by them. Only the deploying service principal should have full management permissions.

Question: Which `databricks.yml` configuration correctly implements these permission requirements?

A) Set `run_as` to the `data-ops` group and leave permissions unspecified so the group inherits run access B) Add a `permissions` block with `CAN_MANAGE` for `data-ops` and rely on Unity Catalog to restrict edit access C) Deploy the job with no permissions block and then manually configure permissions through the Databricks workspace UI D) Add a `permissions` block granting `CAN_VIEW` and `CAN_MANAGE_RUN` to `group_name: data-ops` while the deploying service principal retains `CAN_MANAGE` as the owner

Answer

Correct Answer: D
The DAB `permissions` block on a job resource supports fine-grained access levels including `CAN_VIEW`, `CAN_MANAGE_RUN`, and `CAN_MANAGE`. Granting `CAN_MANAGE_RUN` to the `data-ops` group allows them to trigger and monitor job runs without being able to edit the job definition. The deploying service principal automatically retains `CAN_MANAGE` as the resource owner, ensuring only the deployment pipeline can modify the job configuration.

Question 57 ***(Medium)*** **Scenario**: A team deployed a new version of their DAB-managed production pipeline 30 minutes ago. Monitoring alerts now indicate that the new version is producing incorrect aggregation results due to a logic error in a transformation. The team needs to revert to the previous working version as quickly as possible.

Question: What is the fastest way to roll back the deployment using Declarative Automation Bundles?

A) Check out the previous Git commit in the repository and run `databricks bundle deploy --target production` to redeploy the last known good version B) Run `databricks bundle destroy --target production` to remove the broken deployment, then manually recreate the resources from the Databricks UI C) Use `databricks bundle run --target production --version previous` to automatically roll back to the prior deployed version D) Navigate to the Databricks workspace UI, locate each deployed resource, and manually revert the settings to match the previous configuration

Answer

Correct Answer: A
Since DAB deployments are driven from source-controlled configuration files, the fastest rollback strategy is to check out the previous known-good commit in Git and redeploy with `databricks bundle deploy`. This restores all resource configurations -- jobs, pipelines, clusters -- to their previous state in a single command. Option C uses a `--version` flag that does not exist in the DAB CLI. Options B and D are slow, error-prone, and defeat the purpose of infrastructure-as-code.

Developing Code for Data Processing using Python and SQL (Question 58)

Question 58 ***(Medium)*** **Scenario**: A data engineering team is building a Lakeflow (DLT) pipeline that ingests clickstream events from Kafka and produces aggregated session metrics consumed by two downstream systems. A real-time dashboard queries the latest session data every 30 seconds, and a nightly batch process generates a daily summary report from the same session metrics.

Question: How should the team define the session metrics dataset in the Lakeflow pipeline to best serve both downstream consumers?

A) Define it as a materialized view so that both the dashboard and batch process always read a consistent, pre-computed result B) Define it as a streaming table with Change Data Feed enabled so the batch process can read incremental changes C) Define it as a streaming table so the real-time dashboard reads low-latency results, while the batch process queries the same table on its own schedule D) Define two separate datasets: a streaming table for the dashboard and a materialized view for the batch process

Answer

Correct Answer: C
A streaming table continuously processes incoming records with low latency, which satisfies the real-time dashboard's requirement for fresh data. The nightly batch process can simply query the same streaming table at its scheduled time without requiring a separate materialized view. Option D introduces unnecessary duplication, and option A (materialized view) only recomputes on pipeline refresh rather than processing incrementally, adding latency unsuitable for the 30-second dashboard polling.

Cost & Performance Optimization (Questions 59–60)

Question 59 ***(Medium)*** **Scenario**: A data engineer is evaluating whether to enable Photon on a production SQL warehouse that runs a mix of workloads. The warehouse processes nightly ETL jobs that perform large-scale joins and aggregations across Delta tables with hundreds of columns, as well as ad-hoc exploratory queries from analysts that use many UDFs written in Python.

Question: Which workload pattern benefits most from enabling Photon on this warehouse?

A) The ad-hoc exploratory queries, because Photon accelerates Python UDF execution through native compilation B) The nightly ETL jobs, because Photon accelerates scan-heavy operations, joins, and aggregations on Delta tables C) Both workloads benefit equally, because Photon replaces the entire Spark execution engine D) Neither workload benefits significantly, because Photon only optimizes streaming workloads

Answer

Correct Answer: B
Photon is a vectorized query engine written in C++ that significantly accelerates scan-intensive operations, hash joins, aggregations, and shuffle operations on Delta tables -- exactly the patterns in the nightly ETL jobs. Photon does not accelerate Python UDFs, which still execute in the standard Python runtime, so the ad-hoc queries with Python UDFs see limited benefit. Option C is incorrect because Photon does not replace the entire engine; unsupported operations fall back to standard Spark execution.

Question 60 ***(Medium)*** **Scenario**: A data engineer manages a 5TB Delta table that was originally optimized using `OPTIMIZE ... ZORDER BY (region, event_date)`. The team has decided to migrate to liquid clustering by running `ALTER TABLE events CLUSTER BY (region, event_date)`. After migration, the engineer notices that running `OPTIMIZE events` no longer requires specifying `ZORDER BY` columns.

Question: How does `OPTIMIZE` behave differently after the table has been converted to liquid clustering?

A) OPTIMIZE no longer performs any file compaction; it only updates clustering metadata in the Delta log B) OPTIMIZE applies Z-ORDER clustering automatically using the columns defined in the

CLUSTER BY clause, with no behavioral change to the compaction process C) OPTIMIZE rewrites all data files on every run to ensure complete clustering, which increases write amplification compared to Z-ORDER D) OPTIMIZE incrementally clusters only the files that need reorganization, avoiding full rewrites and enabling more efficient partial clustering over time

Answer

Correct Answer: D
With liquid clustering, `OPTIMIZE` uses an incremental approach that identifies and rewrites only the files that would benefit from better clustering, rather than rewriting the entire table or requiring a full sort. This is a key behavioral difference from Z-ORDER, which rewrites all files in the targeted partitions on each OPTIMIZE run. The incremental strategy reduces write amplification and makes it practical to run OPTIMIZE more frequently without excessive I/O cost.

← Back to Mock Exam 2

Supplementary questions (not in the timed set) These five questions were added for the Nov 30, 2025 guide and are not part of the timed set or the practice-quiz bank. Each is labelled with its Oct 9, 2026 exam section — the former *Data Sharing and Federation* questions now belong to **Data Ingestion & Acquisition** (12 %).

Question DSF-1 *(Medium — Data Ingestion & Acquisition)* **Scenario**: A data engineering team needs to give a partner company read-only access to a Delta table that's updated every hour. The partner uses Databricks too, but is in a different cloud account. The team wants to avoid copying data and wants UC to enforce all access.

Question: Which pattern fits best?

A) Export the table to S3 as Parquet and email the partner a presigned URL
B) Create a Databricks-to-Databricks (D2D) OpenSharing share with the partner's UC sharing identifier and grant the share to a `RECIPIENT`
C) Replicate the table to the partner's account using Lakeflow Pipelines
D) Set up a foreign catalog in the partner's workspace pointing at the team's table

Answer

Correct Answer: B
Databricks-to-Databricks OpenSharing is purpose-built for this: it shares the live Delta table (the partner sees updates as they land), uses UC identities on both sides, and the provider grants a `SHARE` to a `RECIPIENT` identified by the partner's UC sharing identifier. No data copy, full UC audit trail. Option A loses governance and freshness. Option C (Lakeflow Pipelines replication) creates a parallel pipeline you have to maintain AND loses freshness — pipelines run on a schedule, while D2D Share is live. Option D is Lakehouse Federation, which is for the *consumer* to query *external* (non-Delta) sources — not for cross-Databricks data sharing.

Question DSF-2 *(Medium — Data Ingestion & Acquisition)* **Scenario**: An analytics team wants to query a Snowflake `orders` table for ad-hoc analysis without ETL'ing it into Delta. They need filter pushdown to keep performance reasonable, and credentials must be governed centrally.

Question: Which combination is correct?

A) Create a Databricks JDBC connection in `spark.conf` and read with `spark.read.format("jdbc")`
B) Create a Unity Catalog `CONNECTION` of type `SNOWFLAKE`, then `CREATE FOREIGN CATALOG` over it; query as `catalog.schema.table`
C) Use OpenSharing — Snowflake supports being a OpenSharing share recipient
D) Use Auto Loader with the Snowflake source connector

Answer

Correct Answer: B
Lakehouse Federation is the documented path for federated, read-mostly access to external databases. The two-step DDL — `CREATE CONNECTION ... TYPE SNOWFLAKE ...` followed by `CREATE FOREIGN CATALOG ... USING CONNECTION ...` — stores the credential in UC (auditable, rotatable), exposes the Snowflake database as a UC catalog, and supports filter / projection / aggregation pushdown to Snowflake where the dialect allows. Option A scatters credentials in cluster config and bypasses UC. Option C is wrong direction — Snowflake is not a OpenSharing share recipient platform. Option D is for object-store file ingestion, not relational sources.

Question PERF-1 *(Hard — Cost & Performance Optimization)* **Scenario**: A 2 TB Delta table is partitioned by `event_date` and Z-ORDERed by `customer_id`. Queries filter on both `event_date` (high selectivity) and `region` (low cardinality, 5 distinct values). The team is considering migrating to liquid clustering.

Question: Which migration approach gives the best query performance with the least operational disruption?

A) Run `OPTIMIZE table ZORDER BY (customer_id, region, event_date)` to add `region` to the Z-ORDER
B) Drop the table and recreate it with `CLUSTER BY (customer_id, event_date, region)` and a one-time `OPTIMIZE` to materialise clustering
C) Run `ALTER TABLE table CLUSTER BY (customer_id, event_date, region)` and let liquid clustering organise data incrementally on subsequent writes; back-fill with `OPTIMIZE table FULL`
D) Switch to bucketing on `customer_id` with 100 buckets

Answer

Correct Answer: C
Liquid clustering supports **`ALTER TABLE ... CLUSTER BY`** for migration without recreating the table. New writes adopt the clustering; an explicit `OPTIMIZE table FULL` triggers a one-time reorganisation of existing data. This avoids the disruption of B (recreate) and the partitioning lock-in of A (Z-ORDER is non-adaptive and partition columns can't be Z-ORDERed). D is a Hive-era pattern not relevant to Delta on Databricks.

Question COST-1 *(Medium — Cost & Performance Optimization)* **Scenario**: A nightly Lakeflow Job runs a Spark batch pipeline that takes 45 minutes on a Standard (DBR) all-purpose cluster of 8 i3.xlarge workers. The team wants to halve the cost.

Question: Which two changes typically yield the biggest cost reduction with the least risk?

A) Switch to a Job cluster (cheaper DBU rate per node) AND enable autoscaling 2–8 workers
B) Reduce worker count to 4 and keep the cluster as all-purpose

C) Move to Photon-enabled instance type AND enable single-user access mode
D) Switch to spot instances AND disable autoscaling

Answer

Correct Answer: A
Two independent levers, both safe: (1) Job clusters cost ~50 % less per DBU than all-purpose clusters because they're not designed for shared interactive use; (2) autoscaling drops idle nodes so you only pay for what you use during low-throughput phases of the job. B caps capacity (may slow the job → cost trade-off neutralised). C may help but Photon depends on workload shape and single-user mode is unrelated to cost. D introduces spot-eviction risk and removes elasticity.

Question MON-1 ***(Medium — Monitoring and Alerting)*** **Scenario**: A Lakeflow Job that processes daily orders has been failing intermittently for the past week. The team needs both proactive alerting (so on-call is paged before downstream teams notice) AND a way to query historical failure patterns to identify the root cause.

Question: Which combination satisfies both requirements?

A) Configure Lakeflow Jobs notifications (email + webhook to PagerDuty) for failures AND query `system.lakeflow.job_run_timeline` for historical failure analytics
B) Add a Slack webhook inside the job's last task that fires on success; absence of message indicates failure
C) Set up a SQL Alert that polls `SHOW JOBS` every 5 minutes
D) Rely on Databricks support to surface failures automatically

Answer

Correct Answer: A
Lakeflow Jobs notifications fire on success/failure/duration thresholds and route via email or webhook (PagerDuty, Slack, custom HTTP). For historical analytics, system tables under `system.lakeflow.*` expose job run history queryable in SQL. B is anti-pattern ("absence of signal" alerts are notoriously unreliable). C `SHOW JOBS` doesn't surface run-level state; the system tables do. D is not how Databricks works.

Mock Exam 2 Debrief — Databricks Data Engineer Professional

Use this after Mock Exam 1's debrief, working from your *second* timed sitting.

Tip

If your Mock-2 score is **higher than Mock-1**, the gap is closing — keep the study cadence. If it's **lower or flat**, your study plan needs a course correction toward the weakest sections.

Note

Questions are labelled with the **9 sections of the Oct 9, 2026 exam guide**. The mock was written for earlier guides, so its mix does not match the official weights exactly — use the counts below when you score yourself, and use the practice questions for the skills that are

new in 2026.

Section → study-resource mapping Same as Mock Exam 1's debrief — see the Mock Exam 1 debrief for the full section → resource table.

Per-question quick map Each question is mapped to its primary exam section. Questions that touch two sections are listed under the one their correct answer depends on. The supplementary questions at the end of `questions.md` are listed last.

Q#	Difficulty	Section	Topic folder	Cheat sheet
1	Medium	Data Ingestion	Section 2 folder	Auto Loader quick ref
2	Hard	Data Ingestion	Section 2 folder	Auto Loader quick ref
3	Hard	Data Modeling	Section 9 folder	Delta Lake commands
4	Hard	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
5	Hard	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
6	Medium	Cost & Performance	Section 5 folder	Performance optimization
7	Medium	Cost & Performance	Section 5 folder	Performance optimization
8	Hard	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
9	Medium	Data Ingestion	Section 2 folder	Auto Loader quick ref
10	Medium	Data Manipulation	Section 3 folder	Delta Lake commands
11	Hard	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
12	Medium	Data Manipulation	Section 3 folder	Delta Lake commands
13	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
14	Medium	Data Ingestion	Section 2 folder	Auto Loader quick ref
15	Medium	Data Manipulation	Section 3 folder	Delta Lake commands
16	Hard	Data Modeling	Section 9 folder	Delta Lake commands
17	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
18	Hard	Data Modeling	Section 9 folder	Delta Lake commands
19	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
20	Medium	Debugging & Deploying	Section 8 folder	Spark configurations
21	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
22	Hard	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
23	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
24	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
25	Medium	Security & Compliance	Section 6 folder	Unity Catalog quick ref
26	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
27	Medium	Cost & Performance	Section 5 folder	Performance optimization
28	Medium	Data Governance	Section 7 folder	Unity Catalog quick ref
29	Hard	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
30	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
31	Hard	Data Modeling	Section 9 folder	Delta Lake commands
32	Hard	Data Modeling	Section 9 folder	Delta Lake commands
33	Hard	Data Modeling	Section 9 folder	Delta Lake commands
34	Medium	Data Modeling	Section 9 folder	Delta Lake commands
35	Medium	Data Manipulation	Section 3 folder	Delta Lake commands
36	Hard	Data Modeling	Section 9 folder	Delta Lake commands
37	Medium	Cost & Performance	Section 5 folder	Performance optimization
38	Medium	Data Manipulation	Section 3 folder	Delta Lake commands
39	Hard	Data Modeling	Section 9 folder	Delta Lake commands
40	Medium	Data Governance	Section 7 folder	Unity Catalog quick ref
41	Hard	Security & Compliance	Section 6 folder	Unity Catalog quick ref
42	Hard	Data Ingestion	Section 2 folder	Auto Loader quick ref
43	Hard	Monitoring	Section 4 folder	Unity Catalog quick ref

Q#	Difficulty	Section	Topic folder	Cheat sheet
44	Medium	Security & Compliance	Section 6 folder	Unity Catalog quick ref
45	Hard	Security & Compliance	Section 6 folder	Unity Catalog quick ref
46	Hard	Debugging & Deploying	Section 8 folder	Spark configurations
47	Hard	Monitoring	Section 4 folder	Unity Catalog quick ref
48	Medium	Monitoring	Section 4 folder	Unity Catalog quick ref
49	Hard	Debugging & Deploying	Section 8 folder	Spark configurations
50	Hard	Monitoring	Section 4 folder	Unity Catalog quick ref
51	Hard	Cost & Performance	Section 5 folder	Performance optimization
52	Medium	Debugging & Deploying	Section 8 folder	Spark configurations
53	Hard	Debugging & Deploying	Section 8 folder	Spark configurations
54	Medium	Debugging & Deploying	Section 8 folder	Spark configurations
55	Hard	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
56	Medium	Debugging & Deploying	Section 8 folder	Spark configurations
57	Medium	Debugging & Deploying	Section 8 folder	Spark configurations
58	Medium	Developing Code	Section 1 folder	Lakeflow Pipelines quick ref
59	Medium	Cost & Performance	Section 5 folder	Performance optimization
60	Medium	Cost & Performance	Section 5 folder	Performance optimization
DSF-1	—	Data Ingestion	Section 2 folder	Auto Loader quick ref
DSF-2	—	Data Ingestion	Section 2 folder	Auto Loader quick ref
PERF-1	—	Cost & Performance	Section 5 folder	Performance optimization
COST-1	—	Cost & Performance	Section 5 folder	Performance optimization
MON-1	—	Monitoring	Section 4 folder	Unity Catalog quick ref

Per-section question counts (for self-scoring)

Exam section	Official weight	Questions in this mock (of 60
1. Developing Code for Data Processing using Python and SQL	23 %	16
2. Data Ingestion & Acquisition	12 %	5
3. Data Manipulation	12 %	5
4. Monitoring and Alerting	10 %	4
5. Cost & Performance Optimization	15 %	7
6. Ensuring Data Security and Compliance	8 %	4
7. Data Governance	5 %	2
8. Debugging and Deploying	10 %	8
9. Data Modeling	5 %	9

Study plan by miss count (60-question mock)

Missed	Status	Action
0–6 (≥ 90 %)	Excellent	Schedule the real exam this week
7–12 (80–89 %)	Likely passing	Spot-fill the 2-3 weakest domains; re-confirm by re-attempting the misse
13–17 (72–78 %)	On the bubble	1-2 more weeks of focused study; consider an Academy advanced course
18–24 (60–70 %)	Not ready	Comprehensive review; re-take Mock 1 + 2 after 3-4 weeks
25+ (< 60 %)	Significant gap	Restart from `shared/fundamentals/`; plan 4-6 weeks

Common Mock-2 traps

- **Lakeflow Pipelines event-log queries** — multiple questions test reading the pipeline event log; know the column shape

- **Declarative Automation Bundles `mode: development` vs `mode: production`** — dev pauses schedules and prefixes resources; production uses the run-as identity
- **AUTO CDC (formerly `APPLY CHANGES INTO`)** vs hand-written MERGE — AUTO CDC is preferred for SCD inside declarative pipelines
- **OpenSharing identifier vs activation URL** — D2D uses sharing identifiers; open OpenSharing uses activation URLs

---